

---

# DATA SCIENCE

Série 1 — Volume II

## *Modélisation statistique*

---

Régression · Modèles linéaires généralisés · Analyse multivariée  
Séries temporelles · Survie · Statistique bayésienne

De la régression linéaire aux méthodes bayésiennes,  
avec Dar Jasmin, de 2016 à 2025.

# Table des matières

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>Introduction : de la description à la modélisation</b>                                      | <b>1</b>  |
| Là où le volume I nous a laissés . . . . .                                                     | 1         |
| À qui s'adresse ce volume ? . . . . .                                                          | 1         |
| La carte du volume . . . . .                                                                   | 2         |
| Cinq idées qui reviennent dans tout le volume . . . . .                                        | 3         |
| Comment travailler avec ce volume . . . . .                                                    | 3         |
| <b>Les données et l'environnement du volume</b>                                                | <b>5</b>  |
| Dar Jasmin, de 2016 à 2025 . . . . .                                                           | 5         |
| Le tableau des clients . . . . .                                                               | 6         |
| L'enquête de satisfaction . . . . .                                                            | 7         |
| Les ventes mensuelles . . . . .                                                                | 8         |
| L'environnement de travail . . . . .                                                           | 9         |
| <b>Chapitre 1 : Régression linéaire</b>                                                        | <b>11</b> |
| Le chemin de ce chapitre . . . . .                                                             | 11        |
| 1.1 Le modèle linéaire et les moindres carrés . . . . .                                        | 12        |
| 1.1.1 Un exemple minuscule, entièrement à la main . . . . .                                    | 12        |
| 1.1.2 Le modèle linéaire : ce que l'on suppose . . . . .                                       | 15        |
| 1.1.3 Les équations normales : démonstration . . . . .                                         | 16        |
| 1.1.4 La géométrie des moindres carrés : une projection . . . . .                              | 17        |
| 1.1.5 Propriétés statistiques : sans biais, variance, et le théorème de Gauss-Markov . . . . . | 19        |
| 1.1.6 Décomposition de la variance et coefficient de détermination $R^2$ . . . . .             | 20        |
| 1.1.7 Première étude : le panier des clients de Dar Jasmin . . . . .                           | 21        |
| 1.1.8 Logarithmes : interpréter et prédire . . . . .                                           | 25        |
| 1.1.9 Interactions : quand un effet dépend d'un autre . . . . .                                | 26        |
| 1.2 Inférence sur les coefficients . . . . .                                                   | 27        |
| 1.2.1 La loi des estimateurs sous l'hypothèse de normalité . . . . .                           | 28        |
| 1.2.2 Tester un coefficient, lire un intervalle . . . . .                                      | 29        |
| 1.2.3 Le test $F$ : tester plusieurs coefficients à la fois . . . . .                          | 30        |
| 1.2.4 Intervalle de confiance et intervalle de prédiction . . . . .                            | 31        |
| 1.2.5 Vérifier la théorie par simulation : la couverture des intervalles . . . . .             | 34        |
| 1.2.6 Quand on ne veut pas supposer la normalité : le bootstrap des couples . . . . .          | 34        |
| 1.2.7 Retour aux questions de Yasmine . . . . .                                                | 35        |
| 1.3 Diagnostics : le modèle est-il fiable ? . . . . .                                          | 36        |
| 1.3.1 Les résidus : brut, standardisé, studentisé . . . . .                                    | 37        |
| 1.3.2 Les graphiques de résidus : voir avant de tester . . . . .                               | 38        |
| 1.3.3 Les tests de diagnostic (et pourquoi ils ne remplacent pas les graphiques) . . . . .     | 39        |
| 1.3.4 Effet de levier et observations influentes . . . . .                                     | 41        |
| 1.3.5 La multicollinéarité : quand deux variables disent la même chose . . . . .               | 44        |
| 1.3.6 Variance non constante : que faire ? . . . . .                                           | 46        |

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| 1.3.7 Courbure et variables manquantes : lire les graphiques de résidus partiels . . | 47        |
| 1.4 Sélection de variables et comparaison de modèles . . . . .                       | 48        |
| 1.4.1 Le surajustement : mieux s'ajuster n'est pas mieux prédire . . . . .           | 49        |
| 1.4.2 Les critères d'information : AIC et BIC . . . . .                              | 50        |
| 1.4.3 La validation croisée . . . . .                                                | 52        |
| 1.4.4 Comparer plusieurs modèles sur les données de l'enquête . . . . .              | 53        |
| 1.4.5 La sélection automatique, et pourquoi il faut s'en méfier . . . . .            | 55        |
| 1.4.6 Verdict : retrouver la vérité . . . . .                                        | 56        |
| 1.5 ⊕ Pour aller plus loin : la régularisation (Ridge, Lasso, Elastic Net) . . . . . | 59        |
| 1.5.1 La régression Ridge . . . . .                                                  | 59        |
| 1.5.2 Le Lasso : la pénalité qui élimine des variables . . . . .                     | 62        |
| 1.5.3 L'Elastic Net : un compromis . . . . .                                         | 64        |
| 1.5.4 Application : beaucoup de variables, peu de clients . . . . .                  | 65        |
| 1.5.5 Précautions . . . . .                                                          | 68        |
| 1.6 ⊕ Pour aller plus loin : la régression robuste . . . . .                         | 69        |
| 1.6.1 La fragilité des moindres carrés . . . . .                                     | 70        |
| 1.6.2 Les M-estimateurs et la fonction de Huber . . . . .                            | 70        |
| 1.6.3 Une contamination : les erreurs de saisie . . . . .                            | 72        |
| 1.6.4 La limite : les points à fort levier . . . . .                                 | 75        |
| 1.6.5 Une autre voie : la régression quantile . . . . .                              | 75        |
| 1.7 ⊕ Pour aller plus loin : les modèles à effets mixtes et hiérarchiques . . . . .  | 77        |
| 1.7.1 Un jeu de données groupé : les points relais . . . . .                         | 78        |
| 1.7.2 Trois façons de traiter les groupes . . . . .                                  | 79        |
| 1.7.3 Le modèle à intercept aléatoire . . . . .                                      | 80        |
| 1.7.4 La mise en commun partielle : le rétrécissement . . . . .                      | 81        |
| 1.7.5 Pente aléatoire : chaque relais a sa propre sensibilité au retard . . . . .    | 84        |
| 1.7.6 Pourquoi ignorer les groupes est dangereux : une simulation . . . . .          | 86        |
| 1.8 Exercices corrigés . . . . .                                                     | 88        |
| Énoncés . . . . .                                                                    | 88        |
| Corrigés . . . . .                                                                   | 90        |
| Bilan du chapitre 1 . . . . .                                                        | 95        |
| <b>Chapitre 2 : Modèles linéaires généralisés</b> . . . . .                          | <b>97</b> |
| Le chemin de ce chapitre . . . . .                                                   | 97        |
| 2.1 Le cadre des GLM . . . . .                                                       | 98        |
| 2.1.1 Pourquoi la droite ne suffit plus . . . . .                                    | 98        |
| 2.1.2 Les trois ingrédients d'un GLM . . . . .                                       | 101       |
| 2.1.3 La famille exponentielle . . . . .                                             | 102       |
| 2.1.4 Les fonctions de lien . . . . .                                                | 104       |
| 2.1.5 L'estimation : maximum de vraisemblance et IRLS . . . . .                      | 105       |
| 2.1.6 Les trois tests . . . . .                                                      | 107       |
| 2.2 Régression logistique . . . . .                                                  | 108       |
| 2.2.1 De la probabilité à la cote . . . . .                                          | 108       |
| 2.2.2 Le modèle . . . . .                                                            | 109       |
| 2.2.3 Un premier modèle : l'offre de bienvenue seule . . . . .                       | 110       |
| 2.2.4 Le modèle complet . . . . .                                                    | 111       |
| 2.2.5 L'estimation à la main : IRLS contre statsmodels . . . . .                     | 112       |
| 2.2.6 Probabilités et effets marginaux . . . . .                                     | 113       |
| 2.2.7 Utiliser plus d'information : les notes de l'enquête . . . . .                 | 114       |
| 2.2.8 Évaluer un modèle de classement : matrice de confusion, ROC, AUC, calibration  | 115       |
| 2.2.9 Pièges de la régression logistique . . . . .                                   | 118       |
| 2.3 Régression de Poisson et Gamma . . . . .                                         | 119       |

|                                                                                         |            |
|-----------------------------------------------------------------------------------------|------------|
| 2.3.1 Compter : le modèle de Poisson . . . . .                                          | 119        |
| 2.3.2 Quand les clients n'ont pas été observés aussi longtemps : l'exposition . . . . . | 121        |
| 2.3.3 Quand la variance dépasse la moyenne : la surdispersion . . . . .                 | 122        |
| 2.3.4 Modéliser des montants : la loi Gamma . . . . .                                   | 125        |
| 2.3.5 Comment choisir sa famille ? . . . . .                                            | 128        |
| 2.4 Déviance, qualité d'ajustement, vérification du modèle . . . . .                    | 129        |
| 2.4.1 La déviance . . . . .                                                             | 129        |
| 2.4.2 Comparer des modèles emboîtés : le test du rapport de vraisemblance . . . . .     | 131        |
| 2.4.3 Les résidus d'un GLM . . . . .                                                    | 132        |
| 2.4.4 Mesurer la qualité d'ajustement . . . . .                                         | 135        |
| 2.4.5 Comparer des modèles non emboîtés : AIC et BIC . . . . .                          | 137        |
| 2.4.6 Observations influentes . . . . .                                                 | 138        |
| 2.4.7 Une démarche en quatre temps . . . . .                                            | 139        |
| 2.5 ⊕ Pour aller plus loin : les modèles additifs généralisés (GAM) . . . . .           | 139        |
| 2.5.1 Le problème . . . . .                                                             | 140        |
| 2.5.2 Fonctions de base : un lissage comme une régression . . . . .                     | 140        |
| 2.5.3 La pénalisation : beaucoup de flexibilité, mais disciplinée . . . . .             | 141        |
| 2.5.4 Les GAM avec R et mgcv . . . . .                                                  | 144        |
| 2.5.5 Un GAM à plusieurs termes : l'âge des clients . . . . .                           | 146        |
| 2.5.6 Les pièges des GAM . . . . .                                                      | 146        |
| 2.6 ⊕ Pour aller plus loin : zéros en excès, surdispersion et loi de Tweedie . . . . .  | 147        |
| 2.6.1 Les modèles à zéros en excès : ZIP et modèle de barrière . . . . .                | 147        |
| 2.6.2 Nos comptages ont-ils des zéros « en trop » ? . . . . .                           | 148        |
| 2.6.3 Un montant avec des zéros : la loi de Tweedie . . . . .                           | 150        |
| 2.6.4 Une alternative : le modèle à deux parties . . . . .                              | 152        |
| 2.6.5 Comment choisir ? . . . . .                                                       | 153        |
| 2.7 Exercices corrigés . . . . .                                                        | 154        |
| Énoncés . . . . .                                                                       | 154        |
| Corrigés . . . . .                                                                      | 155        |
| Bilan du chapitre 2 . . . . .                                                           | 161        |
| La vérité dévoilée . . . . .                                                            | 162        |
| <b>Chapitre 3 : Analyse multivariée</b> . . . . .                                       | <b>164</b> |
| Le chemin de ce chapitre . . . . .                                                      | 164        |
| 3.1 Analyse en composantes principales . . . . .                                        | 165        |
| 3.1.1 Le problème : huit colonnes qui se ressemblent . . . . .                          | 165        |
| 3.1.2 Un exemple à la main : cinq clientes, deux variables . . . . .                    | 166        |
| 3.1.3 La théorie : l'ACP est un problème d'optimisation . . . . .                       | 168        |
| 3.1.4 L'ACP par la décomposition en valeurs singulières . . . . .                       | 169        |
| 3.1.5 Faut-il standardiser ? Le piège des unités . . . . .                              | 170        |
| 3.1.6 Lire une ACP : éboulis, saturations, scores . . . . .                             | 171        |
| 3.1.7 Compression et reconstruction . . . . .                                           | 174        |
| 3.1.8 Les pièges de l'ACP . . . . .                                                     | 175        |
| 3.2 Analyse factorielle . . . . .                                                       | 176        |
| 3.2.1 Un exemple à la main : trois thermomètres, un facteur . . . . .                   | 176        |
| 3.2.2 Le modèle général . . . . .                                                       | 177        |
| 3.2.3 Estimer les saturations . . . . .                                                 | 178        |
| 3.2.4 Combien de facteurs ? . . . . .                                                   | 179        |
| 3.2.5 La rotation : rendre les facteurs lisibles . . . . .                              | 180        |
| 3.2.6 ACP contre analyse factorielle : que choisir ? . . . . .                          | 182        |
| 3.2.7 La révélation : qu'avait-on programmé ? . . . . .                                 | 183        |
| 3.2.8 Application : construire deux échelles de mesure . . . . .                        | 184        |



|                                                                                                |            |
|------------------------------------------------------------------------------------------------|------------|
| 3.3 Classification non supervisée . . . . .                                                    | 186        |
| 3.3.1 Ressembler, c'est être proche : la notion de distance . . . . .                          | 186        |
| 3.3.2 Les k-means . . . . .                                                                    | 186        |
| 3.3.3 Combien de groupes ? . . . . .                                                           | 189        |
| 3.3.4 La classification hiérarchique . . . . .                                                 | 191        |
| 3.3.5 Une segmentation honnête des clientes de Dar Jasmin . . . . .                            | 193        |
| 3.3.6 Les pièges de la classification . . . . .                                                | 195        |
| 3.4 ⊕ Pour aller plus loin : l'analyse des correspondances (AC et ACM) . . . . .               | 197        |
| 3.4.1 Le point de départ : un tableau de contingence . . . . .                                 | 197        |
| 3.4.2 La méthode : une SVD sur les résidus standardisés . . . . .                              | 198        |
| 3.4.3 Lire la carte . . . . .                                                                  | 200        |
| 3.4.4 L'analyse des correspondances multiples (ACM) . . . . .                                  | 201        |
| 3.5 ⊕ Pour aller plus loin : l'analyse discriminante . . . . .                                 | 204        |
| 3.5.1 La règle de Bayes avec des groupes gaussiens . . . . .                                   | 204        |
| 3.5.2 Estimer le modèle . . . . .                                                              | 205        |
| 3.5.3 LDA et régression logistique : deux routes vers la même frontière . . . . .              | 208        |
| 3.5.4 Fisher : la projection qui sépare le mieux les groupes . . . . .                         | 209        |
| 3.5.5 Les pièges de l'analyse discriminante . . . . .                                          | 210        |
| 3.6 Exercices corrigés . . . . .                                                               | 211        |
| Énoncés . . . . .                                                                              | 211        |
| Corrigés . . . . .                                                                             | 213        |
| Bilan du chapitre 3 . . . . .                                                                  | 219        |
| <b>Chapitre 4 : Séries temporelles</b> . . . . .                                               | <b>220</b> |
| Le chemin de ce chapitre . . . . .                                                             | 220        |
| 4.1 Stationnarité, autocorrélation, décomposition . . . . .                                    | 221        |
| 4.1.1 Pourquoi l'ordre compte : le test du mélange . . . . .                                   | 221        |
| 4.1.2 Première lecture : tendance, saison, bruit, et pourquoi on prend le logarithme . . . . . | 222        |
| 4.1.3 La stationnarité . . . . .                                                               | 224        |
| 4.1.4 L'autocorrélation : mesurer la mémoire . . . . .                                         | 226        |
| 4.1.5 Le bruit blanc et le test de Ljung-Box . . . . .                                         | 228        |
| 4.1.6 Rendre une série stationnaire : différencier, et tester . . . . .                        | 229        |
| 4.1.7 Décomposer : tendance, saison, reste . . . . .                                           | 232        |
| 4.2 Modèles ARIMA et saisonniers . . . . .                                                     | 235        |
| 4.2.1 Les deux briques : AR et MA . . . . .                                                    | 235        |
| 4.2.2 L'AR(1) et le MA(1) en profondeur . . . . .                                              | 236        |
| 4.2.3 ARMA, ARIMA et SARIMA . . . . .                                                          | 238        |
| 4.2.4 La méthode de Box-Jenkins appliquée aux ventes de Dar Jasmin . . . . .                   | 240        |
| 4.2.5 Variables explicatives : la régression à erreurs ARMA . . . . .                          | 242        |
| 4.2.6 Le diagnostic des résidus . . . . .                                                      | 243        |
| 4.2.7 Deux visions du monde : saison stochastique ou déterministe ? . . . . .                  | 244        |
| 4.2.8 Le même travail en R : <code>auto.arima</code> . . . . .                                 | 245        |
| 4.3 Prédiction et évaluation des prévisions . . . . .                                          | 246        |
| 4.3.1 Comment un modèle prévoit : l'exemple de l'AR(1) . . . . .                               | 246        |
| 4.3.2 Prévoir en logarithme, annoncer en dinars . . . . .                                      | 248        |
| 4.3.3 Le découpage temporel : ne jamais mélanger le passé et l'avenir . . . . .                | 249        |
| 4.3.4 Des références simples, puis des mesures d'erreur . . . . .                              | 250        |
| 4.3.5 Vingt-quatre mois suffisent-ils pour conclure ? . . . . .                                | 252        |
| 4.3.6 La validation à origine glissante . . . . .                                              | 253        |
| 4.3.7 Application : les ventes de 2026 . . . . .                                               | 255        |
| 4.3.8 Révélation : comment les données ont été fabriquées . . . . .                            | 257        |
| 4.4 ⊕ Pour aller plus loin : séries multivariées, cointégration et GARCH . . . . .             | 260        |

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| 4.4.1 Plusieurs séries à la fois : le modèle VAR . . . . .                                | 260        |
| 4.4.2 Cointégration : des séries qui dérivent ensemble . . . . .                          | 263        |
| 4.4.3 Quand la variabilité change : le modèle GARCH . . . . .                             | 265        |
| 4.5 ⊕ Pour aller plus loin : modèles d'espace d'états et filtre de Kalman . . . . .       | 268        |
| 4.5.1 L'idée : un état caché, des observations bruitées . . . . .                         | 268        |
| 4.5.2 Le filtre de Kalman, à la main . . . . .                                            | 268        |
| 4.5.3 Le filtre de Kalman en numpy, et comparaison avec statsmodels . . . . .             | 269        |
| 4.5.4 Régime permanent et lissage exponentiel . . . . .                                   | 271        |
| 4.5.5 Un modèle structurel pour les ventes de Dar Jasmin . . . . .                        | 271        |
| 4.5.6 Combler un trou : les données manquantes . . . . .                                  | 273        |
| 4.6 ⊕ Pour aller plus loin : Prophet et les bibliothèques modernes de prévision . . . . . | 274        |
| 4.6.1 Prophet : l'idée . . . . .                                                          | 274        |
| 4.6.3 Évaluation honnête : tout le monde au même concours . . . . .                       | 277        |
| 4.6.4 Les autres bibliothèques, et comment choisir . . . . .                              | 279        |
| 4.7 Exercices du chapitre 4 . . . . .                                                     | 280        |
| Énoncés . . . . .                                                                         | 280        |
| Corrigés . . . . .                                                                        | 282        |
| Bilan du chapitre 4 . . . . .                                                             | 288        |
| <b>Chapitre 5 : Analyse de survie</b> . . . . .                                           | <b>289</b> |
| Le chemin de ce chapitre . . . . .                                                        | 289        |
| Les données de ce chapitre . . . . .                                                      | 290        |
| 5.1 Censure et fonctions de survie . . . . .                                              | 290        |
| 5.1.1 Un problème que la moyenne ne sait pas résoudre . . . . .                           | 290        |
| 5.1.2 Le vocabulaire . . . . .                                                            | 293        |
| 5.1.3 Trois fonctions pour décrire une durée . . . . .                                    | 293        |
| 5.1.4 Les différentes sortes de censure . . . . .                                         | 295        |
| 5.1.5 La vraisemblance avec censure . . . . .                                             | 297        |
| 5.2 L'estimateur de Kaplan-Meier . . . . .                                                | 298        |
| 5.2.1 Le principe, à la main . . . . .                                                    | 299        |
| 5.2.2 L'estimateur sur les 2 000 clients, comparé aux bibliothèques . . . . .             | 300        |
| 5.2.3 Précision : la variance de Greenwood et les intervalles de confiance . . . . .      | 303        |
| 5.2.4 Médiane, quantiles et durée moyenne « restreinte » . . . . .                        | 304        |
| 5.2.5 Comparer des groupes : courbes et test du log-rank . . . . .                        | 305        |
| 5.2.6 Quand les clients entrent tard : la troncature à gauche . . . . .                   | 309        |
| 5.2.7 Pièges et analyses de sensibilité . . . . .                                         | 310        |
| 5.3 Le modèle de Cox à risques proportionnels . . . . .                                   | 311        |
| 5.3.1 Le modèle . . . . .                                                                 | 311        |
| 5.3.2 La vraisemblance partielle : estimer $\beta$ sans connaître $h_0$ . . . . .         | 312        |
| 5.3.3 Ajuster le modèle sur les 2 000 clients . . . . .                                   | 314        |
| 5.3.4 Les ex aequo : Breslow, Efron et les autres . . . . .                               | 317        |
| 5.3.5 Vérifier l'hypothèse : les risques sont-ils vraiment proportionnels ? . . . . .     | 318        |
| 5.3.6 Variables qui changent avec le temps, et le piège de l'« immortalité » . . . . .    | 321        |
| 5.3.7 Prédire : la courbe de survie d'un client donné . . . . .                           | 322        |
| 5.3.8 Le modèle sépare-t-il bien les clients ? L'indice de concordance . . . . .          | 323        |
| 5.3.9 Un second jeu de données : la récurrence (Rossi) . . . . .                          | 325        |
| 5.4 Modèles de durée paramétriques . . . . .                                              | 326        |
| 5.4.1 Le modèle à temps de vie accéléré . . . . .                                         | 326        |
| 5.4.2 La Weibull : à la fois « risques proportionnels » et « temps accéléré » . . . . .   | 327        |
| 5.4.3 Le maximum de vraisemblance, écrit à la main . . . . .                              | 327        |
| 5.4.4 Choisir entre plusieurs lois . . . . .                                              | 329        |
| 5.4.5 Extrapoler et chiffrer : la valeur vie client . . . . .                             | 332        |

|                                                                                                   |            |
|---------------------------------------------------------------------------------------------------|------------|
| 5.4.6 Une variable manquante : le service . . . . .                                               | 334        |
| 5.4.7 Révéler la vérité . . . . .                                                                 | 334        |
| 5.5 ⊕ Pour aller plus loin : les risques concurrents . . . . .                                    | 336        |
| 5.5.1 Deux façons de sortir . . . . .                                                             | 336        |
| 5.5.2 Pourquoi « 1 – Kaplan-Meier » trompe . . . . .                                              | 337        |
| 5.5.3 Une simulation avec une vérité connue . . . . .                                             | 338        |
| 5.5.4 Modéliser : deux questions, deux modèles . . . . .                                          | 341        |
| 5.5.5 Pièges . . . . .                                                                            | 342        |
| 5.6 Exercices corrigés . . . . .                                                                  | 343        |
| Corrigés . . . . .                                                                                | 344        |
| Bilan du chapitre 5 . . . . .                                                                     | 349        |
| <b>Chapitre 6 : Statistique bayésienne et simulation</b>                                          | <b>351</b> |
| Le chemin de ce chapitre . . . . .                                                                | 351        |
| 6.1 Inférence bayésienne et lois a priori . . . . .                                               | 352        |
| 6.1.1 La formule de Bayes, version « paramètre » . . . . .                                        | 352        |
| 6.1.2 Un exemple minuscule, calculé à la main . . . . .                                           | 353        |
| 6.1.3 La loi Bêta : le coup de génie de la conjugaison . . . . .                                  | 354        |
| 6.1.4 Application : l'offre de bienvenue fonctionne-t-elle ? . . . . .                            | 356        |
| 6.1.5 Intervalle de crédibilité : équi-queue ou HPD ? . . . . .                                   | 358        |
| 6.1.6 Le choix de l'a priori : liberté, responsabilité, et sensibilité . . . . .                  | 358        |
| 6.1.7 Intervalle de crédibilité contre intervalle de confiance : une expérience . . . . .         | 360        |
| 6.1.8 Autres couples conjugués : Poisson-Gamma et Normal-Normal . . . . .                         | 361        |
| 6.1.9 La loi prédictive : prédire du nouveau . . . . .                                            | 363        |
| 6.2 Méthodes de Monte-Carlo . . . . .                                                             | 364        |
| 6.2.1 Le principe : une espérance se calcule en moyennant des simulations . . . . .               | 364        |
| 6.2.2 La vitesse de convergence : $1/\sqrt{n}$ . . . . .                                          | 365        |
| 6.2.3 Application : faut-il envoyer l'offre de bienvenue à tout le monde ? . . . . .              | 367        |
| 6.2.4 Fabriquer des tirages : l'inversion et le rejet . . . . .                                   | 369        |
| 6.2.5 L'échantillonnage préférentiel ( <i>importance sampling</i> ) . . . . .                     | 370        |
| 6.2.6 Réduire la variance : faire mieux avec les mêmes tirages . . . . .                          | 372        |
| 6.2.7 Le bootstrap et le bootstrap bayésien : deux Monte-Carlo que vous connaissez déjà . . . . . | 373        |
| 6.3 MCMC : Metropolis-Hastings et échantillonnage de Gibbs . . . . .                              | 374        |
| 6.3.1 Pourquoi les chaînes de Markov ? . . . . .                                                  | 374        |
| 6.3.2 Les chaînes de Markov : loi stationnaire et réversibilité . . . . .                         | 375        |
| 6.3.3 L'algorithme de Metropolis-Hastings . . . . .                                               | 376        |
| 6.3.4 Régler l'algorithme : le pas de la marche aléatoire . . . . .                               | 378        |
| 6.3.5 Application : la régression logistique bayésienne du rachat . . . . .                       | 380        |
| 6.3.6 L'échantillonnage de Gibbs . . . . .                                                        | 384        |
| 6.3.7 En pratique : PyMC, Stan et l'HMC (non exécuté) . . . . .                                   | 386        |
| 6.4 Vérification des modèles bayésiens . . . . .                                                  | 387        |
| 6.4.1 L'algorithme a-t-il convergé ? Les diagnostics de convergence . . . . .                     | 387        |
| 6.4.2 Le modèle est-il plausible ? Les vérifications prédictives a posteriori . . . . .           | 390        |
| 6.4.3 L'a priori est-il raisonnable ? La vérification prédictive <i>a priori</i> . . . . .        | 393        |
| 6.4.4 Comparer des modèles : facteur de Bayes et WAIC . . . . .                                   | 395        |
| 6.4.5 Le flux de travail bayésien, en une page . . . . .                                          | 398        |
| 6.5 ⊕ Pour aller plus loin : la théorie des valeurs extrêmes . . . . .                            | 398        |
| 6.5.1 Le problème : la queue d'une loi normale est trompeuse . . . . .                            | 399        |
| 6.5.2 Le théorème fondamental : la loi du maximum . . . . .                                       | 399        |
| 6.5.3 Les données : des retards de livraison simulés, à queue lourde . . . . .                    | 400        |
| 6.5.4 Première approche : les maxima par blocs . . . . .                                          | 401        |

|                   |                                                                                        |            |
|-------------------|----------------------------------------------------------------------------------------|------------|
| 6.5.5             | Seconde approche : les excès au-dessus d'un seuil (POT)                                | 403        |
| 6.6               | ⊕ Pour aller plus loin : les copules et la modélisation de la dépendance               | 406        |
| 6.6.1             | La corrélation ne dit pas tout                                                         | 407        |
| 6.6.2             | Le théorème de Sklar                                                                   | 407        |
| 6.6.3             | La copule gaussienne                                                                   | 408        |
| 6.6.4             | La copule de Clayton : la dépendance dans les queues                                   | 408        |
| 6.6.5             | Deux copules, même tau, comportements extrêmes opposés                                 | 409        |
| 6.6.6             | Estimer une copule à partir de données                                                 | 411        |
| 6.6.7             | Applications                                                                           | 412        |
| 6.7               | Exercices du chapitre 6                                                                | 415        |
|                   | Énoncés                                                                                | 415        |
|                   | Corrigés                                                                               | 416        |
|                   | Bilan du chapitre 6                                                                    | 422        |
| <b>Chapitre 7</b> | <b>⊕ Inférence causale</b>                                                             | <b>424</b> |
|                   | Le chemin de ce chapitre                                                               | 424        |
| 7.1               | Raisonnement en causes : résultats potentiels, expérience randomisée, DAG et confusion | 425        |
| 7.1.1             | Une conclusion trop rapide                                                             | 425        |
| 7.1.2             | Les résultats potentiels                                                               | 426        |
| 7.1.3             | Pourquoi la randomisation résout tout                                                  | 428        |
| 7.1.4             | Une vraie expérience : l'offre de bienvenue                                            | 430        |
| 7.1.5             | Quand on ne peut pas tirer au sort : les graphes causaux                               | 433        |
| 7.1.6             | La fourche : confusion et paradoxe de Simpson                                          | 435        |
| 7.1.7             | La chaîne : ne pas ajuster sur un médiateur                                            | 436        |
| 7.1.8             | La collision : ne pas conditionner sur un effet commun                                 | 437        |
| 7.1.9             | Le critère de la porte dérobée                                                         | 439        |
| 7.2               | Scores de propension : comparer ce qui est comparable                                  | 441        |
| 7.2.1             | Pourquoi un score ? La malédiction de la dimension                                     | 441        |
| 7.2.2             | Estimer le score et vérifier le chevauchement                                          | 442        |
| 7.2.3             | L'appariement                                                                          | 444        |
| 7.2.4             | La pondération par l'inverse du score (IPW)                                            | 445        |
| 7.2.5             | Le meilleur des deux mondes : l'estimateur doublement robuste                          | 448        |
| 7.2.6             | Ce que le score de propension ne fait pas                                              | 451        |
| 7.3               | Différences de différences : l'évolution des uns contre l'évolution des autres         | 452        |
| 7.3.1             | Les données, et le calcul à la main                                                    | 453        |
| 7.3.2             | À quelle échelle les tendances sont-elles parallèles ?                                 | 454        |
| 7.3.3             | La régression à effets fixes (TWFE)                                                    | 456        |
| 7.3.4             | L'étude d'événement : regarder les tendances avant et après                            | 457        |
| 7.3.5             | Quand les tendances ne sont pas parallèles                                             | 458        |
| 7.3.6             | Ce que la DiD ne règle pas                                                             | 461        |
| 7.4               | Variables instrumentales : un coup de pouce aléatoire                                  | 461        |
| 7.4.1             | Le problème : suivre le compte, cause ou symptôme ?                                    | 462        |
| 7.4.2             | Les conditions d'un bon instrument                                                     | 463        |
| 7.4.3             | L'estimateur de Wald : un rapport de deux différences                                  | 464        |
| 7.4.4             | Les moindres carrés en deux étapes (2SLS)                                              | 465        |
| 7.4.5             | Ce que l'on estime vraiment : l'effet pour les « complaisants »                        | 467        |
| 7.4.6             | Quand l'instrument est faible                                                          | 468        |
| 7.4.7             | Quand l'exclusion est fautive                                                          | 470        |
| 7.4.8             | Quelle méthode, quand ? Panorama et limites                                            | 470        |
| 7.5               | Exercices corrigés                                                                     | 472        |
|                   | Énoncés                                                                                | 472        |
|                   | Corrigés                                                                               | 473        |

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| Bilan du chapitre 7 . . . . .                                                             | 478        |
| <b>Chapitre 8 : ☉ Plans d'expériences</b>                                                 | <b>480</b> |
| Le chemin de ce chapitre . . . . .                                                        | 480        |
| 8.1 Les principes d'un bon plan d'expériences . . . . .                                   | 481        |
| 8.1.1 Le vocabulaire . . . . .                                                            | 481        |
| 8.1.2 Première règle : randomiser . . . . .                                               | 481        |
| 8.1.3 Deuxième règle : répéter . . . . .                                                  | 482        |
| 8.1.4 Troisième règle : bloquer . . . . .                                                 | 483        |
| 8.1.5 Faire varier un seul facteur à la fois : une fausse bonne idée . . . . .            | 483        |
| 8.2 L'analyse de la variance (ANOVA) . . . . .                                            | 485        |
| 8.2.1 Le problème : quatre agencements de vitrine . . . . .                               | 485        |
| 8.2.2 Décomposer la variabilité : un exemple de neuf nombres . . . . .                    | 486        |
| 8.2.3 Le modèle et le test F . . . . .                                                    | 488        |
| 8.2.4 L'ANOVA, le test de Student et la régression sont le même outil . . . . .           | 489        |
| 8.2.5 Vérifier les hypothèses . . . . .                                                   | 490        |
| 8.2.6 Quels groupes diffèrent ? Les comparaisons multiples . . . . .                      | 491        |
| 8.2.7 La taille de l'effet . . . . .                                                      | 493        |
| 8.2.8 Les blocs : retirer du bruit connu . . . . .                                        | 494        |
| 8.2.9 Deux facteurs et leur interaction . . . . .                                         | 496        |
| 8.2.10 Combien d'observations ? La puissance d'une ANOVA . . . . .                        | 499        |
| 8.2.11 La même chose en R . . . . .                                                       | 500        |
| 8.2.12 Ce que cachaient les données . . . . .                                             | 501        |
| 8.3 Les plans factoriels . . . . .                                                        | 502        |
| 8.3.1 L'expérience de Yasmine : trois facteurs, huit combinaisons . . . . .               | 502        |
| 8.3.2 Les effets, à la main . . . . .                                                     | 503        |
| 8.3.3 Le lien avec la régression, et la précision des effets . . . . .                    | 505        |
| 8.3.4 Lire l'expérience : quels effets comptent ? . . . . .                               | 507        |
| 8.3.5 Ce que l'expérience ne détecte pas : la leçon de la puissance . . . . .             | 509        |
| 8.3.6 Sans répétition : le plan $2^4$ et le diagramme demi-normal . . . . .               | 510        |
| 8.3.7 Ce que cachaient les données . . . . .                                              | 512        |
| 8.4 Plans fractionnaires et surfaces de réponse . . . . .                                 | 513        |
| 8.4.1 Le problème : $2^k$ explose . . . . .                                               | 513        |
| 8.4.2 Une demi-fraction : le plan $2^{4-1}$ . . . . .                                     | 513        |
| 8.4.3 Résolution : mesurer la qualité d'une fraction . . . . .                            | 515        |
| 8.4.4 Plus fractionnaire encore : le plan $2^{5-2}$ et le piège de la confusion . . . . . | 515        |
| 8.4.5 Optimiser un réglage : les surfaces de réponse . . . . .                            | 517        |
| 8.4.6 Ajuster le modèle quadratique . . . . .                                             | 519        |
| 8.4.7 Trouver l'optimum : point stationnaire et analyse canonique . . . . .               | 520        |
| 8.4.8 ☉ Pour aller plus loin : les plans optimaux . . . . .                               | 522        |
| 8.4.9 Ce que cachaient les données . . . . .                                              | 524        |
| 8.5 Exercices du chapitre 8 . . . . .                                                     | 524        |
| Énoncés . . . . .                                                                         | 524        |
| Corrigés . . . . .                                                                        | 526        |
| Bilan du chapitre 8 . . . . .                                                             | 530        |
| <b>Chapitre 9 : ☉ Statistique spatiale</b>                                                | <b>532</b> |
| Le chemin de ce chapitre . . . . .                                                        | 532        |
| 9.1 Données spatiales et cartes . . . . .                                                 | 533        |
| 9.1.1 Pourquoi l'espace change tout . . . . .                                             | 533        |
| 9.1.2 Trois types de données spatiales . . . . .                                          | 534        |
| 9.1.3 Repérer un point : coordonnées et distances . . . . .                               | 534        |

|                                                                                                |            |
|------------------------------------------------------------------------------------------------|------------|
| 9.1.4 Une carte sans fond de carte . . . . .                                                   | 537        |
| 9.2 Autocorrélation spatiale . . . . .                                                         | 540        |
| 9.2.1 Définir « voisin » : la matrice de poids . . . . .                                       | 540        |
| 9.2.2 L'indice de Moran . . . . .                                                              | 542        |
| 9.2.3 L'indice de Geary . . . . .                                                              | 547        |
| 9.2.4 Où sont les îlots ? Les indices locaux (LISA) . . . . .                                  | 548        |
| 9.2.5 Pourquoi cela compte : la régression qui voit des relations qui n'existent pas . . . . . | 550        |
| 9.3 Variogramme et krigeage . . . . .                                                          | 552        |
| 9.3.1 De la corrélation entre voisines à une fonction de la distance . . . . .                 | 552        |
| 9.3.2 Le variogramme empirique . . . . .                                                       | 553        |
| 9.3.3 Modèles de variogramme et ajustement . . . . .                                           | 555        |
| 9.3.4 Le krigeage : la meilleure moyenne pondérée . . . . .                                    | 558        |
| 9.3.5 Une carte de prédiction et une carte d'incertitude . . . . .                             | 560        |
| 9.3.6 Évaluer honnêtement : la validation croisée . . . . .                                    | 562        |
| 9.3.7 Les limites : tendance, anisotropie, extrapolation . . . . .                             | 563        |
| 9.4 Processus ponctuels . . . . .                                                              | 566        |
| 9.4.1 Le hasard complet : le processus de Poisson spatial . . . . .                            | 566        |
| 9.4.2 Le test des quadrats . . . . .                                                           | 568        |
| 9.4.3 Le plus proche voisin : l'indice de Clark-Evans . . . . .                                | 569        |
| 9.4.4 La fonction K de Ripley . . . . .                                                        | 572        |
| 9.4.5 Attention : agrégat ou densité variable ? . . . . .                                      | 575        |
| 9.5 Exercices du chapitre 9 . . . . .                                                          | 577        |
| Énoncés . . . . .                                                                              | 577        |
| Corrigés . . . . .                                                                             | 578        |
| Bilan du chapitre 9 . . . . .                                                                  | 584        |
| <b>Projet du volume : le plan 2026 de Dar Jasmin</b> . . . . .                                 | <b>585</b> |
| P.1 Le cahier des charges . . . . .                                                            | 585        |
| P.2 Étape 1 : charger et contrôler . . . . .                                                   | 586        |
| P.3 Étape 2 : prévoir les ventes de 2026 (question 1) . . . . .                                | 587        |
| P.3.1 Regarder, puis fixer des repères . . . . .                                               | 587        |
| P.3.2 Ajuster et comparer des modèles SARIMA . . . . .                                         | 588        |
| P.3.3 Prévoir 2026 . . . . .                                                                   | 589        |
| P.4 Étape 3 : l'offre de bienvenue fait-elle revenir les clients ? (question 2) . . . . .      | 590        |
| P.5 Étape 4 : combien rapporte un client ? (question 3) . . . . .                              | 591        |
| P.6 Étape 5 : combien de temps un client reste-t-il ? (question 4) . . . . .                   | 593        |
| P.7 Étape 6 : que vaut un client, et l'offre est-elle rentable ? . . . . .                     | 594        |
| P.8 Le rapport pour Yasmine, et ce qui avait été programmé . . . . .                           | 595        |
| Ce qui avait été programmé . . . . .                                                           | 596        |
| P.9 Un détour par de vraies données . . . . .                                                  | 597        |
| P.10 Limites, et la suite . . . . .                                                            | 598        |
| <b>Points clés et auto-évaluation</b> . . . . .                                                | <b>599</b> |
| Les points clés, chapitre par chapitre . . . . .                                               | 599        |
| Quarante-deux questions . . . . .                                                              | 600        |
| Régression linéaire (chapitre 1) . . . . .                                                     | 600        |
| Modèles linéaires généralisés (chapitre 2) . . . . .                                           | 601        |
| Analyse multivariée (chapitre 3) . . . . .                                                     | 601        |
| Séries temporelles (chapitre 4) . . . . .                                                      | 601        |
| Analyse de survie (chapitre 5) . . . . .                                                       | 601        |
| Statistique bayésienne et simulation (chapitre 6) . . . . .                                    | 602        |
| Chapitres facultatifs (7, 8, 9) . . . . .                                                      | 602        |

Vérifier les réponses chiffrées . . . . . 602

Corrigé . . . . . 603

Votre grille d'auto-évaluation . . . . . 606

Et maintenant ? . . . . . 607





# Introduction : de la description à la modélisation

« Tous les modèles sont faux, mais certains sont utiles. » *George Box*

## Là où le volume I nous a laissés

Le volume I vous a donné les gestes de base : **calculer** (mathématiques), **raisonner sous incertitude** (probabilités), **tirer des conclusions honnêtes d'un échantillon** (statistique), **coder** (Python, R), **interroger des données** (SQL) et **travailler proprement** (Git, notebooks, projet reproductible).

À la fin de ce volume, vous saviez répondre à des questions du type : « Les paniers Instagram sont-ils plus petits que ceux de la boutique ? » (un test de Welch), ou « Chaque jour de retard fait-il baisser la satisfaction, et de combien ? » (une pente, avec son intervalle de confiance). Ce sont des questions à **une ou deux variables**.

La vraie vie en pose d'autres :

- « **Toutes choses égales par ailleurs**, combien le canal d'acquisition change-t-il le panier d'un client, une fois qu'on tient compte de son âge ? »
- « Quelle est la **probabilité** qu'un client rachète dans les douze mois, selon son profil ? »
- « Quelles seront mes **ventes** des douze prochains mois ? »
- « **Combien de temps** un client reste-t-il, sachant que certains sont encore là aujourd'hui, donc que leur durée est inconnue ? »

Pour répondre, il ne suffit plus de comparer des moyennes. Il faut un **modèle** : une description mathématique simplifiée du mécanisme qui a produit les données, assez simple pour être comprise, assez riche pour être utile. Ce volume est consacré à la construction, à l'évaluation et à l'interprétation honnête de tels modèles.

**Intuition.** Un modèle est une **carte**. Une carte est utile *parce qu'elle simplifie* : une carte à l'échelle 1:1 serait inutilisable. Mais une carte qui oublie un fleuve vous fait tomber à l'eau. Tout le métier du modélisateur tient en deux questions : *qu'a-t-on le droit de simplifier ?* et *comment vérifier qu'on n'a pas oublié le fleuve ?*

## À qui s'adresse ce volume ?

Aux lecteurs qui ont lu le volume I, ou qui ont un bagage équivalent : algèbre linéaire de base (produit de matrices, valeurs propres), dérivées, probabilités usuelles, intervalles de confiance et tests, et un peu de Python. Chaque fois qu'un résultat du volume I est utilisé, nous le disons (« volume I, section 3.4 »), pour que vous puissiez aller le relire.

| Ce que vous devez maîtriser                                  | Où le revoir dans le volume I |
|--------------------------------------------------------------|-------------------------------|
| Produit matriciel, inverse, valeurs et vecteurs propres, SVD | section 1.1                   |
| Dérivées, gradient, descente de gradient                     | sections 1.2 et 1.3           |
| Lois usuelles, espérance, variance, covariance               | sections 2.2 et 2.3           |
| Loi des grands nombres, théorème central limite              | section 2.4                   |
| Estimation, maximum de vraisemblance                         | section 3.2                   |
| Intervalle de confiance, bootstrap                           | section 3.3                   |
| Tests, p-valeurs, tests multiples                            | sections 3.4 et 3.5           |
| pandas, graphiques                                           | sections 4.4 et 4.5           |

**Les chapitres complémentaires.** Les chapitres 7 (inférence causale), 8 (plans d'expériences) et 9 (statistique spatiale), ainsi que toutes les sections marquées  $\oplus$ , sont **facultatifs**. On peut lire le volume sans eux. Ils sont là pour celles et ceux qui veulent aller plus loin.

## La carte du volume

Le volume contient six chapitres principaux, qui forment trois blocs, et trois chapitres complémentaires.

| BLOC A : relier des variables<br>↪ incertitude                                                                                                                                                                 | BLOC B : structures des données                                                                                                                                                                                                 | BLOC C :         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| ↪ _____<br>Chapitre 1 : Régression linéaire<br>↪ Statistique<br>  (réponse continue)<br>↪ bayésienne et simulation<br>▼<br>Chapitre 2 : Modèles linéaires généralisés<br>(réponse binaire, comptage, positive) | Chapitre 3 : Analyse multivariée<br>(beaucoup de variables,<br>sans réponse à prédire)<br>Chapitre 4 : Séries temporelles<br>(observations ordonnées dans le temps)<br>Chapitre 5 : Analyse de survie<br>(durées, avec censure) | Chapitre 6 :     |
| (+) Chapitre 7 : Inférence causale<br>↪ Statistique spatiale                                                                                                                                                   | (+) Chapitre 8 : Plans d'expériences                                                                                                                                                                                            | (+) Chapitre 9 : |
| Projet du volume : une étude de modélisation complète pour Dar Jasmin                                                                                                                                          |                                                                                                                                                                                                                                 |                  |

| Chapitre                                | Question centrale                                                              | Vous saurez...                                                                |
|-----------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>1. Régression linéaire</b>           | Comment expliquer une quantité continue par d'autres variables ?               | estimer, interpréter, tester et vérifier un modèle linéaire                   |
| <b>2. Modèles linéaires généralisés</b> | Et si la réponse est un oui/non, un comptage, un montant strictement positif ? | choisir une loi et un lien, ajuster et diagnostiquer un GLM                   |
| <b>3. Analyse multivariée</b>           | Comment résumer et regrouper des dizaines de variables ?                       | réduire la dimension (ACP, analyse factorielle) et segmenter (classification) |
| <b>4. Séries temporelles</b>            | Comment modéliser et prévoir ce qui évolue dans le temps ?                     | décomposer, ajuster un ARIMA saisonnier et évaluer des prévisions             |

| Chapitre                  | Question centrale                                                            | Vous saurez...                                                              |
|---------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 5. Analyse de survie      | Comment traiter des durées dont certaines ne sont pas terminées ?            | estimer des courbes de survie et mesurer l'effet de variables sur le risque |
| 6. Statistique bayésienne | Comment exprimer l'incertitude sur un paramètre par une loi de probabilité ? | raisonner à la Bayes et écrire un MCMC                                      |
| Projet                    | Peut-on tout assembler ?                                                     | mener une étude complète : GLM, séries temporelles, survie                  |

## Cinq idées qui reviennent dans tout le volume

Avant de commencer, voici cinq idées qui traversent chaque chapitre. Elles valent mieux que n'importe quelle formule.

1. **Un modèle = une partie « signal » + une partie « bruit ».** Chaque chapitre propose une façon différente, mais toujours explicite, de dire quelle part est le signal et à quoi ressemble le bruit.
2. **Estimer, c'est choisir les paramètres qui rendent les données les plus plausibles.** Presque tous les modèles de ce volume (régression, GLM, ARIMA, survie) sont ajustés par **maximum de vraisemblance** (volume I, section 3.2) ; la régression linéaire en est le cas particulier le plus simple, et le bayésien en est le prolongement.
3. **Un modèle s'évalue sur des données qu'il n'a pas vues.** Mesurer la qualité d'un modèle sur les données qui ont servi à l'ajuster est un auto-compliment. Validation hors échantillon, validation croisée, rétro-test pour les séries : le même principe sous trois formes.
4. **Les hypothèses se vérifient.** Chaque modèle suppose quelque chose (linéarité, indépendance, stationnarité, risques proportionnels...). Chaque chapitre contient des **diagnostics**, c'est-à-dire des moyens de regarder si l'hypothèse tient.
5. **Association n'est pas causalité.** Un modèle prédictif décrit comment les variables varient ensemble ; il ne dit pas ce qui arriverait si on intervenait. La différence est subtile et essentielle : elle fait l'objet du chapitre 7, et elle est signalée chaque fois qu'elle compte.

## Comment travailler avec ce volume

Le rythme est celui du volume I : pour chaque notion, une **intuition**, un **exemple minuscule calculé à la main**, la **démonstration** quand elle éclaire, puis l'**application en code** sur des données, suivie de sa sortie réelle. Les encadrés gardent les mêmes pictogrammes :

| Encadré                 | Rôle                                                                |
|-------------------------|---------------------------------------------------------------------|
| ■ Intuition             | l'idée en langage courant                                           |
| ■ Démonstration         | le raisonnement rigoureux (on peut le sauter à la première lecture) |
| ■ Application           | une petite étude réelle avec du code                                |
| △ Piège                 | l'erreur classique et comment l'éviter                              |
| ■ Expérience / remarque | une simulation ou un commentaire                                    |
| ✓ À retenir             | le résumé de la section                                             |
| ■ Repère                | un guide de lecture, ou une section optionnelle                     |

Une particularité importante de ce volume : **les données sont simulées**. C'est un choix délibéré. Quand on simule, on connaît la **vérité** (les vrais coefficients, la vraie forme de la saisonnalité...), ce qui permet de vérifier que la méthode la retrouve, ou de comprendre pourquoi elle la retrouve mal. À la fin de chaque étude importante, nous dévoilons ce qui avait été programmé. Dans la vie réelle, on n'a jamais cette chance : c'est précisément pourquoi il faut s'entraîner ici. Quand nous utilisons des données réelles (certains jeux embarqués dans les bibliothèques), nous le disons.

**Simulé ne veut pas dire facile.** Un jeu simulé est plus *propre* que la réalité : pas de valeurs saisies de travers, pas de variables oubliées par le service de collecte. Lorsque vous appliquerez ces méthodes à de vrais jeux, comptez sur une part de travail supplémentaire pour le nettoyage et la vérification (volume I, projet de clôture, étape 1).

# Les données et l'environnement du volume

## Dar Jasmin, de 2016 à 2025

Au volume I, Dar Jasmin était une petite boutique observée sur une année. Ici, nous suivons **dix ans d'activité** : la boutique ouverte à Tunis en 2016, le site web, puis la page Instagram. Yasmine a fait croître son affaire, a vécu un arrêt brutal au printemps 2020, et a lancé une **offre de bienvenue** tirée au sort pour les nouveaux clients.

Trois jeux de données, fournis dans le dossier `donnees/`, servent à presque tous les chapitres. Ils sont produits par le script `build/donnees2.py` avec des graines fixes : vous obtiendrez exactement les mêmes chiffres que dans le livre.

| Fichier                               | Contenu                                                                                                                             | Lignes  | Utilisé surtout dans    |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------|-------------------------|
| <code>clients.csv</code>              | un client par ligne : âge, ville, canal d'acquisition, offre de bienvenue, commandes, panier, dépense, rachat, durée de la relation | 2 000   | chapitres 1, 2, 5, 6, 7 |
| <code>enquete_satisfaction.csv</code> | huit questions de satisfaction (notes de 1 à 5) pour les clients qui ont répondu                                                    | ≈ 1 200 | chapitre 3              |
| <code>ventes_mensuelles.csv</code>    | chiffre d'affaires et nombre de commandes par mois, 2016 à 2025                                                                     | 120     | chapitre 4              |

Chargeons-les et regardons-les, avec les trois gestes habituels : la forme, les types, les valeurs manquantes.

```
import numpy as np
import pandas as pd

clients = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
ventes = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"])

for nom, tab in [("clients", clients), ("enquete", enquete), ("ventes", ventes)]:
    print(f"{nom:8s} {tab.shape[0]:5d} lignes, {tab.shape[1]:2d} colonnes,
          ↳ {int(tab.isna().sum().sum())} valeur(s) manquante(s)")
print()
print(clients.head(5).to_string(index=False))
```

|                |                                                    |            |                   |                  |                 |                 |  |
|----------------|----------------------------------------------------|------------|-------------------|------------------|-----------------|-----------------|--|
| clients        | 2000 lignes, 12 colonnes, 0 valeur(s) manquante(s) |            |                   |                  |                 |                 |  |
| enquete        | 1212 lignes, 9 colonnes, 0 valeur(s) manquante(s)  |            |                   |                  |                 |                 |  |
| ventes         | 120 lignes, 5 colonnes, 0 valeur(s) manquante(s)   |            |                   |                  |                 |                 |  |
| id_client      | age                                                | ville      | canal_acquisition | date_inscription | offre_bienvenue | nb_commandes_an |  |
| ↪ panier_moyen | depense_annuelle                                   | rachat_12m | duree_mois        | churn            |                 |                 |  |
| ↪ 1            | 19                                                 | Autre      | Instagram         | 2020-03-11       | 0               | 4               |  |
| ↪ 40.95        |                                                    | 116.50     | 0                 | 15.89            | 1               | 2               |  |
| ↪ 2            | 43                                                 | Tunis      | Site              | 2019-01-10       |                 |                 |  |
| ↪ 78.34        |                                                    | 148.92     | 1                 | 33.68            | 0               |                 |  |
| ↪ 3            | 35                                                 | Bizerte    | Boutique          | 2020-07-02       | 1               | 6               |  |
| ↪ 73.73        |                                                    | 509.05     | 0                 | 23.22            | 0               |                 |  |
| ↪ 4            | 42                                                 | Sousse     | Instagram         | 2024-08-01       | 0               | 0               |  |
| ↪ 0.00         |                                                    | 0.00       | 0                 | 0.96             | 0               |                 |  |
| ↪ 5            | 21                                                 | Sfax       | Instagram         | 2024-12-16       | 0               | 7               |  |
| ↪ 46.57        |                                                    | 321.77     | 0                 | 12.48            | 0               |                 |  |

Le tableau des clients

Voici les variables de clients.csv, avec leur type statistique, car c'est lui qui décidera du modèle (chapitres 1 et 2) :

| Variable          | Type                  | Signification                                                                                              |
|-------------------|-----------------------|------------------------------------------------------------------------------------------------------------|
| age               | quantitative          | âge à l'inscription (18 à 75 ans)                                                                          |
| ville             | qualitative nominale  | Tunis, Sousse, Sfax, Nabeul, Bizerte, Autre                                                                |
| canal_acquisition | qualitative nominale  | canal par lequel le client est arrivé : Instagram, Site, Boutique                                          |
| date_inscription  | date                  | entre janvier 2019 et juin 2025                                                                            |
| offre_bienvenue   | binaire (0/1)         | 1 si le client a reçu une offre de bienvenue ; <b>attribuée au hasard</b>                                  |
| nb_commandes_an   | comptage (0, 1, 2...) | nombre de commandes sur l'année                                                                            |
| panier_moyen      | quantitative positive | montant moyen d'une commande en DT (0 si aucune commande)                                                  |
| depense_annuelle  | quantitative ≥ 0      | dépense totale de l'année en DT : beaucoup de zéros, très asymétrique                                      |
| rachat_12m        | binaire (0/1)         | le client a-t-il racheté dans les douze mois ?                                                             |
| duree_mois        | quantitative positive | durée de la relation, en mois, <b>jusqu'au départ ou jusqu'à la fin de l'observation</b>                   |
| churn             | binaire (0/1)         | 1 : le départ a été observé ; 0 : le client est encore là au 31/12/2025, donc sa durée est <b>censurée</b> |

Cette dernière paire de colonnes, duree\_mois et churn, mérite un instant. Un client inscrit en 2024 et toujours actif fin 2025 a une durée de relation d'au moins dix-huit mois, mais nous ne connaissons pas sa durée finale. Ignorer ces clients, ou les traiter comme s'ils étaient partis, fausserait toute analyse : c'est le sujet du chapitre 5.

```

resume = clients[["age", "nb_commandes_an", "panier_moyen", "depense_annuelle",
↳ "duree_mois"]].describe().round(1)
print(resume.to_string())
print()
print("part des clients sans commande :", round((clients["nb_commandes_an"] == 0).mean(), 3))
print("part des clients qui ont racheté :", round(clients["rachat_12m"].mean(), 3))
print("départs observés / censurés      :", int(clients["churn"].sum()), "/", int((1 -
↳ clients["churn"]).sum()))
print()
print(clients["canal_acquisition"].value_counts().to_string())

```

|       | age    | nb_commandes_an | panier_moyen | depense_annuelle | duree_mois |
|-------|--------|-----------------|--------------|------------------|------------|
| count | 2000.0 | 2000.0          | 2000.0       | 2000.0           | 2000.0     |
| mean  | 35.8   | 3.9             | 53.3         | 247.0            | 23.4       |
| std   | 10.5   | 3.6             | 32.1         | 297.2            | 17.2       |
| min   | 18.0   | 0.0             | 0.0          | 0.0              | 0.0        |
| 25%   | 28.0   | 1.0             | 36.2         | 62.0             | 10.0       |
| 50%   | 35.0   | 3.0             | 52.0         | 156.5            | 19.5       |
| 75%   | 43.0   | 5.0             | 70.2         | 326.8            | 32.8       |
| max   | 73.0   | 29.0            | 245.1        | 2576.3           | 82.7       |

```

part des clients sans commande : 0.13
part des clients qui ont racheté : 0.509
départs observés / censurés      : 977 / 1023

```

```

canal_acquisition
Instagram      816
Site           680
Boutique       504

```

Quelques constats qui guideront les choix de modèles : environ un client sur huit n'a passé aucune commande (les dépenses ont donc beaucoup de zéros) ; les dépenses sont très asymétriques (la moyenne dépasse nettement la médiane, comme au volume I) ; un peu plus de la moitié des durées de relation (1 023 sur 2 000) sont censurées : ces clients sont encore là, leur durée finale est inconnue.

L'offre de bienvenue est la variable la plus précieuse du jeu pour la **causalité** : elle a été attribuée par tirage au sort (une pièce lancée pour chaque nouveau client), donc elle est, par construction, indépendante de l'âge, de la ville et de tout le reste. Vérifions que le tirage a bien équilibré les groupes :

```

equilibre = clients.groupby("offre_bienvenue").agg(
    clients=("id_client", "count"),
    age_moyen=("age", "mean"),
    part_instagram=("canal_acquisition", lambda s: (s == "Instagram").mean()),
    part_tunis=("ville", lambda s: (s == "Tunis").mean()),
).round(3)
print(equilibre.to_string())

```

|                 | clients | age_moyen | part_instagram | part_tunis |
|-----------------|---------|-----------|----------------|------------|
| offre_bienvenue |         |           |                |            |
| 0               | 985     | 36.114    | 0.403          | 0.303      |
| 1               | 1015    | 35.397    | 0.413          | 0.317      |

Les deux groupes se ressemblent : c'est exactement ce que la randomisation promet (nous y reviendrons au chapitre 7).

## L'enquête de satisfaction

Six clients sur dix (1 212 sur 2 000) ont répondu à huit questions, notées de 1 (très insatisfait) à 5 (très satisfait). Les quatre premières portent sur les **produits** (qualité, finition, authenticité,

rapport qualité/prix) ; les quatre suivantes sur le **service** (délais de livraison, emballage, relation client, facilité de retour). Cette structure est cachée dans les données ; l'analyse factorielle du chapitre 3 la retrouvera.

```
print(enquete.drop(columns="id_client").describe().loc[["count", "mean", "std", "min",
↳ "max"]].round(2).to_string())
print()
print("corrélation moyenne entre q1..q4 :",
↳ round(enquete[["q1", "q2", "q3", "q4"]].corr().values[np.triu_indices(4, 1)].mean(), 2))
print("corrélation moyenne entre q5..q8 :",
↳ round(enquete[["q5", "q6", "q7", "q8"]].corr().values[np.triu_indices(4, 1)].mean(), 2))
print("corrélation moyenne entre (q1..q4) et (q5..q8) :", round(enquete[[f"q{i}" for i in range(1,
↳ 5)]]).corrwith(enquete["q5"]).mean(), 2))
```

|       | q1      | q2      | q3      | q4      | q5      | q6      | q7      | q8      |
|-------|---------|---------|---------|---------|---------|---------|---------|---------|
| count | 1212.00 | 1212.00 | 1212.00 | 1212.00 | 1212.00 | 1212.00 | 1212.00 | 1212.00 |
| mean  | 3.63    | 3.64    | 3.59    | 3.60    | 3.56    | 3.54    | 3.51    | 3.54    |
| std   | 0.92    | 0.90    | 0.93    | 0.91    | 0.94    | 0.93    | 0.94    | 0.94    |
| min   | 1.00    | 1.00    | 1.00    | 1.00    | 1.00    | 1.00    | 1.00    | 1.00    |
| max   | 5.00    | 5.00    | 5.00    | 5.00    | 5.00    | 5.00    | 5.00    | 5.00    |

```
corrélation moyenne entre q1..q4 : 0.42
corrélation moyenne entre q5..q8 : 0.48
corrélation moyenne entre (q1..q4) et (q5..q8) : 0.12
```

Les questions d'un même thème sont nettement plus corrélées entre elles qu'avec celles de l'autre thème : deux « blocs » se dessinent, sans que nous ayons eu à le dire.

## Les ventes mensuelles

```
print(ventes.head(6).to_string(index=False))
print("...")
print(ventes.tail(3).to_string(index=False))
annuel = ventes.groupby(ventes["mois"].dt.year)["ca"].sum().round(0).astype(int)
print()
print(annuel.to_string())
```

|     | mois       | ca     | nb_commandes | promo | covid |
|-----|------------|--------|--------------|-------|-------|
|     | 2016-01-01 | 620.0  | 14           | 0     | 0     |
|     | 2016-02-01 | 757.5  | 11           | 0     | 0     |
|     | 2016-03-01 | 1007.0 | 13           | 0     | 0     |
|     | 2016-04-01 | 1018.5 | 17           | 0     | 0     |
|     | 2016-05-01 | 1067.8 | 13           | 0     | 0     |
|     | 2016-06-01 | 1155.4 | 11           | 0     | 0     |
| ... |            |        |              |       |       |
|     | 2025-10-01 | 1804.4 | 31           | 0     | 0     |
|     | 2025-11-01 | 2608.5 | 47           | 0     | 0     |
|     | 2025-12-01 | 4175.0 | 63           | 1     | 0     |

| mois | ca    |
|------|-------|
| 2016 | 12658 |
| 2017 | 13846 |
| 2018 | 15280 |
| 2019 | 16828 |
| 2020 | 15680 |
| 2021 | 20847 |
| 2022 | 22934 |
| 2023 | 22884 |
| 2024 | 25249 |
| 2025 | 27630 |



On voit une croissance régulière, interrompue en 2020 : les mois de mars à juin 2020 (colonne covid) correspondent à l'arrêt de l'activité. La colonne promo marque les mois où une promotion a eu lieu. Nous décomposerons cette série (tendance, saisonnalité, bruit) au chapitre 4.

**Les chiffres du volume I et ceux-ci ne sont pas les mêmes, et c'est normal.** Au volume I, nous avons un échantillon de 400 commandes de l'année 2025. Ici, `ventes_mensuelles.csv` donne le chiffre d'affaires sur dix ans : l'année 2025 y est du même ordre de grandeur que les 24 098 DT observés au volume I, mais les deux jeux sont indépendants. Ne cherchez pas à les rapprocher ligne à ligne.

## L'environnement de travail

Ce volume utilise les mêmes outils que le volume I (Python, R, un terminal), avec quelques bibliothèques de plus. Si vous avez suivi le chapitre 6 du volume I, vous savez créer un environnement virtuel ; voici les commandes (non exécutées ici : elles installent des paquets sur *votre* machine).

```
bash noexec python -m venv .venv source .venv/bin/activate          # sous Windows :
.venv\Scripts\activate pip install numpy pandas scipy matplotlib seaborn statsmodels
scikit-learn lifelines arch
```

Rôle de chaque nouvelle bibliothèque :

| Bibliothèque | Pour quoi faire                                                                                         | Chapitres |
|--------------|---------------------------------------------------------------------------------------------------------|-----------|
| statsmodels  | régression, GLM, séries temporelles, survie, tests : la bibliothèque statistique de référence en Python | 1 à 5     |
| scikit-learn | réduction de dimension, classification, régularisation, validation croisée                              | 1, 3      |
| lifelines    | analyse de survie (certaines fonctions seulement dans ce livre)                                         | 5         |
| arch         | modèles GARCH                                                                                           | 4         |

Pour les lecteurs de R, les paquets `MASS`, `lme4`, `mgcv`, `survival` et `forecast` sont utilisés pour **comparer** certains résultats avec ceux de Python : c'est une excellente façon de vérifier qu'on a bien compris le modèle, puisque deux logiciels indépendants doivent donner les mêmes nombres.

Quelques versions de référence, pour que vous sachiez avec quoi les sorties du livre ont été produites :

```
import sys, platform
import scipy, statsmodels, sklearn
print("Python      :", platform.python_version())
print("numpy       :", np.__version__)
print("pandas      :", pd.__version__)
print("scipy        :", scipy.__version__)
print("statsmodels  :", statsmodels.__version__)
print("scikit-learn:", sklearn.__version__)
```

```
Python      : 3.13.3
numpy       : 2.5.3
pandas      : 3.0.6
scipy       : 1.18.1
```

```
statsmodels : 0.15.0  
scikit-learn: 1.9.1
```

**Des versions différentes donnent parfois de très légères différences** dans la dernière décimale des estimations (algorithmes d'optimisation, arrondis), jamais dans les conclusions. Si vos chiffres s'écartent d'un ou deux chiffres significatifs, c'est un signal ; s'ils s'écartent de la dernière décimale, c'est normal.

**Prêt ?** Au chapitre 1, nous commençons par le modèle le plus simple et le plus important de toute la statistique : la droite des moindres carrés, que vous avez déjà croisée plusieurs fois au volume I, enfin généralisée à plusieurs variables.

# Chapitre 1 : Régression linéaire

« Tous les modèles sont faux, mais certains sont utiles. » — George Box

Au volume I, vous avez appris à **décrire** des données (chapitre 3 : moyennes, corrélations) et à **comparer** des groupes (tests de Student, de Welch). Mais Yasmine pose rarement des questions aussi simples. Elle demande plutôt :

- « **Combien** dépense un client de plus de 50 ans acquis par Instagram, *par rapport à* un client de 30 ans acquis en boutique ? »
- « Quand je dis que les clients Instagram dépensent moins, est-ce vraiment le **canal**, ou est-ce parce qu'ils sont plus jeunes ? »
- « Si je ne connais que l'âge et le canal d'un nouveau client, **quel panier** puis-je prévoir ? Avec quelle marge d'erreur ? »

Pour répondre, il faut un outil qui relie **une quantité à expliquer à plusieurs variables explicatives en même temps**, qui mesure l'effet de chacune *toutes choses égales par ailleurs*, et qui dit honnêtement quelle confiance accorder aux chiffres. Cet outil est la **régression linéaire**, le cheval de trait de la statistique appliquée : il sert tous les jours, il est au cœur de modèles plus sophistiqués (chapitres suivants et volume III), et le comprendre à fond rend tout le reste plus facile.

## Le chemin de ce chapitre

- **1.1 Le modèle linéaire et les moindres carrés** : écrire le modèle, calculer les coefficients (à la main, puis en code), comprendre pourquoi la solution est la bonne (projection, théorème de Gauss-Markov), interpréter les coefficients, y compris avec des variables qualitatives et des transformations logarithmiques.
- **1.2 Inférence sur les coefficients** : erreurs standard, tests  $t$  et  $F$ , intervalles de confiance et intervalles de prédiction : que peut-on conclure, et avec quelle incertitude ?
- **1.3 Diagnostics** : vérifier que le modèle n'est pas faux de façon grave : résidus, effet de levier, observations influentes, multicolinéarité.
- **1.4 Sélection de variables et comparaison de modèles** : quelles variables garder ? Surapprentissage, critères AIC/BIC, validation croisée, et les pièges de la sélection automatique.
- **⊕ Pour aller plus loin** : la **régularisation** (Ridge, Lasso, Elastic Net, 1.5), la **régression robuste** (1.6), et les **modèles à effets mixtes** pour les données groupées (1.7).
- **1.8 Exercices corrigés.**

**Le fil conducteur : le panier des clients de Dar Jasmin.** Nous travaillons sur **2 000 clients** de Dar Jasmin, observés sur une année (âge, canal d'acquisition, ville, dépenses, etc.) et, pour certains, leur réponse à un petit questionnaire de satisfaction. Ces données sont **simulées** (graine fixe) : ainsi, nous connaissons la vérité, et nous pourrions à la fin de l'étude **vérifier** que la méthode la retrouve. C'est un luxe que la vie réelle n'offre jamais, et il rend très instructif l'examen de ce que la régression fait bien... ou moins bien.

**Les fichiers de données.** Ce chapitre lit `donnees/clients.csv` (un client par ligne) et, à partir de 1.4, `donnees/enquete_satisfaction.csv` (réponses à huit questions de satisfaction). Au 1.7, un petit jeu supplémentaire (`donnees/ch01-relais.csv`) est simulé sous vos yeux. Le code est exécuté depuis la racine du volume, d'où les chemins `donnees/...`

**Outils.** Nous utilisons `statsmodels` (le module de référence pour les modèles statistiques en Python : tableaux de résultats, tests, diagnostics), `numpy` (pour tout recalculer à la main afin de bien comprendre), `scikit-learn` (pour la régularisation) et `matplotlib`. Quelques blocs R (`lm`, `lme4`) montrent que l'on obtient exactement les mêmes nombres dans l'autre grand langage de la statistique (section 4.2 du volume I).

**Notations.** Nous écrivons les vecteurs en gras minuscule ( $\mathbf{y}$ ,  $\boldsymbol{\beta}$ ), les matrices en gras majuscule ( $\mathbf{X}$ ),  $n$  pour le nombre d'observations et  $p$  pour le nombre de **colonnes** de  $\mathbf{X}$  (constante comprise). Si l'algèbre linéaire vous semble lointaine, relisez les sections 1.1.2 (matrices) et 1.1.3 (valeurs propres) du volume I ; pour la minimisation d'une fonction, le chapitre 1.3 du même volume.

## 1.1 Le modèle linéaire et les moindres carrés

**Intuition.** Vous avez un nuage de points et vous voulez y poser **la droite qui colle le mieux**. Mais « le mieux » est vague : on peut passer par certains points et en rater d'autres. Le critère des **moindres carrés** tranche : on mesure l'écart vertical entre chaque point et la droite, on **élève chaque écart au carré** (pour que les écarts positifs et négatifs ne s'annulent pas, et pour punir plus sévèrement les gros écarts), on additionne, et on choisit la droite qui rend cette somme **la plus petite possible**. Tout ce chapitre est l'étude de cette idée, avec une ou plusieurs variables explicatives.

### 1.1.1 Un exemple minuscule, entièrement à la main

Yasmine a noté, pour quatre commandes, le **nombre d'articles** et le **montant total** en DT :

| Commande | Articles $x$ | Montant $y$ (DT) |
|----------|--------------|------------------|
| A        | 1            | 22               |
| B        | 2            | 41               |
| C        | 3            | 66               |
| D        | 4            | 79               |

Elle cherche une droite  $y = \beta_0 + \beta_1 x$  :  $\beta_0$  serait un coût fixe (emballage, livraison) et  $\beta_1$  le prix moyen d'un article supplémentaire. Chaque commande donne une équation, que l'on empile en **écriture matricielle** (volume I, section 1.1.2) :

$$\underbrace{\begin{pmatrix} 22 \\ 41 \\ 66 \\ 79 \end{pmatrix}}_{\mathbf{y}} \approx \underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}}_{\mathbf{X}} \underbrace{\begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}}_{\boldsymbol{\beta}}$$

La première colonne de  $\mathbf{X}$  ne contient que des 1 : elle sert à porter la constante  $\beta_0$ . Nous démontrerons plus loin (1.1.3) que la meilleure droite au sens des moindres carrés est donnée par la formule

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Calculons-la à la main. D'abord les deux produits matriciels :

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 22 + 41 + 66 + 79 \\ 22 \cdot 1 + 41 \cdot 2 + 66 \cdot 3 + 79 \cdot 4 \end{pmatrix} = \begin{pmatrix} 208 \\ 618 \end{pmatrix}.$$

Le déterminant de  $\mathbf{X}^\top \mathbf{X}$  vaut  $4 \times 30 - 10 \times 10 = 20$ , donc l'inverse d'une matrice  $2 \times 2$  s'obtient en échangeant les termes de la diagonale, en changeant le signe des deux autres et en divisant par le déterminant :

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{20} \begin{pmatrix} 30 & -10 \\ -10 & 4 \end{pmatrix}, \quad \hat{\beta} = \frac{1}{20} \begin{pmatrix} 30 \cdot 208 - 10 \cdot 618 \\ -10 \cdot 208 + 4 \cdot 618 \end{pmatrix} = \frac{1}{20} \begin{pmatrix} 60 \\ 392 \end{pmatrix} = \begin{pmatrix} 3 \\ 19,6 \end{pmatrix}.$$

La meilleure droite est donc  $\hat{y} = 3 + 19,6x$  : un coût fixe de 3 DT et 19,60 DT par article. Les valeurs ajustées sont 22,6; 42,2; 61,8; 81,4, et les **résidus** (écarts entre le réel et l'ajusté) valent -0,6; -1,2; +4,2; -2,4. Leur somme est nulle, et la somme de leurs carrés vaut  $0,36 + 1,44 + 17,64 + 5,76 = 25,2$ . Vérifions tout cela par le code.

```
import numpy as np

x = np.array([1, 2, 3, 4])
y = np.array([22, 41, 66, 79])
X = np.column_stack([np.ones(4), x])      # la colonne de 1, puis x

XtX = X.T @ X
Xty = X.T @ y
beta = np.linalg.solve(XtX, Xty)          # résout (X'X) beta = X'y

print("X'X =\n", XtX)
print("X'y =", Xty)
print("beta chapeau =", beta)

ajuste = X @ beta
residus = y - ajuste
print("valeurs ajustées :", ajuste)
print("résidus          :", residus.round(2))
print("somme des résidus      :", round(residus.sum(), 10))
print("somme des carrés (SCR) :", round((residus**2).sum(), 2))

X'X =
[[ 4. 10.]
 [10. 30.]]
X'y = [208. 618.]
beta chapeau = [ 3. 19.6]
valeurs ajustées : [22.6 42.2 61.8 81.4]
résidus          : [-0.6 -1.2  4.2 -2.4]
somme des résidus      : 0.0
somme des carrés (SCR) : 25.2
```

Nous avons utilisé `np.linalg.solve(XtX, Xty)` plutôt que d'inverser explicitement la matrice : résoudre un système est plus rapide et surtout **plus précis** que calculer un inverse (c'est la leçon de la section 1.5.4 du volume I sur le conditionnement). Voyons le résultat en image.

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, ENCRE, GRIS = "#2a78d6", "#eb6834", "#1baf7a", "#0b0b0b", "#898781"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
    "#e0e0e0",
    "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
    "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

fig, ax = plt.subplots(figsize=(6.4, 4.2))
xs = np.linspace(0.6, 4.4, 50)
ax.plot(xs, beta[0] + beta[1] * xs, color=BLEU, lw=2)
ax.scatter(x, y, color=ENCRE, zorder=3, s=40)
for xi, yi, ai in zip(x, y, ajuste):
    ax.plot([xi, xi], [yi, ai], color=ORANGE, lw=2) # le résidu : un segment vertical
for xi, yi, nom in zip(x, y, "ABCD"):
    ax.annotate(nom, (xi, yi), textcoords="offset points", xytext=(-12, 4), color=ENCRE)
ax.text(2.45, 30, "droite des moindres carrés\n $\hat{y} = 3 + 19,6 x$ ", color=BLEU)
ax.text(3.08, 60, "résidu", color=ORANGE)
ax.set_xlabel("nombre d'articles")
ax.set_ylabel("montant de la commande (DT)")
ax.set_xlim(0.5, 4.6)
ax.set_ylim(10, 95)
plt.savefig("figures/ch01-droite-4-points.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

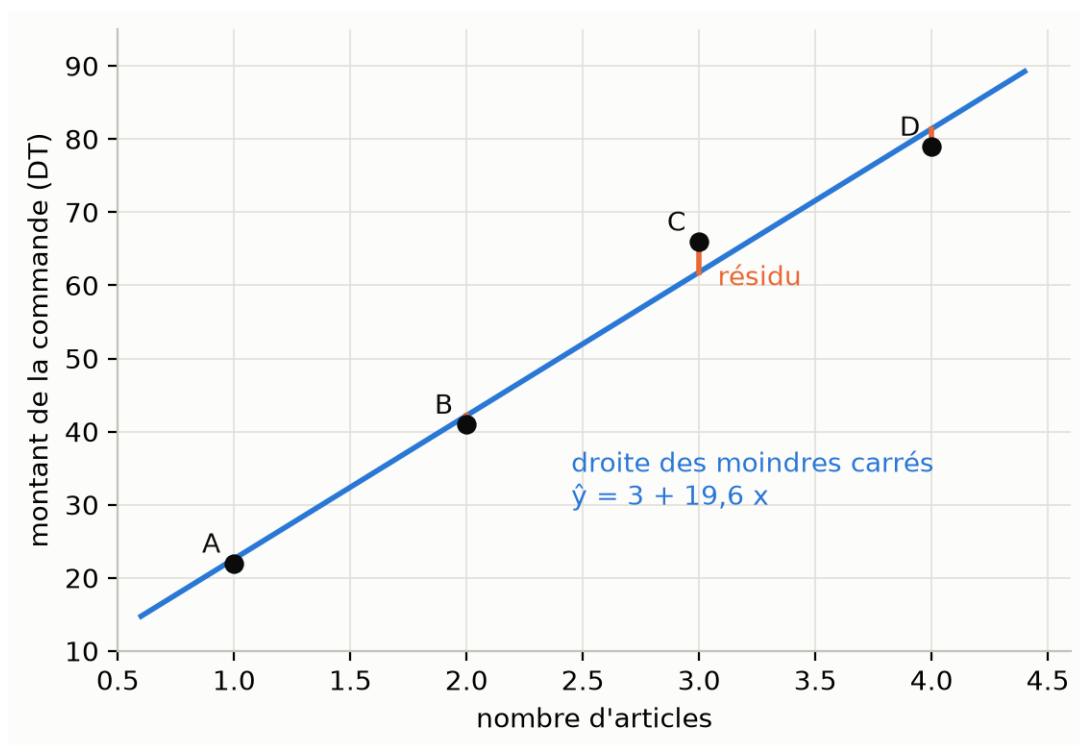


Figure 1: Quatre commandes, la droite des moindres carrés et les résidus (segments verticaux). La droite est celle qui minimise la somme des carrés de ces segments.

**Deux propriétés à remarquer.** (1) La somme des résidus est nulle (aux erreurs d'arrondi près). (2) Le produit  $\mathbf{X}^\top \mathbf{e}$  est nul : les résidus sont *orthogonaux* à chaque colonne de  $\mathbf{X}$ . Nous les démontrons en 1.1.3 et 1.1.4, et elles joueront un rôle central.

```
print("X' e =", (X.T @ residus).round(10))
```

```
X' e = [ 0. -0.]
```

### 1.1.2 Le modèle linéaire : ce que l'on suppose

Passons de quatre points à un vrai modèle statistique. On observe  $n$  individus. Pour l'individu  $i$ , une variable à expliquer  $y_i$  (la **réponse**) et  $p - 1$  variables explicatives  $x_{i1}, \dots, x_{i,p-1}$  (les **prédicteurs**, ou *covariables*). Le **modèle de régression linéaire** postule

$$y_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_{p-1} x_{i,p-1} + \varepsilon_i, \quad \text{soit} \quad \mathbf{y} = \mathbf{X}\beta + \varepsilon.$$

Ici  $\mathbf{X}$  est la **matrice de plan d'expérience** (en anglais *design matrix*), de taille  $n \times p$  (première colonne de 1),  $\beta$  le vecteur des  $p$  coefficients inconnus, et  $\varepsilon$  le vecteur des **erreurs** : tout ce que le modèle n'explique pas (variables oubliées, hasard, erreurs de mesure). Les erreurs ne sont jamais observées ; ce que l'on calcule, ce sont des **résidus**  $\hat{\varepsilon}_i = y_i - \hat{y}_i$ , qui en sont une approximation.

Pour que les formules qui suivent aient un sens statistique, on fait des **hypothèses**. Nous les numérotons, car nous y reviendrons tout le chapitre :

|    | Hypothèse                 | Ce qu'elle dit                                                                                                                                | Ce dont elle sert                                                                |
|----|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| H1 | <b>Linéarité</b>          | $\mathbb{E}[y_i   x_i]$ est une combinaison linéaire des <b>paramètres</b> $\beta_j$                                                          | l'espérance de $y$ est $\mathbf{X}\beta$                                         |
| H2 | <b>Rang plein</b>         | les colonnes de $\mathbf{X}$ sont linéairement indépendantes (rang $\mathbf{X} = p$ , donc $n \geq p$ )                                       | $\mathbf{X}^\top \mathbf{X}$ est inversible : $\hat{\beta}$ existe et est unique |
| H3 | <b>Exogénéité</b>         | $\mathbb{E}[\varepsilon   \mathbf{X}] = \mathbf{0}$ : les erreurs sont « neutres » vis-à-vis des prédicteurs                                  | $\hat{\beta}$ est <b>sans biais</b>                                              |
| H4 | <b>Erreurs sphériques</b> | $\text{Var}(\varepsilon   \mathbf{X}) = \sigma^2 \mathbf{I}_n$ : variance <b>constante</b> (homoscédasticité) et erreurs <b>non corrélées</b> | formule de la variance de $\hat{\beta}$ , théorème de Gauss-Markov               |
| H5 | <b>Normalité</b>          | $\varepsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$                                                                             | tests $t$ et $F$ <b>exacts</b> , intervalles de confiance                        |

« **Linéaire** » veut dire **linéaire dans les paramètres, pas dans les variables**. Le modèle  $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon$  est un modèle linéaire (c'est une combinaison linéaire de  $\beta_0, \beta_1, \beta_2$  ; la colonne  $x^2$  est simplement une colonne de plus dans  $\mathbf{X}$ ). De même  $\log y = \beta_0 + \beta_1 \log x + \varepsilon$ . En revanche,  $y = \beta_0 e^{\beta_1 x} + \varepsilon$  n'est pas linéaire :  $\beta_1$  apparaît dans l'exponentielle. C'est ce qui permet à la régression linéaire d'ajuster des courbes, et pas seulement des droites.

**Le rôle de chaque hypothèse.** On ne les exige pas toutes pour toutes les conclusions. H1 à H3 suffisent à garantir que les coefficients visent juste en moyenne (absence de biais). Il faut H4 en plus pour savoir **quelle précision** on atteint et pour comparer les estimateurs (Gauss-Markov). Il faut H5 pour obtenir des tests et des intervalles de confiance exacts avec  $n$  petit (pour  $n$  grand, le théorème central limite, volume I section 2.4.3, atténue l'importance de H5). La section 1.3 apprendra à **vérifier** ces hypothèses.

### 1.1.3 Les équations normales : démonstration

**Objectif.** Trouver le vecteur  $\beta$  qui minimise la somme des carrés des résidus

$$S(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|^2 = (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta).$$

**Démonstration.** Développons le produit (en rappelant que  $\beta^\top \mathbf{X}^\top \mathbf{y}$  est un nombre, donc égal à sa transposée  $\mathbf{y}^\top \mathbf{X}\beta$ ) :

$$S(\beta) = \mathbf{y}^\top \mathbf{y} - 2\beta^\top \mathbf{X}^\top \mathbf{y} + \beta^\top \mathbf{X}^\top \mathbf{X}\beta.$$

**Gradient** (volume I, section 1.2.2) : les règles  $\nabla_\beta(\mathbf{a}^\top \beta) = \mathbf{a}$  et  $\nabla_\beta(\beta^\top \mathbf{A}\beta) = 2\mathbf{A}\beta$  (pour  $\mathbf{A}$  symétrique) donnent

$$\nabla S(\beta) = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\beta.$$

En un minimum, le gradient s'annule, d'où les **équations normales** :

$$\mathbf{X}^\top \mathbf{X}\hat{\beta} = \mathbf{X}^\top \mathbf{y}$$

**C'est bien un minimum, et il est unique.** La matrice hessienne de  $S$  est  $2\mathbf{X}^\top \mathbf{X}$ . Pour tout vecteur  $\mathbf{v} \neq \mathbf{0}$ ,  $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 \geq 0$ , et cette quantité est **strictement positive** si  $\mathbf{X}\mathbf{v} \neq \mathbf{0}$ , ce qui est garanti par H2 (colonnes indépendantes). Donc le hessien est définie positive :  $S$  est **strictement convexe** (volume I, section 1.3.2), elle possède un unique point critique, et c'est son minimum global. Sous H2,  $\mathbf{X}^\top \mathbf{X}$  est inversible et  $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ .  $\square$

Deux conséquences immédiates. La  $j$ -ième équation normale s'écrit  $\mathbf{x}_j^\top (\mathbf{y} - \mathbf{X}\hat{\beta}) = 0$ , soit

$$\mathbf{X}^\top \hat{\epsilon} = \mathbf{0} :$$

**les résidus sont orthogonaux à chaque colonne de  $\mathbf{X}$ .** En particulier, comme la première colonne est faite de 1,  $\sum_i \hat{\epsilon}_i = 0$  : *tant que le modèle contient une constante*, les résidus sont de somme nulle. Nous avons constaté ces deux faits au 1.1.1.

**Une autre route vers la même solution : la descente de gradient.** Au volume I (section 1.3.3), nous avons appris à descendre une pente pas à pas. La fonction à minimiser ici est  $S$ , de gradient  $-2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta)$ . Pour que la descente converge vite, on **centre** la variable  $x$  (on lui soustrait sa moyenne), ce qui rend les deux directions de  $\mathbf{X}^\top \mathbf{X}$  orthogonales entre elles. Vérifions que l'on retrouve (3 ; 19,6) (après recentrage : constante 52 et pente 19,6).

```
xc = x - x.mean() # x centré : -1,5 ; -0,5 ; 0,5 ; 1,5
Xc = np.column_stack([np.ones(4), xc])
b = np.zeros(2)
pas = 0.1 # taux d'apprentissage
for it in range(200):
    gradient = -2 / len(y) * Xc.T @ (y - Xc @ b) # gradient de la moyenne des carrés
    b = b - pas * gradient
```



```
print("descente de gradient (x centré) :", b.round(4))
print("solution exacte          :", np.linalg.solve(Xc.T @ Xc, Xc.T @ y).round(4))
print("retour à l'échelle d'origine : constante =", round(b[0] - b[1] * x.mean(), 3), "| pente =",
      round(b[1], 3))
```

```
descente de gradient (x centré) : [52.  19.6]
solution exacte                : [52.  19.6]
retour à l'échelle d'origine   : constante = 3.0 | pente = 19.6
```

La descente de gradient retrouve la solution exacte (à la précision près) : il s'agit bien du même problème, résolu autrement. En pratique, les logiciels n'utilisent ni l'inverse ni la descente de gradient pour la régression linéaire classique, mais une **factorisation QR** de  $\mathbf{X}$  (ou une SVD, volume I section 1.1.4), numériquement bien plus stable. La descente de gradient reprend tout son intérêt avec les très grands jeux de données et avec les modèles qui n'ont pas de formule fermée : la régression logistique du chapitre 2, par exemple, s'ajuste par un algorithme itératif de la même famille.

### 1.1.4 La géométrie des moindres carrés : une projection

Les équations normales cachent une image géométrique très puissante. Voyons les  $n$  observations  $\mathbf{y}$  comme **un seul point** (un vecteur) de l'espace  $\mathbb{R}^n$ . Les colonnes de  $\mathbf{X}$  engendrent un sous-espace de dimension  $p$  (ici, un plan si  $p = 2$ ). Les valeurs  $\mathbf{X}\beta$  que le modèle peut produire sont exactement les points de ce sous-espace. Chercher  $\beta$  qui minimise  $\|\mathbf{y} - \mathbf{X}\beta\|$ , c'est **chercher le point du sous-espace le plus proche de  $\mathbf{y}$** , et la géométrie nous dit que c'est le **projeté orthogonal** de  $\mathbf{y}$  sur ce sous-espace.

```
fig, ax = plt.subplots(figsize=(6.4, 4.2))
ax.set_xlim(-0.3, 6.6); ax.set_ylim(-0.9, 3.9); ax.set_aspect("equal"); ax.axis("off")
ax.plot([0, 6.2], [0, 1.24], color=GRIS, lw=4, alpha=0.45, solid_capstyle="round")
ax.text(0.1, -0.5, "sous-espace engendré par les colonnes de X", color=GRIS)
yv = np.array([4.0, 3.2])
d = np.array([5.0, 1.0]) / np.linalg.norm([5.0, 1.0]) # direction du sous-espace
pv = (yv @ d) * d # projeté orthogonal de y
ax.annotate("", xy=pv, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=ENCRE, lw=2))
ax.annotate("", xy=pv, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=BLEU, lw=2))
ax.plot([yv[0], pv[0]], [yv[1], pv[1]], color=ORANGE, lw=2, ls="--")
n = np.array([-d[1], d[0]]) # petit carré : angle droit en y
c = 0.22
ax.plot([pv[0] + c * d[0] * -1, pv[0] - c * d[0] + c * n[0], pv[0] + c * n[0]],
        [pv[1] + c * d[1] * -1, pv[1] - c * d[1] + c * n[1], pv[1] + c * n[1]], color=ORANGE, lw=1.2)
ax.text(yv[0] + 0.12, yv[1] + 0.05, "y (les données)", color=ENCRE)
ax.text(pv[0] + 0.15, pv[1] - 0.55, "ŷ = Xβ (meilleur point du modèle)", color=BLEU)
ax.text(pv[0] + 0.25, 2.0, "résidu ε (perpendiculaire)", color=ORANGE)
plt.savefig("figures/ch01-projection.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**La matrice chapeau.** Comme  $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ , on a  $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$  avec

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \quad (\text{« la matrice chapeau », car elle met un chapeau sur } \mathbf{y}).$$

Elle est **symétrique** ( $\mathbf{H}^\top = \mathbf{H}$ , car  $(\mathbf{X}^\top \mathbf{X})^{-1}$  est symétrique) et **idempotente** :

$$\mathbf{H}^2 = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \underbrace{\mathbf{X}^\top \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1}}_{=\mathbf{I}} \mathbf{X}^\top = \mathbf{H}.$$

Cela caractérise un **projecteur orthogonal** : projeter deux fois revient à projeter une fois. Le résidu est  $\hat{\varepsilon} = \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mathbf{y}$ , et  $\mathbf{I} - \mathbf{H}$  est le projecteur sur le sous-espace **orthogonal**.

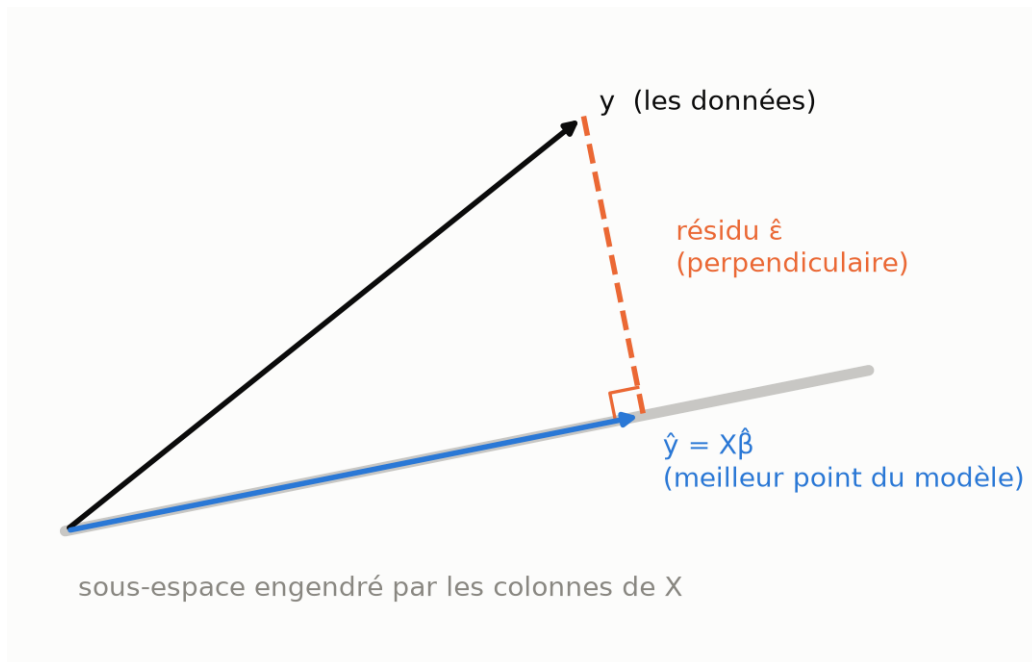


Figure 2: Les moindres carrés comme projection :  $\hat{y}$  est le point du sous-espace le plus proche de  $y$ , et le résidu est perpendiculaire au sous-espace.

Enfin, la trace d'un projecteur est la dimension de son image :

$$\text{tr}(\mathbf{H}) = p, \quad \text{tr}(\mathbf{I} - \mathbf{H}) = n - p.$$

(En effet  $\text{tr}(\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top) = \text{tr}((\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X}) = \text{tr}(\mathbf{I}_p) = p$ , car la trace est invariante par permutation circulaire.) **Pythagore** s'applique alors :  $\|y\|^2 = \|\hat{y}\|^2 + \|\hat{\varepsilon}\|^2$ .

Vérifions ces propriétés sur nos quatre commandes :

```
H = X @ np.linalg.inv(X.T @ X) @ X.T
print("H (arrondie) =\n", H.round(2))
print("symétrique :", np.allclose(H, H.T), "| idempotente :", np.allclose(H @ H, H))
print("trace de H =", round(np.trace(H), 6), "(= p = 2)")
print("H y = valeurs ajustées :", np.allclose(H @ y, ajuste))
print("Pythagore : ||y||² =", int(y @ y), "= ||ŷ||² + ||ε||² =", round(ajuste @ ajuste + residus @
↵ residus, 6))
```

```
H (arrondie) =
[[ 0.7  0.4  0.1 -0.2]
 [ 0.4  0.3  0.2  0.1]
 [ 0.1  0.2  0.3  0.4]
 [-0.2  0.1  0.4  0.7]]
symétrique : True | idempotente : True
trace de H = 2.0 (= p = 2)
H y = valeurs ajustées : True
Pythagore : ||y||² = 12762 = ||ŷ||² + ||ε||² = 12762.0
```

Les éléments diagonaux de  $\mathbf{H}$ , notés  $h_{ii}$  (ici 0,7; 0,3; 0,3; 0,7, de somme  $2 = p$ ), mesurent « l'influence » de chaque observation sur sa propre valeur ajustée : nous les retrouverons en 1.3 sous le nom d'**effet de levier**. On voit déjà que les commandes extrêmes (A et D, avec 1 et 4 articles) ont plus de levier que celles du milieu.

### 1.1.5 Propriétés statistiques : sans biais, variance, et le théorème de Gauss-Markov

Les données sont un échantillon : un autre échantillon donnerait d'autres résidus, donc un autre  $\hat{\beta}$ . L'estimateur  $\hat{\beta}$  est donc une **variable aléatoire** (volume I, section 3.2.1). Quelles sont ses propriétés ?

**Théorème 1 (espérance et variance de  $\hat{\beta}$ ).** Sous H1 à H4,

$$\mathbb{E}[\hat{\beta}] = \beta, \quad \text{Var}(\hat{\beta}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}.$$

*Démonstration.* En posant  $\mathbf{A} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ , on a  $\hat{\beta} = \mathbf{A} \mathbf{y} = \mathbf{A}(\mathbf{X}\beta + \varepsilon) = \beta + \mathbf{A}\varepsilon$ , car  $\mathbf{A}\mathbf{X} = \mathbf{I}_p$ . Donc  $\mathbb{E}[\hat{\beta}] = \beta + \mathbf{A} \mathbb{E}[\varepsilon] = \beta$  (H3). Et  $\text{Var}(\hat{\beta}) = \mathbf{A} \text{Var}(\varepsilon) \mathbf{A}^\top = \sigma^2 \mathbf{A} \mathbf{A}^\top = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$  (H4).  $\square$

Regardons ce que dit la formule de variance : la précision de  $\hat{\beta}_j$  s'améliore quand  $\sigma^2$  est petit (peu de bruit), quand  $n$  est grand ( $\mathbf{X}^\top \mathbf{X}$  croît avec  $n$ ) et quand les valeurs de  $x_j$  sont **bien étalées** (si tous les clients avaient le même âge, on ne pourrait pas estimer l'effet de l'âge). C'est une information précieuse pour **concevoir** une étude.

**Théorème 2 (Gauss-Markov).** Sous H1 à H4,  $\hat{\beta}$  est le **meilleur estimateur linéaire sans biais** (en anglais **BLUE**, *Best Linear Unbiased Estimator*) : pour tout autre estimateur  $\tilde{\beta} = \mathbf{C} \mathbf{y}$  linéaire en  $\mathbf{y}$  et sans biais,  $\text{Var}(\tilde{\beta}) - \text{Var}(\hat{\beta})$  est une matrice semi-définie positive. En particulier, pour toute combinaison  $\mathbf{c}^\top \hat{\beta}$ , la variance de  $\mathbf{c}^\top \hat{\beta}$  est la plus petite possible. *Démonstration.* Écrivons  $\mathbf{C} = \mathbf{A} + \mathbf{D}$ , où  $\mathbf{A} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$  et  $\mathbf{D}$  est une matrice  $p \times n$ . L'absence de biais exige  $\mathbb{E}[\mathbf{C} \mathbf{y}] = \mathbf{C} \mathbf{X} \beta = \beta$  pour tout  $\beta$ , donc  $\mathbf{C} \mathbf{X} = \mathbf{I}$ , c'est-à-dire  $\mathbf{D} \mathbf{X} = \mathbf{0}$  (puisque  $\mathbf{A} \mathbf{X} = \mathbf{I}$ ). Alors

$$\text{Var}(\tilde{\beta}) = \sigma^2 \mathbf{C} \mathbf{C}^\top = \sigma^2 (\mathbf{A} + \mathbf{D})(\mathbf{A} + \mathbf{D})^\top = \sigma^2 (\mathbf{A} \mathbf{A}^\top + \mathbf{A} \mathbf{D}^\top + \mathbf{D} \mathbf{A}^\top + \mathbf{D} \mathbf{D}^\top).$$

Or  $\mathbf{D} \mathbf{A}^\top = \mathbf{D} \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} = \mathbf{0}$ , et sa transposée  $\mathbf{A} \mathbf{D}^\top$  aussi. Il reste  $\text{Var}(\tilde{\beta}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} + \sigma^2 \mathbf{D} \mathbf{D}^\top$ , et  $\mathbf{D} \mathbf{D}^\top$  est semi-définie positive (pour tout  $\mathbf{v}$ ,  $\mathbf{v}^\top \mathbf{D} \mathbf{D}^\top \mathbf{v} = \|\mathbf{D}^\top \mathbf{v}\|^2 \geq 0$ ).  $\square$

Ce théorème est remarquable par ce qu'il **ne demande pas** : aucune hypothèse de normalité. Mais il a aussi des limites, qu'il faut garder en tête : il ne compare que des estimateurs **linéaires et sans biais**. Un estimateur *biaisé* (comme la régression Ridge, section 1.5) peut avoir une erreur quadratique moyenne plus petite. Et si H4 est fautive (variance non constante),  $\hat{\beta}$  reste sans biais mais n'est plus le meilleur.

**Estimer la variance du bruit.** La formule de  $\text{Var}(\hat{\beta})$  contient  $\sigma^2$ , inconnue. On l'estime par

$$s^2 = \frac{\text{SCR}}{n-p} = \frac{\sum_i \hat{\varepsilon}_i^2}{n-p}.$$

**Pourquoi  $n-p$  ?** (c'est la généralisation de la division par  $n-1$  du volume I, section 3.2.3). Le résidu s'écrit  $\hat{\varepsilon} = (\mathbf{I} - \mathbf{H}) \mathbf{y} = (\mathbf{I} - \mathbf{H})(\mathbf{X}\beta + \varepsilon) = (\mathbf{I} - \mathbf{H})\varepsilon$ , car  $(\mathbf{I} - \mathbf{H})\mathbf{X} = \mathbf{0}$ . Donc  $\text{SCR} = \varepsilon^\top (\mathbf{I} - \mathbf{H}) \varepsilon$  (projecteur symétrique idempotent) et

$$\mathbb{E}[\text{SCR}] = \mathbb{E}[\text{tr}(\varepsilon^\top (\mathbf{I} - \mathbf{H}) \varepsilon)] = \text{tr}((\mathbf{I} - \mathbf{H}) \mathbb{E}[\varepsilon \varepsilon^\top]) = \sigma^2 \text{tr}(\mathbf{I} - \mathbf{H}) = \sigma^2(n-p).$$

Intuitivement : les résidus sont plus petits que les vraies erreurs, parce que le modèle a *utilisé*  $p$  degrés de liberté pour s'ajuster aux données. Diviser par  $n-p$  compense exactement ce « trop bon ajustement ».

**Voyons-le en simulation.** Nous connaissons la vérité dans une simulation : prenons les quatre valeurs de  $x$  ci-dessus,  $\beta = (3, 20)$  et  $\sigma = 3$ , fabriquons 20 000 échantillons de quatre montants, et ajustons la droite à chacun. On comparera les moyennes et la variance **observées** des estimateurs aux formules du théorème.

```
rng = np.random.default_rng(1)
beta_vrai, sigma = np.array([3.0, 20.0]), 3.0
R = 20000
Y = X @ beta_vrai + rng.normal(0, sigma, size=(R, 4))      # R échantillons de 4 montants (une
↳ ligne chacun)
Bhat = np.linalg.solve(X.T @ X, X.T @ Y.T)                  # R estimations de beta
Res = Y - Bhat @ X.T
SCR = (Res**2).sum(axis=1)

print("moyenne des beta chapeau :", Bhat.mean(axis=0).round(3), " (vrai :", beta_vrai, ")")
print("variances observées      :", Bhat.var(axis=0, ddof=1).round(3))
print("variances théoriques     :", (sigma**2 * np.diag(np.linalg.inv(X.T @ X))).round(3))
print("covariance observée     :", np.cov(Bhat.T)[0, 1].round(3), "| théorique :", (sigma**2 *
↳ np.linalg.inv(X.T @ X))[0, 1].round(3))
print()
print("sigma² vrai              :", sigma**2)
print("moyenne de SCR/(n-p)     :", (SCR / 2).mean().round(3), " <- sans biais")
print("moyenne de SCR/n        :", (SCR / 4).mean().round(3), " <- sous-estime, facteur (n-p)/n =
↳ 0,5")
```

```
moyenne des beta chapeau : [ 2.978 20.003] (vrai : [ 3. 20.] )
variances observées      : [13.18  1.774]
variances théoriques     : [13.5  1.8]
covariance observée     : -4.414 | théorique : -4.5

sigma² vrai              : 9.0
moyenne de SCR/(n-p)    : 9.07  <- sans biais
moyenne de SCR/n        : 4.535  <- sous-estime, facteur (n-p)/n = 0,5
```

Les moyennes des  $\hat{\beta}_j$  sont proches des vraies valeurs (absence de biais), les variances et covariances observées sont très proches de celles de  $\sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$ , et la division par  $n - p$  donne une estimation centrée de  $\sigma^2$  alors que la division par  $n$  la sous-estime d'un facteur  $(n - p)/n = 1/2$ . La théorie est confirmée par l'expérience.

### 1.1.6 Décomposition de la variance et coefficient de détermination $R^2$

Combien le modèle explique-t-il ? Mesurons la dispersion de  $y$  autour de sa moyenne par la **somme des carrés totale**  $SCT = \sum_i (y_i - \bar{y})^2$ . La régression la décompose en une partie expliquée et une partie résiduelle.

**Théorème (décomposition de la variance).** Si le modèle contient une constante, alors

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{SCT} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{SCE \text{ (expliquée)}} + \underbrace{\sum_i \hat{\varepsilon}_i^2}_{SCR \text{ (résiduelle)}}.$$

*Démonstration.* Écrivons  $y_i - \bar{y} = (\hat{y}_i - \bar{y}) + \hat{\varepsilon}_i$  et élevons au carré : le double produit vaut  $2 \sum_i (\hat{y}_i - \bar{y}) \hat{\varepsilon}_i = 2 \sum_i \hat{y}_i \hat{\varepsilon}_i - 2\bar{y} \sum_i \hat{\varepsilon}_i$ . Le premier terme est nul car  $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}$  est orthogonal à  $\hat{\varepsilon}$  (1.1.3), le second car la somme des résidus est nulle (présence de la constante).  $\square$

Le **coefficient de détermination** est la part de variance expliquée :

$$R^2 = \frac{SCE}{SCT} = 1 - \frac{SCR}{SCT} \in [0, 1].$$

On peut montrer que  $R^2$  est le **carré de la corrélation** entre  $y$  et  $\hat{y}$  : en régression simple, c'est donc le carré du coefficient de corrélation de Pearson entre  $x$  et  $y$  (volume I, section 3.1.7). Pour nos quatre commandes :  $SCT = 900 + 121 + 196 + 729 = 1946$  (les écarts de  $y$  à la moyenne 52 sont  $-30, -11, 14, 27$ ) et  $SCR = 25,2$ , donc  $R^2 = 1 - 25,2/1946 \approx 0,987$ .

```
SCT = ((y - y.mean())**2).sum()
SCR_ = (residus**2).sum()
SCE = ((ajuste - y.mean())**2).sum()
print(f"SCT = {SCT:.1f} | SCE = {SCE:.1f} | SCR = {SCR_:.1f} | SCE + SCR = {SCE + SCR_:.1f}")
print("R² = 1 - SCR/SCT      :", round(1 - SCR_ / SCT, 4))
print("R² = corr(y, ŷ)²      :", round(np.corrcoef(y, ajuste)[0, 1]**2, 4))
print("R² = corr(x, y)² (simple):", round(np.corrcoef(x, y)[0, 1]**2, 4))
```

```
SCT = 1946.0 | SCE = 1920.8 | SCR = 25.2 | SCE + SCR = 1946.0
R² = 1 - SCR/SCT      : 0.9871
R² = corr(y, ŷ)²      : 0.9871
R² = corr(x, y)² (simple): 0.9871
```

**Les limites du  $R^2$ .** (1) Il **ne peut qu'augmenter** quand on ajoute une variable, même une variable sans rapport : un  $R^2$  élevé peut être le signe d'un surajustement. Le  $R^2$  **ajusté**,  $R^2_{aj} = 1 - \frac{SCR/(n-p)}{SCT/(n-1)}$ , corrige ce défaut en comparant des variances *sans biais*. (2) Un  $R^2$  faible n'est pas un échec : quand le phénomène est intrinsèquement bruité (le comportement d'achat d'un individu), un  $R^2$  de 15 % peut déjà être très informatif sur les **effets moyens**. (3) Un  $R^2$  élevé ne prouve ni que le modèle est correct, ni que la relation est causale. Rappelez-vous le quartet d'Anscombe (volume I, section 3.1.7) : quatre jeux très différents donnent la même droite et le même  $R^2$ . **Dessinez toujours.**

### 1.1.7 Première étude : le panier des clients de Dar Jasmin

Passons aux données de Dar Jasmin (simulées, rappelons-le) : 2 000 clients. Yasmine s'intéresse au **panier moyen** (en DT) des clients qui ont commandé au moins une fois dans l'année.

```
import pandas as pd
import statsmodels.formula.api as smf

clients = pd.read_csv("donnees/clients.csv")
print(clients.shape)
print(clients.head(4).to_string(index=False))
print()
print("clients sans aucune commande :", int((clients["nb_commandes_an"] == 0).sum()),
      "sur", len(clients), "(", round(100 * (clients["nb_commandes_an"] == 0).mean(), 1), "%)")
```

```
(2000, 12)
id_client  age  ville canal_acquisition date_inscription  offre_bienvenue  nb_commandes_an
↪ panier_moyen  depense_annuelle  rachat_12m  duree_mois  churn
↪      1    19  Autre      Instagram      2020-03-11           0           4
↪  40.95      116.50           0      15.89      1
↪      2    43  Tunis      Site      2019-01-10           1           2
↪  78.34      148.92           1      33.68      0
↪      3    35 Bizerte      Boutique      2020-07-02           1           6
↪  73.73      509.05           0      23.22      0
↪      4    42  Sousse      Instagram      2024-08-01           0           0
↪   0.00           0.00           0       0.96      0
```

```
clients sans aucune commande : 260 sur 2000 ( 13.0 %)
```

Les clients sans commande ont un panier moyen égal à 0 par convention : ce 0 ne signifie pas « panier nul » mais « pas de panier ». Nous les **écartons** de cette étude (c'est une décision de

périmètre, comme celle du projet du volume I) ; la section 2.6 du chapitre suivant apprendra à traiter ensemble les clients actifs et les autres.

```
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36 # âge centré sur 36 ans (l'âge moyen de la clientèle)
df["log_panier"] = np.log(df["panier_moyen"])
print(len(df), "clients actifs")
print(df["panier_moyen"].describe().round(1).to_string())
print("asymétrie du panier :", round(df["panier_moyen"].skew(), 2), "| du log du panier :",
      ↪ round(df["log_panier"].skew(), 2))
```

```
1740 clients actifs
count    1740.0
mean      61.2
std       26.4
min       17.4
25%      42.7
50%      56.0
75%      73.4
max      245.1
asymétrie du panier : 1.44 | du log du panier : 0.11
```

Le panier est nettement **asymétrique** (queue vers les gros paniers), alors que son logarithme est presque symétrique. Comme au volume I (section 3.1.5), c'est le signe qu'il vaut mieux travailler sur le logarithme : les effets seront alors **multiplicatifs** (un client « 10 % plus dépensier » dépense 10 % de plus, quel que soit son niveau de départ), ce qui est bien plus naturel pour des montants.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.5, 3.6))
ax1.hist(df["panier_moyen"], bins=40, color=BLEU, alpha=0.85)
ax1.set_xlabel("panier moyen (DT)"); ax1.set_ylabel("nombre de clients"); ax1.set_title("Panier :
      ↪ asymétrique")
ax2.hist(df["log_panier"], bins=40, color=AQUA, alpha=0.85)
ax2.set_xlabel("log du panier moyen"); ax2.set_title("Log du panier : presque symétrique")
plt.tight_layout()
plt.savefig("figures/ch01-panier-log.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

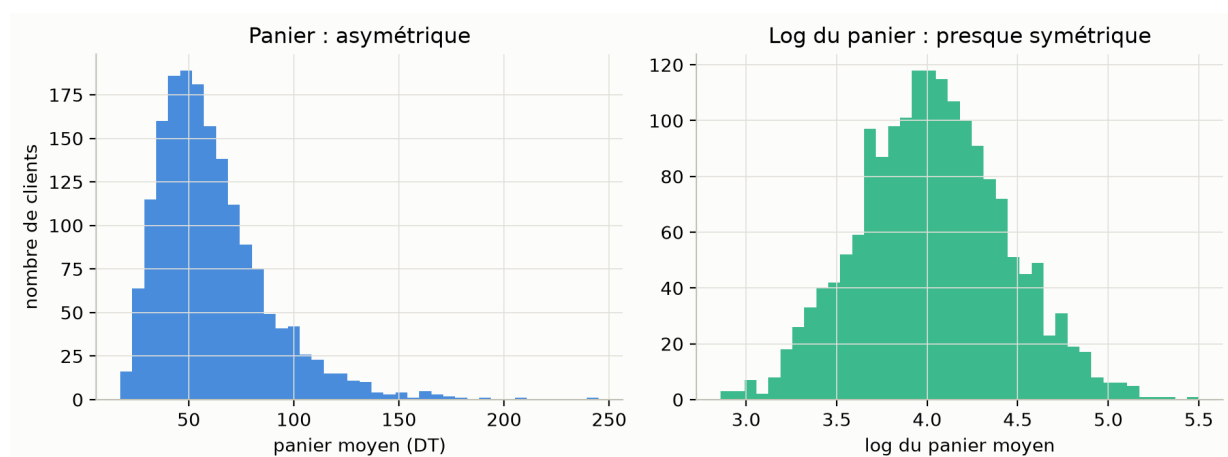


Figure 3: Le panier moyen est asymétrique (à gauche) ; son logarithme est presque symétrique (à droite).

**Régression simple.** Commençons par une seule variable : l'âge (centré, pour que la constante ait un sens : elle sera le log-panier d'un client de 36 ans).

```
m1 = smf.ols("log_panier ~ a", data=df).fit()
print(m1.summary().tables[1])
print("R² =", round(m1.rsquared, 4), "| écart-type résiduel s =", round(np.sqrt(m1.scale), 4))
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      4.0331      0.009     426.820      0.000      4.015      4.052
a              0.0090      0.001     10.050      0.000      0.007      0.011
=====
R² = 0.0549 | écart-type résiduel s = 0.3941
```

La formule `log_panier ~ a` se lit « log-panier expliqué par a » ; `statsmodels` ajoute la constante tout seul. Le tableau donne, pour chaque coefficient, l'estimation (`coef`), son erreur standard (`std err`), une statistique  $t$ , une p-valeur et un intervalle de confiance à 95 % : nous en comprendrons chaque colonne au 1.2. Retenons pour l'instant l'**interprétation** :

- La **constante** vaut environ 4,03 : un client de 36 ans a un log-panier de 4,03, soit un panier typique de  $e^{4,03} \approx 56$  DT.
- Le coefficient de **a** vaut environ 0,009 : chaque année d'âge supplémentaire est associée à une hausse de **0,9 %** du panier (log  $y$  augmente de 0,009, donc  $y$  est multiplié par  $e^{0,009} \approx 1,009$ ).
- Le  $R^2$  est faible (environ 5,5 %) : l'âge à lui seul explique peu de choses, ce qui n'a rien de surprenant.

**Variable qualitative : le codage par indicatrices.** Comment faire entrer le **canal** (Boutique/Site/Instagram), qui n'est pas un nombre ? On le remplace par des **variables indicatrices** (ou *dummies*) : une colonne par modalité *sauf une*, la **modalité de référence**, qui est absorbée par la constante. Avec Boutique pour référence :

| canal     | Site | Instagram |
|-----------|------|-----------|
| Boutique  | 0    | 0         |
| Site      | 1    | 0         |
| Instagram | 0    | 1         |

**Pourquoi pas une colonne par modalité ?** Parce que les trois colonnes s'additionnent exactement à la colonne de 1 de la constante :  $X$  ne serait plus de rang plein, et  $(X^T X)^{-1}$  n'existerait pas (violation de H2 : c'est le « piège de la variable indicatrice »). Retirer une colonne règle le problème.

Ajustons maintenant le modèle à l'âge **et** au canal :

```
m2 = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
print(m2.summary().tables[1])
print("R² =", round(m2.rsquared, 4), "| R² ajusté =", round(m2.rsquared_adj, 4), "| s =",
      round(np.sqrt(m2.scale), 4))
print()
print("Début de la matrice X (les 6 premières lignes) :")
print(m2.model.exog[:6].round(0).astype(int), " <- colonnes :", m2.model.exog_names)
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      4.2206      0.017     241.369      0.000      4.186      4.255
C(canal)[T.Site] -0.1571      0.023     -6.807      0.000     -0.202     -0.112
C(canal)[T.Instagram] -0.3368      0.022    -14.977      0.000     -0.381     -0.293
a              0.0092      0.001     10.862      0.000      0.008      0.011
=====
```



```
=====
R² = 0.1657 | R² ajusté = 0.1643 | s = 0.3705
```

Début de la matrice X (les 6 premières lignes) :

```
[[ 1  0  1 -17]
 [ 1  1  0  7]
 [ 1  0  0 -1]
 [ 1  0  1 -15]
 [ 1  1  0 -6]
 [ 1  0  1 -10]]  <- colonnes : ['Intercept', 'C(canal)[T.Site]', 'C(canal)[T.Instagram]', 'a']
```

L'écriture `C(canal)` demande à `statsmodels` de traiter `canal` comme qualitative. Voici comment lire chaque ligne :

- **Constante** ( $\approx 4,22$ ) : le log-panier d'un client de **36 ans acquis en boutique** (modalité de référence, âge à 0 après centrage), soit  $e^{4,22} \approx 68$  DT.
- **C(canal)[T.Site]** ( $\approx -0,16$ ) : à âge égal, un client acquis par le site a un log-panier inférieur de 0,16 à celui d'un client de la boutique. En pourcentage :  $e^{-0,157} \approx 0,855$ , soit un panier **environ 14,5 % plus petit**.
- **C(canal)[T.Instagram]** ( $\approx -0,34$ ) :  $e^{-0,34} \approx 0,71$  : à âge égal, un panier **29 % plus petit** qu'en boutique.
- **a** ( $\approx 0,009$ ) : à canal égal, +0,9 % de panier par année d'âge.

Et le  $R^2$  a presque triplé par rapport au modèle précédent (de 5,5 % à 16,6 %). C'est la réponse à la première question de Yasmine : *à âge égal*, un client Instagram dépense environ 29 % de moins qu'un client de la boutique.

**Un détail d'interprétation fréquemment raté.** Les coefficients se lisent à l'intérieur du **modèle**. Le coefficient de l'âge ne dit pas « l'effet de l'âge sur le panier » dans l'absolu, mais « l'effet de l'âge **quand on compare des clients de même canal** ». C'est le sens de l'expression « **toutes choses égales par ailleurs** ». Ici, le coefficient de l'âge change à peine entre `m1` et `m2` car l'âge et le canal sont presque indépendants dans nos données ; s'ils étaient liés (par exemple si Instagram attirait surtout les jeunes), le coefficient de l'âge dans `m1` mélangerait l'effet de l'âge et l'effet du canal, et il changerait nettement. Ce mélange, c'est la **confusion**.

**Le théorème de Frisch-Waugh-Lovell** : « **toutes choses égales par ailleurs** » rendu concret. Il existe une manière explicite de retrouver le coefficient de l'âge dans `m2`, qui montre ce que veut dire « tenir compte du canal » :

1. régresser le log-panier sur le **canal** seul et garder les résidus (la partie du log-panier *que le canal n'explique pas*) ;
2. régresser l'âge sur le **canal** seul et garder les résidus (la partie de l'âge *indépendante du canal*) ;
3. régresser les premiers résidus sur les seconds : la pente obtenue est **exactement** le coefficient de l'âge dans le modèle complet.

```
res_y = smf.ols("log_panier ~ C(canal)", data=df).fit().resid      # log-panier, une fois le canal «
↳ retiré »
res_a = smf.ols("a ~ C(canal)", data=df).fit().resid             # âge, une fois le canal «
↳ retiré »
pente_fwl = (res_a @ res_y) / (res_a @ res_a)
print("coefficient de l'âge dans le modèle complet :", round(m2.params["a"], 6))
print("pente obtenue par Frisch-Waugh-Lovell      :", round(pente_fwl, 6))
```

```
coefficient de l'âge dans le modèle complet : 0.00917
pente obtenue par Frisch-Waugh-Lovell      : 0.00917
```



**Pourquoi ça marche (esquisse).** Notons  $\mathbf{M}$  le projecteur sur l'orthogonal des colonnes « canal » (et constante). Par la géométrie du 1.1.4, le coefficient d'une variable  $\mathbf{a}$  dans le modèle  $[\text{canal}, \mathbf{a}]$  s'obtient en projetant  $\mathbf{y}$  sur la partie de  $\mathbf{a}$  qui n'est pas dans l'espace engendré par le canal, soit  $\mathbf{Ma} : \hat{\beta}_a = (\mathbf{Ma})^\top \mathbf{y} / \|\mathbf{Ma}\|^2$ . Comme  $\mathbf{M}$  est symétrique et idempotente,  $(\mathbf{Ma})^\top \mathbf{y} = (\mathbf{Ma})^\top (\mathbf{M}\mathbf{y})$  : c'est la pente de la régression de  $\mathbf{M}\mathbf{y}$  sur  $\mathbf{Ma}$ .

**Calculer à la main ce que fait statsmodels.** Pour boucler la boucle avec les sections précédentes, reconstruisons les coefficients de m2 par les équations normales à partir de la matrice  $\mathbf{X}$  construite par statsmodels :

```
Xd = m2.model.exog # la matrice de plan d'expérience (n x p)
yd = m2.model.endog
beta_main = np.linalg.solve(Xd.T @ Xd, Xd.T @ yd)
print(pd.DataFrame({"à la main": beta_main, "statsmodels": m2.params.to_numpy()}),
      index=m2.params.index).round(6)
print("n =", Xd.shape[0], "| p =", Xd.shape[1])
print("nombre de conditionnement de X'X :", round(np.linalg.cond(Xd.T @ Xd), 1))
```

|                       | à la main | statsmodels |
|-----------------------|-----------|-------------|
| Intercept             | 4.220621  | 4.220621    |
| C(canal)[T.Site]      | -0.157139 | -0.157139   |
| C(canal)[T.Instagram] | -0.336759 | -0.336759   |
| a                     | 0.009170  | 0.009170    |

n = 1740 | p = 4  
nombre de conditionnement de X'X : 1501.8

Les deux colonnes sont identiques : statsmodels ne fait rien de magique, il résout les équations normales (en fait, via une factorisation plus stable). Le nombre de conditionnement de  $\mathbf{X}^\top \mathbf{X}$  (volume I, section 1.5.4) est de l'ordre du millier : cela paraît grand, mais c'est surtout dû au fait que l'âge centré varie de  $-18$  à  $+37$  alors que les indicatrices valent 0 ou 1 (échelles très différentes). Ce n'est pas de la colinéarité entre variables ; nous apprendrons à la diagnostiquer proprement (VIF) en 1.3.5.

### 1.1.8 Logarithmes : interpréter et prédire

Le choix d'une transformation modifie l'interprétation des coefficients. Voici les trois cas les plus courants (où  $\beta$  est le coefficient de  $x$ ) :

| Modèle                                      | Lecture du coefficient $\beta$                                                                                                                                 |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $y = \beta_0 + \beta x$ (niveau-niveau)     | +1 unité de $x \Rightarrow y$ varie de $\beta$ unités                                                                                                          |
| $\log y = \beta_0 + \beta x$ (log-niveau)   | +1 unité de $x \Rightarrow y$ est multiplié par $e^\beta$ , soit une variation de $\approx 100\beta\%$ si $\beta$ est petit (exactement $100(e^\beta - 1)\%$ ) |
| $\log y = \beta_0 + \beta \log x$ (log-log) | +1 % de $x \Rightarrow y$ varie d'environ $\beta\%$ : $\beta$ est une élasticité                                                                               |

«  $\approx 100\beta\%$  » n'est vrai que pour les petits coefficients. Pour  $\beta = -0,34$  (Instagram), l'approximation donne  $-34\%$ , alors que la variation exacte est  $100(e^{-0,34} - 1) \approx -28,8\%$ . Au-delà de  $|\beta| \approx 0,1$ , utilisez toujours  $e^\beta - 1$ .

```
for nom, coef in m2.params.items():
    if nom == "Intercept":
        continue
    print(f"{nom:22s} coefficient = {coef:+.3f} | approximation 100*beta = {100*coef:+.1f}% | exact
          ↳ 100*(exp(beta)-1) = {100*(np.exp(coef)-1):+.1f}%")
```

```

C(canal)[T.Site]      coefficient = -0.157 | approximation 100*beta = -15.7 % | exact
↳ 100*(exp(beta)-1) = -14.5 %
C(canal)[T.Instagram] coefficient = -0.337 | approximation 100*beta = -33.7 % | exact
↳ 100*(exp(beta)-1) = -28.6 %
a                    coefficient = +0.009 | approximation 100*beta = +0.9 % | exact 100*(exp(beta)-1)
↳ = +0.9 %

```

**Prédire en échelle d'origine : le piège de la rétro-transformation.** Le modèle prédit un **log**-panier. Pour revenir en dinars, on pense à prendre l'exponentielle :  $\hat{y} = e^{\hat{\mu}}$ . Mais  $e^{\hat{\mu}}$  est la prédiction de la **médiane** du panier, pas de sa **moyenne**, à cause de l'asymétrie de la loi log-normale.

**Rétro-transformation.** Si  $\log y \sim \mathcal{N}(\mu, \sigma^2)$ , alors  $\mathbb{E}[y] = e^{\mu + \sigma^2/2}$  (c'est la fonction génératrice des moments de la loi normale :  $\mathbb{E}[e^Z] = e^{\mu + \sigma^2/2}$  pour  $Z \sim \mathcal{N}(\mu, \sigma^2)$ ), alors que la médiane est  $e^{\mu}$ . Pour prédire la **moyenne** d'un panier, il faut donc multiplier  $e^{\hat{\mu}}$  par un facteur correctif  $e^{s^2/2}$  (ou, sans supposer la normalité, par le facteur « de Duan »  $\frac{1}{n} \sum_i e^{\hat{\epsilon}_i}$ ).

```

cible = df[(df["canal"] == "Boutique") & (df["age"].between(33, 39))] # clients de la
↳ boutique, 33-39 ans
mu_hat = m2.predict(pd.DataFrame({"a": [0], "canal": pd.Categorical(["Boutique"],
↳ categories=["Boutique", "Site", "Instagram"])})).iloc[0]
s2 = m2.scale
duan = np.exp(m2.resid).mean()
print("panier moyen observé (clients boutique, 33-39 ans) :", round(cible["panier_moyen"].mean(), 2),
↳ "DT (n =", len(cible), ")")
print("exp(mu chapeau) [médiane prédite] :", round(np.exp(mu_hat), 2), "DT")
print("exp(mu chapeau + s^2/2) [moyenne, normale]:", round(np.exp(mu_hat + s2 / 2), 2), "DT")
print("exp(mu chapeau) x facteur de Duan[moyenne, libre] :", round(np.exp(mu_hat) * duan, 2), "DT")

```

```

panier moyen observé (clients boutique, 33-39 ans) : 74.33 DT (n = 110 )
exp(mu chapeau) [médiane prédite] : 68.08 DT
exp(mu chapeau + s^2/2) [moyenne, normale]: 72.91 DT
exp(mu chapeau) x facteur de Duan[moyenne, libre] : 72.96 DT

```

La moyenne observée est nettement plus proche des prédictions **corrigées** que de  $e^{\hat{\mu}}$  : ignorer la correction conduit à sous-estimer systématiquement le panier moyen (ici de plusieurs dinars), ce qui, multiplié par des milliers de clients, devient un manque à gagner dans un budget prévisionnel.

### 1.1.9 Interactions : quand un effet dépend d'un autre

Dans `m2`, l'effet de l'âge est supposé **le même** pour les trois canaux. Mais peut-être que l'âge compte plus chez les clients Instagram que chez ceux de la boutique ? On le teste en ajoutant une **interaction** : le produit de l'âge par les indicatrices du canal.

```

m3 = smf.ols("log_panier ~ a * C(canal)", data=df).fit() # a + C(canal) + a:C(canal)
print(m3.summary().tables[1])

```

|                         | coef    | std err | t       | P> t  | [0.025 | 0.975] |
|-------------------------|---------|---------|---------|-------|--------|--------|
| Intercept               | 4.2205  | 0.017   | 241.183 | 0.000 | 4.186  | 4.255  |
| C(canal)[T.Site]        | -0.1568 | 0.023   | -6.788  | 0.000 | -0.202 | -0.112 |
| C(canal)[T.Instagram]   | -0.3366 | 0.022   | -14.961 | 0.000 | -0.381 | -0.292 |
| a                       | 0.0086  | 0.002   | 5.115   | 0.000 | 0.005  | 0.012  |
| a:C(canal)[T.Site]      | 0.0010  | 0.002   | 0.463   | 0.643 | -0.003 | 0.005  |
| a:C(canal)[T.Instagram] | 0.0005  | 0.002   | 0.235   | 0.814 | -0.004 | 0.005  |

L'écriture  $a * C(\text{canal})$  signifie  $a + C(\text{canal}) + a:C(\text{canal})$ . Les deux lignes  $a:C(\text{canal})[T...]$  mesurent la **différence de pente** de l'âge entre le canal considéré et la boutique. Si ces coefficients sont proches de zéro (et statistiquement indiscernables de zéro, ce que nous formaliserons par un test  $F$  au 1.2.4), le modèle sans interaction suffit. Nous verrons au 1.4 comment choisir objectivement entre  $m_2$  et  $m_3$ .

### À retenir (1.1).

- Le modèle linéaire s'écrit  $y = X\beta + \varepsilon$  ; il est linéaire **dans les paramètres**. Les hypothèses H1 à H5 servent chacune à quelque chose de précis.
- Les **équations normales**  $X^T X \hat{\beta} = X^T y$  donnent  $\hat{\beta} = (X^T X)^{-1} X^T y$ , unique grâce à la convexité ; en pratique on résout, on n'inverse pas.
- Géométriquement,  $\hat{y} = Hy$  est la **projection orthogonale** de  $y$  ; les résidus sont orthogonaux aux colonnes de  $X$  ;  $\text{tr } H = p$ .
- Sous H1-H4 :  $\hat{\beta}$  est sans biais, de variance  $\sigma^2 (X^T X)^{-1}$ , et le **meilleur estimateur linéaire sans biais** (Gauss-Markov). On estime  $\sigma^2$  par  $\text{SCR}/(n - p)$ .
- $R^2 = 1 - \text{SCR}/\text{SCT}$  mesure la part de variance expliquée ; il ne peut qu'augmenter avec le nombre de variables (préférer  $R^2$  ajusté pour comparer).
- Une variable qualitative entre par des **indicateurs** (une modalité de référence) ; un coefficient se lit « toutes choses égales par ailleurs » (Frisch-Waugh-Lovell).
- Avec  $\log y$ , un coefficient  $\beta$  correspond à un effet multiplicatif  $e^\beta$  ; pour prédire la **moyenne** en dinars, corrigez la rétro-transformation.

## 1.2 Inférence sur les coefficients

**Intuition.** Les coefficients de  $m_2$  ( $-0,34$  pour Instagram,  $+0,009$  par année d'âge...) sont calculés sur **un** échantillon de 1 740 clients. Avec un autre échantillon, on aurait obtenu d'autres valeurs. La question de l'inférence est : *de combien ces chiffres peuvent-ils bouger ? et donc que peut-on affirmer sur la vraie valeur ?* C'est exactement l'esprit du chapitre 3 du volume I (intervalles de confiance, tests), appliqué maintenant à chaque coefficient d'un modèle.

On part des mêmes données qu'en 1.1. Voici la préparation (identique), que nous ne détaillerons plus :

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, ENCRE, GRIS = "#2a78d6", "#eb6834", "#1baf7a", "#0b0b0b", "#898781"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
    "#e1e0d9",
    "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
    "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])

m1 = smf.ols("log_panier ~ a", data=df).fit()
```

```
m2 = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
m3 = smf.ols("log_panier ~ a * C(canal)", data=df).fit()
print(len(df), "clients | n - p pour m2 =", int(m2.df_resid))
```

```
1740 clients | n - p pour m2 = 1736
```

### 1.2.1 La loi des estimateurs sous l'hypothèse de normalité

Au 1.1.5, nous avons établi l'espérance et la variance de  $\hat{\beta}$ . Pour fabriquer des tests et des intervalles exacts, il faut sa loi entière : c'est le rôle de l'hypothèse H5 (erreurs normales).

**Théorème 3.** Sous H1 à H5 :

1.  $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1})$  ;
2.  $\frac{(n-p)s^2}{\sigma^2} = \frac{\text{SCR}}{\sigma^2} \sim \chi_{n-p}^2$  ;
3.  $\hat{\beta}$  et  $s^2$  sont **indépendants**.

*Démonstration.* (1) On a vu que  $\hat{\beta} = \beta + \mathbf{A}\varepsilon$  avec  $\mathbf{A} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$  : c'est une transformation linéaire d'un vecteur gaussien, donc un vecteur gaussien, d'espérance  $\beta$  et de variance  $\sigma^2 \mathbf{A} \mathbf{A}^\top = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$ .

- (3) Le résidu est  $\hat{\varepsilon} = (\mathbf{I} - \mathbf{H})\varepsilon$ . Le couple  $(\hat{\beta}, \hat{\varepsilon})$  est gaussien (transformation linéaire de  $\varepsilon$ ), et sa covariance croisée vaut  $\text{Cov}(\mathbf{A}\varepsilon, (\mathbf{I} - \mathbf{H})\varepsilon) = \sigma^2 \mathbf{A}(\mathbf{I} - \mathbf{H})^\top = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{I} - \mathbf{H}) = \mathbf{0}$ , car  $\mathbf{X}^\top (\mathbf{I} - \mathbf{H}) = \mathbf{0}$  (1.1.4). Pour des vecteurs gaussiens **conjoins**, covariance nulle  $\Leftrightarrow$  indépendance. Comme  $s^2$  ne dépend que de  $\hat{\varepsilon}$ , on obtient (3).

- (4)  $\text{SCR}/\sigma^2 = (\varepsilon/\sigma)^\top (\mathbf{I} - \mathbf{H})(\varepsilon/\sigma)$ , où  $\varepsilon/\sigma \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$  et  $\mathbf{I} - \mathbf{H}$  est un projecteur orthogonal de rang  $n - p$ . Dans une base orthonormée adaptée au projecteur, c'est la somme de  $n - p$  carrés de lois  $\mathcal{N}(0, 1)$  indépendantes : une loi  $\chi_{n-p}^2$  (théorème de Cochran).  $\square$

Pour un coefficient  $\beta_j$ , la formule (1) donne  $\hat{\beta}_j \sim \mathcal{N}(\beta_j, \sigma^2 c_{jj})$  où  $c_{jj}$  est le  $j$ -ième élément diagonal de  $(\mathbf{X}^\top \mathbf{X})^{-1}$ . Comme  $\sigma$  est inconnu, on le remplace par  $s$  : l'**erreur standard** de  $\hat{\beta}_j$  est  $\text{se}(\hat{\beta}_j) = s\sqrt{c_{jj}}$ . Et le rapport d'une normale à la racine d'un  $\chi^2$  indépendant, divisé par ses degrés de liberté, est une loi de Student (volume I, section 3.3.3) :

$$T_j = \frac{\hat{\beta}_j - \beta_j}{\text{se}(\hat{\beta}_j)} \sim t_{n-p}$$

Reconstruisons à la main le tableau de statsmodels pour m2, colonne par colonne :

```
X = m2.model.exog
y = m2.model.endog
n, p = X.shape
XtX_inv = np.linalg.inv(X.T @ X)
beta = XtX_inv @ X.T @ y
residus = y - X @ beta
s2 = residus @ residus / (n - p)
se = np.sqrt(s2 * np.diag(XtX_inv))
t = beta / se
pval = 2 * stats.t.sf(np.abs(t), df=n - p)
crit = stats.t.ppf(0.975, df=n - p)
tab = pd.DataFrame({"coef": beta, "std err": se, "t": t, "P>|t|": pval,
                    "IC bas": beta - crit * se, "IC haut": beta + crit * se}, index=m2.params.index)
print(tab.round(4))
print()
print("identique à statsmodels :", np.allclose(tab["std err"], m2.bse), np.allclose(tab["t"],
    ↪ m2.tvalues),
```

```
np.allclose(tab[["IC bas", "IC haut"]].to_numpy(), m2.conf_int().to_numpy())
print(f"s = {np.sqrt(s2):.4f} | quantile t(n-p, 97,5 %) = {crit:.4f} (presque 1,96 : n - p = {n - p}
↳ est grand)")
```

|                       | coef    | std err | t        | P> t | IC bas  | IC haut |
|-----------------------|---------|---------|----------|------|---------|---------|
| Intercept             | 4.2206  | 0.0175  | 241.3693 | 0.0  | 4.1863  | 4.2549  |
| C(canal)[T.Site]      | -0.1571 | 0.0231  | -6.8065  | 0.0  | -0.2024 | -0.1119 |
| C(canal)[T.Instagram] | -0.3368 | 0.0225  | -14.9770 | 0.0  | -0.3809 | -0.2927 |
| a                     | 0.0092  | 0.0008  | 10.8618  | 0.0  | 0.0075  | 0.0108  |

identique à statsmodels : True True True

s = 0.3705 | quantile t(n-p, 97,5 %) = 1.9613 (presque 1,96 : n - p = 1736 est grand)

Chaque colonne a désormais une origine claire : l'erreur standard vient de  $s\sqrt{c_{jj}}$ , la statistique  $t$  est le rapport coefficient / erreur standard, la p-valeur est la probabilité qu'un Student à  $n - p$  degrés de liberté dépasse  $|t|$  en valeur absolue, et l'intervalle de confiance est  $\hat{\beta}_j \pm t_{n-p, 0,975} se(\hat{\beta}_j)$  (c'est la construction du volume I, section 3.3.3, appliquée à chaque coefficient).

### 1.2.2 Tester un coefficient, lire un intervalle

Le **test de Student** d'un coefficient teste  $H_0 : \beta_j = 0$  contre  $H_1 : \beta_j \neq 0$  : « une fois les autres variables prises en compte, cette variable apporte-t-elle une information ? ». La statistique est  $t_j = \hat{\beta}_j / se(\hat{\beta}_j)$ , et on rejette  $H_0$  au niveau 5 % si  $|t_j| > t_{n-p, 0,975} \approx 1,96$ .

Dans le tableau précédent, les trois coefficients du canal et de l'âge ont des  $|t|$  très supérieurs à 1,96 (la plus petite valeur, pour l'âge, est de l'ordre de 11) : les p-valeurs sont minuscules : elles s'affichent 0.000 dans le tableau de statsmodels (ce qui signifie « inférieur à 0,0005 », et non « exactement nul »), et 0.0 dans notre tableau arrondi à quatre décimales.

**Rappels du volume I, appliqués ici.** (1) Une p-valeur n'est **pas** la probabilité que  $H_0$  soit vraie (3.5.2). (2) « Significatif » n'est pas « important » (3.5.3) : avec 1 740 clients, même un très petit effet serait détecté. Il faut donc toujours lire **l'estimation et son intervalle**, pas seulement le test. (3) Si l'on teste beaucoup de coefficients, il faut se méfier des faux positifs (3.5.5) : nous y reviendrons au 1.4.

L'**intervalle de confiance** est bien plus informatif que la p-valeur. Pour Instagram, l'intervalle sur le log-panier est environ  $[-0,381 ; -0,293]$  ; en passant à l'exponentielle (une fonction croissante conserve les bornes), on obtient une **fourchette sur l'effet multiplicatif** :

```
ic = m2.conf_int()
for nom in ["C(canal)[T.Site]", "C(canal)[T.Instagram]"]:
    bas, haut = ic.loc[nom]
    print(f"{nom:24s} effet sur le panier : {100*(np.exp(m2.params[nom])-1):+.1f} % IC95 % :
↳ [{100*(np.exp(bas)-1):+.1f} % ; {100*(np.exp(haut)-1):+.1f} %]")
bas, haut = ic.loc["a"]
print(f"'a (10 ans de plus)':24s} effet sur le panier : {100*(np.exp(10*m2.params['a'])-1):+.1f} %
↳ IC95 % : [{100*(np.exp(10*bas)-1):+.1f} % ; {100*(np.exp(10*haut)-1):+.1f} %]")
```

|                       |                               |                              |
|-----------------------|-------------------------------|------------------------------|
| C(canal)[T.Site]      | effet sur le panier : -14.5 % | IC95 % : [-18.3 % ; -10.6 %] |
| C(canal)[T.Instagram] | effet sur le panier : -28.6 % | IC95 % : [-31.7 % ; -25.4 %] |
| a (10 ans de plus)    | effet sur le panier : +9.6 %  | IC95 % : [+7.8 % ; +11.4 %]  |

La phrase honnête à transmettre à Yasmine est donc : « à âge égal, un client acquis par Instagram dépense environ 29 % de moins qu'un client de la boutique ; avec 95 % de confiance, la vraie différence se situe entre 25 % et 32 % de moins ». Et pour l'âge : « dix ans de plus sont associés à un panier de 8 à 11 % plus élevé ».

**Un test peut aussi porter sur une combinaison de coefficients.** Par exemple : « le Site et Instagram ont-ils le même panier (à âge égal) ? ». L'hypothèse est  $H_0 : \beta_{\text{Site}} - \beta_{\text{Instagram}} = 0$ , c'est-à-dire  $H_0 : \mathbf{c}^T \boldsymbol{\beta} = 0$  avec  $\mathbf{c} = (0, 1, -1, 0)^T$ . La variance de  $\mathbf{c}^T \hat{\boldsymbol{\beta}}$  est  $\sigma^2 \mathbf{c}^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{c}$  : les covariances entre coefficients **comptent** (on ne peut pas se contenter de lire les deux erreurs standard du tableau).

```
c = np.array([0, 1, -1, 0])
diff = c @ beta
se_diff = np.sqrt(s2 * c @ XtX_inv @ c)
print(f"beta_Site - beta_Instagram = {diff:.4f} | erreur standard = {se_diff:.4f} | t =
↳ {diff/se_diff:.2f} | p = {2*stats.t.sf(abs(diff/se_diff), n-p):.2e}")
print(m2.t_test("C(canal)[T.Site] - C(canal)[T.Instagram] = 0"))
```

```
beta_Site - beta_Instagram = 0.1796 | erreur standard = 0.0207 | t = 8.69 | p = 8.16e-18
Test for Constraints
```

|    | coef   | std err | t     | P> t  | [0.025 | 0.975] |
|----|--------|---------|-------|-------|--------|--------|
| c0 | 0.1796 | 0.021   | 8.691 | 0.000 | 0.139  | 0.220  |

### 1.2.3 Le test $F$ : tester plusieurs coefficients à la fois

Comment tester que le **canal** compte, quand il se traduit par *deux* coefficients (Site et Instagram) ? Faire deux tests  $t$  séparés ne répond pas à la question (et multiplie les risques de faux positif). On utilise un **test  $F$  de modèles emboîtés** : on compare le modèle complet  $M_1$  (avec  $p_1$  paramètres) au modèle réduit  $M_0$  (avec  $p_0 < p_1$  paramètres, obtenu en imposant  $q = p_1 - p_0$  contraintes, par exemple « les deux coefficients du canal sont nuls »).

**Statistique de Fisher.** En notant  $\text{SCR}_0$  et  $\text{SCR}_1$  les sommes de carrés résiduelles des deux modèles (le modèle réduit ajuste toujours moins bien :  $\text{SCR}_0 \geq \text{SCR}_1$ ),

$$F = \frac{(\text{SCR}_0 - \text{SCR}_1)/q}{\text{SCR}_1/(n - p_1)} \sim F_{q, n-p_1} \quad \text{sous } H_0.$$

*Pourquoi cette loi ?* Notons  $\mathbf{H}_0$  et  $\mathbf{H}_1$  les matrices chapeau des deux modèles (l'image de  $\mathbf{H}_0$  est contenue dans celle de  $\mathbf{H}_1$ ). Sous  $H_0$ , la vraie moyenne  $\mathbf{X}\boldsymbol{\beta}$  appartient au petit sous-espace, donc  $\text{SCR}_0 - \text{SCR}_1 = \boldsymbol{\varepsilon}^T (\mathbf{H}_1 - \mathbf{H}_0) \boldsymbol{\varepsilon}$  et  $\text{SCR}_1 = \boldsymbol{\varepsilon}^T (\mathbf{I} - \mathbf{H}_1) \boldsymbol{\varepsilon}$ . Les matrices  $\mathbf{H}_1 - \mathbf{H}_0$  et  $\mathbf{I} - \mathbf{H}_1$  sont des projecteurs orthogonaux entre eux, de rangs  $q$  et  $n - p_1$  : par le théorème de Cochran, ce sont deux variables **indépendantes** de lois  $\sigma^2 \chi_q^2$  et  $\sigma^2 \chi_{n-p_1}^2$ . Leur rapport, une fois divisé par les degrés de liberté, est par définition une loi de Fisher.  $\square$

L'intuition est limpide : le numérateur mesure **combien l'ajustement se dégrade** (par contrainte) quand on retire les variables ; le dénominateur est l'échelle de bruit du modèle complet. Si retirer les variables ne dégrade pas plus que du bruit,  $F$  est proche de 1.

```
# Le canal compte-t-il ? M0 : log_panier ~ a (m1) contre M1 : log_panier ~ a + canal (m2)
scr0, scr1 = m1.ssr, m2.ssr
q = int(m2.df_model - m1.df_model)
F = ((scr0 - scr1) / q) / (scr1 / m2.df_resid)
pF = stats.f.sf(F, q, m2.df_resid)
print(f"SCR0 = {scr0:.2f} | SCR1 = {scr1:.2f} | q = {q} | F = {F:.2f} | p = {pF:.2e}")
print()
print(sm.stats.anova_lm(m1, m2).round(4))
```

```
SCR0 = 269.94 | SCR1 = 238.30 | q = 2 | F = 115.26 | p = 9.98e-48
```

|   | df_resid | ssr      | df_diff | ss_diff | F      | Pr(>F) |
|---|----------|----------|---------|---------|--------|--------|
| 0 | 1738.0   | 269.9406 | 0.0     | NaN     | NaN    | NaN    |
| 1 | 1736.0   | 238.2975 | 2.0     | 31.6431 | 115.26 | 0.0    |



La statistique  $F$  calculée à la main coïncide avec celle de `anova_lm`. Le canal est donc **très significatif** dans son ensemble.

Le même outil teste d'autres questions :

- **Le test global** du tableau de résultats (F-statistic dans `summary()`) compare le modèle complet au modèle réduit à la **seule constante** : « au moins une variable explicative est-elle utile ? ».
- **L'interaction âge × canal** (1.1.9) :  $m_3$  contre  $m_2$ , avec  $q = 2$  contraintes (« les deux différences de pente sont nulles »).
- **Un cas particulier** : quand  $q = 1$ ,  $F = t^2$  (le test  $F$  d'un seul coefficient est le carré du test  $t$ ).

```
print("Interaction âge x canal (m2 contre m3) :")
print(sm.stats.anova_lm(m2, m3).round(4))
print()
a_t = m2.tvalues["a"]
print("cas q = 1 : t² de l'âge =", round(a_t**2, 3), "| F de la suppression de l'âge =",
      ↪ round(sm.stats.anova_lm(sm.ols("log_panier ~ C(canal)", df).fit(), m2).loc[1, "F"], 3))
print()
print("test global (m2) : F =", round(m2.fvalue, 2), "| p =", f"{m2.f_pvalue:.2e}")
```

```
Interaction âge x canal (m2 contre m3) :
   df_resid      ssr df_diff  ss_diff      F Pr(>F)
0    1736.0  238.2975     0.0      NaN     NaN   NaN
1    1734.0  238.2677     2.0   0.0299   0.1087  0.897
```

```
cas q = 1 : t² de l'âge = 117.979 | F de la suppression de l'âge = 117.979
```

```
test global (m2) : F = 114.93 | p = 6.88e-68
```

Pour l'interaction, la p-valeur est grande : rien n'indique que l'effet de l'âge diffère selon le canal. Le modèle plus simple  $m_2$  suffit.

**Que se passe-t-il si les erreurs ne sont pas normales ?** Pour un grand échantillon, le théorème central limite (volume I, section 2.4.3) assure que  $\hat{\beta}$  est approximativement normal même si les erreurs ne le sont pas, de sorte que les tests  $t$  et  $F$  restent *approximativement* valides (avec les lois asymptotiques). Pour un petit échantillon avec des erreurs très asymétriques, il vaut mieux recourir au **bootstrap** (voir plus bas).

### 1.2.4 Intervalle de confiance et intervalle de prédiction

Deux questions très différentes se cachent derrière « prédire » :

1. **Quel est le panier moyen** des clients Instagram de 25 ans ? Il s'agit d'estimer une **espérance**  $\mathbf{x}_0^\top \beta$  : on veut un **intervalle de confiance de la moyenne**.
2. **Quel sera le panier** d'un nouveau client Instagram de 25 ans ? Il s'agit de prévoir une **observation**  $y_0 = \mathbf{x}_0^\top \beta + \varepsilon_0$  : on veut un **intervalle de prédiction**, plus large, car il faut ajouter le bruit individuel  $\varepsilon_0$ .

**Les deux formules.** La valeur prédite est  $\hat{y}_0 = \mathbf{x}_0^\top \hat{\beta}$ , de variance  $\sigma^2 \mathbf{x}_0^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_0$ . Posons  $h_0 = \mathbf{x}_0^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_0$  (le « levier » du point  $\mathbf{x}_0$ ). Alors

IC de la moyenne :  $\hat{y}_0 \pm t_{n-p, 0.975} s \sqrt{h_0}$ , intervalle de prédiction :  $\hat{y}_0 \pm t_{n-p, 0.975} s \sqrt{1 + h_0}$ .

Dans le second cas, l'erreur de prévision est  $y_0 - \hat{y}_0 = \varepsilon_0 - \mathbf{x}_0^\top (\hat{\beta} - \beta)$  : somme de deux

termes **indépendants** ( $\varepsilon_0$  est un nouveau bruit, indépendant de l'échantillon), d'où la variance  $\sigma^2(1 + h_0)$ .

**À la main, sur les quatre commandes du 1.1.1.** Quel montant prévoir pour une commande de 5 articles ? On a  $\hat{y}_0 = 3 + 19,6 \times 5 = 101$  DT,  $s^2 = \text{SCR}/(n - p) = 25,2/2 = 12,6$ , et  $\mathbf{x}_0 = (1, 5)^\top$  donne  $h_0 = \frac{1}{20}(30 - 2 \cdot 10 \cdot 5 + 4 \cdot 25) = \frac{30}{20} = 1,5$  (calcul avec  $(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{20} \begin{pmatrix} 30 & -10 \\ -10 & 4 \end{pmatrix}$ ). Avec  $n - p = 2$  degrés de liberté,  $t_{2,0,975} \approx 4,303$  : IC de la moyenne  $101 \pm 4,303\sqrt{12,6 \times 1,5} \approx 101 \pm 18,7$  ; intervalle de prédiction  $101 \pm 4,303\sqrt{12,6 \times 2,5} \approx 101 \pm 24,2$ . Vérifions :

```
x4 = np.array([1, 2, 3, 4]); y4 = np.array([22, 41, 66, 79])
d4 = pd.DataFrame({"x": x4, "y": y4})
mp = smf.ols("y ~ x", d4).fit()
nouveau = pd.DataFrame({"x": [5]})
pred = mp.get_prediction(nouveau).summary_frame(alpha=0.05)
print(pred.round(2).to_string())
x0 = np.array([1, 5]); X4 = np.column_stack([np.ones(4), x4])
h0 = x0 @ np.linalg.inv(X4.T @ X4) @ x0
s2_4 = mp.scale; tq = stats.t.ppf(0.975, 2)
print("à la main : h0 =", round(h0, 3), "| IC moyenne ±", round(tq*np.sqrt(s2_4*h0), 2), "| IC
↳ prédiction ±", round(tq*np.sqrt(s2_4*(1+h0)), 2))
```

|   | mean  | mean_se | mean_ci_lower | mean_ci_upper | obs_ci_lower | obs_ci_upper |
|---|-------|---------|---------------|---------------|--------------|--------------|
| 0 | 101.0 | 4.35    | 82.29         | 119.71        | 76.85        | 125.15       |

à la main : h0 = 1.5 | IC moyenne ± 18.71 | IC prédiction ± 24.15

Ces intervalles sont énormes parce que  $n = 4$  et que  $x_0 = 5$  est **hors de la plage** des données (1 à 4) :  $h_0 = 1,5$  est grand. C'est une propriété générale :  $h_0$  **croît quand  $x_0$  s'éloigne du centre des données**, si bien que l'incertitude explose en **extrapolation**.

**Sur les clients de Dar Jasmin.** Dessinons, pour les clients acquis par Instagram, le nuage log-panier contre âge avec la droite ajustée par  $m_2$ , la bande de confiance de la moyenne et la bande de prédiction.

```
grille = pd.DataFrame({"age": np.arange(18, 76)})
grille["a"] = grille["age"] - 36
grille["canal"] = pd.Categorical(["Instagram"] * len(grille), categories=["Boutique", "Site",
↳ "Instagram"])
pf = m2.get_prediction(grille).summary_frame(alpha=0.05)

insta = df[df["canal"] == "Instagram"]
fig, ax = plt.subplots(figsize=(7.4, 4.6))
ax.scatter(insta["age"], insta["log_panier"], s=9, color=GRIS, alpha=0.55, label="clients Instagram")
ax.fill_between(grille["age"], pf["obs_ci_lower"], pf["obs_ci_upper"], color=ORANGE, alpha=0.15,
↳ label="intervalle de prédiction à 95 % (un client)")
ax.fill_between(grille["age"], pf["mean_ci_lower"], pf["mean_ci_upper"], color=BLEU, alpha=0.45,
↳ label="intervalle de confiance à 95 % (le panier moyen)")
ax.plot(grille["age"], pf["mean"], color=BLEU, lw=2)
ax.set_xlabel("âge (ans)")
ax.set_ylabel("log du panier moyen")
ax.legend(frameon=False, loc="upper left", fontsize=8.5)
ax.set_ylim(2.3, 5.9)
plt.savefig("figures/ch01-bandes-prediction.png", dpi=200, bbox_inches="tight")
dans = ((insta["log_panier"] >= np.interp(insta["age"], grille["age"], pf["obs_ci_lower"])) &
↳ (insta["log_panier"] <= np.interp(insta["age"], grille["age"], pf["obs_ci_upper"])))
print(f"part des {len(insta)} clients Instagram situés dans l'intervalle de prédiction à 95 % :
↳ {100*dans:.1f} %")
```

part des 687 clients Instagram situés dans l'intervalle de prédiction à 95 % : 94.8 %



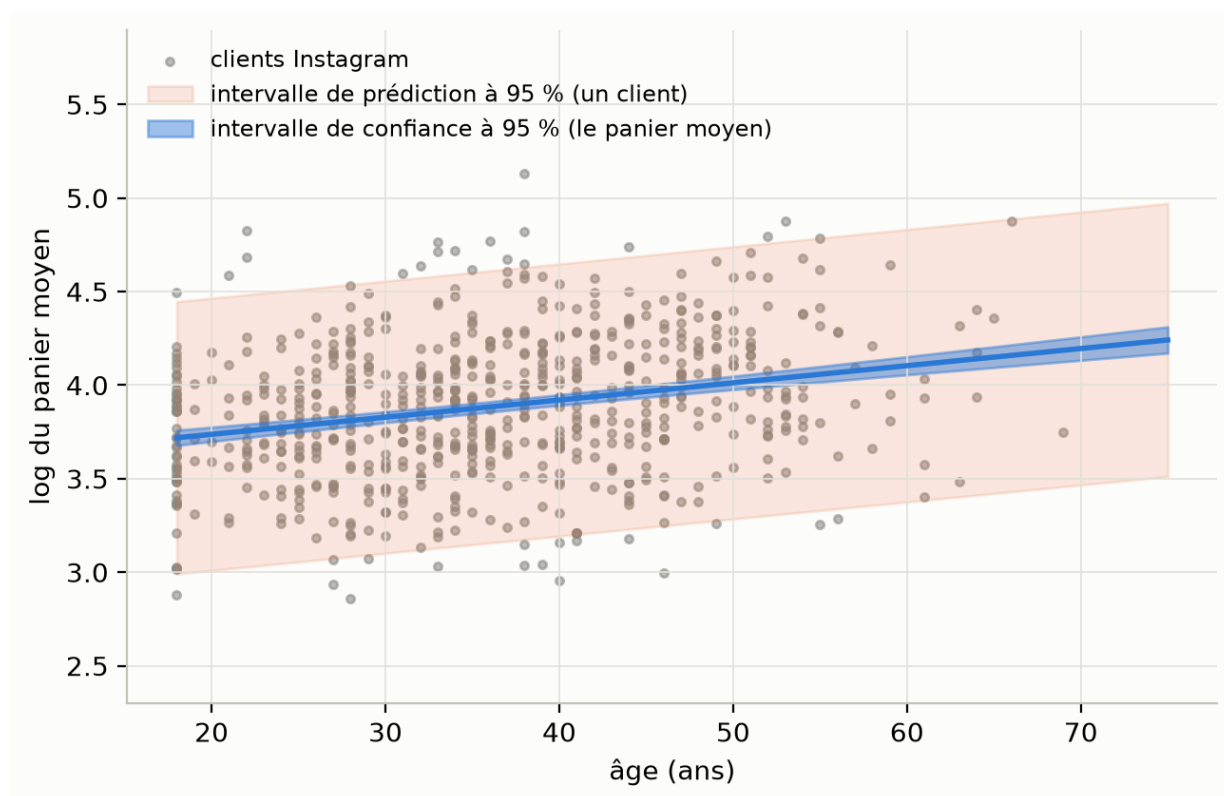


Figure 4: Clients Instagram : log du panier selon l'âge. La bande bleue (intervalle de confiance de la moyenne) est étroite ; la bande orange (prédiction pour un client) est beaucoup plus large et contient environ 95 % des points.

Deux observations. La bande de **confiance** est étroite et se resserre autour de l'âge moyen (là où  $h_0$  est minimal), tout en s'évasant aux âges extrêmes. La bande de **prédiction** est quasi parallèle à la droite et très large : elle est dominée par le terme « 1 » (le bruit individuel), que **rien** ne peut réduire, même avec un échantillon infini. Retenons : *on peut connaître très précisément le panier moyen d'un groupe, et pourtant prévoir très mal le panier d'un individu.*

En dinars, il suffit d'appliquer l'exponentielle aux bornes (la transformation est croissante) :

```
nouveau = pd.DataFrame({"a": [25 - 36], "canal": pd.Categorical(["Instagram"],
↳ categories=["Boutique", "Site", "Instagram"])}))
r = m2.get_prediction(nouveau).summary_frame(alpha=0.05).iloc[0]
print(f"log-panier prédit : {r['mean']:.3f}")
print(f"panier médian prédit : {np.exp(r['mean']):.1f} DT | IC95 % de la médiane :
↳ [{np.exp(r['mean_ci_lower']):.1f} ; {np.exp(r['mean_ci_upper']):.1f}] DT")
print(f"un nouveau client Instagram de 25 ans : panier entre {np.exp(r['obs_ci_lower']):.1f} et
↳ {np.exp(r['obs_ci_upper']):.1f} DT avec 95 % de confiance")
```

```
log-panier prédit : 3.783
panier médian prédit : 43.9 DT | IC95 % de la médiane : [42.5 ; 45.4] DT
un nouveau client Instagram de 25 ans : panier entre 21.2 et 91.0 DT avec 95 % de confiance
```

Remarquez le vocabulaire : l'exponentielle des bornes de l'IC de  $E[\log y]$  donne un intervalle pour la **médiane** de  $y$  (et non pour sa moyenne, cf. 1.1.8). L'intervalle de prédiction, lui, se transforme sans difficulté, car il concerne une observation.

### 1.2.5 Vérifier la théorie par simulation : la couverture des intervalles

Tout ceci repose sur H1-H5. Que vaut vraiment « 95 % de confiance » ? Vérifions-le comme on vérifie un théorème : en **répétant l'expérience** un grand nombre de fois quand on connaît la vérité. Utilisons le plan d'expérience réel  $X$  de  $m_2$  (mêmes 1 740 clients), des coefficients vrais  $\beta^*$  choisis par nous, un bruit normal d'écart-type 0,37, et générons 4 000 jeux de données. Pour chacun, nous construisons l'intervalle à 95 % du coefficient d'Instagram et vérifions s'il contient la vraie valeur.

```
rng = np.random.default_rng(12)
beta_etoile = np.array([4.22, -0.17, -0.34, 0.009]) # nos « vrais » coefficients
sigma_v = 0.37
R = 4000
Ysim = X @ beta_etoile + rng.normal(0, sigma_v, size=(R, n)) # R jeux de données de n clients
↳ (lignes)
B = Ysim @ (XtX_inv @ X.T).T # R estimations de beta (chaque ligne
↳ = un jeu)
E = Ysim - B @ X.T
S2 = (E**2).sum(axis=1) / (n - p)
SE = np.sqrt(S2[:, None] * np.diag(XtX_inv)[None, :])
Tst = (B - beta_etoile) / SE # R x p statistiques t « vraies »
↳ (centrées sur la vérité)
low, high = B - crit * SE, B + crit * SE
couverture = ((low <= beta_etoile) & (beta_etoile <= high)).mean(axis=0)
print("couverture empirique de l'IC à 95 % :")
print(pd.Series(couverture, index=m2.params.index).round(3).to_string())
print("statistique t pour Instagram : moyenne =", Tst[:, 2].mean().round(3), "| écart-type =", Tst[:,
↳ 2].std().round(3),
      "| écart-type théorique de t(n-p) =", round(np.sqrt((n - p) / (n - p - 2)), 3))
print("part de |t| > 1,96 quand H0 est vraie (erreur de type I) pour l'âge :", (np.abs(Tst[:, 3]) >
↳ 1.96).mean().round(3))
```

```
couverture empirique de l'IC à 95 % :
Intercept          0.944
C(canal)[T.Site]    0.951
C(canal)[T.Instagram] 0.948
a                   0.950
statistique t pour Instagram : moyenne = 0.008 | écart-type = 1.01 | écart-type théorique de t(n-p) =
↳ 1.001
part de |t| > 1,96 quand H0 est vraie (erreur de type I) pour l'âge : 0.05
```

Les intervalles à 95 % couvrent la vraie valeur dans environ 95 % des jeux de données, pour chacun des quatre coefficients, et la statistique  $t$  centrée sur la vérité se comporte comme une loi de Student (moyenne nulle, écart-type voisin de 1). Quand l'hypothèse nulle est vraie, le test  $t$  à 5 % se trompe dans environ 5 % des cas : c'est exactement ce que promet la théorie. **Mais** n'oublions pas que cette simulation **respecte** H1 à H5 par construction. Dans la vraie vie, la couverture dépend de la qualité du modèle : c'est tout l'objet de la section 1.3.

### 1.2.6 Quand on ne veut pas supposer la normalité : le bootstrap des couples

Le bootstrap du volume I (section 3.3.5) s'étend à la régression : on tire au hasard, **avec remise**, des **clients entiers** (le vecteur  $(y_i, x_i)$ , d'où le nom de « bootstrap des couples »), on réajuste le modèle sur chaque rééchantillon, et on observe la variabilité des coefficients. Aucune formule, aucune hypothèse de normalité.

```
rng = np.random.default_rng(5)
B_boot = 2000
coefs = np.empty((B_boot, p))
for b in range(B_boot):
    idx = rng.integers(0, n, n)
    coefs[b] = np.linalg.lstsq(X[idx], y[idx], rcond=None)[0]
ic_boot = np.percentile(coefs, [2.5, 97.5], axis=0)
```

```
comp = pd.DataFrame({"IC t (bas)": m2.conf_int()[0].to_numpy(), "IC t (haut)":
↪ m2.conf_int()[1].to_numpy(),
                    "IC bootstrap (bas)": ic_boot[0], "IC bootstrap (haut)": ic_boot[1],
                    "se (formule)": m2.bse.to_numpy(), "se (bootstrap)": coefs.std(axis=0, ddof=1)},
                    index=m2.params.index)
print(comp.round(4).to_string())
```

|                       | IC t (bas) | IC t (haut) | IC bootstrap (bas) | IC bootstrap (haut) | se (formule) |
|-----------------------|------------|-------------|--------------------|---------------------|--------------|
| ↪ se (bootstrap)      |            |             |                    |                     |              |
| Intercept             | 4.1863     | 4.2549      | 4.1868             | 4.2545              | 0.0175       |
| ↪ 0.0172              |            |             |                    |                     |              |
| C(canal)[T.Site]      | -0.2024    | -0.1119     | -0.2021            | -0.1131             | 0.0231       |
| ↪ 0.0227              |            |             |                    |                     |              |
| C(canal)[T.Instagram] | -0.3809    | -0.2927     | -0.3794            | -0.2931             | 0.0225       |
| ↪ 0.0216              |            |             |                    |                     |              |
| a                     | 0.0075     | 0.0108      | 0.0075             | 0.0108              | 0.0008       |
| ↪ 0.0008              |            |             |                    |                     |              |

Les deux approches donnent des intervalles quasi identiques : ici, la formule théorique est fiable. Le bootstrap devient précieux quand les hypothèses sont douteuses (erreurs très asymétriques, petit échantillon, quantité d'intérêt compliquée comme un rapport de coefficients).

### 1.2.7 Retour aux questions de Yasmine

Nous pouvons maintenant répondre honnêtement aux trois questions de l'introduction du chapitre :

1. « **Combien dépense un client Instagram de 50 ans par rapport à un client de la boutique de 30 ans ?** » C'est une combinaison linéaire de coefficients : effet du canal ( $-0,34$ ) plus 20 ans d'âge ( $+20 \times 0,009$ ). Son intervalle de confiance s'obtient par la formule de variance d'une combinaison (1.2.2).
2. « **Est-ce le canal ou l'âge ?** » Le test  $F$  du canal est très significatif à **âge égal** (1.2.3) : ce n'est pas l'âge. Et réciproquement.
3. « **Quel panier prévoir pour un nouveau client ?** » Un intervalle de prédiction, large (1.2.4).

```
c1 = np.array([0, 0, 1, 20.0]) # coef_Instagram + 20 * coef_age (par rapport à la
↪ référence « Boutique, 30 ans »)
est = c1 @ beta
se_c = np.sqrt(s2 * c1 @ XtX_inv @ c1)
bas, haut = est - crit * se_c, est + crit * se_c
print(f"effet estimé : {est:+.3f} (log) -> panier {100*(np.exp(est)-1):+.1f} % IC95 % :
↪ [{100*(np.exp(bas)-1):+.1f} % ; {100*(np.exp(haut)-1):+.1f} %]")
print(m2.t_test("C(canal)[T.Instagram] + 20*a = 0").summary_frame().round(4).to_string())
```

```
effet estimé : -0.153 (log) -> panier -14.2 % IC95 % : [-18.8 % ; -9.4 %]
coef std err t P>|t| Conf. Int. Low Conf. Int. Upp.
c0 -0.1534 0.028 -5.4802 0.0 -0.2083 -0.0985
```

Un client Instagram de 50 ans dépense donc, en moyenne géométrique, environ **14 % de moins** qu'un client de la boutique de 30 ans : les 20 ans d'âge de plus compensent un peu plus de la moitié de l'écart de canal. Cette comparaison de deux profils précis, avec son intervalle (de  $-19\%$  à  $-9\%$ ), est typiquement ce qu'une simple comparaison de moyennes de groupes ne peut pas donner.

#### À retenir (1.2).

- Sous  $H1-H5$ ,  $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T\mathbf{X})^{-1})$ ,  $SCR/\sigma^2 \sim \chi_{n-p}^2$ , et les deux sont indépendants : d'où  $T_j = (\hat{\beta}_j - \beta_j)/se(\hat{\beta}_j) \sim t_{n-p}$  avec  $se(\hat{\beta}_j) = s\sqrt{c_{jj}}$ .

- Le **test  $t$**  teste un coefficient ; le **test  $F$**  de modèles emboîtés,  $F = \frac{(SCR_0 - SCR_1)/q}{SCR_1/(n-p_1)}$ , teste  $q$  contraintes à la fois (par exemple tout un facteur qualitatif, ou une interaction). Pour  $q = 1$ ,  $F = t^2$ .
- Préférez les **intervalles de confiance** aux seules p-valeurs ; en échelle logarithmique, transformez les bornes par  $e^{(\cdot)}$  pour parler en pourcentage.
- **Intervalle de confiance de la moyenne** ( $\hat{y}_0 \pm t s \sqrt{h_0}$ ) et **intervalle de prédiction** ( $\hat{y}_0 \pm t s \sqrt{1 + h_0}$ ) répondent à deux questions différentes ; le second ne peut pas rétrécir en dessous du bruit individuel, et les deux **s'élargissent en extrapolation**.
- Une simulation confirme que « 95 % » signifie bien 95 % de couverture... **quand le modèle est correct** ; le bootstrap des couples est une alternative sans hypothèse de normalité.

### 1.3 Diagnostics : le modèle est-il fiable ?

**Intuition.** Un logiciel produit un tableau de coefficients quoi qu'on lui donne, même si le modèle est absurde. Les p-valeurs et les intervalles du 1.2 ne valent **que si les hypothèses H1 à H5 sont à peu près vraies**. Faire des diagnostics, c'est la visite médicale du modèle : on regarde ce qui reste *après* l'ajustement (les résidus), parce que les erreurs d'un modèle bien spécifié ne doivent contenir **aucune structure**. Si l'on voit une courbe, un entonnoir ou un point isolé dans les résidus, c'est que le modèle a raté quelque chose.

Voici la préparation (mêmes données qu'en 1.1 et 1.2). Nous chargeons en plus les **ventes mensuelles** de Dar Jasmin, qui nous serviront de deuxième terrain d'observation :

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, ENCRE, GRIS, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#0b0b0b", "#898781",
↳ "#e34948"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ "#e1e0d9",
        "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
        "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])

m_niv = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit() # panier en DT : modèle « niveau »
m2 = smf.ols("log_panier ~ a + C(canal)", data=df).fit() # log du panier : le modèle de 1.1
↳ et 1.2

ventes = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"])
ventes["t"] = np.arange(len(ventes)) # numéro du mois : 0, 1, ..., 119
print(len(df), "clients actifs |", len(ventes), "mois de ventes")
```

1740 clients actifs | 120 mois de ventes

### 1.3.1 Les résidus : brut, standardisé, studentisé

Le résidu brut  $\hat{\varepsilon}_i = y_i - \hat{y}_i$  approche l'erreur  $\varepsilon_i$ . Mais attention : les résidus **n'ont pas tous la même variance**, même si les erreurs vraies, elles, l'ont.

**Variance d'un résidu.** On a vu (1.1.5) que  $\hat{\varepsilon} = (\mathbf{I} - \mathbf{H})\varepsilon$ . Donc

$$\text{Var}(\hat{\varepsilon}) = (\mathbf{I} - \mathbf{H}) \sigma^2 \mathbf{I} (\mathbf{I} - \mathbf{H})^\top = \sigma^2 (\mathbf{I} - \mathbf{H}), \quad \text{soit} \quad \text{Var}(\hat{\varepsilon}_i) = \sigma^2 (1 - h_{ii}),$$

où  $h_{ii}$  est le  $i$ -ième élément diagonal de la matrice chapeau (le **levier** de l'observation  $i$ , étudié plus loin).

Un point à fort levier « attire » la droite vers lui : son résidu est mécaniquement plus petit. Pour comparer les résidus entre eux, on les **standardise** :

| Résidu                                              | Formule                                                       | Usage                                                                                                                                                                  |
|-----------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| brut                                                | $\hat{\varepsilon}_i = y_i - \hat{y}_i$                       | calcul de base                                                                                                                                                         |
| <b>standardisé</b> ( <i>studentisé en interne</i> ) | $r_i = \frac{\hat{\varepsilon}_i}{s \sqrt{1 - h_{ii}}}$       | variance $\approx 1$ pour tous : comparable                                                                                                                            |
| <b>studentisé externe</b>                           | $t_i = \frac{\hat{\varepsilon}_i}{s_{(i)} \sqrt{1 - h_{ii}}}$ | $s_{(i)}$ = écart-type estimé <b>sans</b> l'observation $i$ ; suit exactement une loi $t_{n-p-1}$ si le modèle est correct : sert à <b>tester</b> si $i$ est aberrante |

Vérifions la formule du résidu standardisé à la main, puis avec statsmodels, en reprenant les quatre commandes du 1.1.1 (où les leviers valaient 0,7 ; 0,3 ; 0,3 ; 0,7) :

```
x4 = np.array([1, 2, 3, 4]); y4 = np.array([22, 41, 66, 79])
X4 = np.column_stack([np.ones(4), x4])
H4 = X4 @ np.linalg.inv(X4.T @ X4) @ X4.T
h4 = np.diag(H4)
e4 = y4 - H4 @ y4
n4, p4 = X4.shape
s4 = np.sqrt(e4 @ e4 / (n4 - p4))
r4 = e4 / (s4 * np.sqrt(1 - h4)) # standardisés
infl4 = sm.OLS(y4, X4).fit().get_influence()
print("leviers h_ii          :", h4.round(2))
print("résidus bruts        :", e4.round(2))
print("standardisés (main)   :", r4.round(3), "| statsmodels :",
      infl4.resid_studentized_internal.round(3))

leviers h_ii          : [0.7 0.3 0.3 0.7]
résidus bruts         : [-0.6 -1.2  4.2 -2.4]
standardisés (main)   : [-0.309 -0.404  1.414 -1.234] | statsmodels : [-0.309 -0.404  1.414 -1.234]
```

Pour le résidu studentisé externe, on n'a pas besoin de refaire  $n$  régressions :  $s_{(i)}^2 = (\text{SCR} - \hat{\varepsilon}_i^2 / (1 - h_{ii})) / (n - p - 1)$  (résultat classique, que statsmodels utilise aussi). Nous le vérifierons en 1.3.4 sur un exemple plus fourni. Sur ces *quatre* points, il est d'ailleurs dégénéré pour la commande C : les trois autres points (1,22), (2,41), (4,79) sont **exactement alignés** (pente 19), donc  $s_{(C)} = 0$  et la statistique serait infinie. C'est une bonne illustration de ce qu'un test d'aberrance demande plus de points que cela.

### 1.3.2 Les graphiques de résidus : voir avant de tester

Trois graphiques suffisent à repérer l'essentiel :

1. **Résidus contre valeurs ajustées** : doit ressembler à un **nuage sans structure** centré sur 0. Une **courbe** signale une non-linéarité (H1) ; un **entonnoir** (dispersion qui augmente), une variance non constante (H4).
2. **Diagramme quantile-quantile (Q-Q) des résidus standardisés** : les points doivent suivre la diagonale si les erreurs sont normales (H5). Une queue relevée indique une asymétrie ou des valeurs extrêmes.
3. **Échelle-position** ( $\sqrt{|r_i|}$  contre valeurs ajustées) : une tendance croissante confirme une variance non constante.

Mettons en concurrence **deux modèles pour le même phénomène** : le panier en dinars ( $m_{niv}$ ) et son logarithme ( $m2$ ). Le premier est celui que l'on écrirait « naturellement » ; le second celui que nous avons choisi en 1.1.7 parce que le panier est asymétrique. Les diagnostics vont nous dire si ce choix était justifié.

```
def diagnostics(modele, axes, titre):
    ajuste = modele.fittedvalues
    r = modele.get_influence().resid_studentized_internal
    ax1, ax2, ax3 = axes
    ax1.scatter(ajuste, r, s=7, color=BLEU, alpha=0.45)
    lis = sm.nonparametric.lowess(r, ajuste, frac=0.4)
    ax1.plot(lis[:, 0], lis[:, 1], color=ORANGE, lw=2)
    ax1.axhline(0, color=GRIS, lw=1)
    ax1.set_xlabel("valeurs ajustées"); ax1.set_ylabel("résidu standardisé"); ax1.set_title(titre + "
↪ : résidus / ajustées", fontsize=10)
    th, emp = stats.probplot(r, dist="norm")[0] # quantiles théoriques, quantiles observés
    ax2.scatter(th, emp, s=7, color=BLEU, alpha=0.45)
    lim = [min(th.min(), emp.min()), max(th.max(), emp.max())]
    ax2.plot(lim, lim, color=ORANGE, lw=2)
    ax2.set_xlabel("quantiles théoriques (loi normale)"); ax2.set_ylabel("quantiles observés");
    ↪ ax2.set_title(titre + " : Q-Q", fontsize=10)
    ax3.scatter(ajuste, np.sqrt(np.abs(r)), s=7, color=BLEU, alpha=0.45)
    lis = sm.nonparametric.lowess(np.sqrt(np.abs(r)), ajuste, frac=0.4)
    ax3.plot(lis[:, 0], lis[:, 1], color=ORANGE, lw=2)
    ax3.set_xlabel("valeurs ajustées"); ax3.set_ylabel("racine de |résidu standardisé|");
    ↪ ax3.set_title(titre + " : échelle-position", fontsize=10)

fig, axes = plt.subplots(2, 3, figsize=(12.5, 7.2))
diagnostics(m_niv, axes[0], "panier en DT")
diagnostics(m2, axes[1], "log du panier")
plt.tight_layout()
plt.savefig("figures/ch01-diagnostics-niveau-log.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

La différence est nette. Pour le panier en dinars (ligne du haut), les résidus sont très **asymétriques** : une nuée de points très hauts (jusqu'à 7 écarts-types), mais aucun point aussi bas, et le diagramme Q-Q montre une **queue droite très relevée** et une queue gauche trop courte. La dispersion augmente aussi avec la valeur ajustée : le lissage de l'échelle-position monte de 0,6 à 0,85 environ (l'entonnoir est modeste à l'œil, mais le test de Breusch-Pagan, ci-dessous, ne s'y trompe pas). Pour le log du panier (ligne du bas), le nuage est homogène, le lissage orange reste plat, et les points suivent la diagonale. La transformation logarithmique n'était donc pas un détail esthétique : elle rend le modèle **valide**.



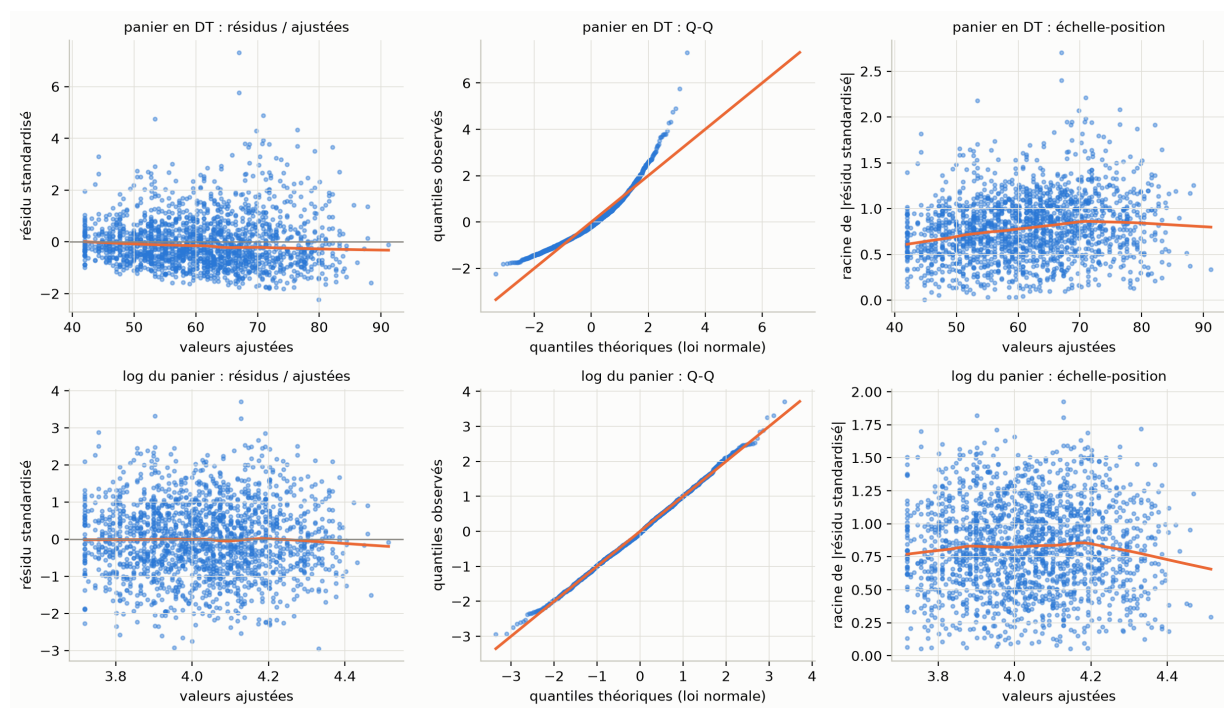


Figure 5: Diagnostics comparés. Ligne du haut : modèle sur le panier en DT (résidus asymétriques, dispersion croissante, queue lourde à droite). Ligne du bas : modèle sur le log du panier (nuage homogène, points alignés sur la diagonale). La courbe orange est un lissage local.

**Pourquoi le log résout le problème.** Quand les effets sont **multiplicatifs** ( $y = \mu \cdot \eta$  avec  $\eta$  un facteur aléatoire autour de 1), l'écart-type de  $y$  est proportionnel à sa moyenne  $\mu$  : les gros paniers fluctuent plus que les petits, exactement l'entonnoir observé. En passant au logarithme,  $\log y = \log \mu + \log \eta$  : le bruit devient **additif** et de variance constante.

### 1.3.3 Les tests de diagnostic (et pourquoi ils ne remplacent pas les graphiques)

Des tests formalisent ce que l'œil voit.

**Variance non constante : le test de Breusch-Pagan.** Idée : si la variance dépend des variables explicatives, alors les carrés des résidus  $\hat{\varepsilon}_i^2$  (estimations grossières de la variance) doivent être **prévisibles** à partir de  $X$ . On régresse donc  $\hat{\varepsilon}_i^2$  sur les mêmes variables ; si cette régression a un  $R^2$  non négligeable, on rejette l'homoscédasticité. La statistique est  $LM = n R_{aux}^2 \sim \chi_{p-1}^2$  sous  $H_0$  (variance constante). Calculons-la à la main pour les deux modèles et comparons à statsmodels :

```
def breusch_pagan_main(modele):
    e2 = modele.resid.to_numpy() ** 2
    Xm = modele.model.exog
    aux = sm.OLS(e2, Xm).fit()
    LM = len(e2) * aux.rsquared
    return LM, stats.chi2.sf(LM, Xm.shape[1] - 1)

for nom, m in [("panier en DT", m_niv), ("log du panier", m2)]:
    LM, p = breusch_pagan_main(m)
    LM_sm, p_sm = sm.stats.diagnostic.het_breuschpagan(m.resid, m.model.exog)[:2]
    print(f"{nom:14s} LM (main) = {LM:6.2f} p = {p:.2e} | statsmodels : LM = {LM_sm:6.2f} p = {p_sm:.2e}")
```

```

panier en DT    LM (main) = 35.90  p = 7.86e-08 | statsmodels : LM = 35.90  p = 7.86e-08
log du panier  LM (main) =  1.92  p = 5.90e-01 | statsmodels : LM =  1.92  p = 5.90e-01

```

**Normalité des erreurs.** Le test de Shapiro-Wilk et celui de Jarque-Bera (fondé sur l'asymétrie et l'aplatissement, volume I, section 3.7.5) comparent les résidus à une loi normale. Avec un grand  $n$ , ils détectent des écarts minuscules sans importance pratique : le diagramme Q-Q reste l'outil principal.

```

for nom, m in [("panier en DT", m_niv), ("log du panier", m2)]:
    r = m.resid
    jrb, pjb, sk, ku = sm.stats.jarque_bera(r)
    print(f"{nom:14s} asymétrie = {sk:5.2f} | aplatissement (excès) = {ku-3:5.2f} | Shapiro p =
    ↪ {stats.shapiro(r).pvalue:.2e} | Jarque-Bera p = {pjb:.2e}")

panier en DT    asymétrie =  1.40 | aplatissement (excès) =  3.84 | Shapiro p = 1.25e-29 | Jarque-Bera
↪ p = 0.00e+00
log du panier  asymétrie =  0.11 | aplatissement (excès) = -0.06 | Shapiro p = 2.34e-01 | Jarque-Bera
↪ p = 1.63e-01

```

**Autocorrélation des erreurs (H4 : erreurs non corrélées).** Quand les observations sont ordonnées dans le temps, les erreurs successives peuvent se ressembler. Le test de **Durbin-Watson** mesure cela :  $DW = \frac{\sum_{i \geq 2} (\hat{\varepsilon}_i - \hat{\varepsilon}_{i-1})^2}{\sum_i \hat{\varepsilon}_i^2} \approx 2(1 - \hat{\rho})$ , où  $\hat{\rho}$  est l'autocorrélation d'ordre 1 des résidus. Une valeur proche de 2 indique l'absence d'autocorrélation ; **inférieure à 2**, une autocorrélation positive. Les clients n'ont pas d'ordre naturel, donc ce test n'a pas de sens pour eux : utilisons plutôt les ventes mensuelles, avec le modèle d'une **tendance simple**  $\text{ventes}_t = \beta_0 + \beta_1 t + \varepsilon_t$ , en niveau puis en logarithme.

```

mv_niv = smf.ols("ca ~ t", data=ventes).fit()
mv_log = smf.ols("np.log(ca) ~ t", data=ventes).fit()
for nom, m in [("ca ~ t", mv_niv), ("log(ca) ~ t", mv_log)]:
    dw = sm.stats.durbin_watson(m.resid)
    print(f"{nom:12s} R² = {m.rsquared:.3f} | Durbin-Watson = {dw:.2f} (rho ≈ {1 - dw/2:.2f}) |
    ↪ Breusch-Pagan p = {sm.stats.diagnostic.het_breuschpagan(m.resid, m.model.exog)[1]:.3f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.8))
ax1.plot(ventes["mois"], mv_log.resid, color=BLEU, lw=1.2)
ax1.axhline(0, color=GRIS, lw=1)
ax1.set_ylabel("résidu de log(ca) ~ t"); ax1.set_title("Résidus dans le temps : une saisonnalité
↪ subsiste", fontsize=10)
pd.plotting.autocorrelation_plot(pd.Series(mv_log.resid.to_numpy()), ax=ax2, color=BLEU)
ax2.set_xlim(0, 36); ax2.set_title("Autocorrélation des résidus (décalages en mois)", fontsize=10)
plt.tight_layout()
plt.savefig("figures/ch01-residus-ventes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

ca ~ t          R² = 0.464 | Durbin-Watson = 1.56 (rho ≈ 0.22) | Breusch-Pagan p = 0.004
log(ca) ~ t     R² = 0.493 | Durbin-Watson = 1.56 (rho ≈ 0.22) | Breusch-Pagan p = 0.889
figure enregistrée

```

Le  $R^2$  du modèle en log est un peu meilleur, et c'est le seul des deux dont la variance est constante (Breusch-Pagan :  $p = 0,889$ , contre 0,004 en niveau). La valeur de Durbin-Watson, identique pour les deux modèles à deux décimales près (ce n'est pas une erreur de copie : la même structure temporelle persiste dans les deux jeux de résidus), est nettement inférieure à 2 : les erreurs sont **positivement autocorrélées** ( $\hat{\rho} \approx 0,22$  d'un mois sur le suivant). Mais la figure raconte une histoire bien plus riche que ce seul nombre. Le test de Durbin-Watson ne regarde que le **décalage d'un mois** ; or le graphique d'autocorrélation (à droite) montre trois pics très nets aux décalages **12, 24 et 36 mois** (autour de 0,6), et des creux négatifs vers 10 et 22 mois. La cause est identifiable



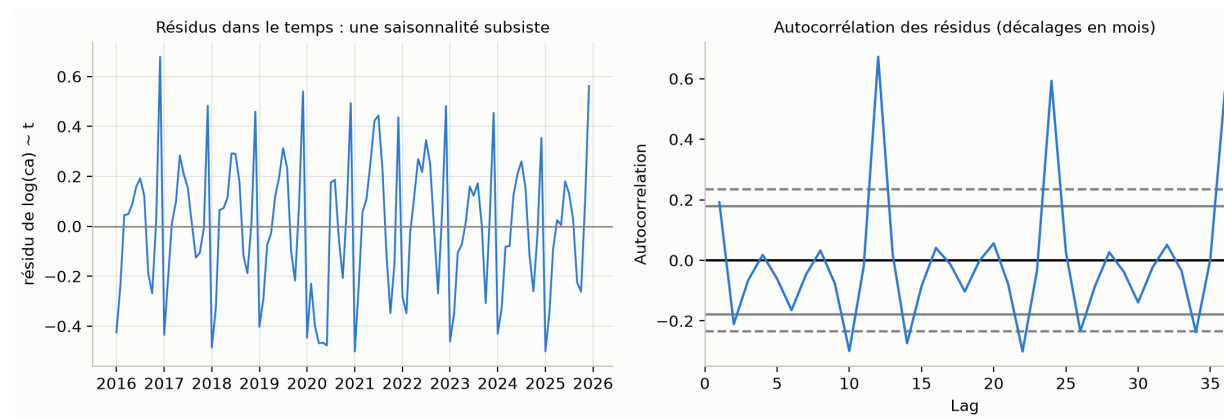


Figure 6: À gauche : les résidus du modèle de tendance sur le log des ventes forment des vagues saisonnières (ils ne sont pas du bruit pur). À droite : leur autocorrélation, avec des pics très nets aux décalages 12, 24 et 36 mois.

: la **saisonnalité annuelle** (le pic de décembre revient chaque année), absente du modèle de tendance, se retrouve intégralement dans les résidus. Un seul test peut donc passer à côté d'une structure massive : *regardez toujours l'autocorrélation complète*.

Quand H4 (erreurs non corrélées) est violée, les estimations restent sans biais, mais **les erreurs standard sont fausses** (en général trop optimistes) : les p-valeurs du 1.2 ne sont plus fiables. La remédiation n'est pas un bricolage de régression ordinaire, mais les **modèles de séries temporelles** du chapitre 4 (section 4.1 : autocorrélation et décomposition saisonnière ; section 4.2 : ARIMA saisonniers).

**Un test n'est pas un diagnostic.** (1) Avec un grand échantillon, un test rejette pour un écart minuscule et sans conséquence ; avec un petit échantillon, il laisse passer des défauts graves. (2) Un test ne dit pas **comment réparer**. (3) Les tests eux-mêmes supposent des hypothèses. Le bon réflexe : **graphique d'abord**, test ensuite pour *confirmer ce qu'on voit*, jugement enfin sur l'ampleur de l'écart.

### 1.3.4 Effet de levier et observations influentes

Certaines observations pèsent beaucoup plus que d'autres dans l'ajustement. Il faut distinguer trois notions, qu'on confond souvent :

- Un **point aberrant** (*outlier*) a un résidu **grand** : il est mal expliqué par le modèle.
- Un point à **fort levier** (*leverage*) a des valeurs de **x inhabituelles** (loin du centre des données) : il *peut* tirer la droite vers lui.
- Un point **influent** change **beaucoup** les résultats quand on le retire. Il est typiquement à la fois aberrant *et* à fort levier.

**Le levier.**  $h_{ii} = \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i$  est le  $i$ -ième élément diagonal de **H**. Comme  $\hat{y}_i = \sum_j h_{ij} y_j$ , on a  $\partial \hat{y}_i / \partial y_i = h_{ii}$  : c'est le **poids de  $y_i$  dans sa propre prédiction**. Propriétés :  $\frac{1}{n} \leq h_{ii} \leq 1$  (avec constante) et  $\sum_i h_{ii} = \text{tr } \mathbf{H} = p$ , donc le levier moyen vaut  $p/n$ . Règle empirique : un levier supérieur à  $2p/n$  est « élevé ».

**La distance de Cook** combine résidu et levier :

$$D_i = \frac{(\hat{\beta} - \hat{\beta}_{(i)})^\top \mathbf{X}^\top \mathbf{X} (\hat{\beta} - \hat{\beta}_{(i)})}{p s^2} = \frac{r_i^2}{p} \cdot \frac{h_{ii}}{1 - h_{ii}},$$

où  $\hat{\beta}_{(i)}$  est l'estimateur calculé sans l'observation  $i$  et  $r_i$  le résidu standardisé. La première expression est la définition (« de combien l'ajustement bouge si j'enlève  $i$ , en unités d'erreur standard ») ; la seconde, un raccourci de calcul qui évite  $n$  régressions. Il découle de l'identité  $\hat{\beta} - \hat{\beta}_{(i)} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i \hat{e}_i / (1 - h_{ii})$  (formule de Sherman-Morrison pour la mise à jour d'une inverse de matrice).

**Un petit exemple pour voir.** Six points : cinq bien alignés sur une droite de pente environ 2, et un sixième, **très à droite** ( $x = 12$ ), qui s'écarte de la tendance.

```
xs = np.array([1, 2, 3, 4, 5, 12.0])
ys = np.array([2.1, 3.9, 6.2, 7.8, 10.1, 14.0]) # le dernier point devrait être vers 24 si la
        ↪ tendance se poursuivait
Xs = np.column_stack([np.ones(6), xs])
ms = sm.OLS(ys, Xs).fit()
m5 = sm.OLS(ys[:5], Xs[:5]).fit() # sans le sixième point
infl = ms.get_influence()
print("pente avec les 6 points :", round(ms.params[1], 3), "| sans le sixième point :",
      ↪ round(m5.params[1], 3))
print("leviers h_ii :", infl.hat_matrix_diag.round(3), "(moyenne p/n =", round(2/6, 3), ")")
print("résidus bruts :", ms.resid.round(2))
print("résidus standardisés :", infl.resid_studentized_internal.round(2))
print("distances de Cook :", infl.cooks_distance[0].round(2))
print("prévision en x = 12 sans le sixième point :", round(m5.params[0] + m5.params[1] * 12, 1), "|
      ↪ observé :", ys[5])
# vérification de la formule de Cook par réajustement sans l'observation i
# résidu studentisé externe : formule sans réajustement, comparée à statsmodels
h6, e6 = infl.hat_matrix_diag, ms.resid
s_sans = np.sqrt((e6 @ e6 - e6**2 / (1 - h6)) / (6 - 2 - 1))
print("studentisés externes (main) :", (e6 / (s_sans * np.sqrt(1 - h6))).round(2))
print("studentisés externes (statsm.) :", infl.resid_studentized_external.round(2))
cook_main = []
for i in range(6):
    mi = sm.OLS(np.delete(ys, i), np.delete(Xs, i, axis=0)).fit()
    d = ms.params - mi.params
    cook_main.append(d @ Xs.T @ Xs @ d / (2 * ms.scale))
print("Cook par réajustement :", np.round(cook_main, 2))

fig, ax = plt.subplots(figsize=(6.6, 4.2))
xx = np.linspace(0, 13, 50)
ax.plot(xx, ms.params[0] + ms.params[1] * xx, color=ORANGE, lw=2, label="droite avec les 6 points")
ax.plot(xx, m5.params[0] + m5.params[1] * xx, color=BLEU, lw=2, label="droite sans le sixième point")
ax.scatter(xs[:5], ys[:5], color=ENCRE, s=36, zorder=3)
ax.scatter(xs[5:], ys[5:], color=ROUGE, s=60, zorder=3)
ax.annotate("point à fort levier", (12, 14.0), textcoords="offset points", xytext=(-95, -35),
        ↪ color=ROUGE,
        arrowprops=dict(arrowstyle="->", color=ROUGE))
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.legend(frameon=False, loc="upper left")
ax.set_xlim(0, 13); ax.set_ylim(0, 27)
plt.savefig("figures/ch01-levier.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

```
pente avec les 6 points : 1.029 | sans le sixième point : 1.99
leviers h_ii : [0.325 0.247 0.196 0.17 0.17 0.892] (moyenne p/n = 0.333 )
résidus bruts : [-1.65 -0.88 0.39 0.96 2.24 -1.07]
résidus standardisés : [-1.23 -0.62 0.27 0.65 1.5 -1.99]
distances de Cook : [3.600e-01 6.000e-02 1.000e-02 4.000e-02 2.300e-01 1.643e+01]
prévision en x = 12 sans le sixième point : 23.9 | observé : 14.0
studentisés externes (main) : [-1.34 -0.56 0.23 0.59 1.96 -17.24]
studentisés externes (statsm.) : [-1.34 -0.56 0.23 0.59 1.96 -17.24]
Cook par réajustement : [3.600e-01 6.000e-02 1.000e-02 4.000e-02 2.300e-01 1.643e+01]
figure enregistrée
```

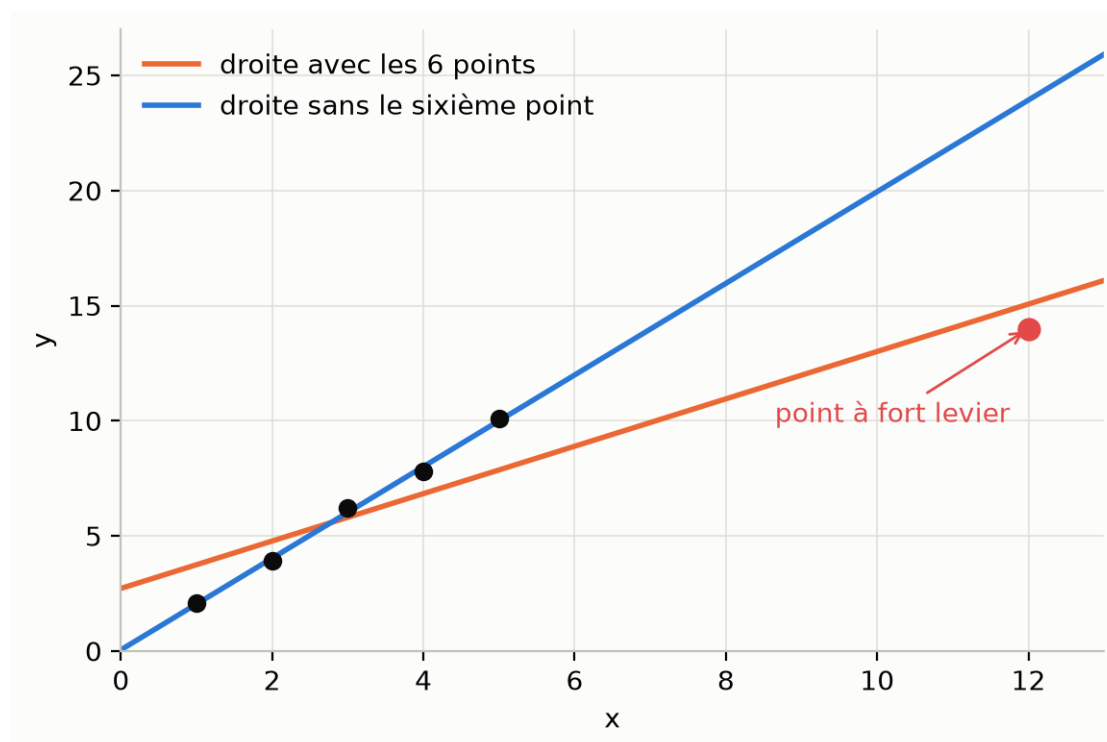


Figure 7: Un point à fort levier (rouge,  $x = 12$ ) tire la droite orange vers lui. Sans ce point, la droite bleue suit le reste du nuage, et prévoit environ 24 en  $x = 12$ .

Remarquez la leçon, qui surprend toujours : **le point influent a un résidu petit**. Comme la droite est tirée vers lui, son résidu brut est modeste (voyez la ligne des résidus), alors qu'il est très loin de ce que la tendance des cinq autres points prévoyait. C'est son **levier** (proche de 0,9, très au-dessus de la moyenne  $p/n \approx 0,33$ ) et sa **distance de Cook** qui le trahissent. Un diagnostic fondé sur les seuls résidus l'aurait laissé passer. Et la formule de Cook, retrouvée ici par réajustement sans chaque point, confirme le raccourci.

**Sur les clients de Dar Jasmin.** Appliquons ces outils au modèle  $m2$ . Avec 1 740 clients, un seul client ne pèse presque rien :

```
infl = m2.get_influence()
h = infl.hat_matrix_diag
cook = infl.cooks_distance[0]
n, p = m2.model.exog.shape
print(f"n = {n}, p = {p} | levier moyen p/n = {p/n:.4f} | levier maximal = {h.max():.4f} | seuil 2p/n
↳ = {2*p/n:.4f}")
print(f"distance de Cook maximale = {cook.max():.4f} | seuil usuel 4/n = {4/n:.4f} | seuil d'alerte
↳ forte : 1")
print(f"nombre de clients au-dessus de 4/n : {(cook > 4/n).sum()} sur {n} ({100*(cook >
↳ 4/n).mean():.1f} %)")
print()
top = pd.DataFrame({"age": df["age"].to_numpy(), "canal": df["canal"].to_numpy(), "panier":
↳ df["panier_moyen"].to_numpy(),
                    "résidu stud. ext.": infl.resid_studentized_external, "levier": h, "Cook":
↳ cook}).sort_values("Cook", ascending=False).head(5)
print(top.round(4).to_string())
print()
print(m2.outlier_test().sort_values("unadj_p").head(3).round(4))
```

```
n = 1740, p = 4 | levier moyen p/n = 0.0023 | levier maximal = 0.0089 | seuil 2p/n = 0.0046
distance de Cook maximale = 0.0067 | seuil usuel 4/n = 0.0023 | seuil d'alerte forte : 1
nombre de clients au-dessus de 4/n : 84 sur 1740 (4.8 %)
```

|      | age           | canal     | panier  | résidu stud. | ext.    | levier | Cook   |
|------|---------------|-----------|---------|--------------|---------|--------|--------|
| 864  | 43            | Site      | 245.06  |              | 3.7254  | 0.0019 | 0.0067 |
| 747  | 48            | Boutique  | 25.53   |              | -2.9552 | 0.0030 | 0.0066 |
| 1060 | 66            | Instagram | 131.04  |              | 1.9415  | 0.0061 | 0.0058 |
| 140  | 50            | Site      | 189.89  |              | 2.8562  | 0.0027 | 0.0055 |
| 439  | 22            | Instagram | 124.39  |              | 2.8921  | 0.0025 | 0.0052 |
|      | student_resid |           | unadj_p | bonf(p)      |         |        |        |
| 1001 | 3.7254        |           | 0.0002  | 0.3502       |         |        |        |
| 180  | 3.3254        |           | 0.0009  | 1.0000       |         |        |        |
| 1741 | 3.2648        |           | 0.0011  | 1.0000       |         |        |        |

Aucun client n'a un levier ou une distance de Cook préoccupants : le plus grand levier (0,0089) est certes environ quatre fois le levier moyen (0,0023), mais il reste inférieur à 1 % (aucun client ne pèse même 1 % dans sa propre prédiction), et la distance de Cook maximale (0,0067) est plus de cent fois sous le seuil d'alerte de 1. Deux remarques de méthode :

- Le seuil «  $D_i > 4/n$  » est une **règle de dépistage**, pas un test : avec  $n$  grand elle signale toujours quelques points (ici environ 5 % des clients), simplement parce que certains points sont toujours un peu plus éloignés que d'autres. Ce qui compte est qu'**aucun** ne soit isolé du lot. Regardez les valeurs, pas seulement le seuil.
- Le test des résidus studentisés externes, ajusté pour le nombre de tests (correction de **Bonferroni**, volume I, section 3.5.5), est un vrai test de « valeur aberrante » : la plus grande valeur absolue (environ 3,73) a une p-valeur brute de 0,0002, mais parmi 1 740 observations on s'attend à de telles valeurs : une fois la correction de Bonferroni appliquée, la p-valeur ajustée est de 0,35, rien de significatif.

**Que faire d'une observation influente ?** Ne la supprimez **pas** automatiquement. (1) Vérifiez qu'il ne s'agit pas d'une **erreur de saisie** (un panier de 5 000 DT au lieu de 50). (2) Si elle est authentique, regardez comment les conclusions changent **avec et sans** elle, et rapportez les deux. (3) Si elle représente un phénomène réel mais rare, envisagez une méthode **robuste** (section 1.6). Retirer un point « parce qu'il gêne » est l'une des formes les plus courantes de falsification involontaire.

### 1.3.5 La multicollinéarité : quand deux variables disent la même chose

Si deux variables explicatives sont presque redondantes, le modèle ne peut pas **départager** leurs effets. Les prédictions restent bonnes, mais les coefficients individuels deviennent **instables** et leurs erreurs standard explosent.

**Pourquoi les erreurs standard explosent.** Par le théorème de Frisch-Waugh-Lovell (1.1.7), le coefficient de la variable  $x_j$  est la pente de la régression sur la partie de  $x_j$  qui n'est pas expliquée par les autres variables, soit  $\tilde{x}_j = \mathbf{M}_{-j}x_j$  (les résidus de la régression de  $x_j$  sur les autres colonnes). Donc

$$\text{Var}(\hat{\beta}_j) = \frac{\sigma^2}{\|\tilde{x}_j\|^2} = \frac{\sigma^2}{\text{SCT}_j (1 - R_j^2)},$$

où  $\text{SCT}_j = \sum_i (x_{ij} - \bar{x}_j)^2$  et  $R_j^2$  est le  $R^2$  de la régression de  $x_j$  sur les autres variables explicatives. Quand  $R_j^2 \rightarrow 1$  (colinéarité), la partie « propre » de  $x_j$  disparaît et la variance explose. Le **facteur d'inflation de la variance** est

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

Règles empiriques : VIF > 5 : à surveiller ; VIF > 10 : problématique.

**Provoquons le problème.** Ajoutons au modèle m2 l'âge **exprimé en mois**, mesuré avec un petit bruit (le client indique son âge « à peu près » ; l'âge en mois est quasiment 12× l'âge en années) :

```
rng = np.random.default_rng(3)
df["age_mois"] = 12 * df["age"] + rng.normal(0, 3, len(df))      # presque redondant avec l'âge en
↳ années
print("corrélation âge (années) / âge (mois) :", round(np.corrcoef(df["age"], df["age_mois"])[0, 1],
↳ 4))

mc = smf.ols("log_panier ~ a + C(canal) + age_mois", data=df).fit()
print(pd.DataFrame({"m2 : coef": m2.params, "m2 : se": m2.bse, "avec age_mois : coef": mc.params,
↳ "avec age_mois : se": mc.bse}).round(4).to_string())
Xc = mc.model.exog
vif = pd.Series([variance_inflation_factor(Xc, i) for i in range(Xc.shape[1])], index=mc.params.index)
print()
print("VIF :", vif.round(1).to_dict())
print("R² de m2 :", round(m2.rsquared, 4), "| R² avec age_mois :", round(mc.rsquared, 4), "
↳ (prédictions aussi bonnes)")
```

```
corrélation âge (années) / âge (mois) : 0.9997
m2 : coef m2 : se avec age_mois : coef avec age_mois : se
C(canal)[T.Instagram] -0.3368 0.0225 -0.3377 0.0225
C(canal)[T.Site] -0.1571 0.0231 -0.1580 0.0231
Intercept 4.2206 0.0175 3.2552 1.2998
a 0.0092 0.0008 -0.0176 0.0361
age_mois NaN NaN 0.0022 0.0030
```

```
VIF : {'Intercept': 1.0, 'C(canal)[T.Site]': 1.5, 'C(canal)[T.Instagram]': 1.5, 'a': 1828.5,
↳ 'age_mois': 1828.6}
R² de m2 : 0.1657 | R² avec age_mois : 0.166 (prédictions aussi bonnes)
```

Les prédictions sont aussi bonnes (le  $R^2$  ne bouge pas), mais regardez les coefficients de l'âge : l'erreur standard de  $a$  est multipliée par plus de quarante, et le coefficient lui-même est devenu **négatif** et non significatif : pourtant, nous savons que l'effet de l'âge est positif. Les deux variables se « disputent » le même effet. Les VIF, de l'ordre de 1 800, le disent sans ambiguïté, alors que ceux des variables de canal restent bas (1,5). Les coefficients du **canal** sont, eux, intacts, car ils ne sont pas corrélés avec ce couple : la multicolinéarité n'abîme que les coefficients des variables concernées.

**Une colinéarité plus sournoise : les termes polynomiaux non centrés.** Si l'on ajoute le carré de l'âge pour tester une courbure, âge et âge<sup>2</sup> sont très corrélés (les deux croissent ensemble). Le **centrage** règle le problème :

```
df["age2_brut"] = df["age"] ** 2
df["a2"] = df["a"] ** 2
mq_brut = smf.ols("log_panier ~ age + age2_brut + C(canal)", data=df).fit()
mq_centre = smf.ols("log_panier ~ a + a2 + C(canal)", data=df).fit()
for nom, m in [("non centré (âge, âge²)", mq_brut), ("centré (a, a²)", mq_centre)]:
    Xq = m.model.exog
    v = [round(float(variance_inflation_factor(Xq, i)), 1) for i in range(Xq.shape[1])]
    print(f"{nom:24s} VIF = {v} | p-valeur du terme quadratique = {m.pvalues.iloc[-1]:.3f}")
```

```
non centré (âge, âge²) VIF = [1.0, 1.5, 1.5, 33.9, 33.9] | p-valeur du terme quadratique = 0.580
centré (a, a²) VIF = [1.0, 1.5, 1.5, 1.0, 1.0] | p-valeur du terme quadratique = 0.580
```

Le terme quadratique n'est pas significatif dans les deux cas (la p-valeur du terme du second degré est identique avec ou sans centrage, comme il se doit : on décrit le *même* modèle), mais les VIF passent de plus de 30 à 1 après centrage, ce qui rend les coefficients lisibles. L'âge n'a pas de

courbure détectable : la relation avec le log-panier est bien linéaire (H1 est plausible).

**Que faire en cas de multicollinéarité ?** (1) **Retirer** une des variables redondantes, ou les **combinaison** en une seule (moyenne, indice). (2) **Centrer** les variables avant de créer des puissances ou des interactions. (3) Si l'on tient à garder toutes les variables et que l'objectif est la **prédiction**, la **régularisation** (Ridge, section 1.5) stabilise les coefficients. (4) Si l'objectif est d'**interpréter** un effet précis, il faut accepter qu'on ne peut pas séparer l'effet de deux variables quasi identiques : il est plus honnête de le dire.

### 1.3.6 Variance non constante : que faire ?

Pour `m_niv`, nous avons vu que le défaut est net. Trois remèdes :

1. **Transformer la réponse** (le logarithme, ici : c'est la solution privilégiée, car elle rend le modèle plus naturel).
2. **Changer de famille de lois** (modèles linéaires généralisés, chapitre 2 : par exemple une régression Gamma, qui suppose précisément que l'écart-type est proportionnel à la moyenne).
3. **Garder le modèle en niveau et corriger les erreurs standard** par la méthode « robuste à l'hétéroscédasticité ». C'est utile quand on tient à l'échelle d'origine.

**Erreurs standard de White (« sandwich »).** Si  $\text{Var}(\varepsilon) = \Omega$  est diagonale mais avec des  $\sigma_i^2$  différents, alors  $\hat{\beta}$  reste sans biais, mais sa variance devient

$$\text{Var}(\hat{\beta}) = (\mathbf{X}^T \mathbf{X})^{-1} \left( \sum_i \sigma_i^2 \mathbf{x}_i \mathbf{x}_i^T \right) (\mathbf{X}^T \mathbf{X})^{-1}$$

(le « sandwich » : deux tranches de pain  $(\mathbf{X}^T \mathbf{X})^{-1}$  autour de la garniture). On remplace  $\sigma_i^2$  par un estimateur :  $\hat{\varepsilon}_i^2$  (version **HC0**), ou, plus prudent pour les échantillons de taille modérée,  $\hat{\varepsilon}_i^2 / (1 - h_{ii})^2$  (version **HC3**). La formule classique  $\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$  n'en est qu'un cas particulier (variances égales).

Comparons, pour `m_niv`, les erreurs standard classiques, robustes (HC3), et celles d'un bootstrap des couples (qui ne fait aucune hypothèse sur la variance) :

```
m_hc3 = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit(cov_type="HC3")
Xn, yn = m_niv.model.exog, m_niv.model.endog
rng = np.random.default_rng(8)
cb = np.array([np.linalg.lstsq(Xn[idx], yn[idx], rcond=None)[0] for idx in (rng.integers(0, len(yn),
    ↳ len(yn)) for _ in range(2000))])
print(pd.DataFrame({"se classique": m_niv.bse, "se robuste (HC3)": m_hc3.bse, "se bootstrap":
    ↳ cb.std(axis=0, ddof=1)}).round(3).to_string())
```

|                       | se classique | se robuste (HC3) | se bootstrap |
|-----------------------|--------------|------------------|--------------|
| Intercept             | 1.149        | 1.306            | 1.274        |
| C(canal)[T.Site]      | 1.518        | 1.688            | 1.659        |
| C(canal)[T.Instagram] | 1.478        | 1.511            | 1.522        |
| a                     | 0.055        | 0.055            | 0.055        |

Les erreurs standard robustes sont plus proches du bootstrap que les erreurs classiques pour la constante, le coefficient du Site et celui d'Instagram : la formule classique **sous-estimait** l'incertitude de ces coefficients (de 12 % environ pour la constante et le Site). Pour le coefficient de l'âge, tout concorde. L'écart n'est pas énorme, mais il va dans le sens qu'annonce la théorie. Dans les modèles où l'hétéroscédasticité est plus forte, il peut être considérable.



### 1.3.7 Courbure et variables manquantes : lire les graphiques de résidus partiels

Pour déceler une relation **non linéaire** avec une variable précise  $x_j$ , on trace les **résidus partiels** :  $\hat{\varepsilon}_i + \hat{\beta}_j x_{ij}$  contre  $x_{ij}$ . Si la relation est linéaire, le nuage suit la droite de pente  $\hat{\beta}_j$  ; une courbure systématique suggère d'ajouter un terme (carré, logarithme, ou une transformation plus flexible, cf. les GAM de la section 2.5).

```
fig, ax = plt.subplots(figsize=(6.6, 4.0))
partiel = m2.resid + m2.params["a"] * df["a"]
ax.scatter(df["age"], partiel, s=7, color=GRIS, alpha=0.45)
lis = sm.nonparametric.lowess(partiel, df["age"], frac=0.5)
ax.plot(lis[:, 0], lis[:, 1], color=ORANGE, lw=2.2, label="lissage local")
xa = np.linspace(18, 75, 50)
ax.plot(xa, m2.params["a"] * (xa - 36), color=BLEU, lw=2, ls="--", label="droite du modèle")
ax.set_xlabel("âge (ans)"); ax.set_ylabel("résidu partiel (effet de l'âge)")
ax.legend(frameon=False, loc="upper left")
plt.savefig("figures/ch01-residus-partiels.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

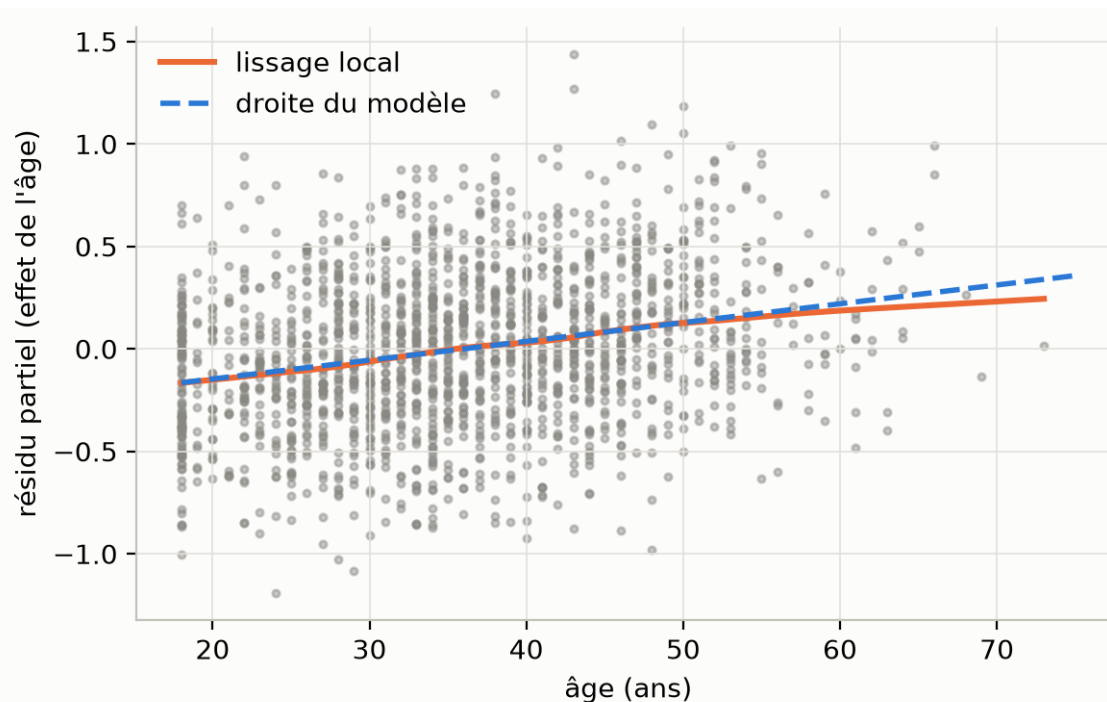


Figure 8: Résidus partiels pour l'âge : le lissage local (orange) suit la droite du modèle (bleu pointillé) : pas de courbure détectable.

Le lissage orange épouse la droite bleue : la linéarité en l'âge est acceptable (ce que le test du terme quadratique, plus haut, confirmait). Notez que le lissage s'écarte un peu aux âges extrêmes, où il y a peu de clients : c'est du bruit d'échantillonnage, pas de la structure.

**Verdict sur m2.** Le modèle `log_panier ~ a + C(canal)` passe les contrôles : linéarité plausible, variance constante (Breusch-Pagan non significatif), résidus proches de la normale, aucune observation influente, pas de colinéarité entre ses variables. Les intervalles du 1.2 sont donc fiables. Ce n'est **pas** une preuve que le modèle est « vrai » (des variables importantes peuvent manquer), seulement que ses hypothèses ne sont pas manifestement violées.

**À retenir (1.3).**

- Les résidus d'un bon modèle ne contiennent **aucune structure**. Regardez : résidus contre ajustées (courbure, entonnoir), Q-Q (normalité), échelle-position (variance).
- $\text{Var}(\hat{\varepsilon}_i) = \sigma^2(1 - h_{ii})$  : on **standardise** (ou studentise) les résidus avant de les comparer. Un point à fort levier a un résidu brut **petit** : ne vous fiez pas aux seuls résidus.
- **Levier**  $h_{ii}$  (valeurs inhabituelles de  $x$ ), **résidu** (écart à la prédiction) et **influence** (changement des résultats si on retire le point, **distance de Cook**) sont trois notions distinctes. Ne supprimez jamais un point sans enquêter.
- **VIF**  $= 1/(1 - R_j^2)$  mesure l'inflation de variance due à la colinéarité ; **centrer** avant de créer des puissances.
- Si la variance n'est pas constante : transformer (log), changer de famille (GLM), ou corriger les erreurs standard (**sandwich**, HC3). Si les erreurs sont **autocorrélées** (séries temporelles) : chapitre 4.
- Un test (Breusch-Pagan, Shapiro, Durbin-Watson) **complète** un graphique, il ne le remplace pas.

## 1.4 Sélection de variables et comparaison de modèles

**Intuition.** Face à dix variables possibles, laquelle garder ? Tout garder semble prudent, mais chaque variable inutile ajoute du **bruit** à l'estimation (les erreurs standard grossissent) et peut conduire à des prédictions pires que celles d'un modèle plus simple. À l'inverse, oublier une variable utile introduit un **biais**. Choisir un modèle, c'est trouver le bon compromis entre **trop simple** (qui rate des effets réels) et **trop complexe** (qui s'ajuste au hasard de l'échantillon, c'est le *surajustement*). Il existe pour cela des outils objectifs : des critères d'information (AIC, BIC), la validation croisée, des tests emboîtés.

Nous gardons la même préparation. Pour avoir plus de variables à départager, nous ajoutons les **notes de l'enquête de satisfaction** (volume II, donnees/enquete\_satisfaction.csv) : 60 % des clients y ont répondu (au hasard), avec huit questions notées de 1 à 5. Les questions q1 à q4 portent sur les **produits**, q5 à q8 sur le **service et la livraison** ; nous en tirons deux scores moyens.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from sklearn.model_selection import KFold
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, ENCRE, GRIS, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#0b0b0b", "#898781",
↳ "#e34948"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ "#e1e0d9",
        "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
        "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
```



```

enq = pd.read_csv("donnees/enquete_satisfaction.csv")
enq["score_produit"] = enq[["q1", "q2", "q3", "q4"]].mean(axis=1)
enq["score_service"] = enq[["q5", "q6", "q7", "q8"]].mean(axis=1)
bq = df.merge(enq[["id_client", "score_produit", "score_service"]], on="id_client")  # clients
↳ actifs ayant répondu
print(len(enq), "répondants à l'enquête |", len(df), "clients actifs |", len(bq), "clients actifs
↳ répondants")
print(bq[["score_produit", "score_service", "a"]].describe().round(2).loc[["mean", "std", "min",
↳ "max"]].to_string())

```

```

1212 répondants à l'enquête | 1740 clients actifs | 1076 clients actifs répondants
      score_produit  score_service      a
mean              3.64              3.55  0.17
std               0.69              0.74 10.51
min               1.25              1.25 -18.00
max               5.00              5.00 37.00

```

Nous travaillerons sur ces **bq** : les clients actifs qui ont répondu à l'enquête. La question est : quelles variables (âge, canal, ville, offre de bienvenue, scores produit et service, courbure de l'âge...) méritent de figurer dans le modèle du log-panier ?

### 1.4.1 Le surajustement : mieux s'ajuster n'est pas mieux prédire

Rappelons l'enjeu (volume I, chapitre 3 : biais et variance d'un estimateur). Un modèle très flexible épouse les données d'entraînement, y compris leur bruit. Pour le **voir**, faisons une expérience : on tire au hasard **40 clients** pour entraîner un modèle polynomial en l'âge de degré 0, 1, 2..., 8, et on mesure l'erreur sur les clients **non utilisés** pour l'entraînement. On répète 300 fois, et on moyenne.

```

z = ((df["age"] - 36) / 10).to_numpy()  # âge centré et réduit (évite les problèmes
↳ numériques des puissances)
yy = df["log_panier"].to_numpy()
rng = np.random.default_rng(21)
degres = np.arange(0, 9)
err_app, err_test = np.zeros((300, len(degres))), np.zeros((300, len(degres)))
for r in range(300):
    idx = rng.permutation(len(z))
    tr, te = idx[:40], idx[40:]
    for j, d in enumerate(degres):
        coef = np.polyfit(z[tr], yy[tr], d)
        err_app[r, j] = np.mean((yy[tr] - np.polyval(coef, z[tr])) ** 2)
        err_test[r, j] = np.mean((yy[te] - np.polyval(coef, z[te])) ** 2)
res = pd.DataFrame({"degré": degres, "erreur d'apprentissage": err_app.mean(axis=0), "erreur sur
↳ nouveaux clients": err_test.mean(axis=0)})
print(res.round(4).to_string(index=False))

fig, ax = plt.subplots(figsize=(6.8, 4.2))
ax.plot(degres, err_app.mean(axis=0), "o-", color=BLEU, lw=2)
ax.plot(degres, err_test.mean(axis=0), "o-", color=ORANGE, lw=2)
ax.text(3.0, 0.108, "erreur sur les clients d'entraînement", color=BLEU, ha="left", va="center")
ax.text(0.1, 0.228, "erreur sur de\nnouveaux clients", color=ORANGE, ha="left", va="center")
ax.text(4.5, 0.33, "au-delà du degré 4, l'erreur\nexplose (axe coupé) :\n1,44 au degré 5,\nplus de 12
↳ 000 au degré 8", color=ORANGE, ha="left", va="center")
ax.set_xlabel("degré du polynôme en l'âge (complexité du modèle)")
ax.set_ylabel("erreur quadratique moyenne")
ax.set_xlim(-0.3, 8.5)
ax.set_ylim(0.10, 0.42)
plt.savefig("figures/ch01-surajustement.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

| degré | erreur d'apprentissage | erreur sur nouveaux clients |
|-------|------------------------|-----------------------------|
| 0     | 0.1610                 | 0.1684                      |
| 1     | 0.1490                 | 0.1629                      |
| 2     | 0.1454                 | 0.1696                      |
| 3     | 0.1423                 | 0.1853                      |
| 4     | 0.1389                 | 0.2958                      |
| 5     | 0.1353                 | 1.4399                      |
| 6     | 0.1311                 | 50.3002                     |
| 7     | 0.1261                 | 612.0571                    |
| 8     | 0.1220                 | 12583.0488                  |

figure enregistrée

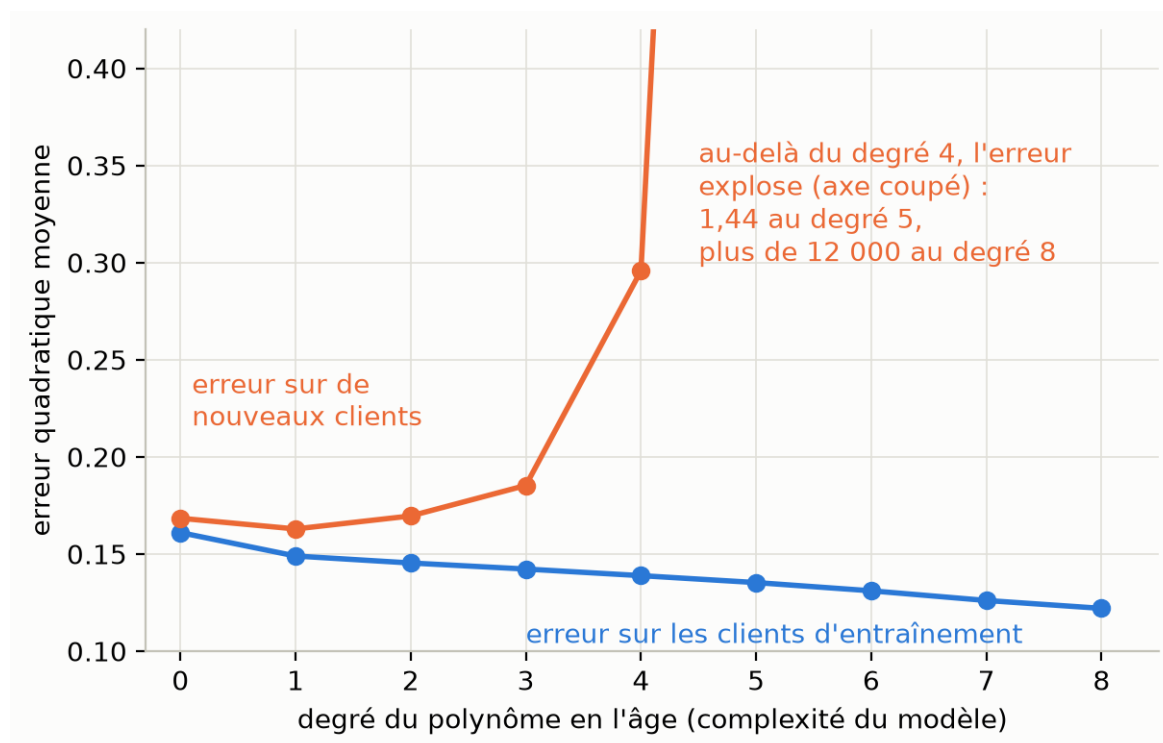


Figure 9: Erreur d'apprentissage (bleu) et erreur sur de nouveaux clients (orange) selon le degré du polynôme, avec 40 clients d'entraînement. L'erreur d'apprentissage ne fait que baisser ; l'erreur de prédiction est minimale au degré 1 puis explose (axe coupé).

C'est le dessin classique du **surajustement** : l'erreur sur les données d'entraînement (bleu) **ne peut que baisser** quand on ajoute des paramètres, alors que l'erreur sur de nouvelles données (orange) atteint son minimum pour un modèle **simple** (le **degré 1**, avec une erreur de 0,163 : l'âge n'explique qu'une faible partie de la variation, et la vraie relation est linéaire) puis **remonte**, d'abord doucement, puis **vertigineusement** : 1,44 au degré 5 et plus de 12 000 au degré 8. Cette explosion vient de ce que les polynômes de haut degré, ajustés sur 40 clients, oscillent violemment dès qu'on sort de la zone dense des données (certains clients de test ont des âges bien plus extrêmes que ceux de l'entraînement) : c'est le surajustement dans sa forme la plus spectaculaire. L'écart entre les deux courbes mesure l'**optimisme** de l'erreur d'apprentissage. Retenez : *on ne peut jamais juger un modèle sur les données qui ont servi à l'ajuster* ; il faut une mesure corrigée de l'optimisme (critères d'information) ou une mesure sur des données mises de côté (validation).

#### 1.4.2 Les critères d'information : AIC et BIC

On cherche une note qui récompense l'ajustement mais **pénalise la complexité**. Le  $R^2$  ajusté (1.1.6) en est une, mais la statistique propose mieux : des critères fondés sur la **vraisemblance**.

**La vraisemblance du modèle linéaire gaussien.** Sous  $H_5$ , les  $y_i$  sont indépendants de loi  $\mathcal{N}(\mathbf{x}_i^\top \beta, \sigma^2)$ . La log-vraisemblance est

$$\ell(\beta, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\beta\|^2.$$

La maximiser en  $\beta$  revient à **minimiser les moindres carrés** : l'estimateur du maximum de vraisemblance est  $\hat{\beta}$  (volume I, section 3.2.5). En  $\sigma^2$ , on obtient  $\hat{\sigma}_{MV}^2 = \text{SCR}/n$  (le facteur  $1/n$  et non  $1/(n-p)$  : l'estimateur du maximum de vraisemblance est biaisé, comme nous l'avons vu en 1.1.5). En reportant, la valeur maximale de la log-vraisemblance est

$$\hat{\ell} = -\frac{n}{2} \left( \log(2\pi) + \log \frac{\text{SCR}}{n} + 1 \right).$$

On pénalise ensuite par le **nombre de paramètres estimés**  $k$ . Par convention (celle de statsmodels), on compte les  $p$  coefficients :  $k = p$ . (Certains logiciels comptent aussi  $\sigma^2$ , donc  $k = p + 1$  : cela ajoute la **même constante** à tous les modèles — 2 à l'AIC,  $\log n$  au BIC — sans changer le moindre classement ; mais deux logiciels peuvent afficher des valeurs différentes pour le même modèle.)

$$\boxed{\text{AIC} = -2\hat{\ell} + 2k},$$

$$\boxed{\text{BIC} = -2\hat{\ell} + k \log n}.$$

**Plus petit = meilleur.** Les deux critères ont la même forme (ajustement + pénalité), avec une pénalité par paramètre de 2 pour l'AIC et de  $\log n$  pour le BIC. Dès que  $n \geq 8$ ,  $\log n > 2$  : le BIC est **plus sévère**, et choisit donc des modèles plus petits.

**D'où viennent-ils ?** (1) L'**AIC** (Akaike) estime, à une constante près, la **qualité prédictive** attendue du modèle sur de nouvelles données :  $-2\hat{\ell}$  est trop optimiste (c'est l'erreur d'apprentissage), et on montre (asymptotiquement) que l'optimisme moyen vaut environ  $2k$ , d'où la correction. Il vise la **prédiction**. (2) Le **BIC** (Schwarz) approche la probabilité *a posteriori* d'un modèle dans une approche bayésienne (chapitre 6) ; il est **consistant** : si le vrai modèle fait partie des candidats, il le retrouve avec une probabilité qui tend vers 1 quand  $n$  grandit, ce que l'AIC ne garantit pas (il a tendance à garder quelques variables en trop). Les deux peuvent être en désaccord : c'est normal, ils ne visent pas la même chose.

Vérifions la formule à la main sur un modèle de bq :

```
m_ex = smf.ols("log_panier ~ a + C(canal)", data=bq).fit()
n, p = m_ex.model.exog.shape
scr = m_ex.ssr
ll = -n / 2 * (np.log(2 * np.pi) + np.log(scr / n) + 1)
k = p                                     # convention de statsmodels : on ne compte pas
                                         # sigma^2
print(f"log-vraisemblance : à la main = {ll:.3f} | statsmodels = {m_ex.llf:.3f}")
print(f"AIC : à la main = {-2*ll + 2*k:.3f} | statsmodels = {m_ex.aic:.3f}")
print(f"BIC : à la main = {-2*ll + k*np.log(n):.3f} | statsmodels = {m_ex.bic:.3f}")
```

```
log-vraisemblance : à la main = -431.472 | statsmodels = -431.472
AIC : à la main = 870.944 | statsmodels = 870.944
BIC : à la main = 890.868 | statsmodels = 890.868
```

**Deux précautions.** (1) On ne peut comparer les AIC/BIC de deux modèles que s'ils sont ajustés **sur exactement les mêmes données et la même variable réponse** (comparer l'AIC d'un modèle sur  $y$  à celui d'un modèle sur  $\log y$  n'a aucun sens, sans correction). (2) La

valeur absolue d'un AIC ne signifie rien ; seules les **différences** comptent. Une différence de moins de 2 est négligeable ; de plus de 10, très forte.

### 1.4.3 La validation croisée

Les critères d'information reposent sur des hypothèses (modèle bien spécifié, grand échantillon). La **validation croisée** est plus directe : on **simule** la prédiction sur des données nouvelles en réservant une partie des données.

**K-fold.** On découpe l'échantillon en  $K$  paquets (*folds*) de taille égale (typiquement  $K = 10$ ). Pour chaque paquet, on ajuste le modèle sur les  $K - 1$  autres et on mesure l'erreur de prédiction sur le paquet laissé de côté. La moyenne des erreurs estime l'erreur de généralisation.

**Leave-one-out et la formule de PRESS.** Si  $K = n$  (un client par paquet), on parle de *leave-one-out*. Pour la régression linéaire, il n'est pas nécessaire de refaire  $n$  ajustements :

**Théorème (PRESS).** L'erreur de prédiction de l'observation  $i$  par le modèle ajusté **sans elle** est  $y_i - \mathbf{x}_i^\top \hat{\beta}_{(i)} = \frac{\hat{\varepsilon}_i}{1 - h_{ii}}$ . La somme des carrés PRESS =  $\sum_i (\hat{\varepsilon}_i / (1 - h_{ii}))^2$  s'obtient donc **avec un seul ajustement**. *Démonstration.* On a vu (1.3.4) que  $\hat{\beta} - \hat{\beta}_{(i)} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i \hat{\varepsilon}_i / (1 - h_{ii})$ . Multiplions à gauche par  $\mathbf{x}_i^\top$  :  $\mathbf{x}_i^\top \hat{\beta} - \mathbf{x}_i^\top \hat{\beta}_{(i)} = h_{ii} \hat{\varepsilon}_i / (1 - h_{ii})$ . Donc  $y_i - \mathbf{x}_i^\top \hat{\beta}_{(i)} = \hat{\varepsilon}_i + h_{ii} \hat{\varepsilon}_i / (1 - h_{ii}) = \hat{\varepsilon}_i / (1 - h_{ii})$ .  $\square$

Un client à fort levier est mal prédit quand on le retire, d'où le diviseur  $1 - h_{ii}$  : l'erreur « honnête » est supérieure au résidu brut. Vérifions par une boucle explicite de  $n$  ajustements :

```
Xb, yb = m_ex.model.exog, m_ex.model.endog
h = m_ex.get_influence().hat_matrix_diag
press_formule = np.sum((m_ex.resid / (1 - h)) ** 2)
loo = np.empty(len(yb))
for i in range(len(yb)):                                # n ajustements, un par client écarté
    mask = np.arange(len(yb)) != i
    b_i = np.linalg.lstsq(Xb[mask], yb[mask], rcond=None)[0]
    loo[i] = yb[i] - Xb[i] @ b_i
print(f"PRESS (formule, 1 ajustement) = {press_formule:.4f}")
print(f"PRESS (boucle, {len(yb)} ajustements) = {np.sum(loo**2):.4f}")
print(f"erreur quadratique d'apprentissage (SCR) = {scr:.4f} <- plus optimiste")

PRESS (formule, 1 ajustement) = 141.5167
PRESS (boucle, 1076 ajustements) = 141.5167
erreur quadratique d'apprentissage (SCR) = 140.4879 <- plus optimiste
```

Écrivons maintenant une fonction de validation croisée K-fold. **Règle importante : utiliser les mêmes paquets pour tous les modèles comparés** (comparaison appariée).

```
def cv_rmse(formule, data, K=10, graine=0):
    """RMSE de validation croisée K-fold (sur l'échelle du log-panier), mêmes paquets pour tous les
    ↪ modèles."""
    kf = KFold(n_splits=K, shuffle=True, random_state=graine)
    erreurs = []
    for tr, te in kf.split(data):
        m = smf.ols(formule, data=data.iloc[tr]).fit()
        pred = np.asarray(m.predict(data.iloc[te]))
        if pred.size == 1:                                # modèle à constante seule : predict
            ↪ renvoie un seul nombre
            pred = np.repeat(pred, len(te))
        erreurs.append(data.iloc[te]["log_panier"].to_numpy() - pred)
    e = np.concatenate(erreurs)
    return np.sqrt(np.mean(e ** 2))
```

```
print("RMSE par validation croisée 10-fold, modèle log_panier ~ a + C(canal) :",
      ↪ round(cv_rmse("log_panier ~ a + C(canal)", bq), 4))
print("RMSE d'apprentissage                                :", round(np.sqrt(scr /
      ↪ n), 4))
```

```
RMSE par validation croisée 10-fold, modèle log_panier ~ a + C(canal) : 0.3629
RMSE d'apprentissage                                : 0.3613
```

### 1.4.4 Comparer plusieurs modèles sur les données de l'enquête

Mettons les outils à l'épreuve. Voici huit modèles candidats, du plus simple au plus chargé :

|           | Variables                                                                |
|-----------|--------------------------------------------------------------------------|
| <b>M0</b> | constante seule                                                          |
| <b>M1</b> | âge                                                                      |
| <b>M2</b> | âge + canal                                                              |
| <b>M3</b> | âge + canal + score produit                                              |
| <b>M4</b> | M3 + score service                                                       |
| <b>M5</b> | M3 + ville + offre de bienvenue                                          |
| <b>M6</b> | M3 + âge <sup>2</sup> + interaction âge × canal                          |
| <b>M7</b> | tout : âge, âge <sup>2</sup> , canal, ville, offre, scores, interactions |

```
bq = bq.copy()
bq["a2"] = bq["a"] ** 2
candidats = {
    "M0": "log_panier ~ 1",
    "M1": "log_panier ~ a",
    "M2": "log_panier ~ a + C(canal)",
    "M3": "log_panier ~ a + C(canal) + score_produit",
    "M4": "log_panier ~ a + C(canal) + score_produit + score_service",
    "M5": "log_panier ~ a + C(canal) + score_produit + C(ville) + offre_bienvenue",
    "M6": "log_panier ~ a * C(canal) + a2 + score_produit",
    "M7": "log_panier ~ a * C(canal) + a2 + C(ville) + offre_bienvenue + score_produit +
    ↪ score_service",
}
lignes, fits = [], {}
for nom, f in candidats.items():
    m = smf.ols(f, data=bq).fit()
    fits[nom] = m
    h = m.get_influence().hat_matrix_diag
    lignes.append({"modèle": nom, "paramètres p": int(m.df_model) + 1, "R²": m.rsquared, "R² ajusté":
    ↪ m.rsquared_adj,
                  "AIC": m.aic, "BIC": m.bic, "RMSE appr.": np.sqrt(m.mse_resid * m.df_resid / m.nobs),
                  "RMSE L00 (PRESS)": np.sqrt(np.mean((m.resid / (1 - h)) ** 2)), "RMSE CV10":
                  ↪ cv_rmse(f, bq)})
tab = pd.DataFrame(lignes).set_index("modèle")
with pd.option_context("display.width", 200):
    print(tab.round(4).to_string())
print()
for crit in ["R² ajusté", "AIC", "BIC", "RMSE L00 (PRESS)", "RMSE CV10"]:
    meilleur = tab[crit].idxmax() if crit == "R² ajusté" else tab[crit].idxmin()
    print(f"meilleur modèle selon {crit:18s} : {meilleur}")
```

|        | paramètres p | R <sup>2</sup> | R <sup>2</sup> ajusté | AIC       | BIC       | RMSE appr. | RMSE L00 (PRESS) | RMSE CV10 |
|--------|--------------|----------------|-----------------------|-----------|-----------|------------|------------------|-----------|
| modèle |              |                |                       |           |           |            |                  |           |
| M0     | 1            | 0.0000         | 0.0000                | 1082.3441 | 1087.3251 | 0.3997     | 0.4001           | 0.4005    |
| M1     | 2            | 0.0663         | 0.0654                | 1010.5132 | 1020.4752 | 0.3863     | 0.3870           | 0.3872    |

|                                             |    |        |        |          |          |        |        |        |
|---------------------------------------------|----|--------|--------|----------|----------|--------|--------|--------|
| M2                                          | 4  | 0.1829 | 0.1807 | 870.9438 | 890.8679 | 0.3613 | 0.3627 | 0.3629 |
| M3                                          | 5  | 0.2542 | 0.2514 | 774.7862 | 799.6912 | 0.3452 | 0.3469 | 0.3478 |
| M4                                          | 6  | 0.2567 | 0.2532 | 773.1610 | 803.0470 | 0.3446 | 0.3466 | 0.3474 |
| M5                                          | 11 | 0.2548 | 0.2478 | 785.8815 | 840.6726 | 0.3451 | 0.3487 | 0.3503 |
| M6                                          | 8  | 0.2549 | 0.2500 | 779.7843 | 819.6323 | 0.3451 | 0.3476 | 0.3485 |
| M7                                          | 15 | 0.2582 | 0.2484 | 788.9871 | 863.7021 | 0.3443 | 0.3491 | 0.3506 |
| meilleur modèle selon $R^2$ ajusté : M4     |    |        |        |          |          |        |        |        |
| meilleur modèle selon AIC : M4              |    |        |        |          |          |        |        |        |
| meilleur modèle selon BIC : M3              |    |        |        |          |          |        |        |        |
| meilleur modèle selon RMSE L00 (PRESS) : M4 |    |        |        |          |          |        |        |        |
| meilleur modèle selon RMSE CV10 : M4        |    |        |        |          |          |        |        |        |

Lisons ce tableau avec méthode.

- Le  $R^2$  et le RMSE d'apprentissage s'améliorent (ou restent égaux) à chaque ajout de variables : ils recommandent toujours le modèle le plus gros (M7). Pas un critère de sélection.
- Les critères corrigés (AIC, BIC,  $R^2$  ajusté, validation croisée) arrêtent leur progression bien avant. Retenez l'ordre de grandeur : l'ajout du **score produit** (M2 → M3) est un gain énorme, l'ajout du **score service** (M3 → M4) est marginal, et tout ce qui vient après (ville, offre, âge<sup>2</sup>, interactions) n'apporte rien (voire détériore).
- Le BIC, plus sévère, désigne le modèle M3. Les autres critères (AIC,  $R^2$  ajusté, PRESS, validation croisée) désignent M4, qui ajoute le score service, mais de justesse : l'AIC de M4 est inférieur de 1,6 point à celui de M3 (différence inférieure à 2 : négligeable, selon la règle du 1.4.2), et le RMSE de validation croisée passe de 0,3478 à 0,3474. M3 et M4 prédisent donc pratiquement aussi bien ; à prédiction égale, on préfère le plus simple (principe de parcimonie), ce que fait le BIC. Les trois mesures hors échantillon (PRESS, CV10) sont très proches l'une de l'autre, ce qui rassure sur la fiabilité du classement.

Les tests emboîtés (1.2.3) répondent à des questions précises sur des modèles qui se contiennent :

```
print("M2 -> M3 (ajout du score produit) :")
print(sm.stats.anova_lm(fits["M2"], fits["M3"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M4 (ajout du score service) :")
print(sm.stats.anova_lm(fits["M3"], fits["M4"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M5 (ajout de la ville et de l'offre de bienvenue, 6 paramètres) :")
print(sm.stats.anova_lm(fits["M3"], fits["M5"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M6 (âge² et interactions, 3 paramètres) :")
print(sm.stats.anova_lm(fits["M3"], fits["M6"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
```

```
M2 -> M3 (ajout du score produit) :
df_resid df_diff      F Pr(>F)
0    1072.0     0.0     NaN   NaN
1    1071.0     1.0 102.2966   0.0
```

```
M3 -> M4 (ajout du score service) :
df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN   NaN
1    1070.0     1.0  3.6111 0.0577
```

```
M3 -> M5 (ajout de la ville et de l'offre de bienvenue, 6 paramètres) :
df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN   NaN
1    1065.0     6.0  0.1493 0.9892
```

```
M3 -> M6 (âge² et interactions, 3 paramètres) :
df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN   NaN
1    1068.0     3.0  0.3316 0.8025
```

Le score produit est hautement significatif ; le score service est **à la limite** (p-valeur autour de 0,05) ; la ville, l'offre de bienvenue, la courbure de l'âge et les interactions n'ont **aucun** effet décelable. C'est cohérent avec les critères d'information : *deux méthodes différentes, la même conclusion*. Le cas du score service illustre bien la tension entre AIC et BIC : sa  $t$ -statistique, voisine de 1,9, dépasse le seuil  $\sqrt{2} \approx 1,41$  que l'AIC applique implicitement (un paramètre en plus est conservé si  $t^2 > 2$ ) mais pas le seuil  $\sqrt{\log n} \approx 2,6$  du BIC pour  $n \approx 1000$ .

### 1.4.5 La sélection automatique, et pourquoi il faut s'en méfier

Avec beaucoup de variables, comparer à la main devient impossible, d'où la tentation des procédures automatiques : sélection **ascendante** (*forward* : on part de rien et on ajoute à chaque étape la variable qui améliore le plus), **descendante** (*backward*), **pas à pas** (*stepwise*), ou **tous les sous-ensembles**. Elles sont commodes, mais elles posent un problème **statistique profond** : *le modèle final est choisi sur les données, puis ses p-valeurs sont lues comme si on l'avait choisi à l'avance*. Les p-valeurs et les intervalles du modèle final sont donc **trop optimistes**.

Une simulation le montre. On génère  $n = 100$  observations d'une variable réponse  $y$  qui **n'a aucun lien** avec 20 variables explicatives (toutes du pur bruit). On applique la sélection ascendante par p-valeur (on ajoute la variable la plus significative tant que sa p-valeur est inférieure à 0,05). Combien de « découvertes » la méthode va-t-elle faire ? On répète 500 fois.

```
def selection_ascendante(X, y, alpha=0.05):
    """Sélection ascendante par p-valeur. X : matrice n x q sans constante. Retourne la liste des
    ↪ colonnes retenues."""
    n, q = X.shape
    retenues = []
    while len(retenues) < q:
        meilleur, p_min = None, 1.0
        for j in range(q):
            if j in retenues:
                continue
            Z = np.column_stack([np.ones(n), X[:, retenues + [j]]])
            b, _, _, _ = np.linalg.lstsq(Z, y, rcond=None)
            e = y - Z @ b
            s2 = e @ e / (n - Z.shape[1])
            se = np.sqrt(s2 * np.linalg.inv(Z.T @ Z)[-1, -1])
            pj = 2 * stats.t.sf(abs(b[-1] / se), n - Z.shape[1])
            if pj < p_min:
                meilleur, p_min = j, pj
        if meilleur is None or p_min >= alpha:
            break
        retenues.append(meilleur)
    return retenues

def ols_pvaleurs(X, y):
    Z = np.column_stack([np.ones(len(y)), X])
    b = np.linalg.lstsq(Z, y, rcond=None)[0]
    e = y - Z @ b
    s2 = e @ e / (len(y) - Z.shape[1])
    se = np.sqrt(s2 * np.diag(np.linalg.inv(Z.T @ Z)))
    return 2 * stats.t.sf(np.abs(b / se), len(y) - Z.shape[1])[1:], 1 - (e @ e) / np.sum((y -
    ↪ y.mean()) ** 2)

rng = np.random.default_rng(99)
R, n_obs, q = 500, 100, 20
nb_retenues, r2_final, sig_final, sig_honnete, nb_honnete = [], [], [], [], []
for _ in range(R):
    X = rng.normal(size=(n_obs, q)); y = rng.normal(size=n_obs) # y est indépendant de tout X
    ret = selection_ascendante(X, y)
    nb_retenues.append(len(ret))
    if ret:
```



```

pv, r2 = ols_pvaleurs(X[:, ret], y)
r2_final.append(r2); sig_final.append(np.mean(pv < 0.05))
# procédure honnête : on choisit sur les 50 premiers, on teste sur les 50 autres
ret_h = selection_ascendante(X[:50], y[:50])
if ret_h:
    pv_h, _ = ols_pvaleurs(X[50:][:, ret_h], y[50:])
    sig_honnête.append(np.sum(pv_h < 0.05)); nb_honnête.append(len(ret_h))
nb_retenues = np.array(nb_retenues)
print(f"y est du pur bruit, indépendant des {q} variables explicatives (n = {n_obs}).")
print(f"part des jeux de données où au moins une variable est « découverte » : {np.mean(nb_retenues >
    ↪ 0):.1%} (théorie pour une seule étape : 1 - 0,95^20 = {1 - 0.95**20:.1%})")
print(f"nombre moyen de variables retenues : {nb_retenues.mean():.2f}")
print(f"R² moyen du modèle final (quand il est non vide) : {np.mean(r2_final):.3f} alors que le vrai
    ↪ R² est 0")
print(f"part de coefficients « significatifs à 5 % » dans le modèle final : {np.mean(sig_final):.1%}
    ↪ (c'est trompeur : ils ont été choisis POUR l'être)")
print(f"procédure honnête (choix sur la moitié A, test sur la moitié B) : {np.sum(sig_honnête) /
    ↪ np.sum(nb_honnête):.1%} de significatifs parmi les variables retenues (≈ 5 % attendu)")

```

```

y est du pur bruit, indépendant des 20 variables explicatives (n = 100).
part des jeux de données où au moins une variable est « découverte » : 63.4% (théorie pour une seule
    ↪ étape : 1 - 0,95^20 = 64.2%)
nombre moyen de variables retenues : 1.09
R² moyen du modèle final (quand il est non vide) : 0.093 alors que le vrai R² est 0
part de coefficients « significatifs à 5 % » dans le modèle final : 100.0% (c'est trompeur : ils ont
    ↪ été choisis POUR l'être)
procédure honnête (choix sur la moitié A, test sur la moitié B) : 5.6% de significatifs parmi les
    ↪ variables retenues (≈ 5 % attendu)

```

Dans près des deux tiers des jeux de données, la procédure « découvrir » au moins une variable explicative, alors qu'il n'y en a **aucune** ; le  $R^2$  du modèle final est nettement positif (environ 0,09 en moyenne, alors que la vraie valeur est 0) ; et, dans le modèle final, les coefficients retenus apparaissent **tous** significatifs, ce qui est trompeur : ils n'ont été retenus que parce qu'ils l'étaient. La procédure « honnête », qui **sépare** les données utilisées pour choisir des données utilisées pour tester, retrouve le taux d'erreur attendu de 5 %.

**Les défauts de la sélection automatique.** (1) Les **p-valeurs et intervalles** du modèle final sont **trop optimistes** (biais de sélection). (2) Les coefficients retenus sont **gonflés** en valeur absolue (on garde ceux qui, par chance, sont grands). (3) Le résultat est **instable** : un autre échantillon donne un autre modèle. (4) Les procédures ne connaissent pas le **sens** des variables : on risque de retirer une variable de confusion essentielle. (5) Plus on essaie de modèles, plus on a de chances d'en trouver un « bon » par hasard (c'est le problème des tests multiples du volume I, section 3.5.5).

**Bonnes pratiques.** (1) **Commencez par la connaissance du domaine** : quelles variables ont un sens ? (2) Fixez les modèles candidats à **l'avance**, en petit nombre, et comparez-les par AIC/BIC et validation croisée. (3) Si vous devez explorer beaucoup de variables, **mettez de côté un échantillon de test** avant de commencer, et ne l'utilisez qu'**une fois**, à la fin. (4) Pour la prédiction avec beaucoup de variables, préférez la **régularisation** (section 1.5), qui fait une sélection plus stable. (5) Décrivez honnêtement tout ce qui a été essayé.

#### 1.4.6 Verdict : retrouver la vérité

Nous avons un luxe : les données sont simulées, nous connaissons donc le vrai modèle (détails dans la documentation de `build/donnees2.py`). Les clients actifs ont été produits par



$$\log(\text{panier}) = 4,00 + 0,008 (\text{âge} - 36) + \delta_{\text{canal}} + 0,12 F_1 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0,35^2),$$

avec  $\delta = +0,22$  pour la Boutique,  $+0,05$  pour le Site,  $-0,12$  pour Instagram. Ici  $F_1$  est un « **goût pour les produits** » non observé (de moyenne 0 et d'écart-type 1) ; ni la ville, ni l'offre de bienvenue, ni le score de service **n'interviennent** dans le panier. Comparons au modèle M3 retenu par le BIC :

```
m3 = fits["M3"]
ic = m3.conf_int()
vrai_site, vrai_insta = 0.05 - 0.22, -0.12 - 0.22 # effets du Site et d'Instagram
↳ RELATIVEMENT à la Boutique
vrai = {"Intercept": np.nan, "C(canal)[T.Site]": vrai_site, "C(canal)[T.Instagram]": vrai_insta, "a":
↳ 0.008}
comp = pd.DataFrame({"estimation (M3)": m3.params, "IC95 bas": ic[0], "IC95 haut": ic[1]})
comp["vérité"] = pd.Series(vrai)
comp["IC contient la vérité ?"] = [("oui" if (lo <= v <= hi) else "non") if not np.isnan(v) else "-"]
↳ for lo, hi, v in zip(comp["IC95 bas"], comp["IC95 haut"], comp["vérité"])]
print(comp.round(4).to_string())
print()
print("Variables retenues dans M3 : âge, canal, score produit. Écartées par les critères : ville,
↳ offre, score service, âge², interactions.")
print("Dans M7 (le modèle « tout »), p-valeurs des variables sans effet réel :")
print(fits["M7"].pvalues[["C(ville)[T.Bizerte]", "C(ville)[T.Nabeul]", "C(ville)[T.Sfax]",
↳ "C(ville)[T.Sousse]", "C(ville)[T.Tunis]", "offre_bienvenue", "a2",
↳ "score_service"]].round(3).to_string())
```

|                       | estimation (M3) | IC95 bas | IC95 haut | vérité | IC contient la vérité ? |
|-----------------------|-----------------|----------|-----------|--------|-------------------------|
| Intercept             | 3.6543          | 3.5385   | 3.7701    | NaN    | —                       |
| C(canal)[T.Site]      | -0.1573         | -0.2114  | -0.1032   | -0.170 | oui                     |
| C(canal)[T.Instagram] | -0.3519         | -0.4045  | -0.2993   | -0.340 | oui                     |
| a                     | 0.0097          | 0.0078   | 0.0117    | 0.008  | oui                     |
| score_produit         | 0.1557          | 0.1255   | 0.1859    | NaN    | —                       |

Variables retenues dans M3 : âge, canal, score produit. Écartées par les critères : ville, offre, score service, âge², interactions.

Dans M7 (le modèle « tout »), p-valeurs des variables sans effet réel :

|                     |       |
|---------------------|-------|
| C(ville)[T.Bizerte] | 0.901 |
| C(ville)[T.Nabeul]  | 0.934 |
| C(ville)[T.Sfax]    | 0.526 |
| C(ville)[T.Sousse]  | 0.808 |
| C(ville)[T.Tunis]   | 0.883 |
| offre_bienvenue     | 0.788 |
| a2                  | 0.300 |
| score_service       | 0.051 |

Les effets du **canal** et de l'**âge** sont retrouvés : les intervalles à 95 % contiennent les vraies valeurs ( $-0,17$  pour le Site,  $-0,34$  pour Instagram,  $+0,008$  par année d'âge). La méthode a bien écarté les variables **sans effet réel** (ville, offre, courbure, interactions). L'intercept ne se compare pas directement, car dans M3 il correspond à un score produit de 0 (une valeur impossible sur une échelle de 1 à 5) : un bon exemple de la nécessité de **centrer** les variables pour interpréter la constante.

Reste le cas du **score service** : il n'a aucun effet direct dans le vrai modèle du panier, et pourtant l'AIC et la validation croisée le **gardent**, de justesse, avec une  $t$ -statistique voisine de 1,9. C'est la **conséquence mécanique** de la corrélation entre les facteurs « produit » et « service » dans la population (corrélation de 0,3 entre  $F_1$  et  $F_2$ ) : le score de service est un peu informatif sur  $F_1$ , donc sur le panier. Et c'est un rappel qu'une **association n'est pas un effet**.

**Un dernier point, sur le score produit.** Le vrai effet est de **0,12 par écart-type de  $F_1$** . Or le coefficient estimé (0,15 environ) est un effet **par point de note**, ce qui est autre chose : la note moyenne

est un reflet **imparfait** de  $F_1$  (chaque question est bruitée). Un calcul direct à partir de la façon dont les notes ont été simulées (volume II, `donnees2.py` :  $q_j \approx 3,6 + 0,95 (\lambda_j F_1 + \sqrt{1 - \lambda_j^2} \eta_j)$ , avec des poids  $\lambda_j = 0,8; 0,7; 0,75; 0,6$ ) permet de prédire le coefficient attendu **par point de note** :

```
lam = np.array([0.80, 0.70, 0.75, 0.60])
var_bruit = np.mean(1 - lam**2) / 4           # variance du bruit de la moyenne de 4
↳ questions (en unités de F1)
charge = lam.mean()                           # poids moyen de F1 dans la moyenne des
↳ questions
fiabilite = charge**2 / (charge**2 + var_bruit) # part de la variance de la note qui
↳ provient de F1
cov_F_score = 0.95 * charge
var_score = 0.95**2 * (charge**2 + var_bruit)
pente_F_sur_score = cov_F_score / var_score    # E[F1 | score] = pente * (score - moyenne)
print(f"fiabilité du score produit : {fiabilite:.2f}")
print(f"effet attendu par point de note : 0,12 x {pente_F_sur_score:.3f} = {0.12 *
↳ pente_F_sur_score:.3f} | estimé dans M3 : {m3.params['score_produit']:.3f} (IC95 % :
↳ [{ic.loc['score_produit', 0]:.3f} ; {ic.loc['score_produit', 1]:.3f}])")

fiabilité du score produit : 0.81
effet attendu par point de note : 0,12 x 1.192 = 0.143 | estimé dans M3 : 0.156 (IC95 % : [0.125 ;
↳ 0.186])
```

La valeur attendue tombe à l'intérieur de l'intervalle de confiance. La **fiabilité** (la part de la variance de la note qui provient du vrai facteur, environ 0,8) est une notion générale : quand une variable explicative est mesurée avec du bruit, son coefficient est **atténué** par rapport à l'effet de la grandeur « vraie » (*erreur de mesure*) ; ici, la conversion d'échelle (par point de note plutôt que par écart-type de  $F_1$ ) cache cet effet, mais il est bien là.

**Une dernière subtilité (hors programme, mais honnête).** Nous n'observons le panier que pour les clients **actifs** (ayant commandé), et l'activité dépend elle-même de  $F_1$  dans la simulation (le goût pour les produits augmente le nombre de commandes). En ne gardant que les clients actifs, nous opérons une **sélection** qui peut biaiser légèrement la relation entre les notes et le panier. Le biais est ici faible et invisible dans les intervalles ; mais en pratique, restreindre un échantillon sur une variable liée à la réponse est une source classique de biais, que la section 2.6 (modèles à zéros excédentaires) permettra de traiter proprement.

#### À retenir (1.4).

- Un modèle plus complexe s'ajuste toujours mieux aux données d'**entraînement** ( $R^2$  croissant) mais pas forcément aux **nouvelles** données : c'est le **surajustement**.
- **AIC** =  $-2\hat{\ell} + 2k$  vise la prédiction ; **BIC** =  $-2\hat{\ell} + k \log n$  est plus sévère et consistant. Comparez les modèles sur les mêmes données et la même réponse ; seules les **différences** comptent.
- La **validation croisée** estime l'erreur sur de nouvelles données ; pour la régression linéaire, le leave-one-out s'obtient d'un seul ajustement :  $\text{PRESS} = \sum_i (\hat{\epsilon}_i / (1 - h_{ii}))^2$ .
- Les **tests F emboîtés** comparent deux modèles qui se contiennent.
- La **sélection automatique** fabrique des « découvertes » dans le bruit, gonfle les coefficients et rend les p-valeurs trompeuses : fixez des modèles candidats à l'avance, séparez choix et test, expliquez ce qui a été essayé.
- Sur nos données, les critères retrouvent la vérité : âge, canal et score produit comptent ; ville, offre de bienvenue et courbure n'ont aucun effet.

## 1.5 ⊕ Pour aller plus loin : la régularisation (Ridge, Lasso, Elastic Net)

**Section optionnelle.** Elle prolonge directement 1.3 (multicolinéarité) et 1.4 (surajustement) et sert de pont vers l'apprentissage automatique (volume III). On peut la sauter sans perdre le fil du chapitre.

**Intuition.** Quand un modèle a beaucoup de variables pour peu de données, les moindres carrés ont **trop de liberté** : ils utilisent chaque coefficient pour coller au bruit, d'où des coefficients énormes et instables. La **régularisation** consiste à leur mettre une **laisse** : on minimise l'erreur *plus* une pénalité qui grandit avec la taille des coefficients. On accepte un petit biais volontaire (les coefficients sont « rétrécis » vers 0) en échange d'une **forte baisse de variance**. C'est une application directe du dilemme biais-variance, et la preuve que le théorème de Gauss-Markov (1.1.5) a des limites : il ne dit rien des estimateurs **biaisés**.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from sklearn.linear_model import Ridge, RidgeCV, Lasso, LassoCV, ElasticNetCV, lasso_path,
↳ LinearRegression
from sklearn.preprocessing import StandardScaler
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ENCRE, GRIS, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7",
↳ "#0b0b0b", "#898781", "#e34948"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ "#e1e0d9",
        "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
        "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)
```

### 1.5.1 La régression Ridge

**Le principe.** On ajoute à la somme des carrés une pénalité sur la **norme au carré** des coefficients (sans la constante) :

$$\hat{\beta}_{\text{ridge}}(\lambda) = \arg \min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|^2, \quad \lambda \geq 0.$$

Le **paramètre de régularisation**  $\lambda$  règle la sévérité de la laisse :  $\lambda = 0$  redonne les moindres carrés,  $\lambda \rightarrow \infty$  écrase tous les coefficients vers 0.

**Solution explicite.** Le gradient de la fonction objectif est  $-2X^T(y - X\beta) + 2\lambda\beta$ . En l'annulant :

$$(X^T X + \lambda I) \hat{\beta}_{\text{ridge}} = X^T y \implies \hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

Pour  $\lambda > 0$ ,  $X^T X + \lambda I$  est **toujours inversible** (ses valeurs propres sont  $\geq \lambda > 0$ ), même si  $X$  n'est pas de rang plein ou si  $p > n$  : la régularisation résout aussi le problème de la colinéarité parfaite.

Deux règles pratiques, essentielles : (1) **la constante n'est pas pénalisée** (on centre simplement  $y$  et les colonnes de  $X$ ) ; (2) **il faut standardiser les variables** (moyenne 0, écart-type 1) : sinon la pénalité frappe plus les variables exprimées dans de petites unités (un âge en années contre un revenu en milliers de dinars) et le résultat dépend des unités choisies.

**Voir ce que fait la pénalité : la SVD.** Écrivons la décomposition en valeurs singulières de  $\mathbf{X}$  centrée :  $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$  (volume I, section 1.1.4), où les  $d_j$  sont les valeurs singulières. Alors

$$\mathbf{X}\hat{\beta}_{\text{ridge}} = \sum_{j=1}^p \mathbf{u}_j \frac{d_j^2}{d_j^2 + \lambda} \mathbf{u}_j^\top \mathbf{y}.$$

**Démonstration.**  $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} = \mathbf{V}(\mathbf{D}^2 + \lambda \mathbf{I})^{-1} \mathbf{V}^\top$  et  $\mathbf{X}^\top \mathbf{y} = \mathbf{V}\mathbf{D}\mathbf{U}^\top \mathbf{y}$ , donc  $\mathbf{X}\hat{\beta}_{\text{ridge}} = \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda \mathbf{I})^{-1} \mathbf{D}\mathbf{U}^\top \mathbf{y}$ , matrice diagonale  $d_j^2/(d_j^2 + \lambda)$ .  $\square$

Les moindres carrés correspondent à  $\lambda = 0$  : tous les facteurs valent 1. Ridge **rétrécit** chaque direction principale  $\mathbf{u}_j$  d'un facteur  $d_j^2/(d_j^2 + \lambda) \in (0, 1)$  : **très peu** pour les directions où les données varient beaucoup ( $d_j$  grand), **beaucoup** pour celles où elles varient peu ( $d_j$  petit), c'est-à-dire précisément les directions de **quasi-colinéarité**, où l'estimation par moindres carrés est la plus instable.

**Pourquoi un estimateur biaisé peut être meilleur.** L'erreur quadratique moyenne (EQM) d'un estimateur se décompose en biais<sup>2</sup> + variance (volume I, section 3.2.2). Prenons le cas le plus simple, où les colonnes de  $\mathbf{X}$  sont orthonormales ( $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$ ) : alors  $\hat{\beta}_j^{\text{MCO}} = \beta_j + \eta_j$  avec  $\text{Var } \eta_j = \sigma^2$ , et  $\hat{\beta}_j^{\text{ridge}} = \hat{\beta}_j^{\text{MCO}}/(1 + \lambda)$ .

**L'EQM de Ridge est inférieure à celle des MCO pour un  $\lambda$  petit.** Pour Ridge, biais =  $-\frac{\lambda}{1 + \lambda}\beta_j$  et variance =  $\frac{\sigma^2}{(1 + \lambda)^2}$ , donc

$$\text{EQM}(\lambda) = \frac{\lambda^2 \beta_j^2 + \sigma^2}{(1 + \lambda)^2}.$$

Sa dérivée en  $\lambda = 0$  vaut  $-2\sigma^2 < 0$  : **l'EQM décroît dès qu'on s'écarte de  $\lambda = 0$** , quel que soit  $\beta_j$ . Le minimum est atteint en  $\lambda^* = \sigma^2/\beta_j^2$  : la bonne régularisation est d'autant plus forte que le bruit est grand et le signal faible. Le gain n'est pas un accident : *il existe toujours un  $\lambda > 0$  pour lequel Ridge bat les moindres carrés en EQM.*

**Voyons-le par simulation.** Deux variables explicatives très corrélées (corrélation 0,98),  $n = 30$ , des vrais coefficients (1, 1) et  $\sigma = 1$ . On répète 4 000 fois l'expérience et on mesure l'EQM des estimateurs pour plusieurs  $\lambda$  :

```
rng = np.random.default_rng(4)
n_s, rho = 30, 0.98
cov = np.array([[1, rho], [rho, 1]])
Xs = rng.multivariate_normal([0, 0], cov, size=n_s)
Xs = (Xs - Xs.mean(axis=0)) / Xs.std(axis=0) # variables centrées et standardisées,
↳ FIXES pour toute l'expérience
beta_vrai = np.array([1.0, 1.0]) / np.sqrt(n_s) * 3 # vrais coefficients (de petite
↳ taille relative au bruit)
R = 4000
resultats = {}
for lam in [0, 0.5, 1, 2, 5, 10, 20, 50]:
    M = np.linalg.solve(Xs.T @ Xs + lam * np.eye(2), Xs.T) # (X'X + lambda I)^-1 X'
    erreurs = np.empty((R, 2))
    for r in range(R):
        y_s = Xs @ beta_vrai + rng.normal(0, 1, n_s)
        erreurs[r] = M @ y_s - beta_vrai
    if lam == 0:
        erreurs_mco = erreurs.copy()
```

```

resultats[lam] = (np.mean(np.sum(erreurs**2, axis=1)), np.sum(erreurs.mean(axis=0)**2),
↳ np.sum(erreurs.var(axis=0)))
tab = pd.DataFrame(resultats, index=["EQM totale", "biais² total", "variance totale"]).T
tab.index.name = "lambda"
print("corrélation entre les deux variables :", round(np.corrcoef(Xs.T)[0, 1], 3))
print(tab.round(4).to_string())
print("corrélation entre les deux estimations MCO du même échantillon :",
↳ round(np.corrcoef(erreurs_mco.T)[0, 1], 3))

```

```

corrélation entre les deux variables : 0.984
      EQM totale  biais² total  variance totale
lambda
0.0          2.0874         0.0002          2.0872
0.5          0.5132         0.0001          0.5131
1.0          0.2281         0.0003          0.2278
2.0          0.0899         0.0007          0.0892
5.0          0.0332         0.0038          0.0294
10.0         0.0292         0.0128          0.0164
20.0         0.0487         0.0378          0.0109
50.0         0.1310         0.1259          0.0052
corrélation entre les deux estimations MCO du même échantillon : -0.984

```

Pour  $\lambda = 0$  (les moindres carrés), la **variance** domine : les deux estimations sont fortement corrélées négativement entre elles (si l'une surestime, l'autre sous-estime : voir la corrélation affichée), ce qui rend chacune très instable. Quand  $\lambda$  augmente, le **biais²** augmente et la **variance** chute ; l'EQM totale passe par un **minimum** vers  $\lambda = 10$  (0,029, contre 2,09 pour les moindres carrés : soixante-dix fois moins), avant de remonter quand le biais devient excessif ( $\lambda = 50$  : 0,13). Le meilleur estimateur au sens de l'EQM est donc **biaisé** : ce n'est pas en contradiction avec Gauss-Markov, qui ne parlait que des estimateurs **sans biais**.

**À la main contre scikit-learn.** Vérifions notre formule sur les données de Dar Jasmin, avec les notes de l'enquête (nous introduisons ici le jeu de variables de la section suivante) :

```

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
enq = pd.read_csv("donnees/enquete_satisfaction.csv")
enq["score_produit"] = enq[["q1", "q2", "q3", "q4"]].mean(axis=1)
enq["score_service"] = enq[["q5", "q6", "q7", "q8"]].mean(axis=1)
bq = df.merge(enq[["id_client", "score_produit", "score_service"]],
↳ on="id_client").reset_index(drop=True)

# Un petit jeu de variables : l'âge, le canal, les scores
Xp = pd.DataFrame({"age": bq["a"], "Site": (bq["canal"] == "Site").astype(float), "Instagram":
↳ (bq["canal"] == "Instagram").astype(float),
↳ "score_produit": bq["score_produit"], "score_service": bq["score_service"]})
yv = bq["log_panier"].to_numpy()
sc = StandardScaler().fit(Xp)
Z = sc.transform(Xp)
yc = yv - yv.mean()
lam = 50.0
beta_main = np.linalg.solve(Z.T @ Z + lam * np.eye(Z.shape[1]), Z.T @ yc)
sk = Ridge(alpha=lam).fit(Z, yv)
print(pd.DataFrame({"à la main": beta_main, "scikit-learn": sk.coef_, "MCO (λ = 0)":
↳ LinearRegression().fit(Z, yv).coef_}, index=Xp.columns).round(5).to_string())

```

|           | à la main | scikit-learn | MCO ( $\lambda = 0$ ) |
|-----------|-----------|--------------|-----------------------|
| age       | 0.09800   | 0.09800      | 0.10255               |
| Site      | -0.06342  | -0.06342     | -0.07490              |
| Instagram | -0.15797  | -0.15797     | -0.17241              |

|               |         |         |         |
|---------------|---------|---------|---------|
| score_produit | 0.09827 | 0.09827 | 0.10322 |
| score_service | 0.02016 | 0.02016 | 0.02035 |

Les deux colonnes de Ridge coïncident : `scikit-learn` fait exactement le calcul de la formule. Les coefficients de Ridge sont **rétrécis** par rapport aux moindres carrés, modestement ici ( $\lambda = 50$  est petit devant  $n \approx 1\,000$ ) : environ 4 % pour l'âge, 15 % pour le Site, 8 % pour Instagram. Le rétrécissement est le plus marqué pour les indicatrices du canal, qui sont **corrélées entre elles** (les clients Site ne sont pas Instagram) : leurs directions ont de petites valeurs singulières, exactement celles que le facteur  $d_j^2 / (d_j^2 + \lambda)$  pénalise le plus.

### 1.5.2 Le Lasso : la pénalité qui élimine des variables

Ridge rétrécit tous les coefficients, mais **n'en met jamais aucun exactement à 0**. Le **Lasso** (*Least Absolute Shrinkage and Selection Operator*) remplace le carré par la **valeur absolue** :

$$\hat{\beta}_{\text{lasso}}(\alpha) = \arg \min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \alpha \sum_j |\beta_j|.$$

(La normalisation  $1/(2n)$  est celle de `scikit-learn`.) Ce changement apparemment minuscule a une conséquence majeure : le Lasso produit des solutions **creuses**, où beaucoup de coefficients sont **exactement nuls**. C'est une régression qui **sélectionne** ses variables tout en les ajustant.

**Pourquoi des zéros ? La géométrie.** Minimiser la somme des carrés sous la contrainte  $\sum_j \beta_j^2 \leq r^2$  (Ridge) ou  $\sum_j |\beta_j| \leq r$  (Lasso) donne le même résultat que les versions pénalisées. Les courbes de niveau de la somme des carrés sont des **ellipses** centrées sur la solution des moindres carrés ; la solution contrainte est le **premier point de contact** entre une ellipse qui grandit et la région admissible. Pour Ridge, la région est un **disque** : le contact est un point lisse, quelconque, où aucun coefficient n'est nul. Pour le Lasso, la région est un **losange** dont les **sommets sont sur les axes** : une ellipse qui grandit touche très souvent un sommet, où un coefficient est exactement nul.

```
def solution_contrainte(centre, A, region, r, n_pts=4000):
    """Point de la frontière de la région (norme p = 1 ou 2, rayon r) qui minimise (b-c)'A(b-c)."""
    t = np.linspace(0, 2 * np.pi, n_pts)
    if region == 2:
        pts = r * np.column_stack([np.cos(t), np.sin(t)])
    else:
        c, s_ = np.cos(t), np.sin(t)
        pts = r * np.column_stack([c, s_]) / (np.abs(c) + np.abs(s_))[:, None]
    d = pts - centre
    val = np.einsum("ij,jk,ik->i", d, A, d)
    k = np.argmin(val)
    return pts[k], val[k]

centre = np.array([2.0, 0.30]) # solution des moindres carrés
theta = np.deg2rad(-30)
Rot = np.array([np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)])
A = Rot @ np.diag([1.0, 3.0]) @ Rot.T # ellipses allongées
fig, axes = plt.subplots(1, 2, figsize=(10, 4.6))
for ax, region, titre in zip(axes, [2, 1], ["Ridge : contrainte en disque", "Lasso : contrainte en
↳ losange"]):
    r = 1.15
    t = np.linspace(0, 2 * np.pi, 400)
    if region == 2:
        ax.fill(r * np.cos(t), r * np.sin(t), color=BLEU, alpha=0.18)
        ax.plot(r * np.cos(t), r * np.sin(t), color=BLEU, lw=1.8)
    else:
```



```

sommets = np.array([[r, 0], [0, r], [-r, 0], [0, -r], [r, 0]])
ax.fill(sommets[:, 0], sommets[:, 1], color=BLEU, alpha=0.18)
ax.plot(sommets[:, 0], sommets[:, 1], color=BLEU, lw=1.8)
sol, val = solution_contrainte(centre, A, region, r)
for niveau in [val * 0.25, val * 0.6, val, val * 2.4, val * 5]:      # ellipses de niveau
    u = np.linspace(0, 2 * np.pi, 400)
    w, V = np.linalg.eigh(A)
    el = np.sqrt(niveau) * (V @ np.diag(1 / np.sqrt(w)) @ np.vstack([np.cos(u), np.sin(u)])) +
    centre
    ax.plot(el[:, 0], el[:, 1], color=ORANGE if np.isclose(niveau, val) else GRIS, lw=1.6 if
    np.isclose(niveau, val) else 0.9)
ax.scatter(*centre, color=ENCRE, zorder=5, s=30); ax.annotate("moindres carrés", centre,
    textcoords="offset points", xytext=(6, 6), fontsize=9)
ax.scatter(*sol, color=ROUGE, zorder=6, s=45); ax.annotate("solution\npénalisée", sol,
    textcoords="offset points", xytext=(14, -40) if region == 2 else (16, -42), color=ROUGE,
    fontsize=9)
ax.axhline(0, color=GRIS, lw=0.8); ax.axvline(0, color=GRIS, lw=0.8)
ax.set_xlim(-1.6, 3.1); ax.set_ylim(-1.6, 1.9); ax.set_aspect("equal")
ax.set_xlabel("coefficient β1"); ax.set_ylabel("coefficient β2"); ax.set_title(titre,
    fontsize=10.5)
print(f"{titre}: solution = ({sol[0]:.3f}, {sol[1]:.3f})")
plt.tight_layout()
plt.savefig("figures/ch01-geometrie-ridge-lasso.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

Ridge : contrainte en disque: solution = (1.071, 0.420)

Lasso : contrainte en losange: solution = (1.150, 0.000)

figure enregistrée

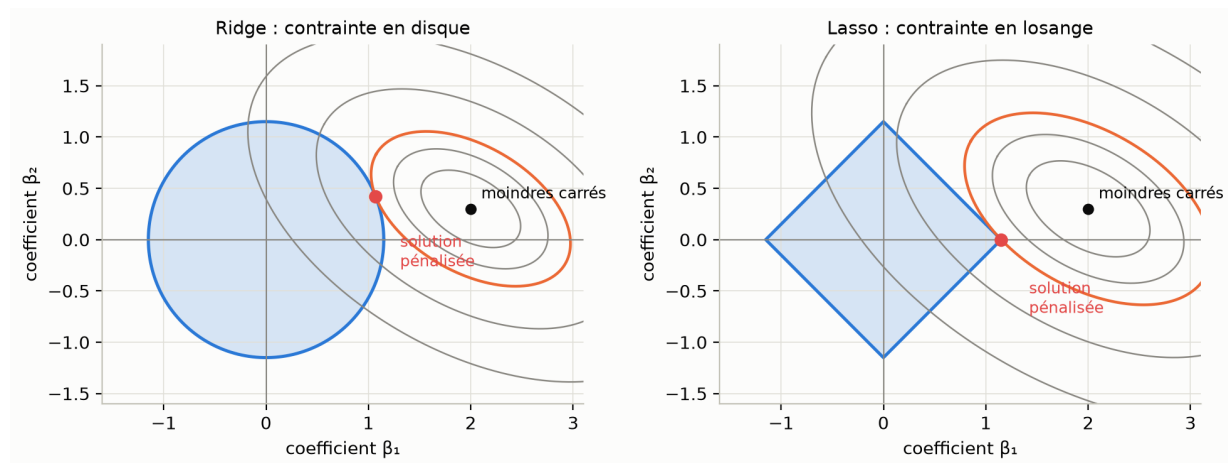


Figure 10: La géométrie de la régularisation. Les ellipses sont les courbes de niveau de la somme des carrés, centrées sur la solution des moindres carrés. À gauche (Ridge), la région admissible est un disque et le point de contact a ses deux coordonnées non nulles. À droite (Lasso), la région est un losange : le contact se fait au sommet, sur l'axe, avec  $\beta_2 = 0$  exactement.

**Comment calcule-t-on le Lasso ?** Il n'a pas de formule fermée (la valeur absolue n'est pas dérivable en 0), mais un algorithme très simple : la **descente par coordonnées**. On optimise un coefficient à la fois, les autres étant fixés, et on recommence jusqu'à stabilisation. Pour des variables standardisées ( $\frac{1}{n}\mathbf{x}_j^\top \mathbf{x}_j = 1$ ), la mise à jour du coefficient  $j$  est donnée par un **seuillage doux** :

$$\beta_j \leftarrow S(\rho_j, \alpha), \quad \rho_j = \frac{1}{n} \mathbf{x}_j^\top (\mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \beta_k), \quad S(\rho, \alpha) = \text{signe}(\rho) \max(|\rho| - \alpha, 0).$$

**Pourquoi le seuillage doux ?** Fixons les autres coefficients : on minimise en  $\beta_j$  la fonction  $\frac{1}{2}(\beta_j - \rho_j)^2 + \alpha|\beta_j|$  (en développant et en utilisant  $\frac{1}{n}\mathbf{x}_j^\top \mathbf{x}_j = 1$ ). Pour  $\beta_j > 0$ , la dérivée est  $\beta_j - \rho_j + \alpha$ , nulle en  $\beta_j = \rho_j - \alpha$ , valable si  $\rho_j > \alpha$  ; symétriquement pour  $\beta_j < 0$ . Si  $|\rho_j| \leq \alpha$ , la dérivée à droite en 0 ( $-\rho_j + \alpha$ ) est  $\geq 0$  et la dérivée à gauche ( $-\rho_j - \alpha$ ) est  $\leq 0$  : le minimum est exactement en 0. Le Lasso **ramène à 0** toute variable dont la corrélation (partielle) avec le résidu est inférieure au seuil  $\alpha$ , et **rétrécit de  $\alpha$**  les autres.

Écrivons-la à la main et comparons à `scikit-learn` :

```
def lasso_coordonnees(Z, y, alpha, n_iter=500):
    n_, p_ = Z.shape
    beta = np.zeros(p_)
    for _ in range(n_iter):
        for j in range(p_):
            r_partiel = y - Z @ beta + Z[:, j] * beta[j]           # résidu en retirant l'effet de
            ↪ toutes les variables sauf j
            rho = Z[:, j] @ r_partiel / n_
            beta[j] = np.sign(rho) * max(abs(rho) - alpha, 0.0)      # seuillage doux
    return beta

alpha = 0.05
b_main = lasso_coordonnees(Z, yc, alpha)
b_sk = Lasso(alpha=alpha, max_iter=100000, tol=1e-12).fit(Z, yv).coef_
print(pd.DataFrame({"à la main": b_main, "scikit-learn": b_sk, "MCO": LinearRegression().fit(Z,
    ↪ yv).coef_}, index=Xp.columns).round(5).to_string())
print("coefficients exactement nuls (à la main) :", list(Xp.columns[b_main == 0]))
```

|               | à la main | scikit-learn | MCO      |
|---------------|-----------|--------------|----------|
| age           | 0.05297   | 0.05297      | 0.10255  |
| Site          | -0.00000  | -0.00000     | -0.07490 |
| Instagram     | -0.07486  | -0.07486     | -0.17241 |
| score_produit | 0.05465   | 0.05465      | 0.10322  |
| score_service | 0.00000   | 0.00000      | 0.02035  |

coefficients exactement nuls (à la main) : ['Site', 'score\_service']

Les deux versions coïncident, ce qui valide notre algorithme. Mais regardez ce que fait ce seuil  $\alpha = 0,05$ , choisi arbitrairement : le Lasso met à zéro `score_service`, qui (dans le vrai modèle) n'a pas d'effet direct sur le panier, mais aussi `Site`, qui a un vrai effet ( $-15\%$  environ, 1.2.2), et il **divise par deux** à peu près les autres coefficients. Cette pénalité est donc **trop forte** : un  $\alpha$  mal choisi supprime de vraies variables et sous-estime les effets. D'où l'importance de choisir  $\alpha$  par validation croisée, ce que nous faisons maintenant.

### 1.5.3 L'Elastic Net : un compromis

Le Lasso a un défaut : face à un **groupe de variables très corrélées**, il en choisit une presque au hasard et élimine les autres, de façon instable. Ridge, au contraire, répartit le poids entre elles. L'**Elastic Net** combine les deux pénalités :

$$\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \alpha \left( \rho \sum_j |\beta_j| + \frac{1-\rho}{2} \sum_j \beta_j^2 \right), \quad \rho \in [0, 1].$$

Pour  $\rho = 1$  c'est le Lasso, pour  $\rho \rightarrow 0$ , Ridge. Il fournit des solutions **creuses** tout en étant **stables** en présence de variables corrélées. On choisit  $\alpha$  et  $\rho$  par validation croisée.



### 1.5.4 Application : beaucoup de variables, peu de clients

Mettons les trois méthodes à l'épreuve dans un scénario réaliste d'**analyse exploratoire** : on dispose de **35 variables candidates** pour prévoir le log-panier d'un client, et de seulement **120 clients** pour apprendre le modèle. Parmi ces 35 variables : 4 portent un vrai signal (l'âge, deux indicatrices de canal, le score produit), quelques-unes sont des **variables pertinentes mais inutiles** (ville, offre de bienvenue, score service, courbure de l'âge, interactions), et **20 sont du pur bruit** que nous ajoutons (nombres tirés au hasard : des « variables » sans aucun rapport avec les clients). On teste sur les 956 autres clients de l'enquête.

```
rng = np.random.default_rng(31)
ville = bq["id_client"].map(clients.set_index("id_client")["ville"])
villes = pd.get_dummies(ville, prefix="ville", drop_first=True, dtype=float)      # 5 indicatrices
↳ (Autre = référence)
F = pd.DataFrame({
    "age": bq["a"].astype(float), "age_carre": bq["a"].astype(float) ** 2, "age_cube":
    ↳ bq["a"].astype(float) ** 3,
    "canal_Site": (bq["canal"] == "Site").astype(float), "canal_Instagram": (bq["canal"] ==
    ↳ "Instagram").astype(float),
    "offre_bienvenue":
    ↳ bq["id_client"].map(clients.set_index("id_client")["offre_bienvenue"]).astype(float),
    "score_produit": bq["score_produit"], "score_service": bq["score_service"],
    "age_x_Site": bq["a"] * (bq["canal"] == "Site").astype(float), "age_x_Instagram": bq["a"] *
    ↳ (bq["canal"] == "Instagram").astype(float),
})
F = pd.concat([F, villes], axis=1)
bruit = pd.DataFrame(rng.normal(size=(len(F), 20)), columns=[f"bruit_{i+1:02d}" for i in range(20)])
F = pd.concat([F, bruit], axis=1)
signal = ["age", "canal_Site", "canal_Instagram", "score_produit"]                # les variables qui
↳ ont un VRAI effet dans la simulation
print("nombre de variables candidates :", F.shape[1], "| dont du pur bruit :", 20, "| clients :",
    ↳ len(F))

ordre = rng.permutation(len(F))
tr, te = ordre[:120], ordre[120:]
sc = StandardScaler().fit(F.iloc[tr])
Ztr, Zte = sc.transform(F.iloc[tr]), sc.transform(F.iloc[te])
ytr, yte = yv[tr], yv[te]
print("entraînement :", len(tr), "clients | test :", len(te), "clients")
```

```
nombre de variables candidates : 35 | dont du pur bruit : 20 | clients : 1076
entraînement : 120 clients | test : 956 clients
```

Entraînons cinq modèles : les **moindres carrés avec les 35 variables**, un modèle de **référence** qui ne contient que les vraies variables (inaccessible en pratique, puisqu'on ne sait pas lesquelles sont vraies), puis **Ridge**, **Lasso** et **Elastic Net** avec  $\lambda$  choisi par validation croisée à 10 paquets sur l'échantillon d'entraînement.

```
def rmse(y, p):
    return float(np.sqrt(np.mean((y - p) ** 2)))

noms = list(F.columns)
idx_signal = [noms.index(v) for v in signal]
modeles = {}
modeles["MC0, 35 variables"] = LinearRegression().fit(Ztr, ytr)
ref = LinearRegression().fit(Ztr[:, idx_signal], ytr)
ridge = RidgeCV(alphas=np.logspace(-1, 3.5, 60), cv=10).fit(Ztr, ytr)
lasso = LassoCV(alphas=100, cv=10, random_state=0, max_iter=100000).fit(Ztr, ytr)
enet = ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8, 0.95], alphas=60, cv=10, random_state=0,
    ↳ max_iter=100000).fit(Ztr, ytr)
modeles.update({"Ridge ( $\lambda$  par CV)": ridge, "Lasso ( $\alpha$  par CV)": lasso, "Elastic Net (CV)": enet})
```

```

lignes = [{"modèle": "constante seule", "RMSE apprentissage": rmse(ytr, np.full(len(ytr),
↪ ytr.mean())), "RMSE test": rmse(yte, np.full(len(yte), ytr.mean())),
          "variables non nulles": 0, "dont bruit": 0}]
lignes.append({"modèle": "référence : 4 vraies variables", "RMSE apprentissage": rmse(ytr,
↪ ref.predict(Ztr[:, idx_signal])),
              "RMSE test": rmse(yte, ref.predict(Zte[:, idx_signal])), "variables non nulles": 4,
              ↪ "dont bruit": 0})
for nom, m in modeles.items():
    coef = m.coef_
    non_nuls = np.abs(coef) > 1e-10
    lignes.append({"modèle": nom, "RMSE apprentissage": rmse(ytr, m.predict(Ztr)), "RMSE test":
↪ rmse(yte, m.predict(Zte)),
                  "variables non nulles": int(non_nuls.sum()), "dont bruit": int(sum(non_nuls[i] for
↪ i, n in enumerate(noms) if n.startswith("bruit"))))
res = pd.DataFrame(lignes).set_index("modèle")
print(res.round(4).to_string())
print()
print(f"Ridge : lambda choisi = {ridge.alpha_:.1f} | Lasso : alpha choisi = {lasso.alpha_:.4f} |
↪ Elastic Net : alpha = {enet.alpha_:.4f}, rho = {enet.ll_ratio_}")

```

| modèle                         | RMSE apprentissage | RMSE test | variables non nulles | dont bruit |
|--------------------------------|--------------------|-----------|----------------------|------------|
| constante seule                | 0.3803             | 0.4039    | 0                    | 0          |
| référence : 4 vraies variables | 0.3093             | 0.3612    | 4                    | 0          |
| MCO, 35 variables              | 0.2728             | 0.4152    | 35                   | 20         |
| Ridge ( $\lambda$ par CV)      | 0.3200             | 0.3765    | 35                   | 20         |
| Lasso ( $\alpha$ par CV)       | 0.3032             | 0.3609    | 15                   | 7          |
| Elastic Net (CV)               | 0.3038             | 0.3608    | 15                   | 7          |

Ridge : lambda choisi = 134.0 | Lasso : alpha choisi = 0.0250 | Elastic Net : alpha = 0.0266, rho = 0.95

Lisons ce tableau, qui contient presque toute la leçon :

- Les **moindres carrés avec 35 variables** ont l'erreur d'apprentissage la plus faible (0,273 : ils s'ajustent au bruit) mais l'erreur de test **la pire** (0,415), **plus mauvaise que celle de la simple constante** (0,404) : avec 35 variables pour 120 clients, le modèle prédit moins bien que si l'on n'avait pas de modèle du tout. C'est le surajustement dans toute sa gloire.
- **Ridge** corrige une grande partie du problème en rétrécissant (erreur de test 0,377), mais il garde les 35 variables avec un coefficient non nul (y compris les 20 de bruit).
- Le **Lasso** et l'**Elastic Net** font mieux encore : ils ne gardent que 15 variables, et leur erreur de test (0,361) **égale** celle de la **référence** (0,361), un modèle qui connaît à l'avance les vraies variables, ce qu'on ne sait jamais faire en pratique. (La très légère avance du Lasso sur la référence, 0,3609 contre 0,3612, est dans le bruit d'échantillonnage : il ne faut pas en tirer de conclusion.)
- Ces méthodes ne font pas une sélection parfaite : sur les 15 variables conservées, **7 sont du bruit**. Mais elles les gardent avec des coefficients **petits**.

Regardons les coefficients retenus par le Lasso, comparés à ceux des moindres carrés :

```

coef = pd.DataFrame({"MCO": modeles["MCO, 35 variables"].coef_, "Ridge": ridge.coef_, "Lasso":
↪ lasso.coef_, "Elastic Net": enet.coef_}, index=noms)
vus = coef[(coef["Lasso"].abs() > 1e-10) | coef.index.isin(signal)]
print("coefficients (variables standardisées) pour les variables retenues par le Lasso ou vraiment
↪ utiles :")
print(vus.round(3).to_string())
print()
print("variables de bruit : plus grand |coefficient| MCO =",
↪ round(coef.loc[coef.index.str.startswith("bruit"), "MCO"].abs().max(), 3),
      "| Ridge =", round(coef.loc[coef.index.str.startswith("bruit"), "Ridge"].abs().max(), 3),
      "| Lasso =", round(coef.loc[coef.index.str.startswith("bruit"), "Lasso"].abs().max(), 3))

```

coefficients (variables standardisées) pour les variables retenues par le Lasso ou vraiment utiles :

|                 | MCO    | Ridge  | Lasso  | Elastic Net |
|-----------------|--------|--------|--------|-------------|
| age             | 0.165  | 0.033  | 0.050  | 0.050       |
| age_carre       | 0.052  | 0.015  | 0.011  | 0.010       |
| canal_Site      | -0.174 | -0.033 | -0.091 | -0.090      |
| canal_Instagram | -0.218 | -0.056 | -0.132 | -0.131      |
| score_produit   | 0.106  | 0.053  | 0.096  | 0.095       |
| age_x_Instagram | -0.005 | 0.022  | 0.026  | 0.026       |
| ville_Sousse    | -0.067 | -0.021 | -0.013 | -0.013      |
| ville_Tunis     | -0.037 | 0.013  | 0.004  | 0.003       |
| bruit_01        | 0.073  | 0.024  | 0.025  | 0.024       |
| bruit_02        | 0.060  | 0.012  | 0.013  | 0.012       |
| bruit_11        | -0.034 | -0.016 | -0.005 | -0.004      |
| bruit_13        | -0.052 | -0.013 | -0.004 | -0.003      |
| bruit_14        | 0.020  | 0.020  | 0.021  | 0.021       |
| bruit_19        | 0.013  | 0.015  | 0.004  | 0.003       |
| bruit_20        | -0.054 | -0.021 | -0.014 | -0.014      |

variables de bruit : plus grand |coefficient| MCO = 0.073 | Ridge = 0.024 | Lasso = 0.025

Le Lasso retient **les quatre vraies variables** (âge, Site, Instagram, score produit), avec des coefficients de 0,05 à 0,13 en valeur absolue, et laisse passer 7 variables de bruit dont les coefficients sont **au plus 0,025**, c'est-à-dire plus de deux fois plus petits que le plus petit coefficient d'une vraie variable : l'ordre de grandeur permet de les distinguer. Avec les moindres carrés, au contraire, les variables de bruit atteignent 0,073, un coefficient du même ordre que celui du score produit (0,106) : on ne peut plus faire la différence. Remarquez aussi que les coefficients MCO des vraies variables sont **gonflés** par rapport à ceux du Lasso (l'âge : 0,165 contre 0,050) : c'est le **biais de sélection** vu au 1.4.5, qui joue ici à l'envers, puisque la régularisation le contrôle.

**Les chemins de régularisation.** Pour voir la régularisation à l'œuvre, on trace comment chaque coefficient évolue quand la pénalité varie, de très forte (tous les coefficients à 0) à très faible (on retrouve les moindres carrés).

```

alphas_l, coefs_l, _ = lasso_path(Ztr, ytr - ytr.mean(), alphas=80)
alphas_r = np.logspace(-1, 4, 80)
coefs_r = np.array([np.linalg.solve(Ztr.T @ Ztr + a * np.eye(Ztr.shape[1])), Ztr.T @ (ytr -
    ↳ ytr.mean()) for a in alphas_r]).T
couleurs = {"age": VIOLET, "canal_Site": AQUA, "canal_Instagram": ORANGE, "score_produit": BLEU}
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4), sharey=True)
for ax, al, cf, titre, choisi in [(ax1, alphas_r, coefs_r, "Ridge", ridge.alpha_), (ax2, alphas_l,
    ↳ coefs_l, "Lasso", lasso.alpha_)]:
    for i, nom in enumerate(noms):
        if nom in couleurs:
            continue
        ax.plot(np.log10(al), cf[i], color=GRIS, lw=0.7, alpha=0.55)
    for i, nom in enumerate(noms):
        if nom in couleurs:
            ax.plot(np.log10(al), cf[i], color=couleurs[nom], lw=2.2, label=nom)
    ax.axvline(np.log10(choisi), color=ENCRE, ls="--", lw=1)
    ax.text(np.log10(choisi), ax.get_ylim()[1] * 0.92, "choisi par\n validation croisée",
    ↳ fontsize=8.5, va="top")
    ax.axhline(0, color=GRIS, lw=0.8)
    ax.set_xlabel("pénalité : log10(λ)" if titre == "Ridge" else "pénalité : log10(α)")
    ax.set_title(titre + " : coefficients (variables standardisées)", fontsize=10.5)
ax1.set_ylabel("valeur du coefficient")
ax1.legend(frameon=False, fontsize=8.5, loc="lower left")
ax1.invert_xaxis(); ax2.invert_xaxis()
plt.tight_layout()
plt.savefig("figures/ch01-chemins-regularisation.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

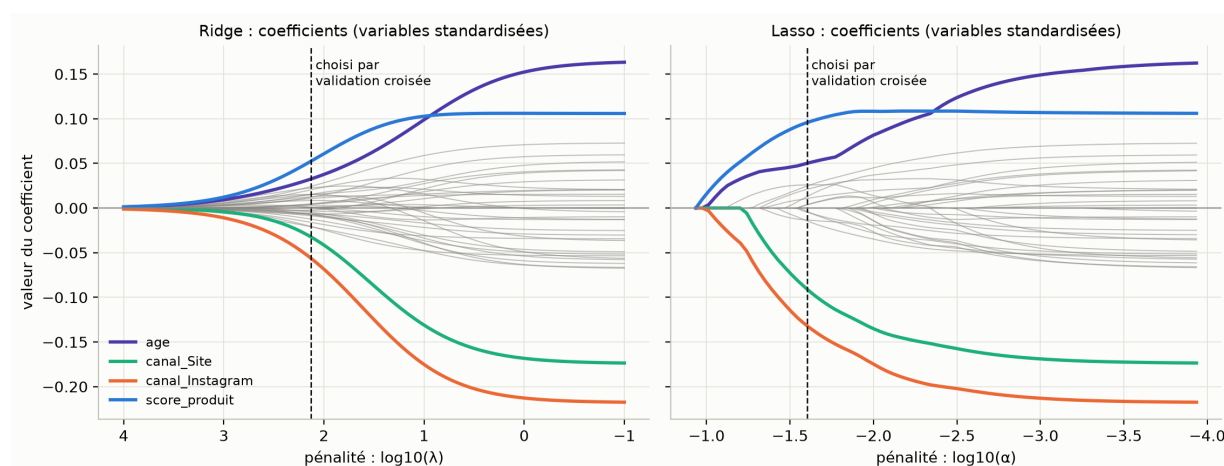


Figure 11: Chemins de régularisation (à gauche : Ridge ; à droite : Lasso). Les coefficients des quatre vraies variables sont en couleur, ceux des 31 autres variables en gris. Plus on va vers la droite (pénalité faible), plus les coefficients grossissent. Le Lasso met à zéro les coefficients un par un ; Ridge les rétrécit tous ensemble sans jamais les annuler. La ligne pointillée marque la pénalité choisie par validation croisée.

Sur la gauche de chaque graphique, la pénalité est énorme et tous les coefficients valent 0 ; en allant vers la droite, la pénalité se relâche et les coefficients se déploient. Avec le Lasso, les variables **entrent une par une** dans le modèle, à peu près dans l'ordre de leur pouvoir prédictif : le score produit, l'âge et Instagram apparaissent en premier ; le Site, dont l'effet est pourtant réel, n'apparaît qu'à peu près en même temps que les premières variables de bruit (il est partiellement redondant avec Instagram, comme on l'a vu avec Ridge) ; les variables de bruit entrent avec de petits coefficients. À la valeur choisie par validation croisée (pointillés), on est dans la zone où le signal est capté et où la plus grande partie du bruit est encore écartée. Avec Ridge, tous les coefficients se rétrécissent **ensemble** : le bruit n'est pas éliminé, seulement atténué.

### 1.5.5 Précautions

- **Standardisez toujours** les variables (et estimez la moyenne et l'écart-type sur l'échantillon d'**entraînement** seulement, jamais sur les données de test : sinon on laisse fuir de l'information du test dans le modèle).
- Choisissez  $\lambda$  **par validation croisée** sur l'entraînement ; ne regardez l'échantillon de test qu'à la fin.
- Les coefficients régularisés sont **biaisés** par construction : on ne les interprète pas comme des effets (« à toutes choses égales par ailleurs ») avec la même confiance. Il n'y a pas de p-valeurs ni d'intervalles de confiance « standard » : les formules du 1.2 ne s'appliquent plus, et la sélection par le Lasso soulève précisément le problème de l'inférence **après sélection** discuté au 1.4.5.
- La régularisation vise la **prédiction**. Pour estimer un effet précis et le décrire à Yasmine, on revient en général à un modèle plus simple et interprétable, éventuellement **choisi** avec l'aide du Lasso, mais **réajusté** sur de nouvelles données.
- **Lien avec le bayésien (chapitre 6)**. Ridge équivaut à une approche bayésienne où chaque coefficient a une loi *a priori* **normale** centrée en 0 (le coefficient est « probablement petit »), et le Lasso à une loi *a priori* de **Laplace** (plus piquée en 0, d'où les zéros). La section 6.1 reprendra cette interprétation :  $\lambda$  y correspond au rapport entre la variance du bruit et celle de l'*a priori*.

**À retenir (1.5).**

- Ridge :  $\hat{\beta} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$  ; il **rétrécit** les directions peu variables ( $d_j^2 / (d_j^2 + \lambda)$ ), est toujours défini et réduit la variance au prix d'un **biais** : l'EQM s'améliore pour un  $\lambda$  petit (et optimal en  $\sigma^2 / \beta^2$  dans le cas orthonormal).
- Lasso : pénalité  $\sum |\beta_j|$  ; il produit des solutions **creuses** (seuillage doux, descente par coordonnées) et sert de **sélection de variables**. Elastic Net : mélange des deux, stable pour les variables corrélées.
- Standardiser, ne pas pénaliser la constante, choisir  $\lambda$  par validation croisée, évaluer sur des données de test non utilisées.
- Beaucoup de variables pour peu de données : les moindres carrés surajustent (ici, pire que la simple constante) ; la régularisation rejoint la prédiction du meilleur modèle possible. Mais ses coefficients ne s'interprètent pas comme ceux d'un modèle ordinaire.

**1.6 ⊕ Pour aller plus loin : la régression robuste**

**Section optionnelle.** Elle prolonge la discussion des observations aberrantes et influentes de 1.3.4 et s'appuie sur l'idée, vue au volume I (section 3.1.3), que la médiane est plus robuste que la moyenne.

**Intuition.** Les moindres carrés élèvent chaque écart **au carré** : un client dont le panier est dix fois trop grand (une virgule mal placée dans un export) pèse **cent fois** plus qu'un client ordinaire. Quelques erreurs de saisie peuvent suffire à fausser tout le modèle. La régression **robuste** limite l'influence des points extrêmes : elle cherche la droite qui colle à **la majorité** des données, sans se laisser tirer par quelques cas isolés. Elle fait pour la régression ce que la médiane fait pour la moyenne.

Voici la préparation habituelle :

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ENCRE, GRIS, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7",
↳ "#0b0b0b", "#898781", "#e34948"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ "#e1e0d9",
        "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
        "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy().reset_index(drop=True)
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
m_propre = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
print(len(df), "clients | coefficients du modèle sur les données propres :")
print(m_propre.params.round(4).to_string())
```

```
1740 clients | coefficients du modèle sur les données propres :
Intercept          4.2206
```

```
C(canal)[T.Site]      -0.1571
C(canal)[T.Instagram] -0.3368
a                      0.0092
```

### 1.6.1 La fragilité des moindres carrés

Reprenons l'exemple du volume I (3.1.3) : cinq commandes de 20, 25, 30, 35 et 400 DT. La **moyenne** (102 DT) est entraînée par la dernière valeur ; la **médiane** (30 DT) ne bouge pas. On dit que la médiane a un **point de rupture** de 50 % (il faut corrompre la moitié des données pour la faire dériver à l'infini) alors que celui de la moyenne est de **0 %** (une seule valeur suffit). Les moindres carrés, qui généralisent la moyenne, héritent de cette fragilité.

**Pourquoi ? La fonction d'influence.** Un estimateur  $\hat{\theta}$  minimise  $\sum_i \rho(e_i)$ . Son équation d'estimation est  $\sum_i \psi(e_i) \mathbf{x}_i = \mathbf{0}$  où  $\psi = \rho'$  (comme les équations normales du 1.1.3, qui correspondent à  $\rho(e) = e^2/2$  et  $\psi(e) = e$ ). La fonction  $\psi$  mesure l'**influence** d'un résidu sur l'estimation. Pour les moindres carrés,  $\psi(e) = e$  n'est **pas bornée** : plus un point est extrême, plus il tire. Un estimateur robuste choisit un  $\rho$  dont la dérivée  $\psi$  est **bornée** : au-delà d'un seuil, un résidu de plus en plus grand n'a pas plus d'influence.

### 1.6.2 Les M-estimateurs et la fonction de Huber

La fonction de **Huber** est quadratique pour les petits résidus (comme les moindres carrés : efficace quand tout va bien) et **linéaire** pour les grands (comme la valeur absolue : l'influence est plafonnée). Avec un seuil  $c$  (en unités d'écart-type du résidu) :

$$\rho_c(u) = \begin{cases} \frac{1}{2}u^2 & \text{si } |u| \leq c \\ c|u| - \frac{1}{2}c^2 & \text{si } |u| > c \end{cases} \quad \psi_c(u) = \begin{cases} u & \text{si } |u| \leq c \\ c \operatorname{sign}(u) & \text{si } |u| > c \end{cases} \quad u = \frac{e}{s}$$

où  $s$  est une estimation **robuste** de l'échelle des résidus : on prend le **MAD** (écart absolu médian) :  $s = \text{médiane}(|e_i - \text{médiane}(e)|) / 0,6745$  (le facteur 0,6745 rend  $s$  cohérent avec l'écart-type pour des erreurs normales). Le seuil usuel  $c = 1,345$  garantit que, **si les erreurs sont réellement normales**, l'estimateur de Huber conserve **95 % de l'efficacité** des moindres carrés : on perd très peu quand tout va bien, et on gagne beaucoup quand il y a des aberrations.

**L'algorithme : les moindres carrés repondérés (IRLS).** Les équations d'estimation  $\sum_i \psi(u_i) \mathbf{x}_i = \mathbf{0}$  s'écrivent  $\sum_i w_i u_i \mathbf{x}_i = \mathbf{0}$  avec les **poids**  $w_i = \psi(u_i) / u_i$  : une régression **pondérée**. Pour Huber,  $w_i = 1$  si  $|u_i| \leq c$  et  $w_i = c / |u_i|$  sinon : chaque point reçoit un poids d'autant plus petit qu'il est loin de la droite. Comme les poids dépendent des résidus qui dépendent de  $\beta$ , on **itère** : (1) calculer les résidus, (2) en déduire les poids, (3) refaire une régression pondérée, jusqu'à stabilisation.

Dessignons les fonctions de perte et de poids de trois méthodes : les moindres carrés, Huber, et la fonction « bisquare » de Tukey (plus radicale : les points très éloignés reçoivent un poids **nul**).

```
u = np.linspace(-6, 6, 400)
c_h, c_t = 1.345, 4.685
rho_mco = u**2 / 2
rho_huber = np.where(np.abs(u) <= c_h, u**2 / 2, c_h * np.abs(u) - c_h**2 / 2)
rho_tukey = np.where(np.abs(u) <= c_t, c_t**2 / 6 * (1 - (1 - (u / c_t)**2)**3), c_t**2 / 6)
w_mco = np.ones_like(u)
w_huber = np.where(np.abs(u) <= c_h, 1.0, c_h / np.abs(u))
w_tukey = np.where(np.abs(u) <= c_t, (1 - (u / c_t)**2)**2, 0.0)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.0))
for rho, nom, col in [(rho_mco, "moindres carrés", GRIS), (rho_huber, "Huber (c = 1,345)", BLEU),
                      (rho_tukey, "Tukey bisquare (c = 4,685)", ORANGE)]:
```



```

ax1.plot(u, rho, color=col, lw=2.2, label=nom)
ax1.set_ylim(0, 9); ax1.set_xlabel("résidu standardisé u"); ax1.set_ylabel("perte  $\rho(u)$ ");
↪ ax1.set_title("Perte attribuée à un résidu", fontsize=10.5)
ax1.legend(frameon=False, loc="upper center", fontsize=9)
for w, col, nom in [(w_mco, GRIS, "moindres carrés : poids constant égal à 1"), (w_huber, BLEU, "Huber
↪ : poids  $c/|u|$  au-delà de 1,345"), (w_tukey, ORANGE, "Tukey : poids nul au-delà de 4,685")]:
    ax2.plot(u, w, color=col, lw=2.2, label=nom)
ax2.set_ylim(-0.05, 1.15); ax2.set_xlabel("résidu standardisé u"); ax2.set_ylabel("poids  $w(u)$ ");
↪ ax2.set_title("Poids donné à l'observation", fontsize=10.5)
ax2.legend(frameon=False, loc="lower center", fontsize=8.5, bbox_to_anchor=(0.5, 0.08))
plt.tight_layout()
plt.savefig("figures/ch01-fonctions-robustes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

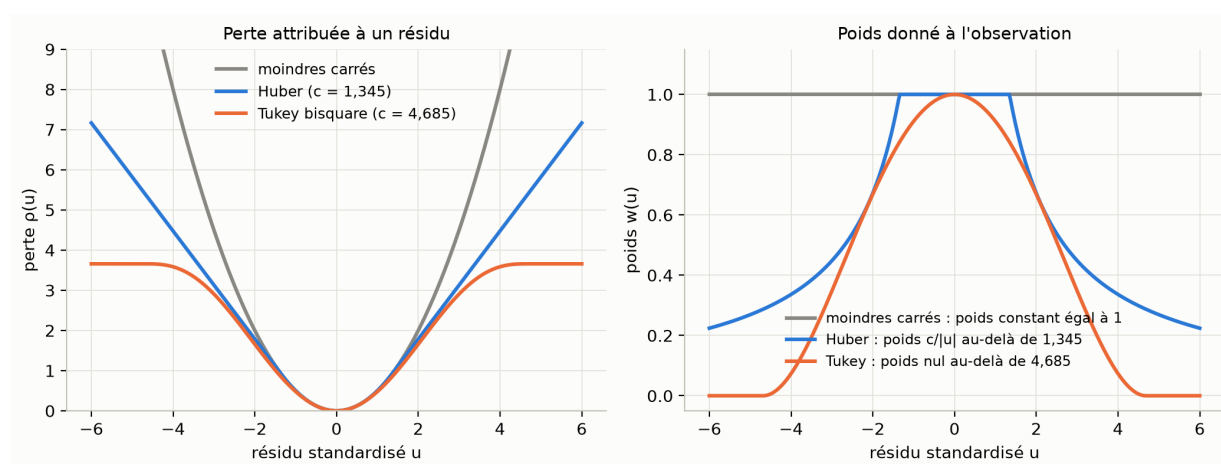


Figure 12: Fonctions de perte (à gauche) et poids (à droite). Les moindres carrés (gris) pénalisent quadratiquement et donnent le même poids à tous. Huber (bleu) devient linéaire au-delà de 1,345 écart-type. Tukey (orange) plafonne la perte et annule le poids des points très éloignés.

Écrivons l'algorithme IRLS à la main et comparons à statsmodels (RLM, *robust linear model*) sur les données propres :

```

def huber_irls(X, y, c=1.345, n_iter=100, tol=1e-10):
    beta = np.linalg.lstsq(X, y, rcond=None)[0]           # départ : les moindres carrés
    for _ in range(n_iter):
        e = y - X @ beta
        s = np.median(np.abs(e - np.median(e))) / 0.6745  # échelle robuste (MAD)
        u = e / s
        w = np.where(np.abs(u) <= c, 1.0, c / np.abs(u))    # poids de Huber
        W = X.T * w                                         # X' diag(w)
        beta_new = np.linalg.solve(W @ X, W @ y)          # régression pondérée
        if np.max(np.abs(beta_new - beta)) < tol:
            beta = beta_new
            break
    beta = beta_new
    return beta, w, s

Xp, yp = m_propre.model.exog, m_propre.model.endog
b_main, w_main, s_main = huber_irls(Xp, yp)
rlm = sm.RLM(yp, Xp, M=sm.robust.norms.HuberT(t=1.345)).fit()
print(pd.DataFrame({"MCO": m_propre.params, "Huber (à la main)": b_main, "RLM statsmodels":
↪ rlm.params}).round(4).to_string())
print(f"échelle robuste s : {s_main:.4f} | écart-type résiduel des MCO :
↪ {np.sqrt(m_propre.scale):.4f}")

```

```
print(f"part des clients dont le poids est inférieur à 1 : {np.mean(w_main < 1):.1%}")
```

|                       | MCO     | Huber (à la main) | RLM statsmodels |
|-----------------------|---------|-------------------|-----------------|
| Intercept             | 4.2206  | 4.2171            | 4.2172          |
| C(canal)[T.Site]      | -0.1571 | -0.1602           | -0.1602         |
| C(canal)[T.Instagram] | -0.3368 | -0.3331           | -0.3332         |
| a                     | 0.0092  | 0.0090            | 0.0090          |

échelle robuste s : 0.3667 | écart-type résiduel des MCO : 0.3705  
part des clients dont le poids est inférieur à 1 : 18.1%

Sur des données **propres**, Huber et les moindres carrés donnent presque les mêmes coefficients : c'est l'efficacité de 95 % en action : on ne paie quasiment rien pour la robustesse. (L'écart minime, de l'ordre de  $10^{-4}$ , entre notre version et RLM tient à des choix de détail sur l'estimation de l'échelle.) Remarquez aussi que **18 % des clients reçoivent un poids inférieur à 1 même sur ces données propres** : ce n'est pas un signe d'anomalie, c'est exactement la proportion attendue pour des erreurs normales, puisque  $\mathbb{P}(|Z| > 1,345) \approx 17,9\%$ . Huber n'écarte pas 18 % des données : il en réduit un peu le poids.

### 1.6.3 Une contamination : les erreurs de saisie

Corrompons maintenant les données, comme le ferait un export mal formaté. Dans 6 % des clients tirés au hasard, la virgule du panier est mal placée : le panier est **multiplié par 10** (54 DT devient 540 DT). Nous gardons la version propre (df) et fabriquons une copie contaminée (dfc).

```
rng = np.random.default_rng(77)
dfc = df.copy()
cible = rng.choice(len(df), size=int(0.06 * len(df)), replace=False) # 6 % de clients touchés
dfc.loc[cible, "panier_moyen"] = dfc.loc[cible, "panier_moyen"] * 10
dfc["log_panier"] = np.log(dfc["panier_moyen"])
contamine = np.zeros(len(df), dtype=bool); contamined[cible] = True
print(len(cible), "paniers corrompus sur", len(df), f"({contamine.mean():.1%})")

m_mco_c = smf.ols("log_panier ~ a + C(canal)", data=dfc).fit()
rlm_c = smf.rlm("log_panier ~ a + C(canal)", data=dfc, M=sm.robust.norms.HuberT(t=1.345)).fit()
tab = pd.DataFrame({"MCO données propres": m_propre.params, "MCO contaminé": m_mco_c.params, "Huber"
    ↳ "contaminé": rlm_c.params})
tab_se = pd.DataFrame({"MCO données propres": m_propre.bse, "MCO contaminé": m_mco_c.bse, "Huber"
    ↳ "contaminé": rlm_c.bse})
print("Coefficients :"); print(tab.round(4).to_string())
print("\nErreurs standard :"); print(tab_se.round(4).to_string())
print(f"\nécart-type résiduel : MCO propre = {np.sqrt(m_propre.scale):.3f} | MCO contaminé =
    ↳ {np.sqrt(m_mco_c.scale):.3f} | échelle robuste de Huber = {rlm_c.scale:.3f}")
print(f"panier médian prédit pour un client Boutique de 36 ans : propre =
    ↳ {np.exp(m_propre.params['Intercept']):.1f} DT | MCO contaminé =
    ↳ {np.exp(m_mco_c.params['Intercept']):.1f} DT | Huber contaminé =
    ↳ {np.exp(rlm_c.params['Intercept']):.1f} DT")
poids = rlm_c.weights
print(f"poids de Huber moyen : clients corrompus = {poids[contamine].mean():.2f} | clients intacts =
    ↳ {poids[~contamine].mean():.2f}")
```

104 paniers corrompus sur 1740 (6.0%)

Coefficients :

|                       | MCO données propres | MCO contaminé | Huber contaminé |
|-----------------------|---------------------|---------------|-----------------|
| Intercept             | 4.2206              | 4.3898        | 4.2628          |
| C(canal)[T.Site]      | -0.1571             | -0.1853       | -0.1573         |
| C(canal)[T.Instagram] | -0.3368             | -0.3919       | -0.3527         |
| a                     | 0.0092              | 0.0088        | 0.0091          |

Erreurs standard :

|           | MCO données propres | MCO contaminé | Huber contaminé |
|-----------|---------------------|---------------|-----------------|
| Intercept | 0.0175              | 0.0314        | 0.0201          |



|                       |        |        |        |
|-----------------------|--------|--------|--------|
| C(canal)[T.Site]      | 0.0231 | 0.0414 | 0.0265 |
| C(canal)[T.Instagram] | 0.0225 | 0.0403 | 0.0258 |
| a                     | 0.0008 | 0.0015 | 0.0010 |

écart-type résiduel : MCO propre = 0.370 | MCO contaminé = 0.665 | échelle robuste de Huber = 0.402  
 panier médian prédit pour un client Boutique de 36 ans : propre = 68.1 DT | MCO contaminé = 80.6 DT |  
 ↳ Huber contaminé = 71.0 DT  
 poids de Huber moyen : clients corrompus = 0.24 | clients intacts = 0.97

Les conséquences pour les moindres carrés sont de deux types. (1) Un **biais** : la constante est tirée vers le haut (de 4,22 à 4,39, soit +18 % sur le panier médian prédit : 80,6 DT au lieu de 68,1 DT ; en moyenne on attendrait  $0,06 \times \ln 10 \approx 0,14$ , le reste vient de la répartition aléatoire des erreurs entre les canaux), les effets du Site et d'Instagram sont gonflés, et les erreurs standard grossissent ; (2) surtout, l'**écart-type résiduel** passe de 0,37 à 0,665 : les moindres carrés *attribuent à tort un bruit énorme* à tout le modèle, donc des intervalles de confiance beaucoup trop larges, des prévisions moins précises, et des effets réels (le canal, l'âge) qui deviennent plus difficiles à détecter. Huber, lui, **donne un poids faible aux clients corrompus** (0,24 en moyenne, contre 0,97 aux clients intacts) : sa constante (4,26) et son effet d'Instagram (−0,353) restent proches de ceux des données propres (4,22 et −0,337), alors que ceux des moindres carrés dérivent (4,39 et −0,392) ; son échelle (0,40) est proche de l'écart-type propre (0,37) alors que celui des moindres carrés double (0,665). Ses erreurs standard (0,020 pour la constante) sont proches de celles des données propres (0,017) et loin de celles des moindres carrés contaminés (0,031).

Visualisons-le sur un modèle simple (le log-panier selon l'âge seul, pour pouvoir tracer les droites) :

```
fig, ax = plt.subplots(figsize=(7.4, 4.6))
ax.scatter(df.loc[~contamine, "age"], df.loc[~contamine, "log_panier"], s=7, color=GRIS, alpha=0.5,
           label="paniers corrects")
ax.scatter(df.loc[contamine, "age"], df.loc[contamine, "log_panier"], s=14, color=ROUGE, alpha=0.8,
           label="paniers corrompus (× 10)")
s1 = smf.ols("log_panier ~ a", data=df).fit()
s2 = smf.ols("log_panier ~ a", data=dfc).fit()
s3 = smf.rlm("log_panier ~ a", data=dfc, M=sm.robust.norms.HuberT(t=1.345)).fit()
xa = np.linspace(18, 75, 50)
for m, col, nom, ls in [(s1, ENCRE, "MCO, données propres", "-"), (s2, ORANGE, "MCO, données
    contaminées", "-"), (s3, BLEU, "Huber, données contaminées", "--)]:
    ax.plot(xa, m.params["Intercept"] + m.params["a"] * (xa - 36), color=col, lw=2.2, ls=ls, label=nom)
ax.set_xlabel("âge (ans)"); ax.set_ylabel("log du panier moyen")
ax.legend(frameon=False, fontsize=8.5, loc="lower right", ncol=2)
ax.set_ylim(1.5, 7.6)
plt.savefig("figures/ch01-robuste-contamination.png", dpi=200, bbox_inches="tight")
print("pente (MCO propre) =", round(s1.params["a"], 4), "| (MCO contaminé) =", round(s2.params["a"],
    ↳ 4), "| (Huber contaminé) =", round(s3.params["a"], 4))
print("constante (MCO propre) =", round(s1.params["Intercept"], 3), "| (MCO contaminé) =",
    ↳ round(s2.params["Intercept"], 3), "| (Huber contaminé) =", round(s3.params["Intercept"], 3))
```

pente (MCO propre) = 0.009 | (MCO contaminé) = 0.0086 | (Huber contaminé) = 0.0092  
 constante (MCO propre) = 4.033 | (MCO contaminé) = 4.171 | (Huber contaminé) = 4.07

Les points rouges forment une bande décalée d'environ  $\ln 10 \approx 2,3$  au-dessus du nuage principal. La droite des moindres carrés (orange) est visiblement tirée vers le haut ; celle de Huber (bleu pointillé) reste beaucoup plus proche de la droite que l'on obtiendrait sans contamination (noire) : la constante passe de 4,03 (données propres) à 4,17 pour les moindres carrés mais seulement à 4,07 pour Huber, et la pente est presque intacte (0,0092 pour Huber, 0,0086 pour les moindres carrés, 0,0090 sans contamination). Huber n'est pas totalement insensible (les points corrompus gardent un petit poids, 0,24 en moyenne), mais l'essentiel du dégât est évité.

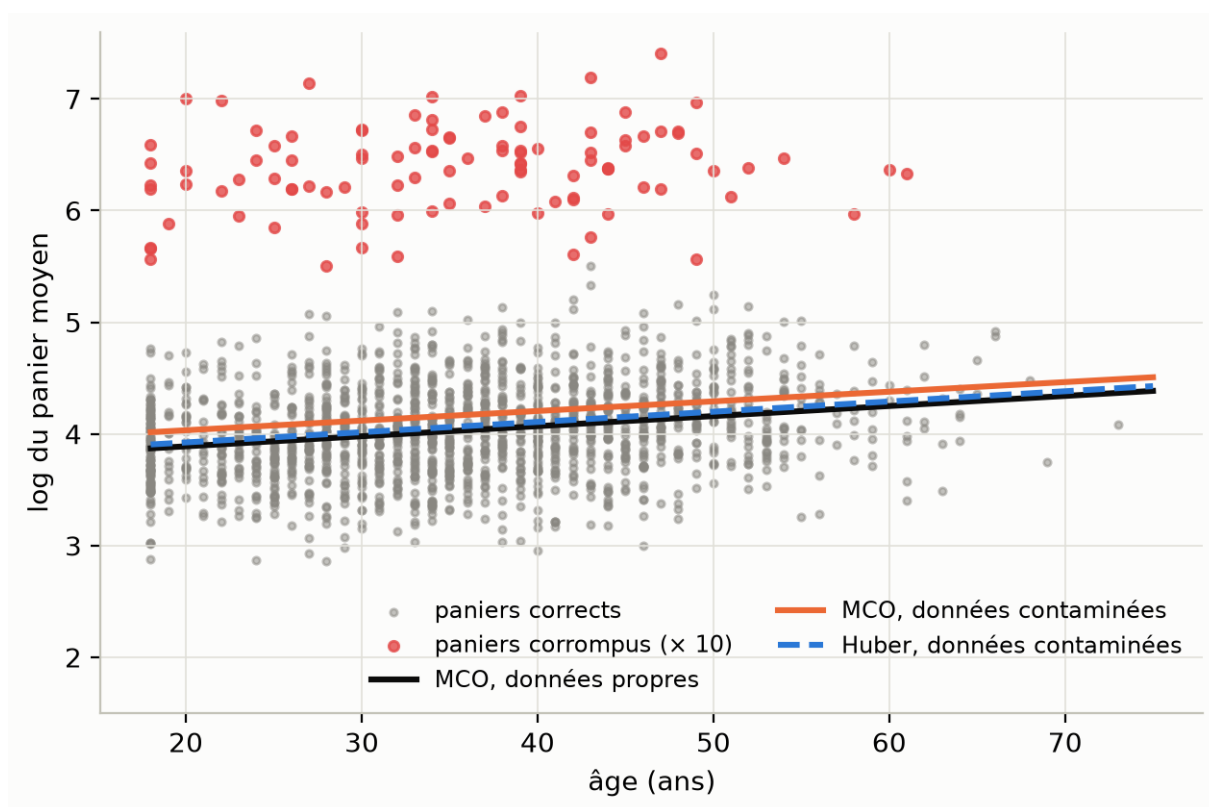


Figure 13: Nuage du log-panier selon l'âge, avec 6 % de paniers corrompus (rouge, environ 2,3 plus haut). La droite des moindres carrés sur les données contaminées (orange) est décalée vers le haut par rapport à la droite sur données propres (noire) ; la droite de Huber sur les mêmes données contaminées (bleu pointillé) reste beaucoup plus proche de la droite propre.

### 1.6.4 La limite : les points à fort levier

Les M-estimateurs de Huber protègent contre les **valeurs aberrantes de la réponse**  $y$  (les « aberrations verticales »), mais **pas contre les points à fort levier** (valeurs aberrantes des *variables explicatives*, 1.3.4). Leur fonction  $\psi$  borne l'influence du résidu, mais pas celle de la position  $x_i$  : un point très éloigné en  $x$  attire la droite, et se retrouve avec un résidu qui n'a pas l'air grand. Faisons une seconde contamination : cette fois, ce sont des âges mal saisis. Pour 1 % des clients, l'âge est écrit avec un chiffre de trop (35 devient 350).

```
rng = np.random.default_rng(78)
dfl = df.copy()
cible_l = rng.choice(len(df), size=int(0.01 * len(df)), replace=False)
dfl.loc[cible_l, "age"] = dfl.loc[cible_l, "age"] * 10
dfl["a"] = dfl["age"] - 36
print(len(cible_l), "âges corrompus ; âge maximal après contamination :", int(dfl["age"].max()),
      ↪ "ans")
m_mco_l = smf.ols("log_panier ~ a + C(canal)", data=dfl).fit()
rlm_l = smf.rlm("log_panier ~ a + C(canal)", data=dfl, M=sm.robust.norms.HuberT(t=1.345)).fit()
print(pd.DataFrame({"MCO propre": m_propre.params, "MCO âges corrompus": m_mco_l.params, "Huber âges
  ↪ corrompus": rlm_l.params}).round(4).to_string())
print(f"coefficient de l'âge : propre = {m_propre.params['a']:.4f} | MCO corrompu =
  ↪ {m_mco_l.params['a']:.4f} | Huber corrompu = {rlm_l.params['a']:.4f}")
print(f"levier maximal : {m_mco_l.get_influence().hat_matrix_diag.max():.3f} (levier moyen =
  ↪ {4/len(dfl):.4f})")
```

```
17 âges corrompus ; âge maximal après contamination : 640 ans
               MCO propre  MCO âges corrompus  Huber âges corrompus
Intercept          4.2206             4.2156             4.2126
C(canal)[T.Site]   -0.1571            -0.1575            -0.1603
C(canal)[T.Instagram] -0.3368           -0.3349            -0.3338
a                   0.0092             0.0008             0.0009
coefficient de l'âge : propre = 0.0092 | MCO corrompu = 0.0008 | Huber corrompu = 0.0009
levier maximal : 0.126 (levier moyen = 0.0023)
```

Les deux méthodes sont mises en échec : le coefficient de l'âge est **écrasé vers 0** (par les moindres carrés comme par Huber), parce que ces 17 clients, supposés avoir jusqu'à 640 ans, ont un panier ordinaire : sur ces points, la droite « âge ↗, panier ↗ » est contredite, et leur très fort levier leur donne le pouvoir de l'aplatir. Les estimateurs robustes **à fort levier** (MM-estimateurs, moindres carrés tronqués) existent, mais dans ce cas la bonne réponse est plus simple : **vérifier les données**. Un âge de 350 ans est **impossible** : une règle de validation élémentaire ( $\text{age} \leq 100$ ) l'élimine.

**Robuste ne veut pas dire infallible.** (1) Une méthode robuste ne remplace pas la **vérification des données** : un âge de 350 ans doit être détecté et corrigé, pas « absorbé » par un estimateur. (2) Les méthodes robustes protègent contre un certain **type** d'anomalie (ici, les erreurs sur  $y$ ) et pas contre tous. (3) Elles sont moins efficaces que les moindres carrés si les données sont parfaitement propres (un peu : 5 % pour Huber). (4) Leurs erreurs standard demandent des formules spécifiques ; RLM les fournit, mais leurs propriétés sont asymptotiques. (5) Si les « aberrations » sont **réelles** (quelques très gros clients), il ne faut pas les cacher : il faut **les étudier**, car elles peuvent être ce qu'il y a de plus important commercialement.

### 1.6.5 Une autre voie : la régression quantile

Une approche voisine consiste à modéliser non plus la **moyenne** conditionnelle mais la **médiane** conditionnelle, en minimisant la **somme des valeurs absolues** des résidus (perte  $\rho(e) = |e|$ , dont l'influence  $\psi = \text{signe}(e)$  est bornée) : c'est la **régression médiane** (LAD, *least absolute deviations*), robuste aux aberrations verticales. Elle se généralise à un **quantile** quelconque  $\tau \in (0, 1)$  avec la

perte asymétrique  $\rho_{\tau}(e) = e(\tau - \mathbb{1}_{e < 0})$  : on modélise alors le  $\tau$ -ième quantile de la réponse, ce qui décrit **toute la distribution** et non son seul centre.

Voyons-le sur les paniers. Dans le modèle en **dinars**, que dit le canal Instagram sur les paniers faibles, moyens et élevés ? Et dans le modèle en **log** ?

```
taus = [0.1, 0.25, 0.5, 0.75, 0.9]
res_niveau, res_log = [], []
for tau in taus:
    qn = smf.quantreg("panier_moyen ~ a + C(canal)", data=df).fit(q=tau)
    ql = smf.quantreg("log_panier ~ a + C(canal)", data=df).fit(q=tau)
    res_niveau.append((tau, qn.params["C(canal)"][T.Instagram]),
    ↪ *qn.conf_int().loc["C(canal)"][T.Instagram]))
    res_log.append((tau, ql.params["C(canal)"][T.Instagram]),
    ↪ *ql.conf_int().loc["C(canal)"][T.Instagram]))
rn = pd.DataFrame(res_niveau, columns=["tau", "coef", "bas", "haut"]).set_index("tau")
rl = pd.DataFrame(res_log, columns=["tau", "coef", "bas", "haut"]).set_index("tau")
mco_n = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit()
print("Effet d'Instagram (par rapport à la Boutique) sur le panier en DT, par quantile :")
print(rn.round(2).to_string())
print(f"(MCO, effet sur la moyenne : {mco_n.params['C(canal)'][T.Instagram]:.2f} DT)")
print("\nEffet d'Instagram sur le log du panier, par quantile :")
print(rl.round(3).to_string())
print(f"(MCO, effet sur la moyenne du log : {m_propre.params['C(canal)'][T.Instagram]:.3f})")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.0))
for ax, r, ref, titre, yl in [(ax1, rn, mco_n.params["C(canal)"][T.Instagram], "Panier en DT", "effet
    ↪ d'Instagram (DT)"),
    (ax2, rl, m_propre.params["C(canal)"][T.Instagram], "Log du panier",
    ↪ "effet d'Instagram (log)")]:
    ax.fill_between(r.index, r["bas"], r["haut"], color=BLEU, alpha=0.2)
    ax.plot(r.index, r["coef"], "o-", color=BLEU, lw=2, label="régression quantile")
    ax.axhline(ref, color=ORANGE, lw=1.8, ls="--", label="moindres carrés (moyenne)")
    ax.set_xlabel("quantile  $\tau$  du panier (0,1 = petits paniers ; 0,9 = grands paniers)");
    ↪ ax.set_ylabel(yl); ax.set_title(titre, fontsize=10.5)
ax1.legend(frameon=False, fontsize=8.5, loc="lower left")
plt.tight_layout()
plt.savefig("figures/ch01-regression-quantile.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

Effet d'Instagram (par rapport à la Boutique) sur le panier en DT, par quantile :

|                              | coef   | bas    | haut   |
|------------------------------|--------|--------|--------|
| tau                          |        |        |        |
| 0.10                         | -13.50 | -16.18 | -10.82 |
| 0.25                         | -14.40 | -16.92 | -11.89 |
| 0.50                         | -18.27 | -21.44 | -15.09 |
| 0.75                         | -24.22 | -28.42 | -20.02 |
| 0.90                         | -33.87 | -40.70 | -27.04 |
| (MCO, effet sur la moyenne : | -20.81 | DT)    |        |

Effet d'Instagram sur le log du panier, par quantile :

|                                     | coef    | bas    | haut   |
|-------------------------------------|---------|--------|--------|
| tau                                 |         |        |        |
| 0.10                                | -0.373  | -0.448 | -0.298 |
| 0.25                                | -0.331  | -0.387 | -0.275 |
| 0.50                                | -0.310  | -0.369 | -0.252 |
| 0.75                                | -0.339  | -0.400 | -0.278 |
| 0.90                                | -0.372  | -0.446 | -0.299 |
| (MCO, effet sur la moyenne du log : | -0.337) |        |        |

figure enregistrée

Lisons la figure. En **dinars** (à gauche), l'effet d'Instagram n'est pas le même sur les petits et les grands paniers : le déficit est de 13,5 DT pour les petits paniers (quantile 10 %) et de 33,9 DT pour les gros (quantile 90 %), à comparer aux 20,8 DT de l'effet moyen des moindres carrés. La

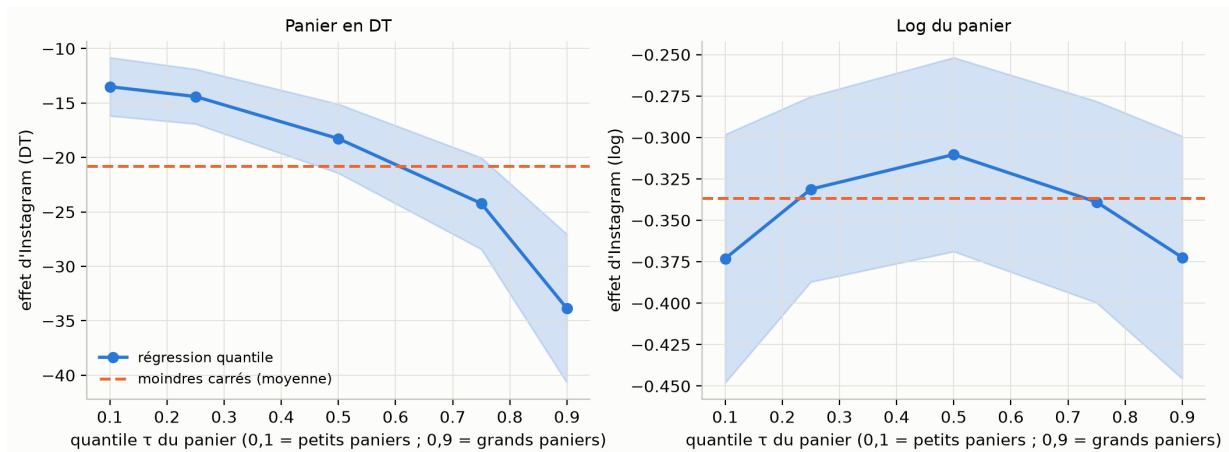


Figure 14: Effet du canal Instagram selon le quantile du panier. À gauche (en dinars) : l'effet est faible pour les petits paniers et de plus en plus négatif pour les grands. À droite (en log) : l'effet est à peu près constant, autour de  $-0,34$ , avec une bande de confiance qui contient la valeur des moindres carrés. Les bandes bleues sont les intervalles de confiance à 95 %.

moyenne (moindres carrés) ne voit qu'un effet moyen. En **log** (à droite), l'effet varie peu le long de la distribution (de  $-0,31$  à  $-0,37$ , des intervalles de confiance qui se chevauchent largement et contiennent la valeur des moindres carrés,  $-0,337$ ) : on peut le considérer comme **constant**. Les deux lectures sont **cohérentes** : un effet **multiplicatif** constant ( $-29$  % du panier, quel que soit son niveau) se traduit en dinars par un effet **proportionnel** au niveau (un gros panier perd plus de dinars qu'un petit). C'est exactement ce que le modèle simulé a programmé, et c'est une raison de plus de préférer le log : l'effet s'y résume par un **seul nombre**.

#### À retenir (1.6).

- Les moindres carrés ont un **point de rupture de 0 %** : une seule aberration peut les fausser, car leur fonction d'influence  $\psi(e) = e$  n'est pas bornée.
- Les **M-estimateurs** (Huber, Tukey) minimisent  $\sum \rho(e_i/s)$  avec un  $\rho$  à influence bornée ; on les calcule par **moindres carrés repondérés** (IRLS), avec une échelle robuste (MAD). Huber avec  $c = 1,345$  garde 95 % d'efficacité si les erreurs sont normales.
- Sur nos données contaminées, Huber **ignore presque** les paniers corrompus ; les moindres carrés sont biaisés et croient à un bruit énorme.
- Les M-estimateurs ne protègent **pas** contre les points à **fort levier** : il faut vérifier les données d'abord.
- La **régression quantile** (médiane ou autres quantiles) est robuste elle aussi, et décrit l'effet d'une variable sur **toute la distribution**.

## 1.7 ⊕ Pour aller plus loin : les modèles à effets mixtes et hiérarchiques

**Section optionnelle.** Elle s'adresse à qui travaille sur des données **groupées** (clients dans des villes, élèves dans des classes, mesures répétées sur les mêmes personnes). Elle prépare les modèles hiérarchiques bayésiens du chapitre 6.

**Intuition.** La régression ordinaire suppose que les observations sont **indépendantes** (hypothèse H4 du 1.1.2). Mais les 20 commandes retirées dans le **même point relais** se ressemblent : elles partagent la même équipe, le même emplacement, la même ambiance. Les

traiter comme 20 observations indépendantes revient à compter vingt fois une information qui n'existe qu'une fois : on se croit plus sûr de soi qu'on ne l'est. Le **modèle mixte** reconnaît explicitement que chaque groupe a sa propre « personnalité » (un niveau de base propre) et la modélise comme un **effet aléatoire**, tiré d'une loi commune. Chaque groupe est ainsi à la fois **différent** des autres et **lié** à eux.

### 1.7.1 Un jeu de données groupé : les points relais

Dar Jasmin livre en partie via **30 points relais** (commerces partenaires où les clients viennent retirer leur colis). Pour chaque commande retirée, on dispose du **délai de livraison** (de 1 à 10 jours) et de la **note de satisfaction** (sur 20). Les relais sont de tailles très inégales (certains ont reçu 4 commandes, d'autres près de 60), et certains sont en zone **urbaine**. Les données sont simulées (graine fixe) selon un modèle que nous connaissons, comme d'habitude :

$$\text{note}_{ij} = 12 + 1,2 \text{urbain}_j + u_{0j} + (-0,7 + u_{1j})(\text{délai}_{ij} - 5) + \varepsilon_{ij},$$

où  $i$  numérote les commandes du relais  $j$ ,  $u_{0j} \sim \mathcal{N}(0, 1,6^2)$  est le **niveau de base propre au relais** (« certains relais notent plus généreusement »),  $u_{1j} \sim \mathcal{N}(0, 0,3^2)$  la **sensibilité propre au retard** de ce relais, et  $\varepsilon_{ij} \sim \mathcal{N}(0, 1,8^2)$  le bruit.

```
import warnings
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ENCRE, GRIS, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7",
↳ "#0b0b0b", "#898781", "#e34948"
STYLE = {"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color":
↳ "#e1e0d9",
        "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb", "axes.facecolor": "#fcfcfb",
        "savefig.facecolor": "#fcfcfb", "font.size": 10, "axes.edgecolor": "#c3c2b7"}
plt.rcParams.update(STYLE)

# Simulation des 30 points relais (graine fixe)
rng = np.random.default_rng(41)
J = 30
tailles = np.clip(np.round(rng.lognormal(np.log(12), 0.8, J)), 3, 60).astype(int) # commandes par
↳ relais (très inégal)
urbain = (rng.random(J) < 0.5).astype(int)
u0 = rng.normal(0, 1.6, J) # vrai niveau de
↳ base de chaque relais
u1 = rng.normal(0, 0.3, J) # vraie sensibilité
↳ au retard de chaque relais
lignes = []
for j in range(J):
    delai = rng.integers(1, 11, tailles[j])
    note = 12 + 1.2 * urbain[j] + u0[j] + (-0.7 + u1[j]) * (delai - 5) + rng.normal(0, 1.8, tailles[j])
    for d, y in zip(delai, note):
        lignes.append((f"R{j + 1:02d}", urbain[j], int(d), round(float(y), 2)))
rel = pd.DataFrame(lignes, columns=["relais", "urbain", "delai", "note"])
rel.to_csv("donnees/ch01-relais.csv", index=False) # fichier fourni avec le livre (lu aussi
↳ par R plus bas)
rel["dc"] = rel["delai"] - 5 # délai centré : 0 = délai de 5 jours
print(rel.shape, "| relais :", rel["relais"].nunique(), "| relais urbains :", int(urbain.sum()))
```



```
print("commandes par relais : min =", tailles.min(), "| médiane =", int(np.median(tailles)), "| max
↳ =", tailles.max())
print("écart-type RÉALISÉ des 30 vrais niveaux de base u0 :", round(u0.std(ddof=1), 2), "(la loi dont
↳ ils sont tirés a un écart-type de 1,6)")
print(rel.head(5).to_string(index=False))
```

```
(484, 5) | relais : 30 | relais urbains : 14
commandes par relais : min = 4 | médiane = 11 | max = 58
écart-type RÉALISÉ des 30 vrais niveaux de base u0 : 1.31 (la loi dont ils sont tirés a un écart-type
↳ de 1,6)
relais  urbain  delai  note  dc
R01      0      6  12.57  1
R01      0      4  11.23 -1
R01      0      8   4.98  3
R01      0     10   7.56  5
R02      1      6   9.91  1
```

Notons déjà un fait important. Les 30 niveaux de base  $u_{0j}$  ont été **tirés** d'une loi d'écart-type 1,6, mais avec seulement 30 relais, leur écart-type **réalisé** n'est que de 1,31. Avec peu de groupes, les composantes de variance sont **mal connues** : nous y reviendrons.

### 1.7.2 Trois façons de traiter les groupes

Pour estimer le niveau de base de chaque relais, trois stratégies s'offrent à nous :

1. **Mise en commun totale** (*complete pooling*) : on ignore les groupes (régression ordinaire sur les 484 commandes). On suppose que tous les relais sont identiques.
2. **Pas de mise en commun** (*no pooling*) : on estime chaque relais **séparément** (une indicatrice par relais). On suppose qu'ils n'ont rien en commun.
3. **Mise en commun partielle** (*partial pooling*) : le **modèle mixte**. Chaque relais a son propre niveau, mais ces niveaux sont supposés tirés d'une même loi : l'information d'un relais **aide** à estimer les autres.

Commençons par les deux premières, et regardons les conséquences sur les erreurs standard :

```
m_commun = smf.ols("note ~ dc + urbain", data=rel).fit() # ignore les groupes
m_separe = smf.ols("note ~ dc + C(relais)", data=rel).fit() # une indicatrice
↳ par relais (urbain y est absorbé)
print("Mise en commun totale (MCO, groupes ignorés) :")
print(pd.DataFrame({"coef": m_commun.params, "erreur standard": m_commun.bse, "IC bas":
↳ m_commun.conf_int()[0], "IC haut": m_commun.conf_int()[1]}).round(3).to_string())
print(f"\nrésidu : s = {np.sqrt(m_commun.scale):.2f}")
print("\nPas de mise en commun : l'effet « urbain » n'est plus estimable (il est confondu avec les
↳ indicatrices de relais).")
print(f"coefficient du délai avec indicatrices de relais : {m_separe.params['dc']:.3f} (erreur
↳ standard {m_separe.bse['dc']:.3f})")
```

```
Mise en commun totale (MCO, groupes ignorés) :
      coef  erreur standard  IC bas  IC haut
Intercept  12.644           0.152  12.345  12.943
dc         -0.721           0.039  -0.796  -0.645
urbain      0.482           0.224   0.043   0.922
```

```
résidu : s = 2.44
```

```
Pas de mise en commun : l'effet « urbain » n'est plus estimable (il est confondu avec les indicatrices
↳ de relais).
coefficient du délai avec indicatrices de relais : -0.703 (erreur standard 0.034)
```

**Le problème de la mise en commun totale.** Les p-valeurs et intervalles de ce premier modèle supposent 484 observations **indépendantes**. Or elles sont groupées en 30 relais. Pour une variable qui ne varie **qu'entre** relais (comme urbain), l'information ne vient que de 30 unités, pas de 484 : la mise en commun totale **sous-estime fortement** son incertitude. Nous le démontrons par simulation plus bas (1.7.6).

Le **pas de mise en commun** a l'inconvénient inverse : il ne peut pas estimer l'effet d'une variable de niveau groupe (urbain), et il estime très mal le niveau d'un petit relais (4 commandes) : l'estimation est trop bruitée.

### 1.7.3 Le modèle à intercept aléatoire

Le **modèle mixte à intercept aléatoire** s'écrit

$$y_{ij} = \mathbf{x}_{ij}^T \boldsymbol{\beta} + u_j + \varepsilon_{ij}, \quad u_j \sim \mathcal{N}(0, \tau^2), \quad \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2),$$

les  $u_j$  et les  $\varepsilon_{ij}$  étant indépendants. Les  $\boldsymbol{\beta}$  sont les **effets fixes** (communs à tous), les  $u_j$  des **effets aléatoires**. Deux paramètres de variance :  $\tau^2$  (variance **entre** relais) et  $\sigma^2$  (variance **à l'intérieur** d'un relais).

**Ce que cela implique pour les observations.** Pour deux commandes  $i \neq i'$  du **même** relais,  $\text{Cov}(y_{ij}, y_{i'j}) = \text{Var}(u_j) = \tau^2$  : elles sont **corrélées**, avec une corrélation

$$\rho = \frac{\tau^2}{\tau^2 + \sigma^2} \quad (\text{coefficient de corrélation intraclasse, ICC}).$$

Pour deux commandes de relais différents, la covariance est nulle. Le vecteur des commandes d'un relais de  $n_j$  commandes a donc pour matrice de variance  $\mathbf{V}_j = \sigma^2 \mathbf{I} + \tau^2 \mathbf{1}\mathbf{1}^T$  (« symétrie composée »). Par conséquent, la **variance de la moyenne** de  $n_j$  commandes est  $\tau^2 + \sigma^2/n_j$ , et non  $\sigma^2/n_j$  : il y a un *plancher*,  $\tau^2$ , que l'on ne peut jamais dépasser en ajoutant des commandes dans un même relais. L'**effectif effectif** d'un échantillon de  $m$  commandes par groupe est  $m/(1 + (m-1)\rho)$  (« effet de plan »).

**L'ICC répond à la question « les groupes comptent-ils ? ».** Comparons deux situations. D'abord un cas où la réponse est **non** : les clients de Dar Jasmin sont répartis en six **villes**. Le panier dépend-il de la ville, au-delà de l'âge et du canal ?

```
clients = pd.read_csv("donnees/clients.csv")
cl = clients[clients["nb_commandes_an"] > 0].copy().reset_index(drop=True)
cl["canal"] = pd.Categorical(cl["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
cl["a"] = cl["age"] - 36
cl["log_panier"] = np.log(cl["panier_moyen"])
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    m_ville = smf.mixedlm("log_panier ~ a + C(canal)", cl, groups=cl["ville"]).fit(reml=True)
tau2_v, sigma2_v = m_ville.cov_re.iloc[0, 0], m_ville.scale
print(f"clients groupés par ville : variance entre villes tau² = {tau2_v:.5f} | variance résiduelle
↳ sigma² = {sigma2_v:.4f}")
print(f"ICC = tau²/(tau² + sigma²) = {tau2_v / (tau2_v + sigma2_v):.4f}")
```

```
clients groupés par ville : variance entre villes tau² = 0.00000 | variance résiduelle sigma² = 0.1373
ICC = tau²/(tau² + sigma²) = 0.0000
```

L'ICC est estimé à **0** (la variance entre villes est estimée au bord de l'espace des paramètres : exactement 0) : une fois l'âge et le canal pris en compte, la ville n'explique rien du panier. Le



modèle mixte aurait ici été superflu, et nos analyses des sections précédentes (sans effet de ville) étaient légitimes. Retenons que **regrouper n'implique pas toujours une dépendance** : le modèle mixte est un outil à utiliser quand l'ICC est notable.

Pour les points relais, la situation est tout autre :

```
m_ri = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"]).fit(reml=True)
tau2, sigma2 = m_ri.cov_re.iloc[0, 0], m_ri.scale
print(m_ri.summary().tables[1])
print(f"\nvariance entre relais tau² = {tau2:.3f} (écart-type {np.sqrt(tau2):.2f}) | variance
↪ résiduelle sigma² = {sigma2:.3f} (écart-type {np.sqrt(sigma2):.2f})")
print(f"ICC = {tau2 / (tau2 + sigma2):.3f} : environ {100 * tau2 / (tau2 + sigma2):.0f} % de la
↪ variance des notes (après effet du délai et du type de relais) est due au relais")
```

|           | Coef.  | Std.Err. | z       | P> z  | [0.025 | 0.975] |
|-----------|--------|----------|---------|-------|--------|--------|
| Intercept | 12.489 | 0.364    | 34.284  | 0.000 | 11.775 | 13.202 |
| dc        | -0.710 | 0.034    | -21.047 | 0.000 | -0.776 | -0.644 |
| urbain    | 0.377  | 0.534    | 0.705   | 0.481 | -0.671 | 1.424  |
| Group Var | 1.711  | 0.280    |         |       |        |        |

```
variance entre relais tau² = 1.711 (écart-type 1.31) | variance résiduelle sigma² = 4.350 (écart-type
↪ 2.09)
ICC = 0.282 : environ 28 % de la variance des notes (après effet du délai et du type de relais) est
↪ due au relais
```

Plus du quart (28 %) de la variance résiduelle des notes est **entre relais** : deux commandes du même relais sont corrélées (corrélation  $\approx 0,28$ ). Avec 16 commandes par relais en moyenne, l'**effet de plan** est considérable :  $1 + (m-1)\rho \approx 1 + 15 \times 0,28 \approx 5,2$  : pour une variable qui ne varie qu'entre relais, les 484 commandes **valent environ  $484/5 \approx 93$  observations indépendantes**. Remarquez le résultat sur urbain : sa erreur standard est de 0,534 avec le modèle mixte, contre 0,224 pour la régression ordinaire du 1.7.2, soit **2,4 fois plus**. L'intervalle de confiance ordinaire (de 0,04 à 0,92) **exclut** zéro et laisse croire à un effet « significatif » ; l'intervalle mixte (de -0,67 à 1,42) **ne l'exclut pas**. C'est l'effet de plan en action : les 484 commandes n'apportent, sur le type de relais, que l'information de 30 relais.

**Comment estime-t-on ce modèle ? La méthode REML.** La variance de  $\hat{\beta}$  dépend de  $\mathbf{V} = \text{diag}(\mathbf{V}_j)$ , qui dépend de  $\tau^2$  et  $\sigma^2$ , inconnus. Étant donné  $\mathbf{V}$ , l'estimateur des effets fixes est celui des **moindres carrés généralisés** (qui pondère selon la fiabilité de chaque observation)

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{V}^{-1} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{V}^{-1} \mathbf{y}.$$

Les paramètres de variance sont estimés par maximum de vraisemblance, mais la version « classique » **sous-estime** les variances (comme la division par  $n$  au lieu de  $n-p$ , 1.1.5) : on lui préfère la **vraisemblance restreinte (REML)**, qui maximise la vraisemblance des **résidus** (les combinaisons des données indépendantes des effets fixes) et corrige ce biais. statsmodels (reml=True, par défaut) et R (lme4) utilisent la REML.

#### 1.7.4 La mise en commun partielle : le rétrécissement

Le grand intérêt du modèle mixte est ce qu'il fait des **niveaux des groupes**. Une fois les effets fixes estimés, on prédit l'effet aléatoire  $u_j$  de chaque relais par le **meilleur prédicteur linéaire sans biais (BLUP)**.

**Le BLUP est une moyenne rétrécie.** Soit  $\bar{r}_j$  la moyenne, pour le relais  $j$ , des résidus « fixes »  $r_{ij} = y_{ij} - \mathbf{x}_{ij}^\top \hat{\beta}$  (c'est l'estimation « sans mise en commun » du niveau de base du relais). Le

BLUP est

$$\hat{u}_j = \underbrace{\frac{\tau^2}{\tau^2 + \sigma^2/n_j}}_{B_j \in (0,1)} \bar{r}_j.$$

*Démonstration.* Pour un groupe de  $n_j$  observations,  $\mathbf{V}_j = \sigma^2 \mathbf{I} + \tau^2 \mathbf{1}\mathbf{1}^\top$  vérifie  $\mathbf{V}_j \mathbf{1} = (\sigma^2 + n_j \tau^2) \mathbf{1}$ , donc  $\mathbf{V}_j^{-1} \mathbf{1} = \mathbf{1}/(\sigma^2 + n_j \tau^2)$ . Le BLUP est  $\hat{u}_j = \tau^2 \mathbf{1}^\top \mathbf{V}_j^{-1} \mathbf{r}_j = \tau^2 \frac{n_j \bar{r}_j}{\sigma^2 + n_j \tau^2} = \frac{\tau^2}{\tau^2 + \sigma^2/n_j} \bar{r}_j$ .  $\square$

Le facteur  $B_j$  est la **fiabilité** de la moyenne du groupe. Pour un **grand** groupe ( $n_j \rightarrow \infty$ ),  $B_j \rightarrow 1$  : on fait confiance à sa moyenne. Pour un **petit** groupe,  $B_j$  est petit : sa moyenne est bruitée, on la **rapproche de la moyenne générale** (0). C'est un compromis entre « ce relais est unique » ( $B_j = 1$ , pas de mise en commun) et « tous les relais sont pareils » ( $B_j = 0$ , mise en commun totale).

Vérifions la formule, puis comparons les deux estimations à la **vérité**, que nous connaissons ici (les vrais  $u_{0j}$ ) :

```
beta_ri = m_ri.fe_params
rel["r"] = rel["note"] - (beta_ri["Intercept"] + beta_ri["dc"] * rel["dc"] + beta_ri["urbain"] *
    rel["urbain"]) # résidus « fixes »
groupes = rel.groupby("relais")["r"].agg(["mean", "size"]).rename(columns={"mean": "r_bar", "size":
    "n_j"})
groupes["B"] = tau2 / (tau2 + sigma2 / groupes["n_j"])
groupes["u_formule"] = groupes["B"] * groupes["r_bar"]
groupes["u_statsmodels"] = [m_ri.random_effects[g]["Group"] for g in groupes.index]
groupes["u_vrai"] = u0
print("formule du BLUP = statsmodels :", np.allclose(groupes["u_formule"], groupes["u_statsmodels"]))
print(groupes.sort_values("n_j").iloc[[0, 1, 2, 14, 27, 28, 29]].round(3).to_string())
erreur_sans = np.sqrt(np.mean((groupes["r_bar"] - groupes["u_vrai"])**2))
erreur_part = np.sqrt(np.mean((groupes["u_statsmodels"] - groupes["u_vrai"])**2))
print(f"l'erreur quadratique moyenne par rapport aux vrais niveaux u0 : sans mise en commun =
    {erreur_sans:.3f} | mise en commun partielle = {erreur_part:.3f}")
petits = groupes["n_j"] <= 8
print(f" pour les {petits.sum()} petits relais (8 commandes ou moins) : sans =
    {np.sqrt(np.mean((groupes.loc[petits, 'r_bar'] - groupes.loc[petits, 'u_vrai']**2)):.3f} |
    partielle = {np.sqrt(np.mean((groupes.loc[petits, 'u_statsmodels'] - groupes.loc[petits,
    'u_vrai']**2)):.3f}")
```

|        | r_bar  | n_j | B     | u_formule | u_statsmodels | u_vrai |
|--------|--------|-----|-------|-----------|---------------|--------|
| relais |        |     |       |           |               |        |
| R01    | -1.983 | 4   | 0.611 | -1.213    | -1.213        | -0.937 |
| R05    | 0.997  | 4   | 0.611 | 0.610     | 0.610         | -0.683 |
| R10    | -2.442 | 4   | 0.611 | -1.493    | -1.493        | 0.413  |
| R19    | -0.426 | 11  | 0.812 | -0.346    | -0.346        | 0.624  |
| R12    | -0.644 | 31  | 0.924 | -0.595    | -0.595        | -0.548 |
| R16    | 1.386  | 56  | 0.957 | 1.326     | 1.326         | 1.867  |
| R29    | 1.257  | 58  | 0.958 | 1.204     | 1.204         | 0.725  |

```
erreur quadratique moyenne par rapport aux vrais niveaux u0 : sans mise en commun = 0.900 | mise en
    commun partielle = 0.765
pour les 10 petits relais (8 commandes ou moins) : sans = 1.273 | partielle = 1.031
```

La formule reproduit exactement les prédictions de statsmodels. Surtout, l'erreur par rapport aux **vrais** niveaux est plus faible avec la mise en commun partielle, et le gain est **le plus net pour les petits relais**, dont l'estimation individuelle était la plus bruitée. Voyons le mécanisme sur un graphique : pour chaque relais (classés par taille), on relie son estimation « sans mise en commun

» à son estimation rétrécie.

```
ordre = groupes.sort_values("n_j").reset_index()
fig, ax = plt.subplots(figsize=(8.8, 4.8))
x = np.arange(len(ordre))
for i, ligne in ordre.iterrows():
    ax.plot([i, i], [ligne["r_bar"], ligne["u_statsmodels"]], color=GRIS, lw=1.2, zorder=1)
ax.scatter(x, ordre["r_bar"], facecolors="none", edgecolors=ORANGE, s=45, lw=1.6, zorder=3,
    label="sans mise en commun (moyenne du relais)")
ax.scatter(x, ordre["u_statsmodels"], color=BLEU, s=32, zorder=4, label="mise en commun partielle
    (BLUP)")
ax.scatter(x, ordre["u_vrai"], marker="_", color=ENCRE, s=130, lw=2, zorder=5, label="vérité (connue
    car simulée)")
ax.axhline(0, color=GRIS, lw=1)
ax.set_xticks(x); ax.set_xticklabels(ordre["n_j"], fontsize=7.5)
ax.set_xlabel("nombre de commandes du relais (relais classés du plus petit au plus grand)")
ax.set_ylabel("niveau de base du relais (écart à la moyenne, en points)")
ax.set_ylim(-3.3, 3.8) # marge en haut pour la légende
ax.legend(frameon=False, loc="upper left", fontsize=8.5)
plt.savefig("figures/ch01-retrecissement.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

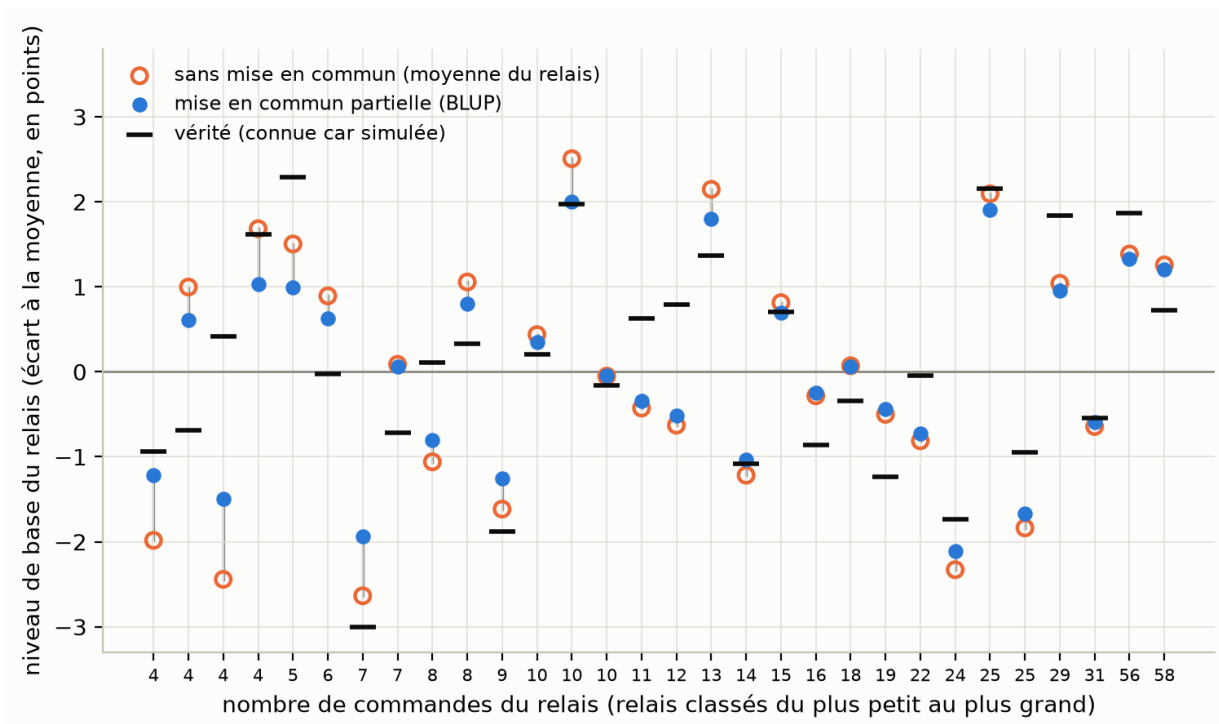


Figure 15: Niveau de base de chacun des 30 relais, classés par nombre de commandes. Les cercles orange sont les moyennes de chaque relais, les points bleus leurs versions rétrécies par le modèle mixte, les traits noirs la vérité. Les segments gris relient les deux estimations. Le rétrécissement est fort pour les petits relais (à gauche) et faible pour les grands (à droite).

Le dessin raconte l'idée centrale. À gauche, les **petits** relais (4 à 8 commandes) : leur moyenne brute (cercle orange) est très variable, parfois extrême ; le modèle la **ramène nettement vers 0** (point bleu), ce qui la rapproche souvent de la vérité (trait noir). À droite, les **grands** relais : l'estimation brute est déjà fiable et le rétrécissement est faible. Ce principe, « **emprunter de la force aux autres groupes** », est l'un des plus puissants de la statistique moderne : on le retrouvera dans le cadre bayésien (chapitre 6, avec la loi *a priori*  $u_j \sim \mathcal{N}(0, \tau^2)$ ).

### 1.7.5 Pente aléatoire : chaque relais a sa propre sensibilité au retard

Jusqu'ici, tous les relais partagent la même pente ( $-0,7$  point par jour de retard, en moyenne). Mais les vraies données ont été produites avec une **pente propre à chaque relais** ( $u_{1j}$ ). On l'inclut par une **pente aléatoire** :  $y_{ij} = \beta_0 + \beta_1 x_{ij} + \dots + u_{0j} + u_{1j} x_{ij} + \varepsilon_{ij}$ , où le couple  $(u_{0j}, u_{1j})$  est tiré d'une loi normale bidimensionnelle de matrice de covariance  $G$ .

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    m_rs = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"],
        ↪ re_formula="~dc").fit(reml=True, method="lbfgs")
print("Converge :", m_rs.converged)
print(m_rs.summary().tables[1])
G = m_rs.cov_re
print(f"écart-type des niveaux de base : {np.sqrt(G.iloc[0, 0]):.3f} (vrai : 1,6) | écart-type des
    ↪ pentes : {np.sqrt(G.iloc[1, 1]):.3f} (vrai : 0,3)")
print(f"corrélacion niveau de base / pente : {G.iloc[0, 1] / np.sqrt(G.iloc[0, 0] * G.iloc[1, 1]):.2f}
    ↪ (vraie : 0) | écart-type résiduel : {np.sqrt(m_rs.scale):.3f} (vrai : 1,8)")
```

Converge : True

|                | Coef.  | Std.Err. | z       | P> z  | [0.025 | 0.975] |
|----------------|--------|----------|---------|-------|--------|--------|
| Intercept      | 12.596 | 0.339    | 37.166  | 0.000 | 11.932 | 13.260 |
| dc             | -0.760 | 0.064    | -11.806 | 0.000 | -0.886 | -0.634 |
| urbain         | 0.277  | 0.501    | 0.553   | 0.581 | -0.705 | 1.259  |
| Group Var      | 1.427  | 0.252    |         |       |        |        |
| Group x dc Cov | 0.072  | 0.044    |         |       |        |        |
| dc Var         | 0.079  | 0.017    |         |       |        |        |

écart-type des niveaux de base : 1.195 (vrai : 1,6) | écart-type des pentes : 0.281 (vrai : 0,3)  
 corrélacion niveau de base / pente : 0.21 (vraie : 0) | écart-type résiduel : 1.930 (vrai : 1,8)

Les composantes de variance sont raisonnablement proches des vraies valeurs, compte tenu du petit nombre de relais : l'écart-type des niveaux de base est estimé à 1,19 (vrai : 1,6, mais **réalisé** : 1,31 : l'estimation colle à ce qui a effectivement été tiré), celui des pentes à 0,28 (vrai : 0,3), l'écart-type résiduel à 1,93 (vrai : 1,8). La corrélation estimée niveau/pente (0,21), alors que la vraie est nulle, est du bruit d'échantillonnage (son erreur est large avec 30 relais). Les **effets fixes** sont bien retrouvés pour le délai :  $-0,76$  point par jour (vrai :  $-0,7$ , intervalle  $[-0,89 ; -0,63]$ ) ; en revanche l'effet « urbain » est estimé à 0,28 avec une erreur standard de 0,50 : **positif mais très imprécis** (la vraie valeur, 1,2, est à moins de deux erreurs standard). Avec 30 relais, on ne peut tout simplement pas mesurer un effet de niveau groupe de cette taille avec précision.

**Comparer les deux modèles par un test du rapport de vraisemblance.** Les modèles à intercept aléatoire seul et à intercept + pente aléatoire sont emboîtés. On compare leurs vraisemblances. Comme les deux modèles ont les **mêmes effets fixes**, la REML conviendrait aussi ; l'usage prudent, que nous suivons, est de comparer les modèles ajustés par **maximum de vraisemblance** (ML), car la vraisemblance REML ne se compare pas entre modèles d'effets fixes différents.

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    ml_ri = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"]).fit(reml=False)      #
    ↪ optimiseur par défaut (voir la remarque ci-dessous)
    ml_rs = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"],
    ↪ re_formula="~dc").fit(reml=False, method="lbfgs")
LR = 2 * (ml_rs.llf - ml_ri.llf)
# Sous H0, la variance de la pente est sur le bord de l'espace des paramètres (>= 0) : loi limite =
    ↪ mélange 50/50 de chi²(1) et chi²(2)
p_val = 0.5 * stats.chi2.sf(LR, 1) + 0.5 * stats.chi2.sf(LR, 2)
print(f"log-vraisemblance (ML) : intercept aléatoire = {ml_ri.llf:.2f} | intercept + pente aléatoires
    ↪ = {ml_rs.llf:.2f}")
print(f"LR = {LR:.2f} | p-valeur (mélange de chi²) = {p_val:.2e}")
print(f"AIC : {-2 * ml_ri.llf + 2 * 5:.1f} (intercept aléatoire, 5 paramètres) contre {-2 * ml_rs.llf
    ↪ + 2 * 7:.1f} (intercept + pente, 7 paramètres)")
```

```
log-vraisemblance (ML) : intercept aléatoire = -1067.99 | intercept + pente aléatoires = -1047.41
LR = 41.15 | p-valeur (mélange de  $\chi^2$ ) = 6.50e-10
AIC : 2146.0 (intercept aléatoire, 5 paramètres) contre 2108.8 (intercept + pente, 7 paramètres)
```

**Un piège de l'optimisation.** En préparant ce bloc, la même ligne ajustée avec `method="lbfgs"` (l'optimiseur que nous avons utilisé pour la pente aléatoire) a renvoyé une log-vraisemblance **infinie** pour le modèle à intercept aléatoire, sans la moindre erreur : l'optimiseur avait convergé vers une solution dégénérée (variance entre groupes égale à 0). Avec l'optimiseur par défaut, ou avec `bfgs`, on obtient  $-1067,99$ , la valeur que donne aussi `lme4`. **Un optimiseur peut échouer en silence.** Vérifiez toujours qu'un ajustement est plausible (log-vraisemblance finie, variances raisonnables) et, dans le doute, comparez deux optimiseurs ou le résultat de `lme4`.

**Un piège du test.** Tester qu'une **variance** est nulle ( $H_0 : \tau_1^2 = 0$ ) place l'hypothèse nulle **au bord** de l'espace des paramètres (une variance est  $\geq 0$ ) : la loi du rapport de vraisemblance n'est pas un simple  $\chi^2$  mais un **mélange** de  $\chi^2$  (ici 50/50 entre  $\chi_1^2$  et  $\chi_2^2$ ). Utiliser un  $\chi_2^2$  naïf est **conservateur** (p-valeur trop grande).

Visualisons maintenant le résultat : les droites de **chaque relais** (prédites avec leurs effets aléatoires), autour de la droite moyenne :

```
fig, ax = plt.subplots(figsize=(8.0, 4.8))
dd = np.arange(-4, 5.01, 1.0)
for j, g in enumerate(sorted(rel["relais"].unique())):
    re = m_rs.random_effects[g]
    u_urb = urbain[j]
    y_g = m_rs.fe_params["Intercept"] + m_rs.fe_params["urbain"] * u_urb + re["Group"] +
    ↪ (m_rs.fe_params["dc"] + re["dc"]) * dd
    ax.plot(dd + 5, y_g, color=ORANGE if u_urb else AQUA, lw=0.9, alpha=0.55)
ax.plot(dd + 5, m_rs.fe_params["Intercept"] + m_rs.fe_params["dc"] * dd, color=ENCRE, lw=3,
    ↪ label="effet moyen (effets fixes), relais non urbain")
ax.scatter(rel["delai"], rel["note"], s=6, color=GRIS, alpha=0.35)
ax.plot([], [], color=ORANGE, lw=1.4, label="relais urbains"); ax.plot([], [], color=AQUA, lw=1.4,
    ↪ label="relais non urbains")
ax.set_xlabel("délai de livraison (jours)"); ax.set_ylabel("note de satisfaction (sur 20)")
ax.legend(frameon=False, fontsize=8.5, loc="lower left")
ax.set_xlim(0.7, 10.3)
plt.savefig("figures/ch01-droites-par-relais.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**Les mêmes résultats en R avec `lme4`.** Le paquet `lme4` est la référence pour ces modèles. Les nombres doivent coïncider avec ceux de `statsmodels` (le fichier `donnees/ch01-relais.csv` vient d'être écrit par notre bloc Python) :

```
suppressMessages(library(lme4))
d <- read.csv("donnees/ch01-relais.csv")
d$dc <- d$delai - 5
m <- lmer(note ~ dc + urbain + (1 + dc | relais), data = d)
print(round(fixef(m), 4))
print(VarCorr(m), digits = 4)
cat("log-vraisemblance REML :", round(as.numeric(logLik(m)), 3), "\n")
```

|             |         |               |
|-------------|---------|---------------|
| (Intercept) | dc      | urbain        |
| 12.5960     | -0.7598 | 0.2769        |
| Groups      | Name    | Std.Dev. Corr |

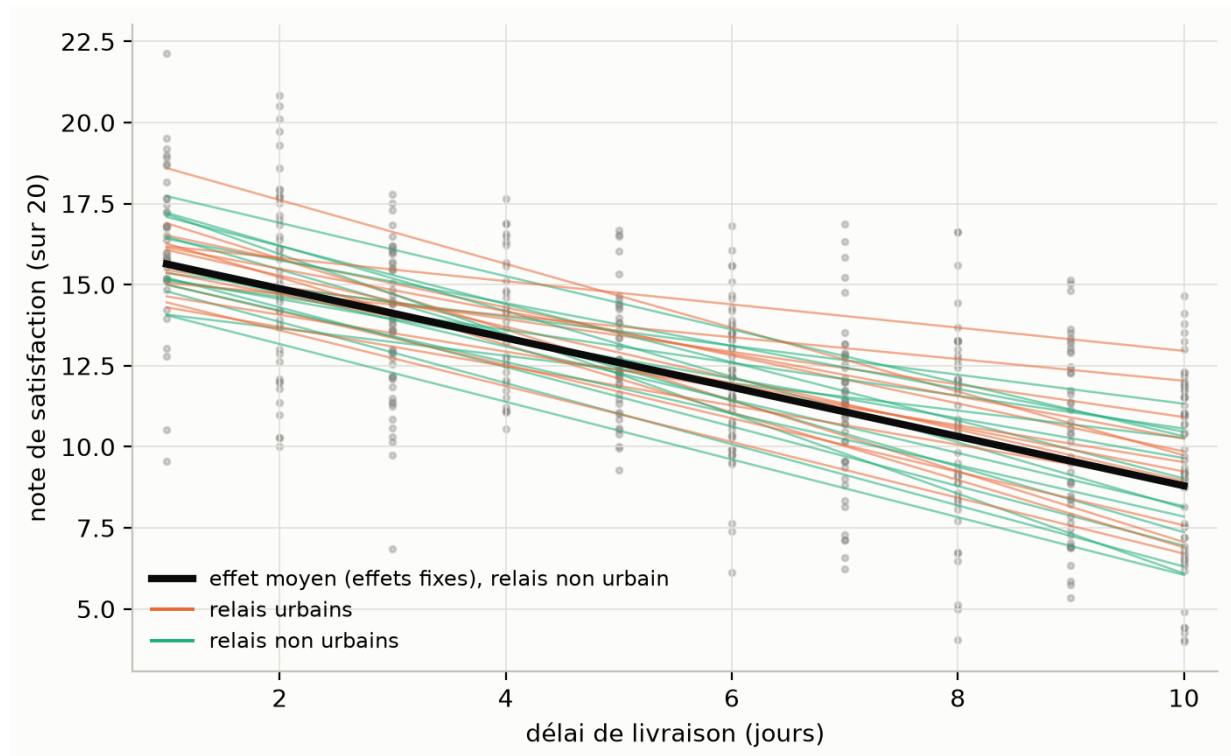


Figure 16: Droites de régression propres à chaque relais (orange : urbains, vert : non urbains) autour de l'effet moyen (noir). Les relais diffèrent par leur niveau de base (décalage vertical) et par leur sensibilité au retard (pente).

```
relais (Intercept) 1.1948
          dc        0.2812  0.21
Residual          1.9299
log-vraisemblance REML : -1049.555
```

### 1.7.6 Pourquoi ignorer les groupes est dangereux : une simulation

Terminons par la démonstration promise : l'effet d'ignorer la structure de groupes sur la fiabilité des tests. Prenons les **mêmes** tailles de relais et la même variance entre relais, mais supposons que le type de relais (urbain ou non) n'a **aucun** effet réel. Un bon test doit alors rejeter l'hypothèse « aucun effet » dans 5 % des cas. Combien rejettent-ils réellement ? On répète 300 fois.

```
rng2 = np.random.default_rng(2024)
R = 300
rej_mco, rej_mixte, z_mixte, tau2_est = 0, 0, [], []
degenere_lbfgs, degenere_default = 0, 0
groupe = np.repeat(np.arange(J), tailles)
urb_obs = urbain[groupe]
dc_obs = rel["dc"].to_numpy()
for r in range(R):
    u = rng2.normal(0, 1.6, J)
    y = 12 + u[groupe] - 0.7 * dc_obs + rng2.normal(0, 1.8, len(groupe)) # AUCUN
    d_sim = pd.DataFrame({"note": y, "dc": dc_obs, "urbain": urb_obs, "g": groupe})
    p_mco = sm.OLS(y, sm.add_constant(d_sim[["dc", "urbain"]])).fit().pvalues["urbain"]
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        f = smf.mixedlm("note ~ dc + urbain", d_sim, groups=d_sim["g"]).fit(reml=True) #
    optimiseur par défaut
```



```

if r < 60:
    ↪ même ajustement avec lbfgs, sur 60 répétitions
    f2 = smf.mixedlm("note ~ dc + urbain", d_sim, groups=d_sim["g"]).fit(reml=True,
    ↪ method="lbfgs")
    degenerer_lbfgs += f2.cov_re.iloc[0, 0] < 1e-6
    degenerer_defaut += f.cov_re.iloc[0, 0] < 1e-6
    rej_mco += p_mco < 0.05
    rej_mixte += f.pvalues["urbain"] < 0.05
    z_mixte.append(f.params["urbain"] / f.bse["urbain"])
    tau2_est.append(f.cov_re.iloc[0, 0])
print(f"quand AUCUN effet n'existe, part de tests « significatifs à 5 % » sur la variable urbain, sur
    ↪ {R} simulations :")
print(f" régression ordinaire (groupes ignorés) : {rej_mco / R:.1%}")
print(f" modèle mixte (intercept aléatoire) : {rej_mixte / R:.1%} | écart-type de la
    ↪ statistique z : {np.std(z_mixte):.2f} (théorie : 1)")
print(f"variance entre relais estimée en moyenne : {np.mean(tau2_est):.2f} (vraie valeur :
    ↪ {1.6**2:.2f})")
print(f"sur 60 répétitions, ajustements qui dégénèrent (variance estimée = 0) : optimiseur par défaut
    ↪ = {degenerer_defaut}/60 | lbfgs = {degenerer_lbfgs}/60")

```

```

quand AUCUN effet n'existe, part de tests « significatifs à 5 % » sur la variable urbain, sur 300
    ↪ simulations :
    régression ordinaire (groupes ignorés) : 55.3%
    modèle mixte (intercept aléatoire) : 6.7% | écart-type de la statistique z : 1.04 (théorie : 1)
variance entre relais estimée en moyenne : 2.54 (vraie valeur : 2.56)
sur 60 répétitions, ajustements qui dégénèrent (variance estimée = 0) : optimiseur par défaut = 0/60 |
    ↪ lbfgs = 43/60

```

La régression ordinaire, qui ignore le groupage, déclare un effet « significatif » dans **plus de la moitié** des simulations alors qu'il n'y a **aucun effet** : plus de dix fois le taux nominal de 5 %. C'est la conséquence directe de l'effet de plan : des **faux positifs en rafale**. Le modèle mixte, lui, se tient près du niveau nominal (un peu au-dessus de 5 %, ce qui est attendu : l'approximation de Wald est légèrement optimiste avec seulement 30 groupes) ; sa statistique z a un écart-type voisin de 1, et sa variance entre relais est estimée sans biais notable. Voilà pourquoi, dès que les données sont groupées, **les erreurs standard ordinaires ne sont pas fiables pour les variables de niveau groupe**.

La dernière ligne de la sortie confirme l'avertissement du 1.7.5 : avec l'optimiseur `lbfgs`, la plupart des ajustements de ce modèle pourtant simple dégénèrent (43 sur 60 : la variance entre relais est estimée à 0, ce qui fausse tout), alors que l'optimiseur par défaut ne dégénère jamais (0 sur 60). Un résultat qui paraît plausible peut être faux : c'est pourquoi nous avons systématiquement comparé aux valeurs de `lme4`.

#### Précautions.

- **Peu de groupes** (moins de 5 à 10) : les variances entre groupes sont très mal estimées, et les approximations de Wald des effets fixes sont trop optimistes. Dans ce cas, envisagez de traiter le groupe comme un effet **fixe** (une indicatrice par groupe) ou une approche bayésienne.
- **Effets fixes ou aléatoires ?** Traiter les groupes comme **aléatoires** est pertinent si l'on veut généraliser à d'autres groupes de la même population (les 30 relais sont un échantillon des relais possibles) et si les groupes sont nombreux. Si les groupes sont **tous** ceux qui existent (les 6 villes) et peu nombreux, des effets fixes suffisent.
- **Convergence** : l'optimisation de ces modèles est parfois délicate (variances proches de 0, corrélations proches de  $\pm 1$ ). Essayez un autre optimiseur (`method="bfgs"`, `"powell"`...), simplifiez la structure aléatoire, ou comparez à `lme4`.
- **Interprétation** : les effets fixes sont des effets **moyens** dans la population de groupes ;

les effets aléatoires sont des **écarts** propres à chaque groupe.

### À retenir (1.7).

- Des observations **groupées** (clients d'une même ville, commandes d'un même relais) ne sont pas indépendantes ; l'ignorer rend les **erreurs standard trop petites**, surtout pour les variables de niveau groupe.
- Le **modèle mixte** ajoute des **effets aléatoires** :  $y_{ij} = \mathbf{x}_{ij}^\top \beta + u_j + \varepsilon_{ij}$ ,  $u_j \sim \mathcal{N}(0, \tau^2)$ . L'ICC =  $\tau^2 / (\tau^2 + \sigma^2)$  mesure la part de variance due aux groupes (elle était nulle pour les villes, notable pour les relais).
- Les effets aléatoires sont prédits par **rétrécissement** :  $\hat{u}_j = \frac{\tau^2}{\tau^2 + \sigma^2/n_j} \bar{r}_j$ . Les petits groupes sont davantage ramenés vers la moyenne : c'est la **mise en commun partielle**, qui bat à la fois « tous pareils » et « tous différents ».
- On peut ajouter des **pentés aléatoires**. Les paramètres se comparent par un test du rapport de vraisemblance (**attention** à la loi limite au bord de l'espace des paramètres) ; l'estimation se fait par **REML**.
- `statsmodels.MixedLM` et `lme4` en R donnent les mêmes résultats.

## 1.8 Exercices corrigés

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 1 à 8 et 11 à 12 se résolvent surtout à la main ; les exercices 9, 10 et 13 demandent le code.

### Énoncés

**Exercice 1 ★ (moindres carrés à la main).** Trois commandes ont  $x$  articles et un montant  $y$  en DT : (1, 2), (2, 3), (4, 7). (a) Écrivez  $\mathbf{X}$  et  $\mathbf{y}$ , calculez  $\mathbf{X}^\top \mathbf{X}$  et  $\mathbf{X}^\top \mathbf{y}$ , puis  $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1)$ . (b) Calculez les valeurs ajustées et les résidus, et vérifiez que  $\mathbf{X}^\top \hat{\varepsilon} = \mathbf{0}$ . (c) Calculez le  $R^2$ . (d) Quel montant prévoit-on pour 3 articles ?

**Exercice 2 ★ (lire un modèle en log).** Le modèle `log_panier ~ a + C(canal)` du 1.1.7 donne : constante 4,2206, Site -0,1571, Instagram -0,3368, âge centré 0,0092 par année. (a) Exprimez **exactement** (en pourcentage) l'effet du Site et d'Instagram par rapport à la boutique. (b) Quel effet pour 10 années d'âge de plus ? (c) Quel est le panier **médian** prédit d'une cliente de 46 ans acquise par le Site ? (d) Pourquoi dit-on « médian » et non « moyen » ?

**Exercice 3 ★★ (régression simple).** Dans la régression simple  $y = \beta_0 + \beta_1 x + \varepsilon$ , démontrez que  $\hat{\beta}_1 = r \frac{s_y}{s_x}$  et que  $R^2 = r^2$ , où  $r$  est le coefficient de corrélation de Pearson entre  $x$  et  $y$ . Vérifiez-le sur les quatre commandes du 1.1.1 ( $x = 1, 2, 3, 4$  ;  $y = 22, 41, 66, 79$ ).

**Exercice 4 ★★ (test et intervalle à la main).** Pour Instagram, `statsmodels` donne le coefficient -0,3368 et son erreur standard 0,0225, avec  $n - p = 1736$  degrés de liberté (quantile de Student à 97,5 % : 1,961). (a) Calculez la statistique  $t$  et dites si l'on rejette  $H_0 : \beta = 0$  à 5 %. (b) Donnez l'intervalle de confiance à 95 % de  $\beta$ , puis de l'effet multiplicatif  $e^\beta$  exprimé en pourcentage. (c) Pour l'âge : coefficient 0,0092, erreur standard 0,00085 : quelle est la statistique  $t$  ?

**Exercice 5 ★★ (test  $F$ ).** Le modèle réduit `log_panier ~ a` a une somme des carrés résiduelle  $\text{SCR}_0 = 269,94$  ; le modèle complet `log_panier ~ a + C(canal)` a  $\text{SCR}_1 = 238,30$ , avec  $n - p_1 = 1736$ . (a) Calculez la statistique  $F$  du test « le canal est inutile ». Combien y a-t-il de contraintes



$q$  ? (b) La valeur critique de  $F_{2,1736}$  à 5 % est environ 3,0 : concluez. (c) Si  $SCR_1$  valait 269,70 au lieu de 238,30 (le canal améliorerait à peine l'ajustement), que vaudrait  $F$  ? Que concluriez-vous ?

**Exercice 6 ★★ (levier).** Quatre clients ont pour âge centré  $x = (1, 2, 3, 10)$ . (a) Calculez le levier  $h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_k (x_k - \bar{x})^2}$  de chacun. Vérifiez que leur somme vaut  $p = 2$ . (b) Quel client a le plus d'influence potentielle ? (c) Si  $y_4$  augmente de 1, de combien augmente la valeur ajustée  $\hat{y}_4$  ?

**Exercice 7 ★★ (PRESS).** Reprenez les trois points de l'exercice 1. (a) Calculez les leviers  $h_{ii}$ . (b) Calculez les erreurs de prédiction « sans le point »  $\hat{\varepsilon}_i / (1 - h_{ii})$  et la somme PRESS. (c) Vérifiez l'une d'elles en réajustant la droite sur les deux autres points. (d) Comparez PRESS à la somme des carrés résiduelle : que constatez-vous ?

**Exercice 8 ★★ (multicolinéarité).** (a) Une variable explicative  $x_j$  est expliquée à 99,5 % par les autres ( $R_j^2 = 0,995$ ) : donnez son VIF et le facteur par lequel son erreur standard est multipliée. (b) Avec deux variables explicatives de corrélation 0,9, que vaut leur VIF ? (c) **Code.** Les huit questions de l'enquête de satisfaction (q1 à q8) sont-elles gravement colinéaires entre elles ? Calculez leurs VIF.

**Exercice 9 ★★ (prédire une moyenne en dinars).** Avec le modèle  $\log\_panier \sim a + C(\text{canal})$  : (a) donnez, pour un client **Instagram de 25 ans**, le panier médian prédit ; (b) calculez une prédiction du panier **moyen** avec la correction de Duan ; (c) comparez à la moyenne observée des clients Instagram âgés de 23 à 27 ans. Laquelle des deux prédictions est la plus proche, et pourquoi ?

**Exercice 10 ★★ (régression sur les ventes mensuelles).** Avec `donnees/ventes_mensuelles.csv` (120 mois), ajustez  $\log(ca) \sim t + C(\text{mois}) + \text{promo} + \text{covid}$ , où  $t$  est le numéro du mois (0 à 119), `mois` le mois civil (1 à 12), `promo` indique un mois de promotion et `covid` les mois de mars à juin 2020. (a) Quelle est la croissance annuelle estimée ? (b) Quel est l'effet estimé de décembre par rapport à janvier (en facteur multiplicatif) ? (c) Quel est l'effet du confinement de 2020 sur les ventes, en pourcentage ? (d) Que disent le  $R^2$  et le test de Durbin-Watson sur ce modèle ? Reste-t-il de la structure dans les résidus ?

**Exercice 11 ★★ (Ridge, cas orthonormal).** Les colonnes de  $\mathbf{X}$  sont orthonormales ( $\mathbf{X}^T \mathbf{X} = \mathbf{I}$ ). (a) Montrez que  $\hat{\beta}_j^{\text{ridge}} = \hat{\beta}_j^{\text{MCO}} / (1 + \lambda)$ . (b) Pour un coefficient vrai  $\beta = 2$  et  $\sigma = 1$ , calculez l'erreur quadratique moyenne  $EQM(\lambda) = \frac{\lambda^2 \beta^2 + \sigma^2}{(1 + \lambda)^2}$  pour  $\lambda = 0, 0,25$  et 1. (c) Quel  $\lambda$  la minimise ? Vérifiez par le code sur une grille.

**Exercice 12 ★★ (modèle mixte, calcul à la main).** Dans un modèle à intercept aléatoire,  $\tau^2 = 2,0$  (variance entre groupes) et  $\sigma^2 = 6,0$  (variance résiduelle). (a) Calculez l'ICC. (b) Un relais a  $n_j = 9$  commandes et une moyenne de résidus « fixes »  $\bar{r}_j = 3,2$  : calculez le facteur de rétrécissement  $B_j$  et le BLUP  $\hat{u}_j$ . (c) Même question pour un relais de  $n_j = 2$  commandes avec la même moyenne de résidus. (d) Avec  $J = 20$  relais de 9 commandes chacun, quel est l'effectif effectif pour estimer une variable qui ne varie qu'entre relais ?

**Exercice 13 ★★ (robustesse, simulation).** Simulez  $n = 100$  points  $y = 2 + 1,5x + \varepsilon$  ( $x$  uniforme sur  $[0, 10]$ ,  $\varepsilon \sim \mathcal{N}(0, 1)$ , graine 5), puis ajoutez 25 à  $y$  pour 5 points tirés au hasard. Comparez les coefficients des moindres carrés, de Huber et de la régression médiane sur les données propres et corrompues. Que constatez-vous ?

## Corrigés

**Corrigé 1.** (a)  $X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \end{pmatrix}$ ,  $y = (2, 3, 7)^\top$ .  $X^\top X = \begin{pmatrix} 3 & 7 \\ 7 & 21 \end{pmatrix}$  (déterminant  $63 - 49 = 14$ ),  $X^\top y = (12, 2 + 6 + 28)^\top = (12, 36)^\top$ . Donc  $\hat{\beta} = \frac{1}{14} \begin{pmatrix} 21 & -7 \\ -7 & 3 \end{pmatrix} \begin{pmatrix} 12 \\ 36 \end{pmatrix} = \frac{1}{14} \begin{pmatrix} 252 - 252 \\ -84 + 108 \end{pmatrix} = \begin{pmatrix} 0 \\ 12/7 \end{pmatrix}$  : la droite passe **exactement par l'origine** :  $\hat{y} = \frac{12}{7}x \approx 1,714x$ . (b) Ajustées :  $\frac{12}{7}$ ,  $\frac{24}{7}$ ,  $\frac{48}{7}$  ; résidus :  $\frac{2}{7}$ ,  $-\frac{3}{7}$ ,  $\frac{1}{7}$  (soit 0,286 ; -0,429 ; 0,143). Somme : 0 ✓.  $\sum x_i \hat{\epsilon}_i = \frac{2}{7} - \frac{6}{7} + \frac{4}{7} = 0$  ✓. (c)  $SCR = \frac{4+9+1}{49} = \frac{2}{7}$  ;  $\bar{y} = 4$ ,  $SCT = 4 + 1 + 9 = 14$  ;  $R^2 = 1 - \frac{2/7}{14} = \frac{48}{49} \approx 0,980$ . (d)  $\hat{y}(3) = \frac{36}{7} \approx 5,14$  DT.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import matplotlib
matplotlib.use("Agg")

x = np.array([1.0, 2, 4]); y = np.array([2.0, 3, 7])
X = np.column_stack([np.ones(3), x])
beta = np.linalg.solve(X.T @ X, X.T @ y)
res = y - X @ beta
print("beta =", beta.round(4), "| 12/7 =", round(12 / 7, 4))
print("résidus =", res.round(4), "| X'e =", (X.T @ res).round(10))
print("SCR =", round(res @ res, 4), "| R² =", round(1 - res @ res / ((y - y.mean())**2).sum(), 4), "|
↳ prévision en x = 3 :", round(beta[0] + 3 * beta[1], 4))
```

```
beta = [0.      1.7143] | 12/7 = 1.7143
résidus = [ 0.2857 -0.4286  0.1429] | X'e = [-0.   0.]
SCR = 0.2857 | R² = 0.9796 | prévision en x = 3 : 5.1429
```

**Corrigé 2.** (a) Site :  $e^{-0,1571} - 1 = -14,5\%$  ; Instagram :  $e^{-0,3368} - 1 = -28,6\%$  (et non  $-15,7\%$  et  $-33,7\%$  : l'approximation  $100\beta$  est mauvaise pour de grands coefficients, 1.1.8). (b)  $e^{10 \times 0,0092} - 1 = +9,6\%$ . (c) Pour 46 ans,  $a = 10$  :  $\hat{\mu} = 4,2206 - 0,1571 + 10 \times 0,0092 = 4,1555$ , donc  $e^{4,1555} \approx 63,8$  DT. (d) Parce que  $e^{\hat{\mu}}$  est la prédiction de la **médiane** de  $y$  : pour la moyenne il faudrait la correction  $e^{s^2/2}$  ou celle de Duan (1.1.8, et exercice 9).

```
for nom, b in [("Site", -0.1571), ("Instagram", -0.3368)]:
    print(f"{nom:10s}: {100 * (np.exp(b) - 1):+.1f} %")
print(f"10 ans d'âge : {100 * (np.exp(10 * 0.0092) - 1):+.1f} %")
print(f"panier médian prédit, 46 ans, Site : {np.exp(4.2206 - 0.1571 + 10 * 0.0092):.1f} DT")
```

```
Site      : -14.5 %
Instagram : -28.6 %
10 ans d'âge : +9.6 %
panier médian prédit, 46 ans, Site : 63.8 DT
```

**Corrigé 3.** On sait (1.1.1, équations normales) que  $\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}$  avec  $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$  et  $S_{xx} = \sum (x_i - \bar{x})^2$ . Or  $r = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}}$  et  $\frac{s_y}{s_x} = \sqrt{S_{yy}/S_{xx}}$ , donc  $r \frac{s_y}{s_x} = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}} \sqrt{\frac{S_{yy}}{S_{xx}}} = \frac{S_{xy}}{S_{xx}} = \hat{\beta}_1$ . Pour le  $R^2$  :  $\hat{y}_i - \bar{y} = \hat{\beta}_1(x_i - \bar{x})$ , donc  $SCE = \hat{\beta}_1^2 S_{xx} = \frac{S_{xy}^2}{S_{xx}}$  et  $R^2 = \frac{SCE}{SCT} = \frac{S_{xy}^2}{S_{xx}S_{yy}} = r^2$ . □

```
x4 = np.array([1.0, 2, 3, 4]); y4 = np.array([22.0, 41, 66, 79])
r = np.corrcoef(x4, y4)[0, 1]
```

```
b1 = np.polyfit(x4, y4, 1)[0]
print(f"r = {r:.5f} | r·sy/sx = {r * y4.std(ddof=1) / x4.std(ddof=1):.4f} | pente MCO = {b1:.4f}")
print(f"r² = {r**2:.5f} | R² du modèle = {sm.OLS(y4, sm.add_constant(x4)).fit().rsquared:.5f}")
```

```
r = 0.99350 | r·sy/sx = 19.6000 | pente MCO = 19.6000
r² = 0.98705 | R² du modèle = 0.98705
```

**Corrigé 4.** (a)  $t = -0,3368/0,0225 = -14,97$  ;  $|t| \gg 1,961$  : on rejette  $H_0$  (p-valeur de l'ordre de  $10^{-48}$ ). (b)  $-0,3368 \pm 1,961 \times 0,0225 = [-0,381 ; -0,293]$  ; en pourcentage :  $e^{-0,381} - 1 = -31,7\%$  et  $e^{-0,293} - 1 = -25,4\%$  : un effet de  $-25\%$  à  $-32\%$ . (c)  $t = 0,0092/0,00085 \approx 10,8$ .

```
b, se, crit = -0.3368, 0.0225, stats.t.ppf(0.975, 1736)
print(f"t = {b / se:.2f} | p = {2 * stats.t.sf(abs(b / se), 1736):.1e} | quantile = {crit:.3f}")
lo, hi = b - crit * se, b + crit * se
print(f"IC de beta : [{lo:.3f} ; {hi:.3f}] | IC de l'effet : [{100 * (np.exp(lo) - 1):.1f} % ; {100 * (np.exp(hi) - 1):.1f} %]")
print(f"t de l'âge = {0.0092 / 0.00085:.2f}")
```

```
t = -14.97 | p = 9.8e-48 | quantile = 1.961
IC de beta : [-0.381 ; -0.293] | IC de l'effet : [-31.7 % ; -25.4 %]
t de l'âge = 10.82
```

**Corrigé 5.** (a)  $q = 2$  (les deux coefficients du canal).  $F = \frac{(269,94 - 238,30)/2}{238,30/1736} = \frac{15,82}{0,1373} \approx 115,3$ . (b)  $115 \gg 3,0$  : on rejette l'hypothèse « le canal est inutile » (p-valeur de l'ordre de  $10^{-47}$ ). (c)  $F = \frac{(269,94 - 269,70)/2}{269,70/1736} = \frac{0,12}{0,1554} \approx 0,77$ , inférieur à 3,0 (et même à 1) : le canal n'apporte pas plus que du bruit, on ne rejette pas  $H_0$ .

```
def F_stat(scr0, scr1, q, ddl):
    F = ((scr0 - scr1) / q) / (scr1 / ddl)
    return F, stats.f.sf(F, q, ddl)
print("cas observé : F = %.2f, p = %.1e" % F_stat(269.94, 238.30, 2, 1736))
print("cas (c) : F = %.2f, p = %.2f" % F_stat(269.94, 269.70, 2, 1736))
print("valeur critique F(2, 1736) à 5 % :", round(stats.f.ppf(0.95, 2, 1736), 3))
```

```
cas observé : F = 115.25, p = 1.0e-47
cas (c) : F = 0.77, p = 0.46
valeur critique F(2, 1736) à 5 % : 3.001
```

**Corrigé 6.** (a)  $\bar{x} = 4$ ,  $\sum (x_k - \bar{x})^2 = 9 + 4 + 1 + 36 = 50$ .  $h_{11} = \frac{1}{4} + \frac{9}{50} = 0,43$  ;  $h_{22} = \frac{1}{4} + \frac{4}{50} = 0,33$  ;  $h_{33} = \frac{1}{4} + \frac{1}{50} = 0,27$  ;  $h_{44} = \frac{1}{4} + \frac{36}{50} = 0,97$ . Somme :  $2,00 = p$  ✓. (b) Le client d'âge 10, éloigné des autres : levier de 0,97 (presque 1). (c)  $\partial \hat{y}_4 / \partial y_4 = h_{44} = 0,97$  : la droite est presque **forcée** de passer par ce point ;  $\hat{y}_4$  augmente de 0,97.

```
x6 = np.array([1.0, 2, 3, 10]); X6 = np.column_stack([np.ones(4), x6])
h6 = np.diag(X6 @ np.linalg.inv(X6.T @ X6) @ X6.T)
print("leviers :", h6.round(3), "| somme =", h6.sum().round(3))
y6 = np.array([2.1, 3.9, 6.2, 20.0]); y6b = y6 + np.array([0, 0, 0, 1.0])
f1, f2 = sm.OLS(y6, X6).fit(), sm.OLS(y6b, X6).fit()
print("variation de la valeur ajustée du 4e point quand y4 augmente de 1 :", round(f2.fittedvalues[3] - f1.fittedvalues[3], 3))
```

```
leviers : [0.43 0.33 0.27 0.97] | somme = 2.0
variation de la valeur ajustée du 4e point quand y4 augmente de 1 : 0.97
```

**Corrigé 7.** (a)  $\bar{x} = 7/3$ ,  $S_{xx} = \frac{16}{9} + \frac{1}{9} + \frac{25}{9} = \frac{42}{9} = \frac{14}{3}$  ;  $h_{11} = \frac{1}{3} + \frac{16/9}{14/3} = \frac{5}{7} \approx 0,714$  ;  $h_{22} = \frac{1}{3} + \frac{1/9}{14/3} = \frac{5}{14} \approx 0,357$  ;  $h_{33} = \frac{1}{3} + \frac{25/9}{14/3} = \frac{13}{14} \approx 0,929$  (somme = 2 ✓). (b) Erreurs sans le point :  $\frac{2/7}{2/7} = 1$  ;  $\frac{-3/7}{9/14} = -\frac{2}{3}$  ;  $\frac{1/7}{1/14} = 2$ . PRESS =  $1 + \frac{4}{9} + 4 = \frac{49}{9} \approx 5,44$ . (c) Sans le point (1,2) : la droite

passé par (2, 3) et (4, 7), d'équation  $y = 2x - 1$ , et prévoit 1 en  $x = 1$  : l'erreur est  $2 - 1 = 1$  ✓. (d) PRESS (5,44) est près de **vingt fois** plus grand que la somme des carrés résiduelle ( $2/7 \approx 0,286$ ) : avec 3 points seulement, l'ajustement « d'apprentissage » est trompeusement optimiste, surtout pour le point  $x = 4$  à fort levier (0,93), dont l'erreur sans lui est de 2 contre un résidu de 0,14.

```
Xa = np.column_stack([np.ones(3), np.array([1.0, 2, 4])]); ya = np.array([2.0, 3, 7])
H = Xa @ np.linalg.inv(Xa.T @ Xa) @ Xa.T
e = ya - H @ ya; h = np.diag(H)
print("leviers :", h.round(3))
print("erreurs sans le point (formule) :", (e / (1 - h)).round(4), "| PRESS =", round(np.sum((e / (1 - h))**2), 4), "| 49/9 =", round(49 / 9, 4))
loo = []
for i in range(3):
    m = np.arange(3) != i
    b = np.linalg.lstsq(Xa[m], ya[m], rcond=None)[0]
    loo.append(ya[i] - Xa[i] @ b)
print("erreurs sans le point (réajustement) :", np.round(loo, 4), "| SCR =", round(e @ e, 4))
```

```
leviers : [0.714 0.357 0.929]
erreurs sans le point (formule) : [ 1.      -0.6667  2.      ] | PRESS = 5.4444 | 49/9 = 5.4444
erreurs sans le point (réajustement) : [ 1.      -0.6667  2.      ] | SCR = 0.2857
```

**Corrigé 8.** (a)  $VIF = \frac{1}{1-0,995} = 200$  ; l'erreur standard est multipliée par  $\sqrt{200} \approx 14$ . (b) Avec deux variables,  $R_j^2 = r^2 = 0,81$ , donc  $VIF = \frac{1}{0,19} \approx 5,3$  : à surveiller (règle empirique :  $> 5$ ). (c) Voir le code ci-dessous.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
print("VIF (a) :", 1 / (1 - 0.995), "| racine :", round(np.sqrt(200), 1), "| VIF (b) :", round(1 / (1 - 0.9**2), 2))
enq = pd.read_csv("donnees/enquete_satisfaction.csv")
Q = sm.add_constant(enq[[f"q{i}" for i in range(1, 9)]])
vif = pd.Series([variance_inflation_factor(Q.to_numpy(), i) for i in range(1, Q.shape[1])],
                 index=Q.columns[1:])
print(vif.round(2).to_string())
```

```
VIF (a) : 199.99999999999983 | racine : 14.1 | VIF (b) : 5.26
q1    1.60
q2    1.42
q3    1.47
q4    1.27
q5    1.71
q6    1.47
q7    1.63
q8    1.46
```

Les VIF des huit questions restent **modestes** (inférieurs à 2) : les questions d'un même bloc (produits d'un côté, service de l'autre) sont corrélées, mais pas au point de rendre un modèle instable. On peut donc les utiliser ensemble, ou les résumer par un score moyen comme au 1.4 (ce que l'analyse factorielle du chapitre 3 formalisera : section 3.2).

**Corrigé 9.** (a) Panier médian prédit :  $e^{\hat{\mu}}$  avec  $\hat{\mu}$  la prédiction du modèle. (b) La prédiction de la moyenne s'obtient en multipliant par le facteur de Duan  $\frac{1}{n} \sum_i e^{\hat{\epsilon}_i}$ . (c) On compare à la moyenne observée.

```
clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Instagram"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
m = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
profil = pd.DataFrame({"a": [25 - 36], "canal": pd.Categorical(["Instagram"], categories=["Boutique", "Site", "Instagram"])}))
```

```

mu = float(m.predict(profil).iloc[0])
duan = float(np.exp(m.resid).mean())
obs = df[(df["canal"] == "Instagram") & (df["age"].between(23, 27))]["panier_moyen"]
print(f"(a) panier médian prédit      : {np.exp(mu):.2f} DT")
print(f"(b) panier moyen prédit (Duan) : {np.exp(mu) * duan:.2f} DT   [facteur de Duan = {duan:.4f} ;
↳ facteur exp(s²/2) = {np.exp(m.scale / 2):.4f}]")
print(f"(c) moyenne observée (Instagram, 23-27 ans, n = {len(obs)}) : {obs.mean():.2f} DT (erreur
↳ standard de cette moyenne : {obs.std() / np.sqrt(len(obs)):.2f} | médiane observée :
↳ {obs.median():.2f} DT")
# Test décisif : sur TOUS les clients, quelle prédiction reproduit la moyenne observée ?
mediane_pred = np.exp(m.fittedvalues)
print(f"tous les clients (n = {len(df)}) : moyenne observée = {df['panier_moyen'].mean():.2f} |
↳ moyenne de exp(mu) = {mediane_pred.mean():.2f} | moyenne de exp(mu) x Duan = {(mediane_pred *
↳ duan).mean():.2f}")

```

```

(a) panier médian prédit      : 43.95 DT
(b) panier moyen prédit (Duan) : 47.10 DT   [facteur de Duan = 1.0718 ; facteur exp(s²/2) = 1.0710]
(c) moyenne observée (Instagram, 23-27 ans, n = 86) : 44.49 DT (erreur standard de cette moyenne :
↳ 1.40) | médiane observée : 42.69 DT
tous les clients (n = 1740) : moyenne observée = 61.23 | moyenne de exp(mu) = 57.12 | moyenne de
↳ exp(mu) x Duan = 61.22

```

Sur ce petit groupe (86 clients), la **moyenne observée (44,49) tombe plus près de la prédiction médiane (43,95) que de la prédiction corrigée (47,10)** : ce n'est pas une contradiction de la théorie, mais du bruit d'échantillonnage. L'erreur standard de cette moyenne observée est de 1,40 DT (écart-type des paniers d'environ 13 DT, divisé par  $\sqrt{86}$ ) : la prédiction corrigée (47,10) est à 1,9 erreur standard de la moyenne observée, la prédiction médiane à 0,4 : sur un groupe aussi petit, ces deux écarts sont tous deux plausibles par hasard. Pour **départager** vraiment, il faut un grand échantillon : sur les 1 740 clients, la moyenne observée est de 61,23 DT ; la moyenne des prédictions  $e^{\hat{\mu}}$  (la médiane de chacun) n'en donne que 57,12 DT, soit un défaut de 7 %, alors que les prédictions corrigées  $e^{\hat{\mu}} \times \text{Duan}$  redonnent 61,22 DT. C'est la confirmation de la théorie du 1.1.8 : la rétro-transformation **naïve vise la médiane, pas la moyenne**, et sous-estime systématiquement le panier moyen ; la correction de Duan la rétablit. (En attendant, retenez qu'une comparaison sur 86 observations ne départage pas des écarts de 7 %.)

### Corrigé 10.

```

v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"])
v["t"] = np.arange(len(v))
v["m"] = v["mois"].dt.month
mv = smf.ols("np.log(ca) ~ t + C(m) + promo + covid", data=v).fit()
ic = mv.conf_int()
print(f"(a) croissance annuelle : exp(12 x {mv.params['t']:.4f}) - 1 = {100 * (np.exp(12 *
↳ mv.params['t']) - 1):.1f} % par an   (IC95 % du coefficient mensuel : [{ic.loc['t', 0]:.4f} ;
↳ {ic.loc['t', 1]:.4f}])")
print(f"(b) décembre / janvier : x {np.exp(mv.params['C(m)[T.12]']):.2f}")
print(f"(c) confinement : {100 * (np.exp(mv.params['covid']) - 1):.0f} % sur les ventes   (IC95 % :
↳ [{100 * (np.exp(ic.loc['covid', 0]) - 1):.0f} % ; {100 * (np.exp(ic.loc['covid', 1]) - 1):.0f}
↳ %])")
print(f"    promotion : {100 * (np.exp(mv.params['promo']) - 1):.1f} %   (IC95 % : [{100 *
↳ (np.exp(ic.loc['promo', 0]) - 1):.1f} % ; {100 * (np.exp(ic.loc['promo', 1]) - 1):.1f} %])")
r = mv.resid.to_numpy()
print(f"(d) R² = {mv.rsquared:.3f} (contre 0,493 pour la tendance seule) | écart-type résiduel =
↳ {np.sqrt(mv.scale):.3f} | Durbin-Watson = {sm.stats.durbin_watson(r):.2f} | autocorrélation
↳ d'ordre 1 des résidus = {np.corrcoef(r[1:], r[:-1])[0, 1]:.2f}")

```

```

(a) croissance annuelle : exp(12 x 0.0072) - 1 = 9.0 % par an   (IC95 % du coefficient mensuel :
↳ [0.0068 ; 0.0076])
(b) décembre / janvier : x 2.57
(c) confinement : -42 % sur les ventes   (IC95 % : [-46 % ; -37 %])
    promotion : +5.9 %   (IC95 % : [+1.3 % ; +10.7 %])

```

(d)  $R^2 = 0.964$  (contre 0,493 pour la tendance seule) | écart-type résiduel = 0.078 | Durbin-Watson = 0.93 | autocorrélation d'ordre 1 des résidus = 0.54

- (a) Les ventes croissent d'environ **9 % par an** (0,72 % par mois). (b) Décembre est environ **2,6 fois** plus fort que janvier, à tendance égale (la saisonnalité est massive). (c) Le confinement de 2020 correspond à une baisse d'environ **42 %** des ventes sur les quatre mois concernés (intervalle large, de -46 % à -37 %). L'effet des promotions est estimé à environ +6 %, avec un intervalle de confiance **large** (de +1 % à +11 %) : un mois de promotion est un événement rare (une quinzaine de mois sur 120) et la dépendance entre mois rend l'intervalle encore plus incertain. (d) Le  $R^2$  monte à **0,96** : tendance, saisonnalité et chocs expliquent presque tout. Mais le **Durbin-Watson est de 0,93** : l'autocorrélation d'ordre 1 des résidus est **0,54**. Il reste donc de la structure : les erreurs successives se ressemblent, de sorte que les p-valeurs et intervalles ci-dessus sont **trop optimistes** (1.3.3). C'est exactement le sujet du chapitre 4 : modéliser cette dépendance (modèles ARIMA, section 4.2). (Une dernière remarque : les données ayant été simulées, nous pouvons vérifier que les valeurs estimées retrouvent la vérité : croissance de 0,75 % par mois, facteur saisonnier décembre/janvier de 1,55/0,62  $\approx$  2,5, effet du confinement de -0,55 en log (soit -42 %), effet des promotions de +0,10 en log (soit +10,5 % : dans l'intervalle, mais près de sa borne haute), autocorrélation des erreurs de 0,5.)

**Corrigé 11.** (a) Avec  $X^T X = I$ , la formule de Ridge  $(X^T X + \lambda I)^{-1} X^T y$  devient  $\frac{1}{1+\lambda} X^T y = \frac{1}{1+\lambda} \hat{\beta}^{MCO}$ . (b) Pour  $\beta = 2, \sigma = 1$  :  $EQM(0) = 1$  ;  $EQM(0,25) = \frac{0,0625 \times 4 + 1}{1,5625} = \frac{1,25}{1,5625} = 0,8$  ;  $EQM(1) = \frac{4 + 1}{4} = 1,25$ . (c) La dérivée s'annule en  $\lambda^* = \sigma^2 / \beta^2 = 0,25$  ; à ce point l'EQM vaut 0,8 (une baisse de 20 % par rapport aux moindres carrés), et elle redevient supérieure à 1 pour  $\lambda > 0,5$  ( $\lambda = 1$  donne 1,25).

```
beta_v, sigma_v = 2.0, 1.0
eqm = lambda lam: (lam**2 * beta_v**2 + sigma_v**2) / (1 + lam)**2
for lam in [0, 0.25, 0.5, 1]:
    print(f"lambda = {lam:4.2f} : EQM = {eqm(lam):.4f}")
grille = np.linspace(0, 3, 30001)
print("lambda optimal sur la grille :", round(grille[np.argmin(eqm(grille))], 4), "| sigma^2/beta^2 =",
      sigma_v**2 / beta_v**2)
rng = np.random.default_rng(1)
est = beta_v + rng.normal(0, sigma_v, 200000) # estimations MCO simulées
print("EQM simulée : MCO =", round(np.mean((est - beta_v)**2), 3), "| Ridge (lambda = 0,25) =",
      round(np.mean((est / 1.25 - beta_v)**2), 3))
```

```
lambda = 0.00 : EQM = 1.0000
lambda = 0.25 : EQM = 0.8000
lambda = 0.50 : EQM = 0.8889
lambda = 1.00 : EQM = 1.2500
lambda optimal sur la grille : 0.25 | sigma^2/beta^2 = 0.25
EQM simulée : MCO = 0.998 | Ridge (lambda = 0,25) = 0.8
```

**Corrigé 12.** (a)  $ICC = \frac{2}{2+6} = 0,25$ . (b)  $B_j = \frac{\tau^2}{\tau^2 + \sigma^2/n_j} = \frac{2}{2+6/9} = \frac{2}{2,667} = 0,75$  ;  $\hat{u}_j = 0,75 \times 3,2 = 2,4$  : on retient 75 % de l'écart observé. (c) Avec  $n_j = 2$  :  $B_j = \frac{2}{2+3} = 0,4$  et  $\hat{u}_j = 0,4 \times 3,2 = 1,28$  : un petit relais est rétréci bien davantage (60 % de l'écart est écarté, contre 25 % pour le grand). (d) Effet de plan 1 +  $(m-1)\rho = 1 + 8 \times 0,25 = 3$  ;  $n = 20 \times 9 = 180$  commandes valent  $180/3 = 60$  observations indépendantes.

```
tau2, sigma2 = 2.0, 6.0
print("ICC =", tau2 / (tau2 + sigma2))
for n_j in [9, 2]:
    B = tau2 / (tau2 + sigma2 / n_j)
```



```
print(f"n_j = {n_j}: B = {B:.3f} | BLUP = {B * 3.2:.3f}")
deff = 1 + (9 - 1) * tau2 / (tau2 + sigma2)
print("effet de plan =", deff, "| effectif effectif =", 180 / deff)
```

```
ICC = 0.25
n_j = 9: B = 0.750 | BLUP = 2.400
n_j = 2: B = 0.400 | BLUP = 1.280
effet de plan = 3.0 | effectif effectif = 60.0
```

### Corrigé 13.

```
rng = np.random.default_rng(5)
n = 100
x = rng.uniform(0, 10, n)
y_propre = 2 + 1.5 * x + rng.normal(0, 1, n)
y_corr = y_propre.copy()
idx = rng.choice(n, 5, replace=False)
y_corr[idx] += 25
X = sm.add_constant(x)
lignes = {}
for nom, y in [("données propres", y_propre), ("données corrompues", y_corr)]:
    lignes[(nom, "MCO")] = sm.OLS(y, X).fit().params
    lignes[(nom, "Huber")] = sm.RLM(y, X, M=sm.robust.norms.HuberT()).fit().params
    lignes[(nom, "médiane")] = sm.QuantReg(y, X).fit(q=0.5).params
tab = pd.DataFrame(lignes, index=["constante", "pente"]).T
print("vrais coefficients : constante = 2, pente = 1,5")
print(tab.round(3).to_string())
```

```
vrais coefficients : constante = 2, pente = 1,5
```

|                    |         | constante | pente |
|--------------------|---------|-----------|-------|
| données propres    | MCO     | 2.356     | 1.450 |
|                    | Huber   | 2.375     | 1.441 |
|                    | médiane | 2.297     | 1.436 |
| données corrompues | MCO     | 2.633     | 1.637 |
|                    | Huber   | 2.424     | 1.451 |
|                    | médiane | 2.297     | 1.450 |

Sur les données **propres**, les trois méthodes donnent des résultats très proches. Après corruption de 5 % des points, les **moindres carrés dérivent** (la pente passe de 1,45 à 1,64 et la constante de 2,36 à 2,63), alors que **Huber et la régression médiane** restent presque inchangés (pente entre 1,44 et 1,45 dans les deux cas). C'est l'intuition du 1.6 : l'influence des points aberrants est plafonnée par les méthodes robustes.

## Bilan du chapitre 1

Vous savez maintenant :

- **écrire et résoudre** un modèle linéaire  $y = X\beta + \varepsilon$  par les équations normales, comprendre pourquoi la solution est une **projection orthogonale** et démontrer le théorème de **Gauss-Markov** ;
- **interpréter** les coefficients (variables qualitatives par indicatrices, « toutes choses égales par ailleurs », modèles en logarithmes, rétro-transformation pour prédire une moyenne) ;
- **faire de l'inférence** : erreurs standard, tests  $t$  et  $F$  de modèles emboîtés, intervalles de confiance et de **prédiction**, bootstrap des couples, en sachant sous quelles hypothèses ils sont valables ;
- **poser un diagnostic** : graphiques de résidus, tests de Breusch-Pagan et de Durbin-Watson, **levier** et **distance de Cook**, **VIF**, erreurs standard robustes, et savoir remédier aux défauts



- (transformer, centrer, changer de modèle) ;
- **choisir un modèle** sans tricher : AIC, BIC, validation croisée (PRESS), tests emboîtés, et se méfier de la sélection automatique ;
  - (en option) **régulariser** (Ridge, Lasso, Elastic Net) quand les variables sont nombreuses, **résister** aux aberrations (Huber, régression quantile) et **modéliser des données groupées** (effets mixtes, rétrécissement).

Le chapitre 2 généralise la régression à des réponses qui ne sont **ni continues ni normales** : un client rachète-t-il (oui/non) ? combien de commandes passe-t-il (un entier) ? C'est le cadre des **modèles linéaires généralisés**, dont la régression linéaire de ce chapitre est le cas particulier le plus simple.

# Chapitre 2 : Modèles linéaires généralisés

« La régression linéaire répond à la question : *de combien la moyenne change-t-elle ?* Les modèles linéaires généralisés répondent à la même question, **pour des données qui ne sont pas des mesures continues et symétriques.** »

Au chapitre 1, nous avons appris à expliquer une variable **continue** (le montant d'une commande, une durée) par d'autres variables, avec une droite, un plan, un hyperplan. Mais regardez les questions que Yasmine se pose vraiment :

- « *Ce client va-t-il racheter dans les douze mois ?* » : la réponse est **oui ou non** (0 ou 1) ;
- « *Combien de commandes va-t-il passer cette année ?* » : la réponse est un **nombre entier** (0, 1, 2, 3...) ;
- « *Combien va-t-il dépenser ?* » : la réponse est un **montant positif**, très asymétrique, parfois nul.

Ces trois variables n'ont rien de « normal » : elles ne prennent que deux valeurs, ou seulement des valeurs entières, ou seulement des valeurs positives. Une droite des moindres carrés leur va très mal : elle prédit des probabilités négatives ou supérieures à 1, elle ignore que la dispersion croît avec la moyenne, elle suppose des erreurs symétriques qui n'existent pas.

Les **modèles linéaires généralisés** (*generalized linear models*, GLM) corrigent cela avec une idée très simple, due à Nelder et Wedderburn (1972) : on garde ce qui marchait dans la régression linéaire (un **prédicteur linéaire**  $x^T \beta$ , qui combine les variables explicatives par une somme pondérée) et on change deux choses : la **loi** de la variable à expliquer, et le **lien** entre la moyenne et le prédicteur linéaire. La régression linéaire n'est plus qu'un cas particulier ; la **régression logistique** (oui/non), la **régression de Poisson** (comptages) et la **régression Gamma** (montants positifs) en sont trois autres, avec **une seule théorie** et **un seul algorithme** d'estimation.

## Le chemin de ce chapitre

- **2.1 Le cadre des GLM** : pourquoi la droite ne suffit plus ; les trois ingrédients (loi, prédicteur linéaire, lien) ; la famille exponentielle ; l'algorithme des moindres carrés repondérés itérés (IRLS), écrit à la main.
- **2.2 Régression logistique** : expliquer un oui/non ; cotes (*odds*) et rapports de cotes ; effets marginaux ; ROC, AUC, calibration.
- **2.3 Régression de Poisson et Gamma** : expliquer un comptage (avec exposition, surdispersion, loi binomiale négative) et un montant positif.
- **2.4 Déviance, qualité d'ajustement, vérification du modèle** : comparer des modèles emboîtés, lire les résidus, détecter un modèle qui ne tient pas.
- **⊕ Pour aller plus loin** : les modèles additifs généralisés, GAM (2.5) ; les modèles à excès de zéros, surdispersés, et la loi de Tweedie (2.6).

- 2.7 Exercices corrigés.

**Le fil conducteur : 2 000 clients de Dar Jasmin.** Nous travaillons sur le fichier `donnees/clients.csv` : un client par ligne, avec son âge, sa ville, son canal d'acquisition, sa dépense annuelle, s'il a racheté, combien de commandes il a passées. Une colonne est particulière : `offre_bienvenue` (0 ou 1) a été **attribuée au hasard** (Yasmine a tiré à pile ou face l'envoi d'un bon de bienvenue). Nous y reviendrons : un tirage au hasard permet de répondre à « *l'offre fait-elle racheter ?* » sans arrière-pensée.

**Les données sont simulées.** Comme dans le volume I, le jeu de données est fabriqué par un programme (graine fixe), ce qui permet à chacun de retrouver les mêmes nombres. Il a un avantage pédagogique énorme : **nous connaissons la vérité**. À la fin du chapitre, nous la dévoilerons, pour voir ce que nos modèles ont retrouvé, et ce qu'ils ont manqué. Un fichier complémentaire (`donnees/ch02-sessions.csv`, section 2.5) est également simulé ; le jeu `donnees/enquete_satisfaction.csv` (notes de 1 à 5 de 1 200 répondants) sert à la section 2.2.

**Outils.** Nous utilisons `statsmodels` (formules à la R : " $y \sim x_1 + x_2$ "), `numpy` et `scipy`. Quelques blocs en R (`glm`, `mgcv`) montrent que les mêmes modèles s'écrivent presque pareil dans l'autre grand langage de la statistique. Pour la théorie des chapitres précédents, nous renvoyons au volume I (vraisemblance : section 3.2 ; tests : section 3.4) et au chapitre 1 de ce volume (régression linéaire, moindres carrés).

## 2.1 Le cadre des GLM

**Intuition.** Une régression linéaire dit : « *la moyenne de Y est une combinaison linéaire des variables explicatives, et autour de cette moyenne, Y fluctue comme une loi normale* ». Un GLM relâche les deux morceaux de cette phrase : la loi autour de la moyenne peut être **Bernoulli**, **Poisson**, **Gamma...** ; et c'est une **transformation** de la moyenne (le *lien*) qui est une combinaison linéaire. Tout le reste (l'estimation, les tests, les diagnostics) est la même mécanique que pour la droite des moindres carrés, appliquée avec des **poids** qui changent à chaque itération.

### 2.1.1 Pourquoi la droite ne suffit plus

Regardons d'abord nos trois variables à expliquer, sans rien modéliser.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

clients = pd.read_csv("donnees/clients.csv")
print(clients[["age", "canal_acquisition", "offre_bienvenue", "rachat_12m", "nb_commandes_an",
↪ "depense_annuelle"]].head(6).to_string())
print()
n = clients["nb_commandes_an"]
d = clients["depense_annuelle"]
print("rachat_12m      : valeurs", sorted(clients["rachat_12m"].unique().tolist()), "| proportion de
↪ 1 :", round(clients["rachat_12m"].mean(), 3))
print("nb_commandes_an : moyenne", round(n.mean(), 2), "| variance", round(n.var(), 2), "| part de
↪ zéros", round((n == 0).mean(), 3))
```

```
print("depense_annuelle : moyenne", round(d.mean(), 1), "| médiane", round(d.median(), 1), "|
↳ écart-type", round(d.std(), 1),
    "| part de zéros", round((d == 0).mean(), 3))
```

|   | age | canal_acquisition | offre_bienvenue | rachat_12m | nb_commandes_an | depense_annuelle |
|---|-----|-------------------|-----------------|------------|-----------------|------------------|
| 0 | 19  | Instagram         | 0               | 0          | 4               | 116.50           |
| 1 | 43  | Site              | 1               | 1          | 2               | 148.92           |
| 2 | 35  | Boutique          | 1               | 0          | 6               | 509.05           |
| 3 | 42  | Instagram         | 0               | 0          | 0               | 0.00             |
| 4 | 21  | Instagram         | 0               | 0          | 7               | 321.77           |
| 5 | 30  | Site              | 0               | 1          | 13              | 1141.53          |

```
rachat_12m      : valeurs [0, 1] | proportion de 1 : 0.509
nb_commandes_an : moyenne 3.88 | variance 13.27 | part de zéros 0.13
depense_annuelle : moyenne 247.0 | médiane 156.5 | écart-type 297.2 | part de zéros 0.13
```

Trois variables, trois natures. `rachat_12m` ne prend que les valeurs 0 et 1 (51 % de clients ont racheté). `nb_commandes_an` est un comptage : en moyenne 3,88 commandes, mais avec une variance de 13,27, soit **3,4 fois la moyenne** (nous y reviendrons), et 13 % de clients à zéro. `depense_annuelle` est un montant : moyenne de 247 DT mais médiane de 156,5 DT et écart-type de 297 DT (supérieur à la moyenne) : la signature d'une forte asymétrie à droite, avec elle aussi 13 % de zéros (ce sont les mêmes clients : pas de commande, pas de dépense).

**Premier problème : les probabilités sortent de [0, 1].** Pour expliquer `rachat_12m` (0 ou 1) par une droite, on ajuste ce qu'on appelle un **modèle de probabilité linéaire** :  $P(Y = 1) = \beta_0 + \beta_1 x_1 + \dots$ , estimé par moindres carrés ordinaires. Utilisons les clients qui ont répondu à l'enquête de satisfaction (section 2.2) : on dispose pour eux de deux notes moyennes, `note_produits` (questions 1 à 4) et `note_service` (questions 5 à 8).

```
enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
enquete["note_produits"] = enquete[["q1", "q2", "q3", "q4"]].mean(axis=1)
enquete["note_service"] = enquete[["q5", "q6", "q7", "q8"]].mean(axis=1)
repondants = clients.merge(enquete[["id_client", "note_produits", "note_service"]], on="id_client")
print("répondants :", len(repondants))

formule = "rachat_12m ~ note_produits + note_service + offre_bienvenue + age"
lineaire = smf.ols(formule, repondants).fit()
logistique = smf.glm(formule, repondants, family=sm.families.Binomial()).fit()

p_lin = lineaire.fittedvalues
print("probabilités « prédites » par la droite : min", round(p_lin.min(), 3), "| max",
    ↳ round(p_lin.max(), 3))
print("nombre de clients avec une probabilité < 0 :", int((p_lin < 0).sum()), "| > 1 :", int((p_lin >
    ↳ 1).sum()))

# Un client « de rêve » : notes parfaites, offre reçue, 20 ans
reve = pd.DataFrame({"note_produits": [5.0], "note_service": [5.0], "offre_bienvenue": [1], "age":
    ↳ [20]})
print("client de rêve : droite =", round(float(lineaire.predict(reve).iloc[0]), 3), "| régression
    ↳ logistique =", round(float(logistique.predict(reve).iloc[0]), 3))
# Un client « de cauchemar »
cauchemar = pd.DataFrame({"note_produits": [1.0], "note_service": [1.0], "offre_bienvenue": [0],
    ↳ "age": [65]})
print("client de cauchemar : droite =", round(float(lineaire.predict(cauchemar).iloc[0]), 3), "|
    ↳ régression logistique =", round(float(logistique.predict(cauchemar).iloc[0]), 3))
```

```
répondants : 1212
probabilités « prédites » par la droite : min -0.071 | max 0.95
nombre de clients avec une probabilité < 0 : 3 | > 1 : 0
client de rêve : droite = 1.005 | régression logistique = 0.905
client de cauchemar : droite = -0.355 | régression logistique = 0.021
```

Soyons précis sur ce que montre ce résultat. Sur nos 1 212 répondants, la droite prédit des probabilités comprises entre  $-0,071$  et  $0,95$  : seuls **3 clients** reçoivent une « probabilité » négative, aucun ne dépasse 1. L'inconvénient reste donc modeste *tant que l'on reste au milieu des données* (c'est pourquoi certains praticiens, notamment en économie, utilisent parfois cette approche). Mais dès que l'on s'approche des extrêmes, le défaut de principe apparaît : pour le client « de rêve » (notes parfaites, offre reçue, 20 ans), la droite annonce **1,005**, une probabilité supérieure à 1 ; pour le client « de cauchemar », elle annonce  $-0,355$ , une probabilité négative. La régression logistique, elle, répond  $0,905$  et  $0,021$  : des valeurs plausibles, et jamais hors de  $[0, 1]$ . Il y a un second défaut, plus discret : la variance d'un 0/1 vaut  $p(1 - p)$ , elle **dépend de la moyenne**, donc les erreurs-types de la droite sont incorrectes.

**Deuxième problème : la dispersion n'est pas constante.** Pour un comptage, plus la moyenne est grande, plus les valeurs s'étalent (pensez à un client qui commande en moyenne 1 fois par an : il commande 0, 1 ou 2 fois ; un client qui commande en moyenne 20 fois : entre 10 et 30). La régression linéaire suppose une variance **constante** (homoscédasticité). Pour un oui/non, la variance est  $p(1 - p)$  : elle dépend de la moyenne  $p$ , elle aussi. Regardons les comptages de commandes, par canal.

```
par_canal = clients.groupby("canal_acquisition")["nb_commandes_an"].agg(moyenne="mean",
    ↪ variance="var", clients="count")
par_canal["variance / moyenne"] = par_canal["variance"] / par_canal["moyenne"]
print(par_canal.round(2).to_string())

# Fréquences observées et fréquences d'une loi de Poisson de même moyenne
from scipy import stats
lam = n.mean()
valeurs = np.arange(0, 11)
obs = np.array([(n == k).mean() for k in valeurs])
poi = stats.poisson.pmf(valeurs, lam)
tab = pd.DataFrame({"observé": obs, "Poisson(%.2f)" % lam: poi}, index=valeurs)
tab.loc["11 et +"] = [(n >= 11).mean(), stats.poisson.sf(10, lam)]
print()
print(tab.round(3).to_string())
```

| canal_acquisition | moyenne | variance | clients | variance / moyenne |
|-------------------|---------|----------|---------|--------------------|
| Boutique          | 4.14    | 13.75    | 504     | 3.32               |
| Instagram         | 3.46    | 11.84    | 816     | 3.42               |
| Site              | 4.20    | 14.32    | 680     | 3.41               |

|         | observé | Poisson(3.88) |
|---------|---------|---------------|
| 0       | 0.130   | 0.021         |
| 1       | 0.154   | 0.080         |
| 2       | 0.158   | 0.155         |
| 3       | 0.130   | 0.201         |
| 4       | 0.096   | 0.195         |
| 5       | 0.088   | 0.151         |
| 6       | 0.060   | 0.098         |
| 7       | 0.048   | 0.054         |
| 8       | 0.038   | 0.026         |
| 9       | 0.020   | 0.011         |
| 10      | 0.018   | 0.004         |
| 11 et + | 0.060   | 0.002         |

La variance des comptages est **3,3 à 3,4 fois leur moyenne dans chacun des trois canaux**. Or une loi de Poisson impose variance = moyenne. Le second tableau le montre sans ambiguïté : une loi de Poisson de moyenne 3,88 prévoirait 2,1 % de clients à zéro commande, alors qu'on en observe 13,0 % ; elle prévoirait 0,2 % de clients à 11 commandes ou plus, alors qu'on en observe 6,0 %. La distribution observée est plus **aplatie** que Poisson, avec trop de valeurs à *chaque* extrémité : c'est la **surdispersion**. Nous la traiterons à la section 2.3.

**Troisième problème : des montants positifs, asymétriques, avec des zéros.** Une dépense annuelle ne peut pas être négative, sa dispersion augmente avec le niveau (les gros clients varient en DT bien plus que les petits) et il y a un paquet de clients à zéro. Une loi normale n'est pas du tout adaptée. Résumons tout cela en une figure.

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.9))
BLEU, ORANGE, AQUA, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#e34948"

# (a) probabilités prédites : droite (en abscisse) contre logistique (en ordonnée), pour chaque
↳ répondant
ax = axes[0]
ax.axvspan(-0.5, 0, color=ROUGE, alpha=0.10, lw=0); ax.axvspan(1, 1.1, color=ROUGE, alpha=0.10, lw=0)
ax.scatter(lineaire.fittedvalues, logistique.fittedvalues, s=5, color="#898781", alpha=0.5)
x_reve = float(lineaire.predict(reve).iloc[0]); y_reve = float(logistique.predict(reve).iloc[0])
x_cauch = float(lineaire.predict(cauchemar).iloc[0]); y_cauch =
↳ float(logistique.predict(cauchemar).iloc[0])
ax.scatter([x_reve, x_cauch], [y_reve, y_cauch], s=50, color=ORANGE, zorder=3)
ax.annotate("client de rêve", (x_reve, y_reve), (0.52, 0.99), fontsize=8, color=ORANGE,
↳ arrowprops=dict(arrowstyle="-", color=ORANGE, lw=0.8))
ax.annotate("client de cauchemar", (x_cauch, y_cauch), (-0.42, 0.20), fontsize=8, color=ORANGE,
↳ arrowprops=dict(arrowstyle="-", color=ORANGE, lw=0.8))
ax.plot([0, 1], [0, 1], color=BLEU, lw=1, ls=":")
ax.set_xlim(-0.45, 1.1); ax.set_ylim(0, 1.05)
ax.set_xlabel("probabilité prédite par la droite (OLS)"); ax.set_ylabel("probabilité prédite par la
↳ logistique")
ax.set_title("(a) un oui/non : la droite peut sortir de [0 ; 1]")

# (b) comptages : observé contre Poisson
ax = axes[1]
x = np.arange(0, 13)
ax.bar(x - 0.2, [(n == k).mean() for k in x], width=0.4, color=BLEU, label="observé")
ax.bar(x + 0.2, stats.poisson.pmf(x, lam), width=0.4, color=ORANGE, label="Poisson de même moyenne")
ax.set_xlabel("nombre de commandes dans l'année"); ax.set_ylabel("proportion de clients")
ax.set_title("(b) un comptage : trop dispersé pour Poisson"); ax.legend(frameon=False, fontsize=8)

# (c) dépense annuelle
ax = axes[2]
ax.hist(d, bins=np.arange(0, 1300, 40), color=AQUA)
ax.set_xlabel("dépense annuelle (DT)"); ax.set_ylabel("nombre de clients")
ax.set_title("(c) un montant : positif, asymétrique, zéros")
plt.tight_layout()
plt.savefig("figures/ch02-pourquoi-glm.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Lisons les trois panneaux. En (a), chaque point est un répondant : si les deux modèles s'accordaient partout, les points suivraient la diagonale pointillée. C'est presque le cas au milieu du nuage (entre 0,2 et 0,8), mais la logistique s'incurve aux extrémités pour rester dans [0,1], alors que la droite continue tout droit : les deux points orange sont tombés dans les zones rouges (valeurs impossibles). En (b), la distribution observée est bien plus étalée que celle de Poisson. En (c), la dépense est positive, très asymétrique ; le pic dans la première classe contient entre autres les 13 % de clients à zéro. **Chacun de ces trois problèmes a sa solution dans le cadre GLM** : la logistique (section 2.2), Poisson et sa variante binomiale négative (2.3), la loi Gamma pour les montants positifs (2.3) et la loi de Tweedie pour les montants avec des zéros (2.6).

### 2.1.2 Les trois ingrédients d'un GLM

Un modèle linéaire généralisé est défini par **trois ingrédients**.

1. **Une composante aléatoire.** Les observations  $Y_1, \dots, Y_n$  sont **indépendantes**, et chaque  $Y_i$

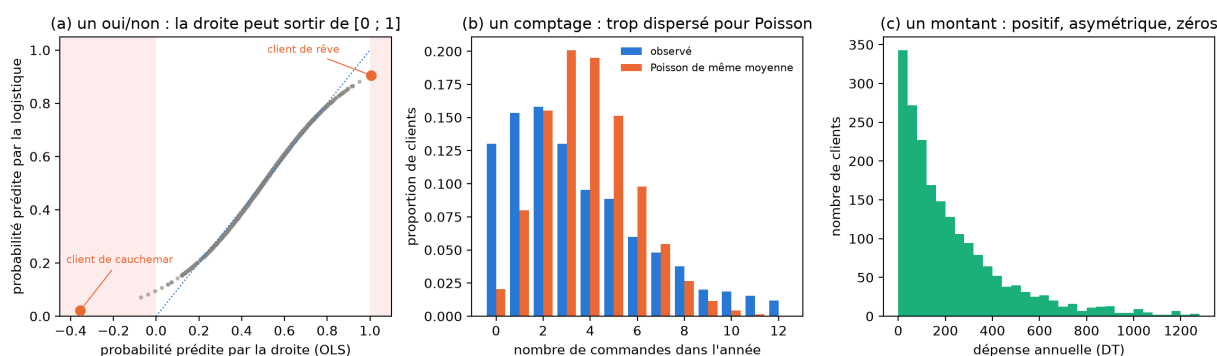


Figure 17: Trois variables, trois problèmes pour la droite des moindres carrés. (a) Probabilité de racheter prédite par la droite (en abscisse) et par la logistique (en ordonnée) pour chacun des 1 212 répondants : les zones rouges sont les valeurs impossibles pour une probabilité ; les deux points orange sont des profils extrêmes, pour lesquels la droite sort de  $[0 ; 1]$  alors que la logistique reste raisonnable. (b) Nombre de commandes : la loi de Poisson de même moyenne est trop étroite. (c) Dépense annuelle : asymétrie à droite et pic en zéro.

suit une loi d'une même *famille exponentielle* (voir 2.1.3), de moyenne  $\mu_i = E[Y_i]$ . Exemples : normale, Bernoulli, binomiale, Poisson, Gamma.

2. **Un prédicteur linéaire.** Pour le client  $i$  de variables explicatives  $x_i = (1, x_{i1}, \dots, x_{ip})$ ,

$$\eta_i = x_i^\top \beta = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}.$$

3. **Une fonction de lien**  $g$ , monotone et dérivable, qui relie la moyenne au prédicteur :

$$g(\mu_i) = \eta_i \quad \Leftrightarrow \quad \mu_i = g^{-1}(\eta_i).$$

La régression linéaire du chapitre 1 est le cas où la loi est **normale** et le lien est l'**identité** ( $g(\mu) = \mu$ ).

**Où est passé l'erreur  $\varepsilon$  ?** On écrit souvent la régression linéaire  $Y = x^\top \beta + \varepsilon$ . Dans un GLM, on ne peut plus « ajouter une erreur » : on ne peut pas ajouter un bruit symétrique à un 0/1. On décrit plutôt la **loi conditionnelle** de  $Y$  sachant  $x$  (sa moyenne, via le lien, et sa forme, via la famille). C'est un changement de point de vue qui sera très utile au chapitre 6.

### 2.1.3 La famille exponentielle

Les lois qui nous intéressent ont toutes la **même forme**. On dit qu'une loi appartient à la *famille exponentielle à dispersion* si sa densité (ou sa fonction de masse) s'écrit

$$f(y; \theta, \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{\phi} + c(y, \phi) \right\},$$

où  $\theta$  est le **paramètre naturel** (il porte la moyenne),  $\phi > 0$  le **paramètre de dispersion**,  $b$  une fonction dérivable deux fois, et  $c$  une fonction qui ne dépend pas de  $\theta$ . Le point remarquable est que  $b$  **détermine à elle seule** la moyenne et la variance.

**Proposition.**  $E[Y] = b'(\theta)$  et  $\text{Var}(Y) = \phi b''(\theta)$ .



*Démonstration.* Notons  $\ell(\theta) = \log f(y; \theta, \phi) = \frac{y\theta - b(\theta)}{\phi} + c(y, \phi)$ . Sa dérivée (le **score**) est  $U = \frac{\partial \ell}{\partial \theta} = \frac{y - b'(\theta)}{\phi}$ . Deux propriétés classiques de la vraisemblance (volume I, section 3.2.5, en donne le contexte) nous suffisent. (i)  $E[U] = 0$  : en effet  $\int f dy = 1$  pour tout  $\theta$ , et en dérivant sous l'intégrale,  $0 = \int \frac{\partial f}{\partial \theta} dy = \int \frac{\partial \log f}{\partial \theta} f dy = E[U]$ . Donc  $E[Y] = b'(\theta)$ . (ii)  $E[U^2] = -E[\frac{\partial^2 \ell}{\partial \theta^2}]$  : on dérive une seconde fois l'identité précédente. Ici  $\frac{\partial^2 \ell}{\partial \theta^2} = -\frac{b''(\theta)}{\phi}$  ne dépend pas de  $y$ , et  $E[U^2] = \frac{\text{Var}(Y)}{\phi^2}$ . L'égalité devient  $\frac{\text{Var}(Y)}{\phi^2} = \frac{b''(\theta)}{\phi}$ , c'est-à-dire  $\text{Var}(Y) = \phi b''(\theta)$ .  $\square$

Comme  $\mu = b'(\theta)$ , on peut exprimer  $b''(\theta)$  en fonction de  $\mu$  : on l'appelle la **fonction de variance**  $V(\mu)$ , et l'on obtient la relation fondamentale

$$\text{Var}(Y) = \phi V(\mu).$$

La famille choisie impose donc **comment la variance dépend de la moyenne**. Voici les quatre cas du chapitre, avec leur écriture canonique (vous pouvez vérifier chaque ligne en développant la densité).

| Loi                                        | $\theta$             | $b(\theta)$          | $\phi$     | $V(\mu)$       | Lien canonique |
|--------------------------------------------|----------------------|----------------------|------------|----------------|----------------|
| Normale( $\mu, \sigma^2$ )                 | $\mu$                | $\theta^2/2$         | $\sigma^2$ | 1              | identité       |
| Bernoulli( $p$ )                           | $\log \frac{p}{1-p}$ | $\log(1 + e^\theta)$ | 1          | $\mu(1 - \mu)$ | logit          |
| Poisson( $\lambda$ )                       | $\log \lambda$       | $e^\theta$           | 1          | $\mu$          | log            |
| Gamma<br>(moyenne $\mu$ ,<br>forme $\nu$ ) | $-1/\mu$             | $-\log(-\theta)$     | $1/\nu$    | $\mu^2$        | inverse        |

Prenons la loi de Poisson à la main :  $f(y; \lambda) = e^{-\lambda} \lambda^y / y! = \exp\{y \log \lambda - \lambda - \log y!\}$ . On lit  $\theta = \log \lambda$ ,  $b(\theta) = e^\theta$  (puisque  $\lambda = e^\theta$ ),  $\phi = 1$  et  $c(y) = -\log y!$ . Alors  $b'(\theta) = e^\theta = \lambda$  et  $b''(\theta) = e^\theta = \lambda$  : la moyenne **et** la variance valent  $\lambda$ , comme nous le savons depuis le volume I.

Vérifions numériquement les quatre lignes du tableau : on dérive  $b$  numériquement, et l'on compare à la moyenne et à la variance d'un grand échantillon simulé.

```
rng = np.random.default_rng(21)

def d1(f, x):
    h = 1e-4 * abs(x)          # pas relatif : theta varie de 1/60 à 5 selon les lois
    return (f(x + h) - f(x - h)) / (2 * h)

def d2(f, x):
    h = 1e-4 * abs(x)
    return (f(x + h) - 2 * f(x) + f(x - h)) / h**2

N = 400_000
cas = [
    # nom, theta, b, phi, échantillon simulé
    ("Normale(5 ; sigma²=4)", 5.0, lambda t: t**2 / 2, 4.0, rng.normal(5, 2, N)),
    ("Bernoulli(0,3)", np.log(0.3 / 0.7), lambda t: np.log1p(np.exp(t)), 1.0, rng.binomial(1, 0.3,
    ↪ N)),
    ("Poisson(3)", np.log(3.0), lambda t: np.exp(t), 1.0, rng.poisson(3, N)),
    ("Gamma(moyenne 60 ; forme 4)", -1 / 60, lambda t: -np.log(-t), 1 / 4, rng.gamma(4, 60 / 4, N)),
]
```

```
lignes = []
for nom, theta, b, phi, echantillon in cas:
    lignes.append({"loi": nom, "b'(theta)": d1(b, theta), "phi * b''(theta)": phi * d2(b, theta),
                  "moyenne simulée": echantillon.mean(), "variance simulée": echantillon.var()})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

|  | loi                         | b'(theta) | phi * b''(theta) | moyenne simulée | variance simulée |
|--|-----------------------------|-----------|------------------|-----------------|------------------|
|  | Normale(5 ; sigma²=4)       | 5.0       | 4.00             | 4.998           | 4.005            |
|  | Bernoulli(0,3)              | 0.3       | 0.21             | 0.300           | 0.210            |
|  | Poisson(3)                  | 3.0       | 3.00             | 3.002           | 2.991            |
|  | Gamma(moyenne 60 ; forme 4) | 60.0      | 900.00           | 60.024          | 900.824          |

Les deux colonnes de gauche (calculées uniquement à partir de  $b : b'(\theta)$  et  $\phi b''(\theta)$ ) coïncident avec la moyenne et la variance des 400 000 tirages, à l'erreur de simulation près : par exemple, pour la loi Gamma, la formule donne une variance de 900 ( $= \mu^2/\nu = 60^2/4$ ) et la simulation 900,8 ; pour Bernoulli,  $0,3 \times 0,7 = 0,21$  dans les deux cas. Une seule fonction  $b$  suffit donc à décrire la moyenne et la variance de chaque loi.

**Pourquoi se donner ce mal ?** Parce que tout ce qui suit (le score, l'information de Fisher, l'algorithme d'estimation, la déviance) ne dépend que de  $b$  et du lien. On écrit **une seule fois** la théorie, et elle vaut pour toutes les lois de la famille.

### 2.1.4 Les fonctions de lien

Le **lien** transforme la moyenne en prédicteur linéaire,  $g(\mu) = \eta$ . Il a deux rôles : garantir que la moyenne prédite reste dans son domaine (une probabilité dans  $]0,1[$ , un nombre positif...) et fixer **l'échelle sur laquelle les effets sont additifs**.

| Lien     | $g(\mu)$                 | $\mu = g^{-1}(\eta)$    | Domaine de $\mu$ | Un effet $+\beta$ sur $\eta$ signifie                           |
|----------|--------------------------|-------------------------|------------------|-----------------------------------------------------------------|
| identité | $\mu$                    | $\eta$                  | $\mathbb{R}$     | la moyenne augmente de $\beta$                                  |
| log      | $\log \mu$               | $e^\eta$                | $]0, \infty[$    | la moyenne est <b>multipliée</b> par $e^\beta$                  |
| logit    | $\log \frac{\mu}{1-\mu}$ | $\frac{1}{1+e^{-\eta}}$ | $]0, 1[$         | la <b>cote</b> $\frac{\mu}{1-\mu}$ est multipliée par $e^\beta$ |
| inverse  | $1/\mu$                  | $1/\eta$                | $]0, \infty[$    | l'inverse de la moyenne augmente de $\beta$                     |

Le **lien canonique** d'une famille est celui qui rend  $\theta = \eta$ , c'est-à-dire  $g = (b')^{-1}$  : logit pour Bernoulli, log pour Poisson, inverse pour Gamma, identité pour la normale. Il a de belles propriétés mathématiques (l'équation du score se simplifie, l'estimation converge très bien), mais ce n'est **pas une obligation** : le choix du lien est un choix de modélisation, à faire selon l'interprétation voulue. On utilise très souvent le lien **log pour la loi Gamma** (effets multiplicatifs sur des montants), bien que son lien canonique soit l'inverse.

**Exemple chiffré : le lien log.** Supposons qu'un client Instagram passe en moyenne 3 commandes par an, et qu'un client de la boutique en passe 3,6. Avec un lien log,  $\log \mu_{\text{boutique}} - \log \mu_{\text{Instagram}} = \beta$  donne  $\beta = \log(3,6/3) = \log 1,2 \approx 0,182$ . On lit : « la boutique passe **20 % de commandes en plus** », un effet **multiplicatif** : si Instagram passait 10 commandes, la boutique en passerait 12. Avec un lien identité, on aurait dit « 0,6 commande de plus », un

effet **additif** : or il est peu plausible que l'écart reste de 0,6 pour des clients bien plus actifs.

### 2.1.5 L'estimation : maximum de vraisemblance et IRLS

Les paramètres  $\beta$  sont estimés par **maximum de vraisemblance** (volume I, section 3.2). La log-vraisemblance de l'échantillon est  $\ell(\beta) = \sum_i \left[ \frac{y_i \theta_i - b(\theta_i)}{\phi} + c(y_i, \phi) \right]$ , où  $\theta_i$  dépend de  $\beta$  à travers  $\mu_i = g^{-1}(x_i^\top \beta)$ .

**Les équations du score.** Par la règle de dérivation en chaîne,  $\frac{\partial \ell}{\partial \beta_j} = \sum_i \frac{\partial \ell_i}{\partial \theta_i} \frac{\partial \theta_i}{\partial \mu_i} \frac{\partial \mu_i}{\partial \eta_i} \frac{\partial \eta_i}{\partial \beta_j}$ . Or  $\frac{\partial \ell_i}{\partial \theta_i} = \frac{y_i - \mu_i}{\phi}$ ;  $\frac{\partial \mu_i}{\partial \theta_i} = b''(\theta_i) = V(\mu_i)$ , donc  $\frac{\partial \theta_i}{\partial \mu_i} = \frac{1}{V(\mu_i)}$ ; et  $\frac{\partial \eta_i}{\partial \beta_j} = x_{ij}$ . On obtient

$$\frac{\partial \ell}{\partial \beta_j} = \frac{1}{\phi} \sum_{i=1}^n \frac{(y_i - \mu_i) x_{ij}}{V(\mu_i)} \frac{\partial \mu_i}{\partial \eta_i}, \quad j = 0, \dots, p.$$

Ces  $p + 1$  équations ne sont **pas linéaires** en  $\beta$  (sauf pour la loi normale avec lien identité), et n'ont en général pas de solution explicite : il faut une méthode numérique.

On les résout par la **méthode de Newton** (volume I, section 1.5 : on remplace la courbe par sa tangente ; ici, comme pour la descente de gradient de la section 1.3, on cherche à annuler le gradient, mais en utilisant aussi la courbure), avec une variante : on remplace la dérivée seconde de  $\ell$  par son **espérance**, l'**information de Fisher**  $I(\beta) = \frac{1}{\phi} X^\top W X$ , où  $W$  est la matrice diagonale des **poids**

$$W_i = \frac{1}{V(\mu_i)} \left( \frac{\partial \mu_i}{\partial \eta_i} \right)^2.$$

C'est le **score de Fisher** (*Fisher scoring*) : à partir d'une valeur courante  $\beta^{(t)}$ ,

$$\beta^{(t+1)} = \beta^{(t)} + (X^\top W X)^{-1} X^\top W \tilde{e}, \quad \tilde{e}_i = (y_i - \mu_i) \frac{\partial \eta_i}{\partial \mu_i}.$$

Introduisons la **réponse de travail**  $z_i = \eta_i + (y_i - \mu_i) \frac{\partial \eta_i}{\partial \mu_i}$ . Un calcul direct montre que la mise à jour s'écrit

$$\boxed{\beta^{(t+1)} = (X^\top W X)^{-1} X^\top W z}$$

c'est-à-dire **exactement la solution des moindres carrés pondérés** de la régression de  $z$  sur  $X$  avec les poids  $W$ . À chaque itération, on recalcule  $\mu$ ,  $W$  et  $z$  avec la valeur courante de  $\beta$ , puis on résout une régression pondérée : c'est l'algorithme des **moindres carrés repondérés itérés** (*iteratively reweighted least squares, IRLS*). Avec le lien canonique, Fisher scoring et Newton-Raphson coïncident.

**Lecture intuitive de  $z$  et  $W$ .**  $z$  est « la réponse que l'on aurait observée si le modèle était linéaire autour du point courant » : on prend la prédiction  $\eta_i$  et l'on corrige par l'écart  $y_i - \mu_i$ , converti à l'échelle du prédicteur.  $W_i$  dit **à quel point on fait confiance** à l'observation  $i$  : un poids élevé là où la variance est faible. Les observations les plus informatives pèsent le plus.

**Un calcul à la main.** Prenons le plus petit exemple possible : quatre clients, dont trois ont racheté,  $y = (1, 0, 1, 1)$ , et un modèle logistique **sans variable** :  $\eta_i = \beta$  pour tous. On sait que la solution

est  $\hat{p} = 3/4$ , donc  $\hat{\beta} = \log \frac{3/4}{1/4} = \log 3 \approx 1,0986$ . Voyons IRLS la retrouver. Pour la loi de Bernoulli avec lien logit,  $\frac{\partial \mu}{\partial \eta} = \mu(1 - \mu)$ , donc  $W = \mu(1 - \mu)$  et  $z_i = \eta + \frac{y_i - \mu}{\mu(1 - \mu)}$ .

- **Départ**  $\beta^{(0)} = 0$  :  $\mu = 0,5$ ,  $W = 0,25$ . Les réponses de travail valent  $z = 0 + \frac{1-0,5}{0,25} = 2$  pour un  $y = 1$  et  $-2$  pour le  $y = 0$ . Régression pondérée sans variable = moyenne pondérée :  $\beta^{(1)} = \frac{0,25(2 - 2 + 2 + 2)}{4 \times 0,25} = 1,000$ .
- **Itération 2** :  $\mu = \frac{1}{1+e^{-1}} \approx 0,7311$ ,  $W \approx 0,1966$ ,  $z \approx 2,368$  (pour  $y = 1$ ) et  $\approx -2,718$  (pour  $y = 0$ ), d'où  $\beta^{(2)} = \frac{3 \times 2,368 - 2,718}{4} \approx 1,0963$ .

On s'approche de 1,0986 très vite. Voici le même calcul en code.

```
y = np.array([1, 0, 1, 1])
beta = 0.0
for it in range(1, 6):
    eta = beta                                # modèle sans variable : eta = beta pour tous
    mu = 1 / (1 + np.exp(-eta))
    w = np.full(len(y), mu * (1 - mu))        # un poids par client : (dmu/deta)^2 / V(mu) = mu(1-mu)
    z = eta + (y - mu) / w                    # réponse de travail
    beta = np.sum(w * z) / np.sum(w)
    print(f"itération {it} : mu = {mu:.4f}    W = {w[0]:.4f}    beta = {beta:.6f}")
print("valeur exacte : log(3) =", round(np.log(3), 6))
```

```
itération 1 : mu = 0.5000    W = 0.2500    beta = 1.000000
itération 2 : mu = 0.7311    W = 0.1966    beta = 1.096339
itération 3 : mu = 0.7496    W = 0.1877    beta = 1.098611
itération 4 : mu = 0.7500    W = 0.1875    beta = 1.098612
itération 5 : mu = 0.7500    W = 0.1875    beta = 1.098612
valeur exacte : log(3) = 1.098612
```

Quatre itérations suffisent pour obtenir six décimales exactes : 1,000000, puis 1,096339, puis 1,098611, puis 1,098612. Les deux premières valeurs sont exactement celles de notre calcul à la main (1,000 puis 1,0963). Remarquez la **vitesse de convergence** : le nombre de décimales exactes double presque à chaque pas, signature de la méthode de Newton. Comparez avec la descente de gradient du volume I, dont l'erreur ne diminue que d'un facteur constant à chaque pas.

Écrivons maintenant l'algorithme **en général** : une fonction `irls` qui prend une matrice  $X$ , un vecteur  $y$  et une « famille » (le lien, sa dérivée, la fonction de variance). Nous la réutiliserons aux sections 2.2 et 2.3.

```
from scipy.special import expit

FAMILLES = {
    # lien, lien inverse, dmu/deta (en fonction de eta), fonction de variance V(mu), valeur initiale
    # de mu
    "binomial": dict(lien=lambda m: np.log(m / (1 - m)), inverse=expit,
                    dmu_deta=lambda e: expit(e) * (1 - expit(e)), variance=lambda m: m * (1 - m),
                    init=lambda y: (y + 0.5) / 2),
    "poisson": dict(lien=np.log, inverse=np.exp, dmu_deta=np.exp, variance=lambda m: m,
                    init=lambda y: y + 0.5),
    "gamma_log": dict(lien=np.log, inverse=np.exp, dmu_deta=np.exp, variance=lambda m: m**2,
                      init=lambda y: y),
}

def irls(X, y, famille, tol=1e-10, max_iter=50):
    """Moindres carrés repondérés itérés. Renvoie les coefficients, leur matrice de covariance (phi =
    ↪ 1) et le détail."""
    f = FAMILLES[famille]
    mu = f["init"](y)
    eta = f["lien"](mu)
```

```

for it in range(1, max_iter + 1):
    w = f["dmu_deta"](eta) ** 2 / f["variance"](mu)           # poids W_i
    z = eta + (y - mu) / f["dmu_deta"](eta)                   # réponse de travail z_i
    XtW = X.T * w
    beta = np.linalg.solve(XtW @ X, XtW @ z)                  # régression pondérée de z sur X
    nouveau_eta = X @ beta
    converge = np.max(np.abs(nouveau_eta - eta)) < tol
    eta, mu = nouveau_eta, f["inverse"](nouveau_eta)
    if converge:
        break
w = f["dmu_deta"](eta) ** 2 / f["variance"](mu)
cov = np.linalg.inv((X.T * w) @ X)                           # (X'WX)^-1 : covariance si phi = 1
return dict(beta=beta, cov=cov, mu=mu, eta=eta, w=w, iterations=it)

# Test 1 : l'exemple à quatre clients
un = np.ones((4, 1))
r = irls(un, y.astype(float), "binomial")
print("logistique, 4 clients : beta =", r["beta"].round(6), "en", r["iterations"], "itérations")

# Test 2 : Poisson sans variable sur les vraies données : la solution doit être log(moyenne)
nb = clients["nb_commandes_an"].to_numpy(dtype=float)
r = irls(np.ones((len(nb), 1)), nb, "poisson")
print("Poisson, 2000 clients : beta =", r["beta"].round(6), "| log(moyenne) =",
      round(float(np.log(nb.mean()))), 6, "| itérations :", r["iterations"])

logistique, 4 clients : beta = [1.098612] en 5 itérations
Poisson, 2000 clients : beta = [1.356608] | log(moyenne) = 1.356608 | itérations : 6

```

Dans les deux cas, IRLS retrouve la solution connue à l'avance, en 5 et 6 itérations. Pour Poisson sans variable, la solution doit être  $\hat{\beta} = \log \bar{y}$  (on vérifie :  $\log 3,88 \approx 1,3566$ ), ce qui confirme que notre fonction fait bien ce qu'on attend d'un logiciel.

**Ce que fait vraiment statsmodels.** La fonction `sm.GLM(...).fit()` utilise exactement cet algorithme (IRLS par défaut). Les seules différences sont des détails d'ingénierie : valeurs initiales, critère d'arrêt, calcul numérique plus soigné. À la section 2.2.5, nous comparerons notre fonction à la sienne sur un vrai modèle.

### 2.1.6 Les trois tests

Une fois  $\hat{\beta}$  obtenu, la théorie du maximum de vraisemblance (volume I, section 3.2) donne la précision et les tests, pour des échantillons assez grands :

- **Précision.**  $\widehat{\text{Var}}(\hat{\beta}) \approx \phi (X^T W X)^{-1}$  : c'est exactement la matrice que `irls` renvoie (pour  $\phi = 1$ ), évaluée en  $\hat{\beta}$ . Les **erreurs-types** sont les racines de sa diagonale.
- **Test de Wald** pour un coefficient :  $z_j = \hat{\beta}_j / \text{se}(\hat{\beta}_j)$ , comparé à une loi normale. C'est le test affiché dans toutes les sorties de logiciel. Il est simple, mais peut être trompeur quand l'effet est grand ou l'échantillon petit.
- **Test du rapport de vraisemblance** (RV) pour comparer deux modèles emboîtés :  $2(\ell_1 - \ell_0) \approx \chi_q^2$ , où  $q$  est le nombre de paramètres en plus. Il est en général plus fiable que Wald ; nous l'étudions au 2.4 via la **déviance**.
- **Test du score**, qui ne demande d'ajuster que le modèle réduit : peu utilisé directement, il est à l'origine de nombreux tests classiques (par exemple le khi-deux de Pearson).

Lorsque  $\phi$  est inconnu (Gamma, normale), on l'estime ; lorsque  $\phi = 1$  est imposé (Bernoulli, Poisson), il faut **vérifier** que cette hypothèse est raisonnable : c'est le problème de la **surdispersion**, que nous rencontrerons dès la section 2.3.

**Les pièges du cadre GLM.** (1) L'**indépendance des observations** est supposée : des mesures répétées sur un même client ne la respectent pas (voir les modèles mixtes, section 1.7). (2) Le choix de la famille impose la relation **variance-moyenne** : une mauvaise famille donne des erreurs-types fausses, même si les coefficients sont corrects. (3) Les coefficients ne se lisent **jamais sur l'échelle de  $\mu$**  mais sur celle du prédicteur : log-cote pour la logistique, log-taux pour Poisson.

### À retenir

- Un GLM = **loi** de la famille exponentielle + **prédicteur linéaire**  $x^\top \beta$  + **lien**  $g(\mu) = x^\top \beta$ .
- Pour la famille exponentielle,  $E[Y] = b'(\theta)$  et  $\text{Var}(Y) = \phi V(\mu)$  : la famille fixe la **forme de la variance** (constante,  $\mu$ ,  $\mu^2$ ,  $\mu(1 - \mu)$ ...).
- Le lien fixe l'échelle où les effets sont additifs : identité (additif), log (multiplicatif), logit (multiplicatif sur les cotes).
- L'estimation est un **maximum de vraisemblance** résolu par **IRLS** : une suite de régressions pondérées. Les erreurs-types viennent de  $(X^\top W X)^{-1}$ .
- Trois tests (Wald, rapport de vraisemblance, score) ; le rapport de vraisemblance, via la déviance, est le plus fiable.

## 2.2 Régression logistique

**Intuition.** Yasmine envoie un bon de bienvenue à la moitié de ses nouveaux clients, tirés au sort. Douze mois plus tard, elle observe pour chaque client un seul bit d'information : a-t-il **racheté** (1) ou non (0) ? Elle veut savoir de combien l'offre augmente les chances de rachat, et comment les autres caractéristiques (âge, canal d'acquisition, satisfaction) y contribuent. La régression logistique modélise la **probabilité** de racheter, mais pas directement : elle modélise une transformation de cette probabilité, la **cote**, qui vit sur toute la droite réelle et qui se prête à un modèle linéaire.

### 2.2.1 De la probabilité à la cote

Si un événement a la probabilité  $p$ , sa **cote** (*odds*) est  $\frac{p}{1-p}$  : le rapport entre les chances que l'événement arrive et les chances qu'il n'arrive pas. Une probabilité de 0,75 donne une cote de 3 (« trois contre un »). Le **logit** est le logarithme de la cote :  $\text{logit}(p) = \log \frac{p}{1-p}$ .

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import expit

clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram",
↪ "Site"]) # référence : Boutique

p = np.array([0.05, 0.10, 0.25, 0.50, 0.60, 0.75, 0.90, 0.95])
tab = pd.DataFrame({"probabilité p": p, "cote p/(1-p)": p / (1 - p), "logit = log(cote)": np.log(p /
↪ (1 - p))})
print(tab.round(3).to_string(index=False))
```

| probabilité p | cote p/(1-p) | logit = log(cote) |
|---------------|--------------|-------------------|
| 0.05          | 0.053        | -2.944            |
| 0.10          | 0.111        | -2.197            |
| 0.25          | 0.333        | -1.099            |
| 0.50          | 1.000        | 0.000             |
| 0.60          | 1.500        | 0.405             |
| 0.75          | 3.000        | 1.099             |
| 0.90          | 9.000        | 2.197             |
| 0.95          | 19.000       | 2.944             |

Lisez le tableau en trois temps. Une probabilité de 0,5 correspond à une cote de 1 (« une chance contre une ») et à un logit de 0. Les probabilités symétriques ont des logits **opposés** : 0,25 donne -1,099 et 0,75 donne +1,099 ; 0,05 donne -2,944 et 0,95 donne +2,944. Enfin, le logit **étire les extrémités** : passer de 90 % à 95 % ne change presque rien en probabilité (5 points), mais la cote passe de 9 à 19 et le logit de 2,2 à 2,9. C'est précisément ce qui permet d'écrire un modèle linéaire sans jamais sortir de ]0, 1[.

Le logit est la fonction qui transforme une probabilité ( $]0, 1[$ ) en un nombre réel quelconque ( $] - \infty, +\infty[$ ) ; sa réciproque, la fonction **logistique**  $\text{expit}(\eta) = \frac{1}{1 + e^{-\eta}}$ , fait le chemin inverse. C'est elle qui donne à la courbe sa forme de S.

### 2.2.2 Le modèle

Le client  $i$  a pour caractéristiques  $x_i$  et un résultat  $Y_i \in \{0, 1\}$ . Le modèle de **régression logistique** (cas Bernoulli + lien logit, le lien canonique du tableau de 2.1.3) pose

$$Y_i \sim \text{Bernoulli}(p_i), \quad \text{logit}(p_i) = \log \frac{p_i}{1 - p_i} = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip},$$

soit, de façon équivalente,  $p_i = \frac{1}{1 + e^{-x_i^T \beta}}$ .

**Une deuxième lecture : la variable latente.** Imaginons que chaque client ait une « envie de racheter » continue  $Y_i^* = x_i^T \beta + \varepsilon_i$ , que nous n'observons pas ; il rachète si cette envie dépasse zéro. Si  $\varepsilon_i$  suit la **loi logistique** (de fonction de répartition  $F(t) = 1/(1 + e^{-t})$ ), alors  $P(Y_i = 1) = P(\varepsilon_i > -x_i^T \beta) = F(x_i^T \beta)$  par symétrie : on retrouve exactement le modèle logistique. (Si  $\varepsilon_i$  était normale, on obtiendrait le modèle **probit**, très voisin.) Cette image aide à comprendre pourquoi la courbe est un S : près de  $p = 0,5$ , un petit changement de l'envie fait basculer beaucoup de clients ; près de 0 ou de 1, il en faut beaucoup plus.

**Comment lire un coefficient ?** Si la variable  $x_j$  augmente d'une unité, toutes choses égales par ailleurs, le logit augmente de  $\beta_j$ , c'est-à-dire que la **cote est multipliée par  $e^{\beta_j}$**  :  $e^{\beta_j}$  est un **rapport de cotes** (*odds ratio*, OR). Un OR de 1 signifie « pas d'effet » ; supérieur à 1, l'effet favorise l'événement ; inférieur à 1, il le défavorise. Mais sur l'échelle de la **probabilité**, l'effet n'est pas constant :

```
beta = 0.5 # un effet de +0,5 sur le logit : OR = exp(0,5) ≈ 1,65
p0 = np.array([0.02, 0.10, 0.30, 0.50, 0.70, 0.90, 0.98])
p1 = expit(np.log(p0 / (1 - p0)) + beta)
print("OR =", round(float(np.exp(beta)), 3))
print(pd.DataFrame({"p avant": p0, "p après": p1, "gain (points)": 100 * (p1 -
↪ p0})).round(3).to_string(index=False))
```

```
OR = 1.649
p avant  p après  gain (points)
0.02     0.033    1.255
```



|      |       |        |
|------|-------|--------|
| 0.10 | 0.155 | 5.483  |
| 0.30 | 0.414 | 11.404 |
| 0.50 | 0.622 | 12.246 |
| 0.70 | 0.794 | 9.369  |
| 0.90 | 0.937 | 3.686  |
| 0.98 | 0.988 | 0.777  |

Un même effet de +0,5 sur le logit (un OR de 1,649) produit un gain de **12,2 points** quand la probabilité de départ est de 50 %, mais seulement 5,5 points à 10 %, 3,7 points à 90 % et 1,3 point à 2 %. L'effet en points de pourcentage est donc **maximal au milieu** et s'écroule vers 0 et 1 (pour un petit effet, il vaut environ  $\beta p(1 - p)$  : ici  $0,5 \times 0,25 = 12,5$  à  $p = 0,5$ , très proche de 12,2). Conclusion pratique : **un coefficient logistique est constant sur l'échelle du logit, pas sur celle de la probabilité** ; pour parler en points de pourcentage, il faut préciser *pour quel client*.

### 2.2.3 Un premier modèle : l'offre de bienvenue seule

Commençons par le modèle le plus simple, une seule variable binaire : `offre_bienvenue`. Avant de l'ajuster, calculons tout à la main à partir du tableau croisé.

```
croise = pd.crosstab(clients["offre_bienvenue"], clients["rachat_12m"])
print(croise)
sans, avec = croise.loc[0], croise.loc[1]
p_sans, p_avec = sans.sum() / sans.sum(), avec[1] / avec.sum()
cote_sans, cote_avec = sans[1] / sans[0], avec[1] / avec[0]
print()
print(f"sans offre : {p_sans:.4f} ont racheté | cote = {cote_sans:.4f} | logit =
↳ {np.log(cote_sans):.4f}")
print(f"avec offre : {p_avec:.4f} ont racheté | cote = {cote_avec:.4f} | logit =
↳ {np.log(cote_avec):.4f}")
print(f"rapport de cotes (a*d)/(b*c) = {cote_avec / cote_sans:.4f} | log = {np.log(cote_avec /
↳ cote_sans):.4f}")
print(f"différence de risque = {p_avec - p_sans:.4f} | risque relatif = {p_avec / p_sans:.4f}")

m_offre = smf.glm("rachat_12m ~ offre_bienvenue", clients, family=sm.families.Binomial()).fit()
print()
print(m_offre.params.round(4).to_string())
print("exp(coefficient de l'offre) =", round(float(np.exp(m_offre.params["offre_bienvenue"])), 4))
```

| rachat_12m      | 0   | 1   |
|-----------------|-----|-----|
| offre_bienvenue |     |     |
| 0               | 544 | 441 |
| 1               | 437 | 578 |

```
sans offre : 0.4477 ont racheté | cote = 0.8107 | logit = -0.2099
avec offre : 0.5695 ont racheté | cote = 1.3227 | logit = 0.2796
rapport de cotes (a*d)/(b*c) = 1.6316 | log = 0.4895
différence de risque = 0.1217 | risque relatif = 1.2719
```

```
Intercept          -0.2099
offre_bienvenue      0.4895
exp(coefficient de l'offre) = 1.6316
```

Sans offre, 441 clients sur 985 ont racheté (44,77 %) ; avec l'offre, 578 sur 1 015 (56,95 %). Les cotes correspondantes sont 0,8107 et 1,3227, d'où un rapport de cotes de  $1,3227/0,8107 = 1,6316$ , ou encore  $(578 \times 544)/(437 \times 441)$  : le « produit en croix » du tableau. Le modèle logistique donne Intercept = -0,2099, qui est le logit du groupe sans offre, et `offre_bienvenue` = 0,4895, qui est le logarithme du rapport de cotes ; son exponentielle est 1,6316. L'égalité est **exacte**, pas approchée.

**Pourquoi retrouve-t-on exactement le tableau ?** Avec une seule variable binaire, le modèle a **deux** paramètres  $(\beta_0, \beta_1)$  pour **deux** groupes : il est dit **saturé**. Les équations du score, qui pour le lien canonique s'écrivent simplement  $\sum_i (y_i - \hat{p}_i) x_{ij} = 0$  (voir 2.2.5), imposent alors que la proportion prédite de chaque groupe égale la proportion observée. Donc  $\hat{\beta}_0 = \text{logit}(\hat{p}_{\text{sans}})$  et  $\hat{\beta}_0 + \hat{\beta}_1 = \text{logit}(\hat{p}_{\text{avec}})$ , d'où  $\hat{\beta}_1 = \text{logarithme du rapport de cotes du tableau}$ .

**Rapport de cotes  $\neq$  risque relatif.** Les clients à qui l'on a envoyé l'offre ont des cotes de rachat multipliées par environ 1,6 ; pourtant la probabilité de rachat n'est « multipliée » que par environ 1,27. Le rapport de cotes **exagère** le risque relatif quand l'événement est fréquent (ici, environ la moitié des clients rachètent). Dire « l'offre rend le rachat 63 % plus probable » serait faux : elle le rend plus probable de **12 points de pourcentage**, ou de 27 % en valeur relative. Quand l'événement est rare (moins de 10 %), l'OR et le RR sont presque identiques ; sinon, il faut les distinguer.

### 2.2.4 Le modèle complet

Ajoutons l'âge et le canal d'acquisition. Avec statsmodels, une formule à la R suffit ; la catégorie de référence du canal est la boutique (nous l'avons fixée en tête de liste).

```
modele = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
  ↪ family=sm.families.Binomial()).fit()
print(modele.summary().tables[1])
print()
print("log-vraisemblance :", round(modele.llf, 2), "| déviance :", round(modele.deviance, 2), "| AIC
  ↪ :", round(modele.aic, 2), "| n =", int(modele.nobs))
```

|                    | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|--------------------|---------|---------|--------|-------|--------|--------|
| Intercept          | 0.5885  | 0.185   | 3.175  | 0.001 | 0.225  | 0.952  |
| canal[T.Instagram] | -0.4630 | 0.116   | -4.008 | 0.000 | -0.689 | -0.237 |
| canal[T.Site]      | -0.1677 | 0.120   | -1.402 | 0.161 | -0.402 | 0.067  |
| offre_bienvenue    | 0.4933  | 0.091   | 5.423  | 0.000 | 0.315  | 0.672  |
| age                | -0.0155 | 0.004   | -3.562 | 0.000 | -0.024 | -0.007 |

log-vraisemblance : -1355.42 | déviance : 2710.83 | AIC : 2720.83 | n = 2000

Lecture ligne à ligne (les coefficients sont des log-cotes) :

- offre\_bienvenue : +0,493 (erreur-type 0,091,  $z = 5,42$ ) : l'offre augmente nettement la cote de rachat.
- canal[T.Instagram] : -0,463 ( $z = -4,01$ ) : à âge et offre fixés, les clients acquis par Instagram rachètent moins que ceux de la boutique.
- canal[T.Site] : -0,168 ( $p = 0,161$ ) : on ne peut pas distinguer le site de la boutique.
- age : -0,0155 par année ( $z = -3,56$ ) : les clients plus âgés rachètent un peu moins.
- Intercept : 0,5885 est le logit d'un client de la boutique, sans offre, **d'âge 0** : une extrapolation sans signification (on gagnerait à *centrer* l'âge, par exemple en soustrayant 36).

Sous le tableau, trois nombres : la log-vraisemblance -1 355,42, la déviance 2 710,83 et l'AIC 2 720,83. Remarquez que, pour des données binaires, la déviance est exactement  $-2\ell = 2 \times 1\,355,42$  (la vraisemblance du modèle saturé vaut 0), et que l'AIC est  $-2\ell + 2k = 2\,710,83 + 2 \times 5 = 2\,720,83$  avec  $k = 5$  paramètres. Nous reviendrons sur la déviance à la section 2.4.

Les coefficients sont des **log-cotes**, peu parlants. On les convertit en rapports de cotes, avec leur intervalle de confiance à 95 % (on prend l'exponentielle des bornes de l'intervalle du coefficient).

```
ic = modele.conf_int()
rapports = pd.DataFrame({"OR": np.exp(modele.params), "IC95 bas": np.exp(ic[0]), "IC95 haut":
    np.exp(ic[1]), "p-valeur": modele.pvalues})
print(rapports.round(3).to_string())
print()
print("OR pour +10 ans d'âge :", round(float(np.exp(10 * modele.params["age"])), 3))
```

|                    | OR    | IC95 bas | IC95 haut | p-valeur |
|--------------------|-------|----------|-----------|----------|
| Intercept          | 1.801 | 1.253    | 2.590     | 0.001    |
| canal[T.Instagram] | 0.629 | 0.502    | 0.789     | 0.000    |
| canal[T.Site]      | 0.846 | 0.669    | 1.069     | 0.161    |
| offre_bienvenue    | 1.638 | 1.370    | 1.957     | 0.000    |
| age                | 0.985 | 0.976    | 0.993     | 0.000    |

OR pour +10 ans d'âge : 0.856

Les rapports de cotes se lisent directement :

- **Offre** : OR = 1,64, intervalle à 95 % de 1,37 à 1,96. L'offre multiplie la cote de rachat par un facteur compris, avec 95 % de confiance, entre 1,4 et 2,0.
- **Instagram** par rapport à la boutique : OR = 0,63 (de 0,50 à 0,79) : la cote de rachat est environ **37 % plus basse**.
- **Site** par rapport à la boutique : OR = 0,85, intervalle de 0,67 à 1,07 : l'intervalle contient 1, aucun effet net n'est démontré.
- **Âge** : OR = 0,985 par année ; pour **dix ans** de plus,  $e^{10\beta} = 0,856$  : la cote est réduite d'environ 14 %.

L'intervalle d'un OR n'est pas symétrique autour de l'OR : c'est l'exponentielle d'un intervalle symétrique pour le log-OR (intervalle de Wald, 2.1.6).

### 2.2.5 L'estimation à la main : IRLS contre statsmodels

Reprenons la fonction `irls` écrite à la section 2.1.5, et appliquons-la à ce modèle. Il suffit de construire la matrice  $X$  (une colonne de 1, puis les variables ; `patsy` fait ce travail comme `statsmodels`).

```
from patsy import dmatrices

Y, X = dmatrices("rachat_12m ~ offre_bienvenue + age + canal", clients, return_type="dataframe")
r = irls(X.to_numpy(), Y.to_numpy().ravel(), "binomial")
comparaison = pd.DataFrame({
    "coef (IRLS main)": r["beta"], "coef (statsmodels)": modele.params.to_numpy(),
    "se (IRLS main)": np.sqrt(np.diag(r["cov"])), "se (statsmodels)": modele.bse.to_numpy(),
}, index=X.columns)
print(comparaison.round(6).to_string())
print()
print("itérations de notre IRLS :", r["iterations"], "| écart maximal sur les coefficients :",
    f"{np.max(np.abs(r['beta'] - modele.params.to_numpy())):.2e}")
print("écart maximal sur les erreurs-types :", f"{np.max(np.abs(np.sqrt(np.diag(r['cov'])) -
    modele.bse.to_numpy())):.2e}")
```

|                    | coef (IRLS main) | coef (statsmodels) | se (IRLS main) | se (statsmodels) |
|--------------------|------------------|--------------------|----------------|------------------|
| Intercept          | 0.588499         | 0.588499           | 0.185372       | 0.185372         |
| canal[T.Instagram] | -0.462955        | -0.462955          | 0.115509       | 0.115509         |
| canal[T.Site]      | -0.167719        | -0.167719          | 0.119619       | 0.119619         |
| offre_bienvenue    | 0.493258         | 0.493258           | 0.090953       | 0.090953         |
| age                | -0.015498        | -0.015498          | 0.004351       | 0.004351         |

itérations de notre IRLS : 5 | écart maximal sur les coefficients : 3.13e-14  
 écart maximal sur les erreurs-types : 4.44e-09

Les deux méthodes donnent les **mêmes coefficients** (écart maximal de l'ordre de  $10^{-14}$ ) et les **mêmes erreurs-types** (écart maximal de l'ordre de  $10^{-9}$ ), après 5 itérations seulement. Notre petite fonction, écrite à partir de la théorie de la section 2.1, reproduit donc fidèlement ce que fait le logiciel : il n'y a pas de magie derrière `.fit()`. Les erreurs-types viennent de la matrice  $(X^T W X)^{-1}$  calculée au point final.

**Une propriété du lien canonique.** Pour la régression logistique, l'équation du score (2.1.5) se simplifie : comme  $\frac{\partial \mu_i}{\partial \eta_i} = p_i(1 - p_i) = V(p_i)$ , on obtient  $\sum_i (y_i - \hat{p}_i) x_{ij} = 0$  pour chaque colonne  $x_j$ , y compris la colonne de 1. Deux conséquences : (1) la somme des probabilités prédites est égale au nombre de « oui » observés ; (2) les résidus  $y_i - \hat{p}_i$  sont **orthogonaux** à chaque variable explicative. Vérifions-le.

```
residus = clients["rachat_12m"] - modele.fittedvalues
print("somme des résidus (y - p̂)           :", round(float(residus.sum()), 8))
print("somme des p̂ =", round(float(modele.fittedvalues.sum()), 3), "| nombre de 1 observés =",
      ↪ int(clients["rachat_12m"].sum()))
for col in ["offre_bienvenue", "age"]:
    print(f"somme des résidus × {col:16s}:", round(float((residus * clients[col]).sum()), 6))

somme des résidus (y - p̂)           : 0.0
somme des p̂ = 1019.0 | nombre de 1 observés = 1019
somme des résidus × offre_bienvenue : 0.0
somme des résidus × age              : 0.0
```

Les sommes de résidus sont nulles (à la précision de l'arrondi) et la somme des probabilités prédites, 1 019, est **exactement** le nombre de clients qui ont racheté. C'est un excellent réflexe de vérification : si, après un ajustement logistique **avec constante**, ces deux nombres diffèrent, c'est que l'algorithme n'a pas convergé.

## 2.2.6 Probabilités et effets marginaux

Un rapport de cotes dit « de combien la cote est multipliée » ; mais Yasmine veut savoir *de combien de points de pourcentage* l'offre augmente la probabilité de rachat. Deux façons de répondre.

**Pour un profil donné.** Prenons un client de 36 ans acquis par Instagram, avec ou sans l'offre.

```
profil = pd.DataFrame({"offre_bienvenue": [0, 1], "age": [36, 36],
                      "canal": pd.Categorical(["Instagram", "Instagram"], categories=["Boutique",
                                             ↪ "Instagram", "Site"])}))
p_profil = modele.predict(profil)
b = modele.params
a_la_main = expit(b["Intercept"] + b["canal[T.Instagram]"] + 36 * b["age"] + b["offre_bienvenue"] *
                  ↪ np.array([0, 1]))
print("probabilités prédites (sans offre, avec offre) :", p_profil.round(4).to_numpy(), "| à la main
      ↪ :", a_la_main.round(4))
print("gain en points de pourcentage pour ce profil   :", round(100 * float(p_profil.iloc[1] -
      ↪ p_profil.iloc[0]), 2))
```

```
probabilités prédites (sans offre, avec offre) : [0.3936 0.5152] | à la main : [0.3936 0.5152]
gain en points de pourcentage pour ce profil   : 12.17
```

**En moyenne sur tous les clients** (effet marginal moyen). On calcule, pour *chaque* client, sa probabilité prédite en lui attribuant l'offre, puis sans l'offre, et l'on moyenne la différence. Comme l'offre a été **attribuée au hasard**, cette quantité estime directement l'effet causal moyen de l'offre sur la probabilité de rachat.

```
avec_offre = clients.assign(offre_bienvenue=1)
sans_offre = clients.assign(offre_bienvenue=0)
```

```

effet_moyen = (modele.predict(avec_offre) - modele.predict(sans_offre)).mean()
print(f"effet marginal moyen de l'offre (modèle) : {100 * effet_moyen:.2f} points")
print(f"différence brute des taux de rachat      : {100 * (p_avec - p_sans):.2f} points")

# effet marginal moyen de l'âge : +1 an
effet_age = (modele.predict(clients.assign(age=clients["age"] + 1)) - modele.predict(clients)).mean()
print(f"effet marginal moyen d'un an de plus      : {100 * effet_age:.3f} point (soit {100 * 10 *
↪ effet_age:.2f} points pour 10 ans)")

# contrôle avec la fonction de statsmodels (modèle Logit)
logit = smf.logit("rachat_12m ~ offre_bienvenue + age + canal", clients).fit(dis=0)
print()
print(logit.get_margeff(at="overall", dummy=True).summary_frame().round(4).to_string())

```

```

effet marginal moyen de l'offre (modèle) : 12.07 points
différence brute des taux de rachat      : 12.17 points
effet marginal moyen d'un an de plus      : -0.376 point (soit -3.76 points pour 10 ans)

```

|                    | dy/dx   | Std. Err. | z       | Pr(> z ) | Conf. Int. Low | Conf. Int. Hi. |
|--------------------|---------|-----------|---------|----------|----------------|----------------|
| canal[T.Instagram] | -0.1126 | 0.0277    | -4.0625 | 0.0000   | -0.1669        | -0.0583        |
| canal[T.Site]      | -0.0405 | 0.0287    | -1.4108 | 0.1583   | -0.0967        | 0.0158         |
| offre_bienvenue    | 0.1207  | 0.0220    | 5.4768  | 0.0000   | 0.0775         | 0.1640         |
| age                | -0.0038 | 0.0010    | -3.6063 | 0.0003   | -0.0058        | -0.0017        |

Pour ce profil (Instagram, 36 ans), l'offre fait passer la probabilité de rachat de 39,4 % à 51,5 %, soit un gain de **12,2 points** (le calcul à la main coïncide avec predict). En moyenne sur les 2 000 clients, l'effet marginal de l'offre est de **12,07 points**, très proche de la différence brute des taux (12,17 points) : c'est normal puisque l'offre a été tirée au sort. La fonction get\_margeff de statsmodels donne le même chiffre (0,1207) avec un intervalle de confiance de **7,8 à 16,4 points**. Pour l'âge, un an de plus réduit la probabilité de rachat d'environ **0,38 point** en moyenne (0,0038 dans le tableau de statsmodels), soit environ 3,8 points pour dix ans. Notez que ces effets, exprimés en points, sont **moyens** : ils varient d'un client à l'autre (2.2.2).

**Pourquoi la différence brute et l'effet du modèle sont proches.** Quand le traitement est attribué au hasard, il est (en moyenne) indépendant de l'âge et du canal : ajuster pour ces variables précise l'estimation, mais ne la déplace pas beaucoup. Dans une étude **observationnelle** (où les clients choisissent), les deux chiffres pourraient être très différents ; c'est le sujet du chapitre 7 (inférence causale).

### 2.2.7 Utiliser plus d'information : les notes de l'enquête

Nous avons, pour environ 60 % des clients, deux notes issues du questionnaire de satisfaction : note\_produits (moyenne des questions 1 à 4) et note\_service (questions 5 à 8). Ajoutons-les au modèle, **sur les répondants seulement** (les clients qui n'ont pas répondu n'ont pas de notes).

```

enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
enquete["note_produits"] = enquete[["q1", "q2", "q3", "q4"]].mean(axis=1)
enquete["note_service"] = enquete[["q5", "q6", "q7", "q8"]].mean(axis=1)
repondants = clients.merge(enquete[["id_client", "note_produits", "note_service"]], on="id_client")

m_base = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", repondants,
↪ family=sm.families.Binomial()).fit()
m_riche = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service",
↪ repondants,
                    family=sm.families.Binomial()).fit()
ic = m_riche.conf_int()
print(pd.DataFrame({"coef": m_riche.params, "OR": np.exp(m_riche.params), "IC95 bas": np.exp(ic[0]),
↪ "IC95 haut": np.exp(ic[1]),
                    "p": m_riche.pvalues}).round(3).to_string())

```

```
print()
print("répondants :", int(m_riche.nobs), "| AIC modèle de base :", round(m_base.aic, 1), "| AIC modèle
↳ avec notes :", round(m_riche.aic, 1))
print("écart-types des notes :", repondants[["note_produits",
↳ "note_service"]].std().round(2).to_dict())
print("OR de l'offre, sur ces mêmes répondants : sans les notes =",
↳ round(float(np.exp(m_base.params["offre_bienvenue"])), 3), "| avec les notes =",
↳ round(float(np.exp(m_riche.params["offre_bienvenue"])), 3))
```

|                    | coef   | OR    | IC95 bas | IC95 haut | p     |
|--------------------|--------|-------|----------|-----------|-------|
| Intercept          | -3.626 | 0.027 | 0.010    | 0.069     | 0.000 |
| canal[T.Instagram] | -0.474 | 0.622 | 0.458    | 0.845     | 0.002 |
| canal[T.Site]      | -0.244 | 0.783 | 0.570    | 1.075     | 0.131 |
| offre_bienvenue    | 0.607  | 1.834 | 1.442    | 2.334     | 0.000 |
| age                | -0.018 | 0.983 | 0.971    | 0.994     | 0.003 |
| note_produits      | 0.764  | 2.148 | 1.781    | 2.589     | 0.000 |
| note_service       | 0.420  | 1.522 | 1.285    | 1.803     | 0.000 |

```
répondants : 1212 | AIC modèle de base : 1652.3 | AIC modèle avec notes : 1545.2
écart-types des notes : {'note_produits': 0.69, 'note_service': 0.73}
OR de l'offre, sur ces mêmes répondants : sans les notes = 1.749 | avec les notes = 1.834
```

Sur les 1 212 répondants, les deux notes sont fortement associées au rachat : un point de plus sur la **note « produits »** multiplie la cote par **2,15** (de 1,78 à 2,59), un point de plus sur la **note « service »** par **1,52** (de 1,29 à 1,80). Comme les notes n'ont pas la même dispersion (écarts-types 0,69 et 0,73), comparons à écart-type égal :  $e^{0,764 \times 0,69} \approx 1,69$  pour les produits,  $e^{0,420 \times 0,73} \approx 1,36$  pour le service. Les deux comptent, avec un avantage à la qualité des produits. L'AIC chute de 1 652,3 à 1 545,2 : le gain est considérable (plus de 100 points).

Un détail instructif : l'OR de l'offre passe de **1,749** (sans les notes) à **1,834** (avec les notes), alors que l'offre est tirée au hasard et n'est donc pas corrélée aux notes : aucun biais de confusion ne peut expliquer ce changement. C'est un phénomène classique, la **non-collapsibilité** du rapport de cotes : quand on ajoute au modèle une variable qui prédit bien le résultat, l'OR de la variable de traitement change, même sans confusion, parce que l'on passe d'un OR *moyenné sur la population* à un OR *conditionnel aux notes*. Ce n'est pas le cas des différences de probabilités ou des coefficients d'une régression linéaire. Une raison de plus de préférer les effets marginaux en points de pourcentage pour communiquer.

### 2.2.8 Évaluer un modèle de classement : matrice de confusion, ROC, AUC, calibration

Un modèle logistique rend une **probabilité**. On peut s'en servir de deux façons : (a) comme **score** pour classer les clients (qui relancer en priorité ?), (b) comme **règle de décision** : on prédit « rachète » si  $\hat{p}$  dépasse un seuil. Les deux se jugent différemment.

**Matrice de confusion.** À un seuil donné, on compare la prédiction (0/1) au résultat réel.

```
y = repondants["rachat_12m"].to_numpy()
score = m_riche.fittedvalues.to_numpy()

def matrice(y, score, seuil):
    pred = (score >= seuil).astype(int)
    vp, fp = int(((pred == 1) & (y == 1)).sum()), int(((pred == 1) & (y == 0)).sum())
    fn, vn = int(((pred == 0) & (y == 1)).sum()), int(((pred == 0) & (y == 0)).sum())
    return vp, fp, fn, vn

for seuil in (0.5, 0.4, 0.6):
    vp, fp, fn, vn = matrice(y, score, seuil)
    print(f"seuil {seuil} : VP={vp} FP={fp} FN={fn} VN={vn} | exactitude={(vp + vn) / len(y):.3f} | "
          f"sensibilité={vp / (vp + fn):.3f} | spécificité={vn / (vn + fp):.3f} | précision={vp / (vp "
          f"+ fp):.3f}")
```



```

seuil 0.5 : VP=406 FP=228 FN=206 VN=372 | exactitude=0.642 | sensibilité=0.663 | spécificité=0.620 |
↳ précision=0.640
seuil 0.4 : VP=512 FP=353 FN=100 VN=247 | exactitude=0.626 | sensibilité=0.837 | spécificité=0.412 |
↳ précision=0.592
seuil 0.6 : VP=270 FP=112 FN=342 VN=488 | exactitude=0.625 | sensibilité=0.441 | spécificité=0.813 |
↳ précision=0.707

```

Parmi les 1 212 répondants, 612 ont racheté (50,5 %) : un modèle qui répondrait toujours « oui » aurait une exactitude de 50,5 %. Au seuil de 0,5, la matrice donne 406 vrais positifs, 228 faux positifs, 206 faux négatifs et 372 vrais négatifs : exactitude 64,2 %, sensibilité 66,3 % (parmi ceux qui rachètent, 66 % sont repérés), spécificité 62,0 %. En **baissant le seuil à 0,4**, on repère davantage de rachats (sensibilité 83,7 %) au prix de davantage de fausses alertes (spécificité 41,2 %) ; en **montant à 0,6**, l'inverse (sensibilité 44,1 %, spécificité 81,3 %, mais précision de 70,7 %). L'exactitude, elle, reste presque la même (entre 62,5 % et 64,2 %) : c'est un résumé qui **cache** le compromis. Le bon seuil dépend du coût de chaque type d'erreur.

**Courbe ROC et AUC.** Plutôt que de choisir un seuil, on regarde **tous** les seuils à la fois : la courbe ROC trace la sensibilité (taux de vrais positifs) en fonction de 1 – spécificité (taux de faux positifs). L'**AUC** (*area under the curve*) est l'aire sous cette courbe. Elle a une interprétation très parlante : c'est la **probabilité qu'un client pris au hasard parmi ceux qui ont racheté ait un score plus élevé qu'un client pris au hasard parmi ceux qui n'ont pas racheté** (avec une demi-part pour les ex aequo). Vérifions-le en calculant l'AUC de trois façons.

```

from sklearn.metrics import roc_auc_score

def auc_paires(y, s):
    pos, neg = s[y == 1], s[y == 0]
    return (pos[:, None] > neg[None, :]).mean() + 0.5 * (pos[:, None] == neg[None, :]).mean()

# courbe ROC « à la main » : on trie les clients par score décroissant et on cumule
ordre = np.argsort(-score)
y_trie = y[ordre]
tpr = np.concatenate([[0], np.cumsum(y_trie) / y.sum()])
fpr = np.concatenate([[0], np.cumsum(1 - y_trie) / (1 - y).sum()])
auc_trapezes = np.trapezoid(tpr, fpr)

print("AUC par comparaison de paires :", round(float(auc_paires(y, score)), 4))
print("AUC par aire sous la courbe   :", round(float(auc_trapezes), 4))
print("AUC de scikit-learn           :", round(float(roc_auc_score(y, score)), 4))
print("AUC du modèle de base (sans les notes) :", round(float(roc_auc_score(y, m_base.fittedvalues)),
↳ 4))

```

```

AUC par comparaison de paires : 0.6975
AUC par aire sous la courbe   : 0.6975
AUC de scikit-learn           : 0.6975
AUC du modèle de base (sans les notes) : 0.599

```

Les trois calculs donnent la **même valeur**, 0,6975 : l'aire sous la courbe ROC est bien la probabilité qu'un client qui a racheté ait un score supérieur à celui d'un client qui n'a pas racheté. Un modèle sans pouvoir de classement aurait une AUC de 0,5 ; un modèle parfait, 1. Le modèle sans les notes de l'enquête n'atteint que **0,599** : les notes font passer l'AUC de 0,60 à 0,70, un gain sensible. Les deux valeurs restent modestes : prédire un comportement individuel est difficile (et notre jeu simulé contient réellement beaucoup de hasard).

**Calibration.** Un bon classement ne suffit pas toujours : si l'on veut utiliser  $\hat{p}$  comme une **vraie probabilité** (par exemple pour calculer un gain espéré), il faut qu'elle soit *calibrée* : parmi les clients à qui le modèle donne 70 %, environ 70 % doivent avoir racheté. On le vérifie en regroupant les clients par dixièmes de score.



```

dec = pd.qcut(score, 10, labels=False)
calib = pd.DataFrame({"p prédite": score, "observé": y, "dixième": dec}).groupby("dixième").agg(
    p_predite=("p prédite", "mean"), observe=("observé", "mean"), n=("observé", "size"))
print(calib.round(3).to_string())

fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
ax = axes[0]
ordre_b = np.argsort(-m_base.fittedvalues.to_numpy())
yb = y[ordre_b]
tpr_b = np.concatenate([[0], np.cumsum(yb) / y.sum()]); fpr_b = np.concatenate([[0], np.cumsum(1 -
    ↪ yb) / (1 - y).sum())])
ax.plot(fpr, tpr, color=BLEU, lw=2); ax.plot(fpr_b, tpr_b, color=ORANGE, lw=2); ax.plot([0, 1], [0,
    ↪ 1], color="#898781", ls=":")
ax.text(0.45, 0.30, f"avec les notes : AUC = {roc_auc_score(y, score):.2f}", color=BLEU, fontsize=9)
ax.text(0.45, 0.22, f"sans les notes : AUC = {roc_auc_score(y, m_base.fittedvalues):.2f}",
    ↪ color=ORANGE, fontsize=9)
ax.text(0.45, 0.14, "hasard : AUC = 0,50", color="#898781", fontsize=9)
ax.set_xlabel("taux de faux positifs (1 - spécificité)"); ax.set_ylabel("taux de vrais positifs
    ↪ (sensibilité)"); ax.set_title("Courbe ROC")
ax = axes[1]
ax.plot([0, 1], [0, 1], color="#898781", ls=":")
ax.plot(calib["p_predite"], calib["observe"], "o-", color=AQUA, lw=1.8)
ax.set_xlabel("probabilité prédite (moyenne par dixième)"); ax.set_ylabel("fréquence observée de
    ↪ rachat"); ax.set_title("Calibration")
plt.tight_layout()
plt.savefig("figures/ch02-roc-calibration.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

| dixième | p_predite | observe | n   |
|---------|-----------|---------|-----|
| 0       | 0.212     | 0.287   | 122 |
| 1       | 0.308     | 0.207   | 121 |
| 2       | 0.377     | 0.397   | 121 |
| 3       | 0.433     | 0.421   | 121 |
| 4       | 0.485     | 0.537   | 121 |
| 5       | 0.537     | 0.479   | 121 |
| 6       | 0.582     | 0.595   | 121 |
| 7       | 0.635     | 0.628   | 121 |
| 8       | 0.698     | 0.661   | 121 |
| 9       | 0.783     | 0.836   | 122 |

figure enregistrée

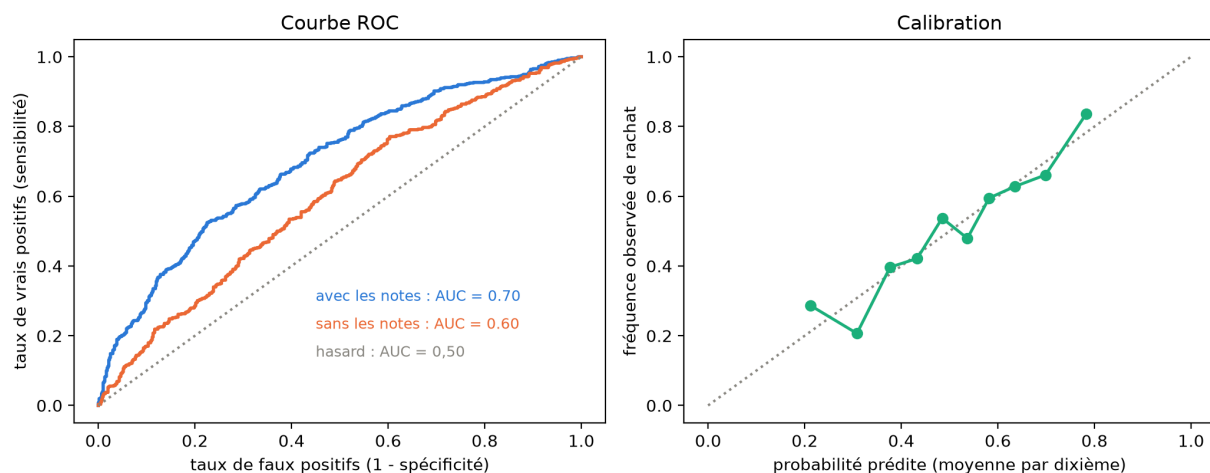


Figure 18: À gauche : courbes ROC du modèle de rachat avec et sans les notes de l'enquête. À droite : courbe de calibration, la diagonale pointillée représente un modèle parfaitement calibré.

À gauche, la courbe bleue (avec les notes) est partout au-dessus de la courbe orange (sans les notes) : à taux de fausses alertes égal, elle repère davantage de rachats. À droite, les points suivent globalement la diagonale : le modèle est **assez bien calibré**. Quelques écarts sont visibles (deuxième dixième : 30,8 % prédits, 20,7 % observés), mais avec environ 121 clients par dixième, l'erreur-type d'une proportion observée est d'au plus  $\sqrt{0,25/121} \approx 4,5$  points : un écart de 10 points (environ deux erreurs-types) sur dix classes n'a rien d'alarmant. Nous verrons au 2.4.4 un test formel (Hosmer-Lemeshow).

**Évaluer sur les données qui ont servi à ajuster est optimiste.** Le modèle a vu les réponses qu'on lui demande de « prédire ». Pour une estimation honnête, on sépare les données : on ajuste sur une partie (apprentissage) et l'on évalue sur l'autre (test). C'est le sujet central du volume III ; voici un avant-goût.

```
rng = np.random.default_rng(5)
idx = rng.permutation(len(repondants))
n_app = int(0.7 * len(repondants))
app, test = repondants.iloc[idx[:n_app]], repondants.iloc[idx[n_app:]]
m_app = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service", app,
               family=sm.families.Binomial()).fit()
print("AUC en apprentissage :", round(float(roc_auc_score(app["rachat_12m"], m_app.fittedvalues)),
               4), "(", len(app), "clients)")
print("AUC en test          :", round(float(roc_auc_score(test["rachat_12m"], m_app.predict(test))),
               4), "(", len(test), "clients)")
```

```
AUC en apprentissage : 0.6841 ( 848 clients )
AUC en test          : 0.7193 ( 364 clients )
```

L'AUC en test (0,719) est *supérieure* à celle d'apprentissage (0,684) : ce n'est pas une anomalie. Avec 364 clients en test (environ 180 par classe), l'incertitude sur une AUC est de l'ordre de  $\sqrt{0,7 \times 0,3/180} \approx 0,03$  : un seul découpage aléatoire ne permet de conclure ni à du sur-apprentissage ni à son absence. En moyenne, l'AUC en test est plutôt un peu inférieure à celle en apprentissage, et pour une estimation fiable on **répète** le découpage (validation croisée, volume III). Retenez le principe : **on n'évalue pas un modèle sur les données qui l'ont ajusté**.

### 2.2.9 Pièges de la régression logistique

**La séparation parfaite.** Si une combinaison des variables sépare **parfaitement** les 0 des 1, la vraisemblance n'a pas de maximum : elle augmente sans fin quand les coefficients grossissent. Un exemple minuscule : six clients, trois qui n'ont pas racheté avec 1, 2 et 3 achats préalables, trois qui ont racheté avec 4, 5 et 6.

```
import warnings
x_sep = np.array([1.0, 2, 3, 4, 5, 6])
y_sep = np.array([0, 0, 0, 1, 1, 1])
with warnings.catch_warnings(record=True) as avertissements:
    warnings.simplefilter("always")
    try:
        res = sm.GLM(y_sep, sm.add_constant(x_sep), family=sm.families.Binomial()).fit()
        print("pente estimée :", round(float(res.params[1]), 1), "| erreur-type de la pente :",
              round(float(res.bse[1]), 1))
        print("probabilités prédites (arrondies) :", res.fittedvalues.round(3).tolist())
    except Exception as e:
        print(type(e).__name__, ":", e)
    for message in sorted({str(a.message) for a in avertissements}):
        print("AVERTISSEMENT :", message)
```

```
pente estimée : 41.2 | erreur-type de la pente : 25719.8
probabilités prédites (arrondies) : [0.0, 0.0, 0.0, 1.0, 1.0, 1.0]
```

AVERTISSEMENT : Perfect separation or prediction detected, parameter may not be identified

statsmodels n'a pas planté, mais il a **averti** : les probabilités prédites sont arrondies à 0,0 et 1,0 (le modèle classe parfaitement), et la pente estimée (41,2) n'a pas de sens : son erreur-type est de plusieurs dizaines de milliers. L'algorithme ne s'est pas arrêté parce qu'il a trouvé un maximum, mais parce que la vraisemblance est devenue quasi plate : la « vraie » solution est infinie, et les valeurs affichées dépendent des détails de l'algorithme. **Ne faites jamais confiance à un coefficient accompagné de cet avertissement.** Les remèdes classiques sont de regrouper ou retirer la variable en cause, d'ajouter une pénalité (régularisation, section 1.5) ou d'utiliser la régression logistique de Firth (par exemple le paquet R `logistf`).

**Autres pièges.** (1) **Choisir le seuil à 0,5 par réflexe** : le bon seuil dépend des coûts relatifs d'un faux positif et d'un faux négatif (envoyer une relance inutile coûte peu, rater un client précieux coûte beaucoup). (2) **Les événements rares** : avec 1 % de « oui », une exactitude de 99 % est obtenue en répondant toujours « non » ; il faut regarder la sensibilité, la précision, l'AUC. (3) **Interpréter un OR comme un risque relatif** (2.2.3). (4) **Oublier la forme fonctionnelle** : le modèle suppose que l'effet de chaque variable est **linéaire sur le logit** ; si l'effet est en cloche (section 2.5), la logistique « simple » passe à côté.

#### À retenir

- Le modèle logistique pose  $\text{logit}(p) = x^T \beta$  : les coefficients sont des **log-cotes**,  $e^{\beta_j}$  est un **rapport de cotes**.
- Le rapport de cotes n'est pas un risque relatif : il le surestime quand l'événement est fréquent. Pour parler en points de pourcentage, calculez des **probabilités prédites** et des **effets marginaux moyens**.
- Avec un traitement tiré au hasard, l'effet marginal moyen de la variable de traitement estime son effet causal moyen.
- L'estimation se fait par IRLS ; avec le lien canonique,  $\sum_i (y_i - \hat{p}_i) x_{ij} = 0$  : les probabilités prédites ont la bonne moyenne.
- Un modèle se juge sur le **classement** (AUC, sensibilité/spécificité à un seuil) **et** sur la **calibration**, de préférence sur des données **non utilisées** pour l'ajustement.
- Attention à la séparation parfaite, au choix du seuil, aux événements rares.

## 2.3 Régression de Poisson et Gamma

**Intuition.** Yasmine voudrait comprendre **combien** de commandes passe un client dans l'année, et **combien** il dépense. Dans les deux cas, la moyenne est strictement positive et les effets sont plutôt **multiplicatifs** : un client plus actif commande 20 % de plus, quel que soit son niveau de départ. C'est exactement le travail du **lien logarithme**. Pour les comptages, on le combine avec la loi de **Poisson** (ou une variante plus souple) ; pour les montants positifs, avec la loi **Gamma**.

### 2.3.1 Compter : le modèle de Poisson

Pour un comptage  $Y_i \in \{0, 1, 2, \dots\}$  (volume I, section 2.2 pour la loi de Poisson), le modèle de **régression de Poisson** pose

$$Y_i \sim \text{Poisson}(\mu_i), \quad \log \mu_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}.$$

C'est le cas  $b(\theta) = e^\theta$  du tableau de 2.1.3, avec son lien canonique, le log. Un effet  $\beta_j$  s'interprète de façon **multiplicative** : quand  $x_j$  augmente d'une unité, le nombre moyen de commandes est

multiplié par  $e^{\beta_j}$ , appelé **rapport de taux** (*rate ratio*, RR).

Commençons par le cas le plus simple : une seule variable catégorielle, le canal d'acquisition.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram",
↪ "Site"])

moy = clients.groupby("canal", observed=True)["nb_commandes_an"].mean()
m_canal = smf.glm("nb_commandes_an ~ canal", clients, family=sm.families.Poisson()).fit()
print("moyennes observées par canal :", moy.round(4).to_dict())
print("rapports de moyennes / Boutique :", (moy / moy["Boutique"]).round(4).to_dict())
print("exp(coefficients) Poisson :", np.exp(m_canal.params).round(4).to_dict())
print("exp(Intercept) = moyenne de la boutique :", round(float(np.exp(m_canal.params["Intercept"])),
↪ 4))

moyennes observées par canal : {'Boutique': 4.1369, 'Instagram': 3.4645, 'Site': 4.1971}
rapports de moyennes / Boutique : {'Boutique': 1.0, 'Instagram': 0.8375, 'Site': 1.0145}
exp(coefficients) Poisson : {'Intercept': 4.1369, 'canal[T.Instagram]': 0.8375, 'canal[T.Site]':
↪ 1.0145}
exp(Intercept) = moyenne de la boutique : 4.1369
```

Les exponentielles des coefficients de Poisson sont **exactement** les rapports des moyennes observées :  $3,4645/4,1369 = 0,8375$  pour Instagram contre la boutique,  $4,1971/4,1369 = 1,0145$  pour le site, et  $e^{\text{Intercept}} = 4,1369$  est la moyenne de la boutique. C'est le même phénomène qu'en 2.2.3 : avec une variable catégorielle seule, le modèle est saturé et, le log étant le lien canonique de Poisson, l'équation du score  $\sum_i (y_i - \hat{\mu}_i) x_{ij} = 0$  impose que la moyenne prédite de chaque groupe soit sa moyenne observée.

Ajoutons maintenant l'âge et l'offre de bienvenue.

```
poisson = smf.glm("nb_commandes_an ~ age + canal + offre_bienvenue", clients,
↪ family=sm.families.Poisson()).fit()
ic = poisson.conf_int()
rr = pd.DataFrame({"coef": poisson.params, "RR": np.exp(poisson.params), "IC95 bas": np.exp(ic[0]),
↪ "IC95 haut": np.exp(ic[1]),
↪ "p": poisson.pvalues})
print(rr.round(4).to_string())
```

|                    | coef    | RR     | IC95 bas | IC95 haut | p      |
|--------------------|---------|--------|----------|-----------|--------|
| Intercept          | 1.4637  | 4.3218 | 3.9509   | 4.7275    | 0.0000 |
| canal[T.Instagram] | -0.1762 | 0.8385 | 0.7923   | 0.8873    | 0.0000 |
| canal[T.Site]      | 0.0151  | 1.0152 | 0.9594   | 1.0742    | 0.6008 |
| age                | -0.0010 | 0.9990 | 0.9969   | 1.0011    | 0.3675 |
| offre_bienvenue    | -0.0189 | 0.9813 | 0.9385   | 1.0259    | 0.4051 |

À offre et âge fixés, un client acquis par **Instagram** passe en moyenne **16 % de commandes en moins** qu'un client de la boutique (RR = 0,84, intervalle de 0,79 à 0,89). Le site ne se distingue pas de la boutique (RR = 1,015,  $p = 0,60$ ). L'**âge** n'a pas d'effet détectable sur le nombre de commandes (RR = 0,999 par an,  $p = 0,37$ ), pas plus que l'**offre de bienvenue** (RR = 0,98,  $p = 0,41$ ) : l'offre augmente la *probabilité* de racheter (section 2.2), mais pas le *nombre* de commandes. « Pas d'effet détectable » n'est pas « effet nul » : les intervalles de confiance disent quelle taille d'effet reste plausible (pour l'offre, de -6 % à +3 %).

**Ces p-valeurs sont à prendre avec beaucoup de précaution.** Elles supposent que la variance est égale à la moyenne, comme l'impose la loi de Poisson. Nous allons voir que cette hypothèse est ici très fautive (2.3.3), ce qui rend les erreurs-types trop petites et les p-valeurs trop optimistes.

### 2.3.2 Quand les clients n'ont pas été observés aussi longtemps : l'exposition

Jusqu'ici, tous les clients ont été observés pendant **12 mois**. Que faire si certains l'ont été trois mois, d'autres douze ? On ne peut pas comparer directement leurs nombres de commandes : un client observé quatre fois plus longtemps a, toutes choses égales, quatre fois plus de commandes. On modélise alors le **taux** (commandes par mois) et non le nombre brut : si  $t_i$  est la durée d'observation (l'**exposition**) et  $\lambda_i$  le taux mensuel,  $\mu_i = t_i \lambda_i$ , donc

$$\log \mu_i = \log t_i + x_i^T \beta.$$

Le terme  $\log t_i$  est un **décalage** (*offset*) : une variable dont le coefficient est imposé égal à 1. Voyons ce qu'il se passe si on l'oublie. Nous simulons (graine 23) 1 500 nouveaux clients observés 3, 6, 9 ou 12 mois ; une partie des clients (38 %) est abonnée à la newsletter, mais les clients anciens (longue observation) sont **plus souvent abonnés**. Le **vrai** effet de la newsletter est de multiplier le taux de commandes par 1,2.

```
rng = np.random.default_rng(23)
n = 1500
mois_obs = rng.choice([3, 6, 9, 12], size=n)
p_newsletter = pd.Series(mois_obs).map({3: 0.15, 6: 0.30, 9: 0.45, 12: 0.65}).to_numpy()
newsletter = rng.binomial(1, p_newsletter)
taux_mensuel = 0.4 * 1.2 ** newsletter # commandes par mois : vrai RR = 1,2
sim = pd.DataFrame({"mois": mois_obs, "newsletter": newsletter, "commandes": rng.poisson(taux_mensuel
↪ * mois_obs)})

naif = smf.glm("commandes ~ newsletter", sim, family=sm.families.Poisson()).fit()
avec_offset = smf.glm("commandes ~ newsletter", sim, family=sm.families.Poisson(),
↪ offset=np.log(sim["mois"])).fit()
libre = smf.glm("commandes ~ newsletter + np.log(mois)", sim, family=sm.families.Poisson()).fit()

print("part de clients abonnés à la newsletter :", round(float(sim["newsletter"].mean()), 3))
print("exposition moyenne : sans newsletter =", round(sim.loc[sim.newsletter == 0, "mois"].mean(),
↪ 2), "mois | avec newsletter =", round(sim.loc[sim.newsletter == 1, "mois"].mean(), 2), "mois")
g = sim.groupby("newsletter").agg(commandes=("commandes", "sum"), clients_mois=("mois", "sum"))
g["taux par client-mois"] = g["commandes"] / g["clients_mois"]
print(g.round(4).to_string())
print()
print("vrai RR de la newsletter : 1.2")
print("RR sans tenir compte de la durée : ", round(float(np.exp(naif.params["newsletter"])), 3))
print("RR avec décalage log(mois) :",
↪ round(float(np.exp(avec_offset.params["newsletter"])), 3),
    " IC95 [", " ", ".join(str(round(float(v), 3)) for v in
    ↪ np.exp(avec_offset.conf_int().loc["newsletter"])), " ]")
print("RR avec log(mois) comme variable libre : ", round(float(np.exp(libre.params["newsletter"])),
↪ 3),
    "| coefficient de log(mois) :", round(float(libre.params["np.log(mois)"]), 3), "(attendu : 1)")
```

```
part de clients abonnés à la newsletter : 0.381
exposition moyenne : sans newsletter = 6.44 mois | avec newsletter = 9.05 mois
      commandes  clients_mois  taux par client-mois
newsletter
0           2360           5979           0.3947
1           2451           5166           0.4744

vrai RR de la newsletter : 1.2
```

|                                        |         |                                                |
|----------------------------------------|---------|------------------------------------------------|
| RR sans tenir compte de la durée       | : 1.69  |                                                |
| RR avec décalage log(mois)             | : 1.202 | IC95 [ 1.136, 1.272 ]                          |
| RR avec log(mois) comme variable libre | : 1.196 | coefficient de log(mois) : 1.017 (attendu : 1) |

Les clients abonnés à la newsletter (38 % de l'échantillon) sont observés en moyenne **9,05 mois**, contre **6,44 mois** pour les autres. Le modèle sans décalage compare donc des clients observés pendant des durées très différentes et attribue à la newsletter l'effet de la durée : il annonce un RR de **1,69** alors que le vrai est de 1,2. Avec le décalage log(mois), on retrouve **1,202**, avec un intervalle de confiance (1,136 à 1,272) qui contient la vérité ; c'est aussi le rapport des deux taux du tableau,  $0,4744/0,3947 = 1,202$  commande par client-mois. Enfin, si l'on laisse le modèle estimer librement le coefficient de log(mois), il trouve **1,017** : très proche de 1, ce qui justifie l'imposition du décalage. Ici la durée d'observation est une **variable de confusion** : elle influence à la fois le traitement (les anciens sont plus souvent abonnés) et le résultat (ils ont eu plus de temps pour commander).

**L'offset, c'est un taux.** Les trois modèles se rejoignent dans l'idée : la bonne quantité à comparer est le nombre de commandes **par client-mois**. Avec un décalage, le modèle de Poisson compare exactement des taux. (Pour un modèle à une seule variable binaire, le RR du décalage est le rapport des taux observés dans le tableau ci-dessus.)

### 2.3.3 Quand la variance dépasse la moyenne : la surdispersion

La loi de Poisson impose  $\text{Var}(Y) = \mu$  ( $\phi = 1$ ). Mais en 2.1.1 nous avons vu que la variance des commandes valait 3,4 fois la moyenne. Un modèle de Poisson peut avoir de bons coefficients et de très mauvaises erreurs-types : il faut **mesurer** cette surdispersion.

**Deux statistiques simples.** Si le modèle est correct, la statistique de Pearson  $X^2 = \sum_i \frac{(y_i - \hat{\mu}_i)^2}{\hat{\mu}_i}$  et la déviance valent à peu près leurs degrés de liberté ( $n - p$ ) ; leurs rapports aux degrés de liberté doivent être proches de 1. On en déduit une estimation de la dispersion,  $\hat{\phi} = X^2 / (n - p)$ .

```
y = clients["nb_commandes_an"].to_numpy()
mu = poisson.fittedvalues.to_numpy()
print("Pearson X² =", round(poisson.pearson_chi2, 1), "| degrés de liberté =", int(poisson.df_resid),
      ↪ "| phi estimé =", round(poisson.pearson_chi2 / poisson.df_resid, 3))
print("déviante =", round(poisson.deviance, 1), "| déviante / ddl =", round(poisson.deviance /
      ↪ poisson.df_resid, 3))

# Test de Cameron et Trivedi (1990) : sous Poisson, E[(y-mu)² - y] = 0 ; sous « variance = mu + alpha
      ↪ mu² », E[(y-mu)² - y] = alpha mu²
aux = ((y - mu) ** 2 - y) / mu
reg = sm.OLS(aux, mu).fit()          # régression de ((y-mu)²-y)/mu sur mu, sans constante : la pente
      ↪ estime alpha
print("alpha estimé par Cameron-Trivedi :", round(float(reg.params[0]), 3), "| t =",
      ↪ round(float(reg.tvalues[0]), 2))
```

```
Pearson X² = 6774.2 | degrés de liberté = 1995 | phi estimé = 3.396
déviante = 6293.0 | déviante / ddl = 3.154
alpha estimé par Cameron-Trivedi : 0.609 | t = 12.47
```

La statistique de Pearson vaut 6 774,2 pour 1 995 degrés de liberté :  $\hat{\phi} = 3,40$ , et la déviance divisée par les degrés de liberté vaut 3,15. Les deux sont **très éloignés de 1**. Le test de Cameron-Trivedi estime directement le paramètre de surdispersion :  $\hat{\alpha} = 0,609$  avec une statistique  $t$  de 12,5 : la surdispersion est hautement significative. Nos comptages ont donc une variance qui vaut en gros  $\mu + 0,6\mu^2$ , et non  $\mu$ .

**Trois façons de réagir.**



1. **Quasi-Poisson** : on garde les coefficients de Poisson et l'on **gonfle les erreurs-types** par  $\sqrt{\hat{\phi}}$ . C'est la solution la plus simple.
2. **Erreurs-types robustes** (dites « sandwich ») : on ne suppose plus rien sur la forme de la variance et l'on estime directement la variabilité de  $\hat{\beta}$ .
3. **Loi binomiale négative** : on remplace Poisson par une loi plus dispersée.

Écrivons d'abord la troisième. La loi binomiale négative (NB2) est un **mélange de Poissons** : on suppose que le client  $i$  a un taux de commandes *propre*  $\lambda_i$ , **aléatoire**, de moyenne  $\mu_i$  et distribué selon une loi Gamma, et que, sachant  $\lambda_i$ , son nombre de commandes est de Poisson.

**Proposition.** Si  $Y | \lambda \sim \text{Poisson}(\lambda)$  et  $\lambda \sim \text{Gamma}$  de moyenne  $\mu$  et de variance  $\alpha\mu^2$ , alors  $E[Y] = \mu$  et  $\text{Var}(Y) = \mu + \alpha\mu^2$ .

*Démonstration.* Par la loi de l'espérance totale,  $E[Y] = E[E[Y | \lambda]] = E[\lambda] = \mu$ . Par la loi de la variance totale,  $\text{Var}(Y) = E[\text{Var}(Y | \lambda)] + \text{Var}(E[Y | \lambda]) = E[\lambda] + \text{Var}(\lambda) = \mu + \alpha\mu^2$ .  $\square$

Le paramètre  $\alpha \geq 0$  mesure l'hétérogénéité entre clients ;  $\alpha = 0$  redonne Poisson. La variance est une **fonction quadratique** de la moyenne, ce qui convient à beaucoup de comptages réels. Vérifions la proposition par simulation, puis ajustons les quatre modèles.

```
rng = np.random.default_rng(3)
mu0, alpha0, N = 4.0, 0.6, 400_000
lam = rng.gamma(shape=1 / alpha0, scale=mu0 * alpha0, size=N) # Gamma de moyenne mu0 et de
↳ variance alpha0 * mu0^2
tirages = rng.poisson(lam)
print(f"simulation : moyenne = {tirages.mean():.3f} | variance = {tirages.var():.3f}")
print(f"formule : moyenne = {mu0:.3f} | variance = {mu0 + alpha0 * mu0**2:.3f}")

formule = "nb_commandes_an ~ age + canal + offre_bienvenue"
quasi = smf.glm(formule, clients, family=sm.families.Poisson()).fit(scale="X2")
robuste = smf.glm(formule, clients, family=sm.families.Poisson()).fit(cov_type="HC0")
nb = smf.negativebinomial(formule, clients).fit(dis=0)

comp = pd.DataFrame({
    "coef Poisson": poisson.params, "coef NB": nb.params[poisson.params.index],
    "se Poisson": poisson.bse, "se quasi-Poisson": quasi.bse, "se robuste": robuste.bse, "se NB":
    ↳ nb.bse[poisson.params.index],
})
print()
print(comp.round(4).to_string())
print()
print("ratio se quasi-Poisson / se Poisson :", round(float((quasi.bse / poisson.bse).mean()), 3), "|
↳ racine de phi =", round(float(np.sqrt(poisson.pearson_chi2 / poisson.df_resid)), 3))
print("alpha (binomiale négative) =", round(float(nb.params["alpha"]), 3), "| IC95 :",
↳ [round(float(v), 3) for v in nb.conf_int().loc["alpha"]])
lr = 2 * (nb.llf - poisson.llf)
log10_p = stats.norm.logsf(np.sqrt(lr)) / np.log(10) # 0,5 × P(khi-deux à 1 ddl > lr) = P(N(0,1)
↳ > sqrt(lr)) ; en log pour éviter le dépassement de capacité
print(f"AIC Poisson = {poisson.aic:.1f} | AIC binomiale négative = {nb.aic:.1f} | RV : 2(l_NB -
↳ l_Poisson) = {lr:.1f}, log10(p) = {log10_p:.0f}")
```

simulation : moyenne = 4.004 | variance = 13.640

formule : moyenne = 4.000 | variance = 13.600

|                    | coef Poisson | coef NB | se Poisson | se quasi-Poisson | se robuste | se NB  |
|--------------------|--------------|---------|------------|------------------|------------|--------|
| Intercept          | 1.4637       | 1.4681  | 0.0458     | 0.0844           | 0.0837     | 0.0840 |
| canal[T.Instagram] | -0.1762      | -0.1765 | 0.0289     | 0.0532           | 0.0529     | 0.0520 |
| canal[T.Site]      | 0.0151       | 0.0148  | 0.0288     | 0.0531           | 0.0529     | 0.0534 |
| age                | -0.0010      | -0.0011 | 0.0011     | 0.0020           | 0.0019     | 0.0020 |
| offre_bienvenue    | -0.0189      | -0.0156 | 0.0227     | 0.0419           | 0.0419     | 0.0411 |

ratio se quasi-Poisson / se Poisson : 1.843 | racine de phi = 1.843



```
alpha (binomiale négative) = 0.584 | IC95 : [0.528, 0.639]
AIC Poisson = 11715.5 | AIC binomiale négative = 9768.7 | RV : 2(l_NB - l_Poisson) = 1948.8, log10(p)
↪ = -425
```

Trois constats. (1) La simulation confirme la proposition : variance simulée 13,64 contre 13,60 par la formule  $\mu + \alpha\mu^2$  avec  $\mu = 4$  et  $\alpha = 0,6$ . (2) Les **coefficients** de Poisson et de la binomiale négative sont presque identiques (par exemple  $-0,176$  pour Instagram dans les deux cas) : la surdispersion ne biaise pas la moyenne estimée, à condition qu'elle soit bien modélisée par ailleurs. (3) Les **erreurs-types** de Poisson sont trop petites : 0,0289 pour Instagram, contre 0,0532 (quasi-Poisson), 0,0529 (robuste) et 0,0520 (binomiale négative). Les trois corrections s'accordent, et le ratio quasi-Poisson/Poisson vaut exactement  $\sqrt{\hat{\phi}} = 1,843$  comme annoncé. Ici les conclusions ne changent pas (l'effet d'Instagram reste significatif :  $z = -0,176/0,053 \approx -3,3$ ), mais dans un cas limite, l'erreur-type trop petite de Poisson aurait transformé un effet douteux en effet « significatif ».

La binomiale négative estime  $\hat{\alpha} = 0,584$  (intervalle de 0,528 à 0,639), cohérent avec l'estimation de Cameron-Trivedi, et fait chuter l'AIC de 11 715,5 à 9 768,7 : un gain de près de 1 950 points. Le rapport de vraisemblance vaut 1 948,8, soit une p-valeur d'environ  $10^{-425}$  : la surdispersion est incontestable.

**Une subtilité du test du rapport de vraisemblance.** Tester  $\alpha = 0$  revient à tester un paramètre **au bord** de son domaine ( $\alpha \geq 0$ ). Dans ce cas, la loi de  $2(\ell_{NB} - \ell_{Poisson})$  sous  $H_0$  n'est pas un khi-deux à 1 ddl, mais un **mélange** à parts égales d'une masse en 0 et d'un khi-deux à 1 ddl : la p-valeur correcte est la moitié de celle du khi-deux, d'où le facteur 0,5 dans le code. Ici l'écart est si grand que la conclusion ne change pas.

Pour juger si la binomiale négative décrit **mieux la distribution entière** des comptages, comparons les fréquences observées aux fréquences que chaque modèle prédit (moyenne, sur tous les clients, des probabilités prédites de chaque valeur).

```
valeurs = np.arange(0, 16)
obs = np.array([(y == k).mean() for k in valeurs])
mu_p = poisson.fittedvalues.to_numpy()
mu_nb = np.asarray(nb.predict())
a = float(nb.params["alpha"])
pred_p = np.array([stats.poisson.pmf(k, mu_p).mean() for k in valeurs])
pred_nb = np.array([stats.nbinom.pmf(k, 1 / a, (1 / a) / ((1 / a) + mu_nb)).mean() for k in valeurs])
print(pd.DataFrame({"observé": obs, "Poisson": pred_p, "binomiale négative": pred_nb},
↪ index=valeurs).round(3).head(8).to_string())
print()
print("écart absolu moyen aux fréquences observées : Poisson =", round(float(np.abs(obs -
↪ pred_p).mean()), 4), "| binomiale négative =", round(float(np.abs(obs - pred_nb).mean()), 4))

fig, ax = plt.subplots(figsize=(7.2, 4.2))
BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
ax.bar(valeurs, obs, color="#c3c2b7", width=0.8, label="observé")
ax.plot(valeurs, pred_p, "o-", color=ORANGE, lw=1.8, ms=4, label="Poisson ajusté")
ax.plot(valeurs, pred_nb, "s-", color=BLEU, lw=1.8, ms=4, label="binomiale négative ajustée")
ax.set_xlabel("nombre de commandes dans l'année"); ax.set_ylabel("proportion de clients")
ax.set_title("Quelle loi décrit le mieux les comptages ?"); ax.legend(frameon=False)
plt.tight_layout()
plt.savefig("figures/ch02-poisson-vs-nb.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

|   | observé | Poisson | binomiale négative |
|---|---------|---------|--------------------|
| 0 | 0.130   | 0.022   | 0.133              |
| 1 | 0.154   | 0.082   | 0.157              |
| 2 | 0.158   | 0.156   | 0.147              |

|   |       |       |       |
|---|-------|-------|-------|
| 3 | 0.130 | 0.199 | 0.126 |
| 4 | 0.096 | 0.192 | 0.103 |
| 5 | 0.088 | 0.149 | 0.081 |
| 6 | 0.060 | 0.097 | 0.063 |
| 7 | 0.048 | 0.055 | 0.048 |

écart absolu moyen aux fréquences observées : Poisson = 0.033 | binomiale négative = 0.0035  
figure enregistrée

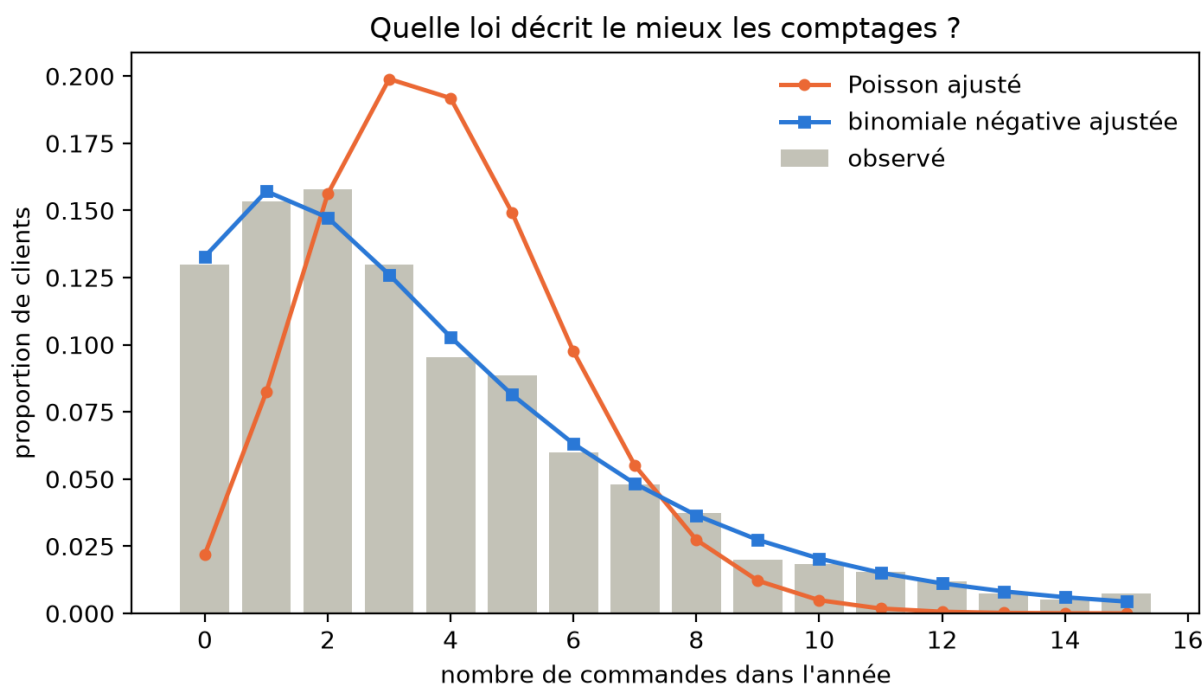


Figure 19: Distribution du nombre de commandes par client (barres) et distributions prédites par le modèle de Poisson (orange) et par le modèle binomial négatif (bleu).

La loi de Poisson ajustée (courbe orange) a la mauvaise forme : elle prévoit un pic à 3 commandes (environ 20 % des clients) qui n'existe pas dans les données, seulement 2,2 % de clients à zéro commande (13,0 % observés) et pas assez de clients très actifs. La binomiale négative (bleu) suit les barres presque parfaitement : 13,3 % à zéro, 15,7 % à une commande, une queue correcte. L'écart absolu moyen aux fréquences observées est **dix fois plus petit** (0,0035 contre 0,033). Remarquez que la binomiale négative reproduit à elle seule l'**excès de zéros** apparent : nous verrons à la section 2.6 si un modèle « à zéros en excès » apporte quelque chose de plus.

### 2.3.4 Modéliser des montants : la loi Gamma

Les dépenses sont **positives**, **asymétriques**, et la dispersion croît avec le niveau : un client qui dépense 1 000 DT varie en DT bien plus qu'un client qui dépense 50 DT. Une propriété remarquable de la loi **Gamma** est que son **coefficient de variation**  $\sqrt{\text{Var}}/\mu$  est **constant** : avec  $\text{Var}(Y) = \phi\mu^2$ , il vaut  $\sqrt{\phi}$ , quel que soit  $\mu$ . C'est exactement ce que l'on observe souvent avec des montants (une incertitude *proportionnelle* au niveau). Le modèle de **régression Gamma avec lien log** pose

$$Y_i \sim \text{Gamma}(\text{moyenne } \mu_i, \text{Var} = \phi\mu_i^2), \quad \log \mu_i = x_i^\top \beta.$$

Un coefficient  $\beta_j$  multiplie encore la moyenne par  $e^{\beta_j}$ . On ne peut pas inclure les clients à zéro (la loi Gamma est strictement positive) : nous nous limitons donc aux **acheteurs**, et les effets seront à lire « *parmi les clients qui ont acheté* ». La section 2.6 montrera comment traiter les zéros.

```

acheteurs = clients[clients["depense_annuelle"] > 0].copy()
print("acheteurs :", len(acheteurs), "sur", len(clients), "| dépense moyenne =",
      ↪ round(acheteurs["depense_annuelle"].mean(), 1), "DT | médiane =",
      ↪ round(acheteurs["depense_annuelle"].median(), 1), "DT")

formule_d = "depense_annuelle ~ age + canal + offre_bienvenue"
gamma = smf.glm(formule_d, acheteurs,
                 ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
print()
print(gamma.summary().tables[1])
phi = gamma.scale
print()
print("dispersion phi estimée =", round(phi, 3), "| coefficient de variation implicite =",
      ↪ round(float(np.sqrt(phi)), 3))

```

acheteurs : 1740 sur 2000 | dépense moyenne = 283.9 DT | médiane = 193.4 DT

|                    | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|--------------------|---------|---------|--------|-------|--------|--------|
| Intercept          | 5.6086  | 0.098   | 57.234 | 0.000 | 5.417  | 5.801  |
| canal[T.Instagram] | -0.4988 | 0.061   | -8.181 | 0.000 | -0.618 | -0.379 |
| canal[T.Site]      | -0.1518 | 0.063   | -2.425 | 0.015 | -0.275 | -0.029 |
| age                | 0.0075  | 0.002   | 3.295  | 0.001 | 0.003  | 0.012  |
| offre_bienvenue    | -0.0098 | 0.048   | -0.203 | 0.839 | -0.104 | 0.085  |

dispersion phi estimée = 1.009 | coefficient de variation implicite = 1.004

Parmi les 1 740 acheteurs (87 % des clients), la dépense moyenne est de 283,9 DT, la médiane de 193,4 DT. Les effets se lisent en pourcentage de la dépense moyenne, **parmi les acheteurs** :

- **Instagram** :  $e^{-0,499} = 0,61$  : les acheteurs acquis par Instagram dépensent environ **39 % de moins** que ceux de la boutique ( $p < 0,001$ ).
- **Site** :  $e^{-0,152} = 0,86$  : environ 14 % de moins que la boutique ( $p = 0,015$ ).
- **Âge** : +0,0075 par an, soit +0,75 % par an et  $e^{0,075} \approx +7,8$  % pour dix ans ( $p = 0,001$ ) : les clients plus âgés dépensent un peu plus.
- **Offre** : aucun effet détectable ( $p = 0,84$ ).

La dispersion estimée vaut  $\hat{\phi} = 1,009$ , donc un **coefficient de variation d'environ 1** : l'écart-type de la dépense est à peu près égal à sa moyenne (comme pour une loi exponentielle, cas particulier de la Gamma de forme 1). C'est une forte dispersion, qui s'explique ici par le fait qu'une dépense annuelle cumule un nombre de commandes (très variable) et un panier moyen (variable lui aussi).

**L'estimation à la main.** Notre fonction `irls` (2.1.5) gère aussi la loi Gamma avec lien log. Pour la loi Gamma, la dispersion  $\phi$  est inconnue ; on l'estime par  $\hat{\phi} = X^2/(n - p)$  et l'on multiplie la matrice de covariance par  $\hat{\phi}$ .

```

from patsy import dmatrices

Yd, Xd = dmatrices(formule_d, acheteurs, return_type="dataframe")
r = irls(Xd.to_numpy(), Yd.to_numpy().ravel(), "gamma_log")
mu_d = r["mu"]
phi_main = np.sum((Yd.to_numpy().ravel() - mu_d)**2 / mu_d**2) / (len(acheteurs) - Xd.shape[1])
se_main = np.sqrt(np.diag(r["cov"]) * phi_main)
print(pd.DataFrame({"coef main": r["beta"], "coef statsmodels": gamma.params.to_numpy(), "se main":
                    ↪ se_main, "se statsmodels": gamma.bse.to_numpy(),
                    index=Xd.columns).round(5).to_string())
print("phi (main) =", round(float(phi_main), 4), "| phi (statsmodels) =", round(phi, 4), "|
      ↪ itérations :", r["iterations"])

```

|                    | coef main | coef statsmodels | se main | se statsmodels |
|--------------------|-----------|------------------|---------|----------------|
| Intercept          | 5.60858   | 5.60858          | 0.09799 | 0.09799        |
| canal[T.Instagram] | -0.49879  | -0.49879         | 0.06097 | 0.06097        |
| canal[T.Site]      | -0.15181  | -0.15181         | 0.06260 | 0.06260        |
| age                | 0.00754   | 0.00754          | 0.00229 | 0.00229        |
| offre_bienvenue    | -0.00980  | -0.00980         | 0.04819 | 0.04819        |

phi (main) = 1.0088 | phi (statsmodels) = 1.0088 | itérations : 11

Notre IRLS et statsmodels donnent les mêmes coefficients, les mêmes erreurs-types (à cinq décimales) et la même dispersion ( $\hat{\phi} = 1,0088$ ). Notez le nombre d'itérations : **11**, contre 5 pour la régression logistique. Le lien log n'est pas le lien canonique de la loi Gamma, donc le score de Fisher n'est plus identique à la méthode de Newton et la convergence est un peu moins rapide.

**Pourquoi ne pas simplement prendre le logarithme ?** Une autre approche, très répandue, consiste à régresser  $\log Y$  sur  $x$  par moindres carrés, puis à lire  $e^{\hat{\beta}_i}$ . Les deux méthodes ne répondent **pas** à la même question :

- La régression de  $\log Y$  modélise  $E[\log Y | x]$ . Revenir à l'échelle des DT par  $e^{\hat{\beta}^T x}$  donne l'estimation de la **médiane** (ou de la moyenne géométrique), **pas de la moyenne** : par l'inégalité de Jensen,  $E[\log Y] \leq \log E[Y]$ , donc on sous-estime systématiquement la moyenne. Pour une loi lognormale de paramètre  $\sigma^2$ , le facteur manquant est  $e^{\sigma^2/2}$ .
- La régression Gamma modélise  $\log E[Y | x]$  : la moyenne, directement, sans correction.

Les deux coïncident presque pour les rapports de moyennes si la dispersion est constante, mais pas pour les **prédictions en DT** (et donc pas pour un chiffre d'affaires total).

```
ols_log = smf.ols("np.log(depense_annuelle) ~ age + canal + offre_bienvenue", acheteurs).fit()
print(pd.DataFrame({"effet (Gamma, log)": gamma.params, "effet (OLS sur log y)":
    ↳ ols_log.params}).round(4).to_string())
print()
y_d = acheteurs["depense_annuelle"]
naif_log = np.exp(ols_log.fittedvalues)
lissage = naif_log * np.mean(np.exp(ols_log.resid)) # correction de Duan (« smearing »)
print("dépense moyenne observée", round(float(y_d.mean()), 1), "DT")
print("moyenne des prédictions Gamma (lien log)",
    ↳ round(float(gamma.fittedvalues.mean()), 1), "DT")
print("moyenne des prédictions exp(OLS sur log y), sans correction :", round(float(naif_log.mean()),
    ↳ 1), "DT")
print("idem avec la correction de Duan", round(float(lissage.mean()), 1),
    ↳ "DT")
print()
par_canal = pd.DataFrame({"observée": y_d.groupby(acheteurs["canal"], observed=True).mean(),
    "Gamma": gamma.fittedvalues.groupby(acheteurs["canal"], observed=True).mean(),
    "exp(OLS log)": naif_log.groupby(acheteurs["canal"], observed=True).mean()})
print(par_canal.round(1).to_string())
```

|                    | effet (Gamma, log) | effet (OLS sur log y) |
|--------------------|--------------------|-----------------------|
| Intercept          | 5.6086             | 5.1977                |
| canal[T.Instagram] | -0.4988            | -0.4608               |
| canal[T.Site]      | -0.1518            | -0.1381               |
| age                | 0.0075             | 0.0075                |
| offre_bienvenue    | -0.0098            | -0.0238               |

|                                                             |            |
|-------------------------------------------------------------|------------|
| dépense moyenne observée                                    | : 283.9 DT |
| moyenne des prédictions Gamma (lien log)                    | : 284.0 DT |
| moyenne des prédictions exp(OLS sur log y), sans correction | : 189.9 DT |
| idem avec la correction de Duan                             | : 283.1 DT |

|          | observée | Gamma | exp(OLS log) |
|----------|----------|-------|--------------|
| canal    |          |       |              |
| Boutique | 355.0    | 356.6 | 234.6        |

|           |       |       |       |
|-----------|-------|-------|-------|
| Instagram | 216.8 | 217.0 | 148.2 |
| Site      | 307.2 | 306.1 | 204.1 |

Les effets sont du même ordre pour les deux méthodes (Instagram :  $-0,499$  pour Gamma,  $-0,461$  pour la régression sur  $\log y$  ; âge :  $0,0075$  dans les deux cas), mais les **prédictions en DT** n'ont rien à voir. La moyenne des prédictions de la loi Gamma ( $284,0$  DT) coïncide avec la dépense moyenne observée ( $283,9$  DT), et suit de près la moyenne observée de chaque canal ( $356,6$  contre  $355,0$  pour la boutique,  $217,0$  contre  $216,8$  pour Instagram). En revanche  $e^{\text{ajusté}}$  de la régression sur  $\log y$  prédit en moyenne **189,9 DT**, soit un tiers de moins : c'est la médiane conditionnelle, pas la moyenne. L'écart est visible dans l'ordonnée à l'origine ( $5,609$  contre  $5,198$ , soit un facteur  $e^{0,41} \approx 1,5$ ) ; il peut être corrigé par le facteur de lissage de Duan (le résultat remonte à  $283,1$  DT), mais la régression Gamma n'a pas besoin de correction.

**Quand préférer quoi ?** Si l'on s'intéresse à la **moyenne** (chiffre d'affaires prévu, coût moyen), préférez Gamma avec lien log : elle modélise directement ce que l'on cherche. Si l'on s'intéresse à la **valeur typique** et que le logarithme est approximativement normal, la régression sur  $\log y$  est correcte, simple et rapide. Dans tous les cas, **ne retransformez jamais**  $e^{\hat{y}_{\log}}$  en la présentant comme « la dépense moyenne prévue ».

### 2.3.5 Comment choisir sa famille ?

| La variable à expliquer est...                | Famille            | Lien usuel       | Variance $V(\mu)$   | Exemple                     |
|-----------------------------------------------|--------------------|------------------|---------------------|-----------------------------|
| continue, symétrique, de dispersion constante | Normale            | identité         | 1                   | un écart de prix            |
| un 0/1                                        | Bernoulli          | logit            | $\mu(1 - \mu)$      | racheter ou non             |
| un comptage (variance $\approx$ moyenne)      | Poisson            | log              | $\mu$               | commandes par heure         |
| un comptage surdispersé                       | binomiale négative | log              | $\mu + \alpha\mu^2$ | commandes par client        |
| un montant positif, CV constant               | Gamma              | log (ou inverse) | $\mu^2$             | dépense des acheteurs       |
| un montant $\geq 0$ avec des zéros            | Tweedie (2.6)      | log              | $\mu^p, 1 < p < 2$  | dépense de tous les clients |

**Un fil conducteur : la puissance de la variance.** Dans ce tableau, la variance prend la forme  $\mu^p$  avec  $p = 0$  (normale),  $p = 1$  (Poisson),  $p = 2$  (Gamma). La loi de Tweedie (section 2.6) est la famille qui comble les valeurs de  $p$  entre 1 et 2 et couvre ainsi le cas des montants avec des zéros.

#### À retenir

- Poisson :  $\log \mu = x^T \beta$  ;  $e^{\beta_j}$  est un **rapport de taux**. Pour des durées d'observation différentes, ajoutez un **décalage**  $\log t_i$  : l'oublier peut produire un effet très faux.
- Poisson impose variance = moyenne. **Mesurez** la surdispersion ( $X^2/\text{ddl}$ , test de Cameron-Trivedi). Remèdes : quasi-Poisson ou erreurs-types robustes (mêmes coefficients, erreurs-types corrigées), ou **binomiale négative** ( $\text{Var} = \mu + \alpha\mu^2$ , un mélange

de Poissons).

- Gamma (lien log) : pour des montants strictement positifs au coefficient de variation à peu près constant ; elle modélise la **moyenne**.
- Régresser  $\log y$  modélise autre chose (la médiane) ; ne retransformez pas sans correction.
- Le choix de la famille est le choix de la relation variance–moyenne.

## 2.4 Déviance, qualité d'ajustement, vérification du modèle

**Intuition.** Un modèle ajusté, ce n'est pas un modèle **validé**. Dans la régression linéaire, on jugeait un modèle par la somme des carrés des résidus (RSS) et par le  $R^2$ . Dans un GLM, l'équivalent de la RSS est la **déviance** : une mesure de l'écart entre le modèle ajusté et le meilleur modèle imaginable. Elle sert à **comparer** des modèles emboîtés (test du rapport de vraisemblance), et les **résidus** servent à détecter ce que le modèle n'a pas compris. Dans tous les cas, la règle est la même : on regarde ce qui reste *après* l'ajustement.

### 2.4.1 La déviance

Pour chaque observation, on peut imaginer un modèle « parfait » qui prédit exactement  $y_i$  : c'est le **modèle saturé** (un paramètre par observation). Sa log-vraisemblance  $\ell_{\text{sat}}$  est la plus grande possible. La **déviance** d'un modèle ajusté mesure son retard sur ce modèle saturé :

$$D = 2[\ell_{\text{sat}} - \ell(\hat{\beta})] = \sum_{i=1}^n d(y_i, \hat{\mu}_i),$$

où  $d(y, \mu)$  est la **déviance unitaire** de l'observation (dépend de la famille). C'est un  $\chi^2$  pour des données groupées assez grandes. La déviance joue pour un GLM le rôle de la **somme des carrés des résidus** : pour la loi normale,  $d(y, \mu) = (y - \mu)^2$  et  $D = \text{RSS}/\sigma^2$ .

| Famille   | Déviance unitaire $d(y, \mu)$                                                                   |
|-----------|-------------------------------------------------------------------------------------------------|
| Normale   | $(y - \mu)^2$                                                                                   |
| Poisson   | $2[y \log \frac{y}{\mu} - (y - \mu)]$                                                           |
| Bernoulli | $2[y \log \frac{y}{\mu} + (1 - y) \log \frac{1-y}{1-\mu}]$ (avec la convention $0 \log 0 = 0$ ) |
| Gamma     | $2[-\log \frac{y}{\mu} + \frac{y-\mu}{\mu}]$                                                    |

**Un calcul à la main.** Quatre clients ont passé  $y = (2, 5, 3, 8)$  commandes ; le modèle sans variable (Poisson) estime  $\hat{\mu} = \bar{y} = 4,5$  pour tous. La déviance est  $D = 2 \sum_i [y_i \log \frac{y_i}{\hat{\mu}} - (y_i - \hat{\mu})]$  ; comme  $\sum (y_i - \hat{\mu}) = 0$ , il reste  $D = 2 \sum y_i \log \frac{y_i}{4,5}$ . Terme par terme :  $2 \log \frac{2}{4,5} = -1,622$  ;  $5 \log \frac{5}{4,5} = 0,527$  ;  $3 \log \frac{3}{4,5} = -1,216$  ;  $8 \log \frac{8}{4,5} = 4,603$ . Leur somme vaut 2,2915, d'où  $D = 4,583$ . La statistique de Pearson, elle, vaut  $\sum \frac{(y_i - \hat{\mu})^2}{\hat{\mu}} = \frac{6,25+0,25+2,25+12,25}{4,5} = 4,667$  : proche de la déviance, comme c'est généralement le cas.

Vérifions cela avec le code, puis avec les modèles des sections précédentes.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import xlogy

def deviance_unitaire(famille, y, mu):
    """Déviance unitaire d(y, mu) pour chaque observation (xlogy gère la convention 0 log 0 = 0)."""
    if famille == "normale":
        return (y - mu) ** 2
    if famille == "poisson":
        return 2 * (xlogy(y, y / mu) - (y - mu))
    if famille == "bernoulli":
        return 2 * (xlogy(y, y / mu) + xlogy(1 - y, (1 - y) / (1 - mu)))
    if famille == "gamma":
        return 2 * (-np.log(y / mu) + (y - mu) / mu)

y4 = np.array([2.0, 5, 3, 8])
mu4 = np.full(4, y4.mean())
print("déviance à la main      :", round(float(deviance_unitaire("poisson", y4, mu4).sum()), 4))
print("Pearson X² à la main   :", round(float(((y4 - mu4) ** 2) / mu4).sum()), 4))
m4 = sm.GLM(y4, np.ones((4, 1)), family=sm.families.Poisson()).fit()
print("déviance statsmodels  :", round(float(m4.deviance), 4), "| Pearson statsmodels :",
      ↪ round(float(m4.pearson_chi2), 4))

# Les trois modèles des sections 2.2 et 2.3, sur les vraies données
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram",
      ↪ "Site"])
logi = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
      ↪ family=sm.families.Binomial()).fit()
poi = smf.glm("nb_commandes_an ~ age + canal + offre_bienvenue", clients,
      ↪ family=sm.families.Poisson()).fit()
acheteurs = clients[clients["depense_annuelle"] > 0]
gam = smf.glm("depense_annuelle ~ age + canal + offre_bienvenue", acheteurs,
      ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")

lignes = []
for nom, fam, res, y in [("logistique", "bernoulli", logi, clients["rachat_12m"]), ("Poisson",
      ↪ "poisson", poi, clients["nb_commandes_an"]),
      ↪ ("Gamma", "gamma", gam, acheteurs["depense_annuelle"])] :
    d_main = deviance_unitaire(fam, y.to_numpy(float), res.fittedvalues.to_numpy()).sum()
    mu_nul = np.full(len(y), y.mean())
    d_nul = deviance_unitaire(fam, y.to_numpy(float), mu_nul).sum()
    lignes.append({"modèle": nom, "déviance (main)": d_main, "déviance (statsmodels)": res.deviance,
      ↪ "déviance nulle (main)": d_nul,
      ↪ "déviance nulle (statsmodels)": res.null_deviance, "part de déviance expliquée": 1
      ↪ - d_main / d_nul})

print()
print(pd.DataFrame(lignes).round(3).to_string(index=False))

```

```

déviance à la main      : 4.5829
Pearson X² à la main   : 4.6667
déviance statsmodels  : 4.5829 | Pearson statsmodels : 4.6667

```

|   | modèle     | déviance (main) | déviance (statsmodels) | déviance nulle (main) | déviance nulle (statsmodels) | part de déviance expliquée |
|---|------------|-----------------|------------------------|-----------------------|------------------------------|----------------------------|
| ↪ | logistique | 2710.830        | 2710.830               | 2771.867              | 2771.867                     | 0.022                      |
| ↪ | Poisson    | 6292.954        | 6292.954               | 6357.639              | 6357.639                     | 0.010                      |
| ↪ | Gamma      | 1389.349        | 1389.349               | 1473.452              | 1473.452                     | 0.057                      |



Dans l'exemple à quatre clients, notre calcul (4,5829) et celui de `statsmodels` coïncident, de même que la statistique de Pearson (4,6667). Pour les trois modèles des sections précédentes, la déviance calculée à la main est **identique** à celle du logiciel (par exemple 2 710,83 pour la logistique, 6 292,95 pour Poisson, 1 389,35 pour Gamma), ainsi que la déviance nulle. La part de déviance expliquée est de **2,2 %** pour la logistique, **1,0 %** pour Poisson et **5,7 %** pour Gamma : des valeurs faibles, typiques de résultats individuels très aléatoires (rachat ou non, nombre de commandes d'un client).

« **Part de déviance expliquée** ». Le rapport  $1 - D/D_{\text{nulle}}$  est l'analogue du  $R^2$  : il mesure la fraction de la déviance du modèle sans variable (le plus simple possible) que l'on a réussi à « expliquer ». C'est un pseudo- $R^2$ , et il est **souvent faible** pour des données individuelles discrètes : un faible pseudo- $R^2$  n'est pas un défaut en soi (le hasard est important), mais il rappelle l'ampleur du travail qui reste.

**La déviance d'un modèle binaire n'est pas un test d'ajustement.** Pour des 0/1 individuels, la déviance vaut  $-2\ell$  et ne suit pas un  $\chi^2$  ; il est **inutile** de la comparer à ses degrés de liberté (voir 2.4.4 pour le bon outil). Pour des comptages ou des données groupées à effectifs assez grands, en revanche, déviance et Pearson divisées par les degrés de liberté doivent être proches de 1 si le modèle est correct.

### 2.4.2 Comparer des modèles emboîtés : le test du rapport de vraisemblance

Deux modèles sont **emboîtés** si le plus petit s'obtient en fixant certains coefficients du plus grand à zéro. Si le petit modèle est correct, la différence de déviance suit, pour de grands échantillons, un  $\chi^2$  dont le nombre de degrés de liberté est le nombre de coefficients retirés :

$$D_{\text{réduit}} - D_{\text{complet}} = 2[\ell_{\text{complet}} - \ell_{\text{réduit}}] \approx \chi_q^2.$$

Pour retirer une variable à  $q$  niveaux (le canal en a trois :  $q = 2$ ), cette différence permet de tester **globalement** son utilité, ce que les tests de Wald coefficient par coefficient ne font pas.

```
def test_rv(complet, reduit, ddl):
    delta = reduit.deviance - complet.deviance
    return delta, ddl, stats.chi2.sf(delta, ddl)

formule_c = "rachat_12m ~ offre_bienvenue + age + canal"
complet = smf.glm(formule_c, clients, family=sm.families.Binomial()).fit()
essais = [("canal (2 ddl)", "rachat_12m ~ offre_bienvenue + age", 2),
          ("age (1 ddl)", "rachat_12m ~ offre_bienvenue + canal", 1),
          ("offre_bienvenue (1 ddl)", "rachat_12m ~ age + canal", 1)]
wald = complet.wald_test_terms(scalar=True).table
lignes = []
for nom, f_reduite, ddl in essais:
    reduit = smf.glm(f_reduite, clients, family=sm.families.Binomial()).fit()
    delta, q, p_rv = test_rv(complet, reduit, ddl)
    terme = nom.split(" ")[0]
    lignes.append({"variable retirée": nom, "D réduit - D complet": delta, "p (rapport de
↪ vraisemblance)": p_rv,
                  "p (Wald)": wald.loc[terme, "pvalue"]})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
print()
print("déviance du modèle complet :", round(complet.deviance, 2), "| déviance sans l'offre :",
↪ round(smf.glm("rachat_12m ~ age + canal", clients, family=sm.families.Binomial()).fit().deviance,
↪ 2))
```

| variable retirée        | D réduit - D complet | p (rapport de vraisemblance) | p (Wald) |
|-------------------------|----------------------|------------------------------|----------|
| canal (2 ddl)           | 17.7355              | 0.0001                       | 0.0001   |
| age (1 ddl)             | 12.7919              | 0.0003                       | 0.0004   |
| offre_bienvenue (1 ddl) | 29.6401              | 0.0000                       | 0.0000   |

déviance du modèle complet : 2710.83 | déviance sans l'offre : 2740.47

Retirer le canal (2 coefficients) augmente la déviance de 17,74 ( $p = 0,0001$ ) ; retirer l'âge, de 12,79 ( $p = 0,0003$ ) ; retirer l'offre, de 29,64 ( $p < 0,0001$ ) : la déviance du modèle complet est de 2 710,83, contre 2 740,47 sans l'offre. Les p-valeurs du rapport de vraisemblance et celles de Wald (qui testent les mêmes hypothèses) sont ici très proches (par exemple 0,0003 contre 0,0004 pour l'âge) : avec 2 000 observations et des effets modestes, les deux tests s'accordent. La différence apparaît pour de petits échantillons ou des effets très forts, où le rapport de vraisemblance est le plus fiable.

**Quand la dispersion  $\phi$  est inconnue** (Gamma, normale, quasi-Poisson), la différence de déviance divisée par  $\hat{\phi}$  n'est plus un  $\chi^2$  exact : on utilise un test  $F$  (comme en régression linéaire),

$$F = \frac{(D_{\text{réduit}} - D_{\text{complet}})/q}{\hat{\phi}_{\text{complet}}} \sim F_{q, n-p}.$$

Appliquons-le à la régression Gamma de 2.3 : le canal (2 ddl) et l'offre (1 ddl) ont-ils un effet sur la dépense des acheteurs ?

```
f_gamma = "depense_annuelle ~ age + canal + offre_bienvenue"
g_complet = smf.glm(f_gamma, acheteurs,
  ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
phi_g = g_complet.scale
for nom, f_reduite, ddl in [("canal (2 ddl)", "depense_annuelle ~ age + offre_bienvenue", 2),
  ↪ ("offre_bienvenue (1 ddl)", "depense_annuelle ~ age + canal", 1)]:
  g_reduit = smf.glm(f_reduite, acheteurs,
  ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
  F = (g_reduit.deviance - g_complet.deviance) / ddl / phi_g
  print(f"{nom:24s}: F = {F:7.2f} sur ({ddl}, {int(g_complet.df_resid)}) ddl | p = {stats.f.sf(F,
  ↪ ddl, g_complet.df_resid):.2e}")
```

```
canal (2 ddl)          : F = 37.09 sur (2, 1735) ddl | p = 1.69e-16
offre_bienvenue (1 ddl): F = 0.04 sur (1, 1735) ddl | p = 8.39e-01
```

Le canal a un effet très net sur la dépense des acheteurs ( $F = 37,09$  sur (2; 1 735) degrés de liberté,  $p \approx 10^{-16}$ ) ; l'offre, aucun ( $F = 0,04$ ,  $p = 0,84$ ). Pour un seul degré de liberté,  $F$  est le carré de la statistique  $t$  de Wald et les deux tests donnent la même p-valeur : on retrouve ici exactement la p-valeur 0,839 du tableau de la section 2.3.4.

### 2.4.3 Les résidus d'un GLM

Dans la régression linéaire, le résidu est  $y_i - \hat{y}_i$ , et son graphique contre la valeur ajustée doit ressembler à un nuage sans structure. Dans un GLM, la variance varie avec la moyenne, donc les résidus bruts  $y_i - \hat{\mu}_i$  ne sont pas comparables entre eux. On les **standardise** :

- **Résidu de Pearson** :  $r_i^P = \frac{y_i - \hat{\mu}_i}{\sqrt{\phi V(\hat{\mu}_i)}}$  (on divise par l'écart-type attendu). La somme de leurs carrés est la statistique de Pearson  $X^2$ .
- **Résidu de déviance** :  $r_i^D = \text{signe}(y_i - \hat{\mu}_i) \sqrt{d(y_i, \hat{\mu}_i)}$ . La somme de leurs carrés est la déviance  $D$ . Leur loi est souvent plus proche de la normale que celle des résidus de Pearson.

```

r_p = poi.resid_pearson.to_numpy()
r_d = poi.resid_deviance.to_numpy()
print("Poisson : somme des carrés des résidus de Pearson =", round(float((r_p ** 2).sum()), 1), "| X² =", round(float(poi.pearson_chi2), 1))
print("Poisson : somme des carrés des résidus de déviance =", round(float((r_d ** 2).sum()), 1), "| D =", round(float(poi.deviance), 1))
print("résidus de Pearson : moyenne =", round(float(r_p.mean()), 3), "| écart-type =", round(float(r_p.std()), 3), "(attendu : environ 1 si le modèle est correct)")
print("part de |résidus de Pearson| > 2 : ", round(float((np.abs(r_p) > 2).mean()), 3), "(attendu pour une loi normale : environ 0,046)")

```

```

Poisson : somme des carrés des résidus de Pearson = 6774.2 | X² = 6774.2
Poisson : somme des carrés des résidus de déviance = 6293.0 | D = 6293.0
résidus de Pearson : moyenne = -0.0 | écart-type = 1.84 (attendu : environ 1 si le modèle est correct)
part de |résidus de Pearson| > 2 : 0.175 (attendu pour une loi normale : environ 0,046)

```

Les deux sommes de carrés reproduisent exactement  $X^2 = 6774,2$  et  $D = 6293,0$ . L'écart-type des résidus de Pearson vaut **1,84** au lieu de 1 (c'est  $\sqrt{\hat{\phi}} = \sqrt{3,40}$ ), et 17,5 % d'entre eux dépassent 2 en valeur absolue, contre environ 4,6 % pour une loi normale : les résidus sont beaucoup trop dispersés, ce qui confirme la surdispersion vue en 2.3.3.

**Le problème des données discrètes.** Pour un 0/1 ou un comptage, les résidus de Pearson et de déviance prennent des valeurs **discrètes** : même avec le bon modèle, leur graphique ne ressemble pas à un nuage normal. On utilise alors les **résidus quantiles aléatoires** de Dunn et Smyth (1996), qui ont une propriété remarquable : si le modèle est correct, ils suivent **exactement** une loi normale standard, quelle que soit la famille.

**Construction.** Soit  $F_i$  la fonction de répartition prédite par le modèle pour l'observation  $i$  (Poisson de moyenne  $\hat{\mu}_i$ , etc.). Si  $Y_i$  est continue et que le modèle est correct,  $U_i = F_i(Y_i)$  suit une loi uniforme sur  $[0, 1]$  (c'est la **transformée intégrale de probabilité**), et  $\Phi^{-1}(U_i)$  suit une loi normale standard ( $\Phi$  : fonction de répartition normale). Si  $Y_i$  est discrète,  $F_i(Y_i)$  n'est pas uniforme (il prend un nombre fini de valeurs) ; on **randomise** : on tire  $U_i$  uniformément dans l'intervalle  $[F_i(y_i - 1), F_i(y_i)]$  (la marche de la fonction de répartition au point  $y_i$ ), puis on pose  $r_i = \Phi^{-1}(U_i)$ . Si le modèle est correct,  $U_i$  est uniforme et  $r_i \sim \mathcal{N}(0, 1)$ .

Appliquons-les à nos quatre modèles : la logistique, la régression de Poisson, la binomiale négative, la régression Gamma. Si le modèle est bon, le graphique « quantiles théoriques contre quantiles observés » (QQ-plot) suit la diagonale.

```

rng = np.random.default_rng(44)

def residus_quantiles(u_bas, u_haut):
    u = rng.uniform(u_bas, u_haut)
    return stats.norm.ppf(np.clip(u, 1e-12, 1 - 1e-12))

y_b = clients["rachat_12m"].to_numpy()
p_b = logi.fittedvalues.to_numpy()
rq_logi = residus_quantiles(np.where(y_b == 1, 1 - p_b, 0.0), np.where(y_b == 1, 1.0, 1 - p_b))

y_c = clients["nb_commandes_an"].to_numpy()
mu_p = poi.fittedvalues.to_numpy()
rq_poi = residus_quantiles(stats.poisson.cdf(y_c - 1, mu_p), stats.poisson.cdf(y_c, mu_p))

nbm = smf.negativebinomial("nb_commandes_an ~ age + canal + offre_bienvenue", clients).fit(dis=0)
a_nb, mu_nb = float(nbm.params["alpha"]), np.asarray(nbm.predict())
n_nb, p_nb = 1 / a_nb, (1 / a_nb) / ((1 / a_nb) + mu_nb)
rq_nb = residus_quantiles(stats.nbinom.cdf(y_c - 1, n_nb, p_nb), stats.nbinom.cdf(y_c, n_nb, p_nb))

y_g = acheteurs["depense_annuelle"].to_numpy()

```

```

mu_g = gam.fittedvalues.to_numpy()
u_g = stats.gamma.cdf(y_g, a=1 / gam.scale, scale=mu_g * gam.scale)
rq_gam = stats.norm.ppf(np.clip(u_g, 1e-12, 1 - 1e-12))

# Même le modèle SANS variable (probabilité constante) donne des résidus quantiles « parfaits » pour
↳ un 0/1
p_nul = np.full(len(y_b), y_b.mean())
rq_nul = residus_quantiles(np.where(y_b == 1, 1 - p_nul, 0.0), np.where(y_b == 1, 1.0, 1 - p_nul))

resume = []
for nom, r in [("logistique", rq_logi), ("logistique sans variable", rq_nul), ("Poisson", rq_poi),
↳ ("binomiale négative", rq_nb), ("Gamma", rq_gam)]:
    resume.append({"modèle": nom, "moyenne": r.mean(), "écart-type": r.std(), "part de |r| > 2":
↳ (np.abs(r) > 2).mean(),
                    "p (Kolmogorov-Smirnov)": stats.kstest(r, "norm").pvalue})
print(pd.DataFrame(resume).round(3).to_string(index=False))

fig, axes = plt.subplots(1, 4, figsize=(14, 3.7), sharex=True, sharey=True)
BLEU, ORANGE = "#2a78d6", "#eb6834"
for ax, (nom, r, c) in zip(axes, [("logistique", rq_logi, BLEU), ("Poisson", rq_poi, ORANGE),
↳ ("binomiale négative", rq_nb, BLEU), ("Gamma", rq_gam, BLEU)]):
    r = np.sort(r)
    theo = stats.norm.ppf((np.arange(1, len(r) + 1) - 0.5) / len(r))
    ax.plot(theo, r, ".", color=c, ms=3)
    ax.plot([-4, 4], [-4, 4], color="#52514e", lw=1)
    ax.set_title(nom); ax.set_xlabel("quantiles de la loi normale")
axes[0].set_ylabel("résidus quantiles aléatoires")
axes[0].set_xlim(-4, 4); axes[0].set_ylim(-4, 4)
plt.tight_layout()
plt.savefig("figures/ch02-residus-quantiles.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

|  | modèle                   | moyenne | écart-type | part de $ r  > 2$ | p (Kolmogorov-Smirnov) |
|--|--------------------------|---------|------------|-------------------|------------------------|
|  | logistique               | 0.010   | 1.009      | 0.048             | 0.896                  |
|  | logistique sans variable | -0.020  | 1.008      | 0.040             | 0.257                  |
|  | Poisson                  | -0.148  | 1.675      | 0.228             | 0.000                  |
|  | binomiale négative       | 0.004   | 0.991      | 0.045             | 0.849                  |
|  | Gamma                    | 0.087   | 0.833      | 0.023             | 0.000                  |

figure enregistrée

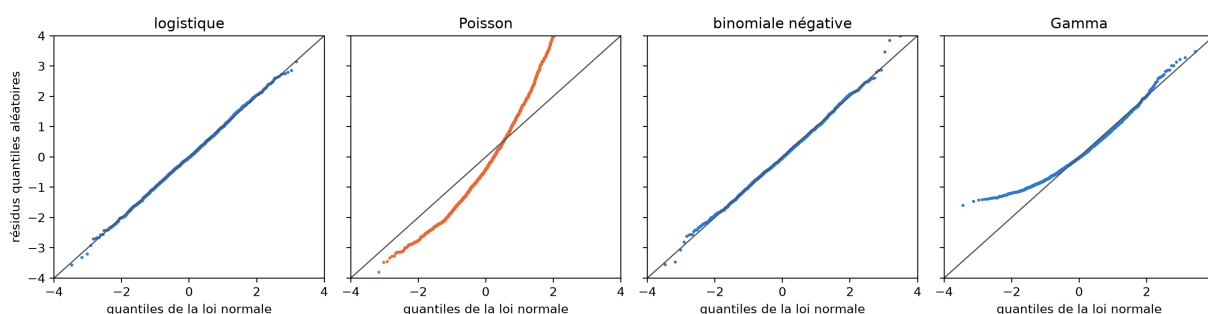


Figure 20: QQ-plots des résidus quantiles aléatoires pour quatre modèles. Si le modèle est correct, les points suivent la diagonale. Le modèle de Poisson (orange) s'en écarte nettement ; le modèle Gamma s'écarte aussi, dans la queue inférieure.

Lisons le tableau et les QQ-plots.

- **Poisson** : écart-type des résidus de 1,68, 22,8 % de résidus au-delà de  $\pm 2$  (pour 4,6 % attendus), p-valeur de Kolmogorov-Smirnov nulle. La courbe est nettement plus raide que la diagonale : les résidus sont trop dispersés, le modèle sous-estime la variabilité.
- **Binomiale négative** : moyenne 0,004, écart-type 0,991, 4,5 % au-delà de  $\pm 2$ ,  $p = 0,85$  : les

résidus sont **indiscernables d'une loi normale**. La famille est adaptée.

- **Gamma** : écart-type 0,83 et p-valeur nulle. La courbe est plate dans la queue inférieure (les résidus ne descendent pas sous  $-1,6$  environ) et trop haute dans la queue supérieure. Le modèle Gamma avec un coefficient de variation de 1 attend beaucoup de petites dépenses, alors qu'une dépense annuelle d'acheteur vaut au moins un panier, et qu'un panier est rarement très petit. Les **moyennes** prédites sont bonnes (nous l'avons vu en 2.3.4), mais la **forme** de la loi est fausse : la régression Gamma reste valide pour estimer l'effet des variables sur la moyenne (c'est une méthode de quasi-vraisemblance), mais il ne faudrait pas s'en servir pour simuler des dépenses ou calculer des intervalles de prévision. La section 2.6 propose un modèle plus adapté.
- **Logistique** : écart-type 1,009,  $p = 0,90$ . **Attention : ce résultat ne prouve rien**. Pour un résultat 0/1, les résidus quantiles aléatoires sont normaux dès que la probabilité *moyenne* est correcte : le modèle **sans aucune variable** donne lui aussi des résidus parfaits (écart-type 1,008,  $p = 0,26$ ). Pour une réponse binaire, le QQ-plot ne détecte rien ; il faut regarder la calibration, le test de Hosmer-Lemeshow et les résidus par classes (2.4.4).

#### 2.4.4 Mesurer la qualité d'ajustement

**Surdispersion.** Pour un modèle de comptage, on a déjà un indicateur : Pearson  $X^2/\text{ddl}$  doit être proche de 1. Pour la binomiale négative, la variance est  $\mu + \alpha\mu^2$  ; on calcule donc  $X^2$  avec cette variance.

```
x2_nb = np.sum((y_c - mu_nb) ** 2 / (mu_nb + a_nb * mu_nb ** 2))
ddl_nb = len(y_c) - len(nbm.params) + 1      # on retire le paramètre alpha du compte des
↳ coefficients
print("Poisson          : X²/ddl =", round(poi.pearson_chi2 / poi.df_resid, 3))
print("binomiale négative : X²/ddl =", round(float(x2_nb / ddl_nb), 3))

Poisson          : X²/ddl = 3.396
binomiale négative : X²/ddl = 1.044
```

Le rapport vaut **3,40** pour Poisson (nettement supérieur à 1 : surdispersion) et **1,04** pour la binomiale négative : avec la bonne forme de variance, la dispersion est correctement décrite.

**Le test de Hosmer-Lemeshow pour la régression logistique.** Pour des 0/1 individuels, on regroupe les clients par classes de probabilité prédite (dix classes de même effectif) et l'on compare, dans chaque classe  $k$ , le nombre observé de « oui »  $O_k$  au nombre attendu  $E_k = \sum_{i \in k} \hat{p}_i$ . La statistique

$$HL = \sum_{k=1}^g \frac{(O_k - E_k)^2}{E_k(1 - \bar{p}_k)} \approx \chi_{g-2}^2$$

(où  $\bar{p}_k = E_k/n_k$  est la probabilité moyenne de la classe) est grande si le modèle est mal calibré. C'est la version formelle de la courbe de calibration de la section 2.2.8.

```
def hosmer_lemeshow(y, p, g=10):
    classes = pd.qcut(p, g, labels=False, duplicates="drop")
    d = pd.DataFrame({"y": y, "p": p, "k": classes}).groupby("k").agg(0=("y", "sum"), E=("p", "sum"),
    ↳ n=("y", "size"))
    d["pbar"] = d["E"] / d["n"]
    hl = (((d["0"] - d["E"]) ** 2) / (d["E"] * (1 - d["pbar"]))).sum()
    ddl = len(d) - 2
    return hl, ddl, stats.chi2.sf(hl, ddl), d

hl, ddl, p_hl, tab = hosmer_lemeshow(clients["rachat_12m"].to_numpy(), logi.fittedvalues.to_numpy())
print(f"modèle de rachat : HL = {hl:.2f} sur {ddl} ddl, p = {p_hl:.3f}")

# Un modèle mal spécifié : sessions de navigation (simulées), effet de la durée en « cloche » mais
↳ modèle linéaire sur le logit
```

```

sessions = pd.read_csv("donnees/ch02-sessions.csv")
lineaire = smf.glm("achat ~ duree_min", sessions, family=sm.families.Binomial()).fit()
bosse = smf.glm("achat ~ duree_min + I(duree_min**2)", sessions, family=sm.families.Binomial()).fit()
for nom, res in [("linéaire sur le logit", lineaire), ("avec terme quadratique", bosse)]:
    hl_s, ddl_s, p_s, _ = hosmer_lemeshow(sessions["achat"].to_numpy(), res.fittedvalues.to_numpy())
    print(f"sessions, {nom:24s} : HL = {hl_s:6.2f} sur {ddl_s} ddl, p = {p_s:.4f} | AIC = {res.aic:.1f}")

```

```

modèle de rachat : HL = 6.15 sur 8 ddl, p = 0.631
sessions, linéaire sur le logit      : HL = 243.18 sur 8 ddl, p = 0.0000 | AIC = 1960.3
sessions, avec terme quadratique    : HL = 54.33 sur 8 ddl, p = 0.0000 | AIC = 1787.7

```

Pour le modèle de rachat,  $HL = 6,15$  sur 8 degrés de liberté ( $p = 0,63$ ) : rien n'indique une mauvaise calibration, ce qui confirme la lecture de la courbe de calibration de 2.2.8. Pour les sessions de navigation, au contraire, le modèle linéaire sur le logit est rejeté de façon écrasante ( $HL = 243$ ,  $AIC = 1960,3$ ). Ajouter un terme quadratique améliore beaucoup les choses ( $HL$  tombe à 54,3 et l'AIC à 1 787,7, soit **172 points** de moins), mais le test rejette encore : le modèle quadratique n'est **pas suffisant** non plus. Un test qui rejette dit « quelque chose ne va pas », pas « quoi » : regardons le graphique.

**Le graphique de résidus par classes.** Pour voir ce que le test détecte, on regroupe les sessions par tranches de durée et l'on compare, pour chaque tranche, la fréquence d'achat observée à celle prédite par le modèle linéaire.

```

tranches = pd.cut(sessions["duree_min"], bins=[0, 3, 5, 7, 9, 11, 13, 15, 18, 22, 40])
g = sessions.assign(p_lin=lineaire.fittedvalues, p_bosse=bosse.fittedvalues,
    ↪ tranche=tranches).groupby("tranche", observed=True).agg(
    duree=("duree_min", "mean"), observe=("achat", "mean"), p_lineaire=("p_lin", "mean"),
    ↪ p_bosse=("p_bosse", "mean"), n=("achat", "size"))
print(g.round(3).to_string())

fig, ax = plt.subplots(figsize=(7.2, 4.2))
BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
ax.plot(g["duree"], g["observe"], "o", color="#52514e", label="fréquence observée (par tranche)")
ax.plot(g["duree"], g["p_lineaire"], "-", color=ORANGE, lw=2, label="modèle linéaire sur le logit")
ax.plot(g["duree"], g["p_bosse"], "-", color=BLEU, lw=2, label="avec terme quadratique")
ax.set_xlabel("durée de la session (minutes)"); ax.set_ylabel("probabilité d'achat")
ax.set_title("Un modèle mal spécifié se voit dans les résidus par classes"); ax.legend(frameon=False,
    ↪ fontsize=9)
plt.tight_layout()
plt.savefig("figures/ch02-residus-par-classes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

| tranche  | duree  | observe | p_lineaire | p_bosse | n   |
|----------|--------|---------|------------|---------|-----|
| (0, 3]   | 2.265  | 0.189   | 0.630      | 0.174   | 37  |
| (3, 5]   | 4.240  | 0.333   | 0.584      | 0.343   | 114 |
| (5, 7]   | 6.113  | 0.381   | 0.539      | 0.489   | 231 |
| (7, 9]   | 8.100  | 0.650   | 0.490      | 0.581   | 240 |
| (9, 11]  | 10.051 | 0.740   | 0.443      | 0.600   | 223 |
| (11, 13] | 11.930 | 0.566   | 0.398      | 0.553   | 182 |
| (13, 15] | 14.071 | 0.320   | 0.350      | 0.422   | 147 |
| (15, 18] | 16.367 | 0.140   | 0.301      | 0.226   | 171 |
| (18, 22] | 19.996 | 0.079   | 0.232      | 0.037   | 101 |
| (22, 40] | 26.831 | 0.019   | 0.139      | 0.001   | 54  |

figure enregistrée

Le modèle linéaire (orange) prédit une probabilité d'achat **décroissante** avec la durée, alors que les fréquences observées montent jusqu'à environ 10 minutes puis redescendent : il prédit 0,63 pour les sessions de 2 minutes (observé : 0,19) et 0,44 autour de 10 minutes (observé : 0,74). Les résidus par classes dessinent une structure nette (négatifs, puis positifs, puis négatifs) : c'est



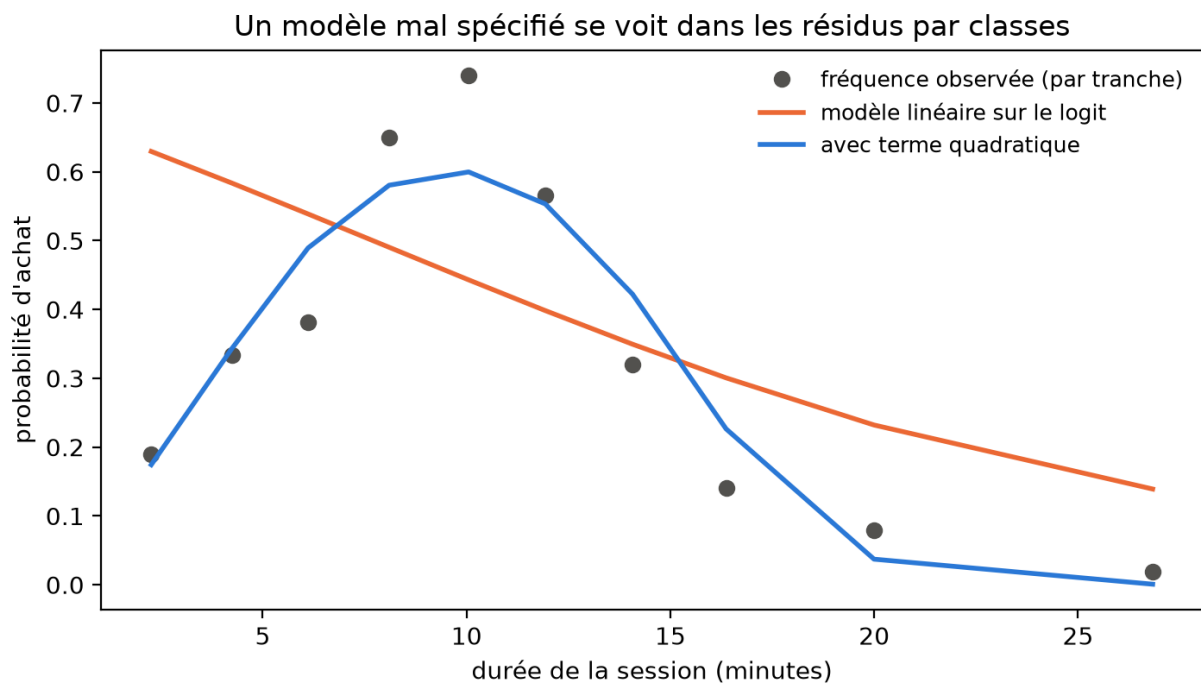


Figure 21: Fréquence d'achat observée par tranche de durée de session (points), modèle linéaire sur le logit (orange) et modèle avec terme quadratique (bleu). Le modèle linéaire rate la bosse ; le terme quadratique la capte mieux, mais pas parfaitement.

la signature d'un **mauvais choix de forme fonctionnelle**. Le terme quadratique (bleu) capte la bosse, mais pas parfaitement : il sous-estime le sommet (0,60 prédit, 0,74 observé à 10 minutes) et descend trop bas dans la queue (0,001 prédit contre 0,019 observé au-delà de 22 minutes). Une parabole sur le logit retombe trop vite : il faut une forme plus souple, celle des modèles additifs généralisés (section 2.5).

### 2.4.5 Comparer des modèles non emboîtés : AIC et BIC

Le test du rapport de vraisemblance ne vaut que pour des modèles emboîtés. Pour comparer des modèles quelconques (par exemple Poisson et binomiale négative, ou deux ensembles de variables différents), on utilise des **critères d'information**, qui pénalisent la vraisemblance par le nombre de paramètres  $k$  :

$$\text{AIC} = -2\ell + 2k, \quad \text{BIC} = -2\ell + k \log n.$$

Plus la valeur est **petite**, mieux c'est. Le BIC pénalise plus lourdement la complexité dès que  $n > 7$ , et conduit à des modèles plus parcimonieux. Ils ne mesurent que la qualité *relative* des modèles comparés : le meilleur d'une liste de mauvais modèles reste un mauvais modèle (d'où l'importance des résidus).

```
formules = {
    "aucune variable": "rachat_12m ~ 1",
    "+ offre": "rachat_12m ~ offre_bienvenue",
    "+ offre + âge": "rachat_12m ~ offre_bienvenue + age",
    "+ offre + âge + canal": "rachat_12m ~ offre_bienvenue + age + canal",
    "+ offre × canal (interactions)": "rachat_12m ~ offre_bienvenue * canal + age",
}
lignes = []
ajustes = {}
for nom, f in formules.items():
```



```

r = smf.glm(f, clients, family=sm.families.Binomial()).fit()
ajustes[nom] = r
lignes.append({"modèle": nom, "paramètres": len(r.params), "déviante": r.deviance, "AIC": r.aic,
↪ "BIC": r.bic_2llf})
res = pd.DataFrame(lignes)
res["ΔAIC"] = res["AIC"] - res["AIC"].min()
res["ΔBIC"] = res["BIC"] - res["BIC"].min()
print(res.round(1).to_string(index=False))
delta_inter = ajustes["+ offre + âge + canal"].deviance - ajustes["+ offre × canal
↪ (interactions)"].deviance
print(f"\ntest du rapport de vraisemblance pour les 2 interactions : différence de déviante =
↪ {delta_inter:.2f}, p = {stats.chi2.sf(delta_inter, 2):.3f}")

```

|  | modèle                         | paramètres | déviante | AIC    | BIC    | ΔAIC | ΔBIC |
|--|--------------------------------|------------|----------|--------|--------|------|------|
|  | aucune variable                | 1          | 2771.9   | 2773.9 | 2779.5 | 57.8 | 30.6 |
|  | + offre                        | 2          | 2742.1   | 2746.1 | 2757.3 | 30.1 | 8.5  |
|  | + offre + âge                  | 3          | 2728.6   | 2734.6 | 2751.4 | 18.5 | 2.5  |
|  | + offre + âge + canal          | 5          | 2710.8   | 2720.8 | 2748.8 | 4.8  | 0.0  |
|  | + offre × canal (interactions) | 7          | 2702.1   | 2716.1 | 2755.3 | 0.0  | 6.4  |

test du rapport de vraisemblance pour les 2 interactions : différence de déviante = 8.77, p = 0.012

L'AIC diminue à chaque variable ajoutée (de  $\Delta = 57,8$  pour le modèle sans variable jusqu'à 0 pour le modèle avec interactions) et **retient donc le modèle le plus complexe**. Le BIC, plus sévère, est minimal pour le modèle **sans interactions** (celui avec les interactions a  $\Delta BIC = 6,4$ ). Le test du rapport de vraisemblance donne  $p = 0,012$  pour les deux interactions. C'est ici un cas limite où les critères divergent. Que faire ? Une interaction que l'on n'avait **pas prévue** et qui n'est « significative » qu'à  $p = 0,012$  est typiquement un résultat à regarder avec méfiance (volume I, section 3.5.5 : quand on teste beaucoup d'effets, quelques-uns paraissent significatifs par hasard) ; on garde de préférence le modèle plus simple, plus facile à expliquer, sauf raison métier d'attendre une interaction. Nous verrons en 2.7 ce qu'il en était réellement.

## 2.4.6 Observations influentes

Une observation peut peser démesurément sur l'ajustement : par son **levier** (ses variables explicatives sont extrêmes) et par son **résidu** (le modèle la prédit mal). La **distance de Cook** combine les deux et mesure de combien les coefficients bougeraient si on retirait cette observation.

```

infl = logi.get_influence()
levier = infl.hat_matrix_diag
cook = infl.cooks_distance[0]
print("levier moyen =", round(float(levier.mean()), 4), "(= p/n =", round(5 / len(clients), 4), ") |
↪ levier maximal =", round(float(levier.max()), 4))
print("distance de Cook maximale =", round(float(cook.max()), 4), "| seuil usuel 4/n =", round(4 /
↪ len(clients), 4), "| clients au-dessus du seuil :", int((cook > 4 / len(clients)).sum()))
pire = clients.loc[int(np.argmax(cook)), ["age", "canal_acquisition", "offre_bienvenue",
↪ "rachat_12m"]]
print("client le plus influent :", pire.to_dict())
print("probabilité de rachat prédite pour ce client :",
↪ round(float(logi.fittedvalues.iloc[int(np.argmax(cook))]), 3))

```

```

levier moyen = 0.0025 (= p/n = 0.0025 ) | levier maximal = 0.0076
distance de Cook maximale = 0.0023 | seuil usuel 4/n = 0.002 | clients au-dessus du seuil : 3
client le plus influent : {'age': 68, 'canal_acquisition': 'Boutique', 'offre_bienvenue': 0,
↪ 'rachat_12m': 1}
probabilité de rachat prédite pour ce client : 0.386

```

Le levier moyen vaut exactement  $p/n = 5/2000 = 0,0025$  (c'est toujours le cas), et le levier maximal 0,0076, soit environ trois fois la moyenne : aucun client n'est extrême dans ses variables. La distance de Cook maximale n'est que de 0,0023, à peine au-dessus du seuil usuel  $4/n = 0,002$

(que trois clients dépassent) : des valeurs de l'ordre du millième n'inquiètent pas. Le client le plus influent (68 ans, acquis en boutique, sans offre) avait une probabilité prédite de 38,6 % de racheter, et il a racheté : un âge élevé, donc proche de l'extrémité de la plage d'âges, et un résultat un peu surprenant, mais rien d'aberrant ; à en juger par sa distance de Cook, le retirer ne déplacerait les coefficients que de très peu.

### 2.4.7 Une démarche en quatre temps

#### La checklist de vérification d'un GLM.

1. **La famille et le lien conviennent-ils ?** Résidus quantiles aléatoires (QQ-plot) ;  $X^2/ddl$  pour la surdispersion ; histogramme des observations.
2. **La forme de chaque effet est-elle bonne ?** Résidus (ou fréquences observées) contre chaque variable explicative, par classes ; Hosmer-Lemeshow pour la calibration ; ajouter un terme quadratique ou un GAM (2.5) en cas de courbure.
3. **Y a-t-il des observations influentes ?** Levier, distance de Cook ; refaire l'ajustement sans les observations suspectes pour voir si les conclusions changent.
4. **Le modèle est-il utile ?** Pseudo- $R^2$ , AUC ou erreur de prévision, **sur des données de test** ; comparaison avec un modèle de référence simple.

#### À retenir

- La **déviance**  $D = 2(\ell_{\text{sat}} - \ell)$  est l'équivalent GLM de la somme des carrés des résidus. Le **rapport de vraisemblance** (différence de déviances entre modèles emboîtés) est le test de référence ; avec une dispersion estimée, on utilise un test  $F$ .
- Les **résidus de Pearson et de déviance** servent aux comptages ; pour des données discrètes, préférez les **résidus quantiles aléatoires** : normaux si le modèle est bon.
- Pour des 0/1 individuels, la déviance **n'est pas** un test d'ajustement ; utilisez la calibration et le test de Hosmer-Lemeshow.
- **AIC / BIC** comparent des modèles quelconques (plus petit = mieux), mais ne disent rien de la qualité absolue.
- Contrôlez toujours : **dispersion, forme des effets, observations influentes, performance hors échantillon.**

## 2.5 ⊕ Pour aller plus loin : les modèles additifs généralisés (GAM)

**Section optionnelle.** Elle prolonge naturellement 2.4 : vous y verrez comment *corriger* un modèle dont la forme fonctionnelle est mauvaise, sans deviner la bonne formule. Vous pouvez passer directement à 2.6.

**Intuition.** Dans un GLM, chaque variable continue agit de façon **linéaire** sur le prédicteur :  $\eta = \beta_0 + \beta_1 x_1 + \dots$ . En 2.4, nous avons vu que cela peut être faux : la probabilité d'achat d'une session de navigation **monte puis redescend** avec sa durée, et ni la droite ni la parabole n'ont suffi. Un **modèle additif généralisé** (*generalized additive model*, GAM) remplace chaque terme linéaire  $\beta_j x_j$  par une **fonction lisse inconnue**  $f_j(x_j)$ , que les données dessinent elles-mêmes :

$$g(\mu_i) = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \dots$$

On garde l'**additivité** (les effets des variables s'additionnent, donc restent faciles à lire un par un) et l'on perd la **linéarité**.

### 2.5.1 Le problème

Reprenons les 1 500 sessions du fichier `donnees/ch02-sessions.csv` (simulées, graine 252) : pour chaque session, sa durée en minutes et un indicateur d'achat. Nous connaissons la vérité (c'est la force des données simulées) :

$$\text{logit } P(\text{achat}) = -1,8 + 3e^{-((d-10)/5)^2} - 0,04d,$$

où  $d$  est la durée : une « bosse » autour de 10 minutes (trop court, le visiteur n'a pas assez regardé ; trop long, il est perdu), sur une pente légèrement décroissante. Nous allons voir si l'on peut la retrouver **sans connaître cette formule**.

### 2.5.2 Fonctions de base : un lissage comme une régression

L'idée est d'écrire la fonction inconnue comme une combinaison de **fonctions de base** connues,  $f(x) = \sum_{k=1}^K \gamma_k B_k(x)$ . Une fois les  $B_k$  choisies,  $f$  est **linéaire dans les paramètres**  $\gamma_k$  : on a de nouveau un GLM, avec pour variables explicatives les  $B_k(x)$ .

**Le plus petit exemple : la base « chapeau ».** Plaçons des **nœuds** en 0, 10, 20, 30 minutes. La fonction  $B_k(x) = \max(0, 1 - |x - t_k|/10)$  vaut 1 au nœud  $t_k$  et décroît linéairement jusqu'à 0 aux nœuds voisins. Avec les coefficients  $\gamma = (-1,8; 1,0; -1,2; -2,8)$ , la fonction  $f$  est la **ligne brisée** qui passe par ces valeurs aux nœuds. Calculons-la en 5 et en 14 minutes : à  $x = 5$ , seuls  $B_0$  et  $B_1$  sont non nuls (poids 0,5 et 0,5), donc  $f(5) = 0,5 \times (-1,8) + 0,5 \times 1,0 = -0,4$  ; à  $x = 14$ ,  $B_1 = 0,6$  et  $B_2 = 0,4$ , donc  $f(14) = 0,6 \times 1,0 + 0,4 \times (-1,2) = 0,12$ .

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.special import expit

noeuds = np.array([0.0, 10.0, 20.0, 30.0])
def chapeau(x, k):
    return np.maximum(0.0, 1 - np.abs(x - noeuds[k]) / 10)

gamma = np.array([-1.8, 1.0, -1.2, -2.8])
for x in (5.0, 10.0, 14.0, 25.0):
    poids = [round(float(chapeau(x, k)), 2) for k in range(4)]
    f = sum(gamma[k] * chapeau(x, k) for k in range(4))
    print(f"x = {x:4.1f} : poids des 4 fonctions de base = {poids} -> f(x) = {f:.3f}")

x = 5.0 : poids des 4 fonctions de base = [0.5, 0.5, 0.0, 0.0] -> f(x) = -0.400
x = 10.0 : poids des 4 fonctions de base = [0.0, 1.0, 0.0, 0.0] -> f(x) = 1.000
x = 14.0 : poids des 4 fonctions de base = [0.0, 0.6, 0.4, 0.0] -> f(x) = 0.120
x = 25.0 : poids des 4 fonctions de base = [0.0, 0.0, 0.5, 0.5] -> f(x) = -2.000
```

Le code confirme nos deux calculs à la main :  $f(5) = -0,400$  et  $f(14) = 0,120$ . En 10 minutes (un nœud), un seul poids est non nul (égal à 1) et  $f(10) = \gamma_1 = 1,0$  ; en 25 minutes, les poids sont 0,5 et 0,5 sur les deux derniers nœuds, d'où  $f(25) = -2,0$ . La courbe  $f$  est la ligne brisée qui relie les points  $(0; -1,8)$ ,  $(10; 1,0)$ ,  $(20; -1,2)$ ,  $(30; -2,8)$  : estimer les  $\gamma_k$  par maximum de vraisemblance revient à *dessiner* cette courbe à partir des données.

La base « chapeau » donne des courbes anguleuses. Pour obtenir des courbes **lisses**, on remplace les triangles par des polynômes de degré 3 raccordés aux nœuds : les **B-splines cubiques**. Leur principe est le même (chacune est non nulle sur un petit intervalle seulement, ce qui rend le calcul stable), et patsy les fournit par `bs(x, df=...)`. Le nombre de fonctions de base (`df`) règle

la **souplesse** : peu de fonctions donnent une courbe rigide, beaucoup de fonctions une courbe très flexible. Ajustons une régression logistique sur des bases de 3, 6 et 15 fonctions, sans pénalité, et comparons-les à la vérité.

```

sessions = pd.read_csv("donnees/ch02-sessions.csv")
def vrai_p(d):
    return expit(-1.8 + 3 * np.exp(-((d - 10) / 5) ** 2) - 0.04 * d)

grille = np.linspace(1, 35, 200)
p_vrai_obs = vrai_p(sessions["duree_min"].to_numpy()) # vraie probabilité de chaque session
rmse = lambda p_hat: float(np.sqrt(np.mean((p_hat - p_vrai_obs) ** 2)))

modeles = {
    "linéaire": "achat ~ duree_min",
    "quadratique": "achat ~ duree_min + I(duree_min**2)",
    "spline, 3 fonctions de base": "achat ~ bs(duree_min, df=3, lower_bound=0, upper_bound=40)",
    "spline, 6 fonctions de base": "achat ~ bs(duree_min, df=6, lower_bound=0, upper_bound=40)",
    "spline, 15 fonctions de base": "achat ~ bs(duree_min, df=15, lower_bound=0, upper_bound=40)",
}
courbes, lignes = {}, []
for nom, f in modeles.items():
    r = smf.glm(f, sessions, family=sm.families.Binomial()).fit()
    courbes[nom] = r.predict(pd.DataFrame({"duree_min": grille})).to_numpy()
    lignes.append({"modèle": nom, "paramètres": len(r.params), "AIC": r.aic, "erreur quadratique vs
    ↪ vérité": rmse(r.fittedvalues.to_numpy())})
print(pd.DataFrame(lignes).round(4).to_string(index=False))

```

|  | modèle                       | paramètres | AIC       | erreur quadratique vs vérité |
|--|------------------------------|------------|-----------|------------------------------|
|  | linéaire                     | 2          | 1960.2911 | 0.1962                       |
|  | quadratique                  | 3          | 1787.7267 | 0.0776                       |
|  | spline, 3 fonctions de base  | 4          | 1739.8372 | 0.0630                       |
|  | spline, 6 fonctions de base  | 7          | 1703.4622 | 0.0306                       |
|  | spline, 15 fonctions de base | 16         | 1708.0771 | 0.0491                       |

L'« erreur quadratique vs vérité » est l'écart (en probabilité) entre la probabilité prédite et la vraie probabilité, moyenné sur les 1 500 sessions : un chiffre qu'on ne peut calculer que parce que les données sont simulées. La droite est très loin (0,196, AIC 1 960,3) ; la parabole progresse (0,078, AIC 1 787,7) ; la spline à 3 fonctions de base (un polynôme de degré 3) fait mieux (0,063, AIC 1 739,8). **La spline à 6 fonctions de base est la meilleure** (erreur 0,031, AIC 1 703,5). Avec 15 fonctions, l'AIC remonte (1 708,1) et, surtout, **l'erreur par rapport à la vérité remonte aussi** (0,049) : plus de souplesse n'est pas toujours mieux, la courbe commence à suivre le bruit. C'est le compromis biais-variance en action.

### 2.5.3 La pénalisation : beaucoup de flexibilité, mais disciplinée

Choisir le nombre de fonctions de base est un arbitrage **biais-variance** (volume I, section 3.2.2) : trop peu et la courbe est biaisée, trop et elle suit le bruit. Plutôt que de choisir *exactement* le bon nombre, les GAM utilisent une idée plus élégante : prendre **beaucoup** de fonctions, mais **pénaliser la courbure**. On maximise

$$\ell(\gamma) - \frac{\lambda}{2} \int f''(x)^2 dx = \ell(\gamma) - \frac{\lambda}{2} \gamma^\top S \gamma,$$

où  $\ell$  est la log-vraisemblance,  $f''$  la dérivée seconde de  $f$  (sa courbure), et  $S$  une matrice connue qui ne dépend que de la base. Le paramètre  $\lambda \geq 0$  règle l'arbitrage :

- $\lambda = 0$  : aucune pénalité, la courbe est libre de zigzaguer (comme les 15 fonctions ci-dessus) ;
- $\lambda \rightarrow \infty$  : la courbure est interdite, la courbe devient une **droite** (une droite a une dérivée seconde nulle) : on retombe sur le GLM ordinaire ;

- entre les deux : un lissage adapté aux données.

La mise à jour d'IRLS (2.1.5) devient  $\hat{\gamma} = (X^T W X + \lambda S)^{-1} X^T W z$  : les mêmes régressions pondérées, avec un terme de plus. La complexité réelle de la courbe se mesure alors par ses **degrés de liberté effectifs** (edf), la trace de la matrice qui transforme les observations en valeurs ajustées : de 1 (droite) à  $K$  (aucune pénalité). On choisit  $\lambda$  en optimisant un critère (AIC, validation croisée généralisée, REML). Avec statsmodels, nous balayons une grille de valeurs de  $\lambda$  (alpha) et gardons l'AIC minimal.

```
from statsmodels.gam.api import GLMGam, BSplines

base = BSplines(sessions[["duree_min"]], df=[12], degree=[3])           # 12 fonctions de base : large,
↳ la pénalité fera le tri
y_s = sessions["achat"].to_numpy()
X0 = np.ones((len(sessions), 1))                                     # constante seule : tout le reste
↳ est dans le lissage

lignes, ajustes = [], {}
for alpha in [0.01, 0.1, 1, 10, 30, 100, 1000, 10000]:
    res = GLMGam(y_s, exog=X0, smoother=base, alpha=alpha, family=sm.families.Binomial()).fit()
    ajustes[alpha] = res
    lignes.append({"alpha (lambda)": alpha, "edf du lissage": res.edf.sum() - 1, "AIC": res.aic,
                  "erreur quadratique vs vérité": rmse(res.fittedvalues)})
tab_alpha = pd.DataFrame(lignes)
print(tab_alpha.round(4).to_string(index=False))
alpha_opt = float(tab_alpha.loc[tab_alpha["AIC"].idxmin(), "alpha (lambda)"])
gam = ajustes[alpha_opt]
print("\nalpha retenu (AIC minimal) :", alpha_opt, "| edf du lissage =", round(float(gam.edf.sum() -
↳ 1), 2))
```

| alpha (lambda) | edf du lissage | AIC       | erreur quadratique vs vérité |
|----------------|----------------|-----------|------------------------------|
| 0.01           | 10.9622        | 1709.0463 | 0.0347                       |
| 0.10           | 10.6634        | 1708.4685 | 0.0343                       |
| 1.00           | 9.2056         | 1705.9029 | 0.0325                       |
| 10.00          | 6.5311         | 1702.7064 | 0.0269                       |
| 30.00          | 5.1937         | 1704.1220 | 0.0264                       |
| 100.00         | 3.8272         | 1711.9038 | 0.0380                       |
| 1000.00        | 1.9769         | 1773.7390 | 0.0995                       |
| 10000.00       | 1.2678         | 1900.7931 | 0.1715                       |

alpha retenu (AIC minimal) : 10.0 | edf du lissage = 6.53

Quand la pénalité  $\lambda$  augmente, les degrés de liberté effectifs diminuent de 11,0 (presque les 12 fonctions de base, sans contrainte pour  $\lambda = 0,01$ ) à 1,27 (presque une droite, pour  $\lambda = 10\,000$ ). L'AIC est minimal pour  $\lambda = 10$  : **6,53 degrés de liberté effectifs**, AIC = 1 702,7. C'est mieux que la meilleure spline non pénalisée (6 fonctions : AIC 1 703,5), et surtout on n'a pas eu à choisir un nombre de fonctions de base : on a pris une base large et laissé la pénalité décider. L'erreur par rapport à la vérité est de 0,027 (la valeur la plus basse du tableau, 0,026, est obtenue pour  $\lambda = 30$  : l'AIC et l'erreur « vraie » ne coïncident pas parfaitement, mais choisissent des valeurs voisines).

Le test de Hosmer-Lemeshow de la section 2.4.4 (la fonction `hosmer_lemeshow` y est définie) dit-il toujours que le modèle est mauvais ?

```
for nom, p in [("linéaire", smf.glm("achat ~ duree_min", sessions,
↳ family=sm.families.Binomial()).fit().fittedvalues.to_numpy()),
              ("quadratique", smf.glm("achat ~ duree_min + I(duree_min**2)", sessions,
↳ family=sm.families.Binomial()).fit().fittedvalues.to_numpy()),
              ("GAM pénalisé", gam.fittedvalues)]:
    hl, ddl, p_val, _ = hosmer_lemeshow(y_s, p)
    print(f"{nom:14s}: HL = {hl:7.2f} sur {ddl} ddl, p = {p_val:.4f}")
```

```
linéaire      : HL = 243.18 sur 8 ddl, p = 0.0000
quadratique   : HL = 54.33 sur 8 ddl, p = 0.0000
GAM pénalisé  : HL = 6.29 sur 8 ddl, p = 0.6146
```

Le test de Hosmer-Lemeshow, qui rejetait la droite ( $HL = 243$ ) et même la parabole ( $HL = 54,3$ ), **ne rejette plus le GAM** :  $HL = 6,29$  sur 8 degrés de liberté,  $p = 0,61$ . Le modèle capte maintenant la forme de la relation.

Dessignons le tout : à gauche, les bases non pénalisées de 3, 6 et 15 fonctions ; à droite, le GAM pénalisé, comparé à la vérité.

```
p_gam = gam.predict(exog=np.ones((len(grille), 1)), exog_smooth=grille[:, None])
tranches = pd.cut(sessions["duree_min"], bins=[0, 3, 5, 7, 9, 11, 13, 15, 18, 22, 36])
pts = sessions.groupby(tranches, observed=True).agg(x=("duree_min", "mean"), y=("achat", "mean"))

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
BLEU, ORANGE, AQUA, VIOLET, GRIS = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#52514e"
ax = axes[0]
ax.scatter(pts["x"], pts["y"], color=GRIS, s=24, zorder=3)
ax.plot(grille, vrai_p(grille), color=GRIS, ls=":", lw=1.3)
for nom, c in [("spline, 3 fonctions de base", ORANGE), ("spline, 6 fonctions de base", BLEU),
               ("spline, 15 fonctions de base", VIOLET)]:
    ax.plot(grille, courbes[nom], color=c, lw=1.8)
ax.text(24, 0.62, "3 fonctions : trop rigide", color=ORANGE, fontsize=9)
ax.text(24, 0.55, "6 fonctions", color=BLEU, fontsize=9)
ax.text(24, 0.48, "15 fonctions : plus nerveux", color=VIOLET, fontsize=9)
ax.set_xlabel("durée de la session (minutes)"); ax.set_ylabel("probabilité d'achat");
    ax.set_title("Splines sans pénalité")
ax = axes[1]
ax.scatter(pts["x"], pts["y"], color=GRIS, s=24, zorder=3, label="fréquence observée (par tranche)")
ax.plot(grille, vrai_p(grille), color=GRIS, ls=":", lw=1.3, label="vérité (connue car simulée)")
ax.plot(grille, p_gam, color=AQUA, lw=2, label=f"GAM pénalisé (edf = {gam.edf.sum() - 1:.1f})")
ax.set_xlabel("durée de la session (minutes)"); ax.set_title("Spline pénalisée (GAM)")
ax.legend(frameon=False, fontsize=8, loc="upper right")
plt.tight_layout()
plt.savefig("figures/ch02-gam-sessions.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
print("sessions de plus de 30 minutes :", int((sessions["duree_min"] > 30).sum()), "sur",
      len(sessions), "| de moins de 3 minutes :", int((sessions["duree_min"] < 3).sum()))
print("probabilité prédite par le GAM à 35 min :", round(float(p_gam[-1]), 3), "| vraie :",
      round(float(vrai_p(35.0)), 3))
```

```
figure enregistrée
sessions de plus de 30 minutes : 10 sur 1500 | de moins de 3 minutes : 33
probabilité prédite par le GAM à 35 min : 0.135 | vraie : 0.039
```

À gauche, les trois bases sans pénalité. La spline à 3 fonctions (orange) est trop rigide : elle sous-estime le sommet et s'effondre vers 0 à gauche. Celle à 6 fonctions (bleu) suit bien les points. Celle à 15 fonctions (violet) est « nerveuse » : elle oscille entre 3 et 6 minutes et explose aux deux bords (elle prévoit 0,67 pour une session d'une minute, alors que la vraie valeur est d'environ 0,15). À droite, le GAM pénalisé (vert, 6,5 degrés de liberté effectifs) reproduit presque exactement la vérité (pointillés) : le sommet vers 10 minutes, la décroissance, et la longue queue. Un défaut subsiste : la courbe remonte à droite, à 35 minutes (0,135 prédit pour 0,039 en réalité). C'est un effet de bord : seules 10 sessions sur 1 500 durent plus de 30 minutes, donc la courbe y est très peu contrainte (le même phénomène se voit à gauche, où 33 sessions seulement durent moins de 3 minutes). Les courbes lisses sont peu fiables aux extrémités des données.



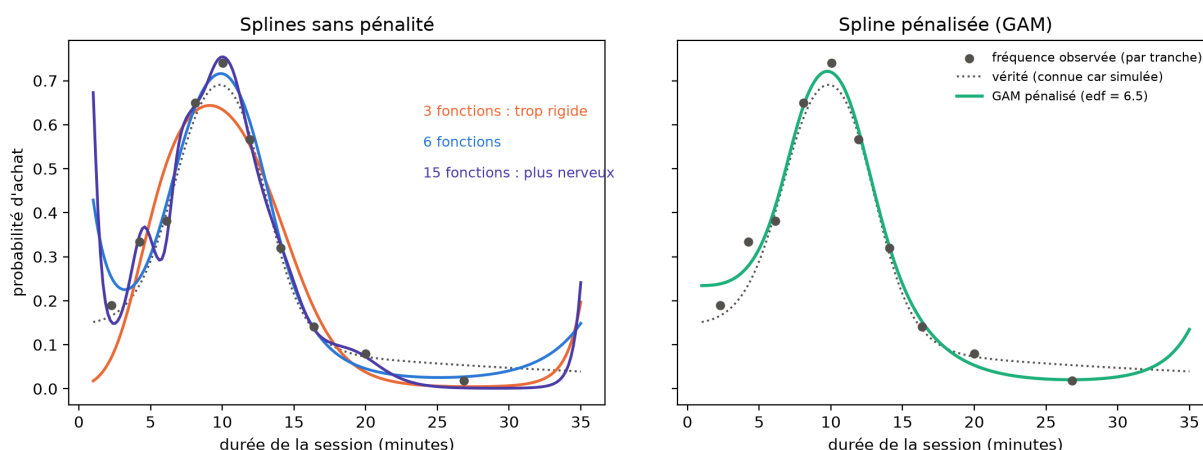


Figure 22: Probabilité d'achat selon la durée de la session. Points : fréquences observées par tranche ; pointillés : la vraie courbe. À gauche, des splines sans pénalité de souplesse croissante ; à droite, le GAM pénalisé.

### 2.5.4 Les GAM avec R et mgcv

Le paquet R **mgcv** (Simon Wood) est la référence pour les GAM : il choisit automatiquement  $\lambda$  par REML, donne des **intervalles de confiance** et des tests sur les termes lisses. Le même modèle s'écrit en une ligne : `gam(achat ~ s(duree_min), family = binomial, method = "REML")`.

```
suppressPackageStartupMessages(library(mgcv))
sessions <- read.csv("donnees/ch02-sessions.csv")
m <- gam(achat ~ s(duree_min), family = binomial, data = sessions, method = "REML")
print(summary(m))

grille <- data.frame(duree_min = seq(1, 35, length.out = 200))
p <- predict(m, grille, type = "link", se.fit = TRUE)
vrai <- plogis(-1.8 + 3 * exp(-((grille$duree_min - 10) / 5)^2) - 0.04 * grille$duree_min)
tranche <- cut(sessions$duree_min, breaks = c(0, 3, 5, 7, 9, 11, 13, 15, 18, 22, 36))
pts <- aggregate(cbind(duree_min, achat) ~ tranche, data = sessions, FUN = mean)

png("figures/ch02-gam-mgcv.png", width = 1300, height = 800, res = 200)
par(mar = c(4.2, 4.2, 2.2, 0.8))
plot(grille$duree_min, plogis(p$fit), type = "n", ylim = c(0, 0.85), xlab = "durée de la session
  (minutes)",
  ylab = "probabilité d'achat", main = "GAM (mgcv, REML) avec intervalle de confiance à 95 %")
polygon(c(grille$duree_min, rev(grille$duree_min)), c(plogis(p$fit - 1.96 * p$se.fit),
  rev(plogis(p$fit + 1.96 * p$se.fit))),
  col = "#cde2fb", border = NA)
lines(grille$duree_min, plogis(p$fit), col = "#2a78d6", lwd = 2)
lines(grille$duree_min, vrai, col = "#52514e", lty = 3, lwd = 1.5)
points(pts$duree_min, pts$achat, pch = 16, col = "#52514e")
invisible(legend("topright", legend = c("GAM ajusté", "intervalle à 95 %", "vérité (simulée)",
  "fréquences observées"),
  lty = c(1, NA, 3, NA), pch = c(NA, 15, NA, 16), col = c("#2a78d6", "#cde2fb",
  "#52514e", "#52514e"), bty = "n", cex = 0.8))
invisible(dev.off())
cat("figure enregistrée\n")
dans_bande <- vrai >= plogis(p$fit - 1.96 * p$se.fit) & vrai <= plogis(p$fit + 1.96 * p$se.fit)
cat("part de la grille où la vraie courbe est dans la bande :", round(mean(dans_bande), 3), "\n")
```

Family: binomial  
Link function: logit

Formula:



```

achat ~ s(duree_min)

Parametric coefficients:
      Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.4779    0.0696  -6.867 6.58e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
      edf Ref.df Chi.sq p-value
s(duree_min) 6.5  7.429 243.6 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.211  Deviance explained = 17.5%
-REML = 858.59  Scale est. = 1          n = 1500
figure enregistrée
part de la grille où la vraie courbe est dans la bande : 1

```

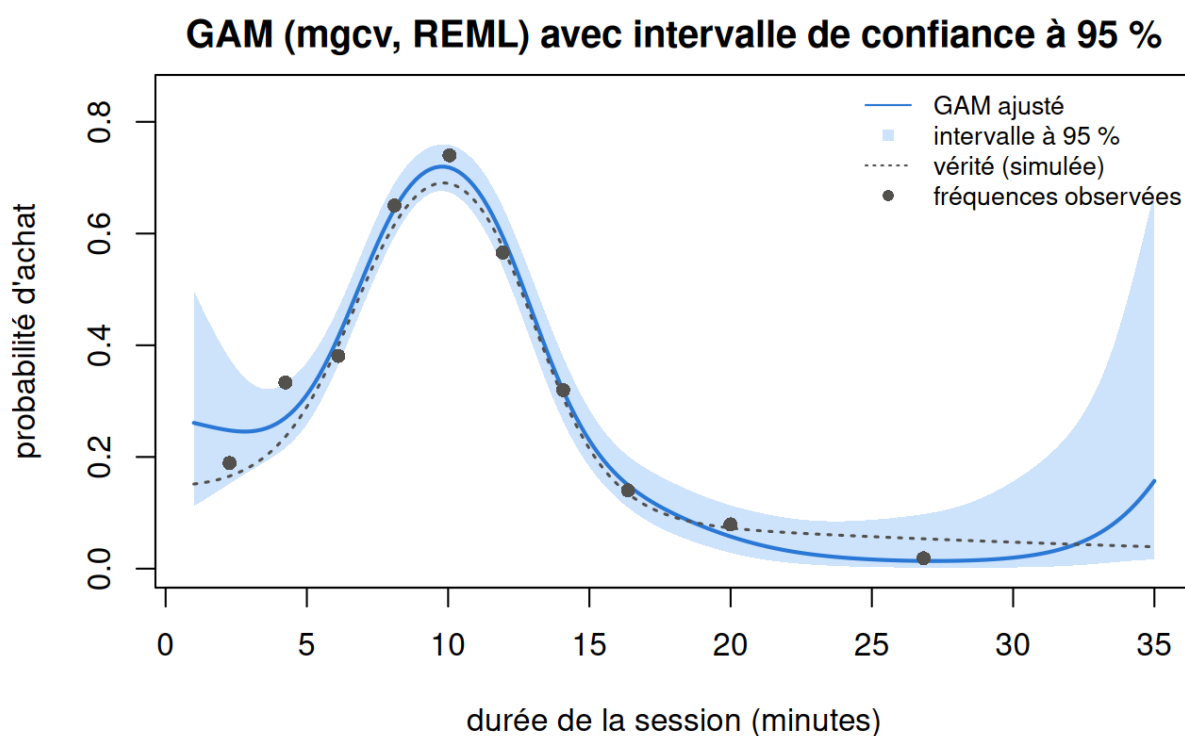


Figure 23: Le même GAM ajusté avec mgcv (REML) : courbe estimée, bande de confiance à 95 % et vraie courbe (pointillés).

mgcv choisit  $\lambda$  par REML et obtient **6,5 degrés de liberté effectifs**, la même complexité que notre balayage par AIC (6,53) : deux méthodes différentes aboutissent à la même courbe. Le test sur le terme lisse est sans appel ( $\chi^2 = 243,6$ ,  $p < 2 \cdot 10^{-16}$ ) : la durée de la session a un effet. Le modèle explique 17,5 % de la déviance. La bande de confiance à 95 % est **étroite au voisinage du sommet** (on y a beaucoup d'observations) et **très large aux extrémités** : de 0,11 à 0,50 environ à une minute, et jusqu'à plus de 0,6 à 35 minutes, ce qui traduit honnêtement le manque de données (voir 2.5.3, effet de bord). La vraie courbe reste dans la bande **en chacun des 200 points de la grille** (la part vaut 1).

### 2.5.5 Un GAM à plusieurs termes : l'âge des clients

Un GAM sert aussi à **vérifier** une hypothèse de linéarité. Dans le modèle de rachat de 2.2, l'effet de l'âge a été supposé linéaire sur le logit. Laissons-le libre avec  $s(\text{age})$ , et comparons au modèle linéaire (test du rapport de vraisemblance approché).

```
library(mgcv)
clients <- read.csv("donnees/clients.csv")
clients$canal <- factor(clients$canal_acquisition, levels = c("Boutique", "Instagram", "Site"))
lineaire <- gam(rachat_12m ~ offre_bienvenue + canal + age, family = binomial, data = clients, method
  ↪ = "REML")
souple <- gam(rachat_12m ~ offre_bienvenue + canal + s(age), family = binomial, data = clients, method
  ↪ = "REML")
print(round(summary(souple)$s.table, 3))
print(anova(lineaire, souple, test = "Chisq"))
print(AIC(lineaire, souple))
```

```
      edf Ref.df Chi.sq p-value
s(age) 1.579  1.971  13.87  0.001
Analysis of Deviance Table

Model 1: rachat_12m ~ offre_bienvenue + canal + age
Model 2: rachat_12m ~ offre_bienvenue + canal + s(age)
  Resid. Df Resid. Dev    Df Deviance Pr(>Chi)
1    1995.0     2710.8
2    1993.6     2709.4  1.364    1.4133  0.3311
      df      AIC
lineaire 5.000000 2720.83
souple   5.971442 2721.36
```

Le terme lisse  $s(\text{age})$  utilise 1,58 degré de liberté effectif : une courbe presque droite (une droite correspondrait à 1). Le test sur le terme lisse ( $p = 0,001$ ) dit seulement que l'âge a un effet, pas qu'il est non linéaire. La question de la linéarité est tranchée par la comparaison des deux modèles : passer du modèle linéaire au modèle lisse réduit la déviance de seulement 1,41 pour 1,36 degré de liberté supplémentaire ( $p = 0,33$ ), et l'AIC préfère le modèle linéaire (2 720,8 contre 2 721,4). **Rien n'indique que l'effet de l'âge soit non linéaire** : l'hypothèse de linéarité faite en 2.2 est raisonnable.

### 2.5.6 Les pièges des GAM

- **Trop de souplesse.** Sans pénalité (ou avec une pénalité trop faible), la courbe épouse le bruit : on obtient des bosses sans signification. Vérifiez toujours que les degrés de liberté effectifs sont raisonnables, regardez la bande de confiance, et méfiez-vous des courbes qui oscillent.
- **L'extrapolation.** Une spline n'est fiable que dans l'**intervalle des données** : en dehors, elle n'est contrainte par rien. (C'est d'ailleurs pourquoi statsmodels refuse de prédire hors de la plage observée.)
- **Les effets centrés.** Chaque courbe lisse est définie à une **constante près** (la constante est dans l'ordonnée à l'origine) : on lit sa *forme*, pas son niveau absolu.
- **Les interactions.** Un GAM additif ne contient pas d'interaction entre variables ; elles s'ajoutent explicitement (surfaces lisses à deux variables, `te()` ou `ti()` dans `mgcv`).
- **La concurvité.** C'est l'analogue non linéaire de la colinéarité : si une variable est une fonction lisse d'une autre, les deux courbes ne sont plus séparables.

#### À retenir

- Un GAM remplace  $\beta_j x_j$  par une fonction lisse  $f_j(x_j)$  :  $g(\mu) = \beta_0 + \sum_j f_j(x_j)$ . On garde

l'additivité (lecture variable par variable) et la famille/le lien des GLM.

- Une courbe lisse s'écrit sur une **base de fonctions** (B-splines) : c'est une régression sur des variables construites.
- La **pénalité de courbure**  $\lambda \gamma^\top S \gamma$  règle la souplesse (de la droite,  $\lambda \rightarrow \infty$ , à l'interpolation,  $\lambda = 0$ ). La complexité réelle se lit dans les **degrés de liberté effectifs**.
- Les GAM détectent les formes qu'une droite ou une parabole manquent, et permettent de **tester la linéarité** d'un effet.
- Soyez prudent : sur-ajustement, extrapolation, interprétation à une constante près.

## 2.6 ⊕ Pour aller plus loin : zéros en excès, surdispersion et loi de Tweedie

**Section optionnelle.** Elle traite un cas très fréquent en assurance, en vente et en santé : des variables **positives avec un paquet de zéros** (aucune commande, aucun sinistre, aucune dépense). Elle s'appuie sur les sections 2.3 et 2.4.

**Intuition.** Parmi nos 2 000 clients, 13 % n'ont passé aucune commande dans l'année, donc dépensé 0 DT. Ces zéros posent deux questions de nature différente. *Pour un comptage* (nombre de commandes) : ces zéros sont-ils **trop nombreux** pour la loi choisie ? Y a-t-il des clients « structurellement » inactifs, qui ne commanderont jamais, mélangés à des clients actifs qui, eux, peuvent aussi tomber par hasard sur zéro ? *Pour un montant* (dépense annuelle) : comment modéliser une variable qui est **zéro avec une probabilité positive, et continue et positive** sinon ? La loi Gamma ne peut pas prendre la valeur 0 ; la loi normale est absurde. Deux familles de réponses existent : **séparer** le problème en deux (modèles à deux parties) ou le traiter d'un coup avec une loi adaptée (**Tweedie**).

### 2.6.1 Les modèles à zéros en excès : ZIP et modèle de barrière

**Le modèle à inflation de zéros (ZIP, zero-inflated Poisson).** On suppose que chaque client appartient, avec la probabilité  $\pi$ , à un groupe « dormant » qui donne toujours zéro, et avec la probabilité  $1 - \pi$  à un groupe actif dont le nombre de commandes suit une loi de Poisson de moyenne  $\mu$ . Un zéro peut donc venir des deux groupes :

$$P(Y = 0) = \pi + (1 - \pi)e^{-\mu}, \quad P(Y = k) = (1 - \pi) \frac{e^{-\mu} \mu^k}{k!} \quad (k \geq 1).$$

**Moyenne et variance.**  $E[Y] = (1 - \pi)\mu$  et  $E[Y^2] = (1 - \pi)(\mu + \mu^2)$ , donc

$$\text{Var}(Y) = (1 - \pi)(\mu + \mu^2) - (1 - \pi)^2 \mu^2 = (1 - \pi) \mu (1 + \pi \mu).$$

Le rapport  $\text{Var}/E = 1 + \pi \mu \geq 1$  : l'inflation de zéros **produit de la surdispersion**. C'est pour cela qu'il est facile de confondre les deux phénomènes.

**Un calcul à la main.** Avec  $\pi = 0,2$  et  $\mu = 3$  :  $P(Y = 0) = 0,2 + 0,8 e^{-3} = 0,2 + 0,8 \times 0,0498 = 0,2398$ . Un Poisson(3) seul n'aurait que 0,0498 de zéros : c'est près de cinq fois plus. La moyenne vaut  $0,8 \times 3 = 2,4$  et la variance  $0,8 \times 3 \times (1 + 0,2 \times 3) = 3,84$  : le rapport variance/moyenne est de 1,6.

**Le modèle de barrière (hurdle).** Il sépare le problème en deux étapes : une régression logistique pour « zéro ou non », puis, *sachant* que le résultat est positif, une loi **tronquée en zéro** pour la valeur (Poisson tronqué, ou binomiale négative tronquée). Dans un modèle de barrière, **tous** les zéros viennent de la première étape ; dans un ZIP, ils viennent des deux groupes. Le choix est

une question de **mécanisme** : les zéros sont-ils un état à part (dormants) ou simplement la queue basse du même comportement ?

Vérifions d'abord la formule du ZIP par simulation.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import expit

rng = np.random.default_rng(26)
pi, mu = 0.2, 3.0
actif = rng.random(400_000) >= pi
y = np.where(actif, rng.poisson(mu, 400_000), 0)
print(f"ZIP simulé : P(0) = {np.mean(y == 0):.4f} | moyenne = {y.mean():.3f} | variance = {y.var():.3f} | variance / moyenne = {y.var() / y.mean():.3f}")
print(f"ZIP théorique : P(0) = {pi + (1 - pi) * np.exp(-mu):.4f} | moyenne = {(1 - pi) * mu:.3f} | variance = {(1 - pi) * mu * (1 + pi * mu):.3f} | variance / moyenne = {1 + pi * mu:.3f}")
print(f"Poisson(3) : P(0) = {np.exp(-mu):.4f}")

ZIP simulé : P(0) = 0.2407 | moyenne = 2.398 | variance = 3.836 | variance / moyenne = 1.600
ZIP théorique : P(0) = 0.2398 | moyenne = 2.400 | variance = 3.840 | variance / moyenne = 1.600
Poisson(3) : P(0) = 0.0498
```

La simulation confirme les formules : 24,07 % de zéros (théorie : 23,98 %), moyenne 2,398 (2,400), variance 3,836 (3,840). Le rapport variance/moyenne vaut 1,600, soit bien  $1 + \pi\mu = 1 + 0,2 \times 3$  : un jeu de données qui ne contiendrait *aucune* hétérogénéité de taux, mais seulement des dormants, paraîtrait déjà surdispersé. Pour comparaison, une loi de Poisson(3) n'aurait que 4,98 % de zéros.

## 2.6.2 Nos comptages ont-ils des zéros « en trop » ?

En 2.3.3 nous avons constaté que la binomiale négative reproduit déjà les 13 % de zéros. Un modèle à inflation de zéros apporte-t-il quelque chose de plus ? Comparons quatre modèles sur nb\_commandes\_an : Poisson, ZIP, binomiale négative, ZINB (binomiale négative avec inflation de zéros). Pour la partie « inflation », nous prenons une probabilité  $\pi$  constante.

```
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram", "Site"])
X = sm.add_constant(pd.get_dummies(clients[["age", "canal"]], drop_first=True, dtype=float))
y_c = clients["nb_commandes_an"]
infl = np.ones((len(clients), 1))

poi = sm.Poisson(y_c, X).fit(dis=0)
zip_ = sm.ZeroInflatedPoisson(y_c, X, exog_infl=infl).fit(dis=0, maxiter=300)
nb = sm.NegativeBinomial(y_c, X).fit(dis=0)
zinb = sm.ZeroInflatedNegativeBinomialP(y_c, X, exog_infl=infl, p=2).fit(dis=0, maxiter=500)

lignes = []
for nom, r in [("Poisson", poi), ("ZIP", zip_), ("binomiale négative", nb), ("ZINB", zinb)]:
    pi_hat = float(expit(r.params["inflate_const"])) if "inflate_const" in r.params.index else 0.0
    lignes.append({"modèle": nom, "paramètres": len(r.params), "log-vraisemblance": r.llf, "AIC": r.aic, "pi estimé": pi_hat})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
print()
print("part de zéros observée :", round(float((y_c == 0).mean()), 4))
print("alpha (NB) =", round(float(nb.params["alpha"]), 3), "| alpha (ZINB) =", round(float(zinb.params["alpha"]), 3))
```

|  | modèle             | paramètres | log-vraisemblance | AIC        | pi estimé |
|--|--------------------|------------|-------------------|------------|-----------|
|  | Poisson            | 4          | -5853.0778        | 11714.1557 | 0.0000    |
|  | ZIP                | 5          | -5536.1403        | 11082.2806 | 0.1181    |
|  | binomiale négative | 5          | -4878.4227        | 9766.8453  | 0.0000    |
|  | ZINB               | 6          | -4878.4227        | 9768.8453  | 0.0000    |

part de zéros observée : 0.13

alpha (NB) = 0.584 | alpha (ZINB) = 0.584

Lisons le tableau. Le ZIP améliore beaucoup l'AIC de Poisson (11 082,3 contre 11 714,2, soit 632 points) en estimant  $\hat{\pi} = 0,118$  : si l'on s'arrêtait là, on conclurait à 12 % de clients « dormants ». Mais la **binomiale négative simple** fait bien mieux (AIC 9 766,8), et le **ZINB** n'apporte rien : sa log-vraisemblance est *identique* à celle de la binomiale négative (-4 878,42), sa probabilité d'inflation estimée est nulle, et son AIC est supérieur de 2 (un paramètre inutile de plus). Le  $\hat{\alpha}$  est le même (0,584). **Conclusion : il n'y a pas d'excès de zéros dans nos comptages** ; les 13 % de zéros sont l'extrémité basse d'une distribution surdispersée. L'« inflation » de 11,8 % du ZIP était un artefact : le ZIP essayait d'absorber par un mélange la surdispersion que seule la binomiale négative représente bien.

**Ne pas conclure trop vite à une « inflation ».** Un excès de zéros par rapport à Poisson est le signe habituel de la **surdispersion**, et une binomiale négative l'absorbe souvent seule. Un modèle ZIP/ZINB se justifie quand on a une **raison métier** (il existe un groupe qui ne peut pas acheter) *et* un gain d'ajustement net (AIC) par rapport à la binomiale négative simple.

Voyons un cas où l'inflation est **réelle**. Nous simulons (graine 27) 1 500 comptes dont **25 % sont dormants** (toujours zéro commande) ; les autres commandent selon une loi de Poisson dont la moyenne augmente avec un score d'engagement  $x$ .

```
rng = np.random.default_rng(27)
n = 1500
x = rng.normal(size=n)
dormant = rng.random(n) < 0.25
y_sim = np.where(dormant, 0, rng.poisson(np.exp(0.9 + 0.4 * x)))
Xs = sm.add_constant(pd.DataFrame({"x": x}))
infl = np.ones((n, 1))

poi_s = sm.Poisson(y_sim, Xs).fit(dis=0)
nb_s = sm.NegativeBinomial(y_sim, Xs).fit(dis=0)
zip_s = sm.ZeroInflatedPoisson(y_sim, Xs, exog_infl=infl).fit(dis=0, maxiter=300)

print("part de zéros observée :", round(float((y_sim == 0).mean()), 3), "| part de dormants simulée
↳ :", round(float(dormant.mean()), 3))
for nom, r in [("Poisson", poi_s), ("binomiale négative", nb_s), ("ZIP", zip_s)]:
    print(f"{nom:20s}: AIC = {r.aic:8.1f}")
print()
print("ZIP : pi estimé =", round(float(expt(zip_s.params["inflate_const"])), 3), "| proportion
↳ réellement simulée de dormants :", round(float(dormant.mean()), 3), "(probabilité programmée :
↳ 0.25)")
print("ZIP : coefficients du comptage (const, x) =", zip_s.params[["const",
↳ "x"]].round(3).to_numpy(), "(vrais : 0.9, 0.4)")
print("binomiale négative : coefficients (const, x) =", nb_s.params[["const",
↳ "x"]].round(3).to_numpy(), "| alpha =", round(float(nb_s.params["alpha"]), 3))
```

part de zéros observée : 0.316 | part de dormants simulée : 0.23

Poisson : AIC = 5786.0

binomiale négative : AIC = 5505.3

ZIP : AIC = 5292.1

```
ZIP : pi estimé = 0.233 | proportion réellement simulée de dormants : 0.23 (probabilité programmée :
↪ 0.25)
ZIP : coefficients du comptage (const, x) = [0.871 0.418] (vrais : 0.9, 0.4)
binomiale négative : coefficients (const, x) = [0.606 0.407] | alpha = 0.448
```

Ici, avec 31,6 % de zéros observés (dont 23,0 % de vrais dormants, car le tirage a donné 23 % de dormants et non exactement 25 %), les trois modèles se classent dans l'ordre inverse du précédent : Poisson (AIC 5 786,0), binomiale négative (5 505,3) et **ZIP, nettement meilleur** (5 292,1). Le ZIP retrouve la proportion de dormants ( $\hat{\pi} = 0,233$  pour 0,230 réellement simulés) et les coefficients du comptage (0,871 et 0,418 pour 0,9 et 0,4). La binomiale négative, elle, donne une constante de 0,606 : elle n'estime pas le taux des clients actifs, mais la moyenne sur **tous** les comptes, dormants compris, soit environ  $\log(0,77) + 0,9 \approx 0,64$ , et son  $\hat{\alpha} = 0,448$  n'a aucune réalité (il n'y a pas d'hétérogénéité de taux : il y a des dormants). Moralité : le bon modèle dépend du **mécanisme** qui a produit les zéros, pas du seul nombre de zéros.

### 2.6.3 Un montant avec des zéros : la loi de Tweedie

Revenons à `depense_annuelle` : positive, asymétrique, et 13 % de zéros exacts. Le fil conducteur est ici de se demander **comment la dépense de l'année se fabrique** : un client passe un nombre aléatoire  $N$  de commandes, chacune d'un montant aléatoire  $X_i > 0$ , et la dépense est la **somme**

$$Y = X_1 + X_2 + \dots + X_N \quad (Y = 0 \text{ si } N = 0).$$

C'est une **somme aléatoire**, ou *loi de Poisson composée*. Si  $N$  suit une loi de Poisson de paramètre  $\lambda$  et les  $X_i$  une loi Gamma (de forme  $\alpha$  et d'échelle  $\theta$ ), on obtient une loi qui a une **masse en zéro** et une densité sur  $]0, \infty[$  : c'est la **loi de Tweedie**, avec des propriétés remarquables.

#### Propriétés de la loi de Poisson-Gamma composée.

- $P(Y = 0) = P(N = 0) = e^{-\lambda}$ .
- $E[Y] = \lambda\alpha\theta$  (formule de Wald : nombre moyen de termes  $\times$  moyenne d'un terme) et  $\text{Var}(Y) = \lambda E[X^2] = \lambda\alpha(\alpha + 1)\theta^2$ .
- Reparamétrons par la moyenne  $\mu$ , un paramètre de dispersion  $\phi$  et une **puissance**  $p = \frac{\alpha + 2}{\alpha + 1} \in ]1, 2[$ , en posant  $\lambda = \frac{\mu^{2-p}}{\phi(2-p)}$ ,  $\alpha = \frac{2-p}{p-1}$ ,  $\theta = \phi(p-1)\mu^{p-1}$ . On vérifie que  $\lambda\alpha\theta = \mu$  et, puisque  $\alpha + 1 = \frac{1}{p-1}$ , que  $\text{Var}(Y) = \mu(\alpha + 1)\theta = \phi\mu^p$ .

Autrement dit, la loi de Tweedie est une **famille exponentielle** de fonction de variance  $V(\mu) = \mu^p$ , avec  $1 < p < 2$  : elle interpole entre Poisson ( $p = 1$ ) et Gamma ( $p = 2$ ), les deux cas du tableau de 2.3.5. Et sa probabilité de zéro est  $P(Y = 0) = \exp(-\mu^{2-p}/(\phi(2-p)))$  : **plus la moyenne est grande, moins il y a de zéros**.

Vérifions ces formules par simulation (graine 28), pour deux valeurs de la moyenne, avec  $p = 1,5$  et  $\phi = 20$ . Pour générer  $Y$ , on tire  $N$ , puis, sachant  $N = n > 0$ , la somme de  $n$  lois Gamma( $\alpha, \theta$ ) est une loi Gamma( $n\alpha, \theta$ ).

```
rng = np.random.default_rng(28)
p_tw, phi_tw, N = 1.5, 20.0, 400_000
lignes = []
for mu_tw in (50.0, 200.0):
    lam = mu_tw ** (2 - p_tw) / (phi_tw * (2 - p_tw))
    alpha = (2 - p_tw) / (p_tw - 1)
    theta = phi_tw * (p_tw - 1) * mu_tw ** (p_tw - 1)
    n_cmd = rng.poisson(lam, N)
    y_tw = np.where(n_cmd > 0, rng.gamma(np.maximum(n_cmd, 1) * alpha, theta), 0.0)
    lignes.append({"moyenne mu": mu_tw, "lambda": lam, "forme alpha": alpha, "échelle theta": theta,
                  "P(0) simulé": np.mean(y_tw == 0), "P(0) = exp(-lambda)": np.exp(-lam),
```



```

      "moyenne simulée": y_tw.mean(), "variance simulée": y_tw.var(), "phi * mu^p":
      ↪ phi_tw * mu_tw ** p_tw})
print(pd.DataFrame(lignes).round(3).T.to_string(header=False))

```

|                     |          |           |
|---------------------|----------|-----------|
| moyenne mu          | 50.000   | 200.000   |
| lambda              | 0.707    | 1.414     |
| forme alpha         | 1.000    | 1.000     |
| échelle theta       | 70.711   | 141.421   |
| P(0) simulé         | 0.492    | 0.243     |
| P(0) = exp(-lambda) | 0.493    | 0.243     |
| moyenne simulée     | 49.984   | 200.039   |
| variance simulée    | 7048.066 | 56394.773 |
| phi * mu^p          | 7071.068 | 56568.542 |

Pour  $\mu = 50 : \lambda = 0,707$  commande en moyenne,  $\alpha = 1$  (les montants sont exponentiels),  $\theta = 70,7$  ; la proportion de zéros simulée est de 49,2 % pour 49,3 % prévus ( $e^{-\lambda}$ ), la moyenne de 49,98 et la variance de 7 048 pour  $\phi\mu^p = 7 071$ . Pour  $\mu = 200 : 24,3$  % de zéros pour 24,3 % prévus, moyenne 200,04, variance 56 395 pour 56 569. Les formules sont donc vérifiées. Remarquez au passage la dernière propriété : quand la moyenne passe de 50 à 200, la proportion de zéros **tombe de 49 % à 24 %** : les gros clients sont moins souvent à zéro.

**Estimer la puissance  $p$ .** Le paramètre  $p$  n'est pas connu : on le choisit par maximum de vraisemblance (on calcule la log-vraisemblance pour plusieurs valeurs de  $p$  et l'on garde la meilleure). La densité de Tweedie est une série infinie, mais le paquet R `mgcv` la calcule précisément et l'estime : nous l'utilisons ici. (La fonction `Tweedie` de `statsmodels` ajuste très bien les coefficients à  $p$  fixé ; mais sa log-vraisemblance est approchée, ce qui la rend peu fiable pour comparer des valeurs de  $p$ .)

```

suppressPackageStartupMessages(library(mgcv))
clients <- read.csv("donnees/clients.csv")
clients$canal <- factor(clients$canal_acquisition, levels = c("Boutique", "Instagram", "Site"))
f <- depense_annuelle ~ age + canal + offre_bienvenue

puissances <- c(1.2, 1.3, 1.4, 1.45, 1.5, 1.6, 1.7, 1.8)
logv <- sapply(puissances, function(p) as.numeric(logLik(gam(f, family = Tweedie(p = p, link =
  ↪ "log"), data = clients, method = "REML"))))
print(data.frame(p = puissances, log_vraisemblance = round(logv, 1)))

m <- gam(f, family = tw(), data = clients, method = "REML")           # tw() estime p en même temps que
  ↪ les coefficients
p_hat <- m$family$getTheta(TRUE)
cat("puissance p estimée :", round(p_hat, 3), "| dispersion phi :", round(m$scale, 2), "\n")
print(round(summary(m)$p.table, 4))

mu <- fitted(m)
zeros_pred <- mean(exp(-mu^(2 - p_hat) / (m$scale * (2 - p_hat))))
cat("part de zéros prédite par le modèle de Tweedie :", round(zeros_pred, 3), "| observée :",
  ↪ round(mean(clients$depense_annuelle == 0), 3), "\n")

```

|   | p    | log_vraisemblance |
|---|------|-------------------|
| 1 | 1.20 | -12492.1          |
| 2 | 1.30 | -12352.9          |
| 3 | 1.40 | -12304.5          |
| 4 | 1.45 | -12299.8          |
| 5 | 1.50 | -12306.0          |
| 6 | 1.60 | -12352.6          |
| 7 | 1.70 | -12460.1          |
| 8 | 1.80 | -12680.0          |

puissance p estimée : 1.447 | dispersion phi : 19.78

|             | Estimate | Std. Error | t value | Pr(> t ) |
|-------------|----------|------------|---------|----------|
| (Intercept) | 5.4730   | 0.0877     | 62.4383 | 0.0000   |



|                 |         |        |          |        |
|-----------------|---------|--------|----------|--------|
| age             | 0.0081  | 0.0021 | 3.9372   | 0.0001 |
| canalInstagram  | -0.5553 | 0.0547 | -10.1570 | 0.0000 |
| canalSite       | -0.1509 | 0.0542 | -2.7858  | 0.0054 |
| offre_bienvenue | -0.0142 | 0.0435 | -0.3260  | 0.7444 |

part de zéros prédite par le modèle de Tweedie : 0.153 | observée : 0.13

La log-vraisemblance est maximale pour  $p = 1,45$  ( $-12\,299,8$ ) et chute de 4,7 unités à  $p = 1,4$  et de 6,2 à  $p = 1,5$ , puis nettement plus loin (de 192 unités à  $p = 1,2$  et de 380 à  $p = 1,8$ ) : la puissance est bien déterminée. `tw()` l'estime à  $\hat{p} = 1,447$ , avec  $\hat{\phi} = 19,78$  : la variance de la dépense est environ  $19,8 \mu^{1,447}$ , entre celle de Poisson ( $p = 1$ ) et celle de Gamma ( $p = 2$ ). Les coefficients se lisent comme des effets multiplicatifs sur la **dépense moyenne de tous les clients** (zéros compris) : Instagram,  $e^{-0,555} = 0,57$  (43 % de dépense en moins que la boutique, contre  $-39$  % parmi les seuls acheteurs en 2.3.4 : l'effet est plus fort car il inclut aussi une moindre probabilité d'acheter) ; Site,  $e^{-0,151} = 0,86$  ; âge,  $+0,8$  % par année ; offre, aucun effet détectable ( $p = 0,74$ ).

Le contrôle de la dernière ligne est instructif : le modèle de Tweedie prédit **15,3 % de zéros**, alors qu'on en observe **13,0 %**. L'écart est modeste, mais il est dû à la nature approximative du modèle (pitfall 4 plus bas) : le vrai nombre de commandes est surdispersé, pas simplement de Poisson.

## 2.6.4 Une alternative : le modèle à deux parties

Au lieu d'une seule loi, on peut décomposer  $E[Y | x] = P(Y > 0 | x) \times E[Y | Y > 0, x]$  et modéliser chaque facteur à part : une régression **logistique** pour savoir si le client achète, puis une régression **Gamma** (lien log) pour le montant des acheteurs. C'est le modèle de barrière appliqué à une variable continue. Comparons-le à Tweedie sur ce qui importe : la dépense moyenne prédite, par canal et par classe de risque.

```
f_rhs = "age + canal + offre_bienvenue"
clients["achete"] = (clients["depense_annuelle"] > 0).astype(int)
partiel = smf.glm("achete ~ " + f_rhs, clients, family=sm.families.Binomial()).fit()
acheteurs = clients[clients["achete"] == 1]
partie2 = smf.glm("depense_annuelle ~ " + f_rhs, acheteurs,
    ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
mu_deux = partiel.predict(clients) * partie2.predict(clients)           # E[Y|x] = P(achete|x)
    ↪ × E[montant|achete, x]

tw = smf.glm("depense_annuelle ~ " + f_rhs, clients, family=sm.families.Tweedie(var_power=1.45,
    ↪ link=sm.families.links.Log())).fit(scale="X2")
mu_tw = tw.fittedvalues

print("coefficients de Tweedie (p = 1,45) :", tw.params.round(4).to_dict())
print()
tab = pd.DataFrame({"observée": clients.groupby("canal", observed=True)["depense_annuelle"].mean(),
    "Tweedie": mu_tw.groupby(clients["canal"], observed=True).mean(),
    "deux parties": mu_deux.groupby(clients["canal"], observed=True).mean()})
tab.loc["tous les clients"] = [clients["depense_annuelle"].mean(), mu_tw.mean(), mu_deux.mean()]
print(tab.round(1).to_string())

# calibration par dixième de dépense prédite
classes = pd.qcut(mu_deux, 10, labels=False)
dec = pd.DataFrame({"observée": clients["depense_annuelle"], "Tweedie": mu_tw, "deux parties":
    ↪ mu_deux, "dixième": classes}).groupby("dixième").mean()
print()
print(dec.round(1).to_string())
print()
print("écart absolu moyen, par dixième : Tweedie =", round(float((dec["Tweedie"] -
    ↪ dec["observée"]).abs().mean(), 2),
    "| deux parties =", round(float((dec["deux parties"] - dec["observée"]).abs().mean(), 2))
print("part de zéros prédite par le modèle à deux parties :", round(float(1 -
    ↪ partiel.fittedvalues.mean(), 3))
```

coefficients de Tweedie (p = 1,45) : {'Intercept': 5.4729, 'canal[T.Instagram]': -0.5553,  
↪ 'canal[T.Site]': -0.151, 'age': 0.0081, 'offre\_bienvenue': -0.0142}

|                  | observée | Tweedie | deux parties |
|------------------|----------|---------|--------------|
| canal            |          |         |              |
| Boutique         | 316.3    | 316.7   | 317.2        |
| Instagram        | 182.5    | 182.8   | 183.0        |
| Site             | 272.9    | 272.3   | 271.7        |
| tous les clients | 247.0    | 247.0   | 247.0        |

|         | observée | Tweedie | deux parties |
|---------|----------|---------|--------------|
| dixième |          |         |              |
| 0       | 170.6    | 163.0   | 162.4        |
| 1       | 173.0    | 175.8   | 175.7        |
| 2       | 199.4    | 186.9   | 187.4        |
| 3       | 192.2    | 202.6   | 203.7        |
| 4       | 205.4    | 244.2   | 243.5        |
| 5       | 268.8    | 266.7   | 266.1        |
| 6       | 316.1    | 280.3   | 280.0        |
| 7       | 280.8    | 294.8   | 294.7        |
| 8       | 338.7    | 313.1   | 313.3        |
| 9       | 331.8    | 345.3   | 345.9        |

écart absolu moyen, par dixième : Tweedie = 16.3 | deux parties = 16.49  
part de zéros prédite par le modèle à deux parties : 0.13

Les coefficients de statsmodels à  $p = 1,45$  (5,4729 ; −0,5553 ; −0,151 ; 0,0081 ; −0,0142) coïncident avec ceux de mgcv à trois ou quatre décimales : deux logiciels, un même modèle. Quant à la **comparaison** : les moyennes prédites par canal sont presque identiques pour les deux approches, et très proches de l'observé (boutique : 316,7 pour Tweedie, 317,2 pour deux parties, 316,3 observé ; Instagram 182,8, 183,0 et 182,5 ; site 272,3, 271,7 et 272,9), et les deux reproduisent exactement la moyenne globale (247,0 DT). Par dixième de dépense prédite, l'écart absolu moyen à l'observé est de 16,3 DT (Tweedie) et 16,5 DT (deux parties) : indiscernables. Les écarts dixième par dixième (par exemple 205 observé contre 244 prédits dans le cinquième dixième) sont de l'ordre de l'erreur d'échantillonnage d'une moyenne sur 200 clients (environ  $297/\sqrt{200} \approx 21$  DT). **La seule différence nette** est la part de zéros prédite : le modèle à deux parties retrouve exactement les 13,0 % (c'est garanti par construction : la logistique avec constante reproduit la proportion observée), alors que Tweedie en prédit 15,3 %. Le choix se fait donc sur des critères autres que l'ajustement de la moyenne : Tweedie est plus parcimonieux (un seul modèle) ; le modèle à deux parties permet de séparer ce qui joue sur la décision d'acheter de ce qui joue sur le montant.

2.6.5 Comment choisir ?

| Situation                                                               | Modèle                                   | Pourquoi                                                         |
|-------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------|
| comptage, un peu plus de zéros que Poisson                              | <b>binomiale négative</b>                | la surdispersion explique souvent les zéros                      |
| comptage avec un groupe « qui ne peut pas »                             | <b>ZIP / ZINB</b>                        | les zéros ont deux origines (groupe dormant, hasard)             |
| comptage où les zéros sont un état à part                               | <b>barrière (hurdle)</b>                 | une étape « zéro ou non », puis une loi tronquée                 |
| montant $\geq 0$ , somme de petits montants (sinistres, achats)         | <b>Tweedie</b> ( $1 < p < 2$ )           | un seul modèle, une seule moyenne, structure « somme aléatoire » |
| montant $\geq 0$ avec une grosse part de zéros et un mécanisme distinct | <b>deux parties</b> (logistique + Gamma) | flexibilité : chaque partie a ses propres variables              |

**Les pièges.** (1) Les effets d'un modèle à **deux parties** se lisent en deux temps (probabilité d'acheter, puis montant) ; ceux de Tweedie portent sur la **moyenne globale** : ce ne sont pas les mêmes questions. (2) Dans un ZIP, les variables de la partie « inflation » et celles de la partie « comptage » peuvent différer, mais leurs effets sont difficiles à séparer : prudence avec les données peu nombreuses. (3) Vérifiez toujours la **part de zéros prédite** par le modèle contre la part observée : c'est un contrôle simple et très révélateur. (4) La loi de Tweedie suppose un mécanisme « somme aléatoire » avec une forme constante : si le comptage lui-même est surdispersé, elle n'est qu'une approximation.

### À retenir

- Un excès de zéros par rapport à Poisson est le signe habituel de la **surdispersion** : essayez la **binomiale négative** avant un modèle à inflation de zéros. Un ZIP se justifie par un mécanisme (groupe dormant) *et* un gain d'AIC.
- Un ZIP mélange un groupe « toujours zéro » (probabilité  $\pi$ ) et un Poisson ; il produit une surdispersion  $\text{Var}/E = 1 + \pi\mu$ . Un modèle de barrière sépare « zéro ou non » d'une loi tronquée.
- La loi de **Tweedie** ( $1 < p < 2$ ) est la somme aléatoire Poisson-Gamma : masse en 0 (probabilité  $e^{-\lambda}$ ), variance  $\phi\mu^p$ , moyenne modélisée avec un lien log. Estimez  $p$  par vraisemblance.
- Le **modèle à deux parties** (logistique  $\times$  Gamma) est la solution flexible.
- Contrôlez toujours la **part de zéros prédite**.

## 2.7 Exercices corrigés

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse ou démonstration.

### Énoncés

**Exercice 1 ★ (cotes et logit).** (a) Un client a une probabilité de rachat de 0,8 : quelle est sa cote, quel est son logit ? (b) Un modèle logistique donne  $\text{logit}(p) = -0,4 + 0,9x$ , où  $x$  est le nombre de commandes passées l'an dernier. Calculez  $p$  pour  $x = 0, 1, 2$ . (c) Quel est le rapport de cotes associé à une commande de plus ? De combien de points de pourcentage la probabilité augmente-t-elle de  $x = 0$  à  $x = 1$ , puis de  $x = 1$  à  $x = 2$  ? Pourquoi ne sont-ils pas égaux ?

**Exercice 2 ★ (rapport de cotes, risque relatif).** Sur 200 clients abonnés à la newsletter, 60 ont acheté ; sur 300 clients non abonnés, 45 ont acheté. Calculez à la main la différence de risque, le risque relatif et le rapport de cotes, puis un intervalle de confiance à 95 % de ce dernier (erreur-type du log-OR :  $\sqrt{1/a + 1/b + 1/c + 1/d}$ ). Vérifiez avec une régression logistique.

**Exercice 3 ★ (régression de Poisson).** Un modèle de Poisson pour le nombre de commandes annuelles donne  $\log \mu = 1,2 - 0,01 \times \text{âge} + 0,2 \times \text{site}$  (la variable « site » vaut 1 pour un client acquis par le site, 0 pour la boutique). (a) Nombre moyen de commandes d'un client de 40 ans acquis par le site, et d'un client de 40 ans acquis en boutique. (b) Par quel facteur le nombre moyen est-il multiplié pour 10 ans de plus ? (c) Probabilité qu'un client de 40 ans acquis par le site ne passe **aucune** commande, si la loi de Poisson est correcte.

**Exercice 4 ★★ (IRLS à la main).** Trois clients :  $x = (0, 1, 2)$  et  $y = (1, 3, 5)$  commandes. On ajuste un modèle de Poisson,  $\log \mu = \beta_0 + \beta_1 x$ . (a) À partir de  $\beta^{(0)} = (0, 0)$ , calculez à la main **une**

**itération** d'IRLS (poids, réponse de travail, régression pondérée). (b) Écrivez le code complet de l'algorithme jusqu'à convergence et comparez à statsmodels.

**Exercice 5 ★★ (déviance et rapport de vraisemblance).** Avec les données de l'exercice 4 : (a) calculez à la main la déviance du modèle **sans variable** ( $\hat{\mu} = \bar{y}$ ) et la statistique de Pearson. (b) Quelle est la déviance du modèle avec  $x$  ? Testez, par le rapport de vraisemblance, l'utilité de  $x$  (1 degré de liberté).

**Exercice 6 ★★ (décalage).** Trois transporteurs ont livré des colis et enregistré des retards : A a livré 200 milliers de colis pour 30 retards ; B, 50 milliers pour 12 retards ; C, 400 milliers pour 40 retards. (a) Calculez les taux de retard par millier de colis et les rapports de taux B/A et C/A. (b) Ajustez une régression de Poisson avec et sans décalage (*offset*) : que concluriez-vous dans chaque cas ?

**Exercice 7 ★★ (surdispersion).** Une régression de Poisson, sur  $n = 305$  clients avec 5 paramètres, donne une statistique de Pearson  $X^2 = 540$ . Le coefficient de la variable « Instagram » est 0,30 avec une erreur-type de 0,12. (a) Estimez la dispersion  $\hat{\phi}$ . (b) Corrigez l'erreur-type et la statistique  $z$ . La conclusion change-t-elle au seuil de 5 % ?

**Exercice 8 ★★ (régression sur  $\log y$ ).** On régresse le logarithme des dépenses sur des variables explicatives ; pour un client donné, le modèle prédit  $\log \hat{y} = 5,0$  et l'écart-type résiduel est  $\hat{\sigma} = 0,9$ . (a) Que vaut  $e^{5,0}$  ? Est-ce la dépense moyenne prévue ? (b) Si les résidus sont normaux, quelle est la dépense moyenne prévue ? Vérifiez par simulation.

**Exercice 9 ★★ (comparer des modèles).** Les déviances de trois modèles de rachat sont  $D_{\text{complet}} = 2710,83$  (offre, âge, canal : 5 paramètres),  $D_{\text{sans offre}} = 2740,47$  (4 paramètres) et  $D_{\text{sans canal}} = 2728,57$  (3 paramètres), pour  $n = 2000$ . (a) Testez par le rapport de vraisemblance l'utilité de l'offre, puis du canal. (b) Calculez la différence d'AIC et de BIC dans chaque cas. Les critères s'accordent-ils avec les tests ?

**Exercice 10 ★★★ (sur les données : la ville).** Le fichier `clients.csv` contient la ville de chaque client. La ville améliore-t-elle le modèle de rachat `offre + âge + canal` ? Utilisez le test du rapport de vraisemblance, l'AIC et le BIC, et interprétez.

**Exercice 11 ★★ (choisir un seuil).** Avec le modèle de rachat enrichi des notes de l'enquête (section 2.2.7), trouvez le seuil qui maximise l'**indice de Youden**  $J = \text{sensibilité} + \text{spécificité} - 1$ , et comparez-le au seuil de 0,5.

**Exercice 12 ★★ (GAM : choisir la souplesse).** Sur les sessions de navigation (`ch02-sessions.csv`), ajustez des régressions logistiques sur des B-splines de 4 à 12 fonctions de base et choisissez le nombre qui minimise l'AIC. Quel est l'avantage de la pénalisation sur cette recherche ?

**Exercice 13 ★★ (zéros en excès).** Dans un modèle ZIP de moyenne de la partie Poisson  $\mu = 4$ , quelle probabilité d'inflation  $\pi$  donne un rapport variance/moyenne égal à 2 ? Quelle est alors la probabilité d'observer un zéro ? Vérifiez par simulation.

**Exercice 14 ★★★ (démonstration et Tweedie).** (a) Montrez que la loi Gamma de moyenne  $\mu$  et de forme  $\nu$  appartient à la famille exponentielle, avec  $\theta = -1/\mu$ ,  $b(\theta) = -\log(-\theta)$  et  $\phi = 1/\nu$ , et retrouvez  $\text{Var}(Y) = \mu^2/\nu$  avec la proposition de 2.1.3. (b) Pour une loi de Tweedie de moyenne  $\mu = 100$ ,  $\phi = 15$ ,  $p = 1,3$ , calculez la probabilité de zéro, puis les paramètres  $(\lambda, \alpha, \theta)$  de la somme Poisson-Gamma correspondante, et vérifiez par simulation.

## Corrigés

**Corrigé 1.** (a)  $\text{Cote} = 0,8/0,2 = 4$  ;  $\text{logit} = \log 4 \approx 1,386$ . (b)  $p(0) = \text{expit}(-0,4) = 0,401$  ;  $p(1) = \text{expit}(0,5) = 0,622$  ;  $p(2) = \text{expit}(1,4) = 0,802$ . (c) Le rapport de cotes est  $e^{0,9} = 2,46$  pour

une commande de plus. La probabilité augmente de 22,1 points puis de 18,0 points : l'effet est **constant sur le logit** mais pas sur la probabilité (il est maximal autour de  $p = 0,5$  et diminue quand on s'en éloigne : 2.2.2).

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import expit

print("cote =", 0.8 / 0.2, "| logit =", round(float(np.log(4)), 4))
p_x = expit(-0.4 + 0.9 * np.array([0, 1, 2]))
print("probabilités :", p_x.round(4), "| OR =", round(float(np.exp(0.9)), 4))
print("gains en points :", (100 * np.diff(p_x)).round(2))
```

```
cote = 4.0 | logit = 1.3863
probabilités : [0.4013 0.6225 0.8022] | OR = 2.4596
gains en points : [22.11 17.97]
```

**Corrigé 2.**  $\hat{p}_1 = 60/200 = 0,30$  et  $\hat{p}_0 = 45/300 = 0,15$ . Différence de risque = 0,15 (15 points)  
; risque relatif =  $0,30/0,15 = 2,0$  ; rapport de cotes =  $\frac{0,30/0,70}{0,15/0,85} = \frac{0,4286}{0,1765} = 2,43$ . Erreur-type  
du log-OR :  $\sqrt{\frac{1}{60} + \frac{1}{140} + \frac{1}{45} + \frac{1}{255}} = 0,2235$  ; intervalle de log OR :  $0,887 \pm 1,96 \times 0,2235$ , soit  
[0,449; 1,325], donc OR dans [1,57; 3,76]. Le RR (2,0) est plus petit que l'OR (2,43) : l'OR exagère  
le risque relatif car l'événement n'est pas rare (15 % à 30 %).

```
a, b, c, d = 60, 140, 45, 255 # avec : achat / non-achat ; sans : achat /
↪ non-achat
p1, p0 = a / (a + b), c / (c + d)
or_ = (a * d) / (b * c)
se = np.sqrt(1 / a + 1 / b + 1 / c + 1 / d)
print(f"différence de risque = {p1 - p0:.3f} | RR = {p1 / p0:.3f} | OR = {or_:.4f}")
print(f"IC95 % de l'OR : [{np.exp(np.log(or_) - 1.96 * se):.3f} ; {np.exp(np.log(or_) + 1.96 *
↪ se):.3f}] (erreur-type du log-OR = {se:.4f})")
tab2 = pd.DataFrame({"newsletter": [1] * 200 + [0] * 300, "achat": [1] * 60 + [0] * 140 + [1] * 45 +
↪ [0] * 255})
m2 = smf.glm("achat ~ newsletter", tab2, family=sm.families.Binomial()).fit()
print("régression logistique : OR =", round(float(np.exp(m2.params["newsletter"])), 4), "| IC95 % :",
↪ np.exp(m2.conf_int().loc["newsletter"]).round(3).tolist())
```

```
différence de risque = 0.150 | RR = 2.000 | OR = 2.4286
IC95 % de l'OR : [1.567 ; 3.764] (erreur-type du log-OR = 0.2235)
régression logistique : OR = 2.4286 | IC95 % : [1.567, 3.764]
```

**Corrigé 3.** (a) Site, 40 ans :  $\mu = e^{1,2-0,4+0,2} = e^{1,0} = 2,718$  commandes ; boutique :  $e^{0,8} = 2,226$ .  
(b) Pour 10 ans de plus :  $e^{-0,1} = 0,905$ , soit 9,5 % de commandes en moins. Le site est associé à  
 $e^{0,2} = 1,221$  fois plus de commandes que la boutique. (c)  $P(Y = 0) = e^{-\mu} = e^{-2,718} = 0,066$  : 6,6  
% de zéros attendus (sous Poisson).

```
mu_site, mu_boutique = np.exp(1.2 - 0.01 * 40 + 0.2), np.exp(1.2 - 0.01 * 40)
print("moyennes (site, boutique) :", round(float(mu_site), 3), round(float(mu_boutique), 3))
print("facteur pour +10 ans :", round(float(np.exp(-0.1)), 4), "| facteur site/boutique :",
↪ round(float(np.exp(0.2)), 4))
print("P(Y = 0 | site, 40 ans) =", round(float(stats.poisson.pmf(0, mu_site)), 4))
```

```
moyennes (site, boutique) : 2.718 2.226
facteur pour +10 ans : 0.9048 | facteur site/boutique : 1.2214
```

$P(Y = 0 \mid \text{site}, 40 \text{ ans}) = 0.066$

**Corrigé 4.** (a) Avec  $\beta^{(0)} = (0, 0) : \eta = 0, \mu = e^0 = 1$  pour tous, donc  $W_i = \mu_i = 1$  (pour Poisson avec lien log,  $W = \mu$ ) et  $z_i = \eta_i + (y_i - \mu_i)/\mu_i = (0, 2, 4)$ . La régression de  $z$  sur  $x$  (poids tous égaux à 1) a pour pente  $\frac{\sum (x-1)(z-2)}{\sum (x-1)^2} = \frac{(-1)(-2)+0+(1)(2)}{2} = 2$  et pour ordonnée  $\bar{z} - 2\bar{x} = 2 - 2 = 0$  : donc  $\beta^{(1)} = (0; 2)$ . (b) La deuxième itération, avec  $\mu = (1; e^2; e^4)$ , donne déjà des valeurs bien plus raisonnables ; l'algorithme converge en quelques pas (voir le code).

```
x4 = np.array([0.0, 1, 2]); y4 = np.array([1.0, 3, 5])
X4 = np.column_stack([np.ones(3), x4])
beta = np.zeros(2)
for it in range(1, 9):
    eta = X4 @ beta
    mu = np.exp(eta)
    w = mu                                # poids de Poisson (lien log)
    z = eta + (y4 - mu) / mu              # réponse de travail
    beta = np.linalg.solve(X4.T @ (w[:, None] * X4), X4.T @ (w * z))
    if it <= 4:
        print(f"itération {it} : beta = {beta.round(5)}")
m4 = sm.GLM(y4, X4, family=sm.families.Poisson()).fit()
print("après 8 itérations :", beta.round(6), "| statsmodels :", m4.params.round(6))
```

```
itération 1 : beta = [0. 2.]
itération 2 : beta = [-0.17925  1.63377]
itération 3 : beta = [0.00434  1.15568]
itération 4 : beta = [0.1661  0.82978]
après 8 itérations : [0.207929 0.723349] | statsmodels : [0.207929 0.723349]
```

**Corrigé 5.** (a) Sans variable,  $\hat{\mu} = 3 : D = 2[1 \log \frac{1}{3} + 3 \log 1 + 5 \log \frac{5}{3} - (9 - 9)] = 2[-1,0986 + 0 + 2,5541] = 2,911$  ; Pearson =  $\frac{(1-3)^2+0+(5-3)^2}{3} = 2,667$ . (b) La déviance du modèle avec  $x$ , calculée ci-dessous, est beaucoup plus petite : la différence  $D_0 - D_1$  se compare à  $\chi^2_1$ .

```
mu0 = np.full(3, y4.mean())
d0 = 2 * np.sum(y4 * np.log(y4 / mu0) - (y4 - mu0))
print("déviance sans variable :", round(float(d0), 4), "| Pearson :", round(float(np.sum((y4 - mu0)
    ** 2 / mu0)), 4))
m0 = sm.GLM(y4, np.ones((3, 1)), family=sm.families.Poisson()).fit()
print("statsmodels : déviance nulle =", round(float(m0.deviance), 4), "| déviance avec x =",
    round(float(m4.deviance), 4))
delta = m0.deviance - m4.deviance
print(f"rapport de vraisemblance : {delta:.4f} sur 1 ddl, p = {stats.chi2.sf(delta, 1):.4f}")
```

```
déviance sans variable : 2.911 | Pearson : 2.6667
statsmodels : déviance nulle = 2.911 | déviance avec x = 0.1363
rapport de vraisemblance : 2.7748 sur 1 ddl, p = 0.0958
```

La déviance du modèle avec  $x$  est de 0,136 : il épouse presque parfaitement les trois points (la pente estimée est 0,7233, soit un facteur  $e^{0,7233} = 2,06$  par unité de  $x$ ). La différence  $2,911 - 0,136 = 2,77$  donne pourtant  $p = 0,096$  : au seuil de 5 %, on **ne rejette pas** l'absence d'effet de  $x$ . Avec seulement 3 observations, le test a très peu de puissance (et l'approximation par un  $\chi^2$  est de toute façon douteuse pour un si petit échantillon) : une pente énorme n'est pas « significative » faute de données.

**Corrigé 6.** (a) Taux par millier de colis :  $A = 30/200 = 0,15$  ;  $B = 12/50 = 0,24$  ;  $C = 40/400 = 0,10$ . Rapports de taux :  $B/A = 1,6$ ,  $C/A = 0,667$  : B est le transporteur le plus mauvais, C le meilleur. (b) Avec le décalage  $\log(\text{exposition})$ , le modèle (saturé : 3 paramètres pour 3 transporteurs) reproduit exactement ces rapports. **Sans** décalage, il compare des nombres bruts de retards (30, 12, 40) : il conclut que B a  $12/30 = 0,4$  fois les retards de A et C 1,33 fois : un renversement complet,



parce que C livre beaucoup plus de colis.

```
transp = pd.DataFrame({"transporteur": ["A", "B", "C"], "milliers": [200, 50, 400], "retards": [30,
↳ 12, 40]})
transp["taux"] = transp["retards"] / transp["milliers"]
print(transp.to_string(index=False))
avec = smf.glm("retards ~ transporteur", transp, family=sm.families.Poisson(),
↳ offset=np.log(transp["milliers"])).fit()
sans = smf.glm("retards ~ transporteur", transp, family=sm.families.Poisson()).fit()
print("rapports de taux avec décalage (B/A, C/A) :", np.exp(avec.params[["transporteur[T.B]",
↳ "transporteur[T.C]"]]).round(3).tolist())
print("rapports sans décalage (B/A, C/A) :", np.exp(sans.params[["transporteur[T.B]",
↳ "transporteur[T.C]"]]).round(3).tolist())
```

```
transporteur  milliers  retards  taux
            A      200      30  0.15
            B       50      12  0.24
            C      400      40  0.10
rapports de taux avec décalage (B/A, C/A) : [1.6, 0.667]
rapports sans décalage (B/A, C/A)          : [0.4, 1.333]
```

**Corrigé 7.** (a)  $\hat{\phi} = X^2/(n-p) = 540/(305-5) = 1,8$ . (b) L'erreur-type corrigée est  $0,12 \times \sqrt{1,8} = 0,161$ , et  $z = 0,30/0,161 = 1,86$  au lieu de 2,5. La p-valeur passe de 0,012 à 0,063 : l'effet est **significatif** à 5 % avec la loi de Poisson, mais **ne l'est plus** après correction. C'est exactement le danger de la surdispersion non corrigée.

```
phi = 540 / (305 - 5)
se_c = 0.12 * np.sqrt(phi)
z_nc, z_c = 0.30 / 0.12, 0.30 / se_c
print(f"phi = {phi:.2f} | erreur-type corrigée = {se_c:.4f}")
print(f"z non corrigé = {z_nc:.2f} (p = {2 * stats.norm.sf(z_nc):.4f}) | z corrigé = {z_c:.2f} (p = {2
↳ * stats.norm.sf(z_c):.4f})")
```

```
phi = 1.80 | erreur-type corrigée = 0.1610
z non corrigé = 2.50 (p = 0.0124) | z corrigé = 1.86 (p = 0.0624)
```

**Corrigé 8.** (a)  $e^{5,0} = 148,4$  DT : c'est la **médiane** prévue (si les résidus de  $\log y$  sont symétriques), pas la moyenne. (b) Pour  $\log Y \sim \mathcal{N}(5; 0,9^2)$ ,  $E[Y] = e^{5+\sigma^2/2} = e^{5+0,405} = 148,4 \times 1,499 = 222,5$  DT : la moyenne est **50 % plus grande** que  $e^5$ . Ne pas retransformer sans correction revient à sous-estimer systématiquement la dépense moyenne (2.3.4).

```
rng = np.random.default_rng(8)
y_sim = rng.lognormal(mean=5.0, sigma=0.9, size=1_000_000)
print("exp(5,0) =", round(float(np.exp(5.0)), 1), "| médiane simulée :",
↳ round(float(np.median(y_sim)), 1))
print("exp(5 + sigma^2/2) =", round(float(np.exp(5.0 + 0.9 ** 2 / 2)), 1), "| moyenne simulée :",
↳ round(float(y_sim.mean()), 1))
```

```
exp(5,0) = 148.4 | médiane simulée : 148.6
exp(5 + sigma^2/2) = 222.5 | moyenne simulée : 222.6
```

**Corrigé 9.** (a) Offre :  $\Delta D = 2740,47 - 2710,83 = 29,64$  sur 1 ddl,  $p = 5,2 \times 10^{-8}$  ; canal :  $\Delta D = 2728,57 - 2710,83 = 17,74$  sur 2 ddl,  $p = 1,4 \times 10^{-4}$ . Les deux variables sont nécessaires. (b)  $\Delta AIC = \Delta D - 2q$  et  $\Delta BIC = \Delta D - q \log n$  (avec  $\log 2000 = 7,60$ ) : pour l'offre,  $\Delta AIC = 27,64$  et  $\Delta BIC = 22,04$  ; pour le canal,  $\Delta AIC = 13,74$  et  $\Delta BIC = 2,54$ . Tous les écarts sont positifs : les critères gardent chaque variable, en accord avec les tests ; le BIC est beaucoup moins enthousiaste pour le canal (+2,5 seulement), qui est le cas le plus marginal.

```
n = 2000
for nom, d_red, d_comp, q in [("offre", 2740.47, 2710.83, 1), ("canal", 2728.57, 2710.83, 2)]:
    dd = d_red - d_comp
```



```
print(f"{nom:6s}: ΔD = {dd:6.2f} sur {q} ddl | p = {stats.chi2.sf(dd, q):.2e} | ΔAIC = {dd - 2 *
↳ q:6.2f} | ΔBIC = {dd - q * np.log(n):6.2f}")
```

```
offre : ΔD = 29.64 sur 1 ddl | p = 5.20e-08 | ΔAIC = 27.64 | ΔBIC = 22.04
canal : ΔD = 17.74 sur 2 ddl | p = 1.41e-04 | ΔAIC = 13.74 | ΔBIC = 2.54
```

**Corrigé 10.** Ajoutons la ville (6 modalités, donc 5 paramètres de plus) au modèle de rachat et testons.

```
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram",
↳ "Site"])
base = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
↳ family=sm.families.Binomial()).fit()
avec_ville = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + C(ville)", clients,
↳ family=sm.families.Binomial()).fit()
dd = base.deviance - avec_ville.deviance
print(f"ΔD = {dd:.2f} sur 5 ddl | p (rapport de vraisemblance) = {stats.chi2.sf(dd, 5):.3f}")
print(f"AIC : sans ville = {base.aic:.1f} | avec ville = {avec_ville.aic:.1f} | BIC : sans =
↳ {base.bic_1lf:.1f} | avec = {avec_ville.bic_1lf:.1f}")
ic = avec_ville.conf_int()
print(pd.DataFrame({"OR": np.exp(avec_ville.params), "IC95 bas": np.exp(ic[0]), "IC95 haut":
↳ np.exp(ic[1])}).filter(like="ville", axis=0).round(3).to_string())
```

```
ΔD = 1.13 sur 5 ddl | p (rapport de vraisemblance) = 0.952
AIC : sans ville = 2720.8 | avec ville = 2729.7 | BIC : sans = 2748.8 | avec = 2785.7
      OR  IC95 bas  IC95 haut
C(ville)[T.Bizerte]  0.985    0.687    1.411
C(ville)[T.Nabeul]   1.002    0.713    1.407
C(ville)[T.Sfax]     0.901    0.648    1.253
C(ville)[T.Sousse]   1.062    0.774    1.457
C(ville)[T.Tunis]    1.023    0.773    1.354
```

La ville n'améliore pas le modèle :  $\Delta D = 1,13$  pour 5 degrés de liberté ( $p = 0,95$ , un résultat parfaitement banal si la ville n'a aucun effet). L'AIC **se dégrade** (de 2 720,8 à 2 729,7) et le BIC encore davantage (de 2 748,8 à 2 785,7) : les cinq paramètres de plus ne rapportent presque rien en vraisemblance. Les rapports de cotes de toutes les villes par rapport à la ville de référence (Autre) sont compris entre 0,90 et 1,06, avec des intervalles de confiance qui contiennent tous 1. **Conclusion : la ville n'a pas d'effet détectable sur le rachat**, et l'on garde le modèle sans elle. (Nous verrons plus bas que, dans la simulation, la ville n'a effectivement aucun rôle.)

**Corrigé 11.** On balaye les seuils de 0,05 à 0,95 et l'on garde celui qui maximise  $J$ .

```
enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
enquete["note_produits"] = enquete[["q1", "q2", "q3", "q4"]].mean(axis=1)
enquete["note_service"] = enquete[["q5", "q6", "q7", "q8"]].mean(axis=1)
rep = clients.merge(enquete[["id_client", "note_produits", "note_service"]], on="id_client")
riche = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service", rep,
↳ family=sm.families.Binomial()).fit()
yr, sr = rep["rachat_12m"].to_numpy(), riche.fittedvalues.to_numpy()

seuils = np.arange(0.05, 0.96, 0.01)
sens = np.array([(sr >= s) & (yr == 1)].sum() / (yr == 1).sum() for s in seuils)
spec = np.array([(sr < s) & (yr == 0)].sum() / (yr == 0).sum() for s in seuils)
J = sens + spec - 1
k = int(np.argmax(J))
print(f"seuil optimal (Youden) = {seuils[k]:.2f} | sensibilité = {sens[k]:.3f} | spécificité =
↳ {spec[k]:.3f} | J = {J[k]:.3f}")
k5 = int(np.argmin(np.abs(seuils - 0.5)))
print(f"seuil 0,50 : sensibilité = {sens[k5]:.3f} | spécificité = {spec[k5]:.3f} | J =
↳ {J[k5]:.3f}")
print("part de rachat dans l'échantillon :", round(float(yr.mean()), 3))
```

```
seuil optimal (Youden) = 0.57 | sensibilité = 0.526 | spécificité = 0.772 | J = 0.298
seuil 0,50          : sensibilité = 0.663 | spécificité = 0.620 | J = 0.283
part de rachat dans l'échantillon : 0.505
```

Le seuil de Youden est de **0,57** : sensibilité 0,526, spécificité 0,772,  $J = 0,298$ . Au seuil de 0,5,  $J = 0,283$ . Le gain est minuscule : la surface de  $J$  est très plate autour de son maximum. Gardez deux idées en tête. (1) Un seuil « optimal » choisi sur les données qui ont servi à l'ajuster est un peu optimiste (2.2.8). (2) L'indice de Youden donne le même poids aux deux types d'erreur ; dans la pratique, le bon seuil dépend de leurs **coûts** (ici : relancer inutilement un client coûte peu, laisser partir un client précieux coûte cher).

**Corrigé 12.** On ajuste une spline pour chaque  $df$  et l'on compare les AIC.

```
sessions = pd.read_csv("donnees/ch02-sessions.csv")
lignes = []
for k in range(4, 13):
    r = smf.glm(f"achat ~ bs(duree_min, df={k}, lower_bound=0, upper_bound=40)", sessions,
    ↪ family=sm.families.Binomial()).fit()
    lignes.append({"fonctions de base": k, "paramètres": len(r.params), "AIC": r.aic})
res = pd.DataFrame(lignes)
res["ΔAIC"] = res["AIC"] - res["AIC"].min()
print(res.round(1).to_string(index=False))
print("nombre de fonctions de base retenu :", int(res.loc[res["AIC"].idxmin(), "fonctions de base"]))
```

| fonctions de base | paramètres | AIC    | ΔAIC |
|-------------------|------------|--------|------|
| 4                 | 5          | 1722.1 | 18.7 |
| 5                 | 6          | 1704.2 | 0.8  |
| 6                 | 7          | 1703.5 | 0.0  |
| 7                 | 8          | 1703.6 | 0.1  |
| 8                 | 9          | 1703.9 | 0.4  |
| 9                 | 10         | 1705.7 | 2.2  |
| 10                | 11         | 1707.4 | 4.0  |
| 11                | 12         | 1709.1 | 5.7  |
| 12                | 13         | 1707.7 | 4.2  |

nombre de fonctions de base retenu : 6

L'AIC est minimal pour 6 fonctions de base (1 703,5), mais les valeurs pour 5, 7 et 8 fonctions n'en sont qu'à 0,8, 0,1 et 0,4 point : un **plateau**, pas un minimum net. À 4 fonctions la courbe est trop rigide (+18,7) ; à partir de 9 fonctions l'AIC remonte de 2 à 6 points. L'avantage de la **pénalisation** est précisément d'éviter cette recherche à tâtons : on prend une base large et c'est le paramètre de lissage  $\lambda$ , choisi automatiquement, qui règle la souplesse (nous avons obtenu 6,5 degrés de liberté effectifs, au milieu de ce plateau).

**Corrigé 13.** Le rapport variance/moyenne d'un ZIP vaut  $1 + \pi\mu$ . Avec  $\mu = 4$  :  $1 + 4\pi = 2$ , donc  $\pi = 0,25$ . La probabilité d'un zéro est  $\pi + (1 - \pi)e^{-\mu} = 0,25 + 0,75e^{-4} = 0,25 + 0,75 \times 0,0183 = 0,2637$ .

```
rng = np.random.default_rng(13)
N = 1_000_000
y_zip = np.where(rng.random(N) < 0.25, 0, rng.poisson(4, N))
print(f"variance / moyenne simulée = {y_zip.var() / y_zip.mean():.3f} (attendu 2) | P(0) simulée =
↪ {np.mean(y_zip == 0):.4f} | formule = {0.25 + 0.75 * np.exp(-4):.4f}")
```

variance / moyenne simulée = 2.001 (attendu 2) | P(0) simulée = 0.2641 | formule = 0.2637

**Corrigé 14.** (a) La densité de la loi Gamma de moyenne  $\mu$  et de forme  $\nu$  est  $f(y) = \frac{1}{\Gamma(\nu)} \left(\frac{\nu}{\mu}\right)^\nu y^{\nu-1} e^{-\nu y/\mu}$ , donc  $\log f = \nu(-\frac{y}{\mu} - \log \mu) + (\nu-1) \log y + \nu \log \nu - \log \Gamma(\nu)$ . Avec  $\theta = -1/\mu$ ,  $\phi = 1/\nu$  et  $b(\theta) = -\log(-\theta) = \log \mu$ , on obtient  $\frac{y\theta - b(\theta)}{\phi} = \nu(-\frac{y}{\mu} - \log \mu)$  : le reste ne dépend

pas de  $\theta$ , c'est  $c(y, \phi)$ . Alors  $b'(\theta) = -1/\theta = \mu$  et  $b''(\theta) = 1/\theta^2 = \mu^2$ , d'où  $\text{Var}(Y) = \phi b''(\theta) = \mu^2/\nu$  : le coefficient de variation  $1/\sqrt{\nu}$  est constant. (b)  $P(Y = 0) = \exp(-\mu^{2-p}/(\phi(2-p)))$  avec  $\mu^{0,7} = 100^{0,7} = 25,12$ , donc  $\lambda = 25,12/(15 \times 0,7) = 2,392$  et  $P(Y = 0) = e^{-2,392} = 0,091$ . Puis  $\alpha = \frac{2-p}{p-1} = \frac{0,7}{0,3} = 2,333$  et  $\theta = \phi(p-1)\mu^{p-1} = 15 \times 0,3 \times 100^{0,3} = 4,5 \times 3,981 = 17,91$  ; on vérifie  $\lambda\alpha\theta = 2,392 \times 2,333 \times 17,91 \approx 100$ .

```
mu_t, phi_t, p_t = 100.0, 15.0, 1.3
lam = mu_t ** (2 - p_t) / (phi_t * (2 - p_t)); alpha = (2 - p_t) / (p_t - 1); theta = phi_t * (p_t - 1) * mu_t ** (p_t - 1)
print(f"lambda = {lam:.3f} | alpha = {alpha:.3f} | theta = {theta:.3f} | lambda*alpha*theta = {lam * alpha * theta:.2f} | P(0) = exp(-lambda) = {np.exp(-lam):.4f}")
rng = np.random.default_rng(14)
N = 500_000
n_cmd = rng.poisson(lam, N)
y_t = np.where(n_cmd > 0, rng.gamma(np.maximum(n_cmd, 1) * alpha, theta), 0.0)
print(f"simulation : P(0) = {np.mean(y_t == 0):.4f} | moyenne = {y_t.mean():.2f} | variance = {y_t.var():.1f} | phi * mu^p = {phi_t * mu_t ** p_t:.1f}")

lambda = 2.392 | alpha = 2.333 | theta = 17.915 | lambda*alpha*theta = 100.00 | P(0) = exp(-lambda) = 0.0914
simulation : P(0) = 0.0918 | moyenne = 99.82 | variance = 5962.3 | phi * mu^p = 5971.6
```

La simulation confirme le calcul :  $P(Y = 0) = 0,0918$  simulée pour 0,0914 théorique, moyenne 99,82 pour 100, variance 5 962 pour  $\phi\mu^p = 5 972$ . Les paramètres  $(\lambda, \alpha, \theta) = (2,392; 2,333; 17,91)$  sont donc cohérents avec la loi de Tweedie annoncée.

## Bilan du chapitre 2

Vous savez maintenant :

- **reconnaître** les situations où la régression linéaire ne convient pas (0/1, comptages, montants positifs avec zéros) et formuler un **GLM** : une **loi** de la famille exponentielle, un **prédicteur linéaire**, un **lien** ;
- **démontrer** que, pour la famille exponentielle,  $E[Y] = b'(\theta)$  et  $\text{Var}(Y) = \phi V(\mu)$ , et **estimer** un GLM par maximum de vraisemblance avec l'algorithme **IRLS**, que vous avez programmé à la main ;
- **interpréter** une régression logistique (cotes, rapports de cotes, probabilités prédites, effets marginaux, ROC, AUC, calibration), et ne pas confondre rapport de cotes et risque relatif ;
- **modéliser** des comptages (Poisson, décalage pour l'exposition, binomiale négative en cas de **surdispersion**) et des montants positifs (Gamma avec lien log, qui modélise la moyenne) ;
- **vérifier** un modèle : déviance, rapport de vraisemblance, AIC/BIC, résidus de Pearson et **résidus quantiles aléatoires**, test de Hosmer-Lemeshow, calibration, observations influentes ;
- (en option) **assouplir** un effet par un **GAM**, et traiter des **zéros en excès** (ZIP, barrière, **Tweedie**, modèle à deux parties).

Le chapitre 3 passe de la modélisation d'**une** variable à celle de **plusieurs variables à la fois** : l'analyse multivariée (ACP, analyse factorielle, classification) cherche la structure cachée dans un grand tableau de variables corrélées. Le chapitre 4 traitera ensuite des données ordonnées dans le temps (séries temporelles), et le projet de clôture du volume réunira les modèles linéaires généralisés et les séries temporelles dans une étude complète.

## La vérité dévoilée

Nos données sont simulées : nous connaissons les paramètres qui les ont produites. Il est temps de les confronter aux estimations du chapitre. Le tableau ci-dessous compare, pour chaque modèle, l'estimation, son erreur-type, la **vraie valeur** (celle du programme de simulation, build/donnees2.py) et l'écart exprimé en erreurs-types.

```
import statsmodels.api as sm
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Instagram",
↪ "Site"])

# (1) rachat : logistique (vérité sur l'échelle du logit, canal relatif à la boutique)
logi = smf.glm("rachat_l2m ~ offre_bienvenue + age + canal", clients,
↪ family=sm.families.Binomial()).fit()
# (2) nombre de commandes : binomiale négative (log de la moyenne)
nbm = smf.negativebinomial("nb_commandes_an ~ offre_bienvenue + age + canal", clients).fit(dis=0)
# (3) dépense moyenne de tous les clients : Tweedie p = 1,45 (log de la moyenne)
twd = smf.glm("depense_annuelle ~ offre_bienvenue + age + canal", clients,
↪ family=sm.families.Tweedie(var_power=1.45, link=sm.families.links.Log())).fit(scale="X2")

verite = {
    "rachat (logit)": (logi, {"offre_bienvenue": 0.55, "age": -0.015, "canal[T.Instagram]": -0.30,
↪ "canal[T.Site]": -0.30}),
    "commandes (log moyenne)": (nbm, {"offre_bienvenue": 0.0, "age": -0.005, "canal[T.Instagram]":
↪ -0.05, "canal[T.Site]": 0.10}),
    "dépense (log moyenne)": (twd, {"offre_bienvenue": 0.0, "age": 0.003, "canal[T.Instagram]": -0.39,
↪ "canal[T.Site]": -0.07}),
}

lignes = []
for nom, (res, vrai) in verite.items():
    for param, v in vrai.items():
        est, se = float(res.params[param]), float(res.bse[param])
        lignes.append({"modèle": nom, "paramètre": param, "estimation": est, "erreur-type": se,
↪ "vérité": v, "écart (en erreurs-types)": (est - v) / se})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
print()
print("alpha (binomiale négative) :", round(float(nbm.params["alpha"]), 3), "| IC95 % :",
↪ nbm.conf_int().loc["alpha"].round(3).tolist())
# Écarts du modèle « commandes » : les moyennes par canal sont-elles conformes à la vérité programmée ?
var_f = 0.22 ** 2 + 0.10 ** 2 + 2 * 0.22 * 0.10 * 0.3 # variance de 0,22 F1 +
↪ 0,10 F2
eff_canal = {"Boutique": 0.05, "Instagram": 0.0, "Site": 0.15}
lignes = []
for canal, e in eff_canal.items():
    g = clients[clients["canal"] == canal]["nb_commandes_an"]
    ages = clients.loc[clients["canal"] == canal, "age"]
    attendu = np.exp(1.25 + e + var_f / 2) * np.mean(np.exp(-0.005 * (ages - 36))) # E[N] sous la
↪ vérité programmée
    lignes.append({"canal": canal, "clients": len(g), "commandes attendues": attendu, "commandes
↪ observées": g.mean(),
↪ "écart (en erreurs-types de la moyenne)": (g.mean() - attendu) / (g.std() /
↪ np.sqrt(len(g)))})
print()
print(pd.DataFrame(lignes).round(3).to_string(index=False))
print()
print("hétérogénéité attendue : (1 + 0,5) × (1 + variance relative de exp(0,22 F1 + 0,10 F2)) - 1 =",
↪ round((1 + 0.5) * np.exp(0.22 ** 2 + 0.10 ** 2 + 2 * 0.22 * 0.10 * 0.3) - 1, 3))
```

|  | modèle         | paramètre          | estimation | erreur-type | vérité | écart (en erreurs-types) |
|--|----------------|--------------------|------------|-------------|--------|--------------------------|
|  | rachat (logit) | offre_bienvenue    | 0.493      | 0.091       | 0.550  | -0.624                   |
|  | rachat (logit) | age                | -0.015     | 0.004       | -0.015 | -0.114                   |
|  | rachat (logit) | canal[T.Instagram] | -0.463     | 0.116       | -0.300 | -1.411                   |
|  | rachat (logit) | canal[T.Site]      | -0.168     | 0.120       | -0.300 | 1.106                    |

|                         |                    |        |       |        |        |
|-------------------------|--------------------|--------|-------|--------|--------|
| commandes (log moyenne) | offre_bienvenue    | -0.016 | 0.041 | 0.000  | -0.379 |
| commandes (log moyenne) | age                | -0.001 | 0.002 | -0.005 | 1.938  |
| commandes (log moyenne) | canal[T.Instagram] | -0.176 | 0.052 | -0.050 | -2.430 |
| commandes (log moyenne) | canal[T.Site]      | 0.015  | 0.053 | 0.100  | -1.597 |
| dépense (log moyenne)   | offre_bienvenue    | -0.014 | 0.051 | 0.000  | -0.277 |
| dépense (log moyenne)   | age                | 0.008  | 0.002 | 0.003  | 2.112  |
| dépense (log moyenne)   | canal[T.Instagram] | -0.555 | 0.064 | -0.390 | -2.570 |
| dépense (log moyenne)   | canal[T.Site]      | -0.151 | 0.064 | -0.070 | -1.270 |

alpha (binomiale négative) : 0.584 | IC95 % : [0.528, 0.639]

| canal     | clients | commandes attendues | commandes observées | écart (en erreurs-types de la moyenne) |
|-----------|---------|---------------------|---------------------|----------------------------------------|
| Boutique  | 504     | 3.818               | 4.137               | 1.930                                  |
| Instagram | 816     | 3.620               | 3.464               | -1.293                                 |
| Site      | 680     | 4.219               | 4.197               | -0.154                                 |

hétérogénéité attendue :  $(1 + 0,5) \times (1 + \text{variance relative de } \exp(0,22 F1 + 0,10 F2)) - 1 = 0.611$

Voici le bilan, sans fard.

**Ce qui est retrouvé.** Dans le modèle de **rachat**, toutes les estimations sont à moins de 1,5 erreur-type de la vérité. Le coefficient de l'offre (0,493 pour 0,55) est un peu **atténué** : les facteurs latents de goût pour les produits et de sensibilité au service, qui influencent réellement le rachat, ne sont pas dans ce modèle, et omettre une variable qui explique le résultat atténue, dans un modèle logistique, les coefficients des autres variables (c'est la non-collapsibilité vue en 2.2.7 ; un calcul approché donne un facteur voisin de 0,93, soit environ 0,51, compatible avec 0,493). L'offre n'a, comme programmé, **aucun effet** sur le nombre de commandes ( $-0,016$ ,  $z = -0,38$ ) ni sur la dépense ( $-0,014$ ,  $z = -0,28$ ), et la ville n'a aucun rôle (exercice 10). Le paramètre de surdispersion  $\hat{\alpha} = 0,584$  (intervalle de 0,528 à 0,639) **exclut** la valeur 0,5 programmée pour l'hétérogénéité de la loi Gamma, mais ce n'est pas une erreur du modèle : l'hétérogénéité totale comprend aussi celle que créent les deux facteurs latents omis, et le calcul de la dernière ligne donne  $(1 + 0,5) \times \exp(0,0716) - 1 = 0,611$ , **à l'intérieur** de l'intervalle estimé.

**Ce qui s'écarte, et pourquoi.** Quatre des douze écarts dépassent environ deux erreurs-types : l'effet d'Instagram sur le nombre de commandes ( $-0,176$  pour  $-0,05$ ,  $z = -2,4$ ) et sur la dépense ( $-0,555$  pour  $-0,39$ ,  $z = -2,6$ ), et l'effet de l'âge sur la dépense ( $z = 2,1$ ) et sur les commandes ( $z = 1,9$ ). Ces écarts ne sont **pas indépendants** : le deuxième tableau ci-dessus (commandes attendues et observées par canal) montre que, dans cet échantillon, les clients de la **boutique** ont passé en moyenne 4,14 commandes alors que la vérité en prévoit 3,82 (1,9 erreur-type de plus), tandis que ceux d'Instagram en ont passé un peu moins que prévu (3,46 pour 3,62,  $-1,3$  erreur-type). C'est une fluctuation d'échantillonnage (assez rare, mais pas invraisemblable) qui se propage aux effets sur les commandes **et** sur la dépense, puisque la dépense est le produit du nombre de commandes par le panier. Les erreurs-types du modèle de Tweedie reposent de plus sur une forme de variance seulement approximative (2.6.3), ce qui peut les rendre un peu optimistes (nous ne l'avons pas vérifié ici). Retenez la leçon : **un estimateur peut s'écarter de plus de deux erreurs-types de la vérité sans qu'il y ait de défaut dans le modèle**, parce que l'échantillon est une réalisation parmi d'autres ; avec douze comparaisons corrélées, ce n'est pas un signal d'alarme.

# Chapitre 3 : Analyse multivariée

« Une variable, on la regarde. Deux variables, on les relie. Vingt variables, il faut apprendre à les **résumer**. »

Les chapitres 1 et 2 avaient un point commun : une variable à **expliquer** (la dépense, le rachat, le nombre de commandes) et des variables **explicatives**. Ici, le décor change. Nous n'avons plus de variable réponse : nous avons **un tableau de variables**, et la question est « que contient ce tableau ? ». Quelles variables disent la même chose ? Peut-on les remplacer par deux ou trois indicateurs qui conservent l'essentiel ? Les clients se rangent-ils naturellement en familles ? C'est ce qu'on appelle l'**analyse multivariée**, ou, dans le vocabulaire de l'apprentissage automatique, l'apprentissage **non supervisé** : on cherche de la structure sans qu'aucune « bonne réponse » ne guide la recherche.

Yasmine a posé ces questions sans le savoir. Elle a envoyé à ses clientes un petit questionnaire de huit questions et se retrouve avec huit colonnes de notes qui se ressemblent beaucoup : « Les clientes qui aiment la qualité des bols aiment aussi celle des foutahs, non ? » Elle voudrait **un ou deux chiffres par cliente** plutôt que huit. Et elle voudrait aussi savoir si ses 2 000 clientes forment des « types » (les occasionnelles, les fidèles, les grosses acheteuses) à qui s'adresser différemment.

## Le chemin de ce chapitre

- **3.1 Analyse en composantes principales (ACP)** : la technique reine pour **résumer** des variables numériques corrélées. Nous la construirons à la main sur cinq clientes, la démontrerons proprement (c'est un problème d'optimisation, avec les valeurs propres du volume I), puis l'appliquerons au questionnaire.
- **3.2 Analyse factorielle** : un modèle voisin mais différent. L'ACP **résume** ; l'analyse factorielle **explique** les corrélations par des causes cachées (des « facteurs »), comme la qualité du service. Nous apprendrons à les estimer, à les faire tourner (rotation), et à vérifier si l'on retrouve ce que la simulation avait programmé.
- **3.3 Classification non supervisée** : les **k-means** (écrits en vingt lignes de NumPy) et la **classification hiérarchique**, avec leurs pièges, et une segmentation honnête des clientes de Dar Jasmin.
- **3.4 Analyse des correspondances (AC et ACM)** : l'équivalent de l'ACP pour des **variables qualitatives** (canal, ville, tranche d'âge).
- **3.5 Analyse discriminante** : quand les groupes sont connus d'avance et que l'on cherche la meilleure frontière entre eux ; le lien avec la régression logistique de la section 2.2.
- **3.6 Exercices corrigés.**

**Comment travailler avec ce chapitre.** Les algorithmes de ce chapitre sont courts : nous les écrirons **nous-mêmes** en NumPy avant d'utiliser les versions de `scikit-learn`, puis nous comparerons les deux. Si votre version et celle de la bibliothèque donnent les mêmes nombres, vous avez compris l'algorithme. Un conseil : à chaque figure, **prédisez ce que vous**

**allez voir** avant de lire la légende.

**Ce que vous devez avoir en tête du volume I.** Le chapitre s'appuie sur les valeurs propres et la décomposition en valeurs singulières (volume I, sections 1.1.3 et 1.1.4), sur la covariance et la corrélation (volume I, sections 2.3 et 3.1.7) et sur les multiplicateurs de Lagrange (volume I, section 1.3.5). Si ces notions sont floues, relisez-les d'abord : l'ACP n'en est, au fond, que l'application directe.

**Les données.** Deux fichiers du dossier `donnees/`, **entièrement simulés** :

- `enquete_satisfaction.csv` : 1 212 répondantes (60 % des clientes, tirées au hasard) et leurs huit notes de 1 à 5, `q1` à `q8`. Les quatre premières portent sur les **produits** (qualité, finitions...), les quatre dernières sur le **service** (livraison, emballage...).
- `clients.csv` : les 2 000 clientes, avec âge, canal d'acquisition, nombre de commandes, panier moyen, dépense annuelle et durée de la relation.

Comme les données sont simulées, nous connaissons **la vérité** et nous la révélerons à la fin de chaque étude pour juger de la qualité de nos méthodes. Dans la vraie vie, cette vérité n'est jamais disponible : c'est précisément ce qui rend les méthodes de ce chapitre difficiles à valider, et ce qui impose la prudence que nous cultiverons à chaque page.

## 3.1 Analyse en composantes principales

**Intuition.** Vous photographiez une sculpture. Sous certains angles, la photo est plate et on ne comprend rien ; sous d'autres, on voit tout de suite la forme. L'**analyse en composantes principales** (ACP) cherche l'**angle de prise de vue qui garde le plus d'information possible** quand on aplatit un nuage de points en haute dimension vers une dimension plus petite. Ici, « information » veut dire **variance** : on garde les directions dans lesquelles les données s'étalent le plus, et on jette celles où elles sont presque constantes.

### 3.1.1 Le problème : huit colonnes qui se ressemblent

Reprenons le questionnaire de Yasmine. Voici la **matrice de corrélation** de ses huit questions (volume I, section 3.1.7) :

```
import numpy as np
import pandas as pd

q = pd.read_csv("donnees/enquete_satisfaction.csv")
items = [f"q{j}" for j in range(1, 9)]
Q = q[items]
print(Q.shape)
print(Q.corr().round(2).to_string())
```

```
(1212, 8)
      q1    q2    q3    q4    q5    q6    q7    q8
q1  1.00  0.48  0.50  0.41  0.11  0.11  0.09  0.11
q2  0.48  1.00  0.44  0.33  0.12  0.11  0.09  0.08
q3  0.50  0.44  1.00  0.34  0.14  0.15  0.13  0.12
q4  0.41  0.33  0.34  1.00  0.12  0.15  0.06  0.06
q5  0.11  0.12  0.14  0.12  1.00  0.47  0.56  0.48
q6  0.11  0.11  0.15  0.15  0.47  1.00  0.46  0.43
q7  0.09  0.09  0.13  0.06  0.56  0.46  1.00  0.45
q8  0.11  0.08  0.12  0.06  0.48  0.43  0.45  1.00
```



Regardez la structure en blocs. Les questions q1 à q4 sont toutes corrélées entre elles (de 0,3 à 0,5 environ), de même que q5 à q8 (de 0,4 à 0,55), alors que les corrélations **entre** les deux groupes sont faibles (autour de 0,1). Autrement dit, ce tableau de huit colonnes contient probablement **deux informations** : un niveau d'appréciation des produits, un niveau d'appréciation du service. Si c'est vrai, deux nombres par répondante devraient suffire à résumer les huit notes sans trop de perte.

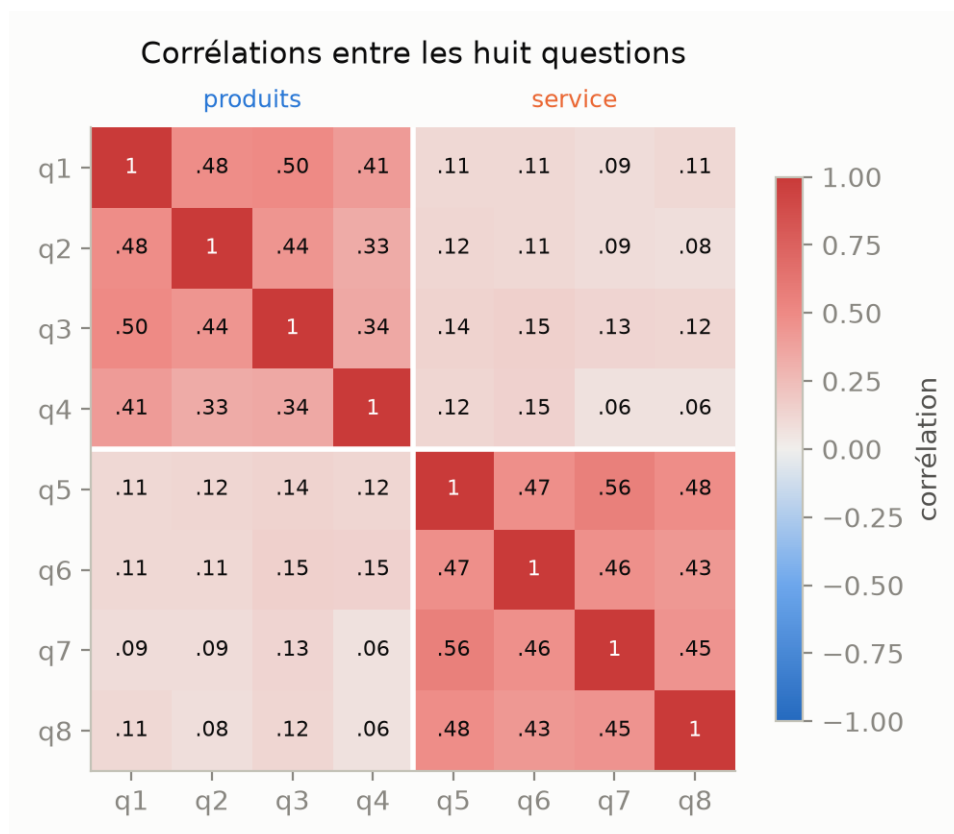


Figure 24: Matrice de corrélation des huit questions du questionnaire : deux blocs de questions corrélées entre elles (q1 à q4, q5 à q8), faiblement corrélés entre eux.

L'ACP est la méthode qui transforme cette intuition visuelle en calcul : elle trouve automatiquement les « bonnes » directions.

### 3.1.2 Un exemple à la main : cinq clientes, deux variables

Avant les huit dimensions, restons dans le plan, où l'on peut tout dessiner. Cinq clientes, deux variables : le **nombre de commandes** dans l'année et le **panier moyen**, exprimé ici en **dizaines de DT** pour garder des nombres simples (nous verrons en 3.1.5 pourquoi ce choix d'unité n'est pas innocent).

| Client | Commandes (x) | Panier en dizaines de DT (y) |
|--------|---------------|------------------------------|
| A      | 4             | 4                            |
| B      | 5             | 3                            |
| C      | 6             | 6                            |
| D      | 7             | 5                            |
| E      | 8             | 7                            |

**Étape 1 : centrer.** Les moyennes sont  $\bar{x} = 6$  et  $\bar{y} = 5$ . Les écarts à la moyenne valent donc  $x_c = (-2, -1, 0, 1, 2)$  et  $y_c = (-1, -2, 1, 0, 2)$ . Le nuage est maintenant centré sur l'origine.

**Étape 2 : la matrice de covariance.** Avec le diviseur  $n - 1 = 4$  (volume I, section 3.2.3) :

$$s_{xx} = \frac{4 + 1 + 0 + 1 + 4}{4} = 2,5, \quad s_{yy} = \frac{1 + 4 + 1 + 0 + 4}{4} = 2,5, \quad s_{xy} = \frac{2 + 2 + 0 + 0 + 4}{4} = 2.$$

$$S = \begin{pmatrix} 2,5 & 2 \\ 2 & 2,5 \end{pmatrix}.$$

**Étape 3 : les valeurs propres.** On résout  $\det(S - \lambda I) = 0$  :  $(2,5 - \lambda)^2 - 4 = 0$ , donc  $2,5 - \lambda = \pm 2$ , soit  $\lambda_1 = 4,5$  et  $\lambda_2 = 0,5$ .

**Étape 4 : les vecteurs propres.** Pour  $\lambda_1 = 4,5$ ,  $(S - 4,5I)v = 0$  donne  $-2v_1 + 2v_2 = 0$ , donc  $v_1 = v_2$  : le vecteur unitaire est  $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)$ . Pour  $\lambda_2 = 0,5$ , on trouve  $\mathbf{v}_2 = \frac{1}{\sqrt{2}}(1, -1)$ . Ils sont bien orthogonaux.

**Étape 5 : interpréter.** La variance totale est  $2,5 + 2,5 = 5$  (c'est la trace de  $S$ ) et  $\lambda_1 + \lambda_2 = 5$  aussi : l'ACP **redistribue** la variance totale entre les nouveaux axes, sans en perdre. Le premier axe en porte  $4,5/5 = 90\%$ . Sa direction  $(1, 1)/\sqrt{2}$  dit : « plus on commande, plus le panier est gros », c'est l'axe « **valeur de la cliente** ». Le deuxième axe,  $(1, -1)/\sqrt{2}$ , oppose les grosses commandeuses à petit panier aux petites commandeuses à gros panier.

**Étape 6 : les scores.** Le *score* d'une cliente sur l'axe  $j$  est sa coordonnée dans la nouvelle base :  $t_{ij} = \mathbf{v}_j^T \mathbf{x}_{c,i}$ . Pour la cliente A,  $\mathbf{x}_c = (-2, -1)$ , donc  $t_{A1} = (-2 - 1)/\sqrt{2} \approx -2,12$  et  $t_{A2} = (-2 + 1)/\sqrt{2} \approx -0,71$ . Vérifions par le calcul :

```
X = np.array([[4, 4], [5, 3], [6, 6], [7, 5], [8, 7]], dtype=float)
Xc = X - X.mean(axis=0)
S = np.cov(Xc, rowvar=False)           # diviseur n-1 par défaut
print("S =\n", S)

valeurs, vecteurs = np.linalg.eigh(S)   # eigh : pour les matrices symétriques, valeurs
↳ croissantes
ordre = np.argsort(valeurs)[::-1]       # on range par valeur propre décroissante
valeurs, vecteurs = valeurs[ordre], vecteurs[:, ordre]
print("valeurs propres :", valeurs.round(3))
print("part de variance :", (valeurs / valeurs.sum()).round(3))
print("vecteurs propres (en colonnes) :\n", vecteurs.round(3))

scores = Xc @ vecteurs
print("scores :\n", scores.round(2))
print("variance des scores :", scores.var(axis=0, ddof=1).round(3))
print("corrélation des scores :", np.corrcoef(scores.T)[0, 1].round(6))
```

```
S =
[[2.5 2. ]
 [2.  2.5]]
valeurs propres : [4.5 0.5]
part de variance : [0.9 0.1]
vecteurs propres (en colonnes) :
[[ 0.707 -0.707]
 [ 0.707  0.707]]
scores :
[[-2.12  0.71]
 [-2.12 -0.71]]
```

```
[ 0.71  0.71]
[ 0.71 -0.71]
[ 2.83  0.  ]
variance des scores : [4.5 0.5]
corrélation des scores : -0.0
```

Les valeurs propres sont bien 4,5 et 0,5. Le signe d'un vecteur propre est arbitraire (si  $\mathbf{v}$  convient,  $-\mathbf{v}$  aussi) : ne vous étonnez pas de voir  $(-0,707; -0,707)$  au lieu de  $(0,707; 0,707)$ , c'est le même axe. Remarquez aussi que la **variance de chaque score** est égale à la valeur propre correspondante, et que les deux scores sont **non corrélés**. Ce n'est pas un hasard, nous le démontrons plus bas.

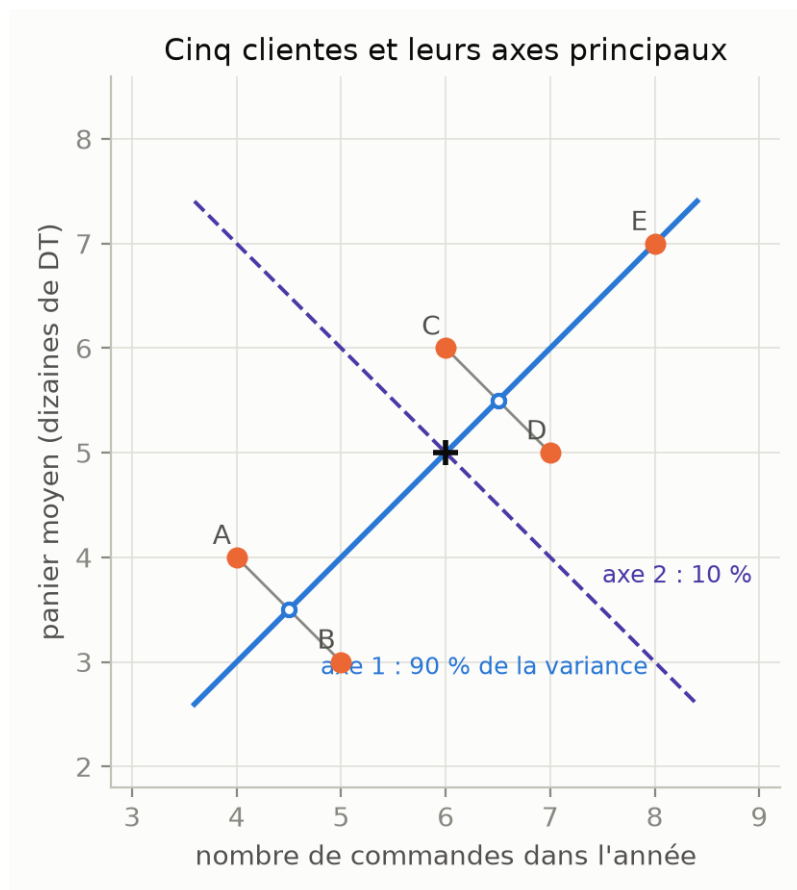


Figure 25: Les cinq clientes dans le plan commandes x panier (en dizaines de DT), les deux axes principaux (en trait plein, le premier ; en pointillés, le second) et les projections sur le premier axe.

**Ce que la figure montre.** Projeter les cinq points sur le premier axe (la droite de plus grande dispersion) conserve 90 % de la variance : on résume le nuage par **un seul nombre par cliente** en ne perdant que 10 %. Les segments gris sont ce qu'on jette. Projeter sur le second axe, au contraire, écraserait presque tout.

### 3.1.3 La théorie : l'ACP est un problème d'optimisation

Cherchons la direction (un vecteur unitaire  $\mathbf{v}$ ) sur laquelle la variance des projections est maximale. Pour un tableau centré  $X_c$  de  $n$  lignes et  $p$  colonnes, de matrice de covariance  $S = \frac{1}{n-1} X_c^T X_c$ , la projection de la ligne  $i$  est  $\mathbf{v}^T \mathbf{x}_{c,i}$  et la variance des projections vaut

$$\text{Var}(X_c \mathbf{v}) = \frac{1}{n-1} \mathbf{v}^\top X_c^\top X_c \mathbf{v} = \mathbf{v}^\top S \mathbf{v}.$$

**Théorème (première composante).** Le maximum de  $\mathbf{v}^\top S \mathbf{v}$  sous la contrainte  $\|\mathbf{v}\| = 1$  est la plus grande valeur propre  $\lambda_1$  de  $S$ , atteinte en un vecteur propre associé.

**Démonstration.** On utilise les multiplicateurs de Lagrange (volume I, section 1.3.5). Le lagrangien est  $\mathcal{L}(\mathbf{v}, \lambda) = \mathbf{v}^\top S \mathbf{v} - \lambda(\mathbf{v}^\top \mathbf{v} - 1)$ . Comme  $S$  est symétrique, le gradient en  $\mathbf{v}$  vaut  $2S\mathbf{v} - 2\lambda\mathbf{v}$ . Il s'annule si et seulement si  $S\mathbf{v} = \lambda\mathbf{v}$  : **tout point critique est un vecteur propre**. En un tel point, la valeur à maximiser est  $\mathbf{v}^\top S \mathbf{v} = \lambda \mathbf{v}^\top \mathbf{v} = \lambda$ . Le maximum est donc la plus grande des valeurs propres.  $\square$

**Les composantes suivantes.** On cherche ensuite le vecteur unitaire de variance maximale **parmi ceux orthogonaux au premier**. Le même raisonnement, avec une contrainte d'orthogonalité supplémentaire, conduit au deuxième plus grand  $\lambda_2$  et à son vecteur propre. Et ainsi de suite : les directions de l'ACP sont les vecteurs propres de  $S$  rangés par valeur propre décroissante.

Trois conséquences à connaître, toutes immédiates :

1. **Les scores sont non corrélés.** Si  $t_j = X_c \mathbf{v}_j$ , alors  $\text{Cov}(t_i, t_j) = \mathbf{v}_i^\top S \mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j = 0$  pour  $i \neq j$ , car les vecteurs propres d'une matrice symétrique sont orthogonaux (volume I, section 1.1.3).
2. **La variance du score  $j$  vaut  $\lambda_j$  :**  $\text{Var}(t_j) = \mathbf{v}_j^\top S \mathbf{v}_j = \lambda_j$ .
3. **La variance totale est conservée :**  $\sum_j \lambda_j = \text{tr}(S) = \sum_j s_{jj}$ . La part de variance expliquée par la composante  $j$  est donc  $\lambda_j / \text{tr}(S)$ .

**Lecture géométrique.** L'ACP est une **rotation** du nuage de points : on choisit de nouveaux axes, orthogonaux, le premier dans la direction de plus grande dispersion, le deuxième dans la plus grande dispersion restante, etc. Aucune information n'est perdue tant que l'on garde **tous** les axes ; c'est quand on **en jette** (les derniers) que l'on compresse.

### 3.1.4 L'ACP par la décomposition en valeurs singulières

En pratique, on ne forme pas la matrice de covariance : on décompose directement le tableau centré par la **SVD** (volume I, section 1.1.4),  $X_c = U\Sigma V^\top$ . Alors  $X_c^\top X_c = V\Sigma^2 V^\top$ , donc les colonnes de  $V$  sont les vecteurs propres de  $S$ , et

$$\lambda_j = \frac{\sigma_j^2}{n-1}, \quad \text{scores} = X_c V = U\Sigma.$$

C'est numériquement plus stable (nous avons vu au volume I, section 1.5.4, que former  $X^\top X$  élève au carré le conditionnement). Vérifions sur les huit questions que les deux routes donnent les mêmes valeurs propres :

```
Qc = (Q - Q.mean()).to_numpy()
n = len(Qc)

# route 1 : valeurs propres de la matrice de covariance
S_q = np.cov(Qc, rowvar=False)
lam_cov = np.sort(np.linalg.eigvalsh(S_q))[:-1]

# route 2 : SVD du tableau centré
U, sing, Vt = np.linalg.svd(Qc, full_matrices=False)
lam_svd = sing**2 / (n - 1)
```

```
print("valeurs propres (covariance) :", lam_cov.round(4))
print("valeurs propres (SVD)       :", lam_svd.round(4))
print("écart maximal               :", np.abs(lam_cov - lam_svd).max())
print("scores via U*Sigma = Xc*V   :", np.allclose(U * sing, Qc @ Vt.T))
```

```
valeurs propres (covariance) : [2.4143 1.6167 0.5921 0.5059 0.4802 0.4677 0.4032 0.3815]
valeurs propres (SVD)       : [2.4143 1.6167 0.5921 0.5059 0.4802 0.4677 0.4032 0.3815]
écart maximal               : 4.440892098500626e-15
scores via U*Sigma = Xc*V   : True
```

Les deux routes coïncident à la précision machine. Comparons enfin avec la bibliothèque, qui utilise la SVD en interne :

```
from sklearn.decomposition import PCA
```

```
pca = PCA().fit(Q)
print("part de variance (sklearn) :", pca.explained_variance_ratio_.round(3))
print("même chose, à la main      :", (lam_svd / lam_svd.sum()).round(3))
```

```
part de variance (sklearn) : [0.352 0.236 0.086 0.074 0.07 0.068 0.059 0.056]
même chose, à la main      : [0.352 0.236 0.086 0.074 0.07 0.068 0.059 0.056]
```

### 3.1.5 Faut-il standardiser ? Le piège des unités

L'ACP maximise la variance. Or la variance dépend des **unités** : exprimer un panier en centimes de dinar plutôt qu'en dinars multiplie sa variance par dix mille, et l'ACP ne verrait plus que lui. Voyons-le sur la table des clientes, avec cinq variables d'échelles très différentes.

```
c = pd.read_csv("donnees/clients.csv")
cols = ["age", "nb_commandes_an", "panier_moyen", "depense_annuelle", "duree_mois"]
X = c[cols]
print("variances :")
print(X.var().round(1).to_string())
```

```
variances :
age                110.5
nb_commandes_an    13.3
panier_moyen       1029.6
depense_annuelle   88322.3
duree_mois         296.0
```

La dépense annuelle (en DT, jusqu'à plus de 2 000) a une variance plusieurs milliers de fois supérieure à celle du nombre de commandes (88 322 contre 13,3). Sans précaution, l'ACP va simplement redécouvrir... la dépense.

```
def acp_covariance(M):
    """Valeurs propres (décroissantes) et vecteurs propres de la matrice de covariance."""
    w, V = np.linalg.eigh(np.cov(M, rowvar=False))
    o = np.argsort(w)[::-1]
    w, V = w[o], V[:, o]
    # le signe d'un vecteur propre est arbitraire : par convention, on rend positive sa plus grande
    # composante (en valeur absolue)
    V = V * np.sign(V[np.abs(V).argmax(axis=0), np.arange(V.shape[1])])
    return w, V

# 1) sans standardiser : ACP sur la matrice de covariance
w_brut, V_brut = acp_covariance(X.to_numpy())
# 2) en standardisant : ACP sur la matrice de corrélation
Z = ((X - X.mean()) / X.std()).to_numpy()
w_std, V_std = acp_covariance(Z)
```

```
print("sans standardiser : part de la 1re composante =", round(w_brut[0] / w_brut.sum(), 3))
print("en standardisant : parts de variance =", (w_std / w_std.sum()).round(3))
directions = pd.DataFrame({"1re direction, sans standardiser": V_brut[:, 0], "1re direction,
↪ standardisé": V_std[:, 0]}, index=cols)
print(directions.round(2).to_string())
```

```
sans standardiser : part de la 1re composante = 0.988
en standardisant : parts de variance = [0.442 0.214 0.194 0.125 0.025]
      1re direction, sans standardiser  1re direction, standardisé
age                                     0.00                        0.10
nb_commandes_an                       0.01                        0.58
panier_moyen                          0.06                        0.49
depense_annuelle                      1.00                        0.64
duree_mois                            0.00                        0.07
```

Sans standardisation, la première composante absorbe près de 99 % de la variance et pointe presque entièrement sur `depense_annuelle` (saturation de 1,0, contre 0,06 pour le panier) : c'est un artefact de l'unité, pas une découverte. Après standardisation (centrer **et réduire** chaque variable à un écart-type de 1), chaque variable pèse autant au départ, et la première composante n'absorbe plus que 44 % de la variance et mélange nombre de commandes, panier et dépense (0,58, 0,49 et 0,64), tandis que l'âge et la durée de la relation y pèsent très peu : c'est l'**activité d'achat**, une information qui a un vrai sens.

**La règle pratique.** Standardisez dès que vos variables ont des **unités ou des échelles différentes**. Faire l'ACP sur les données standardisées revient à diagonaliser la **matrice de corrélation** (c'est la convention de la plupart des logiciels : on dit « ACP normée »). Ne pas standardiser n'a de sens que si toutes les variables sont dans la même unité et que les différences d'échelle sont **informatives** (par exemple, des notes toutes sur 5).

**Un détail à retenir.** Pour une ACP normée de  $p$  variables, la variance totale est  $p$  (chaque variable standardisée a une variance de 1). La **règle de Kaiser** dit alors : une composante est « intéressante » si sa valeur propre dépasse 1, c'est-à-dire si elle explique plus de variance qu'une seule variable d'origine. Nous verrons au 3.1.6 pourquoi cette règle est pratique mais grossière.

### 3.1.6 Lire une ACP : éboulis, saturations, scores

Revenons au questionnaire (les notes sont toutes sur 5, mais nous standardiserons quand même pour travailler sur la matrice de corrélation). Trois outils pour lire une ACP.

#### 1. L'éboulis des valeurs propres : combien de composantes garder ?

```
Zq = ((Q - Q.mean()) / Q.std()).to_numpy()
w, V = acp_covariance(Zq)
part = w / w.sum()
tab = pd.DataFrame({"valeur propre": w, "part de variance": part, "part cumulée": part.cumsum()},
                    index=[f"CP{j}" for j in range(1, 9)])
print(tab.round(3).to_string())
```

|     | valeur propre | part de variance | part cumulée |
|-----|---------------|------------------|--------------|
| CP1 | 2.797         | 0.350            | 0.350        |
| CP2 | 1.894         | 0.237            | 0.586        |
| CP3 | 0.712         | 0.089            | 0.675        |
| CP4 | 0.578         | 0.072            | 0.748        |
| CP5 | 0.562         | 0.070            | 0.818        |
| CP6 | 0.550         | 0.069            | 0.887        |

|     |       |       |       |
|-----|-------|-------|-------|
| CP7 | 0.475 | 0.059 | 0.946 |
| CP8 | 0.433 | 0.054 | 1.000 |

Trois critères pour choisir le nombre de composantes à garder :

- **L'éboulis (coude)** : on cherche le point où la courbe des valeurs propres cesse de chuter brutalement pour devenir un plateau. Ici, il y a un « coude » net après la deuxième composante.
- **La règle de Kaiser** : on garde les valeurs propres supérieures à 1. Elle en retient deux.
- **L'analyse parallèle**, plus rigoureuse : on compare les valeurs propres observées à celles que donneraient des données **sans structure** (huit variables indépendantes, même effectif). On ne garde que les composantes qui font mieux que le hasard.

```
rng = np.random.default_rng(3)
n, p = Zq.shape
simulees = np.empty((500, p))
for b in range(500):
    bruit = rng.normal(size=(n, p))
    simulees[b] = np.sort(np.linalg.eigvalsh(np.corrcoef(bruit, rowvar=False)))[:-1]
seuil95 = np.percentile(simulees, 95, axis=0)

comparaison = pd.DataFrame({"observée": w, "seuil du hasard (95 %)": seuil95,
                           index=[f"CP{j}" for j in range(1, 9)]})
comparaison["on garde ?"] = comparaison["observée"] > comparaison["seuil du hasard (95 %)"]
print(comparaison.round(3).to_string())
```

|     | observée | seuil du hasard (95 %) | on garde ? |
|-----|----------|------------------------|------------|
| CP1 | 2.797    | 1.160                  | True       |
| CP2 | 1.894    | 1.107                  | True       |
| CP3 | 0.712    | 1.067                  | False      |
| CP4 | 0.578    | 1.033                  | False      |
| CP5 | 0.562    | 1.003                  | False      |
| CP6 | 0.550    | 0.976                  | False      |
| CP7 | 0.475    | 0.947                  | False      |
| CP8 | 0.433    | 0.915                  | False      |

**Pourquoi des valeurs propres de données aléatoires dépassent 1.** Même avec des variables parfaitement indépendantes, les corrélations empiriques ne sont pas exactement nulles ( $n$  est fini). Les valeurs propres de la matrice de corrélation sont alors étalées autour de 1 : la première est supérieure à 1, la dernière inférieure. L'analyse parallèle mesure **de combien** : c'est un seuil calibré sur le bruit, plus honnête que le « 1 » de Kaiser.

## 2. Les saturations (loadings) : que veut dire chaque axe ?

Les colonnes de  $V$  sont les **directions** des composantes. Pour interpréter, on préfère souvent les **corrélations entre les variables d'origine et les composantes**. Pour des variables standardisées,  $\text{cor}(x_i, t_j) = v_{ij}\sqrt{\lambda_j}$  (démonstration :  $\text{Cov}(x_i, t_j) = (Sv_j)_i = \lambda_j v_{ij}$ , puis on divise par les écarts-types 1 et  $\sqrt{\lambda_j}$ ).

```
charges = V[:, :2] * np.sqrt(w[:2])          # corrélation variable-composante
scores_q = Zq @ V                          # coordonnées des répondantes

# contrôle numérique : la corrélation empirique donne la même chose
verif = np.array([np.corrcoef(Zq[:, i], scores_q[:, j])[0, 1] for j in range(2)] for i in range(8)])
print("formule = corrélation empirique :", np.allclose(np.abs(charges), np.abs(verif)))

lect = pd.DataFrame(charges, index=items, columns=["CP1", "CP2"]).round(2)
lect["cos² (qualité sur CP1-CP2)"] = (lect["CP1"]**2 + lect["CP2"]**2).round(2)
print(lect.to_string())
```



| formule = corrélation empirique : True |      |       |                                        |
|----------------------------------------|------|-------|----------------------------------------|
|                                        | CP1  | CP2   | cos <sup>2</sup> (qualité sur CP1-CP2) |
| q1                                     | 0.56 | 0.58  | 0.65                                   |
| q2                                     | 0.52 | 0.54  | 0.56                                   |
| q3                                     | 0.57 | 0.51  | 0.58                                   |
| q4                                     | 0.48 | 0.47  | 0.45                                   |
| q5                                     | 0.68 | -0.45 | 0.66                                   |
| q6                                     | 0.64 | -0.39 | 0.56                                   |
| q7                                     | 0.64 | -0.48 | 0.64                                   |
| q8                                     | 0.61 | -0.44 | 0.57                                   |

La première composante a des corrélations **de même signe pour les huit questions** : c'est une dimension de **satisfaction générale** (les répondantes qui notent bien une chose notent bien le reste). La deuxième **oppose** les questions sur les produits (q1 à q4) à celles sur le service (q5 à q8) : c'est la dimension « produit plutôt que service, ou l'inverse ». Le signe global d'un axe est arbitraire (nous avons adopté la convention « la plus grande saturation est positive », ce qui rend CP1 positive pour toutes les questions), mais les **contrastes** entre variables, eux, ont un sens. Le cos<sup>2</sup> indique quelle fraction de la variance de la question est restituée par le plan des deux premières composantes.

**Ne confondez pas direction et corrélation.** Les éléments de  $V$  sont des coefficients de combinaison linéaire (la composante est  $\sum_i v_{ij} x_i$ ) ; les corrélations  $v_{ij} \sqrt{\lambda_j}$  sont les quantités que l'on compare entre variables. Les deux sont proportionnelles pour une composante donnée, mais seules les corrélations sont comparables d'une composante à l'autre.

### 3. Le biplot : variables et individus sur la même carte.

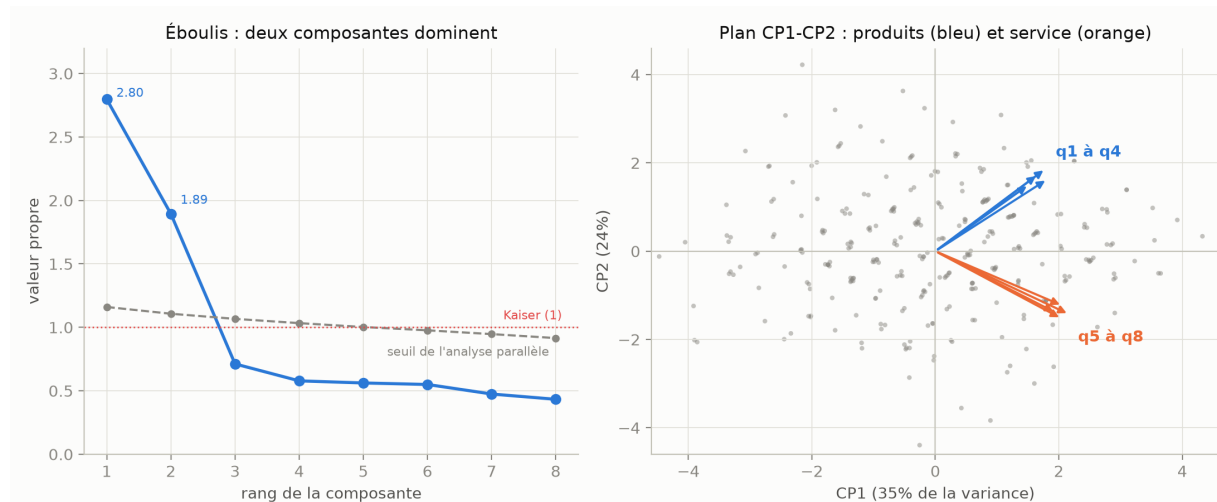


Figure 26: À gauche : éboulis des valeurs propres du questionnaire, avec le seuil de Kaiser et le seuil de l'analyse parallèle. À droite : biplot du plan CP1-CP2 : un point par répondante (échantillon de 300), une flèche par question.

Le biplot superpose les **scores** des répondantes (les points) et les **corrélations** des variables (les flèches). Deux flèches proches signalent des questions corrélées ; une flèche longue est bien représentée par le plan ; une répondante située du côté d'une flèche a une note élevée à cette question. Remarquez que les flèches se répartissent en **deux faisceaux** : produits d'un côté du deuxième axe, service de l'autre. L'ACP a retrouvé les deux blocs que l'œil voyait dans la matrice de corrélation.

### 3.1.7 Compression et reconstruction

Garder  $k$  composantes revient à **approcher** le tableau centré par sa projection :  $\hat{X}_c = T_k V_k^\top = X_c V_k V_k^\top$  (matrice de rang  $k$ ). Le théorème d'**Eckart-Young** dit que c'est la **meilleure** approximation de rang  $k$  au sens des moindres carrés, et que l'erreur est exactement la somme des carrés des valeurs singulières jetées. En termes de valeurs propres de la matrice de corrélation, pour des données standardisées :

$$\sum_{i,l} (Z_{il} - \hat{Z}_{il})^2 = (n-1) \sum_{j>k} \lambda_j.$$

Vérifions-le numériquement sur le questionnaire :

```
n = len(Zq)
for k in range(1, 9):
    Vk = V[:, :k]
    Z_hat = Zq @ Vk @ Vk.T                                # reconstruction avec k composantes
    erreur = ((Zq - Z_hat) ** 2).sum()
    theorique = (n - 1) * w[k:].sum()
    print(f"k = {k} : erreur = {erreur:10.2f}    (n-1) x somme des valeurs propres jetées =
    ↪ {theorique:10.2f}")
```

|                  |         |                                            |         |
|------------------|---------|--------------------------------------------|---------|
| k = 1 : erreur = | 6300.88 | (n-1) x somme des valeurs propres jetées = | 6300.88 |
| k = 2 : erreur = | 4007.48 | (n-1) x somme des valeurs propres jetées = | 4007.48 |
| k = 3 : erreur = | 3145.48 | (n-1) x somme des valeurs propres jetées = | 3145.48 |
| k = 4 : erreur = | 2445.15 | (n-1) x somme des valeurs propres jetées = | 2445.15 |
| k = 5 : erreur = | 1764.95 | (n-1) x somme des valeurs propres jetées = | 1764.95 |
| k = 6 : erreur = | 1099.10 | (n-1) x somme des valeurs propres jetées = | 1099.10 |
| k = 7 : erreur = | 524.38  | (n-1) x somme des valeurs propres jetées = | 524.38  |
| k = 8 : erreur = | 0.00    | (n-1) x somme des valeurs propres jetées = | 0.00    |

L'erreur de reconstruction décroît à mesure qu'on garde de composantes, jusqu'à devenir nulle pour  $k = 8$  (on garde tout). Elle est **exactement** égale à ce qu'annonce le théorème : la valeur propre d'une composante mesure donc très concrètement **ce que l'on perd** en l'abandonnant.

**Application : un indice de satisfaction pour chaque répondante.** Yasmine voulait « un ou deux chiffres par cliente ». Les deux premiers scores sont ces chiffres. Les relie-t-on aux comportements d'achat ? Joignons-les au fichier des clientes et comparons les scores moyens selon qu'une cliente a racheté ou non dans les 12 mois.

```
scores_df = pd.DataFrame(scores_q[:, :2], columns=["CP1", "CP2"])
scores_df["id_client"] = q["id_client"].to_numpy()
fusion = scores_df.merge(c[["id_client", "rachat_12m", "duree_mois", "panier_moyen"]],
    ↪ on="id_client")
print(fusion.groupby("rachat_12m")[["CP1", "CP2"]].mean().round(2).to_string())
print()
print(fusion[["CP1", "CP2", "duree_mois", "panier_moyen"]].corr().round(2).loc[["CP1",
    ↪ "CP2"]].to_string())
```

|            | CP1   | CP2   |
|------------|-------|-------|
| rachat_12m |       |       |
| 0          | -0.47 | -0.11 |
| 1          | 0.46  | 0.10  |

|     | CP1  | CP2  | duree_mois | panier_moyen |
|-----|------|------|------------|--------------|
| CP1 | 1.0  | -0.0 | 0.14       | 0.21         |
| CP2 | -0.0 | 1.0  | -0.09      | 0.13         |

La satisfaction générale (CP1) est nettement plus élevée chez celles qui rachètent, ce qui est co-

hérent. Grâce à la convention de signe adoptée plus haut, un score élevé sur CP1 veut dire une cliente plus satisfaite (c'est pourquoi on lit toujours les **corrélations** de l'axe avant d'interpréter ses scores). Le deuxième axe, lui, sépare à peine les deux groupes (-0,11 contre 0,10, pour un écart-type de l'ordre de 1,4) : le contraste produit/service n'est pas, à lui seul, lié au rachat. Mais regardez les corrélations : CP2 est faiblement **positive** avec le panier (0,13) et **négative** avec la durée de la relation (-0,09). Les deux dimensions semblent donc avoir des rôles distincts : nous y reviendrons en 3.2, quand nous saurons séparer proprement les deux facteurs.

### 3.1.8 Les pièges de l'ACP

1. **Le signe et l'ordre ne sont pas la vérité.** Un axe peut être retourné sans changer le modèle, et deux valeurs propres voisines donnent des axes **instables** (un peu de bruit les mélange). Si  $\lambda_2 \approx \lambda_3$ , n'interprétez pas séparément CP2 et CP3.
2. **Linéaire seulement.** L'ACP ne voit que les relations **linéaires**. Un nuage en forme de croissant (corrélation nulle, structure évidente) lui échappe.
3. **Sensible aux valeurs extrêmes.** Quelques points très éloignés peuvent à eux seuls orienter un axe, puisque la variance est un critère quadratique.
4. **Interprétable ne veut pas dire vrai.** Une composante est une combinaison **mathématiquement optimale**, pas forcément une **cause** réelle. Si vous voulez dire « il existe un facteur caché de qualité de service », il faut un autre outil, qui postule explicitement des facteurs : l'analyse factorielle (section 3.2).
5. **Pas d'inférence.** L'ACP est une technique **descriptive** : elle n'a ni p-valeur ni intervalle de confiance (sauf à utiliser le bootstrap, volume I, section 3.3.5). Elle décrit **ce tableau**, pas la population.
6. **Des notes ordinales.** Nos questions sont notées de 1 à 5, ce qui n'est pas une variable continue au sens strict (volume I, section 3.1.1). L'ACP sur ces notes fonctionne bien en pratique (on traite la note comme un nombre), mais pour des échelles très courtes ou très déséquilibrées, on préfère des corrélations adaptées (polychoriques).

#### À retenir

- L'ACP cherche des **axes orthogonaux de variance maximale** : ce sont les vecteurs propres de la matrice de covariance, rangés par valeur propre décroissante ; la valeur propre est la **variance** du score correspondant.
- On la calcule par la **SVD** du tableau centré ; scores =  $X_c V = U \Sigma$  et  $\lambda_j = \sigma_j^2 / (n - 1)$ .
- **Standardisez** quand les unités diffèrent : sinon l'ACP ne retrouve que la variable de plus grande variance.
- Pour choisir le nombre de composantes : éboulis, règle de Kaiser, mieux, **analyse parallèle**.
- On interprète les **corrélations variable-composante** ; les scores sont les coordonnées des individus ; le **biplot** réunit les deux.
- Garder  $k$  composantes est la **meilleure approximation de rang  $k$**  ; l'erreur vaut la somme des valeurs propres jetées.
- L'ACP **résume**, elle ne **postule pas de causes** : c'est le rôle de l'analyse factorielle.

## 3.2 Analyse factorielle

**Intuition.** Quand vous prenez la température d'un malade avec trois thermomètres, les trois valeurs diffèrent un peu, mais elles « parlent » toutes de la **même chose** : la vraie température, que vous ne voyez pas directement. L'**analyse factorielle** repose sur la même idée : derrière des variables observées corrélées, il y a un petit nombre de **facteurs cachés** (la qualité du service, le goût pour les produits...). Chaque variable observée est un **thermomètre imparfait** d'un facteur : elle contient une part du facteur et une part qui lui est propre (le bruit de mesure, l'effet de la formulation de la question). L'ACP **résume** des variables ; l'analyse factorielle **modélise** la façon dont elles ont été engendrées.

### 3.2.1 Un exemple à la main : trois thermomètres, un facteur

Trois questions sur le service, standardisées (moyenne 0, variance 1). Leurs corrélations sont  $r_{12} = 0,72$ ,  $r_{13} = 0,63$  et  $r_{23} = 0,56$ . Supposons qu'un seul facteur  $f$  (de variance 1) explique tout :

$$x_i = \ell_i f + \varepsilon_i, \quad \text{Var}(\varepsilon_i) = \psi_i, \quad \varepsilon_i \text{ indépendants entre eux et de } f.$$

Les  $\ell_i$  s'appellent les **saturations** (*loadings*) et  $\psi_i$  l'**unicité** de la variable  $i$ . Comme  $x_i$  est standardisée,  $1 = \text{Var}(x_i) = \ell_i^2 + \psi_i$ . Et pour deux variables différentes, comme les bruits sont indépendants :

$$r_{ij} = \text{Cov}(x_i, x_j) = \ell_i \ell_j.$$

Nous avons donc trois équations pour trois inconnues :  $\ell_1 \ell_2 = 0,72$ ,  $\ell_1 \ell_3 = 0,63$ ,  $\ell_2 \ell_3 = 0,56$ . En multipliant les deux premières et en divisant par la troisième :

$$\ell_1^2 = \frac{r_{12} r_{13}}{r_{23}} = \frac{0,72 \times 0,63}{0,56} = 0,81, \quad \text{donc } \ell_1 = 0,9, \quad \ell_2 = \frac{0,72}{0,9} = 0,8, \quad \ell_3 = \frac{0,63}{0,9} = 0,7.$$

La **communalité**  $h_i^2 = \ell_i^2$  est la part de la variance de la variable expliquée par le facteur commun : 0,81, 0,64, 0,49. Les **unicités** sont les compléments à 1 :  $\psi = (0,19; 0,36; 0,51)$ . La première question est donc un excellent thermomètre du facteur (81 % de sa variance en vient), la troisième un thermomètre médiocre (49 %).

```
import numpy as np
import pandas as pd

r12, r13, r23 = 0.72, 0.63, 0.56
l1 = np.sqrt(r12 * r13 / r23)
l = np.array([l1, r12 / l1, r13 / l1])
print("saturations :", l.round(2))
print("communalités :", (l**2).round(2))
print("unicités      :", (1 - l**2).round(2))
# contrôle : on reconstruit les corrélations
print("r12, r13, r23 reconstruits :", (l[0]*l[1]).round(2), (l[0]*l[2]).round(2),
      ↪ (l[1]*l[2]).round(2))
```

```
saturations : [0.9 0.8 0.7]
communalités : [0.81 0.64 0.49]
unicités      : [0.19 0.36 0.51]
r12, r13, r23 reconstruits : 0.72 0.63 0.56
```

**Pourquoi trois variables au minimum ?** Avec deux variables, on a une équation ( $\ell_1 \ell_2 = r_{12}$ ) pour deux inconnues : une infinité de solutions. Avec trois, le modèle à un facteur est exactement identifié. Avec quatre ou plus, il devient **testable** : il impose des contraintes sur les corrélations (comme  $r_{12}r_{34} = r_{13}r_{24}$ ), qui peuvent être vraies ou fausses. C'est ce qui distingue l'analyse factorielle de l'ACP : c'est un **modèle**, que les données peuvent contredire.

### 3.2.2 Le modèle général

Avec  $p$  variables et  $k$  facteurs, on écrit, pour un vecteur d'observations centré  $\mathbf{x} \in \mathbb{R}^p$  :

$$\mathbf{x} = \Lambda \mathbf{f} + \varepsilon, \quad \mathbb{E}[\mathbf{f}] = 0, \text{Var}(\mathbf{f}) = I_k, \quad \mathbb{E}[\varepsilon] = 0, \text{Var}(\varepsilon) = \Psi \text{ (diagonale)}, \quad \mathbf{f} \perp \varepsilon.$$

$\Lambda$  est la matrice  $p \times k$  des saturations. La matrice de covariance de  $\mathbf{x}$  s'en déduit immédiatement :

$$\Sigma = \Lambda \Lambda^\top + \Psi.$$

Tout est là : le modèle dit que les **covariances** (les termes hors diagonale de  $\Sigma$ ) viennent **uniquement** des facteurs communs ; les termes diagonaux sont la somme d'une part commune  $\sum_j \ell_{ij}^2 = h_i^2$  et d'une part propre  $\psi_i$ .

**L'indétermination des rotations.** Soit  $T$  une matrice orthogonale  $k \times k$  ( $TT^\top = I$ ). Remplaçons  $\Lambda$  par  $\Lambda^* = \Lambda T$ . Alors  $\Lambda^* \Lambda^{*\top} = \Lambda T T^\top \Lambda^\top = \Lambda \Lambda^\top$  : **la même matrice**  $\Sigma$ , donc exactement le même ajustement aux données. Les saturations ne sont donc définies qu'à **une rotation près** ; les communalités  $h_i^2 = \sum_j \ell_{ij}^2$ , elles, ne changent pas (la norme de chaque ligne est conservée par une rotation). Cette liberté n'est pas un défaut mais une chance : parmi toutes les solutions équivalentes, on choisira celle qui est **la plus facile à interpréter** (section 3.2.5).

Vérifions-le numériquement sur une rotation quelconque :

```
rng = np.random.default_rng(0)
Lam = np.array([[0.8, 0.1], [0.7, 0.2], [0.1, 0.9], [0.2, 0.7]])      # saturations d'un modèle à 2
                                ↳ facteurs
Psi = np.diag(1 - (Lam**2).sum(axis=1))
Sigma = Lam @ Lam.T + Psi

angle = 0.7                                                          # rotation de 0,7 radian
T = np.array([[np.cos(angle), -np.sin(angle)], [np.sin(angle), np.cos(angle)]])
Lam_rot = Lam @ T
Sigma_rot = Lam_rot @ Lam_rot.T + Psi
print("même matrice Sigma :", np.allclose(Sigma, Sigma_rot))
print("communalités avant :", (Lam**2).sum(axis=1).round(3))
print("communalités après :", (Lam_rot**2).sum(axis=1).round(3))
print("saturations avant :\n", Lam.round(2))
print("saturations après :\n", Lam_rot.round(2))

même matrice Sigma : True
communalités avant : [0.65 0.53 0.82 0.53]
communalités après : [0.65 0.53 0.82 0.53]
saturations avant :
[[0.8 0.1]
 [0.7 0.2]
 [0.1 0.9]
 [0.2 0.7]]
```

```
saturations après :
[[ 0.68 -0.44]
 [ 0.66 -0.3 ]
 [ 0.66  0.62]
 [ 0.6   0.41]]
```

**Combien de paramètres, combien de contraintes ?** Le modèle a  $pk$  saturations et  $p$  unicités, auxquelles il faut retrancher  $k(k-1)/2$  pour lever l'indétermination de la rotation. La matrice  $\Sigma$  compte  $p(p+1)/2$  paramètres libres. Le modèle est donc plus économe que  $\Sigma$  libre de

$$\text{ddl} = \frac{(p-k)^2 - (p+k)}{2}$$

degrés de liberté. Pour  $p = 8$  questions : 20 degrés de liberté avec  $k = 1$  facteur, 13 avec  $k = 2$ , 7 avec  $k = 3$ . C'est ce nombre qui permet de **tester** l'ajustement (section 3.2.4).

### 3.2.3 Estimer les saturations

Deux grandes familles de méthodes.

**1. Les axes principaux itérés** (méthode de la « matrice de corrélation réduite »). Si le modèle est exact,  $R - \Psi = \Lambda\Lambda^\top$  est une matrice de rang  $k$  : on devrait pouvoir la reconstruire avec ses  $k$  premières valeurs propres. L'algorithme alterne :

1. partir d'une estimation des communalités (par exemple  $h_i^2 = \text{le } R^2 \text{ de la régression de la variable } i \text{ sur toutes les autres}$ ) ;
2. remplacer la diagonale de  $R$  par ces communalités, extraire les  $k$  premières valeurs propres/vecteurs propres :  $\Lambda = V_k \sqrt{D_k}$  ;
3. recalculer les communalités  $h_i^2 = \sum_j \ell_{ij}^2$  ; recommencer jusqu'à stabilisation.

C'est l'ACP appliquée à une matrice dont la diagonale a été corrigée pour ne garder que la variance commune. Écrivons-la :

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
items = [f"q{j}" for j in range(1, 9)]
Q = q[items]
Zq = ((Q - Q.mean()) / Q.std()).to_numpy()
R = np.corrcoef(Zq, rowvar=False)
n, p = Zq.shape

def axes_principaux(R, k, iterations=200, tol=1e-9):
    """Axes principaux itérés : renvoie les saturations (non tournées) et les communalités."""
    h2 = 1 - 1 / np.diag(np.linalg.inv(R))          # départ : R² de chaque variable sur les autres
    for _ in range(iterations):
        Rr = R.copy()
        np.fill_diagonal(Rr, h2)                    # corrélation « réduite »
        w, V = np.linalg.eigh(Rr)
        o = np.argsort(w)[-1:-k]
        L = V[:, o] * np.sqrt(np.maximum(w[o], 0))
        h2_nouveau = (L**2).sum(axis=1)
        if np.abs(h2_nouveau - h2).max() < tol:
            break
        h2 = h2_nouveau
    return L, h2_nouveau

L_paf, h2_paf = axes_principaux(R, k=2)
print("communalités (axes principaux) :", h2_paf.round(3))
```

```
communalités (axes principaux) : [0.57  0.413 0.447 0.277 0.571 0.418 0.522 0.408]
```

**2. Le maximum de vraisemblance.** On suppose  $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \Sigma)$  avec  $\Sigma = \Lambda\Lambda^\top + \Psi$  et on maximise la vraisemblance (volume I, section 3.2.5) par rapport à  $\Lambda$  et  $\Psi$ . Pas de formule fermée : on utilise un algorithme itératif. Avantage décisif : on dispose d'un **test d'ajustement** (section 3.2.4). Nous utilisons l'implémentation de `scikit-learn`, et nous comparons les deux méthodes :

```
from sklearn.decomposition import FactorAnalysis

fa_ml = FactorAnalysis(n_components=2, rotation=None, random_state=0).fit(Zq)
L_ml = fa_ml.components_.T                # p x k
L_ml = L_ml * np.sign(L_ml.sum(axis=0))    # signe arbitraire : on rend positive la somme de
↳ chaque colonne
h2_ml = (L_ml**2).sum(axis=1)

comparaison = pd.DataFrame({"axes principaux": h2_paf, "maximum de vraisemblance": h2_ml},
↳ index=items)
print(comparaison.round(3).to_string())

# les saturations dépendent de la rotation, mais la matrice reproduite Lambda Lambda' n'en dépend pas
ecart = np.abs(L_paf @ L_paf.T - L_ml @ L_ml.T).max()
print("écart maximal entre les deux corrélations reproduites :", ecart.round(3))
```

|    | axes principaux | maximum de vraisemblance |
|----|-----------------|--------------------------|
| q1 | 0.570           | 0.564                    |
| q2 | 0.413           | 0.414                    |
| q3 | 0.447           | 0.448                    |
| q4 | 0.277           | 0.279                    |
| q5 | 0.571           | 0.572                    |
| q6 | 0.418           | 0.413                    |
| q7 | 0.522           | 0.526                    |
| q8 | 0.408           | 0.405                    |

écart maximal entre les deux corrélations reproduites : 0.005

Les deux méthodes donnent des communalités presque identiques (au troisième chiffre près pour la plupart des questions), et reproduisent la matrice de corrélation à moins de 0,01 près. Les communalités sont modestes : entre 0,28 et 0,57. Chaque question est un thermomètre bruité de son facteur, ce qui est typique d'un questionnaire où l'on n'a posé que quatre questions par dimension. La question q4 est la plus « bruyante » (0,28) : 72 % de sa variance lui est propre.

### 3.2.4 Combien de facteurs ?

Même question qu'en ACP, avec plus de rigueur disponible, puisque l'on a un modèle.

- **Éboulis, Kaiser, analyse parallèle** : les trois outils de la section 3.1.6 s'appliquent aux valeurs propres de  $R$ . Ils conduisaient à **deux** composantes.
- **Le test du rapport de vraisemblance.** On teste  $H_0$  : «  $k$  facteurs suffisent » contre «  $\Sigma$  est une matrice quelconque ». Avec la correction de Bartlett, la statistique est

$$\chi^2 = \left( n - 1 - \frac{2p+5}{6} - \frac{2k}{3} \right) \left[ \ln |\hat{\Sigma}| - \ln |R| + \text{tr}(\hat{\Sigma}^{-1}R) - p \right], \quad \hat{\Sigma} = \hat{\Lambda}\hat{\Lambda}^\top + \hat{\Psi},$$

et elle suit approximativement une loi du  $\chi^2$  à ddl =  $\frac{(p-k)^2 - (p+k)}{2}$  degrés de liberté. Une p-valeur **petite** signale que  $k$  facteurs sont **insuffisants**.

- **Les critères d'information.** On compare  $\chi^2 - \text{ddl} \times \ln n$  (de type BIC) : le modèle qui le minimise offre le meilleur compromis entre ajustement et complexité.

Calculons tout cela pour  $k = 1, 2, 3$  :

```
from scipy import stats
```



```
def test_ajustement(R, k, n):
    fa = FactorAnalysis(n_components=k, rotation=None, random_state=0).fit(Zq)
    L = fa.components_.T
    Sigma_hat = L @ L.T + np.diag(fa.noise_variance_)
    p = R.shape[0]
    F = np.linalg.slogdet(Sigma_hat)[1] - np.linalg.slogdet(R)[1] +
    np.trace(np.linalg.solve(Sigma_hat, R)) - p
    chi2 = (n - 1 - (2 * p + 5) / 6 - 2 * k / 3) * F
    ddl = ((p - k) ** 2 - (p + k)) / 2
    return chi2, ddl, stats.chi2.sf(chi2, ddl), chi2 - ddl * np.log(n)

lignes = []
for k in (1, 2, 3):
    chi2, ddl, pval, bic = test_ajustement(R, k, n)
    lignes.append({"k": k, "chi2": chi2, "ddl": int(ddl), "p-valeur": pval, "chi2 - ddl*ln(n)": bic})
print(pd.DataFrame(lignes).set_index("k").round(3).to_string())
```

|   | chi2    | ddl | p-valeur | chi2 - ddl*ln(n) |
|---|---------|-----|----------|------------------|
| k |         |     |          |                  |
| 1 | 947.357 | 20  | 0.000    | 805.357          |
| 2 | 18.443  | 13  | 0.141    | -73.857          |
| 3 | 8.749   | 7   | 0.271    | -40.951          |

Lecture du tableau :

- **Avec un seul facteur**,  $\chi^2 \approx 947$  pour 20 degrés de liberté : la p-valeur est nulle à trois décimales. Un seul facteur est **massivement rejeté** : les huit questions ne forment pas une seule dimension.
- **Avec deux facteurs**,  $\chi^2 \approx 18,4$  pour 13 degrés de liberté, soit  $p \approx 0,14$  : on **ne rejette pas** le modèle. Rappelons qu'ici, c'est « ne pas rejeter » qui est la bonne nouvelle : le modèle à deux facteurs est compatible avec les données (volume I, section 3.4.1 : ne pas rejeter n'est pas prouver, mais c'est tout ce que l'on peut espérer).
- **Avec trois facteurs**, le modèle est évidemment encore compatible ( $p \approx 0,27$ ), mais le critère de type BIC remonte (de  $-73,9$  à  $-41,0$ ) : le troisième facteur n'améliore pas assez l'ajustement pour justifier ses paramètres supplémentaires.

Les trois outils (éboulis/analyse parallèle, test d'ajustement, BIC) convergent : **deux facteurs**.

### 3.2.5 La rotation : rendre les facteurs lisibles

Regardons les saturations « brutes » du modèle à deux facteurs :

```
brut = pd.DataFrame(L_ml, index=items, columns=["F1", "F2"])
print(brut.round(2).to_string())
```

|    | F1   | F2    |
|----|------|-------|
| q1 | 0.48 | 0.58  |
| q2 | 0.42 | 0.49  |
| q3 | 0.47 | 0.47  |
| q4 | 0.36 | 0.38  |
| q5 | 0.67 | -0.35 |
| q6 | 0.58 | -0.27 |
| q7 | 0.63 | -0.37 |
| q8 | 0.56 | -0.30 |

Les saturations brutes se ressemblent beaucoup à celles de l'ACP : un premier axe « général » (toutes les questions du même côté) et un second axe qui **oppose** produits et service. Ce n'est pas un hasard : l'algorithme choisit la solution où le premier facteur capte le maximum de variance commune, ce qui produit un facteur général. Mais, comme nous l'avons montré en 3.2.2, **toute rotation de ces axes explique les données aussi bien**. On peut donc faire tourner les axes jusqu'à

ce que chaque question soit proche d'un **seul** facteur : c'est ce qu'on appelle la **structure simple** (Thurstone).

La **rotation varimax** (Kaiser, 1958) cherche la rotation orthogonale qui **maximise la variance des carrés des saturations** dans chaque colonne. Intuition : la variance des carrés est grande quand certaines saturations sont proches de 1 et les autres proches de 0, c'est-à-dire quand les saturations sont « tranchées ». Voici l'algorithme, qui s'écrit en dix lignes grâce à la SVD :

```
def varimax(L, gamma=1.0, iterations=100, tol=1e-9):
    """Rotation varimax orthogonale (algorithme de Kaiser, version SVD). Renvoie L tournée et la
    ↪ rotation."""
    p, k = L.shape
    T = np.eye(k)
    critere = 0.0
    for _ in range(iterations):
        Lr = L @ T
        u, s, vt = np.linalg.svd(L.T @ (Lr**3 - (gamma / p) * Lr @ np.diag((Lr**2).sum(axis=0))))
        T = u @ vt
        if s.sum() < critere * (1 + tol):
            break
        critere = s.sum()
    return L @ T, T

L_vm, T_vm = varimax(L_ml)

def aligner(M):
    """Colonne 1 = le bloc de questions q1-q4 ; convention de signe : la plus grande saturation de
    ↪ chaque colonne est positive."""
    M = M[:, np.argsort(-np.abs(M[:,4]).mean(axis=0))]
    return M * np.sign(M[np.abs(M).argmax(axis=0), [0, 1]])

tourne = pd.DataFrame(aligner(L_vm), index=items, columns=["F1 (produits)", "F2 (service)"])
print(tourne.round(2).to_string())
print("communalités inchangées :", np.allclose((L_vm**2).sum(axis=1), h2_ml))

# comparaison avec la rotation varimax de scikit-learn
L_sk = FactorAnalysis(n_components=2, rotation="varimax", random_state=0).fit(Zq).components_.T
print("écart maximal avec la solution de scikit-learn :", np.abs(aligner(L_vm) -
    ↪ aligner(L_sk)).max().round(4))
```

|    | F1 (produits) | F2 (service) |
|----|---------------|--------------|
| q1 | 0.75          | 0.07         |
| q2 | 0.64          | 0.07         |
| q3 | 0.66          | 0.12         |
| q4 | 0.52          | 0.08         |
| q5 | 0.09          | 0.75         |
| q6 | 0.11          | 0.63         |
| q7 | 0.05          | 0.72         |
| q8 | 0.07          | 0.63         |

communalités inchangées : True  
écart maximal avec la solution de scikit-learn : 0.001

La figure montre ce que fait la rotation : **les points ne bougent pas** (leurs positions relatives sont identiques), ce sont les **axes** qui tournent. À gauche, les deux groupes de questions sont écartés des axes, ce qui rend leur lecture confuse ; à droite, chaque groupe se range le long d'un axe. Après rotation, les saturations se lisent presque comme un tableau de 0 et de 1 :  $F_1$  pour un groupe de questions,  $F_2$  pour l'autre.

**Rotation orthogonale ou oblique ?** Varimax impose des facteurs **non corrélés** (la rotation est orthogonale). Or, dans la réalité, il est courant que « le goût pour les produits » et « l'appréciation du service » soient un peu corrélés : une cliente globalement contente aura

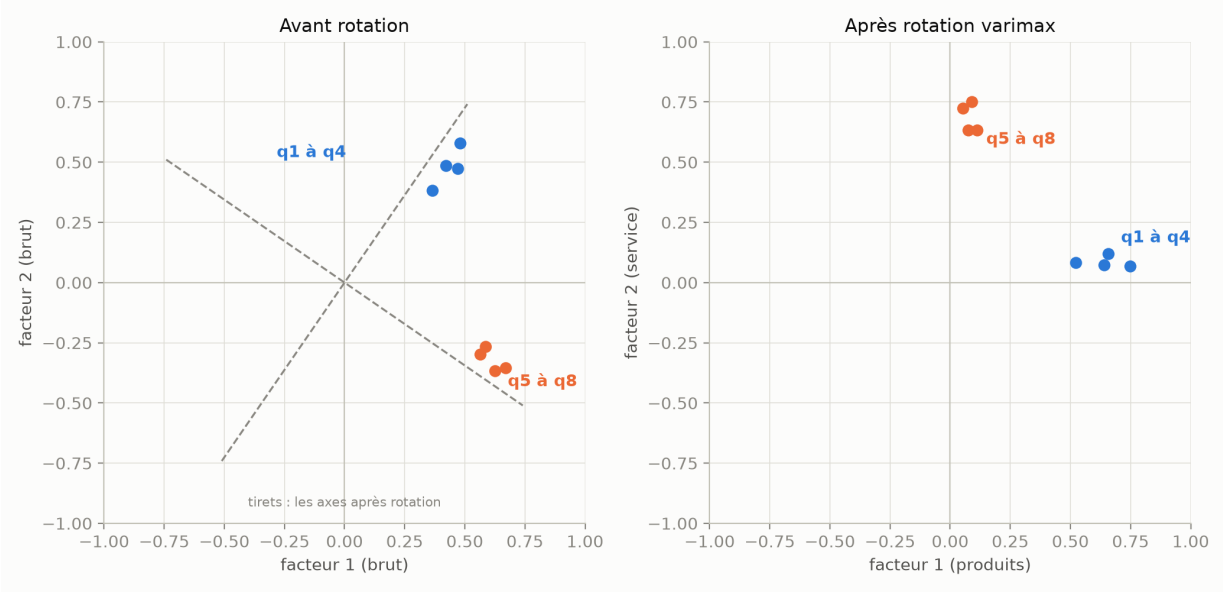


Figure 27: Les huit questions dans le plan des deux facteurs : avant rotation (à gauche) et après rotation varimax (à droite). La rotation fait tourner les axes sans modifier la position relative des points ; après rotation, chaque question est proche d'un seul axe.

tendance à bien noter les deux. Les rotations **obliques** (promax, oblimin) autorisent des facteurs corrélés et produisent souvent une structure plus nette, au prix d'une interprétation un peu plus délicate (deux tableaux : les saturations « de structure » et « de configuration »). Nous reviendrons sur ce point à la section 3.2.7.

3.2.6 ACP contre analyse factorielle : que choisir ?

Les deux méthodes donnent souvent des résultats voisins, mais elles ne répondent pas à la même question.

|                         | ACP                                | Analyse factorielle                                                   |
|-------------------------|------------------------------------|-----------------------------------------------------------------------|
| Objectif                | Résumer les variables              | Expliquer les corrélations par des facteurs                           |
| Modèle                  | Aucun : transformation exacte      | $\Sigma = \Lambda\Lambda^T + \Psi$ (testable)                         |
| Variance expliquée      | Totale (diagonale de R à 1)        | Commune seulement ( $\Psi$ absorbe le reste)                          |
| Les composantes sont... | Des combinaisons des variables     | Des causes supposées des variables                                    |
| Unicité de la solution  | Unique (à un signe près)           | À une rotation près                                                   |
| Test d'ajustement       | Non                                | Oui (maximum de vraisemblance)                                        |
| Quand ?                 | Compresser, visualiser, prétraiter | Construire une échelle de mesure (questionnaires), tester une théorie |

Comparons, sur nos données, la façon dont chacune des deux méthodes « voit » la qualité de la représentation de chaque question :

```

w, V = np.linalg.eigh(R)
o = np.argsort(w)[-1]
w, V = w[o], V[:, o]
Lacp = V[:, :2] * np.sqrt(w[:2])
h2_acp = (Lacp**2).sum(axis=1)

tab = pd.DataFrame({"part de variance (ACP, 2 comp.)": h2_acp, "communalité (AF, 2 facteurs)": h2_ml},
    ↪ index=items)
print(tab.round(2).to_string())
print("moyennes :", tab.mean().round(3).to_dict())

```

|            | part de variance (ACP, 2 comp.)                                                     | communalité (AF, 2 facteurs) |
|------------|-------------------------------------------------------------------------------------|------------------------------|
| q1         | 0.66                                                                                | 0.56                         |
| q2         | 0.57                                                                                | 0.41                         |
| q3         | 0.59                                                                                | 0.45                         |
| q4         | 0.45                                                                                | 0.28                         |
| q5         | 0.66                                                                                | 0.57                         |
| q6         | 0.57                                                                                | 0.41                         |
| q7         | 0.64                                                                                | 0.53                         |
| q8         | 0.56                                                                                | 0.41                         |
| moyennes : | { 'part de variance (ACP, 2 comp.)': 0.586, 'communalité (AF, 2 facteurs)': 0.453 } |                              |

L'ACP attribue toujours aux composantes **plus** de variance que l'analyse factorielle n'en attribue aux facteurs : l'ACP considère que toute la variance d'une question est « à expliquer », alors que l'analyse factorielle admet qu'une partie est du bruit propre à la question (l'unicité). Quand les variables sont très corrélées, l'écart est petit ; quand elles le sont peu, comme ici (les corrélations intra-groupe sont autour de 0,4), il est notable.

### 3.2.7 La révélation : qu'avait-on programmé ?

Les données sont simulées. Voici la vérité : deux facteurs latents (la qualité des produits, la qualité du service) avec une corrélation de **0,30**, et des saturations vraies de (0,80; 0,70; 0,75; 0,60) pour q1 à q4 sur le premier facteur et (0,80; 0,70; 0,75; 0,65) pour q5 à q8 sur le second. Chaque note a ensuite été **arrondie et bornée** entre 1 et 5, comme dans un vrai questionnaire. Comparons à ce que la méthode a estimé :

```

vrai = np.array([0.80, 0.70, 0.75, 0.60, 0.80, 0.70, 0.75, 0.65])
principale = pd.concat([tourne.iloc[:4, 0], tourne.iloc[4:, 1]]) # saturation sur le « bon »
    ↪ facteur
croisee = pd.concat([tourne.iloc[:4, 1], tourne.iloc[4:, 0]]).abs() # saturation sur l'autre
    ↪ facteur
bilan = pd.DataFrame({"vraie": vrai, "estimée (varimax)": principale, "saturation croisée": croisee})
print(bilan.round(2).to_string())
print("erreur absolue moyenne sur les saturations principales :", np.abs(bilan["vraie"] -
    ↪ bilan["estimée (varimax)"]).mean().round(3))

```

|                                                                | vraie | estimée (varimax) | saturation croisée |
|----------------------------------------------------------------|-------|-------------------|--------------------|
| q1                                                             | 0.80  | 0.75              | 0.07               |
| q2                                                             | 0.70  | 0.64              | 0.07               |
| q3                                                             | 0.75  | 0.66              | 0.12               |
| q4                                                             | 0.60  | 0.52              | 0.08               |
| q5                                                             | 0.80  | 0.75              | 0.09               |
| q6                                                             | 0.70  | 0.63              | 0.11               |
| q7                                                             | 0.75  | 0.72              | 0.05               |
| q8                                                             | 0.65  | 0.63              | 0.07               |
| erreur absolue moyenne sur les saturations principales : 0.055 |       |                   |                    |

**Pourquoi l'estimation n'est pas exacte.** Trois raisons. (1) L'échantillon est fini (1 212 répondantes) : toute estimation fluctue. (2) Les notes ont été **arrondies** à des entiers de 1 à 5, ce

qui atténue les corrélations et donc les saturations (l'arrondi est une forme de bruit de mesure). (3) Les facteurs vrais sont **corrélés** (0,30) alors que varimax les impose orthogonaux : il compense en attribuant de petites **saturations croisées** aux questions de l'autre groupe, ce qui explique les valeurs non nulles de la dernière colonne.

### 3.2.8 Application : construire deux échelles de mesure

Le but pratique d'une analyse factorielle de questionnaire est de **construire des échelles** : un score « produits » et un score « service », moyennes des questions de chaque groupe. Il faut alors vérifier que les questions d'un groupe forment bien un ensemble **cohérent**. L'indicateur classique est l'**alpha de Cronbach** :

$$\alpha = \frac{k}{k-1} \left( 1 - \frac{\sum_{i=1}^k \text{Var}(x_i)}{\text{Var}(\sum_{i=1}^k x_i)} \right).$$

Il vaut 0 si les questions ne sont pas corrélées et tend vers 1 si elles le sont toutes parfaitement ; au-dessus de 0,7 on parle de cohérence acceptable (convention usuelle, sans valeur de loi universelle).

```
def alpha_cronbach(D):
    D = np.asarray(D, dtype=float)
    k = D.shape[1]
    return k / (k - 1) * (1 - D.var(axis=0, ddof=1).sum() / D.sum(axis=1).var(ddof=1))

produits = Q[["q1", "q2", "q3", "q4"]]
service = Q[["q5", "q6", "q7", "q8"]]
print("alpha, échelle produits :", round(alpha_cronbach(produits), 3))
print("alpha, échelle service :", round(alpha_cronbach(service), 3))
print("alpha des 8 questions ensemble :", round(alpha_cronbach(Q), 3))

scores = pd.DataFrame({"id_client": q["id_client"], "score_produits": produits.mean(axis=1),
    ↪ "score_service": service.mean(axis=1)})
print("corrélation entre les deux échelles :",
    ↪ round(scores["score_produits"].corr(scores["score_service"]), 3))

alpha, échelle produits : 0.741
alpha, échelle service : 0.784
alpha des 8 questions ensemble : 0.732
corrélation entre les deux échelles : 0.19
```

Les deux échelles sont **cohérentes** ( $\alpha = 0,74$  et  $0,78$ , au-dessus du repère de  $0,7$ ). Leur corrélation, de  $0,19$ , est plus faible que la corrélation **vraie** de  $0,30$  entre les facteurs : c'est l'effet classique d'**atténuation par l'erreur de mesure** (chaque score contient du bruit propre aux questions, qui dilue la corrélation entre les facteurs sous-jacents). Remarquez que l'alpha des huit questions réunies reste honorable ( $0,73$ ), mais qu'il serait trompeur de conclure à « une seule dimension » : un alpha élevé est compatible avec **plusieurs** dimensions corrélées. C'est l'analyse factorielle, pas l'alpha, qui répond à la question « combien de dimensions ? ».

Reste la question de fond : ces deux scores sont-ils liés au comportement des clientes ? Joignons-les au fichier des clientes.

```
c = pd.read_csv("donnees/clients.csv")
f = scores.merge(c[["id_client", "rachat_12m", "panier_moyen", "duree_mois", "nb_commandes_an",
    ↪ "age"]], on="id_client")
f = f[f["panier_moyen"] > 0] # clientes ayant commandé
print(f[["score_produits", "score_service", "panier_moyen", "duree_mois", "nb_commandes_an",
    ↪ "age"]].corr().round(2))
```

```
.loc[["score_produits", "score_service"], ["panier_moyen", "duree_mois", "nb_commandes_an",
↪ "age"]].to_string())
print()
print(f.groupby("rachat_12m")[["score_produits", "score_service"]].mean().round(2).to_string())
```

|                | panier_moyen | duree_mois | nb_commandes_an | age   |
|----------------|--------------|------------|-----------------|-------|
| score_produits | 0.24         | 0.04       | 0.20            | 0.03  |
| score_service  | 0.09         | 0.16       | 0.16            | -0.00 |

|            | score_produits | score_service |
|------------|----------------|---------------|
| rachat_12m |                |               |
| 0          | 3.46           | 3.41          |
| 1          | 3.80           | 3.68          |

Les deux échelles ne sont **pas redondantes**, elles sont associées à des comportements différents. Le score « produits » est le mieux corrélé au **panier moyen** (0,24 contre 0,09 pour le score « service »), tandis que le score « service » est le mieux corrélé à la **durée de la relation** (0,16 contre 0,04). Les deux sont liés au nombre de commandes (0,20 et 0,16), et les clientes qui rachètent sont plus satisfaites sur les deux plans. Cela confirme l'indice entrevu en 3.1.7 : les deux dimensions ont des rôles distincts.

**Révélation.** C'est exactement ce qui avait été programmé : le facteur « produits » influence le panier (et le nombre de commandes), le facteur « service » influence la durée de la relation (et le rachat). Notez que l'analyse factorielle ne **savait rien** de ces liens : elle a simplement séparé deux dimensions, et c'est en les confrontant ensuite à des comportements que leur sens apparaît.

**Ce que l'on peut dire, et ne pas dire.** Ces corrélations sont **modestes** (entre 0,1 et 0,25) et elles ne prouvent aucune causalité : une cliente satisfaite rachète peut-être pour des raisons qui jouent aussi sur ses réponses. Mais Yasmine dispose désormais de deux indicateurs fiables, chacun résumant quatre questions, à suivre dans le temps, et d'une hypothèse de travail claire : le **produit** fait le panier, le **service** fait la fidélité. Pour vérifier la part de chacun une fois les autres facteurs contrôlés, il faudra un modèle de régression (chapitres 1 et 2) ou, pour la durée de la relation, un modèle de survie (chapitre 5).

### À retenir

- Le modèle factoriel écrit  $\Sigma = \Lambda\Lambda^T + \Psi$  : les covariances viennent de **facteurs communs**, le reste est de l'**unicité**. La **communalité** d'une variable est la part de sa variance expliquée par les facteurs.
- Les saturations ne sont définies qu'à **une rotation orthogonale près** : on choisit la rotation la plus lisible (**varimax** = structure simple ; **obliques** si les facteurs sont corrélés).
- On estime par **axes principaux** ou **maximum de vraisemblance** ; ce dernier fournit un **test d'ajustement** et permet de comparer des modèles à  $k$  différents.
- **ACP** = résumer ; **analyse factorielle** = modéliser des causes cachées. Elles diffèrent par la variance expliquée (totale contre commune) et par le statut des axes.
- Pour un questionnaire : construire des **échelles**, mesurer leur cohérence (**alpha de Cronbach**), et ne pas confondre alpha élevé et unidimensionnalité.

### 3.3 Classification non supervisée

**Intuition.** Yasmine a devant elle un millier de clientes et voudrait savoir « qui ressemble à qui ». On ne lui a donné aucune étiquette (« cliente fidèle », « cliente de passage ») : on veut que ce soient **les données elles-mêmes** qui proposent des groupes. C'est la **classification non supervisée** (*clustering*) : regrouper les individus de façon que ceux d'un même groupe se ressemblent plus entre eux qu'avec ceux des autres groupes. Deux familles de méthodes, simples et complémentaires : les **k-means** (on fixe le nombre de groupes et on optimise) et la **classification hiérarchique** (on construit un arbre de fusions successives).

**Avertissement de départ.** Un algorithme de classification **rend toujours des groupes**, même quand il n'y en a aucun. Donnez-lui un nuage uniforme, il le découpera en morceaux avec le même aplomb. Tout ce que nous apprendrons dans cette section — choisir le nombre de groupes, mesurer leur qualité, tester leur stabilité — sert à répondre à la vraie question : *ces groupes existent-ils, ou est-ce moi qui les ai tracés ?*

#### 3.3.1 Ressembler, c'est être proche : la notion de distance

Un groupe, c'est un ensemble de points **proches**. Il faut donc définir « proche ». La distance la plus courante est la distance **euclidienne** : pour deux clientes décrites par  $p$  variables  $\mathbf{x}$  et  $\mathbf{y}$ ,

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2}.$$

Comme en ACP (section 3.1.5), **les unités comptent** : si le panier est en DT et le nombre de commandes en unités, la distance sera dominée par le panier. On **standardise** donc presque toujours les variables avant de classer. Autre limite : la distance euclidienne n'a de sens que pour des variables **numériques**. Pour des variables mixtes (numériques et qualitatives), on utilise des distances adaptées (par exemple celle de Gower) ; pour des variables qualitatives pures, on peut passer par l'analyse des correspondances multiples (section 3.4).

#### 3.3.2 Les k-means

**L'objectif.** On cherche à découper  $n$  points en  $k$  groupes  $C_1, \dots, C_k$  de centres (moyennes)  $\mu_1, \dots, \mu_k$  de façon à minimiser la **variance intra-groupe** (aussi appelée inertie intra-classe) :

$$W(C, \mu) = \sum_{j=1}^k \sum_{i \in C_j} \|\mathbf{x}_i - \mu_j\|^2.$$

Trouver la partition qui minimise  $W$  est un problème difficile (il y a un nombre astronomique de partitions possibles) ; on le résout **approximativement** par l'algorithme de **Lloyd**, qui alterne deux étapes simples :

1. **Affectation** : chaque point rejoint le groupe dont le centre est le plus proche.
2. **Mise à jour** : chaque centre est remplacé par la **moyenne** des points de son groupe.

On répète jusqu'à ce que plus aucun point ne change de groupe.

**Un exemple à la main.** Six clientes décrites par une seule variable (leur panier moyen, en dizaines de DT) : 1, 2, 4, 9, 11, 12. On veut  $k = 2$  groupes et on choisit, au hasard, les centres initiaux  $\mu_1 = 1$  et  $\mu_2 = 4$ .



- *Affectation 1.* Les points 1 et 2 sont plus proches de  $\mu_1 = 1$  ; le point 4 est son propre centre ; 9, 11 et 12 sont plus proches de 4 que de 1. Groupes :  $\{1, 2\}$  et  $\{4, 9, 11, 12\}$ .
- *Mise à jour 1.* Les nouvelles moyennes sont  $\mu_1 = 1,5$  et  $\mu_2 = (4 + 9 + 11 + 12)/4 = 9$ .
- *Affectation 2.* Le point 4 est maintenant à 2,5 de  $\mu_1$  et à 5 de  $\mu_2$  : **il change de groupe**. Groupes :  $\{1, 2, 4\}$  et  $\{9, 11, 12\}$ .
- *Mise à jour 2.* Moyennes :  $\mu_1 = 7/3 \approx 2,33$  et  $\mu_2 = 32/3 \approx 10,67$ .
- *Affectation 3.* Plus personne ne change : l'algorithme s'arrête.

Suivons la variance intra-groupe  $W$  : avec les centres de départ (1; 4) et les groupes  $\{1, 2\}, \{4, 9, 11, 12\}$ ,  $W = (0 + 1) + (0 + 25 + 49 + 64) = 139$  ; après la première mise à jour (centres 1,5 et 9),  $W = 0,5 + 38 = 38,5$  ; après la deuxième affectation,  $W = 19,75$  ; après la deuxième mise à jour,  $W = 28/3 \approx 9,33$ .  $W$  **n'a fait que diminuer** :  $139 \rightarrow 38,5 \rightarrow 19,75 \rightarrow 9,33$ .

Le code suivant écrit l'algorithme et suit  $W$  à chaque étape :

```
import numpy as np
import pandas as pd

x = np.array([1.0, 2.0, 4.0, 9.0, 11.0, 12.0])
centres = np.array([1.0, 4.0])

for tour in range(1, 6):
    groupes = np.abs(x[:, None] - centres[None, :]).argmin(axis=1)      # affectation
    W = sum((x[groupe == j] - centres[j]) ** 2).sum() for j in range(2)) # inertie avec les
    ↪ centres actuels
    nouveaux = np.array([x[groupe == j].mean() for j in range(2)])      # mise à jour
    W_apres = sum((x[groupe == j] - nouveaux[j]) ** 2).sum() for j in range(2))
    print(f"tour {tour} : groupes = {groupes.tolist()}, W avant mise à jour = {W:.2f}, après =
    ↪ {W_apres:.2f}, centres = {nouveaux.round(2).tolist()}")
    if np.allclose(nouveaux, centres):
        print("convergence")
        break
    centres = nouveaux

tour 1 : groupes = [0, 0, 1, 1, 1, 1], W avant mise à jour = 139.00, après = 38.50, centres = [1.5, 9.0]
tour 2 : groupes = [0, 0, 0, 1, 1, 1], W avant mise à jour = 19.75, après = 9.33, centres = [2.33,
↪ 10.67]
tour 3 : groupes = [0, 0, 0, 1, 1, 1], W avant mise à jour = 9.33, après = 9.33, centres = [2.33, 10.67]
convergence
```

#### Pourquoi l'algorithme converge : $W$ ne peut que diminuer.

- **L'étape d'affectation ne peut pas augmenter  $W$**  : à centres fixés, chaque point contribue par son carré de distance à son centre ; en le plaçant dans le groupe du centre le plus proche, on minimise sa contribution.
- **L'étape de mise à jour ne peut pas augmenter  $W$**  : à groupes fixés, la moyenne est le point qui minimise la somme des carrés des distances aux points du groupe. En effet, pour tout point  $\mathbf{m}$ ,

$$\sum_{i \in C} \|\mathbf{x}_i - \mathbf{m}\|^2 = \sum_{i \in C} \|\mathbf{x}_i - \bar{\mathbf{x}}_C\|^2 + |C| \|\bar{\mathbf{x}}_C - \mathbf{m}\|^2,$$

(on développe en insérant  $\bar{\mathbf{x}}_C$  ; le terme croisé s'annule parce que  $\sum_i (\mathbf{x}_i - \bar{\mathbf{x}}_C) = 0$ ), donc le minimum est atteint en  $\mathbf{m} = \bar{\mathbf{x}}_C$ .

$W$  est donc une suite **décroissante et minorée par 0**. Comme il n'y a qu'un nombre **fini** de partitions possibles et que  $W$  décroît strictement tant que la partition change, l'algorithme s'arrête après un nombre fini d'étapes.  $\square$

**Mais attention** : il s'arrête sur un **minimum local**, pas forcément global. Le résultat dépend des centres de départ.

**Les centres de départ : k-means++ et redémarrages.** Deux précautions pour éviter les mauvais minima locaux. D'abord, **plusieurs départs aléatoires** (on garde la solution de plus petit  $W$ ). Ensuite, l'initialisation **k-means++** : le premier centre est tiré au hasard parmi les points, puis chaque centre suivant est tiré avec une probabilité proportionnelle au **carré de la distance** au centre le plus proche déjà choisi. Les centres de départ sont ainsi bien étalés. Voici l'algorithme complet en NumPy :

```
def kmeans(X, k, rng, n_init=10, max_iter=100):
    """k-means (algorithme de Lloyd, initialisation k-means++, n_init redémarrages)."""
    meilleur = None
    for _ in range(n_init):
        # initialisation k-means++
        centres = [X[rng.integers(len(X))]]
        for _ in range(k - 1):
            d2 = ((X[:, None, :] - np.array(centres)[None, :, :]) ** 2).sum(axis=2).min(axis=1)
            centres.append(X[rng.choice(len(X), p=d2 / d2.sum())])
        centres = np.array(centres)
        for iteration in range(max_iter):
            d = ((X[:, None, :] - centres[None, :, :]) ** 2).sum(axis=2)
            groupes = d.argmax(axis=1)
            nouveaux = np.array([X[groupe].mean(axis=0) if (groupe == j).any() else centres[j]
                                for j in range(k)])
            if np.allclose(nouveaux, centres):
                break
            centres = nouveaux
            W = ((X - centres[groupe]) ** 2).sum()
            if meilleur is None or W < meilleur["W"]:
                meilleur = {"W": W, "groupes": groupes, "centres": centres, "iterations": iteration + 1}
    return meilleur
```

**Un jeu de données avec de vrais groupes.** Pour voir l'algorithme réussir, il faut des données qui contiennent des groupes. Simulons (graine fixée) 600 clientes issues de trois profils distincts, décrites par leur nombre de commandes annuel et leur panier moyen :

- les **occasionnelles** (300) : peu de commandes, petit panier ;
- les **fidèles** (200) : beaucoup de commandes, panier moyen ;
- les **cadeaux** (100) : peu de commandes mais très gros panier (offrir un margoum).

```
rng = np.random.default_rng(11)
profils = {"occasionnelles": (300, (1.5, 35), (0.7, 8)),
          "fidèles": (200, (6.0, 50), (1.4, 10)),
          "cadeaux": (100, (3.0, 110), (0.9, 15))}
blocs, vrais = [], []
for numero, (nom, (n_p, moyennes, ecarts)) in enumerate(profils.items()):
    blocs.append(rng.normal(moyennes, ecarts, size=(n_p, 2)))
    vrais += [numero] * n_p
sim = np.vstack(blocs)
sim[:, 0] = np.clip(sim[:, 0], 0, None)
vrais = np.array(vrais)
Zs = (sim - sim.mean(axis=0)) / sim.std(axis=0) # on standardise

resultat = kmeans(Zs, 3, np.random.default_rng(0))
print("inertie intra-groupe W :", round(resultat["W"], 1))
print("itérations de la meilleure exécution :", resultat["iterations"])
print("effectifs des groupes trouvés :", np.bincount(resultat["groupes"]).tolist())

# comparaison avec scikit-learn
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Zs)
print("inertie scikit-learn :", round(km.inertia_, 1))

# accord avec les vrais profils (indice de Rand ajusté : 1 = partitions identiques, 0 = accord du
↳ hasard)
```

```
from sklearn.metrics import adjusted_rand_score
print("indice de Rand ajusté (k-means de ce chapitre vs vrais profils) :",
      round(adjusted_rand_score(vrais, resultat["groupes"]), 3))
```

```
inertie intra-groupe W : 188.1
itérations de la meilleure exécution : 6
effectifs des groupes trouvés : [99, 191, 310]
inertie scikit-learn : 188.1
indice de Rand ajusté (k-means de ce chapitre vs vrais profils) : 0.941
```

Notre implémentation trouve **exactement** la même inertie que celle de `scikit-learn` (188,1). Les effectifs trouvés (99, 191 et 310) sont proches des vrais (100, 200 et 300) et l'accord avec les profils programmés est excellent ( $ARI \approx 0,94$ ). Il n'est pas parfait, et ne le sera jamais : les nuages des « occasionnelles » et des « fidèles » se chevauchent un peu, et les clientes situées à la frontière sont inévitablement rangées au hasard de leur position.

**L'indice de Rand ajusté (ARI).** Pour comparer deux partitions sans avoir à numérotter les groupes de la même façon, on compte les **paires de points** : une paire est « d'accord » si les deux partitions la placent ensemble toutes les deux, ou séparée toutes les deux. L'indice de Rand est la proportion de paires d'accord ; la version **ajustée** retire l'accord attendu du hasard, de sorte que  $ARI = 1$  signifie des partitions identiques et  $ARI \approx 0$  un accord de hasard. Nous nous en servons pour mesurer à la fois l'exactitude (contre la vérité, quand on l'a) et la **stabilité** (entre deux exécutions).

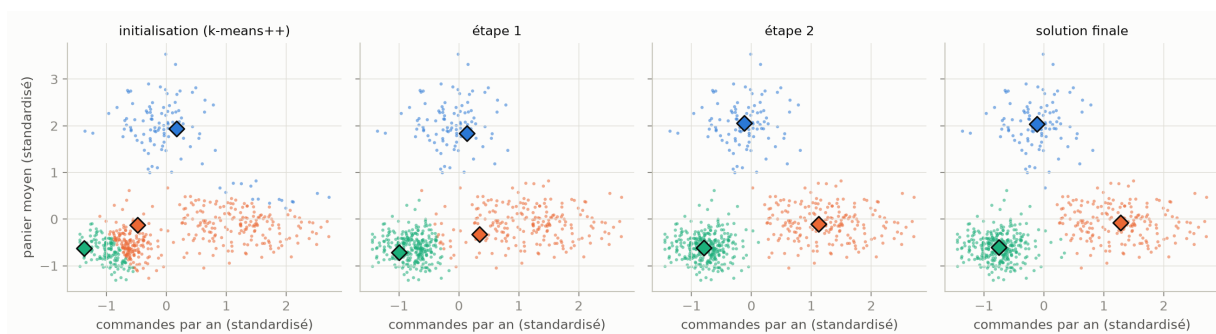


Figure 28: L'algorithme de Lloyd sur les 600 clientes simulées (variables standardisées) : initialisation `k-means++`, premières mises à jour puis solution finale. Les losanges sont les centres ; la couleur d'un point est celle du centre le plus proche.

### 3.3.3 Combien de groupes ?

Il faut choisir  $k$  **avant** de lancer l'algorithme, et la variance intra-groupe  $W$  **ne peut pas servir de critère** : elle décroît toujours quand  $k$  augmente (avec  $k = n$ , chaque point est son propre groupe et  $W = 0$ ). Deux outils.

**1. Le coude.** On trace  $W(k)$  pour  $k = 1, 2, 3, \dots$  et on cherche le point où la décroissance ralentit nettement : ajouter un groupe supplémentaire n'apporte plus grand-chose.

**2. La silhouette.** Pour chaque point  $i$ , on calcule :

- $a(i)$  : la distance moyenne de  $i$  aux **autres points de son propre groupe** (est-il bien « chez lui » ?) ;
- $b(i)$  : la distance moyenne de  $i$  aux points du **groupe voisin le plus proche** (le meilleur « plan B »).

La silhouette du point  $i$  est

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1].$$

Proche de 1 : le point est bien à l'intérieur de son groupe. Proche de 0 : il est à la frontière. Négatif : il est probablement mal classé. La **silhouette moyenne** sur tous les points mesure la qualité globale de la partition ; on choisit le  $k$  qui la maximise.

À la main, sur nos six clientes ( $\{1, 2, 4\}$  et  $\{9, 11, 12\}$ ) : pour le point 4,  $a = (|4 - 1| + |4 - 2|)/2 = 2,5$  et  $b = (|4 - 9| + |4 - 11| + |4 - 12|)/3 = 20/3 \approx 6,67$ , donc  $s(4) = (6,67 - 2,5)/6,67 = 0,625$ . Vérifions avec une fonction écrite à la main, puis avec la bibliothèque.

```
def silhouette_points(X, groupes):
    """Silhouette de chaque point (distances euclidiennes)."""
    D = np.sqrt(((X[:, None, :] - X[None, :, :]) ** 2).sum(axis=2))
    n = len(X)
    s = np.zeros(n)
    for i in range(n):
        propre = groupes == groupes[i]
        if propre.sum() == 1:
            continue
        a = D[i, propre].sum() / (propre.sum() - 1)
        b = min(D[i, groupes == g].mean() for g in set(groupe) if g != groupes[i])
        s[i] = (b - a) / max(a, b)
    return s

from sklearn.metrics import silhouette_score
x2 = x.reshape(-1, 1)
g = np.array([0, 0, 0, 1, 1, 1])
print("silhouette à la main de chaque point :", silhouette_points(x2, g).round(3))
print("silhouette moyenne (à la main)          :", silhouette_points(x2, g).mean().round(4))
print("silhouette moyenne (scikit-learn)       :", round(silhouette_score(x2, g), 4))

silhouette à la main de chaque point : [0.793 0.827 0.625 0.625 0.827 0.793]
silhouette moyenne (à la main)          : 0.7483
silhouette moyenne (scikit-learn)       : 0.7483
```

Appliquons les deux outils aux **trois groupes simulés** : le bon nombre est connu, c'est 3. Sachons-nous le retrouver ?

```
lignes = []
for k in range(1, 8):
    r = kmeans(Zs, k, np.random.default_rng(0))
    sil = silhouette_score(Zs, r["groupes"]) if k > 1 else np.nan
    lignes.append({"k": k, "W": r["W"], "silhouette moyenne": sil})
tab_sim = pd.DataFrame(lignes).set_index("k")
print(tab_sim.round(3).to_string())
```

|   | W        | silhouette moyenne |
|---|----------|--------------------|
| k |          |                    |
| 1 | 1200.000 | NaN                |
| 2 | 607.380  | 0.533              |
| 3 | 188.103  | 0.676              |
| 4 | 140.201  | 0.582              |
| 5 | 123.191  | 0.402              |
| 6 | 108.734  | 0.387              |
| 7 | 97.185   | 0.396              |

Deux lectures concordantes. La **variance intra-groupe** s'effondre de 1 200 à 607 puis à 188 quand

on passe de 1 à 2 puis à 3 groupes (pour  $k = 1$ ,  $W = n \times p = 600 \times 2 = 1\,200$  puisque les variables sont standardisées), puis ne décroît plus que lentement : le **coude** est net en  $k = 3$ . La **silhouette moyenne** atteint son maximum en  $k = 3$  (0,676) et chute ensuite. Les deux outils retrouvent le bon nombre de groupes. Pour interpréter une silhouette moyenne, on utilise des repères conventionnels (Kaufman et Rousseeuw) : au-dessus de 0,5, la structure est réelle ; entre 0,25 et 0,5, elle est faible ; en dessous de 0,25, on ne peut pas dire qu'il y ait une structure. Ces repères restent **indicatifs** : des variables très asymétriques produisent des silhouettes élevées même sans aucun groupe, et l'on ne peut interpréter une silhouette qu'en la comparant à celle d'un nuage témoin sans structure (voir l'exercice 12).

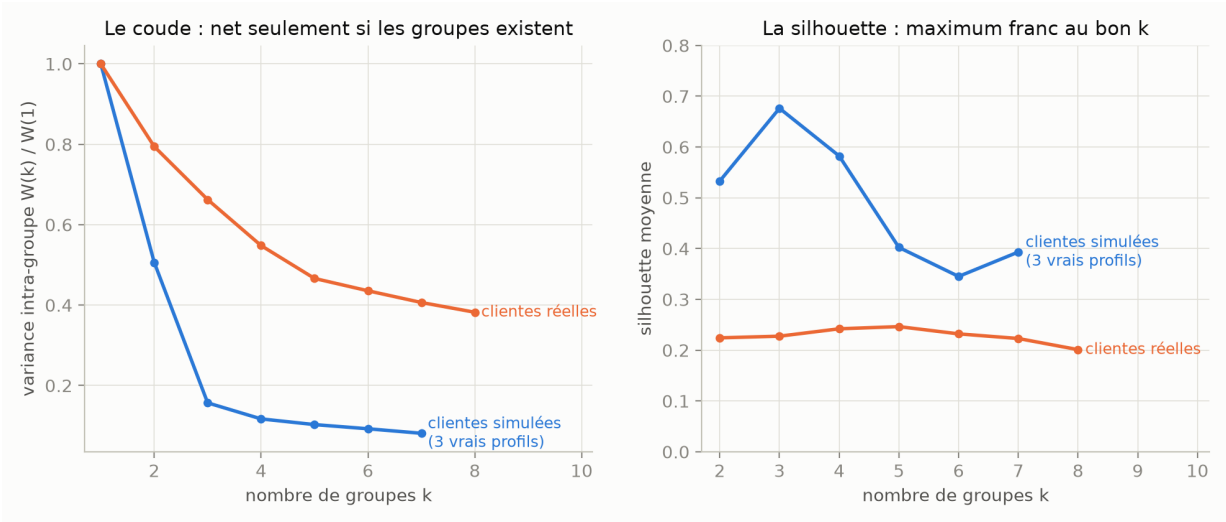


Figure 29: Choix du nombre de groupes : variance intra-groupe (le coude) et silhouette moyenne, pour les clientes simulées avec trois vrais profils et pour les clientes réelles de Dar Jasmin.

### 3.3.4 La classification hiérarchique

Ici, on ne fixe pas  $k$  à l'avance : on construit **un arbre** de regroupements. L'algorithme **ascendant** (agglomératif) est d'une simplicité désarmante :

1. au départ, chaque point est un groupe à lui seul ;
2. on **fusionne** les deux groupes les plus proches ;
3. on recommence jusqu'à n'avoir plus qu'un seul groupe.

Il reste à définir la **distance entre deux groupes** : c'est le **critère d'agrégation** (*linkage*).

| Critère                            | Distance entre deux groupes A et B                                                       | Effet typique                                                         |
|------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Simple</b> ( <i>single</i> )    | $\min_{a \in A, b \in B} d(a, b)$ : les deux points les plus proches                     | groupes <b>allongés</b> (effet de chaîne), sensible aux points isolés |
| <b>Complet</b> ( <i>complete</i> ) | $\max_{a \in A, b \in B} d(a, b)$ : les deux points les plus éloignés                    | groupes <b>compacts</b> de diamètre comparable                        |
| <b>Moyen</b> ( <i>average</i> )    | moyenne des $d(a, b)$                                                                    | compromis                                                             |
| <b>Ward</b>                        | augmentation de $W$ due à la fusion : $\frac{ A  B }{ A  +  B } \ \bar{a} - \bar{b}\ ^2$ | groupes <b>sphériques</b> de taille homogène, comparable aux k-means  |

À la main, sur cinq clientes décrites par une variable :  $A = 0$ ,  $B = 1$ ,  $C = 4$ ,  $D = 6$ ,  $E = 11$ .

- *Fusion 1* : la paire la plus proche est  $\{A, B\}$  (distance 1).
- *Fusion 2* : la plus proche ensuite est  $\{C, D\}$  (distance 2).
- *Fusion 3* : on fusionne  $\{A, B\}$  et  $\{C, D\}$ . La distance entre eux vaut, selon le critère : **simple**  $\min = d(B, C) = 3$  ; **complet**  $\max = d(A, D) = 6$ .
- *Fusion 4* : on rattache  $E$ . Distance **simple**  $= d(D, E) = 5$  ; **complète**  $= d(A, E) = 11$ .

```
from scipy.cluster.hierarchy import linkage
```

```
pts = np.array([[0.0], [1.0], [4.0], [6.0], [11.0]])
for methode in ["single", "complete"]:
    Zlien = linkage(pts, method=methode)
    print(f"critère {methode} : hauteurs des fusions =", Zlien[:, 2].round(2).tolist())
```

```
critère single : hauteurs des fusions = [1.0, 2.0, 3.0, 5.0]
critère complete : hauteurs des fusions = [1.0, 2.0, 6.0, 11.0]
```

Les hauteurs de fusion confirment le calcul à la main : 1, 2, 3, 5 pour le critère simple et 1, 2, 6, 11 pour le critère complet. Le **dendrogramme** dessine cet arbre, la hauteur de chaque fusion étant la distance à laquelle elle s'est produite. **Couper l'arbre** à une hauteur donnée fournit une partition : plus on coupe bas, plus il y a de groupes.

**Lire un dendrogramme.** Une longue branche verticale avant une fusion signale que deux groupes **bien séparés** ont été réunis de force : c'est là qu'il faut couper. Si toutes les fusions se font à des hauteurs graduellement croissantes (pas de saut), c'est que les données ne contiennent pas de groupes nets.

Appliquons-le aux 600 clientes simulées (critère de Ward, variables standardisées), et comparons la partition à trois groupes ainsi obtenue à celle des k-means. (Pour le dessin, nous ne représenterons que 40 clientes : un dendrogramme de 600 feuilles serait illisible.)

```
from scipy.cluster.hierarchy import fcluster, cophenet
from scipy.spatial.distance import pdist
```

```
L = linkage(Zs, method="ward")
hier3 = fcluster(L, t=3, criterion="maxclust") - 1
print("hauteurs des 4 dernières fusions (Ward) :", L[-4:, 2].round(1).tolist())
print("effectifs des 3 groupes hiérarchiques :", np.bincount(hier3).tolist())
print("ARI hiérarchique (Ward) vs vrais profils :", round(adjusted_rand_score(vrais, hier3), 3))
print("ARI k-means vs hiérarchique :", round(adjusted_rand_score(resultat["groupes"],
↪ hier3), 3))
c_corr, _ = cophenet(L, pdist(Zs))
print("corrélation cophénétique (fidélité de l'arbre aux distances) :", round(c_corr, 3))
```

```
hauteurs des 4 dernières fusions (Ward) : [6.1, 9.3, 29.1, 34.3]
effectifs des 3 groupes hiérarchiques : [304, 99, 197]
ARI hiérarchique (Ward) vs vrais profils : 0.964
ARI k-means vs hiérarchique : 0.965
corrélation cophénétique (fidélité de l'arbre aux distances) : 0.82
```

Les **deux dernières fusions** ont lieu à des hauteurs de 29,1 et 34,3, contre 9,3 et 6,1 pour les deux précédentes : l'arbre fait un **grand saut** avant de passer de trois groupes à deux, puis à un. C'est le signe qu'il y a trois groupes. La coupe à trois groupes redonne les trois profils (ARI de 0,96 contre les vrais profils) et coïncide presque avec les k-means (0,97) : sur des groupes aussi bien séparés, les deux méthodes s'accordent. La corrélation cophénétique de 0,82 indique que l'arbre est une bonne représentation des distances.

**La corrélation cophénétique.** La *distance cophénétique* de deux points est la hauteur à laquelle ils sont fusionnés dans l'arbre. La corrélation entre ces distances et les vraies distances mesure **dans quelle mesure l'arbre est fidèle** aux données : proche de 1, l'arbre résume bien les distances ; faible, il les déforme.

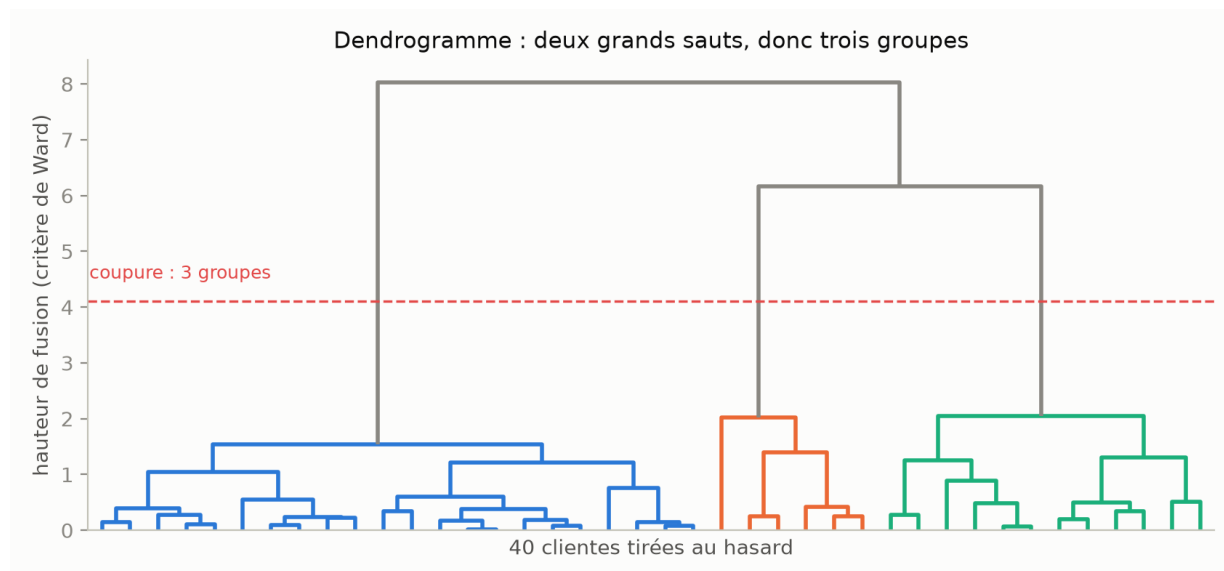


Figure 30: Dendrogramme de 40 clientes simulées (critère de Ward). La ligne en pointillés coupe l'arbre en trois groupes, colorés ; on remarque les deux fusions finales, très hautes, par rapport aux précédentes.

**k-means ou hiérarchique ?** Les k-means passent très bien à l'échelle (des millions de points), mais exigent  $k$  à l'avance et ne trouvent que des groupes « arrondis ». La classification hiérarchique donne une vue **à toutes les échelles** et ne dépend pas d'une initialisation, mais elle exige de stocker les  $n^2/2$  distances (elle ne dépasse guère quelques dizaines de milliers de points) et ses fusions sont définitives : une erreur précoce ne se corrige jamais.

### 3.3.5 Une segmentation honnête des clientes de Dar Jasmin

Passons aux **vraies** clientes (celles du fichier `clients.csv`, qui sont simulées mais **sans** segments programmés, comme dans la plupart des situations réelles). Nous décrivons chaque cliente par quatre variables : nombre de commandes par an, panier moyen, durée de la relation et âge. On écarte la dépense annuelle (elle est quasiment le produit de deux autres variables), et les clientes sans commande (panier nul).

```
c = pd.read_csv("donnees/clients.csv")
act = c[c["nb_commandes_an"] > 0].copy()
var = ["nb_commandes_an", "panier_moyen", "duree_mois", "age"]
Zc = ((act[var] - act[var].mean()) / act[var].std()).to_numpy()
print("clientes retenues :", len(act), "sur", len(c))

lignes = []
for k in range(1, 9):
    r = kmeans(Zc, k, np.random.default_rng(0), n_init=5)
    sil = silhouette_score(Zc, r["groupes"]) if k > 1 else np.nan
    lignes.append({"k": k, "W": r["W"], "silhouette_moyenne": sil})
tab_c = pd.DataFrame(lignes).set_index("k")
print(tab_c.round(3).to_string())
```



```

clientes retenues : 1740 sur 2000
      W  silhouette moyenne
k
1  6956.000           NaN
2  5526.143           0.224
3  4607.880           0.228
4  3812.099           0.242
5  3243.262           0.246
6  3029.588           0.232
7  2826.824           0.223
8  2658.348           0.201

```

Sur les clientes réelles, la variance intra-groupe décroît **régulièrement** de 6 956 à 2 658, sans coude (la courbe orange de la figure est presque une droite), et la silhouette moyenne **ne dépasse jamais** 0,25 (maximum 0,246 pour  $k = 5$ , valeurs voisines pour tous les autres  $k$ ). D'après les repères ci-dessus, on ne peut pas dire qu'il existe une structure en groupes.

Comparez avec le tableau des clientes simulées (3.3.3) : sur des données qui contiennent de vrais groupes, la silhouette atteint un maximum **franc** (0,68) au bon  $k$  ; ici, rien de tel. Les données ne se laissent pas découper naturellement.

Ce constat n'interdit pas de **segmenter** : pour un usage commercial, on a le droit de découper un continuum en tranches commodas, comme on découpe un âge en tranches de dix ans, pourvu que l'on **ne prétende pas** avoir découvert des types naturels de clientes. Prenons  $k = 4$  et décrivons les groupes :

```

r4 = kmeans(Zc, 4, np.random.default_rng(0), n_init=10)
act["segment"] = r4["groupes"]
profil = act.groupby("segment").agg(effectif=("id_client", "size"), commandes=("nb_commandes_an",
↪ "mean"),
                                   panier=("panier_moyen", "mean"), duree=("duree_mois", "mean"),
↪ age=("age", "mean")).round(1)
profil["part"] = (100 * profil["effectif"] / len(act)).round(0)
print(profil.sort_values("panier").to_string())

```

| segment | effectif | commandes | panier | duree | age  | part |
|---------|----------|-----------|--------|-------|------|------|
| 3       | 695      | 3.2       | 46.8   | 16.8  | 28.8 | 40.0 |
| 1       | 282      | 4.0       | 63.0   | 52.9  | 36.6 | 16.0 |
| 2       | 237      | 11.2      | 67.9   | 20.8  | 34.3 | 14.0 |
| 0       | 526      | 3.4       | 76.4   | 17.5  | 45.4 | 30.0 |

Quatre profils lisibles se dégagent, pour peu qu'on les nomme avec prudence :

- **Segment 3** (40 % des clientes) : les plus **jeunes** (29 ans en moyenne) au **plus petit panier** (47 DT) ;
- **Segment 0** (30 %) : les plus **âgées** (45 ans) au **plus gros panier** (76 DT) ;
- **Segment 2** (14 %) : les **habituées**, avec 11 commandes par an en moyenne, contre 3 à 4 pour les autres ;
- **Segment 1** (16 %) : les **anciennes**, dont la relation dure en moyenne 53 mois, contre 17 à 21 pour les autres.

Ces portraits sont **utiles** (on n'enverra pas le même message à une étudiante et à une cliente de longue date), mais ils décrivent des *tendances*, pas des types : chaque segment contient des clientes très variées.

Il reste à tester si ces segments sont **stables** : si l'on tire d'autres clientes dans la même population, retrouve-t-on les mêmes groupes ? On ré-estime les k-means sur 30 échantillons bootstrap (volume I, section 3.3.5), on range **toutes** les clientes d'origine dans le groupe du centre le plus

proche, et on compare à la partition de référence avec l'ARI.

```
def stabilite(X, k, ref, rng, B=30):
    ari = []
    for _ in range(B):
        idx = rng.integers(0, len(X), len(X))
        r = kmeans(X[idx], k, rng, n_init=3)
        d = ((X[:, None, :] - r["centres"][None, :, :]) ** 2).sum(axis=2)
        ari.append(adjusted_rand_score(ref, d.argmin(axis=1)))
    return np.mean(ari), np.min(ari)

m_sim, mn_sim = stabilite(Zs, 3, resultat["groupes"], np.random.default_rng(1))
m_c, mn_c = stabilite(Zc, 4, r4["groupes"], np.random.default_rng(1))
print(f"clientes simulées avec vrais groupes (k=3) : ARI moyen = {m_sim:.3f}, minimum = {mn_sim:.3f}")
print(f"clientes réelles (k=4) : ARI moyen = {m_c:.3f}, minimum = {mn_c:.3f}")

clientes simulées avec vrais groupes (k=3) : ARI moyen = 0.994, minimum = 0.985
clientes réelles (k=4) : ARI moyen = 0.864, minimum = 0.510
```

Les clientes simulées avec de vrais groupes donnent une stabilité **quasi parfaite** (ARI moyen 0,994, minimum 0,985 : on retrouve la même partition quel que soit l'échantillon). Sur les clientes réelles, la stabilité est **honorabile en moyenne** (0,86) mais le **pire cas** est mauvais (0,51) : sur certains échantillons, le découpage est très différent de la référence.

**Stabilité n'est pas existence.** Une partition peut être reproductible sans correspondre à des groupes réels : avec 1 740 clientes, les k-means retrouvent à peu près les mêmes quatre « tranches » à chaque tirage, comme un découpeur de gâteau qui poserait toujours son couteau aux mêmes endroits. La stabilité est une condition **nécessaire** (une partition instable est sûrement arbitraire), jamais **suffisante**. C'est pourquoi on la combine avec la silhouette et le coude.

**Révélation.** Aucun segment n'avait été programmé dans ces données : les variables ont été générées indépendamment de toute notion de « type de cliente ». Les k-means ont pourtant rendu quatre groupes, tout aussi sérieux en apparence que les trois profils simulés. Seuls les **indicateurs** (silhouette inférieure à 0,25, absence de coude, stabilité inégale dans le pire cas) pouvaient nous avertir qu'ils n'étaient pas réels. Voilà pourquoi on ne livre jamais une segmentation sans ces diagnostics.

### 3.3.6 Les pièges de la classification

**1. Les k-means supposent des groupes « arrondis ».** Comme ils minimisent des distances à des centres, ils découpent l'espace en polyèdres et échouent sur des formes allongées, incurvées ou de tailles très inégales. La classification hiérarchique à lien simple, au contraire, suit les chaînes de points voisins. Exemple classique, ci-dessous : deux « croissants » de lune.

```
from sklearn.datasets import make_moons
from sklearn.cluster import AgglomerativeClustering

lune, vrai_lune = make_moons(n_samples=300, noise=0.07, random_state=0)
km_lune = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(lune)
simple_lune = AgglomerativeClustering(n_clusters=2, linkage="single").fit_predict(lune)
print("ARI k-means", round(adjusted_rand_score(vrai_lune, km_lune), 3))
print("ARI lien simple", round(adjusted_rand_score(vrai_lune, simple_lune), 3))

ARI k-means : 0.234
ARI lien simple : 1.0
```

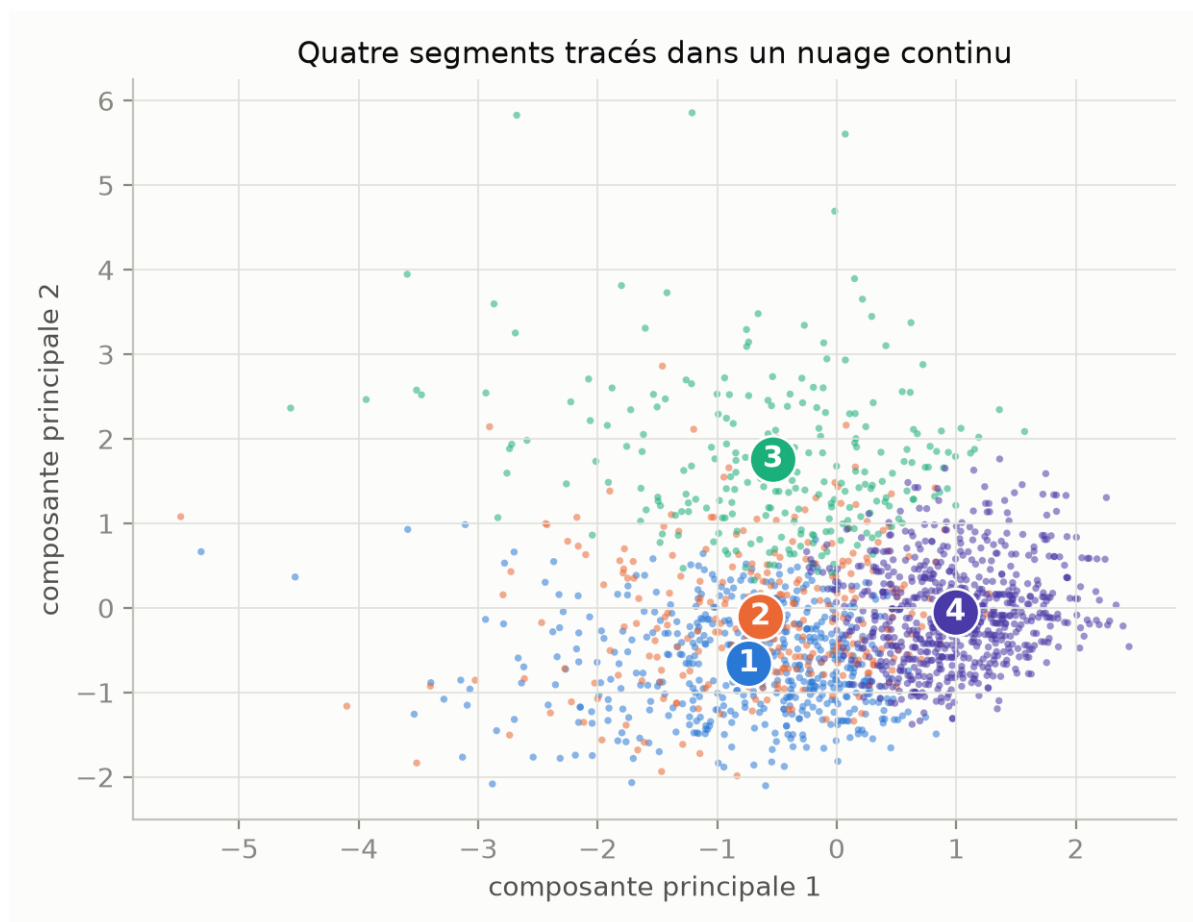


Figure 31: Les clientes de Dar Jasmin (variables standardisées) projetées sur les deux premières composantes principales, colorées par segment k-means ( $k = 4$ ). Les segments se touchent : ils découpent un nuage continu.

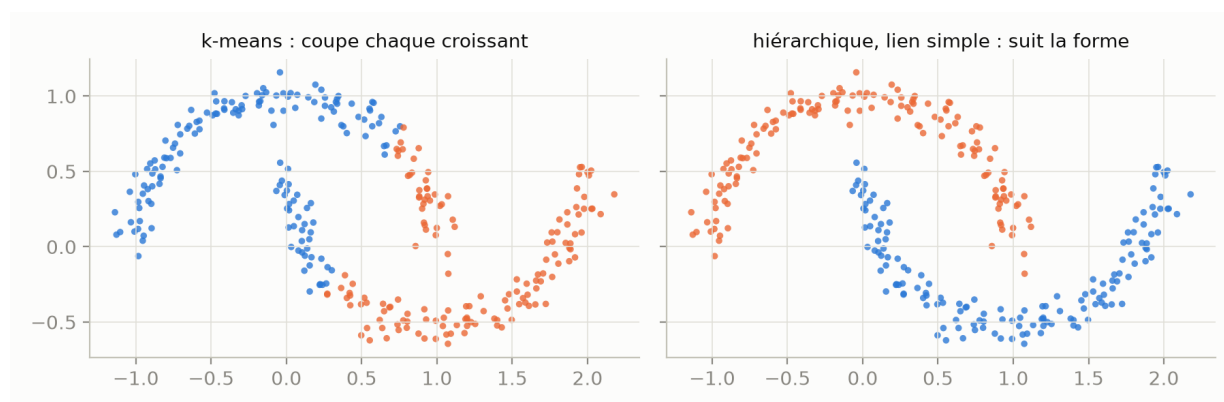


Figure 32: Deux croissants de lune. À gauche, les k-means coupent chaque croissant en deux ; à droite, la classification hiérarchique à lien simple suit la forme des croissants.

**2. L'échelle des variables** décide du résultat : sans standardisation, la variable de plus grande variance fait la loi (section 3.1.5).

**3. Les valeurs extrêmes** attirent un centre (la moyenne est sensible aux extrêmes) ou forment des groupes à un seul point.

**4. La dimension.** Quand le nombre de variables est grand, toutes les distances tendent à se ressembler (le « fléau de la dimension ») et la notion de « proche » perd son sens. On réduit d'abord la dimension (ACP, section 3.1), puis on classe les scores.

**5. Le choix de  $k$  n'est jamais « objectif » :** le coude est une impression, la silhouette un compromis. Documentez le choix, testez plusieurs valeurs, vérifiez la stabilité et l'utilité du découpage pour la décision à prendre.

**6. Pas de vérité à laquelle comparer.** Contrairement à une régression, il n'y a pas de variable à prédire : la qualité d'une classification ne se mesure que par des indicateurs internes (silhouette, stabilité) ou par sa **valeur d'usage**.

#### À retenir

- Classer, c'est regrouper des points **proches** ; **standardisez** d'abord.
- **K-means** : on minimise  $W = \sum_j \sum_{i \in C_j} \|x_i - \mu_j\|^2$  par l'algorithme de Lloyd (affectation / mise à jour) ;  $W$  décroît à chaque étape, mais on n'atteint qu'un **minimum local** : initialisation **k-means++** et plusieurs départs.
- $k$  : jamais choisi avec  $W$  seule ; on regarde le **coude**, la **silhouette**  $s(i) = \frac{b-a}{\max(a,b)}$ , et surtout la **stabilité** (bootstrap + ARI).
- **Hiérarchique** : on fusionne pas à pas ; le **critère d'agrégation** (simple, complet, moyen, Ward) détermine la forme des groupes ; le **dendrogramme** montre toutes les échelles.
- Un algorithme de classification **rend toujours des groupes** : sans indicateurs de qualité et de stabilité, on ne peut pas savoir s'ils existent.

### 3.4 ⊕ Pour aller plus loin : l'analyse des correspondances (AC et ACM)

**Section optionnelle.** Elle étend l'idée de l'ACP aux **variables qualitatives** (canal, ville, tranche d'âge), pour lesquelles moyenne et variance n'ont pas de sens. On peut sauter cette section sans perdre le fil du chapitre. Elle s'appuie sur le test du khi-deux du volume I (section 3.4.6) et sur la décomposition en valeurs singulières (volume I, section 1.1.4).

**Intuition.** L'ACP regarde un nuage de points dans un espace de nombres. Mais comment « dessiner » un tableau croisant le canal d'acquisition (trois modalités) et la taille du panier (quatre tranches) ? L'**analyse factorielle des correspondances** (AFC, ou simplement AC) transforme un tableau de contingence en **deux nuages de points** (un pour les lignes, un pour les colonnes), placés sur une même carte, de façon que **deux modalités proches sur la carte soient fréquemment associées** dans les données. Quant à l'**analyse des correspondances multiples** (ACM), c'est l'AC appliquée à plusieurs variables qualitatives à la fois.

#### 3.4.1 Le point de départ : un tableau de contingence

Prenons les clientes ayant déjà commandé (1 740 sur 2 000) et croisons leur **canal d'acquisition** avec la **tranche de leur panier moyen**, définie par les quartiles : « très petit » pour le quart le plus bas, jusqu'à « très grand » pour le quart le plus haut.

```
import numpy as np
import pandas as pd
from scipy import stats

c = pd.read_csv("donnees/clients.csv")
a = c[c["nb_commandes_an"] > 0].copy()
a["tranche_panier"] = pd.qcut(a["panier_moyen"], 4, labels=["T1 très petit", "T2 petit", "T3 grand",
↵ "T4 très grand"])
a["tranche_age"] = pd.cut(a["age"], [0, 29, 39, 200], labels=["moins de 30", "30-39", "40 et plus"])

N = pd.crosstab(a["canal_acquisition"], a["tranche_panier"])
print(N.to_string())
print()
print("profils-lignes (en %) : répartition des paniers dans chaque canal")
print((100 * N.div(N.sum(axis=1), axis=0)).round(1).to_string())
```

| tranche_panier    | T1 très petit | T2 petit | T3 grand | T4 très grand |
|-------------------|---------------|----------|----------|---------------|
| canal_acquisition |               |          |          |               |
| Boutique          | 46            | 93       | 126      | 184           |
| Instagram         | 258           | 184      | 158      | 87            |
| Site              | 132           | 157      | 152      | 163           |

| tranche_panier    | T1 très petit | T2 petit | T3 grand | T4 très grand |
|-------------------|---------------|----------|----------|---------------|
| canal_acquisition |               |          |          |               |
| Boutique          | 10.2          | 20.7     | 28.1     | 41.0          |
| Instagram         | 37.6          | 26.8     | 23.0     | 12.7          |
| Site              | 21.9          | 26.0     | 25.2     | 27.0          |

Les **profils-lignes** (la répartition des tranches de panier dans chaque canal) sont très différents d'un canal à l'autre : la boutique compte 69 % de paniers « grands » ou « très grands » (T3 et T4) contre 36 % pour Instagram, dont 64 % des paniers sont « petits » ou « très petits » (T1 et T2). Si le canal et le panier étaient **indépendants**, les trois lignes auraient le même profil, égal au profil des totaux. L'analyse des correspondances décrit **comment et dans quelle direction** les profils s'écartent de cette indépendance.

Mesurons d'abord l'écart global avec le test du khi-deux (volume I, section 3.4.6) :

```
chi2, p, ddl, attendus = stats.chi2_contingency(N, correction=False)
n = N.values.sum()
print(f"khi-deux = {chi2:.1f}, ddl = {ddl}, p = {p:.2g}")
print(f"khi-deux / n = {chi2 / n:.4f}")
```

```
khi-deux = 180.7, ddl = 6, p = 2.5e-36
khi-deux / n = 0.1038
```

La quantité  $\chi^2/n$  est appelée **inertie totale** du tableau. Elle ne dépend pas de la taille de l'échantillon et mesure l'intensité de l'association : 0 en cas d'indépendance parfaite. C'est ce nombre que l'analyse des correspondances va **décomposer sur des axes**, exactement comme l'ACP décompose la variance totale.

### 3.4.2 La méthode : une SVD sur les résidus standardisés

Notons  $P = N/n$  la matrice des fréquences,  $\mathbf{r}$  le vecteur des fréquences de lignes (marges de  $P$ ) et  $\mathbf{c}$  celui des colonnes. Sous indépendance, la fréquence attendue de la case  $(i, j)$  est  $r_i c_j$ . La matrice des **résidus standardisés** est

$$S_{ij} = \frac{p_{ij} - r_i c_j}{\sqrt{r_i c_j}}.$$

**L'inertie totale est**  $\|S\|^2 = \chi^2/n$ . Il suffit de développer :  $\sum_{i,j} S_{ij}^2 = \sum_{i,j} \frac{(p_{ij}-r_i c_j)^2}{r_i c_j} = \frac{1}{n} \sum_{i,j} \frac{(n_{ij}-E_{ij})^2}{E_{ij}} = \frac{\chi^2}{n}$ , car  $n p_{ij} = n_{ij}$  et  $n r_i c_j = E_{ij}$  (effectifs attendus).

L'analyse des correspondances **est** la SVD de cette matrice :  $S = U \Sigma V^T$ , avec  $\Sigma = \text{diag}(\sigma_1 \geq \sigma_2 \geq \dots)$ . Comme la norme de Frobenius au carré est la somme des carrés des valeurs singulières, l'inertie totale se **répartit sur les axes** :

$$\frac{\chi^2}{n} = \sum_k \sigma_k^2, \quad \text{l'axe } k \text{ porte la part } \frac{\sigma_k^2}{\sum_l \sigma_l^2}.$$

Les **coordonnées principales** des lignes et des colonnes sur l'axe  $k$  sont, en notant  $D_r = \text{diag}(\mathbf{r})$  et  $D_c = \text{diag}(\mathbf{c})$  :

$$F = D_r^{-1/2} U \Sigma \quad (\text{lignes}), \quad G = D_c^{-1/2} V \Sigma \quad (\text{colonnes}).$$

On dessine  $F$  et  $G$  sur la même carte. Ce choix a une conséquence élégante, la **relation barycentrique** : la coordonnée d'une ligne est, au facteur  $1/\sigma_k$  près, la **moyenne pondérée** des coordonnées des colonnes, les poids étant son profil. Une modalité-ligne est donc attirée vers les modalités-colonnes avec lesquelles elle est fortement associée.

**Un exemple à la main : un tableau  $2 \times 2$ .** Deux canaux, deux tranches de panier, 80 clientes :

|           | Petit panier | Grand panier |
|-----------|--------------|--------------|
| Instagram | 30           | 10           |
| Boutique  | 10           | 30           |

Les marges valent  $\mathbf{r} = (0,5; 0,5)$  et  $\mathbf{c} = (0,5; 0,5)$  ;  $P = \begin{pmatrix} 0,375 & 0,125 \\ 0,125 & 0,375 \end{pmatrix}$  ;  $r_i c_j = 0,25$  partout. Donc

$$S = \frac{P - 0,25}{\sqrt{0,25}} = \frac{1}{0,5} \begin{pmatrix} 0,125 & -0,125 \\ -0,125 & 0,125 \end{pmatrix} = \begin{pmatrix} 0,25 & -0,25 \\ -0,25 & 0,25 \end{pmatrix}.$$

Cette matrice est de rang 1 :  $S = 0,5 \times \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$ , donc  $\sigma_1 = 0,5$  et  $\sigma_1^2 = 0,25$ .

Vérification avec le khi-deux : les effectifs attendus valent 20 partout, d'où  $\chi^2 = 4 \times \frac{(\pm 10)^2}{20} = 20$  et  $\chi^2/n = 20/80 = 0,25$ . ✓ Les coordonnées des lignes sont  $\pm \frac{1}{\sqrt{0,5}} \cdot \frac{1}{\sqrt{2}} \cdot 0,5 = \pm 0,5$  : Instagram à  $-0,5$  d'un côté, Boutique à  $+0,5$  de l'autre. Un tableau  $2 \times 2$  n'a qu'**un seul axe**, sur lequel les deux modalités s'opposent. Plus généralement, un tableau  $I \times J$  a au plus  $\min(I, J) - 1$  axes non triviaux.

Écrivons la méthode et vérifions-la sur cet exemple, puis sur notre tableau :

```
def analyse_correspondances(N):
    """AC par SVD des résidus standardisés. Renvoie l'inertie par axe et les coordonnées
    ↪ principales."""
    N = np.asarray(N, dtype=float)
    P = N / N.sum()
    r, c = P.sum(axis=1), P.sum(axis=0)
    S = (P - np.outer(r, c)) / np.sqrt(np.outer(r, c))
```

```

U, sv, Vt = np.linalg.svd(S, full_matrices=False)
F = (U / np.sqrt(r)[:, None]) * sv          # coordonnées principales des lignes
G = (Vt.T / np.sqrt(c)[:, None]) * sv      # coordonnées principales des colonnes
return {"inertie": sv**2, "F": F, "G": G}

petit = analyse_correspondances([[30, 10], [10, 30]])
print("exemple 2x2 : inertie =", petit["inertie"].round(4), "| coordonnées des lignes sur l'axe 1 =",
      ↪ np.abs(petit["F"][:, 0]).round(3))

ac = analyse_correspondances(N.values)
print("tableau canal x panier : inertie par axe =", ac["inertie"].round(4))
print("somme des inerties =", ac["inertie"].sum().round(4), " et khi-deux/n =", round(chi2 / n, 4))
print("part de l'axe 1 :", f"{100 * ac['inertie'][0] / ac['inertie'].sum():.1f} %")

exemple 2x2 : inertie = [0.25 0. ] | coordonnées des lignes sur l'axe 1 = [0.5 0.5]
tableau canal x panier : inertie par axe = [0.1031 0.0007 0. ]
somme des inerties = 0.1038 et khi-deux/n = 0.1038
part de l'axe 1 : 99.3 %

```

### 3.4.3 Lire la carte

```

lignes = pd.DataFrame(ac["F"][:, :2], index=N.index, columns=["axe 1", "axe 2"])
colonnes = pd.DataFrame(ac["G"][:, :2], index=N.columns, columns=["axe 1", "axe 2"])
print("canaux :")
print(lignes.round(3).to_string())
print("tranches de panier :")
print(colonnes.round(3).to_string())

```

```

canaux :
              axe 1  axe 2
canal_acquisition
Boutique         0.448 -0.026
Instagram        -0.354 -0.015
Site              0.070  0.036
tranches de panier :
              axe 1  axe 2
tranche_panier
T1 très petit   -0.440 -0.024
T2 petit        -0.090  0.046
T3 grand         0.079 -0.010
T4 très grand   0.452 -0.012

```

Sur la carte de gauche, **tout se passe sur le premier axe** (il porte presque toute l'inertie). Les trois canaux s'y rangent dans l'ordre Instagram, Site, Boutique ; et les quatre tranches de panier dans l'ordre T1, T2, T3, T4, **du même côté que** les canaux auxquels elles sont associées : les très grands paniers (T4) sont du côté de la boutique, les très petits (T1) du côté d'Instagram. L'axe 1 est donc un axe « **petits paniers / gros paniers** » commun aux deux variables. Le fait que les tranches de panier s'ordonnent exactement comme leur numérotation est typique : pour une variable ordinaire, l'AC retrouve l'ordre sans qu'on le lui ait dit.

**Révélation.** C'est ce qui avait été programmé : le canal d'acquisition agit sur le panier (en échelle logarithmique : +0,22 pour la boutique, +0,05 pour le site, -0,12 pour Instagram). L'AC a retrouvé cette association sans qu'on lui désigne de variable « à expliquer », et a même restitué l'ordre des canaux.

**Piège numéro un : une carte a toujours l'air de dire quelque chose.** Faites maintenant l'expérience inverse, avec deux variables qui n'ont **aucun lien** dans la simulation : le canal



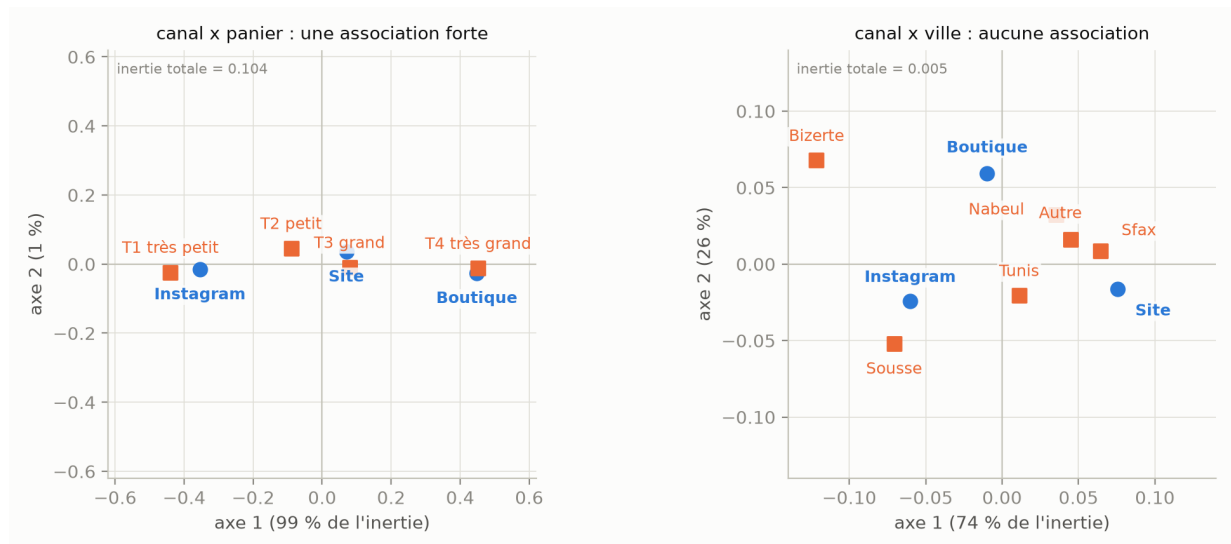


Figure 33: Analyse des correspondances du tableau canal x tranche de panier (à gauche) et, en comparaison, du tableau canal x ville (à droite). Attention aux échelles : à gauche les points sont répartis sur près de  $\pm 0,45$ , à droite sur  $\pm 0,1$  seulement.

d'acquisition et la ville.

```
N2 = pd.crosstab(a["canal_acquisition"], a["ville"])
chi2_b, p_b, ddl_b, _ = stats.chi2_contingency(N2, correction=False)
ac2 = analyse_correspondances(N2.values)
print(f"canal x ville : khi-deux = {chi2_b:.1f}, ddl = {ddl_b}, p = {p_b:.2f}")
print("inertie totale :", round(chi2_b / N2.values.sum(), 4), "(valeur attendue par pur hasard :  

↳ environ ddl/n =", round(ddl_b / N2.values.sum(), 4), ")")
print("part de l'axe 1 :", f"{100 * ac2['inertie'][0] / ac2['inertie'].sum():.0f} %", " part de l'axe  

↳ 2 :", f"{100 * ac2['inertie'][1] / ac2['inertie'].sum():.0f} %")
```

```
canal x ville : khi-deux = 8.1, ddl = 10, p = 0.62
inertie totale : 0.0047 (valeur attendue par pur hasard : environ ddl/n = 0.0057 )
part de l'axe 1 : 74 % part de l'axe 2 : 26 %
```

Le test du khi-deux ne rejette pas l'indépendance, et l'inertie totale est de l'ordre de grandeur de ce que le **hasard seul** produit (en moyenne  $ddl/n$  sous l'indépendance). Pourtant, l'axe 1 « explique » 74 % de l'inertie et l'axe 2 les 26 % restants : un tableau  $3 \times 6$  n'a que deux axes non triviaux, qui se partagent donc **toujours** 100 % de l'inertie, quel que soit le tableau. La carte de droite de la figure montre des points bien répartis, mais sur une échelle minuscule (de l'ordre de  $\pm 0,1$ , contre  $\pm 0,45$  à gauche). **Un pourcentage d'inertie ne prouve jamais rien** : il faut d'abord regarder le khi-deux et l'**inertie totale**, qui mesure l'intensité de l'association.

### 3.4.4 L'analyse des correspondances multiples (ACM)

Pour étudier **plusieurs** variables qualitatives à la fois (disons  $Q$ ), on recode chaque individu par un vecteur d'indicatrices : pour la variable « canal », trois colonnes (0 ou 1), dont une seule vaut 1. On obtient le **tableau disjonctif complet**  $Z$ , de  $n$  lignes et  $J = \sum_q J_q$  colonnes ( $J_q$  modalités pour la variable  $q$ ). L'ACM est simplement l'**analyse des correspondances de ce tableau  $Z$** , avec les mêmes formules.

Appliquons-la à six variables : canal, ville, tranche d'âge, tranche de panier, offre de bienvenue et rachat dans les 12 mois.

```
cols = ["canal_acquisition", "ville", "tranche_age", "tranche_panier", "offre_bienvenue",
        ↪ "rachat_12m"]
D = pd.get_dummies(a[cols].astype(str), dtype=float)          # tableau disjonctif complet
Q_, J = len(cols), D.shape[1]
print("individus :", D.shape[0], "| variables Q =", Q_, "| modalités J =", J)

acm = analyse_correspondances(D.values)
lam = acm["inertie"]
print("inertie totale =", lam.sum().round(4), "  théorie (J - Q)/Q =", round((J - Q_) / Q_, 4))
print("8 premières valeurs propres :", lam[:8].round(4))
print("part de variance des deux premiers axes :", (100 * lam[:2] / lam.sum()).round(1), "%")

individus : 1740 | variables Q = 6 | modalités J = 20
inertie totale = 2.3333  théorie (J - Q)/Q = 2.3333
8 premières valeurs propres : [0.2347 0.1967 0.1826 0.1777 0.1738 0.1682 0.1663 0.1646]
part de variance des deux premiers axes : [10.1  8.4] %
```

Deux remarques sur ces nombres. D'abord, l'inertie totale de l'ACM vaut **toujours**  $(J - Q)/Q$ , indépendamment des données : ici  $(20 - 6)/6 \approx 2,33$  (le nombre de modalités moins le nombre de variables, rapporté au nombre de variables). Ensuite, les valeurs propres décroissent **très lentement**, et les pourcentages d'inertie sont donc faibles (10,1 % et 8,4 % pour les deux premiers axes) : c'est normal en ACM, c'est un effet du codage, pas un signe d'absence de structure. Une correction classique, due à **Benzécri**, ne garde que les valeurs propres supérieures à  $1/Q$  et les transforme en

$$\lambda_k^{\text{corr}} = \left(\frac{Q}{Q-1}\right)^2 \left(\lambda_k - \frac{1}{Q}\right)^2 \quad (\lambda_k > 1/Q),$$

ce qui donne des pourcentages plus réalistes (six valeurs propres dépassent  $1/Q = 0,1667$ ) :

```
seuil = 1 / Q_
gardees = lam[lam > seuil]
corrigees = (Q_ / (Q_ - 1)) ** 2 * (gardees - seuil) ** 2
print("valeurs propres supérieures à 1/Q =", round(seuil, 4), ":", len(gardees))
print("parts corrigées (Benzécri) des deux premiers axes :", (100 * corrigees[:2] /
        ↪ corrigees.sum()).round(1), "%")
```

```
valeurs propres supérieures à 1/Q = 0.1667 : 6
parts corrigées (Benzécri) des deux premiers axes : [77.7 15.1] %
```

Pour savoir **quelle variable construit quel axe**, on calcule les **contributions** : celle de la modalité  $j$  à l'axe  $k$  vaut  $c_j G_{jk}^2 / \sigma_k^2$  (somme égale à 1 sur les modalités d'un même axe). En additionnant par variable, on voit quelles variables « fabriquent » chaque axe.

```
c_mod = D.values.sum(axis=0) / D.values.sum()          # poids des modalités
ctr = (c_mod[:, None] * acm["G"][:, :3] ** 2) / acm["inertie"][:3]
ctr = pd.DataFrame(ctr, index=D.columns, columns=["axe 1", "axe 2", "axe 3"])
par_variable = ctr.groupby([next(v for v in cols if m.startswith(v + "_")) for m in D.columns]).sum()
print((100 * par_variable).round(0).loc[cols].to_string())
```

|                   | axe 1 | axe 2 | axe 3 |
|-------------------|-------|-------|-------|
| canal_acquisition | 36.0  | 2.0   | 17.0  |
| ville             | 1.0   | 11.0  | 27.0  |
| tranche_age       | 11.0  | 17.0  | 37.0  |
| tranche_panier    | 47.0  | 5.0   | 17.0  |
| offre_bienvenue   | 0.0   | 28.0  | 2.0   |
| rachat_12m        | 5.0   | 37.0  | 0.0   |

Avec la correction de Benzécri, le premier axe porte à lui seul près de 78 % de l'« inertie utile » et le deuxième 15 % : l'essentiel de la structure est dans le plan de la carte. Le tableau des contributions

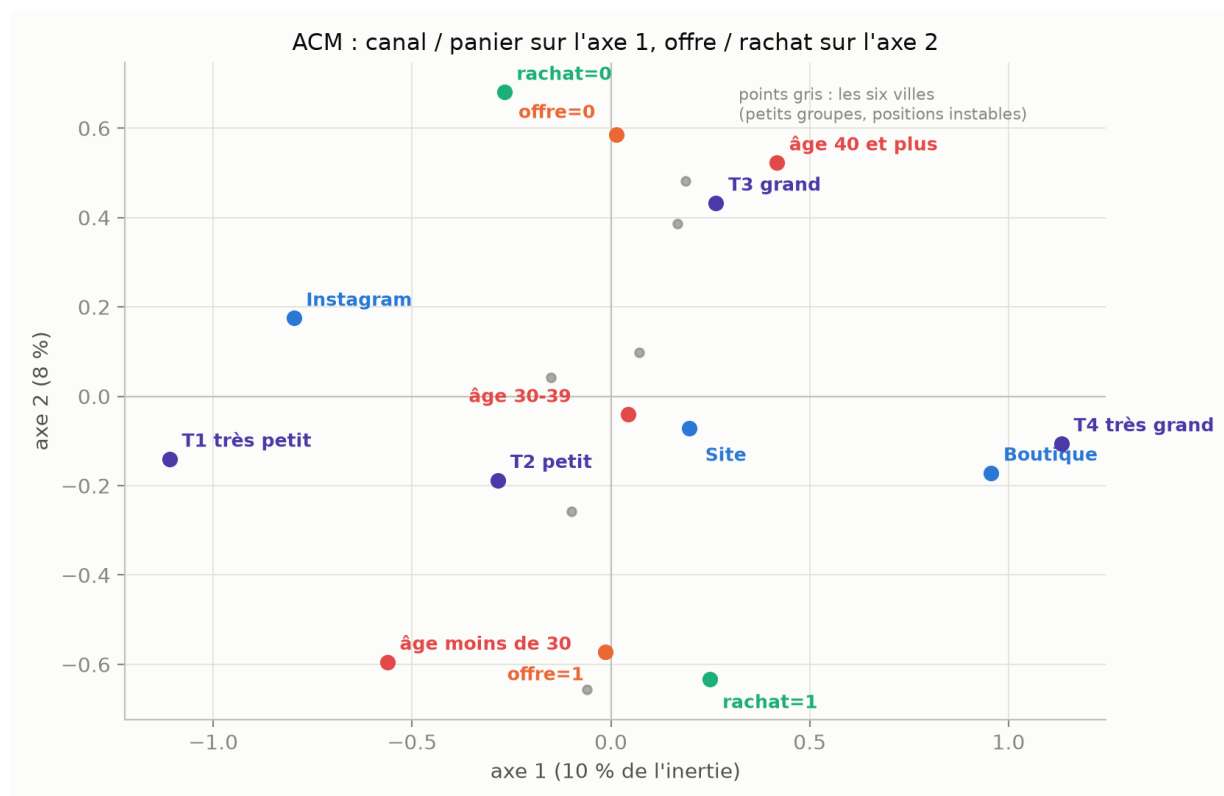


Figure 34: Analyse des correspondances multiples : plan des deux premiers axes. Les modalités des quatre variables d'intérêt sont étiquetées (couleur par variable) ; les six villes sont les points gris.

et la carte se lisent ensemble :

1. **L'axe 1** est construit par la **tranche de panier** (47 %) et le **canal** (36 %) : il oppose Boutique et très grands paniers (à droite) à Instagram et très petits paniers (à gauche), l'association déjà vue en 3.4.3. L'âge y contribue un peu (11 %) : les moins de 30 ans sont du côté des petits paniers.
2. **L'axe 2** est construit par le **rachat** (37 %), l'**offre de bienvenue** (28 %) et, dans une moindre mesure, l'**âge** (17 %) : offre=1 est proche de rachat=1, loin de offre=0 et rachat=0, et les moins de 30 ans sont du même côté que les rachats.
3. **Les villes** contribuent à peine aux deux premiers axes (1 % et 11 %) : leurs points gris sont dispersés (ce sont de petits groupes, dont les coordonnées sont instables) mais ne s'associent à rien de particulier.

**Révélation.** Tout cela correspond à la simulation : le canal détermine le panier ; l'offre de bienvenue augmente la probabilité de rachat ; l'âge agit à la fois sur le panier (les plus âgées dépensent un peu plus) et sur le rachat (les plus jeunes rachètent davantage) ; la ville n'a aucun effet. L'ACM n'a reçu aucune variable « à expliquer » : elle a simplement dessiné les associations qui existent dans le tableau.

**Application.** L'ACM est la porte d'entrée classique pour explorer **un questionnaire avec des modalités qualitatives** (profils de clientes, réponses à choix multiples). Elle est aussi utilisée pour construire des **typologies** : on calcule les coordonnées des individus sur les premiers axes, puis on les classe par k-means ou classification hiérarchique (section 3.3).

Mais prudence : l'ACM décrit **les associations observées**, elle n'établit pas de causalité.

**Autres pièges.** (1) Les **modalités rares** (quelques individus) ont des coordonnées extrêmes et peuvent dominer un axe : regroupez-les ou mettez-les en éléments supplémentaires. (2) L'AC est **descriptive** : testez d'abord l'association (khi-deux), puis décrivez-la. (3) Dans l'ACM, **chaque variable pèse son nombre de modalités** : une variable à 15 modalités dominera une variable à 2.

#### À retenir

- L'AC transforme un **tableau de contingence** en cartes : les modalités fréquemment associées sont **proches**.
- Mathématiquement, c'est la **SVD des résidus standardisés**  $S_{ij} = (p_{ij} - r_i c_j) / \sqrt{r_i c_j}$  ; l'**inertie totale** vaut  $\chi^2/n$  et se répartit sur les axes.
- Avant d'interpréter une carte, **testez l'association** : une carte d'indépendance a l'air tout aussi sérieuse.
- L'**ACM** est l'AC du tableau disjonctif complet ; son inertie totale vaut  $(J - Q)/Q$  ; les pourcentages d'inertie sont faibles par construction (corrigez avec Benzécri) ; les **contributions** disent quelle variable construit quel axe.

### 3.5 ⊕ Pour aller plus loin : l'analyse discriminante

**Section optionnelle.** Elle fait le pont entre ce chapitre (où l'on cherche des groupes inconnus) et le chapitre 2 (où l'on prédit une catégorie à partir de variables). Elle suppose connue la régression logistique de la section 2.2, et reprend la formule de Bayes et la loi normale du volume I (sections 2.1 et 2.2) ; la loi normale **multivariée** est rappelée en 3.5.1.

**Intuition.** Jusqu'ici, nous avons cherché des groupes **sans savoir** à quoi ils ressemblent. Ici, on **connaît** les groupes (les clientes qui ont racheté, celles qui n'ont pas racheté) et l'on veut deux choses : (1) **classer** une nouvelle cliente dans le bon groupe, (2) **décrire** ce qui distingue le mieux les groupes. L'idée de l'**analyse discriminante** est élégante : on suppose que, **dans chaque groupe**, les variables suivent une loi normale, et l'on applique la formule de Bayes pour calculer la probabilité d'appartenir à chaque groupe étant donné ce que l'on observe.

#### 3.5.1 La règle de Bayes avec des groupes gaussiens

Soit  $K$  groupes de probabilités a priori  $\pi_1, \dots, \pi_K$  (la part de chaque groupe dans la population). Supposons qu'au sein du groupe  $k$ , le vecteur de variables  $\mathbf{x} \in \mathbb{R}^p$  suive une loi normale  $\mathcal{N}(\mu_k, \Sigma_k)$ , de densité  $f_k$ . (Rappel : la densité d'une loi normale multivariée est  $f(\mathbf{x}) = (2\pi)^{-p/2} |\Sigma|^{-1/2} \exp(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu))$  ; avec  $p = 1$ , on retrouve la loi normale usuelle de moyenne  $\mu$  et de variance  $\Sigma = \sigma^2$ .) La formule de Bayes donne

$$\mathbb{P}(G = k \mid \mathbf{x}) = \frac{\pi_k f_k(\mathbf{x})}{\sum_l \pi_l f_l(\mathbf{x})}.$$

La règle de classement la plus naturelle consiste à **choisir le groupe de plus grande probabilité a posteriori**, c'est-à-dire de plus grand  $\pi_k f_k(\mathbf{x})$ . En passant au logarithme (qui conserve l'ordre) :

$$\ln(\pi_k f_k(\mathbf{x})) = \ln \pi_k - \frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (\mathbf{x} - \mu_k)^\top \Sigma_k^{-1} (\mathbf{x} - \mu_k) + \text{constante}.$$

**Le cas particulier de la covariance commune (LDA).** Supposons que **tous les groupes aient la même matrice de covariance** :  $\Sigma_k = \Sigma$ . Les termes  $-\frac{1}{2} \ln |\Sigma|$  et  $-\frac{1}{2} \mathbf{x}^\top \Sigma^{-1} \mathbf{x}$  sont alors identiques pour tous les groupes : ils ne changent pas le classement. En développant le carré, il reste la **fonction discriminante linéaire**

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$$

et l'on classe  $\mathbf{x}$  dans le groupe de plus grand  $\delta_k$ . La frontière entre deux groupes  $k$  et  $l$  est l'ensemble des  $\mathbf{x}$  où  $\delta_k = \delta_l$  : c'est une équation du **premier degré** en  $\mathbf{x}$ , donc un **hyperplan**. D'où le nom d'**analyse discriminante linéaire** (LDA, *linear discriminant analysis*). Si l'on abandonne l'hypothèse  $\Sigma_k = \Sigma$ , les termes quadratiques ne s'annulent plus et la frontière est une quadrique : c'est l'**analyse discriminante quadratique** (QDA).

**Un exemple à la main, avec une seule variable.** Yasmine note la satisfaction (de 1 à 5) de chaque cliente. Parmi celles qui **n'ont pas** racheté, la satisfaction moyenne est  $\mu_0 = 3$  ; parmi celles qui ont racheté,  $\mu_1 = 4$ . Dans les deux groupes, l'écart-type est  $\sigma = 0,8$ . Pour deux groupes et une variable, la règle «  $\delta_1(x) > \delta_0(x)$  » équivaut à

$$x > \frac{\mu_0 + \mu_1}{2} + \frac{\sigma^2}{\mu_1 - \mu_0} \ln \frac{\pi_0}{\pi_1}.$$

- **Groupes de même taille** ( $\pi_0 = \pi_1 = 0,5$ ) : le logarithme est nul, et le seuil est le **milieu** des deux moyennes, 3,5. Au-dessus de 3,5, on prédit « rachète ».
- **Beaucoup de non-racheteuses** ( $\pi_0 = 0,7$ ,  $\pi_1 = 0,3$ ) : le seuil devient  $3,5 + \frac{0,64}{1} \ln \frac{0,7}{0,3} \approx 3,5 + 0,64 \times 0,847 \approx 4,04$ . Comme la population compte surtout des non-racheteuses, la règle devient **plus exigeante** avant de prédire « rachète » : le seuil est remonté vers le groupe rare.

```
import numpy as np
import pandas as pd

mu0, mu1, sigma = 3.0, 4.0, 0.8
for pi0 in (0.5, 0.7):
    seuil = (mu0 + mu1) / 2 + sigma**2 / (mu1 - mu0) * np.log(pi0 / (1 - pi0))
    print(f"a priori pi0 = {pi0} : seuil de décision = {seuil:.3f}")

# contrôle : au seuil, les deux probabilités a posteriori sont égales
from scipy.stats import norm
pi0 = 0.7
seuil = (mu0 + mu1) / 2 + sigma**2 / (mu1 - mu0) * np.log(pi0 / (1 - pi0))
p0 = pi0 * norm.pdf(seuil, mu0, sigma)
p1 = (1 - pi0) * norm.pdf(seuil, mu1, sigma)
print("proba a posteriori au seuil :", round(p0 / (p0 + p1), 3), "contre", round(p1 / (p0 + p1), 3))

a priori pi0 = 0.5 : seuil de décision = 3.500
a priori pi0 = 0.7 : seuil de décision = 4.042
proba a posteriori au seuil : 0.5 contre 0.5
```

### 3.5.2 Estimer le modèle

Dans la pratique, les paramètres sont inconnus : on les remplace par leurs estimations sur un échantillon d'apprentissage :

- $\hat{\pi}_k = n_k/n$ , la proportion de chaque groupe ;

- $\hat{\mu}_k$  : la moyenne des observations du groupe  $k$  ;
- $\hat{\Sigma} = \frac{1}{n-K} \sum_k \sum_{i \in G_k} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^\top$  : la **covariance intra-groupe poolée** (pour la LDA), moyenne pondérée des covariances de chaque groupe, avec le diviseur  $n - K$  (le même raisonnement qu'au volume I, section 3.2.3, pour la variance sans biais).

**Le nombre de paramètres.** La LDA estime  $Kp$  moyennes et  $p(p+1)/2$  covariances (une seule matrice) ; la QDA estime  $Kp$  moyennes et  $Kp(p+1)/2$  covariances (une par groupe). Pour  $p = 3$  variables et  $K = 2$  groupes :  $6 + 6 = 12$  paramètres (plus les proportions) pour la LDA, contre  $6 + 12 = 18$  pour la QDA. Avec  $p = 30$  variables, la différence devient énorme (465 paramètres de covariance contre 930). La QDA est plus **flexible** mais plus **variable** : avec peu de données, la LDA, plus simple, est souvent meilleure même quand ses hypothèses ne sont pas tout à fait vraies. C'est le compromis biais-variance du volume I.

**Application : prédire le rachat.** Reprenons le questionnaire de satisfaction (section 3.2.8). Chaque répondante a deux scores (produits, service), un âge, et l'on sait si elle a **racheté dans les 12 mois** (rachat\_12m). On met de côté 362 répondantes pour **tester** la règle sur des données qu'elle n'a jamais vues (l'évaluation sur données de test est approfondie au volume III) ; le reste (850) sert à l'ajuster.

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
c = pd.read_csv("donnees/clients.csv")
q["score_produits"] = q[["q1", "q2", "q3", "q4"]].mean(axis=1)
q["score_service"] = q[["q5", "q6", "q7", "q8"]].mean(axis=1)
d = q[["id_client", "score_produits", "score_service"]].merge(c[["id_client", "age", "rachat_12m"]],
    on="id_client")

X = d[["score_produits", "score_service", "age"]].to_numpy()
y = d["rachat_12m"].to_numpy()
rng = np.random.default_rng(5)
ordre = rng.permutation(len(d))
app, test = ordre[:850], ordre[850:]
print("répondantes :", len(d), "| apprentissage :", len(app), "| test :", len(test))
print("part de rachat dans l'échantillon :", round(y.mean(), 3))
```

```
répondantes : 1212 | apprentissage : 850 | test : 362
part de rachat dans l'échantillon : 0.505
```

Écrivons la LDA nous-mêmes, en trois fonctions :

```
def lda_ajuster(X, y):
    classes = np.unique(y)
    n, K = len(X), len(classes)
    pi = np.array([(y == k).mean() for k in classes])
    mu = np.array([X[y == k].mean(axis=0) for k in classes])
    Sw = sum((X[y == k] - mu[i]).T @ (X[y == k] - mu[i]) for i, k in enumerate(classes)) / (n - K)
    return classes, pi, mu, Sw

def lda_scores(X, modele):
    classes, pi, mu, Sw = modele
    Sinv = np.linalg.inv(Sw)
    return np.column_stack([X @ Sinv @ mu[i] - 0.5 * mu[i] @ Sinv @ mu[i] + np.log(pi[i]) for i in
        range(len(classes))])

def lda_probabilities(X, modele):
    s = lda_scores(X, modele)
    s = s - s.max(axis=1, keepdims=True) # stabilité numérique du softmax
    e = np.exp(s)
    return e / e.sum(axis=1, keepdims=True)
```

```

modele = lda_ajuster(X[app], y[app])
classes, pi, mu, Sw = modele
print("probabilités a priori :", pi.round(3))
print("moyennes (produits, service, âge) :")
print(pd.DataFrame(mu, index=["pas de rachat", "rachat"], columns=["score_produits", "score_service",
↪ "age"]).round(2).to_string())

```

```

probabilités a priori : [0.499 0.501]
moyennes (produits, service, âge) :
      score_produits  score_service    age
pas de rachat      3.46           3.42  36.82
rachat             3.78           3.68  35.60

```

Comparons à la bibliothèque, puis évaluons sur les 362 répondantes de test, en ajoutant la QDA :

```

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.metrics import accuracy_score, roc_auc_score

p_main = lda_probas(X[test], modele)[: , 1]
pred_main = classes[lda_probas(X[test], modele).argmax(axis=1)]
lda_sk = LinearDiscriminantAnalysis().fit(X[app], y[app])
qda_sk = QuadraticDiscriminantAnalysis().fit(X[app], y[app])

print("LDA à la main : même prédictions que scikit-learn ?", np.array_equal(pred_main,
↪ lda_sk.predict(X[test])))
print("écart maximal entre les probabilités :", np.abs(p_main - lda_sk.predict_proba(X[test])[: ,
↪ 1]).max().round(4))
tab = pd.DataFrame({
    "précision (test)": [accuracy_score(y[test], pred_main), accuracy_score(y[test],
↪ qda_sk.predict(X[test]))],
    "AUC (test)": [roc_auc_score(y[test], p_main), roc_auc_score(y[test],
↪ qda_sk.predict_proba(X[test])[: , 1])],
    index=["LDA", "QDA"])
print(tab.round(3).to_string())
print()
print(pd.crosstab(pd.Series(y[test], name="réel"), pd.Series(pred_main, name="prédit")).to_string())

```

```

LDA à la main : même prédictions que scikit-learn ? True
écart maximal entre les probabilités : 0.0005
précision (test)  AUC (test)
LDA              0.657      0.705
QDA              0.638      0.708

```

```

prédit   0    1
réel
0        121  55
1         69 117

```

**Lire la précision et l'AUC.** La **précision** est la proportion de bonnes prédictions. L'**AUC** (aire sous la courbe ROC) mesure la capacité du modèle à **ordonner** les individus : c'est la probabilité qu'une cliente qui rachète reçoive un score plus élevé qu'une cliente qui ne rachète pas (0,5 : hasard ; 1 : parfait). Ces deux mesures et leurs pièges font l'objet du volume III ; retenez simplement ici qu'on les calcule **sur des données de test**, jamais sur celles qui ont servi à l'ajustement.

La règle écrite à la main reproduit celle de la bibliothèque (mêmes prédictions, probabilités identiques à 0,0005 près). Le résultat est modeste : la matrice de confusion montre 121 + 117 = 238 bonnes prédictions sur 362, soit environ deux sur trois (65,7 %). C'est mieux que le hasard (la précision d'un tirage à pile ou face serait de 50 %) mais loin d'une prédiction fiable : 55 non-



racheteuses sont prédites racheteuses, et 69 racheteuses passent inaperçues. C'est normal : la satisfaction mesurée n'est qu'un déterminant du rachat parmi d'autres, et celui-ci est intrinsèquement incertain. Notez aussi que la **QDA ne fait pas mieux** que la LDA (63,8 % contre 65,7 % de précision, des AUC quasi identiques) : ses 6 paramètres de covariance supplémentaires n'apportent rien. L'écart de 2 points est de l'ordre du bruit d'un échantillon de test de 362 personnes, il ne faut donc pas en tirer plus que : « elle ne fait pas mieux ». Cela suggère que l'hypothèse de covariance commune est raisonnable ici.

### 3.5.3 LDA et régression logistique : deux routes vers la même frontière

Calculons le **logarithme du rapport des probabilités a posteriori** (la « log-cote ») de deux groupes, avec la covariance commune :

$$\ln \frac{\mathbb{P}(G = 1 | \mathbf{x})}{\mathbb{P}(G = 0 | \mathbf{x})} = \delta_1(\mathbf{x}) - \delta_0(\mathbf{x}) = \underbrace{\ln \frac{\pi_1}{\pi_0} - \frac{1}{2}(\mu_1 + \mu_0)^\top \beta + \beta^\top \mathbf{x}}_{\beta_0}, \quad \beta = \Sigma^{-1}(\mu_1 - \mu_0).$$

(on vérifie que  $\frac{1}{2}(\mu_0^\top \Sigma^{-1} \mu_0 - \mu_1^\top \Sigma^{-1} \mu_1) = -\frac{1}{2}(\mu_1 + \mu_0)^\top \Sigma^{-1}(\mu_1 - \mu_0)$ .) La log-cote est donc **linéaire en x** : c'est précisément la forme du modèle de la régression logistique de la section 2.2,  $\ln \frac{p}{1-p} = \beta_0 + \beta^\top \mathbf{x}$ .

Les deux méthodes ont donc **la même forme**, mais elles ne **calculent pas** les coefficients de la même façon :

- la **LDA** estime les moyennes et la covariance, puis en déduit  $\beta = \hat{\Sigma}^{-1}(\hat{\mu}_1 - \hat{\mu}_0)$  : elle modélise **comment les x sont distribués dans chaque groupe** (modèle *génératif*) ;
- la **régression logistique** maximise directement la vraisemblance de  $P(G | \mathbf{x})$  (modèle *discriminatif*) et ne dit rien de la distribution des x.

Comparons les coefficients sur nos données :

```
from sklearn.linear_model import LogisticRegression

beta = np.linalg.solve(Sw, mu[1] - mu[0])
beta0 = np.log(pi[1] / pi[0]) - 0.5 * (mu[1] + mu[0]) @ beta
logit = LogisticRegression(C=1e6, max_iter=1000).fit(X[app], y[app])          # C très grand : pas de
↪ régularisation

coef = pd.DataFrame({"LDA (formule)": np.r_[beta0, beta],
                    "LDA (scikit-learn)": np.r_[lda_sk.intercept_, lda_sk.coef_.ravel()],
                    "régression logistique": np.r_[logit.intercept_, logit.coef_.ravel()]},
                    index=["constante", "score_produits", "score_service", "age"])
print(coef.round(3).to_string())
print("AUC de la régression logistique (test) :", round(roc_auc_score(y[test],
↪ logit.predict_proba(X[test])[:, 1]), 3))
```

|                                                | LDA (formule) | LDA (scikit-learn) | régression logistique |
|------------------------------------------------|---------------|--------------------|-----------------------|
| constante                                      | -3.345        | -3.353             | -3.341                |
| score_produits                                 | 0.663         | 0.664              | 0.663                 |
| score_service                                  | 0.401         | 0.402              | 0.399                 |
| age                                            | -0.013        | -0.013             | -0.013                |
| AUC de la régression logistique (test) : 0.705 |               |                    |                       |

Les trois colonnes sont presque identiques. Quand les données sont vraiment gaussiennes avec une covariance commune, la LDA est légèrement **plus efficace** (elle exploite l'hypothèse de forme) ; quand elles ne le sont pas, la régression logistique est plus **robuste** (elle ne fait pas d'hypothèse sur la loi des x). En pratique, avec des variables qui ne sont pas gaussiennes (une

variable 0/1 comme l'offre de bienvenue, par exemple), on préfère la **régression logistique** (section 2.2).

**Révélation.** Dans la simulation, la log-cote de rachat dépend, par unité de facteur latent, de +0,45 pour le facteur « produits », +0,35 pour le facteur « service », et de −0,015 par année d'âge (plus l'effet de l'offre de bienvenue, que nous n'avons pas inclus ici). Les coefficients des **scores** ne sont pas directement comparables à ceux des **facteurs** : un facteur latent a un écart-type de 1, un score est une moyenne de notes de 1 à 5 dont l'écart-type est bien plus petit, de sorte qu'une unité de score correspond à beaucoup plus d'unités de facteur. Ce que l'on peut comparer, c'est le **signe** et l'**ordre** : le score « produits » pèse plus que le score « service », comme les facteurs, et l'âge a un effet négatif d'environ −0,013 par an, très proche des −0,015 programmés (l'âge, lui, est mesuré sans erreur).

### 3.5.4 Fisher : la projection qui sépare le mieux les groupes

Il existe une autre façon, géométrique, d'arriver à la LDA, due à R. A. Fisher. Au lieu d'un modèle de probabilité, on cherche **la direction a sur laquelle projeter les données** pour que les groupes soient le plus séparés possible, c'est-à-dire pour maximiser le rapport

$$J(\mathbf{a}) = \frac{\mathbf{a}^T B \mathbf{a}}{\mathbf{a}^T W \mathbf{a}} = \frac{\text{variance entre les groupes}}{\text{variance à l'intérieur des groupes}},$$

où  $B$  est la covariance **entre** groupes (dispersion des moyennes de groupes autour de la moyenne générale) et  $W$  la covariance **intra**-groupe (la matrice  $\hat{\Sigma}$  de plus haut). En annulant le gradient, on tombe sur un **problème aux valeurs propres généralisé** (comme pour l'ACP, section 3.1.3) :  $W^{-1}B\mathbf{a} = \lambda\mathbf{a}$ . Avec  $K$  groupes, la matrice  $B$  est de rang au plus  $K - 1$  : il y a **au plus  $K - 1$  axes discriminants** utiles. La LDA devient alors une technique de **réduction de dimension supervisée** : contrairement à l'ACP, qui cherche la direction de plus grande variance **totale**, elle cherche la direction qui **sépare les groupes connus**.

Illustrons-le avec les **600 clientes simulées** de la section 3.3, dont on connaît les trois profils (occasionnelles, fidèles, cadeaux). Les variables sont leurs deux caractéristiques standardisées.

```
Sb_n = np.zeros((2, 2))
moy_gen = Zs.mean(axis=0)
Sw_n = np.zeros((2, 2))
for k in range(3):
    Zk = Zs[vrais == k]
    mk = Zk.mean(axis=0)
    Sb_n += len(Zk) * np.outer(mk - moy_gen, mk - moy_gen)
    Sw_n += (Zk - mk).T @ (Zk - mk)
valeurs, vecteurs = np.linalg.eig(np.linalg.solve(Sw_n, Sb_n))
valeurs = np.sort(valeurs.real)[::-1]
print("valeurs propres de W^-1 B :", valeurs.round(3))
print("part du pouvoir discriminant (à la main) :", (valeurs / valeurs.sum()).round(3))

lda3 = LinearDiscriminantAnalysis().fit(Zs, vrais)
print("part du pouvoir discriminant (scikit-learn):", lda3.explained_variance_ratio_.round(3))
print("précision sur les données d'apprentissage : LDA =", round(lda3.score(Zs, vrais), 3),
      " QDA =", round(QuadraticDiscriminantAnalysis().fit(Zs, vrais).score(Zs, vrais), 3))
```

```
valeurs propres de W^-1 B : [6.982 3.679]
part du pouvoir discriminant (à la main) : [0.655 0.345]
part du pouvoir discriminant (scikit-learn): [0.655 0.345]
précision sur les données d'apprentissage : LDA = 0.983 QDA = 0.992
```

Avec trois groupes et deux variables, il y a deux axes discriminants (au plus  $K - 1 = 2$ ). Le premier

porte les deux tiers du pouvoir séparateur (65,5 %), le second le tiers restant (34,5 %) : les deux axes sont utiles, ce qui est naturel puisque les trois profils s'étalent dans les deux directions du plan. Les deux méthodes classent très bien (plus de 98 % des 600 clientes). La QDA fait **un peu mieux** (99,2 % contre 98,3 %, soit cinq clientes de plus bien classées) : les trois profils n'ont pas la même dispersion (les clientes « cadeaux » ont un panier beaucoup plus variable que les autres), l'hypothèse de covariance commune est donc fautive, et des frontières courbes s'adaptent un peu mieux. Prudence : il s'agit de la précision sur les données d'**apprentissage** (voir le piège 5), et l'écart est faible.

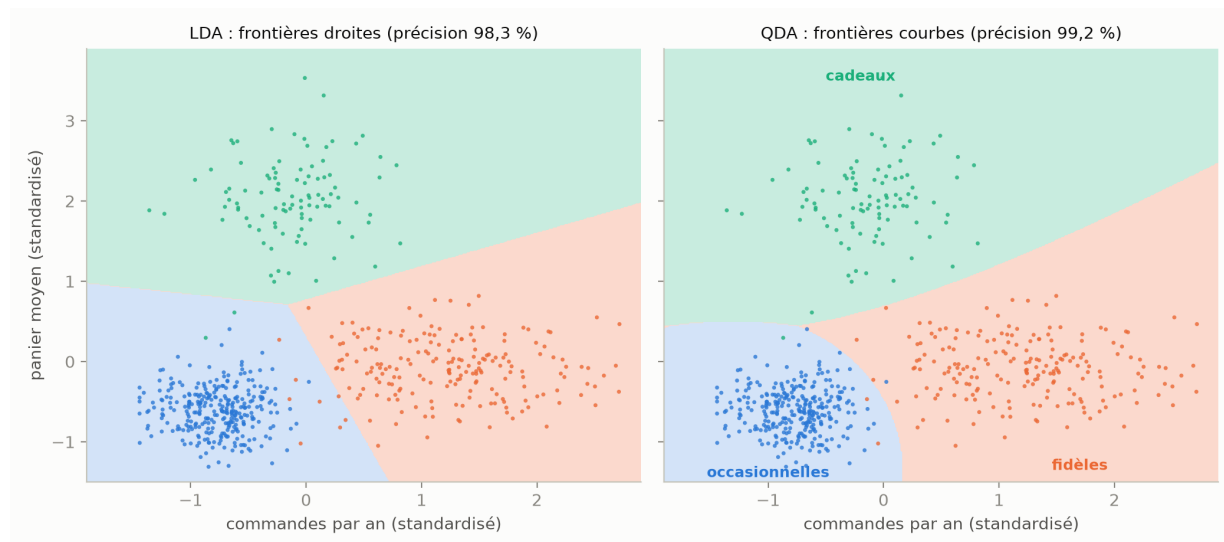


Figure 35: Frontières de décision de la LDA (à gauche, droites) et de la QDA (à droite, courbes) sur les 600 clientes simulées de la section 3.3. Chaque point est coloré par son vrai profil ; la couleur de fond est la classe prédite.

### 3.5.5 Les pièges de l'analyse discriminante

- 1. L'hypothèse gaussienne.** Les variables d'un même groupe doivent être à peu près normales. Elle est violée par des variables binaires, des comptages, des variables très asymétriques (montants) : transformez-les (par exemple en logarithme, volume I, section 3.1.3) ou utilisez la régression logistique (section 2.2).
- 2. La covariance commune.** À vérifier (comparez les covariances de chaque groupe) : si elle est fautive et que l'on a beaucoup de données, préférez la QDA ; sinon, la LDA est plus stable.
- 3. Les probabilités a priori.** Elles déplacent la frontière (voir l'exemple à la main). Par défaut, on prend les proportions de l'échantillon d'apprentissage : cela suppose que les futurs individus auront les mêmes proportions. Si l'on classe des clientes dont la composition diffère de celle de l'échantillon, il faut les modifier.
- 4. Beaucoup de variables, peu d'individus.**  $\hat{\Sigma}$  devient instable, puis non inversible quand  $p \geq n$ . On régularise (méthodes du volume III) ou on réduit d'abord la dimension par ACP.
- 5. L'évaluation sur les données d'apprentissage est trompeuse.** Une règle ajustée sur un échantillon classe presque toujours mieux cet échantillon que de nouvelles données : on évalue sur un échantillon de test ou par validation croisée (volume III).

#### À retenir

- Avec des groupes gaussiens, la **règle de Bayes** classe un individu dans le groupe de plus grande probabilité a posteriori.

- **LDA** (covariance commune) : fonction discriminante  $\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$ , **frontières linéaires** ; **QDA** (une covariance par groupe) : frontières **quadratiques**, plus flexible mais plus variable.
- La log-cote de la LDA est **linéaire** : même forme que la **régression logistique** (section 2.2), estimée autrement (modèle génératif contre discriminatif).
- **Fisher** : la LDA maximise le rapport variance entre groupes / variance intra-groupe ; avec  $K$  groupes, au plus  $K - 1$  axes discriminants.
- Les probabilités a priori déplacent le seuil ; on évalue toujours sur des données de test.

### 3.6 Exercices corrigés

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 13 et 14 portent sur les sections optionnelles 3.5 et 3.4.

#### Énoncés

**Exercice 1 ★ (ACP à la main).** Cinq clientes ont passé 4, 5, 6, 7, 8 commandes dans l'année, pour un panier moyen respectif de 4, 3, 6, 7, 5 dizaines de DT. (a) Centrez les données et calculez la matrice de covariance. (b) Trouvez ses valeurs propres et ses vecteurs propres unitaires. (c) Quelle part de la variance la première composante explique-t-elle ? (d) Calculez les scores des cinq clientes sur cette composante. (e) Vérifiez par le code.

**Exercice 2 ★ (lire un éboulis).** Une ACP normée sur 5 variables a donné les valeurs propres 2,6; 1,1; 0,7; 0,4; 0,2. (a) Quelle est la variance totale ? (b) Calculez la part de variance de chaque composante et les parts cumulées. (c) Combien de composantes retient la règle de Kaiser ? Combien faut-il pour atteindre 80 % de la variance ? (d) Pourquoi les deux critères ne donnent-ils pas la même réponse, et lequel préférez-vous ?

**Exercice 3 ★ (les unités).** Dans la table `clients.csv`, on exprime la dépense annuelle en **centimes** de dinar au lieu de dinars. (a) Que devient la part de variance de la première composante d'une ACP **non standardisée** ? (b) Et d'une ACP **standardisée** ? Justifiez, puis vérifiez par le code.

**Exercice 4 ★★ (propriétés des scores).** Soit  $Z$  un tableau de variables standardisées,  $R$  sa matrice de corrélation,  $\mathbf{v}_j$  et  $\lambda_j$  ses éléments propres, et  $\mathbf{t}_j = Z\mathbf{v}_j$  les scores. (a) Démontrez que  $\text{Var}(\mathbf{t}_j) = \lambda_j$  et que  $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = 0$  pour  $i \neq j$ . (b) Démontrez que la corrélation entre la variable  $z_i$  et le score  $\mathbf{t}_j$  vaut  $v_{ij} \sqrt{\lambda_j}$ . (c) Vérifiez les deux propriétés numériquement sur les cinq variables numériques de `clients.csv` standardisées.

**Exercice 5 ★★ (reconstruction).** Sur les huit questions standardisées du questionnaire : (a) combien de composantes faut-il pour restituer au moins 80 % de la variance ? (b) Calculez l'erreur quadratique moyenne par case (RMSE) de la reconstruction avec 2 composantes, et comparez-la à l'erreur d'une reconstruction **triviale** par la moyenne (c'est-à-dire avec 0 composante). (c) Vérifiez que l'erreur totale vaut  $(n - 1) \sum_{j>k} \lambda_j$ .

**Exercice 6 ★★ (modèle factoriel à un facteur).** Trois questions sur la rapidité de la livraison ont les corrélations  $r_{12} = 0,48$ ,  $r_{13} = 0,40$ ,  $r_{23} = 0,30$ . (a) Calculez les saturations d'un modèle à un facteur, les communalités et les unicités. (b) Quelle question est le meilleur indicateur du facteur ? (c) Que se passe-t-il si  $r_{23}$  valait 0,15 ? Qu'est-ce que cela signifie pour le modèle ?

**Exercice 7 ★★ (invariance par rotation).** Les saturations d'un modèle à deux facteurs pour

quatre questions sont  $\Lambda = \begin{pmatrix} 0,7 & 0,3 \\ 0,6 & 0,4 \\ 0,2 & 0,8 \\ 0,3 & 0,7 \end{pmatrix}$ . (a) Calculez les communalités. (b) On applique une

rotation d'angle  $\theta = 30^\circ$  :  $\Lambda^* = \Lambda T$  avec  $T = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ . Calculez  $\Lambda^*$  et vérifiez que les communalités et la matrice  $\Lambda\Lambda^\top$  sont inchangées. (c) Pourquoi cela empêche-t-il de dire « la vraie valeur » d'une saturation ?

**Exercice 8 ★★ (alpha de Cronbach).** Une échelle de trois questions standardisées a une corrélation moyenne de 0,5 entre questions. (a) Calculez la variance de la somme des trois questions, puis l'alpha de Cronbach. (b) Combien de questions de même corrélation moyenne faut-il pour atteindre  $\alpha \geq 0,9$  ? On rappelle la formule de Spearman-Brown : pour des questions standardisées,  $\alpha = \frac{k\bar{r}}{1 + (k-1)\bar{r}}$ .

**Exercice 9 ★★ (k-means : le piège du minimum local).** Six paniers (en dizaines de DT) : 2, 3, 4, 10, 12, 20. On cherche  $k = 2$  groupes. (a) Déroulez l'algorithme de Lloyd à partir des centres (2; 20) et calculez la variance intra-groupe finale  $W$ . (b) Recommencez à partir des centres (2; 10). (c) Comparez les deux solutions : que conclure sur l'algorithme ?

**Exercice 10 ★★ (silhouette).** Pour la partition  $\{2, 3, 4\}$  et  $\{10, 12, 20\}$  de l'exercice 9, calculez à la main la silhouette des points 4, 10 et 20, puis la silhouette moyenne (code). Quel point est le moins bien classé ?

**Exercice 11 ★★ (critères d'agrégation).** Quatre clientes  $A, B, C, D$  ont les distances deux à deux suivantes :  $AB = 2, AC = 5, AD = 9, BC = 3, BD = 7, CD = 4,5$ . Construisez à la main l'arbre de la classification hiérarchique pour les critères **simple**, **complet** et **moyen**, en donnant les fusions successives et leurs hauteurs. Vérifiez avec `scipy`.

**Exercice 12 ★★★ (une segmentation, ou pas ?).** Yasmine veut des segments de clientes selon deux critères seulement : le nombre de commandes par an et le panier moyen (clientes ayant commandé, variables standardisées). (a) Calculez la silhouette moyenne des k-means pour  $k = 2, \dots, 6$ . (b) Évaluez la stabilité (indice de Rand ajusté entre deux ré-estimations sur des échantillons bootstrap) du découpage en  $k = 3$ . (c) Que conseillez-vous à Yasmine ? Rédigez deux phrases pour elle, sans jargon.

**Exercice 13 ★★ (analyse discriminante, section 3.5).** Les clientes à petit panier ont un panier moyen de 40 DT, celles à gros panier de 55 DT, avec un écart-type de 10 DT dans chaque groupe, et les gros paniers représentent 20 % des commandes. (a) À partir de quel panier la règle de Bayes (LDA à une variable) classe-t-elle une commande dans « gros panier » ? (b) Quelle est la probabilité a posteriori d'être un gros panier pour une commande de 55 DT ? (c) Commentez : une commande de 55 DT, égale à la moyenne du groupe « gros panier », est-elle classée dans ce groupe ?

**Exercice 14 ★★★ (analyse des correspondances, section 3.4).** Un tableau croise le canal (Instagram, Boutique) et trois tranches de délai de livraison (court, moyen, long) :

|           | court | moyen | long |
|-----------|-------|-------|------|
| Instagram | 40    | 30    | 10   |
| Boutique  | 10    | 30    | 30   |

(a) Calculez  $\chi^2$  et l'inertie totale  $\chi^2/n$ . (b) Combien d'axes non triviaux l'AC aura-t-elle ? (c) Vérifiez par le code que la somme des inerties des axes est égale à  $\chi^2/n$ , et dites quelle

modalité de délai est la plus associée à Instagram.

## Corrigés

**Corrigé 1.** (a) Moyennes :  $\bar{x} = 6$ ,  $\bar{y} = 5$ . Écarts :  $x_c = (-2, -1, 0, 1, 2)$  et  $y_c = (-1, -2, 1, 2, 0)$ . Variances :  $s_{xx} = 10/4 = 2,5$ ,  $s_{yy} = (1+4+1+4+0)/4 = 2,5$ ; covariance :  $s_{xy} = (2+2+0+2+0)/4 =$

1,5. Donc  $S = \begin{pmatrix} 2,5 & 1,5 \\ 1,5 & 2,5 \end{pmatrix}$ .

(b)  $\det(S - \lambda I) = (2,5 - \lambda)^2 - 1,5^2 = 0$  donne  $\lambda = 2,5 \pm 1,5$ , soit  $\lambda_1 = 4$  et  $\lambda_2 = 1$ . Pour  $\lambda_1 = 4$  :  $-1,5v_1 + 1,5v_2 = 0$ , donc  $\mathbf{v}_1 = (1, 1)/\sqrt{2}$ . Pour  $\lambda_2 = 1$  :  $\mathbf{v}_2 = (1, -1)/\sqrt{2}$ .

(c) Variance totale = 5 ; la première composante en explique  $4/5 = 80\%$ .

(d)  $t_{i1} = (x_{c,i} + y_{c,i})/\sqrt{2}$  : les sommes  $x_c + y_c$  valent  $(-3, -3, 1, 3, 2)$ , donc les scores sont  $(-3, -3, 1, 3, 2)/\sqrt{2} \approx (-2,12; -2,12; 0,71; 2,12; 1,41)$ . On vérifie que leur variance est  $(9 + 9 + 1 + 9 + 4)/(2 \times 4) = 32/8 = 4 = \lambda_1$  ✓.

```
import numpy as np
X = np.array([[4, 4], [5, 3], [6, 6], [7, 7], [8, 5]], dtype=float)
Xc = X - X.mean(axis=0)
S = np.cov(Xc, rowvar=False)
w, V = np.linalg.eigh(S)
o = np.argsort(w)[-1:]
w, V = w[o], V[:, o]
print("S =", S.tolist())
print("valeurs propres :", w.round(3), "| parts :", (w / w.sum()).round(3))
print("scores sur CP1 (au signe près) :", np.abs(Xc @ V[:, 0]).round(2))
```

```
S = [[2.5, 1.5], [1.5, 2.5]]
valeurs propres : [4. 1.] | parts : [0.8 0.2]
scores sur CP1 (au signe près) : [2.12 2.12 0.71 2.12 1.41]
```

**Corrigé 2.** (a) Pour une ACP normée, la variance totale est le nombre de variables : 5 (et c'est bien la somme  $2,6 + 1,1 + 0,7 + 0,4 + 0,2 = 5$ ). (b) Parts : 52 %, 22 %, 14 %, 8 %, 4 % ; cumul : 52, 74, 88, 96, 100 %. (c) Kaiser garde les valeurs propres  $> 1$  : **deux** composantes (2,6 et 1,1). Pour atteindre 80 %, il en faut **trois** (74 % avec deux, 88 % avec trois). (d) Les deux critères répondent à des questions différentes : Kaiser compare chaque composante à « une variable d'origine » ; la règle des 80 % fixe un objectif de restitution. Ici, la deuxième composante (1,1) dépasse de très peu le seuil de 1 : c'est le cas où l'**analyse parallèle** est utile, car elle compare à ce que ferait le bruit (qui dépasse souvent 1 pour la deuxième valeur propre, comme au 3.1.6).

```
w = np.array([2.6, 1.1, 0.7, 0.4, 0.2])
print("variance totale :", w.sum())
print("parts (%)      :", (100 * w / w.sum()).round(0))
print("cumul (%)      :", (100 * w.cumsum() / w.sum()).round(0))
print("Kaiser (> 1):", int((w > 1).sum()), "| composantes pour 80 % :", int(np.argmax(w.cumsum() /
↳ w.sum() >= 0.8) + 1))
```

```
variance totale : 5.000000000000001
parts (%)       : [52. 22. 14.  8.  4.]
cumul (%)       : [ 52.  74.  88.  96. 100.]
Kaiser (> 1): 2 | composantes pour 80 % : 3
```

**Corrigé 3.** (a) Multiplier une variable par 100 multiplie sa variance par 10 000 : la dépense, qui dominait déjà, **écrase** tout le reste. La première composante, qui expliquait déjà 98,8 % de la variance en dinars, en explique alors pratiquement 100 % (le code arrondit à 1,0) et pointe sur la dépense. (b) Avec la standardisation, chaque variable est divisée par son écart-type : **changer d'unité ne**



**change rien** (la variable standardisée est la même). Les parts de variance sont identiques à celles du 3.1.5.

```
import pandas as pd
c = pd.read_csv("donnees/clients.csv")
cols = ["age", "nb_commandes_an", "panier_moyen", "depense_annuelle", "duree_mois"]
X0 = c[cols].copy()
X1 = X0.copy(); X1["depense_annuelle"] = X1["depense_annuelle"] * 100      # en centimes

def parts(M, standardiser):
    M = (M - M.mean()) / M.std() if standardiser else M - M.mean()
    w = np.sort(np.linalg.eigvalsh(np.cov(M.to_numpy(), rowvar=False)))[::-1]
    return w / w.sum()

print("non standardisé, dinars      :", parts(X0, False)[0].round(5))
print("non standardisé, centimes    :", parts(X1, False)[0].round(5))
print("standardisé, dinars          :", parts(X0, True).round(3))
print("standardisé, centimes        :", parts(X1, True).round(3))

non standardisé, dinars      : 0.98789
non standardisé, centimes    : 1.0
standardisé, dinars          : [0.442 0.214 0.194 0.125 0.025]
standardisé, centimes        : [0.442 0.214 0.194 0.125 0.025]
```

**Corrigé 4.** (a) Les colonnes de  $Z$  sont centrées, donc  $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = \frac{1}{n-1} \mathbf{v}_i^\top Z^\top Z \mathbf{v}_j = \mathbf{v}_i^\top R \mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j$  (car  $R \mathbf{v}_j = \lambda_j \mathbf{v}_j$ ). Les vecteurs propres d'une matrice symétrique sont orthonormés :  $\mathbf{v}_i^\top \mathbf{v}_j = \delta_{ij}$ . D'où  $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = \lambda_j \delta_{ij}$ .

(b)  $\text{Cov}(\mathbf{z}_i, \mathbf{t}_j) = \frac{1}{n-1} \mathbf{z}_i^\top Z \mathbf{v}_j = (R \mathbf{v}_j)_i = \lambda_j v_{ij}$ . Comme  $\text{Var}(\mathbf{z}_i) = 1$  et  $\text{Var}(\mathbf{t}_j) = \lambda_j$ , la corrélation vaut  $\frac{\lambda_j v_{ij}}{1 \times \sqrt{\lambda_j}} = v_{ij} \sqrt{\lambda_j}$ .

(c)

```
Z = ((X0 - X0.mean()) / X0.std()).to_numpy()
R = np.corrcoef(Z, rowvar=False)
w, V = np.linalg.eigh(R)
o = np.argsort(w)[::-1]
w, V = w[o], V[:, o]
T = Z @ V
print("variances des scores :", T.var(axis=0, ddof=1).round(4))
print("valeurs propres      :", w.round(4))
corr_scores = np.corrcoef(T, rowvar=False)
print("plus grande corrélation entre deux scores différents :", np.abs(corr_scores -
    np.eye(5)).max().round(10))
cor_var_comp = np.array([[np.corrcoef(Z[:, i], T[:, j])[0, 1] for j in range(5)] for i in range(5)])
print("cor(variable, composante) = v * sqrt(lambda) ?", np.allclose(cor_var_comp, V * np.sqrt(w)))

variances des scores : [2.2115 1.0702 0.9684 0.6256 0.1242]
valeurs propres      : [2.2115 1.0702 0.9684 0.6256 0.1242]
plus grande corrélation entre deux scores différents : 0.0
cor(variable, composante) = v * sqrt(lambda) ? True
```

**Corrigé 5.** (a) On lit les parts cumulées : 0,748 avec quatre composantes, 0,818 avec cinq : il en faut **cinq**. C'est beaucoup pour huit questions, et c'est instructif : la règle des 80 % est ici bien moins sévère que Kaiser et l'analyse parallèle, qui retiennent deux composantes ; le questionnaire contient deux dimensions nettes, mais chaque question a aussi une grande part de variance qui lui est propre (3.2). (b) La reconstruction avec  $k$  composantes a pour erreur  $(n-1) \sum_{j>k} \lambda_j$  ; la RMSE par case vaut  $\sqrt{\text{erreur}/(np)}$ . Avec 0 composante, on reconstruit chaque case par 0 (la moyenne des variables standardisées) et l'erreur est la variance totale  $(n-1)p$  ; la RMSE vaut alors



à peu près 1 (l'écart-type des variables standardisées). (c) Vérification par le code : avec deux composantes, la RMSE tombe de 1,000 à 0,643, et l'erreur totale (4007,5) coïncide exactement avec la formule.

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
Q = q[[f"q{j}" for j in range(1, 9)]]
Zq = ((Q - Q.mean()) / Q.std()).to_numpy()
n, p = Zq.shape
w, V = np.linalg.eigh(np.corrcoef(Zq, rowvar=False))
o = np.argsort(w)[-1]
w, V = w[o], V[:, o]

cumul = (w / w.sum()).cumsum()
print("composantes pour au moins 80 % :", int(np.argmax(cumul >= 0.8)) + 1, "(parts cumulées :",
      ↪ cumul.round(3).tolist(), ")")

for k in (0, 2):
    Vk = V[:, :k]
    erreur = ((Zq - Zq @ Vk @ Vk.T) ** 2).sum()
    print(f"k = {k} : erreur totale = {erreur:.1f}, théorie (n-1) x somme(valeurs jetées) = {(n - 1) *
          ↪ w[k:].sum():.1f}, RMSE par case = {np.sqrt(erreur / (n * p)):.3f}")

composantes pour au moins 80 % : 5 (parts cumulées : [0.35, 0.586, 0.675, 0.748, 0.818, 0.887, 0.946,
      ↪ 1.0] )
k = 0 : erreur totale = 9688.0, théorie (n-1) x somme(valeurs jetées) = 9688.0, RMSE par case = 1.000
k = 2 : erreur totale = 4007.5, théorie (n-1) x somme(valeurs jetées) = 4007.5, RMSE par case = 0.643
```

**Corrigé 6.** (a)  $\ell_1^2 = \frac{r_{12}r_{13}}{r_{23}} = \frac{0,48 \times 0,40}{0,30} = 0,64$ , donc  $\ell_1 = 0,8$  ;  $\ell_2 = 0,48/0,8 = 0,6$  ;  $\ell_3 = 0,40/0,8 = 0,5$ . Communalités : 0,64; 0,36; 0,25. Unicités : 0,36; 0,64; 0,75. Contrôle :  $\ell_2\ell_3 = 0,30 = r_{23}$  ✓. (b) La question 1 : saturation la plus forte (0,8) et unicité la plus faible. (c) Avec  $r_{23} = 0,15$  :  $\ell_1^2 = 0,48 \times 0,40/0,15 = 1,28 > 1$ . Une communalité supérieure à 1 est **impossible** (elle dépasserait la variance de la variable, soit une unicité négative  $1 - 1,28 = -0,28$ ) : c'est un **cas de Heywood**. Il signale que le modèle à un facteur est incompatible avec ces corrélations (par exemple, la question 1 est trop corrélée aux deux autres par rapport à leur corrélation mutuelle), ou que l'échantillon est trop petit.

```
for r23 in (0.30, 0.15):
    l1_carre = 0.48 * 0.40 / r23
    print(f"r23 = {r23} : l1^2 = {l1_carre:.2f}", "-> impossible (cas de Heywood)" if l1_carre > 1
          ↪ else f"-> l1 = {np.sqrt(l1_carre):.2f}")
```

```
r23 = 0.3 : l1^2 = 0.64 -> l1 = 0.80
r23 = 0.15 : l1^2 = 1.28 -> impossible (cas de Heywood)
```

**Corrigé 7.** (a) Communalités :  $0,49 + 0,09 = 0,58$  ;  $0,36 + 0,16 = 0,52$  ;  $0,04 + 0,64 = 0,68$  ;  $0,09 + 0,49 = 0,58$ . (b) Voir le code :  $\Lambda^*$  est différente de  $\Lambda$  mais communalités et  $\Lambda\Lambda^T$  sont identiques, puisque  $\Lambda^*\Lambda^{*T} = \Lambda T T^T \Lambda^T = \Lambda\Lambda^T$ . (c) Comme l'ajustement est strictement le même pour tous les angles, les données **ne peuvent pas choisir** entre « la saturation de la question 1 sur le facteur 1 vaut 0,7 » et « elle vaut 0,66 » : seule la convention de rotation (varimax, etc.) fixe une valeur.

```
Lam = np.array([[0.7, 0.3], [0.6, 0.4], [0.2, 0.8], [0.3, 0.7]])
th = np.deg2rad(30)
T = np.array([[np.cos(th), -np.sin(th)], [np.sin(th), np.cos(th)]])
Lam2 = Lam @ T
print("communalités avant :", (Lam**2).sum(axis=1).round(3))
print("communalités après :", (Lam2**2).sum(axis=1).round(3))
print("Lambda* =\n", Lam2.round(3))
print("même Lambda Lambda' :", np.allclose(Lam @ Lam.T, Lam2 @ Lam2.T))
```

```
communalités avant : [0.58 0.52 0.68 0.58]
communalités après : [0.58 0.52 0.68 0.58]
Lambda* =
[[ 0.756 -0.09 ]
 [ 0.72   0.046]
 [ 0.573  0.593]
 [ 0.61   0.456]]
même Lambda Lambda' : True
```

**Corrigé 8.** (a) Variance de la somme de  $k = 3$  variables de variance 1 et de covariance 0,5 :  $3 + 3 \times 2 \times 0,5 = 6$ . Alpha :  $\frac{3}{2} \left(1 - \frac{3}{6}\right) = 0,75$ . Cohérent avec Spearman-Brown :  $\frac{3 \times 0,5}{1 + 2 \times 0,5} = 0,75$  ✓. (b) On résout  $\frac{0,5k}{1 + 0,5(k-1)} = 0,9$ , soit  $0,5k = 0,9(0,5 + 0,5k) = 0,45 + 0,45k$ , donc  $0,05k = 0,45$  et  $k = 9$ . Il faut **neuf** questions de corrélation moyenne 0,5 pour atteindre  $\alpha = 0,9$  : l'alpha augmente avec la longueur de l'échelle, ce qui rend toute comparaison entre échelles de longueurs différentes délicate.

```
def alpha_std(k, r): return k * r / (1 + (k - 1) * r)
print("alpha, k = 3 :", alpha_std(3, 0.5))
print("k pour alpha >= 0,9 :", next(k for k in range(2, 50) if alpha_std(k, 0.5) >= 0.9), "| alpha(k = 9) =", round(alpha_std(9, 0.5), 3))
```

```
alpha, k = 3 : 0.75
k pour alpha >= 0,9 : 9 | alpha(k = 9) = 0.9
```

**Corrigé 9.** (a) Centres (2; 20). *Affectation* : 2, 3, 4 vont avec 2 ; 10 est à 8 de 2 et à 10 de 20 : il va avec 2 ; 12 est à 10 de 2 et à 8 de 20 : il va avec 20. Groupes {2, 3, 4, 10} et {12, 20}. *Mise à jour* : centres 4,75 et 16. *Affectation* : 10 est à 5,25 de 4,75 et à 6 de 16 ; 12 est à 7,25 et à 4 : rien ne change. Fin.  $W = (2,75^2 + 1,75^2 + 0,75^2 + 5,25^2) + (4^2 + 4^2) = 7,5625 + 3,0625 + 0,5625 + 27,5625 + 32 = 70,75$ . (b) Centres (2; 10) : groupes {2, 3, 4} et {10, 12, 20} ; centres 3 et 14 ; plus rien ne change.  $W = (1 + 0 + 1) + (16 + 4 + 36) = 58$ . (c) Les deux exécutions **convergent**, mais vers deux solutions différentes (70,75 et 58) : la seconde est meilleure. L'algorithme de Lloyd ne trouve qu'un **minimum local**, qui dépend de l'initialisation : d'où les redémarrages multiples et k-means++.

```
pts = np.array([2, 3, 4, 10, 12, 20.0])
for init in ([2.0, 20.0], [2.0, 10.0]):
    centres = np.array(init)
    while True:
        g = np.abs(pts[:, None] - centres[None, :]).argmin(axis=1)
        nouveaux = np.array([pts[g == j].mean() for j in range(2)])
        if np.allclose(nouveaux, centres):
            break
        centres = nouveaux
    W = sum((pts[g == j] - centres[j]) ** 2).sum() for j in range(2))
    print(f"départ {init} -> groupes {g.tolist()}, centres {centres.round(2).tolist()}, W = {W:.2f}")
```

```
départ [2.0, 20.0] -> groupes [0, 0, 0, 0, 1, 1], centres [4.75, 16.0], W = 70.75
départ [2.0, 10.0] -> groupes [0, 0, 0, 1, 1, 1], centres [3.0, 14.0], W = 58.00
```

**Corrigé 10.** Point 4 (groupe {2, 3, 4}) :  $a = (2 + 1)/2 = 1,5$  ;  $b = (6 + 8 + 16)/3 = 10$  ;  $s = (10 - 1,5)/10 = 0,85$ . Point 10 (groupe {10, 12, 20}) :  $a = (2 + 10)/2 = 6$  ;  $b = (8 + 7 + 6)/3 = 7$  ;  $s = (7 - 6)/7 \approx 0,143$ . Point 20 :  $a = (10 + 8)/2 = 9$  ;  $b = (18 + 17 + 16)/3 = 17$  ;  $s = (17 - 9)/17 \approx 0,471$ . Le point 10 est le moins bien classé : presque à égale distance des deux groupes (son groupe est étiré par le point extrême 20).

```
from sklearn.metrics import silhouette_samples
s = silhouette_samples(pts.reshape(-1, 1), np.array([0, 0, 0, 1, 1, 1]))
print("silhouettes :", dict(zip(pts.astype(int).tolist(), s.round(3).tolist())))
```

```
print("silhouette moyenne :", s.mean().round(3))
```

```
silhouettes : {2: 0.875, 3: 0.909, 4: 0.85, 10: 0.143, 12: 0.444, 20: 0.471}
silhouette moyenne : 0.615
```

**Corrigé 11.** *Lien simple* (distance minimale entre groupes) : fusion de  $\{A, B\}$  à 2 ; puis  $\{A, B\}-C$  vaut  $\min(5; 3) = 3$ ,  $\{A, B\}-D$  vaut  $\min(9; 7) = 7$ ,  $C-D$  vaut 4,5 : on fusionne  $C$  à  $\{A, B\}$  à la hauteur 3 ; enfin  $D$  à la hauteur  $\min(9; 7; 4,5) = 4,5$ . Hauteurs : 2; 3; 4,5 (effet de **chaîne** :  $A, B, C, D$  s'enchaînent). *Lien complet* (distance maximale) :  $\{A, B\}$  à 2 ; puis  $\{A, B\}-C$  vaut  $\max(5; 3) = 5$ ,  $\{A, B\}-D$  vaut 9,  $C-D$  vaut 4,5 : on fusionne  $\{C, D\}$  à 4,5 ; enfin  $\{A, B\}-\{C, D\}$  vaut  $\max(5; 9; 3; 7) = 9$ . Hauteurs : 2; 4,5; 9. *Lien moyen* :  $\{A, B\}$  à 2 ;  $\{A, B\}-C$  vaut  $(5 + 3)/2 = 4$ ,  $\{A, B\}-D$  vaut 8,  $C-D$  vaut 4,5 : on fusionne  $C$  à  $\{A, B\}$  à 4 ; puis  $D$  à  $(9 + 7 + 4,5)/3 \approx 6,83$ . Hauteurs : 2; 4; 6,83. Les **trois arbres différent** : le critère simple et le moyen rattachent  $C$  à  $\{A, B\}$ , le critère complet préfère former  $\{C, D\}$ .

```
from scipy.cluster.hierarchy import linkage
from scipy.spatial.distance import squareform
D = np.array([[0, 2, 5, 9], [2, 0, 3, 7], [5, 3, 0, 4.5], [9, 7, 4.5, 0]], dtype=float)
for m in ["single", "complete", "average"]:
    L = linkage(squareform(D), method=m)
    print(f"{m:9s} : fusions (indices) = {L[:, :2].astype(int).tolist()}, hauteurs = {L[:, :2].round(3).tolist()}")
```

```
single      : fusions (indices) = [[0, 1], [2, 4], [3, 5]], hauteurs = [2.0, 3.0, 4.5]
complete    : fusions (indices) = [[0, 1], [2, 3], [4, 5]], hauteurs = [2.0, 4.5, 9.0]
average      : fusions (indices) = [[0, 1], [2, 4], [3, 5]], hauteurs = [2.0, 4.0, 6.833]
```

**Corrigé 12.** (a) et (b) par le code, ainsi qu'un point de comparaison indispensable : la silhouette qu'obtiendrait un nuage **sans aucun groupe**, mais de même taille et d'asymétrie comparable (variables log-normales, comme le sont le nombre de commandes et le panier). (c) Voir la discussion sous le code.

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, silhouette_score

act = c[c["nb_commandes_an"] > 0]
Z2 = ((act[["nb_commandes_an", "panier_moyen"]] - act[["nb_commandes_an", "panier_moyen"]].mean())
      / act[["nb_commandes_an", "panier_moyen"]].std()).to_numpy()

# nuage témoin : mêmes effectifs, variables asymétriques (log-normales) corrélées, mais AUCUN groupe
rng = np.random.default_rng(0)
temoin = np.exp(rng.multivariate_normal([0, 0], [[0.4, 0.15], [0.15, 0.3]], len(Z2)))
temoin = (temoin - temoin.mean(axis=0)) / temoin.std(axis=0)

lignes = []
for k in range(2, 7):
    sil_reel = silhouette_score(Z2, KMeans(n_clusters=k, n_init=10, random_state=0).fit(Z2).labels_)
    sil_temoin = silhouette_score(temoin, KMeans(n_clusters=k, n_init=10,
    random_state=0).fit(temoin).labels_)
    lignes.append({"k": k, "silhouette (clientes)": sil_reel, "silhouette (nuage sans groupe)":
    sil_temoin})
print(pd.DataFrame(lignes).set_index("k").round(3).to_string())

ref = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Z2)
rng = np.random.default_rng(4)
ari = []
for _ in range(30):
    idx = rng.integers(0, len(Z2), len(Z2))
    km = KMeans(n_clusters=3, n_init=3, random_state=int(rng.integers(10**6))).fit(Z2[idx])
    ari.append(adjusted_rand_score(ref.labels_, km.predict(Z2)))
print(f"stabilité (k = 3) : ARI moyen = {np.mean(ari):.3f}, minimum = {np.min(ari):.3f}")
```

|                                                        | silhouette (clientes) | silhouette (nuage sans groupe) |
|--------------------------------------------------------|-----------------------|--------------------------------|
| k                                                      |                       |                                |
| 2                                                      | 0.422                 | 0.511                          |
| 3                                                      | 0.450                 | 0.496                          |
| 4                                                      | 0.392                 | 0.420                          |
| 5                                                      | 0.375                 | 0.416                          |
| 6                                                      | 0.358                 | 0.390                          |
| stabilité (k = 3) : ARI moyen = 0.920, minimum = 0.824 |                       |                                |

- (a) La silhouette moyenne des clientes atteint son maximum en  $k = 3$  (0,450). Cette valeur tombe dans la zone « structure faible » (0,25 à 0,50) et l'on pourrait être tenté de conclure à trois familles. Mais le **nuage témoin**, qui ne contient **aucun** groupe, obtient une silhouette **plus élevée à chaque  $k$**  (0,511 en  $k = 2$ , 0,496 en  $k = 3$ ). L'explication : des variables fortement **asymétriques** (longue queue vers les grandes valeurs) font que les k-means découpent le nuage en tranches « cœur dense » / « queue », et la silhouette récompense ce découpage. **Moralité : une silhouette ne se lit jamais seule, il faut une valeur de comparaison obtenue sur des données sans structure** ; ici, les clientes ne montrent aucune structure en groupes au-delà de ce que produit une simple asymétrie.
- (b) La stabilité du découpage en trois groupes est bonne (ARI moyen 0,92, minimum 0,82) : la partition est reproductible. Comme au 3.3.5, **stabilité n'est pas existence** : ce sont des tranches reproductibles d'un nuage continu.
- (c) *Conseil à Yasmine* : « Vos clientes ne forment pas des familles distinctes selon le nombre de commandes et le panier : elles se répartissent sur un continuum, avec beaucoup de petites clientes et quelques très grosses. Pour vos relances, vous pouvez néanmoins utiliser trois tranches pratiques (peu actives, régulières, très actives), à condition de les voir comme des repères commodes, pas comme des types de clientes qui existeraient vraiment. »

**Corrigé 13.** (a)  $\text{Seuil} = \frac{40 + 55}{2} + \frac{10^2}{55 - 40} \ln \frac{0,8}{0,2} = 47,5 + 6,667 \times 1,386 \approx 56,74$  DT. (b)  $\mathbb{P}(\text{gros} | 55) = \frac{0,2f_{55}(55)}{0,8f_{40}(55) + 0,2f_{55}(55)}$  : calculé par le code, environ 0,435. (c) **Non** : malgré un panier égal à la moyenne du groupe « gros panier », la commande est classée « petit panier » ( $0,435 < 0,5$ ). Les gros paniers étant quatre fois plus rares, il faut une commande plus élevée ( $\geq 56,74$ ) pour faire basculer la décision : c'est l'effet de la probabilité a priori (3.5.1).

```
from scipy.stats import norm
mu0, mu1, sig, pi0 = 40, 55, 10, 0.8
seuil = (mu0 + mu1) / 2 + sig**2 / (mu1 - mu0) * np.log(pi0 / (1 - pi0))
print("seuil de décision :", round(seuil, 2), "DT")
for x in (50, 55, 60):
    a = pi0 * norm.pdf(x, mu0, sig); b = (1 - pi0) * norm.pdf(x, mu1, sig)
    print(f"panier de {x} DT : P(gros panier | x) = {b / (a + b):.3f}")

seuil de décision : 56.74 DT
panier de 50 DT : P(gros panier | x) = 0.267
panier de 55 DT : P(gros panier | x) = 0.435
panier de 60 DT : P(gros panier | x) = 0.620
```

**Corrigé 14.** (a)  $n = 150$  ; marges des lignes 80 et 70 ; marges des colonnes 50, 60, 40. Effectifs attendus :  $\frac{80 \times 50}{150} \approx 26,67$ ,  $\frac{80 \times 60}{150} = 32$ ,  $\frac{80 \times 40}{150} \approx 21,33$  pour Instagram, et 23,33, 28, 18,67 pour la boutique.  $\chi^2 = \frac{13,33^2}{26,67} + \frac{2^2}{32} + \frac{11,33^2}{21,33} + \frac{13,33^2}{23,33} + \frac{2^2}{28} + \frac{11,33^2}{18,67} \approx 6,67 + 0,13 + 6,02 + 7,62 + 0,14 + 6,88 \approx 27,5$  ; inertie totale  $\chi^2/n \approx 0,183$ . (b) Un tableau  $2 \times 3$  a  $\min(2, 3) - 1 = 1$  axe non trivial. (c) Voir le code ; la modalité de délai la plus proche d'Instagram sur l'axe est « court » (Instagram a 50 % de délais courts contre 14 % pour la boutique).

```

from scipy import stats
N = np.array([[40, 30, 10], [10, 30, 30]], dtype=float)
chi2 = stats.chi2_contingency(N, correction=False)[0]
P = N / N.sum(); r, cc = P.sum(axis=1), P.sum(axis=0)
S = (P - np.outer(r, cc)) / np.sqrt(np.outer(r, cc))
U, sv, Vt = np.linalg.svd(S, full_matrices=False)
F = (U / np.sqrt(r)[:, None]) * sv
G = (Vt.T / np.sqrt(cc)[:, None]) * sv
print("khi-deux =", round(chi2, 2), "| khi-deux / n =", round(chi2 / N.sum(), 4))
print("inerties des axes :", (sv**2).round(4), "| somme :", (sv**2).sum().round(4))
print("axe 1, lignes   :", pd.Series(F[:, 0], index=["Instagram", "Boutique"]).round(3).to_dict())
print("axe 1, colonnes :", pd.Series(G[:, 0], index=["court", "moyen", "long"]).round(3).to_dict())

khi-deux = 27.46 | khi-deux / n = 0.183
inerties des axes : [0.183 0.   ] | somme : 0.183
axe 1, lignes   : {'Instagram': -0.4, 'Boutique': 0.457}
axe 1, colonnes : {'court': -0.535, 'moyen': 0.067, 'long': 0.568}

```

## Bilan du chapitre 3

Vous savez maintenant :

- **résumer** un tableau de variables numériques par une **ACP** : l'écrire comme la diagonalisation de la matrice de covariance (ou la SVD du tableau centré), choisir entre covariance et corrélation (**standardiser** quand les unités diffèrent), choisir le nombre de composantes (éboulis, Kaiser, **analyse parallèle**), lire saturations, scores et biplot, et mesurer ce que l'on perd (somme des valeurs propres jetées) ;
- distinguer l'ACP, qui **résume**, de l'**analyse factorielle**, qui **modélise** les corrélations par des facteurs cachés ( $\Sigma = \Lambda\Lambda^T + \Psi$ ) ; estimer, **tester l'ajustement**, choisir le nombre de facteurs, **faire tourner** les axes (varimax) sans changer l'ajustement, et construire des échelles de mesure (alpha de Cronbach) ;
- **classer sans étiquettes** : les **k-means** (algorithme de Lloyd, k-means++, minimum local), la **classification hiérarchique** (critères d'agrégation, dendrogramme) ; **choisir k** (coude, silhouette) et surtout **vérifier qu'il y a des groupes** (silhouette, stabilité par bootstrap et ARI) ;
- (en option) étendre l'idée aux **variables qualitatives** par l'**analyse des correspondances** (SVD des résidus standardisés, inertie =  $\chi^2/n$ ) et l'**ACM** ;
- (en option) passer des groupes inconnus aux groupes connus avec l'**analyse discriminante** : règle de Bayes avec des classes gaussiennes, LDA (frontières linéaires) et QDA, lien avec la **régression logistique**, projection de Fisher.

Un fil rouge traverse le chapitre : **une méthode descriptive rend toujours un résultat**, même sur du bruit. Un axe, un facteur, un groupe, une carte n'ont de valeur qu'accompagnés de leurs **diagnostics** (parts d'inertie et analyse parallèle, test d'ajustement, silhouette et stabilité, khi-deux avant de lire une carte).

Le chapitre 4 change d'horizon : après les données « en coupe » (un instantané de clientes), les **séries temporelles**, où l'ordre des observations compte et où la mémoire du passé devient l'information principale. Les ventes mensuelles de Dar Jasmin, de 2016 à 2025, nous y attendent.

# Chapitre 4 : Séries temporelles

« Hier explique un peu aujourd'hui. Les séries temporelles sont l'art de mesurer *com-bien*. »

Jusqu'ici, nous avons presque toujours supposé que nos observations étaient **indépendantes** : 400 commandes tirées au hasard, 2 000 clients qui ne se parlent pas. Cette hypothèse a fait toute la force du volume I (la loi des grands nombres, le théorème central limite, les intervalles de confiance) et celle des régressions des chapitres 1 et 2.

Elle tombe dès qu'on observe **la même chose au fil du temps**. Le chiffre d'affaires de Dar Jasmin en mars dépend de celui de février : une bonne année tire les mois vers le haut, décembre est toujours le mois des fêtes, et un choc comme celui de 2020 se prolonge pendant des mois. Les observations sont **dépendantes**, et c'est précisément cette dépendance qui est à la fois le **piège** (les formules du volume I deviennent fausses) et la **ressource** (le passé permet de prévoir l'avenir).

Ce chapitre apprend à lire, à modéliser et à prévoir une série temporelle, avec une seule série en fil rouge : **dix ans de chiffre d'affaires mensuel de Dar Jasmin** (janvier 2016 à décembre 2025).

## Le chemin de ce chapitre

- **4.1 Stationnarité, autocorrélation, décomposition** : que veut dire « avoir une structure stable dans le temps » ? Comment mesurer la mémoire d'une série (la fonction d'autocorrélation) ? Comment séparer tendance, saisonnalité et bruit ?
- **4.2 Modèles ARIMA et saisonniers** : les briques AR et MA, la méthode de Box-Jenkins, l'ajout de variables explicatives (promotions, COVID), le diagnostic des résidus.
- **4.3 Prévision et évaluation** : prévoir, quantifier l'incertitude, comparer honnêtement des modèles (découpage temporel, mesures d'erreur, validation à origine glissante), et une première révélation sur la façon dont les données ont été fabriquées.
- **⊕ Pour aller plus loin** : modèles multivariés VAR, cointégration et GARCH (4.4) ; modèles d'espace d'états et filtre de Kalman (4.5) ; Prophet et les bibliothèques modernes de prévision (4.6).
- **4.7 Exercices corrigés.**

**Comment travailler avec ce chapitre.** Le fil conducteur est un **concours de prévision** : plusieurs modèles prévoient les 24 derniers mois de la série, que nous aurons mis de côté **dès le début**, avant même d'avoir regardé les données. Vous verrez que le modèle qui paraît le meilleur sur le papier n'est pas toujours celui qui gagne, et pourquoi. Tapez le code vous-même, changez les paramètres, regardez ce qui casse.

**Les données.** Le fichier `donnees/ventes_mensuelles.csv` contient 120 mois : mois, ca (chiffre d'affaires en DT), nb\_commandes, promo (1 si une promotion a eu lieu dans le mois) et covid



(1 de mars à juin 2020, quatre mois de fermeture partielle). Comme au chapitre précédent, ces données sont **simulées** avec des graines fixes : nous connaissons donc la vérité, et nous la révélerons à la fin de la section 4.3, pour voir si les méthodes l'ont retrouvée. Les sections 4.4 et 4.5 utilisent aussi des séries simulées à part, annoncées chaque fois.

**Prérequis.** Le volume I (section 2.4 sur la loi des grands nombres, section 3.3 sur les intervalles de confiance, section 3.5 sur les p-valeurs, section 1.1 sur les valeurs propres) et le chapitre 1 de ce volume (la régression linéaire et ses diagnostics, section 1.3). Les modèles ARIMA sont, au fond, des régressions où les variables explicatives sont le passé de la série elle-même.

## 4.1 Stationnarité, autocorrélation, décomposition

**Intuition.** Un statisticien qui regarde une série temporelle se pose trois questions, dans cet ordre. (1) « Cette série a-t-elle des règles stables dans le temps ? » (la **stationnarité**). (2) « Combien de mémoire a-t-elle ? Aujourd'hui ressemble-t-il à hier, à il y a un an ? » (l'**autocorrélation**). (3) « Quelle part de ce que je vois est une tendance, une saison, ou du bruit ? » (la **décomposition**). Les modèles des sections suivantes ne sont que des réponses chiffrées à ces trois questions.

### 4.1.1 Pourquoi l'ordre compte : le test du mélange

Prenez 100 commandes tirées au hasard : vous pouvez les mélanger, l'histogramme reste le même, la moyenne aussi. Prenez les 96 chiffres d'affaires mensuels de 2016 à 2023 : si vous les mélangez, **vous détruisez l'information**. Un décembre n'est plus à côté d'un janvier, la tendance a disparu, la mémoire aussi. C'est la différence fondamentale entre un échantillon et une **série temporelle** : l'ordre est une partie de la donnée.

Une règle d'hygiène avant d'aller plus loin. Nous voulons, en fin de chapitre, juger honnêtement des prévisions. Il faut donc **mettre de côté dès maintenant** les 24 derniers mois (2024 et 2025) et ne plus les regarder tant que nous n'avons pas choisi nos modèles. Tout ce que nous ferons dans les sections 4.1 et 4.2 (graphiques, tests, choix d'un modèle) utilisera **uniquement les 96 premiers mois**. Un analyste qui choisit son modèle après avoir vu l'avenir ne prévoit rien : il décrit le passé.

```
import warnings
warnings.filterwarnings("ignore")          # masque les avertissements de bibliothèques, pour des
↳ sorties lisibles
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.stattools import acf, pacf, adfuller, kpss
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.seasonal import seasonal_decompose, STL

v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"                       # fréquence « début de mois » : indispensable pour les modèles
y = np.log(v["ca"])                       # on travaille sur le logarithme (voir 4.1.2)
train = y[:"2023-12"]                     # 96 mois pour construire les modèles
test = y["2024-01":]                      # 24 mois mis de côté : on n'y touche plus jusqu'en 4.3
print("apprentissage :", train.index[0].strftime("%Y-%m"), "à", train.index[-1].strftime("%Y-%m"),
↳ "-", len(train), "mois")
```



```
print("mis de côtéé :", test.index[0].strftime("%Y-%m"), "à", test.index[-1].strftime("%Y-%m"), "-",
      ↪ len(test), "mois")
print(v.loc[:"2016-06", ["ca", "nb_commandes", "promo", "covid"]])
```

```
apprentissage : 2016-01 à 2023-12 - 96 mois
mis de côtéé : 2024-01 à 2025-12 - 24 mois
      ca  nb_commandes  promo  covid
mois
2016-01-01    620.0         14     0     0
2016-02-01    757.5         11     0     0
2016-03-01   1007.0         13     0     0
2016-04-01   1018.5         17     0     0
2016-05-01   1067.8         13     0     0
2016-06-01   1155.4         11     0     0
```

Chaque ligne est un mois. Le chiffre d'affaires (ca) de janvier 2016 est de 620 DT. Les colonnes promo et covid sont des variables **explicatives** que nous utiliserons en 4.2.

Pour mesurer à quel point l'ordre compte, calculons la **corrélation entre un mois et le suivant** (nous la définirons proprement en 4.1.4) sur la série, puis sur 1 000 mélanges aléatoires de la même série :

```
def acf_manuel(x, k):
    """Autocorrélation d'ordre k : corrélation entre la série et elle-même décalée de k pas."""
    d = np.asarray(x, float) - np.mean(x)
    return (d[:len(d) - k] * d[k:]).sum() / (d ** 2).sum()

rng = np.random.default_rng(1)
r_vrai = acf_manuel(train.values, 1)
r_melanges = [acf_manuel(rng.permutation(train.values), 1) for _ in range(1000)]
print("corrélation mois/mois-suivant, série réelle :", round(r_vrai, 3))
print("après mélange aléatoire (1000 essais) : moyenne", round(np.mean(r_melanges), 3),
      ↪ "| écart-type", round(np.std(r_melanges), 3))
print("pour comparaison, 1/racine(96) =", round(1 / np.sqrt(96), 3))
```

```
corrélation mois/mois-suivant, série réelle : 0.479
après mélange aléatoire (1000 essais) : moyenne -0.011 | écart-type 0.104
pour comparaison, 1/racine(96) = 0.102
```

La série réelle a une corrélation de l'ordre de 0,48 entre un mois et le suivant ; après mélange, elle tombe en moyenne à zéro, avec une dispersion d'environ 0,10, c'est-à-dire  $1/\sqrt{96}$ . Cette valeur n'est pas un hasard : nous la retrouverons en 4.1.4 comme **bande de confiance** du graphique d'autocorrélation.

### 4.1.2 Première lecture : tendance, saison, bruit, et pourquoi on prend le logarithme

Dessignons la série d'apprentissage, d'abord en dinars, puis en logarithme :

```
BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"figure.dpi": 100, "savefig.dpi": 200, "axes.spines.top": False,
  ↪ "axes.spines.right": False,
      "axes.grid": True, "grid.color": "#e1e0d9", "grid.linewidth": 0.6,
      ↪ "axes.titlesize": 11,
      "axes.titleweight": "regular", "legend.frameon": False, "font.size": 10})

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.8))
ca_train = v["ca"][:"2023-12"]
ax1.plot(ca_train.index, ca_train, color=BLEU, lw=1.6)
ax1.set_title("Chiffre d'affaires mensuel (DT)")
ax2.plot(train.index, train, color=BLEU, lw=1.6)
ax2.set_title("Logarithme du chiffre d'affaires")
```

```

for ax in (ax1, ax2):
    ax.axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-06-30"), color=ORANGE, alpha=0.18, lw=0)
ax1.text(pd.Timestamp("2020-03-10"), 3150, "COVID", color=ORANGE, fontsize=9)
ax2.text(pd.Timestamp("2020-03-10"), 8.05, "COVID", color=ORANGE, fontsize=9)
plt.tight_layout()
plt.savefig("figures/ch04-serie.png", bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

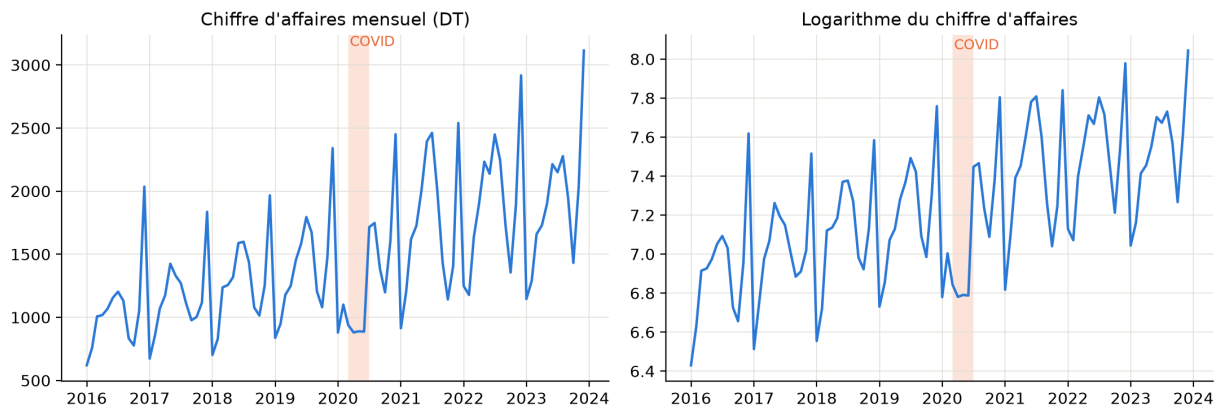


Figure 36: Chiffre d'affaires mensuel de Dar Jasmin de 2016 à 2023 : en dinars (à gauche) l'amplitude des oscillations saisonnières grandit avec le niveau ; en logarithme (à droite), elle reste à peu près constante. La bande orange marque mars-juin 2020.

On lit quatre choses :

1. Une **tendance** : la série monte globalement, d'un peu plus de 1 000 DT par mois en 2016 à près de 2 000 DT en 2023.
2. Une **saisonnalité** : chaque année se ressemble (creux en janvier, plateau haut l'été, pic en décembre).
3. Un **accident** : le trou de mars à juin 2020.
4. Du **bruit** : des écarts irréguliers autour de la tendance et de la saison.

Observez aussi le panneau de gauche : plus la série monte, plus les oscillations saisonnières s'**élargissent** (l'écart entre le mois le plus haut et le mois le plus bas de l'année est d'environ 1 160 DT en 2017 et d'environ 1 970 DT en 2023). C'est un schéma **multiplicatif** : la saison *multiplie* le niveau au lieu de s'y *ajouter*. Le panneau de droite montre le remède : en **logarithme**, un produit devient une somme ( $\log(T \times S \times R) = \log T + \log S + \log R$ ), et les oscillations ont une amplitude à peu près constante. Vérifions-le chiffre en main :

```

g = pd.DataFrame({"annee": ca_train.index.year, "brut": ca_train.values, "log": train.values})
amplitude = g.groupby("annee").agg(niveau_moyen=("brut", "mean"),
                                   amplitude_brute=("brut", lambda s: s.max() - s.min()),
                                   amplitude_log=("log", lambda s: s.max() - s.min()).round(2))
print(amplitude)

```

|       | niveau_moyen | amplitude_brute | amplitude_log |
|-------|--------------|-----------------|---------------|
| annee |              |                 |               |
| 2016  | 1054.87      | 1415.4          | 1.19          |
| 2017  | 1153.82      | 1162.9          | 1.00          |
| 2018  | 1273.32      | 1264.3          | 1.03          |
| 2019  | 1402.31      | 1503.8          | 1.03          |
| 2020  | 1306.68      | 1571.7          | 1.02          |
| 2021  | 1737.24      | 1626.7          | 1.02          |
| 2022  | 1911.13      | 1739.5          | 0.91          |

|      |         |        |      |
|------|---------|--------|------|
| 2023 | 1906.96 | 1968.6 | 1.00 |
|------|---------|--------|------|

L'amplitude brute passe d'environ 1 160 DT (2017) à environ 1 970 DT (2023), en suivant la croissance du niveau moyen ; l'amplitude en logarithme, elle, reste comprise entre 0,9 et 1,2 pour toutes les années. À partir de maintenant, **nous modélisons le logarithme du chiffre d'affaires**.

**Lire une variation en logarithme.** Si  $\log y$  augmente de 0,05, alors  $y$  est multiplié par  $e^{0,05} \approx 1,051$  : une hausse d'environ 5 %. Pour de petites variations, *différence de logarithmes  $\approx$  variation relative*. C'est pourquoi les coefficients des modèles en log se lisent comme des pourcentages.

### 4.1.3 La stationnarité

Pour estimer quelque chose à partir d'une **seule trajectoire** (nous n'avons qu'une seule histoire de Dar Jasmin, on ne peut pas rejouer 2016), il faut que le processus ait des règles qui ne changent pas dans le temps. C'est le sens de la stationnarité.

**Définition (stationnarité faible).** Une série  $(Y_t)$  est **stationnaire au second ordre** si

1.  $\mathbb{E}[Y_t] = \mu$  ne dépend pas de  $t$  ;
2.  $\text{Var}(Y_t) = \sigma^2 < \infty$  ne dépend pas de  $t$  ;
3.  $\text{Cov}(Y_t, Y_{t+k}) = \gamma(k)$  ne dépend que du **décalage**  $k$ , pas de  $t$ .

On appelle  $\gamma(k)$  l'**autocovariance** et  $\rho(k) = \gamma(k)/\gamma(0)$  l'**autocorrélation** d'ordre  $k$ . Une série *strictement* stationnaire a toutes ses lois jointes invariantes par translation ; avec une variance finie, elle est faiblement stationnaire. La réciproque est fautive en général (elle est vraie pour les séries gaussiennes). Nous n'utiliserons que la version faible.

Quatre exemples simulés montrent à quoi cela ressemble. Chacun a 200 points, et nous comparons la moyenne et la variance de la **première moitié** à celles de la **seconde** :

```
rng = np.random.default_rng(4)
n = 200
bruit_blanc = rng.normal(size=n) # Y_t = e_t
marche = np.cumsum(rng.normal(size=n)) # Y_t = Y_{t-1} + e_t
ar1 = np.zeros(n)
for t in range(1, n):
    ar1[t] = 0.8 * ar1[t - 1] + rng.normal() # Y_t = 0,8 Y_{t-1} + e_t
tendance = 0.05 * np.arange(n) + rng.normal(size=n) # Y_t = 0,05 t + e_t

exemples = {"bruit blanc": bruit_blanc, "marche aléatoire": marche, "AR(1), phi = 0,8": ar1, "tendance
↳ + bruit": tendance}
lignes = []
for nom, s in exemples.items():
    lignes.append({"série": nom, "moyenne 1re moitié": s[:100].mean(), "moyenne 2e moitié":
    ↳ s[100:].mean(),
    "variance 1re moitié": s[:100].var(), "variance 2e moitié": s[100:].var()})
print(pd.DataFrame(lignes).round(2).to_string(index=False))

fig, axes = plt.subplots(2, 2, figsize=(11, 5.6), sharex=True)
couleurs = [BLEU, ROUGE, AQUA, VIOLET]
for ax, (nom, s), c in zip(axes.ravel(), exemples.items(), couleurs):
    ax.plot(s, color=c, lw=1.3)
    ax.set_title(nom)
plt.tight_layout()
plt.savefig("figures/ch04-stationnarite.png", bbox_inches="tight")
```

| série            | moyenne 1re moitié | moyenne 2e moitié | variance 1re moitié | variance 2e moitié |
|------------------|--------------------|-------------------|---------------------|--------------------|
| bruit blanc      | -0.06              | 0.16              | 1.02                | 0.94               |
| marche aléatoire | 7.73               | 16.69             | 52.92               | 26.63              |

|                     |       |      |      |      |
|---------------------|-------|------|------|------|
| AR(1), $\phi = 0,8$ | -0.58 | 0.20 | 3.19 | 2.34 |
| tendance + bruit    | 2.46  | 7.45 | 3.03 | 3.16 |

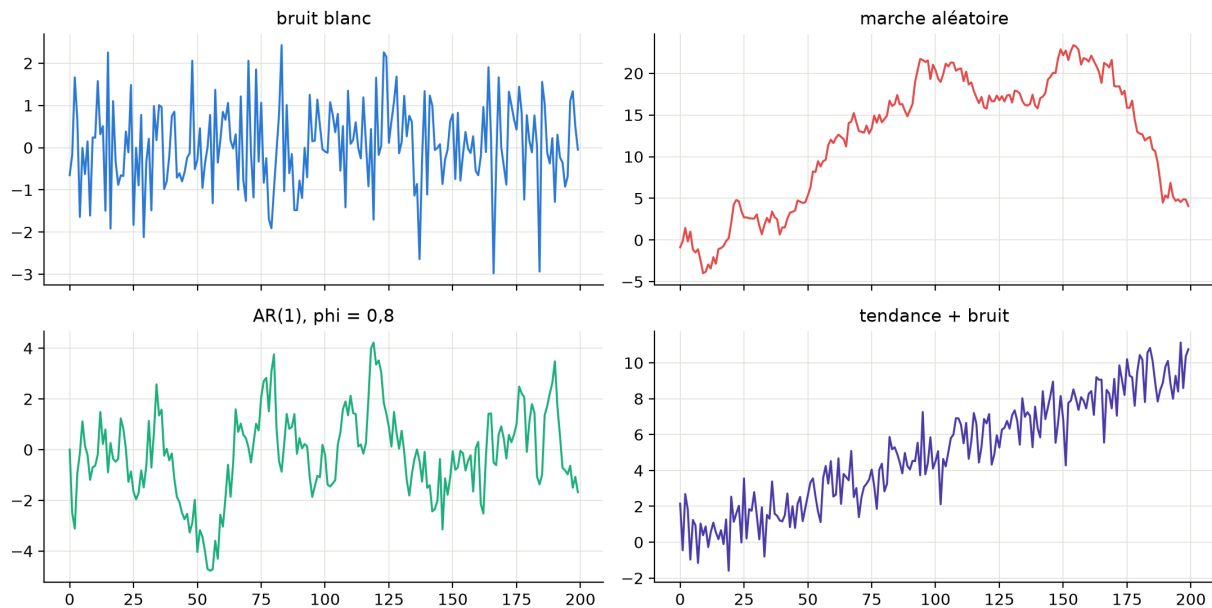


Figure 37: Quatre séries simulées : le bruit blanc et l'AR(1) fluctuent autour d'un niveau fixe (stationnaires) ; la marche aléatoire erre sans revenir ; la série « tendance + bruit » monte (la moyenne change avec le temps).

- Le **bruit blanc** (des chocs indépendants) et l'**AR(1)** (nous l'étudierons en 4.2) gardent des moyennes et des variances comparables d'une moitié à l'autre : ils sont stationnaires.
- La **série avec tendance** a une variance stable mais une moyenne qui change (environ 2,5 puis 7,5) : elle viole la condition 1.
- La **marche aléatoire** a une moyenne et une variance très différentes d'une moitié à l'autre : elle viole les conditions 1 et 2.

La marche aléatoire est l'exemple fondamental de non-stationnarité. Montrons rigoureusement que sa variance explose :

**Démonstration : la variance d'une marche aléatoire croît linéairement.** Soit  $Y_t = Y_{t-1} + \varepsilon_t$  avec  $Y_0 = 0$  et des  $\varepsilon_t$  indépendants de variance  $\sigma^2$ . En remplaçant récursivement,  $Y_t = \varepsilon_1 + \varepsilon_2 + \dots + \varepsilon_t$  : une somme de  $t$  chocs indépendants. Donc  $\text{Var}(Y_t) = t\sigma^2$ , qui dépend de  $t$  et tend vers l'infini. La série ne revient jamais « se stabiliser » : chaque choc a un effet **permanent**. ■

Vérifions par simulation : 5 000 marches aléatoires de 200 pas, et la variance à travers les trajectoires à trois instants :

```
trajectoires = np.cumsum(np.random.default_rng(5).normal(size=(5000, 200)), axis=1)
for t in (50, 100, 200):
    print(f"t = {t:3d} : variance observée = {trajectoires[:, t - 1].var():6.1f} (théorie : t x
    ↳ sigma^2 = {t})")
```

```
t = 50 : variance observée = 50.3 (théorie : t x sigma^2 = 50)
t = 100 : variance observée = 96.4 (théorie : t x sigma^2 = 100)
t = 200 : variance observée = 194.0 (théorie : t x sigma^2 = 200)
```

**Deux façons d'être non stationnaire, deux traitements.** Une série peut avoir une **tendance déterministe** (comme  $Y_t = a + bt + u_t$  avec  $u_t$  stationnaire : on dit « stationnaire autour d'une tendance »), ou une **tendance stochastique** (comme la marche aléatoire : les chocs s'accumulent). Le premier cas se traite en **retirant la tendance par régression** ; le second en **différenciant** ( $Z_t = Y_t - Y_{t-1}$ ). Appliquer le mauvais remède abîme la série : nous le montrerons en 4.1.6.

Et notre série de chiffre d'affaires ? Elle a une tendance, une saison forte et un accident : elle n'est évidemment **pas stationnaire telle quelle**. La question est de savoir *quelle sorte* de non-stationnarité elle a, ce qui détermine le traitement.

#### 4.1.4 L'autocorrélation : mesurer la mémoire

Reprenons la définition :  $\rho(k) = \text{Corr}(Y_t, Y_{t+k})$ , la corrélation entre la série et elle-même décalée de  $k$  pas. On l'estime à partir d'un échantillon  $y_1, \dots, y_n$  par

$$r_k = \frac{\sum_{t=k+1}^n (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^n (y_t - \bar{y})^2}.$$

**Exemple à la main.** Six chiffres : 10, 12, 11, 14, 13, 15. Leur moyenne est  $\bar{y} = 12,5$  ; les écarts sont  $-2,5, -0,5, -1,5, 1,5, 0,5, 2,5$ . Le dénominateur est  $\sum (y_t - \bar{y})^2 = 6,25 + 0,25 + 2,25 + 2,25 + 0,25 + 6,25 = 17,5$ . **Décalage 1** : on multiplie chaque écart par le précédent :  $(-0,5)(-2,5) = 1,25$  ;  $(-1,5)(-0,5) = 0,75$  ;  $(1,5)(-1,5) = -2,25$  ;  $(0,5)(1,5) = 0,75$  ;  $(2,5)(0,5) = 1,25$ . La somme vaut 1,75, donc  $r_1 = 1,75/17,5 = 0,10$ . **Décalage 2** :  $(-1,5)(-2,5) = 3,75$  ;  $(1,5)(-0,5) = -0,75$  ;  $(0,5)(-1,5) = -0,75$  ;  $(2,5)(1,5) = 3,75$ . La somme vaut 6, donc  $r_2 = 6/17,5 \approx 0,343$ .

```
x = np.array([10, 12, 11, 14, 13, 15])
print("r1 =", round(acf_manuel(x, 1), 4), " r2 =", round(acf_manuel(x, 2), 4))
```

```
r1 = 0.1    r2 = 0.3429
```

Le code retrouve exactement les valeurs de la main. Notez que le dénominateur est toujours la somme des  $n$  **termes** (même pour  $k$  grand), ce qui garantit que les autocorrélations estimées forment toujours une suite « possible » (semi-définie positive). Appliquons-le à la série d'apprentissage et comparons avec la fonction de statsmodels :

```
r_perso = np.array([acf_manuel(train.values, k) for k in range(7)])
r_sm = acf(train, nlags=6, fft=False)
print("manuel   :", r_perso.round(3))
print("statsmodels:", r_sm.round(3))
print("identiques :", np.allclose(r_perso, r_sm))
```

```
manuel   : [1.    0.479 0.224 0.305 0.344 0.283 0.191]
statsmodels: [1.    0.479 0.224 0.305 0.344 0.283 0.191]
identiques : True
```

#### L'autocorrélation partielle

L'autocorrélation d'ordre 2 mélange deux effets : l'influence directe de  $y_{t-2}$  sur  $y_t$ , et l'influence **indirecte** passant par  $y_{t-1}$  (qui dépend lui-même de  $y_{t-2}$ ). L'**autocorrélation partielle** d'ordre  $k$ , notée  $\phi_{kk}$ , isole l'effet direct : c'est le **coefficient de  $y_{t-k}$  dans la régression de  $y_t$  sur  $y_{t-1}, \dots, y_{t-k}$** . On peut donc la calculer avec les moindres carrés du chapitre 1 :

```
def pacf_manuel(x, k):
    x = np.asarray(x, float)
    n = len(x)
    cible = x[k:]
    explicatives = np.column_stack([np.ones(n - k)] + [x[k - j:n - j] for j in range(1, k + 1)])
    coef = np.linalg.lstsq(explicatives, cible, rcond=None)[0]
    return coef[-1]  # coefficient de y_{t-k}

p_sm = pacf(train, nlags=3, method="ols")
print("pacf manuelle :", np.round([pacf_manuel(train.values, k) for k in (1, 2, 3)], 4))
print("pacf statsmodels:", p_sm[1:].round(4))

pacf manuelle : [0.5074 0.0042 0.2814]
pacf statsmodels: [0.5074 0.0042 0.2814]
```

Les deux concordent : l'autocorrélation partielle n'est rien d'autre qu'une régression, ce qui annonce les modèles autorégressifs de la section 4.2.

### Les bandes de confiance

Sur un graphique d'autocorrélation, on trace une bande autour de zéro. D'où vient-elle ? Pour un **bruit blanc** (aucune mémoire), les  $r_k$  sont approximativement gaussiens, centrés, de variance  $1/n$  (c'est le théorème central limite du volume I, section 2.4, appliqué à une somme de produits). D'où la bande  $\pm 1.96/\sqrt{n}$  à 95 %. Vérifions par simulation que le taux de fausses alertes est bien d'environ 5 % :

```
rng = np.random.default_rng(2)
n = 96
alertes = np.mean([abs(acf_manuel(rng.normal(size=n), 1)) > 1.96 / np.sqrt(n) for _ in range(2000)])
print("bande :", round(1.96 / np.sqrt(n), 3), "| part de bruits blancs avec |r1| hors bande :",
      ↪ round(alertes, 3))

bande : 0.2 | part de bruits blancs avec |r1| hors bande : 0.046
```

**Une barre qui dépasse n'est pas une catastrophe.** À 95 %, sur 20 décalages, une barre dépasse en moyenne la bande même pour du bruit pur. Ce qui compte, ce sont les barres **nettement** hors bande, ou une structure répétée (tous les 12 mois, par exemple).

Voici l'autocorrélation et l'autocorrélation partielle de notre série d'apprentissage (en logarithme), sur 24 décalages :

```
def tracer_acf(ax, valeurs, n, titre, couleur=BLEU):
    k = np.arange(len(valeurs))
    ax.vlines(k, 0, valeurs, color=couleur, lw=2)
    ax.plot(k, valeurs, "o", color=couleur, ms=3.5)
    ax.axhline(0, color="#898781", lw=0.8)
    b = 1.96 / np.sqrt(n)
    ax.axhspan(-b, b, color=couleur, alpha=0.10, lw=0)
    ax.set_title(titre)
    ax.set_xlabel("décalage (mois)")
    ax.set_ylim(-1, 1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.6))
tracer_acf(ax1, acf(train, nlags=24, fft=False), len(train), "Autocorrélation de log(ca)")
tracer_acf(ax2, pacf(train, nlags=24, method="ols"), len(train), "Autocorrélation partielle de
↪ log(ca)", ORANGE)
plt.tight_layout()
plt.savefig("figures/ch04-acf-serie.png", bbox_inches="tight")
```

L'autocorrélation reste significative sur de nombreux décalages (la série a une longue mémoire,

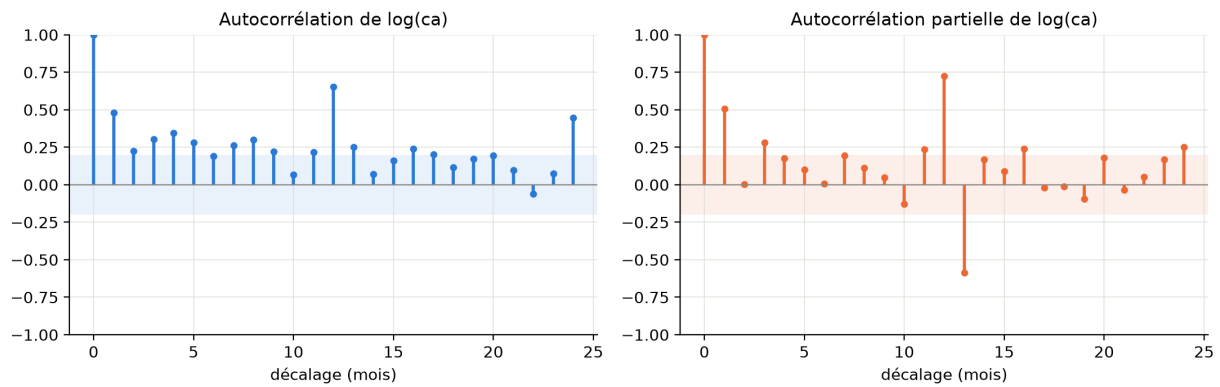


Figure 38: Autocorrélation (à gauche) et autocorrélation partielle (à droite) du logarithme du chiffre d'affaires sur les 96 mois d'apprentissage ; la zone colorée est la bande de confiance à 95 % d'un bruit blanc.

due à la tendance et à la saison), avec des bosses aux multiples de 12 : la signature de la saisonnalité annuelle.

#### 4.1.5 Le bruit blanc et le test de Ljung-Box

Un **bruit blanc** est une suite de variables indépendantes, centrées, de même variance : la série la plus ennuyeuse possible, **sans mémoire**. Il joue un rôle central, car c'est ce qu'il doit **rester** quand un modèle a tout expliqué : si les résidus d'un modèle ne ressemblent pas à un bruit blanc, il reste de l'information à exploiter.

Pour tester cela globalement (plutôt que décalage par décalage), on utilise la statistique de **Ljung-Box** :

$$Q(h) = n(n+2) \sum_{k=1}^h \frac{r_k^2}{n-k}.$$

Sous l'hypothèse « bruit blanc »,  $Q(h)$  suit approximativement une loi du khi-deux à  $h$  degrés de liberté (volume I, section 3.4.6). Une grande valeur de  $Q$  signifie qu'au moins une autocorrélation est trop forte pour être due au hasard. Calculons-la à la main puis avec `statsmodels`, sur un vrai bruit blanc simulé et sur notre série :

```
def ljung_box(x, h):
    x = np.asarray(x, float)
    n = len(x)
    r = np.array([acf_manuel(x, k) for k in range(1, h + 1)])
    Q = n * (n + 2) * np.sum(r ** 2 / (n - np.arange(1, h + 1)))
    return Q, 1 - stats.chi2.cdf(Q, h)

w = np.random.default_rng(6).normal(size=96)
for nom, s in [("bruit blanc simulé", w), ("log(ca), apprentissage", train.values)]:
    Q, p = ljung_box(s, 10)
    print(f"{nom:24s} Q(10) = {Q:7.2f}    p-valeur = {p:.2e}")
sm_lb = acorr_ljungbox(train, lags=[10])
print("statsmodels (log ca)      Q(10) =", round(float(sm_lb['lb_stat'].iloc[0]), 2))
```

|                        |         |       |                     |
|------------------------|---------|-------|---------------------|
| bruit blanc simulé     | Q(10) = | 9.52  | p-valeur = 4.84e-01 |
| log(ca), apprentissage | Q(10) = | 84.00 | p-valeur = 8.20e-14 |
| statsmodels (log ca)   | Q(10) = | 84.0  |                     |



Pour le bruit blanc, la p-valeur est grande : on ne rejette pas l'hypothèse « pas de mémoire ». Pour notre série, elle est pratiquement nulle : la série a de la mémoire, ce que l'œil voyait déjà.

**Quand on teste des résidus de modèle.** Si l'on applique Ljung-Box aux résidus d'un modèle qui a estimé  $m$  paramètres de type AR ou MA, il faut réduire les degrés de liberté à  $h - m$ . Nous le ferons en 4.2.

#### 4.1.6 Rendre une série stationnaire : différencier, et tester

Reprenons la distinction de 4.1.3. Pour une **marche aléatoire**, on **différencie** :  $\nabla Y_t = Y_t - Y_{t-1} = \varepsilon_t$  est un bruit blanc. Pour une série à **saison** de période  $s = 12$ , on peut utiliser la **différence saisonnière**  $\nabla_{12} Y_t = Y_t - Y_{t-12}$  (comparer chaque mois au même mois de l'an passé). Pour notre série en logarithme, cela a une belle interprétation :

- $\nabla \log y_t = \log y_t - \log y_{t-1} \approx$  **taux de croissance mensuel** ;
- $\nabla_{12} \log y_t \approx$  **taux de croissance sur un an**, c'est-à-dire le « +x % par rapport au même mois l'an dernier » des bilans d'entreprise.

Comment décider si une série a besoin d'être différenciée ? On utilise des **tests de racine unitaire**. Le plus célèbre est le **test de Dickey-Fuller**. Son idée : dans le modèle  $Y_t = \rho Y_{t-1} + \varepsilon_t$ , la série est une marche aléatoire si  $\rho = 1$  (racine unitaire) et stationnaire si  $|\rho| < 1$ . En retranchant  $Y_{t-1}$  des deux côtés, on obtient la régression

$$\nabla Y_t = \alpha + \gamma Y_{t-1} + \varepsilon_t, \quad \gamma = \rho - 1,$$

et on teste  $H_0 : \gamma = 0$  (racine unitaire) contre  $\gamma < 0$  (stationnaire), avec la statistique de Student habituelle de  $\hat{\gamma}$ .

**Le piège : cette statistique ne suit PAS la loi de Student.** Sous  $H_0$ , la série n'est pas stationnaire, et les théorèmes du volume I (sur lesquels reposent Student et la loi normale) ne s'appliquent plus. La loi de la statistique est **différente** et se simule. Faisons-le : 5 000 marches aléatoires de 100 pas, la statistique de Dickey-Fuller de chacune, et ses quantiles.

```
def stat_df(s):
    """Statistique t de gamma dans : diff(Y_t) = alpha + gamma * Y_{t-1} + e_t."""
    s = np.asarray(s, float)
    d = np.diff(s)
    X = np.column_stack([np.ones(len(d)), s[:-1]])
    b = np.linalg.lstsq(X, d, rcond=None)[0]
    e = d - X @ b
    s2 = e @ e / (len(d) - 2)
    cov = s2 * np.linalg.inv(X.T @ X)
    return b[1] / np.sqrt(cov[1, 1])

rng = np.random.default_rng(7)
stats_sim = np.array([stat_df(np.cumsum(rng.normal(size=100))) for _ in range(5000)])
q = [1, 5, 10]
print("quantiles à 1 %, 5 %, 10 % de la loi de Dickey-Fuller simulée :", np.percentile(stats_sim,
    ↪ q).round(2))
print("quantiles de la loi normale", stats.norm.ppf([0.01, 0.05,
    ↪ 0.10]).round(2))
print("part des marches aléatoires « rejetées » avec le seuil normal -1,645 :",
    ↪ round(np.mean(stats_sim < -1.645), 3))

fig, ax = plt.subplots(figsize=(7, 3.4))
ax.hist(stats_sim, bins=60, density=True, color=BLEU, alpha=0.75, label="statistique de Dickey-Fuller
    ↪ (simulée)")
```

```

xs = np.linspace(-5, 3, 300)
ax.plot(xs, stats.norm.pdf(xs), color=ORANGE, lw=2, label="loi normale")
ax.vline(np.percentile(stats_sim, 5), color=ROUGE, ls="--", lw=1.2)
ax.text(np.percentile(stats_sim, 5) - 0.1, 0.40, "seuil à 5 %", color=ROUGE, ha="right", fontsize=9)
ax.set_xlabel("valeur de la statistique")
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("figures/ch04-dickey-fuller.png", bbox_inches="tight")

```

quantiles à 1 %, 5 %, 10 % de la loi de Dickey-Fuller simulée : [-3.52 -2.92 -2.56]  
quantiles de la loi normale : [-2.33 -1.64 -1.28]  
part des marches aléatoires « rejetées » avec le seuil normal -1,645 : 0.468

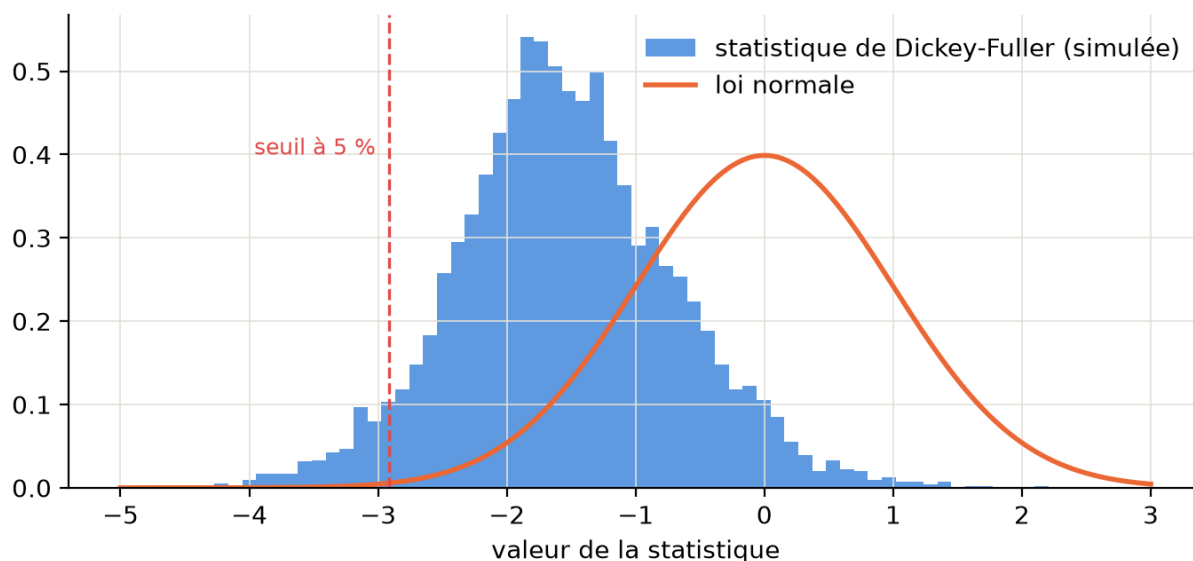


Figure 39: Loi simulée de la statistique de Dickey-Fuller sous l'hypothèse de racine unitaire (histogramme bleu), décalée vers la gauche par rapport à la loi normale (orange). Le seuil à 5 % se situe vers  $-2,9$  et non  $-1,64$ .

Le seuil à 5 % est d'environ  $-2,9$  (et non  $-1,645$ ). Utiliser par erreur le seuil normal ferait « rejeter » la racine unitaire pour près d'une marche aléatoire sur deux : un test nul. Les logiciels utilisent les vraies tables, obtenues par simulation comme ici (Dickey et Fuller, 1979).

Le test de Dickey-Fuller **augmenté** (ADF) ajoute à la régression des différences retardées  $\nabla Y_{t-1}, \nabla Y_{t-2}, \dots$  pour absorber l'autocorrélation. Le test **KPSS** renverse les rôles : son hypothèse nulle est que la série est **stationnaire** (autour d'un niveau ou d'une tendance). Les deux tests se complètent :

| ADF                                   | KPSS                            | Lecture                                                |
|---------------------------------------|---------------------------------|--------------------------------------------------------|
| rejette la racine unitaire            | ne rejette pas la stationnarité | <b>stationnaire</b> (autour de la tendance éventuelle) |
| ne rejette pas                        | rejette                         | <b>racine unitaire</b> : différencier                  |
| les deux rejettent / aucun ne rejette |                                 | <b>verdict ambigu</b> : les données ne tranchent pas   |

Appliquons les deux à notre série d'apprentissage, selon qu'on autorise ou non une tendance déterministe :

```
def tester(s, regression, nom):
    a = adfuller(s, regression=regression, autolag="AIC")
    k = kpss(s, regression=regression, nlags="auto")
    return {"série": nom, "ADF stat": a[0], "ADF p": a[1], "retards": a[2], "seuil 5 %": a[4]["5%"],
            "KPSS stat": k[0], "KPSS p": k[1]}

lignes = [tester(train, "ct", "log(ca), avec tendance"),
           tester(train, "c", "log(ca), niveau seul"),
           tester(train.diff().dropna(), "c", "différence première"),
           tester(train.diff(12).dropna(), "c", "différence saisonnière (12)"),
           tester(train.diff().diff(12).dropna(), "c", "différences 1 et 12")]
tab = pd.DataFrame(lignes).set_index("série")
print(tab.round(3).to_string())
```

| série                       | ADF stat | ADF p | retards | seuil 5 % | KPSS stat | KPSS p |
|-----------------------------|----------|-------|---------|-----------|-----------|--------|
| log(ca), avec tendance      | -3.228   | 0.079 | 12      | -3.465    | 0.042     | 0.10   |
| log(ca), niveau seul        | -1.064   | 0.729 | 12      | -2.897    | 1.561     | 0.01   |
| différence première         | -3.617   | 0.005 | 12      | -2.897    | 0.185     | 0.10   |
| différence saisonnière (12) | -2.418   | 0.137 | 12      | -2.903    | 0.057     | 0.10   |
| différences 1 et 12         | -4.436   | 0.000 | 11      | -2.903    | 0.023     | 0.10   |

**La lecture honnête de ce tableau.** La p-valeur de KPSS est tronquée à 0,10 (borne de la table) ou à 0,01, d'où les valeurs « rondes ».

- **Avec tendance** : ADF obtient une p-valeur de l'ordre de 0,08, **au-dessus de 5 %**, donc il ne rejette pas la racine unitaire ; mais KPSS, de son côté, **ne rejette pas non plus** la stationnarité autour d'une tendance. C'est le cas ambigu de la troisième ligne du tableau. Avec seulement 96 observations, un COVID au milieu et une saison forte (l'ADF a dû utiliser 12 retards pour l'absorber), le test a **peu de puissance** : il ne peut pas trancher.
- **Sans tendance** : les deux tests s'accordent pour dire que la série n'est pas stationnaire autour d'un niveau constant (ce qui est évident : elle monte).
- **Après différenciation** : avec la différence première ou avec les deux différences (1 et 12), les deux tests s'accordent pour la stationnarité. La seule différence saisonnière laisse en revanche un verdict ambigu (l'ADF a une p-valeur de l'ordre de 0,14 : il ne rejette pas la racine unitaire).

**Conclusion** : les données sont compatibles avec **deux** descriptions, « tendance stochastique » (il faut différencier) et « tendance déterministe » (il faut modéliser la tendance et la saison). Plutôt que de forcer un verdict, nous garderons **les deux** candidats en 4.2 et les départagerons en 4.3 par ce qui compte vraiment : la qualité des prévisions.

### Le danger de la sur-différenciation

Que se passe-t-il si l'on différencie une série qui n'en avait pas besoin ? Prenons un bruit blanc pur, déjà stationnaire, et différencions-le :  $\nabla \varepsilon_t = \varepsilon_t - \varepsilon_{t-1}$ .

**Calcul.**  $\text{Var}(\nabla \varepsilon_t) = 2\sigma^2$  et  $\text{Cov}(\nabla \varepsilon_t, \nabla \varepsilon_{t-1}) = \text{Cov}(\varepsilon_t - \varepsilon_{t-1}, \varepsilon_{t-1} - \varepsilon_{t-2}) = -\sigma^2$ . Donc l'autocorrélation d'ordre 1 de la série différenciée vaut  $-\sigma^2/(2\sigma^2) = -0,5$  : on a **fabriqué** une mémoire artificielle (un « MA(1) avec  $\theta = -1$  », que nous étudierons en 4.2).

```
w = np.random.default_rng(9).normal(size=2000)
print("autocorrélation d'ordre 1 de diff(bruit blanc) :", round(acf_manuel(np.diff(w), 1), 3), "
      ↳ théorie : -0,5")
print("autocorrélations de la différence première de log(ca) (décalages 1 à 13) :")
print(acf(train.diff().dropna(), nlags=13)[1:].round(2))
```

```
autocorrélation d'ordre 1 de diff(bruit blanc) : -0.522    théorie : -0,5
autocorrélations de la différence première de log(ca) (décalages 1 à 13) :
[-0.24 -0.31  0.02  0.1   0.04 -0.18  0.04  0.11  0.07 -0.29 -0.25  0.75
 -0.19]
```

Un signe de sur-différenciation est donc une autocorrélation d'ordre 1 **fortement négative** (autour de  $-0,5$ ) et un modèle dont le coefficient MA est proche de  $-1$ . Gardons cela en tête.

#### 4.1.7 Décomposer : tendance, saison, reste

La **décomposition** sépare la série en trois composantes. Sur le logarithme (donc **multiplicative** sur la série d'origine) :

$$\log y_t = T_t + S_t + R_t \quad \Leftrightarrow \quad y_t = e^{T_t} \times e^{S_t} \times e^{R_t}.$$

La méthode **classique** procède en trois temps :

1. **Tendance**  $T_t$  : une moyenne mobile **centrée** sur 12 mois, qui « efface » la saison puisqu'elle contient un exemplaire de chaque mois. Pour une période paire, on utilise la moyenne mobile  $2 \times 12$  (poids  $1/24$  aux extrémités,  $1/12$  pour les 11 mois du milieu) afin de rester centré sur un mois.
2. **Saison**  $S_t$  : on retire la tendance, puis on **moyenne par mois calendaire** (tous les janviers, tous les février...), et on recentre pour que la somme sur l'année soit nulle.
3. **Reste**  $R_t = \log y_t - T_t - S_t$ .

```
def decomposition_manuelle(s, m=12):
    poids = np.r_[0.5, np.ones(m - 1), 0.5] / m                # moyenne mobile 2 x 12
    tendance = pd.Series(np.convolve(s.values, poids, mode="valid"), index=s.index[m // 2: -(m // 2)])
    sans_tendance = s - tendance
    saison = sans_tendance.groupby(sans_tendance.index.month).mean()
    saison = saison - saison.mean()
    return tendance, saison

tendance_m, saison_m = decomposition_manuelle(train)
dec = seasonal_decompose(train, model="additive", period=12)
print("tendance identique à statsmodels :", np.allclose(dec.trend.dropna(),
    ↪ tendance_m.reindex(dec.trend.dropna().index)))
saison_sm = dec.seasonal.groupby(dec.seasonal.index.month).first()
print("saison identique à statsmodels :", np.allclose(saison_sm.values, saison_m.values))
```

```
tendance identique à statsmodels : True
saison identique à statsmodels   : True
```

La méthode classique a deux défauts : elle perd 6 mois à chaque extrémité (la moyenne mobile centrée n'est pas définie), et elle est **sensible aux accidents** : le COVID de 2020 contamine à la fois la tendance et, via les moyennes par mois, la saison. La méthode **STL** (*Seasonal-Trend decomposition using LOESS*) remédie aux deux : elle estime tendance et saison par régressions locales, et son option `robust=True` **réduit le poids des observations aberrantes** pour que l'accident n'abîme pas les composantes.

```
stl = STL(train, period=12, robust=True).fit()
fig, axes = plt.subplots(4, 1, figsize=(10, 7.6), sharex=True)
for ax, (nom, comp, c) in zip(axes, [("log(ca)", train, BLEU), ("tendance", stl.trend, VIOLET),
    ("saison", stl.seasonal, AQUA), ("reste", stl.resid, ORANGE)]):
    ax.plot(comp.index, comp, color=c, lw=1.5)
    ax.set_ylabel(nom)
axes[3].axhline(0, color="#898781", lw=0.8)
axes[3].axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-06-30"), color=ORANGE, alpha=0.18,
    ↪ lw=0)
```

```
plt.tight_layout()
plt.savefig("figures/ch04-decomposition.png", bbox_inches="tight")

indices = (np.exp(stl.seasonal[:12]) * 100).round(0)
indices.index = ["jan", "fév", "mar", "avr", "mai", "juin", "juil", "août", "sept", "oct", "nov",
    ↪ "déc"]
print("indices saisonniers (100 = mois moyen) :")
print(indices.astype(int).to_string())
print()
print("les 5 plus gros résidus négatifs :")
print(stl.resid.nsmallest(5).round(2).to_string())
print("écart-type du reste : tous les mois =", round(stl.resid.std(), 3),
    "| hors mars-juin 2020 =", round(stl.resid.drop(stl.resid["2020-03":"2020-06"].index).std(), 3))
```

```
indices saisonniers (100 = mois moyen) :
jan      65
fév      80
mar     104
avr     106
mai     109
juin    119
juil    119
août    113
sept     84
oct     77
nov     100
déc     160

les 5 plus gros résidus négatifs :
mois
2020-06-01   -0.71
2020-05-01   -0.68
2020-04-01   -0.59
2020-03-01   -0.48
2023-05-01   -0.29
écart-type du reste : tous les mois = 0.146 | hors mars-juin 2020 = 0.077
```

La décomposition raconte l'histoire de la boutique : la **tendance** monte régulièrement, puis semble ralentir à partir de 2021 (STL est un lissage : sa fin de tendance est moins fiable, et nous reverrons en 4.3 si ce ralentissement est réel) ; la **saison** a un profil stable d'une année sur l'autre ; les **résidus** sont petits, sauf les quatre mois de mars à juin 2020 qui ressortent comme les quatre plus gros résidus négatifs. Les **indices saisonniers** se lisent directement : décembre est environ 60 % au-dessus du mois moyen, janvier et février environ 20 à 35 % en dessous.

**À quoi sert la décomposition ?** (1) **À comprendre** : « quelle part de la hausse est de la vraie croissance, quelle part est la saison ? ». (2) **À corriger des variations saisonnières** (CVS) : diviser par l'indice saisonnier pour comparer février à décembre. (3) **À repérer les accidents**, ici le COVID, que nous devons traiter explicitement dans les modèles.

#### À retenir.

- Une série temporelle a des observations **dépendantes** ; l'ordre fait partie de la donnée. Mettez de côté la fin de la série **avant** de choisir un modèle.
- La **stationnarité faible** : moyenne et variance constantes, autocovariance qui ne dépend que du décalage. La marche aléatoire n'est pas stationnaire ( $\text{Var} = t\sigma^2$ ).
- L'**autocorrélation** mesure la mémoire ; l'**autocorrélation partielle** est un coefficient de régression qui isole l'effet direct. Bande de confiance :  $\pm 1,96/\sqrt{n}$ .
- **Ljung-Box** teste globalement l'absence d'autocorrélation ; un bon modèle laisse des

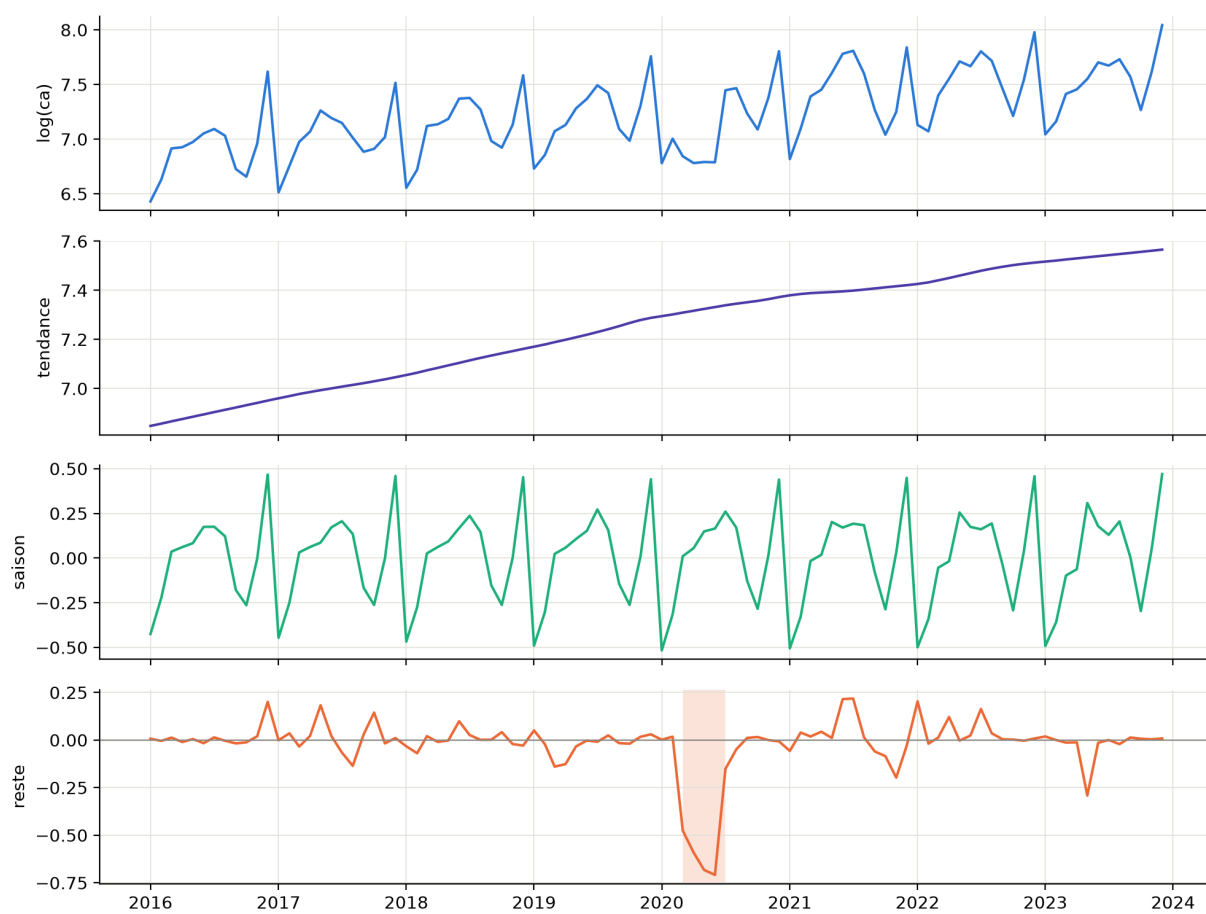


Figure 40: Décomposition STL robuste du logarithme du chiffre d'affaires (96 mois) : série, tendance, saison, reste. Le reste est petit partout, sauf pendant le COVID (bande orange).

résidus « bruit blanc ».

- **Différencier** retire une tendance stochastique ; **sur-différencier** crée une autocorrélation artificielle de  $-0,5$ . La statistique de Dickey-Fuller n'est **pas** de loi normale : son seuil à 5 % est d'environ  $-2,9$ .
- Les tests de racine unitaire (ADF, KPSS) ont **peu de puissance** sur de courtes séries : quand ils ne tranchent pas, gardez les deux hypothèses de travail et départagez-les par la prévision.
- On **décompose** (STL robuste) en tendance, saison et reste ; le logarithme transforme un schéma multiplicatif en schéma additif.

## 4.2 Modèles ARIMA et saisonniers

**Intuition.** Une série temporelle, c'est un enchaînement : ce qui se passe aujourd'hui dépend un peu de ce qui s'est passé hier, et un peu des **surprises** récentes. Les modèles ARIMA formalisent ces deux idées avec deux briques. La brique **AR** (*autorégressive*) dit : « la valeur d'aujourd'hui est un écho de celle d'hier ». La brique **MA** (*moyenne mobile, moving average*) dit : « la valeur d'aujourd'hui garde la trace des chocs d'hier ». On les assemble, on ajoute une différenciation (le **I** de *integrated*) et une version saisonnière, et on obtient la famille la plus utilisée de la prévision statistique.

Nous partons des briques sur des séries **simulées**, où l'on connaît la vérité, pour comprendre ce que chaque modèle fabrique et comment le reconnaître. Puis nous les appliquerons aux ventes de Dar Jasmin.

### 4.2.1 Les deux briques : AR et MA

Dans toute la suite,  $\varepsilon_t$  désigne un **bruit blanc** (4.1.5), les « surprises » du mois : des chocs indépendants, centrés, de variance  $\sigma^2$ .

- **AR(1)** :  $Y_t = \varphi Y_{t-1} + \varepsilon_t$ . La valeur du jour est une fraction  $\varphi$  de celle de la veille, plus une surprise. Si  $\varphi = 0,7$ , 70 % du niveau d'hier « survit » aujourd'hui.
- **AR(p)** :  $Y_t = \varphi_1 Y_{t-1} + \dots + \varphi_p Y_{t-p} + \varepsilon_t$  (une régression sur ses  $p$  propres valeurs passées).
- **MA(1)** :  $Y_t = \varepsilon_t + \theta \varepsilon_{t-1}$ . La valeur du jour est la surprise du jour plus une fraction  $\theta$  de la surprise d'hier. Un choc a donc un effet qui dure **exactement un pas**, puis disparaît.
- **MA(q)** :  $Y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$ .

**Deux mémoires différentes.** Dans un AR, un choc se **propage indéfiniment** en s'estompant (hier m'influence, qui a influencé avant-hier, etc.). Dans un MA( $q$ ), un choc a un effet **limité** à  $q$  pas. Pensez à un AR comme à une cloche qui résonne, et à un MA comme à un coup de tampon qui laisse une trace de durée fixe.

Comment reconnaître l'un ou l'autre sur des données ? Grâce à leurs **signatures** dans l'autocorrélation (ACF) et l'autocorrélation partielle (PACF) de 4.1.4 :

| Modèle         | ACF                                              | PACF                                             |
|----------------|--------------------------------------------------|--------------------------------------------------|
| AR( $p$ )      | décroît progressivement (géométriquement)        | <b>s'annule après le décalage <math>p</math></b> |
| MA( $q$ )      | <b>s'annule après le décalage <math>q</math></b> | décroît progressivement                          |
| ARMA( $p, q$ ) | décroît progressivement                          | décroît progressivement                          |



Vérifions-le. La fonction `ArmaProcess` de `statsmodels` calcule les autocorrélations **théoriques** d'un modèle ; nous les comparons à celles d'une trajectoire simulée de 500 points (les points), pour trois modèles : AR(1) avec  $\varphi = 0,7$ , AR(2) avec  $\varphi_1 = 0,5$  et  $\varphi_2 = 0,3$ , MA(1) avec  $\theta = 0,7$ .

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.arima_process import ArmaProcess, arma_acf, arma_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import acf, pacf
from statsmodels.stats.diagnostic import acorr_ljungbox

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
modeles_sim = {"AR(1), phi = 0,7": ([1, -0.7], [1]),
               "AR(2), phi = (0,5 ; 0,3)": ([1, -0.5, -0.3], [1]),
               "MA(1), theta = 0,7": ([1], [1, 0.7])} # convention de statsmodels : polynômes en
               ↪ B, AR avec signe moins

fig, axes = plt.subplots(2, 3, figsize=(11, 5.2), sharey=True)
for j, (nom, (ar, ma)) in enumerate(modeles_sim.items()):
    x = ArmaProcess(ar, ma).generate_sample(500, distrvs=np.random.default_rng(10 +
    ↪ j).standard_normal)
    for i, (theo, emp, titre, c) in enumerate([(arma_acf(ar, ma, 13), acf(x, nlags=12, fft=False),
    ↪ "ACF", BLEU),
                                           (arma_pacf(ar, ma, 13), pacf(x, nlags=12, method="ols"),
    ↪ "PACF", ORANGE)]):
        ax = axes[i, j]
        ax.vlines(np.arange(13), 0, theo, color=c, lw=2, alpha=0.55)
        ax.plot(np.arange(13), emp, "o", color=ROUGE, ms=3.5)
        ax.axhline(0, color="#898781", lw=0.8)
        ax.axhspan(-1.96 / np.sqrt(500), 1.96 / np.sqrt(500), color=c, alpha=0.10, lw=0)
        ax.set_title(f"{titre} - {nom}", fontsize=9.5)
        if i == 1:
            ax.set_xlabel("décalage")
plt.tight_layout()
plt.savefig("figures/ch04-signatures.png", bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Les signatures du tableau apparaissent nettement : la PACF de l'AR(1) n'a qu'une barre, celle de l'AR(2) en a deux, et l'ACF du MA(1) n'en a qu'une. Retenez cette figure : **c'est l'outil d'identification des modèles** que nous utiliserons en 4.2.4.

## 4.2.2 L'AR(1) et le MA(1) en profondeur

### L'AR(1) : stationnarité, variance, autocorrélations

**Proposition.** Soit  $Y_t = \varphi Y_{t-1} + \varepsilon_t$  avec  $|\varphi| < 1$ . Alors la série admet la représentation  $Y_t = \sum_{j=0}^{\infty} \varphi^j \varepsilon_{t-j}$ , elle est stationnaire, de moyenne 0 et de variance  $\gamma(0) = \frac{\sigma^2}{1 - \varphi^2}$ , et son autocorrélation est  $\rho(k) = \varphi^k$ .

**Démonstration.** En remplaçant  $Y_{t-1}$  par  $\varphi Y_{t-2} + \varepsilon_{t-1}$ , puis  $Y_{t-2}$ , etc., on obtient  $Y_t = \varepsilon_t +$

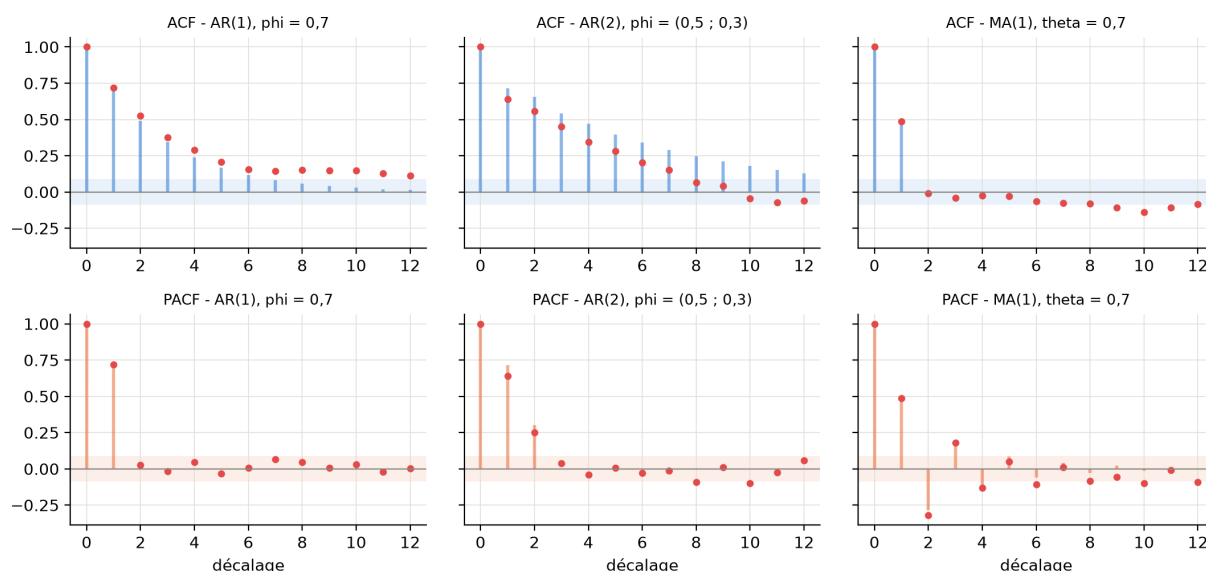


Figure 41: Signatures théoriques (barres) et empiriques (points rouges, 500 observations simulées) : l'AR(1) a une ACF qui décroît et une PACF qui s'arrête au décalage 1 ; l'AR(2) a une PACF qui s'arrête au décalage 2 ; le MA(1) a une ACF qui s'arrête au décalage 1 et une PACF qui décroît.

$\varphi \varepsilon_{t-1} + \varphi^2 \varepsilon_{t-2} + \dots$ . La série  $\sum \varphi^{2j}$  converge si et seulement si  $|\varphi| < 1$ , vers  $1/(1 - \varphi^2)$ . Les  $\varepsilon$  étant non corrélés,  $\text{Var}(Y_t) = \sigma^2 \sum_j \varphi^{2j} = \sigma^2/(1 - \varphi^2)$ , indépendant de  $t$  ; la moyenne est nulle. Pour l'autocovariance, on multiplie l'équation par  $Y_{t-k}$  ( $k \geq 1$ ) et on prend l'espérance :  $\gamma(k) = \varphi \gamma(k-1)$  (car  $\varepsilon_t$  est non corrélé avec  $Y_{t-k}$ ). Donc  $\gamma(k) = \varphi^k \gamma(0)$  et  $\rho(k) = \varphi^k$ . ■  
Si  $|\varphi| \geq 1$ , la somme diverge : la série **n'est pas stationnaire** (pour  $\varphi = 1$ , c'est la marche aléatoire de 4.1.3 ; pour  $|\varphi| > 1$ , elle explose).

**Exemple à la main.** Avec  $\varphi = 0,5$  et  $\sigma = 1$  :  $\gamma(0) = 1/(1 - 0,25) = 1,333$  ;  $\rho(1) = 0,5$ ,  $\rho(2) = 0,25$ ,  $\rho(3) = 0,125$ . La mémoire est divisée par deux à chaque pas, et un choc d'il y a trois mois n'a plus que 12,5 % d'influence.

```
ar, ma = [1, -0.5], [1]
print("autocorrélations théoriques de l'AR(1), phi = 0,5 :", arma_acf(ar, ma, 4).round(4))
x = ArmaProcess(ar, ma).generate_sample(200000, distrvs=np.random.default_rng(3).standard_normal)
print("variance : théorie", round(1 / (1 - 0.25), 3), "| simulation", round(x.var(), 3))
print("autocorrélations simulées (décalages 1 à 3) :", np.round([np.corrcoef(x[:-k], x[k:])[0, 1] for
↪ k in (1, 2, 3)], 3))
```

```
autocorrélations théoriques de l'AR(1), phi = 0,5 : [1.    0.5   0.25  0.125]
variance : théorie 1.333 | simulation 1.328
autocorrélations simulées (décalages 1 à 3) : [0.498 0.248 0.125]
```

Qu'est-ce que la condition  $|\varphi| < 1$  devient pour un  $\text{AR}(p)$  ? On écrit l'équation avec l'**opérateur retard**  $B$  (défini par  $BY_t = Y_{t-1}$ ) :  $\varphi(B)Y_t = \varepsilon_t$  avec  $\varphi(z) = 1 - \varphi_1 z - \dots - \varphi_p z^p$ . La série est stationnaire si et seulement si **toutes les racines du polynôme  $\varphi(z)$  sont en dehors du cercle unité** ( $|z| > 1$ ). Pour l'AR(1), l'unique racine est  $z = 1/\varphi$ , et  $|1/\varphi| > 1 \iff |\varphi| < 1$ . On retrouve la condition. Vérifions pour deux AR(2) :

```
for nom, (phi1, phi2) in {"phi = (0,5 ; 0,3)": (0.5, 0.3), "phi = (0,5 ; 0,6)": (0.5, 0.6)}.items():
    racines = np.roots([-phi2, -phi1, 1])
    # polynôme 1 - phi1 z - phi2 z^2
    verdict = "stationnaire" if np.all(np.abs(racines) > 1) else "NON stationnaire"
    print(f"AR(2) {nom} : modules des racines = {np.abs(racines).round(3)} -> {verdict}")
```

```
AR(2) phi = (0,5 ; 0,3) : modules des racines = [2.84 1.174] -> stationnaire
AR(2) phi = (0,5 ; 0,6) : modules des racines = [1.773 0.94 ] -> NON stationnaire
```

Le second modèle a une racine de module 0,94 < 1 : sa trajectoire diverge, même si chaque coefficient « a l'air raisonnable ».

### Le MA(1) : autocorrélation et inversibilité

**Proposition.** Pour  $Y_t = \varepsilon_t + \theta\varepsilon_{t-1}$  :  $\gamma(0) = \sigma^2(1 + \theta^2)$ ,  $\gamma(1) = \theta\sigma^2$ ,  $\gamma(k) = 0$  pour  $k \geq 2$ .  
Donc  $\rho(1) = \frac{\theta}{1 + \theta^2}$  et  $\rho(k) = 0$  au-delà. Un MA est toujours stationnaire.

**Démonstration.**  $\text{Var}(Y_t) = \sigma^2 + \theta^2\sigma^2$ . Et  $\text{Cov}(Y_t, Y_{t-1}) = \text{Cov}(\varepsilon_t + \theta\varepsilon_{t-1}, \varepsilon_{t-1} + \theta\varepsilon_{t-2}) = \theta\sigma^2$  (seul le terme  $\theta\varepsilon_{t-1} \cdot \varepsilon_{t-1}$  survit). ■

Une subtilité qui surprend : **deux valeurs de  $\theta$  donnent exactement la même autocorrélation**, car  $\theta/(1 + \theta^2)$  est inchangé si l'on remplace  $\theta$  par  $1/\theta$ . Pour  $\theta = 0,5$  comme pour  $\theta = 2$ , on a  $\rho(1) = 0,4$  :

```
for th in (0.5, 2.0):
    print(f"theta = {th} : rho(1) = {arma_acf([1], [1, th], 3)[1]}.3f} (formule : {th / (1 + th ** 2)}.3f}")
```

```
theta = 0.5 : rho(1) = 0.400 (formule : 0.400)
theta = 2.0 : rho(1) = 0.400 (formule : 0.400)
```

Les données ne permettent donc pas de distinguer les deux. Par convention, on retient celui qui est **inversible** :  $|\theta| < 1$ . Pourquoi ? Un MA(1) inversible peut se réécrire  $\varepsilon_t = \sum_{j \geq 0} (-\theta)^j Y_{t-j}$  : on peut **reconstituer les chocs passés à partir des observations**, ce qui est indispensable pour estimer et prévoir. Si  $|\theta| > 1$ , cette somme diverge. La condition est la même que pour l'AR, côté MA : les racines de  $\theta(z) = 1 + \theta_1 z + \dots$  doivent être hors du cercle unité.

### 4.2.3 ARMA, ARIMA et SARIMA

On assemble les briques. Avec l'opérateur retard  $B$  :

- **ARMA**( $p, q$ ) :  $\varphi(B) Y_t = \theta(B) \varepsilon_t$ .
- **ARIMA**( $p, d, q$ ) : on applique d'abord  $d$  différenciations :  $\varphi(B)(1 - B)^d Y_t = \theta(B) \varepsilon_t$ . Le « I » signifie que  $Y$  doit être *intégrée*  $d$  fois pour revenir de la série différenciée à  $Y$ .
- **SARIMA**( $p, d, q$ )( $P, D, Q$ ) <sub>$s$</sub>  : on ajoute des briques et des différences **saisonnnières**, de période  $s$  ( $s = 12$  pour des données mensuelles) :

$$\varphi(B) \Phi(B^s) (1 - B)^d (1 - B^s)^D Y_t = \theta(B) \Theta(B^s) \varepsilon_t.$$

**Le « modèle de la compagnie aérienne ».** Le SARIMA(0, 1, 1)(0, 1, 1)<sub>12</sub>, popularisé par Box et Jenkins sur les passagers aériens, s'écrit  $(1 - B)(1 - B^{12})Y_t = (1 + \theta B)(1 + \Theta B^{12})\varepsilon_t$ . En développant,  $Y_t - Y_{t-1} - Y_{t-12} + Y_{t-13} = \varepsilon_t + \theta\varepsilon_{t-1} + \Theta\varepsilon_{t-12} + \theta\Theta\varepsilon_{t-13}$ . Le membre de gauche compare le mois courant au mois précédent, et compare ce changement au changement observé un an plus tôt. Avec deux paramètres seulement, ce modèle décrit une tendance, une saison qui évolue lentement, et du bruit : il est le point de départ naturel des séries mensuelles.

### Comment estime-t-on ces modèles ?

Pour un AR(1), c'est une régression. Estimer  $\varphi$  revient à régresser  $y_t$  sur  $y_{t-1}$  par moindres carrés (c'est la méthode des *moindres carrés conditionnels*), ce qui donne  $\hat{\varphi} = \sum y_t y_{t-1} / \sum y_{t-1}^2$  pour une

série centrée. Pour les modèles avec une partie MA, les chocs  $\varepsilon_t$  ne sont pas observés, et l'on maximise la **vraisemblance gaussienne** : à chaque date, on calcule la prévision à un pas, et on suppose que l'erreur de prévision suit une loi normale ; la vraisemblance est le produit de ces densités. Le calcul récursif utilise le **filtre de Kalman**, que nous verrons en 4.5. Comparons les deux méthodes sur un AR(1) simulé :

```
x = ArmaProcess([1, -0.7], [1]).generate_sample(300,
↳ distrvs=np.random.default_rng(21).standard_normal)
phi_mco = (x[1:] * x[:-1]).sum() / (x[:-1] ** 2).sum()           # moindres carrés (série centrée en
↳ théorie)
phi_mv = ARIMA(x, order=(1, 0, 0), trend="n").fit().params[0]    # maximum de vraisemblance
print("vraie valeur : 0.7 | moindres carrés :", round(phi_mco, 4), "| maximum de vraisemblance :",
↳ round(phi_mv, 4))
```

vraie valeur : 0.7 | moindres carrés : 0.7119 | maximum de vraisemblance : 0.7097

Les deux estimations sont très proches l'une de l'autre (écart d'environ 0,002) et de la vraie valeur. Qu'en est-il de leur **précision** ? Un résultat classique (Kendall, 1954) dit que l'estimateur des moindres carrés d'un AR(1) est **biaisé vers zéro** :  $E[\hat{\varphi}] \approx \varphi - \frac{1+3\varphi}{n}$ . Vérifions avec 4 000 séries de 100 points et  $\varphi = 0,6$  :

```
rng = np.random.default_rng(11)
estimations = []
for _ in range(4000):
    e = rng.normal(size=150)
    s = np.zeros(150)
    for t in range(1, 150):
        s[t] = 0.6 * s[t - 1] + e[t]
    s = s[50:]
    d = s - s.mean()
    estimations.append((d[1:] * d[:-1]).sum() / (d[:-1] ** 2).sum())
print("phi = 0,6, n = 100 : moyenne des estimations =", round(np.mean(estimations), 4),
      "| formule 0,6 - (1 + 3 x 0,6)/100 =", round(0.6 - (1 + 3 * 0.6) / 100, 4),
      "| écart-type =", round(np.std(estimations), 3))
```

phi = 0,6, n = 100 : moyenne des estimations = 0.571 | formule 0,6 - (1 + 3 x 0,6)/100 = 0.572 |  
↳ écart-type = 0.084

Le biais est petit (environ  $-0,03$  pour  $n = 100$ ), mais réel : avec peu de données, un AR estimé paraît **moins persistant** qu'il ne l'est. Étudions maintenant un ARMA(1,1) ( $\varphi = 0,6$ ,  $\theta = 0,4$ ) : une estimation sur 500 points, puis la distribution des estimations sur 200 séries de 200 points.

```
x = ArmaProcess([1, -0.6], [1, 0.4]).generate_sample(500,
↳ distrvs=np.random.default_rng(12).standard_normal)
ajust = ARIMA(x, order=(1, 0, 1), trend="n").fit()
print(pd.DataFrame({"estimation": ajust.params, "erreur type": ajust.bse}, index=["phi", "theta",
↳ "sigma2"]).round(3))
print()
essais = []
for k in range(200):
    s = ArmaProcess([1, -0.6], [1, 0.4]).generate_sample(200, distrvs=np.random.default_rng(1000 +
↳ k).standard_normal)
    essais.append(ARIMA(s, order=(1, 0, 1), trend="n").fit().params[:2])
essais = np.array(essais)
print("200 séries de 200 points : moyenne des estimations (phi, theta) =", essais.mean(0).round(3),
      "| écart-type =", essais.std(0).round(3))
```

|        | estimation | erreur type |
|--------|------------|-------------|
| phi    | 0.582      | 0.044       |
| theta  | 0.447      | 0.053       |
| sigma2 | 0.951      | 0.062       |

200 séries de 200 points : moyenne des estimations (phi, theta) = [0.591 0.408] | écart-type = [0.076  
↪ 0.08 ]

Sur une seule série de 500 points, les estimations sont proches des vraies valeurs (0,6 et 0,4), à l'intérieur de leurs marges d'erreur. Sur 200 séries de 200 points, la moyenne est proche de la vérité, mais la **dispersion** est importante (de l'ordre de 0,08 pour chaque paramètre) : estimer les deux paramètres d'un ARMA est nettement plus incertain qu'estimer un AR seul, parce que  $\varphi$  et  $\theta$  jouent des rôles voisins.

**Un piège classique : la redondance.** Un ARMA(1,1) avec  $\varphi = \theta'$  (où le MA est « l'opposé » de l'AR) se simplifie :  $(1 - \varphi B)Y_t = (1 - \varphi B)\varepsilon_t$  donne  $Y_t = \varepsilon_t$ . Quand les deux racines sont presque égales, les paramètres ne sont plus identifiables. Si votre modèle estime  $\varphi \approx 0,9$  et  $\theta \approx -0,9$ , il est probablement trop gros : **simplifiez**.

#### 4.2.4 La méthode de Box-Jenkins appliquée aux ventes de Dar Jasmin

Box et Jenkins ont proposé un cycle en quatre temps : (1) **identifier** la structure à partir de l'ACF et de la PACF, (2) **estimer** les paramètres, (3) **diagnostiquer** les résidus, et, si tout est correct, (4) **prévoir**. Appliquons-le à notre série d'apprentissage (96 mois, en logarithme).

**Étape 1 : stationnariser.** D'après 4.1.6, la série a une tendance et une saison. Les deux différences ( $d = 1$  et  $D = 1, s = 12$ ) font disparaître la tendance et la saison **stochastiques**. Regardons ce qui reste :

```
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
y = np.log(v["ca"])
train, test = y["2023-12"], y["2024-01":]
X = v[["promo", "covid"]].astype(float)
Xtr = X["2023-12"]

def tracer_acf(ax, valeurs, n, titre, couleur=BLEU):
    k = np.arange(len(valeurs))
    ax.vlines(k, 0, valeurs, color=couleur, lw=2)
    ax.plot(k, valeurs, "o", color=couleur, ms=3.5)
    ax.axhline(0, color="#898781", lw=0.8)
    b = 1.96 / np.sqrt(n)
    ax.axhspan(-b, b, color=couleur, alpha=0.10, lw=0)
    ax.set_title(titre)
    ax.set_xlabel("décalage (mois)")
    ax.set_ylim(-1, 1)

w = train.diff().diff(12).dropna()
print("série différenciée (1 et 12) :", len(w), "observations ; bande de confiance +/-", round(1.96 /
    ↪ np.sqrt(len(w)), 3))
print("ACF décalages 1 à 14 :", acf(w, nlags=14, fft=False)[1:].round(2))
print("PACF décalages 1 à 14 :", pacf(w, nlags=14, method="ols")[1:].round(2))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.4))
tracer_acf(ax1, acf(w, nlags=24, fft=False), len(w), "ACF de la série différenciée (1 et 12)")
tracer_acf(ax2, pacf(w, nlags=24, method="ols"), len(w), "PACF de la série différenciée (1 et 12)",
    ↪ ORANGE)
plt.tight_layout()
plt.savefig("figures/ch04-acf-diff.png", bbox_inches="tight")
```

série différenciée (1 et 12) : 83 observations ; bande de confiance +/- 0.215  
ACF décalages 1 à 14 : [-0.01 -0.12 0. -0.23 -0.17 -0.08 0.15 0.02 0.04 0.14 -0.03 -0.43  
-0.02 -0.01]

PACF décalages 1 à 14 : [-0.01 -0.12 0. -0.25 -0.19 -0.18 0.09 -0.08 -0.02 0.06 0.01 -0.5 -0.02 -0.14]

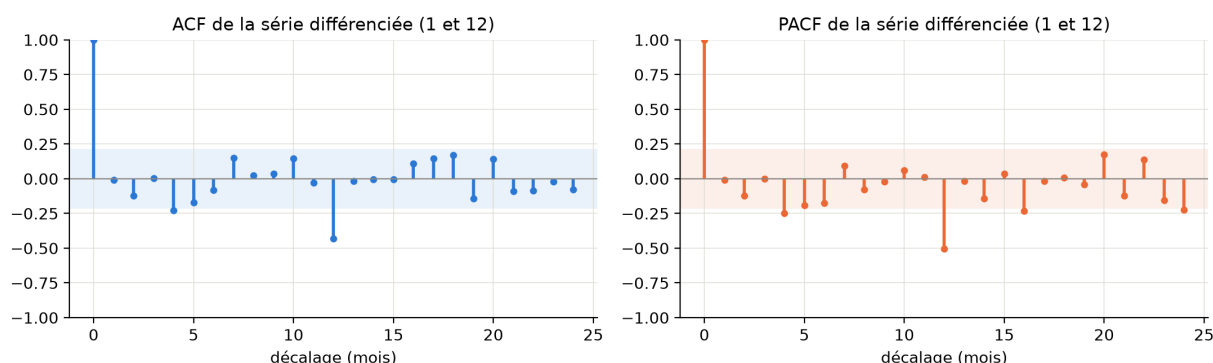


Figure 42: ACF et PACF de la série différenciée (différence première et différence saisonnière) : une barre négative nette au décalage 12 dans les deux graphiques ; les autres barres restent dans la bande ou l'effleurent.

**Lecture.** La série différenciée n'a plus de mémoire à long terme. Il reste une **seule structure nette** : une forte barre négative au décalage 12 dans l'ACF (environ  $-0,43$ ) et dans la PACF (environ  $-0,50$ ). Selon le tableau de 4.2.1, une ACF qui s'arrête au décalage 12 (un seul pic saisonnier) évoque un **MA saisonnier d'ordre 1** ( $Q = 1$ ). Aux décalages courts (1 à 3), rien ne dépasse la bande ; le décalage 4 l'effleure à peine (une barre sur quatorze : à ne pas sur-interpréter). La partie non saisonnière est donc modeste, peut-être nulle. On part donc du modèle de la compagnie aérienne,  $\text{SARIMA}(0, 1, 1)(0, 1, 1)_{12}$ , et de ses voisins.

**Étape 2 : estimer, avec des variables explicatives.** Nous savons qu'il y a eu un accident (COVID) et des promotions. Nous les ajoutons comme **variables explicatives** (*exogènes*) : le modèle devient une régression dont les erreurs suivent un SARIMA. (Nous détaillons cette idée en 4.2.5.) Ajustons une petite grille de modèles avec  $d = 1, D = 1, p, q \in \{0, 1, 2\}$  et  $Q \in \{0, 1\}$ , et comparons-les par **AIC** et **BIC** (chapitre 1, section 1.4 : ces critères mesurent l'ajustement en pénalisant la complexité ; plus petit = meilleur).

```
import itertools

lignes = []
for p, q, Q in itertools.product((0, 1, 2), (0, 1, 2), (0, 1)):
    m = SARIMAX(train, exog=Xtr, order=(p, 1, q), seasonal_order=(0, 1, Q, 12)).fit(dispatch=False,
    ↪ maxiter=200)
    lignes.append({"p": p, "q": q, "Q": Q, "AIC": m.aic, "BIC": m.bic, "convergé":
    ↪ m.mle_retvals["converged"]})
grille = pd.DataFrame(lignes).sort_values("AIC").reset_index(drop=True)
print("les 8 meilleurs modèles par AIC (d = 1, D = 1, s = 12) :")
print(grille.head(8).round(1).to_string())
print()
print("tous convergés :", bool(grille["convergé"].all()), "| meilleur par BIC : (p, q, Q) =",
    ↪ tuple(grille.sort_values("BIC").iloc[0][["p", "q", "Q"]].astype(int)))
```

les 8 meilleurs modèles par AIC (d = 1, D = 1, s = 12) :

|   | p | q | Q | AIC    | BIC    | convergé |
|---|---|---|---|--------|--------|----------|
| 0 | 1 | 1 | 1 | -176.7 | -162.2 | True     |
| 1 | 2 | 1 | 1 | -175.5 | -158.6 | True     |
| 2 | 1 | 2 | 1 | -175.5 | -158.6 | True     |
| 3 | 0 | 2 | 1 | -173.3 | -158.8 | True     |
| 4 | 2 | 2 | 1 | -172.9 | -153.5 | True     |
| 5 | 2 | 0 | 1 | -166.4 | -151.9 | True     |
| 6 | 0 | 1 | 1 | -165.9 | -153.8 | True     |

```
7 0 0 1 -164.7 -155.1 True
```

```
tous convergés : True | meilleur par BIC : (p, q, Q) = (1, 1, 1)
```

Le modèle SARIMAX(1,1,1)(0,1,1)<sub>12</sub> arrive en tête à la fois par l'AIC et par le BIC. Voici ses coefficients :

```
mod_A = SARIMAX(train, exog=Xtr, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(dispatch=False)
print(mod_A.summary().tables[1])
```

|          | coef    | std err | z       | P> z  | [0.025 | 0.975] |
|----------|---------|---------|---------|-------|--------|--------|
| promo    | 0.1061  | 0.022   | 4.887   | 0.000 | 0.064  | 0.149  |
| covid    | -0.5371 | 0.037   | -14.464 | 0.000 | -0.610 | -0.464 |
| ar.L1    | 0.5583  | 0.115   | 4.870   | 0.000 | 0.334  | 0.783  |
| ma.L1    | -0.9919 | 0.331   | -3.001  | 0.003 | -1.640 | -0.344 |
| ma.S.L12 | -0.8889 | 0.262   | -3.389  | 0.001 | -1.403 | -0.375 |
| sigma2   | 0.0047  | 0.002   | 2.383   | 0.017 | 0.001  | 0.009  |

**Les deux coefficients MA sont proches de -1 : un signal d'alerte.** `ma.L1` vaut environ -0,99 et `ma.S.L12` environ -0,89, avec de très grandes incertitudes (erreurs types de l'ordre de 0,3). Rappelez-vous la sur-différenciation de 4.1.6 : différencier une série qui n'en avait pas besoin crée un MA dont le coefficient est -1. Un MA non saisonnier proche de -1 suggère que la **différence première** est de trop (la tendance serait **déterministe**, une droite, et non un cumul de chocs) ; un MA **saisonnier** proche de -1 suggère que la **différence saisonnière** l'est aussi (la saison serait **déterministe** : le même profil chaque année, fixé une fois pour toutes, et non un profil qui évolue). Le modèle a trouvé le meilleur ajustement *parmi les modèles différenciés*, mais ses propres coefficients nous disent qu'il compense une différenciation excessive. Nous allons mettre cette hypothèse à l'épreuve en 4.2.7.

#### 4.2.5 Variables explicatives : la régression à erreurs ARMA

Le modèle que vient d'ajuster SARIMAX est une **régression à erreurs ARIMA** :

$$\log y_t = \beta_1 \text{promo}_t + \beta_2 \text{covid}_t + \eta_t, \quad \eta_t \sim \text{SARIMA}(1,1,1)(0,1,1)_{12}.$$

Les variables explicatives interviennent **en niveau** (pas différenciées) et l'ARIMA décrit ce qui reste. C'est la bonne façon de combiner une régression (chapitre 1) et de la mémoire temporelle : si l'on ignorait la mémoire des erreurs, les erreurs types de la régression seraient fausses (le volume I, section 2.4, suppose l'indépendance).

Les coefficients se lisent comme des pourcentages, puisque la variable dépendante est un logarithme : un coefficient  $\beta$  correspond à un facteur  $e^\beta$ , soit une variation de  $100(e^\beta - 1)\%$  :

```
for nom in ("promo", "covid"):
    b = mod_A.params[nom]
    lo, hi = mod_A.conf_int().loc[nom]
    print(f"{nom:6s} : coefficient = {b:+.3f} -> effet = {100 * (np.exp(b) - 1):+.1f} % (IC 95 % :
    ↳ {100 * (np.exp(lo) - 1):+.1f} % à {100 * (np.exp(hi) - 1):+.1f} %)")
```

```
promo : coefficient = +0.106 -> effet = +11.2 % (IC 95 % : +6.6 % à +16.0 %)
covid : coefficient = -0.537 -> effet = -41.6 % (IC 95 % : -45.7 % à -37.1 %)
```

Une promotion est associée à des ventes environ 11 % plus élevées le mois où elle a lieu, et les quatre mois de COVID à des ventes environ 42 % plus basses, toutes choses égales par ailleurs.



Sans ces deux variables, le modèle aurait dû « expliquer » le COVID par du bruit, ce qui aurait dégradé tous les paramètres. Mesurons-le :

```
sans = SARIMAX(train, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(dispatch=False)
print("AIC sans variables explicatives :", round(sans.aic, 1), "| avec promo et COVID :",
      ↪ round(mod_A.aic, 1))
print("écart-type estimé des chocs (sigma) : sans =", round(np.sqrt(sans.params['sigma2']), 3), "|
      ↪ avec =", round(np.sqrt(mod_A.params['sigma2']), 3))
```

```
AIC sans variables explicatives : -98.1 | avec promo et COVID : -176.7
écart-type estimé des chocs (sigma) : sans = 0.107 | avec = 0.068
```

**Les variables explicatives doivent être connues pour prévoir.** Pour prévoir 2026, il faudra fournir la valeur future de promo (les promotions sont décidées par Yasmine : on peut les supposer connues ou raisonner en scénarios) et de covid (nulle). Une variable explicative inconnue dans le futur est inutilisable telle quelle : il faudrait la prévoir elle-même.

### 4.2.6 Le diagnostic des résidus

Un modèle n'est pas fini quand il est ajusté : il faut vérifier qu'il a **tout expliqué**, c'est-à-dire que ses résidus ressemblent à un bruit blanc gaussien. On regarde quatre choses : les résidus dans le temps (pas de structure, variance stable), l'ACF des résidus (pas de barre hors bande), la normalité (histogramme et QQ-plot) et le test de Ljung-Box. Les premiers résidus d'un SARIMA avec différences (ici 13) n'ont pas de sens (ils servent à initialiser le calcul) : on les ignore.

```
res = mod_A.resid.iloc[13:] # on ignore les 13 premiers (initialisation des
    ↪ différences)
sig = res.std()
fig, axes = plt.subplots(2, 2, figsize=(11, 6))
axes[0, 0].plot(res.index, res / sig, color=BLEU, lw=1.2)
axes[0, 0].axhline(0, color="#898781", lw=0.8)
axes[0, 0].set_title("Résidus standardisés")
axes[0, 1].hist(res / sig, bins=18, density=True, color=BLEU, alpha=0.7)
xs = np.linspace(-4, 4, 200)
axes[0, 1].plot(xs, stats.norm.pdf(xs), color=ORANGE, lw=2)
axes[0, 1].set_title("Histogramme et loi normale")
sm.qqplot(res / sig, line="45", ax=axes[1, 0], markerfacecolor=BLEU, markeredgecolor=BLEU, ms=4)
axes[1, 0].set_title("QQ-plot")
axes[1, 0].set_xlabel("quantiles théoriques")
axes[1, 0].set_ylabel("quantiles observés")
tracer_acf(axes[1, 1], acf(res, nlags=24, fft=False), len(res), "ACF des résidus", ORANGE)
plt.tight_layout()
plt.savefig("figures/ch04-diagnostics.png", bbox_inches="tight")

nb_param = 3 # phi, theta, Theta : les paramètres AR/MA estimés
lb = acorr_ljungbox(res, lags=[12, 24], model_df=nb_param)
print("Ljung-Box sur les résidus (degrés de liberté réduits de", nb_param, ") :")
print(lb.round(3).to_string())
print("Shapiro-Wilk (normalité) : p =", round(stats.shapiro(res).pvalue, 3))
print("plus grand résidu standardisé en valeur absolue :", round((res / sig).abs().max(), 2), "le",
      ↪ (res / sig).abs().idxmax().strftime("%Y-%m"))
```

```
Ljung-Box sur les résidus (degrés de liberté réduits de 3) :
  lb_stat lb_pvalue
12    8.923    0.444
24   22.329    0.381
Shapiro-Wilk (normalité) : p = 0.334
plus grand résidu standardisé en valeur absolue : 2.37 le 2017-12
```

Les résidus ne montrent pas de structure : l'ACF reste dans la bande, les p-valeurs de Ljung-Box

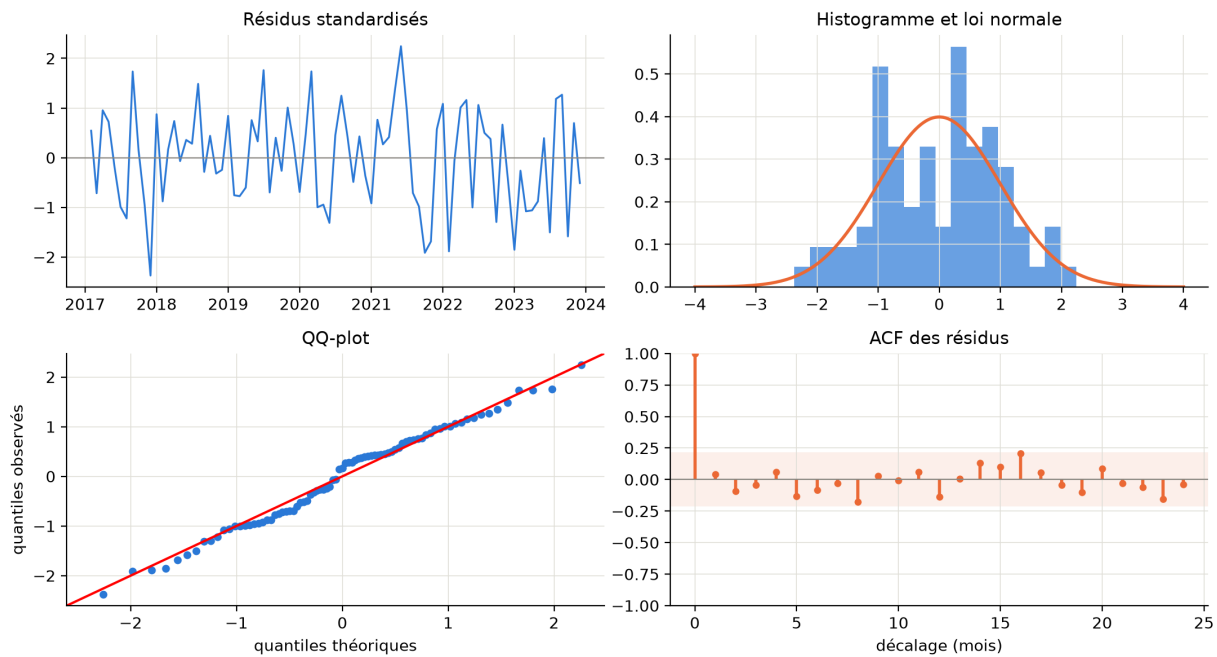


Figure 43: Diagnostic des résidus du SARIMAX(1,1,1)(0,1,1)<sub>12</sub> : résidus standardisés sans structure, histogramme proche de la loi normale, QQ-plot proche de la droite, ACF sans barre hors bande.

sont grandes (on ne rejette pas « bruit blanc »), la normalité n'est pas rejetée. Le modèle a donc **capté l'essentiel de la dynamique linéaire** de la série d'apprentissage.

**Un bon diagnostic ne prouve pas un bon modèle.** Des résidus blancs montrent qu'il ne reste pas d'**autocorrélation exploitable**, pas que le modèle **prévoira bien** : un modèle trop flexible peut avoir des résidus parfaits sur l'apprentissage et prévoir mal (surajustement). Seule la prévision hors échantillon (4.3) le dira.

#### 4.2.7 Deux visions du monde : saison stochastique ou déterministe ?

Nous avons soupçonné plusieurs fois que la tendance et la saison sont **déterministes** (4.1.6 : tests ambigus ; 4.2.4 : deux coefficients MA proches de  $-1$ ). Construisons deux modèles alternatifs, qui **différencient moins** :

- **Modèle B** : SARIMAX(1,0,0)(0,1,1)<sub>12</sub> avec tendance linéaire (trend="ct") : on garde la différence saisonnière mais on modélise la tendance par une droite.
- **Modèle C** : régression sur **tendance linéaire + 11 indicatrices de mois + promo + COVID**, avec des **erreurs AR(1)**. Ici, la saison est un profil fixe (11 coefficients) et aucune différenciation n'est appliquée.

```
mois = pd.get_dummies(pd.Series(v.index.month, index=v.index).astype(str).str.zfill(2), prefix="m",
    <- drop_first=True, dtype=float)
XX = pd.concat([pd.Series(1.0, index=v.index, name="const"),
    pd.Series(np.arange(len(v), dtype=float), index=v.index, name="t"), mois, X], axis=1)

mod_B = SARIMAX(train, exog=Xtr, order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
    <- trend="ct").fit(dispatch=False, maxiter=300)
mod_C = ARIMA(train, exog=XX[:"2023-12"], order=(1, 0, 0), trend="n").fit()

tableau = pd.DataFrame({
    "modèle A : SARIMAX(1,1,1)(0,1,1)": [mod_A.params["promo"], mod_A.params["covid"]],
```

```

    "modèle B : (1,0,0)(0,1,1) + tendance": [mod_B.params["promo"], mod_B.params["covid"]],
    "modèle C : régression + AR(1)": [mod_C.params["promo"], mod_C.params["covid"]], index=["promo",
    ↪ "covid"])
print(tableau.round(3).T.to_string())
print()
print("modèle C : pente de la tendance =", round(mod_C.params["t"], 4), "par mois, soit", round(100 *
    ↪ (np.exp(12 * mod_C.params["t"]) - 1), 1), "% par an ;",
    "AR(1) : phi =", round(mod_C.params["ar.L1"], 3), "| sigma =",
    ↪ round(np.sqrt(mod_C.params["sigma2"]), 3))
print("AIC : A =", round(mod_A.aic, 1), "| B =", round(mod_B.aic, 1), "| C =", round(mod_C.aic, 1))

```

|                                      | promo | covid  |
|--------------------------------------|-------|--------|
| modèle A : SARIMAX(1,1,1)(0,1,1)     | 0.106 | -0.537 |
| modèle B : (1,0,0)(0,1,1) + tendance | 0.100 | -0.506 |
| modèle C : régression + AR(1)        | 0.106 | -0.541 |

modèle C : pente de la tendance = 0.0077 par mois, soit 9.7 % par an ; AR(1) : phi = 0.539 | sigma =  
 ↪ 0.06  
 AIC : A = -176.7 | B = -174.3 | C = -232.5

Les trois modèles s'accordent sur les effets de la promotion (coefficient d'environ 0,10 à 0,11, soit +10 à +11 %) et du COVID (coefficient d'environ -0,51 à -0,54, soit -40 à -42 %). Ils diffèrent sur la **dynamique**. Le modèle C décrit une croissance régulière de l'ordre de 10 % par an, un AR(1) modéré, et il obtient l'AIC le plus bas de loin. Attention : **cette comparaison d'AIC n'a pas de sens**.

**On ne compare pas des AIC entre modèles de différenciations différentes.** L'AIC est calculé à partir de la vraisemblance des **données modélisées**. Le modèle A modélise une série différenciée deux fois, le modèle B une série différenciée une fois, le modèle C la série non différenciée : les vraisemblances portent sur des objets différents, et leurs valeurs ne sont pas comparables (même si le logiciel les affiche sans protester). L'AIC ne peut départager que des modèles de **même d et même D** (comme dans la grille de 4.2.4). Pour comparer A, B et C, il n'y a **qu'un seul juge** légitime : leurs **prévisions** sur des données qu'ils n'ont pas vues. C'est l'objet de 4.3.

#### 4.2.8 Le même travail en R : `auto.arima`

Les statisticiens utilisent aussi R, où le paquet `forecast` propose `auto.arima`, qui automatise la grille de 4.2.4 (et choisit même les différences par des tests de racine unitaire). Voici ce que donne la fonction sur la même série d'apprentissage, avec les mêmes variables explicatives :

```

suppressPackageStartupMessages(library(forecast))
d <- read.csv("donnees/ventes_mensuelles.csv")
d <- d[1:96, ] # les 96 mois d'apprentissage
y <- ts(log(d$ca), start = c(2016, 1), frequency = 12)
xreg <- as.matrix(d[, c("promo", "covid")])
modele <- auto.arima(y, xreg = xreg, seasonal = TRUE, ic = "aicc")
print(modele)

```

```

Series: y
Regression with ARIMA(0,0,1)(0,1,1)[12] errors

Coefficients:
      mal      smal  drift  promo  covid
 0.4942 -0.8836 0.0078 0.0976 -0.5425
s.e. 0.0798 0.2637 0.0004 0.0210 0.0459

sigma^2 = 0.005107: log likelihood = 96.73
AIC=-181.47 AICc=-180.38 BIC=-166.88

```

R a retenu un **ARIMA**(0,0,1)(0,1,1)<sub>12</sub> **avec une dérive** (une tendance linéaire, drift) : pas de différence première, une différence saisonnière, un MA(1) de coefficient d'environ 0,49 et un MA saisonnier d'environ -0,88. Les effets estimés sont voisins des nôtres (promo : 0,098 ; COVID : -0,54). Ce choix automatique, fondé sur des tests de racine unitaire pour  $d$  et  $D$ , **va dans le même sens que nos soupçons** : la tendance se modélise bien par une dérive déterministe, sans différence première. Il diffère de la grille de 4.2.4, où nous avons *imposé*  $d = 1$  ; `auto.arima` utilise de plus l'AICc (une version de l'AIC corrigée pour les petits échantillons). Ce n'est pas une contradiction, mais un rappel : **la sélection automatique n'est pas une vérité**, seulement un point de départ à confronter au diagnostic et à la prévision.

#### À retenir.

- **AR** : la valeur d'aujourd'hui est un écho de ses propres valeurs passées ; **MA** : elle garde la trace des chocs récents. Signatures : un  $AR(p)$  a une PACF qui s'arrête à  $p$ , un  $MA(q)$  une ACF qui s'arrête à  $q$ .
- **AR(1) stationnaire**  $\Leftrightarrow |\varphi| < 1$ , avec  $\gamma(0) = \sigma^2/(1 - \varphi^2)$  et  $\rho(k) = \varphi^k$ . **MA inversible**  $\Leftrightarrow |\theta| < 1$  ; deux valeurs  $\theta$  et  $1/\theta$  ont la même ACF. Pour un  $AR(p)$  ou  $MA(q)$ , les racines du polynôme doivent être hors du cercle unité.
- **SARIMA**( $p, d, q$ )( $P, D, Q$ )<sub>s</sub> :  $\varphi(B)\Phi(B^s)(1-B)^d(1-B^s)^D Y_t = \theta(B)\Theta(B^s)\varepsilon_t$ . L'estimation se fait par maximum de vraisemblance ; l'estimateur d'un  $AR(1)$  est biaisé vers zéro d'environ  $(1 + 3\varphi)/n$ .
- **Box-Jenkins** : stationnariser, identifier avec ACF/PACF, estimer plusieurs candidats, comparer par AIC/BIC (**à même  $d$  et  $D$  seulement**), diagnostiquer les résidus (Ljung-Box avec degrés de liberté réduits du nombre de paramètres AR et MA).
- Une **régression à erreurs ARIMA** (SARIMAX avec variables explicatives) combine les effets de variables connues et la mémoire des erreurs ; les coefficients en log se lisent en pourcentages.
- Un coefficient MA proche de -1 signale souvent une **sur-différenciation** : ici, une saison qui pourrait être déterministe.
- Des résidus blancs ne garantissent pas de bonnes prévisions. **Le juge de paix est la prévision hors échantillon** (4.3).

## 4.3 Prévision et évaluation des prévisions

**Intuition.** Prévoir, ce n'est pas deviner un nombre : c'est annoncer **un nombre et son incertitude**. « Les ventes de décembre seront de 3 300 DT, avec 95 % de chances de tomber entre 2 800 et 3 900 » est une prévision utile ; « 3 300 » tout court n'en est pas une. Et un modèle ne se juge pas à la beauté de son ajustement passé, mais à la qualité des prévisions qu'il fait sur des données qu'il **n'a pas vues**.

Dans cette section, nous faisons trois choses : comprendre comment un modèle ARIMA prévoit et d'où viennent ses intervalles (4.3.1 et 4.3.2) ; mettre en place une **évaluation honnête** (4.3.3 à 4.3.6) ; puis prévoir 2026 pour Yasmine (4.3.7) et **lever le voile** sur la fabrication des données (4.3.8).

### 4.3.1 Comment un modèle prévoit : l'exemple de l'AR(1)

La meilleure prévision ponctuelle de  $Y_{T+h}$  à partir de ce qu'on sait à la date  $T$  est l'**espérance conditionnelle**  $\hat{y}_{T+h} = E[Y_{T+h} \mid Y_T, Y_{T-1}, \dots]$  : c'est elle qui minimise l'erreur quadratique moyenne (chapitre 1, section 1.1, avec le passé comme variables explicatives). Pour un  $AR(1)$  de moyenne  $\mu$ , on peut tout calculer.

**Prévision d'un AR(1).** Soit  $Y_t - \mu = \varphi(Y_{t-1} - \mu) + \varepsilon_t$  avec  $|\varphi| < 1$ . En remplaçant récursivement,  $Y_{T+h} - \mu = \varphi^h(Y_T - \mu) + \sum_{j=0}^{h-1} \varphi^j \varepsilon_{T+h-j}$ . Les chocs futurs  $\varepsilon_{T+1}, \dots, \varepsilon_{T+h}$  sont d'espérance nulle et indépendants du passé, donc

$$\hat{y}_{T+h} = \mu + \varphi^h(Y_T - \mu), \quad \text{Var}(Y_{T+h} - \hat{y}_{T+h}) = \sigma^2 \sum_{j=0}^{h-1} \varphi^{2j} = \sigma^2 \frac{1 - \varphi^{2h}}{1 - \varphi^2}.$$

La prévision **revient vers la moyenne** à vitesse géométrique, et l'incertitude **croît** de  $\sigma^2$  (à un pas) vers la variance de la série elle-même,  $\sigma^2/(1 - \varphi^2)$ . ■

**Exemple à la main.** Avec  $\varphi = 0,6$ ,  $\mu = 0$ ,  $\sigma = 1$  et  $Y_T = 2$  :  $\hat{y}_{T+1} = 0,6 \times 2 = 1,2$  ;  $\hat{y}_{T+2} = 0,6 \times 1,2 = 0,72$  ;  $\hat{y}_{T+3} = 0,432$ . Les demi-largeurs des intervalles à 95 % sont  $1,96\sqrt{1} = 1,96$  ;  $1,96\sqrt{1 + 0,36} \approx 2,286$  ;  $1,96\sqrt{1 + 0,36 + 0,1296} \approx 2,392$ , et tendent vers  $1,96/\sqrt{1 - 0,36} = 2,45$ . L'avenir lointain n'est pas plus prévisible que « la valeur moyenne, avec la dispersion habituelle ».

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
phi, yT = 0.6, 2.0
# un AR(1) dont on FIXE les paramètres (phi = 0,6 ; sigma^2 = 1), pour que statsmodels applique
# exactement la formule
fixe = ARIMA(np.r_[np.zeros(50), yT], order=(1, 0, 0), trend="n").filter([phi, 1.0])
prev = fixe.get_forecast(3)
print("prévisions statsmodels :", prev.predicted_mean.round(3))
print("demi-largeurs (statsmodels) :", ((prev.conf_int()[1, :] - prev.conf_int()[0, :]) / 2).round(3))
print("prévisions (formule) :", np.round([phi ** h * yT for h in (1, 2, 3)], 3))
print("demi-largeurs (formule) :", np.round([1.96 * np.sqrt(sum(phi ** (2 * j) for j in range(h)))
# for h in (1, 2, 3)], 3))
print("limite 1,96 / racine(1 - phi^2) :", round(1.96 / np.sqrt(1 - phi ** 2), 3))

prévisions statsmodels : [1.2  0.72  0.432]
demi-largeurs (statsmodels) : [1.96  2.286  2.392]
prévisions (formule) : [1.2  0.72  0.432]
demi-largeurs (formule) : [1.96  2.286  2.392]
limite 1,96 / racine(1 - phi^2) : 2.45
```

Pour un MA(1), la prévision à un pas utilise le dernier choc, et **au-delà d'un pas la prévision est la moyenne** (le choc n'a plus d'effet). Et pour une série **différenciée** ( $d = 1$ ), la prévision n'est pas stationnaire : c'est la dernière valeur (plus une éventuelle dérive), et l'incertitude ne se stabilise **jamais**. Comparons la demi-largeur d'un intervalle à 95 % pour une marche aléatoire ( $\sigma = 1$ ) et pour l'AR(1) précédent :

```
print(" horizon  marche aléatoire  AR(1), phi = 0,6")
for h in (1, 3, 12, 60):
    print(f"{h:8d}  {1.96 * np.sqrt(h):16.2f}  {1.96 * np.sqrt((1 - phi ** (2 * h)) / (1 - phi **
# 2)):16.2f}")
```

| horizon | marche aléatoire | AR(1), phi = 0,6 |
|---------|------------------|------------------|
| 1       | 1.96             | 1.96             |
| 3       | 3.39             | 2.39             |
| 12      | 6.79             | 2.45             |
| 60      | 15.18            | 2.45             |

**Conséquence pratique.** Un modèle avec une différence (première ou saisonnière) a des intervalles de prévision qui **s'élargissent sans limite** quand l'horizon grandit ; un modèle stationnaire autour d'une tendance et d'une saison **déterministes** a des intervalles qui se stabilisent. Parmi nos trois modèles de 4.2, A (deux différences) et B (une différence saisonnière) sont du premier type, et C (aucune différence) du second. Choisir entre les deux, c'est choisir ce que l'on croit de l'avenir lointain : des chocs qui laissent une trace **permanente** ou qui s'effacent.

### 4.3.2 Prévoir en logarithme, annoncer en dinars

Nos modèles prévoient  $\log(\text{ca})$ . Pour annoncer des dinars, on revient par l'exponentielle. Deux précautions :

- **Les intervalles** se transforment sans difficulté : si  $[L, U]$  est un intervalle à 95 % pour  $\log y$ , alors  $[e^L, e^U]$  en est un pour  $y$  (l'exponentielle est croissante).
- **La prévision ponctuelle**  $e^{\hat{y}}$  est la **médiane** de la loi de  $Y$ , pas sa moyenne : si  $\log Y \sim \mathcal{N}(m, s^2)$ , alors  $\mathbb{E}[Y] = e^{m+s^2/2}$ . Pour des erreurs de l'ordre de 0,07 en log, le facteur  $e^{s^2/2}$  vaut environ 1,0025 : négligeable ici. Il ne le serait pas pour une série plus volatile.

Voyons ce que donnent nos trois modèles (A, B, C de 4.2) sur les 24 mois mis de côté. Rappelons les objets : train (96 mois), test (24 mois), mod\_A, mod\_B, mod\_C ajustés **sur train seulement**.

```
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
y = np.log(v["ca"])
train, test = y[:"2023-12"], y["2024-01":]
X = v[["promo", "covid"]].astype(float)
Xte = X["2024-01":] # les promotions de 2024-2025 sont supposées
↳ connues à l'avance
mois_ind = pd.get_dummies(pd.Series(v.index.month, index=v.index).astype(str).str.zfill(2),
↳ prefix="m", drop_first=True, dtype=float)
XX = pd.concat([pd.Series(1.0, index=v.index, name="const"),
                pd.Series(np.arange(len(v), dtype=float), index=v.index, name="t"), mois_ind, X],
↳ axis=1)

prev_A = mod_A.get_forecast(24, exog=Xte)
prev_B = mod_B.get_forecast(24, exog=Xte)
prev_C = mod_C.get_forecast(24, exog=XX["2024-01":])
previsions = {"A : SARIMAX(1,1,1)(0,1,1)": prev_A, "B : (1,0,0)(0,1,1) + tendance": prev_B, "C :
↳ régression + AR(1)": prev_C}

print("facteur de correction moyenne/médiane exp(s^2/2) pour C, 1er et 24e mois :",
      np.round(np.exp(0.5 * prev_C.se_mean.values[[0, 23]] ** 2), 4))
couverture = {}
for nom, p in previsions.items():
    ic = np.asarray(p.conf_int(alpha=0.05))
    dedans = (test.values >= ic[:, 0]) & (test.values <= ic[:, 1])
    couverture[nom] = dedans.sum()
    print(f"{nom:34s} : {dedans.sum():2d} mois sur 24 dans l'intervalle à 95 % (rapport moyen borne
↳ haute / borne basse : x{np.exp(ic[:, 1] - ic[:, 0]).mean():.2f})")

facteur de correction moyenne/médiane exp(s^2/2) pour C, 1er et 24e mois : [1.0018 1.0026]
A : SARIMAX(1,1,1)(0,1,1) : 22 mois sur 24 dans l'intervalle à 95 % (rapport moyen borne
↳ haute / borne basse : x1.41)
```



B : (1,0,0)(0,1,1) + tendance : 24 mois sur 24 dans l'intervalle à 95 % (rapport moyen borne  
↪ haute / borne basse : x1.43)  
C : régression + AR(1) : 21 mois sur 24 dans l'intervalle à 95 % (rapport moyen borne  
↪ haute / borne basse : x1.32)

**La couverture.** Un intervalle à 95 % **bien calibré** devrait contenir la vraie valeur 95 % du temps. Sur 24 mois, on attend 22 à 23 mois dedans. Observer 21, 22 ou 24 n'est pas une preuve de mauvaise calibration : avec 24 points **corrélés** entre eux, l'incertitude sur la couverture est très grande. Remarquez aussi que les intervalles des modèles qui différencient (A et B, rapports moyens de 1,41 et 1,43) sont plus larges que celui de C (1,32) : conséquence de la remarque de 4.3.1. À retenir : la couverture se vérifie sur de **longues** périodes ou sur **beaucoup** de séries.

### 4.3.3 Le découpage temporel : ne jamais mélanger le passé et l'avenir

En régression ordinaire (chapitre 1), on peut tirer au hasard 20 % des observations pour les tenir à l'écart et évaluer le modèle dessus : les observations sont indépendantes. En série temporelle, **c'est une faute** : un mois « tenu à l'écart » a ses voisins dans l'échantillon d'apprentissage, et le modèle peut les interpoler. On estime alors la capacité du modèle à **combler un trou**, pas à **prévoir**.

Voyons cela sur un cas où le piège est spectaculaire. Ajustons des régressions où la tendance est un **polynôme** de degré 1, 3, 6 ou 10 (plus saison et COVID) sur nos 96 mois d'apprentissage, et évaluons-les de deux façons : **(a)** en tenant à l'écart 24 mois tirés au hasard ; **(b)** en apprenant sur les 72 premiers mois et en prévoyant les 24 suivants (donc toujours *dans* la période d'apprentissage : nous ne touchons pas aux données de test).

```
tt = np.arange(96) / 95 # le temps ramené à [0, 1]
M = mois_ind.values[:96]
covid96 = X["covid"].values[:96]

def dessin(idx, deg):
    return np.column_stack([tt[idx] ** d for d in range(deg + 1)] + [M[idx], covid96[idx]])

def eqm(a, b):
    return np.sqrt(np.mean((np.asarray(a) - np.asarray(b)) ** 2))

rng = np.random.default_rng(0)
lignes = []
for deg in (1, 3, 6, 10):
    hasard = []
    for _ in range(50):
        test_i = rng.choice(96, 24, replace=False)
        app_i = np.setdiff1d(np.arange(96), test_i)
        b = np.linalg.lstsq(dessin(app_i, deg), train.values[app_i], rcond=None)[0]
        hasard.append(eqm(train.values[test_i], dessin(test_i, deg) @ b))
    app_i, test_i = np.arange(72), np.arange(72, 96)
    b = np.linalg.lstsq(dessin(app_i, deg), train.values[app_i], rcond=None)[0]
    lignes.append({"degré du polynôme": deg, "RMSE (log), 24 mois au hasard": np.mean(hasard),
                  "RMSE (log), 24 derniers mois": eqm(train.values[test_i], dessin(test_i, deg) @ b)})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

| degré du polynôme | RMSE (log), 24 mois au hasard | RMSE (log), 24 derniers mois |
|-------------------|-------------------------------|------------------------------|
| 1                 | 0.093                         | 0.102                        |
| 3                 | 0.098                         | 0.123                        |
| 6                 | 0.097                         | 4.442                        |
| 10                | 0.103                         | 157.857                      |

Avec l'évaluation au hasard, tous les polynômes se valent (erreur de l'ordre de 0,1, sans lien



avec le degré) : l'évaluation ne détecte rien. Avec l'évaluation chronologique, la vérité éclate : le polynôme de degré 6 **explose** hors de la zone d'apprentissage (une erreur de 4,4 en logarithme, soit des prévisions fausses d'un facteur de l'ordre de  $e^{4,4} \approx 80$ ), et celui de degré 10 est absurde (erreur de près de 158). Un modèle flexible ajuste parfaitement le passé et **extrapole n'importe comment**.

**La règle d'or.** En série temporelle, les données d'évaluation doivent toujours être **postérieures** aux données d'apprentissage. Même règle pour la validation croisée (4.3.6 : on la fait à origine glissante, jamais au hasard) et pour tout prétraitement (une moyenne ou une normalisation doivent être calculées sur l'apprentissage seul).

#### 4.3.4 Des références simples, puis des mesures d'erreur

Avant de juger un modèle sophistiqué, on le compare à une **référence simple**. Un modèle qui ne bat pas la référence ne mérite pas sa complexité. Pour une série saisonnière, deux références naturelles :

- le **naïf saisonnier** : prévoir pour un mois la valeur du **même mois l'an dernier** ;
- le **naïf saisonnier avec dérive** : idem, plus la croissance annuelle moyenne observée sur tout l'apprentissage (en logarithme : la moyenne de  $y_t - y_{t-12}$ ).

Pour mesurer l'erreur, quatre indicateurs usuels, avec  $e_t = y_t - \hat{y}_t$  l'erreur de prévision sur  $n$  points :

| Mesure      | Définition                                        | Remarque                                                                                                                    |
|-------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>MAE</b>  | $\frac{1}{n} \sum  e_t $                          | dans l'unité de la série ; robuste                                                                                          |
| <b>RMSE</b> | $\sqrt{\frac{1}{n} \sum e_t^2}$                   | pénalise fortement les grosses erreurs                                                                                      |
| <b>MAPE</b> | $\frac{100}{n} \sum  e_t / y_t $                  | en %, mais <b>indéfini</b> si $y_t = 0$ et asymétrique (une surestimation coûte plus qu'une sous-estimation de même taille) |
| <b>MASE</b> | MAE / (MAE dans l'échantillon du naïf saisonnier) | <b>sans unité</b> : $< 1$ signifie « mieux que le naïf saisonnier » ; utilisable pour comparer des séries                   |

**Exemple à la main.** Valeurs réelles 100, 120, 90, 110 ; prévisions 110, 115, 100, 105. Erreurs : -10, 5, -10, 5. MAE =  $(10 + 5 + 10 + 5)/4 = 7,5$ . RMSE =  $\sqrt{(100 + 25 + 100 + 25)/4} = \sqrt{62,5} \approx 7,91$ . MAPE =  $\frac{100}{4} \left( \frac{10}{100} + \frac{5}{120} + \frac{10}{90} + \frac{5}{110} \right) \approx 7,46\%$ . Si le naïf saisonnier avait une MAE de 5 sur l'apprentissage, la MASE vaudrait  $7,5/5 = 1,5$  : **pire** que la référence.

```
def mesures(reel, prevu, ref_apprentissage=None):
    e = np.asarray(reel, float) - np.asarray(prevu, float)
    sortie = {"MAE": np.mean(np.abs(e)), "RMSE": np.sqrt(np.mean(e ** 2)), "MAPE": 100 *
    ↪ np.mean(np.abs(e) / np.abs(reel))}
    if ref_apprentissage is not None:
        sortie["MASE"] = sortie["MAE"] / ref_apprentissage
    return sortie

print({k: round(float(x), 3) for k, x in mesures([100, 120, 90, 110], [110, 115, 100, 105],
    ↪ 5).items())})
```

```
{'MAE': 7.5, 'RMSE': 7.906, 'MAPE': 7.456, 'MASE': 1.5}
```

Appliquons-les à nos cinq prévisions des 24 mois de test, **en dinars** (on revient de l'échelle logarithmique par l'exponentielle) :

```
ca_train = np.exp(train)
echelle = np.mean(np.abs(ca_train.values[12:] - ca_train.values[:-12])) # MAE du naïf saisonnier
↳ dans l'échantillon
g = (train - train.shift(12)).mean() # croissance annuelle
↳ moyenne (en log)
print("croissance annuelle moyenne en apprentissage :", round(g, 3), "soit", round(100 * (np.exp(g) -
↳ 1), 1), "% par an")

naïf = pd.Series(train.iloc[-12:].tolist() * 2, index=test.index) # même mois de l'an
↳ dernier (puis de l'année précédente)
naïf_derive = pd.Series([train.iloc[-12 + (h % 12)] + g * (1 + h // 12) for h in range(24)],
↳ index=test.index)
prev_log = {"naïf saisonnier": naïf, "naïf saisonnier + dérive": naïf_derive}
for nom, p in previsions.items():
    prev_log[nom] = pd.Series(p.predicted_mean.values, index=test.index)

lignes = []
for nom, f in prev_log.items():
    m = mesures(np.exp(test), np.exp(f), echelle)
    m["biais (log)"] = float(np.mean(test - f))
    m["RMSE (log)"] = float(np.sqrt(np.mean((test - f) ** 2)))
    lignes.append(pd.Series(m, name=nom))
tab_test = pd.DataFrame(lignes).sort_values("RMSE")
print(tab_test.round(3).to_string())
```

```
croissance annuelle moyenne en apprentissage : 0.086 soit 9.0 % par an
```

|                               | MAE     | RMSE    | MAPE   | MASE  | biais (log) | RMSE (log) |
|-------------------------------|---------|---------|--------|-------|-------------|------------|
| B : (1,0,0)(0,1,1) + tendance | 136.571 | 169.051 | 6.132  | 0.641 | -0.004      | 0.074      |
| naïf saisonnier + dérive      | 156.641 | 209.579 | 6.700  | 0.735 | 0.013       | 0.085      |
| A : SARIMAX(1,1,1)(0,1,1)     | 173.243 | 225.707 | 8.229  | 0.813 | -0.069      | 0.100      |
| C : régression + AR(1)        | 175.374 | 227.290 | 8.366  | 0.823 | -0.070      | 0.101      |
| naïf saisonnier               | 309.604 | 386.582 | 13.487 | 1.452 | 0.141       | 0.169      |

Lisons ce tableau (trié par RMSE en dinars ; biais (log) est la moyenne de  $y - \hat{y}$  en logarithme, donc un biais **positif** signifie que le modèle **sous-estime**).

- Le **naïf saisonnier simple** est le plus mauvais, avec une MASE de 1,45 (pire que lui-même sur l'apprentissage) et un biais de +0,14 : il répète l'année passée sans tenir compte de la croissance, et donc sous-estime d'environ 14 %.
- Le **naïf saisonnier avec dérive**, trois lignes de code, est déjà très honorable : MASE de 0,74, MAPE de 6,7 %. Il **bat deux de nos trois modèles ARIMA** (A et C) sur cette fenêtre, en dinars comme en MAPE. Voilà pourquoi on ne néglige jamais les références simples.
- Le modèle **B** est le seul à faire nettement mieux que la référence (MASE de 0,64, MAPE de 6,1 %). Les modèles **A** et **C** ont un biais négatif d'environ -0,07 : ils **surestiment** de 7 % en moyenne. Nous verrons en 4.3.8 pourquoi.

**Prévoir en dinars ou en logarithme ?** Nous avons optimisé les modèles en log (erreurs relatives), mais nous les jugeons en dinars (ce qui intéresse Yasmine) : une erreur de 300 DT en décembre et de 300 DT en février n'ont pas le même poids en log, mais le même en dinars. Quand l'objectif métier est en unités de la série, évaluez dans ces unités.

### 4.3.5 Vingt-quatre mois suffisent-ils pour conclure ?

Le tableau précédent donne un classement. Mais il repose sur **un seul** découpage, 24 erreurs **corrélées entre elles** (une erreur en mars annonce souvent une erreur en avril), et sur un seul tirage du hasard. Deux garde-fous.

(1) **Le plancher du bruit.** Même le *meilleur modèle possible* ne peut pas prévoir mieux que le bruit qui reste. Le modèle C estime les erreurs d'un AR(1) autour de la tendance et de la saison ; leur écart-type est  $\sigma / \sqrt{1 - \phi^2}$  :

```
sigma2, phi_C = mod_C.params["sigma2"], mod_C.params["ar.L1"]
print("écart-type de l'écart à la tendance et à la saison (modèle C) :", round(np.sqrt(sigma2 / (1 -
    ↪ phi_C ** 2)), 4))
print("écart-type des chocs d'un mois à l'autre                                :", round(np.sqrt(sigma2), 4))
```

```
écart-type de l'écart à la tendance et à la saison (modèle C) : 0.0715
écart-type des chocs d'un mois à l'autre                        : 0.0603
```

Pour des prévisions à long terme (au-delà de quelques mois), aucun modèle ne peut descendre durablement sous une erreur de l'ordre de 0,07 en logarithme (environ 7 %) : c'est la limite physique. Un écart de 0,01 entre deux modèles n'est pas une différence que 24 points permettent d'établir.

(2) **Un bootstrap par blocs.** Pour quantifier l'incertitude sur l'écart de performance entre deux modèles, on rééchantillonne les **différences d'erreurs quadratiques** par **blocs** de 6 mois consécutifs (pour conserver la corrélation) : si l'intervalle à 95 % de la différence moyenne **contient 0**, l'avantage d'un modèle n'est pas établi.

```
def bootstrap_blocs(e1, e2, L=6, B=4000, graine=0):
    """Intervalle à 95 % de la différence moyenne des erreurs quadratiques (modèle 1 - modèle 2), par
    ↪ blocs de L mois."""
    rng = np.random.default_rng(graine)
    d = np.asarray(e1) ** 2 - np.asarray(e2) ** 2
    n = len(d)
    moyennes = []
    for _ in range(B):
        debuts = rng.integers(0, n - L + 1, int(np.ceil(n / L)))
        idx = np.concatenate([np.arange(s, s + L) for s in debuts])[:n]
        moyennes.append(d[idx].mean())
    return np.percentile(moyennes, [2.5, 97.5]), np.mean(np.array(moyennes) < 0)

erreurs = {k: (test - f).values for k, f in prev_log.items()}
paires = [("B : (1,0,0)(0,1,1) + tendance", "A : SARIMAX(1,1,1)(0,1,1)"),
          ("B : (1,0,0)(0,1,1) + tendance", "C : régression + AR(1)"),
          ("A : SARIMAX(1,1,1)(0,1,1)", "C : régression + AR(1)"),
          ("A : SARIMAX(1,1,1)(0,1,1)", "naïf saisonnier + dérive")]
for a, b in paires:
    (lo, hi), part = bootstrap_blocs(erreurs[a], erreurs[b])
    print(f"{a[:24]:25s} contre {b[:24]:25s} : IC95 = [{lo:+.4f} ; {hi:+.4f}] | part où le 1er est
    ↪ meilleur : {part:.2f}")
```

```
B : (1,0,0)(0,1,1) + ten contre A : SARIMAX(1,1,1)(0,1,1 : IC95 = [-0.0092 ; -0.0011] | part où le
    ↪ 1er est meilleur : 0.99
B : (1,0,0)(0,1,1) + ten contre C : régression + AR(1) : IC95 = [-0.0097 ; -0.0010] | part où le
    ↪ 1er est meilleur : 0.99
A : SARIMAX(1,1,1)(0,1,1 contre C : régression + AR(1) : IC95 = [-0.0007 ; +0.0004] | part où le
    ↪ 1er est meilleur : 0.76
A : SARIMAX(1,1,1)(0,1,1 contre naïf saisonnier + dérive : IC95 = [-0.0028 ; +0.0083] | part où le
    ↪ 1er est meilleur : 0.17
```

Lecture : un intervalle **entièrement négatif** signifie que le premier modèle est meilleur de façon

robuste *sur cette fenêtre* ; un intervalle contenant 0, qu'on ne peut pas conclure. Ici, **B bat A et C** (intervalles négatifs, mais proches de 0 : l'avantage est réel sur cette fenêtre et modeste), tandis que **A et C sont indiscernables**, et que **A ne se distingue pas de la référence simple** (intervalle qui contient 0). Ces résultats portent sur **cette** fenêtre de 24 mois, et nous allons voir en 4.3.8 qu'elle n'est pas anodine.

**Le test de Diebold-Mariano.** On rencontre souvent ce test pour comparer deux prévisions. Il suppose des erreurs d'un **même horizon** (une suite temporelle de différences de pertes) ; appliqué à des prévisions de 1 à 24 mois issues d'une **seule** origine, il mélange des horizons dont la variance n'est pas la même. Le bootstrap par blocs ci-dessus est plus simple et rend le même service ; le test de Diebold-Mariano trouve sa place quand on dispose d'erreurs à horizon fixe pour de nombreuses origines (4.3.6).

### 4.3.6 La validation à origine glissante

Un seul découpage ne donne qu'un seul exemple d'erreur. Pour en avoir plusieurs, on **recommence plusieurs fois** en avançant l'origine : à chaque étape on ajuste le modèle sur les données jusqu'à la date  $T$ , on prévoit les  $H = 12$  mois suivants, on mesure l'erreur, puis on avance  $T$  d'un mois (fenêtre **croissante**). On obtient, pour chaque horizon  $h$  de 1 à 12, une dizaine d'erreurs. C'est la **validation croisée à origine glissante** (*rolling-origin*, ou *time-series cross-validation*).

**Honnêteté.** Ces origines (décembre 2023 à décembre 2024) sont celles de notre jeu de test : nous réutilisons la **même période**, avec un autre usage. Les structures des modèles (A, B, C) ont été choisies en 4.2 sur l'apprentissage seul ; ce qui est ré-estimé à chaque origine, ce sont les paramètres. Il n'y a donc pas de fuite, mais il n'y a pas non plus de nouvelles données : le test de 24 mois est « utilisé » ici et ne servira plus à rien d'autre.

On ajoute une cinquième prévision, une **combinaison** : la moyenne des prévisions des trois modèles A, B, C (la combinaison de prévisions est connue pour être robuste : les erreurs de modèles différents ne se corrigent qu'en partie, mais elles se corrigent).

```
H = 12
origines = list(range(96, 109))                # 13 origines : fin décembre 2023, ..., fin
↳ décembre 2024

def prev_naif(i):
    return y.iloc[i - 12:i - 12 + H].values

def prev_derive(i):
    tr = y.iloc[:i]
    gi = (tr - tr.shift(12)).mean()
    return np.array([tr.iloc[-12 + (h % 12)] + gi * (1 + h // 12) for h in range(H)])

def prev_modele_A(i):
    m = SARIMAX(y.iloc[:i], exog=X.iloc[:i], order=(1, 1, 1), seasonal_order=(0, 1, 1,
↳ 12)).fit(dispatch=False)
    return m.forecast(H, exog=X.iloc[i:i + H]).values

def prev_modele_B(i):
    m = SARIMAX(y.iloc[:i], exog=X.iloc[:i], order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
↳ trend="ct").fit(dispatch=False, maxiter=300)
    return m.forecast(H, exog=X.iloc[i:i + H]).values

def prev_modele_C(i):
    m = ARIMA(y.iloc[:i], exog=XX.iloc[:i], order=(1, 0, 0), trend="n").fit()
    return m.forecast(H, exog=XX.iloc[i:i + H]).values
```

```

fonctions = {"naïf saisonnier": prev_naif, "naïf saisonnier + dérive": prev_derive,
             "A : SARIMAX(1,1,1)(0,1,1)": prev_modele_A, "B : (1,0,0)(0,1,1) + tendance":
             ↪ prev_modele_B,
             "C : régression + AR(1)": prev_modele_C}
P = {nom: np.array([f(i) for i in origines]) for nom, f in fonctions.items()} # forme (13
↪ origines, 12 horizons)
P["combinaison A+B+C"] = (P["A : SARIMAX(1,1,1)(0,1,1)"] + P["B : (1,0,0)(0,1,1) + tendance"] + P["C :
↪ régression + AR(1)"]) / 3
reel = np.array([y.iloc[i:i + H].values for i in origines])

lignes = []
for nom, f in P.items():
    e = reel - f
    lignes.append({"modèle": nom, "RMSE global": np.sqrt((e ** 2).mean()), "h = 1": np.sqrt((e[:, 0]
↪ ** 2).mean()),
                  "h = 6": np.sqrt((e[:, 5] ** 2).mean()), "h = 12": np.sqrt((e[:, 11] ** 2).mean()),
                  ↪ "biais": e.mean()})
tab_glissant = pd.DataFrame(lignes).set_index("modèle").sort_values("RMSE global")
print("RMSE en logarithme, 13 origines x 12 horizons :")
print(tab_glissant.round(4).to_string())

fig, ax = plt.subplots(figsize=(8, 4))
couleurs = {"naïf saisonnier": "#898781", "naïf saisonnier + dérive": VIOLET, "A :
↪ SARIMAX(1,1,1)(0,1,1)": ORANGE,
            "B : (1,0,0)(0,1,1) + tendance": BLEU, "C : régression + AR(1)": ROUGE, "combinaison
↪ A+B+C": AQUA}
for nom, f in P.items():
    ax.plot(range(1, H + 1), np.sqrt(((reel - f) ** 2).mean(axis=0)), "o-", ms=3.5, lw=1.6,
    ↪ color=couleurs[nom], label=nom)
ax.set_xlabel("horizon de prévision (mois)")
ax.set_ylabel("RMSE (en logarithme)")
ax.set_xticks(range(1, H + 1))
ax.legend(fontsize=8, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.17))
ax.grid(True, color="#e1e0d9", lw=0.6)
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-origine-glissante.png", bbox_inches="tight")

```

| RMSE en logarithme, 13 origines x 12 horizons : |             |        |        |        |         |
|-------------------------------------------------|-------------|--------|--------|--------|---------|
|                                                 | RMSE global | h = 1  | h = 6  | h = 12 | biais   |
| modèle                                          |             |        |        |        |         |
| B : (1,0,0)(0,1,1) + tendance                   | 0.0670      | 0.0712 | 0.0581 | 0.0889 | 0.0037  |
| combinaison A+B+C                               | 0.0687      | 0.0708 | 0.0521 | 0.0808 | -0.0295 |
| A : SARIMAX(1,1,1)(0,1,1)                       | 0.0763      | 0.0737 | 0.0572 | 0.0890 | -0.0416 |
| C : régression + AR(1)                          | 0.0807      | 0.0742 | 0.0637 | 0.0903 | -0.0506 |
| naïf saisonnier + dérive                        | 0.0839      | 0.0753 | 0.0801 | 0.1052 | -0.0175 |
| naïf saisonnier                                 | 0.1081      | 0.1213 | 0.1100 | 0.1264 | 0.0707  |

Trois enseignements, que les chiffres du tableau confirment :

1. **Le naïf saisonnier simple est nettement le moins bon** : il ne sait pas que la série croît. Le naïf avec dérive (RMSE de 0,084), trois lignes de code, fait déjà beaucoup mieux, et fait presque jeu égal avec le modèle C (0,081) : **toujours essayer les références simples**.
2. **Les modèles ARIMA/régression sont proches les uns des autres**, avec un avantage au modèle B (RMSE global de 0,067), qui combine différence saisonnière (il s'adapte au niveau de l'an dernier) et tendance déterministe. La **combinaison** A+B+C (0,069) est presque aussi bonne, **sans avoir à choisir**, et elle est la meilleure aux horizons 1, 6 et 12 du tableau.
3. **L'erreur ne croît pas régulièrement avec l'horizon** (le graphique est irrégulier, avec seulement 13 origines) : elle se situe entre 0,05 et 0,10 pour tous les modèles ARIMA, c'est-à-dire autour du plancher de 0,07 évoqué en 4.3.5. Les écarts entre courbes sont petits devant cette bande de bruit.

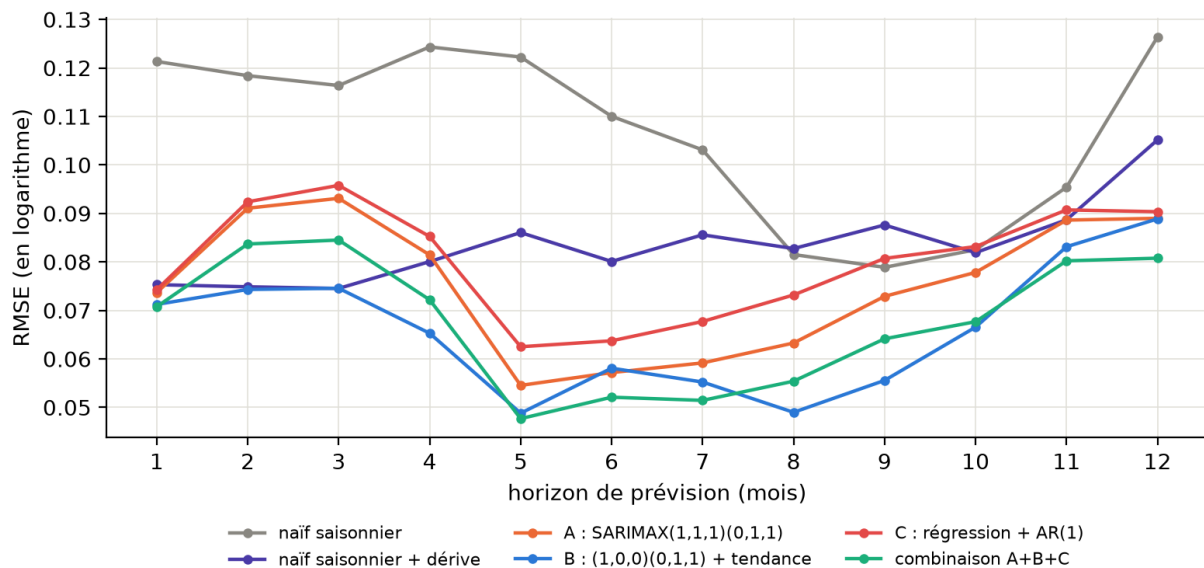


Figure 44: Erreur de prévision (RMSE en logarithme) selon l'horizon, pour six méthodes évaluées à 13 origines glissantes. Le naïf saisonnier est nettement derrière ; les modèles ARIMA et leur combinaison sont proches les uns des autres.

#### 4.3.7 Application : les ventes de 2026

Yasmine prépare son budget. Nous prévoyons maintenant 2026 avec le modèle retenu (B), ajusté sur les 120 mois, en supposant une promotion en décembre 2026 (prévue par Yasmine), pas de COVID. Outre la prévision mois par mois, on lui donne une **prévision de l'année entière** avec son incertitude, obtenue par **simulation** : on tire 2 000 trajectoires futures plausibles du modèle (en respectant la corrélation entre mois) et on additionne les 12 mois de chacune.

```
X_2026 = pd.DataFrame({"promo": [0.0] * 11 + [1.0], "covid": [0.0] * 12},
                      index=pd.date_range("2026-01-01", periods=12, freq="MS"))
final = SARIMAX(y, exog=X, order=(1, 0, 0), seasonal_order=(0, 1, 1, 12), trend="ct").fit(dispatch=False,
                                             maxiter=300)
pf = final.get_forecast(12, exog=X_2026)
ic = np.asarray(pf.conf_int(alpha=0.05))
resume = pd.DataFrame({"prévision (DT)": np.exp(pf.predicted_mean.values), "IC95 bas": np.exp(ic[:, 0]),
                      "IC95 haut": np.exp(ic[:, 1])},
                      index=X_2026.index.strftime("%Y-%m").round(0).astype(int))
print(resume.to_string())

a_fin = final.predicted_state[:, -1] # loi de l'état caché juste après la
↳ dernière observation :
P_fin = final.predicted_state_cov[:, :, -1] # moyenne et covariance
sigma = np.sqrt(final.params["sigma2"])

def une_trajectoire(graine):
    rng = np.random.default_rng(graine) # on tire nous-mêmes l'état initial et les
    ↳ chocs : résultat reproductible
    etat0 = rng.multivariate_normal(a_fin, P_fin, method="eigh")
    chocs = rng.normal(0, sigma, size=(12, 1))
    return final.simulate(12, measurement_shocks=np.zeros((12, 1)), state_shocks=chocs,
                        ↳ initial_state=etat0,
                          anchor="end", exog=X_2026).values

simul = np.array([une_trajectoire(k) for k in range(2000)]).T # forme (12 mois, 2000
↳ trajectoires)
totaux = np.exp(simul).sum(axis=0)
total_2025 = v["ca"]["2025-01":"2025-12"].sum()
```

```

p10, p50, p90 = np.percentile(totaux, [10, 50, 90])
p025, p975 = np.percentile(totaux, [2.5, 97.5])
print(f"\ntotal 2025 observé : {total_2025:,.0f} DT")
print(f"total 2026 prévu : médiane {p50:,.0f} DT | intervalle à 80 % [{p10:,.0f} ; {p90:,.0f}] | à  

↳ 95 % [{p025:,.0f} ; {p975:,.0f}]")
print(f"croissance prévue sur 2025 : {100 * (p50 / total_2025 - 1):+.1f} %")

fig, ax = plt.subplots(figsize=(10, 3.8))
hist = v["ca"]["2023-01":]
ax.plot(hist.index, hist, color=BLEU, lw=1.6, label="observé")
ax.plot(X_2026.index, np.exp(pf.predicted_mean), color=ORANGE, lw=1.8, label="prévision 2026")
ax.fill_between(X_2026.index, np.exp(ic[:, 0]), np.exp(ic[:, 1]), color=ORANGE, alpha=0.2,  

↳ label="intervalle à 95 %")
ax.set_ylabel("chiffre d'affaires mensuel (DT)")
ax.legend(loc="upper left")
ax.grid(True, color="#e1e0d9", lw=0.6)
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-prevision-2026.png", bbox_inches="tight")

```

|         | prévision (DT) | IC95 bas | IC95 haut |
|---------|----------------|----------|-----------|
| 2026-01 | 1414           | 1214     | 1648      |
| 2026-02 | 1694           | 1434     | 2001      |
| 2026-03 | 2130           | 1799     | 2522      |
| 2026-04 | 2207           | 1863     | 2614      |
| 2026-05 | 2495           | 2106     | 2956      |
| 2026-06 | 2881           | 2431     | 3413      |
| 2026-07 | 2864           | 2418     | 3393      |
| 2026-08 | 2637           | 2226     | 3124      |
| 2026-09 | 2067           | 1745     | 2449      |
| 2026-10 | 1873           | 1581     | 2219      |
| 2026-11 | 2602           | 2197     | 3083      |
| 2026-12 | 4195           | 3541     | 4970      |

total 2025 observé : 27,630 DT

total 2026 prévu : médiane 29,169 DT | intervalle à 80 % [27,823 ; 30,597] | à 95 % [27,073 ; 31,397]

croissance prévue sur 2025 : +5.6 %

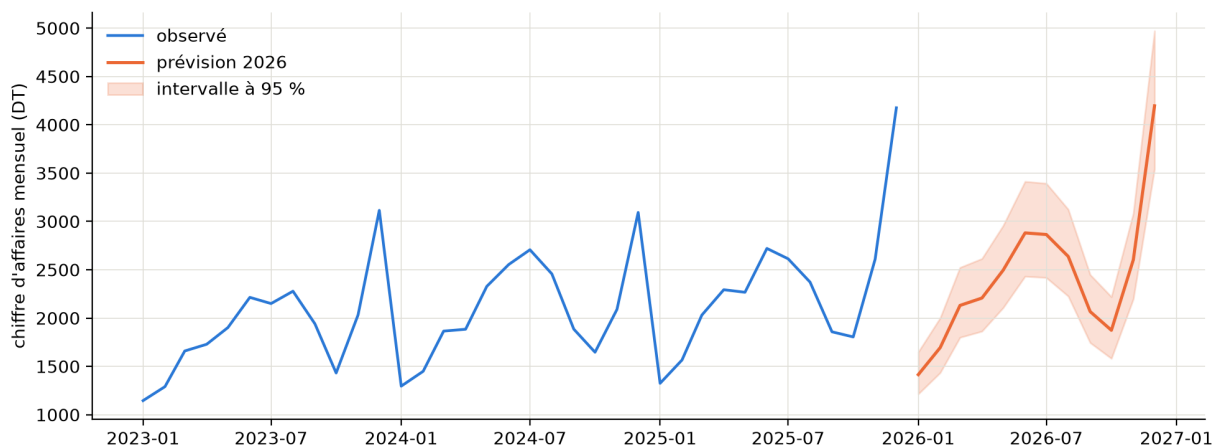


Figure 45: Chiffre d'affaires mensuel depuis 2023 et prévision pour 2026 (courbe orange) avec son intervalle de prévision à 95 %.

Yasmine peut retenir trois messages : (1) **le profil de l'année** : le creux de janvier, le plateau d'été, le pic de décembre ; (2) une **fourchette** de chiffre d'affaires annuel plutôt qu'un point (la fourchette à 80 % est un bon outil de budget) ; (3) une croissance attendue de l'ordre de +5 % sur 2025, avec une incertitude que la fourchette rend visible. Cette croissance est inférieure à la



tendance de long terme de la série (environ 9 % par an, voir 4.3.4) parce que le modèle B repart du **niveau récent**, resté sous la tendance : nous verrons en 4.3.8 que ce choix est un pari sur la persistance de ces écarts.

**Ce que cette prévision suppose.** Que la structure observée de 2016 à 2025 **se prolonge** (même croissance, même saisonnalité), que la promotion de décembre ait lieu, et qu'aucun choc du type COVID n'arrive (un tel choc est par nature imprévisible : les intervalles de prévision ne le contiennent pas). Une prévision n'est pas une promesse : c'est le résultat d'un modèle et d'hypothèses, qu'il faut toujours énoncer à côté du chiffre.

### 4.3.8 Révélation : comment les données ont été fabriquées

Les ventes de Dar Jasmin sont **simulées**. Il est temps de lever le voile. Voici la recette complète : une tendance exponentielle (+0,75 % par mois), une saisonnalité **déterministe** (un facteur multiplicatif par mois), un bruit **AR(1)** (coefficient 0,5, chocs de 0,07), un effet de promotion (+10 % en log), et un choc COVID (−0,55 en log, mars à juin 2020).

```
def simuler_ventes(graine):
    """La recette exacte du fichier ventes_mensuelles.csv (graine 2018)."""
    rng = np.random.default_rng(graine)
    mois = pd.date_range("2016-01-01", "2025-12-01", freq="MS")
    t = np.arange(120)
    saison = np.array([0.62, 0.72, 0.95, 1.00, 1.12, 1.18, 1.22, 1.15, 0.90, 0.78, 1.05, 1.55]) #
    ↪ facteur multiplicatif par mois
    bruit = np.zeros(120)
    for i in range(1, 120):
        bruit[i] = 0.5 * bruit[i - 1] + rng.normal(0, 0.07) # AR(1)
    ↪ : phi = 0,5 ; sigma = 0,07
    promo = (rng.random(120) < 0.15).astype(int)
    covid = ((mois >= "2020-03-01") & (mois <= "2020-06-01")).astype(int)
    log_ca = np.log(1000) + 0.0075 * t + np.log(saison[mois.month - 1]) + bruit + 0.10 * promo - 0.55
    ↪ * covid
    d = pd.DataFrame({"ca": np.round(np.exp(log_ca), 1), "promo": promo, "covid": covid}, index=mois)
    return d, pd.Series(bruit, index=mois)

recette, bruit = simuler_ventes(2018)
print("la recette reproduit exactement le fichier :", np.allclose(recette["ca"].values,
    ↪ v["ca"].values)
    and (recette["promo"].values == v["promo"].values).all() and (recette["covid"].values ==
    ↪ v["covid"].values).all())
```

la recette reproduit exactement le fichier : True

Comparons ce que les modèles ont **estimé** à ce qui a été **programmé** :

```
vrai_saison = np.log(np.array([0.62, 0.72, 0.95, 1.00, 1.12, 1.18, 1.22, 1.15, 0.90, 0.78, 1.05,
    ↪ 1.55]))
estim_saison = np.r_[0.0, mod_C.params[["m_{k:02d}" for k in range(2, 13)]]].values #
    ↪ relatif à janvier
tab_vrai = pd.DataFrame({"programmé": [0.0075, 0.10, -0.55, 0.5, 0.07],
    "modèle A": [np.nan, mod_A.params["promo"], mod_A.params["covid"], np.nan,
    ↪ np.nan],
    "modèle B": [np.nan, mod_B.params["promo"], mod_B.params["covid"], np.nan,
    ↪ np.nan],
    "modèle C": [mod_C.params["t"], mod_C.params["promo"], mod_C.params["covid"],
    ↪ mod_C.params["ar.L1"], np.sqrt(mod_C.params["sigma2"])]},
    index=["pente de la tendance (par mois)", "effet promo (log)", "effet COVID
    ↪ (log)", "AR(1) : phi", "chocs : sigma"])
print(tab_vrai.round(4).to_string())
print()
```

```

saison_cmp = pd.DataFrame({"programmé": vrai_saison - vrai_saison[0], "estimé (modèle C)":
↳ estim_saison},
                           index=["jan", "fév", "mar", "avr", "mai", "juin", "juil", "août", "sept",
↳ "oct", "nov", "déc"])
print("profil saisonnier, en log et relativement à janvier :")
print(saison_cmp.round(2).T.to_string())

```

|                                 | programmé | modèle A | modèle B | modèle C |
|---------------------------------|-----------|----------|----------|----------|
| pente de la tendance (par mois) | 0.0075    | NaN      | NaN      | 0.0077   |
| effet promo (log)               | 0.1000    | 0.1061   | 0.0995   | 0.1063   |
| effet COVID (log)               | -0.5500   | -0.5371  | -0.5055  | -0.5408  |
| AR(1) : phi                     | 0.5000    | NaN      | NaN      | 0.5388   |
| chocs : sigma                   | 0.0700    | NaN      | NaN      | 0.0603   |

profil saisonnier, en log et relativement à janvier :

|                   | jan | fév  | mar  | avr  | mai  | juin | juil | août | sept | oct  | nov  | déc  |
|-------------------|-----|------|------|------|------|------|------|------|------|------|------|------|
| programmé         | 0.0 | 0.15 | 0.43 | 0.48 | 0.59 | 0.64 | 0.68 | 0.62 | 0.37 | 0.23 | 0.53 | 0.92 |
| estimé (modèle C) | 0.0 | 0.18 | 0.46 | 0.51 | 0.59 | 0.66 | 0.70 | 0.61 | 0.37 | 0.20 | 0.48 | 0.94 |

Les modèles ont **retrouvé** la recette : la pente de la tendance (0,0077 estimé pour 0,0075), l'effet de la promotion (0,106 pour 0,10), l'effet du COVID (de  $-0,51$  à  $-0,54$  pour  $-0,55$ ), la saison mois par mois (écarts de quelques centièmes en log), le coefficient AR (0,54 pour 0,5) ; seule la taille des chocs est un peu sous-estimée (0,060 pour 0,07). Et nos soupçons étaient fondés : la tendance et la saison étaient bien **déterministes**, ce que laissent entendre les tests ambigus de 4.1.6 et les MA proches de  $-1$  de 4.2.4. Le modèle C a la **structure exacte** de la vérité.

**Pourtant, C n'a pas gagné le concours de 4.3.4.** Pourquoi un modèle correctement spécifié perd-il ? La réponse tient au bruit. Calculons ce que valait le **bruit programmé** sur les deux périodes :

```

print("bruit AR(1) programmé, moyenne : apprentissage =", round(bruit[:"2023-12"].mean(), 4), "| test
↳ =", round(bruit["2024-01"].mean(), 4))
print("écart-type du bruit sur le test :", round(bruit["2024-01"].std(), 4), "| RMSE du « modèle
↳ parfait » (qui connaîtrait tous les paramètres) :", round(np.sqrt((bruit["2024-01"] **
↳ 2).mean()), 4))
print("biais moyen des prévisions du modèle C sur le test :", round(float(np.mean(test - prev_log["C :
↳ régression + AR(1)"])), 4))

```

```

bruit AR(1) programmé, moyenne : apprentissage = -0.014 | test = -0.0695
écart-type du bruit sur le test : 0.0657 | RMSE du « modèle parfait » (qui connaîtrait tous les
↳ paramètres) : 0.0947
biais moyen des prévisions du modèle C sur le test : -0.0696

```

Le bruit programmé sur les 24 mois de test est **négativement décalé** en moyenne :  $-0,0695$  en log, soit environ 7 % de ventes en dessous de la tendance et de la saison (contre  $-0,014$  pendant l'apprentissage). Le modèle C, qui ne croit qu'à la tendance et à la saison, prévoit « la tendance » : son erreur moyenne ( $y - \hat{y}$  vaut  $-0,0696$  en moyenne, contre  $-0,0695$  pour le bruit) est **exactement ce bruit**. Le « biais » de C n'est pas un défaut du modèle : c'est la trajectoire du hasard. Même un « modèle parfait » qui connaîtrait tous les paramètres aurait une RMSE de 0,095 sur ces 24 mois, **plus** que le modèle B (0,074, en log). C'est la malchance de cet échantillon de test : la série est restée sous sa tendance pendant deux ans, et le modèle B, qui **s'ajuste au niveau récent** grâce à sa différence saisonnière, a bénéficié de cette persistance. (Dans cette situation, il a *parié* sur la persistance des écarts, et il a gagné. Ce pari est bon quand les écarts sont persistants et mauvais quand ils sont transitoires.)

Un test sur **une** trajectoire ne distingue pas « bon modèle » et « modèle chanceux ». La seule façon de le savoir est de **rejouer le hasard** : tirons plusieurs historiques complets avec la même recette (graines différentes), ajustons B et C sur les 96 premiers mois de chacun, prévoyons les 24 suivants, et comparons les erreurs.

```

mois_calendaire = lambda idx: pd.get_dummies(pd.Series(idx.month,
↳ index=idx).astype(str).str.zfill(2), prefix="m", drop_first=True, dtype=float)

def un_essai(graine):
    d, _ = simuler_ventes(graine)
    d.index.freq = "MS"
    yy = np.log(d["ca"])
    XXe = pd.concat([pd.Series(1.0, index=d.index, name="const"), pd.Series(np.arange(120.0),
↳ index=d.index, name="t"),
                    mois_calendaire(d.index), d[["promo", "covid"]].astype(float)], axis=1)
    Xe = d[["promo", "covid"]].astype(float)
    tr, te = yy[:96], yy[96:]
    gg = (tr - tr.shift(12)).mean()
    prevs = {"naïf saisonnier + dérive": np.array([tr.iloc[-12 + (h % 12)] + gg * (1 + h // 12) for h
↳ in range(24)]),
            "B": SARIMAX(tr, exog=Xe[:96], order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
↳ trend="ct").fit(dis=False, maxiter=300).forecast(24, exog=Xe[96:]).values,
            "C": ARIMA(tr, exog=XXe[:96], order=(1, 0, 0), trend="n").fit().forecast(24,
↳ exog=XXe[96:]).values}
    return {k: np.sqrt(np.mean((te.values - f) ** 2)) for k, f in prevs.items()}

essais = pd.DataFrame([un_essai(5000 + k) for k in range(30)])
print("RMSE (log) sur 24 mois de test, moyenne de 30 historiques simulés :")
print(essais.mean().round(4).to_string())
print("\nC bat B dans", int((essais["C"] < essais["B"]).sum()), "historiques sur 30 ;",
      "C bat le naïf avec dérive dans", int((essais["C"] < essais["naïf saisonnier + dérive"]).sum()),
↳ "sur 30")

```

RMSE (log) sur 24 mois de test, moyenne de 30 historiques simulés :

|                          |        |
|--------------------------|--------|
| naïf saisonnier + dérive | 0.1289 |
| B                        | 0.1108 |
| C                        | 0.0878 |

C bat B dans 25 historiques sur 30 ; C bat le naïf avec dérive dans 30 sur 30

Sur 30 historiques, c'est le **modèle de structure exacte (C)** qui a en moyenne la meilleure performance, mais il **ne gagne pas à chaque fois** : le résultat d'un seul découpage dépend beaucoup de la trajectoire du bruit. C'est exactement ce qu'on attend de la statistique : une conclusion se juge sur la **répétition**, pas sur un tirage.

#### À retenir.

- Une prévision est un **couple** (valeur, incertitude). Pour un AR(1) :  $\hat{y}_{T+h} = \mu + \phi^h(Y_T - \mu)$  et la variance de l'erreur croît de  $\sigma^2$  vers  $\sigma^2/(1 - \phi^2)$  ; avec une différence première, elle croît **sans limite**.
- En logarithme,  $e^{\hat{y}}$  est la **médiane** ; les intervalles se transforment par l'exponentielle.
- **Découpage temporel obligatoire** : jamais de mélange au hasard. Un modèle flexible peut avoir une excellente évaluation « au hasard » et extrapoler n'importe comment.
- Toujours comparer à des **références simples** (naïf saisonnier, avec dérive). Mesures : MAE, RMSE, MAPE (attention aux zéros), **MASE** (sans unité,  $< 1$  = bat le naïf).
- Un seul découpage de 24 mois est **bruité** : l'erreur d'un modèle parfait est plafonnée par le bruit de la série ; on utilise un bootstrap par blocs, et surtout la **validation à origine glissante**. Les **combinaisons** de prévisions sont robustes.
- Les modèles ne s'évaluent pas par l'AIC entre familles différentes, mais par la **prévision hors échantillon**, répétée.
- Une prévision se publie avec ses **hypothèses** (promotions connues, pas de choc type COVID).

## 4.4 ⊕ Pour aller plus loin : séries multivariées, cointégration et GARCH

**Section optionnelle.** Les sections 4.1 à 4.3 traitent **une** série à la fois et se suffisent à elles-mêmes. Ici, nous élargissons le cadre dans trois directions : **plusieurs séries qui s'influencent** (VAR), **des séries qui dérivent mais restent liées** (cointégration), et **une variabilité qui change dans le temps** (GARCH). Si vous cherchez l'essentiel, sautez à la section 4.5 ou aux exercices.

### 4.4.1 Plusieurs séries à la fois : le modèle VAR

Le chiffre d'affaires et le **nombre de commandes** sont deux séries liées. Un modèle **VAR** (*vector autoregression*) décrit chacune comme une combinaison du passé de *toutes* les séries. Pour deux séries  $y_{1,t}$  et  $y_{2,t}$ , regroupées dans le vecteur  $\mathbf{y}_t$ , le VAR(1) s'écrit

$$\mathbf{y}_t = \mathbf{c} + A \mathbf{y}_{t-1} + \varepsilon_t, \quad A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}.$$

C'est un AR(1) où le coefficient  $\varphi$  devient une **matrice**. Chaque ligne est une régression linéaire ordinaire (chapitre 1) de  $y_{i,t}$  sur les deux retards : le coefficient  $a_{12}$  dit combien le passé de la série 2 aide à prévoir la série 1, **au-delà** du passé de la série 1 elle-même.

**Stationnarité d'un VAR(1).** Comme pour l'AR(1) ( $|\varphi| < 1$ ), la condition est que **toutes les valeurs propres de  $A$  soient de module strictement inférieur à 1** (volume I, section 1.1.3 : une valeur propre mesure de combien la matrice étire une direction ;  $A^k \rightarrow 0$  si et seulement si ce facteur est inférieur à 1). Pour un VAR( $p$ ), on applique la même condition à la « matrice compagne » du système.

**Exemple à la main.** Soit  $A = \begin{pmatrix} 0,5 & 0,2 \\ 0,1 & 0,4 \end{pmatrix}$ . Sa trace vaut 0,9 et son déterminant  $0,5 \times 0,4 - 0,2 \times 0,1 = 0,18$ . Les valeurs propres sont solutions de  $\lambda^2 - 0,9\lambda + 0,18 = 0$ , soit  $\lambda = \frac{0,9 \pm \sqrt{0,81 - 0,72}}{2} = \frac{0,9 \pm 0,3}{2}$ , donc 0,6 et 0,3 : le système est stationnaire. Si un choc de 1 frappe la série 1 à la date 0, la réponse au pas  $k$  est la première colonne de  $A^k$  : (1;0), puis (0,5; 0,1), puis  $A^2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} = (0,27; 0,09)$ ... C'est la **fonction de réponse impulsionnelle** : un choc sur une série se propage dans l'autre.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import coint, adfuller
from statsmodels.stats.diagnostic import het_arch, acorr_ljungbox
from arch import arch_model
```

```
BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
A = np.array([[0.5, 0.2], [0.1, 0.4]])
print("valeurs propres de A :", np.linalg.eigvals(A).round(3), "-> stationnaire :",
      bool(np.all(np.abs(np.linalg.eigvals(A)) < 1)))
```

```
for k in (1, 2, 3):
    print(f"réponse à un choc unitaire sur la série 1, pas {k} :", (np.linalg.matrix_power(A, k) @
        ↪ np.array([1, 0])).round(3))
```

```
valeurs propres de A : [0.6+0.j 0.3+0.j] -> stationnaire : True
réponse à un choc unitaire sur la série 1, pas 1 : [0.5 0.1]
réponse à un choc unitaire sur la série 1, pas 2 : [0.27 0.09]
réponse à un choc unitaire sur la série 1, pas 3 : [0.153 0.063]
```

### Application : ventes et commandes

Le VAR suppose des séries **stationnaires**. Pour les rendre telles, on utilise l'idée de 4.2.7 : on retire de chaque série (en logarithme) sa **tendance**, sa **saison**, la **promotion** et le **COVID** par régression, et on garde l'**écart** — ce qui reste quand on a enlevé tout ce qu'on sait expliquer. Ces écarts sont stationnaires (nous le vérifions) et ce sont eux qui portent la dynamique conjointe. Nous utilisons les 96 mois d'apprentissage.

```
tr = XX["2023-12"] # tendance, mois, promo, covid (objet défini
    ↪ en 4.3)
ecarts = pd.DataFrame({"e_ca": sm.OLS(np.log(v["ca"][:"2023-12"]), tr).fit().resid,
    "e_nb": sm.OLS(np.log(v["nb_commandes"][:"2023-12"]), tr).fit().resid})
print("écarts-types :", ecarts.std().round(3).to_dict(), "| corrélation instantanée :",
    ↪ round(ecarts.corr().iloc[0, 1], 3))
for c in ecarts:
    print(f"test ADF de {c} : p-valeur = {adfuller(ecarts[c], regression='n', autolag='AIC')[1]}.1e")

modele = VAR(ecarts)
print(modele.select_order(4).summary())
var1 = modele.fit(maxlags=1)
print(pd.concat({"coefficient": var1.params, "p-valeur": var1.pvalues}, axis=1).round(3).to_string())
print("modules des valeurs propres de la matrice A estimée :",
    ↪ np.abs(np.linalg.eigvals(var1.coefs[0])).round(3))
```

```
écarts-types : {'e_ca': 0.072, 'e_nb': 0.226} | corrélation instantanée : 0.235
test ADF de e_ca : p-valeur = 2.6e-07
test ADF de e_nb : p-valeur = 7.5e-08
VAR Order Selection (* highlights the minimums)
```

|   | AIC     | BIC     | FPE        | HQIC    |
|---|---------|---------|------------|---------|
| 0 | -8.241  | -8.186  | 0.0002636  | -8.219  |
| 1 | -8.515* | -8.351* | 0.0002004* | -8.449* |
| 2 | -8.479  | -8.205  | 0.0002077  | -8.369  |
| 3 | -8.493  | -8.109  | 0.0002051  | -8.338  |
| 4 | -8.466  | -7.973  | 0.0002107  | -8.267  |

```
=====
coefficient      p-valeur
   e_ca  e_nb   e_ca  e_nb
const    -0.001 -0.005   0.930  0.825
L1.e_ca    0.520  0.675   0.000  0.036
L1.e_nb    0.026 -0.003   0.368  0.976
modules des valeurs propres de la matrice A estimée : [0.552 0.035]
```

Les deux écarts sont stationnaires. Les critères (AIC, BIC) choisissent **un seul retard**. Lecture des coefficients : l'écart de chiffre d'affaires d'un mois influence celui du mois suivant (coefficient d'environ 0,52, comme le AR(1) du modèle C), et il **prévoit aussi** l'écart du nombre de commandes ( $\approx 0,68$ , p-valeur  $\approx 0,04$ ). L'effet inverse (commandes  $\rightarrow$  chiffre d'affaires) est proche de zéro.

**Causalité au sens de Granger.** On dit que  $x$  *cause*  $y$  au sens de Granger si le passé de  $x$  améliore la prévision de  $y$  au-delà de son propre passé ; on le teste par un test de Fisher sur les coefficients du retard de  $x$  dans l'équation de  $y$  (hypothèse nulle : ils sont tous nuls).

```
g1 = varl.test_causality("e_nb", ["e_ca"], kind="f")
g2 = varl.test_causality("e_ca", ["e_nb"], kind="f")
print(f"le chiffre d'affaires précède le nombre de commandes : F = {g1.test_statistic:.2f}, p = {g1.pvalue:.3f}")
print(f"le nombre de commandes précède le chiffre d'affaires : F = {g2.test_statistic:.2f}, p = {g2.pvalue:.3f}")

irf = varl.irf(8)
fig, axes = plt.subplots(1, 2, figsize=(10, 3.4), sharey=False)
for ax, (j, nom) in zip(axes, [(0, "choc sur l'écart de chiffre d'affaires"), (1, "choc sur l'écart de nombre de commandes")]):
    ax.plot(range(9), irf.irfs[:, 0, j], "o-", color=BLEU, lw=1.6, ms=4, label="réponse du chiffre d'affaires")
    ax.plot(range(9), irf.irfs[:, 1, j], "o-", color=ORANGE, lw=1.6, ms=4, label="réponse du nombre de commandes")
    ax.axhline(0, color="#898781", lw=0.8)
    ax.set_title(nom, fontsize=10)
    ax.set_xlabel("mois après le choc")
    ax.grid(True, color="#e1e0d9", lw=0.6)
    ax.spines[["top", "right"]].set_visible(False)
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.savefig("figures/ch04-var-irf.png", bbox_inches="tight")
```

le chiffre d'affaires précède le nombre de commandes : F = 4.41, p = 0.037

le nombre de commandes précède le chiffre d'affaires : F = 0.81, p = 0.369

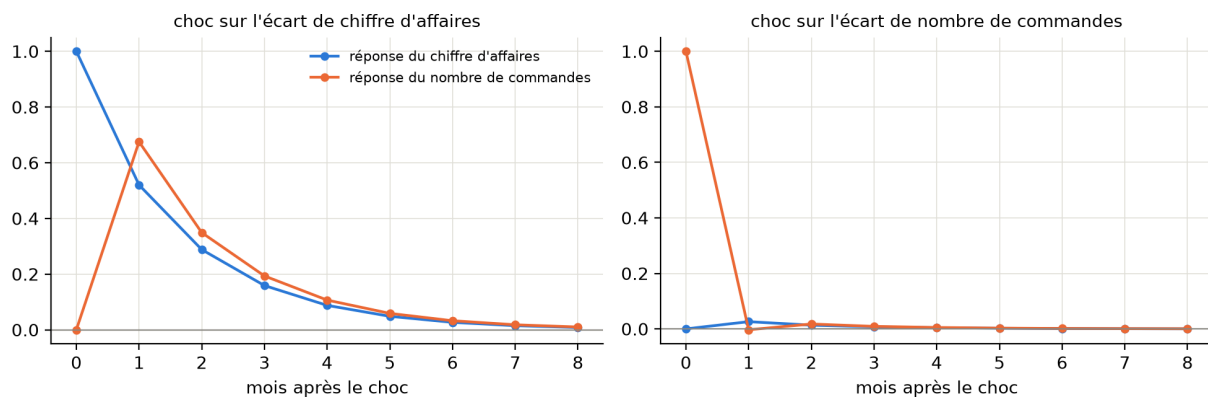


Figure 46: Fonctions de réponse impulsionnelle du VAR(1) : un choc de 1 sur l'écart de chiffre d'affaires (à gauche) se transmet à l'écart de commandes avec un effet plus important, puis s'atténue ; un choc sur les commandes (à droite) a très peu d'effet sur le chiffre d'affaires.

« Cause au sens de Granger » ne veut pas dire « cause ». Le test dit seulement que le passé du chiffre d'affaires **aide à prévoir** le nombre de commandes. Ici, la raison est connue, puisque nous avons fabriqué les données : le nombre de commandes est tiré d'une loi de Poisson dont la moyenne est le chiffre d'affaires du mois divisé par 58 (le panier moyen). Le nombre de commandes est donc une **mesure bruitée** du chiffre d'affaires *du même mois* ; comme le chiffre d'affaires est lui-même autocorrélé, son passé aide à prévoir cette mesure, et un nombre de commandes passé n'apporte rien de plus sur le chiffre d'affaires futur que le chiffre d'affaires passé lui-même. Aucune « influence » ne passe d'un mois à l'autre : c'est de l'**information**, pas



de la causalité. Pour parler de causalité, il faut un schéma d'étude, pas un test de précédence (chapitre 7).

#### 4.4.2 Cointégration : des séries qui dérivent ensemble

Passons à un autre piège, plus dangereux. Deux séries qui montent toutes les deux **semblent** liées, même si elles n'ont rien à voir. Prenons deux marches aléatoires **indépendantes** (4.1.3), régressons l'une sur l'autre, et regardons ce que dit la régression du chapitre 1. Répétons 1 000 fois :

```
rng = np.random.default_rng(31)
n_sim = 1000
t_stats, r2, dw = [], [], []
for _ in range(n_sim):
    a = np.cumsum(rng.normal(size=100))
    b = np.cumsum(rng.normal(size=100))          # indépendante de a par construction
    f = sm.OLS(a, sm.add_constant(b)).fit()
    t_stats.append(f.tvalues[1])
    r2.append(f.rsquared)
    dw.append(sm.stats.durbin_watson(f.resid))
t_stats = np.array(t_stats)
print("régressions d'une marche aléatoire sur une AUTRE, indépendante (100 points) :")
print("part des pentes « significatives » à 5 % :", round(np.mean(np.abs(t_stats) > 1.96), 3), " (on attendrait 0,05)")
print("R2 moyen :", round(np.mean(r2), 3), "| statistique de Durbin-Watson moyenne :", round(np.mean(dw), 3), " (2 = pas d'autocorrélation)")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.4))
a = np.cumsum(np.random.default_rng(5).normal(size=100))
b = np.cumsum(np.random.default_rng(6).normal(size=100))
ax1.plot(a, color=BLEU, lw=1.5, label="série 1")
ax1.plot(b, color=ORANGE, lw=1.5, label="série 2")
ax1.set_title("deux marches aléatoires indépendantes", fontsize=10)
ax1.legend(fontsize=8)
ax2.hist(t_stats, bins=50, color=BLEU, alpha=0.75)
ax2.axvline(-1.96, color=ROUGE, ls="--", lw=1.2)
ax2.axvline(1.96, color=ROUGE, ls="--", lw=1.2)
ax2.set_title("statistique t de la pente, 1 000 régressions", fontsize=10)
ax2.set_xlabel("valeur de t (les tirets marquent ±1,96)")
for ax in (ax1, ax2):
    ax.grid(True, color="#e1e0d9", lw=0.6)
    ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-regression-fallacieuse.png", bbox_inches="tight")
```

```
régressions d'une marche aléatoire sur une AUTRE, indépendante (100 points) :
part des pentes « significatives » à 5 % : 0.769 (on attendrait 0,05)
R2 moyen : 0.244 | statistique de Durbin-Watson moyenne : 0.172 (2 = pas d'autocorrélation)
```

Résultat : près de **quatre régressions sur cinq** déclarent « significatif » un lien qui n'existe pas (au lieu de 5 %), avec un  $R^2$  moyen d'environ 0,25 et une statistique de Durbin-Watson très basse (les résidus sont très autocorrélés). C'est la **régression fallacieuse** (Granger et Newbold, 1974) : quand les séries ne sont pas stationnaires, les p-valeurs de la régression sont **fausses** (les hypothèses du chapitre 1 ne tiennent plus). Règle de prudence : si le  $R^2$  est supérieur à la statistique de Durbin-Watson, méfiez-vous.

Comment distinguer une vraie relation d'une illusion ? Par la **cointégration**. Deux séries non stationnaires (intégrées d'ordre 1) sont **cointégrées** s'il existe une combinaison linéaire  $y_t - \beta x_t$  qui, elle, est **stationnaire** : les deux séries dérivent, mais **elles dérivent ensemble**, comme deux promeneurs liés par une corde élastique. Exemple plausible pour Dar Jasmin : un indice du coût



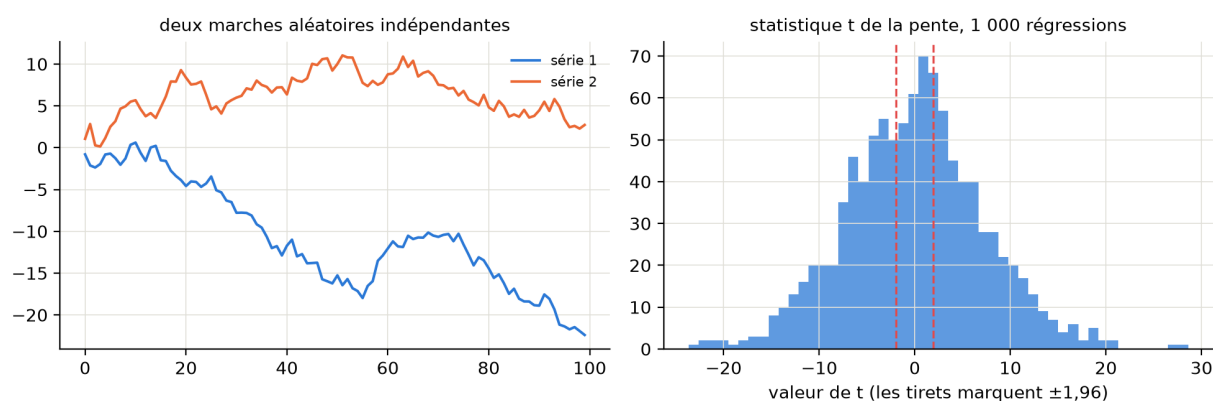


Figure 47: À gauche, deux marches aléatoires indépendantes qui semblent évoluer ensemble. À droite, les statistiques  $t$  de la pente de 1 000 régressions de ce type : elles sont bien plus dispersées que la loi de Student, et une majorité franchit les seuils de  $\pm 1,96$ .

des matières premières ( $x_t$ , une marche aléatoire) et le prix moyen de vente ( $y_t$ ), qui suit le coût avec un écart temporaire. Nous les **simulons** :  $x_t$  est une marche aléatoire,  $y_t = 2 + 1,5x_t + u_t$  avec  $u_t$  un AR(1) de coefficient 0,6.

La **méthode d'Engle et Granger** a deux temps : (1) on régresse  $y$  sur  $x$  ; (2) on teste la **racine unitaire des résidus** (ADF de 4.1.6, avec des seuils adaptés car les résidus sont estimés). S'ils sont stationnaires, il y a cointégration.

```
rng = np.random.default_rng(31)
n = 150
x = np.cumsum(rng.normal(size=n))
u = np.zeros(n)
for t in range(1, n):
    u[t] = 0.6 * u[t - 1] + rng.normal(0, 0.5)
y_coint = 2 + 1.5 * x + u
y_indep = np.cumsum(rng.normal(size=n)) # une série qui n'a rien à voir avec x

for nom, serie in [("y cointégrée avec x", y_coint), ("y indépendante de x", y_indep)]:
    stat, p, _ = coint(serie, x, trend="c")
    print(f"{nom:22s} : statistique d'Engle-Granger = {stat:6.2f} | p-valeur = {p:.3f}")

reg = sm.OLS(y_coint, sm.add_constant(x)).fit()
print("régression de y sur x : constante =", round(reg.params[0], 3), "| pente =",
      round(reg.params[1], 3), " (programmé : 2 et 1,5)")
```

```
y cointégrée avec x      : statistique d'Engle-Granger = -4.54 | p-valeur = 0.001
y indépendante de x     : statistique d'Engle-Granger = -2.55 | p-valeur = 0.257
régression de y sur x   : constante = 2.118 | pente = 1.494 (programmé : 2 et 1,5)
```

Pour la série cointégrée, le test rejette « pas de cointégration » (p-valeur proche de 0,001) et la pente estimée est très proche de 1,5 (les estimateurs de cointégration sont *super-convergers* : ils convergent à la vitesse  $n$  au lieu de  $\sqrt{n}$ ). Pour la série indépendante, le test ne rejette pas.

**Le modèle à correction d'erreur.** Si  $y_t - \beta x_t - c = u_t$  avec  $u_t = \varphi u_{t-1} + e_t$  stationnaire ( $|\varphi| < 1$ ), alors en différenciant  $y_t = \beta x_t + c + u_t$  :  $\Delta y_t = \beta \Delta x_t + \Delta u_t = \beta \Delta x_t + (\varphi - 1) u_{t-1} + e_t$ . En remplaçant  $u_{t-1}$  par l'écart à l'équilibre  $y_{t-1} - \beta x_{t-1} - c$ , on obtient

$$\Delta y_t = \gamma \Delta x_t + \alpha (y_{t-1} - \beta x_{t-1} - c) + e_t, \quad \gamma = \beta, \quad \alpha = \varphi - 1 < 0.$$

Le coefficient  $\alpha$  est la **vitesse de rappel** : une fraction  $|\alpha| = 1 - \varphi$  de l'écart à l'équilibre se

réorbe à chaque période. Ici, on s'attend à  $\alpha = -0,4$  et  $\gamma = 1,5$ .

```
ecart = (y_coint - reg.params[1] * x - reg.params[0])[:-1]          # écart à l'équilibre à la date
↳ t-1
dy, dx = np.diff(y_coint), np.diff(x)
ecm = sm.OLS(dy, sm.add_constant(np.column_stack([ecart, dx]))).fit()
print(pd.DataFrame({"estimation": ecm.params[1:], "erreur type": ecm.bse[1:], "théorie": [0.6 - 1,
↳ 1.5]}),
      index=["alpha (rappel)", "gamma (effet immédiat)"]).round(3).to_string())
```

|                        | estimation | erreur type | théorie |
|------------------------|------------|-------------|---------|
| alpha (rappel)         | -0.417     | 0.068       | -0.4    |
| gamma (effet immédiat) | 1.524      | 0.040       | 1.5     |

Les estimations sont proches de la théorie ( $\hat{\alpha} \approx -0,42$  pour  $-0,4$  ;  $\hat{\gamma} \approx 1,52$  pour  $1,5$ ) : environ 40 % de l'écart entre le prix de vente et son équilibre se résorbent chaque période, et une hausse immédiate du coût se répercute presque un pour un (à  $1,5$ ) sur le prix.

### 4.4.3 Quand la variabilité change : le modèle GARCH

Les modèles précédents supposent que les chocs  $\varepsilon_t$  ont une **variance constante**. Certaines séries violent cette hypothèse de façon flagrante : en finance, les rendements calmes alternent avec des périodes agitées (**clusters de volatilité**). C'est le cas, par exemple, du taux de change qui préoccupe Yasmine quand elle importe de la laine d'Italie : une variation quotidienne du taux dinar/euro est difficile à prévoir **en moyenne**, mais ses variations *absolues* se regroupent.

**Les données de cette sous-section sont simulées** : un jeu de 1 500 « rendements quotidiens » (en %) tiré d'un modèle GARCH(1,1) à paramètres connus (graine 41). Nous n'avons pas de série réelle de taux de change hors ligne ; ce n'est donc **pas** un historique du dinar.

Le modèle **GARCH(1,1)** (Bollerslev, 1986) écrit le rendement  $r_t = \mu + \varepsilon_t$ , avec  $\varepsilon_t = \sigma_t z_t$  ( $z_t$  de loi  $\mathcal{N}(0,1)$ ) et une variance qui **évolue** :

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2.$$

Un gros choc hier ( $\varepsilon_{t-1}^2$  grand) augmente la variance d'aujourd'hui (terme en  $\alpha$ ) ; et la variance est persistante (terme en  $\beta$ ).

**Variance inconditionnelle.** Si  $\alpha + \beta < 1$ , la variance moyenne à long terme existe : en prenant l'espérance de l'équation (avec  $\mathbb{E}[\varepsilon_{t-1}^2] = \mathbb{E}[\sigma_{t-1}^2] = \sigma^2$ ), on obtient  $\sigma^2 = \omega + (\alpha + \beta)\sigma^2$ , d'où  $\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$ . La quantité  $\alpha + \beta$  mesure la **persistance** de la volatilité.

**Exemple à la main.** Avec  $\omega = 0,05$ ,  $\alpha = 0,10$ ,  $\beta = 0,85$  : la variance de long terme est  $0,05 / (1 - 0,95) = 1$ . Si hier le rendement s'est écarté de sa moyenne de  $\varepsilon_{t-1} = 3$  (un choc de 3 écarts-types) alors que la variance valait 1, la variance d'aujourd'hui sera  $0,05 + 0,10 \times 9 + 0,85 \times 1 = 1,8$  : presque le double, d'un seul coup.

```
omega, alpha, beta, mu = 0.05, 0.10, 0.85, 0.02
print("variance de long terme :", round(omega / (1 - alpha - beta), 3), "| variance après un choc de 3
↳ :", round(omega + alpha * 9 + beta * 1, 3))

rng = np.random.default_rng(41)
n = 1500
```

```

sig2 = np.zeros(n)
r = np.zeros(n)
sig2[0] = omega / (1 - alpha - beta)
for t in range(n):
    if t > 0:
        sig2[t] = omega + alpha * (r[t - 1] - mu) ** 2 + beta * sig2[t - 1]
        r[t] = mu + np.sqrt(sig2[t]) * rng.normal()

print("kurtosis (excès) des rendements :", round(pd.Series(r).kurt(), 2), "(0 pour une loi normale)")
print("Ljung-Box (10 retards) sur les rendements : p =", round(float(acorr_ljungbox(r,
    ↪ lags=[10])["lb_pvalue"].iloc[0]), 4))
print("Ljung-Box (10 retards) sur les carrés : p =", f"{float(acorr_ljungbox((r - r.mean()) ** 2,
    ↪ lags=[10])['lb_pvalue'].iloc[0]):.1e}")
print("test ARCH-LM (5 retards) : p =", f"{het_arch(r - r.mean(), nlags=5)[1]:.1e}")

```

```

variance de long terme : 1.0 | variance après un choc de 3 : 1.8
kurtosis (excès) des rendements : 0.25 (0 pour une loi normale)
Ljung-Box (10 retards) sur les rendements : p = 0.0019
Ljung-Box (10 retards) sur les carrés : p = 4.7e-22
test ARCH-LM (5 retards) : p = 1.1e-08

```

Les rendements eux-mêmes n'ont, par construction, **aucune** mémoire dans leur moyenne ; leurs **carrés** (la variance) en ont beaucoup : le test de Ljung-Box appliqué aux carrés, et le test ARCH-LM (qui régresse  $\varepsilon_t^2$  sur ses retards), rejettent massivement l'hypothèse de variance constante (p-valeurs de l'ordre de  $10^{-22}$  et  $10^{-8}$ ). Remarquez que le Ljung-Box appliqué aux rendements bruts rejette *aussi* faiblement (p-valeur d'environ 0,002) alors qu'il n'y a pas de mémoire en moyenne : ce test suppose une variance constante, et il est lui-même perturbé quand elle ne l'est pas. C'est une raison de plus de tester les carrés. Ajustons le GARCH(1,1) avec la bibliothèque arch :

```

fit = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1).fit(dis="off")
res = pd.DataFrame({"programmé": [mu, omega, alpha, beta], "estimé": fit.params.values, "erreur
    ↪ type": fit.std_err.values,
                    index=["mu", "omega", "alpha", "beta"]})
print(res.round(3).to_string())
a_hat, b_hat, w_hat = fit.params["alpha[1]", fit.params["beta[1]", fit.params["omega"]
print("\npersistence alpha + beta :", round(a_hat + b_hat, 3), "(programmé 0,95) | variance de long
    ↪ terme estimée :", round(w_hat / (1 - a_hat - b_hat), 3), "(programmé 1)")

fv = fit.forecast(horizon=20).variance.iloc[-1].values
print("prévision de la variance aux horizons 1, 5, 10 et 20 jours :", fv[[0, 4, 9, 19]].round(3))
z = fit.resid / fit.conditional_volatility
print("diagnostic : Ljung-Box des résidus standardisés AU CARRÉ, p =", round(float(acorr_ljungbox(z
    ↪ ** 2, lags=[10])["lb_pvalue"].iloc[0]), 3))

```

|       | programmé | estimé | erreur | type |
|-------|-----------|--------|--------|------|
| mu    | 0.02      | 0.010  | 0.024  |      |
| omega | 0.05      | 0.041  | 0.013  |      |
| alpha | 0.10      | 0.070  | 0.014  |      |
| beta  | 0.85      | 0.891  | 0.020  |      |

```

persistence alpha + beta : 0.961 (programmé 0,95) | variance de long terme estimée : 1.054 (programmé
    ↪ 1)
prévision de la variance aux horizons 1, 5, 10 et 20 jours : [0.735 0.782 0.83 0.903]
diagnostic : Ljung-Box des résidus standardisés AU CARRÉ, p = 0.548

```

Les estimations se rapprochent des vraies valeurs, avec des écarts de l'ordre de une à deux erreurs types ( $\alpha$  est sous-estimé de deux erreurs types environ,  $\beta$  surestimé d'autant : ils jouent des rôles voisins, comme  $\varphi$  et  $\theta$  en 4.2.3, et leurs erreurs se compensent). C'est la **persistence**  $\alpha + \beta$  (0,961 pour 0,95) qui est bien estimée. Après ajustement, les résidus standardisés au carré n'ont plus de mémoire : le modèle a absorbé le regroupement de volatilité. La prévision de la variance **remonte lentement vers sa valeur de long terme** (d'environ 0,74 à un jour à 0,90 à vingt jours,

pour une cible de 1,05) : comme la prévision d'un AR(1) (4.3.1) qui revient vers sa moyenne, mais lentement, car la persistance est proche de 1.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 5.4), sharex=True)
ax1.plot(r, color=BLEU, lw=0.7)
ax1.set_ylabel("rendement (%)")
ax2.plot(np.sqrt(sig2), color="#898781", lw=1.6, label="volatilité programmée")
ax2.plot(fit.conditional_volatility, color=ORANGE, lw=1.2, label="volatilité estimée par le GARCH")
ax2.set_ylabel("écart-type conditionnel")
ax2.set_xlabel("jour")
ax2.legend(loc="upper right", fontsize=8)
for ax in (ax1, ax2):
    ax.grid(True, color="#e1e0d9", lw=0.6)
    ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-garch.png", bbox_inches="tight")
```

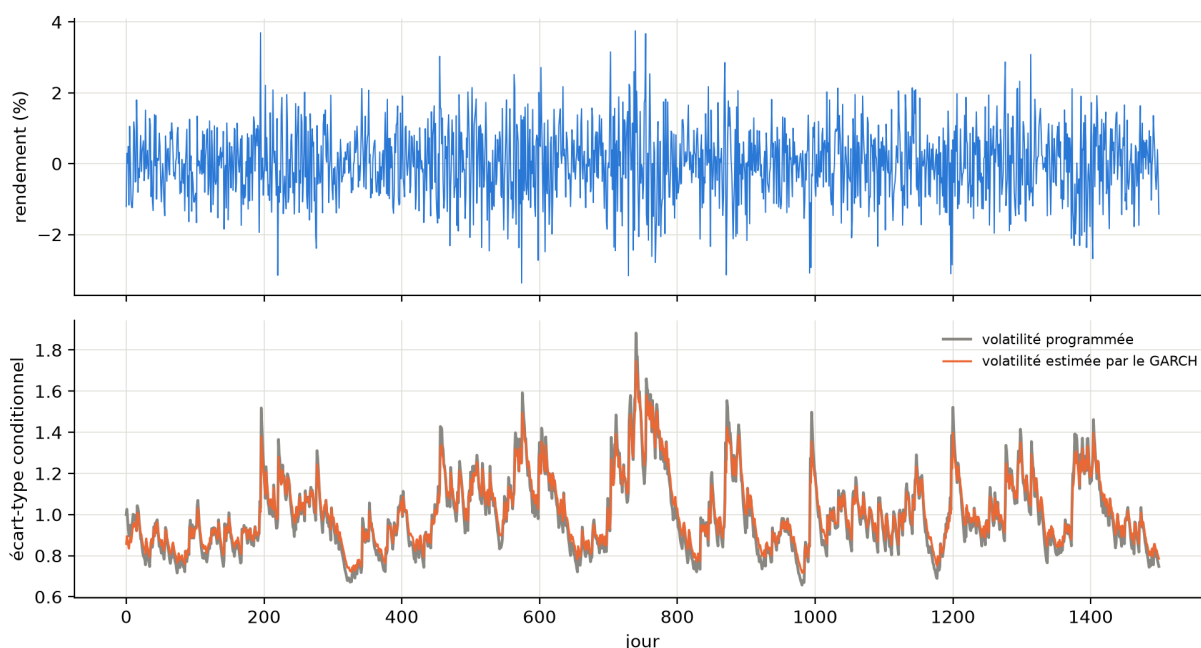


Figure 48: Rendements simulés (en haut), avec des périodes calmes et agitées, et volatilité conditionnelle (en bas) : la volatilité programmée (gris) et celle estimée par le GARCH (orange) se superposent presque.

**Ce que le GARCH modélise, et ce qu'il ne modélise pas.** Il prévoit la **variance** (donc l'incertitude, les intervalles de prévision, le risque), pas le niveau. Il est fondamental en gestion du risque financier (le volume  $V$  y reviendra). Sa validité repose sur l'hypothèse de loi (ici normale) pour les  $z_t$  ; sur des données réelles, les queues sont souvent plus épaisses, et on utilise une loi de Student.

#### À retenir.

- Un **VAR( $p$ )** régresse chaque série sur les retards de **toutes** les séries ; il est stationnaire si les valeurs propres de la matrice (compagne) sont de module  $< 1$  ; les **réponses impulsionnelles** suivent la propagation d'un choc.
- La **causalité de Granger** est une précedence prédictive, **pas** une causalité.
- La **régression entre séries non stationnaires est trompeuse** : environ quatre fois sur cinq, deux marches aléatoires indépendantes paraissent « significativement » liées. Mé-

fiance quand  $R^2 > \text{Durbin-Watson}$ .

- **Cointégration** : deux séries intégrées dont une combinaison est stationnaire ; test d'Engle-Granger ; le **modèle à correction d'erreur** a pour coefficient de rappel  $\alpha = \varphi - 1$ .
- **GARCH(1,1)** :  $\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$  ; variance de long terme  $\omega / (1 - \alpha - \beta)$  ; il modélise la volatilité qui se regroupe. Les rendements ont peu de mémoire, leurs carrés beaucoup.

## 4.5 ☉ Pour aller plus loin : modèles d'espace d'états et filtre de Kalman

**Section optionnelle.** Elle ne demande aucun prérequis nouveau (seulement le chapitre 1 de ce volume et la section 4.2), mais elle est plus abstraite que le reste du chapitre. Elle vaut le détour : les modèles ARIMA de 4.2 sont **un cas particulier** de ce cadre, et le **filtre de Kalman** que vous allez construire est l'algorithme qui a calculé, en coulisses, toutes les vraisemblances de SARIMAX.

### 4.5.1 L'idée : un état caché, des observations bruitées

Yasmine voudrait connaître le **niveau réel** de ses ventes, c'est-à-dire ce que serait son chiffre d'affaires sans les accidents du mois (une grosse commande, un jour de pluie). Elle n'observe que le chiffre d'affaires du mois, qui est le **niveau réel plus du bruit**. Le niveau réel, lui, évolue lentement. C'est un problème à **deux niveaux** :

- une **équation d'observation** : ce que l'on mesure,  $y_t = \mu_t + \varepsilon_t$ , avec  $\varepsilon_t \sim \mathcal{N}(0, \sigma_\varepsilon^2)$  ;
- une **équation d'état** : comment l'état caché évolue,  $\mu_{t+1} = \mu_t + \eta_t$ , avec  $\eta_t \sim \mathcal{N}(0, \sigma_\eta^2)$ .

C'est le **modèle de niveau local** (*local level*). Le niveau  $\mu_t$  est une marche aléatoire (il dérive sans retour), que l'on n'observe qu'à travers le bruit  $\varepsilon_t$ . Tout le problème est de **reconstituer**  $\mu_t$  à partir des  $y_t$  : c'est le rôle du **filtre de Kalman**. L'idée se généralise à un état de plusieurs composantes (niveau, pente, saison, régression), et à des observations manquantes ; nous verrons les deux.

### 4.5.2 Le filtre de Kalman, à la main

À chaque date, le filtre maintient une **croissance** sur l'état : une loi normale  $\mathcal{N}(a, P)$  dont  $a$  est la meilleure estimation et  $P$  l'incertitude. Il alterne deux gestes.

1. **Prédiction.** Entre deux dates, le niveau dérive : l'estimation ne bouge pas ( $a_{t+1|t} = a_{t|t}$ ) mais l'incertitude grandit :  $P_{t+1|t} = P_{t|t} + \sigma_\eta^2$ .
2. **Mise à jour.** On observe  $y_t$ . L'**innovation**  $v_t = y_t - a_{t|t-1}$  est la surprise (ce que l'on n'avait pas prévu). On corrige l'estimation d'une fraction  $K_t$  de cette surprise :

$$a_{t|t} = a_{t|t-1} + K_t v_t, \quad K_t = \frac{P_{t|t-1}}{P_{t|t-1} + \sigma_\varepsilon^2}, \quad P_{t|t} = (1 - K_t) P_{t|t-1}.$$

Le coefficient  $K_t$  s'appelle le **gain de Kalman**. Il vaut entre 0 et 1 et arbitre entre deux sources d'information : l'ancienne croyance (si elle est incertaine,  $P$  grand,  $K$  proche de 1 : on se fie à la mesure) et la mesure (si elle est bruyante,  $\sigma_\varepsilon^2$  grand,  $K$  proche de 0 : on se fie à l'ancienne croyance).

**D'où vient la formule ?** C'est le calcul bayésien le plus simple : un a priori normal  $\mathcal{N}(a, P)$  sur  $\mu$ , une mesure  $y = \mu + \varepsilon$  avec  $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ . La loi *a posteriori* de  $\mu$  est normale, de précision (inverse de la variance) **égale à la somme des précisions** :  $\frac{1}{P_{\text{post}}} = \frac{1}{P} + \frac{1}{\sigma^2}$ , d'où

$P_{\text{post}} = \frac{P\sigma^2}{P + \sigma^2} = (1 - K)P$  ; et de moyenne égale à la **moyenne des deux informations pondérée par leur précision** :  $a_{\text{post}} = \frac{a/P + y/\sigma^2}{1/P + 1/\sigma^2} = a + K(y - a)$ . Le filtre de Kalman est donc une mise à jour bayésienne répétée, avec une étape de « vieillissement » entre deux (nous reverrons ce principe au chapitre 6).

**Exemple à la main.** Prenons  $\sigma_\varepsilon^2 = 4$ ,  $\sigma_\eta^2 = 1$ , et une croyance initiale  $\mathcal{N}(100, 5)$  sur le niveau avant la première mesure. On observe  $y_1 = 103$ ,  $y_2 = 101$ ,  $y_3 = 106$ .

**Date 1.**  $a_{1|0} = 100$ ,  $P_{1|0} = 5$ . Variance de l'innovation :  $F_1 = P + \sigma_\varepsilon^2 = 9$ . Gain  $K_1 = 5/9 = 0,5556$ . Surprise  $v_1 = 103 - 100 = 3$ . Donc  $a_{1|1} = 100 + 0,5556 \times 3 = 101,667$  et  $P_{1|1} = 5 \times (1 - 0,5556) = 2,222$ . **Date 2.** Prédiction :  $a_{2|1} = 101,667$ ,  $P_{2|1} = 2,222 + 1 = 3,222$ .  $F_2 = 7,222$ ,  $K_2 = 3,222/7,222 = 0,4462$ . Surprise  $v_2 = 101 - 101,667 = -0,667$ . Donc  $a_{2|2} = 101,667 + 0,4462 \times (-0,667) = 101,369$  et  $P_{2|2} = 3,222 \times (1 - 0,4462) = 1,785$ . **Date 3.**  $P_{3|2} = 2,785$ ,  $F_3 = 6,785$ ,  $K_3 = 0,4104$ ,  $v_3 = 106 - 101,369 = 4,631$ ,  $a_{3|3} = 101,369 + 0,4104 \times 4,631 = 103,270$ ,  $P_{3|3} = 1,642$ .

Observez deux choses : le gain **diminue** (0,56 puis 0,45 puis 0,41) à mesure que l'on en sait plus, et l'incertitude  $P_{t|t}$  diminue aussi.

### 4.5.3 Le filtre de Kalman en numpy, et comparaison avec statsmodels

Voici le filtre complet pour le niveau local. Il traite aussi les **observations manquantes** (une valeur NaN) : on ne fait alors **que la prédiction**, sans mise à jour, et l'incertitude grandit. À chaque étape, il cumule la **log-vraisemblance** gaussienne des innovations (c'est ce qu'optimisent les logiciels pour estimer les variances).

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.structural import UnobservedComponents

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"

def kalman_niveau_local(y, s2_eps, s2_eta, a0, P0):
    """Filtre de Kalman du niveau local. (a0, P0) = croyance sur le niveau AVANT la première
    ↪ observation."""
    n = len(y)
    sortie = {k: np.zeros(n) for k in ["a_pred", "P_pred", "F", "K", "v", "a_filtre", "P_filtre"]}
    a, P, loglik = a0, P0, 0.0
    for t in range(n):
        sortie["a_pred"][t], sortie["P_pred"][t] = a, P
        if np.isnan(y[t]):
            # observation manquante : pas de mise à jour
            sortie["F"][t] = sortie["K"][t] = sortie["v"][t] = np.nan
            a_f, P_f = a, P
        else:
            F = P + s2_eps
            v = y[t] - a
            K = P / F
            a_f, P_f = a + K * v, P * (1 - K)
            loglik += -0.5 * (np.log(2 * np.pi * F) + v ** 2 / F)
            sortie["F"][t], sortie["K"][t], sortie["v"][t] = F, K, v
        sortie["a_filtre"][t], sortie["P_filtre"][t] = a_f, P_f
        a, P = a_f, P_f + s2_eta
    # prédiction pour la date suivante
```



```

    sortie["loglik"] = loglik
    return sortie

obs = np.array([103.0, 101.0, 106.0])
r = kalman_niveau_local(obs, s2_eps=4.0, s2_eta=1.0, a0=100.0, P0=5.0)
print(pd.DataFrame({k: r[k] for k in ["a_pred", "P_pred", "F", "K", "v", "a_filtre", "P_filtre"]},
    ↪ index=[1, 2, 3]).round(4).to_string())

# la même chose avec statsmodels, avec la même croyance initiale connue
mod = UnobservedComponents(obs, level="llevel")
mod.ssm.initialize_known(np.array([100.0]), np.array([[5.0]]))
mod.ssm.loglikelihood_burn = 0 # par défaut, statsmodels ignore le 1er
    ↪ terme de la vraisemblance
res_sm = mod.smooth([4.0, 1.0]) # variances : irrégulière (4), niveau (1)
print("\nétat filtré statsmodels      :", res_sm.filtered_state[0].round(4), "| variances filtrées :",
    ↪ res_sm.filtered_state_cov[0, 0].round(4))
print("log-vraisemblance : numpy =", round(r["loglik"], 4), "| statsmodels =", round(res_sm.llf, 4))

```

|   | a_pred   | P_pred | F      | K      | v       | a_filtre | P_filtre |
|---|----------|--------|--------|--------|---------|----------|----------|
| 1 | 100.0000 | 5.0000 | 9.0000 | 0.5556 | 3.0000  | 101.6667 | 2.2222   |
| 2 | 101.6667 | 3.2222 | 7.2222 | 0.4462 | -0.6667 | 101.3692 | 1.7846   |
| 3 | 101.3692 | 2.7846 | 6.7846 | 0.4104 | 4.6308  | 103.2698 | 1.6417   |

```

état filtré statsmodels      : [101.6667 101.3692 103.2698] | variances filtrées : [2.2222 1.7846
    ↪ 1.6417]
log-vraisemblance : numpy = -7.9124 | statsmodels = -7.9124

```

Le filtre écrit en numpy retrouve les valeurs calculées à la main (101,667 ; 101,369 ; 103,270) et celles de statsmodels, **à la quatrième décimale**, y compris la log-vraisemblance.

### Estimer les variances

Dans la pratique, on ne connaît pas  $\sigma_\varepsilon^2$  ni  $\sigma_\eta^2$  : on les **estime par maximum de vraisemblance**. Testons sur un niveau local **simulé** de 200 points (variances programmées : 4 et 1) :

```

rng = np.random.default_rng(5)
n = 200
niveau = 100 + np.cumsum(rng.normal(0, 1.0, n)) # sigma_eta = 1
y_sim = niveau + rng.normal(0, 2.0, n) # sigma_eps = 2

mod = UnobservedComponents(y_sim, level="llevel")
ajust = mod.fit(dispatch=False)
print(pd.DataFrame({"programmé": [4.0, 1.0], "estimé": ajust.params, "erreur type": ajust.bse},
    ↪ index=["variance du bruit d'observation", "variance du
    ↪ niveau"]).round(3).to_string())

```

|                                 | programmé | estimé | erreur type |
|---------------------------------|-----------|--------|-------------|
| variance du bruit d'observation | 4.0       | 3.051  | 0.445       |
| variance du niveau              | 1.0       | 1.453  | 0.382       |

**Estimer deux variances est difficile.** Les estimations sont dans la bonne région (3,05 pour 4 et 1,45 pour 1) mais avec de **grandes erreurs types** (la variance d'observation est sous-estimée d'environ deux erreurs types) : à partir de la seule série, la part du « bruit de mesure » et celle de la « vraie dérive » du niveau sont difficiles à séparer, comme  $\varphi$  et  $\theta$  en 4.2.3. Seul le **rapport**  $q = \sigma_\eta^2 / \sigma_\varepsilon^2$  gouverne le comportement du filtre (nous le voyons ci-dessous), ce qui limite l'effet de cette incertitude sur les prévisions.



### 4.5.4 Régime permanent et lissage exponentiel

Quand le filtre a tourné assez longtemps, le gain  $K_t$  se stabilise à une valeur  $K_\infty$ . Calculons-la pour  $\sigma_\varepsilon^2 = 4$  et  $\sigma_\eta^2 = 1$  : en régime permanent, la variance prédite  $p = P_{t+1|t}$  vérifie  $p = \frac{p\sigma_\varepsilon^2}{p + \sigma_\varepsilon^2} + \sigma_\eta^2$ .

En multipliant par  $p + 4$  :  $p(p + 4) = 4p + (p + 4)$ , soit  $p^2 - p - 4 = 0$ , d'où  $p = \frac{1 + \sqrt{17}}{2} \approx 2,562$  et  $K_\infty = \frac{p}{p + 4} \approx 0,390$ .

Or, une fois  $K$  constant, la mise à jour devient  $a_{t|t} = a_{t-1|t-1} + K(y_t - a_{t-1|t-1}) = Ky_t + (1 - K)a_{t-1|t-1}$  : exactement le **lissage exponentiel simple** de paramètre  $\alpha = K_\infty$  (la moyenne mobile exponentielle bien connue des prévisionnistes). Le lissage exponentiel est donc le filtre de Kalman d'un niveau local en régime permanent. Vérifions-le numériquement :

```
r = kalman_niveau_local(y_sim, 4.0, 1.0, a0=100.0, P0=25.0)
p = (1 + np.sqrt(17)) / 2
print("gain final du filtre :", round(r["K"][-1], 5), "| théorie p/(p+4) :", round(p / (p + 4), 5))

alpha = p / (p + 4)
lisse = np.zeros(n)
lisse[0] = r["a_filtre"][0]
for t in range(1, n):
    lisse[t] = alpha * y_sim[t] + (1 - alpha) * lisse[t - 1]
print("écart maximal lissage exponentiel / filtre de Kalman, après 30 pas :", f"{np.abs(lisse[30:] - r['a_filtre'][30:]).max():.1e}")
```

```
gain final du filtre : 0.39039 | théorie p/(p+4) : 0.39039
écart maximal lissage exponentiel / filtre de Kalman, après 30 pas : 2.2e-07
```

Le gain converge vers sa valeur théorique, et le lissage exponentiel avec  $\alpha = 0,39$  est le filtre de Kalman une fois le régime permanent atteint (écart de l'ordre de  $10^{-7}$ ). Plus le niveau bouge vite par rapport au bruit ( $q$  grand), plus  $K_\infty$  est grand : le filtre « oublie » vite le passé.

### 4.5.5 Un modèle structurel pour les ventes de Dar Jasmin

L'intérêt des modèles d'espace d'états est de **mettre bout à bout** des composantes comprises : un niveau, une pente, une saison, des variables explicatives. C'est ce qu'on appelle un **modèle structurel**. Pour nos ventes (en logarithme, 96 mois d'apprentissage), prenons un niveau, une pente, une saison de période 12 et les variables promo et covid. Pour commencer, laissons la pente évoluer aléatoirement (le « *local linear trend* » classique) :

```
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
y = np.log(v["ca"])
train, test = y[:"2023-12"], y["2024-01":]
X = v[["promo", "covid"]].astype(float)
Xtr, Xte = X[:"2023-12"], X["2024-01":]

mod_lt = UnobservedComponents(train, exog=Xtr, level="l1trend", seasonal=12,
    ↳ stochastic_seasonal=False)
fit_lt = mod_lt.fit(dispatch=False, maxiter=300)
print(pd.Series(fit_lt.params, index=mod_lt.param_names).round(5).to_string())
```

```
sigma2.irregular    0.00141
sigma2.level        0.00276
sigma2.trend        0.00000
beta.promo          0.11356
beta.covid          -0.54473
```

L'estimation donne une **variance de la pente égale à zéro** (`sigma2.trend`) : les données disent que la pente est **constante**. Le modèle se simplifie alors en un niveau qui dérive à vitesse constante plus un bruit de niveau (en langage statsmodels : `level="rwdrift"`, une marche aléatoire avec dérive). Gardons ce modèle, plus simple, avec une saison **déterministe** (profil fixe), et regardons ce qu'il estime et comment il prévoit les 24 mois de test :

```
mod_ss = UnobservedComponents(train, exog=Xtr, level="rwdrift", seasonal=12,
    ↪ stochastic_seasonal=False)
fit_ss = mod_ss.fit(dispatch=False, maxiter=300)
print(pd.Series(fit_ss.params, index=mod_ss.param_names).round(4).to_string())

prev_ss = fit_ss.get_forecast(24, exog=Xte).predicted_mean
print("\nRMSE (log) sur les 24 mois de test :", round(np.sqrt(np.mean((test - prev_ss) ** 2)), 4),
    "| biais (y - prévision) :", round(float(np.mean(test - prev_ss)), 4))
print("pour comparaison (4.3) : modèle B 0,0738 ; naïf avec dérive 0,0849 ; modèle A 0,0997 ; modèle C
    ↪ 0,1008")
```

```
sigma2.level    0.0054
beta.promo      0.1090
beta.covid     -0.5263
```

```
RMSE (log) sur les 24 mois de test : 0.0819 | biais (y - prévision) : -0.0364
pour comparaison (4.3) : modèle B 0,0738 ; naïf avec dérive 0,0849 ; modèle A 0,0997 ; modèle C 0,1008
```

Le modèle structurel a une erreur de test d'environ 0,082, entre celle du modèle B (0,074) et celle des modèles A et C (environ 0,10). Il estime que l'effet de la promotion vaut environ +0,11 et celui du COVID environ -0,53, des valeurs voisines de celles de 4.2. Voyons les composantes que l'on peut lire dans ce modèle : le niveau lissé (la « vraie » tendance, débarrassée de la saison et du bruit) et le profil saisonnier.

```
noms = mod_ss.state_names # noms des composantes
    ↪ de l'état caché
niveau_lisse = pd.Series(fit_ss.smoothed_state[noms.index("level")], index=train.index) #
    ↪ composante de niveau
saison_lisse = pd.Series(fit_ss.smoothed_state[noms.index("seasonal")], index=train.index) #
    ↪ composante saisonnière à la date t
print("composantes de l'état :", noms[:4], "...")
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.6), gridspec_kw={"width_ratios": [1.5, 1]})
ax1.plot(train.index, train, color="#c3c2b7", lw=1.2, label="log(ca) observé")
ax1.plot(niveau_lisse.index, niveau_lisse, color=VIOLET, lw=2.2, label="niveau lissé (filtre de
    ↪ Kalman)")
ax1.set_title("Le niveau réel, débarrassé de la saison et du bruit", fontsize=10)
ax1.legend(fontsize=8, loc="upper left")
profil = saison_lisse["2023-01":"2023-12"]
ax2.bar(range(1, 13), 100 * (np.exp(profil.values) - 1), color=AQUA)
ax2.set_xticks(range(1, 13))
ax2.set_xticklabels(["j", "f", "m", "a", "m", "j", "j", "a", "s", "o", "n", "d"])
ax2.set_title("Profil saisonnier (écart au niveau, en %)", fontsize=10)
ax2.axhline(0, color="#898781", lw=0.8)
for ax in (ax1, ax2):
    ax.grid(True, color="#e1e0d9", lw=0.6)
    ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-structurel.png", bbox_inches="tight")
print("écart de décembre au niveau :", round(100 * (np.exp(profil.iloc[-1]) - 1)), "% ; de janvier :",
    ↪ round(100 * (np.exp(profil.iloc[0]) - 1)), "%")
```

```
composantes de l'état : ['level', 'trend', 'seasonal', 'seasonal.L1'] ...
écart de décembre au niveau : 59 % ; de janvier : -38 %
```

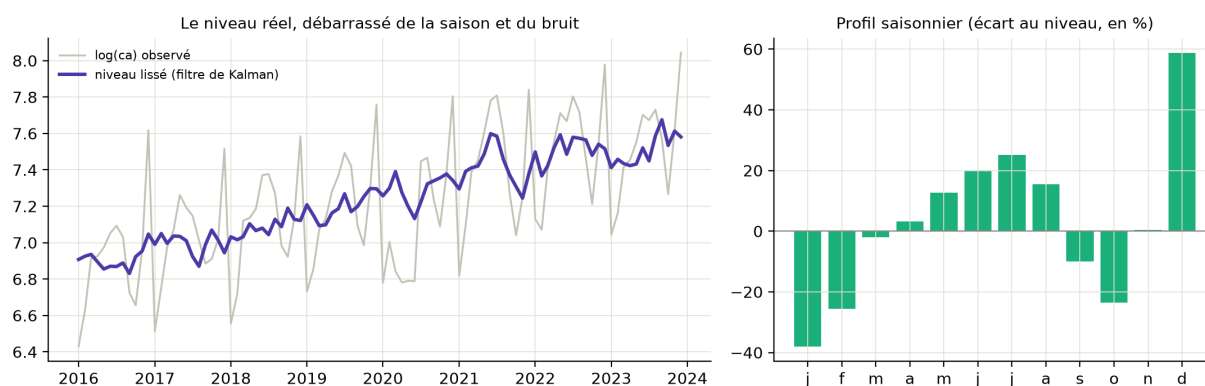


Figure 49: À gauche : le logarithme du chiffre d'affaires (gris) et le niveau lissé par le filtre de Kalman (violet) qui en retire la saison et le bruit. À droite : le profil saisonnier estimé, avec le pic de décembre et le creux de janvier.

#### 4.5.6 Combler un trou : les données manquantes

Un atout décisif du filtre de Kalman est de traiter les **observations manquantes** sans bricolage : à une date sans mesure, il ne fait que prédire (l'incertitude grandit), et le **lissage** (qui utilise aussi l'avenir) reconstitue la valeur manquante avec son intervalle d'incertitude. Un cas réaliste : les registres de Yasmine ont perdu les ventes de **mars à août 2022**. Nous effaçons ces six mois de la série d'apprentissage (nous connaissons les vraies valeurs, ce qui permet de vérifier), ajustons le modèle sur ce qui reste, puis **reconstituons** les mois manquants.

```
trou = train["2022-03":"2022-08"].index
vraies = train[trou].copy()
train_trou = train.copy()
train_trou[trou] = np.nan

mod_na = UnobservedComponents(train_trou, exog=Xtr, level="rwdrift", seasonal=12,
    ↳ stochastic_seasonal=False)
fit_na = mod_na.fit(dis=False, maxiter=300)
lissage = fit_na.smoother_results
estim = pd.Series(lissage.smoothed_forecasts[0], index=train.index)[trou]
se = pd.Series(np.sqrt(lissage.smoothed_forecasts_error_cov[0, 0]), index=train.index)[trou]

# deux méthodes sans modèle, pour comparer : interpolation linéaire, et « même mois l'an dernier »
↳ corrigé de la croissance annuelle
lineaire = pd.Series(np.interp(np.arange(6), [-1, 6], [train["2022-02"].iloc[0],
    ↳ train["2022-09"].iloc[0]]), index=trou)
croissance = (train - train.shift(12))["2022-02"].mean()
an_dernier = pd.Series(train.shift(12)[trou].values + croissance, index=trou)

tab = pd.DataFrame({"vrai (log)": vraies, "Kalman": estim, "± 1,96 se": 1.96 * se, "interp. linéaire":
    ↳ lineaire, "an dernier + croiss.": an_dernier})
print(tab.round(3).to_string())
rmse = lambda a, b: np.sqrt(np.mean((a - b) ** 2))
print("\nRMSE de reconstitution (log) : Kalman =", round(rmse(vraies, estim), 3), "| interpolation
    ↳ linéaire =", round(rmse(vraies, lineaire), 3),
    ↳ "| an dernier corrigé =", round(rmse(vraies, an_dernier), 3))
print("mois réels dans l'intervalle à 95 % de Kalman :", int(((vraies >= estim - 1.96 * se) & (vraies
    ↳ <= estim + 1.96 * se)).sum()), "sur 6")
```

|            | vrai (log) | Kalman | ± 1,96 se | interp. linéaire | an dernier + croiss. |
|------------|------------|--------|-----------|------------------|----------------------|
| mois       |            |        |           |                  |                      |
| 2022-03-01 | 7.400      | 7.371  | 0.142     | 7.127            | 7.491                |
| 2022-04-01 | 7.553      | 7.442  | 0.183     | 7.182            | 7.553                |
| 2022-05-01 | 7.711      | 7.550  | 0.201     | 7.238            | 7.706                |

|            |       |       |       |       |       |
|------------|-------|-------|-------|-------|-------|
| 2022-06-01 | 7.667 | 7.661 | 0.201 | 7.293 | 7.881 |
| 2022-07-01 | 7.803 | 7.722 | 0.183 | 7.349 | 7.909 |
| 2022-08-01 | 7.717 | 7.674 | 0.142 | 7.404 | 7.701 |

RMSE de reconstitution (log) : Kalman = 0.089 | interpolation linéaire = 0.383 | an dernier corrigé = 0.104

mois réels dans l'intervalle à 95 % de Kalman : 6 sur 6

Le filtre reconstitue les six mois avec une erreur quadratique moyenne de 0,09 en logarithme (soit 9 %) et les six vraies valeurs tombent dans son intervalle à 95 %, plus large au milieu du trou (là où l'on est le plus loin des mesures). C'est bien mieux que l'**interpolation linéaire** (0,38), qui ignore la saison et lisse une bosse qui existe réellement. La comparaison avec la méthode « l'an dernier corrigé de la croissance » (0,10) est plus serrée : elle utilise déjà la saison, mais pas l'information des mois qui entourent le trou. Les écarts du filtre ne sont pas nuls : la plus grosse erreur, en mai, est d'environ 0,16.

#### À retenir.

- Un **modèle d'espace d'états** distingue un **état caché** (niveau, pente, saison...) qui évolue, et des **observations bruitées**. Les ARIMA en sont un cas particulier.
- Le **filtre de Kalman** alterne **prédiction** (l'incertitude grandit) et **mise à jour** :  $a \leftarrow a + K(y - a)$ , avec le **gain**  $K = P / (P + \sigma_\epsilon^2)$  qui arbitre entre ancienne croyance et mesure. C'est une mise à jour bayésienne répétée.
- Il fournit aussi la **log-vraisemblance** des données : c'est elle qui sert à estimer les paramètres (variances), dans SARIMAX comme dans UnobservedComponents.
- En régime permanent, le filtre d'un niveau local **est** un lissage exponentiel de paramètre  $\alpha = K_\infty$ .
- Les **modèles structurels** assemblent niveau, pente, saison et variables explicatives en composantes lisibles ; ici, l'estimation de la variance de la pente à zéro a simplifié le modèle.
- Les **observations manquantes** se traitent naturellement (prédire sans mettre à jour, puis lisser), avec une incertitude honnête.

## 4.6 ☉ Pour aller plus loin : Prophet et les bibliothèques modernes de prévision

**Section optionnelle.** Elle répond à une question que se posent beaucoup de lecteurs : « Et les bibliothèques à la mode, qui font tout en trois lignes ? ». Réponse en trois temps : comprendre ce qu'elles font **vraiment** (4.6.1 et 4.6.2), les **évaluer honnêtement** comme les autres (4.6.3), et savoir où chercher la suite (4.6.4).

### 4.6.1 Prophet : l'idée

**Prophet** est une bibliothèque de prévision publiée par des ingénieurs de Facebook (Taylor et Letham, 2018), conçue pour des séries d'entreprise avec des saisons marquées, des jours fériés et des ruptures de tendance. Son modèle est **additif** :

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t,$$

avec  $g(t)$  une tendance **linéaire par morceaux** (la pente peut changer en quelques dates candidates, les *points de rupture*, avec une pénalisation qui évite que toutes les ruptures soient utilisées),  $s(t)$  une saisonnalité décrite par une **série de Fourier** (une somme de sinus et de cosinus

de période 1 an :  $\sum_k a_k \sin(2\pi kt/P) + b_k \cos(2\pi kt/P)$ , et  $h(t)$  l'effet d'événements (jours fériés, promotions) ou de variables explicatives. L'ajustement est bayésien (les coefficients ont des lois a priori ; l'estimation se fait avec le logiciel Stan), et les intervalles de prévision sont obtenus par simulation.

Rien de magique donc : c'est une **régression linéaire** sur des variables bien construites (la tendance par morceaux, les termes de Fourier, les événements), assortie d'une pénalisation, exactement dans l'esprit du chapitre 1 (section 1.5 : la régularisation). Voici le code typique, tel qu'on le trouve dans la documentation de la bibliothèque :

```
“python noexec from prophet import Prophet
```

```
df = v.reset_index().rename(columns={"mois": "ds"}) # Prophet attend les colonnes
« ds » (date) et « y » df["y"] = np.log(df["ca"]) m = Prophet(yearly_seasonality=6,
weekly_seasonality=False, daily_seasonality=False, changepoint_prior_scale=0.05) # nombre
de termes de Fourier ; souplesse de la tendance m.add_regressor("promo") m.add_regressor("covid")
m.fit(df[df["ds"] < "2024-01-01"]) futur = m.make_future_dataframe(periods=24, freq="MS")
futur = futur.merge(df[["ds", "promo", "covid"]], on="ds") prevision = m.predict(futur) #
colonnes yhat, yhat_lower, yhat_upper, trend, yearly...
```

```
> (!) **Ce bloc n'est pas exécuté.** La bibliothèque Prophet n'est pas installée dans l'environnement
```

```
### 4.6.2 Le même modèle, écrit à la main
```

Pour comprendre, le plus sûr est de **construire** le modèle. Nous ajustons, sur  $\log(\text{ca})$  :

- une **tendance linéaire par morceaux** :  $g(t) = k + a \cdot t + \sum_j \delta_j \max(0, t - s_j)$ , où les  $s_j$
- une **saisonnalité de Fourier** à  $K$  harmoniques :  $s(t) = \sum_{k=1}^K [a_k \sin(2\pi k \cdot t / m) + b_k \cos(2\pi k \cdot t / m)]$
- les variables explicatives `promo` et `covid`.

C'est une régression linéaire pénalisée : les moindres carrés avec un terme  $\lambda \sum \delta_j^2$ .

```
python
import warnings
warnings.filterwarnings("ignore")
import itertools
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.structural import UnobservedComponents

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
y = np.log(v["ca"])
train, test = y[:"2023-12"], y["2024-01":]
X = v[["promo", "covid"]].astype(float)
temps = np.arange(120.0) # 0 = janvier 2016
num_mois = v.index.month.values.astype(float)
Xv = X.values

def colonnes(i_t, i_mois, K, ruptures, promo, covid):
```

```

t = i_t / 120.0
fourier = []
for k in range(1, K + 1):
    for f in (np.sin, np.cos):
        c = f(2 * np.pi * k * i_mois / 12)
        if np.abs(c).max() > 1e-9:
            # pour k = 6, sin(pi m) est identiquement nul : on ne pénalise pas
            fourier.append(c)
base = [np.ones_like(t), t] + fourier + [promo, covid]
sauts = [np.maximum(0.0, t - s / 120.0) for s in ruptures]
return np.column_stack(base + sauts), len(base) # (matrice, nombre de colonnes NON pénalisées)

def ajuster_predire(y_app, i_app, i_prev, m_app, m_prev, X_app, X_prev, K, lam, n_rupt=20):
    ruptures = np.linspace(6, 0.85 * len(i_app), n_rupt) # points de rupture candidats dans les 85 premiers mois
    A, nb = colonnes(i_app, m_app, K, ruptures, X_app[:, 0], X_app[:, 1])
    B, _ = colonnes(i_prev, m_prev, K, ruptures, X_prev[:, 0], X_prev[:, 1])
    pen = np.zeros(A.shape[1])
    pen[nb:] = 1.0 # on ne pénalise que les changements de pente
    # moindres carrés pénalisés, résolus de façon stable : on empile sqrt(lambda) * pénalité sous la matrice
    A_aug = np.vstack([A, np.sqrt(lam) * np.diag(pen)])
    y_aug = np.r_[y_app, np.zeros(A.shape[1])]
    beta = np.linalg.lstsq(A_aug, y_aug, rcond=None)[0]
    return B @ beta, A @ beta

lignes = []
for K, lam in itertools.product((1, 2, 3, 4, 5, 6), (0.001, 0.01, 0.1, 1, 10)):
    f, _ = ajuster_predire(train.values[:72], temps[:72], temps[72:96], num_mois[:72], num_mois[72:96])
    lignes.append({"K": K, "lambda": lam, "RMSE": np.sqrt(np.mean((train.values[72:96] - f) ** 2))})
grille = pd.DataFrame(lignes)
print("RMSE (log) de validation, sur les 24 derniers mois de l'apprentissage :")
print(grille.pivot(index="K", columns="lambda", values="RMSE").round(3).to_string())
meilleur = grille.sort_values("RMSE").iloc[0]
K_opt, lam_opt = int(meilleur["K"]), float(meilleur["lambda"])
print(f"réglage retenu : K = {K_opt} harmoniques, lambda = {lam_opt} (RMSE de validation : {meilleur['RMSE']})")

```

RMSE (log) de validation, sur les 24 derniers mois de l'apprentissage :

| lambda | 0.001 | 0.010 | 0.100 | 1.000 | 10.000 |
|--------|-------|-------|-------|-------|--------|
| K      |       |       |       |       |        |
| 1      | 0.386 | 0.347 | 0.323 | 0.307 | 0.293  |
| 2      | 0.294 | 0.284 | 0.277 | 0.271 | 0.260  |
| 3      | 0.183 | 0.190 | 0.197 | 0.199 | 0.192  |
| 4      | 0.133 | 0.125 | 0.133 | 0.145 | 0.141  |
| 5      | 0.139 | 0.096 | 0.099 | 0.112 | 0.110  |
| 6      | 0.153 | 0.094 | 0.090 | 0.104 | 0.103  |

réglage retenu : K = 6 harmoniques, lambda = 0.1 (RMSE de validation : 0.090)

Lecture de la grille. Avec **peu d'harmoniques** ( $K = 1$  à  $3$ ), la saison est trop lisse : une seule ondulation sinusoïdale ne peut pas reproduire le **pic étroit de décembre** et le creux de janvier, et l'erreur de validation est grande (de 0,18 à 0,39). Il en faut davantage : l'erreur chute pour  $K = 4$  puis atteint son minimum pour  $K = 6$ . Or  $K = 6$  harmoniques sur des données mensuelles équivaut à **un effet libre pour chaque mois** (12 paramètres au total), comme les 11 indicatrices de 4.2.7 : c'est la limite du système. Sur des données quotidiennes, 10 harmoniques suffisent à décrire un profil annuel détaillé ; avec seulement 12 points par an, il ne reste rien à *lisser*.

Refaisons l'ajustement sur les 96 mois et prévoyons les 24 mois de test :

```

f_test, ajuste = ajuster_predire(train.values, temps[:96], temps[96:], num_mois[:96], num_mois[96:],
    ↪ Xv[:96], Xv[96:], K_opt, lam_opt)
print("RMSE (log) sur les 24 mois de test :", round(np.sqrt(np.mean((test.values - f_test) ** 2)), 4),
    ↪ "| biais (y - prévision) :", round(float(np.mean(test.values - f_test)), 4))
print("pour comparaison (4.3) : modèle B 0,0738 ; naïf avec dérive 0,0849 ; modèle A 0,0997 ; modèle C
    ↪ 0,1008")
print("RMSE d'ajustement sur l'apprentissage :", round(np.sqrt(np.mean((train.values - ajuste) **
    ↪ 2)), 4))

fig, ax = plt.subplots(figsize=(10, 3.6))
ax.plot(train.index, train, color="#c3c2b7", lw=1.3, label="log(ca) observé (apprentissage)")
ax.plot(train.index, ajuste, color=VIOLET, lw=1.6, label="ajustement (tendance par morceaux +
    ↪ Fourier)")
ax.plot(test.index, test, color=BLEU, lw=1.6, label="log(ca) observé (test)")
ax.plot(test.index, f_test, color=ORANGE, lw=1.8, label="prévision")
ax.legend(fontsize=8, loc="upper left")
ax.grid(True, color="#e1e0d9", lw=0.6)
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.savefig("figures/ch04-fourier.png", bbox_inches="tight")

```

```

RMSE (log) sur les 24 mois de test : 0.0801 | biais (y - prévision) : 0.0173
pour comparaison (4.3) : modèle B 0,0738 ; naïf avec dérive 0,0849 ; modèle A 0,0997 ; modèle C 0,1008
RMSE d'ajustement sur l'apprentissage : 0.0679

```

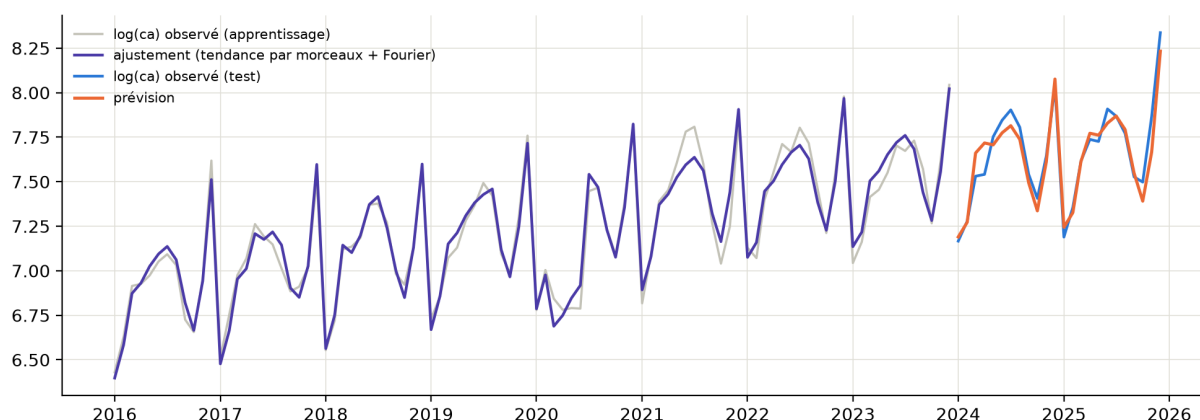


Figure 50: Modèle de type Prophet écrit à la main : ajustement sur l'apprentissage (violet) et prévision des 24 mois de test (orange), comparés aux observations (gris puis bleu).

Le modèle fait presque aussi bien que le meilleur des modèles de 4.3 sur la fenêtre de test (RMSE de 0,080 contre 0,074 pour B, et mieux que A, C et le naïf avec dérive en log). Ce que fait Prophet, au fond, ce n'est pas de la magie : c'est cela, avec des a priori bayésiens, des intervalles simulés, et une interface qui évite d'écrire la matrice de dessin.

### 4.6.3 Évaluation honnête : tout le monde au même concours

Un nouveau venu doit passer **le même concours** que les autres, sans faveur. Nous ajoutons donc à la validation à origine glissante de 4.3.6 deux nouveaux concurrents : le **modèle de type Prophet** que nous venons de construire (avec les réglages  $K$  et  $\lambda$  retenus *sur l'apprentissage*), et le **modèle structurel** de 4.5.5. On reprend les objets de 4.3 ( $P$ ,  $\text{reel}$ ,  $\text{origines}$ ,  $H$ ,  $y$ ,  $X$ ) et on y ajoute deux séries de prévisions.

```

def prev_fourier(i):
    f, _ = ajuster_predire(y.values[:i], temps[:i], temps[i:i + H], num_mois[:i], num_mois[i:i + H],
    ↪ Xv[:i], Xv[i:i + H], K_opt, lam_opt)

```



```

    return f

def prev_structuel(i):
    m = UnobservedComponents(y.iloc[:i], exog=X.iloc[:i], level="rwdrift", seasonal=12,
    ↪ stochastic_seasonal=False).fit(dis=False, maxiter=300)
    return m.forecast(H, exog=X.iloc[i:i + H]).values

P["type Prophet (Fourier + tendance par morceaux)"] = np.array([prev_fourier(i) for i in origines])
P["modèle structurel (niveau local + saison)"] = np.array([prev_structuel(i) for i in origines])

lignes = []
for nom, f in P.items():
    e = reel - f
    lignes.append({"modèle": nom, "RMSE global": np.sqrt((e ** 2).mean()), "h = 1": np.sqrt((e[:, 0]
    ↪ ** 2).mean()),
                    "h = 12": np.sqrt((e[:, 11] ** 2).mean()), "biais": e.mean()})
tableau_final = pd.DataFrame(lignes).set_index("modèle").sort_values("RMSE global")
print("validation à origine glissante (13 origines, 12 horizons), RMSE en logarithme :")
print(tableau_final.round(4).to_string())

fig, ax = plt.subplots(figsize=(9, 3.8))
ordre = tableau_final.index[::-1]
couleurs = [AQUA if "Prophet" in n else (ORANGE if n.startswith(("A", "B", "C", "comb")) else
    ↪ "#898781") for n in ordre]
ax.barh(range(len(ordre)), tableau_final.loc[ordre, "RMSE global"], color=couleurs)
ax.set_yticks(range(len(ordre)))
ax.set_yticklabels(ordre, fontsize=8)
ax.set_xlabel("RMSE global (en logarithme) : plus petit = meilleur")
ax.axvline(0.0715, color=ROUGE, ls="--", lw=1.1)
ax.text(0.0722, len(ordre) - 1, "écart-type du bruit à long terme (0,0715)", color=ROUGE, fontsize=8,
    ↪ va="center")
ax.spines[["top", "right"]].set_visible(False)
ax.grid(True, axis="x", color="#e1e0d9", lw=0.6)
plt.tight_layout()
plt.savefig("figures/ch04-concours.png", bbox_inches="tight")

```

```

validation à origine glissante (13 origines, 12 horizons), RMSE en logarithme :
                                RMSE global   h = 1   h = 12   biais
modèle
type Prophet (Fourier + tendance par morceaux)    0.0665  0.0777  0.0673  0.0125
B : (1,0,0)(0,1,1) + tendance                    0.0670  0.0712  0.0889  0.0037
combinaison A+B+C                                0.0687  0.0708  0.0808 -0.0295
A : SARIMAX(1,1,1)(0,1,1)                        0.0763  0.0737  0.0890 -0.0416
C : régression + AR(1)                           0.0807  0.0742  0.0903 -0.0506
naïf saisonnier + dérive                         0.0839  0.0753  0.1052 -0.0175
modèle structurel (niveau local + saison)         0.0996  0.0786  0.1022 -0.0066
naïf saisonnier                                  0.1081  0.1213  0.1264  0.0707

```

Lecture honnête du tableau :

1. Le modèle de type Prophet, tel que nous l'avons écrit, arrive **en tête** du classement (RMSE de 0,0665), de très peu devant le modèle B (0,0670) : un écart bien inférieur à ce que 13 origines permettent de distinguer (4.3.5). Il est **dans le peloton de tête**, pas au-dessus.
2. Il y est parce que, pour des données mensuelles à saison nette, il se ramène à « **tendance linéaire par morceaux + effets de mois libres + régresseurs** », une structure très proche de la vérité (le modèle C de 4.2.7 en est le cousin direct, sans changement de pente).
3. Le **modèle structurel**, très bon pour reconstituer des trous (4.5.6), fait ici **moins bien que la référence simple avec dérive** (0,0996 contre 0,0839) : son niveau en marche aléatoire « suit le bruit » et l'ancre sur des écarts qui s'effacent. Un bon outil pour une tâche (interpoler) n'est pas forcément le meilleur pour une autre (prévoir).
4. Les meilleurs modèles ont une erreur de 0,067 : entre l'écart-type des chocs mensuels (0,060,

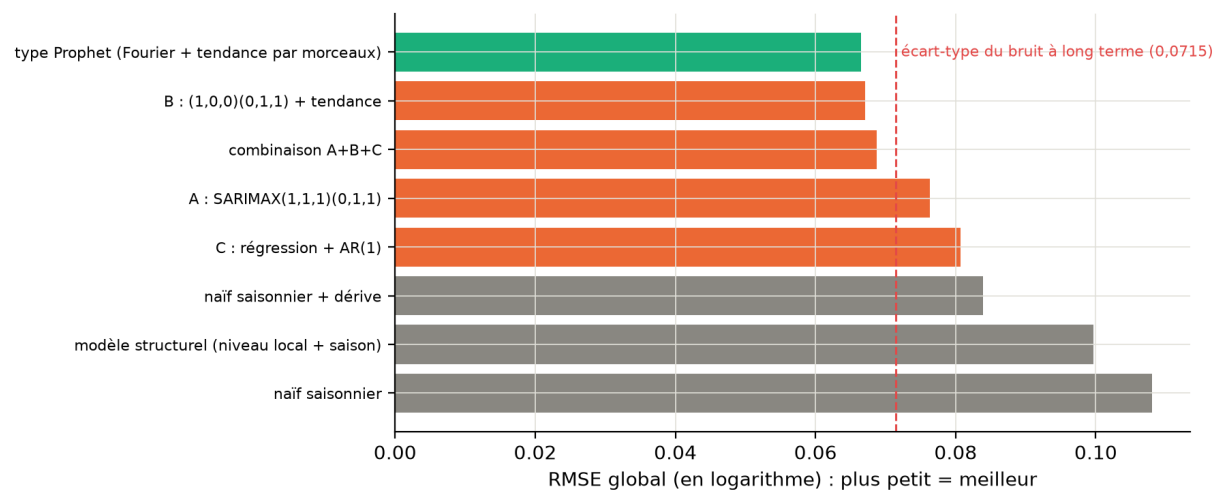


Figure 51: Concours final à origine glissante : RMSE (en logarithme) de huit méthodes, avec en pointillé l'écart-type du bruit de la série estimé en 4.3.5. Les trois meilleures, dont le modèle de type Prophet, sont voisines et au niveau de ce bruit.

le minimum pour une prévision à un mois) et celui de l'écart à la tendance (0,0715, valeur à long terme). Ils font à **peu près ce que le bruit permet : plus de sophistication n'achète pas de précision** quand on est déjà proche de ce que les données permettent.

**Mise en garde sur ce classement.** Il porte sur **une** série, **une** période de test, des réglages choisis par nous. Nous avons vu en 4.3.8 que le classement d'un jeu de test de deux ans peut s'inverser avec un autre tirage du hasard. Ne généralisez pas : un benchmark honnête (nombreuses séries, nombreuses origines, mêmes données pour tous) est un travail à part entière.

4.6.4 Les autres bibliothèques, et comment choisir

Voici un tour d'horizon, **sans exécution** ici (aucune de ces bibliothèques n'est installée dans l'environnement du livre) ; vérifiez la documentation de la version que vous utilisez.

| Bibliothèque                                        | Idée                                                                                                   | Quand l'utiliser                                                       |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| statsmodels (utilisée ici)                          | ARIMA, SARIMAX, espace d'états, ETS, VAR                                                               | comprendre, diagnostiquer, un petit nombre de séries                   |
| statsforecast                                       | versions très rapides d'AutoARIMA, ETS, modèles naïfs                                                  | <b>des milliers de séries</b> à prévoir d'un coup                      |
| sktime, darts                                       | interface unifiée (même syntaxe pour des dizaines de modèles), validation à origine glissante intégrée | comparer plusieurs familles de modèles proprement                      |
| Prophet, NeuralProphet                              | décomposition additive, événements, intervalles                                                        | séries d'entreprise à plusieurs saisons et jours fériés                |
| modèles de fondation (par exemple Chronos, TimesFM) | grands réseaux pré-entraînés sur de nombreuses séries, utilisables sans ré-entraînement                | prévision rapide « prête à l'emploi » à évaluer sur <b>vos</b> données |
| R : forecast, fable                                 | auto.arima, ets, écosystème très complet                                                               | si votre équipe travaille en R                                         |

Quelques principes pour choisir, tirés de ce chapitre :

- **Commencez par les références simples** (naïf saisonnier avec dérive) : elles sont le seuil à battre.
- **Comparez à origine glissante**, jamais sur un seul découpage, avec **le même protocole pour tous**.
- Une bibliothèque qui « fait tout en trois lignes » **cache** les choix (nombre d'harmoniques, flexibilité de la tendance, a priori) : sachez ce qu'elle fait (c'est le sens de 4.6.2).
- À précision comparable, **préférez le modèle que vous pouvez expliquer** : celui dont vous savez dire pourquoi il prévoit ce qu'il prévoit.

#### À retenir.

- **Prophet** est un modèle additif : tendance linéaire par morceaux + saisonnalité de Fourier + événements, ajusté par une régression pénalisée avec des a priori bayésiens. On peut l'écrire à la main avec une régression ridge.
- Sur des données **mensuelles**, avec seulement 12 points par an, une saison nette demande beaucoup d'harmoniques ( $K = 6$  revient à un effet libre par mois).
- Un nouveau modèle se juge **au même concours** que les autres (origine glissante, mêmes données). Sur notre série, le modèle de type Prophet, les modèles ARIMA et leur combinaison sont **tous proches de ce que le bruit permet** : la sophistication n'améliore pas ce que les données ne contiennent pas.
- Choisissez en cascade : références simples, modèles statistiques compréhensibles, puis bibliothèques modernes ; et exigez toujours une évaluation honnête.

## 4.7 Exercices du chapitre 4

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 10 et 13 s'appuient sur des objets définis dans les sections précédentes (la fonction `kalman_niveau_local` de 4.5, la série `v` de 4.3).

### Énoncés

**Exercice 1 ★ (autocorrélation à la main).** Six chiffres d'affaires hebdomadaires (en centaines de DT) : 8, 6, 9, 5, 7, 4. Calculez à la main les autocorrélations  $r_1$  et  $r_2$ . Quel signe attendiez-vous pour  $r_1$  en regardant la série, et pourquoi ?

**Exercice 2 ★ (stationnaire ou non ?).** Pour chaque processus ( $\varepsilon_t$  est un bruit blanc), dites s'il est stationnaire, et si non, quel remède appliquer : (a)  $Y_t = 5 + \varepsilon_t$  ; (b)  $Y_t = Y_{t-1} + \varepsilon_t$  ; (c)  $Y_t = 0,9 Y_{t-1} + \varepsilon_t$  ; (d)  $Y_t = 2t + \varepsilon_t$  ; (e)  $Y_t = 1,1 Y_{t-1} + \varepsilon_t$ .

**Exercice 3 ★ (prévoir avec un AR(1)).** Un processus suit  $Y_t - 10 = 0,8(Y_{t-1} - 10) + \varepsilon_t$  avec  $\sigma = 2$ . (a) Quelle est sa variance, et les autocorrélations  $\rho(1), \rho(2), \rho(3)$  ? (b) On observe  $Y_T = 14$  : donnez les prévisions à 1, 2 et 3 pas, avec les demi-largeurs des intervalles à 95 %.

**Exercice 4 ★★ (MA(1) et inversibilité).** (a) Calculez  $\rho(1)$  pour  $\theta = 0,8$ , puis pour  $\theta = 1,25$ . Que remarquez-vous ? (b) Montrez qu'un MA(1) ne peut jamais avoir  $|\rho(1)| > 0,5$ . Si vous observez  $r_1 = 0,6$  sur une longue série, que concluez-vous ?

**Exercice 5 ★★ (Ljung-Box à la main).** Sur  $n = 50$  résidus, on trouve  $r_1 = 0,30$  et  $r_2 = 0,20$ . Calculez  $Q(2)$  et sa p-valeur (indice : pour 2 degrés de liberté,  $P(\chi_2^2 > x) = e^{-x/2}$ ). Que concluez-vous ?

**Exercice 6 ★★ (du logarithme aux dinars).** Un modèle prévoit  $\log(\text{ca}) = 7,60$  pour décembre, avec une erreur type de prévision de 0,08. Donnez la prévision **médiane** en dinars, la prévision de la **moyenne**, et l'intervalle de prévision à 95 %.

**Exercice 7 ★★ (MASE et MAPE).** Sur l'apprentissage, le naïf saisonnier a une MAE de 150 DT. Le modèle A a une MAE de 120 DT sur le test et le modèle B de 160 DT. (a) Calculez les MASE. (b) Pourquoi le MAPE est-il dangereux quand une valeur réelle est nulle ? (c) Pourquoi favorise-t-il les prévisions trop basses ?

**Exercice 8 ★★ (identifier un modèle).** Trois séries de 300 points, X, Y et Z, ont été simulées avec des modèles ARMA différents. Voici leurs autocorrélations et autocorrélations partielles. **Identifiez** pour chacune le modèle le plus probable (type et ordre), puis vérifiez avec les critères d'information.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.arima_process import ArmaProcess
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import acf, pacf

mystere = {"X": ([1, -0.75], [1]), "Y": ([1], [1, 0.7, 0.5]), "Z": ([1, -0.5, -0.3], [1])}
series = {nom: ArmaProcess(ar, ma).generate_sample(300, distrvs=np.random.default_rng(100 +
    ↪ j).standard_normal)
    for j, (nom, (ar, ma)) in enumerate(mystere.items())}
bande = 1.96 / np.sqrt(300)
print("bande de confiance : +/-", round(bande, 3))
for nom, s in series.items():
    print(f"série {nom} : ACF décalages 1-6 :", acf(s, nlags=6, fft=False)[1:].round(2))
    print(f"          PACF décalages 1-6 :", pacf(s, nlags=6, method="ols")[1:].round(2))
```

```
bande de confiance : +/- 0.113
série X : ACF décalages 1-6 : [0.73 0.58 0.43 0.28 0.2 0.12]
          PACF décalages 1-6 : [ 0.74 0.1 -0.05 -0.1 0.03 -0.03]
série Y : ACF décalages 1-6 : [ 0.6 0.17 -0.19 -0.25 -0.23 -0.17]
          PACF décalages 1-6 : [ 0.6 -0.28 -0.27 0.1 -0.12 -0.12]
série Z : ACF décalages 1-6 : [0.69 0.57 0.48 0.36 0.29 0.28]
          PACF décalages 1-6 : [ 0.69 0.19 0.08 -0.07 -0. 0.09]
```

**Exercice 9 ★★★ (la sur-différenciation, en théorie).** Soit  $Y_t$  un AR(1) stationnaire de coefficient  $\varphi$ , et  $W_t = Y_t - Y_{t-1}$ . (a) Calculez  $\text{Var}(W_t)$  et  $\text{Cov}(W_t, W_{t-1})$  en fonction de  $\gamma(0)$  et de  $\varphi$ , puis montrez que  $\rho_W(1) = -\frac{1-\varphi}{2}$ . (b) Que vaut ce résultat pour  $\varphi = 0$  ? pour  $\varphi \rightarrow 1$  ? Interprétez. (c) Vérifiez par simulation pour  $\varphi = 0,5$  et  $\varphi = 0,9$ .

**Exercice 10 ★★★ (filtre de Kalman à la main).** Niveau local de variances  $\sigma_\varepsilon^2 = 9$  (observation) et  $\sigma_\eta^2 = 3$  (niveau), croyance initiale  $\mathcal{N}(50, 12)$  avant la première mesure. On observe  $y_1 = 56$  puis  $y_2 = 52$ . (a) Calculez à la main les gains  $K_1, K_2$ , les niveaux filtrés  $a_{1|1}, a_{2|2}$  et leurs variances. (b) Quel est le gain en régime permanent ? (c) À quel paramètre de lissage exponentiel cela correspond-il ?

**Exercice 11 ★★★ (régression fallacieuse et remède).** Deux marches aléatoires indépendantes de 200 points sont régressées l'une sur l'autre. (a) Quelle part de régressions « significatives » à 5 % attendez-vous, en niveaux ? (b) Montrez par simulation que la régression **des différences**  $\Delta a_t$  sur  $\Delta b_t$  redonne le bon taux de 5 %. (c) Pourquoi ce remède fonctionne-t-il, et quelle information perd-on ?

**Exercice 12 ★★ (GARCH à la main).** Un GARCH(1,1) a pour paramètres  $\omega = 0,1, \alpha = 0,2,$

$\beta = 0,7$ . (a) Quelle est la variance de long terme ? (b) Hier,  $\varepsilon_{t-1} = 2$  et  $\sigma_{t-1}^2 = 1,5$  : que vaut  $\sigma_t^2$  ? (c) Quelle est la prévision de  $\sigma_{t+1}^2$  ? (d) En combien de pas l'excès de variance (au-dessus de la variance de long terme) est-il divisé par deux ?

**Exercice 13** ★★★ (le concours complet sur une autre série). Appliquez la démarche de 4.3 à la série nb\_commandes (nombre de commandes par mois) : travaillez en logarithme, apprenez sur 2016-2023, prévoyez 2024-2025 avec (i) le naïf saisonnier avec dérive et (ii) un SARIMAX(1,0,0)(0,1,1)<sub>12</sub> avec tendance et variables promo et covid. Calculez la MASE (en nombre de commandes) de chacun. Le modèle bat-il la référence ? Que pensez-vous de la précision atteignable sur cette série, plus bruitée que le chiffre d'affaires ?

## Corrigés

**Corrigé 1.** Moyenne =  $39/6 = 6,5$  ; écarts : 1,5; -0,5; 2,5; -1,5; 0,5; -2,5 ; somme des carrés :  $2,25 + 0,25 + 6,25 + 2,25 + 0,25 + 6,25 = 17,5$ . **Décalage 1** : produits  $(-0,5)(1,5) = -0,75$  ;  $(2,5)(-0,5) = -1,25$  ;  $(-1,5)(2,5) = -3,75$  ;  $(0,5)(-1,5) = -0,75$  ;  $(-2,5)(0,5) = -1,25$  ; somme -7,75, donc  $r_1 = -7,75/17,5 \approx -0,443$ . **Décalage 2** :  $(2,5)(1,5) = 3,75$  ;  $(-1,5)(-0,5) = 0,75$  ;  $(0,5)(2,5) = 1,25$  ;  $(-2,5)(-1,5) = 3,75$  ; somme 9,5, donc  $r_2 \approx 0,543$ . On attendait un  $r_1$  négatif : la série fait du « zigzag » (haut, bas, haut, bas) autour de sa moyenne, et un zigzag correspond à une corrélation négative entre deux mois consécutifs et positive à deux mois d'écart. Avec seulement six points, ces valeurs sont très incertaines (bande de  $\pm 0,8$ ).

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd

def acf_manuel(x, k):
    d = np.asarray(x, float) - np.mean(x)
    return (d[:len(d) - k] * d[k:]).sum() / (d ** 2).sum()

x = [8, 6, 9, 5, 7, 4]
print("r1 =", round(acf_manuel(x, 1), 4), " | r2 =", round(acf_manuel(x, 2), 4), " | bande
↳ 1,96/racine(6) =", round(1.96 / np.sqrt(6), 2))
```

```
r1 = -0.4429 | r2 = 0.5429 | bande 1,96/racine(6) = 0.8
```

**Corrigé 2.** (a) **Stationnaire** : moyenne 5, variance  $\sigma^2$ , pas de mémoire. (b) **Marche aléatoire** : non stationnaire ( $\text{Var} = t\sigma^2$ ) ; remède : **différencier**. (c) **Stationnaire** ( $|\phi| = 0,9 < 1$ ), mais très persistant : la mémoire décroît lentement ( $\rho(k) = 0,9^k$ ), et sur un échantillon court elle se confond facilement avec une racine unitaire (c'est le manque de puissance de 4.1.6). (d) **Non stationnaire** : la moyenne  $2t$  varie ; remède : **retirer la tendance** (régression sur  $t$ ), car la tendance est déterministe, pas stochastique (différencier fonctionnerait aussi, mais sur-différencierait). (e) **Explosif** ( $|\phi| > 1$ ) : non stationnaire, aucune transformation simple ne le « répare » ; c'est rare en pratique.

```
rng = np.random.default_rng(3)
n = 400
e = rng.normal(size=n)
def generer(phi, tendance=0.0, depart=0.0):
    y = np.zeros(n); y[0] = depart
    for t in range(1, n):
        y[t] = phi * y[t - 1] + e[t] + tendance * t
    return y
cas = {"(a) 5 + bruit": 5 + e, "(b) marche aléatoire": generer(1.0), "(c) AR(1), phi = 0,9":
↳ generer(0.9),
      "(d) 2t + bruit": 2 * np.arange(n) + e}
lignes = [{"processus": k, "moyenne 1re moitié": s[:n // 2].mean(), "moyenne 2e moitié": s[n //
↳ 2:].mean(),
```

```
"écart-type 1re moitié": s[:n // 2].std(), "écart-type 2e moitié": s[n // 2:].std()] for k,
    ↪ s in cas.items())
print(pd.DataFrame(lignes).round(1).to_string(index=False))
```

|     | processus        | moyenne 1re moitié | moyenne 2e moitié | écart-type 1re moitié | écart-type 2e moitié |
|-----|------------------|--------------------|-------------------|-----------------------|----------------------|
| (a) | 5 + bruit        | 5.0                | 5.0               | 1.0                   | 1.0                  |
| (b) | marche aléatoire | -2.8               | 7.4               | 4.3                   | 4.1                  |
| (c) | AR(1), phi = 0,9 | 0.3                | 0.2               | 2.2                   | 2.3                  |
| (d) | 2t + bruit       | 199.0              | 599.0             | 115.6                 | 115.5                |

La simulation montre que les moyennes diffèrent nettement d'une moitié à l'autre pour (b) et (d), alors que (a) et (c) se ressemblent. (Pour la marche aléatoire (b), c'est la variance **à travers les trajectoires** qui croît avec  $t$  (4.1.3) ; sur une seule trajectoire, comparer les écarts-types de deux moitiés n'est pas probant, et le tableau ne le montre d'ailleurs pas. C'est le déplacement de la moyenne qui trahit la non-stationnarité.)

**Corrigé 3.** (a)  $\gamma(0) = \frac{\sigma^2}{1 - \varphi^2} = \frac{4}{1 - 0,64} = 11,11$  (écart-type 3,33) ;  $\rho(1) = 0,8$ ,  $\rho(2) = 0,64$ ,  $\rho(3) = 0,512$ . (b)  $\hat{y}_{T+h} = 10 + 0,8^h \times (14 - 10) : 13,2 ; 12,56 ; 12,048$ . Demi-largeurs :  $1,96 \sigma \sqrt{\sum_{j < h} \varphi^{2j}}$  :  $h = 1 : 1,96 \times 2 = 3,92$  ;  $h = 2 : 3,92 \times \sqrt{1,64} = 5,02$  ;  $h = 3 : 3,92 \times \sqrt{1,64 + 0,4096} = 5,61$ . Limite :  $1,96 \times 3,33 = 6,53$ .

```
from statsmodels.tsa.arima.model import ARIMA
phi, mu, sigma, yT = 0.8, 10.0, 2.0, 14.0
print("variance :", round(sigma ** 2 / (1 - phi ** 2), 2), "| rho(1..3) :", [round(phi ** k, 3) for k
    ↪ in (1, 2, 3)])
print("prévisions :", [round(mu + phi ** h * (yT - mu), 3) for h in (1, 2, 3)])
print("demi-largeurs :", np.round([1.96 * sigma * np.sqrt(sum(phi ** (2 * j) for j in range(h))) for h
    ↪ in (1, 2, 3)], 2),
    "| limite :", round(1.96 * sigma / np.sqrt(1 - phi ** 2), 2))
# vérification avec statsmodels (paramètres fixés : constante, phi, sigma2)
fixe = ARIMA(np.r_[np.full(50, mu), yT], order=(1, 0, 0), trend="c").filter([mu, phi, sigma ** 2])
p = fixe.get_forecast(3)
print("statsmodels :", p.predicted_mean.round(3), ((p.conf_int()[ :, 1] - p.conf_int()[ :, 0]) /
    ↪ 2).round(2))
```

```
variance : 11.11 | rho(1..3) : [0.8, 0.64, 0.512]
prévisions : [13.2, 12.56, 12.048]
demi-largeurs : [3.92 5.02 5.61] | limite : 6.53
statsmodels : [13.2 12.56 12.048] [3.92 5.02 5.61]
```

**Corrigé 4.** (a)  $\rho(1) = \frac{0,8}{1 + 0,64} = 0,488$  et  $\rho(1) = \frac{1,25}{1 + 1,5625} = 0,488$  : les deux valeurs donnent la même autocorrélation ( $\theta$  et  $1/\theta$ ). On retient la valeur inversible  $\theta = 0,8$ . (b)  $1 + \theta^2 \geq 2|\theta|$  (car  $(1 - |\theta|)^2 \geq 0$ ), donc  $|\rho(1)| = \frac{|\theta|}{1 + \theta^2} \leq \frac{1}{2}$ , avec égalité pour  $\theta = \pm 1$ . Si l'on observe  $r_1 = 0,6$  sur une longue série (assez longue pour que 0,6 ne soit pas du bruit d'échantillonnage), **ce n'est pas un MA(1)** : il faut un AR (dont la mémoire peut produire une autocorrélation d'ordre 1 proche de 1), ou un ARMA.

```
from statsmodels.tsa.arima_process import arma_acf
for th in (0.8, 1.25):
    print(f"theta = {th} : rho(1) = {arma_acf([1], [1, th], 2)[1] : .4f}")
thetas = np.linspace(-5, 5, 100001)
rho1 = thetas / (1 + thetas ** 2)
print("maximum de |rho(1)| sur theta dans [-5, 5] :", round(np.abs(rho1).max(), 4), "atteint pour
    ↪ theta =", round(abs(thetas[np.abs(rho1).argmax()]), 3))
```



```
theta = 0.8 : rho(1) = 0.4878
theta = 1.25 : rho(1) = 0.4878
maximum de |rho(1)| sur theta dans [-5, 5] : 0.5 atteint pour theta = 1.0
```

**Corrigé 5.**  $Q(2) = n(n+2) \left[ \frac{r_1^2}{n-1} + \frac{r_2^2}{n-2} \right] = 50 \times 52 \times \left[ \frac{0,09}{49} + \frac{0,04}{48} \right] = 2600 \times (0,001837 + 0,000833) = 6,94$ . Pour 2 degrés de liberté,  $p = e^{-6,94/2} = e^{-3,47} \approx 0,031$ . **p < 0,05** : on rejette l'hypothèse « bruit blanc » ; le modèle n'a pas tout expliqué. (Si les résidus provenaient d'un modèle avec deux paramètres AR/MA estimés, il faudrait comparer à un khi-deux à  $2 - 2 = 0$  degré de liberté : le test serait inutilisable à 2 retards ; on utiliserait plus de retards.)

```
from scipy import stats
n, r = 50, np.array([0.30, 0.20])
Q = n * (n + 2) * np.sum(r ** 2 / (n - np.arange(1, 3)))
print("Q(2) =", round(Q, 3), "| p-valeur =", round(np.exp(-Q / 2), 4), "| via scipy :", round(1 -
↪ stats.chi2.cdf(Q, 2), 4))
```

Q(2) = 6.942 | p-valeur = 0.0311 | via scipy : 0.0311

**Corrigé 6.** La **médiane** est  $e^{7,60} \approx 1998$  DT. La **moyenne** vaut  $e^{7,60+0,08^2/2} = e^{7,6032} \approx 2005$  DT (un facteur  $e^{0,0032} \approx 1,003$  : négligeable). L'intervalle à 95 % :  $\exp(7,60 \pm 1,96 \times 0,08) = \exp(7,60 \pm 0,1568) = [1708; 2337]$  DT : environ  $\pm 16\%$  autour de la médiane, de façon **asymétrique** en dinars (l'intervalle est un peu plus étendu vers le haut : +339 DT contre -290 DT).

```
m, s = 7.60, 0.08
print("médiane :", round(np.exp(m), 1), "| moyenne :", round(np.exp(m + s ** 2 / 2), 1))
print("IC95 % :", round(np.exp(m - 1.96 * s), 1), "à", round(np.exp(m + 1.96 * s), 1),
      "| écarts à la médiane :", round(np.exp(m - 1.96 * s) - np.exp(m), 1), "et +", round(np.exp(m +
↪ 1.96 * s) - np.exp(m), 1))
```

médiane : 1998.2 | moyenne : 2004.6  
IC95 % : 1708.2 à 2337.4 | écarts à la médiane : -290.0 et + 339.2

**Corrigé 7.** (a)  $MASE_A = 120/150 = 0,80$  (le modèle fait **20 % mieux** que le naïf saisonnier de l'apprentissage) ;  $MASE_B = 160/150 \approx 1,07$  (**moins bien** que le naïf). (b) Le MAPE divise par  $|y_t|$  : si  $y_t = 0$  (un mois sans vente), le terme est indéfini (division par zéro), et pour des valeurs réelles très petites il explose. (c) Une sous-prévision ne peut pas dépasser **100 %** d'erreur (prévoir 0 pour une valeur de 100 donne 100 %), alors qu'une surestimation est **illimitée** (prévoir 300 pour 100 donne 200 %). En minimisant le MAPE, un modèle est donc incité à prévoir **trop bas**.

```
actuel = 100
for prevu in (0, 50, 150, 300):
    print(f"réel = {actuel}, prévu = {prevu:3d} -> erreur en % du réel = {100 * abs(actuel - prevu) /
↪ actuel:.0f} %")
print("MASE A =", 120 / 150, "| MASE B =", round(160 / 150, 3))
```

```
réel = 100, prévu = 0 -> erreur en % du réel = 100 %
réel = 100, prévu = 50 -> erreur en % du réel = 50 %
réel = 100, prévu = 150 -> erreur en % du réel = 50 %
réel = 100, prévu = 300 -> erreur en % du réel = 200 %
MASE A = 0.8 | MASE B = 1.067
```

**Corrigé 8. Règle** (4.2.1) :  $AR(p) \Rightarrow PACF$  qui s'arrête après  $p$ ,  $ACF$  qui décroît ;  $MA(q) \Rightarrow ACF$  qui s'arrête après  $q$ ,  $PACF$  qui décroît. Lecture des sorties de l'énoncé (bande de  $\pm 0,113$ ) :

- X :  $ACF$  qui décroît lentement (0,73 ; 0,58 ; 0,43 ; 0,28...),  $PACF$  avec **un seul** pic net, au décalage 1 (0,74), puis des valeurs dans la bande : un **AR(1)**, de coefficient voisin de 0,74.
- Z :  $ACF$  qui décroît,  $PACF$  de 0,69 puis 0,19 (au-dessus de la bande), puis dans la bande : un **AR(2)** est plausible (un **AR(1)** est possible : le deuxième pic est faible).



- Y : PACF qui décroît avec alternance de signes (0,60 ; -0,28 ; -0,27 ; puis dans la bande) : une signature de **MA** ; mais l'ACF (0,60 ; 0,17 ; puis -0,19, -0,25, -0,23 au-delà de la bande) ne s'arrête **pas** proprement après le décalage 2. Cette série est **ambiguë** à l'œil : un MA(2), mais peut-être un MA plus long ou un ARMA.

Voilà la réalité de l'identification : avec 300 points, les autocorrélations estimées fluctuent d'environ  $\pm 0,11$ , et les signatures théoriques de la figure de 4.2.1 se brouillent. D'où le rôle des critères d'information, qui comparent les modèles candidats **entre eux** :

```
candidats = {"AR(1)": (1, 0, 0), "AR(2)": (2, 0, 0), "MA(1)": (0, 0, 1), "MA(2)": (0, 0, 2),
             ↪ "ARMA(1,1)": (1, 0, 1)}
ajustements = {nom: {c: ARIMA(s, order=o, trend="n").fit() for c, o in candidats.items()} for nom, s
               ↪ in series.items()}
aic = pd.DataFrame({nom: {c: m.aic for c, m in d.items()} for nom, d in ajustements.items()})
bic = pd.DataFrame({nom: {c: m.bic for c, m in d.items()} for nom, d in ajustements.items()})
print("AIC par modèle (une colonne par série) :")
print(aic.round(1).to_string())
print("\nBIC par modèle :")
print(bic.round(1).to_string())
print("\nmeilleur modèle par AIC :", aic.idxmin().to_dict())
print("meilleur modèle par BIC :", bic.idxmin().to_dict())
print("vérité : X = AR(1), phi = 0,75 | Y = MA(2), theta = (0,7 ; 0,5) | Z = AR(2), phi = (0,5 ; 0,3)")
```

AIC par modèle (une colonne par série) :

|           | X     | Y     | Z     |
|-----------|-------|-------|-------|
| AR(1)     | 850.5 | 906.2 | 882.7 |
| AR(2)     | 849.5 | 883.7 | 873.3 |
| MA(1)     | 944.1 | 929.2 | 964.5 |
| MA(2)     | 897.8 | 864.3 | 930.8 |
| ARMA(1,1) | 850.0 | 896.0 | 871.9 |

BIC par modèle :

|           | X     | Y     | Z     |
|-----------|-------|-------|-------|
| AR(1)     | 858.0 | 913.6 | 890.2 |
| AR(2)     | 860.6 | 894.8 | 884.4 |
| MA(1)     | 951.5 | 936.6 | 971.9 |
| MA(2)     | 908.9 | 875.4 | 941.9 |
| ARMA(1,1) | 861.1 | 907.1 | 883.0 |

meilleur modèle par AIC : {'X': 'AR(2)', 'Y': 'MA(2)', 'Z': 'ARMA(1,1)'}

meilleur modèle par BIC : {'X': 'AR(1)', 'Y': 'MA(2)', 'Z': 'ARMA(1,1)'}

vérité : X = AR(1), phi = 0,75 | Y = MA(2), theta = (0,7 ; 0,5) | Z = AR(2), phi = (0,5 ; 0,3)

**Lecture des critères.** Pour Y, le MA(2) l'emporte nettement, par l'AIC comme par le BIC : le critère résout l'ambiguïté que l'œil ne résolvait pas. Pour X et Z, ce sont des modèles **voisins** qui se disputent la première place avec un ou deux points d'écart : un AR(2) dont le second coefficient est petit ressemble à un AR(1), et un ARMA(1,1) imite un AR(2). **Un écart d'AIC de l'ordre de 1 à 2 ne permet pas de trancher** (la règle empirique est qu'il faut un écart de plus de 2 à 4 points pour préférer un modèle). Le BIC, plus sévère avec la complexité, retrouve la vérité pour X (AR(1)) et pour Y (MA(2)), mais préfère encore l'ARMA(1,1) à l'AR(2) pour Z, avec 1,4 point d'écart (883,0 contre 884,4) : pour ces deux modèles voisins, les données n'ont pas de quoi décider. En pratique, on retient alors le modèle **le plus simple** parmi les ex æquo, et on valide par les résidus et par la prévision.

**Corrigé 9.** (a) Soit  $\gamma(k) = \phi^k \gamma(0)$ .  $\text{Var}(W_t) = 2\gamma(0) - 2\gamma(1) = 2\gamma(0)(1 - \phi)$ .  $\text{Cov}(W_t, W_{t-1}) = \text{Cov}(Y_t - Y_{t-1}, Y_{t-1} - Y_{t-2}) = \gamma(1) - \gamma(2) - \gamma(0) + \gamma(1) = \gamma(0)(2\phi - \phi^2 - 1) = -\gamma(0)(1 - \phi)^2$ .

Donc  $\rho_W(1) = \frac{-\gamma(0)(1 - \phi)^2}{2\gamma(0)(1 - \phi)} = -\frac{1 - \phi}{2}$ . (b) Pour  $\phi = 0$  (Y est un bruit blanc), on retrouve -0,5 (4.1.6). Pour  $\phi \rightarrow 1$  (quasi-marche aléatoire),  $\rho_W(1) \rightarrow 0$  : la différence d'une marche aléatoire est un bruit blanc : **la différenciation est alors le bon remède**. Entre les deux, plus la série est « loin

» d'une racine unitaire, plus la différenciation fabrique une autocorrélation négative artificielle.  
(c) Simulation :

```
from statsmodels.tsa.arima_process import ArmaProcess
for phi in (0.0, 0.5, 0.9):
    x = ArmaProcess([1, -phi], [1]).generate_sample(300000,
    ↳ distrvs=np.random.default_rng(7).standard_normal)
    print(f"phi = {phi} : rho_W(1) simulé = {acf_manuel(np.diff(x), 1):+.3f} | théorie -(1-phi)/2 =
    ↳ {-(1 - phi) / 2:+.3f}")

phi = 0.0 : rho_W(1) simulé = -0.502 | théorie -(1-phi)/2 = -0.500
phi = 0.5 : rho_W(1) simulé = -0.252 | théorie -(1-phi)/2 = -0.250
phi = 0.9 : rho_W(1) simulé = -0.052 | théorie -(1-phi)/2 = -0.050
```

**Corrigé 10.** (a) **Date 1.**  $P_{1|0} = 12$ ,  $F_1 = 12 + 9 = 21$ ,  $K_1 = 12/21 = 0,5714$ ,  $v_1 = 56 - 50 = 6$ ,  
 $a_{1|1} = 50 + 0,5714 \times 6 = 53,43$ ,  $P_{1|1} = 12 \times \frac{9}{21} = 5,143$ . **Date 2.**  $P_{2|1} = 5,143 + 3 = 8,143$ ,  $F_2 = 17,143$ ,  
 $K_2 = 8,143/17,143 = 0,475$ ,  $v_2 = 52 - 53,43 = -1,43$ ,  $a_{2|2} = 53,43 + 0,475 \times (-1,43) = 52,75$ ,  $P_{2|2} =$   
 $8,143 \times \frac{9}{17,143} = 4,275$ . (b) En régime permanent, la variance prédite  $p$  vérifie  $p = \frac{p\sigma_\varepsilon^2}{p + \sigma_\varepsilon^2} + \sigma_\eta^2$ ,  
soit  $p^2 = \sigma_\eta^2 p + \sigma_\eta^2 \sigma_\varepsilon^2$ , donc  $p = \frac{3 + \sqrt{9 + 4 \times 27}}{2} = \frac{3 + \sqrt{117}}{2} \approx 6,908$  et  $K_\infty = \frac{p}{p + 9} \approx 0,434$ . (c)  
Le lissage exponentiel simple de paramètre  $\alpha = K_\infty \approx 0,43$ .

```
r = kalman_niveau_local(np.array([56.0, 52.0]), s2_eps=9.0, s2_eta=3.0, a0=50.0, P0=12.0) #
↳ fonction définie en 4.5.3
print(pd.DataFrame({k: r[k] for k in ["P_pred", "F", "K", "v", "a_filtre", "P_filtre"]}, index=[1,
↳ 2])).round(4).to_string())
p_inf = (3 + np.sqrt(9 + 4 * 27)) / 2
print("régime permanent : p =", round(p_inf, 4), "| K_inf =", round(p_inf / (p_inf + 9), 4))
longue = kalman_niveau_local(np.random.default_rng(1).normal(100, 3, 300), 9.0, 3.0, 100.0, 12.0)
print("gain après 300 observations :", round(longue["K"][-1], 4))
```

|   | P_pred  | F       | K      | v       | a_filtre | P_filtre |
|---|---------|---------|--------|---------|----------|----------|
| 1 | 12.0000 | 21.0000 | 0.5714 | 6.0000  | 53.4286  | 5.1429   |
| 2 | 8.1429  | 17.1429 | 0.4750 | -1.4286 | 52.7500  | 4.2750   |

régime permanent : p = 6.9083 | K\_inf = 0.4343  
gain après 300 observations : 0.4343

**Corrigé 11.** (a) En niveaux, la pente est « significative » dans **plus des trois quarts** des cas (77 % avec 100 points en 4.4.2 ; plus encore avec 200 points, voir la sortie : plus les séries sont longues, plus l'illusion est fréquente), pas 5 %. (b) Sur les différences (qui sont des bruits blancs indépendants), le taux retombe autour de 5 % (la sortie donne une valeur proche, à l'incertitude de la simulation près). (c) La différenciation rend les séries **stationnaires** : les hypothèses du chapitre 1 s'appliquent de nouveau, donc les p-valeurs sont justes. On **perd l'information de long terme** : si les séries sont cointégrées (4.4.2), une régression sur les différences ignore la relation d'équilibre ; il faut alors le modèle à correction d'erreur.

```
import statsmodels.api as sm
rng = np.random.default_rng(55)
niveaux, differences = [], []
for _ in range(2000):
    a = np.cumsum(rng.normal(size=200))
    b = np.cumsum(rng.normal(size=200))
    niveaux.append(abs(sm.OLS(a, sm.add_constant(b)).fit().tvalues[1]) > 1.96)
    differences.append(abs(sm.OLS(np.diff(a), sm.add_constant(np.diff(b))).fit().tvalues[1]) > 1.96)
print("part de pentes « significatives » à 5 % : en niveaux =", np.mean(niveaux).round(3), "| sur les
↳ différences =", np.mean(differences).round(3))
```

part de pentes « significatives » à 5 % : en niveaux = 0.824 | sur les différences = 0.052

**Corrigé 12.** (a)  $\sigma^2 = \frac{0,1}{1 - 0,2 - 0,7} = 1$ . (b)  $\sigma_t^2 = 0,1 + 0,2 \times 2^2 + 0,7 \times 1,5 = 0,1 + 0,8 + 1,05 = 1,95$ .  
 (c)  $\mathbb{E}[\sigma_{t+1}^2] = \omega + (\alpha + \beta)\sigma_t^2 = 0,1 + 0,9 \times 1,95 = 1,855$  (car  $\mathbb{E}[\varepsilon_t^2 | \text{passé}] = \sigma_t^2$ ). (d) L'excès  $\sigma_{t+h}^2 - 1$  est multiplié par  $\alpha + \beta = 0,9$  à chaque pas : il est divisé par deux quand  $0,9^h = 0,5$ , soit  $h = \ln 0,5 / \ln 0,9 \approx 6,6$  : **environ 7 jours**.

```
omega, alpha, beta = 0.1, 0.2, 0.7
var_lt = omega / (1 - alpha - beta)
sig2_t = omega + alpha * 2 ** 2 + beta * 1.5
prev = [omega + (alpha + beta) * sig2_t]
for _ in range(19):
    prev.append(omega + (alpha + beta) * prev[-1])
print("variance de long terme :", round(var_lt, 4), "| sigma_t^2 =", round(sig2_t, 3), "| prévision de
↳ sigma_{t+1}^2 =", round(prev[0], 3))
print("excès sur la variance de long terme aux pas 1, 5, 10, 20 :", np.round(np.array(prev)[[0, 4, 9,
↳ 19]] - var_lt, 3))
print("demi-vie de l'excès :", round(np.log(0.5) / np.log(alpha + beta), 2), "pas")
```

```
variance de long terme : 1.0 | sigma_t^2 = 1.95 | prévision de sigma_{t+1}^2 = 1.855
excès sur la variance de long terme aux pas 1, 5, 10, 20 : [0.855 0.561 0.331 0.115]
demi-vie de l'excès : 6.58 pas
```

**Corrigé 13.** On applique la démarche de 4.3 à  $\log(\text{nb\_commandes})$  : même découpage, un naïf saisonnier avec dérive, un SARIMAX, et la MASE en **nombre de commandes** (échelle : MAE du naïf saisonnier sur l'apprentissage).

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
z = np.log(v["nb_commandes"])
train_z, test_z = z[:"2023-12"], z["2024-01":]
Xe = v[["promo", "covid"]].astype(float)

g = (train_z - train_z.shift(12)).mean()
naif = pd.Series([train_z.iloc[-12 + (h % 12)] + g * (1 + h // 12) for h in range(24)],
↳ index=test_z.index)
modele = SARIMAX(train_z, exog=Xe[:"2023-12"], order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
↳ trend="ct").fit(dispatch=False, maxiter=300)
prevu = pd.Series(modele.get_forecast(24, exog=Xe["2024-01:"]).predicted_mean.values,
↳ index=test_z.index)

nb_train = np.exp(train_z)
echelle = np.mean(np.abs(nb_train.values[12:] - nb_train.values[:-12]))
lignes = {}
for nom, f in [("naïf saisonnier + dérive", naif), ("SARIMAX(1,0,0)(0,1,1)12 + tendance", prevu)]:
    mae = np.mean(np.abs(np.exp(test_z) - np.exp(f)))
    lignes[nom] = {"MAE (commandes)": mae, "MASE": mae / echelle, "RMSE (log)":
↳ np.sqrt(np.mean((test_z - f) ** 2)), "biais (log)": float(np.mean(test_z - f))}
print("MAE du naïf saisonnier sur l'apprentissage :", round(echelle, 2), "commandes")
print(pd.DataFrame(lignes).T.round(3).to_string())
print("écart-type des résidus du SARIMAX (log) :", round(float(np.sqrt(modele.params['sigma2'])), 3),
↳ "| à comparer à celui du chiffre d'affaires (4.2) : environ 0,06 à 0,07")
```

MAE du naïf saisonnier sur l'apprentissage : 7.92 commandes

|                                    | MAE (commandes) | MASE  | RMSE (log) | biais (log) |
|------------------------------------|-----------------|-------|------------|-------------|
| naïf saisonnier + dérive           | 7.786           | 0.983 | 0.273      | -0.139      |
| SARIMAX(1,0,0)(0,1,1)12 + tendance | 6.146           | 0.776 | 0.216      | 0.023       |

écart-type des résidus du SARIMAX (log) : 0.262 | à comparer à celui du chiffre d'affaires (4.2) :  
 ↳ environ 0,06 à 0,07

Le résultat se lit en trois temps. (1) Le **bruit** de cette série est nettement plus grand que celui du chiffre d'affaires : un nombre de commandes est un **comptage** (de loi de Poisson, de variance égale à la moyenne), donc son erreur relative, de l'ordre de  $1/\sqrt{\text{moyenne}}$  (environ 19 % pour 28 commandes par mois), est irréductible. L'écart-type des résidus du SARIMAX est d'environ 0,26 en logarithme, près de quatre fois celui du chiffre d'affaires. (2) Le SARIMAX fait **mieux que la référence simple** (MASE de 0,78 contre 0,98, soit environ 20 % d'erreur de moins ; sa MAE est de 6,1 commandes par mois), mais la référence est déjà **à peine meilleure que le naïf saisonnier de l'apprentissage** (MASE proche de 1) : à ce niveau de bruit, le gain d'un bon modèle se mesure en dizaines de pour cent d'une erreur de toute façon élevée, pas en précision. (3) La bonne conclusion est donc moins « quel modèle ? » que « quelle précision peut-on espérer ? » : une prévision de commandes mensuelles doit s'accompagner d'une **fourchette large**.

## Bilan du chapitre 4

Vous savez maintenant :

- **décrire** une série temporelle (tendance, saison, bruit, accident) et la **décomposer** (classique, STL robuste), en travaillant sur le **logarithme** quand la saison est proportionnelle au niveau ;
- définir la **stationnarité**, mesurer la mémoire par l'**autocorrélation** et l'**autocorrélation partielle**, tester le bruit blanc (**Ljung-Box**) et la racine unitaire (**ADF**, **KPSS**) en connaissant leurs limites (peu de puissance, loi de Dickey-Fuller non normale) ;
- distinguer **tendance déterministe** et **tendance stochastique**, et éviter la **sur-différenciation** ;
- construire des modèles **AR**, **MA**, **ARMA**, **ARIMA**, **SARIMA** et **SARIMAX**, connaître leurs conditions de stationnarité et d'inversibilité, suivre la méthode de **Box-Jenkins** (identification, estimation, diagnostic, prévision) et lire un diagnostic de résidus ;
- **prévoir** avec ses intervalles, passer du logarithme aux dinars, et **évaluer honnêtement** : découpage temporel, références simples, MAE, RMSE, MAPE, **MASE**, bootstrap par blocs, **validation à origine glissante**, combinaison de prévisions ;
- comprendre qu'un seul jeu de test est **bruité** : le meilleur modèle en espérance ne gagne pas toujours (4.3.8) ;
- (en option) modéliser **plusieurs séries** (VAR, causalité de Granger, **cointégration**, régression fallacieuse), la **volatilité** (GARCH), les **modèles d'espace d'états** et le **filtre de Kalman** (lissage exponentiel, données manquantes), et situer **Prophet** et les bibliothèques modernes.

Le chapitre 5 change d'univers : il ne s'agit plus d'une série qui évolue **dans le temps calendaire**, mais d'un événement dont on mesure **la durée d'attente**, avec le défi particulier de la **censure** : combien de temps un client reste-t-il fidèle, quand certains sont encore clients aujourd'hui ?

# Chapitre 5 : Analyse de survie

« La question n'est pas seulement *si* l'événement arrivera, mais *quand*, et ce que l'on peut dire des personnes qu'on n'a pas encore vues partir. »

Yasmine a une inquiétude que tous les commerçants connaissent : **combien de temps un client reste-t-il fidèle ?** Elle sait que 49 % de ses 2 000 clients ont cessé d'acheter à la fin de 2025. Mais les autres ? Certains sont arrivés en 2019 et sont toujours là ; d'autres se sont inscrits il y a trois mois et n'ont tout simplement pas eu le temps de partir. Peut-on dire qu'ils « ne partiront jamais » ? Bien sûr que non : on ne sait pas encore. C'est un problème très particulier : **une partie de l'information est incomplète, et pourtant elle n'est pas sans valeur**, puisque savoir qu'un client est resté au moins 40 mois est un renseignement précieux.

L'**analyse de survie** (on dit aussi *analyse des durées*) est la branche de la statistique qui traite ce problème. Son nom vient de la médecine (durée de survie d'un patient), mais ses applications dépassent largement l'hôpital : durée de vie d'un client, d'un abonnement, d'une machine, délai avant un remboursement anticipé de crédit, durée d'un chômage, temps avant le premier sinistre d'une assurance...

**Ce que ce chapitre suppose.** Les chapitres 1 à 3 du volume I (surtout : lois de probabilité et loi exponentielle, estimation par maximum de vraisemblance, intervalles de confiance, tests, bootstrap) et le chapitre 2 de ce volume (modèles linéaires généralisés) pour l'idée de « modèle de régression avec une fonction de lien ». Aucune connaissance préalable en analyse de survie n'est nécessaire.

## Le chemin de ce chapitre

- **5.1 Censure et fonctions de survie** : pourquoi la moyenne ne marche pas, ce qu'est la censure, et les trois fonctions (survie, risque instantané, risque cumulé) qui décrivent une durée.
- **5.2 L'estimateur de Kaplan-Meier** : estimer la courbe de survie **sans hypothèse de forme**, avec son intervalle de confiance, et comparer des groupes (test du log-rank).
- **5.3 Le modèle de Cox à risques proportionnels** : une régression pour les durées, le modèle le plus utilisé de la discipline. Comment l'ajuster, l'interpréter, le vérifier.
- **5.4 Modèles de durée paramétriques** : exponentiel, Weibull, log-normal ; extrapoler au-delà des données et calculer la **valeur vie client**.
- **⊕ 5.5 Pour aller plus loin : les risques concurrents** : quand plusieurs événements peuvent interrompre la durée et s'excluent mutuellement.
- **5.6 Exercices corrigés et bilan.**

## Les données de ce chapitre

Nous utilisons le fichier `donnees/clients.csv`, déjà présenté en début de volume : 2 000 clients de Dar Jasmin inscrits entre janvier 2019 et juin 2025, observés jusqu'au **31 décembre 2025**. Quatre colonnes comptent ici :

| Colonne                             | Signification                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>duree_mois</code>             | durée pendant laquelle le client a été <b>observé</b> (en mois)                                                               |
| <code>churn</code>                  | <b>1</b> si le départ a été observé, <b>0</b> si le client était encore là à la fin de l'observation (durée <i>censurée</i> ) |
| <code>offre_bienvenue</code>        | 1 si le client a reçu une offre de bienvenue, 0 sinon, <b>attribuée au hasard</b>                                             |
| <code>canal_acquisition, age</code> | canal par lequel le client est arrivé, âge à l'inscription                                                                    |

**Des données simulées.** Comme dans tout le volume, ces données sont **simulées** avec une graine fixe : nous connaissons donc la loi qui les a engendrées. Nous ne la révélerons qu'à la fin du chapitre (section 5.4.7), pour pouvoir vérifier ce que les méthodes retrouvent, et ce qu'elles retrouvent mal. Faites comme si Yasmine ne la connaissait pas.

**Les outils.** Nous écrivons à la main les estimateurs importants (Kaplan-Meier, log-rank, vraisemblance de Cox, maximum de vraisemblance paramétrique) pour comprendre ce qu'ils calculent, puis nous les comparons aux bibliothèques : `statsmodels` (`SurvfuncRight`, `survdiff`, `PHReg`), `lifelines`, et le paquet R `survival`, qui est la référence de la discipline. Chaque fois qu'une comparaison est faite, elle est **exécutée**.

## 5.1 Censure et fonctions de survie

**Intuition.** Une durée est un nombre comme un autre, sauf sur un point : **on n'a pas toujours le temps d'attendre la fin**. Quand l'étude s'arrête, certains clients sont encore là. Pour eux, on ne connaît pas la durée complète, mais on sait quelque chose de précieux : *elle dépasse celle observée*. Toute l'analyse de survie consiste à **utiliser cette information partielle sans la déformer**.

### 5.1.1 Un problème que la moyenne ne sait pas résoudre

Commençons petit, avec huit clients que Yasmine a suivis dans son cahier. Pour chacun, elle a noté le nombre de mois pendant lesquels elle l'a observé, et si elle l'a vu **partir** (il n'a plus jamais commandé) ou s'il était **encore là** quand elle a fermé son cahier.

| Client | Mois observés | Situation                    |
|--------|---------------|------------------------------|
| A      | 3             | parti                        |
| B      | 5             | parti                        |
| C      | 6             | encore là ( <b>censuré</b> ) |
| D      | 8             | parti                        |
| E      | 10            | encore là ( <b>censuré</b> ) |
| F      | 12            | parti                        |

| Client | Mois observés | Situation                    |
|--------|---------------|------------------------------|
| G      | 14            | encore là ( <b>censuré</b> ) |
| H      | 14            | encore là ( <b>censuré</b> ) |

Quelle est la durée de fidélité d'un client « typique » ? Trois réponses viennent naturellement, et **les trois sont fausses**.

```
import numpy as np
import pandas as pd

clients8 = pd.DataFrame({
    "client": list("ABCDEFGH"),
    "mois": [3, 5, 6, 8, 10, 12, 14, 14],
    "parti": [1, 1, 0, 1, 0, 1, 0, 0], # 1 = départ observé, 0 = censuré
})
print("(1) moyenne de toutes les durées observées :", clients8["mois"].mean())
print("(2) moyenne des seuls clients partis      :", clients8.loc[clients8["parti"] == 1,
    ↪ "mois"].mean())
print("(3) clients partis / clients observés      :", clients8["parti"].mean())
```

```
(1) moyenne de toutes les durées observées : 9.0
(2) moyenne des seuls clients partis      : 7.0
(3) clients partis / clients observés      : 0.5
```

- **(1) Tout moyenner.** C'est traiter C, E, G et H comme s'ils étaient partis à l'instant où l'on a cessé de les regarder. Or ils sont toujours là : leur vraie durée est **plus longue** que celle notée. La moyenne de 9 mois est donc **trop faible**.
- **(2) Ne garder que les clients partis.** On jette les censurés, comme s'ils n'avaient pas existé. Mais on jette précisément ceux qui **restent le plus longtemps**. On ne garde que les départs précoces : la moyenne de 7 mois est **encore plus faible**.
- **(3) Compter la proportion de départs.** La moitié des clients sont partis, mais cette proportion dépend uniquement de **la durée pendant laquelle on a regardé**. Avec un cahier fermé plus tard, la proportion serait plus grande ; elle ne mesure aucune propriété du client.

Le dessin rend la situation limpide. Chaque ligne est un client ; le trait s'arrête quand on cesse de l'observer ; le point plein marque un départ, le cercle vide un client **encore présent**, dont l'histoire continue hors du cadre.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
fig, ax = plt.subplots(figsize=(7.2, 3.4))
for i, (nom, m, p) in enumerate(zip(clients8["client"], clients8["mois"], clients8["parti"])):
    y = len(clients8) - i
    ax.plot([0, m], [y, y], color="#898781", lw=1.6, zorder=1)
    if p == 1:
        ax.scatter(m, y, s=70, color=ORANGE, zorder=3)
    else:
        ax.scatter(m, y, s=70, facecolor="white", edgecolor=BLEU, linewidth=2, zorder=3)
        ax.annotate("", xy=(m + 1.6, y), xytext=(m + 0.3, y), arrowprops=dict(arrowstyle="->",
    ↪ color=BLEU, lw=1.4))
ax.set_yticks(range(1, 9))
ax.set_yticklabels(list("HGFEDCBA"))
ax.set_xlabel("mois depuis l'inscription")
ax.set_xlim(0, 17)
ax.text(1.0, 8.9, "● départ observé", color=ORANGE, fontsize=9, va="bottom")
ax.text(6.0, 8.9, "○ encore client à la fermeture du cahier : durée censurée", color=BLEU, fontsize=9,
    ↪ va="bottom")
```



```
ax.set_ylim(0.3, 9.6)
ax.grid(axis="y", visible=False)
plt.tight_layout()
plt.savefig("figures/ch05-clients-censures.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

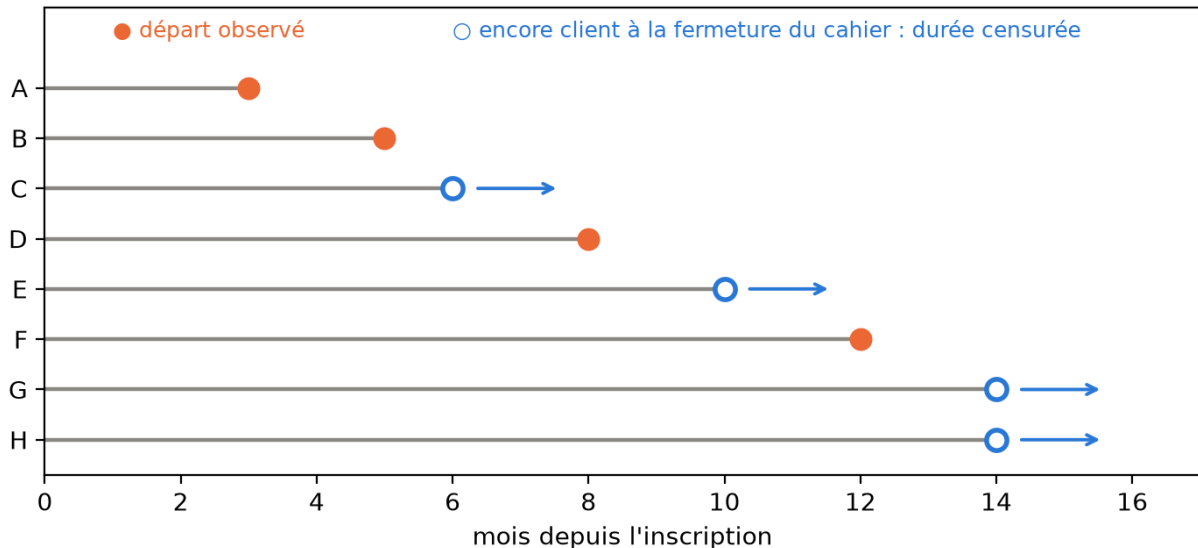


Figure 52: Huit clients suivis : un trait par client, un point plein pour un départ observé, un cercle vide et une flèche pour un client encore présent (durée censurée).

**Une durée censurée n'est pas une donnée manquante.** On ne la supprime pas, et on ne la remplace pas par la durée observée. Elle dit : « *cette personne a survécu au moins jusque-là* ». C'est exactement l'information que les méthodes de ce chapitre exploitent.

**Mesurer l'erreur.** Avec huit clients, on ne peut pas savoir ce qui est « juste ». Faisons donc une **expérience contrôlée**, possible parce que nous simulons : nous créons 5 000 clients dont nous **connaissons les vraies durées** (une loi de Weibull, que nous définirons au 5.1.3), puis nous les « observons » en coupant l'étude à une date quelconque entre 6 et 60 mois après leur inscription.

```
from math import gamma, log

rng = np.random.default_rng(51)
n, k, echelle = 5000, 1.35, 36.0
T_vraie = echelle * rng.weibull(k, n)
C = rng.uniform(6, 60, n)
y = np.minimum(T_vraie, C)
parti = (T_vraie <= C).astype(int)

# durées complètes (inconnues en pratique)
# durée d'observation possible de chaque client
# ce que l'on voit

moy_vraie = echelle * gamma(1 + 1 / k)
print(f"part de clients censurés : {100 * (1 - parti.mean()):.1f} %")
print(f"VRAIE durée moyenne (formule exacte) : {moy_vraie:.1f} mois")
print(f"(1) moyenne de toutes les durées observées : {y.mean():.1f} mois")
print(f"(2) moyenne des seuls partis : {y[parti == 1].mean():.1f} mois")
```

```
part de clients censurés : 44.8 %
VRAIE durée moyenne (formule exacte) : 33.0 mois
(1) moyenne de toutes les durées observées : 21.5 mois
(2) moyenne des seuls partis : 18.5 mois
```

La vérité est de **33 mois** ; les deux moyennes naïves donnent environ 21,5 et 18,5, soit **un tiers à presque la moitié de moins**. Aucune ne converge vers la bonne valeur quand on ajoute des données : ce sont des **estimateurs biaisés**, au sens du volume I (section 3.2). Les méthodes du chapitre corrigent ce biais.

### 5.1.2 Le vocabulaire

Posons les notations, qui serviront jusqu'à la fin du chapitre. Pour chaque client  $i$  :

- $T_i$  est la **durée vraie** (le temps entre l'**origine** et l'**événement**), généralement inconnue ;
- $C_i$  est la **durée de censure** : le temps entre l'origine et l'instant où l'on cesse d'observer ;
- ce que l'on **voit** est le couple  $(Y_i, \delta_i)$  avec

$$Y_i = \min(T_i, C_i), \quad \delta_i = \begin{cases} 1 & \text{si } T_i \leq C_i \text{ (événement observé)} \\ 0 & \text{si } T_i > C_i \text{ (censuré)} \end{cases}$$

Dans notre fichier, `duree_mois` est  $Y$  et `churn` est  $\delta$ . Trois choix, souvent oubliés, définissent proprement une étude de survie :

1. **L'événement** : ici, « le client cesse d'acheter ». À définir précisément (dans le fichier simulé, la colonne est donnée ; dans la réalité, on décide, par exemple, « aucun achat depuis 6 mois »).
2. **L'origine du temps** : ici, la date d'inscription. Chaque client a *sa propre* horloge, qui démarre à *son* inscription ; c'est ce qui permet de comparer des clients arrivés à des dates différentes.
3. **L'unité** : le mois.

### 5.1.3 Trois fonctions pour décrire une durée

Soit  $T$  une durée (positive) de fonction de répartition  $F(t) = P(T \leq t)$  et de densité  $f(t)$ . On définit trois objets, équivalents mais qui éclairent chacun un aspect différent.

**La fonction de survie.**

$$S(t) = P(T > t) = 1 - F(t)$$

C'est la proportion de clients encore là à l'instant  $t$ . Elle vaut 1 en  $t = 0$ , décroît, et tend vers 0 (si tout le monde finit par partir).

**Le risque instantané** (en anglais *hazard*) :

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t} = \frac{f(t)}{S(t)}$$

C'est le **taux de départ, à l'instant  $t$ , parmi ceux qui sont encore là**. Ce n'est pas une probabilité (il peut dépasser 1, il se mesure « par mois ») : c'est une vitesse. Si  $h(10) = 0,03$  par mois, un client encore présent au dixième mois a environ 3 % de chances de partir dans le mois qui suit.

**Le risque cumulé :**

$$H(t) = \int_0^t h(u) du$$

C'est le « total de risque » accumulé depuis l'origine.

**Les trois fonctions sont reliées :**  $S(t) = e^{-H(t)}$ . Partons de  $f = -S'$ , car  $S = 1 - F$  et  $F' = f$ . Alors

$$h(t) = \frac{f(t)}{S(t)} = -\frac{S'(t)}{S(t)} = -\frac{d}{dt} \ln S(t).$$

Intégrons de 0 à  $t$  en utilisant  $S(0) = 1$ , donc  $\ln S(0) = 0$  :

$$H(t) = \int_0^t h(u) du = -\ln S(t) + \ln S(0) = -\ln S(t), \quad \text{d'où} \quad S(t) = e^{-H(t)}.$$

Connaître l'une des trois fonctions suffit donc à retrouver les deux autres :  $f(t) = h(t) e^{-H(t)}$ .

**Exemple à la main : le risque constant.** Supposons que, chaque mois, un client encore présent ait la même chance de partir :  $h(t) = \lambda = 0,03$  par mois. Alors  $H(t) = 0,03t$  et  $S(t) = e^{-0,03t}$  : la loi **exponentielle** du volume I. Au bout de 12 mois,  $S(12) = e^{-0,36} \approx 0,698$  : 70 % des clients sont encore là. La médiane vérifie  $S(t) = 0,5$ , soit  $t = \ln 2 / 0,03 \approx 23,1$  mois. La moyenne vaut  $1/\lambda \approx 33,3$  mois.

```
lam = 0.03
print("S(12)      =", round(np.exp(-lam * 12), 4))
print("médiane   =", round(np.log(2) / lam, 2), "mois")
print("moyenne   =", round(1 / lam, 2), "mois")
```

```
S(12)      = 0.6977
médiane    = 23.1 mois
moyenne    = 33.33 mois
```

Un risque constant a une propriété remarquable (et très restrictive) : la loi exponentielle est **sans mémoire**. Un client qui est là depuis 30 mois a *exactement* la même chance de partir le mois prochain qu'un client arrivé hier. Est-ce réaliste ? En pratique, rarement : les premiers mois d'une relation commerciale sont souvent les plus risqués (on essaie, on est déçu, on part), ou au contraire le risque augmente avec le temps (lassitude, usure). Il faut une famille plus souple.

**La loi de Weibull** généralise l'exponentielle avec un paramètre de **forme**  $k > 0$  et un paramètre d'**échelle**  $\sigma > 0$  :

$$h(t) = \frac{k}{\sigma} \left(\frac{t}{\sigma}\right)^{k-1}, \quad H(t) = \left(\frac{t}{\sigma}\right)^k, \quad S(t) = \exp\left[-\left(\frac{t}{\sigma}\right)^k\right].$$

Le risque est **décroissant** si  $k < 1$ , **constant** si  $k = 1$  (c'est l'exponentielle, avec  $\lambda = 1/\sigma$ ), **croissant** si  $k > 1$ . La médiane est  $\sigma(\ln 2)^{1/k}$  et la moyenne  $\sigma \Gamma(1 + 1/k)$ .

```
import matplotlib.pyplot as plt

t = np.linspace(0.01, 60, 400)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
formes = [(0.7, AQUA, "k = 0,7 : risque décroissant"), (1.0, BLEU, "k = 1 : risque constant"),
          (1.35, ORANGE, "k = 1,35 : risque croissant"), (2.0, VIOLET, "k = 2 : croissant plus vite")]
for kk, couleur, nom in formes:
    h = (kk / 36) * (t / 36) ** (kk - 1)
    S = np.exp(-(t / 36) ** kk)
    ax1.plot(t, h, color=couleur, label=nom)
    ax2.plot(t, S, color=couleur)
ax1.set_ylim(0, 0.12)
ax1.set_xlabel("mois depuis l'inscription")
ax1.set_ylabel("risque instantané h(t), par mois")
ax1.set_title("Le risque : quatre formes")
ax1.legend(fontsize=8, loc="upper center")
ax2.set_xlabel("mois depuis l'inscription")
ax2.set_ylabel("survie S(t)")
ax2.set_title("La survie correspondante (σ = 36 mois)")
ax2.axhline(0.5, color="#c3c2b7", lw=0.8, ls="--")
plt.tight_layout()
plt.savefig("figures/ch05-formes-risque.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

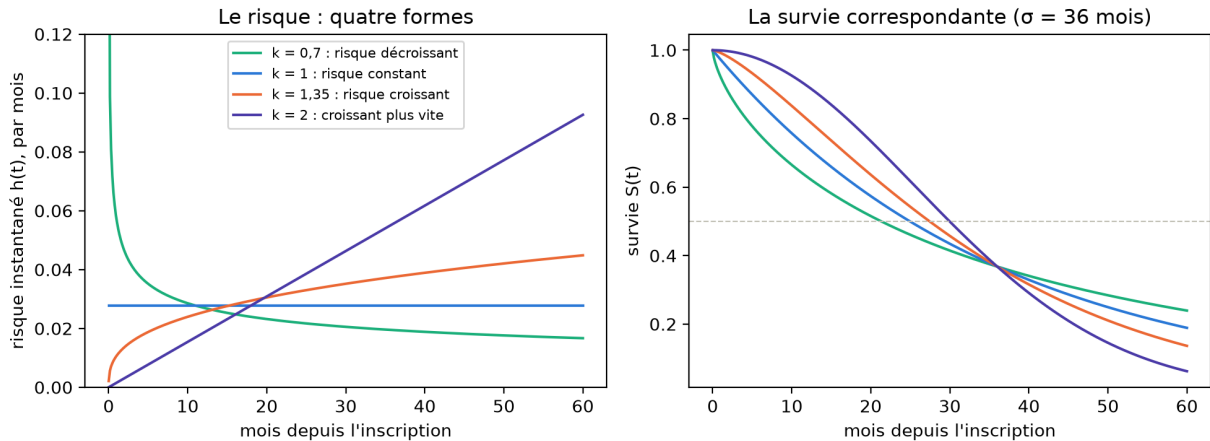


Figure 53: Quatre lois de Weibull de même échelle (36 mois) : à gauche le risque instantané, à droite la courbe de survie correspondante. Le pointillé marque la médiane ( $S = 0,5$ ).

Remarquez que les quatre courbes de survie se croisent au même point, à  $t = \sigma = 36$  mois, où elles valent toutes  $e^{-1} \approx 0,37$  : c'est la conséquence de  $S(\sigma) = \exp[-(\sigma/\sigma)^k] = e^{-1}$ , quel que soit  $k$ . On lit ensuite l'effet de la forme : à échelle égale, un risque qui **monte** ( $k > 1$ ) laisse presque tous les clients survivre au début puis les fait partir massivement plus tard, tandis qu'un risque qui **baisse** ( $k < 1$ ) en élimine beaucoup tôt et laisse un noyau de fidèles.

**La durée moyenne est l'aire sous la courbe de survie :**  $E[T] = \int_0^\infty S(t) dt$ . Pour le voir, écrivons  $T = \int_0^\infty \mathbf{1}_{\{T > t\}} dt$  (l'intégrale de la fonction qui vaut 1 tant que  $t < T$  vaut  $T$ ). L'espérance de l'intégrale est l'intégrale de l'espérance (théorème de Fubini, valable ici car tout est positif) :

$$E[T] = \int_0^\infty E[\mathbf{1}_{\{T > t\}}] dt = \int_0^\infty P(T > t) dt = \int_0^\infty S(t) dt.$$

Cette formule sera notre boussole : estimer la durée moyenne, c'est estimer **l'aire sous la courbe de survie**.

Vérifions-la numériquement pour la Weibull ( $k = 1,35, \sigma = 36$ ) :

```
from scipy.integrate import quad
```

```
aire, _ = quad(lambda s: np.exp(-(s / 36) ** 1.35), 0, np.inf)
print("aire sous S(t)      :", round(aire, 3))
print("formule σ Γ(1 + 1/k) :", round(36 * gamma(1 + 1 / 1.35), 3))
print("médiane σ (ln 2)^(1/k) :", round(36 * log(2) ** (1 / 1.35), 2), "mois")
```

```
aire sous S(t)      : 33.012
formule σ Γ(1 + 1/k) : 33.012
médiane σ (ln 2)^(1/k) : 27.44 mois
```

#### 5.1.4 Les différentes sortes de censure

Nous avons parlé de censure « à droite » sans la définir précisément. Il en existe plusieurs sortes, qu'il faut savoir reconnaître car elles ne se traitent pas de la même façon.

| Type           | Ce que l'on sait                                                          | Exemple chez Dar Jasmin                                                                             |
|----------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| À droite       | $T > c$ : l'événement n'a pas eu lieu à la fin de l'observation           | un client inscrit en mars 2025, toujours actif en décembre                                          |
| À gauche       | $T < c$ : l'événement a <i>déjà</i> eu lieu avant la première observation | on découvre en 2025 qu'un client de la base n'a rien acheté depuis une date inconnue, avant l'étude |
| Par intervalle | $a < T \leq b$                                                            | une enquête annuelle : « le client est encore là en 2023, parti en 2024 »                           |

Ne confondez pas la censure avec la **troncature** (ou **entrée tardive**) : un client qui a rejoint le programme de fidélité en 2022 alors qu'il était client depuis 2018 n'est observé qu'à partir de 2022. Ceux qui sont partis *avant* 2022 n'apparaissent jamais dans la base. La censure, elle, conserve l'individu dans l'échantillon ; la troncature le fait **disparaître** s'il n'a pas « survécu jusqu'à l'entrée ». Il faut alors corriger les ensembles à risque (nous le ferons à la section 5.2 avec des entrées tardives). Dans notre fichier, tous les clients sont observés **dès leur inscription** : il n'y a pas de troncature, et la seule censure est à droite.

On distingue encore, pour la censure à droite :

- la censure **administrative** (type I) : l'étude s'arrête à une date fixée à l'avance, ici le 31 décembre 2025 ;
- la censure **aléatoire** : des individus sortent de l'étude pour une raison propre (déménagement, perte de contact, décès par une autre cause) ;
- la censure de **type II** : on arrête quand un nombre fixé d'événements est atteint (fréquent en essais industriels).

**L'hypothèse de censure non informative.** Toute la théorie qui suit suppose que **la censure est indépendante de la durée** : savoir qu'un client est censuré à  $c$  ne doit rien dire sur son risque de partir juste après  $c$ . Autrement dit, les clients encore sous observation au temps  $c$  sont **représentatifs** de tous ceux qui seraient encore là à  $c$ . Pour la censure administrative, c'est plausible (la date du 31 décembre ne dépend pas des clients). Pour une perte de vue, c'est souvent **faux** : un client qui cesse de répondre est peut-être précisément en train de partir. Cette hypothèse n'est **pas testable** avec les seules données ; il faut la défendre par la connaissance du terrain.

Regardons ce que contient notre fichier : quelle part des durées censurées est purement administrative ?

```
c = pd.read_csv("donnees/clients.csv")
c["date_inscription"] = pd.to_datetime(c["date_inscription"])
c["suivi_possible"] = (pd.Timestamp("2025-12-31") - c["date_inscription"]).dt.days / 30.4375

censures = c[c["churn"] == 0]
admin = (censures["duree_mois"] - censures["suivi_possible"]).abs() < 0.01
print("clients :", len(c), "| départs observés :", int(c["churn"].sum()), "| censurés :",
      ↪ len(censures))
print("censurés administratifs (observés jusqu'au 31/12/2025) :", int(admin.sum()))
print("censurés avant la fin (pertes de vue) :", int((~admin).sum()))
print("durée de suivi possible : de", round(c["suivi_possible"].min(), 1), "à",
      ↪ round(c["suivi_possible"].max(), 1), "mois")
```

clients : 2000 | départs observés : 977 | censurés : 1023  
censurés administratifs (observés jusqu'au 31/12/2025) : 660  
censurés avant la fin (pertes de vue) : 363  
durée de suivi possible : de 6.0 à 84.0 mois

Environ **deux tiers** des durées censurées (660 sur 1 023) sont purement administratives : ces clients sont encore là le 31 décembre 2025. Le tiers restant (363) est constitué de **pertes de vue** : des clients qui sortent de l'observation avant la fin sans que l'on ait vu de départ (changement d'adresse, compte clos par erreur, perte de contact...). Ce n'est donc pas un détail. La durée de suivi possible va de 6 mois (inscrits en juin 2025) à 84 mois (inscrits en janvier 2019).

Que faire de ces pertes de vue ? Nous les traiterons, comme le fait toute analyse standard, comme une censure **non informative**. Ici, c'est justifié par une raison que nous connaissons parce que nous simulons : elles sont indépendantes de la durée par construction. Dans une vraie étude, ce serait **une hypothèse à discuter** : si les clients « perdus de vue » étaient en réalité des clients qui partent sans prévenir, les traiter comme censurés *surestimerait* la survie. Une analyse de sensibilité classique consiste à les recompter comme des départs (le pire cas) pour encadrer la vérité.

### 5.1.5 La vraisemblance avec censure

Comment estimer des paramètres quand une partie des durées est censurée ? Par le **maximum de vraisemblance** (volume I, section 3.2), à condition d'écrire correctement la contribution de chaque client.

- Un client dont le départ est **observé** à  $y_i$  apporte la densité  $f(y_i)$  : « la durée est exactement  $y_i$  ».
- Un client **censuré** à  $y_i$  apporte  $S(y_i) = P(T > y_i)$  : « la durée dépasse  $y_i$  ».

La vraisemblance du paramètre  $\theta$  de la loi de  $T$  est donc, pour des durées indépendantes :

$$L(\theta) = \prod_{i=1}^n f(y_i; \theta)^{\delta_i} S(y_i; \theta)^{1-\delta_i} = \prod_{i=1}^n h(y_i; \theta)^{\delta_i} S(y_i; \theta),$$

puisque  $f = h S$ . En passant au logarithme et en utilisant  $\ln S = -H$  :

$$\ell(\theta) = \sum_{i=1}^n [\delta_i \ln h(y_i; \theta) - H(y_i; \theta)]$$

La loi de la censure  $C$  n'apparaît pas : sous l'hypothèse d'indépendance, elle ne dépend pas de  $\theta$  et se factorise hors de la vraisemblance. C'est cette forme que nous utiliserons aux sections 5.3 (vraisemblance partielle de Cox) et 5.4 (modèles paramétriques).

**Premier calcul : le taux de départ constant.** Pour la loi exponentielle,  $h = \lambda$  et  $H(y) = \lambda y$  :

$$\ell(\lambda) = D \ln \lambda - \lambda \sum_i y_i, \quad D = \sum_i \delta_i \text{ (nombre de départs).}$$

En dérivant,  $\ell'(\lambda) = D/\lambda - \sum y_i = 0$ , donc

$$\hat{\lambda} = \frac{D}{\sum_i y_i} = \frac{\text{nombre de départs observés}}{\text{total des mois d'exposition}}.$$

Le dénominateur est l'**exposition**, en « mois-clients » : chaque client, parti ou censuré, contribue par **tout** le temps qu'il a passé sous observation. La dérivée seconde  $-D/\lambda^2$  donne l'écart-type  $\hat{\lambda}/\sqrt{D}$ .

Sur nos huit clients :  $D = 4$  départs et  $\sum y_i = 3 + 5 + 6 + 8 + 10 + 12 + 14 + 14 = 72$  mois-clients, donc  $\hat{\lambda} = 4/72 \approx 0,056$  par mois. La durée moyenne estimée est  $1/\hat{\lambda} = 18$  mois et la médiane  $18 \ln 2 \approx 12,5$  mois. Vérifions avec un optimiseur, puis appliquons aux 2 000 clients :

```

from scipy.optimize import minimize_scalar

def log_vraisemblance_exp(lam, y, d):
    return d.sum() * np.log(lam) - lam * y.sum()

y8, d8 = clients8["mois"].to_numpy(float), clients8["parti"].to_numpy()
opt = minimize_scalar(lambda l: -log_vraisemblance_exp(l, y8, d8), bounds=(1e-4, 1), method="bounded")
print(f"8 clients :  $\lambda^{\text{optimiseur}} = \{\text{opt.x} : .4f\}$  | formule  $D/\Sigma y = \{d8.sum() / y8.sum() : .4f\}$ ")

y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
lam_hat = d.sum() / y.sum()
se = lam_hat / np.sqrt(d.sum())
print(f"2000 clients : D = {d.sum()}, exposition = {y.sum() : .0f} mois-clients")
print(f" $\lambda^{\text{hat}} = \{\text{lam\_hat} : .5f\}$  par mois (IC95 :  $\{\text{lam\_hat} - 1.96 * \text{se} : .5f\}$  à  $\{\text{lam\_hat} + 1.96 * \text{se} : .5f\}$ )")
print(f"durée moyenne estimée  $1/\lambda^{\text{hat}} = \{1 / \text{lam\_hat} : .1f\}$  mois | médiane  $\ln 2 / \lambda^{\text{hat}} = \{\text{np.log}(2) / \text{lam\_hat} : .1f\}$  mois")
print("pour comparer : moyenne naïve de toutes les durées =", round(y.mean(), 1), "mois")

8 clients :  $\lambda^{\text{optimiseur}} = 0.0556$  | formule  $D/\Sigma y = 0.0556$ 
2000 clients : D = 977, exposition = 46,761 mois-clients
 $\lambda^{\text{hat}} = 0.02089$  par mois (IC95 : 0.01958 à 0.02220)
durée moyenne estimée  $1/\lambda^{\text{hat}} = 47.9$  mois | médiane  $\ln 2 / \lambda^{\text{hat}} = 33.2$  mois
pour comparer : moyenne naïve de toutes les durées = 23.4 mois

```

L'estimation exponentielle donne environ 48 mois de durée moyenne, plus du double de la moyenne naïve (23 mois). Est-elle pour autant fiable ? Elle repose sur une hypothèse très forte : un risque **constant**. Si le vrai risque est croissant ou décroissant, l'estimation est biaisée. Il nous faut donc une méthode qui **ne suppose pas la forme du risque** : c'est l'objet de la section suivante.

### À retenir

- Une durée est **censurée à droite** quand l'événement n'a pas eu lieu à la fin de l'observation : on sait seulement  $T > y$ . Ce n'est pas une donnée manquante.
- Ni la moyenne de toutes les durées, ni celle des seuls événements, ni la proportion d'événements ne convergent vers la vraie durée moyenne : ces trois résumés sont **biaisés** par la censure.
- Trois fonctions décrivent une durée :  $S(t) = P(T > t)$ , le risque instantané  $h(t) = f/S$  et le risque cumulé  $H(t)$ , reliés par  $S(t) = e^{-H(t)}$  ; et  $E[T] = \int_0^\infty S(t) dt$ .
- La loi de Weibull offre un risque décroissant, constant ou croissant selon sa forme  $k$  ; l'exponentielle est le cas  $k = 1$  (sans mémoire).
- La vraisemblance avec censure est  $\ell = \sum [\delta_i \ln h(y_i) - H(y_i)]$  ; pour un risque constant,  $\hat{\lambda} = D / \sum y_i$  (événements par mois d'exposition).
- Toute la théorie suppose une censure **non informative** (indépendante de la durée) : une hypothèse qui se défend, mais ne se teste pas.

## 5.2 L'estimateur de Kaplan-Meier

**Intuition.** Pour savoir quelle part des clients est encore là au bout de 24 mois, on ne peut pas simplement compter ceux qui ont duré 24 mois : beaucoup de clients récents n'ont pas encore atteint cet âge. L'idée de Kaplan et Meier (1958) est de **découper le temps** : à chaque moment où quelqu'un part, on regarde *parmi ceux qui étaient encore sous observation à cet instant* quelle fraction est partie, et on **multiplie** les fractions de survie successives. Chaque client, censuré ou non, participe tant qu'il est observé, puis sort proprement du calcul.



### 5.2.1 Le principe, à la main

Reprenons nos huit clients (section 5.1.1). Les départs ont lieu aux mois 3, 5, 8 et 12. À chacun de ces instants, on se demande : *parmi les clients encore présents et observés juste avant, quelle fraction part ?*

- Mois 3 : les 8 clients sont là. Un part (A). La survie sur cet instant est  $1 - \frac{1}{8} = \frac{7}{8} = 0,875$ .
- Mois 5 : 7 clients sont là (A est parti). B part :  $1 - \frac{1}{7} = \frac{6}{7}$ . La survie cumulée est  $\frac{7}{8} \times \frac{6}{7} = 0,75$ .
- Mois 6 : C est **censuré**. Personne ne part. La courbe ne bouge pas, mais C **quitte l'ensemble à risque** : à partir de maintenant, il n'est plus compté.
- Mois 8 : il reste 5 clients (D, E, F, G, H). D part :  $1 - \frac{1}{5} = \frac{4}{5}$ , survie cumulée  $0,75 \times 0,8 = 0,6$ .
- Mois 10 : E est censuré, sort de l'ensemble à risque.
- Mois 12 : il reste 3 clients (F, G, H). F part :  $1 - \frac{1}{3} = \frac{2}{3}$ , survie cumulée  $0,6 \times \frac{2}{3} = 0,4$ .
- Mois 14 : G et H sont censurés. Il n'y a plus d'autre départ : la courbe reste à 0,4.

Le nombre de clients sous observation juste avant l'instant  $t_j$  s'appelle l'**ensemble à risque** (*risk set*), noté  $n_j$ . Le nombre de départs à  $t_j$  est  $d_j$ . Voilà l'estimateur de Kaplan-Meier :

$$\hat{S}(t) = \prod_{j: t_j \leq t} \left(1 - \frac{d_j}{n_j}\right)$$

où le produit porte sur tous les instants  $t_j \leq t$  où au moins un départ a été observé. Les censures **n'apparaissent pas** dans la formule, mais elles agissent *indirectement* en réduisant  $n_j$  : un client censuré à 6 mois compte dans les  $n_j$  des instants antérieurs à 6 mois, mais plus ensuite.

Mettons ce calcul en code, sous la forme d'un tableau que nous réutiliserons :

```
import numpy as np
import pandas as pd

def tableau_km(y, d):
    """Version pédagogique : une ligne par instant de départ, calcul direct."""
    y, d = np.asarray(y, float), np.asarray(d, int)
    lignes, S, somme_gw = [], 1.0, 0.0
    for t in np.unique(y[d == 1]):
        n = int(np.sum(y >= t))          # ensemble à risque : encore là juste avant t
        dj = int(np.sum((y == t) & (d == 1))) # départs à l'instant t
        cj = int(np.sum((y == t) & (d == 0))) # censures à l'instant t (comptées après les
        ↪ départs)
        S *= 1 - dj / n
        somme_gw += dj / (n * (n - dj)) if n > dj else np.inf
        lignes.append((t, n, dj, cj, round(dj / n, 4), round(S, 4), round(somme_gw, 4)))
    return pd.DataFrame(lignes, columns=["t_j", "n_j (à risque)", "d_j (départs)", "censurés en t_j",
                                         "d_j/n_j", "S(t_j)", "somme Greenwood"])

y8 = np.array([3, 5, 6, 8, 10, 12, 14, 14])
d8 = np.array([1, 1, 0, 1, 0, 1, 0, 0])
print(tableau_km(y8, d8).to_string(index=False))
```

| t_j  | n_j (à risque) | d_j (départs) | censurés en t_j | d_j/n_j | S(t_j) | somme Greenwood |
|------|----------------|---------------|-----------------|---------|--------|-----------------|
| 3.0  | 8              | 1             | 0               | 0.1250  | 0.875  | 0.0179          |
| 5.0  | 7              | 1             | 0               | 0.1429  | 0.750  | 0.0417          |
| 8.0  | 5              | 1             | 0               | 0.2000  | 0.600  | 0.0917          |
| 12.0 | 3              | 1             | 0               | 0.3333  | 0.400  | 0.2583          |

On retrouve les valeurs de la main :  $0,875 \rightarrow 0,75 \rightarrow 0,6 \rightarrow 0,4$ . La dernière colonne servira au 5.2.3.

**Pourquoi cette formule, et pourquoi est-elle « la bonne » ?** Deux justifications.

*Par les probabilités conditionnelles.* Découpons l'axe du temps en intervalles autour des instants de départ. Survivre au-delà de  $t$  revient à survivre à chacun des instants de départ précédents en ayant survécu aux précédents :

$$S(t) = \prod_{j: t_j \leq t} P(T > t_j | T \geq t_j) = \prod_j (1 - h_j), \quad h_j = P(T = t_j | T \geq t_j).$$

Le « risque discret »  $h_j$  se lit directement dans les données : parmi les  $n_j$  clients à risque,  $d_j$  partent, d'où l'estimation naturelle  $\hat{h}_j = d_j/n_j$ .

*Par le maximum de vraisemblance.* Supposons que la loi de  $T$  ne charge que les instants observés, avec des risques  $h_1, \dots, h_J$  libres. À l'instant  $t_j$ , parmi les  $n_j$  clients à risque,  $d_j$  partent (probabilité  $h_j$  chacun) et  $n_j - d_j$  restent (probabilité  $1 - h_j$ ). Les censures n'ajoutent aucun facteur en  $h_j$  au-delà de leur participation aux  $n_j$ . La vraisemblance (section 5.1.5) est donc

$$L(h_1, \dots, h_J) = \prod_{j=1}^J h_j^{d_j} (1 - h_j)^{n_j - d_j}.$$

Chaque facteur est une vraisemblance binomiale ; on le maximise séparément en  $\hat{h}_j = d_j/n_j$ . Kaplan-Meier est donc l'**estimateur du maximum de vraisemblance non paramétrique** de la fonction de survie : on ne suppose *aucune forme* pour  $S$ .

## 5.2.2 L'estimateur sur les 2 000 clients, comparé aux bibliothèques

La version précédente boucle sur chaque instant et chaque client : c'est lisible, mais lent sur 2 000 clients. Voici une version vectorisée, qui prend en charge aussi les **entrées tardives** (nous en aurons besoin au 5.2.6), et que nous réutiliserons pour tout le chapitre.

```
def kaplan_meier(y, d, entree=None):
    """Kaplan-Meier vectorisé. Retourne (t_j, n_j, d_j, S, somme_greenwood).
    entree : date d'entrée dans l'observation (troncature à gauche), 0 par défaut."""
    y, d = np.asarray(y, float), np.asarray(d, int)
    tj = np.unique(y[d == 1])
    y_tries = np.sort(y)
    departes = np.sort(y[d == 1])
    sorties_avant = np.searchsorted(y_tries, tj, side="left")           # clients sortis
    ↪ strictement avant t_j
    entres_avant = len(y) if entree is None else np.searchsorted(np.sort(np.asarray(entree, float)),
    ↪ tj, side="left")
    n = entres_avant - sorties_avant                                   # à risque : entrés avant
    ↪ t_j et pas encore sortis
    dj = np.searchsorted(departes, tj, side="right") - np.searchsorted(departes, tj, side="left")
    S = np.cumprod(1 - dj / n)
    with np.errstate(divide="ignore"):
        gw = np.cumsum(dj / (n * (n - dj)))
    return tj, n, dj, S, gw

def surv_at(t, tj, S, avant=1.0):
    """Valeur de la fonction en escalier S à l'instant t (avant = valeur avant le premier départ)."""
    i = np.searchsorted(tj, t, side="right") - 1
    return avant if i < 0 else S[i]

# 1) la version rapide redonne exactement le tableau des 8 clients
tj8, n8, dj8, S8, gw8 = kaplan_meier(y8, d8)
print("8 clients, S aux instants de départ :", S8.round(4), "| à risque :", n8)

# 2) sur les 2000 clients
```

```
c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
tj, n, dj, S, gw = kaplan_meier(y, d)
print("instants de départ distincts :", len(tj), "pour", int(d.sum()), "départs")
print("instants avec plusieurs départs simultanés (ex aequo) :", int((dj > 1).sum()))
```

```
8 clients, S aux instants de départ : [0.875 0.75  0.6   0.4   ] | à risque : [8 7 5 3]
instants de départ distincts : 882 pour 977 départs
instants avec plusieurs départs simultanés (ex aequo) : 91
```

Comme les durées sont mesurées au centième de mois, quelques départs coïncident (les *ex aequo*) ; la formule les traite d'un seul bloc :  $d_j > 1$ . Comparons maintenant à statsmodels et au paquet R de référence, survival, à plusieurs instants :

```
from statsmodels.duration.survfunc import SurvfuncRight

sf = SurvfuncRight(y, d)
print(f"{'mois':>5} {'à la main':>10} {'statsmodels':>12} {'ET (Greenwood)':>15} {'ET  
↪ statsmodels':>15}")
for t in (12, 24, 36, 48, 60):
    i = np.searchsorted(sf.surv_times, t, side="right") - 1
    s_main = surv_at(t, tj, S)
    et_main = s_main * np.sqrt(surv_at(t, tj, gw, avant=0.0))
    print(f"{'t':>5} {'s_main':>10.5f} {'sf.surv_prob[i]:>12.5f} {'et_main':>15.5f}  
↪ {'sf.surv_prob_se[i]:>15.5f}")
```

| mois | à la main | statsmodels | ET (Greenwood) | ET statsmodels |
|------|-----------|-------------|----------------|----------------|
| 12   | 0.82519   | 0.82519     | 0.00888        | 0.00888        |
| 24   | 0.63376   | 0.63376     | 0.01211        | 0.01211        |
| 36   | 0.45275   | 0.45275     | 0.01396        | 0.01396        |
| 48   | 0.30758   | 0.30758     | 0.01498        | 0.01498        |
| 60   | 0.24754   | 0.24754     | 0.01558        | 0.01558        |

```
library(survival)
clients <- read.csv("donnees/clients.csv")
km_all <- survfit(Surv(duree_mois, churn) ~ 1, data = clients)
print(summary(km_all, times = c(12, 24, 36, 48, 60)))
```

```
Call: survfit(formula = Surv(duree_mois, churn) ~ 1, data = clients)
```

| time | n.risk | n.event | survival | std.err | lower 95% CI | upper 95% CI |
|------|--------|---------|----------|---------|--------------|--------------|
| 12   | 1378   | 323     | 0.825    | 0.00888 | 0.808        | 0.843        |
| 24   | 814    | 286     | 0.634    | 0.01211 | 0.610        | 0.658        |
| 36   | 416    | 200     | 0.453    | 0.01396 | 0.426        | 0.481        |
| 48   | 186    | 111     | 0.308    | 0.01498 | 0.280        | 0.338        |
| 60   | 96     | 31      | 0.248    | 0.01558 | 0.219        | 0.280        |

Les trois sources donnent la même courbe et les mêmes erreurs standard. Lecture : **82,5 %** des clients sont encore là à 12 mois, **63,4 %** à 24 mois, **45,3 %** à 36 mois, et **24,8 %** à 60 mois. La colonne n. risk de R est instructive : à 60 mois, il ne reste que 96 clients sous observation, contre 1 378 à 12 mois. Ce sont les *clients récents*, sortis de l'ensemble à risque par censure, qui font fondre l'effectif.

Voyons maintenant la courbe. Nous y superposons le modèle **exponentiel** de la section 5.1.5 (risque constant,  $\hat{\lambda} = 0,0209$  par mois) pour juger de sa pertinence :

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
z = 1.959964
```

```

bas = S ** np.exp(z * np.sqrt(gw) / np.abs(np.log(S))) # IC log-log, voir 5.2.3
haut = S ** np.exp(-z * np.sqrt(gw) / np.abs(np.log(S)))
temps = np.concatenate([[0], tj])

fig, ax = plt.subplots(figsize=(7.4, 4.2))
ax.step(temps, np.concatenate([[1], S]), where="post", color=BLEU, lw=2)
ax.fill_between(temps, np.concatenate([[1], bas]), np.concatenate([[1], haut]), step="post",
    ↪ color=BLEU, alpha=0.18, lw=0)
grille = np.linspace(0, 84, 200)
ax.plot(grille, np.exp(-(d.sum() / y.sum()) * grille), color=ORANGE, lw=1.8, ls="--")
ax.axhline(0.5, color="#c3c2b7", lw=0.8)
ax.legend(handles=[Line2D([0], [0], color=BLEU, lw=2, label="Kaplan-Meier (bande : IC 95 %)",
    ↪ Line2D([0], [0], color=ORANGE, lw=1.8, ls="--", label="modèle exponentiel (risque
    ↪ constant)"),
    loc="lower left", frameon=False, fontsize=10)
ax.set_xlabel("mois depuis l'inscription")
ax.set_ylabel("proportion de clients encore là")
ax.set_xlim(0, 84)
ax.set_ylim(0, 1.02)
plt.tight_layout()
plt.savefig("figures/ch05-km-global.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

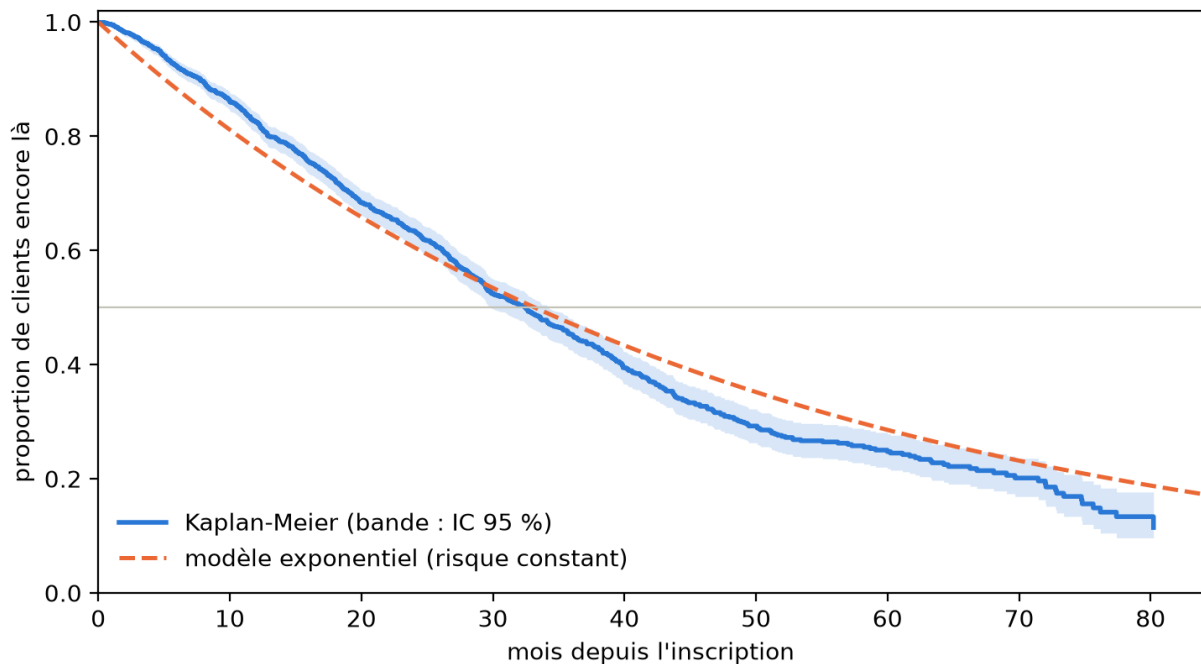


Figure 54: Courbe de Kaplan-Meier des 2 000 clients de Dar Jasmin, avec son intervalle de confiance à 95 %, et courbe du modèle exponentiel ajusté par maximum de vraisemblance.

La courbe exponentielle passe **sous** Kaplan-Meier pendant les 28 premiers mois environ (elle prévoit trop de départs précoces), croise la courbe vers 30 mois, puis reste **au-dessus** ensuite (elle prévoit trop de départs tardifs). Ce schéma est la signature d'un risque qui **augmente** avec l'ancienneté : un risque constant ne peut pas le reproduire. La section 5.4 le confirmera. C'est un avantage décisif de Kaplan-Meier : il ne parie sur aucune forme.

### 5.2.3 Précision : la variance de Greenwood et les intervalles de confiance

Une courbe sans mesure d'incertitude est un dessin, pas un résultat. Que vaut  $\hat{S}(t)$  ? Reprenons le raisonnement binomial.

**La formule de Greenwood.** À l'instant  $t_j$ , parmi  $n_j$  clients à risque, la fraction qui survit est  $\hat{p}_j = 1 - d_j/n_j$ . Conditionnellement à  $n_j$ ,  $d_j$  est binomial :  $\text{Var}(\hat{p}_j) \approx p_j(1 - p_j)/n_j$ . Comme  $\ln \hat{S}(t) = \sum_j \ln \hat{p}_j$ , la **méthode delta** (qui approche la variance d'une fonction régulière  $g$  d'une quantité aléatoire :  $\text{Var } g(X) \approx g'(E[X])^2 \text{Var } X$ ) donne

$$\text{Var}(\ln \hat{p}_j) \approx \frac{\text{Var}(\hat{p}_j)}{p_j^2} = \frac{1 - p_j}{n_j p_j} \approx \frac{d_j}{n_j(n_j - d_j)}.$$

Les facteurs des différents instants sont (conditionnellement) non corrélés, donc les variances s'additionnent :

$$\text{Var}(\ln \hat{S}(t)) \approx \sum_{j: t_j \leq t} \frac{d_j}{n_j(n_j - d_j)}, \quad \text{Var}(\hat{S}(t)) \approx \hat{S}(t)^2 \sum_{j: t_j \leq t} \frac{d_j}{n_j(n_j - d_j)}.$$

C'est la **formule de Greenwood** : la dernière colonne de notre tableau (la « somme Greenwood ») en est la somme partielle.

Sur nos huit clients à  $t = 12$  :  $\hat{S} = 0,4$  et la somme vaut 0,2583, donc l'écart-type est  $0,4\sqrt{0,2583} \approx 0,20$  : énorme, ce qui n'a rien d'étonnant avec huit clients.

Pour un intervalle de confiance, on peut prendre  $\hat{S} \pm 1,96\widehat{se}$  (intervalle **plan**), mais il peut sortir de  $[0, 1]$  et se comporte mal quand  $\hat{S}$  est proche de 0 ou de 1. On préfère transformer d'abord :

- **intervalle « log »** (le défaut de R) :  $\hat{S} \exp(\pm 1,96\sqrt{G(t)})$ , avec  $G(t) = \sum d_j/[n_j(n_j - d_j)]$  ;
- **intervalle « log-log »**, souvent recommandé : on travaille sur  $\ln(-\ln \hat{S})$ , dont la variance est  $G(t)/(\ln \hat{S})^2$ , ce qui donne  $\hat{S}^{\exp(\pm 1,96\sqrt{G(t)}/\ln \hat{S})}$  (les bornes restent toujours dans  $[0, 1]$ ).

```
def intervalles(t, tj, S, gw, z=1.959964):
    s, g = surv_at(t, tj, S), surv_at(t, tj, gw, avant=0.0)
    se = s * np.sqrt(g)
    plan = (s - z * se, s + z * se)
    log_ = (s * np.exp(-z * np.sqrt(g)), s * np.exp(z * np.sqrt(g)))
    expo = np.exp(z * np.sqrt(g) / abs(np.log(s)))
    loglog = (s ** expo, s ** (1 / expo))
    return s, plan, log_, loglog

for t in (36, 60):
    s, plan, log_, loglog = intervalles(t, tj, S, gw)
    print(f"t = {t} mois : S = {s:.4f}")
    print(f"    plan    [{plan[0]:.4f} ; {plan[1]:.4f}]")
    print(f"    log     [{log_[0]:.4f} ; {log_[1]:.4f}]")
    print(f"    log-log [{loglog[0]:.4f} ; {loglog[1]:.4f}]")
print()
s8, plan8, log8, ll8 = intervalles(12, tj8, S8, gw8)
print(f"8 clients, t = 12 : S = {s8:.2f} ; plan [{plan8[0]:.2f} ; {plan8[1]:.2f}] ; log-log
↪ [{ll8[0]:.2f} ; {ll8[1]:.2f}])"
```

```
t = 36 mois : S = 0.4528
    plan    [0.4254 ; 0.4801]
    log     [0.4262 ; 0.4810]
    log-log [0.4252 ; 0.4799]
t = 60 mois : S = 0.2475
    plan    [0.2170 ; 0.2781]
```

```

log      [0.2188 ; 0.2800]
log-log[0.2176 ; 0.2786]

8 clients, t = 12 : S = 0.40 ; plan [0.00 ; 0.80] ; log-log [0.07 ; 0.73]

for (ct in c("plain", "log", "log-log")) {
  f <- survfit(Surv(duree_mois, churn) ~ 1, data = clients, conf.type = ct)
  s <- summary(f, times = c(36, 60))
  cat(sprintf("%-8s t=36 [%%.4f ; %%.4f] t=60 [%%.4f ; %%.4f]\n", ct, s$lower[1], s$upper[1],
    ↪ s$lower[2], s$upper[2]))
}

plain      t=36 [0.4254 ; 0.4801] t=60 [0.2170 ; 0.2781]
log        t=36 [0.4262 ; 0.4810] t=60 [0.2188 ; 0.2800]
log-log    t=36 [0.4252 ; 0.4799] t=60 [0.2176 ; 0.2786]

```

Nos trois intervalles coïncident avec ceux de R à la quatrième décimale. Sur 2 000 clients, ils sont presque identiques ; la différence se voit sur les petits effectifs : avec les huit clients, l'intervalle plan est large (de presque 0 à 0,80), et le log-log est plus raisonnable. Ces intervalles s'interprètent comme au volume I (section 3.3.2) : la *méthode* encadre la vraie valeur dans 95 % des échantillons.

### 5.2.4 Médiane, quantiles et durée moyenne « restreinte »

La **médiane de survie** est le premier instant où la courbe passe sous 0,5. C'est le bon résumé de la « durée typique » : contrairement à la moyenne, elle est définie dès que la courbe descend sous 0,5, même si certains clients ne sont jamais partis. Son intervalle de confiance s'obtient en cherchant où la **bande de confiance** de la courbe coupe le niveau 0,5 (méthode de Brookmeyer et Crowley) :

```

z = 1.959964
S_bas, S_haut = S * np.exp(-z * np.sqrt(gw)), S * np.exp(z * np.sqrt(gw))      # bande « log »
med = tj[S <= 0.5][0]
print(f"médiane de survie : {med:.2f} mois ; IC95 [{tj[S_bas <= 0.5][0]:.1f} ; {tj[S_haut <=
  ↪ 0.5][0]:.1f}]" )
print("statsmodels      :", round(sf.quantile(0.5), 2))

médiane de survie : 32.45 mois ; IC95 [29.9 ; 34.2]
statsmodels      : 32.45

```

```
print(km_all)
```

```
Call: survfit(formula = Surv(duree_mois, churn) ~ 1, data = clients)
```

```

      n events median 0.95LCL 0.95UCL
[1,] 2000    977  32.5   29.9   34.2

```

La médiane est d'environ **32,5 mois** (IC95 : 29,9 à 34,2). La moitié des clients ont quitté Dar Jasmin au bout de deux ans et huit mois et demi. Notez que ce chiffre est bien supérieur à la moyenne « naïve » de 23 mois de la section 5.1.

**Et la durée moyenne ?** Nous savons que  $E[T] = \int_0^\infty S(t) dt$ . Mais Kaplan-Meier ne descend pas jusqu'à zéro : le dernier client est censuré, et la courbe s'arrête à 0,11 (dernier départ au mois 80,2). L'aire totale n'est pas définie ; elle dépend de ce que l'on suppose *au-delà* des données. On calcule donc la **durée moyenne restreinte** (en anglais *restricted mean survival time*, RMST) jusqu'à un horizon  $\tau$  choisi :

$$\text{RMST}(\tau) = \int_0^\tau \hat{S}(t) dt.$$

C'est le **nombre moyen de mois passés dans la clientèle pendant les  $\tau$  premiers mois**. Comme

$\hat{S}$  est une fonction en escalier, l'intégrale est une somme de rectangles. À la main, pour nos huit clients et  $\tau = 14$  :  $3 \times 1 + 2 \times 0,875 + 3 \times 0,75 + 4 \times 0,6 + 2 \times 0,4 = 3 + 1,75 + 2,25 + 2,4 + 0,8 = 10,2$  mois. (Ni 9 mois ni 7 mois : on a bien utilisé l'information des censurés.)

```
def rmst(tj, S, tau):
    """Aire sous la courbe en escalier de 0 à tau."""
    bornes = np.concatenate([[0.0], tj[tj < tau], [tau]])
    valeurs = np.concatenate([[1.0], S[tj < tau]])
    return float(np.sum(valeurs * np.diff(bornes)))

print("8 clients, tau = 14 :", round(rmst(tj8, S8, 14), 2), "mois")
for tau in (24, 36, 60):
    print(f"2000 clients, tau = {tau} : {rmst(tj, S, tau):.2f} mois passés en moyenne dans les {tau} premiers mois")
```

```
8 clients, tau = 14 : 10.2 mois
2000 clients, tau = 24 : 19.76 mois passés en moyenne dans les 24 premiers mois
2000 clients, tau = 36 : 26.19 mois passés en moyenne dans les 36 premiers mois
2000 clients, tau = 60 : 33.94 mois passés en moyenne dans les 60 premiers mois
```

Sur 36 mois, un nouveau client passe en moyenne un peu plus de 26 mois dans la clientèle de Dar Jasmin, sur un maximum possible de 36. C'est une quantité très parlante pour décider (nous la retrouverons au 5.4 pour la valeur vie client), et elle ne dépend d'aucune hypothèse de forme.

### 5.2.5 Comparer des groupes : courbes et test du log-rank

Yasmine a envoyé une **offre de bienvenue** à la moitié de ses clients, **tirée au hasard**. Cette offre prolonge-t-elle la relation ? Et le canal d'acquisition joue-t-il un rôle ? Traçons Kaplan-Meier par groupe, avec les bandes de confiance log-log.

```
def courbe_groupe(y, d, masque):
    t_, n_, d_, S_, g_ = kaplan_meier(y[masque], d[masque])
    b = S_ ** np.exp(1.959964 * np.sqrt(g_) / np.abs(np.log(S_)))
    h = S_ ** np.exp(-1.959964 * np.sqrt(g_) / np.abs(np.log(S_)))
    return t_, S_, b, h

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
off = c["offre_bienvenue"].to_numpy()
for val, couleur, nom in [(0, VIOLET, "sans offre"), (1, AQUA, "avec offre")]:
    t_, S_, b, h = courbe_groupe(y, d, off == val)
    x = np.concatenate([0, t_])
    ax1.step(x, np.concatenate([1, S_]), where="post", color=couleur, lw=2)
    ax1.fill_between(x, np.concatenate([1, b]), np.concatenate([1, h]), step="post",
        color=couleur, alpha=0.15, lw=0)
    ax1.legend(handles=[Line2D([0], [0], color=AQUA, lw=2, label="avec offre"), Line2D([0], [0],
        color=VIOLET, lw=2, label="sans offre")],
        loc="lower left", frameon=False, fontsize=10)
    ax1.set_title("Selon l'offre de bienvenue (attribuée au hasard)")
canaux = [("Boutique", AQUA), ("Site", BLEU), ("Instagram", ORANGE)]
for nom, couleur in canaux:
    t_, S_, b, h = courbe_groupe(y, d, (c["canal_acquisition"] == nom).to_numpy())
    x = np.concatenate([0, t_])
    ax2.step(x, np.concatenate([1, S_]), where="post", color=couleur, lw=2)
    ax2.fill_between(x, np.concatenate([1, b]), np.concatenate([1, h]), step="post",
        color=couleur, alpha=0.15, lw=0)
    ax2.legend(handles=[Line2D([0], [0], color=c_, lw=2, label=n_) for n_, c_ in canaux], loc="lower
        left", frameon=False, fontsize=10)
    ax2.set_title("Selon le canal d'acquisition")
for ax in (ax1, ax2):
    ax.axhline(0.5, color="#c3c2b7", lw=0.8)
    ax.set_xlabel("mois depuis l'inscription")
    ax.set_xlim(0, 84)
    ax.set_ylim(0, 1.02)
```



```
ax1.set_ylabel("proportion de clients encore là")
plt.tight_layout()
plt.savefig("figures/ch05-km-groupes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

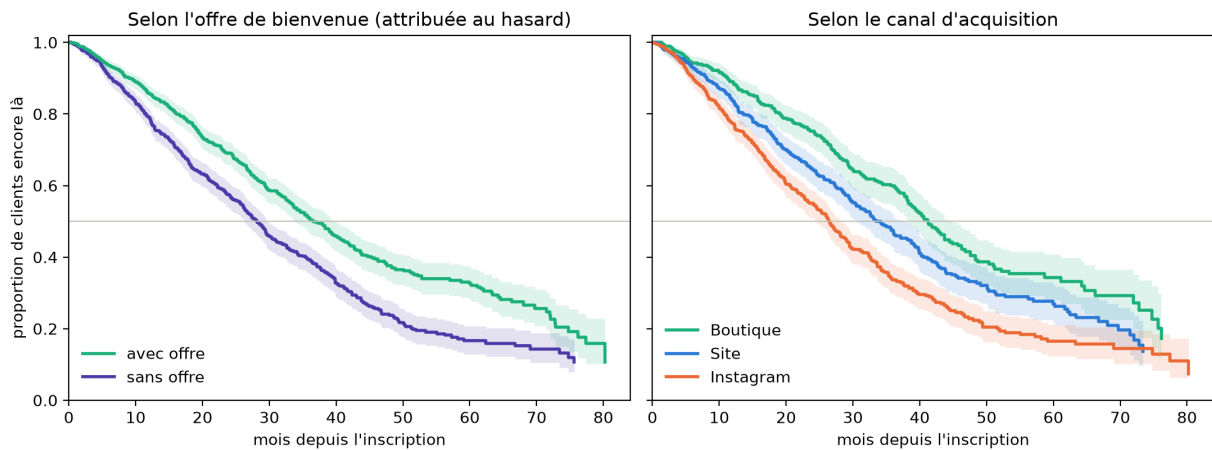


Figure 55: Courbes de Kaplan-Meier par groupe, avec bandes de confiance à 95 % : à gauche selon l'offre de bienvenue, à droite selon le canal d'acquisition.

Les courbes avec et sans offre se séparent nettement et durablement ; à droite, Boutique est au-dessus de Site, lui-même au-dessus d'Instagram (les bandes de Boutique et de Site se chevauchent par endroits). Résumons chaque groupe (médiane de survie et durée moyenne restreinte à 36 mois) :

```
resume = []
for var in ["offre_bienvenue", "canal_acquisition"]:
    for valeur, g in c.groupby(var):
        t_, n_, d_, S_, gw_ = kaplan_meier(g["duree_mois"], g["churn"])
        m = t_[S_ <= 0.5]
        resume.append({"variable": var, "groupe": valeur, "clients": len(g), "départs":
            int(g["churn"].sum()),
            "médiane (mois)": round(m[0], 1) if len(m) else np.nan, "RMST 36 mois":
            round(rmst(t_, S_, 36), 2)})
print(pd.DataFrame(resume).to_string(index=False))
```

| variable          | groupe    | clients | départs | médiane (mois) | RMST 36 mois |
|-------------------|-----------|---------|---------|----------------|--------------|
| offre_bienvenue   | 0         | 985     | 528     | 28.2           | 24.62        |
| offre_bienvenue   | 1         | 1015    | 449     | 36.6           | 27.69        |
| canal_acquisition | Boutique  | 504     | 212     | 41.2           | 29.10        |
| canal_acquisition | Instagram | 816     | 443     | 26.4           | 23.93        |
| canal_acquisition | Site      | 680     | 322     | 34.1           | 26.67        |

Les chiffres sont éloquentes : sans offre, la médiane est de 28 mois ; avec l'offre, de près de 37. Mais est-ce un vrai effet ou le hasard de l'échantillonnage ? Il faut un **test**.

**Le test du log-rank.** C'est le test de référence pour comparer deux courbes de survie (ou plus). Son principe est celui du volume I (section 3.4), appliqué à *chaque instant de départ* : à l'instant  $t_j$ ,  $d_j$  départs ont lieu parmi  $n_j$  clients à risque, dont  $n_{1j}$  dans le groupe 1. Si les deux groupes avaient **le même risque** (hypothèse nulle), les  $d_j$  départs seraient répartis « au hasard » entre les clients à risque, et le nombre de départs dans le groupe 1 suivrait une loi **hypergéométrique** :

$$E_{1j} = d_j \frac{n_{1j}}{n_j}, \quad V_j = d_j \frac{n_{1j}}{n_j} \left(1 - \frac{n_{1j}}{n_j}\right) \frac{n_j - d_j}{n_j - 1}.$$

On additionne, sur tous les instants, les écarts entre départs observés  $O_1$  et attendus  $E_1 = \sum_j E_{1j}$  :

$$\chi^2 = \frac{(O_1 - E_1)^2}{\sum_j V_j} \approx \chi_1^2 \text{ sous } H_0.$$

Pour  $K$  groupes, on utilise la forme quadratique du vecteur  $(O_k - E_k)_{k < K}$  avec la matrice de covariance correspondante :  $\chi_{K-1}^2$ .

**À la main, sur les huit clients.** Imaginons que C, E, F et G aient reçu l'offre, et A, B, D et H non. Aux quatre instants de départ :

| $t_j$ | $n_j$ | $n_{1j}$ (avec offre) | $d_j$ | qui part       | $E_{1j} = d_j n_{1j} / n_j$ | $V_j$ |
|-------|-------|-----------------------|-------|----------------|-----------------------------|-------|
| 3     | 8     | 4                     | 1     | A (sans offre) | 0,500                       | 0,250 |
| 5     | 7     | 4                     | 1     | B (sans offre) | 0,571                       | 0,245 |
| 8     | 5     | 3                     | 1     | D (sans offre) | 0,600                       | 0,240 |
| 12    | 3     | 2                     | 1     | F (avec offre) | 0,667                       | 0,222 |

Le groupe « avec offre » a eu  $O_1 = 1$  départ pour  $E_1 = 2,338$  attendus ;  $\sum V_j = 0,957$  ;  $\chi^2 = (1 - 2,338)^2 / 0,957 = 1,87$ , soit  $p \approx 0,17$  : avec huit clients, aucune conclusion, évidemment. Le code généralise à  $K$  groupes :

```
from scipy import stats

def logrank(y, d, groupe):
    """Test du log-rank à K groupes. Retourne (observés, attendus, chi2, ddl, p)."""
    y, d, groupe = np.asarray(y, float), np.asarray(d, int), np.asarray(groupe)
    niveaux = np.unique(groupe)
    K = len(niveaux)
    O, E, V = np.zeros(K), np.zeros(K), np.zeros((K, K))
    for t in np.unique(y[d == 1]):
        r = np.array([np.sum((y >= t) & (groupe == g)) for g in niveaux], float)
        o = np.array([np.sum((y == t) & (d == 1) & (groupe == g)) for g in niveaux], float)
        n_, dj_ = r.sum(), o.sum()
        O += o
        E += dj_ * r / n_
        if n_ > 1:
            V += dj_ * (n_ - dj_) / (n_ - 1) * (np.diag(r) / n_ - np.outer(r, r) / n_**2)
    x = (O - E)[:K-1]
    chi2 = float(x @ np.linalg.solve(V[:K-1, :K-1], x))
    return O, E, chi2, K - 1, stats.chi2.sf(chi2, K - 1)

groupe8 = np.array(["sans", "sans", "avec", "sans", "avec", "avec", "avec", "sans"]) # A..H
O8, E8, chi8, ddl8, p8 = logrank(y8, d8, groupe8)
print(f"8 clients : observés {O8} | attendus {E8.round(3)} | chi2 = {chi8:.2f} | p = {p8:.3f}")
```

```
8 clients : observés [1. 3.] | attendus [2.338 1.662] | chi2 = 1.87 | p = 0.171
```

Appliquons maintenant le test aux données réelles : à la main, avec statsmodels, avec R.

```
from statsmodels.duration.survfunc import survdiff

O, E, chi2, ddl, p = logrank(y, d, c["offre_bienvenue"])
print(f"À la main : observés (sans, avec) = {O.astype(int)}, attendus = {E.round(1)}, chi2 = {chi2:.2f}, p = {p:.1e}")
print("statsmodels : chi2 = %.2f, p = %.1e" % survdiff(y, d, c["offre_bienvenue"]))
```

```
À la main : observés (sans, avec) = [528 449], attendus = [435.6 541.4], chi2 = 35.54, p = 2.5e-09
statsmodels : chi2 = 35.54, p = 2.5e-09
```

```
print(survdiff(Surv(duree_mois, churn) ~ offre_bienvenue, data = clients))
```

Call:

```
survdiff(formula = Surv(duree_mois, churn) ~ offre_bienvenue,
        data = clients)
```

|                   | N    | Observed | Expected | (O-E)^2/E | (O-E)^2/V |
|-------------------|------|----------|----------|-----------|-----------|
| offre_bienvenue=0 | 985  | 528      | 436      | 19.6      | 35.5      |
| offre_bienvenue=1 | 1015 | 449      | 541      | 15.8      | 35.5      |

Chisq= 35.5 on 1 degrees of freedom, p= 3e-09

Les trois calculs donnent  $\chi^2 \approx 35,5$  ( $p \approx 3 \times 10^{-9}$ ) : dans le groupe avec offre, on observe 449 départs là où l'on en attendrait 541 si l'offre ne changeait rien ; dans le groupe sans offre, 528 au lieu de 436. L'offre étant attribuée **au hasard**, la différence peut être lue comme **l'effet causal de l'offre** sur la durée de la relation (nous reviendrons sur la mesure de cet effet au 5.3).

Pour le canal d'acquisition (trois groupes,  $\chi^2_2$ ) puis les comparaisons deux à deux, avec la correction de **Holm** du volume I (section 3.5.5) :

```
03, E3, chi3, ddl3, p3 = logrank(y, d, c["canal_acquisition"])
print(f"3 canaux : chi2 = {chi3:.1f} à {ddl3} ddl, p = {p3:.1e}")
print(pd.DataFrame({"observés": 03.astype(int), "attendus": E3.round(1)},
    ↪ index=np.unique(c["canal_acquisition"])).T.to_string())

paires = [("Boutique", "Instagram"), ("Boutique", "Site"), ("Site", "Instagram")]
brutes = []
for a, b in paires:
    m = c["canal_acquisition"].isin([a, b]).to_numpy()
    brutes.append(logrank(y[m], d[m], c["canal_acquisition"].to_numpy()[m])[4])
ordre = np.argsort(brutes)
holm, courant = np.empty(3), 0.0
for rang, i in enumerate(ordre):
    courant = max(courant, min(1.0, (3 - rang) * brutes[i]))
    holm[i] = courant
print(pd.DataFrame({"comparaison": [f"{a} / {b}" for a, b in paires], "p brute": brutes, "p Holm":
    ↪ holm}).to_string(index=False,
    float_format=lambda x: f"{x:.2e}"))
```

|                                             |             |           |        |
|---------------------------------------------|-------------|-----------|--------|
| 3 canaux : chi2 = 55.0 à 2 ddl, p = 1.1e-12 |             |           |        |
|                                             | Boutique    | Instagram | Site   |
| observés                                    | 212.0       | 443.0     | 322.0  |
| attendus                                    | 296.5       | 342.4     | 338.1  |
|                                             | comparaison | p brute   | p Holm |
| Boutique / Instagram                        | 5.27e-13    | 1.58e-12  |        |
| Boutique / Site                             | 8.61e-04    | 8.61e-04  |        |
| Site / Instagram                            | 3.02e-05    | 6.05e-05  |        |

Les trois canaux diffèrent ( $\chi^2_2 = 55,0$ ). Deux à deux, toutes les comparaisons restent significatives après correction de Holm : la plus nette oppose Boutique et Instagram ( $p$  de l'ordre de  $10^{-12}$ ), la plus faible Boutique et Site ( $p \approx 9 \times 10^{-4}$ ). Les clients arrivés par la boutique restent le plus longtemps, ceux d'Instagram partent le plus vite : 212 départs observés en Boutique contre 296 attendus sous l'hypothèse « aucune différence », mais 443 contre 342 pour Instagram.

**Autres pondérations.** Le log-rank donne le **même poids** à tous les instants ; il est le plus puissant quand les risques des groupes sont **proportionnels** (section 5.3). D'autres tests pondèrent davantage le début (Gehan-Breslow, Tarone-Ware, Fleming-Harrington) ; ils sont plus sensibles

aux différences précoces et moins aux différences tardives :

```
for nom, w, kw in [("log-rank", None, {}), ("Gehan-Breslow", "gb", {}), ("Tarone-Ware", "tw", {}),
  ↪ ("Fleming-Harrington p=1", "fh", {"fh_p": 1})]:
    chi, pv = survdiff(y, d, c["offre_bienvenue"], weight_type=w, **kw)
    print(f"{nom:<24} chi2 = {chi:6.2f}    p = {pv:.1e}")
```

```
log-rank          chi2 = 35.54    p = 2.5e-09
Gehan-Breslow     chi2 = 31.45    p = 2.0e-08
Tarone-Ware       chi2 = 35.12    p = 3.1e-09
Fleming-Harrington p=1  chi2 = 34.98    p = 3.3e-09
```

```
print(survdiff(Surv(duree_mois, churn) ~ offre_bienvenue, data = clients, rho = 1)$chisq)
```

```
[1] 34.97764
```

Le dernier test (Fleming-Harrington,  $\rho = 1$  dans R) donne le même  $\chi^2$  dans les deux logiciels. Ici, toutes les pondérations concluent de la même façon. Quand elles divergent, c'est un signal qu'il se passe quelque chose de différent selon l'âge de la relation (par exemple des courbes qui se croisent) : on regarde alors les courbes plutôt que de choisir le test qui nous arrange.

**Le choix du test ne se fait pas après avoir vu les résultats.** Décidez de la pondération *avant* (par défaut, le log-rank). Essayer plusieurs tests et ne retenir que le meilleur est un cas de tests multiples (volume I, section 3.5.5).

### 5.2.6 Quand les clients entrent tard : la troncature à gauche

Nous avons promis (5.1.4) de montrer comment traiter les **entrées tardives**. Imaginons que Yasmine n'ait enregistré les clients qu'à partir de leur **adhésion au programme de fidélité**, qui peut intervenir longtemps après le premier achat. On mesure la durée depuis le premier achat, mais un client n'apparaît dans la base que s'il était **encore client** à sa date d'adhésion. Les clients partis avant n'ont jamais été vus.

La règle est simple : **un client n'est dans l'ensemble à risque à l'instant  $t$  que s'il est entré dans l'observation avant  $t$  et n'en est pas encore sorti** :

$$n_j = \#\{i : e_i < t_j \leq y_i\}.$$

Ignorer cette règle (prendre  $e_i = 0$  pour tout le monde) revient à compter, dans les ensembles à risque des premiers mois, des clients qui *n'étaient pas encore observables* ; on sous-estime le risque précoce et on **surestime la survie**. Vérifions par simulation, avec une vérité connue :

```
rng = np.random.default_rng(52)
N = 6000
T = 36 * rng.weibull(1.35, N)                # durées vraies (Weibull, comme en 5.1)
E = rng.uniform(0, 30, N)                  # date d'adhésion au programme (mois après le 1er achat)
vus = T > E                                # seuls les clients encore là à l'adhésion sont enregistrés
Tv, Ev = T[vus], E[vus]
Cv = Ev + rng.uniform(6, 60, vus.sum())    # fin d'observation après l'adhésion
Yv, Dv = np.minimum(Tv, Cv), (Tv <= Cv).astype(int)

tj_n, _, _, S_n, _ = kaplan_meier(Yv, Dv)    # on ignore les entrées tardives
tj_c, _, _, S_c, _ = kaplan_meier(Yv, Dv, entree=Ev) # on les prend en compte
sf_e = SurvfuncRight(Yv, Dv, entry=Ev)
print(f"clients vus : {vus.sum()} sur {N}")
print(f"{'mois':>5} {'vraie S(t)':>11} {'ignorer entrée':>15} {'avec entrée':>12}")
  ↪ {'statsmodels':>12}")
for t in (6, 12, 24, 36):
    i = np.searchsorted(sf_e.surv_times, t, side="right") - 1
    print(f"{'t':>5} {np.exp(-(t / 36) ** 1.35):>11.3f} {surv_at(t, tj_n, S_n):>15.3f} {surv_at(t, tj_c,
  ↪ S_c):>12.3f} {sf_e.surv_prob[i]:>12.3f}")
```

```
clients vus : 4417 sur 6000
mois   vraie S(t)   ignorer entrée   avec entrée   statsmodels
6      0.915        0.988        0.913        0.913
12     0.797        0.940        0.804        0.804
24     0.561        0.759        0.577        0.577
36     0.368        0.490        0.365        0.365
```

Les estimations qui **ignorent** l'entrée surestiment fortement la survie : 0,76 au lieu de 0,56 à 24 mois, par exemple. Celles qui **la prennent en compte** (à la main comme avec statsmodels, qui donnent exactement les mêmes valeurs) restent à environ deux points de la vérité à tous les horizons. Cet exemple n'est pas qu'académique : il explique pourquoi l'on dit que **la survie d'« ex-clients encore actifs » ou de patients « prévalents » est toujours plus belle que la réalité.**

### 5.2.7 Pièges et analyses de sensibilité

**La queue de la courbe est fragile.** Quand l'ensemble à risque devient petit (ici : moins de 100 clients au-delà de 60 mois, 7 avant le dernier départ), un seul départ fait chuter la courbe de plusieurs points. On présente toujours sous la courbe le **nombre de clients à risque** et on arrête l'interprétation là où il devient trop faible.

```
instants = [0, 12, 24, 36, 48, 60, 72]
tab = {}
for nom, masque in [("sans offre", off == 0), ("avec offre", off == 1)]:
    tab[nom] = [int(np.sum(y[masque] >= t)) for t in instants]
print(pd.DataFrame(tab, index=[f"{t} mois" for t in instants]).T.to_string())
```

|            | 0 mois | 12 mois | 24 mois | 36 mois | 48 mois | 60 mois | 72 mois |
|------------|--------|---------|---------|---------|---------|---------|---------|
| sans offre | 985    | 647     | 359     | 179     | 70      | 29      | 13      |
| avec offre | 1015   | 731     | 455     | 237     | 116     | 67      | 24      |

**La censure non informative est une hypothèse, pas un fait.** À la section 5.1.4, nous avons vu que 363 clients sont des *pertes de vue*. Et si, en réalité, tous étaient partis sans prévenir ? Une **analyse de sensibilité du pire cas** consiste à les recompter comme des départs, à la date où l'on les perd de vue : la vraie courbe se trouve alors entre les deux.

```
c["date_inscription"] = pd.to_datetime(c["date_inscription"])
suivi_possible = (pd.Timestamp("2025-12-31") - c["date_inscription"]).dt.days / 30.4375
perdu = ((c["churn"] == 0) & (suivi_possible - c["duree_mois"] > 0.01)).to_numpy()

d_pire = np.where(perdu, 1, d)
tj_p, _, _, S_p, _ = kaplan_meier(y, d_pire)
print(f"pertes de vue recomptées comme départs : {perdu.sum()}")
print(f"{'mois':>5} {'censure non informative':>24} {'pire cas':>10}")
for t in (12, 24, 36):
    print(f"{'t':>5} {surv_at(t, tj, S):>24.3f} {surv_at(t, tj_p, S_p):>10.3f}")
chi_pire = logrank(y, d_pire, c["offre_bienvenue"])[2]
print(f"log-rank offre / sans offre, pire cas : chi2 = {chi_pire:.1f} (standard : 35.5)")
```

```
pertes de vue recomptées comme départs : 363
mois   censure non informative   pire cas
12     0.825                    0.753
24     0.634                    0.526
36     0.453                    0.341
log-rank offre / sans offre, pire cas : chi2 = 27.4 (standard : 35.5)
```

À 36 mois, la survie est de **45 %** sous l'hypothèse standard et de **34 %** dans le pire cas : l'incertitude

liée à nos 363 pertes de vue pèse onze points, bien plus que l'incertitude statistique (l'intervalle à 95 % mesurait moins de trois points de demi-largeur). La vraie survie se situe entre les deux ; elle est plus proche de la première si les pertes de vue sont réellement indépendantes du risque de départ, ce qui est le cas dans notre simulation. Et la **conclusion sur l'offre** survit à l'épreuve : le test du log-rank reste très significatif même dans le pire cas (dernière ligne :  $\chi^2 = 27,4$  contre 35,5), ce qui est cohérent avec le fait que, dans notre simulation, les pertes de vue ne dépendent pas de l'offre. Ce genre de vérification est précieux : il sépare les résultats **robustes** (la conclusion ne bouge pas) des chiffres **fragiles** (le niveau de survie, lui, dépend de l'hypothèse).

#### À retenir

- **Kaplan-Meier** :  $\hat{S}(t) = \prod_{t_j \leq t} (1 - d_j/n_j)$ , où  $n_j$  est l'ensemble à risque (les censurés y restent jusqu'à leur censure, puis en sortent). C'est l'estimateur du maximum de vraisemblance **non paramétrique**.
- **Greenwood** :  $\text{Var } \hat{S} \approx \hat{S}^2 \sum d_j/[n_j(n_j - d_j)]$  ; on préfère les intervalles **log-log**, qui restent dans  $[0, 1]$ .
- La **médiane** se lit sur la courbe ; la **durée moyenne** n'est pas définie si la courbe ne descend pas à 0, d'où la **RMST** (aire jusqu'à un horizon  $\tau$ ).
- Le **log-rank** compare les groupes en cumulant, instant par instant, départs observés et attendus sous  $H_0$  ; il est le plus puissant sous risques proportionnels.
- Une **entrée tardive** impose de n'inclure un client dans l'ensemble à risque qu'**après** son entrée ; l'oublier surestime la survie.
- La queue de la courbe est fragile (peu de clients à risque), et la censure non informative est une hypothèse qu'on peut éprouver par une analyse de sensibilité.

## 5.3 Le modèle de Cox à risques proportionnels

**Intuition.** Kaplan-Meier sait comparer deux ou trois groupes, mais pas dire « à canal et à offre égaux, que change un an de plus d'âge ? ». Il faut un modèle de **régression**. David Cox (1972) a eu l'idée géniale de modéliser le **risque instantané** plutôt que la durée, en ne supposant que ceci : le risque d'un client est le risque d'un client « de référence », multiplié par un facteur qui dépend de ses caractéristiques, **le même à tout âge de la relation**. La forme du risque de référence reste libre, ce qui rend le modèle très robuste. Et miracle : on peut estimer les facteurs *sans jamais estimer ce risque de référence*.

### 5.3.1 Le modèle

Pour un client de caractéristiques  $x = (x_1, \dots, x_p)$ , le **modèle de Cox** s'écrit

$$h(t | x) = h_0(t) \exp(\beta_1 x_1 + \dots + \beta_p x_p) = h_0(t) e^{x^\top \beta}$$

- $h_0(t)$  est le **risque de base** : le risque d'un client dont toutes les caractéristiques valent 0. **On ne lui impose aucune forme** : c'est la partie « non paramétrique » du modèle. Le modèle est donc dit **semi-paramétrique**.
- $e^{x^\top \beta}$  est le **facteur multiplicatif** : il ne dépend pas de  $t$ .

**Comment lire un coefficient.** Prenons deux clients identiques sauf pour la variable  $x_j$ , qui vaut  $a + 1$  pour l'un et  $a$  pour l'autre. Le rapport de leurs risques est

$$\frac{h(t | x_j = a + 1)}{h(t | x_j = a)} = \frac{h_0(t) e^{\beta_j(a+1) + \dots}}{h_0(t) e^{\beta_j a + \dots}} = e^{\beta_j}.$$

Le terme  $h_0(t)$  s'est **simplifié** : ce rapport ne dépend pas de  $t$ . C'est le **rapport de risques** (en anglais *hazard ratio*, HR) :

- $HR = e^{\beta_j} < 1$  : la variable **réduit** le risque de partir à tout instant (protectrice) ;
- $HR > 1$  : elle l'**augmente** ;
- $HR = 1$  ( $\beta_j = 0$ ) : aucun effet.

**Exemple chiffré.** Si  $HR = 0,67$  pour l'offre de bienvenue, cela signifie : à *n'importe quel âge de la relation, parmi deux clients identiques encore là, celui qui a eu l'offre a un risque instantané de partir égal à 67 % de celui qui ne l'a pas eu*, soit un risque réduit d'un tiers.

**Un rapport de risques n'est pas un rapport de probabilités.** « Le risque est réduit d'un tiers » ne veut pas dire « un tiers de départs en moins sur 3 ans » : le risque instantané et la proportion cumulée de départs sont deux choses différentes. C'est la **courbe de survie** (section 5.3.7) qui traduit le HR en pourcentages de clients conservés.

### 5.3.2 La vraisemblance partielle : estimer $\beta$ sans connaître $h_0$

Comment estimer  $\beta$  si  $h_0(t)$  est inconnue ? L'idée de Cox est d'utiliser un **argument conditionnel**, que l'on comprend sur un exemple.

Reprenons nos huit clients (section 5.1.1) et supposons que C, E, F et G aient reçu l'offre ( $x = 1$ ) et A, B, D, H non ( $x = 0$ ). À l'instant  $t = 3$ , **un** départ a lieu, et les huit clients sont à risque. Sachant qu'un départ a lieu à cet instant, quelle est la probabilité que ce soit A ? Chaque client  $k$  à risque a un risque  $h_0(3)e^{\beta x_k}$  : la probabilité que ce soit A est sa part du risque total,

$$\frac{h_0(3) e^{\beta x_A}}{\sum_{k \in \text{à risque}} h_0(3) e^{\beta x_k}} = \frac{e^{\beta x_A}}{\sum_{k \in \text{à risque}} e^{\beta x_k}}.$$

**Le risque de base  $h_0(3)$  s'est simplifié**, en haut comme en bas ! Il en est de même à chaque instant de départ. En multipliant ces probabilités conditionnelles sur tous les départs, on obtient la **vraisemblance partielle** de Cox :

$$L_p(\beta) = \prod_{i: \delta_i=1} \frac{e^{x_i^\top \beta}}{\sum_{k \in R_i} e^{x_k^\top \beta}} \quad \ell_p(\beta) = \sum_{i: \delta_i=1} [x_i^\top \beta - \ln \sum_{k \in R_i} e^{x_k^\top \beta}]$$

où  $R_i$  est l'**ensemble à risque** à l'instant du départ du client  $i$  (ceux encore sous observation juste avant). Les clients censurés n'ont pas de facteur propre, mais ils figurent dans les ensembles à risque tant qu'ils sont observés.

À la main, pour nos huit clients (en notant  $u = e^\beta$ ) :

| Départ        | $t$ | Ensemble à risque                    | Facteur de $L_p$   |
|---------------|-----|--------------------------------------|--------------------|
| A ( $x = 0$ ) | 3   | A à H : 4 avec offre, 4 sans         | $\frac{1}{4 + 4u}$ |
| B ( $x = 0$ ) | 5   | B, C, D, E, F, G, H : 4 avec, 3 sans | $\frac{1}{3 + 4u}$ |
| D ( $x = 0$ ) | 8   | D, E, F, G, H : 3 avec, 2 sans       | $\frac{1}{2 + 3u}$ |
| F ( $x = 1$ ) | 12  | F, G, H : 2 avec, 1 sans             | $\frac{u}{1 + 2u}$ |



D'où  $\ell_p(\beta) = -\ln(4 + 4u) - \ln(3 + 4u) - \ln(2 + 3u) + \beta - \ln(1 + 2u)$ . Où est son maximum ? Pour le trouver, on dérive. Le **score** et l'**information** ont une interprétation simple (cas d'une seule variable) :

$$U(\beta) = \ell'_p(\beta) = \sum_{i: \delta_i=1} [x_i - \bar{x}_{R_i}(\beta)], \quad I(\beta) = -\ell''_p(\beta) = \sum_{i: \delta_i=1} \text{Var}_{R_i}(x; \beta),$$

où  $\bar{x}_{R_i}(\beta) = \sum_{k \in R_i} x_k e^{\beta x_k} / \sum_{k \in R_i} e^{\beta x_k}$  est la **moyenne pondérée** de  $x$  dans l'ensemble à risque (les poids sont les risques relatifs  $e^{\beta x_k}$ ), et  $\text{Var}_{R_i}$  la variance pondérée correspondante. Le score compare, à chaque départ, la valeur de  $x$  du client parti à la valeur « attendue » : si ceux qui partent ont systématiquement un  $x$  plus grand que la moyenne de ceux qui restent,  $\beta$  doit être positif. C'est ce qu'on appelle les **résidus de Schoenfeld**  $x_i - \bar{x}_{R_i}$ , que nous retrouverons au 5.3.5.

```
import numpy as np
import pandas as pd
from scipy import stats
from scipy.optimize import minimize_scalar

y8 = np.array([3, 5, 6, 8, 10, 12, 14, 14], float)
d8 = np.array([1, 1, 0, 1, 0, 1, 0, 0])
x8 = np.array([0, 0, 1, 0, 1, 1, 1, 0], float)      # 1 = a reçu l'offre (C, E, F, G)

def log_vrais_partielle_8(beta):
    u = np.exp(beta)
    return -np.log(4 + 4 * u) - np.log(3 + 4 * u) - np.log(2 + 3 * u) + beta - np.log(1 + 2 * u)

def score_info(beta, y, d, x):
    """Score U(beta) et information I(beta) d'une covariable : boucle sur les départs."""
    U = I = 0.0
    for i in np.where(d == 1)[0]:
        R = y >= y[i]                                # ensemble à risque
        w = np.exp(beta * x[R])
        m = (w * x[R]).sum() / w.sum()                # moyenne pondérée de x dans R
        v = (w * x[R] ** 2).sum() / w.sum() - m**2    # variance pondérée
        U += x[i] - m
        I += v
    return U, I

U0, I0 = score_info(0.0, y8, d8, x8)
print(f"en beta = 0 : score U = {U0:.4f}, information I = {I0:.4f}")
print(f"statistique de score U²/I = {U0**2 / I0:.3f} (comparer au chi2 du log-rank des huit clients  
↪ : 1.87)")

beta = 0.0                                           # algorithme de Newton-Raphson : beta <- beta
↪ + U/I
for it in range(6):
    U, I = score_info(beta, y8, d8, x8)
    print(f"itération {it} : beta = {beta:8.5f}    log-vraisemblance partielle =  
↪ {log_vrais_partielle_8(beta):.5f}")
    beta += U / I
opt = minimize_scalar(lambda b: -log_vrais_partielle_8(b), bounds=(-6, 3), method="bounded")
print(f"optimiseur : beta = {opt.x:.5f} -> HR = {np.exp(opt.x):.3f}")

en beta = 0 : score U = -1.3381, information I = 0.9571
statistique de score U²/I = 1.871 (comparer au chi2 du log-rank des huit clients : 1.87)
itération 0 : beta = 0.00000    log-vraisemblance partielle = -6.73340
itération 1 : beta = -1.39804    log-vraisemblance partielle = -5.79849
itération 2 : beta = -1.45965    log-vraisemblance partielle = -5.79702
itération 3 : beta = -1.46056    log-vraisemblance partielle = -5.79702
itération 4 : beta = -1.46057    log-vraisemblance partielle = -5.79702
itération 5 : beta = -1.46057    log-vraisemblance partielle = -5.79702
optimiseur : beta = -1.46056 -> HR = 0.232
```

Trois enseignements.

1. La **méthode de Newton** converge en quatre itérations (le cinquième chiffre décimal ne bouge plus ensuite) vers  $\hat{\beta} \approx -1,46$ , donc  $\widehat{HR} = e^{-1,46} \approx 0,23$  : l'offre semblerait diviser le risque par quatre. Mais ce n'est qu'un exemple jouet à huit clients ; l'erreur standard est de  $1/\sqrt{I} \approx 1,16$ , de sorte qu'un intervalle de confiance irait de  $e^{-1,46-1,96 \times 1,16} \approx 0,02$  à  $e^{-1,46+1,96 \times 1,16} \approx 2,3$ .
2. La statistique de score en  $\beta = 0$  vaut exactement 1,87 : c'est le **chi-deux du test du log-rank** de la section 5.2.5.
3. Ce n'est pas un hasard.

**Le test du log-rank est le test de score du modèle de Cox.** En  $\beta = 0$ ,  $e^{\beta x_k} = 1$  pour tous : la moyenne pondérée  $\bar{x}_{R_i}(0)$  est la **proportion de clients du groupe 1** dans l'ensemble à risque ( $n_{1j}/n_j$ ), et le score vaut  $\sum_j (d_{1j} - d_j n_{1j}/n_j) = O_1 - E_1$ . L'information est la variance de la loi hypergéométrique. Autrement dit, tester  $H_0 : \beta = 0$  par le score revient à calculer le log-rank. Le test du log-rank est donc exactement ce que « donne » le modèle de Cox lorsqu'on n'a que la variable de groupe.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
bs = np.linspace(-4.5, 2.0, 300)
ll = np.array([log_vrais_partielle_8(b) for b in bs])
fig, ax = plt.subplots(figsize=(6.4, 3.8))
ax.plot(bs, ll, color=BLEU, lw=2)
ax.axvline(opt.x, color=ORANGE, lw=1.4, ls="--")
ax.axvline(0, color="#898781", lw=1)
ax.scatter([opt.x, 0], [log_vrais_partielle_8(opt.x), log_vrais_partielle_8(0)], color=[ORANGE,
    "#898781"], zorder=3)
ax.set_ylim(top=log_vrais_partielle_8(opt.x) + 0.45)
ax.text(opt.x - 0.12, log_vrais_partielle_8(opt.x) + 0.12, "maximum :  $\hat{\beta} = %.2f$ " % opt.x, color=ORANGE,
    fontsize=10, ha="right")
ax.text(0.3, log_vrais_partielle_8(0) + 0.5, " $\beta = 0$  (aucun effet)", color="#52514e", fontsize=10)
ax.set_xlabel(" $\beta$  : effet de l'offre sur le log du risque")
ax.set_ylabel("log-vraisemblance partielle")
plt.tight_layout()
plt.savefig("figures/ch05-cox-vraisemblance.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

La courbe est **plate** autour de son maximum : beaucoup de valeurs de  $\beta$  sont presque aussi vraisemblables. C'est la traduction graphique de l'erreur standard élevée.

### 5.3.3 Ajuster le modèle sur les 2 000 clients

Pour plusieurs variables, les mêmes formules s'écrivent avec des vecteurs : le score est  $U(\beta) = \sum_i [x_i - \bar{x}_{R_i}]$  (un vecteur) et l'information  $I(\beta)$  est la matrice (variances et covariances pondérées dans les ensembles à risque). L'estimateur  $\hat{\beta}$  est asymptotiquement normal de matrice de covariance  $I(\hat{\beta})^{-1}$ , comme tout estimateur du maximum de vraisemblance (volume I, section 3.2).

Voici une implémentation complète, **vectorisée** (on trie les clients par durée décroissante pour obtenir toutes les sommes sur les ensembles à risque par des sommes cumulées). Elle traite les départs simultanés (*ex aequo*) par la correction de **Breslow** : voir 5.3.4.

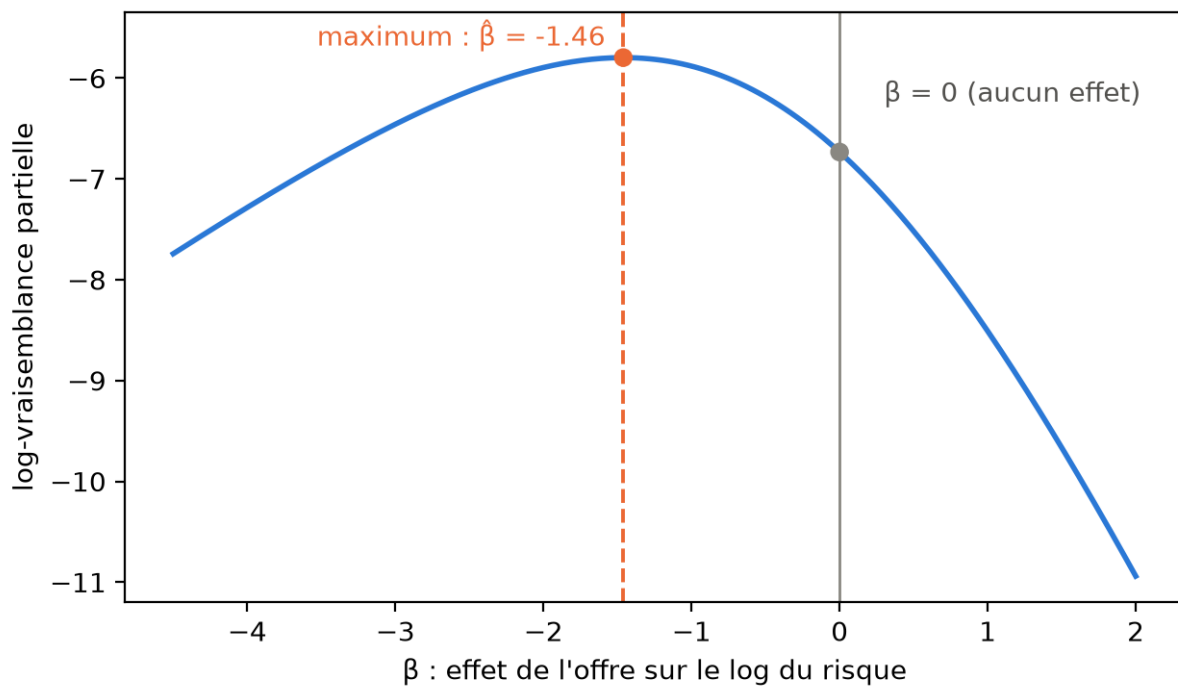


Figure 56: Log-vraisemblance partielle de Cox pour les huit clients, en fonction du coefficient  $\beta$  de l'offre : le maximum se trouve à  $\beta \approx -1,46$  et la courbe est très plate autour, signe d'une grande incertitude.

```
from scipy.optimize import minimize

def cox_ph(y, d, X):
    """Modèle de Cox par maximum de vraisemblance partielle (ex aequo : Breslow).
    y : durées, d : 1 = départ, X : matrice n x p. Retourne un dictionnaire."""
    y, d, X = np.asarray(y, float), np.asarray(d, int), np.asarray(X, float)
    ordre = np.argsort(-y, kind="stable")
    ys, Xs = y[ordre], X[ordre]
    tj = np.unique(y[d == 1])
    rang = np.searchsorted(-ys, -tj, side="right")
    # clients triés par durée DÉCROISSANTE
    # instants de départ distincts
    # nb de clients avec y >= t_j : leur
    ensemble à risque
    dj = np.array([np.sum((y == t) & (d == 1)) for t in tj])
    sx = np.array([X[(y == t) & (d == 1)].sum(axis=0) for t in tj])
    # départs à chaque instant

    def sommes(beta):
        w = np.exp(Xs @ beta)
        s0 = np.cumsum(w)[rang - 1]
        # somme des risques
        relatifs dans R_j
        s1 = np.cumsum(w[:, None] * Xs, axis=0)[rang - 1]
        s2 = np.cumsum(w[:, None, None] * Xs[:, :, None] * Xs[:, None, :], axis=0)[rang - 1]
        # somme de w x
        return s0, s1, s2

    def moins_ll(beta):
        s0, _, _ = sommes(beta)
        return -(sx @ beta).sum() - (dj * np.log(s0)).sum()

    def moins_score(beta):
        s0, s1, _ = sommes(beta)
        return -(sx.sum(axis=0) - (dj[:, None] * s1 / s0[:, None])).sum(axis=0)

    res = minimize(moins_ll, np.zeros(X.shape[1]), jac=moins_score, method="BFGS")
    beta = res.x
    s0, s1, s2 = sommes(beta)
```

```

info = (dj[:, None, None] * (s2 / s0[:, None, None] - s1[:, :, None] * s1[:, None, :]) / s0[:,
↪ None, None] ** 2)).sum(axis=0)
return {"beta": beta, "cov": np.linalg.inv(info), "info": info, "ll": -res.fun,
        "ll0": -moins_ll(np.zeros(X.shape[1])), "tj": tj, "dj": dj, "s0": s0}

c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
X = pd.get_dummies(c[["offre_bienvenue", "age", "canal_acquisition"]], drop_first=True, dtype=float)
print("colonnes :", list(X.columns), "(référence : Boutique)")
fit = cox_ph(y, d, X.to_numpy())

se = np.sqrt(np.diag(fit["cov"]))
z = fit["beta"] / se
tableau = pd.DataFrame({"coef": fit["beta"], "ET": se, "z": z, "p": 2 * stats.norm.sf(np.abs(z)),
                        "HR": np.exp(fit["beta"]), "HR bas": np.exp(fit["beta"] - 1.96 * se), "HR
↪ haut": np.exp(fit["beta"] + 1.96 * se)}, index=X.columns)
print(tableau.round(4).to_string())

```

```

colonnes : ['offre_bienvenue', 'age', 'canal_acquisition_Instagram', 'canal_acquisition_Site']
↪ (référence : Boutique)

```

|                             | coef    | ET     | z       | p      | HR     | HR bas | HR haut |
|-----------------------------|---------|--------|---------|--------|--------|--------|---------|
| offre_bienvenue             | -0.4061 | 0.0645 | -6.2926 | 0.0000 | 0.6662 | 0.5871 | 0.7561  |
| age                         | -0.0136 | 0.0030 | -4.4792 | 0.0000 | 0.9865 | 0.9806 | 0.9924  |
| canal_acquisition_Instagram | 0.6351  | 0.0842 | 7.5467  | 0.0000 | 1.8872 | 1.6002 | 2.2256  |
| canal_acquisition_Site      | 0.3258  | 0.0887 | 3.6735  | 0.0002 | 1.3851 | 1.1641 | 1.6481  |

Vérifions avec statsmodels (PHReg, option ties="breslow") et avec R (coxph), la référence :

```

from statsmodels.duration.hazard_regression import PHReg

ph = PHReg(y, X, status=d, ties="breslow").fit()
print("statsmodels : coef", np.round(ph.params, 5), "| ET", np.round(ph.bse, 5))
print("à la main : coef", np.round(fit["beta"], 5), "| ET", np.round(se, 5))
print(f"log-vraisemblance partielle : à la main {fit['ll']:.3f} | statsmodels {ph.llf:.3f}")

```

```

statsmodels : coef [-0.40612 -0.01363  0.63508  0.3258 ] | ET [0.06454 0.00304 0.08415 0.08869]
à la main : coef [-0.40612 -0.01363  0.63508  0.3258 ] | ET [0.06454 0.00304 0.08415 0.08869]
log-vraisemblance partielle : à la main -6512.361 | statsmodels -6512.361

```

```

clients$canal_acquisition <- factor(clients$canal_acquisition, levels = c("Boutique", "Instagram",
↪ "Site"))
cox_r <- coxph(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data = clients,
↪ ties = "breslow")
print(summary(cox_r))

```

Call:

```
coxph(formula = Surv(duree_mois, churn) ~ offre_bienvenue + age +
      canal_acquisition, data = clients, ties = "breslow")
```

n= 2000, number of events= 977

|                            | coef      | exp(coef) | se(coef) | z      | Pr(> z )     |
|----------------------------|-----------|-----------|----------|--------|--------------|
| offre_bienvenue            | -0.406119 | 0.666231  | 0.064540 | -6.293 | 3.12e-10 *** |
| age                        | -0.013635 | 0.986458  | 0.003044 | -4.479 | 7.49e-06 *** |
| canal_acquisitionInstagram | 0.635078  | 1.887170  | 0.084153 | 7.547  | 4.47e-14 *** |
| canal_acquisitionSite      | 0.325800  | 1.385138  | 0.088688 | 3.674  | 0.000239 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

|                            | exp(coef) | exp(-coef) | lower .95 | upper .95 |
|----------------------------|-----------|------------|-----------|-----------|
| offre_bienvenue            | 0.6662    | 1.5010     | 0.5871    | 0.7561    |
| age                        | 0.9865    | 1.0137     | 0.9806    | 0.9924    |
| canal_acquisitionInstagram | 1.8872    | 0.5299     | 1.6002    | 2.2256    |

```

canal_acquisitionSite      1.3851      0.7219      1.1641      1.6481

Concordance= 0.605 (se = 0.01 )
Likelihood ratio test= 112 on 4 df,  p=<2e-16
Wald test              = 110.3 on 4 df,  p=<2e-16
Score (logrank) test = 111.8 on 4 df,  p=<2e-16

```

Les trois calculs coïncident à la cinquième décimale. Lisons le tableau.

- **Offre de bienvenue** :  $\widehat{HR} \approx 0,67$  (IC95 : de 0,59 à 0,76). À âge et canal égaux, l'offre réduit le risque de départ d'environ **un tiers**, à n'importe quel âge de la relation. Comme l'offre est **attribuée au hasard**, l'interprétation causale est licite ; l'intervalle exclut nettement 1.
- **Âge** :  $\widehat{HR} \approx 0,986$  **par année** : chaque année d'âge de plus réduit le risque d'environ 1,4 %. Pour parler à Yasmine, on exprime l'effet par tranche : *dix ans de plus* correspondent à  $HR = e^{10\hat{\beta}}$ , soit environ 0,87 (calculé ci-dessous).
- **Canal** : par rapport à la boutique (référence), un client arrivé par **Instagram** a un risque environ **1,9 fois plus élevé** et un client arrivé par le **site**, environ **1,4 fois**. C'est cohérent avec les courbes de la section 5.2.

```

i_age = list(X.columns).index("age")
b, s_ = fit["beta"][i_age], se[i_age]
print(f"âge, +10 ans : HR = {np.exp(10 * b):.3f} (IC95 : {np.exp(10 * (b - 1.96 * s_)):.3f} à
↪ {np.exp(10 * (b + 1.96 * s_)):.3f})")

# Tests globaux : rapport de vraisemblance, Wald, score (comparer avec R)
lr = 2 * (fit["ll"] - fit["ll0"])
wald = fit["beta"] @ fit["info"] @ fit["beta"]
print(f"test global (4 ddl) : rapport de vraisemblance = {lr:.2f}, p = {stats.chi2.sf(lr, 4):.1e} ;
↪ Wald = {wald:.2f}")

```

```

âge, +10 ans : HR = 0.873 (IC95 : 0.822 à 0.926)
test global (4 ddl) : rapport de vraisemblance = 112.01, p = 2.7e-23 ; Wald = 110.28

```

**Les trois tests classiques.** Pour un modèle de maximum de vraisemblance, on peut tester  $H_0 : \beta = 0$  de trois façons : par le **rapport de vraisemblance** (comparer  $\ell_p(\hat{\beta})$  à  $\ell_p(0)$ ), par le test de **Wald** ( $\hat{\beta}^T I \hat{\beta}$ , valeur de  $z^2$  pour un seul coefficient), ou par le test de **score** (évaluer le score en  $\beta = 0$  : c'est le log-rank). Ils sont équivalents asymptotiquement et donnent ici des conclusions identiques ; R affiche les trois.

### 5.3.4 Les ex aequo : Breslow, Efron et les autres

Si deux clients partent exactement au même instant, la vraisemblance partielle ne dit pas dans quel ordre : les facteurs de la formule ne sont plus bien définis. Trois approches :

- **Breslow** : on traite tous les ex aequo comme si l'ensemble à risque était *le même* pour chacun (celui d'avant l'instant). Simple, mais approximative quand les ex aequo sont nombreux. C'est ce que nous avons codé.
- **Efron** : on retire progressivement les clients partis simultanément de l'ensemble à risque, en moyenne. Plus précise ; c'est le **défaut de R** (coxph) et de lifelines.
- **Exacte** : on somme sur tous les ordres possibles. Précise mais coûteuse quand les ex aequo sont nombreux.

Dans nos données, mesurées au centième de mois, il y a peu d'ex aequo (91 instants sur 882 comportent plusieurs départs) ; la différence est **négligeable** :

```

efron = PHReg(y, X, status=d, ties="efron").fit()
print("Breslow :", np.round(ph.params, 5))

```

```
print("Efron :", np.round(efron.params, 5))
```

```
Breslow : [-0.40612 -0.01363  0.63508  0.3258 ]
Efron    : [-0.40616 -0.01364  0.63513  0.32584]
```

Mais voyons ce qui se passe quand on **arrondit** les durées au mois supérieur, comme le ferait une base de données qui ne stocke que des mois entiers : les ex aequo deviennent massifs.

```
y_mois = np.ceil(y)
print("départs :", int(d.sum()), "| instants distincts après arrondi :", len(np.unique(y_mois[d ==
↪ 1])))
b_b = PHReg(y_mois, X, status=d, ties="breslow").fit().params
b_e = PHReg(y_mois, X, status=d, ties="efron").fit().params
print(pd.DataFrame({"Breslow": b_b, "Efron": b_e, "écart relatif (%)": 100 * (b_b - b_e) /
↪ np.abs(b_e)}, index=X.columns).round(4).to_string())
```

```
départs : 977 | instants distincts après arrondi : 75
```

|                             | Breslow | Efron   | écart relatif (%) |
|-----------------------------|---------|---------|-------------------|
| offre_bienvenue             | -0.4030 | -0.4075 | 1.1077            |
| age                         | -0.0135 | -0.0136 | 1.1785            |
| canal_acquisition_Instagram | 0.6260  | 0.6327  | -1.0456           |
| canal_acquisition_Site      | 0.3211  | 0.3240  | -0.8909           |

Après arrondi, les 977 départs se répartissent sur seulement 75 instants (une douzaine d'ex aequo par instant en moyenne). Breslow tire alors chaque coefficient **vers zéro** (son coefficient est plus petit en valeur absolue que celui d'Efron, de l'ordre de 1 % ici), alors que sans arrondi, les deux méthodes étaient confondues à la quatrième décimale. L'écart reste modeste parce que les ex aequo sont, ici, une petite fraction de chaque ensemble à risque (quelques dizaines sur plusieurs centaines) ; il deviendrait important si la fraction de clients partant à chaque instant était grande. La règle pratique : **utilisez Efron par défaut**.

### 5.3.5 Vérifier l'hypothèse : les risques sont-ils vraiment proportionnels ?

Tout le modèle repose sur *un seul* pari : l'effet d'une variable est **le même à tout âge de la relation**. L'offre de bienvenue divise-t-elle le risque par 0,67 aussi bien le premier mois que la troisième année ? Si ce n'est pas le cas, le coefficient estimé n'est qu'une **moyenne**, potentiellement trompeuse. Il faut **vérifier**.

(a) **Le graphique log-log**. Si les risques sont proportionnels,  $H(t | x) = H_0(t)e^{x^T \beta}$ , donc  $\ln(-\ln S(t | x)) = \ln H_0(t) + x^T \beta$  : en traçant  $\ln(-\ln \hat{S})$  contre  $\ln t$  pour plusieurs groupes, on doit obtenir des courbes **parallèles** (décalées de  $x^T \beta$ ).

(b) **Les résidus de Schoenfeld**. Le résidu du client  $i$  parti à  $t_i$  est  $r_i = x_i - \bar{x}_{R_i}(\hat{\beta})$  (5.3.2). Si l'effet de  $x_j$  était constant, ces résidus n'auraient aucune tendance en fonction du temps. S'ils croissent avec  $t$ , l'effet réel de  $x_j$  augmente avec le temps ; s'ils décroissent, il diminue. Grambsch et Therneau transforment cette idée en **test de score** : on ajoute au modèle un terme  $\theta_j x_j g(t)$  où  $g$  est une fonction du temps (ici, le **rang** de la durée), et l'on teste  $\theta_j = 0$  au point  $(\hat{\beta}, \theta = 0)$ . Les trois ingrédients se calculent avec les mêmes sommes que précédemment : le score  $U_{\theta_j} = \sum_i g(t_i) r_{ij}$ , et la matrice d'information du modèle étendu.

```
def test_ph(y, d, X, fit):
    """Test de score de proportionnalité des risques (Grambsch-Therneau), g(t) = rang de la durée."""
    y, d, X, beta, V = np.asarray(y, float), np.asarray(d, int), np.asarray(X, float), fit["beta"],
    ↪ fit["cov"]
    ev = np.where(d == 1)[0]
    g = stats.rankdata(y)[ev]
    g = g - g.mean()
    w = np.exp(X @ beta)
```

```

ordre = np.argsort(-y, kind="stable")
ys, Xs, ws = y[ordre], X[ordre], w[ordre]
pos = np.searchsorted(-ys, -y[ev], side="right") - 1           # fin de l'ensemble à risque
↳ de chaque départ
s0 = np.cumsum(ws)[pos]
m1 = np.cumsum(ws[:, None] * Xs, axis=0)[pos] / s0[:, None]
cov = np.cumsum(ws[:, None, None] * Xs[:, :, None] * Xs[:, None, :], axis=0)[pos] / s0[:, None,
↳ None] - m1[:, :, None] * m1[:, None, :]
R = X[ev] - m1           # résidus de Schoenfeld (non
↳ normalisés)
U = (g[:, None] * R).sum(axis=0)           # score des coefficients
↳ dépendant du temps
Ibt = (g[:, None, None] * cov).sum(axis=0)
Itt = (g[:, None, None] ** 2 * cov).sum(axis=0)
Schur = Itt - Ibt.T @ V @ Ibt           # information « nette » (le
↳ beta est re-estimé)
chi2_j = np.array([U[j] ** 2 / Schur[j, j] for j in range(X.shape[1])])
chi2_glob = float(U @ np.linalg.solve(Schur, U))
return chi2_j, chi2_glob

chi2_j, chi2_glob = test_ph(y, d, X, fit)
res_ph = pd.DataFrame({"chi2": chi2_j, "p": stats.chi2.sf(chi2_j, 1)}, index=X.columns)
res_ph.loc["GLOBAL (4 ddl)"] = [chi2_glob, stats.chi2.sf(chi2_glob, 4)]
print(res_ph.round(4).to_string())

```

|                             | chi2   | p      |
|-----------------------------|--------|--------|
| offre_bienvenue             | 0.5588 | 0.4548 |
| age                         | 0.7292 | 0.3931 |
| canal_acquisition_Instagram | 2.2063 | 0.1375 |
| canal_acquisition_Site      | 0.0146 | 0.9037 |
| GLOBAL (4 ddl)              | 4.9502 | 0.2924 |

```
print(cox.zph(cox_r, transform = "rank", terms = FALSE))
```

|                            | chisq  | df | p    |
|----------------------------|--------|----|------|
| offre_bienvenue            | 0.5588 | 1  | 0.45 |
| age                        | 0.7292 | 1  | 0.39 |
| canal_acquisitionInstagram | 2.2063 | 1  | 0.14 |
| canal_acquisitionSite      | 0.0146 | 1  | 0.90 |
| GLOBAL                     | 4.9502 | 4  | 0.29 |

Notre test à la main et `cox.zph` de R donnent les **mêmes** statistiques. Aucune p-valeur n'est petite : la proportionnalité des risques est **plausible**. Ce n'est pas une surprise : les données ont été simulées avec un modèle de Weibull, qui est à risques proportionnels (nous le démontrerons au 5.4).

Que se passe-t-il quand l'hypothèse est **fausse** ? Simulons deux groupes de 750 clients : dans le groupe A, le risque de partir **diminue** avec le temps (Weibull de forme 0,8) ; dans le groupe B, il **augmente** (forme 1,8). Leurs risques se croisent.

```

rng = np.random.default_rng(54)
n = 1500
g = np.repeat([0, 1], n // 2)
T = np.where(g == 0, 30 * rng.weibull(0.8, n), 40 * rng.weibull(1.8, n))
C = rng.uniform(10, 80, n)
yc, dc = np.minimum(T, C), (T <= C).astype(int)
Xc = g[:, None].astype(float)

fit_c = cox_ph(yc, dc, Xc)
chi2_c, _ = test_ph(yc, dc, Xc, fit_c)
print(f"Cox : HR (B contre A) = {np.exp(fit_c['beta'][0]):.2f} -> « B a 27 % de risque en moins »")
print(f"test de proportionnalité : chi2 = {chi2_c[0]:.1f}, p = {stats.chi2.sf(chi2_c[0], 1):.1e}")

```



```
# La vérité : on compare le risque dans les 25 premiers mois puis après
def hr_période(debut, fin):
    garde = yc > debut
    yy = np.minimum(yc[garde], fin)
    dd = dc[garde] * (yc[garde] <= fin)
    ph_ = PHReg(yy, Xc[garde], status=dd, entry=np.full(garde.sum(), float(debut)),
    ↪ ties="efron").fit()
    ↪ return float(np.exp(ph_.params[0]))
print(f"HR sur 0-25 mois : {hr_période(0, 25):.2f} | HR après 25 mois : {hr_période(25,
    ↪ 1000):.2f}")
```

```
Cox : HR (B contre A) = 0.73 -> « B a 27 % de risque en moins »
test de proportionnalité : chi2 = 216.3, p = 5.7e-49
HR sur 0-25 mois : 0.43 | HR après 25 mois : 2.20
```

Le HR global (0,73) est une moyenne qui ne décrit **aucune période réelle** : le groupe B est bien plus protégé au début (HR  $\approx 0,43$  sur les 25 premiers mois), puis devient moins bon que A (HR  $\approx 2,2$  ensuite). Le test de proportionnalité le détecte immédiatement ( $\chi^2$  très grand). Visualisons-le, avec les deux graphiques (courbes de survie qui se croisent ; graphique log-log) :

```
def courbe_loglog(ax, y_, d_, masque, couleur, nom):
    t_, n_, dj_, S_, g_ = kaplan_meier(y_[masque], d_[masque])
    garde = (S_ > 0) & (S_ < 1) & (t_ > 3) # on écarte les tout premiers mois (très peu de
    ↪ départs)
    ax.plot(np.log(t_[garde]), np.log(-np.log(S_[garde])), color=couleur, lw=2, label=nom)

fig, axes = plt.subplots(1, 3, figsize=(13, 3.9))
# (1) Clients : les trois canaux, courbes log-log presque parallèles
for nom, couleur in [("Boutique", AQUA), ("Site", BLEU), ("Instagram", ORANGE)]:
    courbe_loglog(axes[0], y, d, (c["canal_acquisition"] == nom).to_numpy(), couleur, nom)
axes[0].set_title("Clients : les risques sont proportionnels")
# (2) et (3) Groupes simulés qui se croisent
for k_, couleur, nom in [(0, VIOLET, "groupe A (risque décroissant)", (1, AQUA, "groupe B (risque
    ↪ croissant)"))]:
    t_, n_, dj_, S_, g_ = kaplan_meier(yc[g == k_], dc[g == k_])
    axes[1].step(np.concatenate([[0], t_]), np.concatenate([[1], S_]), where="post", color=couleur,
    ↪ lw=2, label=nom)
    courbe_loglog(axes[2], yc, dc, g == k_, couleur, nom)
axes[1].set_title("Simulé : les courbes se croisent")
axes[2].set_title("Simulé : log-log non parallèles")
axes[0].set_xlabel("log(mois)"); axes[2].set_xlabel("log(mois)"); axes[1].set_xlabel("mois")
axes[0].set_ylabel("log(-log S(t))"); axes[1].set_ylabel("survie S(t)")
for ax in axes:
    ax.legend(frameon=False, fontsize=8, loc="best")
plt.tight_layout()
plt.savefig("figures/ch05-cox-loglog.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**Que faire quand les risques ne sont pas proportionnels ?** Plusieurs remèdes, du plus simple au plus fin :

1. **Stratifier** sur la variable fautive : chaque strate (par exemple chaque canal) a son propre risque de base  $h_{0s}(t)$ , mais les autres coefficients sont communs. On n'obtient plus de coefficient pour la variable de stratification, mais on se libère de l'hypothèse pour elle.
2. **Estimer un effet qui change avec le temps** :  $\beta_j(t) = \beta_j + \theta_j g(t)$ , ou un coefficient différent par période (comme ci-dessus).
3. **Changer de résumé** : comparer des durées moyennes restreintes (5.2.4), qui n'exigent pas la proportionnalité.

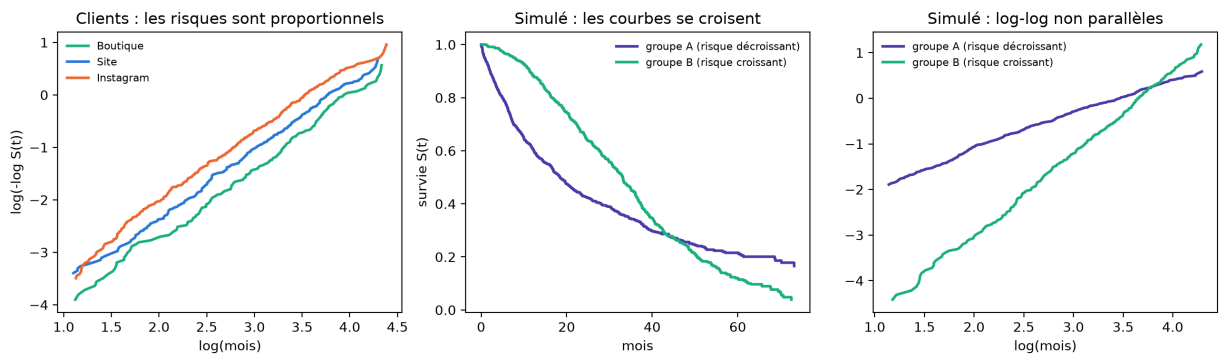


Figure 57: À gauche : graphique log-log des trois canaux de Dar Jasmin (courbes presque parallèles). Au centre et à droite : deux groupes simulés dont les risques se croisent (courbes de survie qui se coupent, courbes log-log non parallèles).

```
strat = PHReg(y, X[["offre_bienvenue", "age"]], status=d, strata=c["canal_acquisition"].to_numpy(),
  ↪ ties="efron").fit()
sans_strat = PHReg(y, X[["offre_bienvenue", "age"]], status=d, ties="efron").fit()
print(pd.DataFrame({"stratifié par canal": strat.params, "non stratifié": sans_strat.params},
  ↪ index=["offre_bienvenue", "age"]).round(4).to_string())
```

|                 | stratifié par canal | non stratifié |
|-----------------|---------------------|---------------|
| offre_bienvenue | -0.4074             | -0.3954       |
| age             | -0.0136             | -0.0120       |

Les coefficients de l'offre et de l'âge changent peu (offre :  $-0,407$  avec stratification,  $-0,395$  sans) ; la différence vient surtout de ce que le canal, facteur important du risque, est omis dans le second modèle.

**Un test non significatif ne prouve pas la proportionnalité.** Avec peu de données, le test manque de puissance ; avec beaucoup, il détecte des écarts sans importance pratique. Combinez le test avec le **graphique**, et posez-vous la question du **mécanisme** : est-il plausible qu'une offre de bienvenue ait le même effet relatif au premier mois et à la cinquième année ?

### 5.3.6 Variables qui changent avec le temps, et le piège de l'« immortalité »

Jusqu'ici, chaque variable était fixée à l'inscription. Mais certaines évoluent : un client reçoit une carte de fidélité au mois 12, passe à un abonnement premium au mois 20, etc. Le modèle de Cox accepte des **covariables dépendant du temps**  $x(t)$  ; on les représente en découpant la vie du client en **épisodes** (début, fin] sur chacun desquels la covariable est constante : c'est le format « **processus de comptage** ».

| Client                                         | Épisode | début | fin | carte de fidélité | départ à la fin de l'épisode ? |
|------------------------------------------------|---------|-------|-----|-------------------|--------------------------------|
| Z (a eu la carte au mois 12, parti au mois 30) | 1       | 0     | 12  | 0                 | non                            |
|                                                | 2       | 12    | 30  | 1                 | oui                            |
| W (jamais de carte, parti au mois 8)           | 1       | 0     | 8   | 0                 | oui                            |

Dans la vraisemblance partielle, à chaque instant de départ, un client est à risque **avec la valeur de sa covariable à cet instant-là**. La date de début de l'épisode joue le rôle d'une **entrée tardive** (comme en 5.2.6).

**Le biais d'immortalité.** Voici l'erreur la plus fréquente, et la plus trompeuse. Yasmine veut savoir si la **carte de fidélité** (remise aux clients encore là au mois 12) réduit les départs. Elle crée une colonne « a la carte » = 1 pour tous ceux qui l'ont **un jour** reçue, et ajuste un modèle de Cox. Problème : pour avoir la carte, il fallait **être encore client au mois 12**. Les porteurs de carte ont donc, par construction, **survécu 12 mois** : ils sont « immortels » pendant cette période. La variable prédit le futur depuis le passé.

Simulons une situation où la carte n'a **strictement aucun effet** (le risque de tout le monde est constant, 3 % par mois) :

```
rng = np.random.default_rng(53)
N = 3000
T = rng.exponential(1 / 0.03, N)           # durées vraies : risque constant 3 % par mois,
↳ indépendantes de la carte
C = rng.uniform(24, 60, N)
Y, D = np.minimum(T, C), (T <= C).astype(int)
carte = (Y > 12) & (rng.random(N) < 0.5)   # carte remise au mois 12 à la moitié des clients
↳ encore là
print("porteurs de carte :", int(carte.sum()), "sur", N)

# Analyse FAUSSE : la carte est traitée comme une variable fixe de départ
naif = PHReg(Y, carte.astype(float)[: , None], status=D, ties="efron").fit()
print(f"analyse naïve : HR = {np.exp(naif.params[0]):.2f} (IC95 {np.exp(naif.params[0] - 1.96 *
↳ naif.bse[0]):.2f} à {np.exp(naif.params[0] + 1.96 * naif.bse[0]):.2f})")

# Analyse JUSTE : variable dépendant du temps, deux épisodes pour les porteurs
porteurs = np.where(carte)[0]
debut = np.concatenate([np.zeros(N), np.full(len(porteurs), 12.0)])
fin = np.concatenate([np.where(carte, 12.0, Y), Y[porteurs]])
statut = np.concatenate([np.where(carte, 0, D), D[porteurs]])
x_carte = np.concatenate([np.zeros(N), np.ones(len(porteurs))])
juste = PHReg(fin, x_carte[: , None], status=statut, entry=debut, ties="efron").fit()
print(f"analyse correcte : HR = {np.exp(juste.params[0]):.2f} (IC95 {np.exp(juste.params[0] - 1.96 *
↳ juste.bse[0]):.2f} à {np.exp(juste.params[0] + 1.96 * juste.bse[0]):.2f})")

porteurs de carte : 995 sur 3000
analyse naïve : HR = 0.46 (IC95 0.41 à 0.50)
analyse correcte : HR = 1.03 (IC95 0.92 à 1.16)
```

L'analyse naïve « découvre » que la carte divise le risque par plus de deux ( $HR \approx 0,46$ ) alors qu'elle n'a **aucun effet** ; l'analyse correcte trouve un HR proche de 1, avec un intervalle de confiance qui contient 1. Cette erreur est célèbre (elle a faussé des études médicales entières, par exemple sur les bénéfices d'un traitement reçu « un jour »). **Règle** : toute variable dont la valeur n'est connue qu'*après* un certain temps de survie doit être traitée comme dépendant du temps.

### 5.3.7 Prédire : la courbe de survie d'un client donné

Le risque de base  $h_0$  a disparu de la vraisemblance, mais on peut l'estimer *après coup*. Pour un client de profil  $x$ ,  $S(t | x) = \exp(-H_0(t) e^{x^\top \beta})$ , où le **risque cumulé de base**  $H_0(t)$  est estimé par l'**estimateur de Breslow** :

$$\hat{H}_0(t) = \sum_{j: t_j \leq t} \frac{d_j}{\sum_{k \in R_j} e^{x_k^\top \hat{\beta}}}$$

C'est une version « pondérée » de l'estimateur de Nelson-Aalen  $\sum d_j/n_j$  (si  $\beta = 0$ , les deux coïncident). Chaque saut est le nombre de départs divisé par la **somme des risques relatifs** des clients à risque.

```
H0 = np.cumsum(fit["dj"] / fit["s0"]) # risque cumulé de base (Breslow)
def survie_pred(t, x):
    i = np.searchsorted(fit["tj"], t, side="right") - 1
    return float(np.exp(-(H0[i] if i >= 0 else 0.0) * np.exp(np.asarray(x, float) @ fit["beta"])))

profils = {"A : Instagram, 25 ans, sans offre": [0, 25, 1, 0],
           "B : Instagram, 25 ans, avec offre": [1, 25, 1, 0],
           "C : Boutique, 45 ans, avec offre": [1, 45, 0, 0]} # ordre des colonnes : offre,
           ↪ age, Instagram, Site
horizons = (12, 24, 36, 60)
print(pd.DataFrame({k: [survie_pred(t, v) for t in horizons] for k, v in profils.items()},
                    ↪ index=[f"{t} mois" for t in horizons]).round(4).T.to_string())
```

|                                   | 12 mois | 24 mois | 36 mois | 60 mois |
|-----------------------------------|---------|---------|---------|---------|
| A : Instagram, 25 ans, sans offre | 0.7108  | 0.4379  | 0.2298  | 0.0686  |
| B : Instagram, 25 ans, avec offre | 0.7966  | 0.5769  | 0.3754  | 0.1678  |
| C : Boutique, 45 ans, avec offre  | 0.9123  | 0.8010  | 0.6735  | 0.4867  |

```
nd <- data.frame(offre_bienvenue = c(0, 1, 1), age = c(25, 25, 45),
                 canal_acquisition = factor(c("Instagram", "Instagram", "Boutique"), levels =
                 ↪ levels(clients$canal_acquisition)))
print(round(t(summary(survfit(cox_r, newdata = nd), times = c(12, 24, 36, 60))$surv), 4))
```

|   | [,1]   | [,2]   | [,3]   | [,4]   |
|---|--------|--------|--------|--------|
| 1 | 0.7108 | 0.4379 | 0.2298 | 0.0686 |
| 2 | 0.7966 | 0.5769 | 0.3754 | 0.1678 |
| 3 | 0.9123 | 0.8010 | 0.6735 | 0.4867 |

Les prédictions à la main et celles de R sont identiques. Elles se traduisent en pourcentages de clients : un client arrivé par Instagram à 25 ans **sans offre** a environ 44 % de chances d'être encore là à 24 mois, contre 58 % **avec** l'offre ; un client de 45 ans arrivé par la boutique avec l'offre : 80 %. Dessinons ces courbes :

```
temps = np.linspace(0, 70, 300)
fig, ax = plt.subplots(figsize=(7.2, 4.2))
for (nom, x), couleur in zip(profils.items(), [ORANGE, VIOLET, AQUA]):
    ax.plot(temps, [survie_pred(t, x) for t in temps], color=couleur, lw=2, label=nom)
ax.axhline(0.5, color="#c3c2b7", lw=0.8)
ax.set_xlabel("mois depuis l'inscription")
ax.set_ylabel("probabilité d'être encore client")
ax.set_ylim(0, 1.02)
ax.legend(frameon=False, loc="lower left", fontsize=9)
plt.tight_layout()
plt.savefig("figures/ch05-cox-profils.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

### 5.3.8 Le modèle sépare-t-il bien les clients ? L'indice de concordance

Un modèle de survie est-il bon ? Une mesure très utilisée est l'**indice de concordance** de Harrell (C-index) : parmi tous les **couples comparables** de clients (c'est-à-dire ceux dont on sait lequel est parti le premier), quelle proportion le modèle ordonne-t-il correctement, c'est-à-dire en attribuant le **risque le plus élevé** à celui qui est parti le plus tôt ? Un couple  $(i, j)$  est comparable si  $y_i < y_j$  et que  $i$  est un départ observé (le client  $j$ , lui, peut être censuré : on sait qu'il est resté plus longtemps). C'est l'extension du « AUC » du volume I à des durées censurées : 0,5 = hasard, 1 =

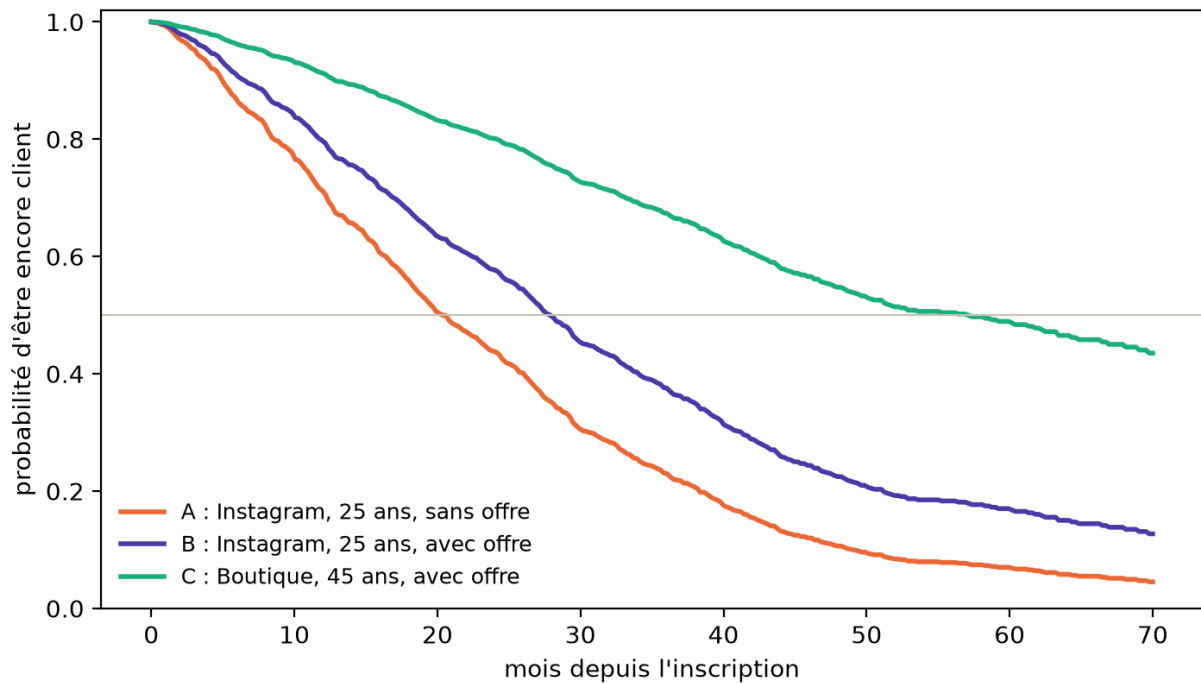


Figure 58: Courbes de survie prédites par le modèle de Cox pour trois profils de clients de Dar Jasmin.

tri parfait.

```
def indice_concordance(y, d, eta):
    """C de Harrell : pourcentage de couples comparables bien ordonnés (les ex aequo de score comptent
    ↪ pour 1/2)."""
    y, d, eta = np.asarray(y), np.asarray(d), np.asarray(eta)
    concordants = egalites = total = 0
    for i in np.where(d == 1)[0]:
        plus_long = y > y[i]
        total += plus_long.sum()
        concordants += (eta[plus_long] < eta[i]).sum()
    ↪ bien ordonné
        egalites += (eta[plus_long] == eta[i]).sum()
    return (concordants + 0.5 * egalites) / total

eta = X.to_numpy() @ fit["beta"]
print(f"indice de concordance : {indice_concordance(y, d, eta):.4f}")
```

```
indice de concordance : 0.6052
```

```
print(round(summary(coxph(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data =
    ↪ clients))$concordance, 4))
```

```
      C   se(C)
0.6052 0.0101
```

Le C-index est d'environ **0,605**, identique à celui de R. C'est **modeste**, et c'est normal : nous n'avons que trois variables, et surtout, la durée d'un client individuel comporte une très grande part d'aléa. Le modèle explique bien les **différences entre groupes** (le HR de 0,67 est très significatif) mais prédit mal le sort d'un client *particulier*. C'est une distinction capitale en pratique : *un résultat significatif n'est pas un modèle prédictif performant* (volume I, section 3.5.3).

### 5.3.9 Un second jeu de données : la récidive (Rossi)

Pour nous assurer que le modèle ne marche pas que sur des données que nous avons simulées, essayons un jeu de données **réel** classique, livré avec la bibliothèque `lifelines` (donc utilisable hors ligne). Il provient de l'étude de **Rossi, Berk et Lenihan (1980)** : 432 hommes libérés de prison dans le Maryland, suivis pendant 52 semaines ; l'événement est une **nouvelle arrestation**. La moitié, tirée au hasard, a reçu une **aide financière** (variable `fin`) pendant sa période de libération. Les autres variables : âge, origine (race), expérience professionnelle (`wexp`), marié (`mar`), libération conditionnelle (`paro`), nombre de condamnations antérieures (`prio`).

```
from lifelines.datasets import load_rossi
from lifelines import CoxPHFitter

rossi = load_rossi()
Xr = rossi.drop(columns=["week", "arrest"])
print(f"{len(rossi)} personnes, {int(rossi['arrest'].sum())} arrestations observées,
      ↳ {int((rossi['arrest'] == 0).sum())} censurées à 52 semaines")
sm_r = PHReg(rossi["week"], Xr, status=rossi["arrest"], ties="efron").fit()
ll_r = CoxPHFitter().fit(rossi, "week", "arrest")
comp = pd.DataFrame({"HR statsmodels": np.exp(sm_r.params), "HR lifelines":
      ↳ ll_r.hazard_ratios_.to_numpy(),
      "ET(log HR)": sm_r.bse, "p": sm_r.pvalues}, index=Xr.columns)
print(comp.round(3).to_string())
print(f"log-vraisemblance : statsmodels {sm_r.llf:.3f} | lifelines {ll_r.log_likelihood_:.3f}")
```

```
432 personnes, 114 arrestations observées, 318 censurées à 52 semaines
HR statsmodels  HR lifelines  ET(log HR)      p
fin             0.684         0.684         0.191  0.047
age             0.944         0.944         0.022  0.009
race            1.369         1.369         0.308  0.308
wexp            0.861         0.861         0.212  0.480
mar             0.648         0.648         0.382  0.256
paro            0.919         0.919         0.196  0.665
prio            1.096         1.096         0.029  0.001
log-vraisemblance : statsmodels -658.748 | lifelines -658.748
```

Les deux bibliothèques donnent les mêmes résultats. L'**aide financière** (`fin`, attribuée au hasard) réduit le risque de nouvelle arrestation d'environ **32 %** ( $HR \approx 0,68$ ), un effet à la limite de la significativité ( $p \approx 0,047$  : avec 114 arrestations seulement, la précision est limitée) ; chaque condamnation antérieure l'augmente d'environ 10 % ; chaque année d'âge de plus le réduit de près de 6 %. Notez que, comme pour notre offre de bienvenue, la variable d'intérêt a été **randomisée** : c'est ce qui autorise la lecture causale (volume II, chapitre 7, pour aller plus loin).

#### À retenir

- Le modèle de Cox :  $h(t | x) = h_0(t)e^{x^T \beta}$ . Le risque de base  $h_0$  est **libre** ; chaque coefficient est un **rapport de risques**  $HR = e^\beta$ , constant dans le temps.
- La **vraisemblance partielle**  $\prod_i e^{x_i^T \beta} / \sum_{k \in R_i} e^{x_k^T \beta}$  ne contient pas  $h_0$ . Son score est  $\sum_i (x_i - \bar{x}_{R_i})$  (les résidus de Schoenfeld) ; le **test du log-rank** est le **test de score** du modèle à une variable de groupe.
- Pour les **ex aequo**, préférez **Efron** à Breslow, surtout quand les durées sont arrondies.
- L'hypothèse de **proportionnalité** se vérifie par le graphique log-log et par le test de score de Grambsch-Therneau (par défaut avec le rang du temps) ; en cas de violation : stratifier, introduire un effet variable dans le temps, ou changer de résumé.
- Une variable connue seulement **après** un certain temps de survie doit être traitée comme **dépendant du temps** (format à épisodes), sinon on tombe dans le **bias d'immortalité**.

- Le **risque de base** s'estime a posteriori par Breslow ; on en déduit  $S(t | x) = \exp[-H_0(t)e^{x^\top \beta}]$  pour tout profil.
- L'**indice de concordance** mesure la capacité à ordonner les clients ; un effet très significatif peut aller avec un C-index modeste.

## 5.4 Modèles de durée paramétriques

**Intuition.** Le modèle de Cox est élégant parce qu'il ne dit rien sur la forme du risque de base. Mais ce silence a un prix : on ne peut rien dire **au-delà** de ce qu'on a observé. Yasmine voudrait savoir combien un client rapporte *au total*, y compris pendant les années à venir que personne n'a encore vécues. Pour extrapoler, il faut **parier sur une forme** de loi. Les modèles paramétriques font ce pari : ils décrivent la durée par une loi connue (exponentielle, Weibull, log-normale...) dont les paramètres dépendent des caractéristiques du client. En échange du pari, on obtient des courbes lisses, des durées moyennes, des extrapolations, et des estimations plus précises *si la forme est bonne*.

### 5.4.1 Le modèle à temps de vie accéléré

Les modèles paramétriques s'écrivent le plus naturellement sur le **logarithme de la durée** (le logarithme transforme une quantité positive et asymétrique en une quantité symétrique, comme pour la régression linéaire du début du volume) :

$$\ln T = \mu(x) + \sigma W, \quad \mu(x) = x^\top \gamma$$

où  $W$  est une variable aléatoire de loi **fixée** (le « bruit ») et  $\sigma > 0$  un paramètre d'**échelle**. C'est le **modèle à temps de vie accéléré** (en anglais *accelerated failure time*, AFT). Le choix de la loi de  $W$  détermine la famille :

| Loi de $W$                            | Loi de $T$            | $S(t   x)$ , avec<br>$z = (\ln t - \mu)/\sigma$ | Forme du risque $h(t)$                                                   |
|---------------------------------------|-----------------------|-------------------------------------------------|--------------------------------------------------------------------------|
| $\sigma = 1$ , valeur extrême         | <b>exponentielle</b>  | $\exp(-e^z)$                                    | constant                                                                 |
| valeur extrême<br>(Gumbel du minimum) | <b>Weibull</b>        | $\exp(-e^z)$                                    | monotone : croissant si<br>$\sigma < 1$ , décroissant si<br>$\sigma > 1$ |
| normale                               | <b>log-normale</b>    | $1 - \Phi(z)$                                   | croît puis décroît                                                       |
| logistique                            | <b>log-logistique</b> | $1/(1 + e^z)$                                   | croît puis décroît (ou<br>décroît seulement si<br>$\sigma \geq 1$ )      |

(On reconnaît la forme de Weibull du 5.1.3 avec la **forme**  $k = 1/\sigma$  et l'**échelle**  $e^\mu$ .)

**Interprétation : l'« accélération ».** Écrivons  $T = e^{x^\top \gamma} T_0$ , où  $T_0 = e^{\sigma W}$  est la durée d'un client « de référence » ( $x = 0$ ). Un client de caractéristiques  $x$  vit  $e^{x^\top \gamma}$  **fois plus longtemps** : les covariables **accélèrent ou ralentissent l'horloge** :

$$S(t | x) = S_0(t e^{-x^\top \gamma}).$$

Le facteur  $e^{\gamma_j}$  est le **facteur d'accélération** : s'il vaut 1,37 pour l'offre de bienvenue, tout se passe comme si le temps s'écoulait 1,37 fois plus lentement pour un client qui l'a reçue ; sa durée **médiane** (et sa durée moyenne) est 37 % plus longue. C'est un langage plus parlant que le rapport de risques : « *l'offre rallonge la relation de 37 %* ».



**Attention au sens des coefficients.** Dans un modèle AFT, un coefficient **positif** signifie une durée **plus longue** (protecteur). Dans le modèle de Cox, un coefficient **positif** signifie un risque **plus élevé** (néfaste). Les signes sont opposés ! Si l'on compare ce que fait Cox et ce que fait un AFT, il faut convertir (ce que nous ferons au 5.4.2).

### 5.4.2 La Weibull : à la fois « risques proportionnels » et « temps accéléré »

Parmi toutes les lois, la Weibull a une propriété unique : elle appartient aux **deux** familles. Montrons-le. Avec  $k = 1/\sigma$  et  $\lambda(x) = e^{x^\top \gamma}$ ,

$$S(t | x) = \exp\left[-\left(\frac{t}{\lambda(x)}\right)^k\right], \quad H(t | x) = \left(\frac{t}{\lambda(x)}\right)^k = t^k e^{-k x^\top \gamma}.$$

Dérivons pour obtenir le risque :

$$h(t | x) = k t^{k-1} e^{-k x^\top \gamma} = \underbrace{k t^{k-1}}_{h_0(t)} \cdot e^{x^\top \beta} \quad \text{avec} \quad \boxed{\beta = -k \gamma = -\gamma/\sigma}.$$

C'est exactement la forme du modèle de Cox, avec un risque de base **imposé**  $h_0(t) = k t^{k-1}$ . Les coefficients des deux modèles se déduisent l'un de l'autre. C'est un excellent test de cohérence : *si les données sont vraiment Weibull, un modèle de Cox et un AFT Weibull doivent donner les mêmes effets.*

### 5.4.3 Le maximum de vraisemblance, écrit à la main

La vraisemblance avec censure (5.1.5) s'écrit pour un AFT, avec  $z_i = (\ln y_i - x_i^\top \gamma)/\sigma$ ,  $f_W$  la densité et  $S_W$  la survie de  $W$  :

$$\ell(\gamma, \sigma) = \sum_i \left[ \delta_i (\ln f_W(z_i) - \ln \sigma - \ln y_i) + (1 - \delta_i) \ln S_W(z_i) \right].$$

(La densité de  $T = e^{\mu + \sigma W}$  est  $f_W(z)/(\sigma t)$  : c'est le changement de variable habituel.) Pour la Weibull,  $\ln f_W(z) = z - e^z$  et  $\ln S_W(z) = -e^z$  ; pour la log-normale,  $f_W = \varphi$  et  $S_W = 1 - \Phi$  ; pour la log-logistique,  $\ln f_W(z) = z - 2 \ln(1 + e^z)$  et  $\ln S_W(z) = -\ln(1 + e^z)$ . Le code suivant écrit ces trois vraisemblances, les maximise numériquement et calcule les erreurs standard par la **hessienne** numérique de  $-\ell$  (l'inverse de l'information, comme au volume I, section 3.2).

```
import numpy as np
import pandas as pd
from math import gamma as Gamma
from scipy import stats
from scipy.optimize import minimize
from statsmodels.tools.numdiff import approx_hess3

c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
X = pd.get_dummies(c[["offre_bienvenue", "age", "canal_acquisition"]], drop_first=True, dtype=float)
Xc = np.column_stack([np.ones(len(c)), X.to_numpy()]) # première colonne : constante
noms = ["(constante)"] + list(X.columns)

def log_vrais_aft(theta, y, d, X, loi):
    """theta = (gamma_0..gamma_p, ln sigma). Retourne la log-vraisemblance (avec censure à droite)."""
    gam, sigma = theta[:-1], np.exp(theta[-1])
    z = (np.log(y) - X @ gam) / sigma
    if loi == "weibull":
        lf, ls = z - np.exp(z), -np.exp(z)
    elif loi == "lognormale":
        lf, ls = stats.norm.logpdf(z), stats.norm.logsf(z)
    elif loi == "loglogistique":
```

```

    lf, ls = z - 2 * np.log1p(np.exp(z)), -np.log1p(np.exp(z))
    return np.sum(d * (lf - np.log(sigma) - np.log(y)) + (1 - d) * ls)

def ajuster(loi, y, d, X, sigma_fixe=None):
    """Maximum de vraisemblance. sigma_fixe=1 donne l'exponentielle (Weibull avec sigma = 1)."""
    p = X.shape[1]
    if sigma_fixe is None:
        f = lambda t: -log_vrais_ajt(t, y, d, X, loi)
        t0 = np.concatenate([[np.log(y.mean())], np.zeros(p - 1)], [0.0]])
    else:
        f = lambda t: -log_vrais_ajt(np.concatenate([t, [np.log(sigma_fixe)]]), y, d, X, loi)
        t0 = np.concatenate([[np.log(y.mean())], np.zeros(p - 1)])
    r = minimize(f, t0, method="Nelder-Mead", options={"xatol": 1e-9, "fatol": 1e-11, "maxiter":
↪ 20000, "maxfev": 20000})
    r = minimize(f, r.x, method="BFGS") # affinage
    H = approx_hess3(r.x, f)
    return {"theta": r.x, "cov": np.linalg.inv(H), "ll": -r.fun, "k": len(r.x)}

ajustements = {loi: ajuster(loi, y, d, Xc) for loi in ("weibull", "lognormale", "loglogistique")}
ajustements["exponentielle"] = ajuster("weibull", y, d, Xc, sigma_fixe=1.0)

wb = ajustements["weibull"]
se = np.sqrt(np.diag(wb["cov"]))
tab = pd.DataFrame({"gamma": wb["theta"][:-1], "ET": se[:-1], "facteur d'accélération e^gamma":
↪ np.exp(wb["theta"][:-1]), index=noms})
print(tab.round(4).to_string())
sigma = np.exp(wb["theta"][-1])
print(f"\nsigma = {sigma:.4f} -> forme k = 1/sigma = {1 / sigma:.4f} (ET de ln sigma :
↪ {se[-1]:.4f})")
print(f"log-vraisemblance = {wb['ll']:.3f}")

```

|                             | gamma   | ET     | facteur d'accélération e^gamma |
|-----------------------------|---------|--------|--------------------------------|
| (constante)                 | 3.5257  | 0.0970 | 33.9765                        |
| offre_bienvenue             | 0.3132  | 0.0489 | 1.3678                         |
| age                         | 0.0102  | 0.0023 | 1.0102                         |
| canal_acquisition_Instagram | -0.4799 | 0.0636 | 0.6188                         |
| canal_acquisition_Site      | -0.2452 | 0.0671 | 0.7825                         |

```

sigma = 0.7563 -> forme k = 1/sigma = 1.3223 (ET de ln sigma : 0.0253)
log-vraisemblance = -4657.378

```

Comparons avec R (survreg, la référence) et avec lifelines :

```

w_r <- survreg(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data = clients,
↪ dist = "weibull")
print(summary(w_r))

```

```

Call:
survreg(formula = Surv(duree_mois, churn) ~ offre_bienvenue +
  age + canal_acquisition, data = clients, dist = "weibull")

```

|                            | Value   | Std. Error | z      | p       |
|----------------------------|---------|------------|--------|---------|
| (Intercept)                | 3.5257  | 0.0970     | 36.34  | < 2e-16 |
| offre_bienvenue            | 0.3132  | 0.0489     | 6.40   | 1.5e-10 |
| age                        | 0.0102  | 0.0023     | 4.43   | 9.3e-06 |
| canal_acquisitionInstagram | -0.4799 | 0.0636     | -7.55  | 4.4e-14 |
| canal_acquisitionSite      | -0.2452 | 0.0671     | -3.66  | 0.00026 |
| Log(scale)                 | -0.2794 | 0.0253     | -11.04 | < 2e-16 |

Scale= 0.756

```

Weibull distribution
Loglik(model)= -4657.4 Loglik(intercept only)= -4714.4
Chisq= 114.02 on 4 degrees of freedom, p= 1e-23

```

```
Number of Newton-Raphson Iterations: 6
n= 2000
```

```
from lifelines import WeibullAFTFitter

df_ll = pd.concat([c[["duree_mois", "churn"]], X], axis=1)
ll_w = WeibullAFTFitter().fit(df_ll, "duree_mois", "churn")
print("lifelines : coefficients de lambda_ :", ll_w.params_["lambda_"].round(4).to_dict())
print(f"lifelines : rho = ln k = {ll_w.params_['rho_', 'Intercept']:.4f} (à la main : -ln sigma =
↳ {-wb['theta'][-1]:.4f}) ; log-vraisemblance {ll_w.log_likelihood_:.3f}")

lifelines : coefficients de lambda_ : {'age': 0.0102, 'canal_acquisition_Instagram': -0.4799,
↳ 'canal_acquisition_Site': -0.2452, 'offre_bienvenue': 0.3132, 'Intercept': 3.5257}
lifelines : rho = ln k = 0.2794 (à la main : -ln sigma = 0.2794) ; log-vraisemblance -4657.378
```

Les trois sources donnent les mêmes coefficients, la même échelle et la même log-vraisemblance. Lecture :

- **Offre de bienvenue** : facteur d'accélération  $e^{0,313} \approx 1,37$  : la durée de vie d'un client qui l'a reçue est **37 % plus longue** (à âge et canal égaux).
- **Canal** : par rapport à la boutique, un client arrivé par Instagram a une durée de vie **38 % plus courte** ( $e^{-0,48} \approx 0,62$ ) et un client arrivé par le site, **22 % plus courte** ( $e^{-0,245} \approx 0,78$ ).
- **Âge** : chaque année en plus allonge la durée d'environ 1 %.
- **Forme** :  $k \approx 1,32 > 1$  : le risque **augmente** avec l'ancienneté, ce qui confirme ce que montrait la comparaison de Kaplan-Meier et de l'exponentielle (5.2.2).

**La vérification croisée avec Cox.** Convertissons ces coefficients AFT en coefficients de risques par  $\beta = -\gamma/\sigma$  et comparons au modèle de Cox de la section 5.3 :

```
beta_ph = -wb["theta"][-1] / sigma
cox_coef = np.array([-0.40612, -0.01363, 0.63508, 0.32580]) # section 5.3.3 (Breslow)
print(pd.DataFrame({"Weibull converti (-gamma/sigma)": beta_ph, "Cox (5.3.3)": cox_coef},
↳ index=noms[1:]).round(4).to_string())
```

|                             | Weibull converti (-gamma/sigma) | Cox (5.3.3) |
|-----------------------------|---------------------------------|-------------|
| offre_bienvenue             | -0.4141                         | -0.4061     |
| age                         | -0.0135                         | -0.0136     |
| canal_acquisition_Instagram | 0.6346                          | 0.6351      |
| canal_acquisition_Site      | 0.3242                          | 0.3258      |

Les deux séries sont presque identiques : les données sont bien compatibles avec une Weibull. Les erreurs standard sont, elles aussi, voisines (environ 0,065 pour l'offre dans les deux cas, en convertissant celle de l'AFT par 0,0489/0,756). Quand la forme paramétrique est bonne, on pourrait espérer un gain de précision par rapport à Cox ; ici il est négligeable : avec 977 départs, le risque de base est déjà estimé très précisément, et Cox ne perd presque rien.

#### 5.4.4 Choisir entre plusieurs lois

Tous ces modèles ont le même nombre de paramètres sauf l'exponentielle (un de moins) ; on peut donc les comparer par la **vraisemblance** et le **critère d'Akaike** ( $AIC = 2k - 2\ell$ , voir la section 1.4 de ce volume ; plus petit = meilleur).

```
lignes = []
for loi, r in ajustements.items():
    lignes.append({"loi": loi, "paramètres": r["k"], "log-vraisemblance": round(r["ll"], 2), "AIC":
↳ round(2 * r["k"] - 2 * r["ll"], 2)})
print(pd.DataFrame(lignes).sort_values("AIC").to_string(index=False))

# Test du rapport de vraisemblance : exponentielle (sigma = 1) contre Weibull
```

```
lr = 2 * (ajustements["weibull"]["ll"] - ajustements["exponentielle"]["ll"])
print(f"\nexponentielle contre Weibull : chi2 = {lr:.1f} (1 ddl), p = {stats.chi2.sf(lr, 1):.1e}")
```

| loi           | paramètres | log-vraisemblance | AIC     |
|---------------|------------|-------------------|---------|
| weibull       | 6          | -4657.38          | 9326.76 |
| loglogistique | 6          | -4663.73          | 9339.45 |
| lognormale    | 6          | -4697.20          | 9406.40 |
| exponentielle | 5          | -4710.58          | 9431.17 |

```
exponentielle contre Weibull : chi2 = 106.4 (1 ddl), p = 6.0e-25
```

```
for (loi in c("weibull", "lognormal", "loglogistic", "exponential")) {
  f <- survreg(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data = clients,
    ↪ dist = loi)
  cat(sprintf("%-12s log-vraisemblance = %.3f   AIC = %.2f\n", loi, f$loglik[2], AIC(f)))
}
```

|             |                               |               |
|-------------|-------------------------------|---------------|
| weibull     | log-vraisemblance = -4657.378 | AIC = 9326.76 |
| lognormal   | log-vraisemblance = -4697.200 | AIC = 9406.40 |
| loglogistic | log-vraisemblance = -4663.725 | AIC = 9339.45 |
| exponential | log-vraisemblance = -4710.583 | AIC = 9431.17 |

La **Weibull** est nettement la meilleure ; la log-logistique arrive deuxième (13 points d'AIC derrière), la log-normale troisième, l'exponentielle dernière. Le test du rapport de vraisemblance rejette l'exponentielle de façon écrasante : le risque n'est pas constant. Les valeurs coïncident avec celles de R.

Un AIC compare des modèles **entre eux** ; il ne dit pas si le meilleur est *bon*. Pour juger l'adéquation, on utilise les **résidus de Cox-Snell**.

**Résidus de Cox-Snell.** Si  $T$  a pour fonction de survie  $S$ , alors  $S(T)$  suit une loi uniforme (transformée intégrale de probabilité) et donc  $H(T) = -\ln S(T)$  suit une loi **exponentielle de paramètre 1**. Pour un modèle correct, les résidus  $r_i = \hat{H}(y_i | x_i)$  se comportent comme un échantillon **censuré** de loi exponentielle(1). On estime donc le risque cumulé de ces résidus par Kaplan-Meier (on garde la censure,  $\delta_i$ ) : le graphique de ce risque cumulé contre  $r$  doit suivre la **diagonale**.

Superposons la comparaison globale (survie marginale prédite par chaque modèle, contre Kaplan-Meier) et les résidus de Cox-Snell, pour la Weibull et pour l'exponentielle :

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"

def survie_aft(t, mu, sigma, loi):
    z = (np.log(t) - mu) / sigma
    return {"weibull": lambda: np.exp(-np.exp(z)), "lognormale": lambda: stats.norm.sf(z),
    ↪ "loglogistique": lambda: 1 / (1 + np.exp(z))}[loi]()

def survie_marginale(t_grille, theta, loi, Xmat):
    mu, sg = Xmat @ theta[:-1], np.exp(theta[-1])
    return np.array([survie_aft(t, mu, sg, loi).mean() for t in t_grille])

def km_simple(t, dd):
    tj = np.unique(t[dd == 1])
    ys, ev = np.sort(t), np.sort(t[dd == 1])
    n_ = len(t) - np.searchsorted(ys, tj, side="left")
    dj_ = np.searchsorted(ev, tj, side="right") - np.searchsorted(ev, tj, side="left")
```

```

    return tj, np.cumprod(1 - dj_ / n_)

grille = np.linspace(0.5, 84, 150)
tj_km, S_km = km_simple(y, d)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))
ax1.step(np.concatenate([[0], tj_km]), np.concatenate([[1], S_km]), where="post", color="#52514e",
        lw=2.2, label="Kaplan-Meier")
for loi, couleur, nom in [("weibull", ORANGE, "Weibull"), ("loglogistique", AQUA, "log-logistique"),
                          ("lognormale", VIOLET, "log-normale")]:
    ax1.plot(grille, survie_marginale(grille, ajustements[loi]["theta"], loi, Xc), color=couleur,
            lw=1.6, label=nom)
th_e = np.concatenate([ajustements["exponentielle"]["theta"], [0.0]])
ax1.plot(grille, survie_marginale(grille, th_e, "weibull", Xc), color=BLEU, lw=1.6, ls="--",
        label="exponentielle")
ax1.set_xlabel("mois depuis l'inscription")
ax1.set_ylabel("proportion de clients encore là")
ax1.set_title("Survie prédite (moyenne sur les 2 000 clients)")
ax1.legend(frameon=False, fontsize=9)
ax1.set_ylim(0, 1.02)

# Résidus de Cox-Snell
for loi, th, couleur, nom in [("weibull", ajustements["weibull"]["theta"], ORANGE, "Weibull"),
                              ("weibull", th_e, BLEU, "exponentielle")]:
    mu, sg = Xc @ th[:-1], np.exp(th[-1])
    r = -np.log(survie_aft(y, mu, sg, loi))
    tj_r, S_r = km_simple(r, d)
    ax2.step(tj_r, -np.log(S_r), where="post", color=couleur, lw=1.8, label=nom)
ax2.plot([0, 3], [0, 3], color="#898781", lw=1, ls=":")
ax2.set_xlim(0, 2.6)
ax2.set_ylim(0, 2.6)
ax2.set_xlabel("résidu de Cox-Snell r")
ax2.set_ylabel("risque cumulé estimé des résidus")
ax2.set_title("Adéquation : suivre la diagonale")
ax2.legend(frameon=False, fontsize=9, loc="upper left")
plt.tight_layout()
plt.savefig("figures/ch05-param-ajustement.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

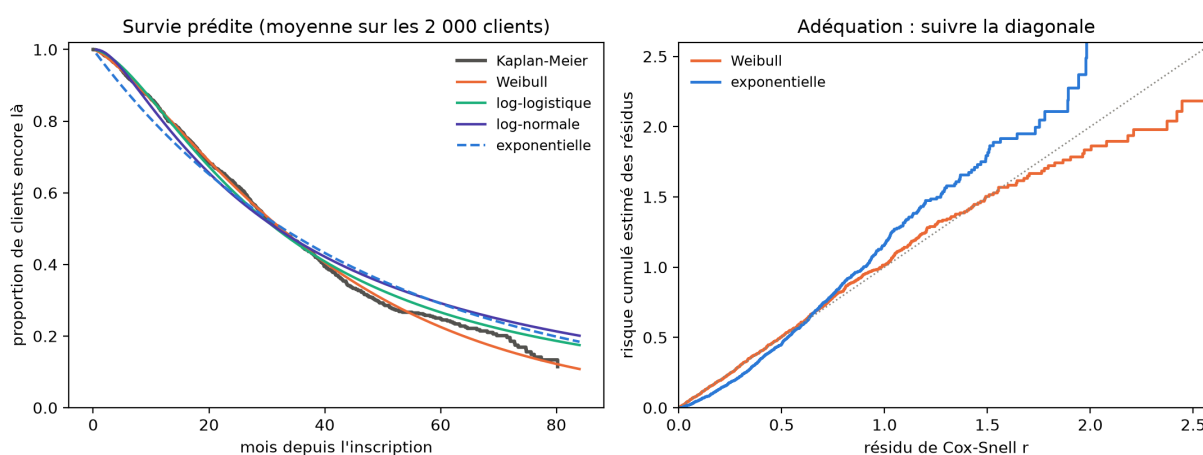


Figure 59: À gauche : survie de Kaplan-Meier des 2 000 clients et survie moyenne prédite par quatre modèles paramétriques. À droite : résidus de Cox-Snell du modèle de Weibull et du modèle exponentiel avec covariables ; le bon modèle suit la diagonale.

À gauche, les quatre modèles sont presque indiscernables de Kaplan-Meier jusqu'à 35 mois environ, sauf l'exponentielle, qui prévoit trop de départs au tout début (le schéma de la section 5.2.2).

Au-delà de 40 mois, les courbes se séparent : la Weibull suit Kaplan-Meier jusqu'à 55 mois environ, puis passe **un peu en dessous** ; les trois autres lois restent **au-dessus**. À droite, les résidus de Cox-Snell du modèle de Weibull suivent la diagonale jusqu'à  $r \approx 1,5$  (au-delà, quelques résidus seulement : le tracé est instable), tandis que ceux du modèle exponentiel s'en écartent dès que  $r$  dépasse environ 0,7.

### 5.4.5 Extrapoler et chiffrer : la valeur vie client

Yasmine veut savoir **ce que rapporte un client en moyenne sur toute sa vie**. Pour cela, il lui faut la survie *au-delà* de la fenêtre observée (84 mois au plus). Seul un modèle paramétrique peut fournir cette extrapolation.

La **durée de vie moyenne** d'un client de profil  $x$  dans un modèle de Weibull est  $E[T | x] = \lambda(x) \Gamma(1 + 1/k)$  (section 5.1.3). La **valeur vie client** actualisée (en anglais *customer lifetime value*, CLV) combine cette survie avec un revenu :

$$CLV(x) = m \sum_{t=0}^{T_{\max}} \frac{S(t | x)}{(1 + r)^t}$$

où  $m$  est la **marge mensuelle** par client encore actif,  $r$  le taux d'actualisation mensuel (un euro dans un an vaut moins qu'un euro aujourd'hui), et  $S(t | x)$  la probabilité d'être encore client au mois  $t$ .

**Des hypothèses, pas des données.** Nous n'avons dans nos fichiers **ni marge ni coût**. Les trois chiffres ci-dessous sont des hypothèses que Yasmine devrait remplacer par les siens : une marge de **6 DT par mois** et par client actif, un taux d'actualisation de **1 % par mois** (environ 13 % par an) et un horizon de 20 ans (240 mois). Le but est de montrer la **mécanique** du calcul, pas de chiffrer vraiment la boutique.

```
m_mensuelle, taux, horizon = 6.0, 0.01, 240
mois = np.arange(0, horizon)
k_hat, th = 1 / sigma, wb["theta"]
lam = np.exp(Xc @ th[:-1]) # échelle e^{x gamma} de chaque client

def S_groupe(masque):
    """Survie moyenne du groupe, d'après le modèle de Weibull ajusté, mois par mois."""
    return np.array([np.mean(np.exp(-(t / lam[masque]) ** k_hat)) for t in mois])

groupes = {"tous les clients": np.ones(len(c), bool),
           "sans offre": (c["offre_bienvenue"] == 0).to_numpy(), "avec offre": (c["offre_bienvenue"]
           <- == 1).to_numpy(),
           "Boutique": (c["canal_acquisition"] == "Boutique").to_numpy(), "Site":
           <- (c["canal_acquisition"] == "Site").to_numpy(),
           "Instagram": (c["canal_acquisition"] == "Instagram").to_numpy())

lignes = []
for nom, masque in groupes.items():
    S = S_groupe(masque)
    clv = m_mensuelle * np.sum(S / (1 + taux) ** mois)
    apres_84 = m_mensuelle * np.sum(S[84:] / (1 + taux) ** mois[84:])
    lignes.append({"groupe": nom, "clients": int(masque.sum()), "durée moyenne (mois)":
    <- np.mean(lam[masque]) * Gamma(1 + 1 / k_hat),
                  "survie à 36 mois": S[36], "CLV (DT)": clv, "part de la CLV après 84 mois (%)": 100
    <- * apres_84 / clv})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

|    | groupe           | clients | durée moyenne (mois) | survie à 36 mois | CLV (DT) | part de la CLV après 84 mois (%) |
|----|------------------|---------|----------------------|------------------|----------|----------------------------------|
| <- | mois (%)         |         |                      |                  |          |                                  |
|    | tous les clients | 2000    | 41.339               | 0.453            | 186.096  | 3.741                            |
|    | sans offre       | 985     | 35.167               | 0.384            | 165.806  | 2.156                            |

|            |      |        |       |         |       |
|------------|------|--------|-------|---------|-------|
| avec offre | 1015 | 47.329 | 0.521 | 205.785 | 4.981 |
| Boutique   | 504  | 53.159 | 0.574 | 223.468 | 6.364 |
| Site       | 680  | 42.068 | 0.471 | 189.751 | 3.545 |
| Instagram  | 816  | 33.431 | 0.364 | 159.967 | 1.672 |

Quelques vérifications de bon sens. La survie moyenne à 36 mois du modèle (0,453) retrouve **exactement** la valeur de Kaplan-Meier de la section 5.2 (0,453), ce qui confirme que le modèle colle aux données observées. La durée moyenne d'un client est de **41 mois**, tandis que la durée moyenne *restreinte* à 60 mois de Kaplan-Meier valait 34 mois : l'écart (7 mois) est la part de la vie *au-delà de 60 mois*, que Kaplan-Meier ne peut pas chiffrer et que le modèle extrapole. Et la durée moyenne **exponentielle** de la section 5.1.5 (48 mois) était trop optimiste.

**L'offre de bienvenue en valait-elle la peine ?** Elle augmente la CLV de 166 à 206 DT par client, soit un gain de **40 DT** d'actualisé. Si l'offre coûte, par hypothèse, 10 DT par client, le gain net est de 30 DT par client. Comme l'offre est randomisée, cette différence est une estimation de **l'effet causal** de l'offre sur la valeur. Voyons si le résultat dépend des hypothèses :

```
S0, S1 = S_groupe(groupe("sans offre")), S_groupe(groupe("avec offre"))
cout_offre = 10.0
print(f"CLV sans offre : {m_mensuelle * np.sum(S0 / (1 + taux) ** mois):.1f} DT | avec offre :
↳ {m_mensuelle * np.sum(S1 / (1 + taux) ** mois):.1f} DT\n")
print("gain net par client (CLV avec offre - CLV sans offre - coût de 10 DT), selon les hypothèses :")
resultats = pd.DataFrame(index=[f"taux {100 * r:.1f} %/mois" for r in (0.005, 0.01, 0.02)],
                          columns=[f"marge {m} DT/mois" for m in (3, 6, 9)], dtype=float)
for r in (0.005, 0.01, 0.02):
    for m in (3, 6, 9):
        v = (1 + r) ** (-mois)
        resultats.loc[f"taux {100 * r:.1f} %/mois", f"marge {m} DT/mois"] = m * np.sum((S1 - S0) * v)
        ↳ - cout_offre
print(resultats.round(1).to_string())
```

CLV sans offre : 165.8 DT | avec offre : 205.8 DT

gain net par client (CLV avec offre - CLV sans offre - coût de 10 DT), selon les hypothèses :

|                 | marge 3 DT/mois | marge 6 DT/mois | marge 9 DT/mois |
|-----------------|-----------------|-----------------|-----------------|
| taux 0.5 %/mois | 16.5            | 42.9            | 69.4            |
| taux 1.0 %/mois | 10.0            | 30.0            | 50.0            |
| taux 2.0 %/mois | 2.4             | 14.7            | 27.1            |

Le gain net reste **positif dans les neuf cas testés**, mais son ordre de grandeur varie d'un facteur 30 : de 69 DT par client (marge de 9 DT, actualisation faible) à seulement 2,4 DT (marge de 3 DT, actualisation de 2 % par mois), c'est-à-dire presque rien. La conclusion « l'offre est rentable » est donc **robuste** ; la conclusion « elle rapporte 30 DT par client » ne l'est pas. Voilà exactement le genre d'information utile : on peut dire à Yasmine que l'offre ne perd pas d'argent dans ce domaine d'hypothèses, et lui demander sa vraie marge pour chiffrer le gain.

**Les limites de l'extrapolation.** La CLV dépend de la survie *au-delà* des données, donc de la forme de loi supposée. La dernière colonne du premier tableau chiffre cette dépendance : la part de la valeur située **après 84 mois** n'est que d'environ 4 % pour l'ensemble des clients (6 % pour la boutique), parce que l'**actualisation** écrase les mois lointains et que beaucoup de clients sont déjà partis. La CLV est donc peu sensible à l'extrapolation. Ce n'est pas le cas de la **durée moyenne**, qui n'est pas actualisée : c'est elle qui réclame de l'extrapolation (7 mois de plus que la RMST à 60 mois). Deux lois qui s'ajustent presque aussi bien sur 84 mois peuvent extrapoler très différemment : la figure du 5.4.4 le montre, avec une Weibull qui passe sous Kaplan-Meier à partir de 55 mois et des lois log-logistique et log-normale qui restent au-dessus. Le bon réflexe : refaire le calcul avec une autre loi (log-logistique) et



comparer ; et ne jamais interpréter une CLV comme une certitude.

### 5.4.6 Une variable manquante : le service

Nos modèles ne connaissent ni la qualité du service reçu, ni la satisfaction du client. Or l'enquête de satisfaction (donnees/enquete\_satisfaction.csv) en mesure une partie : les notes q5 à q8 portent sur le service et la livraison. Environ 60 % des clients ont répondu. Que se passe-t-il si l'on ajoute leur **note moyenne de service** au modèle de survie, sur les répondants ?

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
q["service"] = q[["q5", "q6", "q7", "q8"]].mean(axis=1)
rep = c.merge(q[["id_client", "service"]], on="id_client")
rep["service_std"] = (rep["service"] - rep["service"].mean()) / rep["service"].std()
Xr_base = np.column_stack([np.ones(len(rep)), pd.get_dummies(rep[["offre_bienvenue", "age",
↳ "canal_acquisition"]], drop_first=True, dtype=float).to_numpy()]])
Xr_serv = np.column_stack([Xr_base, rep["service_std"]])
yr, dr = rep["duree_mois"].to_numpy(), rep["churn"].to_numpy()
sans = ajuster("weibull", yr, dr, Xr_base)
avec = ajuster("weibull", yr, dr, Xr_serv)
se_a = np.sqrt(np.diag(avec["cov"]))
print(f"répondants : {len(rep)}")
print(f"sans le service : log-vraisemblance {sans['ll']:.1f} ; forme k = {1 /
↳ np.exp(sans['theta'][-1]):.3f}")
print(f"avec le service : log-vraisemblance {avec['ll']:.1f} ; forme k = {1 /
↳ np.exp(avec['theta'][-1]):.3f}")
print(f"coefficient du service (par écart-type de la note) : {avec['theta'][-2]:.3f} (ET
↳ {se_a[-2]:.3f}) -> durée x {np.exp(avec['theta'][-2]):.2f}")
lr_s = 2 * (avec["ll"] - sans["ll"])
print(f"rapport de vraisemblance : chi2 = {lr_s:.1f} (1 ddl), p = {stats.chi2.sf(lr_s, 1):.1e}")
```

```
répondants : 1212
sans le service : log-vraisemblance -2785.8 ; forme k = 1.312
avec le service : log-vraisemblance -2750.2 ; forme k = 1.368
coefficient du service (par écart-type de la note) : 0.262 (ET 0.031) -> durée x 1.30
rapport de vraisemblance : chi2 = 71.2 (1 ddl), p = 3.2e-17
```

Un client dont la note de service est **un écart-type au-dessus de la moyenne** reste en moyenne environ **30 % plus longtemps** (facteur 1,30) : le service compte, et le test du rapport de vraisemblance le confirme. C'est un exemple concret de ce qu'apportent les **variables explicatives pertinentes** (et du travail de construction d'indicateurs de la section 3.2 de ce volume, l'analyse factorielle, pour condenser huit notes en un score de service).

### 5.4.7 Révéler la vérité

Comme les données sont simulées, nous pouvons maintenant comparer nos estimations à la loi qui les a engendrées (documentée en tête de build/donnees2.py). Les durées sont de loi de **Weibull de forme 1,35**, avec

$$\ln T = 3,6 + 0,30 F_2 + 0,35 \text{ offre} + \{\text{Boutique} + 0,30, \text{Site 0, Instagram} - 0,15\} + 0,008 (\text{âge} - 36) + \sigma W,$$

où  $F_2$  est un **facteur de sensibilité au service** (loi normale centrée réduite) **que le fichier ne contient pas**, et où  $\sigma = 1/1,35 \approx 0,74$ . On peut même reconstruire ce facteur manquant avec le générateur de données, et ajuster le modèle « oracle » qui le connaît :

```
import sys
sys.path.insert(0, "build")
import donnees2
_, F1, F2 = donnees2.clients()
oracle = ajuster("weibull", y, d, np.column_stack([Xc, F2]))
```

```

se_o = np.sqrt(np.diag(oracle["cov"]))

# Valeurs vraies, exprimées dans la même paramétrisation que notre modèle (âge centré en 36 dans le
# ↪ générateur,
# référence = Boutique, donc Instagram = -0.15 - 0.30 et Site = 0 - 0.30)
vrai = {"(constante)": 3.6 + 0.30 - 0.008 * 36, "offre_bienvenue": 0.35, "age": 0.008,
        "canal_acquisition_Instagram": -0.15 - 0.30, "canal_acquisition_Site": 0.0 - 0.30}
lignes = []
for j, nom in enumerate(noms):
    lignes.append({"paramètre": nom, "vérité": vrai[nom], "estimé (sans F2)": wb["theta"][j], "ET":
    ↪ se[j],
                  "écart / ET": (wb["theta"][j] - vrai[nom]) / se[j], "oracle (avec F2)":
    ↪ oracle["theta"][j]})
lignes.append({"paramètre": "forme k = 1/sigma", "vérité": 1.35, "estimé (sans F2)": 1 / sigma, "ET":
    ↪ np.nan,
                  "écart / ET": np.nan, "oracle (avec F2)": 1 / np.exp(oracle["theta"][-1])})
lignes.append({"paramètre": "F2 (facteur manquant)", "vérité": 0.30, "estimé (sans F2)": np.nan,
    ↪ "ET": np.nan,
                  "écart / ET": np.nan, "oracle (avec F2)": oracle["theta"][len(noms)]})
print(pd.DataFrame(lignes).round(3).to_string(index=False))

```

|                             | paramètre             | vérité | estimé (sans F2) | ET    | écart / ET | oracle (avec F2) |
|-----------------------------|-----------------------|--------|------------------|-------|------------|------------------|
|                             | (constante)           | 3.612  | 3.526            | 0.097 | -0.890     | 3.562            |
|                             | offre_bienvenue       | 0.350  | 0.313            | 0.049 | -0.753     | 0.299            |
|                             | age                   | 0.008  | 0.010            | 0.002 | 0.955      | 0.009            |
| canal_acquisition_Instagram |                       | -0.450 | -0.480           | 0.064 | -0.471     | -0.486           |
| canal_acquisition_Site      |                       | -0.300 | -0.245           | 0.067 | 0.817      | -0.242           |
|                             | forme k = 1/sigma     | 1.350  | 1.322            | NaN   | NaN        | 1.400            |
|                             | F2 (facteur manquant) | 0.300  | NaN              | NaN   | NaN        | 0.310            |

Que montre cette comparaison ?

1. **Tous les coefficients du modèle réaliste sont à moins d'une erreur standard de la vérité** (colonne « écart / ET ») : la méthode retrouve ce qu'on a programmé (effet de l'offre : +0,31 estimé contre +0,35 ; les canaux et l'âge aussi).
2. Le modèle **oracle**, qui connaît le facteur manquant, retrouve **son coefficient** ( $\approx 0,31$  pour une vérité de 0,30). Il estime aussi une forme de 1,40, contre 1,32 pour le modèle sans  $F_2$  : la vraie valeur (1,35) se situe entre les deux, et l'écart entre les deux estimations est précisément ce que prédit le point suivant. (De même, sur les répondants à l'enquête, la forme passe de 1,31 à 1,37 quand on ajoute la note de service.)
3. Le **déplacement de la forme** est un phénomène classique : quand une variable qui joue sur le risque est **omise**, le risque observé pour l'ensemble de la population est un **mélange** de risques individuels ; les clients les plus fragiles partent les premiers, de sorte que les survivants sont de plus en plus robustes. La population semble avoir un risque qui augmente **moins vite** que celui de chaque individu. En termes de modèle, on parle d'**hétérogénéité non observée** (ou de **fragilité**, *frailty*). Elle est inévitable : il y a toujours des variables que l'on ne mesure pas.

### À retenir

- Un modèle **paramétrique** parie sur la loi de la durée. Il permet d'**extrapoler**, de calculer des durées moyennes et d'être plus précis que Cox *si la forme est bonne*.
- Le modèle **AFT** s'écrit  $\ln T = x^\top \gamma + \sigma W$  : les covariables **accélèrent ou ralentissent le temps** ;  $e^\gamma$  est le facteur d'accélération (**signe opposé** à celui de Cox).
- La **Weibull** est à la fois AFT et à risques proportionnels, avec  $\beta = -\gamma/\sigma$  : un excellent test de cohérence avec Cox.
- La vraisemblance avec censure s'écrit à la main pour chaque loi ; le résultat doit coïncider avec survreg de R. On choisit entre lois par l'**AIC** et on vérifie l'**adéquation** par

les **résidus de Cox-Snell**.

- La **valeur vie client**  $\sum_t S(t | x) m / (1 + r)^t$  repose sur des hypothèses explicites (marge, actualisation, horizon) et sur une extrapolation : on la présente avec une analyse de sensibilité, jamais comme une certitude.
- Une variable pertinente **omise** (hétérogénéité non observée) déplace la forme apparente du risque ; on ne peut jamais exclure qu'il en existe.

## 5.5 ☉ Pour aller plus loin : les risques concurrents

**Section optionnelle.** Elle prolonge le chapitre quand **plusieurs événements** peuvent mettre fin à la durée et s'excluent mutuellement. Les sections 5.1 à 5.4 se lisent sans elle.

**Intuition.** Jusqu'ici, il n'y avait qu'une façon de « sortir » : le client partait, ou on ne l'avait pas encore vu partir. Mais dans la vie d'une boutique, un compte peut aussi être **fermé de force** (impayés, soupçon de fraude, adresse invalide). Dès qu'une sortie de ce type survient, le client **ne peut plus partir volontairement** : les deux événements sont en **compétition**, et le premier qui arrive élimine l'autre. Si l'on oublie cette compétition, on surestime les probabilités de chaque cause.

### 5.5.1 Deux façons de sortir

Reprenons le fil de Dar Jasmin. Un client peut :

- **cause 1 : partir volontairement** (il ne commande plus et se désinscrit) ;
- **cause 2 : voir son compte fermé de force** (un incident de paiement, par exemple).

On observe un seul de ces événements, **le premier**. Notons  $T_1$  et  $T_2$  les durées « potentielles » jusqu'à chaque cause (la seconde n'est jamais observée si la première survient avant) ; la durée observée est  $T = \min(T_1, T_2)$  (ou la durée de censure, comme avant) et la **cause** est celle qui l'emporte.

Les objets du 5.1 se généralisent :

- le **risque propre à la cause  $k$**  (*cause-specific hazard*) :

$$h_k(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T < t + \Delta t, \text{ cause} = k \mid T \geq t)}{\Delta t} \quad : \text{le taux de sortie par la cause } k \text{ parmi ceux encore là ;}$$

- le **risque total**  $h(t) = h_1(t) + h_2(t)$  : les risques **s'additionnent**, et la survie (« rester client, quelle que soit la cause de départ ») est donc

$$S(t) = \exp \left[ - \int_0^t (h_1(u) + h_2(u)) du \right] = \exp [ - H_1(t) - H_2(t) ] ;$$

- la **fonction d'incidence cumulée** (CIF) de la cause  $k$  : la **probabilité** d'être sorti *par la cause*  $k$  avant  $t$ ,

$$F_k(t) = P(T \leq t, \text{ cause} = k) = \int_0^t h_k(u) S(u) du$$

**Pourquoi cette formule ?** Pour sortir par la cause  $k$  **entre**  $u$  et  $u + du$ , il faut deux choses : être encore là en  $u$  (probabilité  $S(u)$ ), puis sortir par la cause  $k$  à cet instant (probabilité  $h_k(u) du$ ). On additionne sur tous les instants. Le point crucial est que  $S(u)$  **est la survie totale** : elle dépend des *deux* risques. La probabilité d'une cause dépend donc de l'autre. De plus, comme

tout client finit dans l'un des trois états (encore là, sorti par 1, sorti par 2),

$$S(t) + F_1(t) + F_2(t) = 1 \quad \text{à tout instant.}$$

### 5.5.2 Pourquoi « 1 – Kaplan-Meier » trompe

La tentation est de traiter la cause 2 comme une simple *censure* : « pour estimer le départ volontaire, on compte les fermetures forcées comme des clients perdus de vue ». Et d'estimer la probabilité de partir volontairement par  $1 - \hat{S}_{KM}(t)$ . C'est **faux** : on estime ainsi la probabilité de départ volontaire *dans un monde imaginaire où aucune fermeture forcée n'existerait*, que l'on n'a jamais observé. Voyons-le sur dix clients.

| Client | 1             | 2            | 3             | 4       | 5            | 6             | 7       | 8             | 9            | 10      |
|--------|---------------|--------------|---------------|---------|--------------|---------------|---------|---------------|--------------|---------|
| Mois   | 2             | 3            | 4             | 5       | 6            | 7             | 8       | 9             | 10           | 12      |
| Issue  | départ<br>(1) | fermé<br>(2) | départ<br>(1) | censuré | fermé<br>(2) | départ<br>(1) | censuré | départ<br>(1) | fermé<br>(2) | censuré |

L'estimateur d'Aalen-Johansen de la CIF de la cause  $k$  applique la formule précédente en version « escalier » :

$$\hat{F}_k(t) = \sum_{j: t_j \leq t} \hat{S}(t_j^-) \frac{d_{kj}}{n_j}$$

où  $n_j$  est l'ensemble à risque en  $t_j$ ,  $d_{kj}$  le nombre de sorties par la cause  $k$  en  $t_j$ , et  $\hat{S}(t_j^-)$  la survie **totale** de Kaplan-Meier (toutes causes confondues) juste avant  $t_j$ . Calculons à la main :

| $t_j$ | $n_j$ | $d_{1j}$ | $d_{2j}$ | $\hat{S}(t_j^-)$ | ajout à<br>$\hat{F}_1$ | $\hat{F}_1$ | ajout à<br>$\hat{F}_2$ | $\hat{F}_2$ | $\hat{S}(t_j)$ |
|-------|-------|----------|----------|------------------|------------------------|-------------|------------------------|-------------|----------------|
| 2     | 10    | 1        | 0        | 1,000            | 1/10 =<br>0,100        | 0,100       | 0                      | 0           | 0,900          |
| 3     | 9     | 0        | 1        | 0,900            | 0                      | 0,100       | 0,9/9 =<br>0,100       | 0,100       | 0,800          |
| 4     | 8     | 1        | 0        | 0,800            | 0,8/8 =<br>0,100       | 0,200       | 0                      | 0,100       | 0,700          |
| 6     | 6     | 0        | 1        | 0,700            | 0                      | 0,200       | 0,7/6 =<br>0,117       | 0,217       | 0,583          |
| 7     | 5     | 1        | 0        | 0,583            | 0,583/5 =<br>0,117     | 0,317       | 0                      | 0,217       | 0,467          |
| 9     | 3     | 1        | 0        | 0,467            | 0,467/3 =<br>0,156     | 0,472       | 0                      | 0,217       | 0,311          |
| 10    | 2     | 0        | 1        | 0,311            | 0                      | 0,472       | 0,311/2 =<br>0,156     | 0,372       | 0,156          |

(Les censures des mois 5, 8 et 12 réduisent  $n_j$  sans produire de ligne.) À la fin,  $0,156 + 0,472 + 0,372 = 1$  : la propriété  $S + F_1 + F_2 = 1$  est respectée. Comparons avec « 1 – KM » obtenu en traitant les fermetures comme des censures :

```
import numpy as np
import pandas as pd

def incidence_cumulee(t, cause):
    """Aalen-Johansen. t : durées, cause : 0 = censuré, 1, 2, ... Retourne (instants, survie totale,
    ↳ [CIF des causes])."""
```

```

t, cause = np.asarray(t, float), np.asarray(cause, int)
tj = np.unique(t[cause > 0])
ys = np.sort(t)
n_ = len(t) - np.searchsorted(ys, tj, side="left")           # ensemble à risque
causes = sorted(set(cause[cause > 0]))
dk = [np.array([np.sum((t == u) & (cause == k)) for u in tj]) for k in causes]
S = np.cumprod(1 - sum(dk) / n_)                             # survie totale
↪ (Kaplan-Meier toutes causes)
S_avant = np.concatenate([[1.0], S[:-1]])                  # S(t_j^-)
return tj, S, [np.cumsum(S_avant * d_ / n_) for d_ in dk]

t10 = np.array([2, 3, 4, 5, 6, 7, 8, 9, 10, 12.])
c10 = np.array([1, 2, 1, 0, 2, 1, 0, 1, 2, 0])
tj10, S10, (F1_10, F2_10) = incidence_cumulee(t10, c10)
print(pd.DataFrame({"t_j": tj10, "S": S10, "F1": F1_10, "F2": F2_10, "S + F1 + F2": S10 + F1_10 +
↪ F2_10}).round(3).to_string(index=False))

# « 1 - KM » naïf : on traite la cause 2 comme une censure
tj_n, S_n, (F1_naif,) = incidence_cumulee(t10, np.where(c10 == 1, 1, 0))
print(f"\n1 - KM (cause 2 traitée comme censure) à t = 9 : {1 - S_n[-1]:.3f} | incidence cumulée
↪ correcte F1(9) = {F1_10[5]:.3f}")

t_j      S      F1      F2      S + F1 + F2
2.0 0.900 0.100 0.000          1.0
3.0 0.800 0.100 0.100          1.0
4.0 0.700 0.200 0.100          1.0
6.0 0.583 0.200 0.217          1.0
7.0 0.467 0.317 0.217          1.0
9.0 0.311 0.472 0.217          1.0
10.0 0.156 0.472 0.372          1.0

1 - KM (cause 2 traitée comme censure) à t = 9 : 0.580 | incidence cumulée correcte F1(9) = 0.472

```

Le « 1 – KM » donne 0,58 pour la probabilité de départ volontaire à 9 mois, alors que la vraie incidence cumulée est de 0,47. Il est **trop grand** : il fait comme si les trois clients dont le compte a été fermé (aux mois 3, 6, 10) étaient restés exposés au départ volontaire.

**Les risques concurrents ne s'additionnent pas comme les 1 – KM.** Avec la méthode erronée, la somme des deux « probabilités » de sortie peut dépasser 1, ce qui est absurde. Avec les incidences cumulées, la somme  $F_1 + F_2$  est toujours  $1 - S \leq 1$ .

### 5.5.3 Une simulation avec une vérité connue

Pour voir tout cela à l'échelle, simulons 6 000 clients avec deux causes. Nous choisissons les risques de sorte que :

- **cause 1 (départ volontaire)** : durée de loi de Weibull (forme 1,3), allongée par l'offre de bienvenue (offre tirée au hasard) ;
- **cause 2 (fermeture forcée)** : risque **constant** de 0,6 % par mois, multiplié par 2,2 pour les clients arrivés par Instagram, et **sans effet de l'offre** ;
- une censure uniforme entre 12 et 72 mois.

```

rng = np.random.default_rng(55)
n = 6000
offre = rng.integers(0, 2, n)
insta = (rng.random(n) < 0.45).astype(int)
k1 = 1.3
echelle1 = 40 * np.exp(0.40 * offre)                         # l'offre allonge de 49 % le départ
↪ volontaire (exp(0.40))
T1 = echelle1 * rng.weibull(k1, n)                           # durée potentielle de départ volontaire

```

```

taux2 = 0.006 * np.exp(0.8 * insta) # fermeture forcée : taux constant, x 2.2
↳ pour Instagram
T2 = rng.exponential(1 / taux2)
C = rng.uniform(12, 72, n)
t_obs = np.minimum.reduce([T1, T2, C])
cause = np.where(C <= np.minimum(T1, T2), 0, np.where(T1 <= T2, 1, 2))
rc = pd.DataFrame({"duree": t_obs, "cause": cause, "offre": offre, "instagram": insta})
rc.to_csv("donnees/ch05-risques-concurrents.csv", index=False) # fichier relu par R plus bas
print("effectifs par issue (0 = censuré) :", rc["cause"].value_counts().sort_index().to_dict())

tj, S, (F1, F2) = incidence_cumulee(rc["duree"], rc["cause"])
def a(t, tab): return tab[np.searchsorted(tj, t, side="right") - 1]
horizons = (12, 24, 36, 48)
tj_1, S_1, (F1_naif,) = incidence_cumulee(rc["duree"], np.where(rc["cause"] == 1, 1, 0)) # cause 2
↳ traitée comme censure
print(pd.DataFrame({"S (encore client)": [a(t, S) for t in horizons],
                    "F1 (départ volontaire)": [a(t, F1) for t in horizons],
                    "F2 (fermeture forcée)": [a(t, F2) for t in horizons],
                    "1 - KM naïf (cause 1)": [F1_naif[np.searchsorted(tj_1, t, side='right') - 1] for t
                    ↳ in horizons]},
                    index=[f"{t} mois" for t in horizons])).round(3).to_string())

```

```

effectifs par issue (0 = censuré) : {0: 2079, 1: 2655, 2: 1266}
      S (encore client)  F1 (départ volontaire)  F2 (fermeture forcée)  1 - KM naïf (cause 1)
12 mois                0.762                0.143                0.095                0.151
24 mois                0.538                0.299                0.163                0.335
36 mois                0.367                0.422                0.211                0.494
48 mois                0.251                0.514                0.235                0.627

```

Comparons à R (cmprsk::cuminc) et à lifelines. Nous avons déposé la table dans un fichier précisément pour que R puisse la relire :

```

library(cmprsk)
rc <- read.csv("donnees/ch05-risques-concurrents.csv")
ci <- cuminc(rc$duree, rc$cause)
print(round(timepoints(ci, c(12, 24, 36, 48))$est, 4))

```

```

      12      24      36      48
1 1 0.1428 0.2993 0.4217 0.5141
1 2 0.0948 0.1629 0.2109 0.2350

```

```

from lifelines import AalenJohansenFitter

aj1 = AalenJohansenFitter(calculate_variance=False, seed=1).fit(rc["duree"], rc["cause"],
↳ event_of_interest=1)
print("lifelines, F1 aux horizons :", [round(float(aj1.cumulative_density_.loc[:t].iloc[-1, 0]), 4)
↳ for t in horizons])
print("à la main, F1 aux horizons :", [round(float(a(t, F1)), 4) for t in horizons])

```

```

lifelines, F1 aux horizons : [0.1428, 0.2993, 0.4217, 0.5141]
à la main, F1 aux horizons : [0.1428, 0.2993, 0.4217, 0.5141]

```

Les trois méthodes donnent les mêmes incidences. Remarquez la dernière colonne du premier tableau : le « 1 – KM » naïf **surestime** systématiquement la probabilité de départ volontaire (par exemple 0,49 contre 0,42 à 36 mois, et 0,63 contre 0,51 à 48 mois). La simulation nous permet même de comparer à la **vérité** : pour un groupe donné,  $F_k(t) = \int_0^t h_k(u)S(u) du$  s'évalue par une intégrale numérique, en mélangeant les clients d'Instagram (45 %) et des autres canaux.

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

```

```

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
u = np.linspace(0, 100, 20001)
du = u[1] - u[0]

def vraies_cif(off):
    """CIF vraies des deux causes pour le groupe 'off', en moyennant sur la part d'Instagram (45 %)."""
    echelle = 40 * np.exp(0.40 * off)
    h1 = (k1 / echelle) * (u / echelle) ** (k1 - 1)
    H1 = (u / echelle) ** k1
    F1_v = np.zeros_like(u); F2_v = np.zeros_like(u)
    for ins, poids in ((0, 0.55), (1, 0.45)):
        l2 = 0.006 * np.exp(0.8 * ins)
        S_v = np.exp(-H1 - l2 * u)
        F1_v += poids * np.cumsum(h1 * S_v) * du
        F2_v += poids * np.cumsum(l2 * S_v) * du
    return F1_v, F2_v

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
for off, couleur, nom in [(0, VIOLET, "sans offre"), (1, AQUA, "avec offre")]:
    g = rc[rc["offre"] == off]
    tj_g, S_g, (F1_g, F2_g) = incidence_cumulee(g["duree"], g["cause"])
    v1, v2 = vraies_cif(off)
    for ax, F_g, v in ((axes[0], F1_g, v1), (axes[1], F2_g, v2)):
        ax.step(np.concatenate([0, tj_g]), np.concatenate([0, F_g]), where="post", color=couleur,
        ↪ lw=2, label=nom + " (estimée)")
        ax.plot(u, v, color=couleur, lw=1.2, ls=":", label=nom + " (vraie)")
axes[0].set_title("Cause 1 : départ volontaire")
axes[1].set_title("Cause 2 : fermeture forcée")
for ax in axes:
    ax.set_xlim(0, 70); ax.set_xlabel("mois depuis l'inscription")
    ax.legend(frameon=False, fontsize=8, loc="upper left")
axes[0].set_ylabel("probabilité cumulée (incidence)")
axes[0].set_ylim(0, 0.8)
plt.tight_layout()
plt.savefig("figures/ch05-risques-concurrents.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

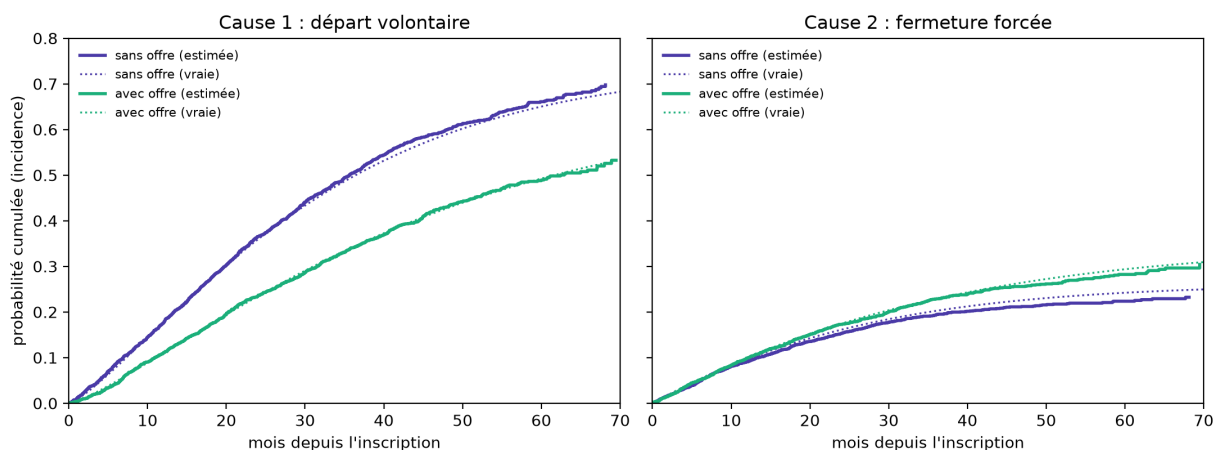


Figure 60: Incidences cumulées estimées (traits pleins) et vraies (pointillés) des deux causes de sortie, selon l'offre de bienvenue : à gauche le départ volontaire, à droite la fermeture forcée.

Observons la **cause 2**, à droite. L'offre n'a *aucun effet sur le risque de fermeture* (nous l'avons simulée ainsi). Pourtant, les deux courbes ne sont pas superposées : les clients avec offre ont une incidence de fermeture **plus élevée** (le test de Gray, plus bas, rejette l'égalité des deux courbes



avec  $p \approx 3 \times 10^{-4}$ ). Pourquoi ? Parce qu'ils partent moins volontairement, donc **restent plus longtemps exposés** au risque de fermeture. La probabilité d'une cause dépend de l'autre : c'est la compétition.

### 5.5.4 Modéliser : deux questions, deux modèles

On peut régresser sur chaque cause de deux façons, qui répondent à **deux questions différentes**.

**Le modèle de Cox par cause.** On ajuste un modèle de Cox pour la cause  $k$  en traitant les sorties par les *autres* causes comme des censures. Les coefficients sont des rapports de **risques propres à la cause** : « *parmi les clients encore là, comment la variable change-t-elle le taux de sortie par la cause  $k$  ?* ». C'est la bonne question pour **comprendre le mécanisme** (l'offre diminue-t-elle le départ volontaire ?).

**Le modèle de Fine et Gray.** Il modélise directement le **risque de sous-distribution** : le taux de sortie par la cause  $k$  parmi ceux qui *n'ont pas encore connu la cause  $k$*  (ceux qui ont eu l'autre cause restent dans l'ensemble à risque, avec un poids qui décroît). Le coefficient décrit alors l'effet de la variable **sur la CIF elle-même**. C'est la bonne question pour **prédire des probabilités** (quelle part de nos clients quittera volontairement d'ici trois ans ?).

```
from statsmodels.duration.hazard_regression import PHReg

covariables = rc[["offre", "instagram"]]
for k in (1, 2):
    m = PHReg(rc["duree"], covariables, status=(rc["cause"] == k).astype(int), ties="efron").fit()
    tab = pd.DataFrame({"HR propre à la cause": np.exp(m.params), "IC95 bas": np.exp(m.params - 1.96 *
    ↪ m.bse),
                        "IC95 haut": np.exp(m.params + 1.96 * m.bse), "p": m.pvalues}, index=["offre",
    ↪ "instagram"])
    print(f"Modèle de Cox par cause : cause {k}")
    print(tab.round(3).to_string(), "\n")
```

```
Modèle de Cox par cause : cause 1
      HR propre à la cause  IC95 bas  IC95 haut    p
offre                0.584    0.540    0.631  0.000
instagram            0.974    0.901    1.053  0.504
```

```
Modèle de Cox par cause : cause 2
      HR propre à la cause  IC95 bas  IC95 haut    p
offre                1.033    0.925    1.154  0.565
instagram            2.316    2.066    2.595  0.000
```

```
cov <- cbind(offre = rc$offre, instagram = rc$instagram)
for (k in 1:2) {
  f <- crr(rc$duree, rc$cause, cov, failcode = k, cencode = 0)
  cat("Fine-Gray, cause", k, "\n")
  print(round(summary(f)$conf.int[, c(1, 3, 4)], 3))      # exp(coef) et son IC95
  cat("\n")
}
print(cuminc(rc$duree, rc$cause, group = rc$offre)$Tests)
```

```
Fine-Gray, cause 1
      exp(coef)  2.5% 97.5%
offre      0.604 0.560 0.652
instagram  0.792 0.733 0.855
```

```
Fine-Gray, cause 2
      exp(coef)  2.5% 97.5%
offre      1.219 1.091 1.361
instagram  2.280 2.036 2.554
```

|   | stat     | pv           | df |
|---|----------|--------------|----|
| 1 | 170.5141 | 0.0000000000 | 1  |
| 2 | 12.9234  | 0.0003244992 | 1  |

Lisons ces deux familles de résultats ensemble.

- **Cause 1 (départ volontaire).** L'offre réduit le risque propre à la cause 1 :  $\widehat{HR} \approx 0,58$  (IC95 : 0,54 à 0,63), ce qui retrouve l'effet programmé ( $e^{-0,40 \times 1,3} \approx 0,59$  ; voir la conversion AFT  $\leftrightarrow$  risques proportionnels au 5.4.2). Le modèle de Fine et Gray donne un rapport de sous-distribution du même ordre (environ 0,60) : la réduction du risque se traduit en réduction de l'incidence.
- **Cause 2 (fermeture forcée).** Les clients d'Instagram ont un risque propre multiplié par 2,3 environ (IC95 : 2,07 à 2,60 ; la valeur programmée est  $e^{0,8} \approx 2,2$ ). Et l'offre n'a **aucun effet sur le risque** de fermeture : HR = 1,03 (IC95 : 0,93 à 1,15). Mais, dans le modèle de Fine et Gray, l'offre **augmente significativement l'incidence** de la cause 2 : rapport de sous-distribution d'environ 1,22 (IC95 : 1,09 à 1,36). C'est exactement l'effet de compétition que montrait la figure : l'offre ne change pas le risque de fermeture, mais, en retenant plus longtemps les clients, elle les expose plus longtemps à ce risque.
- **Instagram et la cause 1 : le piège.** Dans le modèle par cause, Instagram n'a **aucun effet** sur le départ volontaire : HR = 0,97 (IC95 : 0,90 à 1,05), et c'est la vérité. Mais dans le modèle de Fine et Gray, son rapport de sous-distribution est de 0,79 (IC95 : 0,73 à 0,86), **significativement inférieur à 1** : les clients d'Instagram ont une *incidence cumulée de départ volontaire plus faible*. Ce n'est pas qu'ils soient plus fidèles : ils sont **plus souvent éliminés avant** par la fermeture forcée, qui les empêche de partir volontairement. Le coefficient de Fine et Gray mélange l'effet direct sur la cause et l'effet de la compétition.

#### Quelle question posez-vous ?

- « Pourquoi les clients partent-ils ? » (étiologie, mécanisme) : **modèles par cause**. On y lit l'effet de chaque variable sur chaque cause, toutes choses égales.
- « Combien de clients seront perdus par chaque cause ? » (pronostic, planification) : **incidences cumulées** (Aalen-Johansen) et **modèle de Fine et Gray**.
- En cas de doute, présentez **les deux**, avec leurs interprétations.

### 5.5.5 Pièges

**L'indépendance des causes ne se teste pas.** On ne peut observer que la première des deux durées  $T_1, T_2$  : leur loi **jointe** n'est pas identifiable à partir des données. Le modèle par cause n'a pas besoin de cette hypothèse pour estimer les *risques propres* ; mais toute phrase du type « que se passerait-il si l'on supprimait la fermeture forcée ? » en a besoin, et elle est invérifiable. C'est la raison profonde pour laquelle « 1 – KM » n'a pas de sens ici.

**Ne déclarez pas la cause 2 « censure » pour calculer une probabilité.** C'est valable pour estimer un *risque*, jamais pour estimer une *probabilité cumulée*. Si l'on veut une probabilité, on utilise l'incidence cumulée.

**Événement composite.** Une autre option est de regrouper les causes (« tout départ, volontaire ou forcé ») et d'appliquer les méthodes des sections 5.1 à 5.4 : c'est licite si la question porte sur « rester client » et non sur le mécanisme. Il est alors plus sûr de **le dire**.

**À retenir**

- Avec plusieurs causes de sortie **concurrentes**, on observe la première seulement. Les risques propres  $h_k$  s'additionnent ;  $S = e^{-H_1 - H_2}$  ; la CIF de la cause  $k$  est  $F_k(t) = \int_0^t h_k(u)S(u) du$  et  $S + F_1 + F_2 = 1$ .
- **1 – Kaplan-Meier** (autres causes traitées en censure) **surestime** la probabilité d'une cause. Utilisez l'estimateur d'Aalen-Johansen  $\hat{F}_k(t) = \sum_{t_j \leq t} \hat{S}(t_j^-) d_{kj}/n_j$ .
- Le **modèle de Cox par cause** estime l'effet sur le **risque** (mécanisme) ; le modèle de **Fine et Gray** estime l'effet sur l'**incidence cumulée** (pronostic). Les deux peuvent diverger : une variable sans effet direct sur une cause peut modifier son incidence par la compétition.
- La dépendance entre causes n'est pas identifiable : on évite les phrases « si l'autre cause n'existait pas ».

**5.6 Exercices corrigés**

Les exercices sont classés par difficulté : ★ (application directe), ★★ (demande de réfléchir), ★★★ (synthèse). **Cherchez d'abord seul(e)**, à la main quand c'est demandé, avant de lire le corrigé. Les fonctions écrites dans le chapitre (kaplan\_meier, logrank, cox\_ph, test\_ph, ajuster, incidence\_cumulee...) sont réutilisées dans les corrigés.

**Exercice 1 ★ (censure et Kaplan-Meier à la main).** Yasmine suit six clients : (4;parti), (7;censuré), (9;parti), (12;parti), (15;censuré), (20;censuré) (durées en mois). (a) Calculez la moyenne de toutes les durées, puis celle des seuls clients partis. (b) Calculez la courbe de Kaplan-Meier à la main. (c) Donnez la médiane de survie. (d) Calculez la durée moyenne restreinte jusqu'à 20 mois.

**Exercice 2 ★ (relations entre les fonctions).** Un client  $a$ , à l'âge  $t$  de la relation (en mois), un risque instantané  $h(t) = 0,0008 t$ . (a) Déduisez  $H(t)$  et  $S(t)$ . (b) Calculez  $S(24)$  et la médiane. (c) De quelle loi de Weibull s'agit-il ? Donnez sa durée moyenne.

**Exercice 3 ★ (taux constant).** Sur un échantillon, on observe 40 départs pour un total de 1 600 mois-clients d'exposition. (a) Estimez le taux de départ mensuel  $\lambda$  (exponentielle), son écart-type et un intervalle de confiance à 95 %. (b) Estimez  $S(24)$  et la durée moyenne. (c) Testez  $H_0 : \lambda = 0,03$  par le test de Wald et par le rapport de vraisemblance.

**Exercice 4 ★★ (Greenwood).** Huit clients : (2;1), (3;1), (3;1), (5;0), (6;1), (8;0), (9;1), (11;0) (durée ; 1 = parti, 0 = censuré). Construisez le tableau de Kaplan-Meier (avec les ex aequo), puis donnez  $\hat{S}(6)$ , son erreur standard de Greenwood et son intervalle de confiance log-log à 95 %.

**Exercice 5 ★★ (log-rank).** Deux groupes de cinq clients. Groupe A : (3;1), (6;1), (8;0), (10;1), (12;0). Groupe B : (5;1), (7;1), (9;1), (11;0), (13;0). Calculez à la main le test du log-rank (tableau aux instants de départ : ensembles à risque, départs attendus, variances), concluez, puis vérifiez avec statsmodels.

**Exercice 6 ★★ (durée moyenne restreinte).** Les courbes de survie de deux campagnes sont des escaliers. Campagne A :  $S = 1$  jusqu'à 6 mois, 0,8 sur  $[6, 12[$ , 0,5 sur  $[12, 18[$ , 0,3 ensuite. Campagne B :  $S = 1$  jusqu'à 9 mois, 0,9 sur  $[9, 15[$ , 0,7 sur  $[15, 21[$ , 0,6 ensuite. Calculez la durée moyenne restreinte à 24 mois de chaque campagne et leur différence. Que signifie ce nombre ?

**Exercice 7 ★★ (lire une sortie de Cox).** Un modèle de Cox donne : offre de bienvenue  $\hat{\beta} = -0,40$  (ET 0,065), âge  $\hat{\beta} = -0,0136$  (ET 0,0030), canal Instagram (contre Boutique)  $\hat{\beta} = 0,635$  (ET 0,084). (a) Donnez pour chaque variable le rapport de risques, son IC95 et le  $z$  de Wald. (b) Quel est l'effet de dix années d'âge de plus ? (c) Quel est le rapport de risques d'un client Instagram avec offre contre un client Boutique sans offre, à âge égal ? (d) La survie à 24 mois d'un client de référence

est de 0,80 : quelle est celle du même client avec l'offre ? (e) Un AFT Weibull donne  $\hat{\gamma}_{\text{offre}} = 0,35$  avec  $\hat{\sigma} = 0,74$  : quel rapport de risques équivalent ?

**Exercice 8 ★★ (le biais d'immortalité).** Simulez 2 000 clients dont les durées sont exponentielles de taux 2 % par mois (graine 8), censurées uniformément entre 18 et 48 mois. Un cadeau est remis au mois 6 à la moitié des clients encore présents. Estimez l'effet du cadeau (qui n'en a aucun) (a) en traitant « a reçu le cadeau » comme une variable fixe, (b) correctement, avec une variable dépendant du temps. Commentez.

**Exercice 9 ★★ (estimer la forme de Weibull de deux façons).** Pour l'ensemble des 2 000 clients (sans covariables), estimez la forme  $k$  de la loi de Weibull (a) par le graphique de Weibull : régression de  $\ln(-\ln \hat{S}_{KM})$  sur  $\ln t$  entre 6 et 60 mois ; (b) par maximum de vraisemblance. Comparez, et expliquez l'écart avec la valeur  $k = 1,35$  utilisée pour simuler.

**Exercice 10 ★★★ (log-rank et Cox).** (a) Montrez que le score du modèle de Cox à une variable binaire en  $\beta = 0$  vaut  $O_1 - E_1$ . (b) Vérifiez numériquement, sur les 2 000 clients (variable offre\_bienvenue), que la statistique de score  $U^2/I$  est très proche du  $\chi^2$  du log-rank. Pourquoi n'est-elle pas *exactement* égale ?

**Exercice 11 ★★★ (décider : le seuil de rentabilité).** Reprenez les survies moyennes avec et sans offre du 5.4.5. L'offre de bienvenue coûte maintenant 25 DT par client. À partir de quelle **marge mensuelle** par client actif est-elle rentable, pour un taux d'actualisation de 1 % par mois ? Et pour 2 % ?

**Exercice 12 ★★★ (risques concurrents à la main).** Huit clients, durées et causes de sortie (0 = censuré, 1 = départ volontaire, 2 = fermeture forcée) : (1;1), (2;2), (3;1), (4;0), (5;2), (6;1), (7;0), (9;2). Calculez à la main les incidences cumulées  $\hat{F}_1$  et  $\hat{F}_2$  (estimateur d'Aalen-Johansen) et la survie totale. Vérifiez que  $\hat{S} + \hat{F}_1 + \hat{F}_2 = 1$ . Comparez  $\hat{F}_1$  à « 1 - KM » où la cause 2 est traitée comme une censure.

**Exercice 13 ★★★ (proportionnalité des risques sur des données réelles).** Reprenez les données de récurrence de Rossi (5.3.9). Appliquez le test de score de proportionnalité des risques du 5.3.5 à chacune des sept covariables. Quelles variables posent problème ? Que feriez-vous ?

## Corrigés

**Corrigé 1.** (a) Moyenne de toutes les durées :  $(4+7+9+12+15+20)/6 \approx 11,17$  mois ; moyenne des seuls clients partis :  $(4+9+12)/3 \approx 8,33$  mois. Les deux sont biaisées (5.1.1). (b) Départs aux mois 4, 9 et 12. À 4 mois, 6 clients à risque :  $5/6 = 0,833$ . À 9 mois (le client censuré à 7 est sorti), 4 clients à risque :  $0,833 \times 3/4 = 0,625$ . À 12 mois, 3 clients à risque :  $0,625 \times 2/3 = 0,417$ . (c) La courbe passe sous 0,5 au mois 12 : **médiane = 12 mois**. (d)  $RMST(20) = 4 \times 1 + 5 \times 0,833 + 3 \times 0,625 + 8 \times 0,417 = 4 + 4,167 + 1,875 + 3,333 = 13,375$  mois.

```
import numpy as np
import pandas as pd
from scipy import stats

y1 = np.array([4, 7, 9, 12, 15, 20.]); d1 = np.array([1, 0, 1, 1, 0, 0])
tj1, n1, dj1, S1_, gw1 = kaplan_meier(y1, d1)
print("moyenne de toutes les durées :", round(y1.mean(), 2), "| des seuls partis :", round(y1[d1 ==
↪ 1].mean(), 2))
print("S aux instants de départ", tj1, ":", S1_.round(4), "| à risque :", n1)
print("médiane :", tj1[S1_ <= 0.5][0], "| RMST(20) :", round(rmst(tj1, S1_, 20), 3))
```

```
moyenne de toutes les durées : 11.17 | des seuls partis : 8.33
S aux instants de départ [ 4.  9. 12.] : [0.8333 0.625  0.4167] | à risque : [6 4 3]
```

médiane : 12.0 | RMST(20) : 13.375

**Corrigé 2.** (a)  $H(t) = \int_0^t 0,0008u \, du = 0,0004t^2$ , donc  $S(t) = \exp(-0,0004t^2)$ . (b)  $S(24) = \exp(-0,0004 \times 576) = e^{-0,2304} \approx 0,794$  ; la médiane vérifie  $0,0004t^2 = \ln 2$ , donc  $t = \sqrt{\ln 2 / 0,0004} \approx 41,6$  mois. (c) Une Weibull a  $H(t) = (t/\sigma)^k$  : on lit  $k = 2$  et  $\sigma^{-2} = 0,0004$ , soit  $\sigma = 50$  mois. La durée moyenne vaut  $\sigma \Gamma(1 + 1/k) = 50 \Gamma(1,5) \approx 44,3$  mois.

```
from math import gamma, log
from scipy.integrate import quad
print("S(24) =", round(np.exp(-0.0004 * 24 ** 2), 4), "| médiane =", round(np.sqrt(log(2) / 0.0004),
↪ 2))
print("moyenne par la formule :", round(50 * gamma(1.5), 2), "| par intégration de S :",
↪ round(quad(lambda t: np.exp(-0.0004 * t ** 2), 0, np.inf)[0], 2))
```

S(24) = 0.7942 | médiane = 41.63  
moyenne par la formule : 44.31 | par intégration de S : 44.31

**Corrigé 3.** (a)  $\hat{\lambda} = D/E = 40/1600 = 0,025$  par mois ;  $\widehat{se} = \hat{\lambda}/\sqrt{D} = 0,025/\sqrt{40} \approx 0,00395$  ; IC95 :  $0,025 \pm 1,96 \times 0,00395$ , soit  $[0,0173 ; 0,0327]$ . (b)  $\hat{S}(24) = e^{-0,025 \times 24} = e^{-0,6} \approx 0,549$  ; durée moyenne  $1/\hat{\lambda} = 40$  mois. (c) Wald :  $z = (0,025 - 0,03)/0,00395 \approx -1,26$ ,  $p \approx 0,21$ . Rapport de vraisemblance :  $2[\ell(\hat{\lambda}) - \ell(0,03)] = 2[D \ln(\hat{\lambda}/0,03) - (\hat{\lambda} - 0,03)E] = 2[40 \ln(0,8333) + 8] \approx 1,41$ ,  $p \approx 0,23$ . Les deux tests concluent de la même façon : **on ne rejette pas**  $\lambda = 0,03$  (les données sont compatibles aussi avec ce taux).

```
D, E = 40, 1600
lam = D / E; se = lam / np.sqrt(D)
print(f"lambda = {lam:.4f}, ET = {se:.5f}, IC95 = [{lam - 1.96 * se:.4f} ; {lam + 1.96 * se:.4f}]")
print(f"S(24) = {np.exp(-lam * 24):.4f}, durée moyenne = {1 / lam:.1f} mois")
z = (lam - 0.03) / se
lr = 2 * (D * np.log(lam / 0.03) - (lam - 0.03) * E)
print(f"Wald : z = {z:.3f}, p = {2 * stats.norm.sf(abs(z)):.3f} | rapport de vraisemblance : chi2 =
↪ {lr:.3f}, p = {stats.chi2.sf(lr, 1):.3f}")
```

lambda = 0.0250, ET = 0.00395, IC95 = [0.0173 ; 0.0327]  
S(24) = 0.5488, durée moyenne = 40.0 mois  
Wald : z = -1.265, p = 0.206 | rapport de vraisemblance : chi2 = 1.414, p = 0.234

**Corrigé 4.** Départs : mois 2 (1 départ, 8 à risque : 7/8), mois 3 (2 départs, 7 à risque : 5/7), mois 6 (1 départ, 4 à risque, car le client censuré à 5 est sorti : 3/4), mois 9 (1 départ, 2 à risque : 1/2). D'où  $\hat{S}(2) = 0,875$ ,  $\hat{S}(3) = 0,875 \times 5/7 = 0,625$ ,  $\hat{S}(6) = 0,625 \times 3/4 = 0,469$ ,  $\hat{S}(9) = 0,234$ . Greenwood :  $\sum \frac{d_j}{n_j(n_j - d_j)} = \frac{1}{8 \times 7} + \frac{2}{7 \times 5} + \frac{1}{4 \times 3} = 0,0179 + 0,0571 + 0,0833 = 0,1583$  ; l'erreur standard de  $\hat{S}(6)$  vaut  $0,469 \times \sqrt{0,1583} \approx 0,187$ . Intervalle log-log :  $\hat{S}^{\exp(\pm 1,96 \sqrt{G}/|\ln \hat{S}|)}$  avec  $1,96 \times \sqrt{0,1583}/|\ln 0,469| = 0,780/0,757 = 1,03$ , d'où  $[0,469^{2,80} ; 0,469^{1/2,80}] \approx [0,12 ; 0,76]$  : un intervalle immense, faute de données.

```
y4 = np.array([2, 3, 3, 5, 6, 8, 9, 11.]); d4 = np.array([1, 1, 1, 0, 1, 0, 1, 0])
tj4, n4, dj4, S4, gw4 = kaplan_meier(y4, d4)
print(pd.DataFrame({"t_j": tj4, "à risque": n4, "départs": dj4, "S": S4.round(4), "somme Greenwood":
↪ gw4.round(4)}).to_string(index=False))
s, plan, log_, loglog = intervalles(6, tj4, S4, gw4)
print(f"S(6) = {s:.4f} ; ET = {s * np.sqrt(surv_at(6, tj4, gw4)):.4f} ; IC log-log = [{loglog[0]:.3f}
↪ ; {loglog[1]:.3f}])")
```

| t_j | à risque | départs | S      | somme Greenwood |
|-----|----------|---------|--------|-----------------|
| 2.0 | 8        | 1       | 0.8750 | 0.0179          |
| 3.0 | 7        | 2       | 0.6250 | 0.0750          |
| 6.0 | 4        | 1       | 0.4688 | 0.1583          |
| 9.0 | 2        | 1       | 0.2344 | 0.6583          |

$S(6) = 0.4688$  ;  $ET = 0.1865$  ;  $IC \log\text{-}\log = [0.120 ; 0.763]$

**Corrigé 5.** Aux instants de départ 3, 5, 6, 7, 9 et 10, on a respectivement  $n_j = 10, 9, 8, 7, 5, 4$  clients à risque, dont  $n_{Aj} = 5, 4, 4, 3, 2, 2$  dans le groupe A. Chaque instant compte un seul départ :  $E_{Aj} = n_{Aj}/n_j$  et  $V_j = \frac{n_{Aj}}{n_j} (1 - \frac{n_{Aj}}{n_j})$ . Le code donne le détail :

```
y5 = np.array([3, 6, 8, 10, 12, 5, 7, 9, 11, 13.]); d5 = np.array([1, 1, 0, 1, 0, 1, 1, 1, 0, 0])
g5 = np.array(["A"] * 5 + ["B"] * 5)
lignes, O1, E1, V1 = [], 0, 0.0, 0.0
for t in np.unique(y5[d5 == 1]):
    n_ = int(np.sum(y5 >= t)); nA = int(np.sum((y5 >= t) & (g5 == "A")))
    dd = int(np.sum((y5 == t) & (d5 == 1))); dA = int(np.sum((y5 == t) & (d5 == 1) & (g5 == "A")))
    e = dd * nA / n_; v = dd * (nA / n_) * (1 - nA / n_) * (n_ - dd) / (n_ - 1)
    lignes.append((t, n_, nA, dd, dA, round(e, 4), round(v, 4))); O1 += dA; E1 += e; V1 += v
print(pd.DataFrame(lignes, columns=["t_j", "n_j", "n_Aj", "d_j", "départ en A", "E_Aj",
    ↪ "V_j"]).to_string(index=False))
chi5 = (O1 - E1) ** 2 / V1
print(f"O_A = {O1}, E_A = {E1:.3f}, V = {V1:.3f}, chi2 = {chi5:.3f}, p = {stats.chi2.sf(chi5, 1):.3f}")
from statsmodels.duration.survfunc import survdiff
print("statsmodels : chi2 = %.3f, p = %.3f" % survdiff(y5, d5, g5))
```

| t_j  | n_j | n_Aj | d_j | départ en A | E_Aj   | V_j    |
|------|-----|------|-----|-------------|--------|--------|
| 3.0  | 10  | 5    | 1   | 1           | 0.5000 | 0.2500 |
| 5.0  | 9   | 4    | 1   | 0           | 0.4444 | 0.2469 |
| 6.0  | 8   | 4    | 1   | 1           | 0.5000 | 0.2500 |
| 7.0  | 7   | 3    | 1   | 0           | 0.4286 | 0.2449 |
| 9.0  | 5   | 2    | 1   | 0           | 0.4000 | 0.2400 |
| 10.0 | 4   | 2    | 1   | 1           | 0.5000 | 0.2500 |

O\_A = 3, E\_A = 2.773, V = 1.482, chi2 = 0.035, p = 0.852  
statsmodels : chi2 = 0.035, p = 0.852

Le groupe A a 3 départs pour 2,77 attendus :  $\chi^2 \approx 0,03, p \approx 0,85$ . Avec dix clients, on ne peut rien conclure ; c'est tout à fait normal, et c'est la raison pour laquelle on ne compare pas des groupes aussi petits.

**Corrigé 6.**  $RMST_A(24) = 6 \times 1 + 6 \times 0,8 + 6 \times 0,5 + 6 \times 0,3 = 6 + 4,8 + 3 + 1,8 = 15,6$  mois ;  $RMST_B(24) = 9 \times 1 + 6 \times 0,9 + 6 \times 0,7 + 3 \times 0,6 = 9 + 5,4 + 4,2 + 1,8 = 20,4$  mois. La différence est de **4,8 mois** : en moyenne, sur les 24 premiers mois, un client de la campagne B reste 4,8 mois de plus dans la clientèle qu'un client de la campagne A. Contrairement au rapport de risques, cette quantité s'interprète sans hypothèse de proportionnalité et se lit directement en unités de temps (ou, multipliée par la marge mensuelle, en dinars).

```
tjA, SA = np.array([6, 12, 18.]), np.array([0.8, 0.5, 0.3])
tjB, SB = np.array([9, 15, 21.]), np.array([0.9, 0.7, 0.6])
print("RMST(24) A : ", round(rmst(tjA, SA, 24), 2), "| B : ", round(rmst(tjB, SB, 24), 2), "| différence
    ↪ : ", round(rmst(tjB, SB, 24) - rmst(tjA, SA, 24), 2))
```

RMST(24) A : 15.6 | B : 20.4 | différence : 4.8

**Corrigé 7.** (a)  $HR = e^{\hat{\beta}}, IC = e^{\hat{\beta} \pm 1,96 se}, z = \hat{\beta}/se$  : offre 0,670 ([0,590 ; 0,761],  $z = -6,15$ ) ; âge 0,986 ([0,981 ; 0,992],  $z = -4,53$ ) ; Instagram 1,887 ([1,60 ; 2,22],  $z = 7,56$ ). (b)  $e^{10 \times (-0,0136)} \approx 0,873$  : dix ans de plus réduisent le risque d'environ 13 %. (c) Les effets se multiplient :  $e^{-0,40+0,635} = e^{0,235} \approx 1,26$  (le canal Instagram l'emporte sur l'offre). (d)  $S(t|x) = S_0(t)^{\exp(x^T \beta)}$  :  $0,80^{0,670} \approx 0,861$ . (e)  $\hat{\beta} = -\hat{\gamma}/\hat{\sigma} = -0,35/0,74 \approx -0,473$ , soit  $HR = e^{-0,473} \approx 0,623$ .

```
b = np.array([-0.40, -0.0136, 0.635]); se_ = np.array([0.065, 0.0030, 0.084])
print(pd.DataFrame({"HR": np.exp(b), "IC bas": np.exp(b - 1.96 * se_), "IC haut": np.exp(b + 1.96 *
    ↪ se_), "z": b / se_}, index=["offre", "age", "Instagram"]).round(3).to_string())
print("+10 ans : ", round(np.exp(10 * b[1]), 3), "| Instagram avec offre / Boutique sans offre : ",
    ↪ round(np.exp(b[0] + b[2]), 3))
```



```
print("S(24) avec offre :", round(0.80 ** np.exp(b[0]), 3), "| HR depuis AFT :", round(np.exp(-0.35 /
↪ 0.74), 3))
```

```
HR IC bas IC haut z
offre 0.670 0.590 0.761 -6.154
age 0.986 0.981 0.992 -4.533
Instagram 1.887 1.601 2.225 7.560
+10 ans : 0.873 | Instagram avec offre / Boutique sans offre : 1.265
S(24) avec offre : 0.861 | HR depuis AFT : 0.623
```

**Corrigé 8.** Dans l'analyse (a), le cadeau n'est donné qu'à ceux qui ont **survécu** jusqu'au mois 6 : ils ont une avance garantie, et le modèle y voit un effet protecteur qui n'existe pas. Dans l'analyse (b), le cadeau est une variable qui passe de 0 à 1 au mois 6 (deux épisodes pour les clients concernés), et seuls les clients **présents au mois 6** sont comparés entre eux.

```
from statsmodels.duration.hazard_regression import PHReg

rng = np.random.default_rng(8)
N = 2000
T = rng.exponential(1 / 0.02, N)
C = rng.uniform(18, 48, N)
Y, D = np.minimum(T, C), (T <= C).astype(int)
cadeau = (Y > 6) & (rng.random(N) < 0.5)
naif = PHReg(Y, cadeau.astype(float)[: , None], status=D, ties="efron").fit()
porteurs = np.where(cadeau)[0]
debut = np.concatenate([np.zeros(N), np.full(len(porteurs), 6.0)])
fin = np.concatenate([np.where(cadeau, 6.0, Y), Y[porteurs]])
statut = np.concatenate([np.where(cadeau, 0, D), D[porteurs]])
x_tv = np.concatenate([np.zeros(N), np.ones(len(porteurs))])
juste = PHReg(fin, x_tv[: , None], status=statut, entry=debut, ties="efron").fit()
for nom, m in (("naïve (variable fixe)", naif), ("correcte (dépend du temps)", juste)):
    print(f"{nom:<28} HR = {np.exp(m.params[0]):.2f} IC95 [{np.exp(m.params[0] - 1.96 *
↪ m.bse[0]):.2f} ; {np.exp(m.params[0] + 1.96 * m.bse[0]):.2f}])")

naïve (variable fixe) HR = 0.64 IC95 [0.56 ; 0.74]
correcte (dépend du temps) HR = 1.00 IC95 [0.86 ; 1.16]
```

L'analyse naïve conclut à un effet protecteur net (un HR bien inférieur à 1), l'analyse correcte à un effet nul (HR proche de 1, intervalle contenant 1) : c'est le **biais d'immortalité**.

**Corrigé 9.** (a) Si  $S(t) = \exp[-(t/\sigma)^k]$ , alors  $\ln(-\ln S) = k \ln t - k \ln \sigma$  : la pente de la droite est  $k$ . (b) Le maximum de vraisemblance sans covariable est l'estimation de  $k = 1/\hat{\sigma}$  dans ajuster avec une seule colonne de 1.

```
c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
tj_, _, S_, _ = kaplan_meier(y, d)
garde = (tj_ >= 6) & (tj_ <= 60)
pente, ord_, r, _ = stats.linregress(np.log(tj_[garde]), np.log(-np.log(S_[garde])))
mv = ajuster("weibull", y, d, np.ones((len(y), 1)))
print(f"forme par le graphique de Weibull : k = {pente:.3f} (r = {r:.3f}) | forme par maximum de
↪ vraisemblance : k = {1 / np.exp(mv['theta'][-1]):.3f}")
```

```
forme par le graphique de Weibull : k = 1.296 (r = 1.000) | forme par maximum de vraisemblance : k =
↪ 1.281
```

Les deux méthodes donnent une forme voisine de 1,3 (1,30 et 1,28) et légèrement inférieure à la vraie valeur 1,35. La raison est celle du 5.4.7 : en **ignorant les covariables** (offre, canal, âge, facteur de service non observé), le risque de la population est un mélange de risques individuels, dont la croissance apparente est plus lente. L'ajout des covariables mesurées dans le modèle du 5.4.3 faisait passer la forme à 1,32, et l'ajout du facteur non observé (modèle « oracle » du 5.4.7) à 1,40



: la vraie valeur (1,35) n'est approchée, à l'erreur d'échantillonnage près, que si l'on tient compte de *toutes* les covariables, y compris celles qu'on ne mesure pas. Les deux méthodes d'estimation (graphique, maximum de vraisemblance) s'accordent entre elles ; c'est le *modèle* qui est incomplet, pas la méthode.

**Corrigé 10.** (a) En  $\beta = 0$ ,  $e^{\beta x_k} = 1$  : la moyenne pondérée de  $x$  dans l'ensemble à risque  $R_j$  est la proportion  $n_{1j}/n_j$  de clients du groupe 1. Le score est  $\sum_j \sum_{i \text{ parti en } t_j} (x_i - n_{1j}/n_j) = \sum_j (d_{1j} - d_j n_{1j}/n_j) = O_1 - E_1$ . (b) L'information en  $\beta = 0$  est  $\sum_i \text{Var}_{R_i}(x) = \sum_i \frac{n_i}{n} (1 - \frac{n_i}{n})$  (somme sur *chaque* départ), alors que le log-rank utilise la variance hypergéométrique, avec le facteur  $\frac{n_j - d_j}{n_j - 1}$  qui corrige les départs simultanés. Les deux coïncident exactement quand il n'y a **jamais** d'ex aequo.

```
off = c["offre_bienvenue"].to_numpy().astype(float)
U0, I0 = score_info(0.0, y, d, off) # fonction du 5.3.2 : boucle sur chaque
↳ départ (ex aequo à la Breslow)
O, E, chi_lr, _, _ = logrank(y, d, off)
print(f"Cox : score U = {U0:.3f}, information I = {I0:.3f}, U²/I = {U0 ** 2 / I0:.3f}")
print(f"log-rank : O1 - E1 = {O[1] - E[1]:.3f}, chi2 = {chi_lr:.3f}")
```

```
Cox : score U = -92.383, information I = 240.231, U²/I = 35.527
log-rank : O1 - E1 = -92.383, chi2 = 35.535
```

Le score est exactement  $O_1 - E_1$  (même valeur, signe compris, puisque  $x = 1$  désigne le groupe avec offre), et les deux statistiques ne diffèrent que de 0,008 sur 35,5 : la différence vient de la correction des ex aequo dans la variance.

**Corrigé 11.** Le gain actualisé de l'offre vaut  $\Delta = \sum_t (S_1(t) - S_0(t))(1 + r)^{-t}$  « mois de présence actualisés » ; l'offre est rentable si  $m \Delta > 25$ , c'est-à-dire  $m > m^* = 25/\Delta$ .

```
cout = 25.0
for r in (0.01, 0.02):
    v = (1 + r) ** (-mois) # `mois`, S0 et S1 viennent du 5.4.5
    delta = np.sum((S1 - S0) * v)
    print(f"taux {100 * r:.0f} %/mois : gain en mois de présence actualisés = {delta:.3f} -> marge de
    ↳ rentabilité m* = {cout / delta:.2f} DT/mois")
```

```
taux 1 %/mois : gain en mois de présence actualisés = 6.663 -> marge de rentabilité m* = 3.75 DT/mois
taux 2 %/mois : gain en mois de présence actualisés = 4.124 -> marge de rentabilité m* = 6.06 DT/mois
```

Avec une actualisation de 1 % par mois, l'offre devient rentable dès que la marge mensuelle par client actif dépasse environ **3,75 DT** ; avec 2 %, il faut environ **6 DT**. L'offre est donc beaucoup moins coûteuse à justifier si la marge est élevée : c'est la lecture pratique du tableau de sensibilité du 5.4.5.

**Corrigé 12.** Instants de sortie : 1, 2, 3, 5, 6, 9 (les censures aux mois 4 et 7 réduisent seulement les ensembles à risque). Aux mois 1 ( $n = 8$ ) :  $d_1 = 1$  ; 2 ( $n = 7$ ) :  $d_2 = 1$  ; 3 ( $n = 6$ ) :  $d_1 = 1$  ; 5 ( $n = 4$ ) :  $d_2 = 1$  ; 6 ( $n = 3$ ) :  $d_1 = 1$  ; 9 ( $n = 1$ ) :  $d_2 = 1$ . On applique  $\hat{F}_k(t) = \sum \hat{S}(t_j^-) d_{kj}/n_j$ .

```
y12 = np.array([1, 2, 3, 4, 5, 6, 7, 9.]); c12 = np.array([1, 2, 1, 0, 2, 1, 0, 2])
tj12, S12, (F1_12, F2_12) = incidence_cumulee(y12, c12)
print(pd.DataFrame({"t_j": tj12, "S": S12, "F1": F1_12, "F2": F2_12, "S + F1 + F2": S12 + F1_12 +
↳ F2_12}).round(4).to_string(index=False))
tj_n12, S_n12, (F1_naif12,) = incidence_cumulee(y12, np.where(c12 == 1, 1, 0))
print("1 - KM (cause 2 traitée comme censure), aux instants de la cause 1 :", (1 - S_n12).round(4), "|
↳ F1 correcte au dernier instant de cause 1 :", round(F1_12[-2], 4))
```

| t_j | S      | F1     | F2     | S + F1 + F2 |
|-----|--------|--------|--------|-------------|
| 1.0 | 0.8750 | 0.1250 | 0.0000 | 1.0         |
| 2.0 | 0.7500 | 0.1250 | 0.1250 | 1.0         |

```

3.0 0.6250 0.2500 0.1250      1.0
5.0 0.4688 0.2500 0.2812      1.0
6.0 0.3125 0.4062 0.2812      1.0
9.0 0.0000 0.4062 0.5938      1.0
1 - KM (cause 2 traitée comme censure), aux instants de la cause 1 : [0.125  0.2708 0.5139] | F1
↪ correcte au dernier instant de cause 1 : 0.4062

```

La somme  $\hat{S} + \hat{F}_1 + \hat{F}_2$  vaut 1 à chaque instant. À 6 mois, la probabilité de départ volontaire est  $\hat{F}_1(6) = 0,406$  ; en traitant la cause 2 comme une censure, on obtiendrait « 1 – KM » = 0,514, soit 11 points de trop : la compétition des fermetures forcées (aux mois 2, 5 et 9) est ignorée.

**Corrigé 13.** On applique `test_ph` à l'ajustement de Cox des données de Rossi (la fonction `cox_ph` du 5.3.3 accepte n'importe quelle matrice de covariables).

```

from lifelines.datasets import load_rossi

rossi = load_rossi()
Xr = rossi.drop(columns=["week", "arrest"])
fit_r = cox_ph(rossi["week"].to_numpy(), rossi["arrest"].to_numpy(), Xr.to_numpy())
chi_r, chi_glob_r = test_ph(rossi["week"].to_numpy(), rossi["arrest"].to_numpy(), Xr.to_numpy(),
    ↪ fit_r)
out = pd.DataFrame({"HR (Breslow)": np.exp(fit_r["beta"]), "chi2 PH": chi_r, "p":
    ↪ stats.chi2.sf(chi_r, 1)}, index=Xr.columns)
print(out.round(3).to_string())
print(f"test global : chi2 = {chi_glob_r:.2f} (7 ddl), p = {stats.chi2.sf(chi_glob_r, 7):.3f}")

```

```

      HR (Breslow)  chi2 PH      p
fin           0.685    1.371  0.242
age           0.944    0.893  0.345
race          1.369    2.245  0.134
wexp          0.860    4.128  0.042
mar           0.649    0.089  0.765
paro          0.919    0.021  0.886
prio          1.095    1.618  0.203
test global : chi2 = 10.93 (7 ddl), p = 0.142

```

La proportionnalité est plausible pour six des sept covariables. Seule l'expérience professionnelle (`wexp`) a une p-valeur inférieure à 0,05 ( $p = 0,042$ ), et le test **global** ne rejette rien ( $p = 0,14$ ). Avec sept tests, obtenir une p-valeur à 0,04 arrive souvent par hasard (volume I, section 3.5.5) : ce n'est pas une preuve de violation. La démarche prudente : tracer le graphique log-log de `wexp`, et si un doute subsiste, **stratifier** sur cette variable (elle est binaire) ou ajouter un effet qui dépend du temps, puis voir si les autres coefficients, en particulier celui de l'aide financière, bougent. S'ils ne bougent pas, la conclusion principale est robuste.

## Bilan du chapitre 5

Vous savez maintenant :

- **reconnaître la censure** et expliquer pourquoi la moyenne des durées, la moyenne des seuls événements et la proportion d'événements sont toutes **biaisées** ; distinguer censure à droite, à gauche, par intervalle et **troncature** (entrée tardive) ; énoncer l'hypothèse de **censure non informative** ;
- manier les trois fonctions d'une durée, **survie**  $S(t)$ , **risque instantané**  $h(t)$  et **risque cumulé**  $H(t)$ , reliées par  $S = e^{-H}$ , et la formule  $E[T] = \int_0^\infty S(t) dt$  ;
- écrire la **vraisemblance avec censure**  $\ell = \sum [\delta_i \ln h(y_i) - H(y_i)]$  et en tirer le taux constant  $\hat{\lambda} = D / \sum y_i$  ;

- calculer l'**estimateur de Kaplan-Meier** à la main et par code, ses intervalles de confiance (**Greenwood**, log-log), la **médiane** et la **durée moyenne restreinte** ; comparer des groupes par le **log-rank** et ses variantes ;
- ajuster et interpréter un **modèle de Cox** : vraisemblance partielle, rapports de risques, **ex aequo** (Breslow, Efron), **vérification de la proportionnalité** (graphique log-log, test de score de Grambsch-Therneau), **covariables dépendant du temps** et **biais d'immortalité**, courbes de survie prédites, **indice de concordance** ;
- ajuster des **modèles paramétriques** (Weibull, log-normal, log-logistique) par maximum de vraisemblance, lire un **facteur d'accélération**, passer de l'AFT aux risques proportionnels ( $\beta = -\gamma/\sigma$  pour la Weibull), choisir par l'AIC et les **résidus de Cox-Snell**, **extrapoler** et calculer une **valeur vie client** avec une analyse de sensibilité ;
- (en option) traiter des **risques concurrents** : incidences cumulées d'**Aalen-Johansen**, pourquoi «  $1 - \text{KM}$  » est faux, **modèle par cause** contre **Fine et Gray**.

Deux messages à garder en mémoire. **Un chiffre de survie n'a de sens qu'avec son traitement de la censure et ses hypothèses** (non informative, proportionnalité, forme de la loi) : on les énonce, on les vérifie quand c'est possible, on mesure leur influence sinon. Et **la randomisation reste l'arme la plus solide** : l'offre de bienvenue a un effet causal mesurable parce qu'elle a été attribuée au hasard ; pour les variables non randomisées (le canal, l'âge), les résultats sont des **associations**.

Le chapitre 6 change de perspective : au lieu de chercher *une* estimation et son incertitude, la **statistique bayésienne** attribue une **loi de probabilité** aux paramètres eux-mêmes, et la **simulation** (Monte-Carlo, MCMC) permet de calculer ce que l'algèbre ne sait pas faire.

# Chapitre 6 : Statistique bayésienne et simulation

« Toute connaissance est une croyance qu'on a promis de réviser à la lumière des faits.  
»

Dans le volume I, nous avons mesuré l'incertitude avec des **intervalles de confiance** et des **p-valeurs**. Cette façon de faire, dite *fréquentiste*, répond à la question : « *si je refaisais mon étude des milliers de fois, quelle serait la fréquence de telle ou telle erreur de ma méthode ?* » C'est une réponse rigoureuse... mais ce n'est pas celle que Yasmine attend. Yasmine demande plutôt : « *vu ce que j'ai observé, quelle est la probabilité que l'offre de bienvenue fonctionne ?* ». Une **probabilité sur le paramètre lui-même**.

La statistique **bayésienne** répond directement à cette question, en appliquant la formule de Bayes (volume I, section 2.1) non plus à des événements, mais à des **paramètres inconnus**. Elle y ajoute un ingrédient que le fréquentisme refuse : **ce que l'on sait avant de regarder les données** (la loi *a priori*). En contrepartie, elle exige souvent des calculs d'intégrales que personne ne sait faire à la main : c'est là qu'intervient la **simulation** (Monte-Carlo, MCMC), qui est devenue l'outil central de la modélisation moderne, bayésienne ou non.

## Le chemin de ce chapitre

- **6.1 Inférence bayésienne et lois a priori** : la formule de Bayes pour un paramètre, des exemples à la main, les lois conjuguées, le choix de l'a priori, la loi prédictive.
- **6.2 Méthodes de Monte-Carlo** : estimer une espérance ou une probabilité en **simulant**, vitesse de convergence en  $1/\sqrt{n}$ , échantillonnage préférentiel, réduction de variance.
- **6.3 MCMC** : quand on ne sait pas simuler directement, on fabrique une **chaîne de Markov** dont la loi limite est celle qu'on veut. Metropolis-Hastings et Gibbs, écrits en quelques lignes de NumPy.
- **6.4 Vérification des modèles bayésiens** : savoir si la chaîne a convergé, si le modèle est plausible, comment comparer deux modèles.
- **⊕ Pour aller plus loin** : la théorie des **valeurs extrêmes** (6.5) et les **copules** (6.6), qui modélisent les événements rares et la dépendance.
- **6.7 Exercices corrigés**.

**Comment lire ce chapitre.** Les sections 6.1 et 6.2 sont indépendantes l'une de l'autre et se lisent bien d'un trait. La section 6.3 suppose 6.1 (pour le sens de la loi *a posteriori*) et 6.2 (pour le sens d'« échantillon simulé »). La section 6.4 suppose 6.3. Les sections ⊕ 6.5 et 6.6 demandent seulement le chapitre 2 du volume I (et, pour la simulation par inversion, la section 6.2.4) ; elles peuvent être lues à part.

**Les données utilisées.** Principalement `donnees/clients.csv` (2 000 clients de Dar Jasmin, simulés ; voir l'introduction du volume). Nous nous intéressons surtout à `rachat_12m` (le client a-t-il recommandé dans les 12 mois ?), à `offre_bienvenue` (une offre attribuée **au hasard** à la moitié des clients), à `canal_acquisition` et à `nb_commandes_an`. Comme les données sont **simulées**, nous connaissons à la fin la vraie valeur de plusieurs paramètres : ce sera l'occasion de vérifier que la méthode les retrouve.

**Honnêteté sur les outils.** Les bibliothèques bayésiennes professionnelles (PyMC, Stan) **ne sont pas installées** dans l'environnement qui a servi à produire ce livre. Leur code est montré dans la section 6.3.7, marqué « non exécuté ». Tous les algorithmes dont nous donnons les résultats sont **écrits à la main en NumPy** : c'est aussi la meilleure façon de comprendre ce que ces bibliothèques font à votre place.

**Matériel.** Tout le code est en Python (NumPy, SciPy, pandas, matplotlib, statsmodels). Les graines sont fixées : vous retrouverez les mêmes nombres que dans le livre, à de très légères variations près selon les versions des bibliothèques.

## 6.1 Inférence bayésienne et lois a priori

**Intuition.** Un détective arrive sur une scène de crime avec des **soupçons** (« le jardinier a déjà été condamné »). Chaque indice (une empreinte, un alibi) **déplace** ses soupçons : certains suspects deviennent plus probables, d'autres moins. À la fin, il a des soupçons *mis à jour*. L'inférence bayésienne est exactement cela, avec des nombres : on part d'une loi *a priori* (les soupçons), on observe des données (les indices), on obtient une loi *a posteriori* (les soupçons mis à jour). Et la **règle de mise à jour** est la formule de Bayes que vous connaissez depuis le volume I (section 2.1).

### 6.1.1 La formule de Bayes, version « paramètre »

Au volume I, la formule de Bayes disait :  $P(A | B) = \frac{P(B | A) P(A)}{P(B)}$ . Remplaçons  $A$  par « le paramètre vaut  $\theta$  » et  $B$  par « nous avons observé les données  $y$  » :

$$\underbrace{p(\theta | y)}_{\text{a posteriori}} = \frac{\overbrace{p(y | \theta)}^{\text{vraisemblance}} \overbrace{p(\theta)}^{\text{a priori}}}{\underbrace{p(y)}_{\text{constante de normalisation}}} \propto p(y | \theta) p(\theta).$$

Trois ingrédients, trois noms à retenir :

- la **vraisemblance**  $p(y | \theta)$  : la même fonction que celle du maximum de vraisemblance du volume I (section 3.2). Elle dit à quel point les données observées sont plausibles pour chaque valeur de  $\theta$  ;
- la loi **a priori**  $p(\theta)$  : ce que l'on croit de  $\theta$  **avant** de voir les données ;
- la loi **a posteriori**  $p(\theta | y)$  : ce que l'on croit **après**. C'est le résultat d'une analyse bayésienne : toute l'information sur  $\theta$  tient dans cette loi.

Le symbole  $\propto$  se lit « proportionnel à » : le dénominateur  $p(y) = \int p(y | \theta) p(\theta) d\theta$  ne dépend pas de  $\theta$ , il sert seulement à ce que l'aire sous la courbe vaille 1. **Tout le travail bayésien tient dans cette phrase : a posteriori = vraisemblance × a priori, puis on normalise.**

**Le point qui divise.** Pour un fréquentiste,  $\theta$  est une constante inconnue : dire «  $P(\theta > 0,5) = 0,98$  » n'a pas de sens, car  $\theta$  est soit plus grand que 0,5, soit non. Pour un bayésien, la probabilité mesure **notre degré de connaissance**, donc elle s'applique aussi à  $\theta$ . Ce n'est pas une querelle de mots : cela change les phrases qu'on a le droit de prononcer (6.1.4). Dans ce livre, nous adoptons une position pragmatique : les deux approches sont des outils, et les chiffres qu'elles donnent coïncident souvent quand les données sont abondantes.

### 6.1.2 Un exemple minuscule, calculé à la main

Yasmine envoie une offre de bienvenue à **10 nouveaux clients**. Sept d'entre eux repassent commande dans l'année. Appelons  $\theta$  la probabilité qu'un client avec offre rachète. Que sait-on de  $\theta$  ?

**Vraisemblance.** Si les clients sont indépendants, le nombre  $y$  de rachats suit une loi binomiale  $\text{Bin}(10, \theta)$  (volume I, section 2.2) :

$$p(y = 7 | \theta) = \binom{10}{7} \theta^7 (1 - \theta)^3.$$

**A priori.** Yasmine n'a aucune idée précise : elle juge que toutes les valeurs de  $\theta$  entre 0 et 1 sont également plausibles ( $p(\theta) = 1$  sur  $[0, 1]$ ).

Pour calculer à la main sans intégrale, regardons seulement **onze valeurs** possibles :  $\theta \in \{0; 0,1; 0,2; \dots; 1\}$ , chacune avec la probabilité *a priori* 1/11. C'est l'**approximation sur grille**. Il suffit de multiplier, puis de diviser par la somme.

Par exemple pour  $\theta = 0,7$  :  $\theta^7 (1 - \theta)^3 = 0,7^7 \times 0,3^3 = 0,0823543 \times 0,027 = 0,0022236$ . (On peut ignorer le coefficient binomial  $\binom{10}{7}$ , qui est le même pour toutes les valeurs de  $\theta$  et disparaît à la normalisation.) Le code fait les onze calculs :

```
import numpy as np
import pandas as pd

theta = np.round(np.arange(0, 1.01, 0.1), 1)          # les 11 valeurs possibles
prior = np.full(11, 1 / 11)                          # a priori : toutes aussi plausibles
vraisemblance = theta**7 * (1 - theta)**3             # sans le coefficient binomial
produit = prior * vraisemblance
posterieur = produit / produit.sum()                  # normalisation : la somme vaut 1

tab = pd.DataFrame({"theta": theta, "a priori": prior.round(3),
                    "vraisemblance": vraisemblance.round(5), "a posteriori": posterieur.round(3)})
print(tab.to_string(index=False))
print("\nsomme des probabilités a posteriori :", posterieur.sum().round(6))
print("valeur la plus probable :", theta[posterieur.argmax()])
print("espérance a posteriori :", round((theta * posterieur).sum(), 3))
```

| theta | a priori | vraisemblance | a posteriori |
|-------|----------|---------------|--------------|
| 0.0   | 0.091    | 0.00000       | 0.000        |
| 0.1   | 0.091    | 0.00000       | 0.000        |
| 0.2   | 0.091    | 0.00001       | 0.001        |
| 0.3   | 0.091    | 0.00008       | 0.010        |
| 0.4   | 0.091    | 0.00035       | 0.047        |
| 0.5   | 0.091    | 0.00098       | 0.129        |
| 0.6   | 0.091    | 0.00179       | 0.236        |
| 0.7   | 0.091    | 0.00222       | 0.293        |
| 0.8   | 0.091    | 0.00168       | 0.221        |
| 0.9   | 0.091    | 0.00048       | 0.063        |
| 1.0   | 0.091    | 0.00000       | 0.000        |

```

somme des probabilités a posteriori : 1.0
valeur la plus probable : 0.7
espérance a posteriori : 0.667

```

On lit le résultat : avant les données, les onze valeurs sont à égalité ; après, la probabilité se concentre autour de 0,7. Les valeurs extrêmes (0, 1) sont **exclues** : 0 est impossible puisque nous avons vu des rachats, 1 est impossible puisque nous avons vu des non-rachats. La valeur la plus plausible est celle du maximum de vraisemblance,  $\hat{\theta} = 7/10$ , mais l'**espérance a posteriori** est un peu plus basse (la loi est étalée plus du côté bas que du côté haut). Nous allons voir pourquoi.

### 6.1.3 La loi Bêta : le coup de génie de la conjugaison

Onze valeurs de  $\theta$ , c'est grossier. Avec un continuum de valeurs, il faut une intégrale. Heureusement, ici, elle se calcule **exactement**, grâce à un choix d'*a priori* bien assorti à la vraisemblance.

**Définition.** La loi **Bêta**( $a, b$ ) a pour densité sur  $[0, 1]$

$$p(\theta) = \frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a, b)}, \quad B(a, b) = \int_0^1 \theta^{a-1}(1-\theta)^{b-1} d\theta.$$

Sa moyenne est  $\frac{a}{a+b}$ . Quand  $a = b = 1$ , c'est la loi uniforme ; quand  $a$  et  $b$  grandissent, elle se resserre autour de sa moyenne.

**Théorème (conjugaison bêta-binomiale).** Si  $\theta \sim \text{Beta}(a, b)$  et si, sachant  $\theta$ ,  $y \sim \text{Bin}(n, \theta)$ , alors

$$\theta | y \sim \text{Beta}(a + y, b + n - y).$$

*Démonstration.* D'après la formule de Bayes,  $p(\theta | y) \propto p(y | \theta) p(\theta)$ , donc

$$p(\theta | y) \propto \theta^y (1-\theta)^{n-y} \cdot \theta^{a-1} (1-\theta)^{b-1} = \theta^{(a+y)-1} (1-\theta)^{(b+n-y)-1}.$$

On reconnaît, **à une constante près**, la densité d'une loi  $\text{Beta}(a+y, b+n-y)$ . Comme une densité est entièrement déterminée par sa forme et par la condition « l'aire vaut 1 », la constante manquante ne peut être que  $1/B(a+y, b+n-y)$ .  $\square$

Les statisticiens disent que la loi Bêta est **conjuguée** à la loi binomiale : l'*a posteriori* reste dans la même famille que l'*a priori*, seuls les paramètres changent. La mise à jour se résume à une **addition** :

$$a_{\text{post}} = a + \underbrace{y}_{\text{succès}}, \quad b_{\text{post}} = b + \underbrace{n-y}_{\text{échecs}}.$$

On peut donc voir  $a$  et  $b$  comme des **pseudo-observations** :  $\text{Beta}(a, b)$  est l'*a priori* qu'aurait quelqu'un qui aurait déjà vu  $a-1$  succès et  $b-1$  échecs. Notre exemple :  $\text{Beta}(1, 1)$  puis  $y = 7$ ,  $n = 10$ , donc  $\theta | y \sim \text{Beta}(8, 4)$ , de moyenne  $8/12 \approx 0,667$ .

**L'espérance a posteriori est une moyenne pondérée.** La moyenne a posteriori vaut

$$\mathbb{E}[\theta | y] = \frac{a+y}{a+b+n} = \underbrace{\frac{a}{a+b+n}}_{\text{moyenne a priori}} + \underbrace{\frac{n}{a+b+n}}_{1-w} \cdot \underbrace{\frac{y}{n}}_{\hat{\theta}_{\text{EMV}}}.$$

C'est un **compromis** entre ce que l'on croyait (moyenne a priori) et ce que disent les données (la fréquence observée), et le poids de l'*a priori*,  $w = \frac{a+b}{a+b+n}$ , **diminue** quand  $n$  augmente.



Avec 10 observations et un a priori uniforme ( $a + b = 2$ ),  $w = 2/12 \approx 0,17$  : 17 % de l'a priori, 83 % des données. C'est la raison pour laquelle  $0,667 < 0,7$  : l'a priori uniforme tire un peu l'estimation vers 0,5. Avec 1 000 observations,  $w \approx 0,002$  : l'a priori a disparu.

Vérifions que la formule exacte et la grille donnent la même chose, et traçons les trois lois (a priori, vraisemblance normalisée, a posteriori) :

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
                    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
                    "legend.frameon": False, "font.size": 10})

# 1) la grille et la loi exacte donnent-elles la même chose ?
exact = stats.beta(8, 4)
grille_fine = exact.pdf(theta) / exact.pdf(theta).sum()
print("probabilités de grille :", posterior.round(3))
print("Beta(8,4) renormalisée :", grille_fine.round(3))
print("écart maximal :", np.abs(posterieur - grille_fine).max().round(4))
print("moyenne exacte :", round(exact.mean(), 4), "= 8/12 =", round(8 / 12, 4))

# 2) le dessin : a priori plat, vraisemblance, a posteriori ; et un a posteriori avec un a priori
    ↪ informatif
x = np.linspace(0, 1, 500)
fig, ax = plt.subplots(figsize=(7.2, 3.8))
vrais = x**7 * (1 - x)**3
vrais = vrais / np.trapezoid(vrais, x) # vraisemblance normalisée (aire = 1)
ax.plot(x, vrais, color=ORANGE, lw=6, alpha=0.45, solid_capstyle="butt")
ax.plot(x, exact.pdf(x), color=BLEU, lw=2.0)
ax.plot(x, stats.beta(1, 1).pdf(x), color="#898781", lw=1.6, ls="--")
ax.plot(x, stats.beta(12, 8).pdf(x), color=VIOLET, lw=1.8, ls="-.")
ax.text(0.02, 1.12, "a priori plat Beta(1, 1)", color="#52514e")
ax.text(0.995, 4.2, "a posteriori Beta(8, 4)\n= vraisemblance\nnormalisée\n(superposées)",
    ↪ color=BLEU, ha="right", va="top")
ax.annotate("avec un a priori Beta(5, 5) : \na posteriori Beta(12, 8)", xy=(0.50, stats.beta(12,
    ↪ 8).pdf(0.50)), xytext=(0.02, 3.0),
    color=VIOLET, arrowprops={"arrowstyle": "-", "color": VIOLET, "lw": 0.8})
ax.set_xlabel(r"probabilité de rachat  $\theta$ ")
ax.set_ylabel("densité")
ax.set_ylim(0, 4.3)
plt.savefig("figures/ch06-bayes-10-clients.png", dpi=200, bbox_inches="tight")
plt.close()

probabilités de grille : [0.    0.    0.001 0.01  0.047 0.129 0.236 0.293 0.221 0.063 0.    ]
Beta(8,4) renormalisée : [0.    0.    0.001 0.01  0.047 0.129 0.236 0.293 0.221 0.063 0.    ]
écart maximal          : 0.0
moyenne exacte         : 0.6667 = 8/12 = 0.6667
```

L'écart maximal est de l'ordre de la précision d'arrondi : la grille de onze points n'était pas si mauvaise. Remarquez, sur le dessin, que l'a posteriori et la vraisemblance normalisée sont **exactement superposés** : avec un a priori plat, **les données parlent seules** (c'est évident dans la formule : multiplier par une constante ne change pas la forme). Avec un a priori informatif Beta(5,5) (celui d'une personne qui croit déjà à un taux proche de 50 %), l'a posteriori Beta(12,8) a pour moyenne  $12/20 = 0,60$  : il est **tiré vers 0,5**, c'est le compromis de la formule ci-dessus.

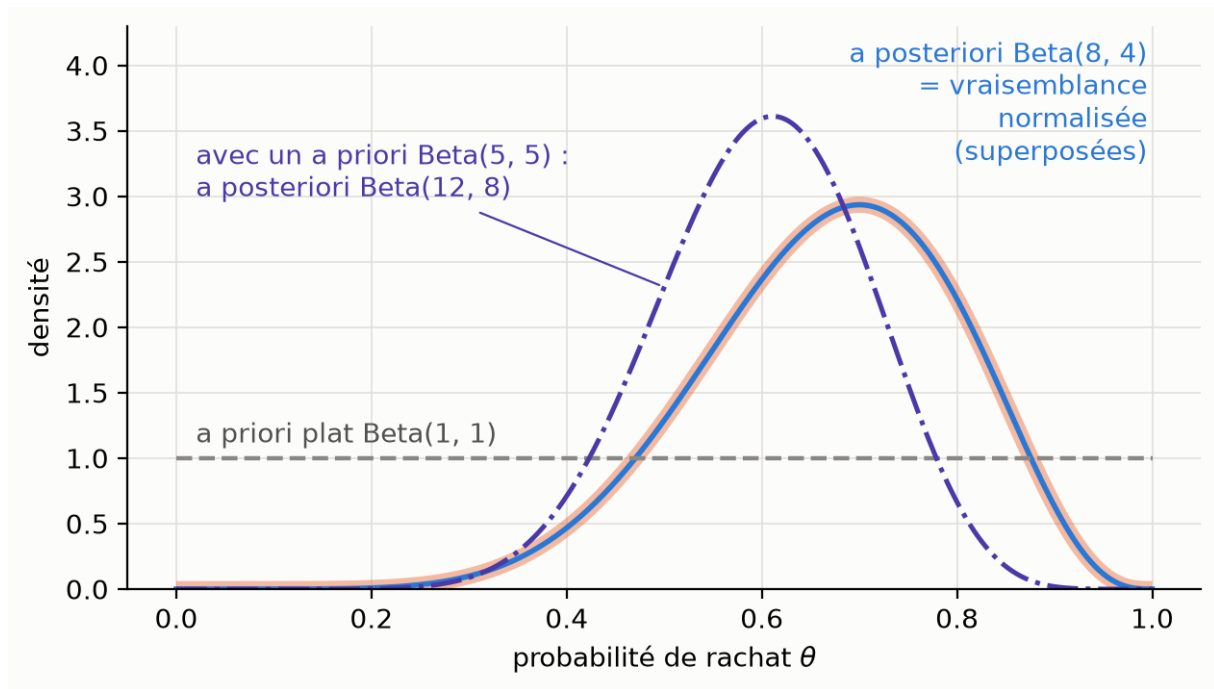


Figure 61: Avec 7 rachats sur 10 : l'a priori plat (tirets gris), la vraisemblance normalisée (orange épais) et l'a posteriori Beta(8, 4) (bleu) sont superposés ; avec un a priori informatif Beta(5, 5), l'a posteriori Beta(12, 8) (violet, tirets-points) est tiré vers 0,5.

**À retenir (6.1.1 à 6.1.3).**  $A \text{ posteriori} \propto \text{vraisemblance} \times \text{a priori}$ . Avec un a priori bêta et des données binomiales, l'a posteriori est une bêta dont les paramètres s'obtiennent en **ajoutant les succès et les échecs observés**. La moyenne a posteriori est un compromis entre a priori et données, où l'a priori pèse de moins en moins quand les données s'accumulent.

#### 6.1.4 Application : l'offre de bienvenue fonctionne-t-elle ?

Passons aux 2 000 clients. L'offre de bienvenue a été attribuée **au hasard** à une moitié d'entre eux : la comparaison des deux groupes est donc une vraie expérience (nous y reviendrons dans le chapitre 7). Les effectifs :

```
clients = pd.read_csv("donnees/clients.csv")
bilan = clients.groupby("offre_bienvenue")["rachat_12m"].agg(clients="size", rachats="sum")
bilan["frequence"] = (bilan["rachats"] / bilan["clients"]).round(4)
print(bilan)
```

| offre_bienvenue | clients | rachats | frequence |
|-----------------|---------|---------|-----------|
| 0               | 985     | 441     | 0.4477    |
| 1               | 1015    | 578     | 0.5695    |

Avec un a priori Beta(1, 1) pour chaque groupe, les deux lois a posteriori sont Beta(1+y, 1+n-y) :

```
post = {}
for groupe, ligne in bilan.iterrows():
    a_post = 1 + ligne["rachats"]
    b_post = 1 + ligne["clients"] - ligne["rachats"]
    post[groupe] = stats.beta(a_post, b_post)
    bas, haut = post[groupe].ppf([0.025, 0.975])
    print(f"offre = {groupe} : a posteriori Beta({a_post:.0f}, {b_post:.0f}) | moyenne
    ↳ {post[groupe].mean():.4f} | ")
```

```
f"intervalle de crédibilité à 95 % : [{bas:.4f} ; {haut:.4f}]"
```

```
offre = 0 : a posteriori Beta(442, 545) | moyenne 0.4478 | intervalle de crédibilité à 95 % : [0.4169
↪ ; 0.4789]
offre = 1 : a posteriori Beta(579, 438) | moyenne 0.5693 | intervalle de crédibilité à 95 % : [0.5388
↪ ; 0.5996]
```

Pour répondre à la question (« l'offre augmente-t-elle le taux de rachat ? »), il faut la loi de la **différence**  $\delta = \theta_1 - \theta_0$ . Elle n'est pas une bêta, mais on peut la **simuler** : on tire de nombreuses valeurs de  $\theta_1$  et de  $\theta_0$  dans leurs lois a posteriori, et on calcule leur différence. C'est la première apparition du Monte-Carlo, que nous étudions à fond en 6.2.

```
rng = np.random.default_rng(61)
S = 200_000
thet1 = post[1].rvs(S, random_state=rng)
thet0 = post[0].rvs(S, random_state=rng)
delta = thet1 - thet0

print(f"moyenne de la différence      : {delta.mean():.4f}")
print(f"IC de crédibilité à 95 %      : [{np.percentile(delta, 2.5):.4f} ; {np.percentile(delta,
↪ 97.5):.4f}]")
print(f"P(l'offre augmente le rachat)  : {(delta > 0).mean():.5f}")
print(f"P(l'effet dépasse 5 points)    : {(delta > 0.05).mean():.4f}")
print(f"P(l'effet dépasse 10 points)   : {(delta > 0.10).mean():.4f}")
```

```
moyenne de la différence      : 0.1214
IC de crédibilité à 95 %      : [0.0778 ; 0.1647]
P(l'offre augmente le rachat) : 1.00000
P(l'effet dépasse 5 points)    : 0.9993
P(l'effet dépasse 10 points)   : 0.8346
```

On peut maintenant prononcer des phrases que le volume I interdisait : « *sachant les données, l'offre augmente le rachat avec une probabilité quasi certaine* » (aucun de nos 200 000 tirages ne donne une différence négative ; l'approximation normale de la différence donne une probabilité d'erreur de l'ordre de  $2 \times 10^{-8}$ ), « *la probabilité que l'effet dépasse 5 points de pourcentage est de 99,9 %, et qu'il dépasse 10 points de 83 %* ». Pour comparer avec l'approche fréquentiste, calculons l'intervalle de confiance de Wald de la différence de proportions (volume I, section 3.4.5) :

```
n1, y1 = bilan.loc[1, "clients"], bilan.loc[1, "rachats"]
n0, y0 = bilan.loc[0, "clients"], bilan.loc[0, "rachats"]
p1, p0 = y1 / n1, y0 / n0
se = np.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
print(f"différence observée          : {p1 - p0:.4f}")
print(f"IC de confiance à 95 %       : [{p1 - p0 - 1.96 * se:.4f} ; {p1 - p0 + 1.96 * se:.4f}]")
```

```
différence observée          : 0.1217
IC de confiance à 95 %       : [0.0782 ; 0.1652]
```

Les deux intervalles sont **presque identiques** (à la troisième décimale). C'est normal : avec 1 000 observations par groupe, l'a priori uniforme ne pèse presque rien. Mais l'**interprétation** diffère profondément :

|                      | Intervalle de confiance<br>(fréquentiste)                                | Intervalle de crédibilité<br>(bayésien)                                          |
|----------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Ce qui est aléatoire | l'intervalle (il change d'un échantillon à l'autre)                      | le paramètre $\theta$ (notre connaissance)                                       |
| Phrase correcte      | « la <i>méthode</i> encadre la vraie valeur dans 95 % des échantillons » | « sachant les données, $\delta$ est dans l'intervalle avec la probabilité 0,95 » |

|                                            | Intervalle de confiance<br>(fréquentiste) | Intervalle de crédibilité<br>(bayésien)       |
|--------------------------------------------|-------------------------------------------|-----------------------------------------------|
| Peut-on dire « $P(\delta > 0) = \dots$ » ? | non                                       | <b>oui</b>                                    |
| Dépend d'un a priori ?                     | non                                       | oui (mais l'effet s'efface quand $n$ grandit) |

**Ce que Yasmine veut entendre.** « L'offre est presque certainement utile, et le gain est de l'ordre de 12 points de pourcentage de rachat (entre 8 et 16 avec 95 % de probabilité). » C'est une phrase bayésienne. Elle est parfaitement naturelle, et la plupart des gens interprètent d'ailleurs *intuitivement* les intervalles de confiance de cette façon (à tort, nous l'avons vu au volume I, section 3.3.2).

### 6.1.5 Intervalle de crédibilité : équi-queue ou HPD ?

Il existe plusieurs façons de fabriquer un intervalle contenant 95 % de la probabilité a posteriori. Les deux principales :

- l'intervalle à **queues égales** : il laisse 2,5 % de probabilité de chaque côté (c'est ce que fait `ppf([0.025, 0.975])`) ;
- l'intervalle de **plus haute densité** (HPD, *highest posterior density*) : le **plus court** intervalle de probabilité 95 %. Tous ses points ont une densité supérieure à ceux qui sont en dehors.

Pour une loi symétrique, ils coïncident. Pour une loi asymétrique (proche de 0 ou de 1, par exemple), ils diffèrent :

```
def hpd(loi, masse=0.95, pas=2000):
    """Plus court intervalle de probabilité `masse` : on fait glisser une fenêtre de quantiles."""
    bas = np.linspace(0, 1 - masse, pas)
    largeurs = loi.ppf(bas + masse) - loi.ppf(bas)
    i = largeurs.argmin()
    return loi.ppf(bas[i]), loi.ppf(bas[i] + masse)

# Un cas asymétrique : un nouveau canal testé auprès de 20 clients, 1 seul rachat
asym = stats.beta(1 + 1, 1 + 19)
print("Beta(2, 20) moyenne :", round(asym.mean(), 4))
print("à queues égales : [%4f ; %4f] largeur %4f" % (*asym.ppf([0.025, 0.975]),
    ↪ np.diff(asym.ppf([0.025, 0.975]))[0]))
h = hpd(asym)
print("HPD          : [%4f ; %4f] largeur %4f" % (*h, h[1] - h[0]))
```

```
Beta(2, 20) moyenne : 0.0909
à queues égales : [0.0117 ; 0.2382] largeur 0.2264
HPD          : [0.0026 ; 0.2080] largeur 0.2054
```

Le HPD est plus court et plus proche de zéro, ce qui correspond mieux à l'idée intuitive d'un intervalle qui « suit » la densité. Pour les lois à peu près symétriques de ce chapitre, la différence est mineure ; l'intervalle à queues égales est le plus répandu.

### 6.1.6 Le choix de l'a priori : liberté, responsabilité, et sensibilité

C'est la grande critique adressée à la méthode : « *l'a priori est subjectif* ». Voyons concrètement ce qu'il change.

**Quatre a priori pour une probabilité de rachat  $\theta$ .**

| A priori               | Paramètres                         | Ce qu'il dit                                                          | Pseudo-observations |
|------------------------|------------------------------------|-----------------------------------------------------------------------|---------------------|
| Uniforme               | Beta(1, 1)                         | « je n'ai aucune idée »                                               | 2                   |
| Jeffreys               | Beta( $\frac{1}{2}, \frac{1}{2}$ ) | invariant par reparamétrisation (plus de poids aux bords)             | 1                   |
| Informatif réaliste    | Beta(20, 20)                       | « d'après mon expérience, autour de 50 %, à quelques points près »    | 40                  |
| Informatif <b>faux</b> | Beta(2, 18)                        | « je suis sûre que très peu de clients rachètent » ( $\approx 10\%$ ) | 20                  |

Comparons les lois a posteriori pour des données de plus en plus abondantes : les **10** premiers clients avec offre, puis les **100** premiers, puis les **1 015**.

```
avec_offre = clients.loc[clients["offre_bienvenue"] == 1, "rachat_12m"].to_numpy()
priors = {"Uniforme Beta(1,1)": (1, 1), "Jeffreys Beta(.5,.5)": (0.5, 0.5),
         "Informatif Beta(20,20)": (20, 20), "Informatif faux Beta(2,18)": (2, 18)}

lignes = []
for n in (10, 100, len(avec_offre)):
    y = avec_offre[:n].sum()
    for nom, (a, b) in priors.items():
        loi = stats.beta(a + y, b + n - y)
        bas, haut = loi.ppf([0.025, 0.975])
        lignes.append({"n": n, "rachats": int(y), "a priori": nom,
                       "moyenne a posteriori": round(loi.mean(), 3),
                       "IC95 bas": round(bas, 3), "IC95 haut": round(haut, 3)})
sens = pd.DataFrame(lignes)
print(sens.to_string(index=False))
print("\nfréquence observée sur les 10 premiers :", avec_offre[:10].mean(), "| sur les 100 :",
      avec_offre[:100].mean(),
      "| sur les", len(avec_offre), ":", round(avec_offre.mean(), 3))
```

| n    | rachats | a priori                   | moyenne a posteriori | IC95 bas | IC95 haut |
|------|---------|----------------------------|----------------------|----------|-----------|
| 10   | 3       | Uniforme Beta(1,1)         | 0.333                | 0.109    | 0.610     |
| 10   | 3       | Jeffreys Beta(.5,.5)       | 0.318                | 0.093    | 0.606     |
| 10   | 3       | Informatif Beta(20,20)     | 0.460                | 0.325    | 0.598     |
| 10   | 3       | Informatif faux Beta(2,18) | 0.167                | 0.058    | 0.317     |
| 100  | 55      | Uniforme Beta(1,1)         | 0.549                | 0.452    | 0.644     |
| 100  | 55      | Jeffreys Beta(.5,.5)       | 0.550                | 0.452    | 0.645     |
| 100  | 55      | Informatif Beta(20,20)     | 0.536                | 0.453    | 0.617     |
| 100  | 55      | Informatif faux Beta(2,18) | 0.475                | 0.387    | 0.564     |
| 1015 | 578     | Uniforme Beta(1,1)         | 0.569                | 0.539    | 0.600     |
| 1015 | 578     | Jeffreys Beta(.5,.5)       | 0.569                | 0.539    | 0.600     |
| 1015 | 578     | Informatif Beta(20,20)     | 0.567                | 0.537    | 0.597     |
| 1015 | 578     | Informatif faux Beta(2,18) | 0.560                | 0.530    | 0.590     |

fréquence observée sur les 10 premiers : 0.3 | sur les 100 : 0.55 | sur les 1015 : 0.569

Lisons ce tableau par bloc :

- $n = 10$  : les a priori comptent *beaucoup*. Les quatre résultats diffèrent nettement, et l'a priori « faux » (qui croit à 10 %) tire fortement la moyenne vers le bas.
- $n = 100$  : les écarts se réduisent, mais l'a priori faux pèse encore.
- $n = 1\,015$  : les quatre moyennes a posteriori sont **quasiment égales** (à moins d'un point de pourcentage près : 0,560 à 0,569), même celle de l'a priori faux. L'a priori a été « oublié ».

**C'est un théorème, pas seulement une observation.** Quand  $n \rightarrow \infty$ , tant que l'a priori n'attribue pas une probabilité nulle à la vraie valeur de  $\theta$ , la loi a posteriori se concentre autour de la vraie valeur, et sa forme devient celle d'une loi normale (théorème de Bernstein-von Mises). Autrement dit : **avec assez de données, des personnes aux a priori différents finissent par être d'accord.** C'est l'argument le plus fort en faveur de la méthode. L'argument le plus fort contre : avec *peu* de données, la conclusion dépend de l'a priori, et il faut alors **le dire**.

### Trois règles d'hygiène.

1. **Toujours dire quel a priori a été utilisé**, et le justifier en une phrase.
2. **Faire une analyse de sensibilité** : refaire le calcul avec deux ou trois a priori raisonnables. Si la conclusion ne change pas, tant mieux ; sinon, c'est une information importante pour le lecteur (« nos données sont insuffisantes pour trancher »).
3. **Ne jamais mettre une probabilité nulle** sur une valeur possible : aucune donnée ne pourrait la réhabiliter (c'est la « règle de Cromwell » : « *je vous en conjure, songez que vous puissiez vous tromper* »).

**Les a priori « faiblement informatifs ».** En pratique, ni l'uniforme total ni l'a priori très précis ne sont idéaux. On préfère souvent des lois larges mais qui **excluent l'absurde** : par exemple, pour un coefficient de régression logistique, une loi normale centrée en 0 avec un écart-type de 1,5 ou 2,5 écarte les rapports de cotes de  $e^{10}$  tout en laissant passer toutes les valeurs plausibles. Nous les utiliserons en 6.3, et nous verrons en 6.4.3 comment *vérifier* qu'un a priori est raisonnable.

### 6.1.7 Intervalle de crédibilité contre intervalle de confiance : une expérience

Un intervalle de crédibilité à 95 % est-il « bon » au sens fréquentiste ? Faisons l'expérience : fixons une vraie valeur  $\theta$ , simulons 20 000 échantillons de taille  $n = 30$ , et comptons dans combien de cas chaque intervalle contient  $\theta$  (la **couverture**). Nous comparons l'intervalle de Wald (volume I, section 3.3.4), l'intervalle de Wilson et l'intervalle de crédibilité Beta(1, 1) à queues égales.

```
from statsmodels.stats.proportion import proportion_confint

def couverture(theta_vrai, n=30, reps=20_000, graine=62):
    rng = np.random.default_rng(graine)
    y = rng.binomial(n, theta_vrai, reps)
    res = {}
    for methode in ("normal", "wilson"):
        bas, haut = proportion_confint(y, n, alpha=0.05, method=methode)
        res[methode] = np.mean((bas <= theta_vrai) & (theta_vrai <= haut))
    loi = stats.beta(1 + y, 1 + n - y)
    bas, haut = loi.ppf(0.025), loi.ppf(0.975)
    res["crédibilité Beta(1,1)"] = np.mean((bas <= theta_vrai) & (theta_vrai <= haut))
    return res

lignes = []
for th in (0.05, 0.10, 0.30, 0.50):
    r = couverture(th)
    lignes.append({"vrai θ": th, "Wald": round(r["normal"], 3), "Wilson": round(r["wilson"], 3),
                "crédibilité": round(r["crédibilité Beta(1,1)"], 3)})
print(pd.DataFrame(lignes).to_string(index=False))
```

| vrai θ | Wald  | Wilson | crédibilité |
|--------|-------|--------|-------------|
| 0.05   | 0.781 | 0.940  | 0.940       |
| 0.10   | 0.806 | 0.974  | 0.974       |
| 0.30   | 0.954 | 0.930  | 0.930       |
| 0.50   | 0.958 | 0.958  | 0.958       |

Pour  $\theta$  proche de 0 (5 % ou 10 %), l'intervalle de Wald est très en dessous de 95 % : il ne couvre la vraie valeur que dans 78 % à 81 % des cas (c'est le défaut connu qu'on avait signalé au volume I). L'intervalle de crédibilité à a priori uniforme, lui, reste dans une fourchette de 93 % à 97 %, ce qui n'a rien d'évident : *il a été construit sans aucun souci de couverture*. Remarquez aussi que sa couverture est **identique** à celle de l'intervalle de Wilson, dans les quatre lignes : ce n'est pas un hasard, les deux intervalles ne diffèrent que de 0,002 au plus pour  $n = 30$  et décident de la même façon pour chaque valeur possible du nombre de succès. L'approche bayésienne et le meilleur intervalle fréquentiste se rejoignent. C'est un résultat général : sous des hypothèses douces, les intervalles de crédibilité des modèles réguliers ont une bonne couverture fréquentiste quand  $n$  est assez grand.

### 6.1.8 Autres couples conjugués : Poisson-Gamma et Normal-Normal

L'idée de conjugaison dépasse le cas bêta-binomial. Voici deux autres couples très utiles ; leurs démonstrations ont **exactement la même structure** (multiplier, reconnaître une forme, normaliser).

**Poisson-Gamma (comptages).** On observe des comptages  $y_1, \dots, y_n$  indépendants de loi Poisson( $\lambda$ ), avec  $\lambda \sim \text{Gamma}(a, \text{taux } b)$  de densité  $\propto \lambda^{a-1} e^{-b\lambda}$ .

**Théorème.**  $\lambda | y \sim \text{Gamma}(a + \sum_i y_i, b + n)$ .

*Démonstration.*  $p(y | \lambda) \propto \prod_i \lambda^{y_i} e^{-\lambda} = \lambda^{\sum y_i} e^{-n\lambda}$ . En multipliant par l'a priori :  $p(\lambda | y) \propto \lambda^{a+\sum y_i-1} e^{-(b+n)\lambda}$ , qui est la densité d'une  $\text{Gamma}(a + \sum y_i, b + n)$ .  $\square$

*Exemple à la main.* Cinq semaines, 3, 5, 4, 6, 2 commandes sur le site. A priori  $\text{Gamma}(a = 2, b = 0,5)$  (moyenne  $a/b = 4$  commandes par semaine, très vague).  $\sum y_i = 20, n = 5$  : a posteriori  $\text{Gamma}(22, 5,5)$ , de moyenne  $22/5,5 = 4$ . (Coïncidence : l'a priori et les données étaient d'accord sur 4.)

Appliquons cela aux **clients de la boutique** : nombre de commandes par an `nb_commandes_an`, modélisé par Poisson( $\lambda$ ) avec le même  $\lambda$  pour tous.

```
boutique = clients.loc[clients["canal_acquisition"] == "Boutique", "nb_commandes_an"].to_numpy()
n_b, s_b = len(boutique), boutique.sum()
a0, b0 = 2.0, 0.5 # a priori Gamma(2, taux 0,5) : moyenne 4, très vague
post_lam = stats.gamma(a=a0 + s_b, scale=1 / (b0 + n_b))
print(f"clients de la boutique : n = {n_b}, total de commandes = {s_b}, moyenne = {boutique.mean():.3f}")
print(f"a posteriori Gamma({a0 + s_b:.0f}, taux {b0 + n_b:.1f}) : moyenne {post_lam.mean():.3f}, "
      f"IC95 [{post_lam.ppf(0.025):.3f} ; {post_lam.ppf(0.975):.3f}]")
print(f"variance observée / moyenne observée = {boutique.var(ddof=1) / boutique.mean():.2f} "
      f"↪ (vaudrait 1 pour une vraie loi de Poisson)")
```

```
clients de la boutique : n = 504, total de commandes = 2085, moyenne = 4.137
a posteriori Gamma(2087, taux 504.5) : moyenne 4.137, IC95 [3.961 ; 4.316]
variance observée / moyenne observée = 3.32 (vaudrait 1 pour une vraie loi de Poisson)
```

L'intervalle est étroit : on est « sûr » de  $\lambda$  à 0,18 près environ. Mais regardez la dernière ligne : pour une vraie loi de Poisson, variance et moyenne sont égales (volume I, section 2.2) ; ici la variance est **plusieurs fois supérieure** à la moyenne. Les clients sont *hétérogènes* (certains commandent beaucoup, d'autres peu), et le modèle de Poisson, qui suppose le même  $\lambda$  pour tous, est **faux**. L'intervalle est donc trop étroit, **et le calcul bayésien ne nous l'a pas dit**. Il ne le dit jamais tout seul : un a posteriori est toujours *conditionnel à la validité du modèle*. Nous verrons en 6.4.2 comment vérifier un modèle et en réparer un (c'est aussi le sujet de la section 2.6 pour les modèles surdispersés).



**Le piège le plus fréquent de la modélisation bayésienne.** Un a posteriori très étroit ne prouve **pas** qu'on est sûr de soi : il prouve qu'on est sûr *si le modèle est juste*. Un modèle faux produit des intervalles étroits et faux, avec la même élégance.

**Normal-Normal (moyenne d'une loi normale, variance connue).** Si  $y_i \sim \mathcal{N}(\mu, \sigma^2)$  avec  $\sigma$  connu et  $\mu \sim \mathcal{N}(\mu_0, \tau_0^2)$ , alors  $\mu | y \sim \mathcal{N}(\mu_n, \tau_n^2)$  avec

$$\frac{1}{\tau_n^2} = \frac{1}{\tau_0^2} + \frac{n}{\sigma^2}, \quad \mu_n = \tau_n^2 \left( \frac{\mu_0}{\tau_0^2} + \frac{n\bar{y}}{\sigma^2} \right).$$

On additionne des **précisions** (l'inverse des variances) et on fait une moyenne **pondérée par les précisions**.

**Démonstration.** On écrit les logarithmes :  $\log p(\mu | y) = \text{cte} - \frac{(\mu - \mu_0)^2}{2\tau_0^2} - \frac{\sum (y_i - \mu)^2}{2\sigma^2}$ . Or  $\sum (y_i - \mu)^2 = \sum (y_i - \bar{y})^2 + n(\bar{y} - \mu)^2$  ; le premier terme ne dépend pas de  $\mu$ . Il reste un polynôme du second degré en  $\mu$  :

$$-\frac{1}{2} \left[ \mu^2 \left( \frac{1}{\tau_0^2} + \frac{n}{\sigma^2} \right) - 2\mu \left( \frac{\mu_0}{\tau_0^2} + \frac{n\bar{y}}{\sigma^2} \right) \right] + \text{cte}.$$

Un polynôme du second degré en  $\mu$  dans l'exponentielle est une densité normale ; en « complétant le carré », on lit sa précision  $\frac{1}{\tau_n^2}$  (le coefficient de  $\mu^2$ ) et sa moyenne  $\mu_n$  (le rapport entre le coefficient de  $\mu$  et celui de  $\mu^2$ ).  $\square$

*Exemple à la main.* On s'intéresse à  $\mu$  = logarithme du panier moyen d'un client de la boutique. Supposons  $\sigma = 0,35$  connu (hypothèse de confort, qu'on lèvera en 6.3), un a priori  $\mathcal{N}(4,0; 0,5^2)$ , et  $n = 5$  clients de moyenne  $\bar{y} = 4,4$ . Les précisions valent  $1/0,5^2 = 4$  pour l'a priori et  $5/0,35^2 = 40,82$  pour les données. Précision totale : 44,82, donc  $\tau_n = 1/\sqrt{44,82} = 0,149$ . Moyenne :  $\mu_n = \frac{4 \times 4,0 + 40,82 \times 4,4}{44,82} = \frac{16 + 179,6}{44,82} = 4,364$ . Les données (91 % du poids) l'emportent largement.

*# Vérification du calcul à la main, puis application aux 504 clients de la boutique*

```
def normal_normal(mu0, tau0, sigma, ybar, n):
    prec = 1 / tau0**2 + n / sigma**2
    return (mu0 / tau0**2 + n * ybar / sigma**2) / prec, 1 / np.sqrt(prec)

mu_n, tau_n = normal_normal(4.0, 0.5, 0.35, 4.4, 5)
print(f"exemple à la main : mu_n = {mu_n:.3f}, tau_n = {tau_n:.3f} (poids des données : {(5 / 0.35**2) / (1 / 0.5**2 + 5 / 0.35**2):.3f})")

acheteurs = clients[(clients["canal_acquisition"] == "Boutique") & (clients["panier_moyen"] > 0)]
ly = np.log(acheteurs["panier_moyen"])
mu_n, tau_n = normal_normal(4.0, 0.5, 0.35, ly.mean(), len(ly))
print(f"{len(ly)} acheteurs de la boutique : moyenne du log-panier = {ly.mean():.4f}")
print(f"a posteriori de mu : N({mu_n:.4f}, {tau_n:.4f}^2) | IC95 [{mu_n - 1.96 * tau_n:.4f} ; {mu_n + 1.96 * tau_n:.4f}]")
print(f"soit un panier « typique » (médiane) de {np.exp(mu_n):.1f} DT, IC95 [{np.exp(mu_n - 1.96 * tau_n):.1f} ; {np.exp(mu_n + 1.96 * tau_n):.1f}] DT")
```

```
exemple à la main : mu_n = 4.364, tau_n = 0.149 (poids des données : 0.911)
449 acheteurs de la boutique : moyenne du log-panier = 4.2183
a posteriori de mu : N(4.2181, 0.0165^2) | IC95 [4.1857 ; 4.2504]
soit un panier « typique » (médiane) de 67.9 DT, IC95 [65.7 ; 70.1] DT
```

Le panier « typique » (médiane, puisque  $\mu$  est le logarithme) d'un acheteur de la boutique est d'environ 68 DT, avec une incertitude d'environ  $\pm 2$  DT. Les données étant simulées, nous connaissons la vérité : le générateur a programmé un log-panier moyen de  $4,00 + 0,22 = 4,22$  pour la

boutique, soit  $e^{4,22} \approx 68,0$  DT. L'intervalle de crédibilité contient bien cette valeur.

### 6.1.9 La loi prédictive : prédire du nouveau

Un estimateur de  $\theta$  n'est pas un but en soi. Yasmine veut des **prédictions** : « sur les 100 prochains clients avec offre, combien rachèteront ? » Deux façons de répondre :

- **Plug-in** : on remplace  $\theta$  par son estimation  $\hat{\theta} = 0,5695$  et on prend  $\text{Bin}(100, \hat{\theta})$ . Cela **ignore** l'incertitude sur  $\theta$ .
- **Prédictive a posteriori** : on **moyenne** la loi binomiale sur toutes les valeurs plausibles de  $\theta$ , pondérées par leur probabilité a posteriori :

$$p(\tilde{y} | y) = \int p(\tilde{y} | \theta) p(\theta | y) d\theta.$$

Pour un a posteriori bêta, cette intégrale est connue : c'est la loi **bêta-binomiale**.

```
post1 = post[1]
n_nouv = 100
plug = stats.binom(n_nouv, avec_offre.mean())
pred = stats.betabinom(n_nouv, post1.args[0], post1.args[1])

for nom, loi in (("plug-in Bin(100,  $\theta$ )", plug), ("prédictive bêta-binomiale", pred)):
    print(f"{nom:28s} moyenne {loi.mean():6.2f} | écart-type {loi.std():5.2f} | "
          f"intervalle à 95 % [{loi.ppf(0.025):.0f} ; {loi.ppf(0.975):.0f}]")

# Même chose par simulation : on tire  $\theta$  dans l'a posteriori, puis y dans Bin(100,  $\theta$ )
rng = np.random.default_rng(63)
theta_s = post1.rvs(100_000, random_state=rng)
y_s = rng.binomial(n_nouv, theta_s)
print(f"simulation : moyenne {y_s.mean():.2f}, écart-type {y_s.std():.2f}, "
      f"intervalle à 95 % [{np.percentile(y_s, 2.5):.0f} ; {np.percentile(y_s, 97.5):.0f}]")

plug-in Bin(100,  $\theta$ )          moyenne 56.95 | écart-type 4.95 | intervalle à 95 % [47 ; 67]
prédictive bêta-binomiale     moyenne 56.93 | écart-type 5.19 | intervalle à 95 % [47 ; 67]
simulation : moyenne 56.94, écart-type 5.20, intervalle à 95 % [47 ; 67]
```

La loi prédictive est **un peu plus large** que le plug-in (écart-type 5,19 contre 4,95) : elle ajoute à la variabilité du tirage binomial l'incertitude sur  $\theta$ . Les intervalles à 95 % sont ici identiques ([47 ; 67]) car, avec 1 015 clients dans l'échantillon, l'incertitude sur  $\theta$  est faible ; avec 10 clients, l'écart serait énorme. La simulation (tirer  $\theta$ , puis  $y$ ) retrouve la loi exacte : 56,94 contre 56,93 pour la moyenne. Retenez le principe, car il est général : **une prédiction honnête incorpore l'incertitude sur les paramètres**. C'est un avantage naturel de l'approche bayésienne : la loi prédictive s'obtient sans effort supplémentaire, simplement en simulant (*simuler  $\theta$ , puis simuler les données sachant  $\theta$* ), ce que nous ferons systématiquement pour vérifier les modèles en 6.4.

#### À retenir (6.1).

- A posteriori  $\propto$  vraisemblance  $\times$  a priori. La vraisemblance est celle du volume I.
- Les **lois conjuguées** (bêta-binomiale, gamma-Poisson, normale-normale) donnent l'a posteriori **exactement**, par simple addition de paramètres ou de précisions.
- Un **intervalle de crédibilité** autorise la phrase «  $\theta$  est dans l'intervalle avec la probabilité 95 % » ; il coïncide numériquement avec l'intervalle de confiance quand l'a priori est plat et les données abondantes.
- L'a priori compte beaucoup avec peu de données et presque pas avec beaucoup. **Dites-le, justifiez-le, testez-en plusieurs.**
- Un a posteriori étroit n'est valable que si le **modèle** est juste (le cas du modèle de Poisson ci-dessus).
- La loi **prédictive** incorpore l'incertitude sur les paramètres.

- Quand l'a posteriori n'est pas une loi connue (régression logistique, modèles hiérarchiques...), il faut **simuler** : c'est l'objet des sections 6.2 et 6.3.

## 6.2 Méthodes de Monte-Carlo

**Intuition.** Comment connaître la probabilité de faire un « double six » avec deux dés ? On peut raisonner  $(1/36)$ . Ou bien... **lancer les dés** dix mille fois et compter. La seconde méthode est moins élégante, mais elle a une qualité inestimable : elle marche **aussi** quand le raisonnement est impossible. Tout le monde ne sait pas calculer la probabilité qu'un portefeuille perde plus de 10 %, ou la probabilité qu'une loi *a posteriori* complexe dépasse un seuil. Mais tout le monde peut **simuler** et compter. Le nom vient du casino de Monte-Carlo, où le hasard est le métier. La méthode a été mise au point à Los Alamos dans les années 1940.

### 6.2.1 Le principe : une espérance se calcule en moyennant des simulations

Presque tout ce que l'on veut calculer en probabilités et en statistique est une **espérance** :

- une probabilité est l'espérance d'un indicateur :  $P(X \in A) = \mathbb{E}[\mathbf{1}_A(X)]$  ;
- une intégrale  $\int g(x)f(x) dx$  est l'espérance  $\mathbb{E}[g(X)]$  pour  $X$  de densité  $f$  ;
- une moyenne a posteriori, un risque, un prix, une probabilité de ruine...

La loi des grands nombres (volume I, section 2.4) dit que la moyenne empirique converge vers l'espérance. D'où l'idée :

**Estimateur de Monte-Carlo.** Pour estimer  $\theta = \mathbb{E}[g(X)]$ , on simule  $X_1, \dots, X_n$  indépendants de même loi que  $X$ , et on prend

$$\hat{\theta}_n = \frac{1}{n} \sum_{i=1}^n g(X_i).$$

**Exemple à la main : estimer  $\pi$  avec des fléchettes.** On lance des fléchettes uniformément dans le carré  $[0, 1]^2$ . La probabilité de tomber dans le quart de disque  $x^2 + y^2 < 1$  est son aire,  $\pi/4$ . Donc  $\pi = 4 \mathbb{E}[\mathbf{1}\{X^2 + Y^2 < 1\}]$ . Voici dix fléchettes (choisies une fois pour toutes) ; on vérifie à la main pour chacune si  $x^2 + y^2 < 1$  :

| fléchette | $(x, y)$      | $x^2 + y^2$ | dedans ? |
|-----------|---------------|-------------|----------|
| 1         | (0,10 ; 0,90) | 0,82        | oui      |
| 2         | (0,80 ; 0,70) | 1,13        | non      |
| 3         | (0,35 ; 0,20) | 0,1625      | oui      |
| 4         | (0,95 ; 0,30) | 0,9925      | oui      |
| 5         | (0,60 ; 0,55) | 0,6625      | oui      |
| 6         | (0,05 ; 0,15) | 0,025       | oui      |
| 7         | (0,90 ; 0,90) | 1,62        | non      |
| 8         | (0,45 ; 0,85) | 0,925       | oui      |
| 9         | (0,70 ; 0,10) | 0,50        | oui      |
| 10        | (0,25 ; 0,65) | 0,485       | oui      |

Huit fléchettes sur dix sont dedans :  $\hat{\pi} = 4 \times 0,8 = 3,2$ . Pas très précis, mais l'idée est là. Laissons maintenant l'ordinateur lancer plus de fléchettes :

```

import numpy as np
import pandas as pd
from scipy import stats

pts = np.array([(0.10, 0.90), (0.80, 0.70), (0.35, 0.20), (0.95, 0.30), (0.60, 0.55),
                (0.05, 0.15), (0.90, 0.90), (0.45, 0.85), (0.70, 0.10), (0.25, 0.65)])
dedans = (pts**2).sum(axis=1) < 1
print("dix fléchettes à la main : dedans =", int(dedans.sum()), "-> pi estimé =", 4 * dedans.mean())

rng = np.random.default_rng(620)
lignes = []
for n in (100, 1_000, 10_000, 100_000, 1_000_000):
    xy = rng.random((n, 2))
    indic = ((xy**2).sum(axis=1) < 1)
    est = 4 * indic.mean()
    se_th = np.sqrt(np.pi * (4 - np.pi) / n)          # écart-type théorique de l'estimateur (voir
    ↪ plus bas)
    lignes.append({"n": n, "estimation": round(est, 5), "erreur": round(est - np.pi, 5), "écart-type
    ↪ théorique": round(se_th, 5)})
print(pd.DataFrame(lignes).to_string(index=False))

```

```
dix fléchettes à la main : dedans = 8 -> pi estimé = 3.2
```

| n       | estimation | erreur   | écart-type théorique |
|---------|------------|----------|----------------------|
| 100     | 3.12000    | -0.02159 | 0.16422              |
| 1000    | 3.15200    | 0.01041  | 0.05193              |
| 10000   | 3.13000    | -0.01159 | 0.01642              |
| 100000  | 3.13760    | -0.00399 | 0.00519              |
| 1000000 | 3.14366    | 0.00207  | 0.00164              |

L'estimation s'approche de  $\pi$  quand  $n$  grandit, **sans jamais être exacte**. Mais à quelle vitesse ? C'est la question suivante.

### 6.2.2 La vitesse de convergence : $1/\sqrt{n}$

Chaque terme  $g(X_i)$  est une variable aléatoire de variance  $\sigma^2 = \text{Var}(g(X))$ . L'estimateur  $\hat{\theta}_n$  est sans biais, et, puisque les  $X_i$  sont indépendants (volume I, section 2.3) :

$$\text{Var}(\hat{\theta}_n) = \frac{\sigma^2}{n}, \quad \text{donc} \quad \text{erreur type} = \frac{\sigma}{\sqrt{n}}.$$

Le théorème central limite (volume I, section 2.4) dit en plus que, pour  $n$  grand,  $\hat{\theta}_n \approx \mathcal{N}(\theta, \sigma^2/n)$  : on obtient donc un **intervalle de confiance** de Monte-Carlo,  $\hat{\theta}_n \pm 1,96 \hat{\sigma} / \sqrt{n}$ , où  $\hat{\sigma}$  est l'écart-type empirique des  $g(X_i)$ . Pour  $\pi$ ,  $g = 4 \cdot \mathbf{1}\{\dots\}$  est une variable qui vaut 4 avec la probabilité  $p = \pi/4$  et 0 sinon, d'où  $\sigma^2 = 16p(1-p) = \pi(4-\pi)$  : c'est la formule utilisée dans le code ci-dessus.

**Conséquence pratique.** Pour diviser l'erreur par 10, il faut **100 fois plus de simulations**. Pour gagner une décimale de précision, 100 fois plus de calcul. C'est lent. Vérifions-le par une expérience : pour chaque  $n$ , on répète 300 fois l'estimation de  $\pi$  et on mesure l'écart-type observé des estimations.

```

rng = np.random.default_rng(621)
tailles = [100, 1_000, 10_000, 100_000]
reps = 300
ecarts = []
for n in tailles:
    estimations = np.array([4 * ((rng.random((n, 2))**2).sum(axis=1) < 1).mean() for _ in range(reps)])
    ecarts.append(estimations.std(ddof=1))
ecarts = np.array(ecarts)
th = np.sqrt(np.pi * (4 - np.pi) / np.array(tailles))
pente = np.polyfit(np.log10(tailles), np.log10(ecarts), 1)[0]

```

```
print(pd.DataFrame({"n": tailles, "écart-type observé": ecarts.round(5), "théorie  $\sigma/\sqrt{n}$ ":
↳ th.round(5)}).to_string(index=False))
print(f"\npente de log(écart-type) en fonction de log(n) : {pente:.3f} (théorie : -0.5)")
```

| n      | écart-type observé | théorie $\sigma/\sqrt{n}$ |
|--------|--------------------|---------------------------|
| 100    | 0.16832            | 0.16422                   |
| 1000   | 0.05164            | 0.05193                   |
| 10000  | 0.01591            | 0.01642                   |
| 100000 | 0.00503            | 0.00519                   |

```
pente de log(écart-type) en fonction de log(n) : -0.508 (théorie : -0.5)
```

La pente vaut presque exactement  $-1/2$  : doubler le nombre de chiffres décimaux coûte un facteur  $10^4$  en simulations.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
↳ "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
    "legend.frameon": False, "font.size": 10})

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.0), gridspec_kw={"width_ratios": [1, 1.1]})

# Panneau de gauche : 1 000 fléchettes
rng = np.random.default_rng(622)
xy = rng.random((1000, 2))
in_ = (xy**2).sum(axis=1) < 1
ax1.scatter(xy[in_, 0], xy[in_, 1], s=7, color=BLEU, alpha=0.8, label="dedans")
ax1.scatter(xy[~in_, 0], xy[~in_, 1], s=7, color=ORANGE, alpha=0.8, label="dehors")
t = np.linspace(0, np.pi / 2, 200)
ax1.plot(np.cos(t), np.sin(t), color="#0b0b0b", lw=1.0)
ax1.set_aspect("equal")
ax1.set_xlim(0, 1)
ax1.set_ylim(0, 1)
ax1.grid(False)
ax1.set_title(f"1 000 fléchettes : {in_.sum()} dedans, soit  $\pi \approx \{4 * in_.mean():.3f\}")
ax1.legend(loc="upper center", bbox_to_anchor=(0.5, -0.06), ncol=2, markerscale=2.5)

# Panneau de droite : écart-type en fonction de n (échelles log-log)
ax2.loglog(tailles, ecarts, "o-", color=BLEU, lw=1.8)
ax2.loglog(tailles, th, "--", color="#898781", lw=1.4)
ax2.text(110, 0.0075, "points bleus : écart-type observé\ntirets gris : théorie  $\sigma/\sqrt{n}$  (pente -1/2)",
↳ color="#52514e", va="center")
ax2.set_xlabel("nombre de simulations n")
ax2.set_ylabel("écart-type de l'estimateur de  $\pi$ ")
ax2.set_title("Erreur de l'estimateur selon n")
plt.tight_layout()
plt.savefig("figures/ch06-monte-carlo-pi.png", dpi=200, bbox_inches="tight")
plt.close()$ 
```

**Pourquoi utiliser une méthode si lente ?** En dimension 1, une méthode déterministe (les rectangles du volume I, section 1.2) est bien plus efficace : l'erreur de la règle des trapèzes décroît comme  $1/n^2$ . Mais la vitesse  $1/\sqrt{n}$  de Monte-Carlo a une propriété extraordinaire : **elle ne dépend pas de la dimension**. Un quadrillage de 10 points par axe demande  $10^d$  évaluations en dimension  $d$  ; Monte-Carlo, lui, garde le même rythme. Illustration : le volume de la boule unité en dimension 10, dont la valeur exacte est  $\pi^5/5! \approx 2,5502$ .

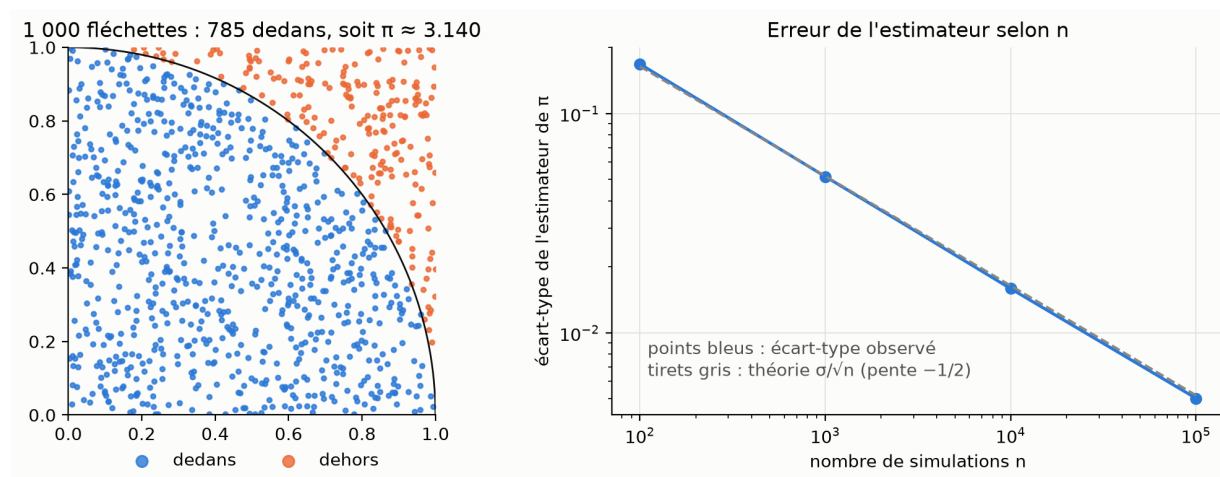


Figure 62: À gauche : 1 000 fléchettes dans le carré, bleues si elles tombent dans le quart de disque, orange sinon. À droite : l'écart-type de l'estimateur de  $\pi$  en fonction du nombre de simulations, en échelles logarithmiques ; il suit la droite de pente  $-1/2$ .

```
from math import gamma, pi

d = 10
rng = np.random.default_rng(624)
n = 1_000_000
pts_d = rng.uniform(-1, 1, size=(n, d))
dans_boule = (pts_d**2).sum(axis=1) < 1
vol_est = 2**d * dans_boule.mean()
se = 2**d * dans_boule.std(ddof=1) / np.sqrt(n)
vol_exact = pi**(d / 2) / gamma(d / 2 + 1)
print(f"boule unité, dimension {d}")
print(f"estimation Monte-Carlo : {vol_est:.4f} ± {1.96 * se:.4f} (IC à 95 %) | valeur exacte :")
print(f"↪ {vol_exact:.4f}")
print(f"fraction de points tombés dans la boule : {dans_boule.mean():.5f} ({dans_boule.sum()}) sur")
print(f"↪ {n}")
print(f"un quadrillage de 10 points par axe aurait demandé {10**d:,} évaluations".replace(",", " "))
```

boule unité, dimension 10  
estimation Monte-Carlo : 2.6020 ± 0.1010 (IC à 95 %) | valeur exacte : 2.5502  
fraction de points tombés dans la boule : 0.00254 (2541 sur 1000000)  
un quadrillage de 10 points par axe aurait demandé 10 000 000 000 évaluations

**Le revers de la médaille.** Remarquez que seule une fraction minuscule des points tombe dans la boule en dimension 10 (2 541 sur un million, soit 0,25 %) : le cube est « presque vide de boule ». Cette inefficacité s'aggrave avec la dimension. Les sections 6.2.5 (échantillonnage préférentiel) et 6.3 (MCMC) sont les réponses à ce problème : mettre les points là où ils comptent.

### 6.2.3 Application : faut-il envoyer l'offre de bienvenue à tout le monde ?

En 6.1.4, nous avons montré que l'offre de bienvenue augmente la probabilité de rachat d'environ 12 points. Mais elle **coûte** quelque chose : la remise accordée. Faut-il l'offrir à tous les futurs clients ?

Posons les hypothèses de l'étude (inventées pour l'exercice, à remplacer par les vraies valeurs de Dar Jasmin) :



- la marge de Yasmine est de 30 % du panier ;
- l'offre coûte **1,50 DT par client** (la remise, quel que soit le client) ;
- un client qui rachète génère un panier tiré au hasard parmi les paniers des acheteurs observés (la distribution empirique, qui est notre meilleure image de la loi des paniers).

Le gain net **par client** de la politique « envoyer l'offre » vaut donc

$$G = (\theta_1 - \theta_0) \times m - c, \quad m = 0,30 \times \text{panier moyen d'un acheteur}, \quad c = 1,50.$$

Les inconnues sont  $\theta_1, \theta_0$  (nous avons leurs lois a posteriori, 6.1.4) et  $m$  (nous le « simulons » par rééchantillonnage des paniers observés, c'est un bootstrap). Monte-Carlo permet de propager **toute** l'incertitude jusqu'au résultat, ce qu'aucune formule simple ne ferait :

```
clients = pd.read_csv("donnees/clients.csv")
# Rappel de 6.1.4 : lois a posteriori Beta(1 + succès, 1 + échecs) pour chaque groupe
resume = clients.groupby("offre_bienvenue")["rachat_l2m"].agg(["size", "sum"])
post0 = stats.beta(1 + resume.loc[0, "sum"], 1 + resume.loc[0, "size"] - resume.loc[0, "sum"])
post1 = stats.beta(1 + resume.loc[1, "sum"], 1 + resume.loc[1, "size"] - resume.loc[1, "sum"])

paniers = clients.loc[clients["panier_moyen"] > 0, "panier_moyen"].to_numpy()
marge, cout = 0.30, 1.50

rng = np.random.default_rng(625)
S = 100_000
theta1 = post1.rvs(S, random_state=rng)
theta0 = post0.rvs(S, random_state=rng)
# incertitude sur la marge moyenne : bootstrap de la moyenne des paniers
m_boot = marge * np.array([rng.choice(paniers, len(paniers)).mean() for _ in range(S // 20)])
m = rng.choice(m_boot, S) # on en tire S valeurs
G = (theta1 - theta0) * m - cout # gain net par client, S scénarios

print(f"marge moyenne par rachat : {marge * paniers.mean():.2f} DT (panier moyen des acheteurs : {paniers.mean():.2f} DT)")
print(f"gain net moyen par client : {G.mean():+.3f} DT")
print(f"intervalle à 90 % : [{np.percentile(G, 5):+.3f} ; {np.percentile(G, 95):+.3f}] DT")
print(f"P(l'offre est rentable) : {(G > 0).mean():.3f}")
print(f"pour 5 000 nouveaux clients : gain attendu {5000 * G.mean():+.0f} DT ; "
      f"dans 95 % des scénarios le gain dépasse {5000 * np.percentile(G, 5):+.0f} DT")

marge moyenne par rachat : 18.37 DT (panier moyen des acheteurs : 61.23 DT)
gain net moyen par client : +0.731 DT
intervalle à 90 % : [+0.062 ; +1.400] DT
P(l'offre est rentable) : 0.964
pour 5 000 nouveaux clients : gain attendu +3656 DT ; dans 95 % des scénarios le gain dépasse +312 DT
```

C'est le genre de réponse qu'attend vraiment une décisionnaire : pas « l'effet est significatif » mais « l'offre est très probablement rentable (environ 96 % de chances), le gain attendu est de 0,73 DT par client, soit 3 700 DT pour 5 000 clients, et dans 95 % des scénarios on gagne au moins 300 DT ». Remarquez que le gain par client reste **modeste** et que l'intervalle à 90 % ([+0,06 ; +1,40] DT) s'approche de zéro : l'offre n'est pas une mine d'or, et un coût de 2 DT (au lieu de 1,50) la rendrait douteuse. Les chiffres de coût et de marge étant inventés, la bonne pratique est de refaire le calcul pour plusieurs valeurs. Observez aussi que **toutes** les sources d'incertitude (les deux taux, la marge) sont propagées d'un seul mouvement, simplement en les simulant ensemble.

**Point de rigueur.** Le calcul suppose que l'effet de l'offre sur le rachat se traduit, pour chaque client qui rachète en plus, par un panier « moyen ». Dans les données simulées, l'offre agit sur la *probabilité* de rachat (c'est vrai par construction) ; une vraie étude vérifierait aussi si les clients « incités » dépensent autant que les autres. Un modèle ne répond qu'à la question qu'on lui a posée.



### 6.2.4 Fabriquer des tirages : l'inversion et le rejet

Jusqu'ici, nous avons utilisé `rng.random`, `stats.beta.rvs`, etc., comme des boîtes noires. Comment un ordinateur fabrique-t-il un tirage dans une loi donnée ? Voici les deux méthodes de base, qui expliquent aussi pourquoi les lois *a posteriori* compliquées posent problème.

**Méthode 1 : l'inversion.** On part d'une loi uniforme  $U \sim \mathcal{U}(0, 1)$ , que l'ordinateur sait fabriquer.

**Théorème (inversion).** Soit  $F$  une fonction de répartition continue et strictement croissante, d'inverse  $F^{-1}$ . Alors  $X = F^{-1}(U)$  a pour fonction de répartition  $F$ .

*Démonstration.* Pour tout  $x$ ,  $P(X \leq x) = P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x)$ , car  $F$  est croissante et  $P(U \leq u) = u$  pour  $u \in [0, 1]$ .  $\square$

*Exemple.* Le délai d'attente d'un colis suit une loi exponentielle de moyenne 3 jours,  $F(x) = 1 - e^{-x/3}$ . En résolvant  $u = 1 - e^{-x/3} : x = -3 \ln(1 - u)$ . On simule donc des délais avec `-3 * np.log(1 - u)`.

```
rng = np.random.default_rng(626)
u = rng.random(100_000)
delaix = -3 * np.log(1 - u) # inversion : F^{-1}(u) pour F(x) = 1 - exp(-x/3)
print(f"moyenne simulée : {delaix.mean():.3f} (théorie : 3)   écart-type : {delaix.std():.3f}
      ↪ (théorie : 3)")
for x in (1, 3, 6, 9):
    print(f"P(délai ≤ {x}) : simulé {np.mean(delaix <= x):.4f} | exact {1 - np.exp(-x / 3):.4f}")
```

```
moyenne simulée : 3.007 (théorie : 3)   écart-type : 3.001 (théorie : 3)
P(délai ≤ 1) : simulé 0.2832 | exact 0.2835
P(délai ≤ 3) : simulé 0.6311 | exact 0.6321
P(délai ≤ 6) : simulé 0.8642 | exact 0.8647
P(délai ≤ 9) : simulé 0.9499 | exact 0.9502
```

L'inversion est parfaite quand on connaît  $F^{-1}$  (exponentielle, Weibull, Cauchy...), mais ce n'est **presque jamais** le cas pour une loi *a posteriori* : on ne sait même pas calculer  $F$ .

**Méthode 2 : le rejet.** Supposons qu'on connaisse la densité cible  $f$  (même à une constante près) et qu'on sache simuler selon une loi plus simple de densité  $g$ , avec  $f(x) \leq M g(x)$  pour tout  $x$ . L'algorithme :

1. tirer  $X \sim g$  et  $U \sim \mathcal{U}(0, 1)$  indépendants ;
2. **accepter**  $X$  si  $U \leq \frac{f(X)}{Mg(X)}$ , sinon recommencer.

**Théorème.** Les valeurs acceptées ont pour densité  $f$ , et la probabilité d'acceptation est  $1/M$ .

*Démonstration.* La probabilité d'accepter un tirage  $X \in dx$  est  $g(x) dx \cdot \frac{f(x)}{Mg(x)} = \frac{f(x)}{M} dx$ .

Sommée sur  $x$ , elle vaut  $\int \frac{f}{M} = \frac{1}{M}$  : c'est la probabilité d'acceptation. La densité de  $X$  **sachant qu'il est accepté** est donc  $\frac{f(x)/M}{1/M} = f(x)$ .  $\square$

*Exemple.* Simulons la loi *a posteriori* Beta(8,4) de 6.1 avec une proposition uniforme sur  $[0, 1]$  ( $g = 1$ ). Il faut  $M \geq \max f$  : le maximum de la densité est atteint en  $\theta = (8 - 1)/(8 + 4 - 2) = 0,7$  et vaut environ 2,94.

```
cible = stats.beta(8, 4)
M = cible.pdf(0.7)
print(f"maximum de la densité Beta(8,4) : M = {M:.4f}   -> probabilité d'acceptation théorique 1/M =
      ↪ {1 / M:.4f}")
```

```

rng = np.random.default_rng(627)
n_essais = 200_000
x = rng.random(n_essais) # proposition : uniforme, g = 1
accepte = rng.random(n_essais) <= cible.pdf(x) / M # test d'acceptation
echantillon = x[accepte]
print(f"taux d'acceptation observé : {accepte.mean():.4f} | tirages conservés :
↳ {len(echantillon)}")
print(f"moyenne : {echantillon.mean():.4f} (exacte {cible.mean():.4f}) | écart-type :
↳ {echantillon.std():.4f} (exact {cible.std():.4f})")
print("quantiles 5 %, 50 %, 95 % :", np.percentile(echantillon, [5, 50, 95]).round(4), "| exacts :",
↳ cible.ppf([0.05, 0.5, 0.95]).round(4))

```

```

maximum de la densité Beta(8,4) : M = 2.9351 -> probabilité d'acceptation théorique 1/M = 0.3407
taux d'acceptation observé : 0.3428 | tirages conservés : 68564
moyenne : 0.6665 (exacte 0.6667) | écart-type : 0.1305 (exact 0.1307)
quantiles 5 %, 50 %, 95 % : [0.4361 0.6763 0.8641] | exacts : [0.4356 0.6762 0.8649]

```

Le rejet marche, mais gaspille : environ deux tirages sur trois sont jetés. En dimension 10 ou 100, la constante  $M$  devient astronomique et l'acceptation tombe à presque zéro : c'est la **malédiction de la dimension** (ce que le volume de la boule laissait deviner). Il faut une autre idée : celle des chaînes de Markov, en 6.3.

### 6.2.5 L'échantillonnage préférentiel (*importance sampling*)

Quand on cherche la probabilité d'un événement **rare**, simuler naïvement est désespérant : on attend longtemps un seul succès. Exemple :  $P(Z > 4)$  pour  $Z \sim \mathcal{N}(0, 1)$ , qui vaut  $3,17 \times 10^{-5}$  (environ une chance sur 31 600). Avec 10 000 tirages, on s'attend à 0,3 succès : le plus souvent, l'estimateur naïf vaut 0.

L'idée de l'échantillonnage préférentiel : **tirer là où l'événement se produit**, puis **corriger** le biais par des poids. Pour estimer  $\mathbb{E}_f[g(X)] = \int g(x)f(x)dx$ , on tire  $X_i \sim h$  (une autre densité, qui charge davantage la zone d'intérêt) et on écrit

$$\int g(x)f(x)dx = \int g(x)\frac{f(x)}{h(x)}h(x)dx = \mathbb{E}_h[g(X)w(X)], \quad w = \frac{f}{h}.$$

L'estimateur  $\hat{\theta} = \frac{1}{n} \sum g(X_i)w(X_i)$  est donc **sans biais** (même démonstration : un changement de variable dans l'intégrale), pourvu que  $h > 0$  partout où  $gf \neq 0$ . Pour  $P(Z > 4)$ , on choisit  $h = \mathcal{N}(4, 1)$  : la moitié des tirages dépasse 4.

```

def naif(n, rng):
    return (rng.standard_normal(n) > 4).mean()

def preferentiel(n, rng):
    x = rng.normal(4, 1, n) # on tire autour de 4
    w = stats.norm.pdf(x) / stats.norm.pdf(x, loc=4, scale=1) # poids f/h
    return np.mean((x > 4) * w)

exact = stats.norm.sf(4)
rng = np.random.default_rng(628)
n, reps = 10_000, 300
a = np.array([naif(n, rng) for _ in range(reps)])
b = np.array([preferentiel(n, rng) for _ in range(reps)])
print(f"valeur exacte P(Z>4) : {exact:.3e}")
print(f"estimateur naïf : moyenne {a.mean():.3e} | écart-type {a.std():.3e} | estimations
↳ nulles : {(a == 0).mean():.0%}")
print(f"échantillonnage préférentiel: moyenne {b.mean():.3e} | écart-type {b.std():.3e}")
print(f"gain en écart-type : facteur {a.std() / b.std():.0f} -> gain en nombre de tirages : facteur
↳ {(a.std() / b.std())**2:.0f}".replace(",", " "))

```

```

valeur exacte P(Z>4)      : 3.167e-05
estimateur naïf           : moyenne 3.567e-05 | écart-type 5.798e-05 | estimations nulles : 69%
échantillonnage préférentiel: moyenne 3.172e-05 | écart-type 6.752e-07
gain en écart-type : facteur 86 -> gain en nombre de tirages : facteur 7 372

```

L'écart-type de l'estimateur naïf ( $5,8 \times 10^{-5}$ ) est plus grand que la quantité à estimer elle-même ( $3,2 \times 10^{-5}$ ), et 69 % des estimations naïves valent exactement 0. Celui de l'échantillonnage préférentiel ( $6,8 \times 10^{-7}$ ) est 86 fois plus petit, ce qui équivaut à  $86^2 \approx 7\,400$  fois plus de tirages pour la même précision. Quand on connaît la forme de la zone d'intérêt, c'est un gain colossal.

**Choisir  $h$  avec soin.** Une proposition mal placée peut être aussi mauvaise que la méthode naïve : trop loin de la zone d'intérêt, quelques tirages portent presque tout le poids et l'estimation devient instable. On mesure cette dégénérescence par la **taille d'échantillon effective** (*effective sample size*), calculée sur les contributions  $c_i = g(X_i) w(X_i)$  :

$$\text{ESS} = \frac{(\sum_i c_i)^2}{\sum_i c_i^2},$$

qui vaut  $n$  si toutes les contributions sont égales et 1 si un seul tirage porte tout. Nous retrouverons cette idée d'ESS en 6.4. Faisons varier le centre  $c$  de la proposition  $h = \mathcal{N}(c, 1)$  (le cas  $c = 0$  est la méthode naïve) :

```

def ess(v):
    return v.sum()**2 / (v**2).sum() if v.sum() > 0 else 0.0

rng = np.random.default_rng(629)
lignes = []
for centre in (0, 2, 3, 4, 5, 6, 8):
    estim, ess_, touche = [], [], []
    for _ in range(300):
        x = rng.normal(centre, 1, 10_000)
        contrib = (x > 4) * stats.norm.pdf(x) / stats.norm.pdf(x, loc=centre, scale=1)
        estim.append(contrib.mean()); ess_.append(ess(contrib)); touche.append((x > 4).mean())
    estim = np.array(estim)
    lignes.append({"centre c": centre, "part de tirages > 4": round(np.mean(touche), 4), "ESS
    ↪ moyenne": round(np.mean(ess_), 1),
                  "écart-type de l'estimateur": f"{estim.std():.2e}", "erreur relative":
    ↪ round(estim.std() / exact, 3)})
print(pd.DataFrame(lignes).to_string(index=False))

```

| centre c | part de tirages > 4 | ESS moyenne | écart-type de l'estimateur | erreur relative |
|----------|---------------------|-------------|----------------------------|-----------------|
| 0        | 0.0000              | 0.3         | 6.16e-05                   | 1.946           |
| 2        | 0.0228              | 187.1       | 2.34e-06                   | 0.074           |
| 3        | 0.1584              | 965.9       | 1.04e-06                   | 0.033           |
| 4        | 0.4996              | 1814.0      | 6.81e-07                   | 0.022           |
| 5        | 0.8413              | 1235.0      | 8.03e-07                   | 0.025           |
| 6        | 0.9771              | 304.9       | 1.68e-06                   | 0.053           |
| 8        | 1.0000              | 3.2         | 3.53e-05                   | 1.114           |

Le tableau raconte toute l'histoire. Avec  $c = 0$  (méthode naïve), pratiquement aucun tirage ne dépasse 4 : l'ESS est inférieure à 1 et l'erreur relative est de l'ordre de 200 %. En se rapprochant de 4, la part de tirages utiles croît, l'ESS atteint **son maximum autour de  $c = 4$**  (environ 1 800 tirages « effectifs » sur 10 000) et l'erreur relative tombe à environ 2 %. Mais si l'on place la proposition **trop loin** ( $c = 8$ ), tous les tirages dépassent 4 et pourtant l'estimateur redevient mauvais : les poids  $f/h$  sont minuscules pour presque tous les tirages et énormes pour de rares valeurs proches de 4. L'ESS s'effondre (environ 3), et l'erreur relative remonte à plus de 100 %. **Centrer la proposition juste sur la zone d'intérêt**, ni trop près ni trop loin, c'est tout l'art.

### 6.2.6 Réduire la variance : faire mieux avec les mêmes tirages

L'erreur est  $\sigma / \sqrt{n}$  ; plutôt que de multiplier  $n$ , on peut **réduire**  $\sigma$ . Prenons une intégrale que l'on sait calculer autrement (pour avoir une référence) :  $I = \int_0^1 e^{-x^2} dx \approx 0,746824$ . Estimateur naïf :  $f(U) = e^{-U^2}$  avec  $U \sim \mathcal{U}(0,1)$ .

**Variables antithétiques.** À chaque tirage  $U$ , on utilise aussi  $1 - U$  (qui suit aussi la loi uniforme) et on moyenne les deux :  $\frac{1}{2}[f(U) + f(1 - U)]$ .

**Pourquoi ça marche.** Si  $\rho = \text{Corr}(f(U), f(1 - U))$ , la variance de la moyenne de ces deux valeurs est  $\frac{\sigma^2}{2}(1 + \rho)$  (volume I, section 2.3). Pour une fonction **monotone** (ici décroissante),  $f(U)$  et  $f(1 - U)$  varient en sens contraire :  $\rho < 0$ , et la variance diminue (elle serait même nulle si  $\rho = -1$ ). Mais attention : on a utilisé **deux** évaluations de  $f$  ; la comparaison honnête se fait à nombre d'évaluations égal.

**Variable de contrôle.** On connaît l'espérance exacte d'une variable  $h(U)$  corrélée à  $f(U)$  (ici  $h(U) = U$ , d'espérance  $1/2$ ). On corrige l'estimateur :

$$\hat{I}_c = \frac{1}{n} \sum [f(U_i) - c(h(U_i) - \mathbb{E}[h])].$$

Il reste sans biais pour tout  $c$ , et sa variance est minimale pour  $c^* = \text{Cov}(f, h) / \text{Var}(h)$ , où elle vaut  $(1 - \rho^2) \sigma^2 / n$  avec  $\rho = \text{Corr}(f(U), h(U))$  : plus  $h$  ressemble à  $f$ , plus on gagne.

**Stratification.** On découpe  $[0, 1]$  en  $K$  tranches égales et on tire le même nombre de points dans chaque, ce qui garantit une couverture régulière de l'intervalle.

On compare les quatre méthodes à budget égal ( $n = 1\,000$  évaluations de  $f$ ), en répétant 500 fois l'expérience :

```
f = lambda x: np.exp(-x**2)
I_exact = 0.7468241328124270 # valeur de référence (fonction erreur)
rng = np.random.default_rng(630)
n, reps, K = 1000, 500, 10

def naif(rng):
    return f(rng.random(n)).mean()

def antithetique(rng):
    u = rng.random(n // 2) # n/2 tirages -> n évaluations
    return 0.5 * (f(u) + f(1 - u)).mean()

def controle(rng):
    u = rng.random(n)
    fu = f(u)
    c = np.cov(fu, u)[0, 1] / u.var(ddof=1) # coefficient optimal estimé
    return (fu - c * (u - 0.5)).mean()

def stratifie(rng):
    m = n // K
    u = (np.arange(K)[:, None] + rng.random((K, m))) / K # m points dans chacune des K
    ↪ tranches
    return f(u).mean()

res = {}
for nom, fct in (("naïf", naif), ("antithétique", antithetique), ("variable de contrôle", controle),
    ↪ ("stratifié", stratifie)):
    est = np.array([fct(rng) for _ in range(reps)])
    res[nom] = {"moyenne": est.mean(), "biais": est.mean() - I_exact, "écart-type": est.std(ddof=1)}
tab = pd.DataFrame(res).T
tab["gain de variance"] = (tab.loc["naïf", "écart-type"] / tab["écart-type"])**2
```

```
with pd.option_context("display.float_format", "{:.6f}".format):
    print(tab.to_string())
```

|                      | moyenne  | biais     | écart-type | gain de variance |
|----------------------|----------|-----------|------------|------------------|
| naïf                 | 0.747244 | 0.000420  | 0.006639   | 1.000000         |
| antithétique         | 0.746821 | -0.000003 | 0.001311   | 25.626675        |
| variable de contrôle | 0.746816 | -0.000008 | 0.000942   | 49.671695        |
| stratifié            | 0.746838 | 0.000013  | 0.000645   | 105.946234       |

Les quatre méthodes sont (à peu près) **sans biais** (la colonne biais est de l'ordre de l'erreur de simulation : l'écart-type d'une moyenne de 500 répétitions est d'environ 0,0003), mais leurs écarts-types diffèrent : la colonne gain de variance dit combien de fois moins de tirages on aurait pu faire pour la même précision. Les antithétiques gagnent un facteur 26 (écart-type divisé par 5), la variable de contrôle un facteur 50, et la stratification un facteur 106 (écart-type divisé par 10). Elles sont ici très efficaces parce que la fonction est lisse et monotone. Dans des problèmes réels à haute dimension, le gain est plus modeste, mais jamais négligeable : **un peu de réflexion avant de simuler vaut souvent mieux qu'un facteur 100 de calcul.**

### 6.2.7 Le bootstrap et le bootstrap bayésien : deux Monte-Carlo que vous connaissez déjà

Le bootstrap du volume I (section 3.3.5) est une méthode de Monte-Carlo : au lieu de simuler à partir d'une loi théorique, on simule à partir de la **loi empirique** (on rééchantillonne les données avec remise). Il existe une version « bayésienne » : plutôt que de tirer des entiers (nombre de fois que chaque observation apparaît), on tire des **poids continus**  $w \sim \text{Dirichlet}(1, \dots, 1)$  et on calcule la statistique pondérée. Les deux donnent des résultats très proches ; le bootstrap bayésien a l'avantage d'une interprétation directe : c'est la loi *a posteriori* d'un modèle sans forme paramétrique, avec un *a priori* très diffus.

Application : la **médiane** du panier des acheteurs de la boutique (une statistique sans formule d'erreur simple).

```
boutique = clients[(clients["canal_acquisition"] == "Boutique") & (clients["panier_moyen"] >
↳ 0)][["panier_moyen"]].to_numpy()
n_b = len(boutique)
rng = np.random.default_rng(631)
B = 5000

med_boot = np.array([np.median(rng.choice(boutique, n_b)) for _ in range(B)])

def mediane_ponderee(x, w):
    ordre = np.argsort(x)
    cw = np.cumsum(w[ordre]) / w.sum()
    return x[ordre][np.searchsorted(cw, 0.5)]

med_bayes = np.array([mediane_ponderee(boutique, rng.dirichlet(np.ones(n_b))) for _ in range(B)])

print(f"{n_b} acheteurs de la boutique ; médiane observée : {np.median(boutique):.2f} DT")
print(f"bootstrap classique : IC95 [{np.percentile(med_boot, 2.5):.2f} ; {np.percentile(med_boot,
↳ 97.5):.2f}] (écart-type {med_boot.std():.2f})")
print(f"bootstrap bayésien : IC95 [{np.percentile(med_bayes, 2.5):.2f} ; {np.percentile(med_bayes,
↳ 97.5):.2f}] (écart-type {med_bayes.std():.2f})")
```

```
449 acheteurs de la boutique ; médiane observée : 67.92 DT
bootstrap classique : IC95 [64.62 ; 70.61] (écart-type 1.56)
bootstrap bayésien : IC95 [64.66 ; 70.61] (écart-type 1.55)
```

**À retenir (6.2).**

- Une espérance se calcule en **moyennant des simulations** :  $\hat{\theta} = \frac{1}{n} \sum g(X_i)$ . Les probabilités, intégrales et moyennes a posteriori sont toutes des espérances.
- L'erreur décroît en  $\sigma / \sqrt{n}$  : **100 fois plus de tirages pour 10 fois moins d'erreur**. Cette vitesse ne dépend pas de la dimension, d'où l'intérêt de la méthode pour les problèmes complexes.
- Pour simuler une loi donnée : **inversion** ( $F^{-1}(U)$ ) si on connaît  $F^{-1}$  ; **rejet** sinon (efficace seulement en petite dimension).
- L'**échantillonnage préférentiel** estime les événements rares en tirant là où ils se produisent et en corrigeant par des poids  $f/h$  ; l'**ESS** détecte les poids dégénérés.
- Les **variables antithétiques, de contrôle et la stratification** réduisent la variance à nombre de tirages égal.
- Le **bootstrap** est un Monte-Carlo sur la loi empirique ; sa version bayésienne tire des poids de Dirichlet.
- Quand ni l'inversion ni le rejet ne marchent (cas général d'une loi a posteriori), il reste les **chaînes de Markov** : section 6.3.

## 6.3 MCMC : Metropolis-Hastings et échantillonnage de Gibbs

**Intuition.** Imaginez un randonneur aveugle lâché dans une chaîne de montagnes, avec pour consigne de **passer plus de temps dans les hautes vallées que dans les basses**, exactement en proportion de l'altitude. Il ne voit pas la carte : il peut seulement, à chaque pas, tâter un point voisin et comparer son altitude à celle de sa position. Règle : *si le point voisin est plus haut, j'y vais toujours ; s'il est plus bas, j'y vais avec une probabilité égale au rapport des altitudes*. Au bout de très longtemps, la proportion du temps passé en chaque lieu est proportionnelle à l'altitude. C'est **Metropolis**. L'« altitude » est la densité a posteriori ; la proportion du temps passé, c'est l'échantillon qu'on cherche.

### 6.3.1 Pourquoi les chaînes de Markov ?

Considérons la régression logistique de `rachat_12m` sur l'offre, le canal et l'âge (nous la ferons en 6.3.5). L'a posteriori est

$$p(\beta | y) \propto \underbrace{\prod_i \sigma(x_i^\top \beta)^{y_i} (1 - \sigma(x_i^\top \beta))^{1-y_i}}_{\text{vraisemblance}} \times \underbrace{\prod_j \mathcal{N}(\beta_j; 0, s^2)}_{\text{a priori}}.$$

Ce n'est **aucune loi connue** : pas de conjugaison, pas de fonction de répartition inversible, et le rejet de 6.2.4 serait inapplicable en dimension 5. Pire : nous ne connaissons même pas la **constante de normalisation**  $p(y)$ , qui est une intégrale en dimension 5. Mais nous savons calculer, pour tout  $\beta$ , la quantité **non normalisée** ci-dessus. Les méthodes MCMC (*Markov chain Monte Carlo*) n'ont besoin que de cela : elles ne s'intéressent qu'aux **rapports**  $p(\beta')/p(\beta)$ , dans lesquels la constante s'annule.

L'idée générale : au lieu de tirer des points *indépendants*, construire une **chaîne de Markov**  $\theta^{(1)}, \theta^{(2)}, \dots$  (chaque point dépend seulement du précédent, volume I, section 2.6.2), conçue pour que sa **loi stationnaire** soit exactement la loi cible. Après un temps de chauffe, les points de la chaîne sont (corrélés) des tirages de la cible.



### 6.3.2 Les chaînes de Markov : loi stationnaire et réversibilité

Rappel (volume I, 2.6.2) : une chaîne à états finis de matrice de transition  $P$  ( $P_{ij} = P(\text{aller de } i \text{ à } j)$ ) admet une **loi stationnaire**  $\pi$  si  $\pi P = \pi$  : si la chaîne a la loi  $\pi$  à un instant, elle l'a encore à l'instant suivant. Sous de bonnes conditions (la chaîne peut aller de tout état à tout état, et n'est pas périodique : on dit qu'elle est *ergodique*), la chaîne **converge** vers  $\pi$  quelle que soit sa position de départ, et la proportion du temps passé dans chaque état tend vers  $\pi$  (théorème ergodique, admis).

Comment fabriquer une chaîne dont la loi stationnaire est une loi  $\pi$  donnée ? Par une condition suffisante simple.

**Théorème (réversibilité  $\Rightarrow$  stationnarité).** Si une matrice de transition  $P$  vérifie l'équation de bilan détaillé

$$\pi_i P_{ij} = \pi_j P_{ji} \quad \text{pour tous } i, j,$$

alors  $\pi P = \pi$ .

*Démonstration.* La  $j$ -ième composante de  $\pi P$  est  $\sum_i \pi_i P_{ij} = \sum_i \pi_j P_{ji} = \pi_j \sum_i P_{ji} = \pi_j$ , car chaque ligne de  $P$  somme à 1.  $\square$

L'équation dit : « en régime stationnaire, le flux de  $i$  vers  $j$  égale le flux de  $j$  vers  $i$  ». Si les flux sont équilibrés pour chaque paire d'états, la population de chaque état est stable.

**Exemple à la main : trois canaux.** Yasmine choisit chaque semaine un canal à mettre en avant parmi 1 = Boutique, 2 = Site, 3 = Instagram, et elle voudrait que ses choix, sur le long terme, suivent des poids (2, 5, 3), c'est-à-dire la loi  $\pi = (0,2; 0,5; 0,3)$ . Elle applique la règle de Metropolis : **proposer** l'un des deux autres canaux au hasard (probabilité  $\frac{1}{2}$  chacun), puis **accepter** avec la probabilité  $\alpha = \min(1, \pi_j/\pi_i)$ , et sinon rester sur place. Construisons la matrice :

- depuis 1 (poids 2) : vers 2,  $\frac{1}{2} \min(1, 5/2) = \frac{1}{2}$  ; vers 3,  $\frac{1}{2} \min(1, 3/2) = \frac{1}{2}$  ; rester : 0 ;
- depuis 2 (poids 5) : vers 1,  $\frac{1}{2} \cdot \frac{2}{5} = 0,2$  ; vers 3,  $\frac{1}{2} \cdot \frac{3}{5} = 0,3$  ; rester : 0,5 ;
- depuis 3 (poids 3) : vers 1,  $\frac{1}{2} \cdot \frac{2}{3} = \frac{1}{3}$  ; vers 2,  $\frac{1}{2} \cdot 1 = 0,5$  ; rester :  $\frac{1}{6}$ .

Vérifions le bilan détaillé à la main :  $\pi_1 P_{12} = 0,2 \times 0,5 = 0,1 = \pi_2 P_{21} = 0,5 \times 0,2$  ✓ ;  $\pi_1 P_{13} = 0,2 \times 0,5 = 0,1 = \pi_3 P_{31} = 0,3 \times \frac{1}{3}$  ✓ ;  $\pi_2 P_{23} = 0,5 \times 0,3 = 0,15 = \pi_3 P_{32} = 0,3 \times 0,5$  ✓. Le code confirme, trouve la loi stationnaire par un calcul d'algèbre linéaire (volume I, section 1.1.3 : vecteur propre associé à la valeur propre 1), et simule la chaîne :

```
import numpy as np
import pandas as pd

poids = np.array([2.0, 5.0, 3.0])
pi = poids / poids.sum()
k = len(poids)

# matrice de transition de Metropolis (proposition : l'un des deux autres états, probabilité 1/2)
P = np.zeros((k, k))
for i in range(k):
    for j in range(k):
        if i != j:
            P[i, j] = 0.5 * min(1.0, poids[j] / poids[i])
    P[i, i] = 1 - P[i].sum()
print("matrice de transition P :\n", P.round(4))

bilan = np.array([pi[i] * P[i, j] - pi[j] * P[j, i] for j in range(k)] for i in range(k))
print("\nbilan détaillé : plus grand |pi_i P_ij - pi_j P_ji| =", np.abs(bilan).max())

# loi stationnaire par algèbre linéaire : vecteur propre à gauche de valeur propre 1
valeurs, vecteurs = np.linalg.eig(P.T)
```



```

stat = np.real(vecteurs[:, np.argmin(np.abs(valeurs - 1))])
stat = stat / stat.sum()
print("loi stationnaire (vecteur propre) :", stat.round(4), "| cible pi :", pi)

# simulation de la chaîne, départ dans l'état 1
rng = np.random.default_rng(630)
etat, compte = 0, np.zeros(k)
n_pas = 100_000
for _ in range(n_pas):
    etat = rng.choice(k, p=P[etat])
    compte[etat] += 1
print("fréquences observées sur", n_pas, "pas :", (compte / n_pas).round(4))

matrice de transition P :
[[0.    0.5   0.5  ]
 [0.2   0.5   0.3  ]
 [0.3333 0.5   0.1667]]

bilan détaillé : plus grand |pi_i P_ij - pi_j P_ji| = 1.3877787807814457e-17
loi stationnaire (vecteur propre) : [0.2 0.5 0.3] | cible pi : [0.2 0.5 0.3]
fréquences observées sur 100000 pas : [0.2009 0.5005 0.2986]

```

Les trois méthodes (vecteur propre, bilan détaillé, simulation) s'accordent : la chaîne visite chaque canal en proportion de son poids. Remarquez que nous n'avons utilisé que les **rapports** de poids  $\pi_j/\pi_i$  : le calcul aurait été le même avec des poids (20, 50, 30). C'est exactement ce qui nous servira quand la constante de normalisation sera inconnue.

### 6.3.3 L'algorithme de Metropolis-Hastings

Généralisons à une loi continue de densité cible  $\pi(\theta)$  (connue à une constante près), en dimension quelconque.

**Metropolis-Hastings.** On se donne une loi de **proposition**  $q(\theta' | \theta)$  dont on sait simuler. Partant de  $\theta^{(t)}$  :

1. tirer une **proposition**  $\theta' \sim q(\cdot | \theta^{(t)})$  ;
2. calculer la probabilité d'acceptation

$$\alpha(\theta, \theta') = \min\left(1, \frac{\pi(\theta') q(\theta | \theta')}{\pi(\theta) q(\theta' | \theta)}\right);$$

3. tirer  $U \sim \mathcal{U}(0, 1)$  ; si  $U \leq \alpha$  :  $\theta^{(t+1)} = \theta'$  (on **accepte**), sinon  $\theta^{(t+1)} = \theta^{(t)}$  (on **reste**, et on **recompte** la valeur).

Quand la proposition est **symétrique** ( $q(\theta' | \theta) = q(\theta | \theta')$ ), par exemple  $\theta' = \theta + \text{bruit gaussien}$ , la « marche aléatoire », le rapport des  $q$  disparaît et on retrouve la règle de Metropolis de l'exemple précédent :  $\alpha = \min(1, \pi(\theta')/\pi(\theta))$ .

**Théorème.** La chaîne de Metropolis-Hastings vérifie le bilan détaillé par rapport à  $\pi$  ; donc  $\pi$  est sa loi stationnaire.

*Démonstration.* Pour  $\theta \neq \theta'$ , la densité de transition est  $K(\theta, \theta') = q(\theta' | \theta) \alpha(\theta, \theta')$ . Alors

$$\pi(\theta) K(\theta, \theta') = \pi(\theta) q(\theta' | \theta) \min\left(1, \frac{\pi(\theta') q(\theta | \theta')}{\pi(\theta) q(\theta' | \theta)}\right) = \min(\pi(\theta) q(\theta' | \theta), \pi(\theta') q(\theta | \theta')).$$

L'expression de droite est **symétrique** en  $(\theta, \theta')$  : elle est égale à  $\pi(\theta') K(\theta', \theta)$ . C'est le bilan détaillé. Il reste à vérifier la stationnarité : pour tout ensemble  $A$ ,  $\int \pi(\theta) K(\theta, A) d\theta = \int_A [\int \pi(\theta) K(\theta, \theta') d\theta'] d\theta + (\text{mass du rejet})$ . Grâce au bilan détaillé, le crochet vaut  $\pi(\theta') \int K(\theta', \theta) d\theta = \pi(\theta') (1 - r(\theta'))$ , où  $r(\theta')$  est la probabilité de rejet en  $\theta'$  ; la part de la

chaîne qui reste sur place (probabilité  $r(\theta)$  en chaque  $\theta$ ) apporte exactement  $\int_A \pi(\theta') r(\theta') d\theta'$ . La somme redonne  $\int_A \pi$ .  $\square$

Pour que la chaîne **converge** effectivement vers  $\pi$  depuis n'importe quel point de départ, il faut en plus qu'elle soit irréductible (elle peut atteindre toute région de probabilité positive) et apériodique ; c'est le cas dès que la proposition gaussienne de la marche aléatoire est utilisée sur un espace continu (théorème admis, voir les références en fin de volume).

**La constante de normalisation disparaît.** Dans  $\alpha$ ,  $\pi$  n'intervient que par le rapport  $\pi(\theta')/\pi(\theta)$  : on peut remplacer  $\pi$  par n'importe quelle fonction proportionnelle (vraisemblance  $\times$  a priori, sans la constante  $p(y)$ ). Pour éviter les dépassements numériques, on travaille toujours avec les **logarithmes** : on accepte si  $\log U < \log \pi(\theta') - \log \pi(\theta)$ .

**Premier test : retrouver la loi Beta(8, 4) de 6.1.** Nous connaissons la bonne réponse (moyenne 2/3, écart-type 0,1307) : nous pouvons donc juger l'algorithme.

```
from scipy import stats

def metropolis_ld(logp, x0, n, pas, rng):
    """Marche aléatoire de Metropolis en dimension 1 : proposition x' = x + pas * N(0, 1)."""
    x, lp = x0, logp(x0)
    chaine = np.empty(n)
    acceptes = 0
    for i in range(n):
        prop = x + pas * rng.standard_normal()
        lpp = logp(prop)
        if np.log(rng.random()) < lpp - lp:
            x, lp = prop, lpp
            acceptes += 1
        chaine[i] = x
    return chaine, acceptes / n

def log_beta84(x):
    # log de la densité NON normalisée de Beta(8, 4)
    return 7 * np.log(x) + 3 * np.log(1 - x) if 0 < x < 1 else -np.inf

rng = np.random.default_rng(631)
chaine, taux = metropolis_ld(log_beta84, x0=0.5, n=20_000, pas=0.2, rng=rng)
exacte = stats.beta(8, 4)
apres = chaine[1000:]
# on jette les 1 000 premiers points (« chauffe »)
print(f"taux d'acceptation : {taux:.3f}")
print(f"moyenne {apres.mean():.4f} (exacte {exacte.mean():.4f}) | écart-type {apres.std():.4f} (exacte {exacte.std():.4f})")
print("quantiles 5 %, 50 %, 95 % :", np.percentile(apres, [5, 50, 95]).round(4), "| exacts :",
      exacte.ppf([0.05, 0.5, 0.95]).round(4))
print(f"test de Kolmogorov-Smirnov contre la loi exacte : statistique {stats.kstest(apres, exacte.cdf).statistic:.4f}")
```

```
taux d'acceptation : 0.589
moyenne 0.6674 (exacte 0.6667) | écart-type 0.1334 (exact 0.1307)
quantiles 5 %, 50 %, 95 % : [0.4321 0.6779 0.8656] | exacts : [0.4356 0.6762 0.8649]
test de Kolmogorov-Smirnov contre la loi exacte : statistique 0.0139
```

La chaîne reproduit la loi cible : moyenne, écart-type et quantiles coïncident à environ le centième. (Le test de Kolmogorov-Smirnov fournit une statistique faible ; sa p-valeur serait trompeuse ici, car les points d'une chaîne sont **corrélés** : le test suppose des observations indépendantes.) Regardons maintenant ce que la chaîne a fait, et, pour comprendre le rôle du **pas**, comparons trois pas.

### 6.3.4 Régler l'algorithme : le pas de la marche aléatoire

Le seul réglage de la marche aléatoire est le **pas** (l'écart-type de la proposition). Il faut le choisir avec soin :

- **pas trop petit** : presque toutes les propositions sont acceptées, mais elles sont minuscules ; la chaîne rampe et met un temps énorme à explorer la loi ;
- **pas trop grand** : presque toutes les propositions tombent dans des zones de faible densité et sont rejetées ; la chaîne reste bloquée sur place ;
- **pas intermédiaire** : un bon compromis.

Pour *mesurer* la qualité de l'exploration, on utilise la **taille d'échantillon effective** (ESS), que nous reverrons en 6.4 :  $n$  points **corrélés** d'une chaîne valent seulement  $ESS = \frac{n}{1 + 2 \sum_{k \geq 1} \rho_k}$  points indépendants, où  $\rho_k$  est l'autocorrélation de la chaîne au décalage  $k$  (volume II, chapitre 4, pour la notion d'autocorrélation). Voici deux petites fonctions pour la calculer (l'autocorrélation par transformée de Fourier rapide ; la somme s'arrête à la première autocorrélation négative) :

```
def autocorr(x, max_lag):
    x = np.asarray(x, dtype=float) - np.mean(x)
    n = len(x)
    f = np.fft.rfft(x, 2 * n) # transformée de Fourier avec remplissage
    ↪ de zéros
    ac = np.fft.irfft(f * np.conj(f))[:n]
    return (ac / ac[0])[: max_lag + 1]

def ess(x):
    rho = autocorr(x, max_lag=min(len(x) // 2, 1000))
    somme = 0.0
    for r in rho[1:]:
        if r < 0:
            break
        somme += r
    return len(x) / (1 + 2 * somme)

lignes = []
traces = {}
for pas in (0.01, 0.05, 0.2, 0.5, 2.0, 10.0):
    rng = np.random.default_rng(632)
    ch, taux = metropolis_ld(log_beta84, x0=0.5, n=20_000, pas=pas, rng=rng)
    traces[pas] = ch
    apres = ch[1000:]
    lignes.append({"pas": pas, "taux d'acceptation": round(taux, 3), "ESS": round(ess(apres)),
                  "ESS / n": round(ess(apres) / len(apres), 3), "autocorr. lag 1":
                  ↪ round(autocorr(apres, 1)[1], 3),
                  "moyenne": round(apres.mean(), 4)})
print(pd.DataFrame(lignes).to_string(index=False))
```

| pas   | taux d'acceptation | ESS  | ESS / n | autocorr. lag 1 | moyenne |
|-------|--------------------|------|---------|-----------------|---------|
| 0.01  | 0.970              | 25   | 0.001   | 0.997           | 0.6900  |
| 0.05  | 0.876              | 444  | 0.023   | 0.944           | 0.6773  |
| 0.20  | 0.594              | 3731 | 0.196   | 0.675           | 0.6662  |
| 0.50  | 0.307              | 3593 | 0.189   | 0.686           | 0.6649  |
| 2.00  | 0.083              | 870  | 0.046   | 0.909           | 0.6612  |
| 10.00 | 0.017              | 217  | 0.011   | 0.978           | 0.6751  |

On lit : avec un pas de 0,01, le taux d'acceptation est proche de 1 mais l'ESS est minuscule (la chaîne ne bouge presque pas) ; avec un pas de 10, le taux d'acceptation s'effondre (1,7 % des propositions seulement tombent dans une zone de densité raisonnable) et l'ESS est de 217, dix-sept fois plus faible qu'à l'optimum ; l'optimum est autour d'un pas de 0,2 à 0,5 (ESS d'environ 3 600 à 3 700, soit 19 % de  $n$ ), avec un taux d'acceptation de 30 à 60 %. Noter que **le taux d'acceptation**

**seul ne suffit pas à juger un réglage** : il vaut 97 % avec le pas de 0,01 (mauvais) et 59 % avec le pas de 0,2 (bon). L'ESS est le vrai juge. Voici les traces correspondantes.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
                    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
                    "legend.frameon": False, "font.size": 10})

fig, axes = plt.subplots(3, 1, figsize=(8.6, 6.6), sharex=True)
for ax, (pas, couleur, etiquette) in zip(axes, [(0.01, ORANGE, "pas trop petit (0,01)"), (0.2, BLEU,
↵ "bon pas (0,2)"), (10.0, VIOLET, "pas trop grand (10)"))]:
    ch = traces[pas][:600]
    ax.plot(ch, color=couleur, lw=0.9)
    ax.set_ylim(0, 1)
    ax.set_ylabel(r"$\theta$")
    ax.set_title(etiquette, loc="left", color=couleur, fontsize=10)
axes[-1].set_xlabel("itération (les 600 premières)")
plt.tight_layout()
plt.savefig("figures/ch06-mh-pas.png", dpi=200, bbox_inches="tight")
plt.close()
```

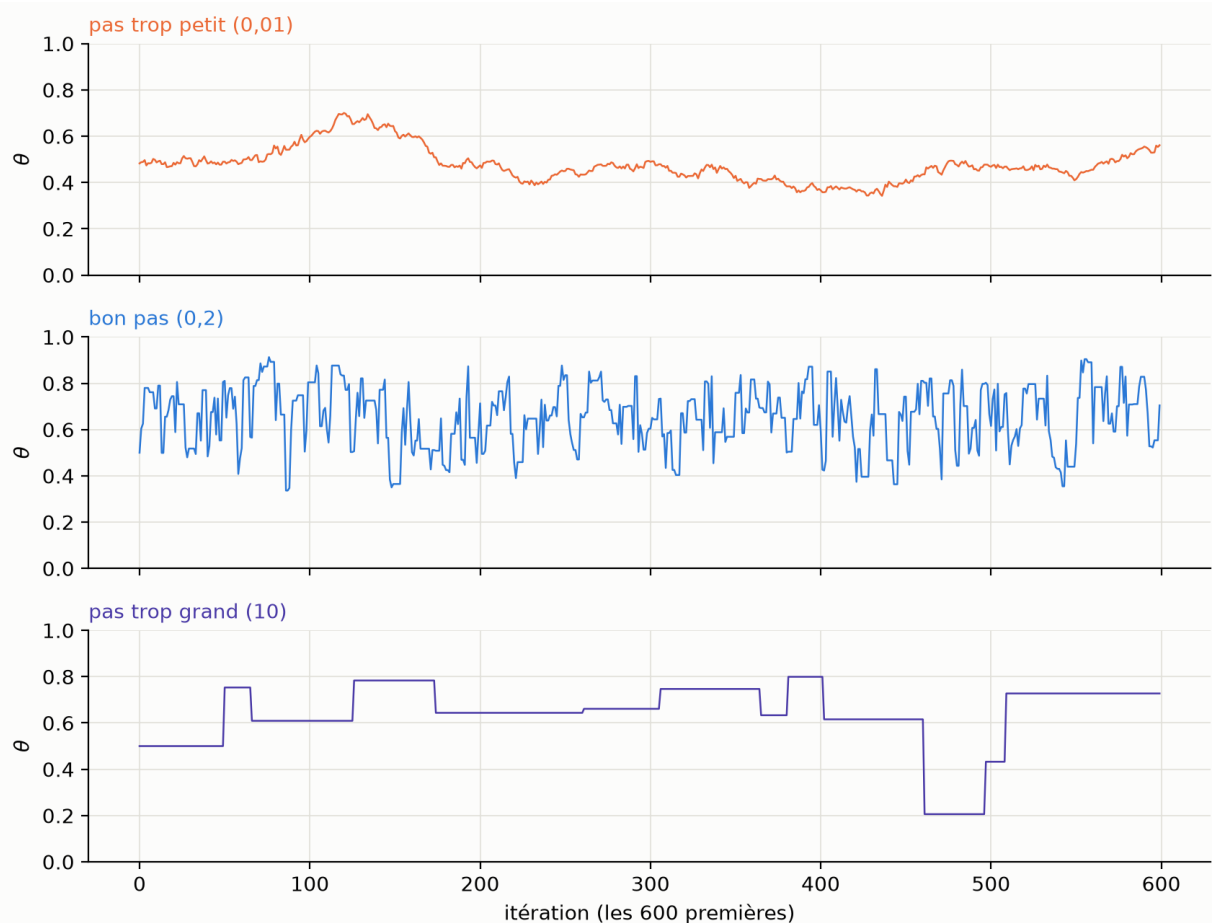


Figure 63: Trois traces de la marche aléatoire de Metropolis pour la loi  $\text{Beta}(8, 4)$ , avec trois pas différents. Pas trop petit : la chaîne rampe lentement. Bon pas : la chaîne explore toute la zone de forte densité. Pas trop grand : la chaîne reste longtemps bloquée sur des paliers.

**Lire une trace.** Une bonne trace ressemble à une « **chenille velue** » : un nuage de points qui oscille de façon irrégulière autour d'une valeur centrale, sans tendance. Une trace **rampante** (trop petit) ressemble à une courbe lisse qui dérive lentement ; une trace **à paliers** (trop grand) a de longs segments plats (la chaîne reste bloquée sur ses refus). Nous verrons en 6.4 comment objectiver cette impression.

**Règles pratiques.** En dimension 1, un taux d'acceptation d'environ 44 % est optimal ; en dimension élevée, des résultats théoriques (Roberts, Gelman et Gilks, 1997) donnent un taux optimal d'environ 23 %, obtenu avec une proposition gaussienne de covariance  $\frac{2,38^2}{d} \hat{\Sigma}$ , où  $d$  est la dimension et  $\hat{\Sigma}$  une estimation de la covariance de la loi cible. Nous utiliserons cette règle pour la régression logistique.

### 6.3.5 Application : la régression logistique bayésienne du rachat

Passons au problème réel. Nous modélisons le rachat dans les 12 mois par

$$y_i \sim \text{Bernoulli}(\sigma(\eta_i)), \quad \eta_i = \beta_0 + \beta_1 \text{offre}_i + \beta_2 \text{Instagram}_i + \beta_3 \text{Site}_i + \beta_4 \text{âge}_i,$$

où  $\sigma(u) = 1/(1 + e^{-u})$ , la Boutique est le canal de référence et l'âge est **standardisé** (centré, divisé par son écart-type : une unité = un écart-type). (La régression logistique est étudiée au chapitre 2, section 2.2 ; ici, on s'intéresse à la façon de **l'estimer** par une méthode bayésienne.) A priori :  $\beta_j \sim \mathcal{N}(0, 2,5^2)$  pour tous les coefficients, un a priori « faiblement informatif » (6.1.6) : il écarte les rapports de cotes extrêmes ( $e^{\pm 5}$ ) sans trop contraindre.

```
import statsmodels.api as sm

clients = pd.read_csv("donnees/clients.csv")
d = clients.copy()
d["age_c"] = (d["age"] - d["age"].mean()) / d["age"].std()
X = pd.get_dummies(d[["offre_bienvenue", "canal_acquisition", "age_c"]],
    ↪ columns=["canal_acquisition"], drop_first=True, dtype=float)
X = X.rename(columns={"canal_acquisition_Instagram": "Instagram", "canal_acquisition_Site": "Site",
    ↪ "offre_bienvenue": "offre"})
X = sm.add_constant(X)
noms = list(X.columns)
Xm, y = X.to_numpy(), d["rachat_12m"].to_numpy()
print("colonnes :", noms, "| n =", len(y), "| rachat moyen :", y.mean().round(3))

S_PRIOR = 2.5
def log_post(beta):
    eta = Xm @ beta
    log_vrais = np.sum(y * eta - np.logaddexp(0, eta))           # somme de y*eta - log(1 + exp(eta))
    log_prior = -0.5 * np.sum(beta**2) / S_PRIOR**2             # normale centrée (constante omise)
    return log_vrais + log_prior

# Point de repère : l'estimation du maximum de vraisemblance (statsmodels)
emv = sm.Logit(y, Xm).fit(dispatch=0)
print("\nlog-posterior en l'EMV :", round(log_post(emv.params), 2))

colonnes : ['const', 'offre', 'age_c', 'Instagram', 'Site'] | n = 2000 | rachat moyen : 0.509

log-posterior en l'EMV : -1355.46
```

Le **log-posterior** est écrit en quelques lignes : c'est toute la modélisation. Il reste à fabriquer la chaîne. Nous prenons une marche aléatoire à **proposition gaussienne multivariée** de covariance  $\frac{2,38^2}{d} \hat{\Sigma}$ , où  $\hat{\Sigma}$  est la covariance estimée des coefficients par le maximum de vraisemblance (l'inverse

de la hessienne, que statsmodels fournit) : c'est un bon « premier dessin » de la forme de l'a posteriori. Pour pouvoir diagnostiquer la convergence en 6.4, nous lançons **quatre chaînes** de points de départ dispersés.

```
def metropolis_multi(logp, x0, n, cov_prop, rng):
    L = np.linalg.cholesky(cov_prop)
    dim = len(x0)
    x, lp = x0.copy(), logp(x0)
    chaine = np.empty((n, dim))
    acceptes = 0
    for i in range(n):
        prop = x + L @ rng.standard_normal(dim)
        lpp = logp(prop)
        if np.log(rng.random()) < lpp - lp:
            x, lp = prop, lpp
            acceptes += 1
        chaine[i] = x
    return chaine, acceptes / n

dim = len(noms)
cov_prop = (2.38**2 / dim) * emv.cov_params()
rng = np.random.default_rng(633)
chaines, taux = [], []
for c in range(4):
    depart = emv.params + rng.normal(0, 1.0, dim)
    ch, tx = metropolis_multi(log_post, depart, n=6000, cov_prop=cov_prop, rng=rng)
    chaines.append(ch)
    taux.append(tx)
chaines = np.array(chaines)
# forme (4 chaînes, 6000
# itérations, 5 paramètres)
print("taux d'acceptation des 4 chaînes :", np.round(taux, 3), "(cible théorique ~ 0,23 à 0,30)")
print("forme du tableau de tirages :", chaines.shape)
```

```
taux d'acceptation des 4 chaînes : [0.284 0.297 0.293 0.274] (cible théorique ~ 0,23 à 0,30)
forme du tableau de tirages : (4, 6000, 5)
```

Les taux d'acceptation sont proches de la valeur théorique. Les 1 000 premières itérations de chaque chaîne (la « chauffe ») sont écartées. Voici le résumé a posteriori, à côté du maximum de vraisemblance :

```
apres = chaines[:, 1000:, :]
tirages = apres.reshape(-1, dim)
resume = pd.DataFrame({
    "EMV": emv.params, "écart-type EMV": emv.bse,
    "moyenne a posteriori": tirages.mean(axis=0), "écart-type a posteriori": tirages.std(axis=0),
    "crédib. 2,5 %": np.percentile(tirages, 2.5, axis=0), "crédib. 97,5 %": np.percentile(tirages,
    97.5, axis=0),
    "ESS": [sum(ess(apres[c, :, j]) for c in range(4)) for j in range(dim)], index=noms)
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 200):
    print(resume.round(3).to_string())
```

|                   | EMV    | écart-type EMV | moyenne a posteriori | écart-type a posteriori | crédib. 2,5 % |
|-------------------|--------|----------------|----------------------|-------------------------|---------------|
| ↳ crédib. 97,5 %  |        | ESS            |                      |                         |               |
| const             | 0.034  | 0.100          | 0.033                | 0.098                   | -0.162        |
| ↳ 0.227 1280.705  |        |                |                      |                         |               |
| offre             | 0.493  | 0.091          | 0.493                | 0.089                   | 0.316         |
| ↳ 0.663 1276.029  |        |                |                      |                         |               |
| age_c             | -0.163 | 0.046          | -0.163               | 0.045                   | -0.256        |
| ↳ -0.076 1114.281 |        |                |                      |                         |               |
| Instagram         | -0.463 | 0.116          | -0.459               | 0.113                   | -0.681        |
| ↳ -0.232 1158.399 |        |                |                      |                         |               |
| Site              | -0.168 | 0.120          | -0.163               | 0.118                   | -0.393        |
| ↳ 0.073 1205.256  |        |                |                      |                         |               |

Les moyennes a posteriori sont **presque identiques** aux estimations du maximum de vraisemblance, et les écarts-types a posteriori sont presque ceux de l'EMV (c'est le théorème de Bernstein-von Mises du 6.1.6 à l'œuvre : avec 2 000 observations et un a priori large, a posteriori et vraisemblance se confondent). Les ESS sont d'environ 1 100 à 1 300 sur 20 000 tirages (l'ESS ne représente qu'environ 6 % du nombre de tirages : les points d'une marche aléatoire sont fortement corrélés). C'est suffisant : l'erreur de simulation sur une moyenne a posteriori vaut  $\sigma/\sqrt{\text{ESS}} \approx 0,089/\sqrt{1\,276} \approx 0,0025$  pour le coefficient de l'offre, soit moins de 3 % de son incertitude a posteriori. (Nous reviendrons en 6.4 sur ce qui rend un ESS « suffisant ».)

Le point clé de l'approche bayésienne est que nous **possédons maintenant des tirages de la loi jointe a posteriori** des cinq coefficients : tout ce que l'on veut calculer devient une moyenne sur ces tirages, sans formule. Par exemple, le **rapport de cotes** de l'offre, et surtout **l'effet de l'offre sur la probabilité de rachat** pour un client de référence (acquis en Boutique, d'âge moyen) :

```
b0, b1 = tirages[:, 0], tirages[:, 1]
rc = np.exp(b1)
print(f"rapport de cotes de l'offre : médiane {np.median(rc):.3f}, IC de crédibilité à 95 %
↳ [{np.percentile(rc, 2.5):.3f} ; {np.percentile(rc, 97.5):.3f}]")
print(f"P(rapport de cotes > 1) = {(rc > 1).mean():.4f} | P(rapport de cotes > 1,5) = {(rc >
↳ 1.5).mean():.4f}")

sigmoide = lambda u: 1 / (1 + np.exp(-u))
effet = sigmoide(b0 + b1) - sigmoide(b0) # variation de probabilité de
↳ rachat (Boutique, âge moyen)
print(f"effet de l'offre sur la probabilité de rachat d'un client Boutique d'âge moyen : "
f"{effet.mean():+.3f} IC95 [{np.percentile(effet, 2.5):+.3f} ; {np.percentile(effet,
↳ 97.5):+.3f}]")

# Comparaison à l'effet brut de 6.1.4 (différence de fréquences)
brut = clients.groupby("offre_bienvenue")["rachat_12m"].mean()
print(f"rappel : différence brute des fréquences (6.1.4) = {brut[1] - brut[0]:+.3f}")
```

rapport de cotes de l'offre : médiane 1.639, IC de crédibilité à 95 % [1.372 ; 1.940]  
P(rapport de cotes > 1) = 1.0000 | P(rapport de cotes > 1,5) = 0.8357  
effet de l'offre sur la probabilité de rachat d'un client Boutique d'âge moyen : +0.120 IC95 [+0.078  
↳ ; +0.161]  
rappel : différence brute des fréquences (6.1.4) = +0.122

Quatre phrases que l'on peut maintenant écrire, avec leur chiffre : le rapport de cotes de l'offre est de l'ordre de 1,6 ; la probabilité qu'il dépasse 1 est quasi certaine ; la probabilité qu'il dépasse 1,5 est d'environ 84 % ; et l'effet de l'offre sur la probabilité de rachat d'un client moyen est de l'ordre de 12 points, en cohérence avec l'estimation brute de 6.1.4 (l'offre ayant été attribuée au hasard, ajuster sur le canal et l'âge ne change presque rien, comme on l'attend). Pour finir, un graphique : pour chaque coefficient, l'intervalle de crédibilité et l'intervalle de confiance du maximum de vraisemblance.

```
fig, ax = plt.subplots(figsize=(7.6, 3.9))
y_pos = np.arange(dim)[::-1]
bas = resume["crédib. 2,5 %"].to_numpy(); haut = resume["crédib. 97,5 %"].to_numpy()
ax.hlines(y_pos + 0.12, bas, haut, color=BLEU, lw=2.4)
ax.plot(resume["moyenne a posteriori"], y_pos + 0.12, "o", color=BLEU, ms=6)
ic = emv.conf_int()
ax.hlines(y_pos - 0.12, ic[:, 0], ic[:, 1], color=ORANGE, lw=2.4)
ax.plot(emv.params, y_pos - 0.12, "s", color=ORANGE, ms=5)
ax.axvline(0, color="#898781", lw=0.8)
ax.set_yticks(y_pos)
ax.set_yticklabels(noms)
ax.set_xlabel("coefficient (échelle du logit)")
ax.text(-1.95, 5.15, "bleu : a posteriori (MCMC), moyenne et intervalle de crédibilité à 95 %",
↳ color=BLEU, fontsize=9, va="center")
```



```
ax.text(-1.95, 4.80, "orange : maximum de vraisemblance et intervalle de confiance à 95 %",
       color=ORANGE, fontsize=9, va="center")
ax.set_ylim(-0.6, 5.4)
ax.set_xlim(-2.0, 1.2)
plt.savefig("figures/ch06-logit-bayes.png", dpi=200, bbox_inches="tight")
plt.close()
```

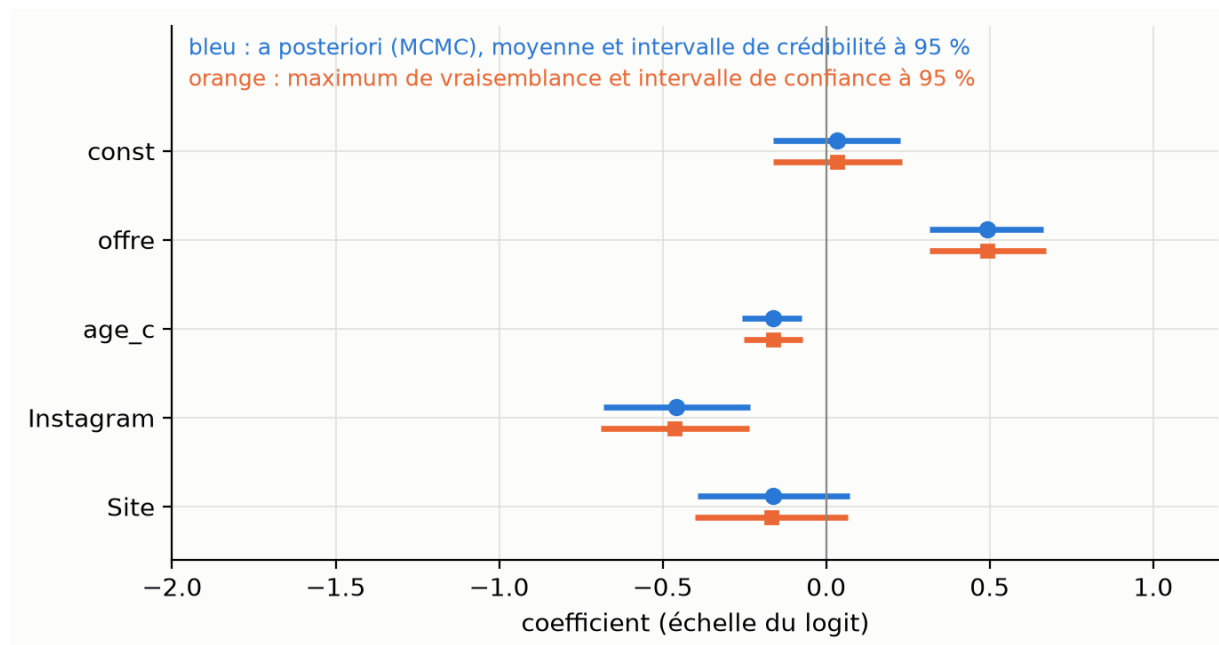


Figure 64: Pour chacun des cinq coefficients de la régression logistique du rachat, l'intervalle de crédibilité à 95 % obtenu par MCMC (bleu) et l'intervalle de confiance à 95 % du maximum de vraisemblance (orange). Les deux approches coïncident presque exactement.

**Que valent ces estimations ? Réponse de la simulation.** Les données étant simulées, nous connaissons les vrais paramètres du générateur : un effet de l'offre de **+0,55** sur le logit, un avantage de la Boutique de **+0,3** par rapport aux deux autres canaux (donc **-0,3** pour Instagram et pour le Site), un effet de l'âge de **-0,015** par an. Notre estimation de l'offre (0,49) est **inférieure à 0,55** mais compatible avec elle (l'intervalle de crédibilité est d'environ  $\pm 0,17$ ). L'effet de l'âge ( $-0,163$  par écart-type, soit  $-0,163/10,5 \approx -0,0155$  par an) retrouve le  $-0,015$  programmé, et les intervalles des canaux contiennent les valeurs programmées ( $-0,3$  pour Instagram comme pour le Site). Mais il y a une raison **systématique**, et pas seulement le hasard, pour laquelle l'effet de l'offre est un peu sous-estimé : le générateur utilise deux facteurs latents (goût pour les produits, sensibilité au service) qui influencent aussi le rachat, et que notre modèle **n'observe pas**. Omettre des variables explicatives *atténue* les coefficients d'une régression logistique, même quand ces variables sont indépendantes de l'offre (c'est la « non-collapsibilité » du rapport de cotes). Vérifions-le sur un très grand échantillon simulé avec la même formule :

```
rng = np.random.default_rng(636)
N = 400_000
F1 = rng.normal(size=N)
F2 = 0.3 * F1 + np.sqrt(1 - 0.3**2) * rng.normal(size=N)      # deux facteurs latents corrélés
offre = rng.integers(0, 2, N).astype(float)
eta = -0.35 + 0.45 * F1 + 0.35 * F2 + 0.55 * offre             # vrai modèle : l'offre vaut 0,55
yy = rng.binomial(1, 1 / (1 + np.exp(-eta)))
marginal = sm.Logit(yy, sm.add_constant(offre)).fit(dis=0).params[1]
condit = sm.Logit(yy, sm.add_constant(np.c_[offre, F1, F2])).fit(dis=0).params[1]
```

```
print(f"coefficient de l'offre sans les facteurs latents : {marginal:.3f}")
print(f"coefficient de l'offre avec les facteurs latents : {condit:.3f} (vrai : 0.55)")
```

```
coefficient de l'offre sans les facteurs latents : 0.499
coefficient de l'offre avec les facteurs latents : 0.548 (vrai : 0.55)
```

Sans les facteurs latents, le coefficient attendu est d'environ 0,50 : notre estimation de 0,49 est exactement ce que la théorie prédit. Voilà un bel exemple de modèle bien estimé mais **mal spécifié** : l'estimation est correcte pour la question posée au modèle, qui n'est pas exactement celle du générateur.

### 6.3.6 L'échantillonnage de Gibbs

Quand le paramètre a plusieurs composantes,  $\theta = (\theta_1, \dots, \theta_d)$ , il est parfois facile de simuler chaque composante **sachant toutes les autres** (on dit : selon sa **loi conditionnelle complète**), même si la loi jointe est compliquée. L'échantillonneur de Gibbs en fait un algorithme : à chaque itération, on met à jour les composantes l'une après l'autre.

**Gibbs.** Partant de  $\theta^{(t)}$  : tirer  $\theta_1^{(t+1)} \sim p(\theta_1 \mid \theta_2^{(t)}, \dots, \theta_d^{(t)})$ , puis  $\theta_2^{(t+1)} \sim p(\theta_2 \mid \theta_1^{(t+1)}, \theta_3^{(t)}, \dots)$ , et ainsi de suite jusqu'à  $\theta_d$ .

**Gibbs est un cas particulier de Metropolis-Hastings, avec acceptation 1.** Mettons à jour la composante  $j$  avec la proposition  $q(\theta' \mid \theta) = \pi(\theta'_j \mid \theta_{-j})$  (les autres composantes inchangées). Le rapport d'acceptation est

$$\frac{\pi(\theta')q(\theta \mid \theta')}{\pi(\theta)q(\theta' \mid \theta)} = \frac{\pi(\theta'_j \mid \theta_{-j}) \pi(\theta_{-j}) \pi(\theta_j \mid \theta_{-j})}{\pi(\theta_j \mid \theta_{-j}) \pi(\theta_{-j}) \pi(\theta'_j \mid \theta_{-j})} = 1,$$

puisque  $\pi(\theta) = \pi(\theta_j \mid \theta_{-j})\pi(\theta_{-j})$ . La proposition est donc **toujours acceptée**, et le théorème de 6.3.3 garantit que  $\pi$  est stationnaire.

**Exemple 1 : la loi normale bivariée.** Simulons  $(X, Y)$  de corrélation  $\rho$  (lois marginales  $\mathcal{N}(0, 1)$ ). Les lois conditionnelles sont connues :  $X \mid Y = y \sim \mathcal{N}(\rho y, 1 - \rho^2)$ , et symétriquement. Gibbs alterne les deux. C'est aussi l'occasion de voir son **point faible** : si les composantes sont fortement corrélées, chaque pas ne peut bouger que dans la direction d'un axe, et la chaîne progresse en zigzag minuscule.

```
def gibbs_binormale(rho, n, rng):
    x = y = 0.0
    s = np.sqrt(1 - rho**2)
    out = np.empty((n, 2))
    for i in range(n):
        x = rng.normal(rho * y, s)          # X | Y
        y = rng.normal(rho * x, s)          # Y | X
        out[i] = (x, y)
    return out

rng = np.random.default_rng(634)
lignes = []
for rho in (0.0, 0.5, 0.9, 0.99):
    ch = gibbs_binormale(rho, 20_000, rng)[1000:]
    lignes.append({"rho vrai": rho, "corrélation estimée": round(np.corrcoef(ch.T)[0, 1], 3),
                  "écart-type de X": round(ch[:, 0].std(), 3), "autocorr. lag 1 de X":
                  round(autocorr(ch[:, 0], 1)[1], 3),
                  "ESS de X (sur 19 000)": round(ess(ch[:, 0]))})
print(pd.DataFrame(lignes).to_string(index=False))
```

| rho vrai | corrélacion estimée | écart-type de X | autocorr. lag 1 de X | ESS de X (sur 19 000) |
|----------|---------------------|-----------------|----------------------|-----------------------|
| 0.00     | -0.003              | 0.997           | -0.006               | 19000                 |
| 0.50     | 0.493               | 1.001           | 0.246                | 11638                 |
| 0.90     | 0.902               | 1.003           | 0.816                | 1966                  |
| 0.99     | 0.990               | 1.011           | 0.980                | 177                   |

Les corrélations estimées retrouvent les  $\rho$  et les écarts-types valent 1, comme prévu. Mais regardez l'ESS : avec  $\rho = 0$  la chaîne est indépendante (ESS proche de  $n$ ), tandis qu'avec  $\rho = 0,9$  elle tombe à environ 2 000 et qu'avec  $\rho = 0,99$  elle s'effondre à moins de 200 tirages efficaces sur 19 000 (1 % du total) : **la corrélation entre composantes ruine Gibbs**. C'est la raison pour laquelle on **reparamétrise** (on décorrèle les paramètres) ou on utilise des méthodes plus sophistiquées comme l'HMC (6.3.7).

**Exemple 2 : une loi normale à moyenne et variance inconnues.** En 6.1.8, nous avons fait l'hypothèse (de confort) que l'écart-type  $\sigma$  du logarithme du panier était *connu*. Levons-la. Les données : les logarithmes des paniers des 449 acheteurs de la boutique. Le modèle :  $y_i \sim \mathcal{N}(\mu, \sigma^2)$ , avec les a priori **indépendants**  $\mu \sim \mathcal{N}(4, 10^2)$  (très large) et la **précision**  $\tau = 1/\sigma^2 \sim \text{Gamma}(a_0 = 0,01, b_0 = 0,01)$  (très diffuse). Les lois conditionnelles complètes sont connues :

- $\mu \mid \tau, y \sim \mathcal{N}(\mu_n, v_n)$  avec  $\frac{1}{v_n} = \frac{1}{\tau_0^2} + n\tau$  et  $\mu_n = v_n \left( \frac{\mu_0}{\tau_0^2} + \tau \bar{y} \right)$  (c'est la formule normale-normale de 6.1.8, avec  $\sigma^2 = 1/\tau$ ) ;
- $\tau \mid \mu, y \sim \text{Gamma}\left(a_0 + \frac{n}{2}, b_0 + \frac{1}{2} \sum_i (y_i - \mu)^2\right)$  (même calcul que gamma-Poisson : multiplier les formes en  $\tau e^{-\tau}$ ).

Il n'y a **pas de formule fermée** pour la loi jointe de  $(\mu, \sigma)$  avec ces a priori indépendants ; Gibbs nous en donne des tirages.

```

acheteurs = clients[(clients["canal_acquisition"] == "Boutique") & (clients["panier_moyen"] > 0)]
ly = np.log(acheteurs["panier_moyen"].to_numpy())
n_l, ybar = len(ly), ly.mean()
mu0, tau0sq, a0, b0 = 4.0, 10.0**2, 0.01, 0.01

def gibbs_normale(n_iter, rng, mu_init, prec_init):
    mu, prec = mu_init, prec_init
    out = np.empty((n_iter, 2))
    for i in range(n_iter):
        v = 1 / (1 / tau0sq + n_l * prec)
        m = v * (mu0 / tau0sq + prec * n_l * ybar)
        mu = rng.normal(m, np.sqrt(v))
        prec = rng.gamma(a0 + n_l / 2, 1 / (b0 + 0.5 * np.sum((ly - mu) ** 2)))
        out[i] = (mu, 1 / np.sqrt(prec))
    return out

rng = np.random.default_rng(635)
g = gibbs_normale(11_000, rng, mu_init=0.0, prec_init=1.0)[1000:] # départ volontairement très
↳ mauvais
mu_s, sig_s = g[:, 0], g[:, 1]

h = stats.t.ppf(0.975, n_l - 1) * ly.std(ddof=1) / np.sqrt(n_l)
print(f"{n_l} acheteurs ; moyenne du log-panier {ybar:.4f} ; écart-type empirique
↳ {ly.std(ddof=1):.4f}")
print(f"Gibbs : mu moyenne {mu_s.mean():.4f} IC95 [{np.percentile(mu_s, 2.5):.4f} ;
↳ {np.percentile(mu_s, 97.5):.4f}]")
print(f"sigma moyenne {sig_s.mean():.4f} IC95 [{np.percentile(sig_s, 2.5):.4f} ;
↳ {np.percentile(sig_s, 97.5):.4f}]")
print(f"classique (Student) : IC95 de mu [{ybar - h:.4f} ; {ybar + h:.4f}]")
print(f"corrélacion a posteriori entre mu et sigma : {np.corrcoef(mu_s, sig_s)[0, 1]:.3f}")
print(f"ESS de mu : {ess(mu_s):.0f}, ESS de sigma : {ess(sig_s):.0f} (sur {len(mu_s)} tirages)")

```

```

449 acheteurs ; moyenne du log-panier 4.2183 ; écart-type empirique 0.3758
Gibbs : mu      moyenne 4.2182 IC95 [4.1838 ; 4.2529]
        sigma moyenne 0.3766 IC95 [0.3525 ; 0.4023]
classique (Student) : IC95 de mu [4.1834 ; 4.2532]
corrélacion a posteriori entre mu et sigma : -0.001
ESS de mu : 10000, ESS de sigma : 9629 (sur 10000 tirages)

```

Les résultats concordent avec le calcul classique. Les ESS sont très proches du nombre de tirages : ici  $\mu$  et  $\sigma$  sont presque indépendants a posteriori (corrélacion proche de 0), donc Gibbs est quasi parfait. Et on peut enfin comparer à la vérité : le générateur combine un bruit de 0,35 et l'effet du facteur latent  $F_1$  (écart-type 0,12), soit un écart-type total de  $\sqrt{0,35^2 + 0,12^2} \approx 0,370$ , et un log-panier moyen de 4,22 pour la Boutique. L'a posteriori contient les deux.

### 6.3.7 En pratique : PyMC, Stan et l'HMC (non exécuté)

Dans les projets réels, on n'écrit pas ses échantillonneurs à la main. Les bibliothèques **PyMC** (Python) et **Stan** (langage dédié, accessible depuis Python, R, etc.) permettent de décrire le modèle et se chargent de l'inférence. Elles utilisent une méthode bien plus puissante que la marche aléatoire : le **Monte-Carlo hamiltonien** (HMC), et sa variante adaptative NUTS. L'idée : au lieu de proposer un pas au hasard, on **simule le mouvement d'une bille** qui roule sur la surface  $-\log \pi(\theta)$  en utilisant le **gradient** de  $\log \pi$  (volume I, section 1.2) ; elle se déplace loin, dans la bonne direction, et les propositions sont presque toujours acceptées. En grande dimension, l'écart d'efficacité avec la marche aléatoire est considérable.

**Non exécuté.** PyMC et Stan **ne sont pas installés** dans l'environnement qui a servi à écrire ce livre. Les deux blocs de code ci-dessous sont donc **montrés sans avoir été exécutés** ; leur résultat attendu est celui de la section 6.3.5 (même modèle, mêmes a priori), mais nous n'avons pas pu le vérifier ici.

```

“python noexec import pymc as pm

```

```

with pm.Model() as modele: beta = pm.Normal("beta", mu=0, sigma=2.5, shape=5) # le même
a priori que dans 6.3.5 eta = pm.math.dot(Xm, beta) pm.Bernoulli("rachat", logit_p=eta, obser-
ved=y) trace = pm.sample(2000, tune=1000, chains=4, random_seed=1) # NUTS par défaut

```

```

```text noexec

```

```

data { int<lower=0> n; matrix[n, 5] X; array[n] int<lower=0, upper=1> y; }
parameters { vector[5] beta; }
model {
  beta ~ normal(0, 2.5);
  y ~ bernoulli_logit(X * beta);
}

```

Comprendre ce que PyMC et Stan font à votre place, c'est comprendre ce chapitre : un log-posterior (que vous venez d'écrire en trois lignes), un algorithme qui explore sa surface, des diagnostics pour savoir s'il a bien exploré (6.4).

#### À retenir (6.3).

- Quand l'a posteriori n'est pas une loi connue, on le **simule avec une chaîne de Markov** dont la loi stationnaire est la cible. Seul le rapport  $\pi(\theta')/\pi(\theta)$  est nécessaire : la constante de normalisation disparaît.
- **Metropolis-Hastings** : proposer, calculer  $\alpha = \min(1, \frac{\pi(\theta')q(\theta|\theta')}{\pi(\theta)q(\theta'|\theta)})$ , accepter ou rester. Il

vérifie le **bilan détaillé**, donc  $\pi$  est stationnaire.

- Le **pas** de la marche aléatoire se règle : trop petit, on rampe ; trop grand, on reste bloqué ; en dimension élevée, viser 23 % d'acceptation avec une covariance  $2,38^2 \hat{\Sigma}/d$ .
- Les points d'une chaîne sont **corrélés** : on les juge par l'ESS, pas par leur nombre. On écarte une période de **chauffe**.
- **Gibbs** met à jour une composante à la fois selon sa loi conditionnelle (acceptation 1) ; il souffre quand les composantes sont très corrélées.
- Une fois les tirages obtenus, **tout** se calcule par moyenne : rapports de cotes, effets sur les probabilités, probabilités de seuils, prédictions.
- PyMC et Stan (HMC/NUTS) font cela mieux et plus vite, mais ne dispensent pas de **vérifier** le résultat (6.4).

## 6.4 Vérification des modèles bayésiens

**Intuition.** Un pilote ne décolle pas sans sa liste de contrôle : carburant, volets, instruments. Un analyste bayésien devrait faire de même, car **deux choses peuvent tourner mal**, et elles sont indépendantes : (1) *l'algorithme* peut avoir mal exploré la loi a posteriori (la chaîne n'a pas convergé), et (2) *le modèle* peut être faux (la loi a posteriori, même parfaitement calculée, répond à une question mal posée). Nous avons vu un exemple de la seconde en 6.1.8 : un modèle de Poisson aux intervalles trop étroits. Cette section donne les outils pour détecter l'une et l'autre.

Dans tout ce qui suit, un échantillon MCMC n'est jamais « bon par nature » : on **démontre** qu'il est bon, ou du moins qu'il ne manifeste aucun signe de mauvaise santé. Un diagnostic qui passe ne prouve pas la convergence (on ne peut jamais prouver qu'une chaîne a tout exploré), mais un diagnostic qui échoue prouve un problème.

### 6.4.1 L'algorithme a-t-il convergé ? Les diagnostics de convergence

Nous reprenons les quatre chaînes de la régression logistique de 6.3.5, lancées depuis des points de départ **dispersés**. Le premier diagnostic, et le plus parlant, est le dessin : la **trace** de chaque chaîne, et la distribution des valeurs prises, chaîne par chaîne.

```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
                    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
                    "legend.frameon": False, "font.size": 10})

j = noms.index("offre") # coefficient de l'offre (chaînes de
↳ 6.3.5)
couleurs = [BLEU, ORANGE, AQUA, VIOLET]
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 3.8), gridspec_kw={"width_ratios": [2.2, 1]})
for c in range(4):
    ax1.plot(chaines[c, :, j], color=couleurs[c], lw=0.6, alpha=0.85)
ax1.axvspan(0, 1000, color="#c3c2b7", alpha=0.35)
ax1.text(500, ax1.get_ylim()[1] - 0.04, "chauffe\n(écartée)", ha="center", va="top", color="#52514e",
↳ fontsize=9)
ax1.set_xlabel("itération")
```

```

ax1.set_ylabel(r"$\beta_{\mathrm{offre}}$")
ax1.set_title("Quatre chaînes, quatre points de départ")
for c in range(4):
    ax2.hist(apres[c, :, j], bins=40, density=True, histtype="step", color=couleurs[c], lw=1.4)
ax2.set_xlabel(r"$\beta_{\mathrm{offre}}$ (après chauffe)")
ax2.set_yticks([])
ax2.set_title("Les quatre distributions")
plt.tight_layout()
plt.savefig("figures/ch06-traces- quatre-chaînes.png", dpi=200, bbox_inches="tight")
plt.close()

```

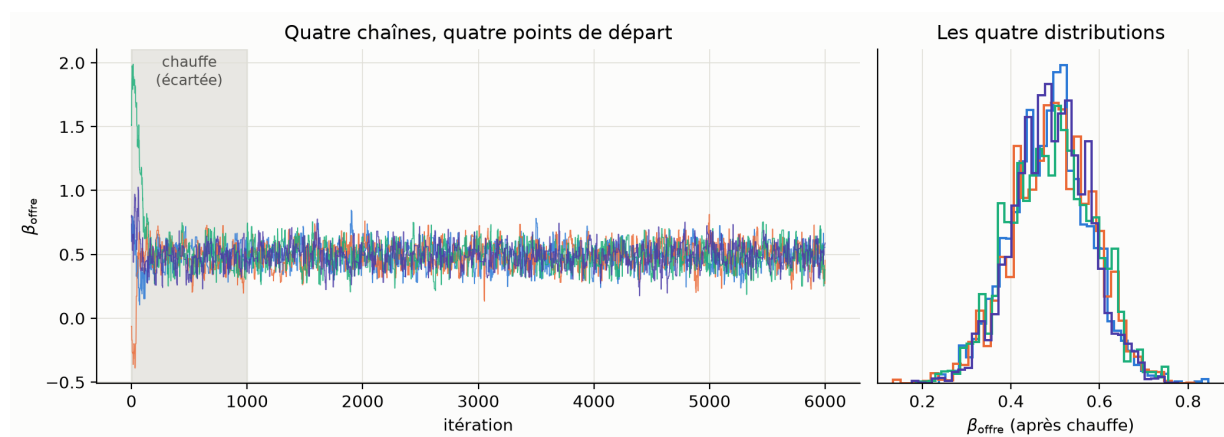


Figure 65: À gauche : la trace du coefficient de l'offre pour les quatre chaînes lancées de points de départ dispersés. La zone grise correspond à la chauffe, écartée. À droite : après la chauffe, les histogrammes des quatre chaînes se superposent.

Ce qu'on cherche (et ce que l'on voit ici) : (i) **les quatre chaînes, parties de points très différents, se retrouvent dans la même zone** après quelques dizaines ou centaines d'itérations (c'est la « chauffe », en anglais *burn-in* ou *warm-up*, que l'on écarte) ; (ii) **elles se mélangent** : on ne distingue plus de quelle chaîne provient chaque point ; (iii) les histogrammes des quatre chaînes **coïncident**. Le dessin ne suffit pourtant pas : une trace peut sembler « stationnaire » à l'œil alors que la chaîne explore à peine un coin de la loi. On quantifie avec le  $\hat{R}$  de Gelman-Rubin.

**L'idée du  $\hat{R}$  (R-chapeau).** Si  $m$  chaînes de longueur  $n$  ont convergé vers la même loi, la variabilité **entre** les chaînes ne doit pas dépasser la variabilité **à l'intérieur** de chaque chaîne. Soient  $\bar{\theta}_j$  et  $s_j^2$  la moyenne et la variance de la chaîne  $j$ , et  $\bar{\theta}$  la moyenne générale. On calcule

$$W = \frac{1}{m} \sum_j s_j^2 \quad (\text{variance intra}), \quad B = \frac{n}{m-1} \sum_j (\bar{\theta}_j - \bar{\theta})^2 \quad (\text{variance inter}),$$

puis la meilleure estimation de la variance de la loi cible,  $\widehat{\text{var}}^+ = \frac{n-1}{n} W + \frac{B}{n}$ , qui **surestime** cette variance tant que les chaînes n'ont pas convergé. On pose

$$\hat{R} = \sqrt{\frac{\widehat{\text{var}}^+}{W}} \geq 1 \quad (\text{approximativement}).$$

Si les chaînes sont bien mélangées,  $B \approx W$  et  $\hat{R} \approx 1$  ; si elles sont chacune bloquées dans un coin différent,  $B \gg W$  et  $\hat{R} \gg 1$ . On utilise aujourd'hui la version « **découpée** » (*split- $\hat{R}$* ) : on coupe chaque chaîne en deux moitiés avant de calculer, ce qui permet de détecter aussi une chaîne qui dérive lentement (ses deux moitiés ne se ressemblent pas). La recommandation actuelle est  $\hat{R} < 1,01$  (l'ancien seuil de 1,1 est jugé trop laxiste).



```
def split_rhat(ch):
    """ch : tableau (m chaînes, n itérations). Split-R-chapeau de Gelman-Rubin."""
    m, n = ch.shape
    moitié = n // 2
    parts = np.concatenate([ch[:, :moitié], ch[:, moitié:2 * moitié]], axis=0) # 2m chaînes de
    ↪ longueur n/2
    M, N = parts.shape
    W = parts.var(axis=1, ddof=1).mean()
    B = N * parts.mean(axis=1).var(ddof=1)
    var_plus = (N - 1) / N * W + B / N
    return np.sqrt(var_plus / W)

def ess_total(ch):
    return sum(ess(ch[c]) for c in range(ch.shape[0])) # somme des ESS de
    ↪ chaque chaîne

lignes = []
for jj, nom in enumerate(noms):
    lignes.append({"paramètre": nom, "R-chapeau (chauffe incluse)": round(split_rhat(chaines[:, :,
    ↪ jj]), 4),
                    "R-chapeau (chauffe écartée)": round(split_rhat(apres[:, :, jj]), 4),
                    "ESS": round(ess_total(apres[:, :, jj]))})
print(pd.DataFrame(lignes).to_string(index=False))
```

| paramètre | R-chapeau (chauffe incluse) | R-chapeau (chauffe écartée) | ESS  |
|-----------|-----------------------------|-----------------------------|------|
| const     | 1.0037                      | 1.0010                      | 1281 |
| offre     | 1.0064                      | 1.0023                      | 1276 |
| age_c     | 1.0163                      | 1.0009                      | 1114 |
| Instagram | 1.0022                      | 1.0037                      | 1158 |
| Site      | 1.0043                      | 1.0024                      | 1205 |

Deux enseignements. D'abord, **écarter la chauffe améliore le diagnostic** : en gardant les 1 000 premières itérations, le  $\hat{R}$  de l'âge est de 1,016 (au-dessus du seuil de 1,01) ; une fois la chauffe retirée, les cinq  $\hat{R}$  sont inférieurs à 1,004 : les quatre chaînes disent la même chose. (L'effet est modeste ici parce que la loi est presque gaussienne et que la chaîne s'y installe vite ; dans un modèle plus difficile, la chauffe peut faire la différence entre  $\hat{R} = 1,5$  et  $\hat{R} = 1,00$ .) Ensuite, les **ESS** sont de l'ordre de 1 100 à 1 300 : pour estimer une moyenne ou un intervalle de crédibilité à 95 %, la règle usuelle est de viser au moins 400 à 1 000 ; nous sommes dans la fourchette. (Pour des quantiles extrêmes, il en faut davantage.)

**Ce que l'ESS que nous calculons simplifie.** Nous additionnons les ESS de chaque chaîne. Les logiciels (Stan, ArviZ) utilisent une version qui combine les autocorrélations de toutes les chaînes et des transformations par rangs : elle est plus fiable pour les lois à queues lourdes. Pour nos besoins, l'ordre de grandeur est le même.

**Que se passe-t-il quand la convergence échoue ?** Un diagnostic n'a de valeur que si on a vu au moins une fois à quoi il ressemble quand il détecte quelque chose. Prenons une loi cible à **deux bosses**, mélange à parts égales de  $\mathcal{N}(-4, 1)$  et  $\mathcal{N}(4, 1)$  (moyenne exacte : 0), et lançons quatre chaînes de Metropolis de points de départ  $-6, -2, 2, 6$ , avec un pas petit (0,5) puis grand (6).

```
def log_bimodale(x):
    return np.logaddexp(stats.norm.logpdf(x, -4, 1), stats.norm.logpdf(x, 4, 1))

lignes, traces_b = [], {}
for pas in (0.5, 6.0):
    rng = np.random.default_rng(641)
    ch = np.array([metropolis_ld(log_bimodale, x0, n=5000, pas=pas, rng=rng)[0] for x0 in (-6, -2, 2,
    ↪ 6)])
    traces_b[pas] = ch
    lignes.append({"pas": pas, "moyennes des 4 chaînes": np.round(ch[:, 1000:].mean(axis=1), 2),
```



```

    "moyenne globale": round(ch[:, 1000:].mean(), 2),
    "R-chapeau": round(split_rhat(ch[:, 1000:]), 3), "ESS": round(ess_total(ch[:,
    ↪ 1000:]))))
print(pd.DataFrame(lignes).to_string(index=False))
print("moyenne exacte de la loi cible : 0.00")

```

| pas | moyennes des 4 chaînes     | moyenne globale | R-chapeau | ESS  |
|-----|----------------------------|-----------------|-----------|------|
| 0.5 | [-4.08, 3.71, 4.06, 3.92]  | 1.90            | 3.072     | 546  |
| 6.0 | [-0.16, 0.16, -0.28, 0.17] | -0.03           | 1.002     | 1430 |

moyenne exacte de la loi cible : 0.00

Avec le petit pas, chaque chaîne reste **piégée dans sa bosse** : une chaîne est restée autour de  $-4$ , les trois autres autour de  $+4$  ; la moyenne globale vaut 1,90 au lieu de 0 ; et le  $\hat{R}$  vaut 3,07, très loin de 1. Remarquez que l'ESS (546) a l'air honorable : elle est calculée chaîne par chaîne et ne voit pas qu'il manque une bosse. **Seule la comparaison entre chaînes (le  $\hat{R}$ ) donne l'alerte.** Fait plus grave : si on n'avait lancé *qu'une seule chaîne*, on aurait trouvé une moyenne proche de  $-4$  ou de  $+4$  avec un bel intervalle de crédibilité étroit, **tout à fait faux** : rien dans une chaîne unique ne signale qu'il manque une bosse. C'est la raison pour laquelle on lance toujours **plusieurs chaînes depuis des points de départ dispersés**. Avec le grand pas, les chaînes sautent d'une bosse à l'autre :  $\hat{R} = 1,002$  et la moyenne globale ( $-0,03$ ) est proche de 0.

**Le diagnostic le plus important : plusieurs chaînes dispersées.** Presque tous les problèmes de convergence se voient en comparant des chaînes parties de points différents. **Ne jamais se contenter d'une seule chaîne.**

#### 6.4.2 Le modèle est-il plausible ? Les vérifications prédictives a posteriori

Même si la chaîne est parfaite, le modèle peut être faux. Le test de bon sens est le suivant : **si le modèle était vrai, les données qu'il simulerait ressembleraient-elles aux données réelles ?** C'est la **vérification prédictive a posteriori** (*posterior predictive check*).

**Procédure.** (1) Tirer un jeu de paramètres  $\theta^{(s)}$  dans la loi a posteriori. (2) Simuler un **jeu de données répliqué**  $y^{\text{rep},(s)}$  avec ce  $\theta^{(s)}$ , de la même taille que les données observées. (3) Calculer une **statistique-test**  $T$  (une caractéristique qui nous inquiète : variance, maximum, proportion de zéros...) sur chaque jeu répliqué. (4) Comparer  $T(y)$  observé à la distribution des  $T(y^{\text{rep}})$ . Le **p-value bayésien**  $p_B = P(T(y^{\text{rep}}) \geq T(y) | y)$  est la fraction de répliques au-dessus de l'observé : une valeur proche de 0 ou de 1 signale un désaccord.

*Remarque* : il ne s'agit pas d'un test au sens du volume I : le  $p_B$  n'est pas uniforme sous le modèle vrai (il est conservateur, car les données servent deux fois : à ajuster le modèle et à le tester). On l'utilise comme **indicateur de désaccord**, pas comme un seuil de décision.

**Retour sur le modèle de Poisson de 6.1.8.** Nous avons modélisé le nombre de commandes annuelles `nb_commandes_an` des 504 clients de la boutique par une loi de Poisson de paramètre  $\lambda$  commun, et obtenu l'a posteriori  $\text{Gamma}(2087; 504,5)$ . Il avait l'air parfait : un intervalle étroit. Soumettons-le à la vérification.

```

clients = pd.read_csv("donnees/clients.csv")
b = clients.loc[clients["canal_acquisition"] == "Boutique", "nb_commandes_an"].to_numpy()
n_b = len(b)

def stats_test(yrep):
    """Trois statistiques sur chaque jeu (lignes) : variance/moyenne, part de zéros, maximum."""
    return np.c_[yrep.var(axis=1, ddof=1) / yrep.mean(axis=1), (yrep == 0).mean(axis=1),
    ↪ yrep.max(axis=1)]

```

```

T_obs = stats_test(b[None, :])[0]
noms_T = ["variance / moyenne", "part de zéros", "maximum"]

rng = np.random.default_rng(642)
lam = stats.gamma(a=2 + b.sum(), scale=1 / (0.5 + n_b)).rvs(2000, random_state=rng)      # a
↳ posteriori de 6.1.8
yrep_pois = rng.poisson(lam[:, None], size=(2000, n_b))                                # 2000 jeux
↳ répliqués
T_pois = stats_test(yrep_pois)

lignes = []
for k_, nom in enumerate(noms_T):
    lignes.append({"statistique": nom, "observée": round(T_obs[k_], 3), "répliques : moyenne":
↳ round(T_pois[:, k_].mean(), 3),
                    "répliques : 2,5 %": round(np.percentile(T_pois[:, k_], 2.5), 3),
                    "répliques : 97,5 %": round(np.percentile(T_pois[:, k_], 97.5), 3),
                    "p bayésien": round((T_pois[:, k_] >= T_obs[k_]).mean(), 3)})
print(pd.DataFrame(lignes).to_string(index=False))

```

| statistique        | observée | répliques : moyenne | répliques : 2,5 % | répliques : 97,5 % | p bayésien |
|--------------------|----------|---------------------|-------------------|--------------------|------------|
| variance / moyenne | 3.324    | 1.000               | 0.879             | 1.127              | 0.0        |
| part de zéros      | 0.109    | 0.016               | 0.006             | 0.030              | 0.0        |
| maximum            | 21.000   | 11.541              | 10.000            | 14.000             | 0.0        |

Le verdict est sans appel. La variance observée est **3,3 fois la moyenne**, alors que les jeux répliqués par le modèle de Poisson ont un rapport proche de 1. Le modèle ne simule **ni autant de zéros** (11 % observés, environ 1,6 % répliqués), **ni de valeurs aussi grandes** (maximum observé de 21, contre 11,5 en moyenne dans les répliques, avec un intervalle à 95 % de [10 ; 14]). Les trois  $p_B$  valent 0 : aucune réplique sur 2 000 n'atteint les données. **Le modèle de Poisson est rejeté**, et la raison est celle qu'on soupçonnait : les clients sont hétérogènes.

**Réparation : la loi binomiale négative.** On suppose que le paramètre de Poisson **varie d'un client à l'autre** selon une loi gamma ; en intégrant cette variation (la marginalisation d'un mélange gamma-Poisson, que nous retrouverons au chapitre 2, section 2.6), le nombre de commandes suit une loi **binomiale négative** de moyenne  $\mu$  et de paramètre de dispersion  $k$  :  $\text{Var} = \mu + \mu^2/k$ . Quand  $k \rightarrow \infty$ , on retrouve Poisson ; plus  $k$  est petit, plus la surdispersion est forte. Il n'y a plus de conjugaison, donc... on utilise notre MCMC. Paramétrons  $\theta = (\log \mu, \log k)$  pour travailler sur des réels, avec des a priori  $\log \mu \sim \mathcal{N}(1, 2^2)$  et  $\log k \sim \mathcal{N}(0, 2^2)$ , larges.

```

def log_post_nb(theta):
    mu, k = np.exp(theta[0]), np.exp(theta[1])
    log_vrais = stats.nbinom.logpmf(b, k, k / (k + mu)).sum()
    log_prior = -0.5 * ((theta[0] - 1) / 2) ** 2 - 0.5 * ((theta[1] - 0) / 2) ** 2
    return log_vrais + log_prior

rng = np.random.default_rng(643)
pilote, _ = metropolis_multi(log_post_nb, np.array([np.log(b.mean()), 0.0]), 3000, np.diag([0.003,
↳ 0.02]), rng)
cov_nb = np.cov(pilote[500:].T) * (2.38**2 / 2)                                     # covariance estimée sur une
↳ course pilote

chaines_nb, taux_nb = [], []
for c in range(4):
    depart = np.array([np.log(b.mean()) + rng.normal(0, 0.3), rng.normal(0, 1.0)])
    ch, tx = metropolis_multi(log_post_nb, depart, 5000, cov_nb, rng)
    chaines_nb.append(ch[1000:]); taux_nb.append(tx)
chaines_nb = np.array(chaines_nb)
print("taux d'acceptation :", np.round(taux_nb, 3))
print("R-chapeau (log mu, log k) :", [round(float(split_rhat(chaines_nb[:, :, i])), 4) for i in
↳ range(2)],
    "| ESS :", [int(round(ess_total(chaines_nb[:, :, i]))) for i in range(2)])

```

```
mu_s, k_s = np.exp(chaines_nb[:, :, 0].ravel()), np.exp(chaines_nb[:, :, 1].ravel())
print(f"mu : moyenne {mu_s.mean():.3f} IC95 [{np.percentile(mu_s, 2.5):.3f} ; {np.percentile(mu_s,
↪ 97.5):.3f}]")
print(f"k : médiane {np.median(k_s):.3f} IC95 [{np.percentile(k_s, 2.5):.3f} ; {np.percentile(k_s,
↪ 97.5):.3f}]")
print(f"rappel Poisson (6.1.8) : IC95 de lambda [3.961 ; 4.316], largeur {4.316 - 3.961:.3f} |
↪ binomiale négative : largeur de mu {np.diff(np.percentile(mu_s, [2.5, 97.5]))[0]:.3f}")
```

```
taux d'acceptation : [0.365 0.352 0.357 0.354]
R-chapeau (log mu, log k) : [1.0029, 1.0019] | ESS : [1951, 2241]
mu : moyenne 4.134 IC95 [3.817 ; 4.457]
k : médiane 1.817 IC95 [1.513 ; 2.199]
rappel Poisson (6.1.8) : IC95 de lambda [3.961 ; 4.316], largeur 0.355 | binomiale négative : largeur
↪ de mu 0.639
```

Les chaînes sont saines ( $\hat{R}$  proche de 1, ESS élevées). La dispersion  $k$  est estimée à environ 1,8 (intervalle environ  $[1,5 ; 2,2]$ ), loin de l'infini de Poisson. Notez surtout que **l'intervalle de crédibilité de la moyenne est plus large que celui du modèle de Poisson** (largeur 0,64 contre 0,36) : le modèle de Poisson **sous-estimait l'incertitude** en la faisant porter par une moyenne unique, alors que les clients varient. C'est l'effet typique d'un modèle trop simple : intervalles trop étroits. Les données étant simulées, la vérité est connue : le générateur utilise justement une dispersion  $k = 2$  (avec, en plus, d'autres sources d'hétérogénéité : le goût pour les produits et l'âge) : le  $k$  estimé est cohérent. Révérifions le modèle réparé avec les mêmes statistiques :

```
idx = rng.choice(len(mu_s), 2000, replace=False)
p_nb = k_s[idx] / (k_s[idx] + mu_s[idx])
yrep_nb = rng.negative_binomial(k_s[idx][:, None], p_nb[:, None], size=(2000, n_b))
T_nb = stats_test(yrep_nb)

lignes = []
for k_, nom in enumerate(noms_T):
    lignes.append({"statistique": nom, "observée": round(T_obs[k_], 3), "répliques : moyenne":
↪ round(T_nb[:, k_].mean(), 3),
                  "répliques : 2,5 %": round(np.percentile(T_nb[:, k_], 2.5), 3),
                  "répliques : 97,5 %": round(np.percentile(T_nb[:, k_], 97.5), 3),
                  "p bayésien": round((T_nb[:, k_] >= T_obs[k_]).mean(), 3)})
print(pd.DataFrame(lignes).to_string(index=False))

fig, axes = plt.subplots(2, 3, figsize=(10.5, 5.2))
for ligne, (T_rep, titre, couleur) in enumerate([(T_pois, "Poisson", ORANGE), (T_nb, "binomiale
↪ négative", BLEU)]):
    for col in range(3):
        ax = axes[ligne, col]
        ax.hist(T_rep[:, col], bins=30, color=couleur, alpha=0.75)
        ax.vline(T_obs[col], color=ROUGE, lw=2.0)
        if ligne == 0:
            ax.set_title(noms_T[col])
            ax.set_yticks([])
            if col == 0:
                ax.set_ylabel(titre, color=couleur, fontsize=11)
            ax.grid(False)
    for ligne in range(2):
        axes[ligne, 0].text(0.97, 0.95, "trait rouge : \nobservé", transform=axes[ligne, 0].transAxes,
↪ ha="right", va="top", color=ROUGE, fontsize=9)
plt.tight_layout()
plt.savefig("figures/ch06-ppc-poisson-binneg.png", dpi=200, bbox_inches="tight")
plt.close()
```

| statistique        | observée | répliques : moyenne | répliques : 2,5 % | répliques : 97,5 % | p bayésien |
|--------------------|----------|---------------------|-------------------|--------------------|------------|
| variance / moyenne | 3.324    | 3.287               | 2.653             | 4.049              | 0.434      |
| part de zéros      | 0.109    | 0.117               | 0.083             | 0.155              | 0.667      |

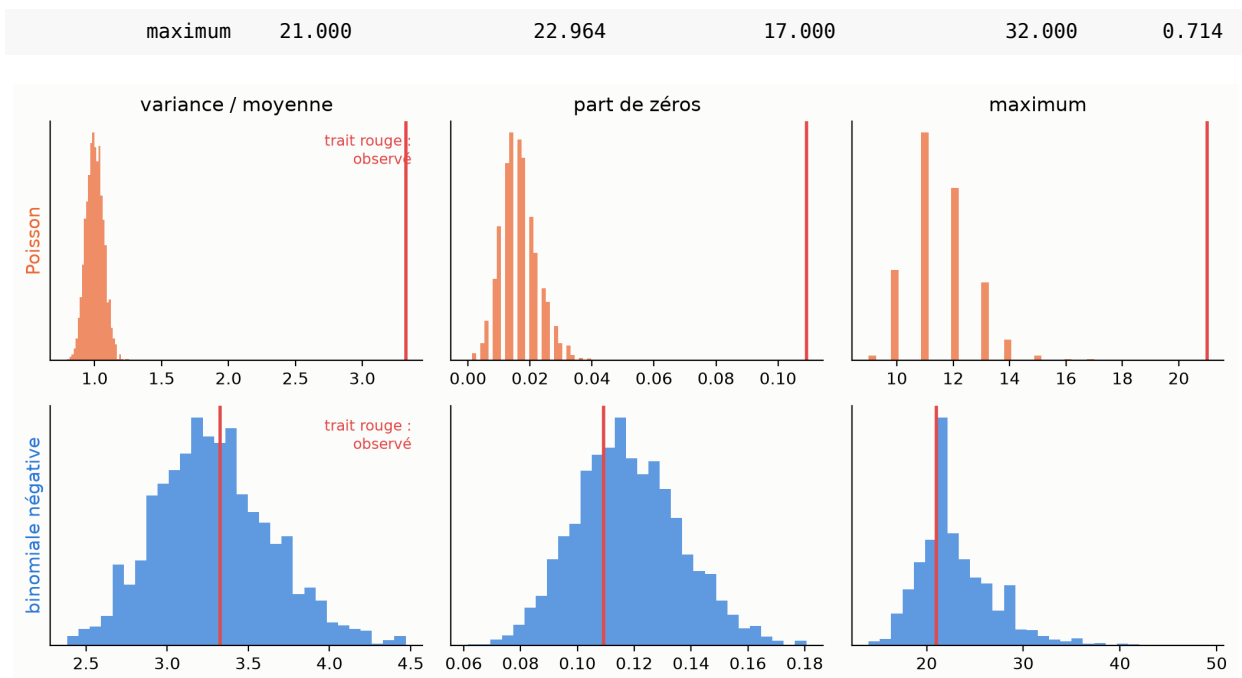


Figure 66: Vérification prédictive a posteriori de deux modèles pour le nombre de commandes annuelles des clients de la boutique. En haut, le modèle de Poisson : les histogrammes des trois statistiques répliquées (variance sur moyenne, part de zéros, maximum) sont loin de la valeur observée (trait rouge). En bas, la binomiale négative : la valeur observée est au milieu des répliques.

Cette fois, les trois statistiques observées tombent **au milieu** des répliques (les  $p_B$  sont loin de 0 et de 1). Le dessin résume la morale : en haut, les données (trait rouge) sont à l'extérieur du nuage des répliques de Poisson ; en bas, elles sont à l'intérieur de celui de la binomiale négative. **Le modèle n'est pas « prouvé vrai »** (aucun ne l'est), mais il a passé trois épreuves que l'autre a échouées.

**Choisir ses statistiques-test.** Une vérification prédictive ne détecte que les défauts que l'on a pensé à tester. Choisissez des statistiques qui **ne sont pas directement ajustées** par le modèle (la moyenne, que le modèle reproduit par construction, ne révélerait rien) et qui correspondent à vos **soucis** : queues, zéros, asymétrie, dépendance dans le temps, etc.

### 6.4.3 L'a priori est-il raisonnable ? La vérification prédictive a priori

Avant même de regarder les données, on peut se demander ce que l'a priori seul implique. On simule des paramètres dans l'a priori, puis des données dans le modèle : on obtient la loi **prédictive a priori**. Si elle attribue des probabilités énormes à des scénarios absurdes, l'a priori est mal choisi.

Pour la régression logistique du rachat, regardons ce que signifient des a priori  $\beta_j \sim \mathcal{N}(0, s^2)$  de plus en plus larges, en calculant, pour chaque tirage de  $\beta$ , la **proportion prédite de clients qui rachètent** :

```
rng = np.random.default_rng(644)
sigmoide = lambda u: 1 / (1 + np.exp(-u))
resultats, lignes = {}, []
for s in (0.5, 1.5, 2.5, 10.0):
    betas = rng.normal(0, s, size=(3000, Xm.shape[1]))
    p_ind = sigmoide(betas @ Xm.T)
# (3000 tirages, 2000 clients)
```

```

    taux = p_ind.mean(axis=1) # taux de rachat prédit pour
    ↪ chaque tirage
    resultats[s] = taux
    lignes.append({"écart-type de l'a priori s": s, "taux prédit : 5 %-95 %": f"{np.percentile(taux,
    ↪ 5):.2f} ; {np.percentile(taux, 95):.2f}",
        "tirages avec taux < 5 % ou > 95 %": f"{np.mean((taux < 0.05) | (taux > 0.95)):.1%}",
        "clients avec p < 1 % ou > 99 % (en moyenne)": f"{np.mean((p_ind < 0.01) | (p_ind >
    ↪ 0.99)):.1%}"})
print(pd.DataFrame(lignes).to_string(index=False))
print("\ntaux de rachat réel observé :", y.mean().round(3))

```

| écart-type de l'a priori s | taux prédit : 5 %-95 % | tirages avec taux < 5 % ou > 95 % | clients avec p < 1 % ou > 99 % (en moyenne) |
|----------------------------|------------------------|-----------------------------------|---------------------------------------------|
| ↪ 0.5                      | [0.28 ; 0.71]          | 0.0%                              |                                             |
| ↪ 1.5                      | [0.12 ; 0.90]          | 2.8%                              |                                             |
| ↪ 2.5                      | [0.05 ; 0.94]          | 9.2%                              |                                             |
| ↪ 10.0                     | [0.01 ; 0.99]          | 19.3%                             |                                             |

taux de rachat réel observé : 0.509

Lecture : avec  $s = 0,5$ , l'a priori est **trop serré** : il n'autorise pratiquement que des taux entre 30 % et 70 %, alors que nous ne savons pas à l'avance qu'ils sont dans cette fourchette. Avec  $s = 10$ , l'a priori est **absurde** : en moyenne, **79 % des clients** y ont une probabilité individuelle de rachat inférieure à 1 % ou supérieure à 99 %, et dans un tirage sur cinq le taux global est inférieur à 5 % ou supérieur à 95 %. Cela n'a rien d'une « ignorance » : c'est une opinion très tranchée, que nous n'avons aucune raison d'avoir. Le choix  $s = 2,5$  de la section 6.3.5 est déjà **généreux** (29 % des clients à probabilité individuelle extrême),  $s = 1,5$  plus prudent (9 %). Les deux autorisent une **large gamme de taux globaux** sans verser dans l'extrême ; nous vérifierons en 6.4.4 que le choix entre eux n'influence pas nos conclusions. Voici le dessin :

```

fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.2), sharey=True)
for ax, s, coul in zip(axes, (0.5, 2.5, 10.0), (ORANGE, BLEU, VIOLET)):
    ax.hist(resultats[s], bins=np.linspace(0, 1, 41), color=coul, alpha=0.8)
    ax.axvline(y.mean(), color="#0b0b0b", lw=1.0, ls="--")
    sf = f"{s:g}".replace(".", "{,}")
    ax.set_title(f"a priori  $\mathcal{N}(0, {sf}^2)$ ", color=coul)
    ax.set_xlabel("taux de rachat prédit (tirage de l'a priori)")
    ax.set_yticks([])
    ax.grid(False)
axes[0].text(0.03, 0.97, "pointillés : \ntaux observé (0,51)", transform=axes[0].transAxes, ha="left",
    ↪ va="top", fontsize=9)
plt.tight_layout()
plt.savefig("figures/ch06-prior-predictive.png", dpi=200, bbox_inches="tight")
plt.close()

```

**Règle pratique.** On ne peut pas vérifier un a priori par les données (ce serait les utiliser deux fois), mais on peut vérifier ses **conséquences** sur des quantités que l'on comprend (un taux, une moyenne, une durée). Si l'a priori produit des scénarios que vous jugeriez ridicules, resserrez-le.

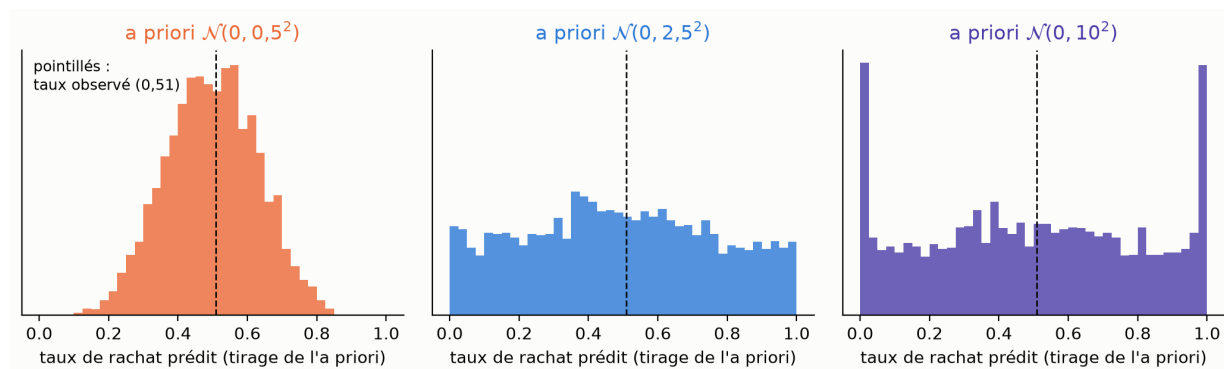


Figure 67: Taux de rachat global prédit par l'a priori seul, pour trois largeurs d'a priori sur les coefficients de la régression logistique. Trop serré (à gauche) : tous les modèles prédisent à peu près le même taux. Raisonnable (au milieu) : une gamme étendue de taux plausibles. Trop large (à droite) : la masse s'accumule près de 0 et de 1, des scénarios absurdes.

#### 6.4.4 Comparer des modèles : facteur de Bayes et WAIC

**Le facteur de Bayes.** Comparons deux modèles (ou deux hypothèses)  $H_0$  et  $H_1$  par la formule de Bayes appliquée aux modèles :  $\frac{P(H_1 | y)}{P(H_0 | y)} = \frac{p(y | H_1)}{p(y | H_0)} \times \frac{P(H_1)}{P(H_0)}$ , où  $p(y | H) = \int p(y | \theta, H) p(\theta | H) d\theta$  est la **vraisemblance marginale** (la moyenne de la vraisemblance sur l'a priori). Le  $BF_{10}$  mesure de combien les données déplacent la cote en faveur de  $H_1$ .

*Exemple à la main.* Une pièce (ou une offre) donne  $y$  succès sur  $n$  essais.  $H_0 : \theta = 0,5$  exactement.  $H_1 : \theta$  est inconnu avec un a priori uniforme sur  $[0, 1]$ . Alors  $p(y | H_0) = \binom{n}{y} 0,5^n$  et

$$p(y | H_1) = \binom{n}{y} \int_0^1 \theta^y (1 - \theta)^{n-y} d\theta = \binom{n}{y} B(y + 1, n - y + 1) = \frac{1}{n + 1}.$$

(Résultat remarquable : avec un a priori uniforme, **chaque** valeur de  $y$  entre 0 et  $n$  est également probable a priori.) Pour  $n = 10$ ,  $y = 7$  :  $p(y | H_0) = 120/1024 = 0,1172$  et  $p(y | H_1) = 1/11 = 0,0909$ . Donc  $BF_{10} = 0,0909/0,1172 = 0,776$  : les données sont **un peu plus probables sous  $H_0$**  que sous  $H_1$ . Sept succès sur dix ne suffisent pas à abandonner la pièce équilibrée ! Le p-value fréquentiste bilatéral est 0,34 : le même message. Le code reproduit ce calcul et l'applique à nos données d'offre.

```
from scipy.special import gammaln

def log_bf10(y_, n_):
    """log du facteur de Bayes de H1 (theta ~ U(0,1)) contre H0 (theta = 0,5)."""
    log_m1 = -np.log(n_ + 1)
    log_m0 = gammaln(n_ + 1) - gammaln(y_ + 1) - gammaln(n_ - y_ + 1) + n_ * np.log(0.5)
    return log_m1 - log_m0

cas = [("7 rachats sur 10", 7, 10), ("offre : 578 sur 1 015", 578, 1015), ("50 400 sur 100 000",
↪ 50_400, 100_000)]
lignes = []
for nom, yy_, nn in cas:
    bf = np.exp(log_bf10(yy_, nn))
    lignes.append({"cas": nom, "fréquence": round(yy_ / nn, 4), "p-valeur (bilatérale)":
↪ f"{stats.binomtest(yy_, nn, 0.5).pvalue:.2g}",
    "BF10": f"{bf:.3g}", "P(H1 | données) si a priori 50/50": f"{bf / (1 + bf):.3f}"})
print(pd.DataFrame(lignes).to_string(index=False))
```

|                       | cas | fréquence | p-valeur (bilatérale) | BF10   | P(H1   données) si a priori 50/50 |
|-----------------------|-----|-----------|-----------------------|--------|-----------------------------------|
| 7 rachats sur 10      |     | 0.7000    | 0.34                  | 0.776  | 0.437                             |
| offre : 578 sur 1 015 |     | 0.5695    | 1.1e-05               | 720    | 0.999                             |
| 50 400 sur 100 000    |     | 0.5040    | 0.012                 | 0.0972 | 0.089                             |

Trois cas, trois leçons. (1) *Peu de données* : le facteur de Bayes de 0,78 et la p-valeur de 0,34 s'accordent : on ne peut pas conclure. (2) *Beaucoup de données et un grand écart* (578 sur 1 015) : les deux approches rejettent fermement  $H_0$ . (3) Le dernier cas est le **paradoxe de Lindley** : avec 100 000 observations et une fréquence de 50,4 %, la p-valeur fréquentiste est petite (0,012 : « significatif au seuil de 5 % », voire de 2 %), alors que le facteur de Bayes (0,097) **favorise  $H_0$  d'un facteur 10** : un écart de 0,4 point est ridicule comparé à ce que prédit l'hypothèse  $H_1$  (qui étale sa probabilité sur tout l'intervalle  $[0, 1]$ ). La p-valeur se laisse impressionner par la taille de l'échantillon ; le facteur de Bayes pénalise l'hypothèse qui « dilue » sa probabilité inutilement. C'est aussi le **point faible** du facteur de Bayes : il **dépend fortement de l'a priori** (ici, de l'étalement uniforme sous  $H_1$ ). Un a priori plus concentré autour de 0,5 donnerait un autre résultat.

**Prudence.** Le facteur de Bayes est très sensible au choix de l'a priori sur les paramètres. Il est difficile à calculer pour des modèles complexes (une intégrale en grande dimension). Beaucoup de praticiens bayésiens lui préfèrent des outils prédictifs comme le WAIC ci-dessous.

**Le WAIC : juger un modèle par ses prédictions.** L'idée : un bon modèle est celui qui **prédit bien des données nouvelles**. Le critère **WAIC** (*widely applicable information criterion*) estime la qualité prédictive hors échantillon à partir des seuls tirages a posteriori :

$$\text{lppd} = \sum_{i=1}^n \log\left(\frac{1}{S} \sum_{s=1}^S p(y_i | \theta^{(s)})\right), \quad p_{\text{WAIC}} = \sum_{i=1}^n \text{Var}_s[\log p(y_i | \theta^{(s)})], \quad \text{WAIC} = -2(\text{lppd} - p_{\text{WAIC}}).$$

Le premier terme mesure l'ajustement aux données (plus il est grand, mieux le modèle colle) ;  $p_{\text{WAIC}}$  est le **nombre effectif de paramètres**, une pénalité qui punit la complexité (c'est l'analogue bayésien de la pénalité de l'AIC, section 1.4). **Plus le WAIC est petit, meilleur est le modèle.** Comparons quatre modèles du rachat, du plus simple au plus complet, en les estimant tous par notre MCMC :

```
def fit_mcmc_logit(colonnes, graine, n_chaines=4, n_iter=3000, chauffe=500, s=2.5):
    Xs = X[colonnes].to_numpy()
    emv_ = sm.Logit(y, Xs).fit(dis=0)
    dim_ = Xs.shape[1]
    cov_ = (2.38**2 / dim_) * np.atleast_2d(emv_.cov_params())
    def lp(beta):
        eta = Xs @ beta
        return np.sum(y * eta - np.logaddexp(0, eta)) - 0.5 * np.sum(beta**2) / s**2
    rng_ = np.random.default_rng(graine)
    tir = []
    for _ in range(n_chaines):
        depart = emv_.params + rng_.normal(0, 0.5, dim_)
        ch, _ = metropolis_multi(lp, depart, n_iter, cov_, rng_)
        tir.append(ch[chauffe:])
    return np.concatenate(tir)[:5], Xs, emv_ # on éclairecit : 1 tirage sur 5

def waic(tir, Xs):
    eta = tir @ Xs.T # (S tirages, n clients)
    ll = y * eta - np.logaddexp(0, eta) # log p(y_i | theta_s)
    S = ll.shape[0]
    lppd_i = np.logaddexp.reduce(ll, axis=0) - np.log(S)
    pw_i = ll.var(axis=0, ddof=1)
    elpd_i = lppd_i - pw_i
    return -2 * elpd_i.sum(), pw_i.sum(), -2 * elpd_i
```



```

modeles = {"M0 : constante": ["const"], "M1 : + offre": ["const", "offre"],
           "M2 : + canal": ["const", "offre", "Instagram", "Site"], "M3 : + âge": ["const", "offre",
           ↪ "Instagram", "Site", "age_c"]}
res, pointwise = [], {}
for i, (nom, cols) in enumerate(modeles.items()):
    tir, Xs, emv_ = fit_mcmc_logit(cols, graine=650 + i)
    w, pw, pt = waic(tir, Xs)
    pointwise[nom] = pt
    res.append({"modèle": nom, "paramètres": len(cols), "p_WAIC": round(pw, 2), "WAIC": round(w, 1),
    ↪ "AIC (EMV)": round(emv_.aic, 1)})
tab = pd.DataFrame(res)
tab["WAIC - min"] = (tab["WAIC"] - tab["WAIC"].min()).round(1)
print(tab.to_string(index=False))

```

```

noms_m = list(modeles)
for a, b_ in (("M0 : constante", "M1 : + offre"), ("M1 : + offre", "M2 : + canal"), ("M2 : + canal",
↪ "M3 : + âge")):
    diff = pointwise[a] - pointwise[b_]
    print(f"{a[:2]} - {b_[:2]} : différence de WAIC = {diff.sum():6.1f}, erreur-type =
    ↪ {np.sqrt(len(diff) * diff.var(ddof=1)):.1f}")

```

|                                | modèle | paramètres | p_WAIC | WAIC   | AIC (EMV)     | WAIC - min |
|--------------------------------|--------|------------|--------|--------|---------------|------------|
| M0 : constante                 |        | 1          | 1.00   | 2773.9 | 2773.9        | 53.1       |
| M1 : + offre                   |        | 2          | 1.89   | 2745.9 | 2746.1        | 25.1       |
| M2 : + canal                   |        | 4          | 4.26   | 2732.1 | 2731.6        | 11.3       |
| M3 : + âge                     |        | 5          | 4.95   | 2720.8 | 2720.8        | 0.0        |
| M0 - M1 : différence de WAIC = |        |            |        | 27.9,  | erreur-type = | 10.9       |
| M1 - M2 : différence de WAIC = |        |            |        | 13.8,  | erreur-type = | 8.5        |
| M2 - M3 : différence de WAIC = |        |            |        | 11.4,  | erreur-type = | 7.0        |

Le WAIC **décroît** à mesure que l'on ajoute l'offre, puis le canal, puis l'âge. Le  $p_{\text{WAIC}}$  est très proche du nombre de paramètres (comme il se doit pour un modèle régulier avec beaucoup de données), et le WAIC est quasiment égal à l'AIC calculé par le maximum de vraisemblance : avec un a priori diffus et beaucoup de données, les deux critères racontent la même histoire. Les dernières lignes comparent les gains avec leur **incertitude** (un WAIC isolé ne signifie rien : seules les **différences entre modèles** comptent, et leur erreur-type). Une différence est jugée « claire » si elle dépasse environ deux erreurs-types. Ici, l'offre apporte un gain net ( $27,9 \pm 10,9$ , soit 2,6 erreurs-types) ; le canal ( $13,8 \pm 8,5$ ) et l'âge ( $11,4 \pm 7,0$ ) apportent des gains d'environ 1,6 erreur-type : **plausibles mais non tranchés** par le WAIC seul. Cela ne contredit pas la section 6.3.5, où les intervalles de crédibilité de l'âge et du canal Instagram excluent 0 : le WAIC répond à la question « *ce coefficient améliore-t-il la prédiction de clients nouveaux ?* », pas à « *ce coefficient est-il différent de zéro ?* ».

**Sensibilité à l'a priori (6.1.6, enfin faite).** Terminons par l'analyse de sensibilité promise : le coefficient de l'offre change-t-il si l'on modifie l'écart-type  $s$  de l'a priori ?

```

lignes = []
for i, s_ in enumerate((0.5, 1.5, 2.5, 10.0)):
    tir, Xs, emv_ = fit_mcmc_logit(["const", "offre", "Instagram", "Site", "age_c"], graine=660 + i,
    ↪ s=s_)
    bo = tir[:, 1]
    lignes.append({"s de l'a priori": s_, "moyenne a posteriori de beta_offre": round(bo.mean(), 3),
    ↪ "IC95 bas": round(np.percentile(bo, 2.5), 3), "IC95 haut": round(np.percentile(bo,
    ↪ 97.5), 3)})
print(pd.DataFrame(lignes).to_string(index=False))
print("rappel : EMV de beta_offre =", round(emv_.params[1], 3))

```

| s de l'a priori | moyenne a posteriori de beta_offre | IC95 bas | IC95 haut |
|-----------------|------------------------------------|----------|-----------|
| 0.5             | 0.476                              | 0.291    | 0.653     |
| 1.5             | 0.488                              | 0.296    | 0.672     |
| 2.5             | 0.492                              | 0.314    | 0.665     |
| 10.0            | 0.494                              | 0.321    | 0.658     |

rappel : EMV de beta\_offre = 0.493

Les résultats sont presque identiques pour les quatre a priori (moyenne a posteriori de 0,476 à 0,494, contre 0,493 pour le maximum de vraisemblance), y compris pour l'a priori très serré ( $s = 0,5$ , qui ramène un peu le coefficient vers 0) et l'a priori très large ( $s = 10$ ) : **nos conclusions ne dépendent pas de l'a priori**, parce que 2 000 observations suffisent à dominer. C'est la situation confortable décrite en 6.1.6, mais elle ne se serait pas produite avec 20 clients.

### 6.4.5 Le flux de travail bayésien, en une page

Voici la liste de contrôle qui résume cette section. Elle est le prolongement bayésien de la démarche du volume I (« décrire, dessiner, contrôler »).

1. **Écrire le modèle** : vraisemblance et a priori. Dire pourquoi.
2. **Vérifier l'a priori** (6.4.3) : la loi prédictive a priori produit-elle des scénarios plausibles ?
3. **Ajuster** avec plusieurs chaînes dispersées (6.3), chauffe écartée.
4. **Diagnostiquer l'algorithme** (6.4.1) : traces en « chenille velue »,  $\hat{R} < 1,01$ , ESS d'au moins quelques centaines pour chaque paramètre d'intérêt. En cas d'échec : changer le pas, reparamétriser, allonger.
5. **Vérifier le modèle** (6.4.2) : jeux répliqués comparables aux données sur des statistiques qui vous inquiètent. En cas d'échec : enrichir le modèle (comme la binomiale négative).
6. **Analyser la sensibilité** (6.1.6) : refaire avec d'autres a priori raisonnables ; la conclusion change-t-elle ?
7. **Comparer** éventuellement des modèles (WAIC, facteur de Bayes en connaissance de cause).
8. **Rapporter** : le modèle, les a priori, les diagnostics, les intervalles de crédibilité, les limites.

#### À retenir (6.4).

- Deux risques indépendants : **l'algorithme** n'a pas convergé, ou **le modèle** est faux. Un diagnostic qui passe ne prouve rien ; un diagnostic qui échoue signale un problème.
- **Convergence** : plusieurs chaînes dispersées, chauffe écartée, traces en « chenille velue »,  $\hat{R} < 1,01$ , ESS suffisante. Une chaîne unique piégée dans un mode ne signale rien : c'est la comparaison entre chaînes qui détecte le problème.
- **Vérification prédictive a posteriori** : simuler des données répliquées avec le modèle ajusté et les comparer aux données sur des statistiques ciblées. Le modèle de Poisson échoue (surdispersion), la binomiale négative passe.
- **Vérification prédictive a priori** : examiner ce que l'a priori implique. Un a priori « vague » mal choisi (écart-type 10 sur un logit) est tout sauf neutre.
- **Facteur de Bayes** (rapport des vraisemblances marginales) : mesure intuitive mais très sensible à l'a priori (paradoxe de Lindley). **WAIC** : qualité prédictive estimée à partir des tirages ; comparer des modèles par leurs **différences** et leurs erreurs-types.
- Un a posteriori n'est valide que **conditionnellement au modèle** : la vérification du modèle est une étape à part entière, jamais facultative.

## 6.5 ☉ Pour aller plus loin : la théorie des valeurs extrêmes

**Section optionnelle.** Elle ne suppose que le chapitre 2 du volume I (lois, fonction de répartition, loi des grands nombres) et le maximum de vraisemblance du chapitre 3. Elle peut se lire indépendamment des sections 6.1 à 6.4. Elle prolonge cependant leur esprit : **simuler** pour comprendre, et **vérifier** par rapport à une vérité connue.

**Intuition.** La plupart de la statistique s'intéresse au **centre** d'une distribution : la moyenne, la médiane, l'écart-type. Mais certaines décisions se jouent dans les **queues** : « *quel est le plus long retard de livraison que je risque une fois par an ? une fois tous les dix ans ?* », « *quelle est la plus grosse commande à prévoir ?* », « *quel niveau de crue une digue doit-elle supporter ?* ». Ce sont des événements **rare**s, que l'on n'a souvent jamais observés. Or **la queue d'une loi n'est pas bien décrite par son centre** : ajuster une loi normale à des données et en extrapoler la queue est l'une des erreurs les plus coûteuses de l'histoire de la gestion du risque. La théorie des valeurs extrêmes fournit des modèles **faits pour la queue**.

### 6.5.1 Le problème : la queue d'une loi normale est trompeuse

Un exemple simple pour fixer les idées. Yasmine suit la **durée de livraison** de chaque colis (en jours). Elle veut savoir quel retard est « tellement long qu'on ne le voit qu'une fois en dix ans ». Elle n'a que dix ans de données : l'événement qu'elle cherche est, au mieux, **à la limite de ce qu'elle a observé**, et souvent au-delà. Il faut donc **extrapoler** hors des données, et la forme de la queue décide du résultat.

**Trois comportements de queue.**

| Type de queue         | La probabilité de dépasser $x$ décroît comme... | Exemples                                                 | Indice $\xi$        |
|-----------------------|-------------------------------------------------|----------------------------------------------------------|---------------------|
| <b>Légère, bornée</b> | s'annule au-delà d'un maximum fini              | temps de trajet maximal physique                         | $\xi < 0$ (Weibull) |
| <b>Exponentielle</b>  | $e^{-x}$ : très vite                            | lois normale, exponentielle, gamma                       | $\xi = 0$ (Gumbel)  |
| <b>Lourde</b>         | $x^{-1/\xi}$ : lentement, comme une puissance   | retards logistiques, pertes d'assurance, crues, montants | $\xi > 0$ (Fréchet) |

Pour une queue lourde, des valeurs **énormes** surviennent bien plus souvent que ce que la loi normale ne laisse imaginer. L'indice  $\xi$  (« indice de queue » ou *shape*) est le paramètre-clé de la théorie.

### 6.5.2 Le théorème fondamental : la loi du maximum

Soit  $X_1, \dots, X_n$  indépendantes de même loi, et  $M_n = \max(X_1, \dots, X_n)$ . Combien vaut sa loi ? Facile :  $P(M_n \leq x) = F(x)^n$ , où  $F$  est la fonction de répartition de chaque  $X_i$ . Mais cela dépend de  $F$  et tend vers 0 ou 1. On cherche une loi **limite** après normalisation.

**Exemple à la main (lois exponentielles).** Soient  $X_i \sim \text{Exp}(1)$  de fonction de répartition  $F(x) = 1 - e^{-x}$ . Normalisons le maximum en le décalant de  $\ln n$  : pour tout réel  $x$ ,

$$P(M_n - \ln n \leq x) = F(x + \ln n)^n = \left(1 - \frac{e^{-x}}{n}\right)^n \xrightarrow{n \rightarrow \infty} \exp(-e^{-x}),$$

car  $(1 - a/n)^n \rightarrow e^{-a}$ . La loi limite, de fonction de répartition  $\exp(-e^{-x})$ , s'appelle la loi de **Gumbel**. On vient de démontrer que le maximum de  $n$  exponentielles, décalé de  $\ln n$ , suit approximativement une loi de Gumbel.

**Théorème de Fisher-Tippett-Gnedenko (admis).** Si, après un changement d'échelle  $M_n \mapsto (M_n - b_n)/a_n$ , le maximum converge en loi vers une loi non dégénérée, celle-ci est nécessairement de la forme **GEV** (*Generalized Extreme Value*) :

$$G(z) = \exp \left\{ - \left[ 1 + \zeta \frac{z - \mu}{\sigma} \right]^{-1/\zeta} \right\}, \quad 1 + \zeta \frac{z - \mu}{\sigma} > 0,$$

avec trois paramètres : **position**  $\mu$ , **échelle**  $\sigma > 0$ , **forme**  $\zeta$  (le cas  $\zeta = 0$  se lit comme la limite  $\exp\{-e^{-(z-\mu)/\sigma}\}$ , la loi de Gumbel).

Autrement dit, comme le théorème central limite pour les moyennes (volume I, section 2.4), il existe un **résultat universel pour les maxima** : quelle que soit la loi d'origine (ou presque), le maximum d'un grand nombre de valeurs suit une loi GEV. Vérifions-le par simulation sur l'exemple exponentiel :

```
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(670)
n, reps = 1000, 20_000
maxima = rng.exponential(1.0, size=(reps, n)).max(axis=1) - np.log(n)      # maximum de 1 000
↳ exponentielles, décalé de ln n
print(f"{reps} maxima de {n} lois exponentielles, décalés de ln({n}) = {np.log(n):.3f}")
for x in (-1, 0, 1, 2, 3):
    print(f"P(M - ln n <= {x:2d}) : simulé {np.mean(maxima <= x):.4f} | Gumbel exp(-exp(-x)) =
    ↳ {np.exp(-np.exp(-x)):.4f}")
print("test de Kolmogorov-Smirnov contre la loi de Gumbel :", f"statistique {stats.kstest(maxima,
↳ stats.gumbel_r.cdf).statistic:.4f}")
```

```
20000 maxima de 1000 lois exponentielles, décalés de ln(1000) = 6.908
P(M - ln n <= -1) : simulé 0.0650 | Gumbel exp(-exp(-x)) = 0.0660
P(M - ln n <= 0) : simulé 0.3685 | Gumbel exp(-exp(-x)) = 0.3679
P(M - ln n <= 1) : simulé 0.6922 | Gumbel exp(-exp(-x)) = 0.6922
P(M - ln n <= 2) : simulé 0.8711 | Gumbel exp(-exp(-x)) = 0.8734
P(M - ln n <= 3) : simulé 0.9496 | Gumbel exp(-exp(-x)) = 0.9514
test de Kolmogorov-Smirnov contre la loi de Gumbel : statistique 0.0044
```

La loi de Gumbel colle presque parfaitement. Pour un **autre type de queue**, le résultat est différent : le maximum de variables à queue lourde (loi de Pareto) converge vers une loi de Fréchet, de  $\zeta > 0$ . Le type de la loi limite est dicté par la **queue** de la loi d'origine : c'est précisément pourquoi on peut modéliser un maximum sans connaître la loi des observations.

### 6.5.3 Les données : des retards de livraison simulés, à queue lourde

Pour que nous puissions **comparer à la vérité**, nous simulons un jeu de données de colis (il s'agit bien d'une **simulation**, de graine 671 ; ce ne sont pas des données réelles de Dar Jasmin). Dix ans (2016-2025) de livraisons : environ 3 colis par jour en moyenne (loi de Poisson), chacun avec une durée de livraison en jours qui suit une loi de **Pareto généralisée** décalée de 1 jour (la durée minimale) :

$$P(\text{durée} > x) = \left( 1 + \zeta \frac{x - 1}{\sigma} \right)^{-1/\zeta}, \quad x \geq 1,$$

avec  $\zeta = 0,25$  (queue lourde modérée : la variance est finie, mais pas le moment d'ordre 4) et  $\sigma = 1,5$  jour. Nous garderons ces valeurs **cachées** de nos calculs : nous les utiliserons à la fin pour juger les estimations.

```

rng = np.random.default_rng(671)
XI0, SIGMA0, COLIS_PAR_JOUR, JOURS = 0.25, 1.5, 3.0, 3652
n_j = rng.poisson(COLIS_PAR_JOUR, JOURS)
n_tot = n_j.sum()
u = rng.random(n_tot)
duree = 1 + SIGMA0 / XI0 * (u ** (-XI0) - 1)          # inversion de la fonction de survie
↳ (6.2.4)
date = pd.Timestamp("2016-01-01") + pd.to_timedelta(np.repeat(np.arange(JOURS), n_j), unit="D")
colis = pd.DataFrame({"date": date, "duree": duree})

print(f"{n_tot} colis sur {JOURS} jours ({n_tot / JOURS:.2f} par jour en moyenne)")
print(colis["duree"].describe().round(2).to_string())
print(f"proportion de colis livrés en plus de 5 jours : {(duree > 5).mean():.3%} en plus de 10 jours
↳ : {(duree > 10).mean():.3%} en plus de 20 jours : {(duree > 20).mean():.3%}")

10981 colis sur 3652 jours (3.01 par jour en moyenne)
count      10981.00
mean        3.04
std         2.89
min         1.00
25%         1.46
50%         2.14
75%         3.53
max         56.32
proportion de colis livrés en plus de 5 jours : 13.114% en plus de 10 jours : 2.723% en plus de 20
↳ jours : 0.373%

```

La médiane est d'environ 2 jours, mais la queue est longue : 13 % des colis dépassent 5 jours, 2,7 % dépassent 10 jours, et 0,37 % (environ un colis sur 270) dépassent 20 jours. Le plus long retard de la décennie est de 56 jours. Voyons comment la forme de cette queue se lit dans les données, puis comment extrapoler.

### 6.5.4 Première approche : les maxima par blocs

On découpe les dix ans en **blocs** (ici : les mois) et on garde le **plus long retard de chaque mois**, ce qui donne 120 maxima. D'après le théorème de la section 6.5.2, ces maxima suivent approximativement une loi GEV, dont on estime les paramètres par **maximum de vraisemblance** (volume I, section 3.2). (Attention : `scipy` paramètre la GEV par  $c = -\xi$ , signe opposé à la convention de ce livre.)

```

maxima_m = colis.groupby(colis["date"].dt.to_period("M"))["duree"].max().to_numpy()
print(f"{len(maxima_m)} maxima mensuels : min {maxima_m.min():.2f}, médiane
↳ {np.median(maxima_m):.2f}, max {maxima_m.max():.2f} jours")

c_hat, mu_hat, sig_hat = stats.genextreme.fit(maxima_m)
xi_hat = -c_hat
print(f"GEV ajustée par maximum de vraisemblance : mu = {mu_hat:.3f}, sigma = {sig_hat:.3f}, xi =
↳ {xi_hat:.3f}")

# Incertitude : bootstrap (on rééchantillonne les 120 maxima, on réajuste)
rng = np.random.default_rng(672)
boot = np.array([stats.genextreme.fit(rng.choice(maxima_m, len(maxima_m))) for _ in range(300)])
xi_boot = -boot[:, 0]
print(f"xi : IC95 bootstrap [{np.percentile(xi_boot, 2.5):.3f} ; {np.percentile(xi_boot, 97.5):.3f}]
↳ (écart-type {xi_boot.std():.3f})")

120 maxima mensuels : min 7.68, médiane 15.54, max 56.32 jours
GEV ajustée par maximum de vraisemblance : mu = 13.741, sigma = 4.772, xi = 0.324
xi : IC95 bootstrap [0.213 ; 0.459] (écart-type 0.066)

```

L'estimation de  $\xi$  est positive, ce qui indique une queue lourde, mais son **intervalle d'incertitude**

**est large** : avec seulement 120 maxima, la forme de la queue est difficile à préciser. C'est la caractéristique majeure de la théorie des extrêmes : **on dispose de très peu de données par construction** (les extrêmes sont rares), et donc l'incertitude est grande.

**Les niveaux de retour.** La quantité que Yasmine veut vraiment est le **niveau de retour**  $z_T$  : la valeur dépassée en moyenne **une fois toutes les  $T$  périodes** (ici,  $T$  mois). Autrement dit, la valeur telle que  $P(\max \text{ mensuel} > z_T) = 1/T$ , soit  $G(z_T) = 1 - 1/T$ . En résolvant l'équation avec la forme de la GEV, on trouve

$$z_T = \mu - \frac{\sigma}{\xi} \left[ 1 - (-\ln(1 - 1/T))^{-\xi} \right].$$

(Démonstration :  $G(z) = 1 - 1/T \iff [1 + \xi \frac{z - \mu}{\sigma}]^{-1/\xi} = -\ln(1 - 1/T)$ , car  $\exp(-y) = 1 - 1/T \iff y = -\ln(1 - 1/T)$  ; on élève à la puissance  $-\xi$  et on isole  $z$ .) Un niveau de retour à **10 ans**, c'est  $T = 120$  mois ; à **100 ans**,  $T = 1\,200$  mois.

Pour montrer le danger de la loi normale, nous comparons trois estimations : (i) la GEV ajustée ; (ii) une loi **normale** ajustée aux mêmes 120 maxima ; (iii) la **vérité** (calculable car nous connaissons le générateur : le maximum mensuel d'un nombre de colis de moyenne  $\lambda_m = 3 \times 30,4375$  suit exactement une GEV de paramètres  $\xi = \xi_0$ ,  $\sigma = \sigma_0 \lambda_m^{\xi_0}$ ,  $\mu = 1 + \sigma_0(\lambda_m^{\xi_0} - 1)/\xi_0$ ).

```
lam_m = COLIS_PAR_JOUR * 30.4375 # colis par mois, en moyenne
sig_vrai = SIGMA0 * lam_m ** XI0
mu_vrai = 1 + SIGMA0 * (lam_m ** XI0 - 1) / XI0
print(f"vérité : GEV(mu = {mu_vrai:.3f}, sigma = {sig_vrai:.3f}, xi = {XI0})")

def niveau_gev(T, mu, sigma, xi):
    return mu - sigma / xi * (1 - (-np.log(1 - 1 / T)) ** (-xi))

m_norm, s_norm = maxima_m.mean(), maxima_m.std(ddof=1)
lignes = []
for T, nom in ((12, "1 an"), (120, "10 ans"), (1200, "100 ans")):
    ni_boot = np.array([niveau_gev(T, b[1], b[2], -b[0]) for b in boot])
    lignes.append({"retour": nom, "vérité": round(niveau_gev(T, mu_vrai, sig_vrai, XI0), 1),
                  "GEV ajustée": round(niveau_gev(T, mu_hat, sig_hat, xi_hat), 1),
                  "IC95 bootstrap GEV": f"[{np.percentile(ni_boot, 2.5):.0f} ; {np.percentile(ni_boot, 97.5):.0f}]",
                  "loi normale ajustée": round(stats.norm.ppf(1 - 1 / T, m_norm, s_norm), 1)})
print(pd.DataFrame(lignes).to_string(index=False))
```

```
vérité : GEV(mu = 13.547, sigma = 4.637, xi = 0.25)
retour  vérité  GEV ajustée  IC95 bootstrap GEV  loi normale ajustée
1 an    29.1    31.5         [27 ; 37]              31.5
10 ans  56.3    68.5         [51 ; 101]             41.1
100 ans 104.2   145.8        [89 ; 289]             48.2
```

Le tableau livre le message central. À 1 an, les deux ajustements donnent la même valeur (31,5 jours, pour une vérité de 29,1) : on est **dans** le domaine des données. Mais à 10 ans puis à 100 ans, **la loi normale s'effondre** : elle prédit 41 puis 48 jours, quand la vérité est de 56 puis 104 jours. Elle se trompe de plus de moitié à 100 ans, et dans le sens **rassurant**. La GEV, elle, se trompe plutôt par excès : 68 jours à 10 ans (vérité 56) et 146 à 100 ans (vérité 104), parce que son  $\hat{\xi} = 0,32$  est un peu supérieur au vrai 0,25 (une petite erreur sur  $\xi$  est amplifiée par l'extrapolation). Mais **son intervalle de confiance contient la vérité** ([51 ; 101] à 10 ans, [89 ; 289] à 100 ans), et il est **très large** à 100 ans : c'est honnête, car on extrapole dix fois au-delà de la durée des données. Retenons : la loi normale ne produit pas seulement des erreurs, elle produit des erreurs **rassurantes** et **sans avertissement** (elle n'a pas d'intervalle qui s'élargisse).

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```



```

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
                    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
                    "legend.frameon": False, "font.size": 10})

n_m = len(maxima_m)
tri = np.sort(maxima_m)
T_emp = (n_m + 1) / (n_m + 1 - np.arange(1, n_m + 1))           # période de retour empirique
↪ (positions de Weibull)
Tg = np.logspace(np.log10(1.2), np.log10(3000), 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.5, 4.2), gridspec_kw={"width_ratios": [1.25, 1]})
ax1.semilogx(Tg, niveau_gev(Tg, mu_vrai, sig_vrai, XI0), color="#0b0b0b", lw=1.4, ls="--")
ax1.semilogx(Tg, niveau_gev(Tg, mu_hat, sig_hat, xi_hat), color=BLEU, lw=2.0)
ax1.semilogx(Tg, stats.norm.ppf(1 - 1 / Tg, m_norm, s_norm), color=ORANGE, lw=2.0)
ax1.semilogx(T_emp, tri, "o", color="#52514e", ms=3.5)
ax1.axvline(120, color="#898781", lw=0.8, ls=":")
ax1.text(125, 5, "10 ans", color="#52514e", fontsize=9)
ax1.text(1.3, 66, "GEV ajustée", color=BLEU, fontsize=10)
ax1.text(1.3, 60, "loi normale ajustée", color=ORANGE, fontsize=10)
ax1.text(1.3, 54, "vérité (tirets)", color="#0b0b0b", fontsize=10)
ax1.text(1.3, 48, "points : maxima observés", color="#52514e", fontsize=9)
ax1.set_xlabel("période de retour (en mois, échelle logarithmique)")
ax1.set_ylabel("retard maximal (jours)")
ax1.set_title("Niveaux de retour")
ax1.set_ylim(0, 70)

# Panneau de droite : fonction d'excès moyen (choix du seuil, voir 6.5.5)
x_all = colis["duree"].to_numpy()
seuils = np.quantile(x_all, np.linspace(0.70, 0.995, 60))
exces_moy = np.array([x_all[x_all > s_].mean() - s_ for s_ in seuils])
ax2.plot(seuils, exces_moy, color=BLEU, lw=1.8)
ax2.set_xlabel("seuil $u$ (jours)")
ax2.set_ylabel("excès moyen au-dessus de $u$ (jours)")
ax2.set_title("Excès moyen : quasi linéaire si la queue est GPD")
plt.tight_layout()
plt.savefig("figures/ch06-extremes-retour.png", dpi=200, bbox_inches="tight")
plt.close()

```

### 6.5.5 Seconde approche : les excès au-dessus d'un seuil (POT)

Garder un seul maximum par mois, c'est **jeter** beaucoup d'information : le deuxième plus long retard d'un mois est peut-être supérieur au maximum d'un autre mois. L'approche **POT** (*peaks over threshold*) exploite **toutes les valeurs qui dépassent un seuil élevé  $u$** .

**Théorème de Pickands-Balkema-de Haan (admis).** Pour un seuil  $u$  assez élevé, la loi des excès  $X - u$  sachant  $X > u$  est approximativement une loi de **Pareto généralisée** (GPD) :

$$P(X - u > y \mid X > u) \approx \left(1 + \xi \frac{y}{\sigma_u}\right)^{-1/\xi}, \quad y > 0,$$

avec le même indice  $\xi$  que la GEV des maxima (le paramètre d'échelle  $\sigma_u$  dépend du seuil).

Un seuil trop bas rend l'approximation GPD fautive (biais) ; un seuil trop haut laisse trop peu de points (variance). C'est le compromis habituel. Deux outils guident le choix :

- le **graphique de l'excès moyen** (*mean residual life plot*, panneau de droite ci-dessus) : pour une GPD, l'excès moyen au-dessus de  $u$  vaut  $\frac{\sigma_u}{1 - \xi}$  et croît **linéairement** avec  $u$ . On choisit



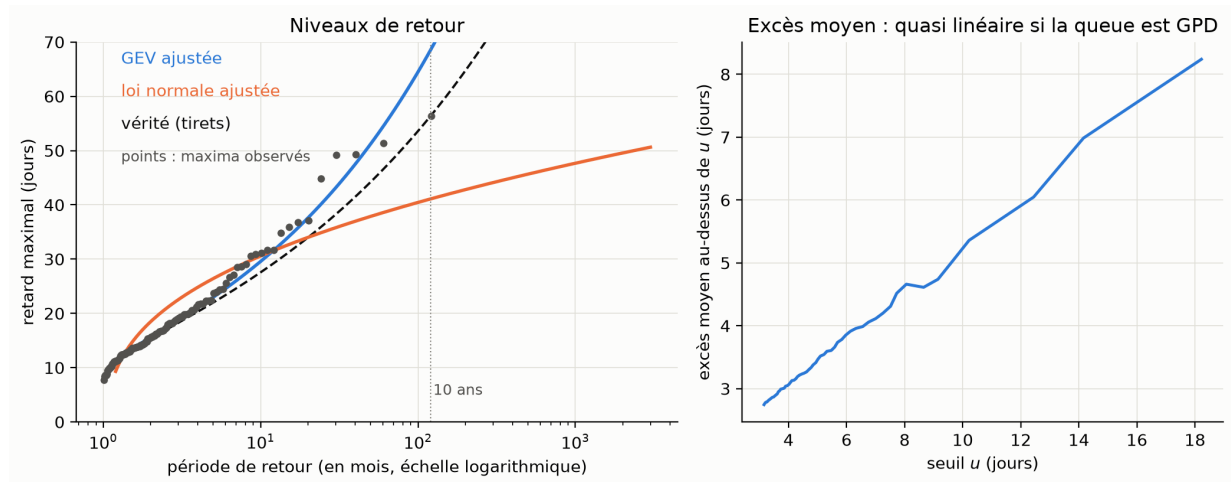


Figure 68: À gauche : niveaux de retour estimés à partir des 120 maxima mensuels de retards de livraison (simulés). La loi normale ajustée (orange) sous-estime fortement les niveaux de retour, alors que la GEV (bleue) reste du bon ordre de grandeur, avec une légère tendance à surestimer, et à intervalle de confiance qui contient la vérité (tirets noirs). À droite : l'excès moyen des retards au-dessus d'un seuil, en fonction du seuil ; une relation à peu près linéaire indique une queue de type GPD.

le plus petit seuil à partir duquel la courbe est à peu près une droite ;

- la **stabilité** de  $\hat{\xi}$  : on estime  $\xi$  pour plusieurs seuils ; au-delà du bon seuil, l'estimation doit se stabiliser.

```
def ajuste_gpd(x, seuil):
    excès = x[x > seuil] - seuil
    xi_, loc_, sc_ = stats.genpareto.fit(excès, floc=0)          # on fixe loc = 0 : les excès sont
    ↪ comptés depuis le seuil
    return xi_, sc_, len(excès)

lignes = []
for q in (0.80, 0.90, 0.93, 0.95, 0.97, 0.98):
    u_ = np.quantile(x_all, q)
    xi_, sc_, k_ = ajuste_gpd(x_all, u_)
    lignes.append({"quantile du seuil": q, "seuil u (jours)": round(u_, 2), "excès": k_, "xi estimé":
    ↪ round(xi_, 3), "sigma_u estimé": round(sc_, 3)})
print(pd.DataFrame(lignes).to_string(index=False))
```

| quantile du seuil | seuil u (jours) | excès | xi estimé | sigma_u estimé |
|-------------------|-----------------|-------|-----------|----------------|
| 0.80              | 4.02            | 2196  | 0.273     | 2.237          |
| 0.90              | 5.69            | 1098  | 0.252     | 2.795          |
| 0.93              | 6.76            | 769   | 0.265     | 2.990          |
| 0.95              | 7.73            | 549   | 0.239     | 3.440          |
| 0.97              | 9.62            | 330   | 0.301     | 3.550          |
| 0.98              | 11.16           | 220   | 0.289     | 4.073          |

L'estimation de  $\xi$  reste entre 0,24 et 0,30 pour tous les seuils raisonnables (la vraie valeur est 0,25) : le choix d'un seuil au 95<sup>e</sup> centile, qui laisse environ 550 excès, est un bon compromis. Passons maintenant à l'estimation du niveau de retour. Si  $\zeta_u = P(X > u)$  est la probabilité d'un dépassement du seuil, la loi de  $X$  au-dessus de  $u$  s'écrit  $P(X > x) = \zeta_u (1 + \xi(x - u)/\sigma_u)^{-1/\xi}$ . Le niveau  $x_m$  dépassé **en moyenne une fois tous les  $m$  colis** vérifie  $P(X > x_m) = 1/m$ , d'où

$$x_m = u + \frac{\sigma_u}{\xi} \left[ (m \zeta_u)^\xi - 1 \right].$$

Une période de retour de 10 ans correspond à  $m =$  nombre de colis en 10 ans (environ 11 000),

de 100 ans à dix fois plus.

```

u_ = np.quantile(x_all, 0.95)
xi_p, sc_p, k_p = ajuste_gpd(x_all, u_)
zeta = (x_all > u_).mean()

def niveau_pot(m, xi_, sc_, u_, zeta_):
    return u_ + sc_ / xi_ * ((m * zeta_) ** xi_ - 1)

# Incertitude : bootstrap sur les colis (on rééchantillonne l'ensemble des colis, on refait
↳ l'ajustement)
rng = np.random.default_rng(673)
boot_pot = []
for _ in range(300):
    xb = rng.choice(x_all, len(x_all))
    ub = np.quantile(xb, 0.95)
    xib, scb, _ = ajuste_gpd(xb, ub)
    boot_pot.append((xib, scb, ub, (xb > ub).mean()))
boot_pot = np.array(boot_pot)

print(f"POT : seuil u = {u_:.2f} j ({k_p} excès, {zeta:.3f} des colis) ; GPD : xi = {xi_p:.3f},
↳ sigma_u = {sc_p:.3f}")
print(f"xi : IC95 bootstrap [{np.percentile(boot_pot[:, 0], 2.5):.3f} ; {np.percentile(boot_pot[:,
↳ 0], 97.5):.3f}] (GEV par blocs : [{np.percentile(xi_boot, 2.5):.3f} ; {np.percentile(xi_boot,
↳ 97.5):.3f}])")

def niveau_vrai_colis(m):
    return 1 + SIGMA0 / XI0 * (m ** XI0 - 1) # S(x) = 1/m pour la vraie GPD décalée

lignes = []
for ans, nom in ((1, "1 an"), (10, "10 ans"), (100, "100 ans")):
    m = n_tot * ans / 10.0 # nombre de colis dans la période de
↳ retour
    lv = np.array([niveau_pot(m, b[0], b[1], b[2], b[3]) for b in boot_pot])
    lignes.append({"retour": nom, "vérité": round(niveau_vrai_colis(m), 1), "POT (GPD)":
↳ round(niveau_pot(m, xi_p, sc_p, u_, zeta), 1),
        "IC95 bootstrap POT": f"[{np.percentile(lv, 2.5):.0f} ; {np.percentile(lv,
↳ 97.5):.0f}]"})
print(pd.DataFrame(lignes).to_string(index=False))

POT : seuil u = 7.73 j (549 excès, 0.050 des colis) ; GPD : xi = 0.239, sigma_u = 3.440
xi : IC95 bootstrap [0.148 ; 0.330] (GEV par blocs : [0.213 ; 0.459])
retour vérité POT (GPD) IC95 bootstrap POT
1 an 29.5 30.8 [27 ; 35]
10 ans 56.4 58.3 [45 ; 73]
100 ans 104.2 105.9 [71 ; 159]

```

La méthode POT donne des estimations très proches de la vérité (58 jours à 10 ans pour une vérité de 56 ; 106 à 100 ans pour 104) et des intervalles d'incertitude **plus étroits** que ceux des maxima par blocs (à 100 ans : [71 ; 159] contre [89 ; 289]), parce qu'elle utilise 549 valeurs au lieu de 120. L'intervalle de  $\xi$  est aussi plus étroit ([0,15 ; 0,33] contre [0,21 ; 0,46]). Une remarque sur les deux colonnes « vérité » : elles diffèrent à peine (56,3 contre 56,4 à 10 ans), car elles mesurent deux choses voisines : l'une, le *maximum mensuel* dépassé une fois tous les 120 mois ; l'autre, le *colis individuel* dépassé une fois tous les 11 000 colis environ. Pour un grand nombre de colis, ces deux notions sont presque identiques.

### Les pièges de l'extrapolation.

1. **Le seuil** : toujours examiner la sensibilité à son choix. Une conclusion qui change radicalement avec le seuil n'est pas fiable.
2. **L'indépendance** : les théorèmes supposent des observations (à peu près) indépen-

dantes. Des extrêmes **regroupés** (une tempête qui donne dix journées d'extrêmes consécutives) doivent être « dégroupés » : on ne garde que le plus grand de chaque grappe.

3. **La stationnarité** : si la loi change avec le temps (saison, tendance), les niveaux de retour d'hier ne valent plus pour demain. On peut faire dépendre  $\mu$  ou  $\sigma$  du temps (tendance, covariables).
4. **L'extrapolation reste une extrapolation** : un niveau de retour à 100 ans estimé sur 10 ans de données est une **projection**, avec une incertitude énorme. L'intervalle, bien plus que l'estimation ponctuelle, est le résultat à communiquer.
5. **Les moments** : pour  $\xi \geq 1/2$  la variance est infinie ; pour  $\xi \geq 1$  la moyenne l'est. Les résumés habituels deviennent alors trompeurs, et « la moyenne des retards » n'a plus de sens statistique stable.

**Lien avec le bayésien.** Quand les données sont rares, les extrêmes se prêtent très bien à l'approche bayésienne : un a priori informatif sur  $\xi$  (« les queues de retards logistiques ont typiquement  $0 < \xi < 0,5$  ») stabilise l'estimation, et le Monte-Carlo (6.2) fournit directement une **loi a posteriori des niveaux de retour**. C'est l'approche standard en hydrologie.

#### À retenir (6.5).

- Les décisions de **risque** dépendent de la **queue** de la distribution, et la loi normale en donne une image **dangereusement optimiste**.
- **Théorème de Fisher-Tippett-Gnedenko** : le maximum de  $n$  observations (normalisé) suit une loi **GEV**, de paramètres position  $\mu$ , échelle  $\sigma$  et forme  $\xi$  ;  $\xi > 0$  signale une queue lourde.
- **Deux approches** : maxima par blocs (GEV) ou excès au-dessus d'un seuil (**POT**, loi de Pareto généralisée). POT utilise davantage de données ; il demande de choisir un seuil.
- Le **niveau de retour**  $z_T$  est la valeur dépassée en moyenne une fois toutes les  $T$  périodes ; on l'estime par une formule explicite dans les paramètres.
- **L'incertitude est grande par construction** : toujours donner un intervalle (bootstrap, vraisemblance profilée, ou a posteriori bayésien).

## 6.6 ☉ Pour aller plus loin : les copules et la modélisation de la dépendance

**Section optionnelle.** Elle suppose les lois jointes et la corrélation (volume I, chapitre 2), la simulation par inversion (6.2.4) et, pour la fin, l'estimation par maximum de vraisemblance. Elle ne dépend pas des sections 6.3 à 6.5.

**Intuition.** Deux transporteurs livrent les colis de Dar Jasmin. Chacun a ses propres retards, parfois très longs. Mais le vrai risque pour Yasmine n'est pas qu'un transporteur ait un mauvais jour : c'est que **les deux aient un très mauvais jour en même temps** (un jour de grève, de tempête, de veille de fête), car alors elle n'a plus de solution de repli. Ce risque ne dépend pas seulement de la loi de chaque transporteur (les « lois marginales »), mais de la façon dont ils sont **liés** : de la **structure de dépendance**. La corrélation ne la résume pas : deux paires de variables peuvent avoir la même corrélation et des comportements extrêmes opposés. Une **copule** isole cette structure de dépendance, séparément des marginales, et permet

de la modéliser.

### 6.6.1 La corrélation ne dit pas tout

Rappelons (volume I, section 3.1.7) que le coefficient de corrélation de Pearson ne mesure que la dépendance **linéaire**, et qu'il est sensible aux marginales. On lui préfère souvent des mesures **de rang**, qui ne changent pas quand on applique une transformation croissante à l'une des variables :

- le **tau de Kendall**  $\tau$  : la probabilité que deux observations soient « concordantes » (l'une est plus grande dans les deux variables) moins la probabilité qu'elles soient « discordantes » ;
- le **rho de Spearman** : la corrélation de Pearson des **rangs**.

*Exemple à la main.* Quatre clients, avec (panier, nombre de commandes) = (10, 1), (20, 3), (30, 2), (40, 4). Il y a  $\binom{4}{2} = 6$  paires. Paires concordantes (les deux variables vont dans le même sens) : {1, 2}, {1, 3}, {1, 4}, {2, 4}, {3, 4} : 5. Paire discordante : {2, 3} (le panier monte de 20 à 30 mais les commandes passent de 3 à 2) : 1. Donc  $\tau = (5 - 1)/6 = 0,667$ .

```
import numpy as np
import pandas as pd
from scipy import stats

panier = [10, 20, 30, 40]
nb = [1, 3, 2, 4]
print("tau de Kendall :", round(stats.kendalltau(panier, nb).statistic, 4), "(à la main : 4/6 =",
      ↪ round(4 / 6, 4), ")")
print("rho de Spearman :", round(stats.spearmanr(panier, nb).statistic, 4))
# invariance par transformation croissante : on passe au logarithme et au carré
print("tau après log et carré :", round(stats.kendalltau(np.log(panier), np.square(nb)).statistic,
      ↪ 4), "(inchangé)")
print("Pearson avant :", round(np.corrcoef(panier, nb)[0, 1], 4), "| après :",
      ↪ round(np.corrcoef(np.log(panier), np.square(nb))[0, 1], 4), "(change)")
```

```
tau de Kendall : 0.6667 (à la main : 4/6 = 0.6667 )
rho de Spearman : 0.8
tau après log et carré : 0.6667 (inchangé)
Pearson avant : 0.8 | après : 0.7592 (change)
```

Le tau de Kendall est inchangé par toute transformation croissante des variables ; la corrélation de Pearson, elle, change. **Ce qui ne bouge pas quand on transforme les marginales, c'est précisément la structure de dépendance.** La copule est l'objet mathématique qui la capture.

### 6.6.2 Le théorème de Sklar

**Le principe : la transformation intégrale de probabilité.** Si  $X$  a une fonction de répartition  $F$  continue, alors  $U = F(X)$  suit la loi uniforme sur  $[0, 1]$ .

*Démonstration.* Pour  $u \in [0, 1]$ ,  $P(F(X) \leq u) = P(X \leq F^{-1}(u)) = F(F^{-1}(u)) = u$ . C'est la réciproque du théorème d'inversion de 6.2.4. □

Passer de  $X$  à  $U = F(X)$  efface la marginale (toute variable continue devient uniforme) mais **garde le rang**, donc la dépendance.

**Définition.** Une **copule** est la fonction de répartition jointe d'un couple  $(U, V)$  dont les deux marginales sont uniformes sur  $[0, 1]$  :  $C(u, v) = P(U \leq u, V \leq v)$ .

**Théorème de Sklar.** Pour tout couple  $(X, Y)$  de fonction de répartition jointe  $H$  et de margi-

nales  $F, G$ , il existe une copule  $C$  telle que

$$H(x, y) = C(F(x), G(y)),$$

unique si  $F$  et  $G$  sont continues. Réciproquement, toute copule  $C$  combinée à des marginales  $F, G$  quelconques définit une loi jointe.

Le théorème sépare l'énoncé « loi jointe » en deux : **(1) les marginales** (le comportement de chaque variable seule) et **(2) la copule** (la dépendance). On peut modéliser l'une et l'autre **séparément**, puis les assembler. C'est ce qui rend l'outil si commode : on choisit une marginale réaliste pour chaque variable (par exemple, une loi de Pareto généralisée pour des retards, comme en 6.5) et une copule pour les relier.

**Trois copules extrêmes (bornes de Fréchet).** *Indépendance* :  $C(u, v) = uv$ . *Dépendance parfaite positive* (comonotonie :  $V = U$ ) :  $C(u, v) = \min(u, v)$ . *Dépendance parfaite négative* ( $V = 1 - U$ ) :  $C(u, v) = \max(u + v - 1, 0)$ . Toute copule est coincée entre les deux dernières :  $\max(u + v - 1, 0) \leq C(u, v) \leq \min(u, v)$ .

### 6.6.3 La copule gaussienne

Idée : prendre le couple normal bivarié, **effacer ses marginales normales** (en les transformant en uniformes), **garder sa dépendance**. Soient  $(Z_1, Z_2)$  normaux standard de corrélation  $\rho$  et  $\Phi$  la fonction de répartition de  $\mathcal{N}(0, 1)$ . Alors  $(U, V) = (\Phi(Z_1), \Phi(Z_2))$  a des marginales uniformes ; sa fonction de répartition est la **copule gaussienne**

$$C_\rho(u, v) = \Phi_2(\Phi^{-1}(u), \Phi^{-1}(v); \rho).$$

Pour simuler un couple  $(X, Y)$  de marginales  $F$  et  $G$  et de dépendance gaussienne : tirer  $(Z_1, Z_2)$ , poser  $U_i = \Phi(Z_i)$ , puis  $X = F^{-1}(U_1)$  et  $Y = G^{-1}(U_2)$  (**inversion**, 6.2.4). Le tau de Kendall d'une copule gaussienne vaut

$$\tau = \frac{2}{\pi} \arcsin \rho \quad \Leftrightarrow \quad \rho = \sin\left(\frac{\pi \tau}{2}\right) \quad (\text{formule admise}).$$

La copule gaussienne est simple et très utilisée, mais elle a un défaut majeur : elle n'a **aucune dépendance de queue**. Quand  $\rho < 1$ , la probabilité que les deux variables soient *simultanément* extrêmes devient négligeable plus vite que ne le ferait une vraie dépendance. En deux mots : un modèle gaussien suppose que « quand ça va très mal, ça va très mal *séparément* ».

### 6.6.4 La copule de Clayton : la dépendance dans les queues

La **copule de Clayton** de paramètre  $\theta > 0$  est

$$C_\theta(u, v) = (u^{-\theta} + v^{-\theta} - 1)^{-1/\theta}.$$

Elle vérifie  $\tau = \frac{\theta}{\theta + 2}$  (formule admise) :  $\theta \rightarrow 0$  donne l'indépendance,  $\theta \rightarrow \infty$  la dépendance parfaite. Sa particularité est la **dépendance de queue inférieure** :  $\lambda_L = \lim_{q \rightarrow 0} P(V \leq q \mid U \leq q) = 2^{-1/\theta} > 0$ . Quand l'une des variables est très petite, l'autre l'est aussi avec une probabilité non négligeable, même à l'extrême.

**Comment la simuler ? L'algorithme de Marshall-Olkin.** Soit  $V \sim \text{Gamma}(1/\theta, \text{taux } 1)$  et  $E_1, E_2 \sim \text{Exp}(1)$  indépendantes de  $V$  et entre elles. Alors

$$U_i = \left(1 + \frac{E_i}{V}\right)^{-1/\theta}, \quad i = 1, 2,$$

est un couple de loi de Clayton.

*Démonstration.* Posons  $\psi(t) = (1 + t)^{-1/\theta}$ . C'est la transformée de Laplace de la loi  $\text{Gamma}(1/\theta, 1)$  :  $\mathbb{E}[e^{-tV}] = \psi(t)$ . Son inverse est  $\psi^{-1}(u) = u^{-\theta} - 1$ . On a  $U_i = \psi(E_i/V)$  et  $\psi$  est décroissante, donc  $U_i \leq u \iff E_i \geq V\psi^{-1}(u)$ , d'où, sachant  $V$ ,  $P(U_i \leq u \mid V) = \exp(-V\psi^{-1}(u))$ . Les  $E_i$  étant indépendantes sachant  $V$ ,

$$P(U_1 \leq u_1, U_2 \leq u_2) = \mathbb{E}[e^{-V\psi^{-1}(u_1)} e^{-V\psi^{-1}(u_2)}] = \psi(\psi^{-1}(u_1) + \psi^{-1}(u_2)) = (u_1^{-\theta} + u_2^{-\theta} - 1)^{-1/\theta}. \square$$

(Le facteur  $V$  est un « facteur commun » : quand  $V$  est petit, les deux  $U_i$  sont petits **ensemble**. C'est exactement l'idée d'un événement qui touche les deux transporteurs à la fois.)

Pour obtenir une dépendance dans la queue **supérieure** (les deux variables sont grandes ensemble), on **retourne** la copule : si  $(U, V)$  suit Clayton, alors  $(1 - U, 1 - V)$  suit la **copule de Clayton retournée** (ou « de survie »), de dépendance de queue supérieure  $\lambda_U = 2^{-1/\theta}$ .

### 6.6.5 Deux copules, même tau, comportements extrêmes opposés

Fixons  $\tau = 0,5$  (une dépendance modérément forte). Alors  $\rho = \sin(\pi/4) = 0,7071$  pour la copule gaussienne et  $\theta = 2\tau/(1 - \tau) = 2$  pour Clayton. Nous simulons 200 000 couples de chacune, et nous comparons à la formule exacte la probabilité que les deux variables soient simultanément parmi les 1 %, 5 % ou 10 % **les plus petites**.

```
from scipy.stats import norm, kendalltau, multivariate_normal

def sim_gauss(rho, n, rng):
    z = rng.multivariate_normal([0, 0], [[1, rho], [rho, 1]], n)
    return norm.cdf(z)

def sim_clayton(theta, n, rng):
    v = rng.gamma(1 / theta, 1.0, n)  # facteur commun
    e = rng.exponential(size=(n, 2))
    return (1 + e / v[:, None]) ** (-1 / theta)

tau = 0.5
rho = np.sin(np.pi * tau / 2)
theta = 2 * tau / (1 - tau)
rng = np.random.default_rng(680)
n = 200_000
ug, uc = sim_gauss(rho, n, rng), sim_clayton(theta, n, rng)
print(f"rho = {rho:.4f}, theta = {theta:.1f}")
print(f"tau de Kendall simulé (sur 5 000 points) : gaussienne {kendalltau(ug[:5000, 0], ug[:5000, 1]).statistic:.3f} | Clayton {kendalltau(uc[:5000, 0], uc[:5000, 1]).statistic:.3f}")

lignes = []
for q in (0.10, 0.05, 0.01):
    z = norm.ppf(q)
    exact_g = multivariate_normal([0, 0], [[1, rho], [rho, 1]]).cdf([z, z])  # C_rho(q, q)
    exact_c = (2 * q ** (-theta) - 1) ** (-1 / theta)  # C_theta(q, q)
    lignes.append({"q": q, "indépendance q²": f"{q**2:.5f}",
                  "gaussienne : exact": f"{exact_g:.5f}", "gaussienne : simulé": f"{np.mean((ug <
                  < q).all(axis=1)):.5f}",
                  "Clayton : exact": f"{exact_c:.5f}", "Clayton : simulé": f"{np.mean((uc <
                  < q).all(axis=1)):.5f}"})
```

```

"Clayton / gaussienne": round(exact_c / exact_g, 2)})
print(pd.DataFrame(lignes).to_string(index=False))
print(f"\ndépendance de queue : lambda_L de Clayton = 2^(-1/theta) = {2 ** (-1 / theta):.3f} ; celle
↳ de la gaussienne = 0")

rho = 0.7071, theta = 2.0
tau de Kendall simulé (sur 5 000 points) : gaussienne 0.500 | Clayton 0.506
q indépendance q² gaussienne : exact gaussienne : simulé Clayton : exact Clayton : simulé Clayton
↳ / gaussienne
0.10      0.01000      0.04739      0.04682      0.07089      0.07184
↳ 1.50
0.05      0.00250      0.01992      0.01959      0.03538      0.03565
↳ 1.78
0.01      0.00010      0.00273      0.00281      0.00707      0.00681
↳ 2.59

dépendance de queue : lambda_L de Clayton = 2^(-1/theta) = 0.707 ; celle de la gaussienne = 0

```

Les formules exactes et les simulations s'accordent. Lecture : avec le **même tau** de 0,5, la probabilité que les deux variables soient *simultanément* parmi les 1 % les plus petites est de 0,71 % avec Clayton contre 0,27 % avec la gaussienne, soit **2,6 fois plus** (dernière colonne), et le rapport **augmente à mesure que l'événement devient plus rare** (1,5 à 10 %, 1,8 à 5 %, 2,6 à 1 %). Pour référence, sous indépendance, cette probabilité serait de 0,01 %. Voici le dessin des deux nuages de points : même dépendance « moyenne », des queues très différentes.

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.facecolor": "#fcfcfb",
                    "figure.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb",
                    "legend.frameon": False, "font.size": 10})

fig, axes = plt.subplots(1, 2, figsize=(9.6, 4.6))
for ax, u_, titre, coul in zip(axes, (ug[:2500], uc[:2500]), ("copule gaussienne ($\\rho=0{,}71$)",
↳ "copule de Clayton ($\\theta=2$)"), (BLEU, ORANGE)):
    ax.scatter(u_[:, 0], u_[:, 1], s=5, color=coul, alpha=0.55)
    coin = (u_ < 0.05).all(axis=1)
    ax.scatter(u_[coin, 0], u_[coin, 1], s=11, color=ROUGE)
    ax.plot([0.05, 0.05, 0], [0, 0.05, 0.05], color=ROUGE, lw=0.8)
    ax.set_title(f"{titre}\\n{coin.sum()} points dans le coin inférieur (rouge)", fontsize=10)
    ax.set_xlabel("$U$ (rang de la première variable)")
    ax.set_aspect("equal")
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.grid(False)
axes[0].set_ylabel("$V$ (rang de la seconde)")
plt.tight_layout()
plt.savefig("figures/ch06-copules-gauss-clayton.png", dpi=200, bbox_inches="tight")
plt.close()

```

**L'enseignement de la crise de 2008.** Avant 2008, de nombreux produits financiers (les « CDO ») évaluaient le risque de défaut simultané de nombreux emprunteurs avec une **copule gaussienne**, calibrée sur des corrélations. En période de crise, les défauts se sont produits **ensemble** (dépendance de queue) bien plus souvent que le modèle ne le prévoyait. Le choix de la copule n'est pas un détail technique : c'est une hypothèse sur la dépendance **dans les extrêmes**, précisément là où les données sont les plus rares.



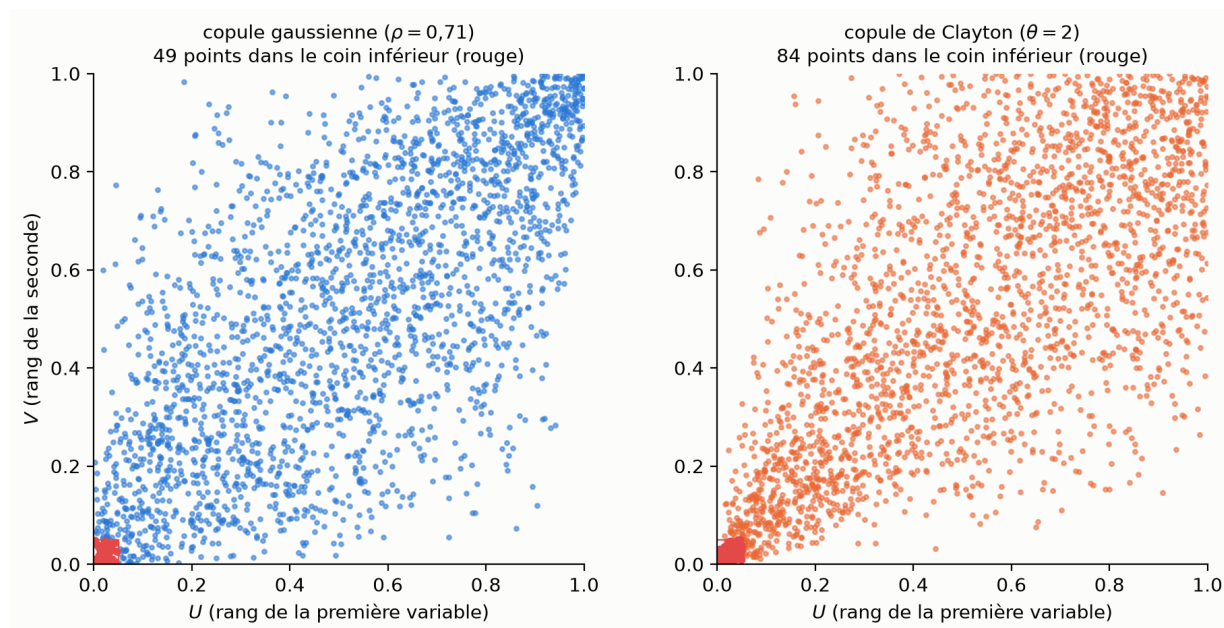


Figure 69: Deux copules de même tau de Kendall (0,5), 2 500 points chacune, dans l'échelle des rangs. À gauche : la copule gaussienne. À droite : la copule de Clayton, dont les points se concentrent dans le coin inférieur gauche (les deux variables très petites ensemble). Les points rouges sont ceux dont les deux coordonnées sont inférieures à 5 %.

### 6.6.6 Estimer une copule à partir de données

Procédure standard (« semi-paramétrique ») :

1. transformer chaque variable en **pseudo-observations**  $\hat{U}_i = \text{rang}(X_i)/(n+1)$  (une estimation de  $F(X_i)$  par le rang, qui efface la marginale sans la modéliser) ;
2. estimer le paramètre de la copule, soit par **inversion du tau de Kendall** (calculer  $\hat{\tau}$  puis résoudre  $\tau(\theta) = \hat{\tau} : \hat{\rho} = \sin(\pi\hat{\tau}/2)$ ,  $\hat{\theta} = 2\hat{\tau}/(1-\hat{\tau})$ ), soit par **maximum de vraisemblance** sur les pseudo-observations ;
3. **choisir** entre familles par la vraisemblance (comme en 1.4 : l'AIC compte les paramètres ; ici une famille à un paramètre contre une autre à un paramètre : la comparaison directe des log-vraisemblances suffit).

La vraisemblance fait appel à la **densité de copule**  $c(u, v) = \partial^2 C / \partial u \partial v$ , que voici : pour Clayton,  $c_\theta(u, v) = (1 + \theta)(uv)^{-\theta-1}(u^{-\theta} + v^{-\theta} - 1)^{-2-1/\theta}$  ; pour la gaussienne, avec  $x = \Phi^{-1}(u)$  et  $y = \Phi^{-1}(v)$ ,  $c_\rho(u, v) = \frac{1}{\sqrt{1-\rho^2}} \exp\left(-\frac{\rho^2(x^2 + y^2) - 2\rho xy}{2(1-\rho^2)}\right)$ . Avant d'attaquer un cas réel, **testons la**

**procédure sur des données simulées dont on connaît la copule** : nous simulons 500 couples gaussiens, puis 500 couples de Clayton (avec des marginales quelconques, exponentielle et log-normale : le rang les efface), et nous demandons à la procédure de retrouver la bonne copule.

```
from scipy.optimize import minimize_scalar

def pseudo_obs(x, y):
    n_ = len(x)
    return stats.rankdata(x) / (n_ + 1), stats.rankdata(y) / (n_ + 1)

def ll_gauss(u, v, rho_):
    x, y = norm.ppf(u), norm.ppf(v)
    return np.sum(-0.5 * np.log(1 - rho_**2) - (rho_**2 * (x**2 + y**2) - 2 * rho_ * x * y) / (2 * (1 - rho_**2)))
```

```

def ll_clayton(u, v, th):
    return np.sum(np.loglp(th) - (th + 1) * (np.log(u) + np.log(v)) - (2 + 1 / th) * np.log(u**-th +
        ↪ v**-th - 1))

def ajuste(x, y):
    u, v = pseudo_obs(x, y)
    tau_ = kendalltau(x, y).statistic
    rho_h, th_h = np.sin(np.pi * tau_ / 2), 2 * tau_ / (1 - tau_)
    th_mv = minimize_scalar(lambda t: -ll_clayton(u, v, t), bounds=(0.01, 30), method="bounded").x
    ↪ # maximum de vraisemblance
    return {"tau": tau_, "rho (inversion)": rho_h, "theta (inversion)": th_h, "theta (max.
        ↪ vraisemblance)": th_mv,
            "logvrais. gaussienne": ll_gauss(u, v, rho_h), "logvrais. Clayton": ll_clayton(u, v, th_h)}

rng = np.random.default_rng(681)
lignes = []
for nom, u_ in (("données gaussiennes (rho = 0,71)", sim_gauss(rho, 500, rng)), ("données Clayton
    ↪ (theta = 2)", sim_clayton(theta, 500, rng))):
    x = -np.log(1 - u_[:, 0]) # marginale exponentielle
    y = np.exp(0.5 * norm.ppf(u_[:, 1])) # marginale log-normale
    r = ajuste(x, y)
    lignes.append({"données": nom, "tau estimé": round(r["tau"], 3), "rho": round(r["rho
    ↪ (inversion)"], 3),
                    "theta (inversion)": round(r["theta (inversion)"], 2), "theta (MV)": round(r["theta
    ↪ (max. vraisemblance)], 2),
                    "logvrais. gaussienne": round(r["logvrais. gaussienne"], 1), "logvrais. Clayton":
    ↪ round(r["logvrais. Clayton"], 1)})
print(pd.DataFrame(lignes).to_string(index=False))

```

|                                  | données | tau estimé | rho   | theta (inversion) | theta (MV) | logvrais. gaussienne |
|----------------------------------|---------|------------|-------|-------------------|------------|----------------------|
| ↪ logvrais. Clayton              |         |            |       |                   |            |                      |
| données gaussiennes (rho = 0,71) |         | 0.452      | 0.652 | 1.65              | 1.08       | 137.0                |
| ↪ 85.6                           |         |            |       |                   |            |                      |
| données Clayton (theta = 2)      |         | 0.564      | 0.774 | 2.58              | 2.34       | 203.4                |
| ↪ 251.5                          |         |            |       |                   |            |                      |

Dans chaque cas, la famille qui a engendré les données obtient la **plus grande log-vraisemblance** (137,0 contre 85,6 pour les données gaussiennes ; 251,5 contre 203,4 pour les données de Clayton) : la procédure sait distinguer. Pour les données de Clayton, les deux estimations de  $\theta$  (2,58 par inversion du tau, 2,34 par maximum de vraisemblance) sont voisines du vrai  $\theta = 2$ , à l'erreur d'échantillonnage près (cet échantillon de 500 points a un tau empirique de 0,56 au lieu de 0,50). Notez qu'en cas de mauvaise famille (Clayton ajustée sur données gaussiennes) les deux méthodes d'estimation de  $\theta$  divergent (1,65 contre 1,08) : c'est un signal de mauvaise spécification. (Avec 500 points, la différence de log-vraisemblance est nette ; avec 50, elle ne le serait pas.)

### 6.6.7 Applications

**Un cas réel (simulé) de dépendance faible.** Dans le fichier `clients.csv`, le panier moyen et le nombre de commandes par an des acheteurs sont-ils liés ? Les clients qui achètent souvent sont-ils aussi ceux qui dépensent plus par commande ? Le nombre de commandes est un entier : de nombreuses égalités rendent les rangs ambigus (la copule n'est pas unique pour des marginales discrètes). Nous **départageons les égalités au hasard**, par une petite astuce standard.

```

clients = pd.read_csv("donnees/clients.csv")
ach = clients[clients["nb_commandes_an"] > 0]
rng = np.random.default_rng(682)
x = ach["panier_moyen"].to_numpy()
y = ach["nb_commandes_an"].to_numpy() + rng.uniform(-0.5, 0.5, len(ach)) # départage aléatoire
↪ des égalités (variable entière)
u, v = pseudo_obs(x, y)

```

```

r = ajuste(x, y)
print(f"len(ach) acheteurs ; tau de Kendall = {r['tau']:.3f} ; rho de Spearman = {stats.spearmanr(x,
↪ y).statistic:.3f}")
print(f"log-vraisemblance : gaussienne {r['logvrais. gaussienne']:.1f}, Clayton {r['logvrais.
↪ Clayton']:.1f}")
q = 0.9
obs = np.mean((u > q) & (v > q))
rho_h = r["rho (inversion)"]
modele = multivariate_normal([0, 0], [[1, rho_h], [rho_h, 1]]).cdf([-norm.ppf(q), -norm.ppf(q)]) #
↪ P(U > q, V > q), symétrie
print(f"part de clients dans les 10 % supérieurs pour LES DEUX variables : observée {obs:.4f} |
↪ indépendance {(1 - q)**2:.4f} | copule gaussienne {modele:.4f}")

```

```

1740 acheteurs ; tau de Kendall = 0.079 ; rho de Spearman = 0.118
log-vraisemblance : gaussienne 12.8, Clayton 3.4
part de clients dans les 10 % supérieurs pour LES DEUX variables : observée 0.0121 | indépendance
↪ 0.0100 | copule gaussienne 0.0142

```

La dépendance est **réelle mais faible** (tau de Kendall de 0,079) : la copule gaussienne prédit 1,42 % de clients dans le décile supérieur des deux variables à la fois, contre 1,00 % sous indépendance ; la fréquence observée (1,21 %, soit 21 clients sur 1 740) se situe entre les deux, et un si petit effectif ne permet pas de trancher. Rien d'alarmant et rien d'exploitable : il est tout aussi important de savoir quand une dépendance est **négligeable**. (Dans le générateur de données, le goût pour les produits, facteur latent, influence légèrement les deux variables.)

**Le vrai problème : deux transporteurs en même temps.** Revenons à notre inquiétude. Yasmine observe depuis 1 500 jours les retards moyens de ses deux transporteurs (simulés ici ; graine 683). Le transporteur A a des retards (en jours) de marginale Pareto généralisée décalée de 1,  $\xi = 0,25$ ,  $\sigma = 1,5$  ; le B de  $\xi = 0,20$ ,  $\sigma = 2$ . **La vraie dépendance** (que Yasmine ignore) est une copule de Clayton **retournée** de  $\theta = 1,5$  : les deux transporteurs sont **simultanément très en retard** bien plus souvent que ne le laisserait croire une dépendance gaussienne. Elle ajuste les deux familles et calcule la probabilité qu'*un même jour*, les deux retards dépassent leur propre 99<sup>e</sup> centile.

```

def gpd_inv(u_, xi_, sg): # fonction de répartition inverse de la GPD
↪ décalée de 1 (6.5)
    return 1 + sg / xi_ * ((1 - u_) ** (-xi_) - 1)

theta_v = 1.5
rng = np.random.default_rng(683)
n_j = 1500
u_vrai = 1 - sim_clayton(theta_v, n_j, rng) # Clayton retournée : dépendance dans la
↪ queue supérieure
A = gpd_inv(u_vrai[:, 0], 0.25, 1.5)
B = gpd_inv(u_vrai[:, 1], 0.20, 2.0)
print(f"{n_j} jours ; retard moyen A : {A.mean():.2f} j, B : {B.mean():.2f} j ; retard maximal A :
↪ {A.max():.1f} j, B : {B.max():.1f} j")

u, v = pseudo_obs(A, B)
tau_h = kendalltau(A, B).statistic
rho_h, th_h = np.sin(np.pi * tau_h / 2), 2 * tau_h / (1 - tau_h)
ll_g = ll_gauss(u, v, rho_h)
ll_cr = ll_clayton(1 - u, 1 - v, th_h) # Clayton retournée : on applique la densité
↪ de Clayton à (1 - u, 1 - v)
ll_c = ll_clayton(u, v, th_h) # Clayton non retournée, pour comparaison
print(f"tau de Kendall estimé : {tau_h:.3f} -> rho = {rho_h:.3f} (gaussienne) ; theta = {th_h:.2f}
↪ (Clayton)")
print(f"log-vraisemblance : gaussienne {ll_g:.1f} | Clayton (queue inférieure) {ll_c:.1f} | Clayton
↪ retournée (queue supérieure) {ll_cr:.1f}")

```

```

1500 jours ; retard moyen A : 2.94 j, B : 3.39 j ; retard maximal A : 23.1 j, B : 31.4 j
tau de Kendall estimé : 0.416 -> rho = 0.608 (gaussienne) ; theta = 1.42 (Clayton)

```

log-vraisemblance : gaussienne 332.6 | Clayton (queue inférieure) 11.0 | Clayton retournée (queue  
↪ supérieure) 431.7

La comparaison des log-vraisemblances désigne sans ambiguïté la **Clayton retournée**, la bonne famille : elle dépasse la gaussienne d'une centaine d'unités de log-vraisemblance (431,7 contre 332,6), alors que la Clayton « classique » (dépendance dans la queue inférieure, ce qui est le mauvais côté) est de loin la pire des trois (11,0). Le  $\hat{\theta} = 1,42$  obtenu est proche du vrai 1,5. Maintenant, le calcul qui compte :

```
qq = 0.99
# P(les deux retards dépassent leur 99e centile) sous chaque modèle (marginales identiques : seules
↪ les copules diffèrent)
p_indep = (1 - qq) ** 2
p_gauss = multivariate_normal([0, 0], [[1, rho_h], [rho_h, 1]]).cdf([-norm.ppf(qq), -norm.ppf(qq)])
p_clay_ret = (2 * (1 - qq) ** (-th_h) - 1) ** (-1 / th_h) # modèle ajusté : P(U>q,V>q) =
↪ C_theta(1-q, 1-q) pour la Clayton retournée
p_vrai = (2 * (1 - qq) ** (-theta_v) - 1) ** (-1 / theta_v) # vérité
p_emp = np.mean((u > qq) & (v > qq))

lignes = [("indépendance", p_indep), ("copule gaussienne ajustée", p_gauss), ("Clayton retournée
↪ ajustée", p_clay_ret),
          ("VÉRITÉ (theta = 1,5)", p_vrai), ("observé sur les 1 500 jours", p_emp)]
tab = pd.DataFrame({"modèle": [l[0] for l in lignes], "P(2 retards extrêmes le même jour)":
↪ [f"{l[1]:.5f}" for l in lignes],
                   "fois plus que l'indépendance": [round(l[1] / p_indep, 1) for l in lignes],
                   "soit une fois tous les... (jours)": [round(1 / l[1]) for l in lignes]})
print(tab.to_string(index=False))
```

|                             | modèle P(2 retards extrêmes le même jour) | fois plus que l'indépendance | soit une |
|-----------------------------|-------------------------------------------|------------------------------|----------|
| ↪ fois tous les... (jours)  |                                           |                              |          |
| indépendance                | 0.00010                                   |                              | 1.0      |
| ↪ 10000                     |                                           |                              |          |
| copule gaussienne ajustée   | 0.00193                                   |                              | 19.3     |
| ↪ 518                       |                                           |                              |          |
| Clayton retournée ajustée   | 0.00615                                   |                              | 61.5     |
| ↪ 163                       |                                           |                              |          |
| VÉRITÉ (theta = 1,5)        | 0.00630                                   |                              | 63.0     |
| ↪ 159                       |                                           |                              |          |
| observé sur les 1 500 jours | 0.00533                                   |                              | 53.3     |
| ↪ 188                       |                                           |                              |          |

C'est la leçon centrale de cette section. À marginales identiques et à dépendance **globale** identique (même tau), la copule gaussienne ajustée prévoit des doubles retards extrêmes **3,3 fois moins fréquents** que la vérité : un jour sur 518 contre un jour sur 159. Pour Yasmine, c'est la différence entre « un tel jour tous les 17 mois » et « un tel jour tous les 5 mois ». Et par rapport à l'indépendance (un jour sur 10 000), les deux modèles de dépendance annoncent un risque 19 à 63 fois plus grand : ignorer la dépendance serait bien pire encore. La Clayton retournée ajustée (un jour sur 163) est quasiment sur la vérité (un jour sur 159). Ce calcul est essentiellement un calcul de **queue conjointe**, et il dépend presque entièrement du **choix de la copule**, pas des marginales. (L'observation directe sur 1 500 jours ne contient que 8 jours de ce type, soit une fréquence de 0,53 % : le modèle de Clayton en prévoyait 9,2, la copule gaussienne 2,9. Huit événements, c'est peu pour trancher à eux seuls, ce qui illustre pourquoi on **modélise** la dépendance plutôt que de compter les événements rares.)

**Limites.** (1) **La dépendance de queue est très difficile à estimer** : elle repose sur les quelques points extrêmes. Choisir une famille par la vraisemblance globale (qui est dominée par le centre des données) n'est pas une garantie sur les queues. (2) Les copules à un paramètre

ont une forme de dépendance rigide ; pour **plus de deux variables**, on utilise des « vines » (lianes de copules à deux variables), et des modèles plus riches. (3) La dépendance peut **changer avec le temps** (les périodes de crise ont leur propre structure) : le modèle estimé sur une période calme est silencieux sur la crise. (4) Les copules décrivent une **association**, pas une causalité (volume III).

#### À retenir (6.6).

- La corrélation de Pearson ne capture que la dépendance linéaire ; le **tau de Kendall** et le **rho de Spearman** sont des mesures de **rang**, invariantes par transformation croissante.
- **Théorème de Sklar** : toute loi jointe = des **marginales** + une **copule** ( $H = C(F, G)$ ). On modélise séparément les deux.
- La copule est la loi jointe des **rangs normalisés**  $U = F(X)$  (transformation intégrale de probabilité).
- **Gaussienne** : pas de dépendance de queue. **Clayton** : dépendance de queue inférieure ( $\lambda_L = 2^{-1/\theta}$ ) ; sa version **retournée** donne une dépendance de queue supérieure.
- On estime une copule sur des **pseudo-observations** (rangs divisés par  $n+1$ ), par inversion du tau ou maximum de vraisemblance ; on **valide la procédure sur des données simulées** de copule connue.
- **À même tau, des copules différentes ont des comportements extrêmes très différents** : la probabilité d'événements simultanément rares dépend presque uniquement de la copule. C'est le risque principal d'un choix par défaut (gaussien).

## 6.7 Exercices du chapitre 6

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 8, 9 et 10 réutilisent les fonctions `metropolis_1d` et `ess` définies en 6.3. Les exercices 13 et 14 se rapportent aux sections optionnelles 6.5 et 6.6.

### Énoncés

**Exercice 1 ★ (bêta-binomial).** Yasmine teste un nouvel emballage : 3 clients sur 8 le jugent « excellent ». Avec l'a priori  $\text{Beta}(2, 2)$  (« je pense plutôt autour de 50 % »), donnez (a) la loi a posteriori, (b) sa moyenne, (c) le poids de l'a priori dans cette moyenne, (d) la comparaison avec l'estimation du maximum de vraisemblance.

**Exercice 2 ★ (gamma-Poisson).** Le site reçoit 2, 4 et 1 commandes lors de trois soirées. A priori  $\lambda \sim \text{Gamma}(3; \text{taux } 1)$  (moyenne 3 commandes par soirée). Donnez la loi a posteriori de  $\lambda$ , sa moyenne, et un intervalle de crédibilité à 95 %.

**Exercice 3 ★★ (normal-normal).** On modélise le log du panier avec  $\sigma = 0,35$  connu et l'a priori  $\mathcal{N}(4,0; 0,5^2)$ . À partir de combien d'observations le poids des données dans la moyenne a posteriori dépasse-t-il 90 % ? Calculez-le à la main, puis vérifiez avec le code.

**Exercice 4 ★★ (test A/B bayésien).** La version A d'une page convertit 30 visiteurs sur 100, la version B 42 sur 110. Avec des a priori  $\text{Beta}(1, 1)$ , calculez par simulation la probabilité que B soit meilleure que A, le gain attendu en points de pourcentage, et la probabilité que le gain dépasse 5 points. Comparez avec le test de proportions fréquentiste.

**Exercice 5 ★ (Monte-Carlo).** Soient  $U_1, U_2$  uniformes indépendantes sur  $[0, 1]$ . (a) Calculez à la main  $\mathbb{E}[\max(U_1, U_2)]$ . (b) Estimez-la par Monte-Carlo avec 100 000 tirages, avec son erreur type



et son intervalle de confiance. (c) Estimez  $P(U_1 + U_2 > 1,5)$  et comparez avec la valeur exacte  $1/8$ .

**Exercice 6 ★★ (échantillonnage préférentiel).** Soit  $X \sim \text{Exp}(1)$ . On veut  $P(X > 5) = e^{-5}$ . (a) Estimez-la naïvement avec  $n = 10\,000$  tirages. (b) Utilisez la proposition «  $5 + \text{Exp}(1)$  ». Que valent les poids ? Que remarquez-vous sur la variance de l'estimateur ?

**Exercice 7 ★★ (inversion).** La durée de vie  $T$  (en mois) d'un abonnement suit une loi de Weibull de fonction de répartition  $F(t) = 1 - \exp(-(t/\lambda)^k)$ , avec  $\lambda = 24$  et  $k = 1,5$ . (a) Déterminez  $F^{-1}$ . (b) Simulez 100 000 durées. (c) Vérifiez la médiane théorique et la moyenne théorique  $\lambda \Gamma(1 + 1/k)$ .

**Exercice 8 ★★ (Metropolis à la main).** On veut une chaîne sur trois états  $\{1, 2, 3\}$  de loi stationnaire proportionnelle à  $(1, 2, 1)$ . La proposition choisit l'un des deux autres états avec probabilité  $\frac{1}{2}$ . (a) Écrivez la matrice de transition de Metropolis. (b) Vérifiez le bilan détaillé. (c) Vérifiez par le code, et par simulation.

**Exercice 9 ★★ (Metropolis sur une échelle logarithmique).** Six semaines de commandes : 3, 5, 4, 6, 2, 5. Modèle :  $\text{Poisson}(\lambda)$ , a priori  $\text{Gamma}(2; \text{taux } 0,5)$ . (a) Donnez la loi a posteriori exacte. (b) Écrivez un Metropolis sur  $\theta = \log \lambda$  (attention à la transformation de la densité) et comparez avec la loi exacte.

**Exercice 10 ★★ (diagnostics).** Une chaîne autorégressive  $x_t = \varphi x_{t-1} + \sqrt{1 - \varphi^2} \varepsilon_t$  ( $\varepsilon_t \sim \mathcal{N}(0, 1)$ ) a pour loi stationnaire  $\mathcal{N}(0, 1)$  et pour autocorrélation  $\rho_k = \varphi^k$ . (a) Montrez que son ESS théorique vaut environ  $n \frac{1 - \varphi}{1 + \varphi}$ . (b) Pour  $\varphi = 0,9$  et  $n = 20\,000$ , comparez avec l'ESS calculée. (c) Quatre chaînes de  $\varphi = 0,99$  lancées de  $-10, -3, 3, 10$  : calculez le  $\hat{R}$  avec et sans élimination des 300 premiers points.

**Exercice 11 ★★ (vérification prédictive).** Modélisez les paniers des acheteurs de la boutique (a) par une loi normale sur le panier en DT, (b) par une loi normale sur le **logarithme** du panier. Avec la statistique-test « asymétrie » (skewness) et le plus petit panier, quel modèle passe la vérification prédictive a posteriori ?

**Exercice 12 ★★ (facteur de Bayes).** Neuf clients sur dix préfèrent le nouvel emballage.  $H_0 : \theta = 0,5$  ;  $H_1 : \theta \sim \mathcal{U}(0, 1)$ . Calculez à la main  $\text{BF}_{10}$ , comparez à la p-valeur exacte bilatérale, puis calculez la probabilité a posteriori de  $H_1$  si l'on pense au départ qu'il y a 1 chance sur 4 que l'emballage ait un effet.

**Exercice 13 ★★ (valeurs extrêmes).** Pour des retards de colis, on a choisi le seuil  $u = 5$  jours ; 5 % des colis le dépassent ( $\zeta_u = 0,05$ ) et la loi des excès est GPD de  $\xi = 0,2$  et  $\sigma_u = 2$  jours. (a) Calculez à la main le retard dépassé en moyenne une fois tous les 1 000 colis. (b) Et tous les 10 000 colis ? (c) Que devient (b) si  $\xi = 0,4$  au lieu de 0,2 ? Qu'en concluez-vous ?

**Exercice 14 ★★ (dépendance de queue de Clayton).** (a) Démontrez que, pour la copule de Clayton,  $P(V \leq q \mid U \leq q) = (2 - q^\theta)^{-1/\theta}$ , puis que cette quantité tend vers  $2^{-1/\theta}$  quand  $q \rightarrow 0$ . (b) Vérifiez numériquement avec  $\theta = 2$  pour  $q = 0,1; 0,01; 0,001$ . (c) Par la formule de survie, que vaut  $P(U > 1 - q, V > 1 - q)$  pour la copule de Clayton *retournée* ? Comparez avec une copule gaussienne de même tau.

## Corrigés

**Corrigé 1.** (a) Succès  $y = 3$ , échecs  $n - y = 5$  :  $\text{Beta}(2 + 3, 2 + 5) = \text{Beta}(5, 7)$ . (b) Moyenne  $5/12 = 0,4167$ . (c) Le poids de l'a priori est  $\frac{a+b}{a+b+n} = \frac{4}{4+8} = \frac{1}{3}$  : un tiers de l'a priori (moyenne 0,5), deux tiers des données (fréquence 0,375) :  $\frac{1}{3} \times 0,5 + \frac{2}{3} \times 0,375 = 0,4167$  ✓. (d) Le maximum

de vraisemblance est  $3/8 = 0,375$  ; l'a posteriori est **tiré vers 0,5**. Avec 8 observations seulement, l'a priori compte beaucoup (voir 6.1.6).

```
import numpy as np
import pandas as pd
from scipy import stats

post = stats.beta(2 + 3, 2 + 5)
print(f"a posteriori Beta(5, 7) : moyenne {post.mean():.4f} | IC95 [{post.ppf(0.025):.3f} ; {post.ppf(0.975):.3f}]")
print(f"poids de l'a priori : {4 / 12:.3f} | EMV : {3 / 8:.3f} | moyenne pondérée : {4 / 12 * 0.5 + 8 / 12 * 3 / 8:.4f}")
```

```
a posteriori Beta(5, 7) : moyenne 0.4167 | IC95 [0.167 ; 0.692]
poids de l'a priori : 0.333 | EMV : 0.375 | moyenne pondérée : 0.4167
```

**Corrigé 2.**  $\sum y_i = 7, n = 3 : \lambda | y \sim \text{Gamma}(3 + 7; 1 + 3) = \text{Gamma}(10; \text{taux } 4)$ , de moyenne  $10/4 = 2,5$  commandes par soirée. Elle est **comprise entre la moyenne a priori (3) et celle des données ( $7/3 = 2,33$ )**, comme il se doit pour un compromis ; elle est plus proche des données car celles-ci pèsent  $3/4$  (3 observations contre un a priori valant « 1 observation » : le taux de l'a priori est 1).

```
post = stats.gamma(a=10, scale=1 / 4)
print(f"Gamma(10, taux 4) : moyenne {post.mean():.3f} | IC95 [{post.ppf(0.025):.3f} ; {post.ppf(0.975):.3f}]")
```

```
Gamma(10, taux 4) : moyenne 2.500 | IC95 [1.199 ; 4.271]
```

**Corrigé 3.** Le poids des données est  $w = \frac{n/\sigma^2}{1/\tau_0^2 + n/\sigma^2}$ . On veut  $w > 0,9 \iff n/\sigma^2 > 9/\tau_0^2 \iff n > 9\sigma^2/\tau_0^2 = 9 \times 0,1225/0,25 = 4,41$ . Il faut donc  $n \geq 5$  observations (avec  $n = 5 : w = 0,911$ , comme en 6.1.8). L'a priori est donc déjà « oublié » à 90 % avec cinq clients seulement : il est assez large par rapport à  $\sigma$ .

```
sigma, tau0 = 0.35, 0.5
for n in range(1, 8):
    w = (n / sigma**2) / (1 / tau0**2 + n / sigma**2)
    print(f"n = {n} : poids des données = {w:.3f}" + (" <- premier n au-dessus de 90 %" if w > 0.9 and (n == 1 or ((n - 1) / sigma**2) / (1 / tau0**2 + (n - 1) / sigma**2)) <= 0.9) else "")
```

```
n = 1 : poids des données = 0.671
n = 2 : poids des données = 0.803
n = 3 : poids des données = 0.860
n = 4 : poids des données = 0.891
n = 5 : poids des données = 0.911 <- premier n au-dessus de 90 %
n = 6 : poids des données = 0.924
n = 7 : poids des données = 0.935
```

**Corrigé 4.** A : Beta(31,71), B : Beta(43,69). On tire dans chaque loi et on compare.

```
rng = np.random.default_rng(690)
S = 400_000
pA = stats.beta(1 + 30, 1 + 70).rvs(S, random_state=rng)
pB = stats.beta(1 + 42, 1 + 68).rvs(S, random_state=rng)
gain = pB - pA
print(f"P(B > A) = {(gain > 0).mean():.4f} | gain moyen = {100 * gain.mean():.1f} points | P(gain > 5 points) = {(gain > 0.05).mean():.4f}")
print(f"IC de crédibilité à 95 % du gain : [{100 * np.percentile(gain, 2.5):.1f} ; {100 * np.percentile(gain, 97.5):.1f}] points")

from statsmodels.stats.proportion import proportions_ztest
z, p = proportions_ztest([42, 30], [110, 100])
```



```
print(f"test fréquentiste : z = {z:.3f}, p-valeur bilatérale = {p:.4f} (unilatérale : {p / 2:.4f})")
```

```
P(B > A) = 0.8925 | gain moyen = 8.0 points | P(gain > 5 points) = 0.6800
IC de crédibilité à 95 % du gain : [-4.7 ; 20.6] points
test fréquentiste : z = 1.248, p-valeur bilatérale = 0.2122 (unilatérale : 0.1061)
```

La probabilité bayésienne que B soit meilleure est de 89 % ; le test fréquentiste donne une p-valeur bilatérale de 0,21 (unilatérale 0,106), **non significative** à 5 %. Remarquez que  $1 - 0,106 = 0,894$  est presque égal à la probabilité bayésienne de 0,8925 : avec un a priori plat et des données assez abondantes, la p-valeur unilatérale et la probabilité a posteriori de l'hypothèse « opposée » **coïncident presque** (c'est un résultat classique pour les proportions). Mais **la p-valeur ne dit pas « la probabilité que B soit meilleure »** (volume I, section 3.5.2), alors que la probabilité bayésienne le dit : « 89 % de chances que B soit meilleure, gain probable de 8 points, avec une probabilité de 68 % que le gain dépasse 5 points ». L'intervalle de crédibilité à 95 % du gain,  $[-4,7 ; +20,6]$  points, est très large : avec une centaine de visiteurs par version, on ne sait pas grand-chose, et c'est exactement ce que dit le test non significatif.

**Corrigé 5.** (a)  $P(\max \leq x) = x^2$ , de densité  $2x$  :  $\mathbb{E}[\max] = \int_0^1 x \cdot 2x dx = 2/3$ . (b), (c) :

```
rng = np.random.default_rng(691)
n = 100_000
u = rng.random((n, 2))
g = u.max(axis=1)
est, se = g.mean(), g.std(ddof=1) / np.sqrt(n)
print(f"E[max] estimée : {est:.4f} ± {1.96 * se:.4f} (IC95) | exacte : {2 / 3:.4f}")
h = (u.sum(axis=1) > 1.5)
est, se = h.mean(), h.std(ddof=1) / np.sqrt(n)
print(f"P(U1 + U2 > 1,5) estimée : {est:.4f} ± {1.96 * se:.4f} | exacte : {1 / 8:.4f}")
```

```
E[max] estimée : 0.6682 ± 0.0015 (IC95) | exacte : 0.6667
P(U1 + U2 > 1,5) estimée : 0.1265 ± 0.0021 | exacte : 0.1250
```

L'intervalle de confiance de Monte-Carlo contient la valeur exacte dans chaque cas (dans 95 % des répétitions de l'expérience, en moyenne).

**Corrigé 6.** (a) L'estimateur naïf a une variance  $p(1-p)/n \approx 6,7 \times 10^{-7}$ , soit un écart-type de  $8,2 \times 10^{-4}$  pour une valeur de  $6,7 \times 10^{-3}$  : environ 12 % d'erreur relative. (b) Tirons  $Y = 5 + E$ ,  $E \sim \text{Exp}(1)$  (densité  $h(y) = e^{-(y-5)}$  pour  $y > 5$ ). Alors  $f(y)/h(y) = e^{-y}/e^{-(y-5)} = e^{-5}$  **pour tout**  $y > 5$  : tous les tirages sont dans la zone d'intérêt (indicatrice égale à 1) et **tous les poids valent**  $e^{-5}$ . L'estimateur vaut donc  $e^{-5}$  **exactement**, avec une **variance nulle** : c'est la proposition idéale, qui est *proportionnelle* à  $g \times f$ . En pratique, on ne la connaît pas (sinon on ne simulerait pas), mais on s'en approche.

```
rng = np.random.default_rng(692)
n = 10_000
naif = (rng.exponential(1, n) > 5).mean()
y = 5 + rng.exponential(1, n)
poids = np.exp(-y) / np.exp(-(y - 5)) # f / h
pref = np.mean((y > 5) * poids)
print(f"exact : {np.exp(-5):.6f} | naïf : {naif:.6f} | préférentiel : {pref:.6f} | poids min / max : {poids.min():.6f} / {poids.max():.6f}")
```

```
exact : 0.006738 | naïf : 0.006700 | préférentiel : 0.006738 | poids min / max : 0.006738 / 0.006738
```

**Corrigé 7.** (a) On résout  $u = 1 - \exp(-(t/\lambda)^k)$  :  $t = \lambda[-\ln(1-u)]^{1/k}$ . (b) et (c) : la médiane théorique est  $\lambda(\ln 2)^{1/k}$ .

```

from math import gamma, log
lam, k = 24, 1.5
rng = np.random.default_rng(693)
u = rng.random(100_000)
t = lam * (-np.log(1 - u)) ** (1 / k)
print(f"médiane simulée {np.median(t):.3f} | théorique {lam * log(2) ** (1 / k):.3f}")
print(f"moyenne simulée {t.mean():.3f} | théorique {lam * gamma(1 + 1 / k):.3f}")
print(f"P(T > 36) simulée {np.mean(t > 36):.4f} | théorique {np.exp(-(36 / lam) ** k):.4f}")

```

```

médiane simulée 18.892 | théorique 18.797
moyenne simulée 21.713 | théorique 21.666
P(T > 36) simulée 0.1587 | théorique 0.1593

```

**Corrigé 8.** Cible  $\pi = (0,25; 0,5; 0,25)$ . (a)  $P_{12} = \frac{1}{2} \min(1, 2) = \frac{1}{2}$ ;  $P_{13} = \frac{1}{2} \min(1, 1) = \frac{1}{2}$ ;  $P_{11} = 0$ . Depuis 2 :  $P_{21} = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$ ,  $P_{23} = \frac{1}{4}$ ,  $P_{22} = \frac{1}{2}$ . Depuis 3 :  $P_{31} = \frac{1}{2}$ ,  $P_{32} = \frac{1}{2} \min(1, 2) = \frac{1}{2}$ ,  $P_{33} = 0$ . (b)  $\pi_1 P_{12} = 0,25 \times \frac{1}{2} = 0,125 = \pi_2 P_{21} = 0,5 \times \frac{1}{4} \checkmark$ ;  $\pi_1 P_{13} = 0,125 = \pi_3 P_{31} \checkmark$ ;  $\pi_2 P_{23} = 0,5 \times \frac{1}{4} = 0,125 = \pi_3 P_{32} = 0,25 \times \frac{1}{2} \checkmark$ .

```

poids = np.array([1.0, 2.0, 1.0]); pi = poids / poids.sum(); k3 = 3
P = np.zeros((k3, k3))
for i in range(k3):
    for j in range(k3):
        if i != j:
            P[i, j] = 0.5 * min(1, poids[j] / poids[i])
        P[i, i] = 1 - P[i].sum()
print(P)
print("bilan détaillé (écart max) :", np.abs(pi[:, None] * P - (pi[:, None] * P).T).max())
rng = np.random.default_rng(694)
etat, compte = 0, np.zeros(3)
for _ in range(60_000):
    etat = rng.choice(3, p=P[etat]); compte[etat] += 1
print("fréquences simulées :", (compte / 60_000).round(3), "| cible :", pi)

```

```

[[0.  0.5  0.5 ]
 [0.25 0.5  0.25]
 [0.5  0.5  0.  ]]
bilan détaillé (écart max) : 0.0
fréquences simulées : [0.251 0.5  0.249] | cible : [0.25 0.5  0.25]

```

**Corrigé 9.** (a)  $\sum y = 25, n = 6 : \lambda \mid y \sim \text{Gamma}(2 + 25; 0,5 + 6) = \text{Gamma}(27; \text{taux } 6,5)$ , de moyenne  $27/6,5 = 4,154$ . (b) Si  $\theta = \log \lambda$ , la densité de  $\theta$  est celle de  $\lambda$  multipliée par le jacobien  $|d\lambda/d\theta| = \lambda = e^\theta$ . L'a posteriori **non normalisé** de  $\theta$  est donc  $\propto e^{(27-1)\theta} e^{-6,5e^\theta} \cdot e^\theta = e^{27\theta - 6,5e^\theta}$  :  $\log \pi(\theta) = 27\theta - 6,5e^\theta$ . **Oublier le jacobien est l'erreur classique** : on échantillonnerait alors une autre loi.

```

def log_cible(theta):
    return 27 * theta - 6.5 * np.exp(theta)

rng = np.random.default_rng(695)
chaine, taux = metropolis_ld(log_cible, x0=np.log(4.0), n=30_000, pas=0.5, rng=rng)
lam_s = np.exp(chaine[2000:])
exacte = stats.gamma(a=27, scale=1 / 6.5)
print(f"taux d'acceptation {taux:.3f}")
print(f"MH : moyenne {lam_s.mean():.4f}, écart-type {lam_s.std():.4f}, quantiles 2,5 / 50 / 97,5 %
↳ {np.percentile(lam_s, [2.5, 50, 97.5]).round(3)}")
print(f"exact : moyenne {exacte.mean():.4f}, écart-type {exacte.std():.4f}, quantiles
↳ {exacte.ppf([0.025, 0.5, 0.975]).round(3)}")

```

```

taux d'acceptation 0.420
MH : moyenne 4.1457, écart-type 0.8114, quantiles 2,5 / 50 / 97,5 % [2.712 4.076 5.874]
exact : moyenne 4.1538, écart-type 0.7994, quantiles [2.737 4.103 5.861]

```

**Corrigé 10.** (a)  $ESS = n / (1 + 2 \sum_{k \geq 1} \rho_k)$  et  $\sum_{k \geq 1} \varphi^k = \frac{\varphi}{1 - \varphi}$ , donc  $1 + 2 \frac{\varphi}{1 - \varphi} = \frac{1 + \varphi}{1 - \varphi}$  et  $ESS = n \frac{1 - \varphi}{1 + \varphi}$ . Pour  $\varphi = 0,9$  :  $n/19$ , soit environ 1 053 sur 20 000. (b), (c) :

```
def arl(phi, n, x0, rng):
    x = np.empty(n); x[0] = x0
    for t in range(1, n):
        x[t] = phi * x[t - 1] + np.sqrt(1 - phi**2) * rng.standard_normal()
    return x

rng = np.random.default_rng(696)
x = arl(0.9, 20_000, 0.0, rng)
print(f"ESS théorique {20_000 * (1 - 0.9) / (1 + 0.9):.0f} | ESS calculée {ess(x):.0f}")

chaines4 = np.array([arl(0.99, 1500, x0, rng) for x0 in (-10, -3, 3, 10)])
print(f"R-chapeau avec les 1 500 points : {split_rhat(chaines4):.3f} | sans les 300 premiers :
↳ {split_rhat(chaines4[:, 300:]):.3f}")
print(f"ESS théorique par chaîne (phi = 0,99) : {1200 * (1 - 0.99) / (1 + 0.99):.0f} sur 1 200 points")

ESS théorique 1053 | ESS calculée 1111
R-chapeau avec les 1 500 points : 1.142 | sans les 300 premiers : 1.063
ESS théorique par chaîne (phi = 0,99) : 6 sur 1 200 points
```

L'ESS calculée est proche de la théorie. Pour  $\varphi = 0,99$ , la chaîne est si lente que l'ESS d'une chaîne de 1 200 points est de l'ordre de 6 seulement : le  $\hat{R}$  **détecte le problème** (valeur nettement supérieure à 1,01, quoi qu'on fasse), parce que chaque chaîne n'a pas eu le temps de quitter son voisinage de départ. C'est exactement la situation où il faut **allonger** ou **reparamétriser**.

**Corrigé 11.** Données : les paniers des 449 acheteurs de la boutique. Sous le modèle normal avec a priori  $p(\mu, \sigma^2) \propto 1/\sigma^2$ , la loi a posteriori est :  $\sigma^2 | y \sim (n-1)s^2/\chi_{n-1}^2$ , puis  $\mu | \sigma^2, y \sim \mathcal{N}(\bar{y}, \sigma^2/n)$ , ce qui donne directement des tirages (sans MCMC).

```
clients = pd.read_csv("donnees/clients.csv")
pan = clients[(clients["canal_acquisition"] == "Boutique") & (clients["panier_moyen"] >
↳ 0)][["panier_moyen"]].to_numpy()

def ppc_normal(y_, graine, S=2000):
    rng = np.random.default_rng(graine)
    n_, ybar, s2 = len(y_), y_.mean(), y_.var(ddof=1)
    sig2 = (n_ - 1) * s2 / rng.chisquare(n_ - 1, S)
    mu = rng.normal(ybar, np.sqrt(sig2 / n_))
    yrep = rng.normal(mu[:, None], np.sqrt(sig2)[:, None], size=(S, n_))
    T = lambda a: np.c_[stats.skew(a, axis=-1), a.min(axis=-1)]
    To, Tr = T(y_[None, :])[0], T(yrep)
    return To, Tr, (Tr >= To).mean(axis=0)

for nom, donnees in (("normale sur le panier (DT)", pan), ("normale sur le log du panier",
↳ np.log(pan))):
    To, Tr, pb = ppc_normal(donnees, 697)
    print(f"{nom:32s} asymétrie : observée {To[0]:6.3f}, répliques {Tr[:, 0].mean():6.3f} (p bayésien
↳ {pb[0]:.3f}) | "
        f"minimum : observé {To[1]:7.3f}, répliques {Tr[:, 1].mean():7.3f} (p bayésien {pb[1]:.3f})")

normale sur le panier (DT)          asymétrie : observée 1.037, répliques 0.003 (p bayésien 0.000) |
↳ minimum : observé 25.530, répliques -11.756 (p bayésien 0.000)
normale sur le log du panier        asymétrie : observée 0.046, répliques 0.003 (p bayésien 0.360) |
↳ minimum : observé 3.240, répliques 3.094 (p bayésien 0.144)
```

Le modèle sur le panier brut échoue : les données ont une asymétrie de 1,04 (queue à droite) que le modèle symétrique ne reproduit jamais (asymétrie répliquée : 0,00 ;  $p_B = 0$ ), et il prédit des **paniers négatifs** (le minimum répliqué vaut en moyenne -11,8 DT, ce qui est absurde), alors

que le plus petit panier observé est de 25,5 DT. Le modèle sur le logarithme (c'est-à-dire une loi log-normale pour le panier) passe les deux vérifications ( $p_B = 0,36$  pour l'asymétrie et 0,14 pour le minimum). C'est la raison pour laquelle on modélise les montants positifs sur une échelle logarithmique.

**Corrigé 12.**  $n = 10, y = 9. p(y | H_0) = \binom{10}{9} 0,5^{10} = 10/1024 = 0,00977 ; p(y | H_1) = 1/11 = 0,0909.$

$BF_{10} = \frac{1/11}{10/1024} = \frac{1024}{110} = 9,31$  : des données environ 9 fois plus probables sous  $H_1$ . La p-valeur

exacte bilatérale est  $2[\binom{10}{9} + \binom{10}{10}]/1024 = 22/1024 = 0,0215$ . Avec une cote a priori de  $\frac{1/4}{3/4} = \frac{1}{3}$

en faveur de  $H_1$ , la cote a posteriori est  $9,31 \times \frac{1}{3} = 3,10$ , soit une probabilité  $3,10/4,10 = 0,756$  pour  $H_1$  : on a gagné en crédibilité mais on est loin de la certitude, alors que la p-valeur de 0,02 « rejette  $H_0$  au seuil de 5 % ». C'est la différence de langage entre « rejeter » et « mettre à jour ses croyances ».

```
from scipy.special import comb
m0 = comb(10, 9) * 0.5**10
m1 = 1 / 11
bf = m1 / m0
print(f"p(y|H0) = {m0:.5f} | p(y|H1) = {m1:.5f} | BF10 = {bf:.3f} (à la main : 1024/110 = {1024 / 110:.3f})")
print(f"p-valeur exacte bilatérale : {stats.binomtest(9, 10, 0.5).pvalue:.4f} (à la main : 22/1024 = {22 / 1024:.4f})")
cote_post = bf * (0.25 / 0.75)
print(f"cote a posteriori {cote_post:.3f} -> P(H1 | données) = {cote_post / (1 + cote_post):.3f}")
```

$p(y|H_0) = 0.00977$  |  $p(y|H_1) = 0.09091$  |  $BF_{10} = 9.309$  (à la main :  $1024/110 = 9.309$ )

p-valeur exacte bilatérale : 0.0215 (à la main :  $22/1024 = 0.0215$ )

cote a posteriori 3.103 ->  $P(H_1 | \text{données}) = 0.756$

**Corrigé 13.** On utilise  $x_m = u + \frac{\sigma_u}{\xi} [(m\zeta_u)^\xi - 1]$ . (a)  $m = 1\,000 : m\zeta_u = 50, 50^{0,2} = e^{0,2 \ln 50} =$

$e^{0,7824} = 2,187 ; x = 5 + \frac{2}{0,2}(2,187 - 1) = 5 + 10 \times 1,187 = 16,87$  jours. (b)  $m = 10\,000 :$

$m\zeta_u = 500, 500^{0,2} = e^{0,2 \ln 500} = 3,466$ , donc  $x = 5 + 10 \times 2,466 = 29,66$  jours. (c) Avec  $\xi = 0,4 :$

$x_{10\,000} = 5 + \frac{2}{0,4}(500^{0,4} - 1) = 5 + 5 \times (12,01 - 1) = 60,1$  jours. **Doubler  $\xi$  double (à peu près)**

**le niveau de retour à 10 000 colis : une petite erreur sur l'indice de queue a une conséquence énorme sur l'extrapolation.**

```
def niveau_pot(m, xi, sc, u, zeta):
    return u + sc / xi * ((m * zeta) ** xi - 1)

for xi_ in (0.2, 0.4):
    print(f"xi = {xi_} : " + " | ".join(f"1 colis sur {m:>6,}: {niveau_pot(m, xi_, 2.0, 5.0, 0.05):6.2f} jours".replace(",", " ") for m in (1_000, 10_000)))
```

xi = 0.2 : 1 colis sur 1 000: 16.87 jours | 1 colis sur 10 000: 29.66 jours

xi = 0.4 : 1 colis sur 1 000: 23.91 jours | 1 colis sur 10 000: 60.06 jours

**Corrigé 14.** (a)  $P(V \leq q | U \leq q) = \frac{C_\theta(q, q)}{q} = \frac{(2q^{-\theta} - 1)^{-1/\theta}}{q}$ . Or  $(2q^{-\theta} - 1)^{-1/\theta} = [q^{-\theta}(2 -$

$q^\theta)]^{-1/\theta} = q(2 - q^\theta)^{-1/\theta}$ ; en divisant par  $q : (2 - q^\theta)^{-1/\theta}$ . Quand  $q \rightarrow 0, q^\theta \rightarrow 0$  (car  $\theta > 0$ ) et l'on obtient  $2^{-1/\theta}$ .  $\square$  (c) Pour la Clayton retournée,  $(U, V) = (1 - U', 1 - V')$  avec  $(U', V')$  de Clayton :

$P(U > 1 - q, V > 1 - q) = P(U' < q, V' < q) = C_\theta(q, q) = (2q^{-\theta} - 1)^{-1/\theta}$ . Avec  $\theta = 2$  ( $\tau = 0,5, \rho = \sin(\pi/4) = 0,7071$  pour la gaussienne), on compare.

```
from scipy.stats import norm, multivariate_normal
theta, rho = 2.0, np.sin(np.pi / 4)
```

```

lignes = []
for q in (0.1, 0.01, 0.001):
    cond = (2 - q**theta) ** (-1 / theta)
    c_qq = (2 * q ** (-theta) - 1) ** (-1 / theta)
    gauss = multivariate_normal([0, 0], [[1, rho], [rho, 1]]).cdf([norm.ppf(q), norm.ppf(q)])
    lignes.append({"q": q, "P(V<=q | U<=q) Clayton": round(cond, 4), "limite 2^(-1/theta)": round(2 **
↪ (-1 / theta), 4),
                  "P(U>1-q, V>1-q) Clayton retournée": f"{c_qq:.3e}", "gaussienne (même tau)":
↪ f"{gauss:.3e}",
                  "rapport": round(c_qq / gauss, 1)})
print(pd.DataFrame(lignes).to_string(index=False))

```

| q           | P(V<=q   U<=q) Clayton | limite 2 <sup>^(-1/theta)</sup> | P(U>1-q, V>1-q) Clayton retournée | gaussienne (même tau) |
|-------------|------------------------|---------------------------------|-----------------------------------|-----------------------|
| 0.100       | 0.7089                 | 0.7071                          | 7.089e-02                         |                       |
| ↪ 4.739e-02 | 1.5                    |                                 |                                   |                       |
| 0.010       | 0.7071                 | 0.7071                          | 7.071e-03                         |                       |
| ↪ 2.735e-03 | 2.6                    |                                 |                                   |                       |
| 0.001       | 0.7071                 | 0.7071                          | 7.071e-04                         |                       |
| ↪ 1.654e-04 | 4.3                    |                                 |                                   |                       |

La probabilité conditionnelle de Clayton converge vers  $2^{-1/2} = 0,707$  (la limite  $\lambda$ ), tandis que celle de la copule gaussienne tend vers 0 quand  $q \rightarrow 0$ . Le rapport entre les probabilités conjointes de la Clayton retournée et de la gaussienne **croît sans cesse** à mesure que  $q$  diminue : c'est la signature de la dépendance de queue.

## Bilan du chapitre 6

Vous savez maintenant :

- **raisonner à la bayésienne** : a posteriori  $\propto$  vraisemblance  $\times$  a priori ; utiliser les **lois conjuguées** (bêta-binomiale, gamma-Poisson, normale-normale) pour obtenir des résultats exacts par simple addition ; lire et interpréter un **intervalle de crédibilité** ; formuler des phrases comme « *la probabilité que l'offre soit utile est de...* » ;
- **choisir et justifier un a priori**, mesurer son influence par une **analyse de sensibilité**, et savoir qu'il s'efface quand les données sont abondantes (Bernstein-von Mises) ;
- **estimer par simulation** (Monte-Carlo) : une espérance, une probabilité, une intégrale ; connaître la vitesse en  $1/\sqrt{n}$  ; fabriquer des tirages par **inversion** ou **rejet** ; utiliser l'**échantillonnage préférentiel** pour les événements rares, avec son **ESS**, et les **techniques de réduction de variance** ;
- **simuler une loi a posteriori** non conjuguée avec une chaîne de Markov : **Metropolis-Hastings** (bilan détaillé, réglage du pas, taux d'acceptation) et **Gibbs** (lois conditionnelles complètes), puis calculer *tout* (rapports de cotes, effets sur les probabilités, prédictions) par de simples moyennes ;
- **vérifier ses modèles** : plusieurs chaînes dispersées, traces,  $\hat{R}$ , ESS ; vérifications prédictives **a priori** et **a posteriori** ; comparaison par facteur de Bayes (et ses limites : sensibilité à l'a priori, paradoxe de Lindley) et par WAIC ;
- (en option) **modéliser les extrêmes** par la loi GEV et les excès au-delà d'un seuil (GPD), estimer des **niveaux de retour** avec leur incertitude ;
- (en option) **modéliser la dépendance** par les **copules**, comprendre la différence entre corrélation et dépendance de queue, et pourquoi le choix de la copule décide du risque de **catastrophes simultanées**.

Les deux fils conducteurs de ce chapitre sont l'**incertitude** (qu'il faut quantifier et propager

jusqu'à la décision) et **la vérification** (un modèle, un a priori ou un algorithme ne sont jamais acceptés sur parole). Ces deux idées ne s'arrêtent pas aux méthodes bayésiennes : elles guident tout le reste du métier.

Le prochain chapitre, optionnel, aborde l'**inférence causale** : comment passer de « *deux variables varient ensemble* » à « *agir sur l'une changera l'autre* », et ce qu'il faut pour que cette inférence soit légitime. Les sections 6.1 et 6.4 (a priori, vérification de modèle) en seront un outil précieux.

# Chapitre 7 : ⊕ Inférence causale

« Les données vous disent ce qui *accompagne* quoi. Pour savoir ce qui *provoque* quoi, il faut en plus une idée de la façon dont les données ont été fabriquées. »

**Chapitre complémentaire.** Ce chapitre est entièrement facultatif : les chapitres 1 à 6 se lisent et se comprennent sans lui. Mais si vous ne devez retenir qu'un chapitre « de métier » de ce volume, c'est peut-être celui-ci : c'est lui qui transforme un modèle statistique correct en **décision correcte**.

Tout au long des chapitres précédents, nous avons posé des questions de **prédiction** et d'**association** : « les clients d'Instagram dépensent-ils moins ? », « quel est le lien entre l'âge et le rachat ? ». Les modèles linéaires et généralisés y répondent très bien. Mais la question que se pose réellement Yasmine, le plus souvent, est d'un autre genre :

- « Si j'envoie une offre de bienvenue à un client, **va-t-il** davantage racheter ? »
- « Si je lance une campagne publicitaire à Sfax, **les commandes vont-elles** augmenter ? »
- « Si mes clients suivent mon compte Instagram, **dépensent-ils plus à cause de cela**, ou est-ce simplement que ce sont déjà mes meilleurs clients qui me suivent ? »

Ce sont des questions **causales** : elles portent sur ce qui se passerait si l'on **agissait**. Et la leçon la plus importante de ce chapitre est la suivante : **aucun modèle, même sophistiqué, ne répond à une question causale à partir des seules données. Il faut y ajouter des hypothèses sur la façon dont ces données ont été produites.** Le chapitre vous apprend à les formuler (avec des graphes), à les justifier (avec la randomisation, quand on le peut) et à les exploiter (quand on ne le peut pas).

## Le chemin de ce chapitre

- **7.1 Raisonner en causes** : les résultats potentiels (le « problème fondamental »), pourquoi la **randomisation** résout tout, une vraie expérience sur l'offre de bienvenue, puis les **graphes causaux** (DAG) et leurs trois structures de base (fourche, chaîne, collision), le paradoxe de Simpson, et le critère de la porte dérobée.
- **7.2 Scores de propension** : quand on ne peut pas randomiser mais que l'on observe les raisons du choix : appariement, pondération, estimateur doublement robuste.
- **7.3 Différences de différences** : une campagne lancée dans certaines villes seulement ; comparer l'évolution des villes traitées à celle des villes témoins.
- **7.4 Variables instrumentales** : quand la confusion n'est **pas observée**, un « coup de pouce » aléatoire peut sauver l'analyse ; avec un panorama honnête des limites de toutes ces méthodes.
- **7.5 Exercices corrigés.**



**Comment travailler avec ce chapitre.** Il repose sur un procédé très particulier : **nous simulons nous-mêmes les données**. Le gros avantage est que nous connaissons la **vraie** valeur de chaque effet, et nous pouvons donc vérifier honnêtement si chaque méthode la retrouve. Dans la vraie vie, ce ne sera jamais le cas : c'est précisément pour cela que le raisonnement causal est difficile. À chaque méthode, nous commencerons donc par faire comme si nous ne connaissions pas la vérité, puis nous la **révélerons**. Gardez bien en tête que cette simulation est une loupe pédagogique, pas une garantie : une méthode qui retrouve la vérité dans une simulation propre n'est pas pour autant infaillible sur des données réelles.

#### Les données de ce chapitre.

- `donnees/clients.csv` (2 000 clients, présenté dans l'introduction du volume) : on y trouve la variable `offre_bienvenue`, **attribuée au hasard**. C'est une vraie expérience randomisée (section 7.1.4).
- `donnees/ch07-observationnel.csv` (4 000 clients) : une version **non randomisée** de la même histoire, où Yasmine a choisi à qui envoyer l'offre (section 7.1.9 et 7.2). Le fichier `ch07-observationnel-verite.csv` contient les « résultats potentiels » que personne ne peut observer en pratique ; nous ne l'ouvrons que pour vérifier.
- `donnees/ch07-panel-villes.csv` : 20 villes suivies pendant 24 mois autour d'une campagne publicitaire (section 7.3).
- `donnees/ch07-iv.csv` (5 000 clients) : une étude du lien entre le suivi du compte Instagram et la dépense (section 7.4).

Les fichiers `ch07-*` sont produits par `build/sim_ch07.py`. Le code de chaque simulation est imprimé dans la section correspondante (7.3 pour le panel de villes, 7.4 pour l'instrument) et exécuté pour vérifier qu'il redonne exactement le fichier ; celui de l'étude observationnelle se trouve dans `build/sim_ch07.py` (fonction `observationnel`).

**Une convention de notation.** Dans tout le chapitre,  $T$  désigne le **traitement** (ce que l'on décide : 1 = offre reçue, 0 = pas d'offre),  $Y$  le **résultat observé** (ce que l'on mesure : dépense, rachat...) et  $X$  les **covariables** (ce que l'on sait du client avant le traitement : âge, canal, engagement...). Dans le volume I, les sections 3.3 à 3.5 (intervalles de confiance, tests, tests multiples) sont les prérequis statistiques ; le chapitre 2 de ce volume (section 2.2, régression logistique) est utilisé pour les scores de propension.

## 7.1 Raisonner en causes : résultats potentiels, expérience randomisée, DAG et confusion

**Intuition.** Dire « l'offre de bienvenue **augmente** la dépense » signifie : *pour le même client, au même moment*, la dépense avec offre est supérieure à la dépense sans offre. Le problème, c'est que **ce même client ne peut pas recevoir et ne pas recevoir l'offre en même temps**. L'une des deux dépenses n'existera jamais. Toute la pensée causale consiste à compenser, par une astuce (la randomisation) ou par une hypothèse (le raisonnement sur un graphe), cette moitié de réalité qui nous manque.

### 7.1.1 Une conclusion trop rapide

Voici l'histoire qui motive tout le chapitre. Yasmine a envoyé, pendant un an, une offre de bienvenue (un bon d'achat) à une partie de ses nouveaux clients. En comparant les dépenses, elle constate que les clients qui ont reçu l'offre dépensent **beaucoup plus** que les autres. Conclusion immédiate : « l'offre rapporte, je l'envoie à tout le monde ».

Avant de croire ce raisonnement, regardons ce qu'il a de fragile. Yasmine n'a pas envoyé l'offre au hasard : elle l'a envoyée à ceux qui lui semblaient les plus prometteurs, ceux qui ouvrent ses e-mails, qui aiment ses publications, qui sont déjà très actifs. Ce sont **ces clients-là** qui dépensent beaucoup, avec ou sans bon d'achat. La comparaison « offre contre pas d'offre » compare donc des clients **différents dès le départ**, et pas seulement à cause de l'offre. Nous allons mettre des chiffres sur cette intuition.

### 7.1.2 Les résultats potentiels

Le langage standard pour parler de causalité est celui des **résultats potentiels** (cadre de Neyman-Rubin). Pour chaque client  $i$ , on imagine deux nombres :

- $Y_i(1)$  : la dépense du client  $i$  **s'il reçoit** l'offre ;
- $Y_i(0)$  : la dépense du client  $i$  **s'il ne la reçoit pas**.

L'**effet causal individuel** de l'offre sur le client  $i$  est  $\tau_i = Y_i(1) - Y_i(0)$ . Un seul des deux résultats est observé, celui qui correspond au traitement réellement reçu  $T_i \in \{0, 1\}$  :

$$Y_i = T_i Y_i(1) + (1 - T_i) Y_i(0).$$

C'est le **problème fondamental de l'inférence causale** (Holland, 1986) : pour chaque client, l'une des deux colonnes est *toujours manquante*. Les effets individuels sont inobservables. On vise donc des **effets moyens** :

- **ATE** (*average treatment effect*) :  $\mathbb{E}[Y(1) - Y(0)]$ , l'effet moyen sur *toute* la population ;
- **ATT** (*on the treated*) :  $\mathbb{E}[Y(1) - Y(0) \mid T = 1]$ , l'effet moyen sur ceux qui ont **effectivement** reçu l'offre ;
- **ATU** (*on the untreated*) :  $\mathbb{E}[Y(1) - Y(0) \mid T = 0]$ , l'effet moyen sur ceux qui ne l'ont pas reçue.

**Une hypothèse cachée : la SUTVA.** Écrire  $Y_i(1)$  et  $Y_i(0)$  suppose deux choses : (1) il n'y a **qu'une seule version** du traitement (un bon d'achat de 10 DT, pas « un bon de 5 DT ou de 20 DT selon les cas ») ; (2) le résultat du client  $i$  ne dépend **pas** du traitement des autres (pas de contagion : si votre voisin reçoit l'offre et vous en parle, la formulation se complique). Dans tout ce chapitre, nous supposons que ces deux conditions sont raisonnablement satisfaites, et nous reviendrons sur la deuxième à propos des campagnes par ville (7.3).

Pour **voir** le problème, jouons à Dieu. Voici huit clients dont nous connaissons, exceptionnellement, les **deux** dépenses potentielles. (Ce tableau est inventé de toutes pièces, et calculable à la main.)

```
import itertools
import numpy as np
import pandas as pd

clients = pd.DataFrame({
    "client": ["Amel", "Bilel", "Chaima", "Dorra", "Ehsan", "Farah", "Ghofrane", "Hichem"],
    "y0": [100, 150, 180, 130, 50, 80, 60, 90],      # dépense SANS offre (DT)
    "y1": [120, 165, 190, 145, 60, 85, 75, 100],    # dépense AVEC offre (DT)
    "offre": [1, 1, 1, 1, 0, 0, 0, 0],              # ce que Yasmine a décidé
})
clients["effet"] = clients["y1"] - clients["y0"]
print("Le tableau vu par Dieu :")
print(clients.to_string(index=False))
print()
print("Le tableau vu par Yasmine (une moitié du tableau manque toujours) :")
```

```
vu = clients.assign(y0=clients["y0"].where(clients["offre"] == 0),
                    y1=clients["y1"].where(clients["offre"] == 1))
print(vu[["client", "offre", "y0", "y1"]].to_string(index=False))
```

Le tableau vu par Dieu :

| client   | y0  | y1  | offre | effet |
|----------|-----|-----|-------|-------|
| Amel     | 100 | 120 | 1     | 20    |
| Bilel    | 150 | 165 | 1     | 15    |
| Chaima   | 180 | 190 | 1     | 10    |
| Dorra    | 130 | 145 | 1     | 15    |
| Ehsan    | 50  | 60  | 0     | 10    |
| Farah    | 80  | 85  | 0     | 5     |
| Ghofrane | 60  | 75  | 0     | 15    |
| Hichem   | 90  | 100 | 0     | 10    |

Le tableau vu par Yasmine (une moitié du tableau manque toujours) :

| client   | offre | y0   | y1    |
|----------|-------|------|-------|
| Amel     | 1     | NaN  | 120.0 |
| Bilel    | 1     | NaN  | 165.0 |
| Chaima   | 1     | NaN  | 190.0 |
| Dorra    | 1     | NaN  | 145.0 |
| Ehsan    | 0     | 50.0 | NaN   |
| Farah    | 0     | 80.0 | NaN   |
| Ghofrane | 0     | 60.0 | NaN   |
| Hichem   | 0     | 90.0 | NaN   |

Calculons à la main ce que Dieu sait. Les effets individuels sont +20, +15, +10, +15 pour les quatre clients qui ont reçu l'offre (**ATT** = 60/4 = 15 DT), et +10, +5, +15, +10 pour les quatre autres (**ATU** = 40/4 = 10 DT). Sur les huit, l'**ATE** vaut 100/8 = 12,5 DT. Voyons-le au calcul, puis comparons avec ce que fait Yasmine : la différence des dépenses moyennes observées.

```
ate = clients["effet"].mean()
att = clients.loc[clients["offre"] == 1, "effet"].mean()
atu = clients.loc[clients["offre"] == 0, "effet"].mean()
print(f"ATE = {ate}    ATT = {att}    ATU = {atu}")

observe = np.where(clients["offre"] == 1, clients["y1"], clients["y0"])
moy_offre = observe[clients["offre"] == 1].mean()
moy_sans = observe[clients["offre"] == 0].mean()
print(f"\nDépense moyenne observée avec offre : {moy_offre}    sans offre : {moy_sans}")
print(f"Différence naïve (ce que calcule Yasmine) : {moy_offre - moy_sans}")
```

ATE = 12.5 ATT = 15.0 ATU = 10.0

Dépense moyenne observée avec offre : 155.0 sans offre : 70.0  
Différence naïve (ce que calcule Yasmine) : 85.0

La comparaison naïve donne **85 DT**, alors que l'effet réel de l'offre n'est que de 15 DT pour ceux qui l'ont reçue ! D'où vient l'écart ? Il se démontre en deux lignes.

**La décomposition du biais de sélection.** Écrivons  $\mu_1 = \mathbb{E}[Y \mid T = 1]$  et  $\mu_0 = \mathbb{E}[Y \mid T = 0]$  les moyennes observées. Chez les traités, on observe  $Y(1)$  ; chez les non-traités,  $Y(0)$ . Donc

$$\mu_1 - \mu_0 = \mathbb{E}[Y(1) \mid T = 1] - \mathbb{E}[Y(0) \mid T = 0].$$

En ajoutant et retranchant  $\mathbb{E}[Y(0) \mid T = 1]$  (ce que les traités auraient dépensé **sans** l'offre, une quantité inobservable) :

$$\underbrace{\mu_1 - \mu_0}_{\text{différence naïve}} = \underbrace{\mathbb{E}[Y(1) - Y(0) \mid T = 1]}_{\text{ATT : l'effet qui nous intéresse}} + \underbrace{\mathbb{E}[Y(0) \mid T = 1] - \mathbb{E}[Y(0) \mid T = 0]}_{\text{biais de sélection}}.$$

Le **biais de sélection** mesure à quel point les traités et les non-traités auraient été **différents même sans le traitement**.

Dans notre petit exemple, les clients choisis par Yasmine auraient dépensé en moyenne  $(100 + 150 + 180 + 130)/4 = 140$  DT *sans* offre, contre  $(50 + 80 + 60 + 90)/4 = 70$  DT pour les autres : biais de sélection = 70 DT. Et  $15 + 70 = 85$  : la décomposition retombe sur la différence naïve.

```
y0_traites = clients.loc[clients["offre"] == 1, "y0"].mean()
y0_non_traites = clients.loc[clients["offre"] == 0, "y0"].mean()
print("Sans offre, les traités auraient dépensé :", y0_traites, "| les non-traités dépensent :",
      ↪ y0_non_traites)
print("Biais de sélection :", y0_traites - y0_non_traites)
print("ATT + biais =", att + (y0_traites - y0_non_traites), "= différence naïve", moy_offre -
      ↪ moy_sans)
```

```
Sans offre, les traités auraient dépensé : 140.0 | les non-traités dépensent : 70.0
Biais de sélection : 70.0
ATT + biais = 85.0 = différence naïve 85.0
```

**Retenir cette équation.** Elle explique tous les résultats trompeurs de ce chapitre : une différence entre groupes est la somme d'un effet **causal** et d'un **biais de sélection**. Tout l'art est de faire disparaître le second terme, ou de l'estimer.

### 7.1.3 Pourquoi la randomisation résout tout

Qu'est-ce qui ferait disparaître le biais de sélection ? Il faudrait que, **sans** le traitement, les deux groupes se ressemblent en moyenne :  $\mathbb{E}[Y(0) | T = 1] = \mathbb{E}[Y(0) | T = 0]$ . Or il existe une façon de **garantir** cela : tirer au sort qui reçoit l'offre. Si l'attribution est aléatoire, elle est **indépendante** de tout ce qui caractérise les clients, y compris de leurs résultats potentiels :  $T \perp (Y(0), Y(1))$ . On a alors  $\mathbb{E}[Y(0) | T = 1] = \mathbb{E}[Y(0) | T = 0] = \mathbb{E}[Y(0)]$ , et le biais de sélection est **nul** :

$$\mu_1 - \mu_0 = \mathbb{E}[Y(1)] - \mathbb{E}[Y(0)] = \text{ATE}.$$

La différence des moyennes, bête et simple, est alors un estimateur **sans biais** de l'effet moyen. Aucun modèle, aucune hypothèse sur la forme de la relation n'est nécessaire : c'est la force de la randomisation.

Sur nos huit clients, on peut **vérifier** cette affirmation exhaustivement. Il y a  $\binom{8}{4} = 70$  façons de choisir les quatre clients qui reçoivent l'offre ; si Yasmine tire au sort l'une d'elles avec la même probabilité, la moyenne des 70 différences naïves possibles doit retomber sur l'ATE.

```
estimations = []
for groupe_offre in itertools.combinations(range(8), 4):
    T = np.zeros(8, dtype=int)
    T[list(groupe_offre)] = 1
    y = np.where(T == 1, clients["y1"], clients["y0"])
    estimations.append(y[T == 1].mean() - y[T == 0].mean())
estimations = np.array(estimations)
print(len(estimations), "attributions possibles")
print("moyenne des 70 différences naïves :", estimations.mean(), " | ATE réel :", ate)
print("plus petite / plus grande :", estimations.min(), "/", estimations.max())
```

```
70 attributions possibles
moyenne des 70 différences naïves : 12.5 | ATE réel : 12.5
plus petite / plus grande : -60.0 / 85.0
```

La moyenne vaut exactement l'ATE : **en moyenne sur les tirages possibles**, l'estimateur est juste.

Mais une expérience particulière n'est qu'un tirage parmi les 70, et celui-ci peut tomber très loin de la vérité (les 70 tirages donnent des estimations de  $-60$  à  $+85$  DT : avec seulement huit clients, on peut même obtenir le mauvais signe) : c'est pourquoi on accompagne toujours l'estimation d'un intervalle de confiance, comme au volume I (section 3.3).

Passons à l'échelle d'une vraie clientèle. Le fichier `ch07-observationnel.csv` contient 4 000 clients dont Yasmine a **ciblé** l'offre, et `ch07-observationnel-verite.csv` leurs deux dépenses potentielles (information que, dans la vraie vie, personne n'a). Rejouons l'histoire de deux façons : avec le ciblage réel de Yasmine, et avec des attributions tirées au sort.

```
obs = pd.read_csv("donnees/ch07-observationnel.csv")
verite = pd.read_csv("donnees/ch07-observationnel-verite.csv")
d = obs.merge(verite, on="id_client")
ate_vrai = (d["y1"] - d["y0"]).mean()
att_vrai = (d["y1"] - d["y0"])[d["offre"] == 1].mean()
print(f"{len(d)} clients ; {d['offre'].mean():.1%} ont reçu l'offre")
print(f"ATE vrai = {ate_vrai:.2f} DT   ATT vrai = {att_vrai:.2f} DT")

naif = d.loc[d["offre"] == 1, "depense"].mean() - d.loc[d["offre"] == 0, "depense"].mean()
print(f"Différence naïve avec le ciblage de Yasmine : {naif:.2f} DT")

rng = np.random.default_rng(1)
diffs_alea = []
for _ in range(2000):
    T = rng.permutation(d["offre"].to_numpy())           # même proportion de traités, mais tirés au
    ↪ sort
    y = np.where(T == 1, d["y1"], d["y0"])
    diffs_alea.append(y[T == 1].mean() - y[T == 0].mean())
diffs_alea = np.array(diffs_alea)
print(f"Avec attribution aléatoire : moyenne {diffs_alea.mean():.2f}, écart-type
    ↪ {diffs_alea.std():.2f}")
print(f" 95 % des tirages entre {np.percentile(diffs_alea, 2.5):.1f} et {np.percentile(diffs_alea,
    ↪ 97.5):.1f}")
```

```
4000 clients ; 46.2% ont reçu l'offre
ATE vrai = 15.53 DT   ATT vrai = 16.64 DT
Différence naïve avec le ciblage de Yasmine : 50.50 DT
Avec attribution aléatoire : moyenne 15.46, écart-type 2.26
 95 % des tirages entre 11.1 et 19.9
```

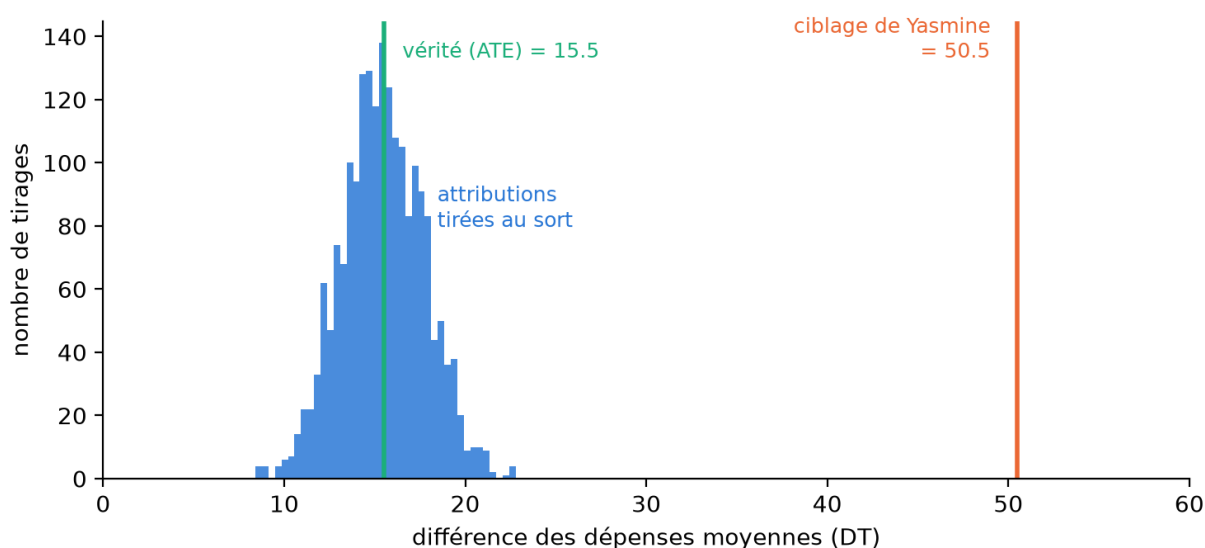


Figure 70: Distribution de la différence naïve quand l'attribution est tirée au sort (2 000 tirages), comparée à la valeur obtenue avec le ciblage de Yasmine et à la vérité.

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
ENCRE, GRIS = "#0b0b0b", "#898781"

fig, ax = plt.subplots(figsize=(8.5, 3.6))
ax.hist(diffs_alea, bins=40, color=BLEU, alpha=0.85)
ax.axvline(ate_vrai, color=AQUA, lw=2)
ax.axvline(naif, color=ORANGE, lw=2)
ymax = ax.get_ylim()[1]
ax.text(ate_vrai + 1, ymax * 0.92, f"vérité (ATE) = {ate_vrai:.1f}", color=AQUA, fontsize=9)
ax.text(naif - 1.5, ymax * 0.92, f"ciblage de Yasmine\n= {naif:.1f}", color=ORANGE, fontsize=9,
       ha="right")
ax.text(diffs_alea.mean() + 3, ymax * 0.55, "attributions\ntirées au sort", color=BLEU, fontsize=9)
ax.set_xlabel("différence des dépenses moyennes (DT)")
ax.set_ylabel("nombre de tirages")
ax.set_xlim(0, 60)
ax.grid(axis="x", visible=False)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
plt.savefig("figures/ch07-randomisation.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Les 2 000 attributions aléatoires se répartissent **autour de la vérité** (en bleu), avec un écart-type d'environ 2,3 DT (95 % des tirages tombent entre 11,1 et 19,9). Le ciblage de Yasmine, lui, donne un résultat très éloigné, **bien en dehors** de cette distribution : ce n'est pas un hasard d'échantillonnage, c'est un **biais** systématique.

**À retenir (7.1.2 et 7.1.3).** Un effet causal compare **deux mondes**, dont un seul est observé. Une différence entre groupes = effet causal + biais de sélection. La **randomisation** annule le biais de sélection *par construction*, sans modèle.

### 7.1.4 Une vraie expérience : l'offre de bienvenue

Bonne nouvelle : pour l'un de ses lancements, Yasmine a **vraiment** tiré au sort. Dans `clients.csv`, la colonne `offre_bienvenue` a été attribuée par pile ou face à chacun des 2 000 clients. Analysons cette expérience comme le ferait un data scientist, en trois temps : vérifier la randomisation, estimer l'effet, interpréter.

**Étape 1 : la randomisation a-t-elle bien « marché » ?** Une randomisation équilibre les groupes *en moyenne* ; sur un échantillon fini, un déséquilibre est possible. On le contrôle avec un tableau d'équilibre. Pour comparer des variables d'unités différentes, on utilise la **différence moyenne standardisée** (SMD) : l'écart des moyennes divisé par l'écart-type typique,

$$\text{SMD} = \frac{\bar{x}_1 - \bar{x}_0}{\sqrt{(s_1^2 + s_0^2)/2}},$$

et l'on considère en pratique qu'une valeur inférieure à 0,1 en valeur absolue est un bon équilibre.

```

from scipy import stats

c = pd.read_csv("donnees/clients.csv")
traite = c["offre_bienvenue"] == 1]

```

```

temoin = c[c["offre_bienvenue"] == 0]
print("effectifs : offre =", len(traite), "| pas d'offre =", len(temoin))

def smd(x1, x0):
    return (x1.mean() - x0.mean()) / np.sqrt((x1.var(ddof=1) + x0.var(ddof=1)) / 2)

lignes = [("age", smd(traite["age"], temoin["age"]), traite["age"].mean(), temoin["age"].mean())]
for modalite in ["Instagram", "Site", "Boutique"]:
    u1 = (traite["canal_acquisition"] == modalite).astype(float)
    u0 = (temoin["canal_acquisition"] == modalite).astype(float)
    lignes.append((f"canal = {modalite}", smd(u1, u0), u1.mean(), u0.mean()))
for v in sorted(c["ville"].unique()):
    u1 = (traite["ville"] == v).astype(float)
    u0 = (temoin["ville"] == v).astype(float)
    lignes.append((f"ville = {v}", smd(u1, u0), u1.mean(), u0.mean()))
equilibre = pd.DataFrame(lignes, columns=["variable", "SMD", "moy. offre", "moy. témoin"]).round(3)
print(equilibre.to_string(index=False))
print()
print("p-valeur (Welch) pour l'âge :", round(stats.ttest_ind(traite["age"], temoin["age"],
    ↪ equal_var=False).pvalue, 3))
print("p-valeur (khi-deux) pour le canal :",
    ↪ round(stats.chi2_contingency(pd.crosstab(c["canal_acquisition"], c["offre_bienvenue"]))[1], 3))
print("p-valeur (khi-deux) pour la ville :", round(stats.chi2_contingency(pd.crosstab(c["ville"],
    ↪ c["offre_bienvenue"]))[1], 3))

```

```

effectifs : offre = 1015 | pas d'offre = 985
      variable      SMD  moy. offre  moy. témoin
      age -0.068      35.397      36.114
canal = Instagram  0.020      0.413      0.403
      canal = Site   0.029      0.347      0.333
canal = Boutique -0.054      0.240      0.264
      ville = Autre -0.021      0.147      0.154
      ville = Bizerte 0.016      0.106      0.102
      ville = Nabeul -0.011      0.124      0.128
      ville = Sfax   -0.012      0.139      0.143
      ville = Sousse -0.011      0.167      0.171
      ville = Tunis  0.032      0.317      0.303

```

```

p-valeur (Welch) pour l'âge : 0.128
p-valeur (khi-deux) pour le canal : 0.473
p-valeur (khi-deux) pour la ville : 0.976

```

Toutes les SMD sont bien inférieures à 0,1 : les deux groupes se ressemblent sur tout ce que nous observons. Les tests de significativité sont ici **secondaires** : le tirage au sort a été fait, donc toute différence est par construction due au hasard, et il est inutile de « tester » l'hypothèse que le hasard est hasardeux. (Si l'on testait pourtant des dizaines de variables à 5 %, on s'attendrait à trouver quelques « différences significatives » par pur hasard : c'est le problème des comparaisons multiples du volume I, section 3.5.5.)

**Étape 2 : estimer l'effet.** Pour le rachat à 12 mois (variable binaire), l'estimateur de l'effet moyen est la différence de proportions  $\hat{p}_1 - \hat{p}_0$ , et son intervalle de confiance de Wald est celui de la section 3.4.5 du volume I :  $\hat{p}_1 - \hat{p}_0 \pm 1,96\sqrt{\hat{p}_1(1 - \hat{p}_1)/n_1 + \hat{p}_0(1 - \hat{p}_0)/n_0}$ .

```

n1, n0 = len(traite), len(temoin)
p1, p0 = traite["rachat_12m"].mean(), temoin["rachat_12m"].mean()
ate_rachat = p1 - p0
se = np.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
print(f"rachat avec offre : {p1:.3f} sans offre : {p0:.3f}")
print(f"effet moyen : {ate_rachat:.3f} (IC 95 % : {ate_rachat - 1.96 * se:.3f} ; {ate_rachat + 1.96 *
    ↪ se:.3f})")
print(f"effet relatif : {ate_rachat / p0:+.1%} | une offre de plus = {ate_rachat:.3f} rachat de plus
    ↪ en moyenne,")

```



```
print(f"soit environ 1 client de plus qui rachète pour {1 / ate_rachat:.1f} offres envoyées")
```

```
rachat avec offre : 0.569   sans offre : 0.448
effet moyen : 0.122   (IC 95 % : 0.078 ; 0.165)
effet relatif : +27.2%   | une offre de plus = 0.122 rachat de plus en moyenne,
soit environ 1 client de plus qui rachète pour 8.2 offres envoyées
```

Même question avec un modèle logistique (chapitre 2, section 2.2). Attention : le coefficient de la régression est un **rapport de cotes** (échelle logarithmique), pas une différence de probabilités. Pour retrouver une différence de probabilités, on calcule l'**effet marginal moyen** :

```
import statsmodels.formula.api as smf

logit = smf.logit("rachat_12m ~ offre_bienvenue", data=c).fit(dis=0)
print("coefficient (log-cote) :", round(logit.params["offre_bienvenue"], 3), "| rapport de cotes :",
      ↪ round(np.exp(logit.params["offre_bienvenue"]), 2))
marg = logit.get_margeff(dummy=True).summary_frame().iloc[0]      # dummy=True : vraie différence de
      ↪ probabilités 1 - 0
print(f"effet marginal moyen : {marg['dy/dx']:.3f}   (IC 95 % : {marg['Conf. Int. Low']:.3f} ;
      ↪ {marg['Conf. Int. Hi.']: .3f})")
```

```
coefficient (log-cote) : 0.49 | rapport de cotes : 1.63
effet marginal moyen : 0.122   (IC 95 % : 0.078 ; 0.165)
```

Les deux approches coïncident (c'est normal : avec une seule variable binaire explicative, le modèle logistique est « saturé » et reproduit exactement les deux proportions ; l'option `dummy=True` demande la vraie différence de probabilités entre  $T = 1$  et  $T = 0$ , et non une dérivée). Pour la dépense annuelle, variable continue et asymétrique, on compare les moyennes avec le test de Welch du volume I (section 3.4.3) :

```
d1, d0 = traite("depense_annuelle"), temoin("depense_annuelle")
res = stats.ttest_ind(d1, d0, equal_var=False)
ic = res.confidence_interval(0.95)
print(f"dépense moyenne avec offre : {d1.mean():.1f} DT   sans offre : {d0.mean():.1f} DT")
print(f"effet moyen : {d1.mean() - d0.mean():+.1f} DT   (IC 95 % : {ic.low:.1f} ; {ic.high:.1f})   p =
      ↪ {res.pvalue:.2f}")
```

```
dépense moyenne avec offre : 243.5 DT   sans offre : 250.5 DT
effet moyen : -7.0 DT   (IC 95 % : -33.1 ; 19.0)   p = 0.60
```

On peut aussi **ajuster** l'estimation sur des covariables. Dans une expérience randomisée, ce n'est pas pour corriger un biais (il n'y en a pas) mais pour gagner en **précision** : les covariables qui expliquent la dépense réduisent le bruit résiduel.

```
simple = smf.ols("rachat_12m ~ offre_bienvenue", data=c).fit(cov_type="HC1")
ajuste = smf.ols("rachat_12m ~ offre_bienvenue + age + C(canal_acquisition) + C(ville)",
      ↪ data=c).fit(cov_type="HC1")
for nom, m in [("sans covariables", simple), ("avec covariables", ajuste)]:
    b, s = m.params["offre_bienvenue"], m.bse["offre_bienvenue"]
    print(f"{nom} : effet = {b:.3f}   erreur-type = {s:.4f}   IC 95 % = [{b - 1.96 * s:.3f} ; {b +
      ↪ 1.96 * s:.3f}])")
```

```
sans covariables : effet = 0.122   erreur-type = 0.0222   IC 95 % = [0.078 ; 0.165]
avec covariables : effet = 0.121   erreur-type = 0.0221   IC 95 % = [0.077 ; 0.164]
```

**Étape 3 : interpréter, et révéler la vérité.** L'offre augmente de façon nette la probabilité de rachat, de l'ordre de 12 points de pourcentage (57 % de rachat avec l'offre, contre 45 % sans), alors qu'on ne détecte **aucun effet sur la dépense annuelle** (l'intervalle de confiance contient très largement 0). Comme les données sont simulées, nous pouvons comparer à la vérité. Dans le modèle de simulation, l'offre augmente de 0,55 la log-cote du rachat et n'intervient pas du tout dans le

nombre de commandes ni dans le panier. L'effet moyen en probabilité se calcule en moyennant sur la population simulée la différence  $\text{expit}(\eta + 0,55) - \text{expit}(\eta)$  :

```
rng = np.random.default_rng(1)
N = 400_000
age = np.clip(np.round(rng.normal(36, 11, N)), 18, 75)
canal = rng.choice(["Instagram", "Site", "Boutique"], N, p=[0.40, 0.35, 0.25])
z = rng.normal(size=(N, 2))
F1 = z[:, 0] # facteurs latents du simulateur (corrélation 0,3)
F2 = 0.3 * z[:, 0] + np.sqrt(1 - 0.3 ** 2) * z[:, 1]
eta = -0.35 + 0.45 * F1 + 0.35 * F2 - 0.015 * (age - 36) + 0.3 * (canal == "Boutique")
expit = lambda x: 1 / (1 + np.exp(-x))
print(f"effet moyen vrai sur la probabilité de rachat : {(expit(eta + 0.55) -
↳ expit(eta)).mean():.4f}")
print("effet vrai sur la dépense annuelle : 0 (par construction du simulateur)")
```

```
effet moyen vrai sur la probabilité de rachat : 0.1239
effet vrai sur la dépense annuelle : 0 (par construction du simulateur)
```

L'estimation expérimentale (0,122) est très proche de la vérité (0,124), et l'intervalle de confiance la contient. Notez aussi ce que l'expérience **ne dit pas** : elle donne l'effet *moyen* ; elle ne dit pas pour qui l'offre marche le mieux, ni *pourquoi* elle marche. Et si l'on fouille dix sous-groupes à la recherche d'un effet, on retombe dans le piège des tests multiples.

**Absence de preuve n'est pas preuve d'absence.** L'intervalle de confiance de l'effet sur la dépense va de  $-33$  à  $+19$  DT environ, pour une dépense moyenne d'environ 247 DT : il exclut un effet massif, mais pas un effet de quelques dizaines de dinars (une dizaine de pour cent de la dépense), dans un sens ou dans l'autre. Ce que l'on peut dire honnêtement : « cette expérience n'a pas détecté d'effet sur la dépense annuelle ». Ici, nous savons que l'effet est réellement nul ; dans la vraie vie, on ne le saurait pas. Ce qui compte pour la décision, c'est la **largeur** de l'intervalle.

**À retenir (7.1.4).** Une expérience randomisée s'analyse simplement : tableau d'équilibre, différence de moyennes, intervalle de confiance. L'ajustement sur covariables, facultatif, améliore la précision. Tout le reste de ce chapitre sert à *approcher* ce résultat quand le tirage au sort n'a pas été possible.

### 7.1.5 Quand on ne peut pas tirer au sort : les graphes causaux

Beaucoup de questions causales ne se prêtent pas à une expérience : on ne peut pas choisir au hasard qui vit à Sfax, ni refaire le passé. Il faut alors **raisonner** sur la façon dont les données ont été produites. L'outil standard est le **graphe orienté acyclique** (DAG, de l'anglais *directed acyclic graph*, popularisé par Judea Pearl) : chaque variable est un **nœud**, et une **flèche**  $A \rightarrow B$  signifie «  $A$  a une influence causale directe sur  $B$  ». Il est acyclique : aucune variable ne peut être sa propre cause en suivant les flèches.

Un graphe est une **hypothèse sur le monde**, pas une conclusion tirée des données. Mais cette hypothèse est explicite, discutable, et elle détermine très précisément **quelles variables il faut ajuster, et lesquelles il ne faut surtout pas ajuster**. Trois structures élémentaires suffisent à comprendre tous les graphes.

```
def dag(ax, positions, aretes, titre="", pointilles=(), styles=None, xlim=(-0.5, 2.5), ylim=(-0.6,
↳ 1.4)):
    """Dessine un DAG : boîtes arrondies, et flèches qui s'arrêtent exactement au bord des boîtes."""
    styles = styles or {}
    ax.set_xlim(*xlim)
```

```

ax.set_ylim(*ylim)
ax.axis("off")
ax.set_title(titre, fontsize=10.5)
fig = ax.figure
rendu = fig.canvas.get_renderer()
demi = {} # demi-largeur et demi-hauteur de chaque
↳ boîte, en points
for nom, (x, y) in positions.items():
    t = ax.text(x, y, nom, ha="center", va="center", fontsize=10, zorder=3,
               bbox=dict(boxstyle="round,pad=0.4", fc="#cde2fb", ec=BLEU, lw=1.2))
    e = t.get_window_extent(rendu)
    demi[nom] = (e.width * 72 / fig.dpi / 2 + 6, e.height * 72 / fig.dpi / 2 + 6)
for a, b in aretes:
    pa, pb = ax.transData.transform(positions[a]), ax.transData.transform(positions[b])
    dx, dy = (pb - pa) * 72 / fig.dpi # direction de la flèche, en points
    longueur = np.hypot(dx, dy)

    def bord(nom): # distance du centre au bord de la boîte, le
        ↳ long de la flèche
        w, h = demi[nom]
        return min(w / abs(dx) if dx else np.inf, h / abs(dy) if dy else np.inf) * longueur + 2

    couleur, trait = styles.get((a, b), (ENCRE, 1.5))
    ax.annotate("", xy=positions[b], xytext=positions[a], zorder=2,
               arrowprops=dict(arrowstyle="->", color=couleur, lw=trait, shrinkA=bord(a),
        ↳ shrinkB=bord(b),
               linestyle="--" if (a, b) in pointilles else "-"))

fig, axes = plt.subplots(1, 3, figsize=(11, 3.1))
fig.subplots_adjust(left=0.01, right=0.99, wspace=0.05, top=0.88) # mise en page fixée AVANT de
↳ dessiner
dag(axes[0], {"Offre": (0, 0.4), "Code utilisé": (1, 0.4), "Dépense": (2, 0.4)},
    [("Offre", "Code utilisé"), ("Code utilisé", "Dépense")], "Chaîne : médiateur")
dag(axes[1], {"Engagement": (1, 1.0), "Offre": (0, 0.0), "Dépense": (2, 0.0)},
    [("Engagement", "Offre"), ("Engagement", "Dépense"), ("Offre", "Dépense")], "Fourche : confusion",
    pointilles={"Offre", "Dépense"})
dag(axes[2], {"Qualité": (0, 1.0), "Attrait": (2, 1.0), "Retenu au catalogue": (1, 0.0)},
    [("Qualité", "Retenu au catalogue"), ("Attrait", "Retenu au catalogue")], "Collision : collider")
plt.savefig("figures/ch07-dag-structures.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

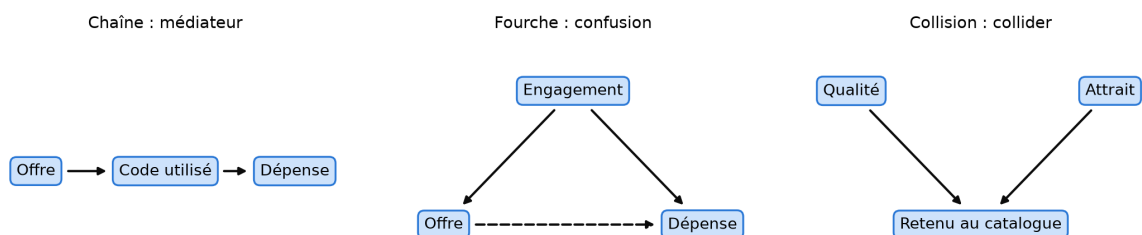


Figure 71: Les trois structures élémentaires d'un graphe causal : la chaîne (la cause agit à travers un médiateur), la fourche (une cause commune crée une association), la collision (deux causes d'un même effet).

- **La chaîne**  $A \rightarrow M \rightarrow B$  :  $A$  agit sur  $B$  à travers  $M$  (le médiateur). Exemple : l'offre incite à utiliser le code, qui fait dépenser.  $A$  et  $B$  sont associés ; si l'on **fige**  $M$ , l'association disparaît

(la voie est bloquée).

- **La fourche**  $A \leftarrow C \rightarrow B$  :  $C$  est une **cause commune** (un *facteur de confusion*). Elle crée une association entre  $A$  et  $B$  **sans** qu'aucune des deux n'agisse sur l'autre. Exemple : l'engagement pousse Yasmine à envoyer l'offre *et* fait dépenser. Si l'on **fige**  $C$ , l'association disparaît.
- **La collision**  $A \rightarrow K \leftarrow B$  :  $K$  est un **effet commun** (un *collider*).  $A$  et  $B$  sont **indépendants**, et ne deviennent associés que si l'on **fige**  $K$  : une surprise à retenir, que nous illustrons plus bas.

Dans les deux premiers cas, « figer » une variable la rend inoffensive ; dans le troisième, c'est au contraire **la figer qui crée le problème**. C'est pourquoi on ne peut pas se contenter de la règle « ajustons sur tout ce que l'on a ». Pour chaque structure, nous allons maintenant **simuler** le phénomène et regarder ce que fait la régression.

### 7.1.6 La fourche : confusion et paradoxe de Simpson

Reprenons des chiffres à la main. Yasmine a envoyé l'offre à 120 clients et pas à 120 autres, et observe le rachat à 12 mois. Les clients viennent de deux canaux : la **boutique** (clients très fidèles, taux de rachat élevé) et **Instagram** (clients plus volatils). Yasmine a envoyé l'offre surtout à des clients d'Instagram, ceux qu'elle « avait envie de convaincre ».

| Canal        | Offre envoyée | Clients    | Rachats   | Taux          |
|--------------|---------------|------------|-----------|---------------|
| Boutique     | oui           | 20         | 18        | 90 %          |
| Boutique     | non           | 100        | 80        | 80 %          |
| Instagram    | oui           | 100        | 40        | 40 %          |
| Instagram    | non           | 20         | 6         | 30 %          |
| <b>Total</b> | <b>oui</b>    | <b>120</b> | <b>58</b> | <b>48,3 %</b> |
| <b>Total</b> | <b>non</b>    | <b>120</b> | <b>86</b> | <b>71,7 %</b> |

Lisez les deux dernières lignes : **globalement**, les clients qui ont reçu l'offre rachètent *moins* (48 % contre 72 %). Mais regardez chaque canal : en boutique, l'offre fait passer le taux de 80 % à 90 % ; sur Instagram, de 30 % à 40 %. L'offre **améliore** le rachat de **10 points dans chaque canal**. Comment le total peut-il dire l'inverse ? Parce que l'offre a été envoyée surtout au canal où l'on rachète peu : les « traités » sont majoritairement des clients d'Instagram, qui auraient peu racheté de toute façon. C'est le **paradoxe de Simpson**, qui n'est un paradoxe que si l'on oublie la fourche  $Canal \rightarrow Offre, Canal \rightarrow Rachat$ .

Quelle est alors la bonne réponse ? Celle qui **compare à canal égal**, puis fait la moyenne. L'effet moyen (ATE) se calcule par **standardisation** : on pondère l'effet dans chaque canal par la part de ce canal dans toute la population (ici 120 clients sur 240 de chaque canal, soit 50 %) :  $0,5 \times 10 \% + 0,5 \times 10 \% = 10$  points. Vérifions au calcul, puis avec une régression logistique.

```
simpson = pd.DataFrame({
    "canal": ["Boutique", "Boutique", "Instagram", "Instagram"],
    "offre": [1, 0, 1, 0],
    "clients": [20, 100, 100, 20],
    "rachats": [18, 80, 40, 6],
})
simpson["taux"] = simpson["rachats"] / simpson["clients"]
glob = simpson.groupby("offre")[["clients", "rachats"]].sum()
print("global :", (glob["rachats"] / glob["clients"]).round(3).to_dict())

par_canal = simpson.pivot(index="canal", columns="offre", values="taux")
par_canal["effet"] = par_canal[1] - par_canal[0]
poids = simpson.groupby("canal")["clients"].sum() / simpson["clients"].sum()
print(par_canal.round(3))
```

```

print("poids des canaux :", poids.round(2).to_dict())
print("effet standardisé :", round((par_canal["effet"] * poids).sum(), 3))

# La même chose avec une régression logistique sur les 240 clients (une ligne par client)
lignes = pd.DataFrame([{"canal": r.canal, "offre": r.offre, "rachat": int(k < r.rachats)}
                        for r in simpson.itertuples() for k in range(r.clients)])
print(len(lignes), "clients, taux de rachat global :", round(lignes["rachat"].mean(), 3))
m_brut = smf.logit("rachat ~ offre", lignes).fit(dis=0)
m_ajuste = smf.logit("rachat ~ offre + C(canal)", lignes).fit(dis=0)
print(f"\ncoefficient de l'offre SANS ajustement sur le canal : {m_brut.params['offre']:+.2f}")
print(f"coefficient de l'offre AVEC ajustement sur le canal : {m_ajuste.params['offre']:+.2f}")

global : {0: 0.717, 1: 0.483}
offre      0      1  effet
canal
Boutique   0.8  0.9    0.1
Instagram  0.3  0.4    0.1
poids des canaux : {'Boutique': 0.5, 'Instagram': 0.5}
effet standardisé : 0.1
240 clients, taux de rachat global : 0.6

coefficient de l'offre SANS ajustement sur le canal : -0.99
coefficient de l'offre AVEC ajustement sur le canal : +0.56

```

Le signe change : sans ajustement, le modèle conclut que l'offre est **nuisible** ; avec le canal, qui est la cause commune, il retrouve un effet **positif**. Morale : la bonne analyse **dépend de l'histoire causale**, pas seulement des chiffres. Les mêmes données, avec une histoire différente, pourraient exiger de ne *pas* ajuster : nous l'illustrons tout de suite.

**Le paradoxe de Simpson n'est pas une bizarrerie arithmétique.** Les mêmes chiffres, lus avec deux histoires causales différentes, appellent deux conclusions opposées. Si le canal était une **conséquence** de l'offre (disons que l'offre fait venir des clients d'un autre canal), il ne faudrait *pas* ajuster dessus. C'est pourquoi aucun test statistique ne peut décider seul s'il faut ou non ajuster.

### 7.1.7 La chaîne : ne pas ajuster sur un médiateur

Autre cas : l'offre agit à **travers** un médiateur. Simulons une expérience où l'offre est **randomisée**, et où elle agit de deux façons : par l'utilisation du code promotionnel (effet de 30 DT quand le code est utilisé), et par un petit effet direct de « bonne image » (8 DT). Les clients les plus motivés (variable motivation, qui influence aussi la dépense) utilisent plus souvent le code.

```

rng = np.random.default_rng(71)
n = 100_000
motivation = rng.normal(size=n)
offre = rng.integers(0, 2, n)
p_code = expit(0.4 + 0.9 * motivation)
↳ l'offre
code_si_offre = rng.random(n) < p_code
code = offre * code_si_offre
bruit = rng.normal(0, 25, n)
depense = 100 + 8 * offre + 30 * code + 20 * motivation + bruit

# inobservée dans la vie réelle
# randomisée
# un code n'existe que si l'on a reçu
# médiateur : 1 si offre ET code utilisé

# Effet total (par résultats potentiels, avec le même bruit) :
y1 = 100 + 8 + 30 * code_si_offre + 20 * motivation + bruit
y0 = 100 + 20 * motivation + bruit
print(f"effet total vrai : {(y1 - y0).mean():.2f} DT    (= 8 + 30 x {code_si_offre.mean():.2f}, où
↳ {code_si_offre.mean():.0%} des clients utilisent le code s'ils reçoivent l'offre)")

df_m = pd.DataFrame({"depense": depense, "offre": offre, "code": code})

```

```
total = smf.ols("depense ~ offre", df_m).fit()
sur_ajuste = smf.ols("depense ~ offre + code", df_m).fit()
print(f"sans ajuster sur le médiateur : effet de l'offre = {total.params['offre']:.2f} (erreur-type
↳ {total.bse['offre']:.2f})")
print(f"en ajustant sur le médiateur : effet de l'offre = {sur_ajuste.params['offre']:.2f}
↳ (erreur-type {sur_ajuste.bse['offre']:.2f})")

# Pourquoi ? Parmi les clients SANS code, les traités et les non-traités n'ont pas la même motivation :
m_traites_sans_code = motivation[(offre == 1) & (code == 0)].mean()
m_temoins = motivation[offre == 0].mean()
print(f"motivation moyenne, offre reçue mais code non utilisé : {m_traites_sans_code:+.2f} | pas
↳ d'offre : {m_temoins:+.2f}")
print(f"écart de dépense dû à cette seule différence de motivation : {20 * (m_traites_sans_code -
↳ m_temoins):+.1f} DT")
```

```
effet total vrai : 25.47 DT (= 8 + 30 x 0.58, où 58% des clients utilisent le code s'ils reçoivent
↳ l'offre)
sans ajuster sur le médiateur : effet de l'offre = 25.62 (erreur-type 0.22)
en ajustant sur le médiateur : effet de l'offre = -0.80 (erreur-type 0.26)
motivation moyenne, offre reçue mais code non utilisé : -0.45 | pas d'offre : -0.00
écart de dépense dû à cette seule différence de motivation : -9.0 DT
```

Sans ajustement, la régression retrouve l'**effet total** (25,6 DT estimés pour 25,5 vrais) (le chiffre que Yasmine cherche : « que rapporte l'envoi d'une offre ? »). En ajoutant le médiateur, on obtient -0,8 DT : un chiffre qui n'est **ni l'effet total** (on a retiré la voie par le code), **ni l'effet direct** de 8 DT que l'on pourrait croire avoir isolé. Pourquoi ? En comparant, parmi les clients sans code, ceux qui ont reçu l'offre à ceux qui ne l'ont pas reçue, on compare des clients **peu motivés** (ceux qui n'ont pas utilisé le code malgré l'offre) à des clients **de motivation moyenne** (tous ceux qui n'ont pas reçu d'offre) : la dernière sortie montre cet écart de motivation, qui à lui seul fait baisser la dépense d'environ 9 DT et masque presque exactement les 8 DT de l'effet direct. Nous avons ouvert, sans le vouloir, un chemin biaisé. Retenez la règle d'or : **n'ajustez jamais sur une variable qui est affectée par le traitement** (variable « post-traitement »), sauf si l'on cherche explicitement un effet direct *et* que l'on sait justifier l'absence de confusion entre médiateur et résultat.

### 7.1.8 La collision : ne pas conditionner sur un effet commun

Dernier cas, le plus surprenant. Dar Jasmin garde au catalogue les prototypes de produits qui ont une bonne **qualité** *ou* un grand **attrait** visuel (ou les deux) : un produit à la fois laid et fragile est abandonné. Imaginons que, dans la réalité, qualité et attrait sont **parfaitement indépendants** : savoir qu'un prototype est beau ne dit rien sur sa solidité.

```
rng = np.random.default_rng(72)
n = 6000
qualite = rng.normal(size=n)
attrait = rng.normal(size=n) # indépendants par construction
score = qualite + attrait + rng.normal(0, 0.5, n)
retenu = score > 0.8 # seuls les prototypes réussis sont gardés

print(f"corrélacion qualite-attrait, tous les prototypes : {np.corrcoef(qualite, attrait)[0,
↳ 1]:+.3f}")
print(f"corrélacion qualite-attrait, produits retenus : {np.corrcoef(qualite[retenu],
↳ attrait[retenu])[0, 1]:+.3f}")
print(f"({retenu.sum()} produits retenus sur {n})")

# Ce que ferait un analyste qui n'observe QUE le catalogue :
cat = pd.DataFrame({"qualite": qualite, "attrait": attrait, "retenu": retenu})
brut = smf.ols("qualite ~ attrait", cat).fit()
dans_catalogue = smf.ols("qualite ~ attrait", cat[cat.retenu]).fit()
print(f"pente de la qualite sur l'attrait, tous : {brut.params['attrait']:+.3f} | catalogue seulement
↳ : {dans_catalogue.params['attrait']:+.3f}")
```



```

corrélation qualité-attrait, tous les prototypes : +0.011
corrélation qualité-attrait, produits retenus      : -0.479
(1811 produits retenus sur 6000)
pente de la qualité sur l'attrait, tous : +0.012 | catalogue seulement : -0.494

```

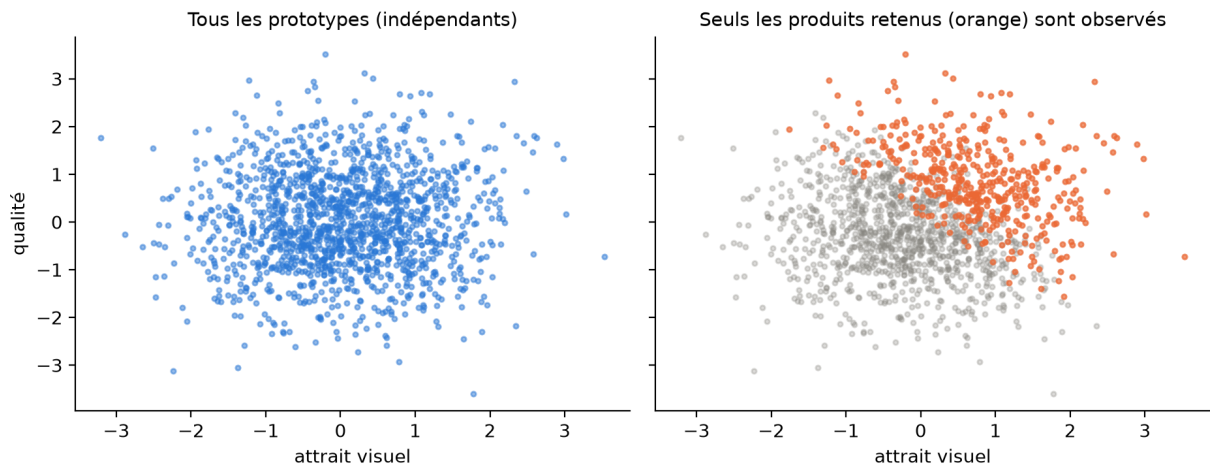


Figure 72: À gauche, tous les prototypes : aucune relation entre qualité et attrait. À droite, seuls les produits retenus au catalogue : une relation négative apparaît.

```

fig, axes = plt.subplots(1, 2, figsize=(9.5, 3.8), sharex=True, sharey=True)
idx = rng.choice(n, 1500, replace=False)
axes[0].scatter(attrait[idx], qualite[idx], s=7, color=BLEU, alpha=0.5)
axes[0].set_title("Tous les prototypes (indépendants)", fontsize=10.5)
r = retenu[idx]
axes[1].scatter(attrait[idx][~r], qualite[idx][~r], s=7, color=GRIS, alpha=0.3)
axes[1].scatter(attrait[idx][r], qualite[idx][r], s=7, color=ORANGE, alpha=0.7)
axes[1].set_title("Seuls les produits retenus (orange) sont observés", fontsize=10.5)
for ax in axes:
    ax.set_xlabel("attrait visuel")
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
axes[0].set_ylabel("qualité")
plt.tight_layout()
plt.savefig("figures/ch07-collider.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Parmi les 1 811 produits **retenus** sur 6 000, la corrélation est de  $-0,48$ , alors qu'elle est nulle ( $+0,01$ ) sur l'ensemble : un produit moins beau doit, pour être gardé, être plus solide, et réciproquement. Aucun lien réel n'existe pourtant entre les deux. L'effet est dû uniquement au **filtre** : on a conditionné sur un effet commun. Ce mécanisme est partout : on ne voit que les clients qui ont **répondu** à l'enquête (tous ne le font pas), les entreprises qui ont **survécu**, les patients **hospitalisés**. Quand on n'observe qu'une population sélectionnée par un effet commun, on crée des associations fantômes. Cette situation s'appelle aussi un **biais de sélection** (au sens des graphes), et on ne peut pas la corriger en ajoutant des variables : il faut éviter de conditionner dessus, ou modéliser la sélection.

**À retenir (7.1.5 à 7.1.8).** Un graphe causal rend explicites les hypothèses. **Fourche** : ajuster sur la cause commune (c'est la correction du biais de confusion). **Chaîne** : ne pas ajuster sur le médiateur si l'on veut l'effet total. **Collision** : ne pas conditionner sur un effet commun. Aucune de ces règles ne se lit dans les données : elles viennent du **raisonnement**.



### 7.1.9 Le critère de la porte dérobée

Ces trois structures sont les briques d'une règle générale, le **critère de la porte dérobée** (*back-door criterion*). Dans un graphe, un **chemin** entre  $T$  et  $Y$  est une suite de flèches reliant les deux, quel que soit leur sens. On distingue :

- les **chemins directs**  $T \rightarrow \dots \rightarrow Y$  (toutes les flèches vont dans le sens de  $T$  vers  $Y$ ) : ce sont eux qui portent l'effet causal ;
- les **chemins « porte dérobée »** : ceux qui **commencent par une flèche entrant dans  $T$**  ( $T \leftarrow \dots$ ) ; ils créent une association non causale (la confusion).

Un ensemble de variables  $S$  **satisfait le critère de la porte dérobée** pour estimer l'effet de  $T$  sur  $Y$  si : (i) aucune variable de  $S$  n'est un **descendant** de  $T$  (pas de variable post-traitement) ; (ii)  $S$  **bloque** tous les chemins porte dérobée entre  $T$  et  $Y$ . Un chemin est bloqué par  $S$  s'il contient une chaîne ou une fourche dont le nœud central est dans  $S$ , ou une collision dont ni le nœud central, ni aucun descendant, n'est dans  $S$ .

**Ce que garantit le critère.** Si  $S$  satisfait le critère, alors, à **valeur de  $S$  fixée**, le traitement est comme tiré au sort :  $Y(t) \perp T \mid S$  (c'est l'hypothèse d'**ignorabilité conditionnelle**). Alors  $\mathbb{E}[Y(t) \mid S = s] = \mathbb{E}[Y \mid T = t, S = s]$ , et en moyennant sur la population on obtient la **formule d'ajustement** :

$$\mathbb{E}[Y(t)] = \sum_s \mathbb{E}[Y \mid T = t, S = s] \mathbb{P}(S = s), \quad \text{ATE} = \sum_s (\mathbb{E}[Y \mid T = 1, S = s] - \mathbb{E}[Y \mid T = 0, S = s]) \mathbb{P}(S = s)$$

C'est exactement la standardisation que nous avons faite à la main pour le paradoxe de Simpson (avec  $S = \text{le canal}$ ). Si  $S$  contient des variables continues, le même principe s'écrit avec des intégrales, et on le met en œuvre par régression, appariement ou pondération.

Appliquons cela au cas d'étude de la suite : l'offre de bienvenue **ciblée** par Yasmine (fichier `ch07-observationnel.csv`). Nous supposons le graphe suivant : l'âge et le canal influencent l'engagement ; l'âge, le canal **et** l'engagement influencent à la fois la décision d'envoyer l'offre (c'est ainsi que Yasmine choisit) et la dépense ; l'offre influence la dépense.

```
fig, ax = plt.subplots(figsize=(8, 3.6))
fig.subplots_adjust(left=0.01, right=0.99, top=0.99, bottom=0.01)
pos = {"Âge": (0, 1.0), "Canal": (0, -0.5), "Engagement": (1.5, 0.25),
       "Offre": (3, 1.0), "Dépense": (3, -0.5)}
aretes = [("Âge", "Engagement"), ("Canal", "Engagement"), ("Âge", "Offre"), ("Canal", "Offre"),
          ("Engagement", "Offre"), ("Âge", "Dépense"), ("Canal", "Dépense"), ("Engagement", "Dépense"),
          ("Offre", "Dépense")]
dag(ax, pos, aretes, styles={"Offre", "Dépense": (ORANGE, 2.6)}, xlim=(-0.6, 3.9), ylim=(-1.0, 1.5))
ax.text(3.15, 0.25, "effet à estimer", color=ORANGE, fontsize=9, va="center")
plt.savefig("figures/ch07-dag-etude.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Les chemins porte dérobée de *Offre* vers *Dépense* passent tous par l'âge, le canal ou l'engagement (par exemple  $\text{Offre} \leftarrow \text{Engagement} \rightarrow \text{Dépense}$ ). L'ensemble  $S = \{\text{âge, canal, engagement}\}$  bloque tous, et ne contient aucun descendant de l'offre : il satisfait le critère. Un ensemble incomplet, comme  $\{\text{âge, canal}\}$ , laisse ouvert le chemin par l'engagement. Mettons-le à l'épreuve en ajustant par régression de façon **progressive**, et en comparant avec la vérité (`ate_vrai`, calculée plus haut avec les résultats potentiels).

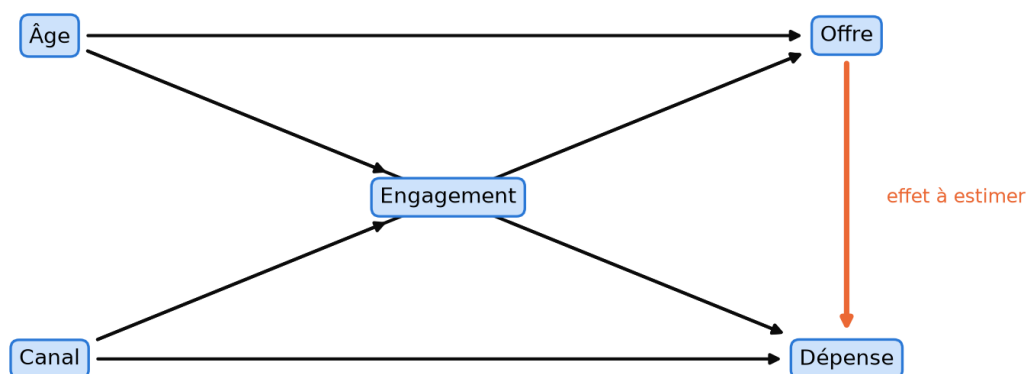


Figure 73: Le graphe supposé de l'étude observationnelle : l'âge, le canal et l'engagement sont des causes communes de l'offre et de la dépense.

```

modeles = [
    ("aucun ajustement", "depense ~ offre"),
    ("+ âge", "depense ~ offre + age"),
    ("+ âge + canal", "depense ~ offre + age + C(canal)"),
    ("+ âge + canal + engagement", "depense ~ offre + age + C(canal) + engagement"),
]
lignes = []
for nom, formule in modeles:
    m = smf.ols(formule, d).fit()
    b, s = m.params["offre"], m.bse["offre"]
    lignes.append({"ensemble d'ajustement": nom, "effet estimé": b, "IC95 bas": b - 1.96 * s, "IC95
    haut": b + 1.96 * s})
tab = pd.DataFrame(lignes).round(1)
print(tab.to_string(index=False))
print(f"\nvérité : ATE = {ate_vrai:.1f}    ATT = {att_vrai:.1f}")
  
```

| ensemble d'ajustement      | effet estimé | IC95 bas | IC95 haut |
|----------------------------|--------------|----------|-----------|
| aucun ajustement           | 50.5         | 46.3     | 54.7      |
| + âge                      | 46.1         | 41.9     | 50.3      |
| + âge + canal              | 47.0         | 42.8     | 51.2      |
| + âge + canal + engagement | 14.8         | 11.0     | 18.7      |

vérité : ATE = 15.5    ATT = 16.6

Voilà la leçon en une table : tant que l'**engagement** manque, l'estimation reste entre 46 et 51 DT, **plus de trois fois** la vérité (15,5 DT), et ajouter l'âge et le canal, variables pourtant « sensées », n'y change presque rien. Dès que l'engagement est inclus, l'estimation tombe près de la vérité et son intervalle de confiance (de 11,0 à 18,7) contient la vérité (15,5). Le **bon ensemble** n'est pas « le plus grand possible » mais celui qui bloque les chemins de confusion.

**Deux pièges de cet exemple.** (1) Le critère suppose que l'on a **mesuré** tous les facteurs de confusion. Ici, l'engagement est observé ; s'il ne l'était pas, aucun ajustement ne pourrait corriger le biais (c'est le sujet de 7.4). (2) Dans une régression linéaire, le coefficient de l'offre est une moyenne **pondérée** des effets individuels : lorsque l'effet varie d'un client à l'autre (ici, il est plus fort sur Instagram), elle ne coïncide pas exactement avec l'ATE. C'est une raison,

parmi d'autres, d'utiliser les méthodes de la section 7.2 qui visent explicitement l'ATE ou l'ATT.

**À retenir (7.1).** (1) Effet causal = comparaison de deux mondes, dont un seul est observé. (2) Différence observée = effet causal + biais de sélection. (3) La **randomisation** supprime le biais de sélection. (4) Sans randomisation, on s'appuie sur un **graphe** et le critère de la porte dérobée : ajuster sur les causes communes, **pas** sur les médiateurs ni sur les effets communs. (5) Aucun ajustement ne corrige une confusion **non mesurée**.

## 7.2 Scores de propension : comparer ce qui est comparable

**Intuition.** Yasmine n'a pas tiré au sort ; elle a **choisi** à qui envoyer l'offre. Mais en observant comment elle a choisi (engagement, âge, canal), on peut reconstituer, pour chaque client, la **probabilité** qu'il ait reçu l'offre : son *score de propension*. Deux clients qui avaient la **même** probabilité de recevoir l'offre, dont l'un l'a reçue et l'autre non, sont presque comme deux clients tirés au sort : la différence entre eux, c'est la chance. Le score de propension résume en **un seul nombre** tout ce qui a guidé le choix.

Nous poursuivons l'étude de la section 7.1.9 : le fichier `ch07-observationnel.csv`, 4 000 clients, et le critère de la porte dérobée, satisfait par l'ensemble  $S = \{\text{âge, canal, engagement}\}$ . L'ajustement par régression y a donné une estimation proche de la vérité. Pourquoi aller plus loin ? Pour trois raisons : la régression suppose une **forme** particulière pour le lien entre covariables et résultat ; elle ne dit rien sur le **chevauchement** entre traités et non-traités (peut-on réellement comparer ces clients ?) ; et elle ne permet pas de cibler explicitement l'ATE ou l'ATT.

### 7.2.1 Pourquoi un score ? La malédiction de la dimension

L'idée naturelle serait de comparer à l'**identique** : pour chaque combinaison d'âge, de canal et d'engagement, comparer les clients qui ont reçu l'offre à ceux qui ne l'ont pas reçue. Voyons ce que cela donne en pratique.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
ENCRE, GRIS = "#0b0b0b", "#898781"

obs = pd.read_csv("donnees/ch07-observationnel.csv")
verite = pd.read_csv("donnees/ch07-observationnel-verite.csv")
d = obs.merge(verite, on="id_client")
ate_vrai = (d["y1"] - d["y0"]).mean()
att_vrai = (d["y1"] - d["y0"])[d["offre"] == 1].mean()

cellules = d.groupby(["age", "canal", "engagement"])["offre"].agg(["size", "sum"])
mixtes = cellules[(cellules["sum"] > 0) & (cellules["sum"] < cellules["size"])]
print(f"{len(d)} clients répartis en {len(cellules)} cellules (âge x canal x engagement)")
print(f"cellules où l'on trouve à la fois un client avec offre et un sans : {len(mixtes)}")
print(f"clients se trouvant dans une telle cellule : {int(mixtes['size'].sum())} sur {len(d)}")
```

4000 clients répartis en 2923 cellules (âge x canal x engagement)  
cellules où l'on trouve à la fois un client avec offre et un sans : 413

clients se trouvant dans une telle cellule : 1035 sur 4000

Avec seulement trois covariables, les 4 000 clients se dispersent en 2 923 cellules, et **seules 413 d'entre elles** contiennent à la fois un client avec offre et un client sans offre : 1 035 clients, soit un peu plus d'un quart, sont comparables « à l'identique ». Les trois quarts restants n'ont aucun jumeau de l'autre groupe. Avec dix ou vingt covariables, ce serait sans espoir. C'est la **malédiction de la dimension**. Le **score de propension** de Rosenbaum et Rubin (1983) l'évite :

$$e(x) = \mathbb{P}(T = 1 \mid X = x).$$

**Pourquoi un nombre suffit : le théorème du score d'équilibrage.** Faisons deux affirmations.

(1) **Le score équilibre les covariables.** Par définition,  $\mathbb{P}(T = 1 \mid X) = e(X)$ , qui est une fonction de  $X$ . Donc, une fois  $e(X)$  fixé,  $X$  n'apporte plus aucune information supplémentaire sur  $T$  :  $\mathbb{P}(T = 1 \mid X, e(X)) = e(X) = \mathbb{P}(T = 1 \mid e(X))$ , c'est-à-dire  $T \perp X \mid e(X)$ . Parmi les clients qui ont le **même score**, ceux qui ont reçu l'offre et les autres ont la **même distribution de covariables**.

(2) **Si l'ignorabilité tient avec  $X$ , elle tient avec  $e(X)$ .** Supposons  $T \perp Y(t) \mid X$ . Alors

$$\mathbb{P}(T = 1 \mid Y(t), e(X)) = \mathbb{E}[\mathbb{P}(T = 1 \mid Y(t), X) \mid Y(t), e(X)] = \mathbb{E}[e(X) \mid Y(t), e(X)] = e(X),$$

qui ne dépend pas de  $Y(t)$  :  $T \perp Y(t) \mid e(X)$ . **Comparer des clients de même score suffit à supprimer le biais de confusion.**

On a donc remplacé un problème en dimension  $p$  par un problème en dimension 1. (Le prix à payer : il faut **estimer**  $e(x)$ , et la qualité de cette estimation conditionne tout.)

Deux hypothèses sont nécessaires, et il faut les avoir en tête à chaque étape :

1. **Ignorabilité conditionnelle** (« pas de confusion non mesurée ») : après avoir tenu compte de  $X$ , l'attribution est comme tirée au sort. Elle est **invérifiable** à partir des données : c'est une hypothèse sur le monde, justifiée par la connaissance du métier (le graphe de 7.1.9).
2. **Chevauchement** (*positivité*) :  $0 < e(x) < 1$  pour tous les  $x$  rencontrés. Si un type de client n'a *jamais* reçu d'offre, on ne peut rien dire de ce qui se serait passé s'il l'avait reçue. Cette hypothèse, elle, se **vérifie en partie** sur les données.

## 7.2.2 Estimer le score et vérifier le chevauchement

Le score est une probabilité qui dépend de covariables : c'est un problème de régression logistique (chapitre 2, section 2.2). Le modèle doit inclure les variables qui ont guidé le choix de l'attribution.

```
from sklearn.metrics import roc_auc_score

modele_ps = smf.logit("offre ~ age + C(canal) + engagement", data=d).fit(dis=0)
print(modele_ps.params.round(3).to_string())
d["ps"] = modele_ps.predict(d)
print(f"\nAUC du modèle d'attribution : {roc_auc_score(d['offre'], d['ps']):.3f}")
print(d.groupby("offre")["ps"].describe().round(3).to_string())
```

```
Intercept                -2.462
C(canal)[T.Instagram]     0.454
C(canal)[T.Site]         -0.060
age                      -0.032
engagement                0.063
```

AUC du modèle d'attribution : 0.761

| count | mean | std | min | 25% | 50% | 75% | max |
|-------|------|-----|-----|-----|-----|-----|-----|
|-------|------|-----|-----|-----|-----|-----|-----|

| offre |        |       |       |       |       |       |       |       |
|-------|--------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 2150.0 | 0.368 | 0.198 | 0.021 | 0.207 | 0.342 | 0.502 | 0.966 |
| 1     | 1850.0 | 0.572 | 0.204 | 0.049 | 0.423 | 0.582 | 0.731 | 0.969 |

Un coefficient positif sur l'engagement et sur Instagram, négatif sur l'âge : le modèle retrouve la manière dont Yasmine choisissait. L'AUC (aire sous la courbe ROC : la probabilité qu'un client avec offre ait un score plus élevé qu'un client sans offre tiré au hasard) mesure à quel point on peut *prédire* l'attribution. Contrairement à un projet de prédiction, **on ne cherche pas ici un score parfait** : un modèle d'attribution qui prédit parfaitement l'offre signifierait au contraire un **manque de chevauchement**.

```
fig, ax = plt.subplots(figsize=(8.5, 3.6))
bins = np.linspace(0, 1, 41)
ax.hist(d.loc[d.offre == 0, "ps"], bins=bins, color=BLEU, alpha=0.75, label="sans offre")
ax.hist(d.loc[d.offre == 1, "ps"], bins=bins, color=ORANGE, alpha=0.75, label="avec offre")
ax.set_xlabel("score de propension (probabilité estimée de recevoir l'offre)")
ax.set_ylabel("nombre de clients")
ax.legend(frameon=False)
ax.grid(axis="x", visible=False)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
plt.savefig("figures/ch07-chevauchement.png", dpi=200, bbox_inches="tight")

bas_commun = max(d.loc[d.offre == 1, "ps"].min(), d.loc[d.offre == 0, "ps"].min())
haut_commun = min(d.loc[d.offre == 1, "ps"].max(), d.loc[d.offre == 0, "ps"].max())
hors = ((d["ps"] < bas_commun) | (d["ps"] > haut_commun)).sum()
print(f"support commun : [{bas_commun:.3f} ; {haut_commun:.3f}] -> {hors} clients en dehors")
```

support commun : [0.049 ; 0.966] -> 22 clients en dehors

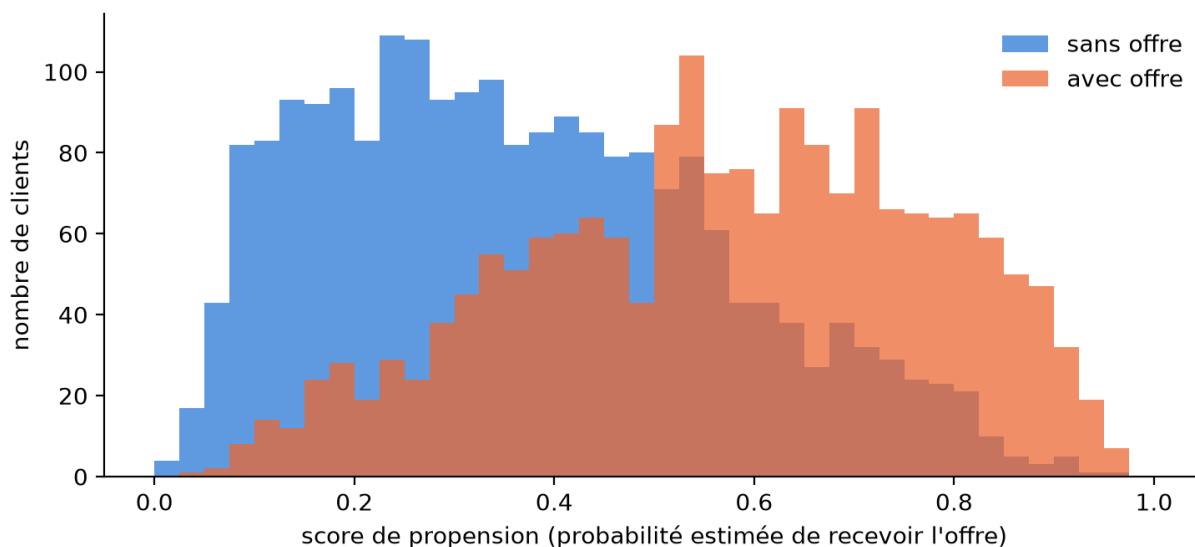


Figure 74: Distribution du score de propension estimé selon que le client a reçu l'offre ou non : les deux distributions sont décalées, mais se chevauchent largement.

Les clients avec offre ont des scores plus élevés, ce qui est normal : c'est la trace du ciblage. L'important est que **les deux histogrammes se recouvrent** sur presque tout l'intervalle : pour (presque) chaque client avec offre, on trouve des clients sans offre qui lui ressemblent. Calcul fait sur les bornes, seuls 22 clients sur 4 000 se trouvent en dehors du support commun (de 0,049 à 0,966) : le chevauchement est bon. C'est le diagnostic de positivité.

**Le diagnostic le plus important.** Avant d'estimer quoi que ce soit, regardez ce graphique. Si les deux distributions sont presque disjointes, aucune méthode ne pourra répondre, et il faut le dire (ou restreindre la population étudiée). Un score proche de 0 ou de 1 pour de nombreux clients annonce des poids énormes et une estimation instable.

### 7.2.3 L'appariement

La première méthode est la plus intuitive : pour chaque client **avec** offre, on cherche un client **sans** offre qui lui **ressemble** (score voisin), et on compare leurs dépenses. L'effet estimé est alors la moyenne de ces différences. Comme on part des traités, on estime l'ATT.

Voici l'algorithme, volontairement écrit à la main pour qu'il n'y ait pas de mystère : appariement au plus proche voisin sur le *logit* du score (qui s'étale mieux que le score), **avec remise** (un même témoin peut servir plusieurs fois), et avec un **calibre** : on refuse les appariements trop lointains (écart de logit supérieur à 0,2 écart-type), faute de quoi on compare des clients qui ne se ressemblent pas.

```
from sklearn.neighbors import NearestNeighbors

def apparier(df, calibre=0.2):
    """Renvoie (effet ATT, nombre de traités appariés, indices des témoins appariés)."""
    logit_ps = np.log(df["ps"] / (1 - df["ps"])).to_numpy()
    T = df["offre"].to_numpy()
    traites, temoins = np.where(T == 1)[0], np.where(T == 0)[0]
    nn = NearestNeighbors(n_neighbors=1).fit(logit_ps[temoins].reshape(-1, 1))
    dist, pos = nn.kneighbors(logit_ps[traites].reshape(-1, 1))
    ok = dist[:, 0] <= calibre * logit_ps.std()
    appar_t = traites[ok]
    appar_c = temoins[pos[ok, 0]]
    y = df["depense"].to_numpy()
    return (y[appar_t] - y[appar_c]).mean(), ok.sum(), appar_t, appar_c

att_appar, n_appar, idx_t, idx_c = apparier(d)
print(f"{n_appar} traités appariés sur {int(d['offre'].sum())}")
print(f"témoins distincts utilisés : {len(np.unique(idx_c))} (un même témoin sert en moyenne  
↳ {len(idx_c) / len(np.unique(idx_c)):.1f} fois)")
print(f"effet estimé par appariement (ATT) : {att_appar:.2f} DT | ATT vrai : {att_vrai:.2f}")

1843 traités appariés sur 1850
témoins distincts utilisés : 814 (un même témoin sert en moyenne 2.3 fois)
effet estimé par appariement (ATT) : 14.41 DT | ATT vrai : 16.64
```

Presque tous les traités (1 843 sur 1 850) ont trouvé un voisin acceptable, et l'estimation (14,4 DT) est du bon ordre de grandeur face à l'ATT vrai (16,6 DT), sans être exacte : combien faut-il s'en méfier ? C'est la question de l'incertitude.

Pour l'incertitude, la formule de la variance n'est pas simple (la procédure inclut l'estimation du score *et* l'appariement). On utilise donc le **bootstrap** (volume I, section 3.3.5) en **refaisant toute la procédure** sur chaque échantillon rééchantillonné : c'est la bonne façon de tenir compte de toutes les sources d'aléa. Nous réutiliserons cette réinitialisation de l'index (`reset_index(drop=True)`) à chaque bootstrap, pour que les lignes dupliquées par le tirage ne se mélangent pas.

```
def ps_et_appariement(df):
    m = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(disps=0)
    df = df.assign(ps=m.predict(df))
    return apparier(df)[0]

rng = np.random.default_rng(2)
boot = []
```

```

for _ in range(200):
    echantillon = d.sample(len(d), replace=True,
    ↪ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True)
    boot.append(ps_et_appariement(echantillon))
boot = np.array(boot)
print(f"erreur-type bootstrap : {boot.std():.2f}    IC 95 % (percentiles) : [{np.percentile(boot,
    ↪ 2.5):.1f} ; {np.percentile(boot, 97.5):.1f}]")

```

```

erreur-type bootstrap : 3.66    IC 95 % (percentiles) : [6.6 ; 20.2]

```

L'intervalle de confiance, de 6,6 à 20,2 DT, contient la vérité (16,6) mais il est **large** : l'erreur-type de 3,7 DT est près de deux fois celle de la régression. C'est une caractéristique de l'appariement au plus proche voisin, qui ne compare chaque traité qu'à *un seul* témoin et gaspille donc de l'information.

Vérifions surtout que l'appariement a bien **rendu les groupes comparables**. On mesure la SMD de chaque covariable **avant** et **après** appariement (les témoins sont comptés autant de fois qu'ils sont utilisés).

```

def smd_pondere(x, T, w):
    """Différence moyenne standardisée entre traités (T=1) et témoins (T=0), avec poids w."""
    x, T, w = np.asarray(x, float), np.asarray(T), np.asarray(w, float)
    m1 = np.average(x[T == 1], weights=w[T == 1])
    m0 = np.average(x[T == 0], weights=w[T == 0])
    v1 = np.average((x[T == 1] - m1) ** 2, weights=w[T == 1])
    v0 = np.average((x[T == 0] - m0) ** 2, weights=w[T == 0])
    return (m1 - m0) / np.sqrt((v1 + v0) / 2)

covariables = pd.DataFrame({
    "âge": d["age"], "engagement": d["engagement"],
    "canal = Instagram": (d["canal"] == "Instagram").astype(float),
    "canal = Site": (d["canal"] == "Site").astype(float),
    "canal = Boutique": (d["canal"] == "Boutique").astype(float)})

poids_appar = np.zeros(len(d))
poids_appar[idx_t] += 1                                # chaque traité apparié compte une fois
np.add.at(poids_appar, idx_c, 1)                        # chaque témoin compte autant de fois qu'il
    ↪ est utilisé
avant = {c: smd_pondere(covariables[c], d["offre"], np.ones(len(d))) for c in covariables}
apres_appar = {c: smd_pondere(covariables[c], d["offre"], poids_appar) for c in covariables}
print(pd.DataFrame({"SMD avant": avant, "SMD après appariement": apres_appar}).round(3).to_string())

```

|                   | SMD avant | SMD après appariement |
|-------------------|-----------|-----------------------|
| âge               | -0.336    | 0.003                 |
| engagement        | 0.915     | 0.006                 |
| canal = Instagram | 0.306     | -0.013                |
| canal = Site      | -0.204    | 0.041                 |
| canal = Boutique  | -0.118    | -0.028                |

Avant l'appariement, l'engagement présente une SMD de 0,92 (un écart considérable : les traités sont presque un écart-type plus engagés que les témoins), et l'âge et le canal Instagram des SMD de -0,34 et +0,31 ; après, toutes les SMD sont inférieures à 0,05 en valeur absolue. L'appariement a bien produit des groupes comparables sur ce que nous avons mesuré. Notez que ce diagnostic porte uniquement sur les covariables **observées** : il ne dit rien sur celles que nous aurions oublié de mesurer.

### 7.2.4 La pondération par l'inverse du score (IPW)

L'appariement jette des données (les traités sans voisin, les témoins jamais utilisés). Une autre façon de rendre les groupes comparables est de les **repondérer** : on donne plus de poids aux



clients **peu probables** dans leur groupe (un client avec offre qui avait peu de chances de la recevoir « représente » beaucoup de clients semblables qui, eux, ne l'ont pas reçue). Ce procédé, la **pondération par l'inverse de la probabilité de traitement** (IPW), s'illustre à la main.

Imaginons deux types de clients, 10 de chaque. Les clients de type A (jeunes, très engagés) reçoivent l'offre avec probabilité 0,8 (8 sur 10) ; ceux de type B, avec probabilité 0,2 (2 sur 10). L'effet réel de l'offre est de +30 DT dans les deux types. Les dépenses moyennes observées sont :

|            | Type A ( $e = 0,8$ ) | Type B ( $e = 0,2$ ) |
|------------|----------------------|----------------------|
| avec offre | 200 DT (8 clients)   | 120 DT (2 clients)   |
| sans offre | 170 DT (2 clients)   | 90 DT (8 clients)    |

La comparaison brute donne  $(8 \times 200 + 2 \times 120)/10 - (2 \times 170 + 8 \times 90)/10 = 184 - 106 = 78$  DT : à nouveau très loin des 30 réels, parce que les traités sont surtout de type A. Pondérons chaque client par  $1/e$  pour les traités et  $1/(1 - e)$  pour les témoins :

- traités de type A : poids  $1/0,8 = 1,25$  ; traités de type B : poids  $1/0,2 = 5$  ;
- témoins de type A : poids  $1/(1 - 0,8) = 5$  ; témoins de type B : poids  $1/(1 - 0,2) = 1,25$ .

La « population pondérée » a alors, dans chaque type, **10 traités et 10 témoins** :  $8 \times 1,25 = 10$  et  $2 \times 5 = 10$  pour les traités, et symétriquement pour les témoins. Le biais de confusion a disparu : le type ne prédit plus l'attribution.

```
groupes = pd.DataFrame({
    "type": ["A", "A", "B", "B"], "offre": [1, 0, 1, 0],
    "n": [8, 2, 2, 8], "depense": [200, 170, 120, 90], "e": [0.8, 0.8, 0.2, 0.2])
groupes["poids"] = np.where(groupes["offre"] == 1, 1 / groupes["e"], 1 / (1 - groupes["e"]))
groupes["effectif_pondere"] = groupes["n"] * groupes["poids"]
print(groupes.to_string(index=False))

def moyenne_ponderee(g):
    return (g["n"] * g["poids"] * g["depense"]).sum() / (g["n"] * g["poids"]).sum()

m1 = moyenne_ponderee(groupes[groupes.offre == 1])
m0 = moyenne_ponderee(groupes[groupes.offre == 0])
brut = ((groupes.n * groupes.depense)[groupes.offre == 1].sum() / groupes.n[groupes.offre == 1].sum()
        - (groupes.n * groupes.depense)[groupes.offre == 0].sum() / groupes.n[groupes.offre ==
        ↪ 0].sum())
print(f"\ndifférence brute : {brut} | après pondération : {m1} - {m0} = {m1 - m0}")
```

| type | offre | n | depense | e   | poids | effectif_pondere |
|------|-------|---|---------|-----|-------|------------------|
| A    | 1     | 8 | 200     | 0.8 | 1.25  | 10.0             |
| A    | 0     | 2 | 170     | 0.8 | 5.00  | 10.0             |
| B    | 1     | 2 | 120     | 0.2 | 5.00  | 10.0             |
| B    | 0     | 8 | 90      | 0.2 | 1.25  | 10.0             |

différence brute : 78.0 | après pondération : 160.0 - 130.0 = 30.0

La pondération retrouve exactement 30 DT. Voici la justification générale.

**Pourquoi l'IPW est sans biais.** Si l'ignorabilité tient avec  $X$ , alors

$$\mathbb{E}\left[\frac{TY}{e(X)}\right] = \mathbb{E}\left[\frac{TY(1)}{e(X)}\right] = \mathbb{E}\left[\mathbb{E}\left[\frac{T}{e(X)} \middle| X, Y(1)\right] Y(1)\right] = \mathbb{E}[Y(1)],$$

car  $\mathbb{E}[T | X, Y(1)] = e(X)$ . De même  $\mathbb{E}[(1 - T)Y/(1 - e(X))] = \mathbb{E}[Y(0)]$ . D'où l'estimateur

de l'ATE (de Horvitz-Thompson)

$$\widehat{ATE}_{IPW} = \frac{1}{n} \sum_i \left( \frac{T_i Y_i}{\hat{e}(X_i)} - \frac{(1 - T_i) Y_i}{1 - \hat{e}(X_i)} \right).$$

En pratique, on préfère la version **normalisée** (de Hájek) qui divise par la somme des poids dans chaque groupe :  $\frac{\sum_i w_i T_i Y_i}{\sum_i w_i T_i} - \frac{\sum_i w_i (1 - T_i) Y_i}{\sum_i w_i (1 - T_i)}$ . Elle est moins sensible aux poids extrêmes. Pour l'ATT, les traités gardent le poids 1 et les témoins reçoivent  $e/(1 - e)$ .

```
e = d["ps"].to_numpy()
T = d["offre"].to_numpy()
Y = d["depense"].to_numpy()

w_ate = np.where(T == 1, 1 / e, 1 / (1 - e))
ht = np.mean(T * Y / e - (1 - T) * Y / (1 - e)) # Horvitz-Thompson
hajek = np.average(Y[T == 1], weights=w_ate[T == 1]) - np.average(Y[T == 0], weights=w_ate[T == 0])
w_att = np.where(T == 1, 1.0, e / (1 - e))
att_ipw = np.average(Y[T == 1]) - np.average(Y[T == 0], weights=w_att[T == 0])

print(f"ATE par IPW (Horvitz-Thompson) : {ht:.2f}")
print(f"ATE par IPW (normalisé) : {hajek:.2f} | ATE vrai : {ate_vrai:.2f}")
print(f"ATT par IPW : {att_ipw:.2f} | ATT vrai : {att_vrai:.2f}")
```

```
ATE par IPW (Horvitz-Thompson) : 13.44
ATE par IPW (normalisé) : 14.36 | ATE vrai : 15.53
ATT par IPW : 11.84 | ATT vrai : 16.64
```

Premier diagnostic des poids : l'**effectif effectif**,  $n_{\text{eff}} = (\sum w_i)^2 / \sum w_i^2$ . Des poids très inégaux signifient que quelques clients pèsent énormément et que l'information réelle est bien inférieure à  $n$ .

```
def effectif_effectif(w):
    return w.sum() ** 2 / (w ** 2).sum()

for nom, mask in [("avec offre", T == 1), ("sans offre", T == 0)]:
    w = w_ate[mask]
    print(f"{nom} : n = {mask.sum()}, poids min/médian/max = {w.min():.2f} / {np.median(w):.2f} / {w.max():.1f}, effectif effectif = {effectif_effectif(w):.0f}")

# Troncature : on borne les poids aux percentiles 1 et 99
bas, haut = np.percentile(w_ate, [1, 99])
w_tronque = np.clip(w_ate, bas, haut)
hajek_tronque = np.average(Y[T == 1], weights=w_tronque[T == 1]) - np.average(Y[T == 0], weights=w_tronque[T == 0])
print(f"\nATE par IPW avec poids tronqués à [{bas:.2f} ; {haut:.1f}] : {hajek_tronque:.2f}")
```

```
avec offre : n = 1850, poids min/médian/max = 1.03 / 1.72 / 20.5, effectif effectif = 1261
sans offre : n = 2150, poids min/médian/max = 1.02 / 1.52 / 29.3, effectif effectif = 1508
```

```
ATE par IPW avec poids tronqués à [1.06 ; 7.4] : 17.52
```

Les poids sont inégaux (de 1 à près de 30, avec une médiane voisine de 1,6), mais pas dramatiquement : l'effectif effectif est d'environ 1 260 pour les 1 850 traités et 1 510 pour les 2 150 témoins, soit une perte d'information de l'ordre d'un quart à un tiers. Le bon chevauchement évite le pire ; si quelques poids dépassaient 50 ou 100, on tronquerait ou on restreindrait la population, au prix d'un léger biais pour gagner beaucoup de stabilité.

Notez que **la troncature a fait passer l'estimation de 14,4 à 17,5 DT** : l'IPW est sensible à quelques poids élevés. C'est son talon d'Achille : un estimateur sans biais, mais **plus variable** que la régres-

sion. Mesurons cette variabilité avec le bootstrap, en réestimant le score à chaque rééchantillonnage.

```
def ipw_ate_att(df):
    e_b = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(dis=0).predict(df).to_numpy()
    Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
    w = np.where(Tb == 1, 1 / e_b, 1 / (1 - e_b))
    ate_b = np.average(Yb[Tb == 1], weights=w[Tb == 1]) - np.average(Yb[Tb == 0], weights=w[Tb == 0])
    att_b = Yb[Tb == 1].mean() - np.average(Yb[Tb == 0], weights=(e_b / (1 - e_b))[Tb == 0])
    return ate_b, att_b

rng = np.random.default_rng(4)
boot_ipw = np.array([ipw_ate_att(d.sample(len(d), replace=True,
    ↪ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True))
    for _ in range(200)])
for nom, i, vrai in [("ATE", 0, ate_vrai), ("ATT", 1, att_vrai)]:
    b = boot_ipw[:, i]
    print(f"IPW {nom} : erreur-type bootstrap {b.std():.2f} IC 95 % : [{np.percentile(b, 2.5):.1f} ;
    ↪ {np.percentile(b, 97.5):.1f}] (vérité {vrai:.1f})")
```

```
IPW ATE : erreur-type bootstrap 2.50 IC 95 % : [9.6 ; 19.1] (vérité 15.5)
IPW ATT : erreur-type bootstrap 3.37 IC 95 % : [4.7 ; 17.8] (vérité 16.6)
```

Les deux intervalles contiennent la vérité. Mais regardez les erreurs-types : 2,5 DT pour l'ATE et 3,4 DT pour l'ATT, plus que les 2,0 DT de la régression de 7.1.9. L'écart de 4,8 DT entre l'ATT estimé par IPW (11,8) et l'ATT vrai (16,6) représente environ 1,4 erreur-type : rien d'anormal, mais une bonne illustration de la précision limitée de la méthode. L'ATT est plus incertain que l'ATE ici, car il ne repose que sur les 1 850 traités et sur des témoins très inégalement pondérés.

Le même diagnostic d'équilibre que pour l'appariement s'applique, et se représente par un **graphique de Love** : une ligne par covariable, la SMD avant (rond gris) et après pondération (rond bleu).

```
apres_ipw = {c: smd_pondere(covariables[c], T, w_ate) for c in covariables}
fig, ax = plt.subplots(figsize=(7.5, 3.4))
noms = list(covariables.columns)
y = np.arange(len(noms))[:-1]
ax.axvline(0, color=GRIS, lw=1)
ax.axvspan(-0.1, 0.1, color="#e1e0d9", alpha=0.6, lw=0)
ax.scatter([avant[c] for c in noms], y, color=GRIS, s=45, label="avant pondération", zorder=3)
ax.scatter([apres_ipw[c] for c in noms], y, color=BLEU, s=45, label="après pondération (IPW)",
    ↪ zorder=3)
ax.set_yticks(y)
ax.set_yticklabels(noms)
ax.set_xlabel("différence moyenne standardisée (SMD)")
ax.legend(frameon=False, loc="lower right", fontsize=9)
ax.grid(axis="y", visible=False)
for s in ("top", "right"):
    ax.spines[s].set_visible(False)
plt.savefig("figures/ch07-love.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Toutes les covariables, qui avaient un déséquilibre marqué (surtout l'engagement), tombent dans la bande de  $\pm 0,1$  après pondération : la pseudo-population est équilibrée.

### 7.2.5 Le meilleur des deux mondes : l'estimateur doublement robuste

Deux stratégies, deux paris. L'**ajustement par régression** parie sur un bon modèle du **résultat** (Y selon X et T). L'**IPW** parie sur un bon modèle de l'**attribution** (T selon X). L'estimateur dou-

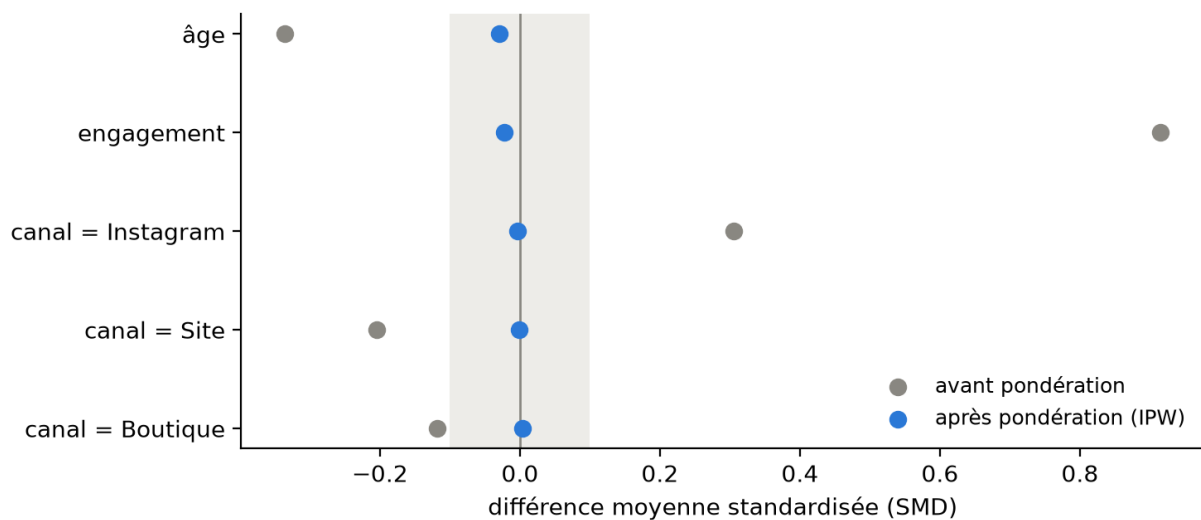


Figure 75: Graphique de Love : la différence moyenne standardisée de chaque covariable avant et après pondération par l'inverse du score ; la bande grise marque l'intervalle de  $\pm 0,1$ .

**blement robuste** (AIPW, pour *augmented* IPW) utilise les deux :

$$\widehat{ATE}_{AIPW} = \frac{1}{n} \sum_i \left[ \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{T_i (Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - T_i) (Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)} \right],$$

où  $\hat{\mu}_t(x)$  est l'espérance estimée du résultat sous le traitement  $t$ . On lit l'estimateur ainsi : on **impute** l'effet par le modèle de résultat ( $\hat{\mu}_1 - \hat{\mu}_0$ ), puis on **corrige** la prédiction par les résidus pondérés de l'IPW. Son nom vient de sa propriété remarquable : l'estimateur est **cohérent si au moins l'un des deux modèles est correct** (pas besoin que les deux le soient). C'est une assurance contre l'erreur de spécification.

**Pourquoi « doublement » ?** Si  $\hat{\mu}_t = \mu_t$  (modèle de résultat correct), les résidus  $Y - \mu_t(X)$  sont de moyenne nulle à  $X$  et  $T$  fixés, donc le terme de correction s'annule en espérance quel que soit  $e$ , et il reste  $E[\mu_1 - \mu_0] = ATE$ . Si au contraire  $\hat{e} = e$  (score correct), le terme de correction compense exactement l'erreur d'un mauvais modèle de résultat :  $E[\frac{T}{e}(Y - \hat{\mu}_1)] = E[Y(1) - \hat{\mu}_1(X)]$ , ce qui annule le biais de  $\hat{\mu}_1$ . Dans les deux cas, on retrouve  $E[Y(1)] - E[Y(0)]$ .

Mettons cette promesse à l'épreuve avec une **expérience** : on estime l'ATE de quatre façons, en rendant volontairement mauvais l'un des deux modèles. Un « mauvais » modèle de résultat est ici un modèle qui ignore les covariables (une constante par groupe) ; un « mauvais » modèle d'attribution est un score constant (il ignore le ciblage).

```
X = np.column_stack([np.ones(len(d)), d["age"], (d["canal"] == "Instagram"), (d["canal"] == "Site"),
↪ d["engagement"]]).astype(float)

def mu_hat(X, Y, T, bon_modele):
    """Prédictions de E[Y | X, T=t] pour t = 0 et 1, par moindres carrés séparés dans chaque groupe."""
    sorties = []
    for t in (0, 1):
        cols = slice(None) if bon_modele else slice(0, 1) # mauvais modèle = constante seule
        beta, *_ = np.linalg.lstsq(X[T == t][:, cols], Y[T == t], rcond=None)
        sorties.append(X[:, cols] @ beta)
    return sorties

def e_hat(X, T, bon_modele):
```

```

if not bon_modele:
    return np.full(len(T), T.mean())
m = smf.logit("offre ~ age + C(canal) + engagement", data=d).fit(dis=0)
return m.predict(d).to_numpy()

def estimateurs(bon_resultat, bon_score):
    mu0, mu1 = mu_hat(X, Y, T, bon_resultat)
    e = e_hat(X, T, bon_score)
    regression = np.mean(mu1 - mu0)
    ipw = np.average(Y[T == 1], weights=1 / e[T == 1]) - np.average(Y[T == 0], weights=1 / (1 - e[T ==
    ↪ 0]))
    aipw = np.mean(mu1 - mu0 + T * (Y - mu1) / e - (1 - T) * (Y - mu0) / (1 - e))
    return regression, ipw, aipw

lignes = []
for br in (True, False):
    for bs in (True, False):
        reg, ipw, aipw = estimateurs(br, bs)
        lignes.append({"modèle de résultat": "correct" if br else "faux", "modèle d'attribution":
        ↪ "correct" if bs else "faux",
            "régression": reg, "IPW": ipw, "doublement robuste": aipw})
print(pd.DataFrame(lignes).round(1).to_string(index=False))
print(f"\nATE vrai : {ate_vrai:.1f}    (différence naïve : {Y[T == 1].mean() - Y[T == 0].mean():.1f})")

```

| modèle de résultat | modèle d'attribution | régression | IPW  | doublement robuste |
|--------------------|----------------------|------------|------|--------------------|
| correct            | correct              | 15.0       | 14.4 | 14.9               |
| correct            | faux                 | 15.0       | 50.5 | 15.0               |
| faux               | correct              | 50.5       | 14.4 | 14.3               |
| faux               | faux                 | 50.5       | 50.5 | 50.5               |

ATE vrai : 15.5 (différence naïve : 50.5)

Lisons la table. Quand le modèle de résultat est faux, la **régression** échoue (elle donne la différence naïve) ; quand le modèle d'attribution est faux, l'**IPW** échoue ; mais l'estimateur **doublement robuste** reste proche de la vérité dès que **l'un des deux** est correct, et n'échoue que lorsque **les deux** sont faux. C'est une assurance, pas une garantie. Calculons l'estimation « principale » avec l'incertitude correspondante par bootstrap :

```

def aipw_complet(df):
    Xb = np.column_stack([np.ones(len(df)), df["age"], (df["canal"] == "Instagram"), (df["canal"] ==
    ↪ "Site"), df["engagement"]]).astype(float)
    Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
    mu0, mu1 = mu_hat(Xb, Yb, Tb, True)
    eb = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(dis=0).predict(df).to_numpy()
    return np.mean(mu1 - mu0 + Tb * (Yb - mu1) / eb - (1 - Tb) * (Yb - mu0) / (1 - eb))

est = aipw_complet(d)
rng = np.random.default_rng(3)
boot = np.array([aipw_complet(d.sample(len(d), replace=True,
    ↪ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True))
    for _ in range(200)])
print(f"ATE doublement robuste : {est:.2f}    erreur-type bootstrap : {boot.std():.2f}    IC 95 % :
    ↪ [{est - 1.96 * boot.std():.1f} ; {est + 1.96 * boot.std():.1f}])")
print(f"ATE vrai : {ate_vrai:.2f}")

```

ATE doublement robuste : 14.89 erreur-type bootstrap : 2.44 IC 95 % : [10.1 ; 19.7]  
ATE vrai : 15.53

Récapitulons toutes les estimations de l'ATE et de l'ATT de la section, face à la vérité :

```

recap = pd.DataFrame({
    "méthode": ["différence naïve", "régression (7.1.9)", "IPW (ATE)", "doublement robuste (ATE)",
    ↪ "appariement (ATT)", "IPW (ATT)"],

```

```

"estimation": [Y[T == 1].mean() - Y[T == 0].mean(),
               smf.ols("depense ~ offre + age + C(canal) + engagement", d).fit().params["offre"],
               hajek, est, att_appar, att_ipw],
"vérité": [ate_vrai, ate_vrai, ate_vrai, ate_vrai, att_vrai, att_vrai]).round(1)
print(recap.to_string(index=False))

```

|  | méthode                  | estimation | vérité |
|--|--------------------------|------------|--------|
|  | différence naïve         | 50.5       | 15.5   |
|  | régression (7.1.9)       | 14.8       | 15.5   |
|  | IPW (ATE)                | 14.4       | 15.5   |
|  | doublément robuste (ATE) | 14.9       | 15.5   |
|  | appariement (ATT)        | 14.4       | 16.6   |
|  | IPW (ATT)                | 11.8       | 16.6   |

Lecture de la table : toutes les méthodes qui tiennent compte de l'engagement ramènent l'ATE vers la vérité (entre 14,4 et 14,9 contre 15,5, alors que la différence naïve est à 50,5), et l'ATT estimé par appariement (14,4) ou par IPW (11,8) est plus bas que l'ATT vrai (16,6) mais dans l'incertitude statistique de chaque méthode. Ces estimations ne sont pas identiques, et il n'y a aucune raison qu'elles le soient : chacune a sa variance et ses hypothèses. L'essentiel est qu'elles **convergent** vers le même ordre de grandeur, très loin de l'estimation naïve.

**À retenir (7.2.1 à 7.2.5).** Le score de propension résume l'attribution en un nombre. Trois usages : **apparier**, **pondérer**, ou combiner avec un modèle de résultat (**doublément robuste**). Dans tous les cas : (1) vérifier le **chevauchement**, (2) vérifier l'**équilibre** après l'ajustement (SMD), (3) accompagner l'estimation d'un intervalle (bootstrap de *toute* la procédure). Ces méthodes ne corrigent que la confusion due aux covariables **observées**.

## 7.2.6 Ce que le score de propension ne fait pas

Terminons par l'avertissement le plus important de la section. Tous les résultats précédents reposaient sur le fait que l'**engagement était observé**. Que se passe-t-il s'il ne l'est pas ? Reprenons l'analyse en le cachant à tous les modèles, ou en ne le mesurant qu'avec du **bruit** (ce qui est le cas typique : un score d'engagement n'est qu'un reflet imparfait de l'enthousiasme réel du client).

```

def estimation_aipw_avec_engagement(bruit_sd, graine=5):
    """AIPW quand l'engagement n'est connu qu'avec un bruit gaussien d'écart-type bruit_sd (None =
    engagement caché)."""
    df = d.copy()
    if bruit_sd is None:
        formule_ps = "offre ~ age + C(canal)"
        cols = [np.ones(len(df)), df["age"], (df["canal"] == "Instagram"), (df["canal"] == "Site")]
    else:
        r = np.random.default_rng(graine)
        df["eng_mesure"] = df["engagement"] + r.normal(0, bruit_sd, len(df))
        formule_ps = "offre ~ age + C(canal) + eng_mesure"
        cols = [np.ones(len(df)), df["age"], (df["canal"] == "Instagram"), (df["canal"] == "Site"),
        df["eng_mesure"]]
    Xb = np.column_stack(cols).astype(float)
    Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
    mu0, mu1 = mu_hat(Xb, Yb, Tb, True)
    eb = smf.logit(formule_ps, data=df).fit(disps=0).predict(df).to_numpy()
    return np.mean(mu1 - mu0 + Tb * (Yb - mu1) / eb - (1 - Tb) * (Yb - mu0) / (1 - eb))

resultats = [("engagement parfaitement mesuré", estimation_aipw_avec_engagement(0.0))]
for sd in (15, 30, 60):
    resultats.append((f"engagement mesuré avec du bruit (écart-type {sd})",
    estimation_aipw_avec_engagement(sd)))
resultats.append(("engagement non observé", estimation_aipw_avec_engagement(None)))
print(pd.DataFrame(resultats, columns=["situation", "ATE doublément
robuste"]).round(1).to_string(index=False))

```

```
print(f"\nATE vrai : {ate_vrai:.1f}")
print(f"écart-type de l'engagement lui-même : {d['engagement'].std():.1f}")
```

| situation                                       | ATE doublement robuste |
|-------------------------------------------------|------------------------|
| engagement parfaitement mesuré                  | 14.9                   |
| engagement mesuré avec du bruit (écart-type 15) | 32.5                   |
| engagement mesuré avec du bruit (écart-type 30) | 41.8                   |
| engagement mesuré avec du bruit (écart-type 60) | 45.6                   |
| engagement non observé                          | 46.9                   |

ATE vrai : 15.5

écart-type de l'engagement lui-même : 15.5

À mesure que l'engagement est de moins en moins bien mesuré, l'estimation, pourtant « doublement robuste », **dérive** vers la différence naïve. Les méthodes sophistiquées ne remplacent pas l'information manquante : un facteur de confusion mal mesuré laisse une **confusion résiduelle**. Les écarts-types de bruit (15, 30, 60) sont à comparer à l'écart-type de l'engagement lui-même (15,5, dernière ligne de la sortie) : avec un bruit de 15, la mesure contient autant de bruit que de signal, et l'estimation, pourtant « doublement robuste », est déjà à 32,5 DT, au milieu du chemin entre la vérité (15,5) et la différence naïve (50,5).

Reste la question honnête : dans la vraie vie, comment sait-on que l'on a mesuré tous les facteurs de confusion importants ? **On ne le sait pas**. On peut seulement (1) s'appuyer sur la connaissance du processus d'attribution (« comment Yasmine a-t-elle décidé ? »), (2) faire des **analyses de sensibilité** (quelle intensité devrait avoir un facteur de confusion caché pour annuler le résultat ?), (3) chercher des situations qui contournent le problème : c'est le rôle des deux sections suivantes.

**Les trois erreurs classiques avec les scores de propension.** (1) **Régler le score pour qu'il prédise bien** : le but est l'équilibre des covariables, pas l'AUC. (2) **Inclure des variables post-traitement** ou des variables qui ne sont causées que du traitement (cela gonfle la variance sans corriger le biais). (3) **Oublier de vérifier l'équilibre** après ajustement. Une analyse par score sans tableau d'équilibre est incomplète.

**À retenir (7.2).** Avec ignorabilité et chevauchement, on peut estimer un effet causal sans randomisation, **à condition d'avoir mesuré les facteurs de confusion**. Cette condition est une hypothèse sur le monde, qu'aucun diagnostic ne confirme complètement.

### 7.3 Différences de différences : l'évolution des uns contre l'évolution des autres

**Intuition.** En juillet 2025, Yasmine lance une campagne publicitaire sur Instagram, mais seulement dans huit grandes villes. Les commandes y augmentent. Est-ce la campagne ? Peut-être, mais les commandes augmentent aussi à l'approche des fêtes, et ces huit villes sont de toute façon plus grandes que les autres. Deux comparaisons naïves sont tentantes : **avant contre après** dans les villes traitées (mais l'été n'est pas l'hiver), ou **traitées contre témoins** après la campagne (mais ces villes étaient déjà plus grosses). La **différence de différences** (DiD) combine les deux pour éliminer les deux pièges à la fois : on compare l'**évolution** des villes traitées à l'**évolution** des villes témoins.

Dans la section précédente, nous avons besoin d'observer tous les facteurs de confusion. Ici, on peut se contenter de beaucoup moins : un facteur de confusion **inobservé** est toléré, **à condition**



qu'il soit constant dans le temps (la taille d'une ville) ou qu'il évolue de la même façon pour les deux groupes (la saison). C'est l'avantage des données en **panel** : suivre les mêmes unités avant et après.

### 7.3.1 Les données, et le calcul à la main

Le fichier `ch07-panel-villes.csv` suit 20 villes pendant 24 mois (janvier 2024 à décembre 2025). Huit d'entre elles (les grandes villes) ont reçu la campagne à partir de juillet 2025 ( $t = 18$ ). Le code qui fabrique ces données est imprimé ci-dessous ; il est exécuté et comparé au fichier, pour que vous puissiez vérifier qu'il n'y a pas de truc caché.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.ticker

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
ENCRE, GRIS = "#0b0b0b", "#898781"

def panel_villes(seed=7002, tendance_diff=0.0):
    """20 villes x 24 mois (2024-2025). Campagne publicitaire Instagram lancée en juillet 2025 dans 8
    ↪ grandes villes."""
    rng = np.random.default_rng(seed)
    villes = ["Tunis", "Ariana", "Ben Arous", "Manouba", "Bizerte", "Nabeul", "Hammamet", "Sousse",
    ↪ "Monastir", "Mahdia",
        "Sfax", "Kairouan", "Gabès", "Gafsa", "Tozeur", "Kasserine", "Le Kef", "Béja",
        ↪ "Jendouba", "Médénine"]
    traitees = {"Tunis", "Ariana", "Ben Arous", "Sousse", "Sfax", "Nabeul", "Monastir", "Bizerte"}
    mois = pd.date_range("2024-01-01", "2025-12-01", freq="MS")
    saison = np.log(np.array([0.70, 0.78, 0.95, 1.00, 1.10, 1.15, 1.20, 1.12, 0.90, 0.80, 1.05, 1.50]))
    effet_commun = saison[mois.month - 1] + 0.004 * np.arange(len(mois))
    lignes = []
    for v in villes:
        T = v in traitees
        niveau = rng.normal(3.3, 0.25) + (0.55 if T else 0.0)
        for t, m in enumerate(mois):
            D = int(T and m >= pd.Timestamp("2025-07-01"))
            log_mu = niveau + effet_commun[t] + 0.15 * D + (tendance_diff * t if T else 0.0)
            lignes.append({"ville": v, "mois": m.strftime("%Y-%m-%d"), "t": t, "groupe_traite":
    ↪ int(T), "campagne": D,
                "commandes": int(rng.poisson(np.exp(log_mu)))})
    return pd.DataFrame(lignes)

p = pd.read_csv("donnees/ch07-panel-villes.csv", parse_dates=["mois"])
regen = panel_villes()
regen["mois"] = pd.to_datetime(regen["mois"])
print("le code ci-dessus reproduit exactement le fichier :", p.equals(regen))
p["apres"] = (p["t"] >= 18).astype(int)
p["vid"] = pd.factorize(p["ville"])[0] # identifiant numérique de ville (pour
    ↪ les erreurs-types)
print(p.head(4).to_string(index=False))
print(f"\n{p['ville'].nunique()} villes, {p['t'].nunique()} mois, {len(p)} lignes ;
    ↪ {p.groupby('ville')['groupe_traite'].first().sum()} villes traitées")
```

le code ci-dessus reproduit exactement le fichier : True

| ville | mois       | t | groupe_traite | campagne | commandes | apres | vid |
|-------|------------|---|---------------|----------|-----------|-------|-----|
| Tunis | 2024-01-01 | 0 | 1             | 0        | 40        | 0     | 0   |
| Tunis | 2024-02-01 | 1 | 1             | 0        | 44        | 0     | 0   |
| Tunis | 2024-03-01 | 2 | 1             | 0        | 47        | 0     | 0   |

Tunis 2024-04-01 3 1 0 61 0 0

20 villes, 24 mois, 480 lignes ; 8 villes traitées

Avant tout calcul, on regarde les moyennes de commandes par ville et par mois, dans chaque groupe, avant ( $t < 18$ ) et après ( $t \geq 18$ ) le lancement :

```
moy = p.groupby(["groupe_traite", "apres"])["commandes"].mean().unstack().round(1)
moy.index = ["témoins (12 villes)", "traitées (8 villes)"]
moy.columns = ["avant", "après"]
print(moy)
```

|                     | avant | après |
|---------------------|-------|-------|
| témoins (12 villes) | 30.6  | 36.1  |
| traitées (8 villes) | 46.9  | 63.0  |

Faisons les trois comparaisons à la main, à partir de ce tableau :

|                 | avant | après | variation |
|-----------------|-------|-------|-----------|
| villes témoins  | 30,6  | 36,1  | +5,5      |
| villes traitées | 46,9  | 63,0  | +16,1     |

1. **Avant contre après, villes traitées seulement** :  $63,0 - 46,9 = +16,1$  commandes par mois. Mais cette hausse mélange l'effet de la campagne et celui de la saison (le second semestre est plus actif que le premier) : on ne peut pas les séparer.
2. **Traitées contre témoins, après** :  $63,0 - 36,1 = +26,9$ . Mais les villes traitées étaient déjà plus grandes avant :  $46,9 - 30,6 = +16,3$  d'écart *avant* la campagne.
3. **La différence de différences** : on retire à la hausse des villes traitées (+16,1) la hausse que les villes témoins ont connue sans campagne (+5,5) :

$$\widehat{\text{DiD}} = (\bar{Y}_{T,\text{après}} - \bar{Y}_{T,\text{avant}}) - (\bar{Y}_{C,\text{après}} - \bar{Y}_{C,\text{avant}}) = 16,1 - 5,5 = +10,6 \text{ commandes par mois.}$$

Les villes témoins jouent le rôle de **mesure de ce qui se serait passé** dans les villes traitées en l'absence de campagne : leur hausse naturelle (saison, tendance) est retranchée.

**Pourquoi ça marche : l'hypothèse des tendances parallèles.** Notons  $Y_{it}(0)$  le résultat de la ville  $i$  au mois  $t$  **sans** campagne. L'effet moyen de la campagne sur les villes traitées (l'ATT) est  $\mathbb{E}[Y_{it}(1) - Y_{it}(0) \mid T_i = 1]$  après le lancement. Le terme  $\mathbb{E}[Y_{it}(0) \mid T_i = 1]$  après le lancement est inobservable (ce qui se serait passé sans campagne). La DiD l'approche en supposant que, **sans campagne**, la variation moyenne des villes traitées aurait été **la même** que celle des villes témoins :

$$\underbrace{\mathbb{E}[Y_{\text{après}}(0) - Y_{\text{avant}}(0) \mid T = 1]}_{\text{inobservable}} = \underbrace{\mathbb{E}[Y_{\text{après}}(0) - Y_{\text{avant}}(0) \mid T = 0]}_{\text{observable}}.$$

C'est l'hypothèse des **tendances parallèles**. Sous cette hypothèse, et si la campagne n'a pas d'effet **avant** son lancement (pas d'anticipation),  $\widehat{\text{DiD}}$  estime l'ATT. Remarquez ce que l'on **n'a pas** besoin de supposer : que les villes traitées et témoins aient le même niveau (elles ne l'ont pas), ni que l'on observe tout ce qui les distingue.

### 7.3.2 À quelle échelle les tendances sont-elles parallèles ?

Regardons les données avant de régresser. Voici le nombre moyen de commandes par ville et par mois dans chaque groupe, en niveau (à gauche) et en échelle logarithmique (à droite).

```

moyenne_mois = p.groupby(["groupe_traite", "mois"])["commandes"].mean().unstack(0)
moyenne_mois.columns = ["témoins", "traitées"]
debut = pd.Timestamp("2025-07-01")

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), sharex=True)
for ax, echelle, titre in [(axes[0], "linear", "Commandes par ville et par mois (niveau)"),
                           (axes[1], "log", "Même chose en échelle logarithmique")]:
    ax.plot(moyenne_mois.index, moyenne_mois["témoins"], color=BLEU, lw=2)
    ax.plot(moyenne_mois.index, moyenne_mois["traitées"], color=ORANGE, lw=2)
    ax.axvline(debut, color=GRIS, lw=1, ls="--")
    ax.set_yscale(echelle)
    if echelle == "log":
        ax.set_yticks([20, 30, 40, 60, 80])
        ax.get_yaxis().set_major_formatter(matplotlib.ticker.ScalarFormatter())
        ax.minorticks_off()
    ax.set_title(titre, fontsize=10.5)
    ax.set_ylabel("commandes (moyenne par ville)")
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
    ax.tick_params(axis="x", labelrotation=30)
axes[0].text(moyenne_mois.index[0], 61, "villes traitées", color=ORANGE, fontsize=9)
axes[0].text(moyenne_mois.index[2], 22, "villes témoins", color=BLEU, fontsize=9)
axes[0].text(debut, axes[0].get_ylim()[1] * 0.98, "lancement", color=GRIS, fontsize=9, va="top")
plt.tight_layout()
plt.savefig("figures/ch07-did-tendances.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

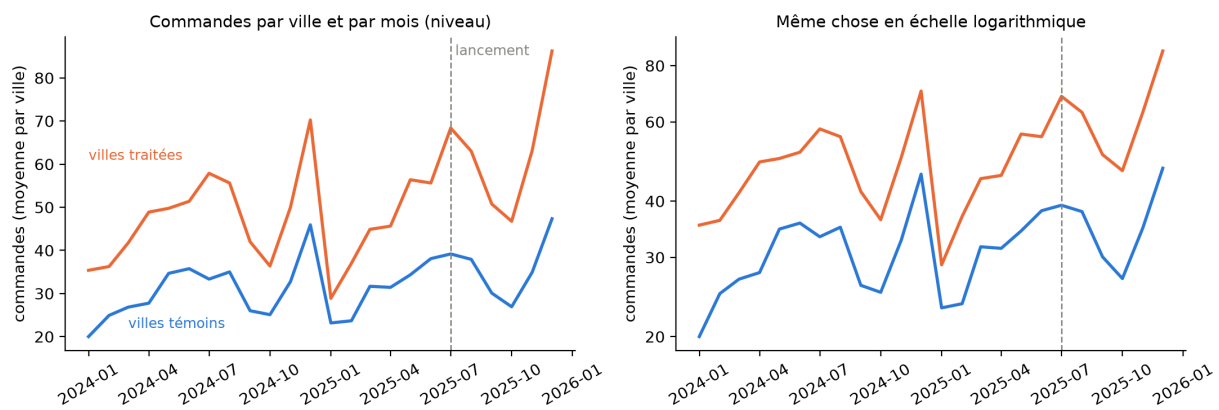


Figure 76: Commandes moyennes par ville et par mois, villes traitées (orange) et témoins (bleu), en niveau à gauche et en échelle logarithmique à droite. La ligne pointillée marque le lancement de la campagne.

Avant le lancement, les deux courbes suivent **la même saisonnalité** (creux en janvier, pic en décembre), mais avec des niveaux différents : les villes traitées sont plus grandes. Les chiffres mensuels sont bruités (ce sont des comptages), si bien que l'effet de la campagne ne saute pas aux yeux sur le graphique : il faut le **mesurer**, ce que font les sections suivantes.

Avant de commenter, mesurons deux façons de comparer les groupes **avant** le lancement : par leur **écart** de niveau (traitées moins témoins) et par leur **rapport** (traitées divisées par témoins).

```

avant_lancement = moyenne_mois[moyenne_mois.index < debut]
ecart = avant_lancement["traitées"] - avant_lancement["témoins"]
rapport = avant_lancement["traitées"] / avant_lancement["témoins"]
print(f"écart de niveau entre les groupes : de {ecart.min():.1f} à {ecart.max():.1f} commandes selon le mois (moyenne {ecart.mean():.1f})")

```

```
print(f"rapport traitées / témoins      : de {rapport.min():.2f} à {rapport.max():.2f} selon le mois
↳ (moyenne {rapport.mean():.2f})")
print(f"variabilité relative (écart-type / moyenne) : écart {ecart.std() / ecart.mean():.0%}, rapport
↳ {rapport.std() / rapport.mean():.0%}")
```

```
écart de niveau entre les groupes : de 5.7 à 24.5 commandes selon le mois (moyenne 16.3)
rapport traitées / témoins      : de 1.25 à 1.77 selon le mois (moyenne 1.54)
variabilité relative (écart-type / moyenne) : écart 30%, rapport 9%
```

La sortie répond. Avant le lancement, l'écart de niveau entre les deux groupes varie beaucoup d'un mois à l'autre (de 5,7 à 24,5 commandes, soit une variabilité relative de 30 %) : il se creuse aux mois d'affluence et se resserre aux mois creux. Leur **rapport**, lui, reste bien plus stable (de 1,25 à 1,77, variabilité relative de 9 %) : les villes traitées commandent environ une fois et demie plus que les témoins, en toute saison.

Voilà un point crucial : l'hypothèse des tendances parallèles **dépend de l'échelle**. Si l'effet de la saison est **multiplicatif** (proportionnel à la taille de la ville : un mois de décembre gonfle les commandes de toutes les villes dans la même **proportion**), alors une grande ville gagne en niveau absolu plus qu'une petite à chaque pic, et les courbes **ne sont pas parallèles en niveau** ; elles le sont **en logarithme**, où un effet multiplicatif devient additif. C'est le cas ici (et pour la plupart des chiffres d'affaires, dont les variations se pensent en pourcentage). Recalculons donc la DiD **en logarithme**, ce qui revient à estimer un effet **relatif**.

```
lm = p.assign(log_cmd=np.log(p["commandes"])).groupby(["groupe_traite",
↳ "apres"])["log_cmd"].mean().unstack()
did_log = (lm.loc[1, 1] - lm.loc[1, 0]) - (lm.loc[0, 1] - lm.loc[0, 0])
print("moyennes du logarithme des commandes :")
print(lm.round(3).rename(index={0: "témoins", 1: "traitées"}, columns={0: "avant", 1: "après"}))
print(f"\nDiD en logarithme : {did_log:.3f} soit un effet relatif de {np.exp(did_log) - 1:+.1%}")
print(f"(rappel : DiD en niveau = {(moy.iloc[1, 1] - moy.iloc[1, 0]) - (moy.iloc[0, 1] - moy.iloc[0,
↳ 0]):.1f} commandes par mois)")
```

```
moyennes du logarithme des commandes :
apres      avant  après
groupe_traite
témoins      3.369  3.537
traitées      3.784  4.089
```

```
DiD en logarithme : 0.138 soit un effet relatif de +14.8%
(rappel : DiD en niveau = 10.6 commandes par mois)
```

La campagne est associée à une hausse relative de 14,8 % des commandes dans les villes traitées. L'estimation en niveau (10,6 commandes par mois, soit environ 23 % des 46,9 commandes d'avant) et celle en pourcentage ne racontent pas tout à fait la même histoire : **la seconde est plus fiable**, parce que l'hypothèse de tendances parallèles est plus plausible à cette échelle (voir le diagnostic ci-dessus).

### 7.3.3 La régression à effets fixes (TWFE)

Le calcul des moyennes ne fournit pas d'erreur-type et ne permet pas d'ajouter des covariables. On le généralise avec une **régression à deux effets fixes** (*two-way fixed effects*, TWFE) : une constante par ville (qui absorbe le niveau de chaque ville), une constante par mois (qui absorbe la saison et la tendance commune), et une variable campagne qui vaut 1 uniquement pour une ville traitée après le lancement :

$$\log Y_{it} = \alpha_i + \gamma_t + \tau D_{it} + \varepsilon_{it}, \quad D_{it} = \mathbb{1}\{i \text{ traitée et } t \geq 18\}.$$

Le coefficient  $\tau$  est l'effet relatif de la campagne. Deux précisions importantes sur l'inférence : les erreurs d'une même ville sont **corrélées dans le temps** (une ville qui commande plus que prévu en janvier a tendance à le faire en février), donc on calcule des erreurs-types **groupées par ville** (*cluster-robust*) ; sans cela, on sous-estime fortement l'incertitude.

```
def twfe(df, debut=18, fin=None, tendance_groupe=False, poisson=False):
    """DiD par régression à effets fixes ville + mois ; erreurs-types groupées par ville."""
    d = df if fin is None else df[df["t"] < fin]
    d = d.assign(camp=((d["groupe_traite"] == 1) & (d["t"] >= debut)).astype(int))
    formule = "commandes ~ camp + C(ville) + C(t)" + (" + groupe_traite:t" if tendance_groupe else "")
    if poisson:
        m = smf.glm(formule, d, family=sm.families.Poisson()).fit(cov_type="cluster",
        ↪ cov_kws={"groups": d["vid"]})
    else:
        m = smf.ols(formule.replace("commandes", "np.log(commandes)"), d).fit(cov_type="cluster",
        ↪ cov_kws={"groups": d["vid"]})
    return m.params["camp"], m.bse["camp"]

b, se = twfe(p)
print(f"TWFE (log) : effet = {b:.3f} erreur-type = {se:.3f} IC 95 % = [{b - 1.96 * se:.3f} ; {b +
↪ 1.96 * se:.3f}] ({np.exp(b) - 1:+.1%}")
b_p, se_p = twfe(p, poisson=True)
print(f"Poisson : effet = {b_p:.3f} erreur-type = {se_p:.3f} IC 95 % = [{b_p - 1.96 * se_p:.3f} ;
↪ {b_p + 1.96 * se_p:.3f}] ({np.exp(b_p) - 1:+.1%}")
print(f"(rappel du calcul à la main : {did_log:.3f}")

TWFE (log) : effet = 0.138 erreur-type = 0.032 IC 95 % = [0.076 ; 0.200] (+14.8%)
Poisson : effet = 0.131 erreur-type = 0.028 IC 95 % = [0.076 ; 0.186] (+14.0%)
(rappel du calcul à la main : 0.138)
```

Le coefficient de la régression coïncide avec la DiD calculée à la main : **dans ce cas simple** (une seule date de lancement, panel complet), la régression à effets fixes redonne exactement la différence de différences des moyennes. Le modèle de **Poisson** (chapitre 2, section 2.3), qui traite les commandes comme un comptage plutôt que de passer au logarithme, donne un résultat voisin ; c'est même préférable quand certains comptages sont proches de zéro (le logarithme de 0 n'existe pas), ce qui n'est pas le cas ici.

**Le nombre de groupes.** Nous n'avons que **20 villes** : les erreurs-types groupées reposent sur un raisonnement asymptotique (beaucoup de groupes) qui est approximatif quand il y en a si peu. Avec moins d'une trentaine de groupes, ou peu de groupes traités (ici 8), l'intervalle de confiance risque d'être trop optimiste ; des méthodes plus robustes (bootstrap par groupes, tests de permutation, section 3.7 du volume I) existent. Gardez en tête que l'intervalle ci-dessus est un ordre de grandeur, pas une certitude à la décimale.

### 7.3.4 L'étude d'événement : regarder les tendances avant et après

L'hypothèse des tendances parallèles porte sur l'**inobservable** (ce qui aurait eu lieu sans la campagne), donc elle ne se teste pas directement. Mais on peut examiner ses **implications observables** : si les villes traitées et témoins évoluaient de façon parallèle *avant* le lancement, un modèle qui laisse l'effet varier mois par mois devrait trouver des coefficients **proches de zéro avant** la campagne. C'est l'**étude d'événement** (*event study*) : on remplace l'unique variable campagne par un coefficient pour chaque mois relatif au lancement, en prenant le mois précédant le lancement (-1) comme référence.

```
def etude_evenement(df, ref=-1):
    """Coefficient mois par mois (traitées vs témoins), relatif au mois de lancement (t = 18) ;
    ↪ référence : k = -1."""
    d = df.copy()
```

```

d["rel"] = d["t"] - 18
termes = []
for k in sorted(d["rel"].unique()):
    if k == ref:
        continue
    nom = f"ev_{'m' if k < 0 else 'p'}{abs(k)}"
    d[nom] = ((d["groupe_traite"] == 1) & (d["rel"] == k)).astype(int)
    termes.append((k, nom))
formule = "np.log(commandes) ~ " + " + ".join(n for _, n in termes) + " + C(ville) + C(t)"
m = smf.ols(formule, d).fit(cov_type="cluster", cov_kwds={"groups": d["vid"]})
return pd.DataFrame({"k": [k for k, _ in termes], "coef": [m.params[n] for _, n in termes],
                    "se": [m.bse[n] for _, n in termes]})

ee = etude_evenement(p)
avant = ee[ee["k"] < 0]
apres = ee[ee["k"] >= 0]
print(f"avant le lancement : coefficient moyen = {avant['coef'].mean():+.3f} ; plus grand écart en
↳ valeur absolue = {avant['coef'].abs().max():.3f}")
print(f"nombre de mois « avant » dont l'IC à 95 % exclut 0 : {int(((avant['coef'].abs() - 1.96 *
↳ avant['se']) > 0).sum())} sur {len(avant)}")
print(f"après le lancement : coefficient moyen = {apres['coef'].mean():+.3f} (effet vrai : 0.150)")

avant le lancement : coefficient moyen = +0.054 ; plus grand écart en valeur absolue = 0.188
nombre de mois « avant » dont l'IC à 95 % exclut 0 : 0 sur 17
après le lancement : coefficient moyen = +0.189 (effet vrai : 0.150)

def tracer_ee(ax, ee, titre):
    ax.axhline(0, color=GRIS, lw=1)
    ax.axvline(-0.5, color=GRIS, lw=1, ls="--")
    ax.errorbar(ee["k"], ee["coef"], yerr=1.96 * ee["se"], fmt="o", color=BLEU, ecolor="#86b6ef",
    ↳ capsize=2, ms=4, lw=1.2)
    ax.plot([-0.5, ee["k"].max() + 0.5], [0.15, 0.15], color=AQUA, lw=1.5, ls=":", label="effet vrai
    ↳ (0,15)")
    ax.legend(frameon=False, loc="lower right", fontsize=9)
    ax.set_title(titre, fontsize=10.5)
    ax.set_xlabel("mois relatif au lancement (0 = juillet 2025)")
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)

fig, ax = plt.subplots(figsize=(8, 3.8))
tracer_ee(ax, ee, "Étude d'événement : effet de la campagne, mois par mois")
ax.set_ylabel("effet relatif (log des commandes)")
plt.savefig("figures/ch07-did-evenement.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Avant le lancement, les coefficients fluctuent autour de zéro (moyenne +0,05, et **aucun** des 17 intervalles de confiance n'exclut zéro) : **pas de tendance différentielle visible** entre les deux groupes. Après le lancement, ils se situent autour de 0,19 en moyenne (la vérité est 0,15), nettement plus haut. Les intervalles sont larges : chaque mois pris isolément est peu informatif ; c'est la **configuration d'ensemble** qui compte. Ce graphique est le **diagnostic standard** d'une DiD, et il devrait figurer dans toute analyse de ce type. Il ne **prouve** pas l'hypothèse (rien ne garantit que les tendances n'auraient pas divergé *après* le lancement pour une autre raison), mais il rend très difficile de croire à une explication par une tendance préexistante.

### 7.3.5 Quand les tendances ne sont pas parallèles

Que se passe-t-il quand l'hypothèse est **fausse** ? Ajoutons un défaut : les grandes villes, déjà plus dynamiques, ont une croissance propre un peu plus rapide que les autres (+1 % par mois, hors

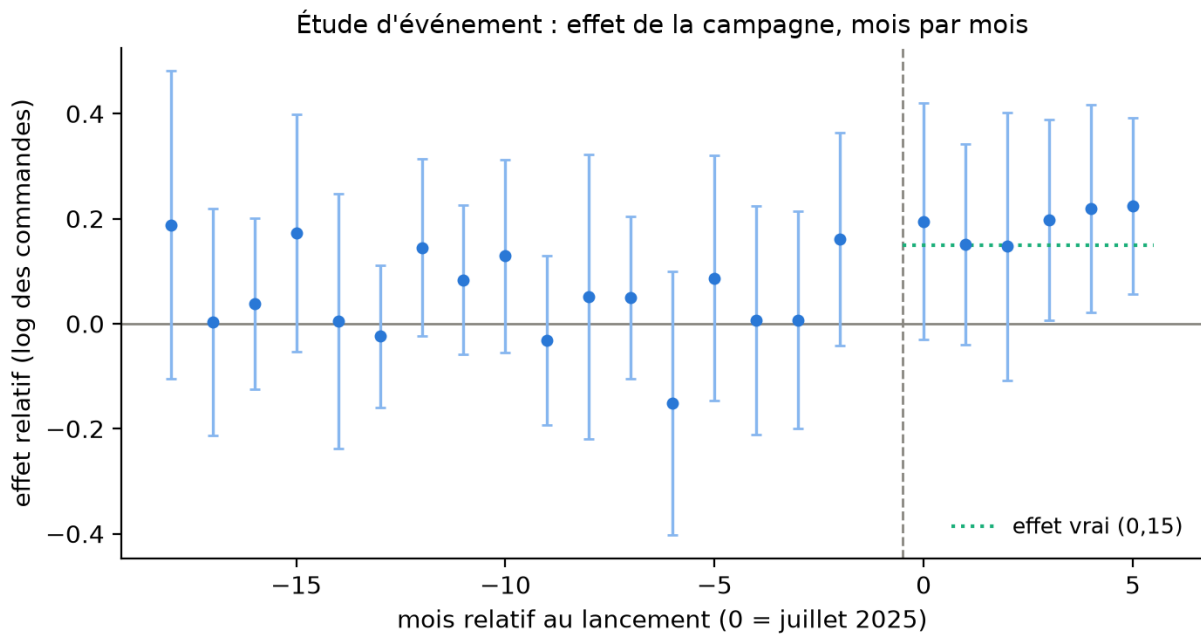


Figure 77: Étude d'événement : coefficients mois par mois (avec intervalle de confiance à 95 %). Avant le lancement, ils fluctuent autour de zéro ; après, ils se situent autour de la vérité.

campagne). Elles auraient donc davantage progressé de toute façon, et la DiD va attribuer cet écart à la campagne. La fonction `panel_villes` imprimée plus haut a un paramètre pour cela.

```
q = panel_villes(tendance_diff=0.01)
q["vid"] = pd.factorize(q["ville"])[0]

b_q, se_q = twfe(q)
print(f"Effet estimé par DiD avec tendances NON parallèles : {b_q:.3f} (erreur-type {se_q:.3f}) | 
↳ effet vrai : 0.150")
ee_q = etude_evenement(q)
avant_q = ee_q[ee_q["k"] < 0]
print(f"mois « avant » dont l'IC à 95 % exclut 0 : {int(((avant_q['coef']).abs() - 1.96 * 
↳ avant_q['se']) > 0).sum())} sur {len(avant_q)}")

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8), sharey=True)
tracer_ee(axes[0], ee, "Tendances parallèles (cas de 7.3.4)")
tracer_ee(axes[1], ee_q, "Villes traitées +1 % par mois d'avance")
axes[0].set_ylabel("effet relatif (log des commandes)")
plt.tight_layout()
plt.savefig("figures/ch07-did-violation.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

```
Effet estimé par DiD avec tendances NON parallèles : 0.315 (erreur-type 0.031) | effet vrai : 0.150
mois « avant » dont l'IC à 95 % exclut 0 : 2 sur 17
figure enregistrée
```

Le résultat est trompeur : l'estimation de l'effet (0,315) est à peu près le **double** de la vérité (0,15), avec une erreur-type (0,031) identique à celle du cas sans défaut : l'intervalle de confiance **exclut la vérité**, et rien dans ce seul chiffre ne l'indique. Mais l'étude d'événement le **révèle** : dans le graphique de droite, les coefficients ont une **pente ascendante** dès avant le lancement (ils passent d'environ  $-0,1$  en début de période à environ  $+0,1$  juste avant), et 2 des 17 intervalles d'avant le lancement excluent déjà zéro. C'est pourquoi on la regarde toujours.

Un second diagnostic complète l'étude d'événement : le **test placebo**. On refait l'analyse **sur la seule période d'avant** en prétendant que la campagne a commencé plus tôt (disons au mois 9).



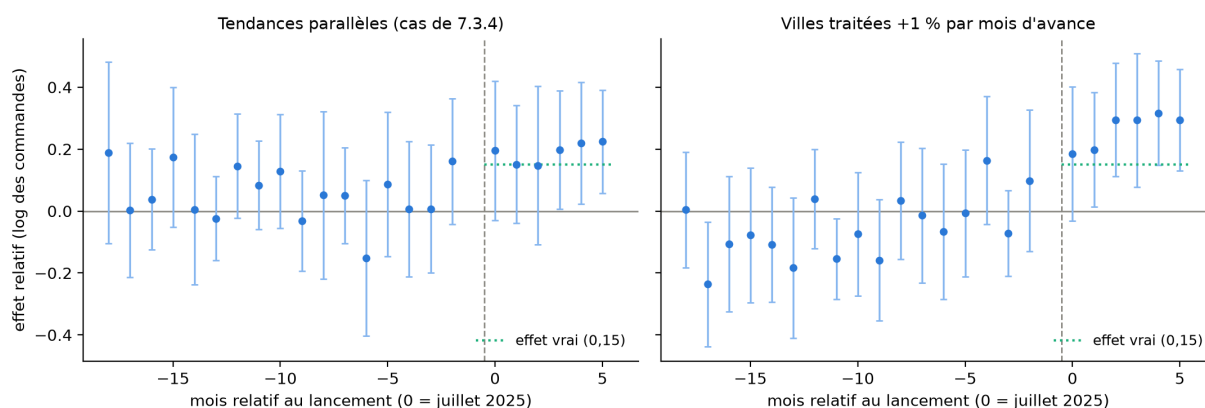


Figure 78: Études d'événement comparées : à gauche les tendances sont parallèles (coefficients proches de zéro avant le lancement) ; à droite, les villes traitées croissaient plus vite, et les coefficients montent déjà avant le lancement.

Comme il ne s'est rien passé, l'effet « placebo » devrait être proche de zéro.

```
for nom, df in [("tendances parallèles", p), ("tendances non parallèles", q)]:
    b_pl, se_pl = twfe(df, debut=9, fin=18)
    print(f"placebo ({nom}) : effet « fictif » = {b_pl:+.3f} (erreur-type {se_pl:.3f}, rapport {b_pl
    ↪ / se_pl:+.1f})")
```

```
placebo (tendances parallèles) : effet « fictif » = -0.062 (erreur-type 0.038, rapport -1.6)
placebo (tendances non parallèles) : effet « fictif » = +0.097 (erreur-type 0.042, rapport +2.3)
```

Quand les tendances sont parallèles, l'effet placebo ( $-0,062$ ) n'est pas distinguable de zéro (rapport à son erreur-type de  $-1,6$ , inférieur à 2 en valeur absolue). Quand elles ne le sont pas, l'effet placebo ( $+0,097$ ) est environ 2,3 fois son erreur-type : le placebo **donne l'alerte**. Notez qu'avec un seul test et un seuil à 2, l'alerte reste un indice, pas une preuve : le placebo du cas parallèle atteint déjà  $-1,6$ .

**Que faire si les tendances ne sont pas parallèles ?** Plusieurs remèdes existent, tous avec un prix. (1) **Ajouter une tendance linéaire propre à chaque groupe** : on peut le faire en ajoutant `groupe_traite:t` dans la régression (essayons ci-dessous). Mais cela suppose que la tendance différentielle est *linéaire* et qu'on peut l'extrapoler après le lancement. (2) **Choisir de meilleurs témoins** : des villes comparables par la taille et la dynamique, ou un **contrôle synthétique** qui construit un témoin sur mesure à partir d'une combinaison pondérée de villes. (3) **Changer de question ou de méthode**. Aucun remède n'est automatique ; ils reposent tous sur d'autres hypothèses.

```
b_t, se_t = twfe(q, tendance_groupe=True)
print(f"avec tendance linéaire propre au groupe traité : effet = {b_t:.3f} (erreur-type {se_t:.3f})
    ↪ | vrai : 0.150")
b_t2, se_t2 = twfe(p, tendance_groupe=True)
print(f"(le même modèle sur les données initiales, où il n'était pas nécessaire : effet = {b_t2:.3f},
    ↪ erreur-type {se_t2:.3f})")
```

```
avec tendance linéaire propre au groupe traité : effet = 0.185 (erreur-type 0.063) | vrai : 0.150
(le même modèle sur les données initiales, où il n'était pas nécessaire : effet = 0.181, erreur-type
    ↪ 0.062)
```

La tendance de groupe ramène l'estimation de 0,315 à 0,185, c'est-à-dire dans l'incertitude statistique de la vérité (0,150 ; l'écart est de 0,035, soit environ la moitié de l'erreur-type), mais au

prix d'une **perte de précision** : l'erreur-type double (0,063 au lieu de 0,031). Et sur les données initiales, où cette tendance n'était pas nécessaire, elle ne fait que gonfler l'erreur-type (0,062 au lieu de 0,032) et déplacer l'estimation (0,181 au lieu de 0,138). C'est le compromis typique entre robustesse et précision.

### 7.3.6 Ce que la DiD ne règle pas

- **Les effets de débordement (SUTVA).** Si la campagne à Tunis fait aussi venir des clients de la banlieue (une ville « témoin »), les témoins sont contaminés et l'effet est sous-estimé. Il faut choisir des témoins suffisamment éloignés ou modéliser le débordement.
- **L'anticipation.** Si les clients retardent leurs achats à l'annonce de la campagne, les mois d'avant sont déjà « traités ». L'étude d'événement le montre.
- **Un choc simultané.** Si, à la même date, un événement touche *seulement* les villes traitées (un concurrent s'installe à Sfax), la DiD l'attribue à la campagne.
- **Les lancements échelonnés.** Si les villes sont traitées à des dates différentes, la régression TWFE ci-dessus peut donner un résultat **trompeur** : elle utilise parfois des villes *déjà traitées* comme témoins de villes traitées plus tard, ce qui fausse la comparaison quand l'effet varie dans le temps. Des estimateurs modernes (Callaway et Sant'Anna, Sun et Abraham, entre autres) corrigent ce problème. Nous ne les présentons pas ici (nous n'avons qu'une seule date de lancement) et **ne les avons pas exécutés** ; si votre situation est échelonnée, il faut les étudier avant de se fier à une régression TWFE simple.
- **Le choix des témoins.** Ici, huit grandes villes ont été choisies pour la campagne *parce qu'elles étaient grandes* : cela ne gêne pas la DiD tant que la croissance n'est pas liée à la taille. Mais si Yasmine avait choisi les villes *parce qu'elles étaient en croissance*, l'hypothèse serait fausse.

**La vérité, révélée.** Dans la simulation, la campagne multiplie les commandes par  $e^{0,15} = 1,162$ , soit +16,2 %. L'estimation par DiD en logarithme est de 0,138 (erreur-type 0,032), soit +14,8 % : proche de la vérité, avec un intervalle de confiance qui la contient. L'estimation avant/après seule (+16,1 commandes, soit +34 % des 46,9 commandes d'avant) et la comparaison traités contre témoins seule (63,0 contre 36,1 commandes, soit +75 %) auraient donné, en termes relatifs, des valeurs respectivement **un peu plus de deux fois** et **plus de quatre fois** trop élevées.

**À retenir (7.3).** La différence de différences compare l'**évolution** des traités à celle des témoins : elle élimine les différences **de niveau** stables et les chocs **communs**. Elle repose sur l'hypothèse invérifiable de **tendances parallèles** (à la bonne échelle) : on la défend avec un **graphique**, une **étude d'événement** et des **tests placebo**. Les erreurs-types doivent être **groupées**, et les lancements échelonnés exigent des méthodes spécifiques.

## 7.4 Variables instrumentales : un coup de pouce aléatoire

**Intuition.** Les méthodes de 7.2 et 7.3 exigent de **mesurer** les facteurs de confusion (ou de les supposer stables dans le temps). Mais le facteur de confusion le plus redoutable est celui que l'on **ne peut pas** mesurer : la passion d'un client pour l'artisanat, son goût, sa motivation. Une **variable instrumentale** est un levier extérieur qui pousse le traitement dans un sens, **sans** passer par la voie cachée. Si un tirage au sort fait suivre le compte à quelques clients de plus, la différence de dépense qui en résulte ne peut pas venir de leur passion : le hasard ne la connaît pas.

### 7.4.1 Le problème : suivre le compte, cause ou symptôme ?

Yasmine remarque que les clients qui **suivent** le compte Instagram de Dar Jasmin dépensent plus. Faut-il investir pour augmenter le nombre d'abonnés ? Tout dépend du sens de la causalité : est-ce que **suivre** fait dépenser (par les publications, les promotions, le sentiment d'appartenance) ? Ou est-ce que les clients **passionnés** à la fois suivent le compte et dépensent beaucoup, sans que l'un cause l'autre ?

Le fichier `ch07-iv.csv` contient 5 000 clients : leur âge, la dépense annuelle, le fait de suivre le compte (`suit_instagram`, choix libre), et une variable particulière, `rappel` : un e-mail invitant à suivre le compte a été envoyé à **la moitié des clients tirée au hasard**. Le code du simulateur est imprimé ci-dessous, avec la vérification qu'il redonne exactement le fichier. (Lisez-le après avoir regardé les résultats : il révèle la vérité. Nous ne l'utiliserons pas pour l'analyse.)

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from statsmodels.sandbox.regression.gmm import IV2SLS
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
ENCRE, GRIS = "#0b0b0b", "#898781"

def iv(n=5000, seed=7003, force=2.0):
    """Suivre le compte Instagram (choix libre, influencé par la passion non observée) ; instrument :
    ↳ rappel e-mail aléatoire."""
    rng = np.random.default_rng(seed)
    age = np.clip(np.round(rng.normal(36, 11, n)), 18, 75).astype(int)
    passion = rng.normal(0, 1, n) # NON OBSERVÉE
    rappel = rng.integers(0, 2, n) # instrument : tiré au hasard
    eta = -0.3 + force * rappel + 0.8 * passion + 0.01 * (age - 36)
    suit = rng.binomial(1, 1 / (1 + np.exp(-eta)))
    depense = 80 + 25 * suit + 30 * passion - 0.8 * (age - 36) + rng.normal(0, 40, n)
    return pd.DataFrame({"id_client": np.arange(1, n + 1), "age": age, "rappel": rappel,
        ↳ "suit_instagram": suit,
        "depense": np.round(depense, 2)})

d = pd.read_csv("donnees/ch07-iv.csv")
print("le code ci-dessus reproduit exactement le fichier :", d.equals(iv()))
print(d.head(5).to_string(index=False))
print(f"\n{len(d)} clients ; {d['suit_instagram'].mean():.1%} suivent le compte ;
↳ {d['rappel'].mean():.1%} ont reçu le rappel")
print(d.groupby("suit_instagram")["depense"].agg(["count", "mean"]).round(1).rename(index={0: "ne
↳ suit pas", 1: "suit"}))
```

le code ci-dessus reproduit exactement le fichier : True

| id_client | age | rappel | suit_instagram | depense |
|-----------|-----|--------|----------------|---------|
| 1         | 39  | 1      | 1              | 5.76    |
| 2         | 40  | 0      | 0              | 124.21  |
| 3         | 37  | 1      | 1              | 142.74  |
| 4         | 35  | 0      | 1              | 69.39   |
| 5         | 58  | 1      | 1              | 51.32   |

5000 clients ; 62.3% suivent le compte ; 48.8% ont reçu le rappel

|                | count | mean  |
|----------------|-------|-------|
| suit_instagram |       |       |
| ne suit pas    | 1886  | 68.7  |
| suit           | 3114  | 109.8 |

La comparaison naïve est à l'œuvre : ceux qui suivent le compte dépensent bien plus. Un modèle de régression qui ajuste sur l'âge (la seule covariable observée) n'y change presque rien :

```
ols = smf.ols("depense ~ suit_instagram + age", d).fit(cov_type="HC1")
b_ols, se_ols = ols.params["suit_instagram"], ols.bse["suit_instagram"]
print(f"MCO : effet de suivre le compte = {b_ols:.1f} DT (erreur-type {se_ols:.1f}, IC 95 % : [{b_ols
↪ - 1.96 * se_ols:.1f} ; {b_ols + 1.96 * se_ols:.1f}])")
```

```
MCO : effet de suivre le compte = 41.9 DT (erreur-type 1.4, IC 95 % : [39.1 ; 44.7])
```

Rien dans ce modèle n'avertit que le chiffre est faux : l'erreur-type est petite, l'intervalle étroit. Mais il suppose que les clients qui suivent le compte et ceux qui ne le suivent pas **ne diffèrent que par l'âge**, ce que l'on a de bonnes raisons de contester. Un facteur non mesuré, la passion, pousse à la fois à suivre et à dépenser : c'est la fourche de 7.1.5, avec une cause commune que **personne ne peut ajuster**.

```
fig, ax = plt.subplots(figsize=(8.2, 2.9))
pos = {"Rappel e-mail\n(instrument Z)": (0, 0.5), "Suit le compte\n(traitement T)": (2.2, 0.5),
↪ "Dépense\n(résultat Y)": (4.4, 0.5),
    "Passion\n(non observée U)": (3.3, 1.7)}
ax.set_xlim(-0.9, 5.4); ax.set_ylim(-0.1, 2.3); ax.axis("off")
rendu = fig.canvas.get_renderer()
demi = {}
for nom, (x, y) in pos.items():
    couleur_boite = "#f7d3c5" if "non observée" in nom else "#cde2fb"
    t = ax.text(x, y, nom, ha="center", va="center", fontsize=9.5, zorder=3,
        bbox=dict(boxstyle="round,pad=0.4", fc=couleur_boite, ec=ORANGE if "non observée" in
↪ nom else BLEU, lw=1.2))
    e = t.get_window_extent(rendu)
    demi[nom] = (e.width * 72 / fig.dpi / 2 + 6, e.height * 72 / fig.dpi / 2 + 6)
for (a, b), couleur in [(("Rappel e-mail\n(instrument Z)", "Suit le compte\n(traitement T)"), ENCRE),
    (("Suit le compte\n(traitement T)", "Dépense\n(résultat Y)"), ORANGE),
    (("Passion\n(non observée U)", "Suit le compte\n(traitement T)"), ENCRE),
    (("Passion\n(non observée U)", "Dépense\n(résultat Y)"), ENCRE)]:
    pa, pb = ax.transData.transform(pos[a]), ax.transData.transform(pos[b])
    dx, dy = (pb - pa) * 72 / fig.dpi
    longueur = np.hypot(dx, dy)
    bord = lambda nom: min(demi[nom][0] / abs(dx) if dx else np.inf, demi[nom][1] / abs(dy) if dy else
↪ np.inf) * longueur + 2
    ax.annotate("", xy=pos[b], xytext=pos[a], zorder=2,
        arrowprops=dict(arrowstyle="->", color=couleur, lw=2.4 if couleur == ORANGE else 1.4,
↪ shrinkA=bord(a), shrinkB=bord(b)))
ax.text(1.1, 0.12, "pas de flèche directe\nde Z vers Y", fontsize=8.5, color=GRIS, ha="center",
↪ va="center")
plt.savefig("figures/ch07-iv-schema.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

### 7.4.2 Les conditions d'un bon instrument

Le schéma résume la structure. Un **instrument**  $Z$  pour estimer l'effet de  $T$  sur  $Y$  doit satisfaire trois conditions :

1. **Pertinence** :  $Z$  influence  $T$ . (Le rappel fait réellement suivre le compte à davantage de clients.) Cette condition **se vérifie** sur les données.
2. **Indépendance** :  $Z$  est indépendante des facteurs cachés  $U$  (« comme tiré au sort »). Ici, c'est vrai **par construction** : nous avons tiré le rappel au hasard. Quand  $Z$  n'est pas randomisé, c'est une hypothèse à défendre.
3. **Exclusion** :  $Z$  n'affecte  $Y$  **que par**  $T$  (aucune flèche directe de  $Z$  vers  $Y$ ). Ici : le rappel ne fait dépenser que parce qu'il fait suivre le compte. Cette condition **ne se vérifie pas** sur les données : c'est un argument de métier. Nous verrons ce qui arrive quand elle est fausse.

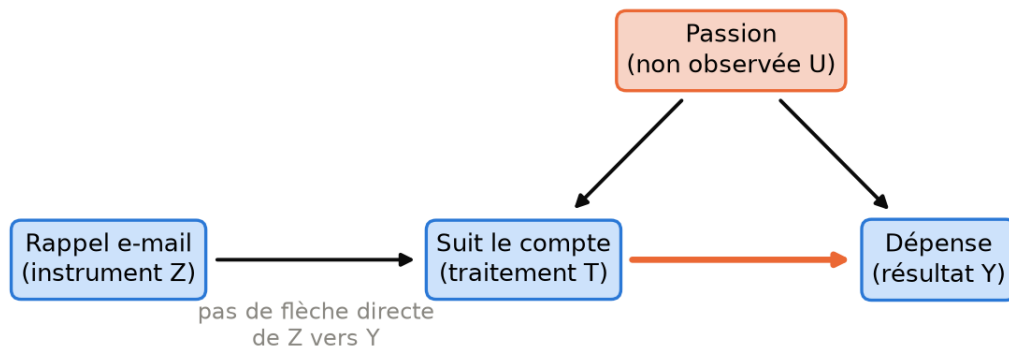


Figure 79: Le schéma d'une variable instrumentale : l'instrument Z (rappel tiré au hasard) agit sur le traitement T, qui agit sur Y ; une cause cachée U agit à la fois sur T et sur Y. Il n'existe aucune flèche directe de Z vers Y, ni de U vers Z.

Vérifions les deux premières. L'indépendance, d'abord : le tirage au sort doit équilibrer les covariables observées.

```

print("âge moyen avec rappel :", round(d.loc[d.rappel == 1, "age"].mean(), 2), "| sans rappel :",
      ↪ round(d.loc[d.rappel == 0, "age"].mean(), 2))
print(f"SMD de l'âge : {(d.loc[d.rappel == 1, 'age'].mean() - d.loc[d.rappel == 0, 'age'].mean()) /
      ↪ np.sqrt((d.loc[d.rappel == 1, 'age'].var() + d.loc[d.rappel == 0, 'age'].var()) / 2):+.3f}")
  
```

```

âge moyen avec rappel : 36.47 | sans rappel : 36.38
SMD de l'âge : +0.008
  
```

La pertinence, ensuite : c'est la **première étape**, la régression du traitement sur l'instrument. On y regarde non seulement le coefficient, mais la **statistique F**, qui mesure la force de l'instrument (nous verrons pourquoi en 7.4.6).

```

etapel = smf.ols("suit_instagram ~ rappel + age", d).fit(cov_type="HCl1")
print("part de clients qui suivent le compte, sans rappel :", round(d.loc[d.rappel == 0,
      ↪ "suit_instagram"].mean(), 3),
      ↪ "| avec rappel :", round(d.loc[d.rappel == 1, "suit_instagram"].mean(), 3))
print(f"première étape : effet du rappel sur la probabilité de suivre = {etapel.params['rappel']:.3f}
      ↪ (erreur-type {etapel.bse['rappel']:.3f})")
print(f"statistique F de l'instrument : {etapel.tvalues['rappel'] ** 2:.0f}")
  
```

```

part de clients qui suivent le compte, sans rappel : 0.441 | avec rappel : 0.813
première étape : effet du rappel sur la probabilité de suivre = 0.372 (erreur-type 0.013)
statistique F de l'instrument : 874
  
```

Le rappel augmente de façon massive la probabilité de suivre le compte : l'instrument est **très pertinent** (une statistique F de plusieurs centaines, à comparer au seuil conventionnel de 10).

### 7.4.3 L'estimateur de Wald : un rapport de deux différences

L'idée est simple. Le rappel est tiré au sort : la comparaison **avec rappel contre sans rappel** est donc une comparaison propre, comme dans une expérience (7.1.4). Elle donne deux différences :

- l'effet du rappel sur le traitement (**première étape**) :  $\pi = \mathbb{E}[T \mid Z = 1] - \mathbb{E}[T \mid Z = 0]$  ;
- l'effet du rappel sur le résultat (**forme réduite**, ou *intention de traiter*) :  $\rho = \mathbb{E}[Y \mid Z = 1] - \mathbb{E}[Y \mid Z = 0]$ .

Si le rappel n'agit sur la dépense **que** par le suivi du compte, alors  $\rho$  est l'effet de suivre multiplié par la proportion de clients que le rappel a fait suivre :  $\rho = \beta \times \pi$ . D'où l'estimateur de Wald :

$$\hat{\beta}_{\text{Wald}} = \frac{\mathbb{E}[Y | Z = 1] - \mathbb{E}[Y | Z = 0]}{\mathbb{E}[T | Z = 1] - \mathbb{E}[T | Z = 0]} = \frac{\rho}{\pi}.$$

**Une démonstration en trois lignes.** Écrivons le modèle structurel  $Y = a + \beta T + \gamma U + \varepsilon$ , où  $U$  est le facteur caché. L'indépendance et l'exclusion donnent  $\text{Cov}(Z, U) = 0$  et  $\text{Cov}(Z, \varepsilon) = 0$ . Donc

$$\text{Cov}(Z, Y) = \beta \text{Cov}(Z, T) + \gamma \underbrace{\text{Cov}(Z, U)}_{=0} + \underbrace{\text{Cov}(Z, \varepsilon)}_{=0} = \beta \text{Cov}(Z, T), \quad \text{soit} \quad \beta = \frac{\text{Cov}(Z, Y)}{\text{Cov}(Z, T)}.$$

Quand  $Z$  est binaire, cette covariance se simplifie en la différence des moyennes selon  $Z$ , d'où le rapport  $\rho/\pi$ . Le facteur caché **disparaît** du calcul : c'est tout l'intérêt de l'instrument. (Il faut en revanche que  $\text{Cov}(Z, T) \neq 0$ , d'où la condition de pertinence.)

Les deux différences se lisent directement dans les moyennes :

```
moy = d.groupby("rappel")[["suit_instagram", "depense"]].mean().round(3)
moy.index = ["sans rappel", "avec rappel"]
print(moy)
pi_hat = moy.loc["avec rappel", "suit_instagram"] - moy.loc["sans rappel", "suit_instagram"]
rho_hat = moy.loc["avec rappel", "depense"] - moy.loc["sans rappel", "depense"]
print(f"\npremière étape pi = {pi_hat:.3f}")
print(f"forme réduite rho = {rho_hat:.2f} DT")
print(f"estimateur de Wald rho / pi = {rho_hat / pi_hat:.1f} DT")
```

|             | suit_instagram | depense |
|-------------|----------------|---------|
| sans rappel | 0.441          | 90.020  |
| avec rappel | 0.813          | 98.825  |

```
première étape pi = 0.372
forme réduite rho = 8.81 DT
estimateur de Wald rho / pi = 23.7 DT
```

À la main : avec le rappel, 81 % des clients suivent le compte contre 44 % sans, soit  $\pi \approx 0,37$  ; la dépense moyenne augmente de  $\rho \approx 8,8$  DT. Le rapport donne un effet de suivre le compte d'environ  $8,8/0,372 \approx 23,7$  DT, **très inférieur** aux 41,9 DT du modèle naïf : une grande partie de l'écart de dépense entre abonnés et non-abonnés venait de la passion, pas du suivi lui-même.

#### 7.4.4 Les moindres carrés en deux étapes (2SLS)

L'estimateur de Wald se généralise à plusieurs covariables et plusieurs instruments avec les **moindres carrés en deux étapes** :

1. **Première étape** : régresser le traitement  $T$  sur l'instrument  $Z$  et les covariables  $W$  (ici l'âge) ; en déduire la **partie de  $T$  expliquée par l'instrument**,  $\hat{T}$  ;
2. **Seconde étape** : régresser  $Y$  sur  $\hat{T}$  et  $W$ .

Comme  $\hat{T}$  ne contient que la variation de  $T$  **causée par l'instrument** (donc indépendante de  $U$ ), le coefficient de  $\hat{T}$  est l'effet recherché. Écrivons-le **à la main**, avec la formule matricielle  $\hat{\beta} = (X^T P_Z X)^{-1} X^T P_Z y$  où  $P_Z = Z(Z^T Z)^{-1} Z^T$  est le projecteur sur les instruments :

```
n = len(d)
y = d["depense"].to_numpy()
X = np.column_stack([np.ones(n), d["suit_instagram"], d["age"]]).astype(float) # régresseurs,
↪ dont le traitement
```



```

Z = np.column_stack([np.ones(n), d["rappel"], d["age"]]).astype(float) # instruments +
↳ covariables exogènes

# Étape 1 : traitement expliqué par l'instrument
pi = np.linalg.lstsq(Z, X[:, 1], rcond=None)[0]
T_chapeau = Z @ pi
X_chapeau = X.copy()
X_chapeau[:, 1] = T_chapeau
# Étape 2 : régression de Y sur le traitement « prédit »
beta = np.linalg.lstsq(X_chapeau, y, rcond=None)[0]

# Même résultat avec la formule matricielle (X' Pz X)^-1 X' Pz y
PzX = Z @ np.linalg.solve(Z.T @ Z, Z.T @ X)
beta_matrice = np.linalg.solve(PzX.T @ X, PzX.T @ y)
# Cas « juste identifié » (un instrument pour un traitement) : (Z' X)^-1 Z' y
beta_iv = np.linalg.solve(Z.T @ X, Z.T @ y)
print("2SLS en deux régressions :", beta.round(3))
print("2SLS, formule matricielle:", beta_matrice.round(3))
print("estimateur IV (Z'X)^-1 Z'y :", beta_iv.round(3))

2SLS en deux régressions : [107.609  23.839 -0.773]
2SLS, formule matricielle: [107.609  23.839 -0.773]
estimateur IV (Z'X)^-1 Z'y : [107.609  23.839 -0.773]

```

Les trois écritures coïncident, et le coefficient du traitement (deuxième nombre) est proche de l'estimateur de Wald. **Attention** aux erreurs-types : si l'on lit simplement la sortie d'une régression de  $Y$  sur  $\hat{T}$ , on obtient des erreurs-types **fausses**, parce que les résidus qu'elle calcule utilisent  $\hat{T}$  au lieu du vrai traitement  $T$ . La bonne variance utilise les résidus  $y - X\hat{\beta}$  calculés avec les **vrais** régresseurs :

$$\widehat{\text{Var}}(\hat{\beta}) = \hat{\sigma}^2 (X^\top P_Z X)^{-1}, \quad \hat{\sigma}^2 = \frac{\|y - X\hat{\beta}\|^2}{n - k}.$$

```

k = X.shape[1]
sigma2_correct = np.sum((y - X @ beta) ** 2) / (n - k) # résidus avec le VRAI traitement
sigma2_naif = np.sum((y - X_chapeau @ beta) ** 2) / (n - k) # résidus avec le traitement «
↳ prédit » (faux)
var = np.linalg.inv(X_chapeau.T @ X_chapeau)
se_correct = np.sqrt(sigma2_correct * var[1, 1])
se_naif = np.sqrt(sigma2_naif * var[1, 1])
print(f"effet du suivi (2SLS) : {beta[1]:.2f} DT")
print(f"erreur-type correcte : {se_correct:.2f} | erreur-type « naïve » de la seconde régression :
↳ {se_naif:.2f}")

# Vérification avec l'implémentation de statsmodels
res = IV2SLS(y, X, Z).fit()
print(f"statsmodels IV2SLS : coefficient = {res.params[1]:.2f} erreur-type = {res.bse[1]:.2f}")
print(f"IC 95 % : [{beta[1] - 1.96 * se_correct:.1f} ; {beta[1] + 1.96 * se_correct:.1f}]")

effet du suivi (2SLS) : 23.84 DT
erreur-type correcte : 3.80 | erreur-type « naïve » de la seconde régression : 4.03
statsmodels IV2SLS : coefficient = 23.84 erreur-type = 3.80
IC 95 % : [16.4 ; 31.3]

```

Notre implémentation reproduit celle de statsmodels (coefficient 23,84, erreur-type 3,80). L'erreur-type « naïve » de la seconde régression (4,03) est ici un peu plus grande que la correcte ; dans d'autres situations elle peut être trop petite : elle n'a **aucune garantie**, il ne faut jamais la lire. Le prix de la robustesse à la confusion cachée se paie en **précision** : l'erreur-type de l'estimateur par variable instrumentale (3,8 DT) est presque trois fois celle des moindres carrés ordinaires (1,4 DT), parce qu'on n'utilise qu'une partie de la variation de  $T$  (celle que le rappel



déclenche).

**À retenir (7.4.1 à 7.4.4).** Une variable instrumentale permet d'estimer un effet causal malgré un facteur de confusion **non observé**, à trois conditions : pertinence (vérifiable), indépendance (garantie si l'instrument est randomisé) et exclusion (invérifiable). Avec un seul instrument, l'estimateur est le **rapport de Wald**  $\rho/\pi$  ; en général, c'est le 2SLS, avec des erreurs-types calculées sur les vrais régresseurs. La précision est plus faible que celle des MCO.

### 7.4.5 Ce que l'on estime vraiment : l'effet pour les « complaisants »

Un point subtil, mais essentiel pour interpréter le résultat. Un instrument ne fait changer de comportement que **certain**s clients : ceux que le rappel **décide** à suivre le compte. Imaginons trois types de clients :

- les **toujours-abonnés** (*always-takers*) suivraient le compte avec ou sans rappel ;
- les **jamais-abonnés** (*never-takers*) ne le suivraient dans aucun cas ;
- les **complaisants** (*compliers*) suivent le compte **si et seulement si** ils reçoivent le rappel.

(Nous supposons qu'il n'existe pas de « contrariants » qui feraient l'inverse du rappel : c'est l'hypothèse de **monotonie**.) Les toujours-abonnés et les jamais-abonnés ne réagissent pas au rappel : ils ne contribuent donc **pas** à la différence  $\rho$ . Seuls les complaisants y contribuent. L'estimateur de Wald estime par conséquent l'effet moyen **parmi les complaisants** (le LATE, pour *local average treatment effect*), et pas l'effet moyen sur tous les clients.

```
rng = np.random.default_rng(74)
n = 400_000
u = rng.normal(size=n)
z = rng.integers(0, 2, n)
v = rng.logistic(size=n)
↳ mondes
d0 = (-0.3 + 0.8 * u + v > 0).astype(int)
d1 = (-0.3 + 2.0 + 0.8 * u + v > 0).astype(int)
tau = 25 + 20 * u
↳ passionnés
T = np.where(z == 1, d1, d0)
Y = 80 + tau * T + 30 * u + rng.normal(0, 40, n)

complaisants = (d0 == 0) & (d1 == 1)
print(f"toujours-abonnés : {(d0 == 1).mean():.1%} | jamais-abonnés : {(d1 == 0).mean():.1%} |
↳ complaisants : {complaisants.mean():.1%}")
print(f"effet moyen sur tous les clients (ATE) : {tau.mean():.2f} DT")
print(f"effet moyen sur les complaisants (LATE vrai) : {tau[complaisants].mean():.2f} DT")
print(f"estimateur de Wald : {(Y[z == 1].mean() - Y[z == 0].mean()) / (T[z
↳ == 1].mean() - T[z == 0].mean()):.2f} DT")
print(f"passion moyenne : complaisants {u[complaisants].mean():+.2f}, toujours-abonnés {u[d0 ==
↳ 1].mean():+.2f}, jamais-abonnés {u[d1 == 0].mean():+.2f}")

toujours-abonnés : 43.4% | jamais-abonnés : 18.1% | complaisants : 38.4%
effet moyen sur tous les clients (ATE) : 24.99 DT
effet moyen sur les complaisants (LATE vrai) : 21.64 DT
estimateur de Wald : 21.75 DT
passion moyenne : complaisants -0.17, toujours-abonnés +0.40, jamais-abonnés -0.60
```

Le rappel convainc 38 % des clients (les complaisants) ; 43 % suivaient déjà le compte et 18 % ne le suivront jamais. L'estimateur de Wald (21,75 DT) tombe sur le **LATE** (21,64 DT) et **non** sur l'ATE (24,99 DT) : les deux diffèrent dès que l'effet varie d'un client à l'autre. Ici, les passionnés profitent davantage du suivi, mais ce sont aussi eux qui suivent le compte **sans** qu'on le leur demande (passion moyenne de +0,40 chez les toujours-abonnés), donc l'instrument n'apprend rien sur

eux ; les complaisants ( $-0,17$ ) sont des clients de passion intermédiaire, un peu en dessous de la moyenne, d'où un effet moyen un peu plus faible.

**Conséquence pratique.** Le LATE est l'effet pour les clients **sur lesquels le levier agit**. Si Yasmine veut savoir ce que rapporterait une campagne d'invitation **comme celle du rappel**, c'est exactement la bonne quantité. Si elle veut l'effet de suivre le compte pour *tous* ses clients, ce n'est pas garanti. Un instrument différent aurait d'autres complaisants, donc un autre LATE.

### 7.4.6 Quand l'instrument est faible

Que se passe-t-il quand l'instrument ne pousse presque personne à changer de comportement ? Le dénominateur de Wald,  $\pi$ , devient très petit, et le rapport  $\rho/\pi$  explose à la moindre fluctuation. Dans notre fichier, le rappel faisait passer la part d'abonnés de 44 % à 81 %. Refaisons l'analyse avec un rappel **beaucoup moins efficace** (la fonction `iv` a un paramètre `force`) sur 5 000 clients :

```
dw = iv(force=0.25)
et1 = smf.ols("suit_instagram ~ rappel + age", dw).fit(cov_type="HC1")
Xw = np.column_stack([np.ones(len(dw)), dw["suit_instagram"], dw["age"]]).astype(float)
Zw = np.column_stack([np.ones(len(dw)), dw["rappel"], dw["age"]]).astype(float)
rw = IV2SLS(dw["depense"].to_numpy(), Xw, Zw).fit()
print(f"part d'abonnés sans rappel : {dw.loc[dw.rappel == 0, 'suit_instagram'].mean():.3f} avec
↪ rappel : {dw.loc[dw.rappel == 1, 'suit_instagram'].mean():.3f}")
print(f"statistique F de l'instrument : {et1.tvalues['rappel'] ** 2:.1f}")
print(f"IV : effet = {rw.params[1]:.1f} DT erreur-type = {rw.bse[1]:.1f} IC 95 % : [{rw.params[1]
↪ - 1.96 * rw.bse[1]:.0f} ; {rw.params[1] + 1.96 * rw.bse[1]:.0f}"])
```

```
part d'abonnés sans rappel : 0.441 avec rappel : 0.484
statistique F de l'instrument : 9.2
IV : effet = 14.9 DT erreur-type = 33.8 IC 95 % : [-51 ; 81]
```

L'estimation ponctuelle (14,9 DT) reste de l'ordre de grandeur de la vérité, mais l'intervalle de confiance est devenu **énorme** : de  $-51$  à  $+81$  DT, il est compatible avec un effet fortement négatif comme avec un effet très élevé. Comme les données sont simulées, on peut aller plus loin : **répéter** l'expérience 500 fois avec de nouvelles données, et regarder la distribution de l'estimateur pour trois forces d'instrument. On compte aussi la fréquence à laquelle l'intervalle de confiance à 95 % **contient** vraiment la vérité (25 DT).

```
def estimation_iv_simple(df):
    """Estimateur IV (Wald) et son erreur-type robuste, sans covariable. Renvoie (estimation,
    ↪ erreur-type, statistique F)."""
    zc = df["rappel"].to_numpy(float) - df["rappel"].mean()
    xc = df["suit_instagram"].to_numpy(float) - df["suit_instagram"].mean()
    yc = df["depense"].to_numpy(float) - df["depense"].mean()
    b = (zc @ yc) / (zc @ xc)
    e = yc - b * xc
    se = np.sqrt(np.sum(zc ** 2 * e ** 2) / abs(zc @ xc))
    r = np.corrcoef(zc, xc)[0, 1]
    F = r ** 2 * (len(df) - 2) / (1 - r ** 2)
    return b, se, F

cas = [("forte (force 2,0, n = 5000)", 5000, 2.0), ("moyenne (force 0,25, n = 5000)", 5000, 0.25),
↪ ("faible (force 0,25, n = 1000)", 1000, 0.25)]
resultats = {}
for nom, n_mc, force in cas:
    est = np.array([estimation_iv_simple(iv(n=n_mc, seed=10_000 + k, force=force)) for k in
    ↪ range(500)])
    couverture = np.mean(np.abs(est[:, 0] - 25) <= 1.96 * est[:, 1])
    resultats[nom] = est[:, 0]
    q5, q25, q50, q75, q95 = np.percentile(est[:, 0], [5, 25, 50, 75, 95])
```

```
print(f"{nom:32s} F médian {np.median(est[:, 2]):6.1f} | estimation médiane {q50:5.1f}, 5%-95% :  
↳ [{q5:7.1f} ; {q95:6.1f}] | l'IC contient 25 dans {couverture:.0%} des cas")
```

```
forte (force 2,0, n = 5000)      F médian  936.7 | estimation médiane  25.1, 5%-95% : [  19.3 ;  
↳ 31.1] | l'IC contient 25 dans 97% des cas  
moyenne (force 0,25, n = 5000) F médian  14.7 | estimation médiane  25.5, 5%-95% : [ -24.7 ;  
↳ 67.5] | l'IC contient 25 dans 98% des cas  
faible (force 0,25, n = 1000)  F médian   3.0 | estimation médiane  30.8, 5%-95% : [ -185.0 ;  
↳ 159.1] | l'IC contient 25 dans 99% des cas
```

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.2), sharey=False)  
for ax, (nom, est), couleur in zip(axes, resultats.items(), [BLEU, VIOLET, ORANGE]):  
    ax.hist(np.clip(est, -150, 200), bins=np.linspace(-150, 200, 60), color=couleur, alpha=0.85)  
    ax.axvline(25, color=AQUA, lw=2)  
    ax.set_title(nom.split(" (")[0].capitalize() + " (" + nom.split(" (")[1], fontsize=9.5)  
    ax.set_xlabel("estimation IV (DT), tronquée à [-150 ; 200]")  
    ax.grid(axis="x", visible=False)  
    for s in ("top", "right"):  
        ax.spines[s].set_visible(False)  
axes[0].set_ylabel("nombre de simulations")  
plt.tight_layout()  
plt.savefig("figures/ch07-iv-faible.png", dpi=200, bbox_inches="tight")  
print("figure enregistrée")
```

figure enregistrée

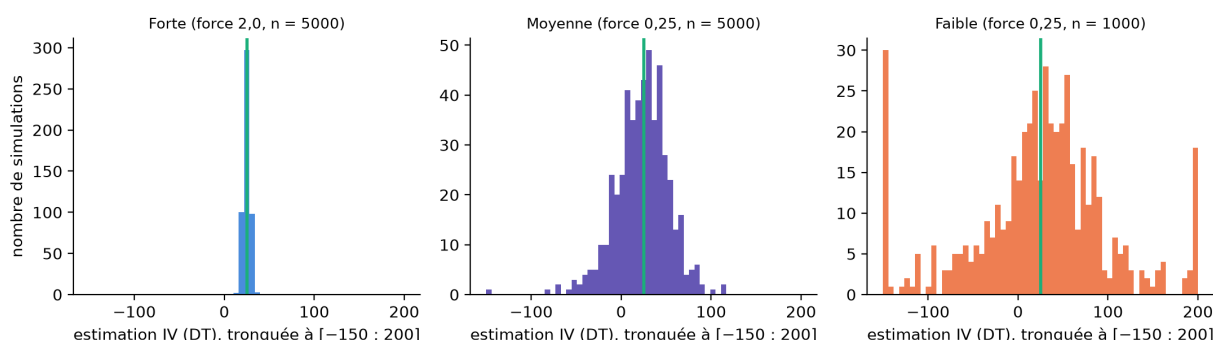


Figure 80: Distribution de l'estimateur IV sur 500 répétitions, pour un instrument fort, moyen et faible. La ligne verte marque la vraie valeur (25 DT). Plus l'instrument est faible, plus la distribution est étalée et plus elle comporte de valeurs aberrantes.

**Lecture.** Avec l'instrument **fort** (F médian de 937), les estimations se concentrent autour de la vérité : médiane de 25,1 DT, et 90 % des estimations entre 19,3 et 31,1. Avec l'instrument **moyen** (F médian de 14,7, au-dessus du seuil habituel de 10), la médiane reste bonne (25,5 DT) mais la dispersion est déjà considérable : 90 % des estimations tombent entre  $-24,7$  et  $+67,5$  DT, soit une fourchette de plus de 90 DT, et plus d'une fois sur vingt l'estimation a le **mauvais signe**. Avec l'instrument **faible** (F médian de 3,0), c'est pire : 90 % des estimations sont entre  $-185$  et  $+159$  DT. Les histogrammes (tronqués) montrent les **queues lourdes** typiques de ce rapport dont le dénominateur peut être proche de zéro.

Le résultat sur la couverture est plus nuancé qu'on ne le dit parfois : les intervalles de confiance à 95 % contiennent bien la vérité (97 %, 98 % et 99 % des cas). Ils sont donc **valides**, mais parce qu'ils sont devenus **immenses** : un intervalle de  $-185$  à  $+159$  DT contient la vérité... et presque tout le reste. L'instrument faible ne produit pas ici des conclusions fausses mais confiantes ; il produit des conclusions **inutilisables**. (Des distorsions plus sournoises existent dans la littérature, notamment avec plusieurs instruments faibles ou une forte confusion ; notre simulation, à un seul instrument, ne les fait pas apparaître, et il ne faut pas en conclure que la faiblesse est sans

danger.)

**La règle du «  $F > 10$  ».** Une règle empirique courante dit qu'un instrument est « assez fort » quand la statistique  $F$  de la première étape dépasse 10. C'est une règle **approximative**, plutôt optimiste (de nombreux auteurs recommandent des seuils plus élevés quand on veut des intervalles de confiance fiables), et elle ne remplace pas le jugement : regardez l'ordre de grandeur du  $F$  et la largeur des intervalles. Quand l'instrument est faible, des méthodes d'inférence robustes existent (test d'Anderson-Rubin) ; nous ne les détaillons pas ici.

### 7.4.7 Quand l'exclusion est fausse

La dernière condition, l'exclusion, est la plus fragile : elle ne se teste pas. Voyons ce que coûte sa violation. Supposons que le rappel ne se contente pas d'inviter à suivre le compte, mais contienne aussi un **bon de réduction** qui fait dépenser **directement** 15 DT de plus, qu'on suive le compte ou non. Cette flèche de  $Z$  vers  $Y$  ne passe pas par  $T$ .

```
dv = iv()
dv["depense"] = dv["depense"] + 15 * dv["rappel"]          # effet direct du rappel sur la
↳ dépense : viole l'exclusion
Xv = np.column_stack([np.ones(len(dv)), dv["suit_instagram"], dv["age"]]).astype(float)
Zv = np.column_stack([np.ones(len(dv)), dv["rappel"], dv["age"]]).astype(float)
rv = IV2SLS(dv["depense"].to_numpy(), Xv, Zv).fit()
print(f"IV avec exclusion violée : effet = {rv.params[1]:.1f} DT (erreur-type {rv.bse[1]:.1f}, IC 95
↳ % : [{rv.params[1] - 1.96 * rv.bse[1]:.0f} ; {rv.params[1] + 1.96 * rv.bse[1]:.0f}]) | vérité :
↳ 25")
print(f"biais théorique : effet direct / première étape = 15 / {pi_hat:.3f} = {15 / pi_hat:.1f} DT")
```

```
IV avec exclusion violée : effet = 64.2 DT (erreur-type 3.8, IC 95 % : [57 ; 72]) | vérité : 25
biais théorique : effet direct / première étape = 15 / 0.372 = 40.3 DT
```

Le résultat (64,2 DT au lieu de 25) est faux de **près de 40 dinars**, soit presque exactement le biais théorique de 40,3 DT, avec une erreur-type aussi petite qu'avant (3,8) : l'instrument violé fournit une estimation précise, mais de la mauvaise quantité. Le biais vaut  $\gamma_Z/\pi$ , où  $\gamma_Z$  est l'effet direct du rappel et  $\pi$  la force de la première étape : même une petite violation de l'exclusion est **amplifiée** par un instrument faible (division par un  $\pi$  petit). Rien dans les données ne signale le problème (avec un seul instrument, l'hypothèse n'est pas testable). Avec **plusieurs** instruments, on peut tester leur cohérence mutuelle (test de suridentification, de Sargan ou de Hansen), mais ce test ne détecte pas les cas où *tous* les instruments sont invalides de la même manière.

**Où trouve-t-on de bons instruments ?** C'est la vraie difficulté. Les cas les plus convaincants sont les **tirages au sort** (comme le rappel ici, ou une loterie administrative) et les **expériences naturelles** (une règle administrative, une distance, un événement météorologique). Les instruments « de bureau », choisis parce qu'ils sont corrélés au traitement et qu'on a un bon argument pour l'exclusion, sont fréquemment contestés. Face à tout instrument, la question à se poser est : « existe-t-il un chemin, même improbable, par lequel il affecterait directement le résultat ? »

### 7.4.8 Quelle méthode, quand ? Panorama et limites

Nous avons vu quatre façons de passer de l'association à la causalité. Chacune échange une hypothèse contre une autre :

| Situation                                | Méthode                                 | Hypothèse-clé<br>(invérifiable)             | Ce qui la fragilise                                            | Voir       |
|------------------------------------------|-----------------------------------------|---------------------------------------------|----------------------------------------------------------------|------------|
| On peut tirer au sort                    | <b>Expérience randomisée</b>            | l'attribution est aléatoire et respectée    | non-respect du protocole, attrition, débordements              | 7.1.4      |
| Tout ce qui guide le choix est mesuré    | <b>Ajustement, scores de propension</b> | ignorabilité conditionnelle + chevauchement | un facteur de confusion non mesuré ou mal mesuré               | 7.1.9, 7.2 |
| Données avant/après avec des témoins     | <b>Différences de différences</b>       | tendances parallèles (à la bonne échelle)   | tendances divergentes, chocs simultanés, lancements échelonnés | 7.3        |
| Un levier extérieur pousse le traitement | <b>Variable instrumentale</b>           | exclusion + indépendance de l'instrument    | effet direct de l'instrument, instrument faible                | 7.4        |

On mentionnera aussi, sans les développer ici, la **régression sur discontinuité** (un seuil arbitraire décide du traitement, par exemple une réduction à partir de 100 DT d'achat) et les **contrôles synthétiques** (un témoin construit sur mesure pour une seule unité traitée) : ce sont des cousins de la DiD et de l'IV, avec leurs propres hypothèses.

#### Les limites, dites franchement.

- **Aucune de ces méthodes ne se vérifie complètement.** Elles traduisent toutes des hypothèses sur le monde (ignorabilité, tendances parallèles, exclusion) en un estimateur. Les diagnostics (équilibre, étude d'événement, force de l'instrument) *réfutent parfois* l'hypothèse, jamais ne la *prouvent*.
- **Ce chapitre était artificiellement simple.** Nous avons simulé les données, donc nous *connaissions* la vérité et la structure. Dans la réalité, le graphe est incertain, les covariables sont imparfaitement mesurées, les effets varient d'un individu à l'autre, et les données sont tronquées ou manquantes.
- **Chaque méthode répond à une question légèrement différente** : ATE, ATT, effet pour les complaisants. Ces effets ne sont pas interchangeables, et il faut dire lequel on estime.
- **La convergence de plusieurs approches rassure.** Si une expérience, une DiD et une analyse par score de propension, qui reposent sur des hypothèses *différentes*, donnent le même ordre de grandeur, la conclusion est beaucoup plus solide. C'est la **triangulation**.
- **Le vocabulaire compte.** Écrire « l'offre *augmente* la dépense » sans préciser l'hypothèse revient à affirmer plus que ce que l'analyse justifie. Préférez « *sous l'hypothèse que...*, l'effet estimé est de... ». La recommandation honnête à un décideur est : « voici l'effet estimé, voici l'hypothèse sur laquelle il repose, et voici ce qui la rendrait fausse ».

**La vérité, révélée.** Dans la simulation, l'effet réel de suivre le compte est de **25 DT**. L'estimation par MCO donnait 42 DT (la passion, non observée, gonflait la comparaison). L'estimateur de Wald et le 2SLS donnent 23,8 DT avec l'instrument fort (23,7 pour Wald, 23,8 pour le 2SLS avec l'âge), très proche de la vérité malgré le facteur de confusion caché, mais avec une erreur-type de 3,8 DT contre 1,4 pour les MCO : la protection contre le biais se paie en précision. Si l'on se limite aux clients « complaisants », la vraie cible serait légèrement différente (21,6 DT dans notre mini-simulation avec effets variables) : l'instrument répond à une question un peu plus étroite.

**À retenir (7.4).** Une variable instrumentale contourne la confusion **non observée** grâce à un levier extérieur (idéalement tiré au sort) qui agit sur le résultat **uniquement** à travers le traitement. L'estimateur est un rapport (Wald / 2SLS) dont la précision dépend de la **force** de l'instrument, et il cible l'effet pour les **complaisants**. L'hypothèse d'exclusion ne se teste pas. Aucune des quatre méthodes du chapitre ne dispense de la question de départ : *comment les données ont-elles été fabriquées ?*

## 7.5 Exercices corrigés

**Comment s'y prendre.** Faites d'abord l'exercice **à la main**, puis vérifiez avec le code. Les exercices sont notés ★ (application directe), ★★ (il faut combiner deux idées) et ★★★ (démonstration ou analyse critique). Les corrigés suivent tous les énoncés.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats

clients = pd.read_csv("donnees/clients.csv")
panel = pd.read_csv("donnees/ch07-panel-villes.csv")
panel["vid"] = pd.factorize(panel["ville"])[0]
print(clients.shape, panel.shape)
```

```
(2000, 12) (480, 7)
```

### Énoncés

**Exercice 1 ★ (résultats potentiels).** Six clients ont les dépenses potentielles suivantes (en DT) : Aya ( $Y(0) = 60$ ,  $Y(1) = 75$ ), Bilel (40, 50), Cyrine (100, 105), Dali (80, 100), Emna (30, 40), Firas (90, 95). Les trois premiers ont reçu l'offre, les trois derniers non. (a) Calculez à la main l'ATE, l'ATT et l'ATU. (b) Quelle est la différence naïve des moyennes observées ? (c) Décomposez-la en ATT plus biais de sélection.

**Exercice 2 ★ (sous-groupes).** Dans l'expérience de `clients.csv`, calculez l'effet de l'offre sur le rachat (`rachat_12m`) **dans chaque canal d'acquisition**, avec intervalle de confiance à 95 %. Yasmine remarque : « l'offre marche presque cinq fois mieux sur Instagram que sur le site ». (a) Testez cette différence. (b) Que devez-vous répondre à Yasmine ?

**Exercice 3 ★ (Simpson).** Une campagne d'e-mails a été envoyée à des clients en ville (1 000 clients dont 600 destinataires) et à la campagne (1 000 clients dont 200 destinataires). En ville, le rachat vaut 50 % chez les destinataires et 40 % chez les autres ; à la campagne, 30 % chez les destinataires et 20 % chez les autres. (a) Calculez le rachat global chez les destinataires et chez les non-destinataires. (b) Que dit la différence globale ? Que dit la différence par zone ? (c) Laquelle croire si la zone est une cause commune de la décision d'envoi et du rachat ? Et si la zone était une conséquence de l'e-mail ?

**Exercice 4 ★★ (choisir l'ensemble d'ajustement).** On simule : un facteur de confusion  $C$  ; un traitement  $T$  qui en dépend ; un médiateur  $M$  affecté par  $T$  ; un résultat  $Y = 5T + 3M + 4C + \varepsilon$  ; et un effet commun  $K = T + Y + \varepsilon'$ . (a) Quel est l'effet **total** de  $T$  sur  $Y$  ? (b) Estimez l'effet par régression avec quatre ensembles d'ajustement :  $\emptyset$ ,  $\{C\}$ ,  $\{C, M\}$ ,  $\{C, K\}$ . (c) Lequel est correct, et que mesurent les autres ?

**Exercice 5 ★★ (stratification sur le score).** Les clients ont été classés en trois strates de score de



propension. Strate A : 400 clients, probabilité d'offre 0,2, dépense moyenne 130 (avec offre) et 100 (sans). Strate B : 400 clients, probabilité 0,5, moyennes 150 et 115. Strate C : 200 clients, probabilité 0,8, moyennes 190 et 150. (a) Calculez la différence naïve. (b) Calculez l'ATE en pondérant les effets par strate. (c) Retrouvez-le par IPW.

**Exercice 6 ★★ (chevauchement).** Reprenez l'étude observationnelle de 7.2 (ch07-observationnel.csv). (a) Restreignez l'analyse aux clients dont le score de propension est compris entre 0,1 et 0,9 : combien en reste-t-il ? (b) Réestimez l'ATE par IPW sur cet échantillon, et comparez avec l'estimation sur tous les clients. (c) Quelle quantité estime-t-on désormais ?

**Exercice 7 ★★ (DiD à la main).** Chiffre d'affaires moyen par ville (en milliers de DT) : villes traitées 120 avant, 150 après ; villes témoins 80 avant, 92 après. (a) Calculez la DiD en niveau. (b) Calculez-la en logarithme et interprétez en pourcentage. (c) Laquelle des deux hypothèses de tendances parallèles est la plus plausible si le chiffre d'affaires évolue en pourcentage ?

**Exercice 8 ★★ (inférence avec peu de groupes).** Dans le panel, ne gardez que les 12 villes témoins, et attribuez au hasard à 4 d'entre elles une « fausse campagne » à partir de  $t = 18$  : il n'y a **aucun effet** à trouver. Répétez 300 fois (graine 81) et comptez à quelle fréquence la DiD avec erreurs-types groupées paraît « significative » à 5 %. (a) Avec le seuil normal  $|t| > 1,96$ . (b) Avec le seuil de Student à 11 degrés de liberté. (c) Que concluez-vous ?

**Exercice 9 ★★ (instrument à la main).** Dans une étude, on observe : avec l'instrument ( $Z = 1$ ), 60 % des clients suivent le compte et la dépense moyenne est de 52 DT ; sans ( $Z = 0$ ), 30 % le suivent et la dépense moyenne est de 46 DT. (a) Calculez l'estimateur de Wald. (b) Quelle proportion de complaisants peut-on au mieux estimer ? (c) Si l'instrument avait en réalité un effet direct de 1,5 DT sur la dépense, de combien l'estimation serait-elle faussée ?

**Exercice 10 ★★★ (démonstration).** Soit  $Z$  binaire de probabilité  $q = \mathbb{P}(Z = 1)$ . (a) Montrez que  $\text{Cov}(Z, Y) = q(1 - q)(\mathbb{E}[Y | Z = 1] - \mathbb{E}[Y | Z = 0])$ . (b) Déduisez que  $\text{Cov}(Z, Y) / \text{Cov}(Z, T)$  est égal au rapport de Wald. (c) Vérifiez-le numériquement sur ch07-iv.csv.

**Exercice 11 ★★★ (médiation).** Reprenez la simulation de 7.1.7 (offre randomisée ; médiateur « code utilisé » ; motivation qui influence à la fois l'utilisation du code et la dépense). Cette fois, **supposez la motivation observée**. (a) Estimez l'effet direct de l'offre en ajustant sur le code et la motivation. (b) Déduisez l'effet indirect (par le code), et comparez avec la valeur vraie  $30 \times \mathbb{P}(\text{code} | \text{offre})$ . (c) Pourquoi l'ajustement sur le médiateur marche-t-il ici et pas en 7.1.7 ?

**Exercice 12 ★★★ (critique d'une étude).** Une collègue veut estimer l'effet de « venir en boutique » (traitement) sur la dépense annuelle (résultat). Elle propose comme instrument la **distance** entre le domicile du client et la boutique. Discutez chacune des trois conditions d'un instrument, proposez une vérification ou un ajustement pour chacune, et dites ce que l'on estimerait si l'instrument était valide.

## Corrigés

**Corrigé 1.** (a) Effets individuels : Aya +15, Bilel +10, Cyrine +5, Dali +20, Emna +10, Firas +5.  $ATT = (15 + 10 + 5)/3 = 10$  ;  $ATU = (20 + 10 + 5)/3 = 11,67$  ;  $ATE = 65/6 = 10,83$ . (b) Observé : traités (les  $Y(1)$  des trois premiers) 75, 50, 105, moyenne 76,67 ; non traités (les  $Y(0)$  des trois derniers) 80, 30, 90, moyenne 66,67 ; différence naïve = 10. (c)  $\mathbb{E}[Y(0) | T = 1] = (60 + 40 + 100)/3 = 66,67$  et  $\mathbb{E}[Y(0) | T = 0] = 66,67$  : le biais de sélection est **nul** (par hasard dans ce petit exemple), donc la différence naïve égale l'ATT (10). Vérifions :

```
ex1 = pd.DataFrame({"client": ["Aya", "Bilel", "Cyrine", "Dali", "Emna", "Firas"],
                    "y0": [60, 40, 100, 80, 30, 90], "y1": [75, 50, 105, 100, 40, 95], "offre": [1, 1,
                    1, 0, 0, 0]})
```



```
ex1["effet"] = ex1["y1"] - ex1["y0"]
obs = np.where(ex1.offre == 1, ex1.y1, ex1.y0)
print(f"ATE = {ex1.effet.mean():.2f} ATT = {ex1.effet[ex1.offre == 1].mean():.2f} ATU =
↳ {ex1.effet[ex1.offre == 0].mean():.2f}")
print(f"différence naïve = {obs[ex1.offre == 1].mean() - obs[ex1.offre == 0].mean():.2f}")
print(f"biais de sélection = {ex1.y0[ex1.offre == 1].mean() - ex1.y0[ex1.offre == 0].mean():.2f}")
```

```
ATE = 10.83 ATT = 10.00 ATU = 11.67
différence naïve = 10.00
biais de sélection = 0.00
```

Moralité : un biais de sélection **nul** n'est pas impossible, mais on ne peut jamais le savoir sans connaître les  $Y(0)$  des traités ; la différence naïve n'est donc fiable que si l'on a une raison de croire que la sélection est ignorable.

**Corrigé 2.** (a) Dans chaque canal, la différence de proportions et son intervalle de Wald :

```
lignes = []
for canal, g in clients.groupby("canal_acquisition"):
    t, u = g[g.offre_bienvenue == 1]["rachat_12m"], g[g.offre_bienvenue == 0]["rachat_12m"]
    diff = t.mean() - u.mean()
    se = np.sqrt(t.var() / len(t) + u.var() / len(u))
    lignes.append({"canal": canal, "clients": len(g), "effet": diff, "IC95 bas": diff - 1.96 * se,
↳ "IC95 haut": diff + 1.96 * se})
print(pd.DataFrame(lignes).round(3).to_string(index=False))

complet = smf.logit("rachat_12m ~ offre_bienvenue * C(canal_acquisition)", clients).fit(dis=0)
reduit = smf.logit("rachat_12m ~ offre_bienvenue + C(canal_acquisition)", clients).fit(dis=0)
lr = 2 * (complet.llf - reduit.llf)
print(f"\ntest de l'interaction offre x canal (rapport de vraisemblance, 2 ddl) : p =
↳ {stats.chi2.sf(lr, 2):.4f}")
```

| canal     | clients | effet | IC95 bas | IC95 haut |
|-----------|---------|-------|----------|-----------|
| Boutique  | 504     | 0.116 | 0.029    | 0.202     |
| Instagram | 816     | 0.196 | 0.129    | 0.263     |
| Site      | 680     | 0.042 | -0.033   | 0.117     |

```
test de l'interaction offre x canal (rapport de vraisemblance, 2 ddl) : p = 0.0105
```

L'effet est de +19,6 points sur Instagram contre +4,2 sur le site, avec des intervalles larges ; le test d'interaction (rapport de vraisemblance, chapitre 2) donne  $p \approx 0,01$ . (b) Que répondre ? **Prudence**. Cette analyse par sous-groupe est *exploratoire* : si Yasmine avait regardé cinq découpages différents (canal, ville, âge, année d'inscription...), le seuil de Bonferroni serait  $0,05/5 = 0,01$ , que  $p = 0,0105$  ne franchit pas (volume I, section 3.5.5). Et l'expérience n'était **pas dimensionnée** pour détecter des différences entre sous-groupes (chaque canal n'a que quelques centaines de clients par bras). **Vérité révélée** : dans le simulateur, l'effet de l'offre est **le même** dans les trois canaux (12,3 à 12,4 points, comme on peut le vérifier par la même intégration qu'en 7.1.4) ; l'écart observé n'est qu'une fluctuation d'échantillonnage, de celles qui arrivent environ une fois sur cent. La bonne réponse : « c'est une piste, pas une conclusion ; on peut la **confirmer** par une nouvelle expérience prévue pour cela ».

**Corrigé 3.** (a) Destinataires :  $600 \times 0,5 + 200 \times 0,3 = 300 + 60 = 360$  rachats sur 800, soit 45 %. Non-destinataires :  $400 \times 0,4 + 800 \times 0,2 = 160 + 160 = 320$  rachats sur 1200, soit 26,7 %. (b) Globalement l'e-mail est associé à **+18,3 points** ; par zone, à **+10 points** dans chaque zone. L'écart global **surestime** l'effet, car les destinataires sont surtout des citoyens, qui rachètent davantage de toute façon (ici le sens est l'inverse du paradoxe de 7.1.6, mais le mécanisme est le même). (c) Si la zone est une cause commune de l'envoi et du rachat, il faut **ajuster** : l'effet standardisé est  $0,5 \times 10 + 0,5 \times 10 = 10$  points. Si la zone était une **conséquence** de l'e-mail (l'e-mail pousserait les clients à déménager en ville !), ajuster sur elle serait une erreur : la différence globale deviendrait

l'effet total.

```
ex3 = pd.DataFrame({"zone": ["ville", "ville", "campagne", "campagne"], "mail": [1, 0, 1, 0],
                    "n": [600, 400, 200, 800], "taux": [0.5, 0.4, 0.3, 0.2]})
ex3["rachats"] = ex3["n"] * ex3["taux"]
glob = ex3.groupby("mail")[["n", "rachats"]].sum()
print("taux global :", (glob.rachats / glob.n).round(3).to_dict())
par_zone = ex3.pivot(index="zone", columns="mail", values="taux")
poids = ex3.groupby("zone")["n"].sum() / ex3["n"].sum()
print("effet par zone :", (par_zone[1] - par_zone[0]).round(2).to_dict(), "| poids :",
      ↪ poids.round(2).to_dict())
print("effet standardisé :", round(((par_zone[1] - par_zone[0]) * poids).sum(), 3))
```

```
taux global : {0: 0.267, 1: 0.45}
effet par zone : {'campagne': 0.1, 'ville': 0.1} | poids : {'campagne': 0.5, 'ville': 0.5}
effet standardisé : 0.1
```

**Corrigé 4.** (a)  $T$  agit directement (5) et par  $M$  (qui vaut  $2T$  en moyenne, et pèse 3 dans  $Y$ ) : effet total =  $5 + 3 \times 2 = 11$ . (b) et (c) :

```
rng = np.random.default_rng(401)
n = 50_000
conf = rng.normal(size=n) # facteur de confusion (nommé « conf »
↪ pour ne pas masquer C() de patsy)
T = (conf + rng.normal(size=n) > 0).astype(int)
M = 2 * T + rng.normal(size=n)
Y = 5 * T + 3 * M + 4 * conf + rng.normal(size=n)
K = T + Y + rng.normal(size=n)
df4 = pd.DataFrame({"Y": Y, "T": T, "M": M, "conf": conf, "K": K})
for nom, f in [{"": "Y", "T": T, "M": M, "conf": conf, "K": K},
               ↪ ~ T + conf + K"]]:
    print(f"ensemble {nom:7s} : effet estimé de T = {smf.ols(f, df4).fit().params['T']:.2f}")

ensemble ∅      : effet estimé de T = 15.50
ensemble {C}    : effet estimé de T = 11.00
ensemble {C, M} : effet estimé de T = 5.01
ensemble {C, K} : effet estimé de T = 0.08
```

Seul  $\{C\}$  donne l'effet **total** (11). Sans ajustement, on garde la **confusion** (15,5 :  $C$  pousse à la fois  $T$  et  $Y$ ) ; avec  $\{C, M\}$ , on retire la voie par le médiateur et on mesure l'effet **direct** (5), ce qui peut être voulu mais ne répond pas à la question « que fait  $T$  ? » ; avec  $\{C, K\}$ , on conditionne sur un effet commun : l'estimation s'effondre vers 0, **en créant un biais**, alors que  $K$  semble une covariable « utile ».

**Corrigé 5.** (a) Nombres de traités par strate :  $0,2 \times 400 = 80$ ,  $0,5 \times 400 = 200$ ,  $0,8 \times 200 = 160$  (total 440) ; de témoins : 320, 200, 40 (total 560). Moyenne des traités =  $(80 \times 130 + 200 \times 150 + 160 \times 190) / 440 = 160,9$  ; moyenne des témoins =  $(320 \times 100 + 200 \times 115 + 40 \times 150) / 560 = 108,9$  ; différence naïve = 52,0. (b) Effets par strate : 30, 35, 40 ; ATE =  $(400 \times 30 + 400 \times 35 + 200 \times 40) / 1000 = 34,0$ . (c) IPW : poids  $1/e$  pour les traités,  $1/(1 - e)$  pour les témoins. Dans chaque strate, le poids total des traités est  $n$  et celui des témoins aussi ; la pseudo-population a donc la même composition dans les deux groupes. L'ATE par IPW coïncide avec 34.

```
ex5 = pd.DataFrame({"strate": ["A", "B", "C"], "n": [400, 400, 200], "e": [0.2, 0.5, 0.8],
                    "m1": [130, 150, 190], "m0": [100, 115, 150]})
ex5["n1"], ex5["n0"] = ex5.n * ex5.e, ex5.n * (1 - ex5.e)
naif = (ex5.n1 * ex5.m1).sum() / ex5.n1.sum() - (ex5.n0 * ex5.m0).sum() / ex5.n0.sum()
ate = ((ex5.m1 - ex5.m0) * ex5.n).sum() / ex5.n.sum()
m1_ipw = (ex5.n1 * ex5.m1 / ex5.e).sum() / (ex5.n1 / ex5.e).sum()
m0_ipw = (ex5.n0 * ex5.m0 / (1 - ex5.e)).sum() / (ex5.n0 / (1 - ex5.e)).sum()
print(f"différence naïve = {naif:.1f} | ATE par strate = {ate:.1f} | ATE par IPW = {m1_ipw -
      ↪ m0_ipw:.1f}")
```

différence naïve = 52.0 | ATE par strate = 34.0 | ATE par IPW = 34.0

**Corrigé 6.** (a)-(b) On refait l'estimation du score, puis on restreint :

```
obs = pd.read_csv("donnees/ch07-observationnel.csv")
verite = pd.read_csv("donnees/ch07-observationnel-verite.csv")
d6 = obs.merge(verite, on="id_client")
d6["ps"] = smf.logit("offre ~ age + C(canal) + engagement", d6).fit(dis=0).predict(d6)
ate_vrai = (d6.y1 - d6.y0).mean()

def ipw_ate(df):
    w = np.where(df.offre == 1, 1 / df.ps, 1 / (1 - df.ps))
    return (np.average(df.depense[df.offre == 1], weights=w[df.offre == 1])
            - np.average(df.depense[df.offre == 0], weights=w[df.offre == 0]))

restreint = d6[(d6.ps >= 0.1) & (d6.ps <= 0.9)]
ate_restraint_vrai = (restreint.y1 - restreint.y0).mean()
print(f"clients conservés : {len(restreint)} sur {len(d6)} ({len(restreint) / len(d6):.1%})")
print(f"IPW, tous les clients : {ipw_ate(d6):.2f} (ATE vrai de la population : {ate_vrai:.2f})")
print(f"IPW, scores dans [0,1 ; 0,9] : {ipw_ate(restreint):.2f} (ATE vrai de ce sous-échantillon :
↳ {ate_restraint_vrai:.2f})")
```

```
clients conservés : 3778 sur 4000 (94.5%)
IPW, tous les clients : 14.36 (ATE vrai de la population : 15.53)
IPW, scores dans [0,1 ; 0,9] : 14.73 (ATE vrai de ce sous-échantillon : 15.59)
```

- (c) En restreignant, on change la **population cible** : on estime l'effet pour les clients dont la probabilité d'offre n'est ni très faible ni très forte (la population de « chevauchement »), et non plus pour tous. Ici, la différence est **minime** : 94,5 % des clients restent, et l'ATE vrai du sous-échantillon (15,59) est presque celui de la population (15,53), parce que le chevauchement était déjà bon ; l'estimation passe de 14,36 à 14,73. Elle serait beaucoup plus importante si l'on devait écarter une grande partie des clients : l'ATE vrai du sous-échantillon pourrait alors différer sensiblement de celui de la population dès que l'effet varie avec le score (ici, il varie avec le canal, et le canal influence le score). Dans tous les cas, l'estimation répond à une question légèrement différente (celle de la population de chevauchement), à **dire explicitement** dans le rapport.

**Corrigé 7.** (a) En niveau :  $(150 - 120) - (92 - 80) = 30 - 12 = 18$  milliers de DT. (b) En logarithme :  $\ln(150/120) - \ln(92/80) = \ln 1,25 - \ln 1,15 = 0,2231 - 0,1398 = 0,0833$  (le code donne 0,0834, sans l'arrondi intermédiaire), soit un effet relatif d'environ +8,7 % ( $e^{0,0833} - 1$ ). Les villes traitées auraient crû de 25 % ; les témoins de 15 % ; sans campagne, les traitées auraient crû de 15 % aussi, soit 138, ce qui donne un effet de  $150 - 138 = 12$  milliers, et non 18. (c) Si le chiffre d'affaires évolue en **pourcentage**, les tendances parallèles sont plausibles **en logarithme** : l'effet de 18 milliers en niveau surestime l'effet réel (il suppose que la ville traitée, plus grosse, aurait crû de 12 milliers comme la petite, alors qu'une croissance de 15 % sur 120 fait 18 : la hausse « naturelle » des grandes villes est plus grande en niveau).

```
a_b, a_a, t_b, t_a = 80, 92, 120, 150
print(f"DiD en niveau : {(t_a - t_b) - (a_a - a_b)} | DiD en log : {np.log(t_a / t_b) - np.log(a_a /
↳ a_b):.4f} (soit {np.exp(np.log(t_a / t_b) - np.log(a_a / a_b)) - 1:+.1%})")
print(f"contrefactuel multiplicatif pour les villes traitées : {t_b * a_a / a_b:.0f} -> effet {t_a -
↳ t_b * a_a / a_b:.0f}")
```

```
DiD en niveau : 18 | DiD en log : 0.0834 (soit +8.7%)
contrefactuel multiplicatif pour les villes traitées : 138 -> effet 12
```

**Corrigé 8.**

```

temoins = panel[panel["groupe_traite"] == 0]
villes = temoins["ville"].unique()
rng = np.random.default_rng(81)
t_stats = []
for _ in range(300):
    faux = set(rng.choice(villes, 4, replace=False))
    d8 = temoins.assign(camp=(temoins["ville"].isin(faux) & (temoins["t"] >= 18)).astype(int))
    m = smf.ols("np.log(commandes) ~ camp + C(ville) + C(t)", d8).fit(cov_type="cluster",
    ↪ cov_kws={"groups": d8["vid"]})
    t_stats.append(m.params["camp"] / m.bse["camp"])
t_stats = np.array(t_stats)
print(f"seuil normal 1,96          : {np.mean(np.abs(t_stats) > 1.96):.1%} de « découvertes »")
print(f"seuil de Student (11 ddl)   : {np.mean(np.abs(t_stats) > stats.t.ppf(0.975, 11)):.1%} de «
    ↪ découvertes »")

```

```

seuil normal 1,96          : 8.0% de « découvertes »
seuil de Student (11 ddl)   : 4.7% de « découvertes »

```

(a)-(b) Avec le seuil normal, le test « découvrir » un effet là où il n'y en a aucun dans environ 8 % des simulations au lieu des 5 % annoncés ; avec le seuil de Student à 11 degrés de liberté, le taux retombe à près de 5 % (4,7 %). (c) Quand le nombre de groupes (ici 12 villes, dont 4 « traitées ») est petit, les erreurs-types groupées sont **trop optimistes** et le seuil normal est trop laxiste : on obtient des faux positifs en excès. Remèdes : seuil de Student avec peu de degrés de liberté, bootstrap par groupes, ou tests de permutation (volume I, section 3.7) : c'est exactement ce que nous venons de faire, puisque la distribution de l'effet « fictif » est la distribution de référence d'un test de permutation.

**Corrigé 9.** (a)  $\hat{\beta} = \frac{52 - 46}{0,60 - 0,30} = \frac{6}{0,3} = 20$  DT. (b) La part de **complaisants** est  $\pi = 0,60 - 0,30 = 30\%$  (sous la monotonie) : l'effet de 20 DT est l'effet **pour ces 30 %** de clients. (c) Un effet direct  $\gamma_Z = 1,5$  ajouterait 1,5 à la forme réduite ( $6 \rightarrow 7,5$ ) sans changer la première étape : l'estimation deviendrait  $7,5/0,3 = 25$ , soit un biais de  $\gamma_Z/\pi = 1,5/0,3 = 5$  DT.

```

pi, rho = 0.60 - 0.30, 52 - 46
print(f"Wald = {rho / pi:.1f} | avec effet direct de 1,5 : {(rho + 1.5) / pi:.1f} | biais = {1.5 /
    ↪ pi:.1f}")

```

```

Wald = 20.0 | avec effet direct de 1,5 : 25.0 | biais = 5.0

```

**Corrigé 10.** (a) Comme  $Z \in \{0, 1\}$  :  $\text{Cov}(Z, Y) = \mathbb{E}[ZY] - \mathbb{E}[Z] \mathbb{E}[Y]$ . On a  $\mathbb{E}[ZY] = q \mathbb{E}[Y | Z = 1]$  et  $\mathbb{E}[Y] = q \mathbb{E}[Y | Z = 1] + (1 - q) \mathbb{E}[Y | Z = 0]$ . Donc  $\text{Cov}(Z, Y) = q \mu_1 - q(q\mu_1 + (1 - q)\mu_0) = q(1 - q)(\mu_1 - \mu_0)$  avec  $\mu_z = \mathbb{E}[Y | Z = z]$ . (b) La même formule vaut pour  $T$  à la place de  $Y$  ; dans le rapport, le facteur  $q(1 - q)$  se **simplifie**, et il reste  $\frac{\mathbb{E}[Y | Z = 1] - \mathbb{E}[Y | Z = 0]}{\mathbb{E}[T | Z = 1] - \mathbb{E}[T | Z = 0]}$ , le rapport de Wald. (c) Vérification :

```

iv_data = pd.read_csv("donnees/ch07-iv.csv")
z, tt, yy = iv_data["rappel"].to_numpy(float), iv_data["suit_instagram"].to_numpy(float),
    ↪ iv_data["depense"].to_numpy()
q = z.mean()
cov_zy, cov_zt = np.cov(z, yy, ddof=0)[0, 1], np.cov(z, tt, ddof=0)[0, 1]
rho = yy[z == 1].mean() - yy[z == 0].mean()
pi = tt[z == 1].mean() - tt[z == 0].mean()
print(f"Cov(Z,Y) = {cov_zy:.4f} vs q(1-q) x rho = {q * (1 - q) * rho:.4f}")
print(f"Cov(Z,Y)/Cov(Z,T) = {cov_zy / cov_zt:.4f} | rapport de Wald rho/pi = {rho / pi:.4f}")

```

```

Cov(Z,Y) = 2.2000 vs q(1-q) x rho = 2.2000
Cov(Z,Y)/Cov(Z,T) = 23.6563 | rapport de Wald rho/pi = 23.6563

```

**Corrigé 11.** (a)-(b) On réutilise la simulation de 7.1.7 (même graine), mais avec la motivation

dans la régression :

```
rng = np.random.default_rng(71)
n = 100_000
motivation = rng.normal(size=n)
offre = rng.integers(0, 2, n)
code_si_offre = rng.random(n) < 1 / (1 + np.exp(-(0.4 + 0.9 * motivation)))
code = offre * code_si_offre
depense = 100 + 8 * offre + 30 * code + 20 * motivation + rng.normal(0, 25, n)
d11 = pd.DataFrame({"depense": depense, "offre": offre, "code": code, "motivation": motivation})

total = smf.ols("depense ~ offre", d11).fit().params["offre"]
direct = smf.ols("depense ~ offre + code + motivation", d11).fit().params["offre"]
print(f"effet total estimé (offre seule) : {total:.2f}")
print(f"effet direct estimé (offre + code + motivation) : {direct:.2f} (vrai : 8)")
print(f"effet indirect = total - direct = {total - direct:.2f} (vrai : 30 x")
↳ f"{code_si_offre.mean():.2f} = {30 * code_si_offre.mean():.2f}")

effet total estimé (offre seule) : 25.62
effet direct estimé (offre + code + motivation) : 8.27 (vrai : 8)
effet indirect = total - direct = 17.35 (vrai : 30 x 0.58 = 17.47)
```

- (c) En 7.1.7, la motivation était **cachée** : conditionner sur le médiateur ouvrait un chemin biaisé offre → code ← motivation → dépense (le code est un effet commun de l'offre et de la motivation). Quand la motivation est **observée et incluse**, ce chemin est bloqué, et le coefficient de l'offre redevient l'effet **direct**. Moralité : l'analyse de médiation exige de contrôler **tous** les facteurs de confusion entre le médiateur et le résultat, une hypothèse supplémentaire, plus exigeante que celle de l'effet total (qui n'en demande aucune dans une expérience randomisée).

**Corrigé 12.** Il n'y a pas de code ici : c'est une analyse critique. *Pertinence* : la distance doit réellement influencer la fréquentation (plausible : plus on habite loin, moins on vient) ; on le **vérifie** avec la première étape et sa statistique *F*. *Indépendance* : les clients **choisissent** où habiter, et ce choix dépend du revenu, du mode de vie, de l'âge : la distance n'est pas tirée au sort. Les citadins aisés habitent peut-être près du centre, où se trouve la boutique, et dépensent plus pour cette raison : l'instrument est corrélé à un facteur de confusion. On peut atténuer en **ajustant** sur les variables socio-économiques observées et sur la ville, mais jamais complètement. *Exclusion* : la distance ne doit affecter la dépense **que** via la fréquentation de la boutique ; or elle peut affecter la dépense par d'autres voies (un client éloigné achète davantage en ligne, ou fait de plus gros achats par déplacement). On peut chercher des **tests de plausibilité** (l'effet de la distance sur la dépense en ligne devrait être nul chez ceux qui ne viennent jamais en boutique) sans pouvoir prouver l'exclusion. *Ce qu'on estimerait si l'instrument était valide* : l'effet moyen de venir en boutique pour les **complaisants**, c'est-à-dire les clients dont la fréquentation **dépend** de la distance (ceux qui viendraient s'ils habitaient près et ne viendraient pas s'ils habitaient loin) : pas pour les habitués, ni pour ceux qui ne viendraient jamais. Pour une variable continue comme la distance, l'interprétation est une moyenne pondérée de tels effets, plus difficile à énoncer.

## Bilan du chapitre 7

Vous savez maintenant :

- **formuler** une question causale avec les **résultats potentiels**, distinguer ATE, ATT et ATU, et comprendre pourquoi l'estimation d'un effet est un problème de **données manquantes** (le problème fondamental) ;

- **décomposer** une différence observée en effet causal et **biais de sélection**, et expliquer pourquoi la **randomisation** supprime le second ;
- **analyser une expérience randomisée** : tableau d'équilibre, effet moyen, intervalle de confiance, ajustement pour la précision ;
- **lire un graphe causal** (DAG) : chaîne, fourche, collision ; savoir ce qu'il faut ajuster (les causes communes) et ce qu'il ne faut **pas** ajuster (médiateurs, effets communs) ; énoncer le **critère de la porte dérobée** ;
- **estimer** un effet à partir de données observationnelles avec un **score de propension** (appariement, IPW, estimateur doublement robuste), et **vérifier** le chevauchement et l'équilibre ;
- **mener une différence de différences** : calcul à la main, régression à effets fixes avec erreurs-types groupées, **étude d'événement**, test placebo, et connaître ses pièges (tendances non parallèles, échelle, peu de groupes, lancements échelonnés) ;
- **utiliser une variable instrumentale** : Wald et 2SLS, conditions de validité, interprétation en **effet pour les complaisants**, danger des instruments faibles et de l'exclusion violée ;
- **comparer** ces méthodes et **dire honnêtement** ce qu'aucune d'elles ne peut garantir : toutes remplacent une information absente par une **hypothèse**.

**La seule phrase à retenir.** Un résultat causal s'écrit toujours sous la forme : « *si* (hypothèse), *alors* (effet estimé, avec son incertitude) ». Ce chapitre vous a appris à écrire des hypothèses honnêtes, et à les mettre à l'épreuve.

# Chapitre 8 : ⊕ Plans d'expériences

« Consulter le statisticien après l'expérience, c'est souvent lui demander de procéder à un examen *post mortem*. Il pourra peut-être dire de quoi l'expérience est morte. » (R. A. Fisher)

**Chapitre complémentaire.** Ce chapitre est **entièrement optionnel** : le reste du volume ne le suppose pas. Il s'adresse à celles et ceux qui ne se contentent pas d'*analyser* des données déjà là, mais qui veulent **les produire** : tester une vitrine, un emballage, un prix, un réglage de four. Il prolonge le test A/B du volume I (section 3.4.5) à plusieurs facteurs à la fois, et il éclaire, par un autre chemin, la question de la causalité abordée au chapitre 7.

Jusqu'ici, les données tombaient du ciel : un fichier de clients, une série de ventes. Mais quand on peut **choisir** ce que l'on mesure, la manière de le mesurer décide de ce que l'on pourra conclure. Une expérience mal conçue ne se rattrape pas avec des mathématiques ; une expérience bien conçue peut se contenter de mathématiques très simples. C'est tout l'art de ce chapitre.

## Le chemin de ce chapitre

- **8.1 Les principes d'un bon plan** : randomisation, répétition, blocage, et pourquoi changer **un seul facteur à la fois** est une fausse bonne idée.
- **8.2 L'analyse de la variance (ANOVA)** : comparer plusieurs groupes d'un coup, démontrer la décomposition de la variance, contrôler les hypothèses, comparer les groupes deux à deux sans tricher, utiliser des **blocs**, étudier **deux facteurs** et leur **interaction**, calculer la **puissance**.
- **8.3 Les plans factoriels** : tester  $k$  facteurs simultanément avec  $2^k$  essais, calculer les effets à **la main**, repérer les effets qui comptent sur un diagramme demi-normal.
- **8.4 Plans fractionnaires et surfaces de réponse** : faire **moins d'essais** en acceptant de confondre certains effets, puis **chercher l'optimum** d'un réglage avec un modèle quadratique ; en option, les plans optimaux.
- **8.5 Exercices corrigés.**

**Comment travailler avec ce chapitre.** Chaque section suit le fil : un petit tableau **calculable à la main**, la théorie qui l'explique, puis le même calcul sur des données simulées. Faites le calcul à la main *avant* de lire la sortie du code : c'est le meilleur moyen de comprendre ce que l'ordinateur fait.

**Les données de ce chapitre.** Les données sont **simulées** (graines fixes, script `build/donnees_ch08.py`) et enregistrées dans `donnees/ch08-*.csv`. Elles racontent des expériences de Yasmine : quatre agencements de vitrine, trois emballages, un test « emballage cadeau × promotion × canal de relance » (plans  $2^3$  puis  $2^4$ ), et un réglage du



four pour ses céramiques. Comme elles sont simulées, **nous connaissons la vérité** : à la fin de chaque étude, nous la dévoilerons pour voir si la méthode l'a retrouvée.

## 8.1 Les principes d'un bon plan d'expériences

**Intuition.** Une expérience est une **question posée à la nature**, et la nature répond à la question exactement telle qu'on l'a posée, y compris quand on l'a mal posée. Un plan d'expériences est la préparation de cette question : *quels* essais faire, *combien*, *dans quel ordre*, pour que la réponse soit à la fois **juste** (sans biais) et **précise** (peu de bruit).

### 8.1.1 Le vocabulaire

Yasmine veut savoir quel agencement de vitrine fait vendre le plus. Les mots du métier :

| Terme                       | Sens                                                                       | Dans l'exemple                                             |
|-----------------------------|----------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Facteur</b>              | une variable que l'on <b>choisit</b> de faire varier                       | l'agencement de la vitrine                                 |
| <b>Niveau</b>               | une valeur possible d'un facteur                                           | « Classique », « Par couleur », « Par thème », « Vedette » |
| <b>Traitement</b>           | une combinaison de niveaux (un seul facteur : un niveau)                   | « vitrine Par thème »                                      |
| <b>Unité expérimentale</b>  | l'objet auquel on applique un traitement, <b>indépendamment</b> des autres | <b>une journée</b> d'ouverture                             |
| <b>Réponse</b>              | ce que l'on mesure                                                         | les ventes du jour (DT)                                    |
| <b>Essai</b> ( <i>run</i> ) | une unité + un traitement + une mesure                                     | « mardi 3, vitrine Vedette : 213 DT »                      |
| <b>Plan</b>                 | la liste des essais, avec leur ordre                                       | 12 jours par agencement, ordre tiré au hasard              |

Le point le plus subtil est l'**unité expérimentale** : c'est la plus petite entité à laquelle on peut affecter un traitement **sans que le traitement d'une unité ne dépende de celui d'une autre**. Ici, on ne peut pas changer la vitrine client par client : tous les clients d'une même journée voient la même vitrine. L'unité est donc la journée, pas le client. Nous verrons en 8.1.4 ce que coûte cette confusion.

**Expérience ou observation ?** Dans une étude **observationnelle** (les clients des volumes précédents), on constate ce qui s'est passé : les clients « Instagram » et « Boutique » diffèrent sans que personne ne l'ait décidé, et les différences observées peuvent venir d'autre chose que du canal. Dans une **expérience**, c'est l'expérimentateur qui **affecte** les traitements. C'est cette affectation, et surtout sa manière d'être faite, qui permet de parler de **cause** (chapitre 7 pour le cas observationnel).

### 8.1.2 Première règle : randomiser

Supposons que Yasmine teste deux vitrines, A et B, sans effet réel : les deux sont aussi efficaces. Elle installe A du **lundi au jeudi** et B du **vendredi au dimanche**, pendant quatre semaines. Or le week-end, la boutique vend plus (disons 60 DT de plus par jour en moyenne, quel que soit l'agencement). Elle va conclure que B est meilleure : le **jour de la semaine** est un **facteur de**

**confusion** : il influence la réponse *et* est lié à l'affectation des traitements (le chapitre 7 en donne la théorie). Voyons l'ampleur du dégât par simulation, en comparant cette affectation à une affectation **tirée au hasard** (14 jours pour A, 14 pour B).

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(81)
jours = np.arange(28)                # 4 semaines ; le jour 0 est un lundi
jour_semaine = jours % 7             # 0 = lundi ... 6 = dimanche
effet_jour = np.where(jour_semaine >= 4, 60, 0) # vendredi, samedi, dimanche : +60 DT

naif_A = jour_semaine <= 3           # A : lundi-jeudi (16 jours) ; B : vendredi-dimanche (12
↳ jours)
ecarts = {"naïve": [], "aléatoire": []}
rejets = {"naïve": 0, "aléatoire": 0}
n_sim = 2000
for _ in range(n_sim):
    y = 200 + effet_jour + rng.normal(0, 25, 28) # AUCUN effet de la vitrine : A = B
    lab_A = {"naïve": naif_A, "aléatoire": rng.permutation(np.r_[np.ones(14, bool), np.zeros(14,
↳ bool))]}
    for nom, A in lab_A.items():
        ecarts[nom].append(y[~A].mean() - y[A].mean())
        rejets[nom] += stats.ttest_ind(y[~A], y[A], equal_var=False).pvalue < 0.05

for nom in ecarts:
    e = np.array(ecarts[nom])
    print(f"affectation {nom:9s}: écart moyen B - A = {e.mean():6.1f} DT ; écart-type = {e.std():5.1f}
↳ ; "
        f"'effet significatif' dans {100 * rejets[nom] / n_sim:5.1f} % des expériences")

affectation naïve      : écart moyen B - A =   60.3 DT ; écart-type =    9.4 ; 'effet significatif' dans
↳ 100.0 % des expériences
affectation aléatoire: écart moyen B - A =   -0.1 DT ; écart-type =   14.8 ; 'effet significatif' dans
↳  5.2 % des expériences
```

On le lit ainsi : avec l'affectation naïve, la différence  $B - A$  est **systématiquement** d'environ 60 DT (le biais) et le test « détecte » un effet qui n'existe pas dans 100 % des 2 000 expériences simulées. Avec l'affectation aléatoire, la différence est **centrée sur zéro** ( $-0,1$  DT en moyenne) et le test se trompe dans 5,2 % des cas, **exactement ce que promet son niveau**  $\alpha = 5\%$ .

**Pourquoi ça marche.** Quand on tire les étiquettes au hasard, le week-end a la **même chance** d'avoir reçu A ou B. Les jours « forts » se répartissent donc équitablement entre les deux traitements *en espérance* : le facteur de confusion, même **inconnu** ou **non mesuré**, cesse d'être confondu avec le traitement. C'est le seul procédé qui protège aussi contre les causes que l'on n'a pas pensé à noter. De plus, c'est le tirage au sort lui-même qui **justifie** les p-valeurs : c'est exactement la logique du test de permutation (volume I, section 3.7.4), où les étiquettes sont mélangées au hasard pour fabriquer la loi de la statistique sous l'hypothèse « aucun effet ».

« **Au hasard** » ne veut pas dire « **n'importe comment** ». Alternier un jour sur deux, choisir les jours « qui s'y prêtent » ou laisser un employé décider sont des procédés **non aléatoires** : ils peuvent coïncider avec un rythme caché (par exemple si un jour sur deux est un jour de livraison). Utilisez un générateur pseudo-aléatoire (`rng.permutation`) et **notez la graine**.

### 8.1.3 Deuxième règle : répéter

Une seule journée par vitrine ne dit rien : la différence entre deux journées vient du **bruit** autant que du traitement. **Répéter** le même traitement sur plusieurs unités permet deux choses : **estimer**

**le bruit** (la variabilité entre unités traitées pareil) et **le réduire** (la moyenne de  $n$  unités a une variance  $\sigma^2/n$ , volume I, section 2.4).

**Répétition ne veut pas dire mesure répétée.** C'est l'erreur la plus fréquente, et elle s'appelle la **pseudo-réplication**. Si Yasmine interroge **40 clients** chaque jour, ces 40 réponses du même jour partagent la même vitrine **et** les mêmes aléas de la journée (météo, jour de marché...) : ce ne sont pas 40 unités indépendantes, mais **une** unité mesurée 40 fois. Voyons ce qu'il en coûte. On simule deux vitrines **sans aucune différence**, 5 jours chacune, 40 clients par jour, avec un aléa propre à chaque jour.

```
rng = np.random.default_rng(83)

faux, bon = 0, 0
n_sim = 3000
for _ in range(n_sim):
    aleas_jour = rng.normal(0, 25, (2, 5)) # un aléa par (vitrine, jour)
    y = 200 + aleas_jour[:, :, None] + rng.normal(0, 40, (2, 5, 40)) # forme : (vitrine, jour, client)
    # (a) on traite les 200 clients de chaque vitrine comme indépendants : FAUX
    faux += stats.ttest_ind(y[0].ravel(), y[1].ravel()).pvalue < 0.05
    # (b) on résume chaque journée par sa moyenne : l'unité expérimentale est le jour (5 contre 5)
    bon += stats.ttest_ind(y[0].mean(axis=1), y[1].mean(axis=1)).pvalue < 0.05

print(f"Test sur 200 clients contre 200 clients : 'effet' trouvé dans {100 * faux / n_sim:.1f} % des cas")
print(f"Test sur 5 jours contre 5 jours : 'effet' trouvé dans {100 * bon / n_sim:.1f} % des cas")
```

```
Test sur 200 clients contre 200 clients : 'effet' trouvé dans 56.9 % des cas
Test sur 5 jours contre 5 jours : 'effet' trouvé dans 5.1 % des cas
```

Alors qu'il n'y a **aucun effet**, le premier test crie victoire dans **57 %** des expériences, au lieu des 5 % annoncés. Il se croit précis parce qu'il compte 400 clients, alors que la précision réelle est celle de 10 journées. Le second test, moins impressionnant en apparence, tient sa promesse (5,1 %). **Règle pratique : le nombre de degrés de liberté de l'erreur se compte en unités expérimentales, pas en mesures.**

### 8.1.4 Troisième règle : bloquer

La répétition réduit le bruit, mais certaines sources de variabilité sont **connues à l'avance** : les semaines ne se ressemblent pas (soldes, fêtes), les lots de matière première non plus. Plutôt que de laisser cette variabilité gonfler le bruit, on l'**isole** : on découpe l'expérience en **blocs** de conditions homogènes (par exemple une semaine par bloc) et, **dans chaque bloc**, on teste **tous** les traitements, en randomisant l'ordre dans le bloc. Les comparaisons se font alors à l'intérieur des blocs, à conditions égales. La section 8.2.8 montre, chiffres à l'appui, ce que cela change.

**La devise de Fisher, en une ligne : Bloquez ce que vous pouvez, randomisez ce que vous ne pouvez pas bloquer, et répétez pour mesurer le bruit.**

### 8.1.5 Faire varier un seul facteur à la fois : une fausse bonne idée

L'instinct dit : « pour savoir ce que fait chaque facteur, changeons-les un par un, les autres restant fixes ». Cette méthode, appelée **OFAT** (*one factor at a time*), semble prudente. Elle est en réalité **inefficace** et **aveugle aux interactions**. Considérons un exemple assez petit pour être calculé à la main. Yasmine hésite entre deux facteurs : l'**emballage** (standard ou cadeau) et le **prix** (normal ou promotion de 10 %). Imaginons que nous connaissions, sans bruit, le nombre moyen de commandes par semaine dans chacune des quatre situations :

|                           | prix normal | promo -10 % |
|---------------------------|-------------|-------------|
| <b>emballage standard</b> | 50          | 62          |
| <b>emballage cadeau</b>   | 55          | 60          |

**La démarche OFAT.** On part de la situation actuelle (standard, prix normal : 50). On change l'emballage : 55, c'est mieux (+5), on adopte le cadeau. Puis, avec le cadeau, on change le prix : 60, c'est mieux (+5), on adopte la promo. Conclusion : « la meilleure combinaison est *cadeau* + *promo*, 60 commandes ». Or la vraie meilleure est **standard + promo : 62**. L'emballage cadeau, qui aide au prix normal, **gêne** quand il y a une promotion (−2) : on dit qu'il y a **interaction** entre les deux facteurs. L'OFAT l'a manquée, car il n'a **jamais** testé « standard + promo ».

```
import pandas as pd

vrai = pd.DataFrame({"prix normal": [50, 55], "promo -10 %": [62, 60]}, index=["standard", "cadeau"])
print(vrai)

effet_A_prix_normal = vrai.loc["cadeau", "prix normal"] - vrai.loc["standard", "prix normal"]
effet_A_promo = vrai.loc["cadeau", "promo -10 %"] - vrai.loc["standard", "promo -10 %"]
effet_B_standard = vrai.loc["standard", "promo -10 %"] - vrai.loc["standard", "prix normal"]
effet_B_cadeau = vrai.loc["cadeau", "promo -10 %"] - vrai.loc["cadeau", "prix normal"]
print()
print("effet du cadeau au prix normal :", effet_A_prix_normal, "| avec la promo :", effet_A_promo)
print("effet de la promo en standard :", effet_B_standard, "| en cadeau :", effet_B_cadeau)

# effets « principaux » et interaction (définitions précisées en 8.3)
A = (effet_A_prix_normal + effet_A_promo) / 2
B = (effet_B_standard + effet_B_cadeau) / 2
AB = (effet_A_promo - effet_A_prix_normal) / 2
print(f"effet principal du cadeau A = {A}, de la promo B = {B}, interaction AB = {AB}")
```

|          | prix normal | promo -10 % |
|----------|-------------|-------------|
| standard | 50          | 62          |
| cadeau   | 55          | 60          |

effet du cadeau au prix normal : 5 | avec la promo : -2  
 effet de la promo en standard : 12 | en cadeau : 5  
 effet principal du cadeau A = 1.5, de la promo B = 8.5, interaction AB = -3.5

Le tableau le dit sans ambiguïté : **l'effet d'un facteur dépend du niveau de l'autre**. Parler de « l'effet du cadeau » tout court n'a alors plus de sens.

**L'OFAT est aussi moins précis.** Supposons un bruit d'écart-type  $\sigma$  sur chaque essai. Avec 4 essais, l'OFAT en consacre deux à la situation de départ (sinon on ne mesure aucun bruit) puis un essai pour chaque facteur modifié : l'effet de A s'estime par  $y_A - \bar{y}_0$ , de variance  $\sigma^2(1 + \frac{1}{2}) = 1,5\sigma^2$ . Le plan **factoriel** utilise les **mêmes 4 essais** (les quatre cases du tableau) et estime l'effet de A par  $\frac{1}{2}[(y_{\text{cadeau, normal}} + y_{\text{cadeau, promo}}) - (y_{\text{std, normal}} + y_{\text{std, promo}})]$ , de variance  $\frac{1}{4} \cdot 4\sigma^2 = \sigma^2$ . Chaque essai y sert **deux fois** : une fois pour chaque facteur. Vérifions par simulation.

```
rng = np.random.default_rng(84)
sigma, n_sim = 4.0, 100000
mu = {"std_normal": 50, "cadeau_normal": 55, "std_promo": 62, "cadeau_promo": 60}

# plan factoriel : 4 essais, un par case
y = {k: v + rng.normal(0, sigma, n_sim) for k, v in mu.items()}
A_fact = ((y["cadeau_normal"] + y["cadeau_promo"]) - (y["std_normal"] + y["std_promo"])) / 2

# OFAT : 4 essais aussi (départ répété 2 fois, puis A changé, puis B changé)
depart = (mu["std_normal"] + rng.normal(0, sigma, n_sim) + mu["std_normal"] + rng.normal(0, sigma,
↵ n_sim)) / 2
```

```
A_ofat = (mu["cadeau_normal"] + rng.normal(0, sigma, n_sim)) - depart

print(f"variance de l'estimation de A : factoriel = {A_fact.var():.1f} (théorie {sigma**2:.1f})")
print(f"                                OFAT      = {A_ofat.var():.1f} (théorie {1.5 * sigma**2:.1f})")
print(f"moyenne estimée de A          : factoriel = {A_fact.mean():.2f} (effet principal vrai 1.5) ; "
      f"                                f"OFAT = {A_ofat.mean():.2f} (effet de A au prix normal : 5)")
```

```
variance de l'estimation de A : factoriel = 16.1 (théorie 16.0)
                                OFAT      = 24.0 (théorie 24.0)
moyenne estimée de A          : factoriel = 1.48 (effet principal vrai 1.5) ; OFAT = 5.01 (effet de A
↪ au prix normal : 5)
```

Deux enseignements. D'abord, à nombre d'essais égal, le plan factoriel est plus précis : la simulation retrouve la variance  $\sigma^2 = 16$  pour le factoriel et  $1,5\sigma^2 = 24$  pour l'OFAT. Ensuite, les deux méthodes n'estiment pas la même chose : l'OFAT mesure l'effet du cadeau **seulement au prix normal** (5), alors que le factoriel mesure son effet **moyen** sur les deux prix (1,5, la simulation donne 1,48) **et**, grâce à l'interaction, sait qu'il est de +5 dans un cas et de -2 dans l'autre. Avec davantage de facteurs, l'avantage du factoriel est encore plus net (section 8.3).

### À retenir.

- Une expérience est caractérisée par ses **facteurs**, ses **unités expérimentales** et sa **réponse** ; l'unité est ce à quoi l'on affecte réellement le traitement.
- **Randomiser** rend le traitement indépendant des facteurs de confusion, même inconnus : sans cela, une différence « significative » peut n'être qu'un artefact.
- **Répéter** pour estimer et réduire le bruit, mais **ne pas confondre** répétition (unités indépendantes) et mesures répétées (pseudo-réplication).
- **Bloquer** pour retirer du bruit les sources de variabilité connues.
- Changer **un facteur à la fois** est moins précis et **aveugle aux interactions** ; les plans **factoriels** font varier tous les facteurs ensemble.

## 8.2 L'analyse de la variance (ANOVA)

**Intuition.** Pour comparer deux groupes, on utilisait un test de Student. Pour en comparer quatre, on pourrait faire les six tests deux à deux, mais cela multiplie les risques de fausse alerte (volume I, section 3.5.5). L'ANOVA pose une seule question globale : « **les moyennes des groupes diffèrent-elles plus que ne le ferait le seul hasard ?** » Et elle y répond en comparant **deux sources de variabilité** : celle **entre** les groupes (l'effet du traitement, s'il existe) et celle **à l'intérieur** des groupes (le bruit). Si la première dépasse nettement la seconde, les groupes ne sont pas interchangeables.

### 8.2.1 Le problème : quatre agencements de vitrine

Yasmine a testé quatre agencements de vitrine (« Classique », « Par couleur », « Par thème », « Vedette »). Pendant 48 jours d'ouverture, elle a tiré au sort l'agencement de chaque journée (12 jours par agencement) et relevé les ventes du jour. Les unités expérimentales sont les journées, comme nous l'avons discuté en 8.1.

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.read_csv("donnees/ch08-vitrines.csv")
print(df.head(6).to_string(index=False))
print()
```

```
resume = df.groupby("agencement")["ventes"].agg(n="count", moyenne="mean", ecart_type="std").round(1)
print(resume.sort_values("moyenne").to_string())
print("\nmoyenne générale :", round(df["ventes"].mean(), 1))
```

```
jour  agencement  ventes
1  Par couleur   226.7
2    Vedette     249.3
3  Par thème     259.0
4    Vedette     272.2
5  Par thème     269.0
6  Par thème     243.2
```

|             | n  | moyenne | ecart_type |
|-------------|----|---------|------------|
| agencement  |    |         |            |
| Classique   | 12 | 196.2   | 38.1       |
| Vedette     | 12 | 215.4   | 28.3       |
| Par couleur | 12 | 224.6   | 40.7       |
| Par thème   | 12 | 237.0   | 25.5       |

moyenne générale : 218.3

Les moyennes diffèrent, mais les écarts-types sont du même ordre que ces différences : difficile de juger à l'œil. Dessinons les données **avant** de calculer quoi que ce soit (volume I, section 3.1).

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb",
                    "axes.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb", "legend.frameon":
                    ↪ False})

ordre = list(resume.sort_values("moyenne").index)
rng = np.random.default_rng(1)
fig, ax = plt.subplots(figsize=(7, 4))
for i, a in enumerate(ordre):
    y = df.loc[df["agencement"] == a, "ventes"]
    ax.scatter(i + rng.uniform(-0.12, 0.12, len(y)), y, s=22, color=BLEU, alpha=0.6, zorder=3)
    ax.hlines(y.mean(), i - 0.3, i + 0.3, color=ORANGE, lw=3, zorder=4)
ax.axhline(df["ventes"].mean(), color="#898781", ls="--", lw=1)
ax.text(len(ordre) - 0.5, df["ventes"].mean() + 3, "moyenne générale", ha="right", color="#52514e",
        ↪ fontsize=8)
ax.set_xticks(range(len(ordre)))
ax.set_xticklabels(ordre)
ax.set_ylabel("ventes du jour (DT)")
ax.set_title("Ventes selon l'agencement (un point = une journée, trait orange = moyenne)")
plt.savefig("figures/ch08-vitrines.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Les nuages se chevauchent beaucoup, mais ils sont ordonnés : « Classique » est en bas, « Par thème » en haut. La différence est-elle réelle ou due au hasard de la répartition des jours ?

### 8.2.2 Décomposer la variabilité : un exemple de neuf nombres

Commençons par un exemple minuscule. Trois agencements, trois journées chacun :

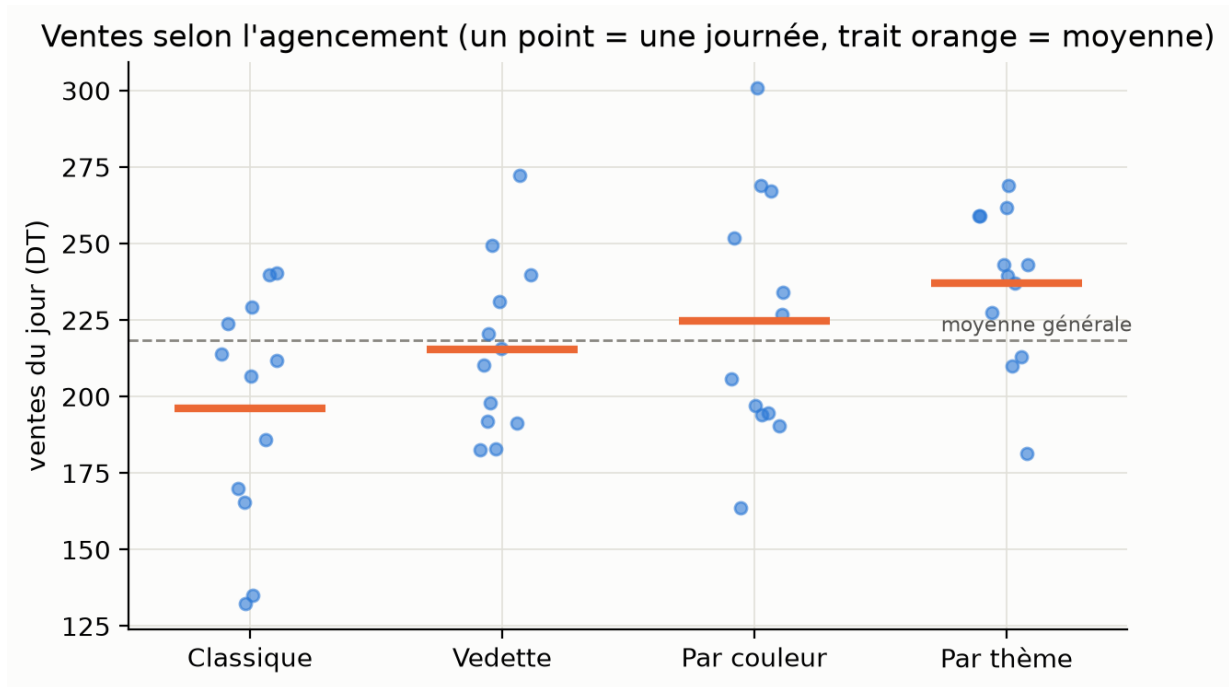


Figure 81: Ventes journalières selon l'agencement de la vitrine : chaque point est une journée, le trait orange est la moyenne du groupe, la ligne en pointillés la moyenne générale.

|             | journée 1 | journée 2 | journée 3 | moyenne |
|-------------|-----------|-----------|-----------|---------|
| Classique   | 190       | 200       | 210       | 200     |
| Par couleur | 205       | 215       | 225       | 215     |
| Par thème   | 230       | 240       | 250       | 240     |

La moyenne des neuf valeurs est  $\bar{y} = (200 + 215 + 240)/3 \approx 218,33$ . Deux sortes d'écarts :

- l'écart **entre** les groupes : chaque moyenne de groupe s'écarte de la moyenne générale  $(-18,33 ; -3,33 ; +21,67)$  ;
- l'écart **à l'intérieur** de chaque groupe : chaque valeur s'écarte de la moyenne de *son* groupe  $(-10, 0, +10, \text{ dans les trois groupes})$ .

On mesure chaque sorte d'écart par une **somme de carrés** (volume I, section 3.1.4 pour la variance) :

$$SS_{\text{entre}} = 3 [(-18,33)^2 + (-3,33)^2 + (21,67)^2] = 2450, \quad SS_{\text{dans}} = 3 \times (100 + 0 + 100) = 600.$$

Et la variabilité **totale** de l'ensemble des neuf valeurs,  $\sum (y - \bar{y})^2$ , vaut  $3050 = 2450 + 600$ . Ce n'est pas un hasard.

```

y = np.array([[190, 200, 210], [205, 215, 225], [230, 240, 250]], dtype=float) # une ligne par groupe
k, n = y.shape
moy_gen, moy_groupes = y.mean(), y.mean(axis=1)

SS_entre = n * ((moy_groupes - moy_gen) ** 2).sum()
SS_dans = ((y - moy_groupes[:, None]) ** 2).sum()
SS_total = ((y - moy_gen) ** 2).sum()
print(f"SS entre = {SS_entre:.0f}, SS dans = {SS_dans:.0f}, somme = {SS_entre + SS_dans:.0f}, SS total
      = {SS_total:.0f}")

```



```

MS_entre, MS_dans = SS_entre / (k - 1), SS_dans / (k * n - k)
F = MS_entre / MS_dans
print(f"F = ({SS_entre:.0f})/({k - 1}) / ({SS_dans:.0f})/({k * n - k}) = {MS_entre:.0f} / {MS_dans:.0f} = \
↳ {F:.2f}")
print(f"p-valeur = {stats.f.sf(F, k - 1, k * n - k):.4f} (scipy f_oneway : \
↳ {stats.f_oneway(*y).pvalue:.4f})")

```

```

SS_entre = 2450, SS_dans = 600, somme = 3050, SS_total = 3050
F = (2450/2) / (600/6) = 1225 / 100 = 12.25
p-valeur = 0.0076 (scipy f_oneway : 0.0076)

```

**Théorème (décomposition de la variance).** Soit  $y_{ij}$  la  $j$ -ième observation du groupe  $i$  ( $i = 1, \dots, k; j = 1, \dots, n_i$ ),  $N = \sum n_i$ ,  $\bar{y}_i$  la moyenne du groupe et  $\bar{y}$  la moyenne générale. Alors

$$\underbrace{\sum_{i,j} (y_{ij} - \bar{y})^2}_{SS_T} = \underbrace{\sum_i n_i (\bar{y}_i - \bar{y})^2}_{SS_B \text{ (entre)}} + \underbrace{\sum_{i,j} (y_{ij} - \bar{y}_i)^2}_{SS_W \text{ (dans)}}.$$

*Démonstration.* On écrit  $y_{ij} - \bar{y} = (y_{ij} - \bar{y}_i) + (\bar{y}_i - \bar{y})$  et on développe le carré :

$$\sum_{i,j} (y_{ij} - \bar{y})^2 = \sum_{i,j} (y_{ij} - \bar{y}_i)^2 + \sum_{i,j} (\bar{y}_i - \bar{y})^2 + 2 \sum_i (\bar{y}_i - \bar{y}) \underbrace{\sum_j (y_{ij} - \bar{y}_i)}_{=0}.$$

Le double produit s'annule, car la somme des écarts d'un groupe à sa propre moyenne est nulle. Et  $\sum_{i,j} (\bar{y}_i - \bar{y})^2 = \sum_i n_i (\bar{y}_i - \bar{y})^2$  puisque le terme ne dépend pas de  $j$ . □

La décomposition est exacte, quelles que soient les données : elle n'a rien de statistique. C'est son **interprétation** qui l'est. Pour en tirer un test, il faut un modèle.

### 8.2.3 Le modèle et le test F

Le **modèle à un facteur** suppose que

$$y_{ij} = \mu + \tau_i + \varepsilon_{ij}, \quad \varepsilon_{ij} \text{ indépendants, } \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2), \quad \sum_i n_i \tau_i = 0.$$

$\mu$  est la moyenne générale,  $\tau_i$  l'**effet** du niveau  $i$  (l'écart de sa moyenne vraie à  $\mu$ ) et  $\varepsilon_{ij}$  le bruit. L'hypothèse nulle « tous les agencements se valent » s'écrit  $H_0 : \tau_1 = \dots = \tau_k = 0$ .

On compare maintenant les sommes de carrés **ramenées à leurs degrés de liberté** :  $MS_B = SS_B / (k - 1)$  et  $MS_W = SS_W / (N - k)$  (*mean squares*, « carrés moyens »). Pourquoi ces diviseurs ?

**Ce que valent les carrés moyens.**

1.  $MS_W$  **estime toujours**  $\sigma^2$ , effet ou non. En effet, dans le groupe  $i$ ,  $\sum_j (y_{ij} - \bar{y}_i)^2 / \sigma^2 \sim \chi_{n_i-1}^2$  (résultat classique pour des observations gaussiennes, que nous admettons ; la loi du khi-deux a été rencontrée au volume I, section 3.4.6, dans un autre rôle) ; les  $k$  groupes étant indépendants, la somme  $SS_W / \sigma^2$  suit une loi  $\chi_{N-k}^2$ , d'espérance  $N - k$ . Donc  $\mathbb{E}[MS_W] = \sigma^2$ .
2.  $MS_B$  **estime**  $\sigma^2$  **plus un terme dû aux effets** : un calcul direct donne

$$\mathbb{E}[MS_B] = \sigma^2 + \frac{\sum_i n_i \tau_i^2}{k - 1}.$$

Sous  $H_0$  (tous les  $\tau_i = 0$ ),  $\mathbb{E}[MS_B] = \sigma^2$  aussi ; sinon  $MS_B$  est plus grand.

3. Sous  $H_0$ ,  $SS_B/\sigma^2 \sim \chi_{k-1}^2$  et  $SS_B$  **est indépendant de**  $SS_W$  (théorème de Cochran : la décomposition en sommes de carrés orthogonales donne des  $\chi^2$  indépendants).

Le quotient de deux  $\chi^2$  indépendants divisés par leurs degrés de liberté suit une **loi de Fisher** :

$$F = \frac{MS_B}{MS_W} \underset{H_0}{\sim} \mathcal{F}(k-1, N-k).$$

Sous  $H_0$ ,  $F$  vaut environ 1 ; si les effets existent,  $F$  est tiré vers le haut. On rejette donc  $H_0$  quand  $F$  est **grand** (test unilatéral à droite).

Appliquons cela aux données de Yasmine, d'abord à la main, ensuite avec les bibliothèques :

```
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

k, N = df["agencement"].nunique(), len(df)
g = df.groupby("agencement")["ventes"]
moy_gen = df["ventes"].mean()
SSB = (g.size() * (g.mean() - moy_gen) ** 2).sum()
SSW = ((df["ventes"] - g.transform("mean")) ** 2).sum()
SST = ((df["ventes"] - moy_gen) ** 2).sum()
MSB, MSW = SSB / (k - 1), SSW / (N - k)
F = MSB / MSW
print(f"SSB = {SSB:.0f}   SSW = {SSW:.0f}   SST = {SST:.0f}   (SSB + SSW = {SSB + SSW:.0f})")
print(f"MSB = {MSB:.0f}   MSW = {MSW:.0f}   F = {F:.3f}   p = {stats.f.sf(F, k - 1, N - k):.4f}   (ddl :
↳ {k - 1} et {N - k})")
print()
modele = ols("ventes ~ C(agencement)", data=df).fit()
print(anova_lm(modele).round(3).to_string())
print("\nscipy f_oneway :", stats.f_oneway(*[x.to_numpy() for _, x in g]))
```

```
SSB = 10595   SSW = 50168   SST = 60763   (SSB + SSW = 60763)
MSB = 3532   MSW = 1140   F = 3.097   p = 0.0363   (ddl : 3 et 44)
```

|               | df   | sum_sq    | mean_sq  | F     | PR(>F) |
|---------------|------|-----------|----------|-------|--------|
| C(agencement) | 3.0  | 10595.062 | 3531.687 | 3.097 | 0.036  |
| Residual      | 44.0 | 50168.171 | 1140.186 | NaN   | NaN    |

```
scipy f_oneway : F_onewayResult(statistic=np.float64(3.097466867203288),
↳ pvalue=np.float64(0.03633773429167083))
```

Les trois calculs coïncident. Le test rejette l'hypothèse « tous les agencements se valent » à 5 %, sans que la preuve soit écrasante :  $p \approx 0,036$  est proche du seuil de 5 %. Nous reviendrons en 8.2.10 sur la puissance de cette expérience.

**Ce que l'ANOVA ne dit pas.** Un  $F$  significatif dit que **au moins un** agencement diffère des autres ; il ne dit pas **lesquels**. Pour cela, il faut des comparaisons deux à deux *corrigées* (8.2.6).

## 8.2.4 L'ANOVA, le test de Student et la régression sont le même outil

Avec deux groupes, l'ANOVA redonne le test de Student à variances égales :  $F = t^2$ , avec la même p-valeur.

```
deux = df[df["agencement"].isin(["Classique", "Par thème"])]
t = stats.ttest_ind(deux.loc[deux["agencement"] == "Par thème", "ventes"],
                    deux.loc[deux["agencement"] == "Classique", "ventes"], equal_var=True)
F2 = stats.f_oneway(*[x["ventes"].to_numpy() for _, x in deux.groupby("agencement")])
print(f"Student : t = {t.statistic:.4f}, t² = {t.statistic ** 2:.4f}, p = {t.pvalue:.5f}")
print(f"ANOVA : F = {F2.statistic:.4f}, p = {F2.pvalue:.5f}")
```

```
Student : t = 3.0796, t² = 9.4837, p = 0.00548
ANOVA   : F = 9.4837, p = 0.00548
```

Avec  $k$  groupes, c'est une **régression linéaire** (chapitre 1) sur des variables indicatrices : on prend un groupe de référence et on code les autres par 0/1. La constante est alors la moyenne du groupe de référence et chaque coefficient est l'écart de moyenne par rapport à lui. Le test  $F$  global de la régression (section 1.2) est le test  $F$  de l'ANOVA, et le  $R^2$  de la régression vaut  $SS_B/SS_T$ .

```
print(modele.params.round(2).to_string())
print(f"\nF global de la régression = {modele.fvalue:.3f}, p = {modele.f_pvalue:.4f}")
print(f"R² = {modele.rsquared:.4f} et SSB/SST = {SSB / SST:.4f}")
```

```
Intercept                196.25
C(agencement)[T.Par couleur]  28.36
C(agencement)[T.Par thème]   40.72
C(agencement)[T.Vedette]    19.19
```

```
F global de la régression = 3.097, p = 0.0363
R² = 0.1744 et SSB/SST = 0.1744
```

La constante est la moyenne du groupe de référence (le premier par ordre alphabétique, « Classique »), et les autres coefficients sont les écarts à ce groupe. L'ANOVA n'est donc pas un outil à part : c'est un **cas particulier du modèle linéaire** dont les variables explicatives sont qualitatives. Cette vue unifiée sera précieuse pour les plans factoriels (8.3), où les « effets » seront exactement des coefficients.

### 8.2.5 Vérifier les hypothèses

Le test  $F$  suppose des erreurs **indépendantes** (garanti par la randomisation et le bon choix de l'unité, 8.1), **gaussiennes** et de **même variance** dans tous les groupes. On les vérifie sur les **résidus**  $e_{ij} = y_{ij} - \bar{y}_i$ .

```
residus = modele.resid
groupes = [x["ventes"].to_numpy() for _, x in df.groupby("agencement")]

print("Shapiro-Wilk (normalité des résidus) : p =", round(stats.shapiro(residus).pvalue, 3))
print("Levene/Brown-Forsythe (variances égales) : p =", round(stats.levene(*groupes,
    ↪ center="median").pvalue, 3))
print("Bartlett (variances égales, sensible à la non-normalité) : p =",
    ↪ round(stats.bartlett(*groupes).pvalue, 3))
sd = df.groupby("agencement")["ventes"].std()
print(f"rapport plus grand / plus petit écart-type : {sd.max() / sd.min():.2f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.6))
(osm, osr), (pente, ordonnee, _) = stats.probplot(residus, dist="norm")
ax1.scatter(osm, osr, s=20, color=BLEU)
ax1.plot(osm, pente * np.asarray(osm) + ordonnee, color=ORANGE, lw=1.5)
ax1.set_xlabel("quantiles théoriques de la loi normale")
ax1.set_ylabel("résidus (DT)")
ax1.set_title("Diagramme quantile-quantile")
ax2.scatter(modele.fittedvalues, residus, s=20, color=BLEU)
ax2.axhline(0, color="#898781", lw=1)
ax2.set_xlabel("valeur ajustée = moyenne du groupe (DT)")
ax2.set_ylabel("résidus (DT)")
ax2.set_title("Résidus selon la valeur ajustée")
plt.tight_layout()
plt.savefig("figures/ch08-anova-diagnostics.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

```

Shapiro-Wilk (normalité des résidus)      : p = 0.521
Levene/Brown-Forsythe (variances égales) : p = 0.276
Bartlett (variances égales, sensible à la non-normalité) : p = 0.369
rapport plus grand / plus petit écart-type : 1.60
figure enregistrée

```

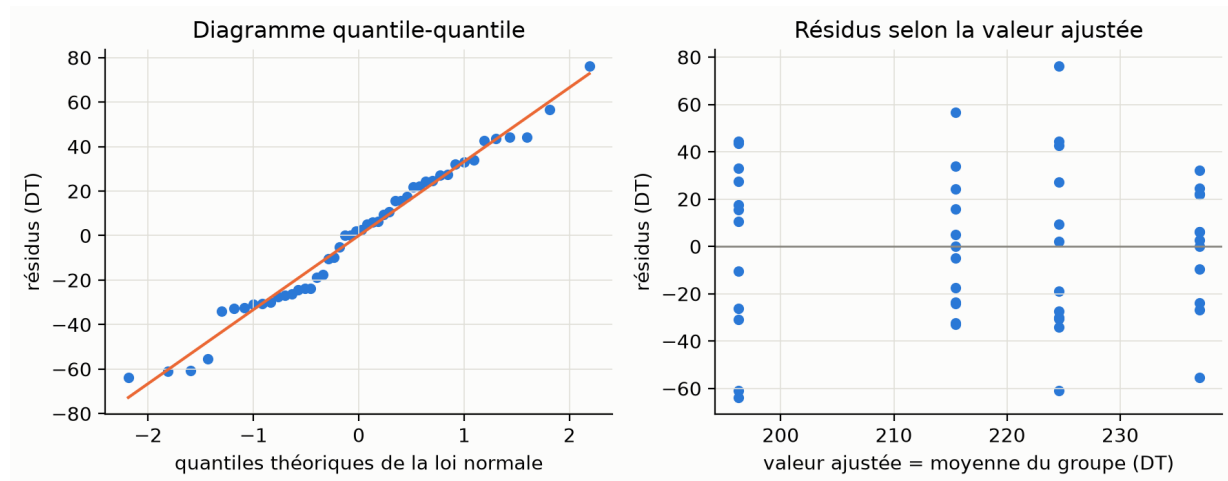


Figure 82: À gauche : diagramme quantile-quantile des résidus de l'ANOVA, qui suivent à peu près la droite. À droite : résidus selon la moyenne du groupe, sans structure visible ni dispersion qui varie nettement d'un groupe à l'autre.

Aucun signal inquiétant : le diagramme quantile-quantile suit la droite et la dispersion des résidus est comparable d'un groupe à l'autre. Si ce n'était pas le cas :

- **variances inégales** : utiliser l'ANOVA de **Welch** (qui n'impose pas l'égalité des variances) ;
- **résidus très non gaussiens** : transformer la réponse (logarithme pour des montants, volume I, section 3.1.5) ou utiliser le test de **Kruskal-Wallis** (volume I, section 3.7.3) ;
- **dépendance** entre unités : c'est un défaut du **plan**, et aucune correction après coup ne le répare.

```

from statsmodels.stats.oneway import anova_oneway

welch = anova_oneway(groupees, use_var="unequal", welch_correction=True)
print(f"ANOVA de Welch : F = {welch.statistic:.3f}, p = {welch.pvalue:.4f}")
print(f"Kruskal-Wallis : H = {stats.kruskal(*groupees).statistic:.3f}, p = {stats.kruskal(*groupees).pvalue:.4f}")

```

```

ANOVA de Welch : F = 3.259, p = 0.0390
Kruskal-Wallis : H = 7.009, p = 0.0716

```

Les ANOVA classique et de Welch concluent de la même façon ( $p = 0,036$  et  $p = 0,039$ ). Le test de Kruskal-Wallis, qui travaille sur les rangs, est un peu moins tranché ( $p = 0,072$ ) : il passe juste au-dessus du seuil de 5 %. Il n'y a pas de contradiction, mais un **avertissement honnête** : la preuve est **limite**, et la conclusion « au moins un agencement diffère » ne doit pas être présentée comme acquise au-delà de tout doute. C'est le genre de nuance qu'une p-valeur isolée ferait oublier.

### 8.2.6 Quels groupes diffèrent ? Les comparaisons multiples

Avec  $k = 4$  niveaux, il y a  $\binom{4}{2} = 6$  paires. Si l'on faisait six tests à 5 %, la probabilité d'au moins une fausse alerte n'est plus 5 % mais proche de  $1 - 0,95^6 \approx 26\%$  (volume I, section 3.5.5). Il faut un procédé qui contrôle le risque **global** (*familywise*). Deux classiques :

- **Bonferroni** : rejeter si  $p < \alpha/m$  ( $m$  = nombre de comparaisons). Simple mais conservateur.
- **Tukey HSD** (*Honestly Significant Difference*), conçu exactement pour comparer **toutes les paires de moyennes** : deux moyennes diffèrent si leur écart dépasse

$$\text{HSD} = q_{1-\alpha; k, N-k} \sqrt{\frac{MS_W}{n}} \quad (\text{groupes de même taille } n),$$

où  $q$  est le quantile de la **loi de l'étendue studentisée** (la loi du plus grand écart entre  $k$  moyennes, divisé par son écart-type estimé). Plus on compare de groupes, plus ce seuil monte.

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

n_par = N // k
q = stats.studentized_range.ppf(0.95, k, N - k)
hsd = q * np.sqrt(MSW / n_par)
print(f"q(0.95 ; k={k}, ddl={N - k}) = {q:.3f} -> HSD = {q:.3f} x sqrt({MSW:.0f}/{n_par}) = \
↳ {hsd:.1f} DT")
lsd = stats.t.ppf(0.975, N - k) * np.sqrt(2 * MSW / n_par)
print(f"(seuil d'un test de Student non corrigé, pour une seule paire : {lsd:.1f} DT)\n")

moy = g.mean()
paires = [(a, b) for i, a in enumerate(moy.index) for b in moy.index[i + 1:]]
for a, b in paires:
    ecart = moy[b] - moy[a]
    print(f"{b:12s} - {a:12s} : écart = {ecart:6.1f} DT    {'> HSD : significatif' if abs(ecart) > hsd \
↳ else '<= HSD'}")

tukey = pairwise_tukeyhsd(df["ventes"], df["agencement"], alpha=0.05)
tab = pd.DataFrame(tukey._results_table.data[1:], columns=tukey._results_table.data[0])
print()
print(tab.round(3).to_string(index=False))

q(0.95 ; k=4, ddl=44) = 3.776 -> HSD = 3.776 x sqrt(1140/12) = 36.8 DT
(seuil d'un test de Student non corrigé, pour une seule paire : 27.8 DT)

Par couleur - Classique : écart = 28.4 DT <= HSD
Par thème - Classique : écart = 40.7 DT > HSD : significatif
Vedette - Classique : écart = 19.2 DT <= HSD
Par thème - Par couleur : écart = 12.4 DT <= HSD
Vedette - Par couleur : écart = -9.2 DT <= HSD
Vedette - Par thème : écart = -21.5 DT <= HSD

group1 group2 meandiff p-adj lower upper reject
Classique Par couleur 28.358 0.183 -8.448 65.165 False
Classique Par thème 40.725 0.025 3.918 77.532 True
Classique Vedette 19.192 0.511 -17.615 55.998 False
Par couleur Par thème 12.367 0.806 -24.440 49.173 False
Par couleur Vedette -9.167 0.910 -45.973 27.640 False
Par thème Vedette -21.533 0.410 -58.340 15.273 False
```

La partie « à la main » et la bibliothèque donnent les mêmes décisions. On voit aussi le prix de la prudence : avec 6 comparaisons, il faut un écart d'environ 37 DT pour conclure, alors qu'un test non corrigé se contenterait d'environ 28 DT (deux écarts, 28,4 et 40,7, l'auraient franchi). Après correction, un seul écart franchit la barre : le plus grand, « Par thème » contre « Classique » (écart de 40,7 DT,  $p$  ajustée = 0,025, intervalle de confiance simultané de 3,9 à 77,5 DT : l'effet est détecté, mais **très imprécis**). Avec la correction de Bonferroni (ici sur des tests de Student séparés), on arrive à la même conclusion : seul « Par thème » contre « Classique » reste significatif ( $p$  brute 0,0055, multipliée par 6 : 0,033).

```
brut = {}
for a, b in paires:
    brut[(a, b)] = stats.ttest_ind(g.get_group(b), g.get_group(a), equal_var=True).pvalue
for (a, b), p in brut.items():
    print(f"{b:12s} - {a:12s} : p brute = {p:.4f} ; p Bonferroni (x6) = {min(1, 6 * p):.4f}")
```

```
Par couleur - Classique : p brute = 0.0919 ; p Bonferroni (x6) = 0.5514
Par thème - Classique : p brute = 0.0055 ; p Bonferroni (x6) = 0.0329
Vedette - Classique : p brute = 0.1752 ; p Bonferroni (x6) = 1.0000
Par thème - Par couleur : p brute = 0.3823 ; p Bonferroni (x6) = 1.0000
Vedette - Par couleur : p brute = 0.5288 ; p Bonferroni (x6) = 1.0000
Vedette - Par thème : p brute = 0.0632 ; p Bonferroni (x6) = 0.3794
```

**Un contraste planifié, pour une question précise.** Si, avant l'expérience, Yasmine s'était demandé « l'agencement *Par thème* fait-il mieux que la moyenne des trois autres ? », elle pouvait tester **un seul** contraste  $c = \bar{y}_{\text{thème}} - \frac{1}{3}(\bar{y}_{\text{classique}} + \bar{y}_{\text{couleur}} + \bar{y}_{\text{vedette}})$ , avec un seul test. Un contraste est une combinaison linéaire des moyennes dont les poids somment à 0 ; son écart-type estimé est  $\sqrt{MS_W \sum_i c_i^2 / n_i}$ , et sa statistique suit une loi de Student à  $N - k$  degrés de liberté. Poser la question **avant** évite d'avoir à corriger des dizaines de comparaisons possibles.

```
poids = pd.Series({"Classique": -1 / 3, "Par couleur": -1 / 3, "Par thème": 1.0, "Vedette": -1 / 3})
estim = (poids * moy).sum()
se = np.sqrt(MSW * (poids ** 2 / g.size()).sum())
t_c = estim / se
ic = (estim - stats.t.ppf(0.975, N - k) * se, estim + stats.t.ppf(0.975, N - k) * se)
print(f"contraste thème - moyenne des autres = {estim:.1f} DT (écart-type {se:.1f})")
print(f"t = {t_c:.2f}, p = {2 * stats.t.sf(abs(t_c), N - k):.4f}, IC95 = [{ic[0]:.1f} ; {ic[1]:.1f}]"
```

```
contraste thème - moyenne des autres = 24.9 DT (écart-type 11.3)
t = 2.21, p = 0.0324, IC95 = [2.2 ; 47.6]
```

Le thème l'emporte d'environ 25 DT sur la moyenne des trois autres ( $p = 0,032$ , intervalle de 2 à 48 DT). Une **seule** question posée à l'avance, donc **un seul** test, sans correction : la conclusion est plus nette que celle des six comparaisons deux à deux, mais elle n'est honnête que parce que la question a été choisie **avant** de voir les données.

```
x = np.arange(len(tab))
fig, ax = plt.subplots(figsize=(7, 3.8))
for i, r in tab.iterrows():
    couleur = ORANGE if r["reject"] else BLEU
    ax.hlines(i, r["lower"], r["upper"], color=couleur, lw=3)
    ax.scatter(r["meandiff"], i, color=couleur, zorder=3)
ax.axvline(0, color="#898781", lw=1)
ax.set_yticks(x)
ax.set_yticklabels([f"{r['group2']}" - f"{r['group1']}" for _, r in tab.iterrows()])
ax.set_xlabel("écart de moyennes (DT) avec intervalle de confiance simultané à 95 % (Tukey)")
ax.set_title("Comparaisons deux à deux : en orange, celles dont l'intervalle exclut 0")
plt.savefig("figures/ch08-tukey.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

### 8.2.7 La taille de l'effet

Une p-valeur dit si l'effet est **déTECTABLE**, pas s'il est **important** (volume I, section 3.5.3). Pour l'ANOVA, la mesure usuelle est la part de variance expliquée par le facteur :

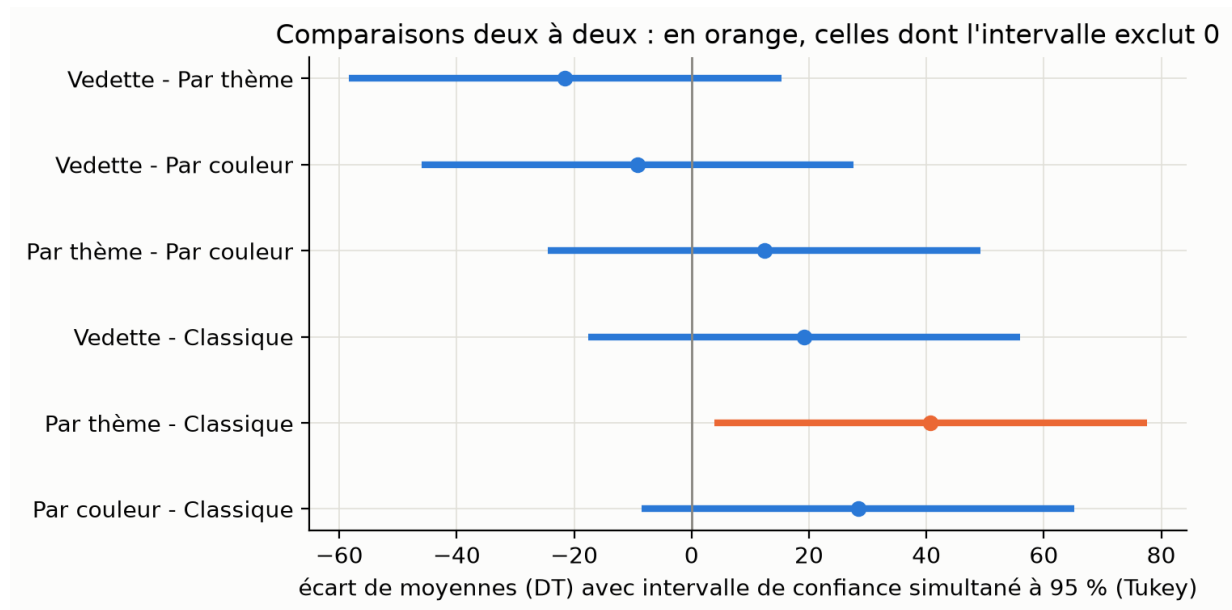


Figure 83: Intervalles de confiance simultanés de Tukey pour les six écarts de moyennes ; seul l'écart Par thème - Classique est entièrement à droite de zéro.

$$\eta^2 = \frac{SS_B}{SS_T} \quad (\text{c'est le } R^2 \text{ de la régression}), \quad \omega^2 = \frac{SS_B - (k - 1)MS_W}{SS_T + MS_W}, \quad f = \sqrt{\frac{\eta^2}{1 - \eta^2}}.$$

$\eta^2$  est **biaisé vers le haut** dans les petits échantillons ;  $\omega^2$  en est une version corrigée. Le  $f$  de Cohen est l'écart-type des moyennes des groupes divisé par  $\sigma$  ; ses repères habituels sont 0,10 (petit), 0,25 (moyen), 0,40 (grand).

```
eta2 = SSB / SST
omega2 = (SSB - (k - 1) * MSW) / (SST + MSW)
f_cohen = np.sqrt(eta2 / (1 - eta2))
print(f"eta² = {eta2:.3f}   omega² = {omega2:.3f}   f de Cohen = {f_cohen:.3f}")
```

```
eta² = 0.174   omega² = 0.116   f de Cohen = 0.460
```

L'agencement explique donc environ 17 % de la variance des ventes journalières ( $\eta^2 = 0,174$ ), ou 12 % après correction du biais ( $\omega^2 = 0,116$ ) : avec  $f = 0,46$ , un effet **grand** selon les repères de Cohen, mais noyé dans un bruit important (les ventes d'une journée varient beaucoup, quel que soit l'agencement). Le  $\omega^2$  est la valeur à retenir : le  $\eta^2$  flatte toujours un peu un petit échantillon.

### 8.2.8 Les blocs : retirer du bruit connu

Yasmine refait l'expérience autrement. Elle sait que les semaines diffèrent beaucoup (soldes, fêtes, météo) : elle découpe l'expérience en **8 semaines** (les **blocs**), et chaque semaine elle teste **chacun des quatre agencements une fois**, dans un ordre tiré au hasard. Soit 32 journées. C'est un **plan en blocs complets randomisés**.

Le modèle ajoute un effet de bloc :

$$y_{ij} = \mu + \tau_i + \beta_j + \varepsilon_{ij}, \quad i = 1..k \text{ (traitements)}, j = 1..b \text{ (blocs)},$$



et la décomposition devient  $SS_T = SS_{\text{trait}} + SS_{\text{blocs}} + SS_E$ , avec  $k - 1$ ,  $b - 1$  et  $(k - 1)(b - 1)$  degrés de liberté. La démonstration est la même qu'en 8.2.2, avec trois sortes d'écarts au lieu de deux. Vérifions-la sur un exemple de neuf nombres, trois traitements et trois semaines :

```
Y = np.array([[10, 12, 14],      # semaine 1 : traitements A, B, C
              [20, 22, 27],      # semaine 2
              [30, 31, 35]], dtype=float)
b_, k_ = Y.shape
mg = Y.mean()
SS_blocs = k_ * ((Y.mean(axis=1) - mg) ** 2).sum()
SS_trait = b_ * ((Y.mean(axis=0) - mg) ** 2).sum()
SS_err = ((Y - Y.mean(axis=1, keepdims=True) - Y.mean(axis=0, keepdims=True) + mg) ** 2).sum()
SS_tot = ((Y - mg) ** 2).sum()
print(f"blocs : {SS_blocs:.2f}   traitements : {SS_trait:.2f}   erreur : {SS_err:.2f}")
print(f"somme = {SS_blocs + SS_trait + SS_err:.2f}   total = {SS_tot:.2f}")
print(f"ddl : blocs {b_ - 1}, traitements {k_ - 1}, erreur {(b_ - 1) * (k_ - 1)}")
```

```
blocs : 602.00   traitements : 44.67   erreur : 3.33
somme = 650.00   total = 650.00
ddl : blocs 2, traitements 2, erreur 4
```

Ici, presque toute la variabilité vient des blocs (les semaines diffèrent beaucoup) ; le résidu, une fois les blocs retirés, est minuscule. Voyons ce que cela change sur les vraies données de l'expérience en blocs :

```
bl = pd.read_csv("donnees/ch08-vitrines-blocs.csv")
print(bl.pivot(index="semaine", columns="agencement",
               ↪ values="ventes").round(0).astype(int).to_string())

sans_bloc = anova_lm(ols("ventes ~ C(agencement)", bl).fit())
avec_bloc = anova_lm(ols("ventes ~ C(agencement) + C(semaine)", bl).fit())
print("\n--- en ignorant les semaines (ANOVA à un facteur) ---")
print(sans_bloc.round(3).to_string())
print("\n--- en tenant compte des semaines (blocs) ---")
print(avec_bloc.round(3).to_string())
```

| agencement | Classique | Par couleur | Par thème | Vedette |
|------------|-----------|-------------|-----------|---------|
| semaine    |           |             |           |         |
| 1          | 202       | 191         | 236       | 208     |
| 2          | 130       | 167         | 192       | 162     |
| 3          | 226       | 256         | 276       | 219     |
| 4          | 142       | 114         | 164       | 134     |
| 5          | 234       | 261         | 272       | 231     |
| 6          | 184       | 220         | 230       | 195     |
| 7          | 132       | 153         | 191       | 174     |
| 8          | 185       | 210         | 217       | 203     |

```

--- en ignorant les semaines (ANOVA à un facteur) ---
              df      sum_sq  mean_sq      F  PR(>F)
C(agencement)  3.0    7932.631  2644.210  1.542  0.225
Residual      28.0   48007.159  1714.541    NaN    NaN

--- en tenant compte des semaines (blocs) ---
              df      sum_sq  mean_sq      F  PR(>F)
C(agencement)  3.0    7932.631  2644.210  15.228  0.0
C(semaine)     7.0   44360.712  6337.245  36.496  0.0
Residual      21.0    3646.447   173.640    NaN    NaN

```

C'est la même mesure, les mêmes 32 ventes, et pourtant la conclusion change du tout au tout. Sans les blocs, l'effet de l'agencement est noyé dans la variabilité entre semaines (qui se retrouve dans le résidu) : le test ne détecte rien ( $F = 1,54$ ,  $p = 0,225$ ). Avec les blocs, la variabilité entre semaines est **isolée** dans sa propre ligne : le carré moyen de l'erreur s'effondre (d'environ 1 715

à 174), et l'effet de l'agencement devient très significatif ( $F = 15,2$ ). Le numérateur, lui, n'a pas bougé (même  $SS_{\text{trait}}$ ) : **seul le bruit a diminué**. On quantifie le gain par l'**efficacité relative** du plan en blocs : le facteur par lequel il aurait fallu multiplier le nombre de répétitions d'un plan sans blocs pour avoir la même précision.

```
b, kk = bl["semaine"].nunique(), bl["agencement"].nunique()
MSE_bloc = avec_bloc.loc["Residual", "mean_sq"]
MS_blocs = avec_bloc.loc["C(semaine)", "mean_sq"]
ER = ((b - 1) * MS_blocs + b * (kk - 1) * MSE_bloc) / ((b * kk - 1) * MSE_bloc)
print(f"efficacité relative du plan en blocs = {ER:.1f}")

# comparaison des agencements à l'intérieur des blocs (Tukey avec le carré moyen de l'erreur du modèle
#   ↪ à blocs)
moyennes = bl.groupby("agencement")["ventes"].mean()
hsd_bloc = stats.studentized_range.ppf(0.95, kk, (b - 1) * (kk - 1)) * np.sqrt(MSE_bloc / b)
print(f"HSD (blocs) = {hsd_bloc:.1f} DT")
print((moyennes - moyennes["Classique"]).round(1).to_string())
```

```
efficacité relative du plan en blocs = 9.0
HSD (blocs) = 18.4 DT
agencement
Classique      0.0
Par couleur    17.1
Par thème      42.9
Vedette        11.3
```

Ici l'efficacité relative est d'environ 9 : un plan sans blocs aurait demandé à peu près neuf fois plus de journées par agencement (de l'ordre de 70 semaines au lieu de 8) pour atteindre la même précision. Et le seuil de Tukey n'est plus que de 18 DT (contre 37 DT dans l'expérience sans blocs) : avec les mêmes quatre agencements, on distingue maintenant des écarts deux fois plus petits. « Par thème » dépasse « Classique » de 43 DT, « Par couleur » de 17 DT, juste sous le seuil, et « Vedette » de 11 DT.

### Deux précautions.

1. Les blocs se **choisissent avant** l'expérience, parce qu'ils sont *connus* comme source de variabilité. On bloque sur ce qui varie beaucoup (la semaine), pas sur n'importe quoi : chaque bloc coûte des degrés de liberté à l'erreur.
2. Le modèle suppose que **l'effet du traitement est le même dans tous les blocs** (pas d'interaction traitement  $\times$  bloc, d'ailleurs non estimable avec une seule observation par case). Si les semaines réagissaient différemment aux agencements, cette hypothèse serait fautive. Avec deux traitements seulement, le plan en blocs est exactement le **test de Student apparié**. Quand les blocs sont des échantillons tirés au sein d'une population plus large, on les traite comme des effets **aléatoires** (modèles mixtes, section 1.7).

### 8.2.9 Deux facteurs et leur interaction

Yasmine s'intéresse maintenant à l'**emballage** (Kraft, Tissu, Coffret) et au **canal** de la commande (Site, Instagram). Pour chacune des 6 combinaisons, elle observe 10 commandes (60 au total) et relève le **panier** en DT. On obtient un plan **factoriel**  $3 \times 2$  **avec répétitions**.

Le modèle à deux facteurs avec interaction s'écrit

$$y_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijr},$$

où  $\alpha_i$  est l'effet de l'emballage  $i$ ,  $\beta_j$  celui du canal  $j$ , et  $(\alpha\beta)_{ij}$  l'**interaction** : ce qu'il faut ajouter à la somme des deux effets principaux pour retrouver la moyenne de la case  $(i, j)$ . S'il n'y a pas

d'interaction, les effets s'**additionnent** ; sinon, l'effet de l'emballage dépend du canal. Les sommes de carrés se décomposent comme en 8.2.2 :  $SS_T = SS_A + SS_B + SS_{AB} + SS_E$ .

```
ec = pd.read_csv("donnees/ch08-emballage-canal.csv")
table = ec.pivot_table(index="emballage", columns="canal", values="panier",
    ↪ aggfunc="mean").loc[["Kraft", "Tissu", "Coffret"]]
table["moyenne ligne"] = table.mean(axis=1)
table.loc["moyenne colonne"] = table.mean()
print(table.round(1).to_string())
```

| canal           | Instagram | Site | moyenne ligne |
|-----------------|-----------|------|---------------|
| emballage       |           |      |               |
| Kraft           | 43.5      | 48.0 | 45.7          |
| Tissu           | 48.0      | 50.3 | 49.2          |
| Coffret         | 69.8      | 60.3 | 65.0          |
| moyenne colonne | 53.8      | 52.9 | 53.3          |

Lisez les moyennes des cases. Sur le **Site**, passer du Kraft au Coffret fait gagner une dizaine de dinars ; sur **Instagram**, le gain est environ **deux fois plus grand**. L'effet de l'emballage dépend du canal : c'est une interaction. Représentons-la, avant de la tester.

```
fig, ax = plt.subplots(figsize=(6.2, 3.8))
for canal, couleur in [("Site", BLEU), ("Instagram", ORANGE)]:
    m = ec[ec["canal"] == canal].groupby("emballage")["panier"].mean().loc[["Kraft", "Tissu",
    ↪ "Coffret"]]
    ax.plot(m.index, m.values, "o-", color=couleur, lw=2)
    ax.text(2.05, m.values[-1], canal, color=couleur, va="center")
ax.set_xlim(-0.2, 2.6)
ax.set_ylabel("panier moyen (DT)")
ax.set_title("Graphique d'interaction : les courbes ne sont pas parallèles")
plt.savefig("figures/ch08-interaction.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**Lire un graphique d'interaction.** Des courbes **parallèles** signifient pas d'interaction (les effets s'additionnent). Des courbes qui **s'écartent** ou **se croisent** signalent une interaction. Ce graphique est le premier outil à regarder pour deux facteurs.

Le test :

```
mod2 = ols("panier ~ C(emballage) * C(canal)", data=ec).fit()
aov2 = anova_lm(mod2)
print(aov2.round(3).to_string())

# vérification à la main de SS_emballage = r * b * somme des (moyenne de l'emballage - moyenne
    ↪ générale)2
r, nb_canaux = 10, 2
mg = ec["panier"].mean()
ss_emb = r * nb_canaux * ((ec.groupby("emballage")["panier"].mean() - mg) ** 2).sum()
print(f"\nSS emballage à la main = {ss_emb:.1f} (tableau : {aov2.loc['C(emballage)',
    ↪ 'sum_sq']:.1f})")
```

|                       | df   | sum_sq   | mean_sq  | F      | PR(>F) |
|-----------------------|------|----------|----------|--------|--------|
| C(emballage)          | 2.0  | 4248.196 | 2124.098 | 39.749 | 0.000  |
| C(canal)              | 1.0  | 12.513   | 12.513   | 0.234  | 0.630  |
| C(emballage):C(canal) | 2.0  | 569.809  | 284.905  | 5.331  | 0.008  |
| Residual              | 54.0 | 2885.656 | 53.438   | NaN    | NaN    |

SS emballage à la main = 4248.2 (tableau : 4248.2)

On lit le tableau **de bas en haut** : on teste d'abord l'**interaction**. Si elle est significative,

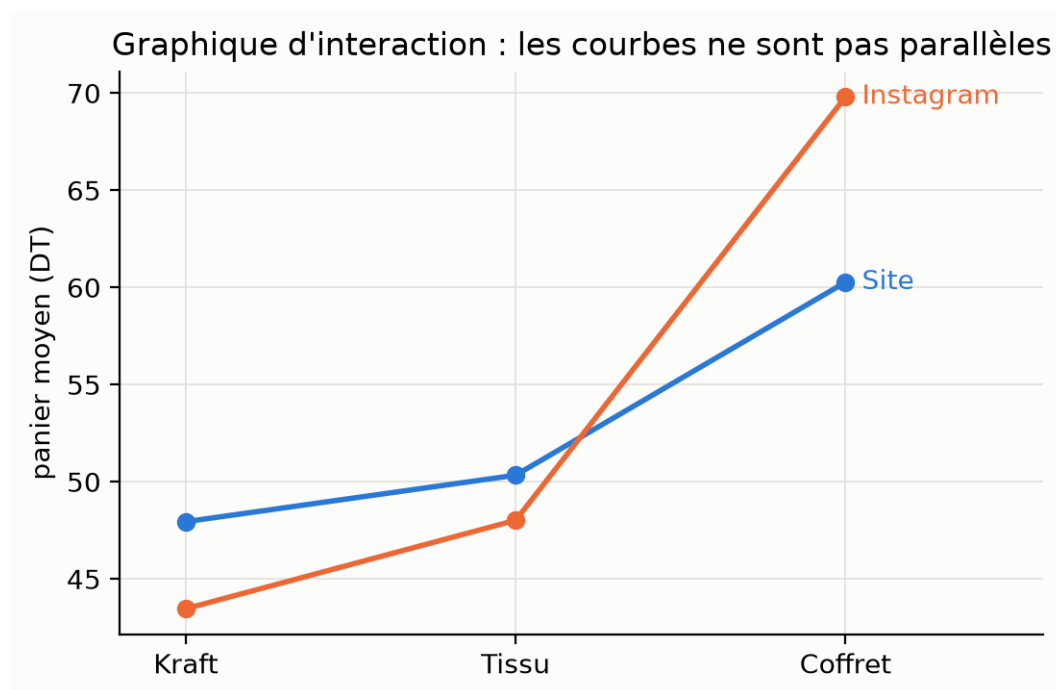


Figure 84: Graphique d'interaction : panier moyen selon l'emballage, une courbe par canal. Les deux courbes montent avec le niveau d'emballage mais celle d'Instagram monte plus fort, donc elles ne sont pas parallèles.

l'interprétation des effets principaux devient **trompeuse**, car un effet principal est une moyenne sur l'autre facteur, qui peut cacher des effets de signes ou d'ampleurs différents. Ici, l'interaction est significative ; l'effet de l'emballage est très fort. En revanche, l'effet principal du **canal** est quasiment nul : ce n'est pas que le canal n'a aucune importance (il joue sur l'effet du coffret), c'est que, **en moyenne sur les emballages**, les deux canaux se compensent. C'est le piège classique. Quand il y a interaction, on étudie les **effets simples** : l'effet d'un facteur à **chaque niveau** de l'autre.

```
for canal in ["Site", "Instagram"]:
    sous = ec[ec["canal"] == canal]
    a = anova_lm(ols("panier ~ C(emballage)", sous).fit())
    m = sous.groupby("emballage")["panier"].mean()
    print(f"{canal:10s}: F emballage = {a.loc['C(emballage)', 'F']:.1f}, p = {a.loc['C(emballage)',
    ↪ 'PR(>F)']:.4f} ; "
          f"gain Coffret - Kraft = {m['Coffret'] - m['Kraft']:.1f} DT")
```

```
Site      : F emballage = 7.1, p = 0.0032 ; gain Coffret - Kraft = 12.3 DT
Instagram : F emballage = 42.1, p = 0.0000 ; gain Coffret - Kraft = 26.3 DT
```

**Plans déséquilibrés.** Ici, chaque case contient 10 observations (plan **équilibré**) : les sommes de carrés sont uniques et les facteurs « orthogonaux ». Quand les effectifs diffèrent selon les cases, la décomposition dépend de l'ordre des facteurs (sommes de carrés de type I, II ou III). Préférez alors l'interprétation par les **coefficients du modèle** et des tests ciblés plutôt que la lecture mécanique du tableau d'ANOVA. C'est une raison de plus de **planifier des plans équilibrés**.

### 8.2.10 Combien d'observations ? La puissance d'une ANOVA

Avant l'expérience, on doit se demander : *si l'effet que je cherche existe vraiment, aurai-je de bonnes chances de le voir ?* C'est la **puissance** (volume I, section 3.5.4). Pour l'ANOVA, sous une hypothèse alternative donnée, la statistique  $F$  suit une loi de Fisher **non centrale**, de paramètre de non-centralité

$$\lambda = \frac{\sum_i n_i \tau_i^2}{\sigma^2} = N f^2,$$

où  $f = \sqrt{\sum_i \tau_i^2 / k} / \sigma$  est l'effet de Cohen (8.2.7). La puissance est  $\mathbb{P}(F > F_{1-\alpha})$  sous cette loi.

Reprenons l'expérience de la vitrine **telle qu'elle a été planifiée** : moyennes vraies 200, 215, 240 et 205 DT, écart-type  $\sigma = 30$  DT, 12 jours par agencement.

```
from statsmodels.stats.power import FTestAnovaPower

mu_vrai = np.array([200, 215, 240, 205])
sigma = 30
f_plan = np.sqrt(((mu_vrai - mu_vrai.mean()) ** 2).mean()) / sigma
print(f"effet de Cohen prévu : f = {f_plan:.3f}")

def puissance(n_par_groupe, f=f_plan, k=4, alpha=0.05):
    N = n_par_groupe * k
    ddl1, ddl2 = k - 1, N - k
    seuil = stats.f.ppf(1 - alpha, ddl1, ddl2)
    return stats.ncf.sf(seuil, ddl1, ddl2, N * f ** 2)    # lambda = N f^2

print(f"puissance avec 12 jours par agencement : {puissance(12):.3f}")
print(f"(statsmodels : {FTestAnovaPower().power(effect_size=f_plan, nobs=48, alpha=0.05,
↪ k_groups=4):.3f})")

# vérification par simulation : on rejoue l'expérience 5000 fois
rng = np.random.default_rng(85)
rejets = 0
for _ in range(5000):
    echantillons = [m + rng.normal(0, sigma, 12) for m in mu_vrai]
    rejets += stats.f_oneway(*echantillons).pvalue < 0.05
print(f"fréquence de rejet simulée : {rejets / 5000:.3f}")

effet de Cohen prévu : f = 0.514
puissance avec 12 jours par agencement : 0.826
(statsmodels : 0.826)
fréquence de rejet simulée : 0.820
```

La puissance **prévue** était d'environ **83 %** : la formule (loi de Fisher non centrale), statsmodels et la simulation (5 000 expériences jouées) s'accordent à quelques millièmes près. Avec l'effet que Yasmine espérait, l'expérience de 12 jours par agencement était donc **bien dimensionnée**. Le  $p = 0,036$  observé, proche du seuil, n'a rien de contradictoire : le  $F$  observé (3,10) est simplement inférieur à celui qu'on attend en moyenne avec l'effet espéré (environ  $1 + \lambda / (k - 1) \approx 5,2$ , avec  $\lambda = N f^2 \approx 12,7$ ) : une fluctuation d'échantillonnage ordinaire, qui arrive environ une fois sur cinq. Et si l'effet réel n'était que **moitié moindre** que celui espéré ?

```
f_moitie = f_plan / 2
print(f"effet moitié moindre : f = {f_moitie:.3f} -> puissance avec 12 jours par agencement =
↪ {puissance(12, f_moitie):.3f}")

def jours_pour_80(f):
    for n_g in range(3, 400):
        if puissance(n_g, f) >= 0.80:
            return n_g
```

```

for f, nom in [(f_plan, "effet prévu"), (f_moitie, "effet moitié moindre")]:
    n_req = jours_pour_80(f)
    print(f"{nom:22s} (f = {f:.3f}) : {n_req} jours par agencement pour 80 % de puissance, soit {4 *
    ↪ n_req} jours au total")
print("statsmodels (N total, avant arrondi à des groupes égaux) :",
      int(np.ceil(FTestAnovaPower().solve_power(effect_size=f_plan, power=0.8, alpha=0.05,
    ↪ k_groups=4))))

```

```

effet moitié moindre : f = 0.257 -> puissance avec 12 jours par agencement = 0.266
effet prévu          (f = 0.514) : 12 jours par agencement pour 80 % de puissance, soit 48 jours au
↪ total
effet moitié moindre (f = 0.257) : 43 jours par agencement pour 80 % de puissance, soit 172 jours au
↪ total
statsmodels (N total, avant arrondi à des groupes égaux) : 46

```

Dessignons la puissance en fonction du nombre de jours, pour plusieurs tailles d'effet :

```

ns = np.arange(4, 45)
fig, ax = plt.subplots(figsize=(6.5, 3.8))
for f, nom, couleur in [(0.10, "petit effet (f = 0,10)", "#898781"), (0.25, "effet moyen (f = 0,25)",
↪ VIOLET),
                        (f_plan, f"effet prévu (f = {f_plan:.2f})".replace(".", ","), ORANGE), (0.40,
↪ "grand effet (f = 0,40)", BLEU)]:
    ax.plot(ns, [puissance(n, f) for n in ns], color=couleur, lw=2, label=nom)
ax.axhline(0.8, color="#c3c2b7", ls="--", lw=1)
ax.axvline(12, color="#c3c2b7", ls=":", lw=1)
ax.set_xlabel("jours par agencement")
ax.set_ylabel("puissance du test F (4 groupes, α = 5 %)")
ax.set_ylim(0, 1.02)
ax.legend(loc="center right", bbox_to_anchor=(1.0, 0.38), fontsize=8)
ax.set_title("La puissance croît avec n, et décroît quand l'effet rétrécit")
plt.savefig("figures/ch08-puissance.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Avec un effet moitié moindre, la puissance de la même expérience tombe à 27 % : plus de deux fois sur trois, elle passerait à côté de l'effet. Pour retrouver 80 %, il faudrait **43 jours par agencement**, soit 172 jours (près de six mois d'ouverture) : diviser l'effet par deux multiplie par environ **quatre** le nombre d'essais nécessaires, car la taille d'échantillon varie comme  $1/f^2$ . C'est la raison pour laquelle on dimensionne l'expérience sur le **plus petit effet qui vaille la peine d'être détecté**.

**La leçon.** On calcule la puissance **avant** de lancer l'expérience, avec l'effet qu'on juge **utile** de détecter (pas celui qu'on espère). Une expérience sous-dimensionnée est pire qu'inutile : elle produit des résultats instables, et quand elle « réussit », elle tend à **surestimer** l'effet (l'effet significatif d'une expérience peu puissante est en moyenne exagéré).

### 8.2.11 La même chose en R

Les statisticiens utilisent souvent R pour l'ANOVA. Voici le même calcul, sur le même fichier, avec aov et TukeyHSD.

```

d <- read.csv("donnees/ch08-vitrines.csv", fileEncoding = "UTF-8")
d$agencement <- factor(d$agencement)
fit <- aov(ventes ~ agencement, data = d)
print(summary(fit))
print(TukeyHSD(fit))

```

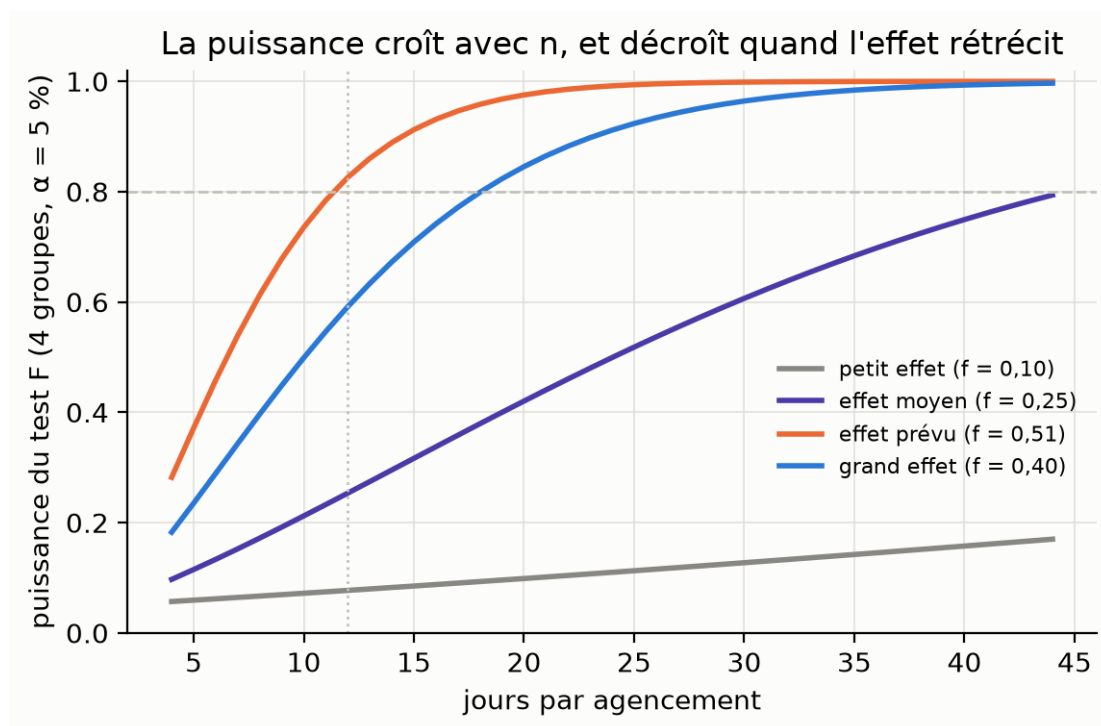


Figure 85: Courbes de puissance du test F à 4 groupes selon le nombre de jours par agencement, pour quatre tailles d'effet ; la ligne pointillée horizontale marque 80 %, la verticale 12 jours.

```

      Df Sum Sq Mean Sq F value Pr(>F)
agencement  3  10595    3532   3.097 0.0363 *
Residuals 44   50168    1140
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Tukey multiple comparisons of means
 95% family-wise confidence level

Fit: aov(formula = ventes ~ agencement, data = d)

$agencement
      diff      lwr      upr    p adj
Par couleur-Classique 28.358333 -8.448152 65.16482 0.1832471
Par thème-Classique  40.725000  3.918514 77.53149 0.0249275
Vedette-Classique   19.191667 -17.614819 55.99815 0.5108611
Par thème-Par couleur 12.366667 -24.439819 49.17315 0.8063634
Vedette-Par couleur  -9.166667 -45.973152 27.63982 0.9096634
Vedette-Par thème   -21.533333 -58.339819 15.27315 0.4103807

```

Même tableau d'analyse de variance, mêmes comparaisons de Tukey (au détail d'arrondi près). Changer d'outil ne change pas les mathématiques.

### 8.2.12 Ce que cachaient les données

Les données étant simulées, nous connaissons la vérité (script `build/donnees_ch08.py`). Pour la vitrine, les moyennes vraies étaient **200, 215, 240 et 205 DT** pour Classique, Par couleur, Par thème et Vedette, avec un bruit de 30 DT. Les écarts **observés** par rapport à « Classique » dans l'expérience à 48 jours (+28, +41 et +19 DT pour Par couleur, Par thème et Vedette) en sont proches mais pas identiques à ceux qui étaient programmés (+15, +40 et +5) : l'erreur d'estimation d'un écart est ici d'environ 14 DT (un écart-type), et le hasard a fait **sur-estimer** l'avantage de « Par couleur » et de « Vedette ». Avec 12 jours par groupe, on ne peut espérer que



des ordres de grandeur. Dans l'expérience en blocs, les effets vrais étaient 0, 15, 40, 5 DT et les écarts estimés sont +17, +43 et +11 DT, avec une erreur d'estimation d'environ 7 DT, deux fois plus petite : c'est le gain de précision apporté par les blocs. Pour l'emballage, l'interaction était programmée : le coffret apporte +10 DT au site mais +22 DT sur Instagram.

### À retenir.

- L'ANOVA **décompose la variabilité** :  $SS_T = SS_B + SS_W$ , exactement ; le test  $F = MS_B/MS_W \sim \mathcal{F}(k-1, N-k)$  sous  $H_0$  compare variabilité entre et à l'intérieur des groupes.
- Avec deux groupes,  $F = t^2$  ; avec  $k$  groupes, l'ANOVA est une **régression sur des indicatrices** ( $R^2 = \eta^2$ ).
- On **vérifie les hypothèses** sur les résidus (normalité, variances égales) ; alternatives : Welch, transformation, Kruskal-Wallis.
- Un  $F$  significatif ne dit pas **quels** groupes diffèrent : utiliser **Tukey** (ou Bonferroni), ou un **contraste planifié** posé *avant* l'expérience.
- **Taille d'effet** :  $\eta^2, \omega^2, f$  ; une p-valeur n'est pas une mesure d'importance.
- Les **blocs** retirent du bruit connu : à nombre d'essais égal, l'effet peut passer de « invisible » à « très net ».
- À **deux facteurs**, regardez d'abord l'**interaction** ; si elle existe, étudiez les **effets simples**, pas les effets principaux.
- **Calculez la puissance avant** l'expérience : une expérience sous-dimensionnée n'est pas fiable.

## 8.3 Les plans factoriels

**Intuition.** En 8.1, nous avons vu que changer un facteur à la fois gaspille des essais et rate les interactions. Un plan **factoriel complet** à  $k$  facteurs, chacun à **deux niveaux**, fait l'inverse : on teste **toutes** les  $2^k$  combinaisons. Chaque essai sert ensuite à estimer **tous** les effets à la fois (les effets principaux comme les interactions), et chaque effet est mesuré sur **tous** les essais, en comparant « la moitié haute » à « la moitié basse ». C'est le plan le plus efficace que l'on puisse imaginer pour un nombre de facteurs modéré.

### 8.3.1 L'expérience de Yasmine : trois facteurs, huit combinaisons

Yasmine veut booster les commandes hebdomadaires de sa boutique en ligne. Trois leviers l'intéressent, chacun à deux niveaux :

| Facteur       | Niveau « - » (-1) | Niveau « + » (+1) |
|---------------|-------------------|-------------------|
| A : emballage | standard          | cadeau            |
| B : prix      | normal            | promotion de 10 % |
| C : relance   | e-mail            | stories Instagram |

Il y a  $2^3 = 8$  combinaisons. Chaque combinaison est testée sur **deux semaines** (deux **répétitions**), tirées au hasard dans le calendrier : 16 semaines au total. La réponse est le nombre de commandes de la semaine.

**Le codage** -1/ +1. On code les niveaux « bas » et « haut » par -1 et +1 (plutôt que 0 et 1). C'est plus qu'une convention : ce codage symétrique rend les colonnes du plan **orthogonales**, ce qui simplifie tous les calculs, comme nous allons le voir.

Voici le plan, avec le tableau des signes de tous les effets possibles. Les colonnes d'interaction sont les **produits** des colonnes des facteurs concernés : si A et B valent  $-1$  et  $+1$ , l'interaction AB vaut  $-1 \times (+1) = -1$ .

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

def plan_2k(k):
    """Plan factoriel 2^k en ordre standard : A varie le plus vite, puis B, etc."""
    return np.array([(1 if (i >> j) & 1 else -1) for j in range(k)] for i in range(2 ** k))

X = plan_2k(3)
S = pd.DataFrame(X, columns=["A", "B", "C"])
S["AB"], S["AC"], S["BC"] = S.A * S.B, S.A * S.C, S.B * S.C
S["ABC"] = S.A * S.B * S.C
S.insert(0, "I", 1)
S.index = ["(1)", "a", "b", "ab", "c", "ac", "bc", "abc"] # notation classique : on nomme les
↳ lettres « hautes »
print(S.to_string())
print("\nS' S (produit scalaire de chaque paire de colonnes) :")
print((S.T @ S).to_string())
```

|     | I | A  | B  | C  | AB | AC | BC | ABC |
|-----|---|----|----|----|----|----|----|-----|
| (1) | 1 | -1 | -1 | -1 | 1  | 1  | 1  | -1  |
| a   | 1 | 1  | -1 | -1 | -1 | -1 | 1  | 1   |
| b   | 1 | -1 | 1  | -1 | -1 | 1  | -1 | 1   |
| ab  | 1 | 1  | 1  | -1 | 1  | -1 | -1 | -1  |
| c   | 1 | -1 | -1 | 1  | 1  | -1 | -1 | 1   |
| ac  | 1 | 1  | -1 | 1  | -1 | 1  | -1 | -1  |
| bc  | 1 | -1 | 1  | 1  | -1 | -1 | 1  | -1  |
| abc | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1   |

S' S (produit scalaire de chaque paire de colonnes) :

|     | I | A | B | C | AB | AC | BC | ABC |
|-----|---|---|---|---|----|----|----|-----|
| I   | 8 | 0 | 0 | 0 | 0  | 0  | 0  | 0   |
| A   | 0 | 8 | 0 | 0 | 0  | 0  | 0  | 0   |
| B   | 0 | 0 | 8 | 0 | 0  | 0  | 0  | 0   |
| C   | 0 | 0 | 0 | 8 | 0  | 0  | 0  | 0   |
| AB  | 0 | 0 | 0 | 0 | 8  | 0  | 0  | 0   |
| AC  | 0 | 0 | 0 | 0 | 0  | 8  | 0  | 0   |
| BC  | 0 | 0 | 0 | 0 | 0  | 0  | 8  | 0   |
| ABC | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 8   |

Les huit lignes sont les huit combinaisons (« a » signifie A haut, B et C bas ; « abc », tout haut ; « (1) », tout bas). Le produit  $S^T S$  vaut  $8 \times$  la matrice identité : **les huit colonnes sont orthogonales** (le produit scalaire de deux colonnes distinctes est nul) et chacune a une norme de  $\sqrt{8}$ . Tout ce chapitre repose sur cette propriété.

### 8.3.2 Les effets, à la main

Les données de l'expérience (ordre des essais tiré au hasard, puis rangées en ordre standard) :

```
f3 = pd.read_csv("donnees/ch08-factoriel-2p3.csv")
print("Les 5 premières semaines dans l'ordre d'exécution (tiré au hasard) :")
print(f3.head(5)[["ordre", "A", "B", "C", "commandes"]].to_string(index=False))

cel = f3.pivot_table(index=["C", "B", "A"], columns="replicat", values="commandes")
cel["moyenne"] = cel.mean(axis=1)
cel.index = S.index # même ordre standard que le tableau des
↳ signes
```

```
print("\nRésultats par combinaison :")
print(cel.round(2).to_string())
```

Les 5 premières semaines dans l'ordre d'exécution (tiré au hasard) :

| ordre | A  | B  | C  | commandes |
|-------|----|----|----|-----------|
| 1     | 1  | -1 | -1 | 57.1      |
| 2     | 1  | 1  | -1 | 56.9      |
| 3     | 1  | 1  | 1  | 78.1      |
| 4     | -1 | -1 | 1  | 45.8      |
| 5     | 1  | 1  | -1 | 68.4      |

Résultats par combinaison :

| replicat | 1    | 2    | moyenne |
|----------|------|------|---------|
| (1)      | 49.9 | 45.9 | 47.90   |
| a        | 57.1 | 65.3 | 61.20   |
| b        | 69.4 | 64.6 | 67.00   |
| ab       | 68.4 | 56.9 | 62.65   |
| c        | 45.8 | 51.5 | 48.65   |
| ac       | 62.1 | 59.8 | 60.95   |
| bc       | 70.1 | 64.4 | 67.25   |
| abc      | 73.2 | 78.1 | 75.65   |

On définit l'**effet principal** d'un facteur comme la **différence moyenne de réponse** quand ce facteur passe de son niveau bas à son niveau haut, **en moyennant sur les niveaux des autres facteurs**. Par exemple, l'effet de A est la moyenne des quatre combinaisons où A est haut, moins la moyenne des quatre où A est bas :

$$\text{effet}(A) = \bar{y}_{A+} - \bar{y}_{A-}.$$

L'**interaction** AB est la demi-différence entre l'effet de A quand B est haut et l'effet de A quand B est bas ; de façon équivalente, c'est la différence entre la moyenne des cellules où AB = +1 et celle où AB = -1. **La même règle s'applique à toutes les colonnes du tableau des signes** : l'effet d'une colonne est la moyenne des réponses là où elle vaut +1 moins la moyenne là où elle vaut -1. Avec  $\bar{y}_i$  les huit moyennes de cellules et  $s_i$  le signe de la colonne :

$$\text{effet} = \frac{1}{4} \sum_{i=1}^8 s_i \bar{y}_i \quad \left( \text{plus généralement } \frac{1}{2^{k-1}} \sum_i s_i \bar{y}_i \right).$$

```
ybar = cel["moyenne"].to_numpy()
effets = {c: (S[c].to_numpy() * ybar).sum() / 4 for c in ["A", "B", "C", "AB", "AC", "BC", "ABC"]}
print("Détail pour A : moyenne des cellules A haut =", round(ybar[S.A.to_numpy() == 1].mean(), 2),
      "; A bas =", round(ybar[S.A.to_numpy() == -1].mean(), 2))
print(pd.Series(effets).round(2).to_string())
print("\nmoyenne générale :", round(ybar.mean(), 2))
```

Détail pour A : moyenne des cellules A haut = 65.11 ; A bas = 57.7

|     |       |
|-----|-------|
| A   | 7.41  |
| B   | 13.46 |
| C   | 3.44  |
| AB  | -5.39 |
| AC  | 2.94  |
| BC  | 3.19  |
| ABC | 3.44  |

moyenne générale : 61.41

Lisez le résultat comme une phrase : « passer de l'emballage standard à l'emballage cadeau fait varier les commandes de ... en moyenne », etc. L'effet du **prix** (B) est le plus grand ; l'interaction AB

est négative : l'effet du cadeau est plus faible en promotion, ce qui rappelle exactement l'exemple de 8.1.5.

**L'algorithme de Yates.** Avant les ordinateurs, on calculait tous les effets avec une procédure d'additions et de soustractions, qui reste élégante. On écrit les moyennes en ordre standard (1),  $a, b, ab, c, ac, bc, abc$ . À chaque étape, la nouvelle colonne contient d'abord les **sommes** de paires voisines, puis leurs **différences** (second moins premier). Après  $k$  étapes, on obtient les « contrastes » dans l'ordre  $I, A, B, AB, C, AC, BC, ABC$  ; il suffit de diviser par  $2^{k-1}$  pour obtenir les effets (et par  $2^k$  pour la moyenne).

```
def yates(y):
    cols, col = [np.array(y, float)], np.array(y, float)
    for _ in range(int(np.log2(len(y)))):
        col = np.concatenate([col[0::2] + col[1::2], col[1::2] - col[0::2]])
        cols.append(col)
    return np.array(cols).T

Y = yates(ybar)
noms = ["I", "A", "B", "AB", "C", "AC", "BC", "ABC"]
tab = pd.DataFrame(Y, index=noms, columns=["moyennes", "étape 1", "étape 2", "contraste (étape 3)"])
tab["effet = contraste / 4"] = tab["contraste (étape 3)"] / 4
tab.loc["I", "effet = contraste / 4"] = tab.loc["I", "contraste (étape 3)"] / 8      # la moyenne se
↳ divise par 8
print(tab.round(2).to_string())
print("\nIdentique aux effets calculés plus haut :", all(np.isclose(tab.loc[c, "effet = contraste /
↳ 4"], effets[c]) for c in effets))
```

|     | moyennes | étape 1 | étape 2 | contraste (étape 3) | effet = contraste / 4 |
|-----|----------|---------|---------|---------------------|-----------------------|
| I   | 47.90    | 109.10  | 238.75  | 491.25              | 61.41                 |
| A   | 61.20    | 129.65  | 252.50  | 29.65               | 7.41                  |
| B   | 67.00    | 109.60  | 8.95    | 53.85               | 13.46                 |
| AB  | 62.65    | 142.90  | 20.70   | -21.55              | -5.39                 |
| C   | 48.65    | 13.30   | 20.55   | 13.75               | 3.44                  |
| AC  | 60.95    | -4.35   | 33.30   | 11.75               | 2.94                  |
| BC  | 67.25    | 12.30   | -17.65  | 12.75               | 3.19                  |
| ABC | 75.65    | 8.40    | -3.90   | 13.75               | 3.44                  |

Identique aux effets calculés plus haut : True

### 8.3.3 Le lien avec la régression, et la précision des effets

Ces effets sont, à un facteur 2 près, les **coefficients de la régression** de la réponse sur les colonnes du tableau des signes (chapitre 1). C'est la clé pour **tester** et **mesurer l'incertitude**. Écrivons le modèle complet :

$$y = \beta_0 + \beta_A x_A + \beta_B x_B + \beta_C x_C + \beta_{AB} x_A x_B + \cdots + \beta_{ABC} x_A x_B x_C + \varepsilon, \quad x \in \{-1, +1\}.$$

**Théorème (effets et coefficients).** Dans un plan factoriel complet  $2^k$  avec  $N$  essais au total, (i)  $\hat{\beta}_j = \frac{1}{N} \sum_i s_{ij} y_i$  ; (ii) l'effet estimé est  $\widehat{\text{effet}}_j = 2\hat{\beta}_j$  ; (iii) les estimateurs sont **non corrélés** et  $\text{Var}(\hat{\beta}_j) = \sigma^2/N$ , donc

$$\text{Var}(\widehat{\text{effet}}_j) = \frac{4\sigma^2}{N}, \quad \text{écart-type d'un effet} = \frac{2\sigma}{\sqrt{N}}.$$

*Démonstration.* Notons  $X$  la matrice  $N \times 2^k$  des colonnes du tableau des signes (répétées si l'on a plusieurs répétitions). Ses colonnes sont orthogonales et chacune a une norme au

carré égale à  $N$  :  $X^T X = NI$ . Les moindres carrés (chapitre 1, section 1.1) donnent  $\hat{\beta} = (X^T X)^{-1} X^T y = \frac{1}{N} X^T y$ , d'où (i). Pour (ii) : quand  $x_j$  passe de  $-1$  à  $+1$ , la réponse prédite varie de  $2\hat{\beta}_j$  (les autres termes se compensent en moyenne grâce à l'orthogonalité). Enfin  $\text{Var}(\hat{\beta}) = \sigma^2 (X^T X)^{-1} = \frac{\sigma^2}{N} I$  : variances égales, covariances nulles ; et  $\text{Var}(2\hat{\beta}_j) = 4\sigma^2/N$ .  $\square$

Deux conséquences remarquables. **La précision est la même pour tous les effets** et ne dépend que du nombre total d'essais  $N$  : tous les essais servent à chaque effet. Et comme les estimateurs sont **non corrélés**, le fait de retirer un effet du modèle ne change pas les estimations des autres. Il reste à estimer  $\sigma^2$  : avec des **répétitions**, on dispose de l'**erreur pure**, la variabilité entre les répétitions d'une même combinaison.

```
mod = smf.ols("commandes ~ A * B * C", data=f3).fit()          # A*B*C = tous les effets principaux et
↳ interactions
N = len(f3)
res = pd.DataFrame({"effet": 2 * mod.params, "ET": 2 * mod.bse, "t": mod.tvalues, "p":
↳ mod.pvalues}).drop("Intercept")
print(res.round(3).to_string())

# erreur pure « à la main » : écarts des deux répétitions à la moyenne de leur combinaison
moy_cel = f3.groupby(["A", "B", "C"])["commandes"].transform("mean")
s2 = ((f3["commandes"] - moy_cel) ** 2).sum() / (N - 8)
print(f"\nerreur pure : s² = {s2:.2f} (ddl = {N - 8}) ; statsmodels : {mod.mse_resid:.2f}")
print(f"écart-type d'un effet = 2 s / sqrt(N) = 2 x {np.sqrt(s2):.2f} / {np.sqrt(N):.0f} = {2 *
↳ np.sqrt(s2 / N):.3f}")
```

|       | effet  | ET   | t      | p     |
|-------|--------|------|--------|-------|
| A     | 7.413  | 2.28 | 3.251  | 0.012 |
| B     | 13.463 | 2.28 | 5.904  | 0.000 |
| A:B   | -5.388 | 2.28 | -2.363 | 0.046 |
| C     | 3.437  | 2.28 | 1.507  | 0.170 |
| A:C   | 2.937  | 2.28 | 1.288  | 0.234 |
| B:C   | 3.188  | 2.28 | 1.398  | 0.200 |
| A:B:C | 3.438  | 2.28 | 1.507  | 0.170 |

```
erreur pure : s² = 20.80 (ddl = 8) ; statsmodels : 20.80
écart-type d'un effet = 2 s / sqrt(N) = 2 x 4.56 / 4 = 2.280
```

La colonne « effet » redonne les effets du calcul à la main (au facteur de moyenne près pour  $I$ ). Les écarts-types des effets sont **tous identiques**, comme le prédit le théorème, et valent environ 2,3 commandes. Chaque effet a **1 degré de liberté** et la même statistique  $t = \text{effet}/\text{ET}$ , à  $N - 8 = 8$  degrés de liberté (erreur pure). Dans un plan factoriel, la décomposition de la variance de 8.2 devient limpide : la somme de carrés de l'effet  $j$  vaut  $N\hat{\beta}_j^2 = N \cdot (\text{effet}_j/2)^2$ .

```
aov = anova_lm(mod)
ss = (N * (res["effet"] / 2) ** 2).round(1)
verif = pd.DataFrame({"SS (tableau d'ANOVA)": aov["sum_sq"].drop("Residual").round(1).to_numpy(),
↳ "SS = N x (effet/2)²": ss.to_numpy()}, index=res.index)
print(verif.to_string())
print(f"SS erreur pure = {aov.loc['Residual', 'sum_sq']:.1f} ; SS total = {((f3['commandes'] -
↳ f3['commandes'].mean()) ** 2).sum():.1f} ; "
↳ f"somme des SS des effets + erreur = {aov['sum_sq'].sum():.1f}")
```

|       | SS (tableau d'ANOVA) | SS = N x (effet/2)² |
|-------|----------------------|---------------------|
| A     | 219.8                | 219.8               |
| B     | 725.0                | 725.0               |
| A:B   | 116.1                | 116.1               |
| C     | 47.3                 | 47.3                |
| A:C   | 34.5                 | 34.5                |
| B:C   | 40.6                 | 40.6                |
| A:B:C | 47.3                 | 47.3                |

SS erreur pure = 166.4 ; SS total = 1396.9 ; somme des SS des effets + erreur = 1396.9

### 8.3.4 Lire l'expérience : quels effets comptent ?

Au seuil de 5 %, trois effets ressortent : le **prix** (B), l'**emballage** (A) et leur **interaction** (AB). Les effets de la relance (C) et des autres interactions sont plus petits que le bruit ne permet de le distinguer ( $p > 0,15$  pour chacun). Représentons les résultats : le **cube des moyennes** (le code de ce dessin est dans `build/fig_ch08.py`) puis le graphique d'interaction AB.

Commandes hebdomadaires moyennes aux 8 sommets du plan  $2^3$

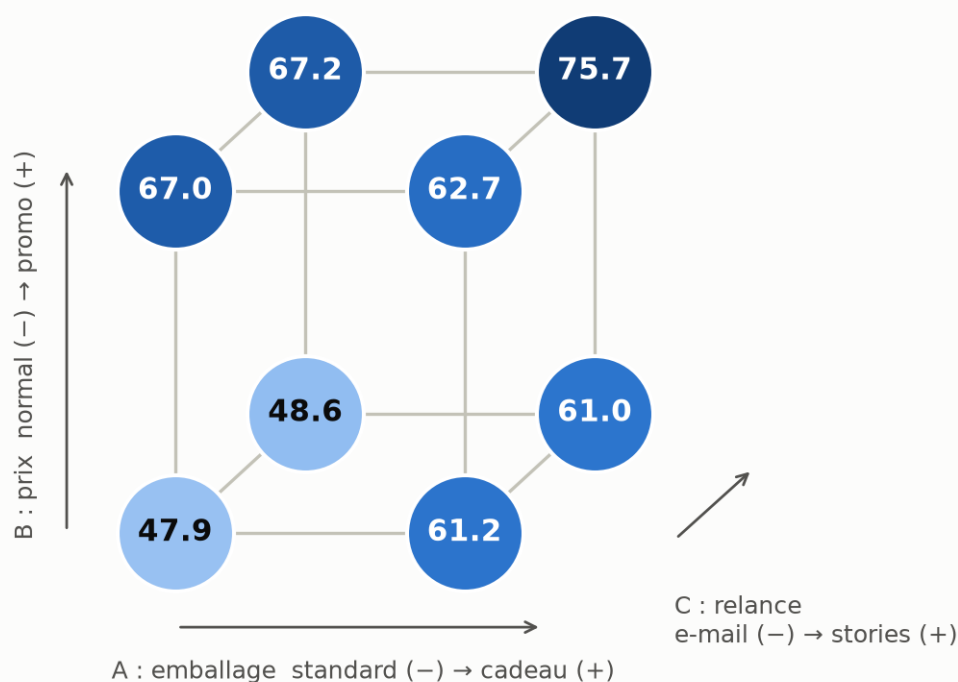


Figure 86: Les huit moyennes de cellules sur un cube dont les arêtes sont les trois facteurs : les valeurs les plus élevées se trouvent du côté « promotion » (haut du cube), et la plus élevée est « tout haut ».

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BLEU, ORANGE, VIOLET = "#2a78d6", "#eb6834", "#4a3aa7"
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "figure.facecolor": "#fcfcfb",
                    "axes.facecolor": "#fcfcfb", "savefig.facecolor": "#fcfcfb", "legend.frameon":
                    False})

fig, ax = plt.subplots(figsize=(5.6, 3.8))
for b, nom, coul in [(-1, "prix normal", BLEU), (1, "promotion", ORANGE)]:
    m = f3[f3["B"] == b].groupby("A")["commandes"].mean()
    ax.plot([-1, 1], m.values, "o-", color=coul, lw=2)
    ax.text(1.06, m.values[1], nom, color=coul, va="center")
ax.set_xticks([-1, 1])
ax.set_xticklabels(["emballage standard", "emballage cadeau"])
```

```
ax.set_xlim(-1.2, 1.9)
ax.set_ylabel("commandes par semaine (moyenne)")
ax.set_title("Interaction emballage × prix (moyennes sur les deux relances)")
plt.savefig("figures/ch08-interaction-AB.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

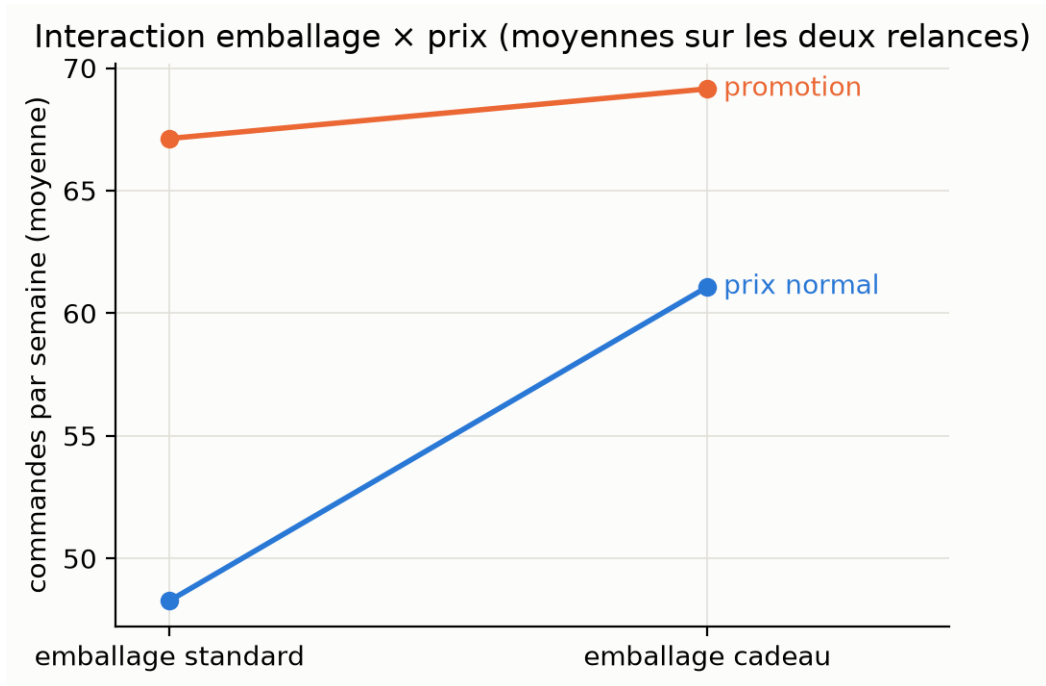


Figure 87: Graphique d'interaction emballage × prix : les deux courbes ne sont pas parallèles ; le cadeau fait nettement monter les commandes au prix normal, beaucoup moins en promotion.

On retrouve la même histoire qu'en 8.1.5 : l'emballage cadeau aide **surtout quand le prix est normal** (il remplace en quelque sorte la promotion) ; en promotion, son apport est bien plus faible. Les deux leviers sont **partiellement substituables**. Un décideur qui n'aurait regardé que les effets principaux aurait conclu « faites les deux » sans voir qu'ils se gênent un peu.

**Modèle réduit et prédiction.** Puisque les effets non significatifs ne se distinguent pas du bruit, on peut les retirer du modèle. Comme les estimateurs sont non corrélés, les effets restants **ne changent pas**, mais l'erreur est estimée avec plus de degrés de liberté. On peut alors prédire la réponse aux huit réglages et choisir le meilleur.

```
red = smf.ols("commandes ~ A + B + A:B", data=f3).fit()
print(red.params.round(3).to_string())
print(f"\nR² complet = {mod.rsquared:.3f} ; R² réduit = {red.rsquared:.3f} ; s (erreur) = {np.sqrt(red.mse_resid):.2f} (ddl {int(red.df_resid)})\n")

grille = pd.DataFrame([(a, b) for b in (-1, 1) for a in (-1, 1)], columns=["A", "B"])
pred = red.get_prediction(grille).summary_frame(alpha=0.05)
grille["prédiction"] = pred["mean"].round(1)
grille["IC95 de la moyenne"] = [f"{lo:.1f} ; {hi:.1f}" for lo, hi in zip(pred["mean_ci_lower"], pred["mean_ci_upper"])]
print(grille.to_string(index=False))
```

|           |        |
|-----------|--------|
| Intercept | 61.406 |
| A         | 3.706  |
| B         | 6.731  |



A:B -2.694

$R^2$  complet = 0.881 ;  $R^2$  réduit = 0.759 ; s (erreur) = 5.29 (ddl 12)

| A  | B  | prédiction | IC95 de la moyenne |
|----|----|------------|--------------------|
| -1 | -1 | 48.3       | [42.5 ; 54.0]      |
| 1  | -1 | 61.1       | [55.3 ; 66.8]      |
| -1 | 1  | 67.1       | [61.4 ; 72.9]      |
| 1  | 1  | 69.2       | [63.4 ; 74.9]      |

Le meilleur réglage prédit est « emballage cadeau **et** promotion » (69,2 commandes), mais son gain sur « promotion seule » (67,1) n'est que de 2 commandes et les deux intervalles de confiance se chevauchent largement : compte tenu du coût d'un emballage cadeau, la décision n'est pas évidente. C'est précisément le genre de conclusion nuancée que seule la prise en compte de l'interaction permet. (Un détail instructif : l'écart-type résiduel passe de 4,56 dans le modèle complet à 5,29 dans le modèle réduit, parce que les effets retirés, C et BC, sont **réels** même s'ils sont non significatifs : ils rejoignent alors l'erreur. Simplifier un modèle n'est pas gratuit.)

### 8.3.5 Ce que l'expérience ne détecte pas : la leçon de la puissance

Le modèle programmé pour simuler ces données (que nous dévoilons maintenant) contenait aussi un effet de la relance C (+4 commandes en effet) et une interaction BC (+3). Le tableau ne les a **pas détectés** :  $p = 0,17$  et  $p = 0,20$ . Absence de preuve n'est pas preuve d'absence. Calculons la puissance de ce plan pour un effet de 4 commandes, avec le vrai bruit  $\sigma = 3,5$  : l'écart-type d'un effet vaut  $2\sigma/\sqrt{N}$  ; la statistique  $t$  suit, sous l'alternative, une loi de Student **non centrale** de paramètre  $\delta = \Delta/(2\sigma/\sqrt{N})$ .

```
sigma, delta_effet = 3.5, 4.0
lignes = []
for r in (1, 2, 3, 4, 6):
    N_r = 8 * r
    ddl = N_r - 8 if r > 1 else None
    if ddl is None:
        # pas de répétition : pas d'erreur pure ; voir 8.3.6
        lignes.append((r, N_r, None, 2 * sigma / np.sqrt(N_r), None))
        continue
    se = 2 * sigma / np.sqrt(N_r)
    seuil = stats.t.ppf(0.975, ddl)
    puissance = stats.nct.sf(seuil, ddl, delta_effet / se) + stats.nct.cdf(-seuil, ddl, delta_effet / se)
    lignes.append((r, N_r, ddl, se, puissance))
tab = pd.DataFrame(lignes, columns=["répétitions r", "essais N = 8 r", "ddl erreur pure", "ET d'un effet", "puissance (effet = 4)"])
print(tab.round(3).to_string(index=False))
```

| répétitions r | essais N = 8 r | ddl erreur pure | ET d'un effet | puissance (effet = 4) |
|---------------|----------------|-----------------|---------------|-----------------------|
| 1             | 8              | NaN             | 2.475         | NaN                   |
| 2             | 16             | 8.0             | 1.750         | 0.520                 |
| 3             | 24             | 16.0            | 1.429         | 0.748                 |
| 4             | 32             | 24.0            | 1.237         | 0.873                 |
| 6             | 48             | 40.0            | 1.010         | 0.971                 |

Sans répétition ( $r = 1$ ), il n'y a pas d'erreur pure, donc pas de test du tout (la case reste vide). Avec deux répétitions, la puissance pour détecter un effet de 4 commandes n'est que de **52 %**, un peu plus de la moitié : l'expérience avait donc à peu près une chance sur deux de rater un effet pourtant réel et non négligeable. Il aurait fallu **quatre répétitions** (32 semaines) pour dépasser 80 % (87 %), trois ne donnant que 75 %. C'est l'arbitrage permanent de l'expérimentateur : *plus de facteurs et peu de répétitions* (on repère les gros effets) ou *moins de facteurs et plus de répétitions* (on repère les petits). La section suivante montre comment obtenir davantage avec moins d'essais.

### 8.3.6 Sans répétition : le plan $2^4$ et le diagramme demi-normal

Les répétitions coûtent cher. Quand on teste plus de facteurs (disons quatre : on ajoute **D**, un message personnalisé), le plan  $2^4$  n'a que 16 essais, **sans répétition**. Il estime 15 effets (4 principaux, 6 interactions d'ordre 2, 4 d'ordre 3 et 1 d'ordre 4) avec 16 essais : plus aucun degré de liberté pour l'erreur pure. Comment tester ? On s'appuie sur un principe d'expérience, observé dans une immense majorité d'études :

**Le principe de parcimonie des effets.** Parmi de nombreux effets possibles, **peu sont réellement actifs**, et les interactions d'ordre élevé (trois facteurs ou plus) sont presque toujours négligeables. Les effets *inactifs* sont donc des **estimations du pur bruit** : ils se répartissent autour de 0 selon une loi normale de même écart-type. Il suffit de repérer ceux qui s'en écartent.

La première idée est le **diagramme demi-normal** (*half-normal plot*). On range les valeurs absolues des  $m$  effets par ordre croissant  $|c|_{(1)} \leq \dots \leq |c|_{(m)}$  et on les compare aux quantiles attendus pour la valeur absolue d'une loi normale :  $z_i = \Phi^{-1}\left(0,5 + 0,5 \frac{i - 0,5}{m}\right)$ . Si **tous** les effets étaient du bruit, les points s'aligneraient sur une droite passant par l'origine, de pente  $\sigma_c$  (l'écart-type d'un effet). Les effets **actifs** sortent de la droite, vers le haut.

La seconde idée, la **méthode de Lenth**, chiffre cette intuition sans modèle d'erreur. Notons  $c_j$  les  $m$  effets estimés :

- $s_0 = 1,5 \times \text{médiane } |c_j|$ , une première estimation de l'écart-type du bruit (robuste : la médiane ignore les quelques effets actifs) ;
- $\text{PSE} = 1,5 \times \text{médiane}\{|c_j| : |c_j| < 2,5 s_0\}$ , l'estimation « **pseudo-erreur standard** » obtenue après avoir écarté les effets manifestement actifs ;
- la **marge d'erreur**  $\text{ME} = t_{0,975;d} \times \text{PSE}$  avec  $d = m/3$ , et la marge **simultanée**  $\text{SME} = t_{\gamma;d} \times \text{PSE}$  avec  $\gamma = \frac{1 + 0,95^{1/m}}{2}$ , qui tient compte du fait qu'on teste  $m$  effets à la fois.

Un effet dont la valeur absolue dépasse la SME est déclaré actif (la ME est plus permissive).

```
g = pd.read_csv("donnees/ch08-factoriel-2p4.csv")
mod4 = smf.ols("commandes ~ A * B * C * D", data=g).fit()
eff = (2 * mod4.params).drop("Intercept")
eff.index = [c.replace(":", "") for c in eff.index]
print("Nombre d'effets estimés :", len(eff), "; ddl de l'erreur :", int(mod4.df_resid), "(aucun :  
↪ modèle saturé)\n")

absolu = eff.abs().sort_values()
m = len(absolu)
s0 = 1.5 * np.median(absolu)
pse = 1.5 * np.median(absolu[absolu < 2.5 * s0])
d = m / 3
ME = stats.t.ppf(0.975, d) * pse
SME = stats.t.ppf((1 + 0.95 ** (1 / m)) / 2, d) * pse
print(f"s0 = {s0:.3f}   PSE = {pse:.3f}   ME = {ME:.2f}   SME = {SME:.2f}\n")
tab = pd.DataFrame({"effet": eff[absolu.index].round(2), "|effet|": absolu.round(2), "actif (|effet|  
↪ > SME)": absolu > SME})
print(tab.iloc[:-1].head(8).to_string())
```

```
Nombre d'effets estimés : 15 ; ddl de l'erreur : 0 (aucun : modèle saturé)
```

```
s0 = 1.012   PSE = 0.900   ME = 2.31   SME = 4.70
```

```
      effet  |effet|  actif (|effet| > SME)
B      12.20    12.20                True
```

|     |       |      |       |
|-----|-------|------|-------|
| A   | 9.53  | 9.53 | True  |
| AB  | -5.68 | 5.68 | True  |
| D   | 5.20  | 5.20 | True  |
| ABC | 1.30  | 1.30 | False |
| ACD | -1.10 | 1.10 | False |
| ABD | -0.92 | 0.92 | False |
| CD  | -0.67 | 0.67 | False |

```

z = stats.norm.ppf(0.5 + 0.5 * (np.arange(1, m + 1) - 0.5) / m)
fig, ax = plt.subplots(figsize=(6.4, 4.4))
actifs = absolu > SME
ax.scatter(z[~actifs], absolu[~actifs], color=BLEU, s=34, zorder=3, label="effets compatibles avec le
↳ bruit")
ax.scatter(z[actifs], absolu[actifs], color=ORANGE, s=44, zorder=4, label="effets actifs (> SME)")
ax.plot([0, z.max()], [0, pse * z.max()], color="#898781", lw=1.2, label="droite du bruit (pente =
↳ PSE)")
ax.axhline(SME, color=VIOLET, ls="--", lw=1)
ax.text(0.05, SME + 0.25, f"SME = {SME:.1f}", color=VIOLET, fontsize=8)
for zi, a, nom in zip(z[actifs], absolu[actifs], absolu.index[actifs]):
    ax.annotate(nom, (zi, a), xytext=(6, -2), textcoords="offset points", fontsize=9, color="#0b0b0b")
ax.set_xlabel("quantile demi-normal théorique")
ax.set_ylabel("|effet| estimé (commandes)")
ax.set_title("Diagramme demi-normal des 15 effets du plan 2^4")
ax.legend(loc="upper left", fontsize=8)
plt.savefig("figures/ch08-demi-normal.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

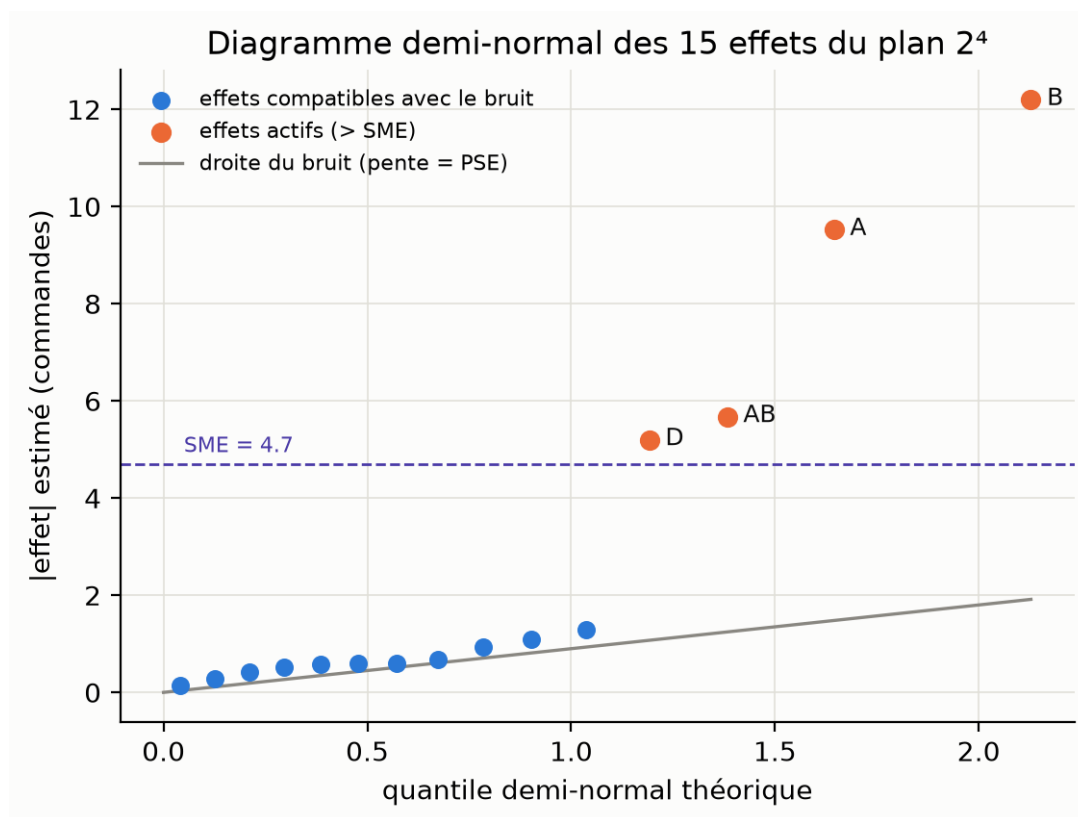


Figure 88: Diagramme demi-normal des 15 effets d'un plan 2<sup>4</sup> non répliqué : onze points suivent la droite du bruit près de l'origine, quatre points actifs (B, A, AB et D) s'en détachent nettement et dépassent le seuil SME.

Onze points alignés sur la droite du bruit, quatre points qui s'en détachent : **B, A, AB et D** (effets de 12,2, 9,5, -5,7 et 5,2 contre une SME de 4,7). Remarquez que D, avec 5,2, ne dépasse la SME que de peu : avec un bruit un peu plus fort, il aurait pu passer inaperçu. Les onze autres, dont toutes les interactions d'ordre 3 et 4, sont indiscernables du bruit. Ce résultat se confirme par une régression sur les seuls effets actifs, en reversant les degrés de liberté « libérés » (les 11 effets retirés) à l'**erreur** :

```
red4 = smf.ols("commandes ~ A + B + A:B + D", data=g).fit()
t4 = pd.DataFrame({"effet": 2 * red4.params, "ET": 2 * red4.bse, "p": red4.pvalues}).drop("Intercept")
print(t4.round(3).to_string())
print(f"s = {np.sqrt(red4.mse_resid):.2f} avec {int(red4.df_resid)} ddl ; R² = {red4.rsquared:.3f}")
```

|     | effet  | ET    | p   |
|-----|--------|-------|-----|
| A   | 9.525  | 0.727 | 0.0 |
| B   | 12.200 | 0.727 | 0.0 |
| A:B | -5.675 | 0.727 | 0.0 |
| D   | 5.200  | 0.727 | 0.0 |

s = 1.45 avec 11 ddl ; R² = 0.981

**Prudence.** Cette démarche **cherche** les effets actifs dans les données : les tests qui suivent sont donc un peu optimistes (on a choisi les effets *parce qu'ils étaient grands*). La méthode de Lenth contrôle ce biais avec la SME, la régression finale non. Un plan non répliqué doit être **confirmé** par quelques essais supplémentaires au réglage retenu.

### 8.3.7 Ce que cachaient les données

Révétons la vérité (build/donnees\_ch08.py).

- **Plan 2<sup>3</sup>** : en unités codées,  $y = 60 + 4A + 6B + 2C - 2,5AB + 1,5BC$ , donc des **effets** de 8 (A), 12 (B), 4 (C), -5 (AB), 3 (BC) et 0 pour AC et ABC, avec un écart-type du bruit  $\sigma = 3,5$ . L'expérience a bien trouvé les trois effets les plus grands (A, B, AB), avec des estimations proches des vraies valeurs (7,4 pour 8, 13,5 pour 12, -5,4 pour -5), et **manqué** C et BC, qui sont plus petits que le bruit ne permet de voir avec 16 essais. Elle a aussi produit deux « effets » (AC et ABC, à environ 3) qui n'existent pas : à environ 1,3 à 1,5 écart-type de zéro (l'écart-type d'un effet est de 2,3), ce sont des fluctuations d'échantillonnage ordinaires, qui n'ont d'ailleurs pas dépassé le seuil de significativité.
- **Plan 2<sup>4</sup>** :  $y = 60 + 4A + 6B - 3AB + 2,5D$ , soit des effets de 8, 12, -6 et 5 ; les 11 autres effets valent 0. La méthode de Lenth a retrouvé **exactement** les quatre effets actifs, sans en inventer un seul, avec des estimations proches de la vérité (9,5, 12,2, -5,7, 5,2) et un bruit de  $\sigma = 1,5$  cette fois plus faible.

#### À retenir.

- Un plan factoriel complet  $2^k$  teste toutes les combinaisons et estime **tous** les effets (principaux et interactions) avec **tous** les essais.
- Codé en  $\pm 1$ , le plan est **orthogonal** :  $X^T X = N I$  ; effet =  $2 \times$  coefficient de régression, **tous les effets ont le même écart-type**  $2\sigma/\sqrt{N}$  et sont non corrélés.
- Les effets se calculent à la main (**contrastes**, algorithme de **Yates**) et se testent par régression avec l'**erreur pure** des répétitions.
- Les **interactions** sont lues sur le **graphique d'interaction** et le **cube** ; ne parlez pas d'un effet principal quand l'interaction domine.
- Sans répétition, le **diagramme demi-normal** et la **méthode de Lenth** repèrent les effets actifs en s'appuyant sur la **parcimonie des effets**.
- Un test non significatif ne prouve pas l'absence d'effet : calculez la **puissance** du plan

pour l'effet qui compte.

## 8.4 Plans fractionnaires et surfaces de réponse

**Intuition.** Le plan factoriel complet est généreux :  $2^k$  essais pour  $k$  facteurs. Mais le nombre d'essais double à chaque facteur ajouté, alors que la plupart des interactions d'ordre élevé sont négligeables. Un **plan fractionnaire** n'exécute qu'une **fraction** ( $\frac{1}{2}$ ,  $\frac{1}{4}$ ...) des combinaisons, bien choisie, en **acceptant de confondre** certains effets entre eux : on économise des essais en renonçant à distinguer des effets que l'on juge de toute façon petits. La seconde moitié de la section est consacrée à un autre but : non plus *repérer les facteurs qui comptent*, mais **trouver le meilleur réglage** de deux facteurs continus, avec les **surfaces de réponse**.

### 8.4.1 Le problème : $2^k$ explose

Combien d'effets contient un plan complet, et de quel ordre ?

```
import numpy as np
import pandas as pd
from math import comb
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

lignes = []
for k in (3, 4, 5, 7):
    ordres = [comb(k, j) for j in range(1, k + 1)]
    lignes.append({"facteurs k": k, "essais 2^k": 2 ** k, "principaux": ordres[0], "interactions
↳ d'ordre 2": ordres[1],
                    "d'ordre 3": ordres[2], "d'ordre 4 et plus": sum(ordres[3:]), "total d'effets":
                    ↳ sum(ordres)})
print(pd.DataFrame(lignes).to_string(index=False))
```

| facteurs k | essais 2 <sup>k</sup> | principaux | interactions d'ordre 2 | d'ordre 3 | d'ordre 4 et plus | total |
|------------|-----------------------|------------|------------------------|-----------|-------------------|-------|
| ↳ d'effets |                       |            |                        |           |                   |       |
| 3          | 8                     | 3          | 3                      | 1         | 0                 | 7     |
| 4          | 16                    | 4          | 6                      | 4         | 1                 | 15    |
| 5          | 32                    | 5          | 10                     | 10        | 6                 | 31    |
| 7          | 128                   | 7          | 21                     | 35        | 64                | 127   |

Avec 7 facteurs, un plan complet demande **128 essais** pour estimer 127 effets, dont **7 seulement** sont des effets principaux et **21** des interactions d'ordre 2 : les 99 autres sont des interactions d'ordre 3 ou plus, presque toujours négligeables en pratique. C'est un énorme gaspillage. Le **principe de hiérarchie** dit que les effets principaux sont plus importants que les interactions d'ordre 2, elles-mêmes plus importantes que celles d'ordre 3, etc. Joint au principe de parcimonie (8.3.6), il justifie de **sacrifier les interactions d'ordre élevé**.

### 8.4.2 Une demi-fraction : le plan $2^{4-1}$

Reprenons les quatre facteurs de l'expérience  $2^4$  de 8.3.6 (A emballage, B prix, C relance, D message personnalisé), mais supposons que Yasmine n'ait pu faire que **8 essais** au lieu de 16. Comment choisir 8 combinaisons parmi 16 ?

Construisons un plan complet  $2^3$  sur A, B, C, et **définissons D comme le produit**  $D = ABC$  : la colonne de D n'est plus libre, elle est **imposée** par les trois autres. C'est le **générateur** de la

fraction. Multiplier les deux membres par  $D$  donne la **relation de définition** :

$$D = ABC \implies D \cdot D = ABC \cdot D \implies I = ABCD,$$

puisque  $D \cdot D = I$  (le produit d'une colonne  $\pm 1$  par elle-même vaut  $+1$ ). Toute la structure de confusion découle de cette seule relation.

**Confusion (alias).** Si  $I = ABCD$ , alors multiplier un effet par  $ABCD$  donne l'effet confondu avec lui :  $A = A \cdot ABCD = BCD$ ,  $B = ACD$ ,  $C = ABD$ ,  $D = ABC$ , et pour les interactions d'ordre 2,  $AB = CD$ ,  $AC = BD$ ,  $AD = BC$ . Les colonnes du tableau des signes de ces effets sont **identiques** dans les 8 essais : il est mathématiquement impossible de les distinguer. Le contraste calculé sur la colonne de  $A$  estime donc **la somme**  $A + BCD$  ; si l'interaction d'ordre 3  $BCD$  est négligeable (c'est l'hypothèse de hiérarchie), c'est bien l'effet de  $A$ .

Vérifions-le numériquement, puis utilisons la **vraie** expérience : parmi les 16 essais de 8.3.6, ne gardons que les 8 pour lesquels  $ABCD = +1$  (c'est exactement la fraction définie par  $I = ABCD$ ) et comparons les effets estimés avec ceux du plan complet.

```
def plan_2k(k):
    return np.array([(1 if (i >> j) & 1 else -1) for j in range(k)] for i in range(2 ** k))

X = plan_2k(3)
A, B, C = X.T
D = A * B * C
print("colonne A = colonne BCD :", np.array_equal(A, B * C * D), "| AB = CD :", np.array_equal(A * B,
    ↪ C * D),
      "| AC = BD :", np.array_equal(A * C, B * D), "| AD = BC :", np.array_equal(A * D, B * C))

g = pd.read_csv("donnees/ch08-factoriel-2p4.csv")
complet = (2 * smf.ols("commandes ~ A * B * C * D", data=g).fit().params).drop("Intercept")
demi = g[g.A * g.B * g.C * g.D == 1] # la demi-fraction I = ABCD : 8 essais sur 16
mod_demi = smf.ols("commandes ~ A * B * C", data=demi).fit() # 8 essais, 8 paramètres : modèle
    ↪ saturé
eff_demi = (2 * mod_demi.params).drop("Intercept")
somme_alias = {"A": ["A", "B:C:D"], "B": ["B", "A:C:D"], "C": ["C", "A:B:D"], "A:B": ["A:B", "C:D"],
    ↪ "A:C": ["A:C", "B:D"], "B:C": ["B:C", "A:D"], "A:B:C": ["A:B:C", "D"]}
noms_alias = {"A": "A + BCD", "B": "B + ACD", "C": "C + ABD", "A:B": "AB + CD", "A:C": "AC + BD",
    ↪ "B:C": "BC + AD", "A:B:C": "ABC + D"}
tab = pd.DataFrame({"contraste estime": [noms_alias[i] for i in eff_demi.index],
    ↪ "demi-fraction (8 essais)": eff_demi.round(2).to_numpy(),
    ↪ "somme des 2 effets du plan complet": [round(sum(complet[e] for e in
    ↪ somme_alias[i]), 2) for i in eff_demi.index]},
    index=eff_demi.index)

print()
print(len(demi), "essais retenus sur", len(g))
print(tab.to_string())
print("\nEffets du plan complet (16 essais) pour mémoire :", complet[["A", "B", "A:B",
    ↪ "D"]].round(2).to_dict())
```

colonne A = colonne BCD : True | AB = CD : True | AC = BD : True | AD = BC : True

8 essais retenus sur 16

|       | contraste estime | demi-fraction (8 essais) | somme des 2 effets du plan complet |
|-------|------------------|--------------------------|------------------------------------|
| A     | A + BCD          | 9.00                     | 9.00                               |
| B     | B + ACD          | 11.10                    | 11.10                              |
| A:B   | AB + CD          | -6.35                    | -6.35                              |
| C     | C + ABD          | -1.20                    | -1.20                              |
| A:C   | AC + BD          | -0.45                    | -0.45                              |
| B:C   | BC + AD          | 0.15                     | 0.15                               |
| A:B:C | ABC + D          | 6.50                     | 6.50                               |

Effets du plan complet (16 essais) pour mémoire : {'A': 9.53, 'B': 12.2, 'A:B': -5.68, 'D': 5.2}

Chaque ligne de la demi-fraction estime **la somme** de deux effets, et la comparaison le confirme **exactement** : l'estimation obtenue avec 8 essais est égale, au centième près, à la somme des deux effets correspondants du plan complet. (On peut le démontrer : sur la fraction,  $A = BCD$ , donc  $\sum_{\text{moitié}} Ay = \frac{1}{2} (\sum_{\text{tous}} Ay + \sum_{\text{tous}} BCDy)$ , ce qui donne  $\hat{\ell}_A = \hat{A} + \widehat{BCD}$ .) Comme les interactions d'ordre 3 sont presque nulles, les estimations de A, B et de D (ligne « ABC + D ») sont proches de celles du plan complet : avec 8 essais au lieu de 16, on aurait tiré les mêmes conclusions. (Le contraste « AB + CD » ne permet pas de dire à lui seul que c'est AB plutôt que CD : c'est la **connaissance du domaine**, ou une expérience complémentaire, qui tranche.)

### 8.4.3 Résolution : mesurer la qualité d'une fraction

Dans  $I = ABCD$ , le seul « mot » a **4 lettres**. On appelle **résolution** d'un plan fractionnaire la **longueur du plus court mot** de sa relation de définition, notée en chiffres romains. Elle résume ce qui est confondu avec quoi :

| Résolution | Plus court mot | Les effets principaux sont confondus avec... | Les interactions d'ordre 2 sont confondues avec... |
|------------|----------------|----------------------------------------------|----------------------------------------------------|
| III        | 3 lettres      | des interactions d'ordre 2                   | des effets principaux                              |
| IV         | 4 lettres      | des interactions d'ordre 3                   | <b>entre elles</b>                                 |
| V          | 5 lettres      | des interactions d'ordre 4                   | des interactions d'ordre 3                         |

**Pourquoi la longueur du plus court mot ?** Un effet de  $j$  lettres est confondu avec le produit de cet effet par chaque mot  $w$  de la relation. Ce produit a  $|j - \ell|$  lettres au moins, où  $\ell$  est la longueur de  $w$ , et au plus  $j + \ell$ . Avec un mot de  $\ell$  lettres, un effet principal ( $j = 1$ ) est donc confondu avec un effet d'ordre au moins  $\ell - 1$ , et une interaction d'ordre 2 avec un effet d'ordre au moins  $\ell - 2$ . Plus  $\ell$  est grand, plus la confusion est entre des effets d'ordre élevé, donc négligeables. D'où la règle de choix : **à nombre d'essais donné, on cherche la résolution la plus élevée.**

Une demi-fraction de résolution IV ( $2^{4-1}$  avec  $I = ABCD$ ) est un excellent compromis : les effets principaux sont libres de toute interaction d'ordre 2.

### 8.4.4 Plus fractionnaire encore : le plan $2^{5-2}$ et le piège de la confusion

Pour cinq facteurs, un plan complet demande 32 essais. Un plan à **8 essais** seulement est possible : on part du plan complet  $2^3$  en A, B, C et on **définit deux facteurs de plus** par  $D = AB$  et  $E = AC$ . C'est un plan  $2^{5-2}$  (un quart de fraction). Les relations de définition sont  $I = ABD$  et  $I = ACE$ , et leur produit  $ABD \cdot ACE = A^2BCDE = BCDE$  est aussi une relation :

$$I = ABD = ACE = BCDE.$$

Le plus court mot a 3 lettres : **résolution III**. Calculons toute la structure de confusion par programme : un effet est confondu avec son produit par chaque mot, et le produit de deux ensembles de lettres est la **différence symétrique** (les lettres communes s'annulent car  $L^2 = I$ ).

```
def produit(u, v):
    return "".join(sorted(set(u) ^ set(v)))          # lettres communes éliminées

mots = ["ABD", "ACE", "BCDE"]
```



```
tous = ["A", "B", "C", "D", "E", "AB", "AC", "AD", "AE", "BC", "BD", "BE", "CD", "CE", "DE", "ABC",
        ↪ "ABD", "ABE",
           "ACD", "ACE", "ADE", "BCD", "BCE", "BDE", "CDE", "ABCD", "ABCE", "ABDE", "ACDE", "BCDE",
        ↪ "ABCDE"]
classes = {}
for e in tous:
    classe = tuple(sorted({e} | {produit(e, w) for w in mots}, key=lambda s: (len(s), s)))
    classes[classe] = classes.get(classe, 0) + 1
print(len(tous), "effets possibles, répartis en", len(classes), "classes de confusion (8 essais = 7
        ↪ contrastes + la moyenne) :\n")
for c in sorted(classes, key=lambda c: (len(c[0]), c[0])):
    print(" " + " = ".join(x or "I" for x in c))
```

31 effets possibles, répartis en 8 classes de confusion (8 essais = 7 contrastes + la moyenne) :

```
I = ABD = ACE = BCDE
A = BD = CE = ABCDE
B = AD = CDE = ABCE
C = AE = BDE = ABCD
D = AB = BCE = ACDE
E = AC = BCD = ABDE
BC = DE = ABE = ACD
BE = CD = ABC = ADE
```

Chaque ligne est une **classe de confusion** : les effets d'une même ligne ne peuvent pas être distingués. La première ligne regroupe les trois mots de la relation de définition avec la **moyenne générale I** : ces trois interactions sont indiscernables de la constante. Remarquez que les effets principaux sont confondus avec des interactions d'ordre 2 (par exemple  $D = AB$  : le facteur D est parfaitement confondu avec l'interaction de A et B). C'est le défaut de la résolution III, et voici ce que cela donne en pratique.

**Une simulation instructive.** Supposons que la vérité soit la suivante : A a un effet de +6, B de +4, A et B ont une interaction de +8, et D n'a aucun effet. (Nous le savons parce que nous simulons ; dans la vraie vie, on l'ignore.) On exécute les 8 essais du plan  $2^{5-2}$ .

```
rng = np.random.default_rng(87)
A, B, C = plan_2k(3).T
D, E = A * B, A * C # générateurs de la fraction
vrai = lambda A, B, C, D, E: 50 + 3 * A + 2 * B + 4 * A * B # effets : A = 6, B = 4, AB = 8,
        ↪ tout le reste 0
y = vrai(A, B, C, D, E) + rng.normal(0, 0.8, 8)

contrastes = {"A": A, "B": B, "C": C, "D (= AB)": D, "E (= AC)": E, "BC (= DE)": B * C, "ABC (= CD =
        ↪ BE)": A * B * C}
estimates = {nom: (col * y).sum() / 4 for nom, col in contrastes.items()}
print(pd.Series(estimates).round(2).to_string())
```

|                 |       |
|-----------------|-------|
| A               | 5.95  |
| B               | 4.76  |
| C               | -0.56 |
| D (= AB)        | 8.17  |
| E (= AC)        | -0.23 |
| BC (= DE)       | 0.09  |
| ABC (= CD = BE) | -0.51 |

Le tableau annonce un « effet de D » d'environ 8 (la colonne D, qui est aussi la colonne AB, capte l'interaction réelle) alors que **D n'a aucun effet** ! Un analyste qui suppose les interactions négligeables conclurait « le message personnalisé fait gagner 8 commandes », et se tromperait. C'est le danger de la résolution III : on ne peut s'y fier que si l'on est certain que les interactions d'ordre 2 sont absentes.

**Le remède : le repliement (*fold-over*).** On exécute une **seconde série de 8 essais** en **inversant tous les signes** du plan ( $A \rightarrow -A$ , etc.). Les relations de définition de longueur impaire changent de signe d'un bloc à l'autre et disparaissent du plan combiné ; il ne reste que  $I = BCDE$  : on obtient un plan de **résolution IV à 16 essais**, dans lequel les effets principaux sont séparés de toutes les interactions d'ordre 2.

```
X1 = plan_2k(3)
bloc1 = pd.DataFrame({"A": X1[:, 0], "B": X1[:, 1], "C": X1[:, 2]})
bloc2 = -bloc1 # repliement : tous les signes inversés
plan = pd.concat([bloc1.assign(bloc=1), bloc2.assign(bloc=2)], ignore_index=True)
plan["D"] = np.where(plan.bloc == 1, plan.A * plan.B, -plan.A * plan.B)
plan["E"] = np.where(plan.bloc == 1, plan.A * plan.C, -plan.A * plan.C)
# vérification : sur les 16 essais, seule la relation BCDE = +1 subsiste
print("BCDE = +1 sur tous les essais :", bool((plan.B * plan.C * plan.D * plan.E == 1).all()),
      "| ABD = +1 :", bool((plan.A * plan.B * plan.D == 1).all()), "| ACE = +1 :", bool((plan.A *
      ↪ plan.C * plan.E == 1).all()))

plan["y"] = vrai(plan.A, plan.B, plan.C, plan.D, plan.E) + rng.normal(0, 0.8, 16)
formule = "y ~ A + B + C + D + E + A:B + A:C + A:D + A:E + B:C + B:D + B:E" # BC=DE, BD=CE,
↪ BE=CD restent confondus deux à deux
fo = smf.ols(formule, data=plan).fit()
t = pd.DataFrame({"effet": 2 * fo.params, "p": fo.pvalues}).drop("Intercept")
print()
print(t.round(3).to_string())
print(f"\n(ddl de l'erreur : {int(fo.df_resid)} ; s = {np.sqrt(fo.mse_resid):.2f})")
```

```
BCDE = +1 sur tous les essais : True | ABD = +1 : False | ACE = +1 : False
```

|     | effet  | p     |
|-----|--------|-------|
| A   | 6.157  | 0.001 |
| B   | 3.624  | 0.004 |
| C   | 0.337  | 0.490 |
| D   | -0.098 | 0.834 |
| E   | -0.107 | 0.820 |
| A:B | 8.082  | 0.000 |
| A:C | 0.686  | 0.209 |
| A:D | -0.381 | 0.441 |
| A:E | -0.223 | 0.640 |
| B:C | -0.329 | 0.500 |
| B:D | -0.281 | 0.560 |
| B:E | -0.780 | 0.167 |

```
(ddl de l'erreur : 3 ; s = 0.86)
```

Après le repliement, l'effet de **D** est proche de zéro et celui de l'interaction **AB** réapparaît (environ 8) : les deux, qui étaient confondus dans le plan à 8 essais, sont maintenant distincts. On a payé 8 essais supplémentaires pour lever l'ambiguïté. Dans la pratique, on n'exécute le repliement **que si** la première série laisse un doute sur un effet important.

### 8.4.5 Optimiser un réglage : les surfaces de réponse

Jusqu'ici, nous cherchions *quels* facteurs comptent. Voici une autre question : **quel réglage donne la meilleure réponse ?** Yasmine cuit ses céramiques de Nabeul au four et veut maximiser le **pourcentage de pièces sans défaut**. Deux réglages continus : la **température** (autour de 1 000 °C) et la **durée** (autour de 6 heures). La **méthodologie des surfaces de réponse** (*response surface methodology*) procède par étapes :

1. un plan factoriel à deux niveaux **avec points au centre** pour détecter si la réponse est une surface **plane** (modèle du premier ordre) ou **courbe** ;
2. si la surface est plane, on suit le **chemin de plus forte pente** (la direction du gradient) jusqu'à ce que la réponse cesse de monter ;

3. au voisinage de l'optimum, la surface est **courbe** : on ajuste un modèle du **second ordre** (quadratique) avec un plan adapté, le **plan composite centré**, puis on cherche son sommet.

On travaille en **unités codées** :  $x_1 = (\text{température} - 1000)/40$  et  $x_2 = \text{durée} - 6$ , de sorte que le centre est (0,0) et le cube vaut  $\pm 1$ . Voici le plan composite centré réalisé : 4 points **factoriels**  $(\pm 1, \pm 1)$ , 4 points **axiaux**  $(\pm\sqrt{2}, 0)$  et  $(0, \pm\sqrt{2})$ , et 5 **répétitions du point central**, soit 13 essais.

```
cc = pd.read_csv("donnees/ch08-ccd-cuisson.csv")
print(cc.sort_values("ordre").to_string(index=False))
```

| ordre | x1      | x2      | temperature_C | duree_h | reussite |
|-------|---------|---------|---------------|---------|----------|
| 1     | 0.0000  | 0.0000  | 1000.0        | 6.00    | 85.5     |
| 2     | -1.0000 | -1.0000 | 960.0         | 5.00    | 73.8     |
| 3     | 1.0000  | -1.0000 | 1040.0        | 5.00    | 73.0     |
| 4     | -1.0000 | 1.0000  | 960.0         | 7.00    | 69.9     |
| 5     | 0.0000  | 0.0000  | 1000.0        | 6.00    | 84.0     |
| 6     | -1.4142 | 0.0000  | 943.4         | 6.00    | 71.4     |
| 7     | 1.0000  | 1.0000  | 1040.0        | 7.00    | 79.8     |
| 8     | 1.4142  | 0.0000  | 1056.6        | 6.00    | 81.2     |
| 9     | 0.0000  | 0.0000  | 1000.0        | 6.00    | 83.0     |
| 10    | 0.0000  | 0.0000  | 1000.0        | 6.00    | 84.0     |
| 11    | 0.0000  | 1.4142  | 1000.0        | 7.41    | 72.1     |
| 12    | 0.0000  | -1.4142 | 1000.0        | 4.59    | 70.9     |
| 13    | 0.0000  | 0.0000  | 1000.0        | 6.00    | 82.5     |

**Pourquoi ces points ?** Un plan à deux niveaux ne peut pas estimer les termes **quadratiques**  $x_1^2$  et  $x_2^2$  :  $(\pm 1)^2 = 1$  pour tous les points, la colonne est constante. Les **points axiaux**  $(\pm\sqrt{2})$  et **centraux** (0) donnent trois niveaux de chaque facteur, ce qui permet de les estimer. Les **répétitions au centre** servent à estimer l'**erreur pure**, donc à tester la qualité de l'ajustement. Le choix  $\alpha = \sqrt{2} = \sqrt[4]{n_{\text{fact}}}$  rend le plan **rotatable** : la précision de la prédiction ne dépend que de la distance au centre, pas de la direction.

**Étape 1 : y a-t-il de la courbure ?** Avec les seuls points factoriels et centraux, on compare la moyenne des points factoriels à celle du centre : si la surface était plane, elles seraient égales en moyenne. L'écart-type est estimé par les 5 répétitions du centre.

```
centre = cc[(cc.x1 == 0) & (cc.x2 == 0)]
fact = cc[(cc.x1.abs() == 1) & (cc.x2.abs() == 1)]
ss_pe = ((centre.reussite - centre.reussite.mean()) ** 2).sum()
df_pe = len(centre) - 1
s_pe = np.sqrt(ss_pe / df_pe)
courbure = fact.reussite.mean() - centre.reussite.mean()
se_c = s_pe * np.sqrt(1 / len(fact) + 1 / len(centre))
t_c = courbure / se_c
print(f"moyenne factoriels = {fact.reussite.mean():.2f} ; moyenne au centre =
↳ {centre.reussite.mean():.2f} ; écart = {courbure:.2f}")
print(f"erreur pure : s = {s_pe:.2f} ({df_pe} ddl) ; t = {t_c:.2f} ; p = {2 * stats.t.sf(abs(t_c),
↳ df_pe):.4f}")
b1 = (fact.x1 * fact.reussite).sum() / len(fact)
b2 = (fact.x2 * fact.reussite).sum() / len(fact)
print(f"pentes du premier ordre (points factoriels) : b1 = {b1:.2f}, b2 = {b2:.2f}")
```

```
moyenne factoriels = 74.12 ; moyenne au centre = 83.80 ; écart = -9.67
erreur pure : s = 1.15 (4 ddl) ; t = -12.53 ; p = 0.0002
pentes du premier ordre (points factoriels) : b1 = 2.27, b2 = 0.72
```

Le centre est **nettement au-dessus** de la moyenne des coins : la réponse a un **sommet** à l'intérieur du carré, et un modèle plan serait faux. Inutile de suivre un chemin de plus forte pente : nous sommes déjà près de l'optimum, et il faut un modèle courbe.

### 8.4.6 Ajuster le modèle quadratique

Le modèle du second ordre pour deux facteurs s'écrit

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2 + \beta_{12} x_1 x_2 + \varepsilon.$$

C'est une **régression linéaire** (linéaire en les  $\beta$ ) sur six colonnes : les moindres carrés du chapitre 1 s'appliquent tels quels.

```
q = smf.ols("reussite ~ x1 + x2 + I(x1**2) + I(x2**2) + x1:x2", data=cc).fit()
noms = {"Intercept": "β0", "x1": "β1 (x1)", "x2": "β2 (x2)", "I(x1 ** 2)": "β11 (x1²)", "I(x2 ** 2)":
↪ "β22 (x2²)", "x1:x2": "β12 (x1 x2)"}
coef = pd.DataFrame({"estimation": q.params, "ET": q.bse, "t": q.tvalues, "p":
↪ q.pvalues}).rename(index=noms)
print(coef.round(3).to_string())
print(f"\nR² = {q.rsquared:.3f} ; R² ajusté = {q.rsquared_adj:.3f} ; s = {np.sqrt(q.mse_resid):.2f}
↪ ({int(q.df_resid)} ddl)")
```

|             | estimation | ET    | t       | p     |
|-------------|------------|-------|---------|-------|
| β0          | 83.800     | 0.490 | 170.916 | 0.000 |
| β1 (x1)     | 2.870      | 0.388 | 7.404   | 0.000 |
| β2 (x2)     | 0.575      | 0.388 | 1.482   | 0.182 |
| β11 (x1²)   | -3.694     | 0.416 | -8.886  | 0.000 |
| β22 (x2²)   | -6.094     | 0.416 | -14.660 | 0.000 |
| β12 (x1 x2) | 2.675      | 0.548 | 4.880   | 0.002 |

$R^2 = 0.980$  ;  $R^2$  ajusté =  $0.966$  ;  $s = 1.10$  (7 ddl)

Les termes quadratiques et le produit  $x_1 x_2$  sont significatifs ; l'effet linéaire de  $x_2$  ne l'est pas. On le **garde** quand même : par le principe de hiérarchie, on ne retire pas un terme d'ordre 1 si des termes d'ordre supérieur qui le contiennent ( $x_2^2, x_1 x_2$ ) sont dans le modèle (retirer  $\beta_2$  reviendrait à imposer que le sommet soit au centre en  $x_2$ , ce qui dépend du choix d'origine du codage).

**Le modèle est-il adapté ?** Avec les répétitions au centre, on peut séparer le résidu en **erreur pure** (variabilité entre répétitions) et **défaut d'ajustement** (écart systématique entre le modèle et les moyennes des points) :

$$SS_{\text{résidu}} = SS_{\text{pur}} + SS_{\text{défaut}}, \quad F = \frac{SS_{\text{défaut}} / (ddl_{\text{res}} - ddl_{\text{pur}})}{SS_{\text{pur}} / ddl_{\text{pur}}}.$$

Si  $F$  est grand, le modèle quadratique est insuffisant (il faudrait un ordre supérieur ou une autre échelle).

```
ss_def, df_def = q.ssr - ss_pe, q.df_resid - df_pe
F_def = (ss_def / df_def) / (ss_pe / df_pe)
print(f"SS résidu = {q.ssr:.2f} = SS erreur pure {ss_pe:.2f} ({df_pe} ddl) + SS défaut d'ajustement
↪ {ss_def:.2f} ({int(df_def)} ddl)")
print(f"F de défaut d'ajustement = {F_def:.2f} ; p = {stats.f.sf(F_def, df_def, df_pe):.3f}")
```

SS résidu = 8.41 = SS erreur pure 5.30 (4 ddl) + SS défaut d'ajustement 3.11 (3 ddl)  
F de défaut d'ajustement = 0.78 ; p = 0.562

Pas de défaut d'ajustement détectable ( $F = 0,78, p = 0,56$ ) : le modèle quadratique décrit correctement la surface sur le domaine étudié. (Attention : avec seulement 4 degrés de liberté d'erreur pure, ce test a peu de puissance ; c'est un garde-fou, pas une preuve.)

### 8.4.7 Trouver l'optimum : point stationnaire et analyse canonique

Écrivons le modèle ajusté sous forme matricielle, avec  $x = (x_1, x_2)^\top$ ,  $b = (\beta_1, \beta_2)^\top$  et la matrice symétrique des termes quadratiques :

$$\hat{y} = \beta_0 + b^\top x + x^\top B x, \quad B = \begin{pmatrix} \beta_{11} & \beta_{12}/2 \\ \beta_{12}/2 & \beta_{22} \end{pmatrix}.$$

**Point stationnaire.** Le gradient de  $\hat{y}$  est  $b + 2Bx$  (volume I, section 1.2 pour le gradient, et section 1.3 pour l'optimisation). Il s'annule au point

$$x_s = -\frac{1}{2}B^{-1}b, \quad \hat{y}_s = \beta_0 + \frac{1}{2}b^\top x_s.$$

(En effet,  $Bx_s = -b/2$  donc  $x_s^\top Bx_s = -b^\top x_s/2$ , et  $\hat{y}_s = \beta_0 + b^\top x_s - b^\top x_s/2$ .) La nature de ce point se lit sur les **valeurs propres** de  $B$  (volume I, section 1.1.3) : le hessien de  $\hat{y}$  vaut  $2B$ . Si **toutes** les valeurs propres sont **négatives**,  $x_s$  est un **maximum** ; toutes **positives**, un **minimum** ; de signes **mixtes**, un **col** (point selle), qui n'est pas un optimum. Les **vecteurs propres** donnent les axes de la surface : le long de l'axe de plus petite valeur propre en valeur absolue, la réponse varie peu (une « **crête** » : plusieurs réglages donnent presque le même résultat).

```
p = q.params
b = np.array([p["x1"], p["x2"]])
B = np.array([[p["I(x1 ** 2)"], p["x1:x2"] / 2], [p["x1:x2"] / 2, p["I(x2 ** 2)"]]])
xs = -0.5 * np.linalg.solve(B, b)
ys = p["Intercept"] + 0.5 * b @ xs
lam, vecs = np.linalg.eigh(B)

print(f"point stationnaire (codé) : x1 = {xs[0]:.3f}, x2 = {xs[1]:.3f} ; distance au centre =
↳ {np.linalg.norm(xs):.2f} (domaine : jusqu'à {np.sqrt(2):.2f})")
print(f"en unités naturelles : température = {1000 + 40 * xs[0]:.0f} °C, durée = {6 + xs[1]:.2f} h")
print(f"réponse prédite au sommet : {ys:.2f} %")
print(f"valeurs propres de B : {lam.round(2)} -> {'maximum' if (lam < 0).all() else 'minimum' if (lam
↳ > 0).all() else 'col'}")
for l, v in zip(lam, vecs.T):
    print(f" valeur propre {l:.6.2f} : axe ({v[0]:+.2f}, {v[1]:+.2f}) ; "
          f"perte de {abs(l):.2f} points par unité² d'écart au sommet le long de cet axe")

nouveau = pd.DataFrame({"x1": [xs[0]], "x2": [xs[1]]})
pred = q.get_prediction(nouveau).summary_frame(alpha=0.05)
print(f"\nIC95 de la réponse moyenne au sommet : [{pred['mean_ci_lower']:[0]:.1f} ;
↳ {pred['mean_ci_upper']:[0]:.1f}])")
print(f"intervalle de prédiction à 95 % pour un nouvel essai : [{pred['obs_ci_lower']:[0]:.1f} ;
↳ {pred['obs_ci_upper']:[0]:.1f}])")

point stationnaire (codé) : x1 = 0.441, x2 = 0.144 ; distance au centre = 0.46 (domaine : jusqu'à 1.41)
en unités naturelles : température = 1018 °C, durée = 6.14 h
réponse prédite au sommet : 84.47 %
valeurs propres de B : [-6.69 -3.1 ] -> maximum
valeur propre -6.69 : axe (-0.41, +0.91) ; perte de 6.69 points par unité² d'écart au sommet le
↳ long de cet axe
valeur propre -3.10 : axe (-0.91, -0.41) ; perte de 3.10 points par unité² d'écart au sommet le
↳ long de cet axe

IC95 de la réponse moyenne au sommet : [83.3 ; 85.6]
intervalle de prédiction à 95 % pour un nouvel essai : [81.6 ; 87.3]
```

Les deux valeurs propres sont négatives ( $-6,7$  et  $-3,1$ ) : le point stationnaire est bien un **maximum**, à environ **1 018 °C** et **6,14 h**, avec un taux prédit de **84,5 %** (intervalle de confiance de la

moyenne : 83,3 à 85,6 %). La surface retombe deux fois plus vite le long du premier axe (direction  $(-0,41; +0,91)$ , surtout la durée) que le long du second (direction  $(-0,91; -0,41)$ , surtout la température) : une durée mal réglée coûte plus cher qu'une température un peu décalée. Le sommet se trouve à l'intérieur du domaine expérimental (distance au centre de 0,46, bien inférieure au rayon  $\sqrt{2}$  des points axiaux) : c'est une **interpolation**, ce qui est crucial ; extrapoler un modèle quadratique au-delà des essais est dangereux, car un polynôme du second degré n'a aucune raison de bien décrire la surface loin des données.

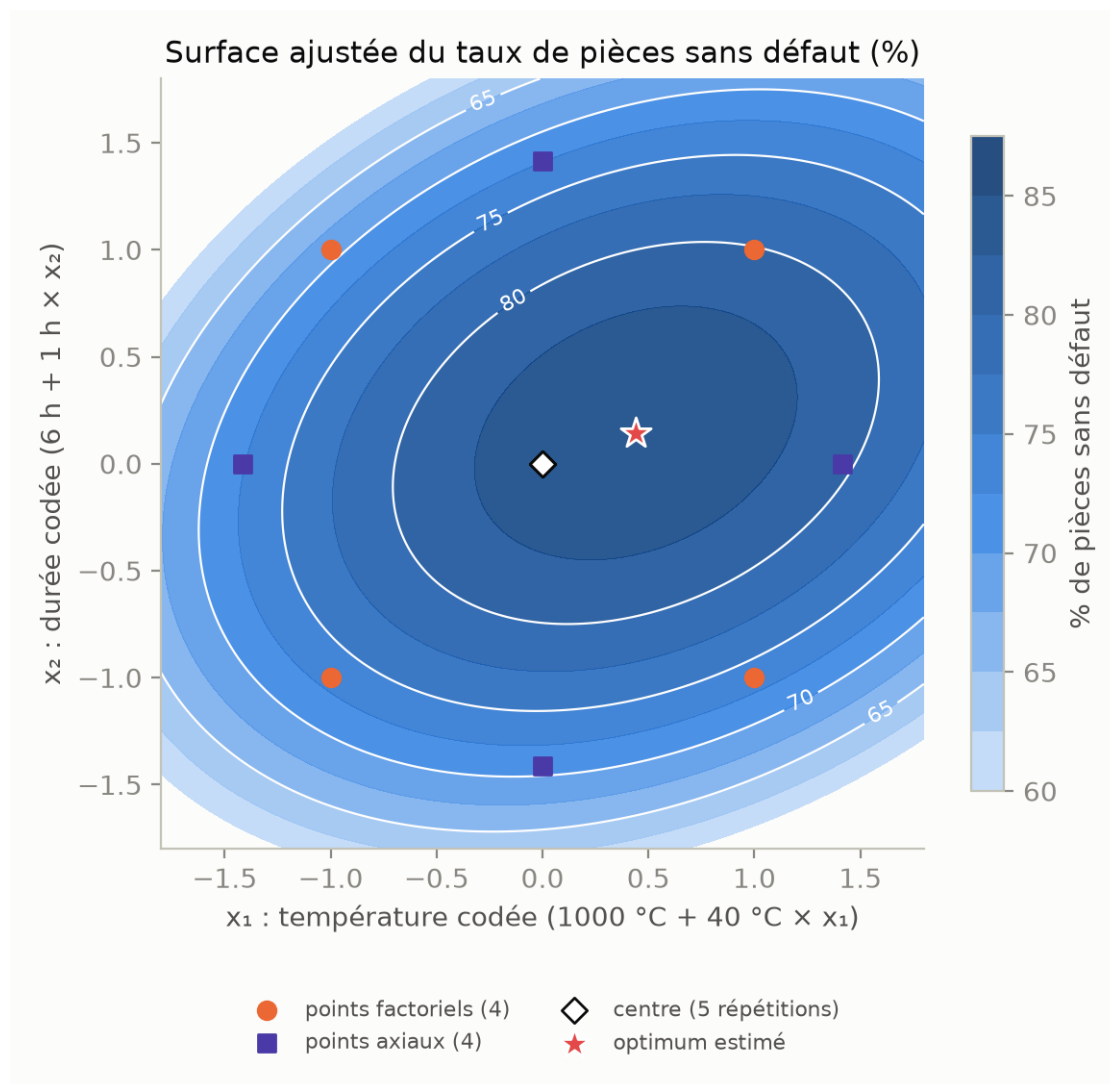


Figure 89: Surface ajustée du taux de pièces sans défaut en fonction de la température et de la durée (unités codées), avec les 13 points du plan composite centré et l'optimum estimé. Les courbes de niveau sont des ellipses allongées et inclinées autour du sommet.

La figure (code dans `build/fig_ch08.py`) montre les courbes de niveau : des **ellipses** centrées sur le sommet, inclinées à cause du terme croisé  $\beta_{12}$  (température et durée **interagissent** : la meilleure durée dépend de la température). La différence entre les deux valeurs propres se voit dans la forme des ellipses : plus étroites dans la direction de forte courbure.

**Confirmer.** Un optimum prédit par un modèle est une **hypothèse** : on la teste par quelques **essais de confirmation** au réglage proposé. Nous simulons ici trois fournées au sommet estimé (avec le vrai processus, que nous connaissons, et son bruit de 0,9) :

```
def vrai_taux(x1, x2):
    return 84 + 3 * x1 + 1 * x2 - 4 * x1 ** 2 - 6 * x2 ** 2 + 2.5 * x1 * x2

rng = np.random.default_rng(88)
confirm = vrai_taux(xs[0], xs[1]) + rng.normal(0, 0.9, 3)
print("trois fournées de confirmation :", confirm.round(1))
print(f"moyenne = {confirm.mean():.1f} %, dans l'intervalle de prédiction : {bool(((confirm >=
↪ pred['obs_ci_lower'])[0]) & (confirm <= pred['obs_ci_upper'])[0])).all())}")
```

```
trois fournées de confirmation : [84.1 84.2 83.8]
moyenne = 84.0 %, dans l'intervalle de prédiction : True
```

### Les pièges de l'optimisation.

- Un **col** ou une **crête** (valeur propre presque nulle) ne désigne pas un optimum unique : il faut se demander quel réglage de la crête est le plus **économique** ou le plus **robuste** (par exemple, moins sensible aux variations de température du four).
- L'optimum n'est valable que **dans le domaine étudié** et pour la **réponse mesurée** : avec **plusieurs réponses** (taux de réussite, coût énergétique, durée), on cherche un compromis (fonctions de désirabilité, courbes de niveau superposées).
- Le modèle quadratique est une **approximation locale** : si l'optimum prédit tombe en dehors du domaine, on déplace le plan dans cette direction et on recommence plutôt que d'extrapoler.

## 8.4.8 ⊕ Pour aller plus loin : les plans optimaux

**Section optionnelle.** Les plans classiques (factoriels, composites) supposent un domaine régulier (un cube) et un nombre d'essais « rond ». Quand le domaine est irrégulier (combinaisons **impossibles**), ou quand le budget impose un nombre d'essais particulier, on peut **calculer** un plan par ordinateur.

L'idée de la **D-optimalité** : la variance des coefficients estimés est proportionnelle à  $(X^T X)^{-1}$  (chapitre 1, section 1.2) ; on cherche les essais qui rendent cette matrice « la plus petite », c'est-à-dire qui **maximisent**  $\det(X^T X)$ . Le **volume** de l'ellipsoïde de confiance de  $\hat{\beta}$  est en effet proportionnel à  $\det(X^T X)^{-1/2}$ . On le fait par un **algorithme d'échange** : on part de  $n$  points tirés dans un ensemble de **candidats**, et on remplace un point par un candidat tant que cela augmente le déterminant.

```
def info(points):
    x1, x2 = points[:, 0], points[:, 1]
    Xm = np.column_stack([np.ones(len(points)), x1, x2, x1 ** 2, x2 ** 2, x1 * x2])    # modèle
    ↪ quadratique
    return np.linalg.det(Xm.T @ Xm)

def echange(candidats, n, rng, depart=20):
    meilleur = (-1, None)
    for _ in range(depart):
        idx = list(rng.choice(len(candidats), n, replace=True))
        change = True
        while change:
            change = False
            for pos in range(n):
                best_j, best_d = idx[pos], info(candidats[idx])
                for j in range(len(candidats)):
                    essai = idx.copy()
                    essai[pos] = j
                    d = info(candidats[essai])
                    if d > best_d * (1 + 1e-9):
```



```

        best_j, best_d, change = j, d, True
        idx[pos] = best_j
        d = info(candidats[idx])
        if d > meilleur[0]:
            meilleur = (d, idx.copy())
    return meilleur

rng = np.random.default_rng(89)
grille = np.array([(a, b) for a in np.linspace(-1, 1, 5) for b in np.linspace(-1, 1, 5)])
d_opt, idx = echange(grille, 9, rng)
factoriel_3x3 = np.array([(a, b) for a in (-1, 0, 1) for b in (-1, 0, 1)])
au_hasard = np.median([info(grille[rng.choice(25, 9, replace=False)]) for _ in range(2000)])
print("9 essais choisis parmi la grille 5 x 5 :")
print(pd.Series([tuple(grille[i]) for i in idx]).value_counts().sort_index().to_string())
print(f"\ndet(X'X) : D-optimal = {d_opt:.0f} ; factoriel 3x3 = {info(factoriel_3x3):.0f} ; "
      f"9 points au hasard (médiane sur 2000 tirages) = {au_hasard:.0f}")

# avec une contrainte : la combinaison « tout haut » (x1 + x2 > 1) est impossible (le four ne le
↳ permet pas)
possible = grille[grille.sum(axis=1) <= 1.0]
d_c, idx_c = echange(possible, 9, rng)
print(f"\nSous la contrainte x1 + x2 <= 1 ({len(possible)} candidats), plan D-optimal à 9 essais (det
↳ = {d_c:.0f}) :")
print(pd.Series([tuple(possible[i]) for i in idx_c]).value_counts().sort_index().to_string())

9 essais choisis parmi la grille 5 x 5 :
(-1.0, -1.0)    1
(-1.0, 0.0)     1
(-1.0, 1.0)     1
(0.0, -1.0)     1
(0.0, 0.0)      1
(0.0, 1.0)      1
(1.0, -1.0)     1
(1.0, 0.0)      1
(1.0, 1.0)      1

det(X'X) : D-optimal = 5184 ; factoriel 3x3 = 5184 ; 9 points au hasard (médiane sur 2000 tirages) = 94

Sous la contrainte x1 + x2 <= 1 (22 candidats), plan D-optimal à 9 essais (det = 1920) :
(-1.0, -1.0)    2
(-1.0, 0.0)     1
(-1.0, 1.0)     1
(0.0, -1.0)     1
(0.0, 0.0)      1
(0.0, 1.0)      1
(1.0, -1.0)     1
(1.0, 0.0)      1

```

Sur le domaine carré, l'algorithme retrouve **exactement** la grille  $3 \times 3$  classique (le plan factoriel à trois niveaux, de même déterminant, 5 184) : les plans classiques ne sont pas arbitraires, ils sont souvent optimaux. Pour mémoire, 9 points tirés au hasard font en médiane 55 fois moins bien (déterminant de 94). L'intérêt de l'optimisation numérique apparaît dès que le domaine est **contraint** : le plan composite centré serait inutilisable tel quel (le coin (1, 1) est impossible), alors que l'algorithme propose immédiatement un plan adapté : la grille  $3 \times 3$  privée du coin impossible, avec le coin opposé  $(-1, -1)$  **répété**. Il faut toutefois rester prudent : un plan *D*-optimal dépend **du modèle supposé** (ici un quadratique) ; si le modèle est faux, l'optimalité ne signifie plus grand-chose.

### 8.4.9 Ce que cachait les données

- **Plan  $2^{4-1}$**  : les effets de la demi-fraction (A, B, AB, D) sont proches de ceux du plan complet de 8.3.6, ce qui était attendu car la vérité (effets de 8, 12, -6, 5 ; le reste nul) respecte la parcimonie. Les confusions A+BCD, etc. n'ont gêné que parce que les interactions d'ordre 3 sont réellement nulles.
- **Plan  $2^{5-2}$**  : nous avons programmé  $A = 6$ ,  $B = 4$ ,  $AB = 8$  et  $D = 0$ . La fraction à 8 essais a attribué à **tort** un effet d'environ 8 à D ; le repliement l'a corrigé. La confusion n'est pas un défaut de calcul, c'est une **conséquence logique** du choix des générateurs.
- **Surface de réponse** : le vrai modèle était  $y = 84 + 3x_1 + x_2 - 4x_1^2 - 6x_2^2 + 2,5x_1x_2$ , dont le sommet exact est en  $(x_1, x_2) = (0,429; 0,173)$ , soit environ **1 017 °C** et **6,17 h**, avec un taux maximal de 84,7 %. Le plan à 13 essais a estimé le sommet en (0,441; 0,144), soit 1 018 °C et 6,14 h, avec un taux de 84,5 % : à environ 1 °C et 0,03 h de la vérité (1 017 °C, 6,17 h, 84,7 %). Les trois fournées de confirmation (84,1 ; 84,2 ; 83,8) tombent dans l'intervalle de prédiction. Ce niveau de précision est celui d'une expérience **bien conçue et peu bruitée** ( $\sigma = 0,9$ ) ; avec un bruit plus fort ou moins de répétitions au centre, le sommet estimé aurait été moins précis.

#### À retenir.

- Un plan **fractionnaire**  $2^{k-p}$  n'exécute qu'une fraction  $2^{-p}$  des combinaisons, définie par des **générateurs** ; les effets sont alors **confondus** (alias) en classes, déterminées par la **relation de définition**.
- La **résolution** (longueur du plus court mot) dit ce qui est confondu avec quoi : **III** (principaux/interactions d'ordre 2), **IV** (principaux libres, interactions d'ordre 2 entre elles), **V** (tout propre jusqu'à l'ordre 2). À nombre d'essais donné, cherchez la résolution la plus élevée.
- La confusion peut **tromper** (résolution III) : un **repliement** (inversion de tous les signes) lève l'ambiguïté au prix d'essais supplémentaires.
- Les **surfaces de réponse** cherchent le meilleur réglage : détecter la **courbure** (points au centre), puis ajuster un **modèle quadratique** avec un **plan composite centré** (points factoriels, axiaux, centraux).
- Le **point stationnaire**  $x_s = -\frac{1}{2}B^{-1}b$  est un maximum si les valeurs propres de  $B$  sont négatives ; vérifiez qu'il est **dans le domaine**, testez le **défaut d'ajustement**, **confirmez** par des essais.
- Les **plans optimaux** ( $D$ -optimalité) calculent un plan quand le domaine ou le budget sortent des cadres classiques ; ils dépendent du modèle supposé.

## 8.5 Exercices du chapitre 8

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 2 à 4 se font avec les mêmes trois groupes ; 7, 9 et 10 se font entièrement à la main.

### Énoncés

**Exercice 1 ★ (concevoir une expérience).** Yasmine veut comparer deux présentations de la page d'accueil de son site, A et B. Elle propose : « affichage A la semaine prochaine, affichage B la semaine suivante, et je compare les commandes ». (a) Quelle est l'unité expérimentale ? (b) Citez deux raisons pour lesquelles la comparaison sera biaisée. (c) Proposez un plan qui applique les trois principes de Fisher (randomisation, répétition, blocage).

**Exercice 2 ★ (ANOVA à la main).** Trois fournisseurs de papier d'emballage, quatre lots chacun ; on mesure la résistance à la déchirure (en newtons) : Fournisseur 1 : 52, 48, 50, 50 ; Fournisseur 2 : 56, 58, 54, 56 ; Fournisseur 3 : 62, 60, 64, 62. Calculez les moyennes,  $SS_B$ ,  $SS_W$ , les carrés moyens et la statistique  $F$ . Combien de degrés de liberté ? Que conclure ?

**Exercice 3 ★★ (taille d'effet).** Avec les données de l'exercice 2, calculez  $\eta^2$  et  $\omega^2$ . Pourquoi  $\omega^2 < \eta^2$  ? Vérifiez ensuite que la régression sur indicatrices redonne le même  $F$  et le même  $R^2$ .

**Exercice 4 ★★ (comparaisons multiples).** Toujours avec les données de l'exercice 2, calculez le seuil HSD de Tukey (utilisez  $q_{0,95;3,9} \approx 3,95$ ) et dites quelles paires de fournisseurs diffèrent. Pourquoi ne pas simplement faire trois tests de Student à 5 % ?

**Exercice 5 ★★ (blocs).** Quatre traitements sont testés dans trois blocs (trois semaines) ; ventes en dizaines de DT :

|           | traitement 1 | traitement 2 | traitement 3 | traitement 4 |
|-----------|--------------|--------------|--------------|--------------|
| semaine 1 | 10           | 14           | 12           | 16           |
| semaine 2 | 20           | 25           | 22           | 27           |
| semaine 3 | 31           | 33           | 32           | 36           |

- (a) Calculez  $SS_{\text{blocs}}$ ,  $SS_{\text{traitements}}$ ,  $SS_E$  et leurs degrés de liberté. (b) Comparez le  $F$  des traitements avec et sans les blocs. (c) Que s'est-il passé ?

**Exercice 6 ★★ (interaction).** On teste deux facteurs, A et B, à deux niveaux ; les moyennes de réponse sont  $\bar{y}_{A-B-} = 20$ ,  $\bar{y}_{A+B-} = 30$ ,  $\bar{y}_{A-B+} = 25$ ,  $\bar{y}_{A+B+} = 15$ . Calculez l'effet principal de A, celui de B et l'interaction AB. L'effet principal de A est nul : A n'a-t-il donc aucune influence ? Quel est le meilleur réglage ?

**Exercice 7 ★★ (algorithme de Yates).** Plan  $2^3$  sans répétition, essais en ordre standard : (1) = 10,  $a = 14$ ,  $b = 12$ ,  $ab = 20$ ,  $c = 11$ ,  $ac = 15$ ,  $bc = 13$ ,  $abc = 21$ . Calculez les sept effets et la moyenne par l'algorithme de Yates, puis par les contrastes. Quels effets sont non nuls ?

**Exercice 8 ★★★ (puissance d'un plan factoriel).** Dans un plan  $2^3$  répliqué  $r$  fois, l'écart-type du bruit est  $\sigma = 3$ . On veut détecter un effet de  $\Delta = 4$  avec une puissance d'au moins 80 %, au seuil de 5 %. (a) Donnez l'écart-type d'un effet en fonction de  $r$ . (b) Estimez à la main un ordre de grandeur de  $r$  avec l'approximation normale. (c) Calculez la valeur exacte avec la loi de Student non centrale.

**Exercice 9 ★★ (confusion).** On veut un plan  $2^{4-1}$  (8 essais, 4 facteurs). (a) Avec le générateur  $D = ABC$ , donnez la relation de définition, la résolution et les alias de AB. (b) Avec  $D = AB$ , mêmes questions. (c) Lequel choisir et pourquoi ?

**Exercice 10 ★★★ (un plan  $2^{6-2}$ ).** On construit 6 facteurs en 16 essais avec les générateurs  $E = ABC$  et  $F = BCD$ . (a) Donnez la relation de définition complète. (b) Quelle est la résolution ? (c) Donnez les alias de A et de AB. (d) Peut-on séparer les interactions AB et CE ?

**Exercice 11 ★★ (surface de réponse).** Un modèle du second ordre ajusté sur un plan composite centré à 2 facteurs ( $\alpha = \sqrt{2}$ ) est  $\hat{y} = 70 + 4x_1 + 2x_2 - 3x_1^2 - x_2^2 + x_1x_2$ . (a) Trouvez le point stationnaire et la réponse prédite. (b) Est-ce un maximum ? (c) Peut-on faire confiance à ce point ?

**Exercice 12 ★★★ (simuler l'effet des blocs).** On compare deux traitements avec 12 unités. L'effet vrai du traitement est de +15, l'écart-type du bruit de 10 et l'écart-type entre blocs (les semaines) de 30. Simulez 3 000 expériences et comparez la puissance (a) d'un plan **complètement randomisé** (12 unités issues de 12 blocs différents, 6 par traitement, analysées par un test de Student

à deux échantillons) et (b) d'un plan **en blocs** (6 blocs, chacun contenant une unité de chaque traitement, analysé par un test de Student apparié).

**Exercice 13 ★★ (méthode de Lenth).** Un plan  $2^3$  non répliqué donne les sept effets 12,0 ; -1,0 ; 0,5 ; 8,0 ; -0,8 ; 0,4 ; 0,6. Calculez  $s_0$  et le PSE de Lenth, et dites quels effets sont actifs (marge d'erreur  $ME = t_{0,975;7/3} \times \text{PSE}$ ).

## Corrigés

**Corrigé 1.** (a) L'unité expérimentale est **la semaine** (tous les visiteurs d'une même semaine voient le même affichage) : elle n'a ici que **deux unités**, une par traitement. (b) D'abord, l'affichage est **confondu avec la semaine** : si la semaine 1 contient une fête ou une promotion, on attribuera à A ce qui vient de la période ; ensuite, il n'y a **aucune répétition** : on ne peut pas estimer le bruit entre semaines, donc aucun test n'est possible. (c) Plan **en blocs** : prendre par exemple 8 semaines ; **dans chaque semaine** (le bloc), afficher A trois ou quatre jours et B les autres, avec un **tirage au sort** des jours ; si l'on peut, afficher A et B en même temps à des visiteurs tirés au hasard (l'unité devient alors le visiteur, plus fine, et la répétition immédiate). On compare A et B **à l'intérieur de chaque semaine** (test apparié), ce qui élimine l'effet de semaine, et on dispose de plusieurs répétitions pour estimer le bruit.

**Corrigé 2.** Moyennes : 50, 56, 62 ; moyenne générale 56.  $SS_B = 4[(50 - 56)^2 + 0 + (62 - 56)^2] = 4 \times 72 = 288$ . Dans chaque groupe, la somme des carrés des écarts vaut  $4 + 4 + 0 + 0 = 8$  (groupe 1),  $0 + 4 + 4 + 0 = 8$  (groupe 2),  $0 + 4 + 4 + 0 = 8$  (groupe 3) :  $SS_W = 24$ . Degrés de liberté :  $k - 1 = 2$  et  $N - k = 12 - 3 = 9$ .  $MS_B = 144$ ,  $MS_W = 24/9 \approx 2,667$  et  $F = 144/2,667 = 54$ . C'est très au-dessus du seuil de la loi  $\mathcal{F}(2, 9)$  (4,26 à 5 %, et  $p \approx 10^{-5}$ ) : les fournisseurs diffèrent nettement.

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

dat = {"F1": [52, 48, 50, 50], "F2": [56, 58, 54, 56], "F3": [62, 60, 64, 62]}
y = np.array(list(dat.values()), dtype=float)
k, n = y.shape
mg = y.mean()
SSB = n * ((y.mean(axis=1) - mg) ** 2).sum()
SSW = ((y - y.mean(axis=1, keepdims=True)) ** 2).sum()
MSB, MSW = SSB / (k - 1), SSW / (k * n - k)
print("moyennes :", y.mean(axis=1), "; moyenne générale :", mg)
print(f"SSB = {SSB:.0f}, SSW = {SSW:.0f}, MSB = {MSB:.0f}, MSW = {MSW:.3f}, F = {MSB / MSW:.1f}")
print(f"seuil F(2, 9) à 5 % = {stats.f.ppf(0.95, k - 1, k * n - k):.2f} ; p = {stats.f.sf(MSB / MSW, k - 1, k * n - k):.1e}")
```

```
moyennes : [50. 56. 62.] ; moyenne générale : 56.0
SSB = 288, SSW = 24, MSB = 144, MSW = 2.667, F = 54.0
seuil F(2, 9) à 5 % = 4.26 ; p = 9.7e-06
```

**Corrigé 3.**  $\eta^2 = SS_B/SS_T = 288/312 \approx 0,923$  : le fournisseur explique 92 % de la variance.  $\omega^2 = (SS_B - (k - 1)MS_W)/(SS_T + MS_W) = (288 - 2 \times 2,667)/(312 + 2,667) \approx 0,898$ .  $\omega^2 < \eta^2$  parce que  $\eta^2$  attribue au facteur une part du **bruit d'échantillonnage** (même sans effet réel,  $SS_B > 0$  en général) ;  $\omega^2$  retranche cette part attendue,  $(k - 1)MS_W$ , et est donc moins optimiste.

```
SST = SSB + SSW
print(f"eta² = {SSB / SST:.4f} ; omega² = {(SSB - (k - 1) * MSW) / (SST + MSW):.4f}")
long = pd.DataFrame({"fournisseur": np.repeat(list(dat), n), "resistance": y.ravel()})
mod = smf.ols("resistance ~ fournisseur", data=long).fit() # (la colonne de texte est traitée
↳ comme un facteur)
```

```
print(f"régression : F = {mod.fvalue:.1f}, R² = {mod.rsquared:.4f} (= eta²)")
```

```
eta² = 0.9231 ; omega² = 0.8983
régression : F = 54.0, R² = 0.9231 (= eta²)
```

**Corrigé 4.**  $MS_W = 2,667$ ,  $n = 4$  :  $\sqrt{MS_W/n} = \sqrt{0,667} \approx 0,816$ , donc  $HSD = 3,95 \times 0,816 \approx 3,22$  N. Les écarts de moyennes sont  $|56 - 50| = 6$ ,  $|62 - 56| = 6$  et  $|62 - 50| = 12$ , tous supérieurs à 3,22 : **les trois fournisseurs diffèrent deux à deux**, le troisième étant le plus résistant. Trois tests de Student à 5 % donneraient un risque global de fausse alerte proche de  $1 - 0,95^3 \approx 14\%$  ; Tukey contrôle ce risque à 5 % pour l'ensemble des comparaisons.

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
q = stats.studentized_range.ppf(0.95, k, k * n - k)
print(f"q = {q:.3f} ; HSD = {q * np.sqrt(MSW / n):.2f}")
res = pairwise_tukeyhsd(long["resistance"], long["fournisseur"])
print(pd.DataFrame(res._results_table.data[1:],
    columns=res._results_table.data[0]).round(3).to_string(index=False))
```

```
q = 3.948 ; HSD = 3.22
group1 group2 meandiff p-adj lower upper reject
    F1    F2      6.0  0.002  2.776  9.224   True
    F1    F3     12.0  0.000  8.776 15.224   True
    F2    F3      6.0  0.002  2.776  9.224   True
```

**Corrigé 5.** Moyennes des blocs : 13, 23,5, 33 ; des traitements : 20,33, 24, 22,0, 26,33 ; moyenne générale 23,17. (a)  $SS_{\text{blocs}} = 4 \sum (\bar{y}_j - \bar{y})^2$ ,  $SS_{\text{trait}} = 3 \sum (\bar{y}_i - \bar{y})^2$ , et  $SS_E$  par différence, avec 2, 3 et  $(2)(3) = 6$  degrés de liberté ; on obtient  $SS_{\text{blocs}} \approx 800,7$ ,  $SS_{\text{trait}} \approx 60,3$  et  $SS_E \approx 2,67$  (leur somme est la somme totale des carrés, vérifiez-le). (b) Avec les blocs :  $F = \frac{60,3/3}{2,67/6} \approx 45,3$ , très significatif ( $p = 0,0002$ ). Sans les blocs, le résidu absorbe la variabilité entre semaines :  $F = \frac{60,3/3}{(800,7 + 2,67)/8} \approx 0,20$  ( $p = 0,89$ ), aucun effet visible. (c) Les semaines diffèrent énormément (13, 23,5, 33), alors que les traitements diffèrent peu : le facteur « semaine » domine, et sans le bloc, l'effet des traitements est invisible.

```
Y = np.array([[10, 14, 12, 16], [20, 25, 22, 27], [31, 33, 32, 36]], dtype=float) # lignes = blocs
b, t = Y.shape
mg = Y.mean()
SSbl = t * ((Y.mean(axis=1) - mg) ** 2).sum()
SStr = b * ((Y.mean(axis=0) - mg) ** 2).sum()
SSe = ((Y - Y.mean(axis=1, keepdims=True) - Y.mean(axis=0, keepdims=True) + mg) ** 2).sum()
print(f"SS blocs = {SSbl:.1f} (ddl {b - 1}) ; SS traitements = {SStr:.1f} (ddl {t - 1}) ; SS erreur = {SSe:.2f} (ddl {(b - 1) * (t - 1)})")
F_avec = (SStr / (t - 1)) / (SSe / ((b - 1) * (t - 1)))
F_sans = (SStr / (t - 1)) / ((SSbl + SSe) / (b * t - t))
print(f"F traitements avec blocs = {F_avec:.2f} (p = {stats.f.sf(F_avec, t - 1, (b - 1) * (t - 1)):.4f})")
print(f"F traitements sans blocs = {F_sans:.2f} (p = {stats.f.sf(F_sans, t - 1, b * t - t):.4f})")
```

```
SS blocs = 800.7 (ddl 2) ; SS traitements = 60.3 (ddl 3) ; SS erreur = 2.67 (ddl 6)
F traitements avec blocs = 45.25 (p = 0.0002)
F traitements sans blocs = 0.20 (p = 0.8933)
```

**Corrigé 6.**  $\text{effet}(A) = \frac{(30+15)-(20+25)}{2} = 0$  ;  $\text{effet}(B) = \frac{(25+15)-(20+30)}{2} = -5$  ;  $AB = \frac{(15-25)-(30-20)}{2} = -10$ . L'effet principal de A est nul **parce que deux effets de signes opposés se compensent** : A fait **monter** la réponse de +10 quand B est bas ( $20 \rightarrow 30$ ) et la fait **baiss**er de 10 quand B est haut ( $25 \rightarrow 15$ ). A a donc une influence majeure, mais elle **dépend** de B : c'est exactement le piège de l'interaction qui annule un effet principal. Le meilleur réglage est  $(A+, B-)$  avec 30.

```
m = {"-","-": 20, ("+", "-"): 30, ("-", "+"): 25, ("+", "+"): 15}
A = ((m[("+", "-")] + m[("+", "+")]) - (m[("-","-")] + m[("-","+")])) / 2
B = ((m[("-","+")] + m[("+", "+")]) - (m[("-","-")] + m[("+", "-")])) / 2
AB = ((m[("+", "+")] - m[("-","+")]) - (m[("+", "-")] - m[("-","-")])) / 2
print("A =", A, "; B =", B, "; AB =", AB, "; meilleur réglage :", max(m, key=m.get))
```

```
A = 0.0 ; B = -5.0 ; AB = -10.0 ; meilleur réglage : ('+', '-')
```

**Corrigé 7.** Moyenne =  $116/8 = 14,5$ . Effets (contraste / 4) :  $A = (14 + 20 + 15 + 21 - 10 - 12 - 11 - 13)/4 = 24/4 = 6$  ;  $B = (12 + 20 + 13 + 21 - 10 - 14 - 11 - 15)/4 = 16/4 = 4$  ;  $C = (11 + 15 + 13 + 21 - 10 - 14 - 12 - 20)/4 = 4/4 = 1$  ;  $AB$  : signes + pour (1),  $ab, c, abc$  :  $(10 + 20 + 11 + 21 - 14 - 12 - 15 - 13)/4 = 8/4 = 2$  ;  $AC = BC = ABC = 0$ . Les effets non nuls sont donc  $A, B, C$  et  $AB$ , avec  $A > B > AB > C$ .

```
def yates(y):
    col = np.array(y, float)
    for _ in range(int(np.log2(len(y)))):
        col = np.concatenate([col[0::2] + col[1::2], col[1::2] - col[0::2]])
    return col
```

```
y = [10, 14, 12, 20, 11, 15, 13, 21]
contr = yates(y)
noms = ["I", "A", "B", "AB", "C", "AC", "BC", "ABC"]
print(dict(zip(noms, np.round(np.r_[contr[0] / 8, contr[1:] / 4], 2))))
```

```
{'I': np.float64(14.5), 'A': np.float64(6.0), 'B': np.float64(4.0), 'AB': np.float64(2.0), 'C':
 np.float64(1.0), 'AC': np.float64(0.0), 'BC': np.float64(0.0), 'ABC': np.float64(0.0)}
```

**Corrigé 8.** (a)  $N = 8r$ , donc l'écart-type d'un effet vaut  $ET = 2\sigma/\sqrt{8r} = 6/\sqrt{8r}$ . (b) Avec l'approximation normale, la puissance de 80 % exige  $\Delta/ET \approx 1,96 + 0,84 = 2,8$ , soit  $ET \leq 4/2,8 = 1,43$  et  $8r \geq (6/1,43)^2 \approx 17,6$ , donc  $r \geq 2,2$  : environ **3 répétitions**. (c) Avec la loi de Student, qui a peu de degrés de liberté quand  $r$  est petit (8 pour  $r = 2$ ), il faut un peu plus de marge. Le calcul exact confirme ce qu'annonce l'approximation :

```
sigma, delta = 3.0, 4.0
for r in (2, 3, 4):
    N = 8 * r
    ddl = N - 8
    se = 2 * sigma / np.sqrt(N)
    seuil = stats.t.ppf(0.975, ddl)
    puissance = stats.nct.sf(seuil, ddl, delta / se) + stats.nct.cdf(-seuil, ddl, delta / se)
    print(f"r = {r} : N = {N}, ET = {se:.3f}, ddl = {ddl}, puissance = {puissance:.3f}")
```

```
r = 2 : N = 16, ET = 1.500, ddl = 8, puissance = 0.648
r = 3 : N = 24, ET = 1.225, ddl = 16, puissance = 0.865
r = 4 : N = 32, ET = 1.061, ddl = 24, puissance = 0.951
```

La puissance est de 65 % pour  $r = 2$ , de 86,5 % pour  $r = 3$  et de 95 % pour  $r = 4$  : le seuil de 80 % est atteint à partir de  $r = 3$  (24 essais), comme l'annonçait l'approximation normale. La loi de Student, qui tient compte du petit nombre de degrés de liberté de l'erreur pure, rend le calcul exact un peu plus exigeant pour les petites valeurs de  $r$ .

**Corrigé 9.** (a)  $D = ABC \Rightarrow I = ABCD$ . Un seul mot de 4 lettres : **résolution IV**.  $AB = AB \cdot ABCD = CD$ . (b)  $D = AB \Rightarrow I = ABD$  (mot de 3 lettres) : **résolution III** ;  $AB = AB \cdot ABD = D$  : l'interaction  $AB$  est confondue avec le **facteur principal D**, ce qui est le pire cas ; de plus,  $A = BD$ ,  $B = AD$ . (c) Le premier : à nombre d'essais égal, la résolution IV garantit que les effets principaux ne sont confondus qu'avec des interactions d'ordre 3 ; la résolution III les confond avec des interactions d'ordre 2, souvent non négligeables.



```
def produit(u, v):
    return "".join(sorted(set(u) ^ set(v)))
for gen, mots in [("D = ABC", ["ABCD"]), ("D = AB", ["ABD"])]:
    print(f"{gen} : I = {' '.join(mots)} ; résolution {min(len(w) for w in mots)} ; "
          f"AB = {' '.join(produit('AB', w) for w in mots)} ; A = {' '.join(produit('A', w) for w
          ↪ in mots)}")
```

D = ABC : I = ABCD ; résolution 4 ; AB = CD ; A = BCD

D = AB : I = ABD ; résolution 3 ; AB = D ; A = BD

**Corrigé 10.** (a) Les mots générateurs :  $E = ABC \Rightarrow I = ABCE$  ;  $F = BCD \Rightarrow I = BCDF$ . Leur produit est aussi un mot :  $ABCE \cdot BCDF = ADEF$  (les lettres B et C, communes, s'éliminent). Relation complète :  $I = ABCE = BCDF = ADEF$ . (b) Mots de longueurs 4, 4 et 4 : **résolution IV**. (c)  $A = A \cdot ABCE = BCE$  ;  $A \cdot BCDF = ABCDF$  ;  $A \cdot ADEF = DEF$  :  $A = BCE = ABCDF = DEF$  (effets d'ordre 3 ou plus : **A est propre**).  $AB = CE = ACDF = BDEF$ . (d) Non : AB est confondue avec CE (elle l'est par le mot ABCE) : en résolution IV, certaines interactions d'ordre 2 sont confondues entre elles ; pour les séparer, il faudrait un plan de résolution V (plus d'essais) ou un repliement bien choisi.

```
mots = ["ABCE", "BCDF", produit("ABCE", "BCDF")]
print("relation de définition : I = " + " ".join(mots))
for e in ["A", "AB"]:
    print(f"{e} = " + " ".join(produit(e, w) for w in mots))
```

relation de définition : I = ABCE = BCDF = ADEF

A = BCE = ABCDF = DEF

AB = CE = ACDF = BDEF

**Corrigé 11.** (a)  $b = (4, 2)^\top$ ,  $B = \begin{pmatrix} -3 & 0,5 \\ 0,5 & -1 \end{pmatrix}$  (le terme croisé  $1 \cdot x_1 x_2$  se répartit en  $0,5 + 0,5$ ).

$\det B = 3 - 0,25 = 2,75$  et  $B^{-1} = \frac{1}{2,75} \begin{pmatrix} -1 & -0,5 \\ -0,5 & -3 \end{pmatrix}$ , donc  $B^{-1}b = \frac{1}{2,75}(-5, -8)^\top$  et  $x_s = -\frac{1}{2}B^{-1}b \approx$

$(0,909; 1,455)$ . La réponse prédite est  $\hat{y}_s = 70 + \frac{1}{2}b^\top x_s = 70 + \frac{1}{2}(4 \times 0,909 + 2 \times 1,455) \approx 73,27$ . (b)

La trace de B vaut  $-4$  et son déterminant  $2,75 > 0$  : les valeurs propres sont  $\frac{-4 \pm \sqrt{16-11}}{2} \approx -0,88$  et  $-3,12$ , **toutes deux négatives** : c'est un **maximum**. (c) La distance au centre est  $\sqrt{0,909^2 + 1,455^2} \approx 1,72$ , **supérieure** au rayon  $\sqrt{2} \approx 1,41$  des points axiaux : le sommet est **en dehors du domaine expérimental**. C'est une extrapolation : on ne doit pas lui faire confiance, mais déplacer le plan dans cette direction et recommencer.

```
b = np.array([4.0, 2.0])
B = np.array([[-3.0, 0.5], [0.5, -1.0]])
xs = -0.5 * np.linalg.solve(B, b)
print("point stationnaire :", xs.round(3), "; réponse :", round(70 + 0.5 * b @ xs, 2))
print("valeurs propres de B :", np.linalg.eigvalsh(B).round(2))
print(f"distance au centre = {np.linalg.norm(xs):.2f} ; rayon du domaine (points axiaux) =
↪ {np.sqrt(2):.2f}")
```

point stationnaire : [0.909 1.455] ; réponse : 73.27

valeurs propres de B : [-3.12 -0.88]

distance au centre = 1.72 ; rayon du domaine (points axiaux) = 1.41

**Corrigé 12.** Dans un plan complètement randomisé, chaque unité vient d'un bloc différent : la variabilité entre blocs (écart-type 30) s'ajoute au bruit (10), soit un écart-type de  $\sqrt{30^2 + 10^2} \approx 31,6$  par unité. L'écart-type de la différence de deux moyennes de 6 unités vaut  $31,6\sqrt{2/6} \approx 18,3$ , pour un effet de 15 : le rapport signal/bruit est d'environ 0,8 et le test est peu puissant. Dans le plan en blocs, la **différence** entre les deux traitements au sein d'un même bloc élimine l'effet



de bloc : l'écart-type d'une différence est de  $10\sqrt{2} \approx 14,1$ , celui de la moyenne des 6 différences  $14,1/\sqrt{6} \approx 5,8$ , soit un rapport signal/bruit de 2,6 : bien meilleur. La simulation le chiffre.

```
rng = np.random.default_rng(90)
effet, sig, sig_bloc, n_par_trait, n_sim = 15.0, 10.0, 30.0, 6, 3000
rej_crd, rej_blocs = 0, 0
for _ in range(n_sim):
    # (a) plan complètement randomisé : 12 unités, chacune avec SON propre effet de bloc (indépendants)
    y1 = rng.normal(0, sig_bloc, n_par_trait) + rng.normal(0, sig, n_par_trait)
    y2 = effet + rng.normal(0, sig_bloc, n_par_trait) + rng.normal(0, sig, n_par_trait)
    rej_crd += stats.ttest_ind(y2, y1).pvalue < 0.05
    # (b) plan en blocs : 6 blocs, une unité de chaque traitement par bloc (l'effet de bloc est PARTAGÉ)
    blocs = rng.normal(0, sig_bloc, n_par_trait)
    z1 = blocs + rng.normal(0, sig, n_par_trait)
    z2 = blocs + effet + rng.normal(0, sig, n_par_trait)
    rej_blocs += stats.ttest_rel(z2, z1).pvalue < 0.05
print(f"puissance, plan complètement randomisé (Student, 6 contre 6) : {rej_crd / n_sim:.3f}")
print(f"puissance, plan en blocs (Student apparié, 6 blocs) : {rej_blocs / n_sim:.3f}")

puissance, plan complètement randomisé (Student, 6 contre 6) : 0.112
puissance, plan en blocs (Student apparié, 6 blocs) : 0.552
```

Les deux plans utilisent **le même nombre d'unités** (12) et le même effet, mais le plan en blocs le détecte dans environ 55 % des expériences, contre environ 11 % pour le plan complètement randomisé : un facteur 5 de puissance obtenu **sans une unité de plus**, simplement en organisant l'expérience. (Un piège à éviter : si l'on appliquait un test de Student à deux échantillons à des données **appariées par un bloc partagé**, on traiterait comme indépendantes des mesures corrélées ; le test deviendrait trop conservateur, avec une puissance inférieure même à son seuil de 5 %. C'est une erreur d'analyse, pas un plan complètement randomisé.)

**Corrigé 13.** Valeurs absolues triées : 0,4; 0,5; 0,6; 0,8; 1,0; 8,0; 12,0 ; médiane = 0,8, donc  $s_0 = 1,5 \times 0,8 = 1,2$  et le seuil  $2,5 s_0 = 3,0$ . On ne garde que les  $|c| < 3$  : 0,4; 0,5; 0,6; 0,8; 1,0, de médiane 0,6 :  $PSE = 1,5 \times 0,6 = 0,9$ . Avec  $d = 7/3 \approx 2,33$  degrés de liberté,  $t_{0,975} \approx 3,76$  (très grand, faute de degrés de liberté) :  $ME \approx 3,39$ . Les effets 12 et 8 dépassent largement la marge ; les cinq autres n'en approchent pas : **deux effets actifs**.

```
c = np.array([12.0, -1.0, 0.5, 8.0, -0.8, 0.4, 0.6])
a = np.abs(c)
s0 = 1.5 * np.median(a)
pse = 1.5 * np.median(a[a < 2.5 * s0])
d = len(c) / 3
ME = stats.t.ppf(0.975, d) * pse
print(f"s0 = {s0:.2f} ; PSE = {pse:.2f} ; d = {d:.2f} ; t = {stats.t.ppf(0.975, d):.2f} ; ME = {ME:.2f}")
print("effets actifs (|c| > ME) :", c[a > ME])

s0 = 1.20 ; PSE = 0.90 ; d = 2.33 ; t = 3.76 ; ME = 3.39
effets actifs (|c| > ME) : [12.  8.]
```

## Bilan du chapitre 8

Vous savez maintenant :

- **concevoir** une expérience : identifier les facteurs, les niveaux, l'**unité expérimentale** et la réponse ; appliquer la **randomisation** (contre la confusion), la **répétition** (contre le bruit, sans pseudo-réplication) et le **blocage** (contre la variabilité connue) ;

- expliquer pourquoi **changer un facteur à la fois** est moins précis et **aveugle aux interactions** ;
- mener et **démontrer** une **ANOVA** : décomposition  $SS_T = SS_B + SS_W$ , test  $F$ , lien avec le test de Student et la **régression**, vérification des hypothèses, **Tukey** et **contrastes**, **tailles d'effet** ( $\eta^2, \omega^2, f$ ) ;
- analyser un plan **en blocs** et un plan à **deux facteurs** avec **interaction** (lire d'abord le graphique d'interaction, étudier les effets simples) ;
- calculer la **puissance** d'une ANOVA ou d'un plan factoriel *avant* l'expérience, et dimensionner le nombre de répétitions ;
- construire un **plan factoriel**  $2^k$  et calculer ses **effets à la main** (contrastes, algorithme de Yates), les relier aux coefficients de la régression, et repérer les effets actifs d'un plan non répliqué (diagramme demi-normal, **méthode de Lenth**) ;
- construire un **plan fractionnaire**, déterminer ses **alias** et sa **résolution**, repérer le danger de la confusion et le lever par un **repliement** ;
- optimiser un réglage par la **méthodologie des surfaces de réponse** : test de courbure, **plan composite centré**, modèle quadratique, test de défaut d'ajustement, **point stationnaire** et analyse canonique, essais de confirmation ;
- (en option) situer les **plans optimaux** ( $D$ -optimalité) pour les domaines contraints.

Ce chapitre a montré qu'un bon plan **simplifie l'analyse** et qu'il détermine, avant la première mesure, ce que l'on pourra conclure. Le chapitre 7 (inférence causale) aborde le problème inverse : que conclure quand on **n'a pas pu** randomiser, comme dans la plupart des données observationnelles de ce livre.

# Chapitre 9 : ⊕ Statistique spatiale

« Tout est lié à tout le reste, mais ce qui est proche est plus lié que ce qui est lointain. »  
— Waldo Tobler, *première loi de la géographie* (1970)

**Chapitre complémentaire.** Ce chapitre est **entièrement facultatif** : rien dans les chapitres 1 à 6 ni dans le projet du volume n'en dépend. Il s'adresse à ceux dont les données ont une **position** : des adresses de clients, des points de livraison, des zones géographiques, des capteurs. Si c'est votre cas, vous verrez que presque tout ce que nous avons appris repose sur une hypothèse que l'espace met à mal : l'**indépendance** des observations.

Dar Jasmin livre maintenant dans tout le nord du pays. Yasmine remarque des choses que les tableaux des chapitres précédents ne savent pas dire :

- « *Les délais de livraison longs sont groupés : quand un client de Zaghouan attend longtemps, ses voisins aussi.* » Un point isolé ne serait pas un problème ; un **paquet** de retards, c'est un problème de tournée ou de route.
- « *Mes ventes par délégation ont l'air de former des îlots : des zones où l'on vend beaucoup, entourées de zones où l'on vend beaucoup.* » Est-ce réel, ou est-ce l'œil qui voit des formes dans le hasard ?
- « *Je voudrais promettre un délai au client avant même d'avoir livré dans son quartier.* » Comment **prédire** une valeur en un endroit où l'on n'a rien mesuré ?
- « *Mes clients de la médina se regroupent-ils vraiment autour de la boutique, ou sont-ils répartis au hasard ?* »

Quatre questions, quatre outils : l'**autocorrélation spatiale**, le **variogramme** et le **krigeage**, les **processus ponctuels**. Les voici, dans l'ordre.

## Le chemin de ce chapitre

- **9.1 Données spatiales et cartes** : les trois grands types de données spatiales, la façon de repérer un point sur la Terre, de calculer une distance **sans se tromper**, et de dessiner une carte honnête sans fond de carte.
- **9.2 Autocorrélation spatiale** : définir « voisin » avec une **matrice de poids**, mesurer la ressemblance entre voisins avec l'**indice de Moran** (démonstré, calculé à la main, puis testé par permutations), l'indice de **Geary**, et repérer où se trouvent les îlots avec les indices **locaux** (LISA).
- **9.3 Variogramme et krigeage** : décrire comment la ressemblance **décroît avec la distance**, ajuster un modèle (pépite, palier, portée), et **prédire** en un point non mesuré avec une **variance d'erreur** (le krigeage, dérivé avec des multiplicateurs de Lagrange).
- **9.4 Processus ponctuels** : quand ce sont les **positions** qui sont aléatoires ; hasard complet, agrégat ou répulsion ? Test des quadrats, plus proche voisin, fonction K de Ripley, enveloppes de Monte-Carlo.

- **9.5 Exercices corrigés**, puis le bilan du chapitre.

**Comment travailler avec ce chapitre.** Aucune bibliothèque de cartographie n'est nécessaire (ni geopandas, ni pykrige) : tout est écrit avec NumPy, SciPy et matplotlib. C'est volontaire. Calculer soi-même un indice de Moran ou résoudre soi-même un système de krigeage est la meilleure garantie de comprendre ce que font ensuite les bibliothèques spécialisées. En contrepartie, les cartes n'ont **pas de fond de carte** (il faudrait télécharger des tuiles) : ce sont des graphiques de coordonnées, sobres mais honnêtes.

**Les données de ce chapitre.** Les données sont **simulées**, avec des graines fixes, de façon que nous connaissions la vérité et puissions vérifier que les méthodes la retrouvent.

- Une **grille de 12 × 12 « délégations » fictives** (144 zones de 5 km de côté) avec des ventes par habitant qui présentent une autocorrélation spatiale *connue* (fichier `donnees/ch09-delegations.csv`), plus une variable de contrôle sans aucune structure spatiale.
- **200 livraisons** dans un carré de 100 km de côté, avec un délai en jours qui varie de façon continue dans l'espace (fichier `donnees/ch09-livraisons.csv`).
- Des **semis de points** (adresses de clients), simulés directement dans les sections qui les utilisent.
- Les **coordonnées géographiques de dix villes tunisiennes**, qui sont des valeurs **approximatives** écrites à la main et arrondies au centième de degré (à vérifier avant tout usage cartographique réel), et le tableau `donnees/clients.csv` du volume pour les effectifs par ville.

Le générateur du chapitre est dans `build/donnees_ch09.py` ; son code est celui qui est imprimé aux sections 9.2 et 9.3.

## 9.1 Données spatiales et cartes

**Intuition.** Dans un tableau ordinaire, l'ordre des lignes n'a aucune importance : on peut les mélanger sans rien changer à la moyenne, à la régression ou au test. Dans des données spatiales, où se trouve chaque ligne fait partie de l'information. Deux clients voisins se ressemblent plus que deux clients éloignés, et c'est précisément ce qui casse l'hypothèse d'**indépendance** sur laquelle reposent presque tous les outils du volume I et des chapitres 1 et 2 de ce volume.

### 9.1.1 Pourquoi l'espace change tout

Un petit exemple pour sentir le problème. Yasmine a mesuré le délai de quatre livraisons : deux à Bizerte (notées A et B, à quelques rues l'une de l'autre) et deux à Sfax (C et D). Les délais sont 6, 7, 3 et 4 jours.

La moyenne est  $(6 + 7 + 3 + 4)/4 = 5$  jours, et l'écart-type est d'environ 1,83 jour. Si les quatre livraisons étaient **indépendantes**, l'écart-type de la moyenne serait  $s/\sqrt{n} \approx 0,91$  jour : la formule du volume I (section 2.4). Mais A et B ont subi la même route embouteillée, C et D la même autoroute fluide. En réalité, nous n'avons pas **quatre** informations indépendantes, mais plutôt **deux** (une par ville). Notre moyenne est beaucoup moins précise que ne le dit la formule.

**Le danger concret.** Avec des observations corrélées dans l'espace, les erreurs-types « habituelles » sont **trop petites**, les intervalles de confiance trop étroits et les p-valeurs trop optimistes. Nous aurons l'impression d'avoir 200 observations alors que nous en avons, au sens

de l'information, bien moins. C'est la raison pour laquelle il faut **mesurer** la dépendance spatiale avant de lui faire confiance, et c'est l'objet de tout ce chapitre.

**La loi de Tobler en une phrase.** « Tout est lié à tout le reste, mais ce qui est proche est plus lié que ce qui est lointain. » Ce n'est pas un théorème, c'est une observation qui se vérifie pour les prix de l'immobilier, la température, les délais de livraison, les maladies contagieuses... et qui justifie tous les outils qui suivent : ils mesurent à *quelle vitesse* la ressemblance s'estompe avec la distance.

### 9.1.2 Trois types de données spatiales

Selon ce qui est aléatoire et ce qui est fixe, on distingue trois grandes familles. Savoir dans quelle famille on se trouve est la **première** question à se poser, car elle détermine les outils.

| Type                                          | Ce qu'on observe                                                     | Ce qui est fixe / aléatoire                             | Exemple Dar Jasmin                                                                | Outils (section)                                |
|-----------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------|
| <b>Géostatistique</b>                         | une valeur en des <b>points</b> d'un phénomène défini <i>partout</i> | les lieux sont choisis (fixes), la valeur est aléatoire | le délai de livraison, mesuré à 200 adresses, qui existe en tout point de la zone | variogramme, krigeage (9.3)                     |
| <b>Données surfaciques</b> ( <i>lattice</i> ) | une valeur par <b>zone</b> d'un découpage                            | le découpage est fixe, la valeur est aléatoire          | les ventes par habitant de chaque délégation                                      | matrice de poids, Moran, Geary, LISA (9.2)      |
| <b>Semis de points</b>                        | les <b>positions</b> elles-mêmes                                     | les lieux sont <b>aléatoires</b>                        | les adresses des clients de la médina                                             | quadrats, plus proche voisin, K de Ripley (9.4) |

La frontière n'est pas étanche : les mêmes ventes peuvent être vues par zone (surfacique) ou, si l'on connaît les adresses, comme un semis de points avec une « marque » (le montant). Mais le **choix du modèle** suit toujours cette question : *qu'est-ce qui est aléatoire ici ?*

**Un test pratique.** Imaginez que l'on refasse l'expérience. Si les **valeurs** changent mais pas les lieux (le même 12 × 12 de délégations, d'autres ventes), c'est de la géostatistique ou du surfacique. Si ce sont les **lieux** qui changeraient (d'autres adresses de clients), c'est un semis de points.

### 9.1.3 Repérer un point : coordonnées et distances

Un point sur Terre se repère par sa **latitude**  $\varphi$  (de  $-90^\circ$  au pôle Sud à  $+90^\circ$  au pôle Nord) et sa **longitude**  $\lambda$  (de  $-180^\circ$  à  $+180^\circ$  à partir du méridien de Greenwich). Voici dix villes tunisiennes, avec des coordonnées **approximatives**, arrondies au centième de degré (nous les avons saisies à la main : elles suffisent pour apprendre, mais à vérifier pour tout usage réel).

```
import numpy as np
import pandas as pd

villes = pd.DataFrame({
    "ville": ["Tunis", "Bizerte", "Nabeul", "Sousse", "Kairouan", "Sfax", "Gafsa", "Gabès", "Djerba",
             "Tozeur"],
    "lat": [36.81, 37.27, 36.46, 35.83, 35.68, 34.74, 34.43, 33.88, 33.88, 33.92],
    "lon": [10.18, 9.87, 10.74, 10.61, 10.10, 10.76, 8.78, 10.10, 10.86, 8.13],
})
```

```
)
print(villes.to_string(index=False))
```

```
ville  lat  lon
Tunis  36.81 10.18
Bizerte 37.27 9.87
Nabeul  36.46 10.74
Sousse  35.83 10.61
Kairouan 35.68 10.10
Sfax    34.74 10.76
Gafsa   34.43 8.78
Gabès   33.88 10.10
Djerba  33.88 10.86
Tozeur  33.92 8.13
```

**Piège n° 1 : un degré n'est pas une distance fixe.** Un degré de latitude vaut toujours à peu près 111 km (la Terre est presque une sphère de rayon  $R \approx 6371$  km, et  $1^\circ = \pi/180$  radian, donc  $R\pi/180 \approx 111,2$  km). Mais les méridiens se **rapprochent** quand on monte vers le pôle : un degré de longitude vaut  $111,2 \times \cos \varphi$  km. À la latitude de Tunis (environ  $37^\circ$ ),  $\cos 37^\circ \approx 0,80$  : un degré de longitude ne vaut que **89 km**.

Faisons un calcul à la main pour Tunis (36,81 ; 10,18) et Bizerte (37,27 ; 9,87) :

- écart de latitude :  $0,46^\circ \times 111,2 \approx 51,2$  km vers le nord ;
- écart de longitude :  $0,31^\circ \times 111,2 \times \cos(37,0^\circ) \approx 27,5$  km vers l'ouest (avec  $\cos 37,0^\circ \approx 0,80$ ) ;
- distance  $\approx \sqrt{51,2^2 + 27,5^2} \approx 58,1$  km.

Si l'on avait oublié le  $\cos \varphi$  et traité les degrés comme des unités égales, on aurait trouvé  $111,2 \times \sqrt{0,46^2 + 0,31^2} \approx 61,7$  km : 6 % d'erreur, sur une distance courte. Et plus le trajet est orienté est-ouest, plus l'erreur grandit (nous le mesurons plus bas).

**La formule de haversine (distance sur la sphère).** La plus courte distance entre deux points d'une sphère de rayon  $R$  est un arc de **grand cercle**. Pour deux points  $(\varphi_1, \lambda_1)$  et  $(\varphi_2, \lambda_2)$  (en radians) :

$$a = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos \varphi_1 \cos \varphi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right), \quad d = 2R \arcsin \sqrt{a}.$$

**Contrôle de bon sens.** Si  $\lambda_1 = \lambda_2$  (même méridien), alors  $a = \sin^2((\varphi_2 - \varphi_1)/2)$  et  $d = 2R \arcsin |\sin((\varphi_2 - \varphi_1)/2)| = R |\varphi_2 - \varphi_1|$  : c'est bien la longueur d'un arc de cercle de rayon  $R$  et d'angle  $|\Delta\varphi|$ . Pour une différence d'un degré, on retrouve  $R\pi/180 \approx 111,2$  km. Le terme  $\cos \varphi_1 \cos \varphi_2$  est exactement le facteur de raccourcissement des méridiens.

```
R = 6371.0 # rayon moyen de la Terre, en km
```

```
def haversine(lat1, lon1, lat2, lon2):
    """Distance de grand cercle en km (arguments en degrés ; NumPy diffuse sur les tableaux)."""
    p1, p2 = np.radians(lat1), np.radians(lat2)
    dphi = p2 - p1
    dlam = np.radians(lon2) - np.radians(lon1)
    a = np.sin(dphi / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dlam / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))
```

```
# Contrôle : un degré le long d'un méridien
```

```
print("1 degré de latitude :", round(float(haversine(36.0, 10.0, 37.0, 10.0)), 1), "km")
print("1 degré de longitude à 36° N :", round(float(haversine(36.0, 10.0, 36.0, 11.0)), 1), "km")
↪ (111,2 x cos 36° =", round(111.19 * np.cos(np.radians(36)), 1), ")")
print("Tunis - Bizerte :", round(float(haversine(36.81, 10.18, 37.27, 9.87)), 1), "km")
```

```
1 degré de latitude : 111.2 km
1 degré de longitude à 36° N : 90.0 km (111,2 x cos 36° = 90.0 )
Tunis - Bizerte : 58.1 km
```

Le calcul à la main (58,1 km) et la formule donnent la même chose. Calculons maintenant la **matrice des distances** entre toutes les villes. Tout le chapitre utilisera des matrices de distances : autant apprendre la technique tout de suite. Le principe est de transformer les vecteurs en colonnes et en lignes pour que NumPy calcule les 10×10 paires d'un coup (la diffusion du chapitre 4.4 du volume I).

```
lat, lon = villes["lat"].to_numpy(), villes["lon"].to_numpy()
D = haversine(lat[:, None], lon[:, None], lat[None, :], lon[None, :]) # (10, 10)
court = ["Tun", "Biz", "Nab", "Sou", "Kai", "Sfa", "Gaf", "Gab", "Dje", "Toz"]
print(pd.DataFrame(D.round(0).astype(int), index=court, columns=court).to_string())
print()
print("matrice symétrique :", np.allclose(D, D.T), "| diagonale nulle :", np.allclose(np.diag(D), 0))
```

|     | Tun | Biz | Nab | Sou | Kai | Sfa | Gaf | Gab | Dje | Toz |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Tun | 0   | 58  | 63  | 116 | 126 | 236 | 293 | 326 | 332 | 371 |
| Biz | 58  | 0   | 119 | 173 | 178 | 292 | 331 | 378 | 387 | 404 |
| Nab | 63  | 119 | 0   | 71  | 104 | 191 | 287 | 293 | 287 | 369 |
| Sou | 116 | 173 | 71  | 0   | 49  | 122 | 228 | 222 | 218 | 310 |
| Kai | 126 | 178 | 104 | 49  | 0   | 121 | 184 | 200 | 212 | 266 |
| Sfa | 236 | 292 | 191 | 122 | 121 | 0   | 185 | 113 | 96  | 258 |
| Gaf | 293 | 331 | 287 | 228 | 184 | 185 | 0   | 136 | 201 | 82  |
| Gab | 326 | 378 | 293 | 222 | 200 | 113 | 136 | 0   | 70  | 182 |
| Dje | 332 | 387 | 287 | 218 | 212 | 96  | 201 | 70  | 0   | 252 |
| Toz | 371 | 404 | 369 | 310 | 266 | 258 | 82  | 182 | 252 | 0   |

```
matrice symétrique : True | diagonale nulle : True
```

Sur quelques paires, mesurons l'erreur que l'on commet avec deux raccourcis tentants : (a) traiter les degrés comme des unités égales (111,2× la distance euclidienne en degrés), (b) projeter à plat avec la **projection équirectangulaire** centrée sur la latitude moyenne  $\varphi_0$ ,  $x = R(\lambda - \lambda_0) \cos \varphi_0$  et  $y = R(\varphi - \varphi_0)$ , puis prendre la distance euclidienne.

```
def naif(lat1, lon1, lat2, lon2):
    return 111.19 * np.hypot(lat2 - lat1, lon2 - lon1)

def equirect(lat1, lon1, lat2, lon2, lat0):
    c = np.cos(np.radians(lat0))
    return R * np.radians(1) * np.hypot(lat2 - lat1, (lon2 - lon1) * c)

lat0 = villes["lat"].mean()
paires = [("Tunis", "Bizerte"), ("Tunis", "Sfax"), ("Tunis", "Tozeur"), ("Bizerte", "Djerba"),
          ("Tozeur", "Djerba")]
lignes = []
for a, b in paires:
    ra, rb = villes.set_index("ville").loc[a], villes.set_index("ville").loc[b]
    vrai = float(haversine(ra.lat, ra.lon, rb.lat, rb.lon))
    n = float(naif(ra.lat, ra.lon, rb.lat, rb.lon))
    e = float(equirect(ra.lat, ra.lon, rb.lat, rb.lon, lat0))
    lignes.append({"paire": f"{a}-{b}", "haversine_km": vrai, "degres_bruts_km": n, "erreur_%": 100 *
                  (n / vrai - 1),
                  "equirect_km": e, "erreur_eq_%": 100 * (e / vrai - 1)})
print(pd.DataFrame(lignes).round(1).to_string(index=False))
```

| paire          | haversine_km | degres_bruts_km | erreur_% | equirect_km | erreur_eq_% |
|----------------|--------------|-----------------|----------|-------------|-------------|
| Tunis-Bizerte  | 58.1         | 61.7            | 6.2      | 58.4        | 0.5         |
| Tunis-Sfax     | 236.0        | 239.0           | 1.3      | 236.1       | 0.0         |
| Tunis-Tozeur   | 371.2        | 394.0           | 6.1      | 371.3       | 0.0         |
| Bizerte-Djerba | 387.4        | 392.7           | 1.4      | 387.5       | 0.0         |



|               |       |       |      |       |      |
|---------------|-------|-------|------|-------|------|
| Tozeur-Djerba | 252.0 | 303.6 | 20.5 | 247.8 | -1.7 |
|---------------|-------|-------|------|-------|------|

Le raccourci « degrés bruts » **surestime** toujours (puisque  $\cos \varphi < 1$ ) : l'erreur va de 1 % à 20 % selon la direction du trajet. Elle est la plus forte pour Tozeur–Djerba, un trajet presque exactement est-ouest (la différence de latitude est minime, celle de longitude est de plus de  $2,7^\circ$ ), et la plus faible pour Tunis–Sfax et Bizerte–Djerba, qui sont surtout nord-sud. La projection équirectangulaire, elle, reste à moins de 2 % d'erreur sur toutes ces paires, ce qui est excellent pour une région de la taille de la Tunisie. C'est pour cela que, dans la pratique, on **projette** les coordonnées dans un système plan local, mesuré en mètres ou en kilomètres (le système UTM, par exemple, fournit des zones de  $6^\circ$  de longitude, et la zone 32 couvre l'essentiel de la Tunisie), puis on travaille avec des distances euclidiennes ordinaires. Pour la suite de ce chapitre, nos zones font 100 km de côté : nous utiliserons directement des coordonnées planes **en kilomètres**.

**À vol d'oiseau n'est pas « en camion ».** Les distances calculées ici sont des distances **à vol d'oiseau**. Entre deux villes, la distance **routière** est plus longue (parfois 30 % de plus), et le temps de trajet dépend de la route. Pour des questions de logistique, la bonne « distance » entre deux points est souvent un temps de parcours, qui n'est pas forcément symétrique ni euclidienne. Les outils de ce chapitre fonctionnent avec n'importe quelle distance... à condition que ses propriétés conviennent (nous y reviendrons aux sections 9.2 et 9.3).

#### 9.1.4 Une carte sans fond de carte

Sans bibliothèque de cartographie, une carte est simplement un nuage de points  $(x, y)$  avec **un rapport d'aspect correct** (un kilomètre vers l'est doit occuper la même longueur à l'écran qu'un kilomètre vers le nord). Pour des coordonnées en degrés, on y parvient en réglant le rapport d'aspect à  $1 / \cos \varphi_0$ , ce qui revient à la projection équirectangulaire. Voyons-le avec les villes de Dar Jasmin, en dessinant un **symbole proportionnel** : la surface du disque est proportionnelle au nombre de clients de la ville (le fichier `clients.csv`, qui vit dans l'univers simulé du volume).

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

clients = pd.read_csv("donnees/clients.csv")
par_ville = clients.groupby("ville").agg(clients=("id_client", "size"), depense=("depense_annuelle",
↪ "mean")).round(1)
print(par_ville.to_string())

carte = villes.merge(par_ville, left_on="ville", right_index=True, how="left") # « Autre » n'a pas
↪ de position
BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"

fig, axes = plt.subplots(1, 2, figsize=(10.5, 5.2))
for ax, titre, corrige in zip(axes, ["Degrés traités comme des unités égales", "Aspect corrigé (1 /
↪ cos de la latitude)"], [False, True]):
    ax.scatter(carte["lon"], carte["lat"], s=18, color=ORANGE, zorder=3)
    ax.scatter(carte["lon"], carte["lat"], s=carte["clients"].fillna(0) * 1.5, color=BLEU,
↪ alpha=0.35, zorder=2)
    for _, r in carte.iterrows():
        decal, ha = ((-6, -11), "right") if r["ville"] == "Kairouan" else ((5, 4), "left") # évite
↪ le disque de Sousse
        ax.annotate(r["ville"], (r["lon"], r["lat"]), xytext=decal, ha=ha, textcoords="offset points",
↪ fontsize=8)
    ax.set_title(titre, fontsize=10)
    ax.set_xlabel("longitude (°E)")
    ax.set_xlim(7.5, 11.9)
    ax.set_ylim(33.5, 37.6)
```

```
ax.set_aspect(1 / np.cos(np.radians(lat0)) if corrige else 1.0) # 1.0 : un degré = un degré,
↳ dans les deux sens
axes[0].set_ylabel("latitude (°N)")
plt.tight_layout()
plt.savefig("figures/ch09-carte-villes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

|         | clients | depense |
|---------|---------|---------|
| ville   |         |         |
| Autre   | 301     | 274.0   |
| Bizerte | 208     | 217.9   |
| Nabeul  | 252     | 243.6   |
| Sfax    | 282     | 245.6   |
| Sousse  | 337     | 247.0   |
| Tunis   | 620     | 245.5   |

figure enregistrée

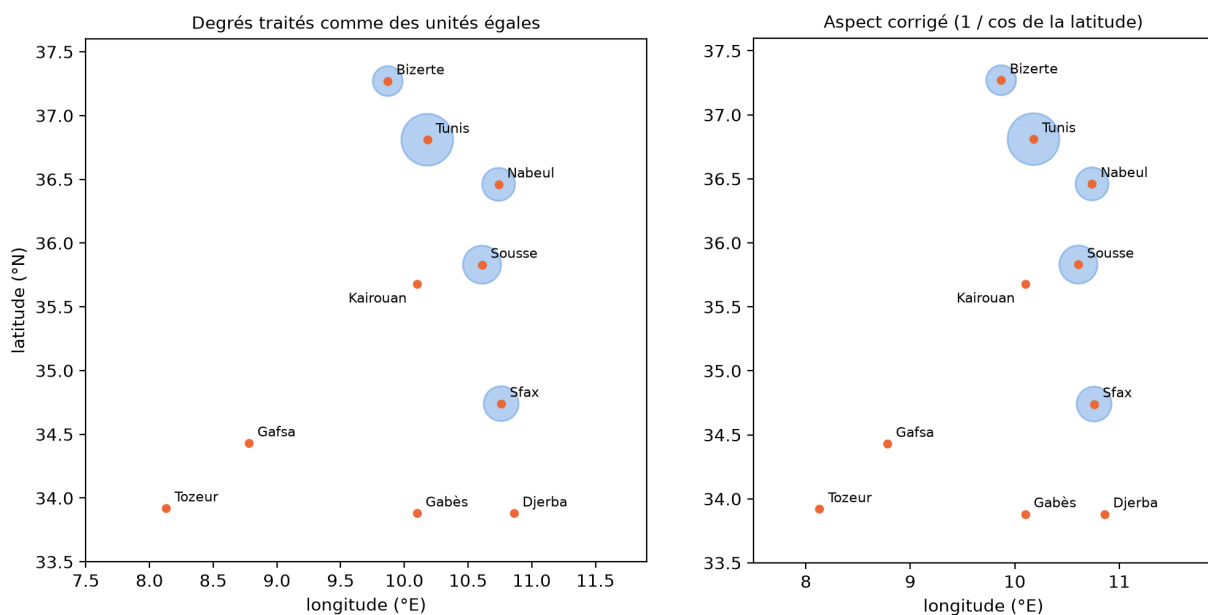


Figure 90: Les dix villes (points orange) et les effectifs de clients (disques bleus, surface proportionnelle). À gauche : un degré de longitude est dessiné aussi long qu'un degré de latitude, ce qui étire la carte d'est en ouest. À droite : le rapport d'aspect  $1/\cos(\varphi)$  rétablit les proportions.

Deux conseils de lecture. Les disques sont **proportionnels en surface** et non en rayon : si l'on doublait le rayon pour doubler la valeur, l'œil verrait une surface quatre fois plus grande, et le graphique mentirait. Et la carte de gauche, qui traite les degrés comme des unités égales (c'est ce que fait un `scatter(lon, lat)` avec `aspect="equal"`), déforme les distances : les villes y paraissent plus éloignées d'est en ouest qu'elles ne le sont.

Le même principe vaut pour les zones : pour afficher une valeur par délégation (données surfaciques), on dessine une grille colorée (une « carte choroplèthe » dans le jargon). Chargeons nos deux jeux de données de travail, la grille de délégations et les 200 livraisons, et regardons-les côte à côte.

```
deleg = pd.read_csv("donnees/ch09-delegations.csv")
liv = pd.read_csv("donnees/ch09-livraisons.csv")
print(deleg.head(4).to_string(index=False))
print()
print(liv.head(4).to_string(index=False))
print()
```

```
print("délégations :", deleg.shape, "| livraisons :", liv.shape)
print(liv["delai_jours"].describe().round(2).to_string())
```

| id | lig | col | x    | y   | population | ventes_hab | ventes_bruit |
|----|-----|-----|------|-----|------------|------------|--------------|
| 0  | 0   | 0   | 2.5  | 2.5 | 3811       | 60.70      | 58.02        |
| 1  | 0   | 1   | 7.5  | 2.5 | 6211       | 64.84      | 43.91        |
| 2  | 0   | 2   | 12.5 | 2.5 | 5702       | 52.04      | 47.84        |
| 3  | 0   | 3   | 17.5 | 2.5 | 5151       | 63.46      | 48.15        |

| x     | y     | delai_jours | pli |
|-------|-------|-------------|-----|
| 69.77 | 31.38 | 5.465       | 4   |
| 12.12 | 32.36 | 5.798       | 1   |
| 93.12 | 78.97 | 5.374       | 2   |
| 1.00  | 19.89 | 2.985       | 0   |

```
délégations : (144, 8) | livraisons : (200, 4)
```

```
count    200.00
mean         4.27
std          1.10
min          1.59
25%          3.62
50%          4.26
75%          5.07
max          7.53
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.6))

# Gauche : carte « choroplèthe » des ventes par habitant, grille 12 x 12 (une case = 5 km x 5 km)
grille = deleg["ventes_hab"].to_numpy().reshape(12, 12)
im = ax1.imshow(grille, origin="lower", extent=(0, 60, 0, 60), cmap="Blues")
ax1.set_title("Ventes par habitant (DT) par délégation")
ax1.set_xlabel("x (km)"); ax1.set_ylabel("y (km)")
fig.colorbar(im, ax=ax1, shrink=0.8, label="DT par habitant")

# Droite : délai de livraison en 200 adresses
sc = ax2.scatter(liv["x"], liv["y"], c=liv["delai_jours"], cmap="Oranges", s=46, edgecolor="#555555",
               linewidth=0.3)
ax2.set_title("Délai de livraison (jours) à 200 adresses")
ax2.set_xlabel("x (km)"); ax2.set_ylabel("y (km)")
ax2.set_aspect("equal")
fig.colorbar(sc, ax=ax2, shrink=0.8, label="jours")
plt.tight_layout()
plt.savefig("figures/ch09-donnees-chapitre.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

```
figure enregistrée
```

La carte de gauche ne ressemble pas à du « sel et poivre » : on y voit nettement une plage de faibles ventes au sud-est et des plages de fortes ventes au centre et au nord. La carte de droite est bien plus difficile à lire : des points de teintes voisines se côtoient, mais d'autres se mêlent, et l'œil hésite entre structure et hasard. C'est exactement pour cela qu'il faut **chiffrer** l'impression plutôt que de s'y fier. Mais avant, une mise en garde sur le dessin lui-même.

**Les cartes mentent facilement.** Trois pièges classiques : (1) **la palette**, car une rampe séquentielle (clair → foncé) convient à une quantité ordonnée, et un arc-en-ciel invente des frontières qui n'existent pas ; (2) **les classes**, car découper une valeur continue en cinq paliers « égaux » ou en quantiles change complètement l'image ; (3) **la surface** : une grande zone peu peuplée attire l'œil autant qu'une petite zone très peuplée. Nous reviendrons sur ce dernier point à la section 9.5 (le problème de l'unité spatiale modifiable).

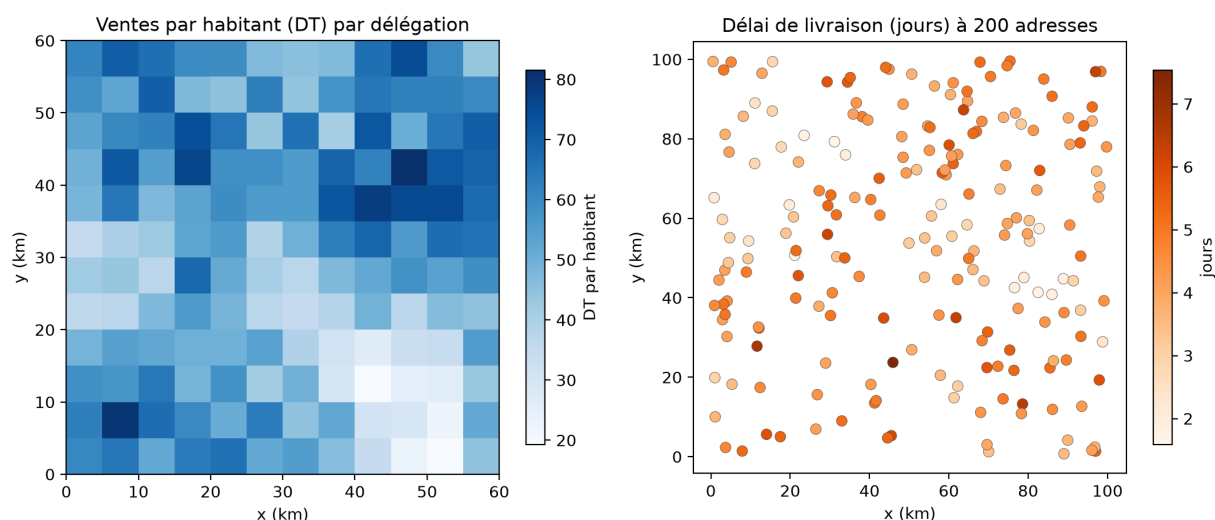


Figure 91: À gauche : ventes par habitant par délégation (grille  $12 \times 12$ ). À droite : délais de livraison observés en 200 adresses d'un carré de 100 km de côté.

### À retenir.

- L'espace casse l'**indépendance** : des observations proches se ressemblent, les erreurs-types usuelles sont trop optimistes.
- Trois familles : **géostatistique** (valeurs en des points d'un champ continu), **surfacique** (une valeur par zone), **semis de points** (les positions sont aléatoires). On se demande d'abord : *qu'est-ce qui est aléatoire ?*
- Un degré de longitude ne vaut pas un degré de latitude :  $111,2 \cos \varphi$  km contre 111,2 km. Pour des distances sur la Terre, on utilise **haversine** (ou on projette dans un plan local en kilomètres).
- Une carte sans fond de carte reste utile : rapport d'aspect correct, symboles proportionnels **en surface**, palette séquentielle.

## 9.2 Autocorrélation spatiale

**Intuition.** La carte des ventes par délégation (9.1) semble faite d'îlots. Pour savoir si cette impression est réelle, on pose une question simple : **la valeur d'une zone ressemble-t-elle à la moyenne de ses voisines ?** Si oui, les valeurs sont *positivement autocorrélées dans l'espace*. Si les zones voisines s'opposent (une forte entourée de faibles), l'autocorrélation est *négative*. Si le voisinage n'apprend rien sur la valeur, il n'y en a pas. L'indice de Moran transforme cette question en un nombre, et un test de permutation dit si ce nombre est plus grand que ce que le hasard produirait.

### 9.2.1 Définir « voisin » : la matrice de poids

Il n'existe pas de notion de voisinage « naturelle » : c'est à nous de la **choisir**, et ce choix s'écrit sous la forme d'une matrice  $W$  de taille  $n \times n$ , la **matrice de poids** (ou de voisinage). Son coefficient  $w_{ij} \geq 0$  dit à quel point la zone  $j$  compte comme voisine de la zone  $i$ , avec  $w_{ii} = 0$  (une zone n'est pas sa propre voisine). Les définitions usuelles :

- **Contiguïté « tour »** (*rook*) :  $w_{ij} = 1$  si les zones partagent une frontière (côté) ; **contiguïté « reine »** (*queen*) : si elles partagent une frontière *ou* un coin (sur une grille : 4 contre 8 voisins).

- **$k$  plus proches voisins** :  $w_{ij} = 1$  si  $j$  est l'un des  $k$  points les plus proches de  $i$ . Pratique pour des points (adresses), mais  $W$  n'est plus symétrique.
- **Bande de distance** :  $w_{ij} = 1$  si la distance entre  $i$  et  $j$  ne dépasse pas un seuil  $d$ .
- **Inverse de la distance** :  $w_{ij} = 1/d_{ij}^\alpha$ , où les points proches pèsent plus.

On **standardise** presque toujours par ligne : on divise chaque ligne par sa somme, de sorte que  $\sum_j w_{ij} = 1$ . Alors  $(Wz)_i$  est la **moyenne** des valeurs des voisins de  $i$ , ce qui rend les formules lisibles. Notons  $S_0 = \sum_{i,j} w_{ij}$  la somme de tous les poids (égale à  $n$  après standardisation par ligne).

Un exemple minuscule que nous garderons pour les calculs à la main : une grille  $3 \times 3$  de neuf zones numérotées de 0 à 8 ligne par ligne, avec la contiguïté « tour ».

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

def contiguite_tour(n_lig, n_col):
    """Matrice binaire des voisins par un côté (haut, bas, gauche, droite)."""
    n = n_lig * n_col
    W = np.zeros((n, n))
    for i in range(n_lig):
        for j in range(n_col):
            for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                if 0 <= i + di < n_lig and 0 <= j + dj < n_col:
                    W[i * n_col + j, (i + di) * n_col + (j + dj)] = 1
    return W

W3 = contiguite_tour(3, 3)
print(W3.astype(int))
print("nombre de voisins par zone :", W3.sum(axis=1).astype(int))
print("S0 (somme de tous les poids) :", int(W3.sum()))
```

```
[[0 1 0 1 0 0 0 0 0]
 [1 0 1 0 1 0 0 0 0]
 [0 1 0 0 0 1 0 0 0]
 [1 0 0 0 1 0 1 0 0]
 [0 1 0 1 0 1 0 1 0]
 [0 0 1 0 1 0 0 0 1]
 [0 0 0 1 0 0 0 1 0]
 [0 0 0 0 1 0 1 0 1]
 [0 0 0 0 0 1 0 1 0]]
nombre de voisins par zone : [2 3 2 3 4 3 2 3 2]
S0 (somme de tous les poids) : 24
```

Les coins ont 2 voisines, les bords 3, le centre 4 : au total  $S_0 = 24$  liens orientés, soit 12 frontières communes comptées dans les deux sens. Pour obtenir la version standardisée par ligne, il suffit de diviser chaque ligne par sa somme.

```
def std_lignes(W):
    """Standardise par ligne : chaque ligne somme à 1 (les lignes de zéros sont laissées intactes)."""
    s = W.sum(axis=1, keepdims=True)
    return np.divide(W, s, out=np.zeros_like(W), where=s > 0)

W3s = std_lignes(W3)
print(np.round(W3s[4], 2), "<- ligne du centre : 1/4 pour chacun de ses 4 voisins")
print("somme de chaque ligne :", W3s.sum(axis=1))
```

```
[0. 0.25 0. 0.25 0. 0.25 0. 0.25 0. ] <- ligne du centre : 1/4 pour chacun de ses 4 voisins
somme de chaque ligne : [1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

### 9.2.2 L'indice de Moran

Soit  $y_1, \dots, y_n$  les valeurs observées,  $\bar{y}$  leur moyenne et  $z_i = y_i - \bar{y}$  les valeurs **centrées**. L'**indice de Moran** est

$$I = \frac{n}{S_0} \frac{\sum_i \sum_j w_{ij} z_i z_j}{\sum_i z_i^2}.$$

Lisons-le morceau par morceau. Le numérateur  $\sum_{ij} w_{ij} z_i z_j$  additionne, pour chaque paire de voisines, le **produit** de leurs écarts à la moyenne : il est positif quand les deux voisines sont du même côté de la moyenne (toutes deux hautes ou toutes deux basses) et négatif quand elles s'opposent. Le dénominateur  $\sum_i z_i^2$  est la variance totale (à un facteur près), qui joue le rôle de **normalisation** :  $I$  ressemble à un coefficient de corrélation entre une zone et ses voisines. Le facteur  $n/S_0$  compense le nombre de liens.

**Une lecture géométrique.** Avec une matrice standardisée par ligne ( $S_0 = n$ ),  $I = \frac{z^T W z}{z^T z}$ . Mais  $Wz$  est le vecteur des **moyennes des valeurs voisines**. Donc  $I$  est exactement la **pente de la droite de régression** de « la moyenne de mes voisines » en fonction de « ma valeur » : le *diagramme de Moran* (nuage de  $Wz$  contre  $z$ ) montre ce que mesure l'indice. Une pente de 0,6 signifie : quand une zone est supérieure à la moyenne de 10, ses voisines le sont, en moyenne, de 6.

**Calcul à la main.** Sur la grille  $3 \times 3$  avec des poids binaires ( $S_0 = 24$ ), prenons les valeurs de 1 à 9 rangées ligne par ligne : une progression régulière, donc des voisines qui se ressemblent.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad \bar{y} = 5, \quad z = \begin{pmatrix} -4 & -3 & -2 \\ -1 & 0 & 1 \\ 2 & 3 & 4 \end{pmatrix}, \quad \sum z_i^2 = 60.$$

La somme  $\sum_{ij} w_{ij} z_i z_j$  compte chaque frontière deux fois, une fois dans chaque sens. Calculons les 12 produits sur les frontières, en les rangeant selon leur orientation.

- **Frontières horizontales** :  $(-4)(-3) = 12$  ;  $(-3)(-2) = 6$  ;  $(-1)(0) = 0$  ;  $(0)(1) = 0$  ;  $(2)(3) = 6$  ;  $(3)(4) = 12$  : total 36.
- **Frontières verticales** :  $(-4)(-1) = 4$  ;  $(-1)(2) = -2$  ;  $(-3)(0) = 0$  ;  $(0)(3) = 0$  ;  $(-2)(1) = -2$  ;  $(1)(4) = 4$  : total 4.

La somme sur les frontières vaut 40, donc  $\sum_{ij} w_{ij} z_i z_j = 2 \times 40 = 80$  et

$$I = \frac{9}{24} \times \frac{80}{60} = 0,5.$$

Dans l'autre sens, rangeons en **damier** : des 1 et des 9 qui alternent, de sorte que chaque zone n'ait que des voisines très différentes. Ici  $\bar{y} = 41/9$ , les zones à 1 ont  $z = -32/9$ , celles à 9 ont  $z = +40/9$ , chaque frontière relie un 1 à un 9 (produit  $-1280/81$ ), il y a 12 frontières donc  $\sum_{ij} w_{ij} z_i z_j = -2 \times 12 \times 1280/81$ , et  $\sum z^2 = 11520/81$ . Tous calculs faits,  $I = \frac{9}{24} \times \frac{-30720}{11520} = -1$  : l'autocorrélation négative maximale. Vérifions les deux calculs, puis la version standardisée par ligne.

```
def moran(y, W):
    """Indice de Moran de y pour la matrice de poids W (standardisée ou non)."""
    z = np.asarray(y, dtype=float) - np.mean(y)
    return len(z) / W.sum() * (z @ W @ z) / (z @ z)

progression = np.arange(1, 10)
damier = np.array([1, 9, 1, 9, 1, 9, 1, 9, 1], dtype=float)
print("progression 1..9, poids binaires      :", round(moran(progression, W3), 4))
print("damier,          poids binaires      :", round(moran(damier, W3), 4))
print("progression 1..9, poids standardisés :", round(moran(progression, W3s), 4))
print("E[I] sans autocorrélation, n = 9      :", round(-1 / (9 - 1), 4))

progression 1..9, poids binaires      : 0.5
damier,          poids binaires      : -1.0
progression 1..9, poids standardisés : 0.5556
E[I] sans autocorrélation, n = 9      : -0.125
```

Les valeurs 0,5 et  $-1$  retrouvent nos calculs à la main. La version standardisée donne un nombre un peu différent : le **choix de W fait partie du modèle** et il doit être indiqué avec le résultat.

**Que vaut  $I$  en l'absence d'autocorrélation ?** Sous l'hypothèse nulle « les valeurs sont échangeables » (aucune relation avec la position), toute permutation des  $y_i$  sur les zones est aussi plausible qu'une autre. Pour  $i \neq j$ ,  $\mathbb{E}[z_i z_j] = \frac{1}{n(n-1)} \sum_{a \neq b} z_a z_b = \frac{(\sum_a z_a)^2 - \sum_a z_a^2}{n(n-1)} = -\frac{\sum_a z_a^2}{n(n-1)}$ , car  $\sum_a z_a = 0$ . Comme  $w_{ii} = 0$ , il vient

$$\mathbb{E}[I] = \frac{n}{S_0} \frac{\sum_{i \neq j} w_{ij} \mathbb{E}[z_i z_j]}{\sum_i z_i^2} = \frac{n}{S_0} \cdot S_0 \cdot \frac{-1}{n(n-1)} = -\frac{1}{n-1}.$$

Ce n'est pas exactement 0 (la contrainte « la moyenne des  $z$  est nulle » induit une très légère corrélation négative), mais pour  $n = 144$  zones,  $-1/143 \approx -0,007$  : négligeable. Pour  $n = 9$ , l'espérance vaut  $-0,125$ , ce qui est loin d'être négligeable : on **ne compare pas** un petit Moran à zéro mais à  $-1/(n-1)$ .

**Donner un sens à la valeur observée : deux voies.** Quelle valeur de  $I$  est « grande » ? Il faut connaître la loi de  $I$  sous l'hypothèse nulle.

1. **Approximation normale.** Sous une hypothèse de normalité des  $y_i$ , on connaît la variance de  $I$  :

$$\text{Var}(I) = \frac{n^2 S_1 - n S_2 + 3 S_0^2}{(n^2 - 1) S_0^2} - \mathbb{E}[I]^2,$$

avec  $S_1 = \frac{1}{2} \sum_{ij} (w_{ij} + w_{ji})^2$  et  $S_2 = \sum_i (\sum_j w_{ij} + \sum_j w_{ji})^2$ . On forme alors  $z_I = (I - \mathbb{E}[I]) / \sqrt{\text{Var}(I)}$  et on le compare à une loi normale centrée réduite.

2. **Permutations.** Plus simple et sans hypothèse de loi : on **mélange** les valeurs sur les zones  $B$  fois, on recalcule  $I$  à chaque fois, et on regarde où tombe la valeur observée parmi ces  $B$  valeurs. C'est le *test de permutation* du volume I (section 3.7.4), appliqué à l'espace. La p-valeur unilatérale pour une autocorrélation positive est  $\frac{1 + \#\{I^* \geq I_{\text{obs}}\}}{B + 1}$ .

Appliquons-les à nos 144 délégations, avec la contiguïté « reine » (8 voisins). Voici d'abord le code qui a fabriqué les ventes (la même fonction est dans `build/donnees_ch09.py`). On part de bruits indépendants  $e_i$  et on les propage dans l'espace par  $z = (I - \rho W)^{-1} e$  : un **modèle autorégressif spatial** (SAR), où chaque zone est un reflet de ses voisines, avec une force  $\rho = 0,9$ .

```
def contiguite_reine(n_lig, n_col):
    """Voisins par un côté ou un coin (8 directions)."""
    n = n_lig * n_col
```



```

W = np.zeros((n, n))
for i in range(n_lig):
    for j in range(n_col):
        for di in (-1, 0, 1):
            for dj in (-1, 0, 1):
                if (di, dj) != (0, 0) and 0 <= i + di < n_lig and 0 <= j + dj < n_col:
                    W[i * n_col + j, (i + di) * n_col + (j + dj)] = 1
return W

def delegations(seed=9, n_lig=12, n_col=12, rho=0.9):
    rng = np.random.default_rng(seed)
    W = std_lignes(contiguïte_reine(n_lig, n_col))
    n = n_lig * n_col
    e = rng.normal(size=n)
    z = np.linalg.solve(np.eye(n) - rho * W, e) # z = (I - rho W)^-1 e
    pop = np.round(rng.lognormal(8.5, 0.5, n)).astype(int)
    lig, col = np.divmod(np.arange(n), n_col)
    return pd.DataFrame({"id": np.arange(n), "lig": lig, "col": col, "x": col * 5.0 + 2.5, "y": lig *
        ↪ 5.0 + 2.5,
                        "population": pop, "ventes_hab": np.round(50 + 8 * z, 2),
                        "ventes_bruit": np.round(rng.normal(50, 8, n), 2)})

deleg = delegations()
fichier = pd.read_csv("donnees/ch09-delegations.csv")
print("identique au fichier fourni :", np.allclose(deleg[["ventes_hab", "ventes_bruit"]],
    ↪ fichier[["ventes_hab", "ventes_bruit"]]))
print(deleg[["ventes_hab", "ventes_bruit"]].describe().round(2).to_string())

```

```

identique au fichier fourni : True
      ventes_hab  ventes_bruit
count      144.00      144.00
mean        52.89       50.22
std         13.51        8.15
min         19.29       29.97
25%         44.29       43.90
50%         53.43       50.36
75%         62.67       56.50
max         81.49       71.64

```

Nous disposons de deux variables : ventes\_hab (structurée dans l'espace) et ventes\_bruit (du bruit indépendant, **sans** structure spatiale ; sa moyenne et son écart-type sont proches de ceux de l'autre variable, sans être identiques). Cette deuxième variable est notre *témoin* : un bon indice doit y voir du hasard.

```

W = std_lignes(contiguïte_reine(12, 12))
n = len(deleg)

def stats_poids(W):
    S0 = W.sum()
    S1 = 0.5 * ((W + W.T) ** 2).sum()
    S2 = ((W.sum(axis=1) + W.sum(axis=0)) ** 2).sum()
    return S0, S1, S2

def moran_normal(y, W):
    """I, E[I], écart-type sous normalité et z_I."""
    n = len(y)
    S0, S1, S2 = stats_poids(W)
    E = -1 / (n - 1)
    var = (n**2 * S1 - n * S2 + 3 * S0**2) / ((n**2 - 1) * S0**2) - E**2
    I = moran(y, W)
    return I, E, np.sqrt(var), (I - E) / np.sqrt(var)

def moran_permutations(y, W, B=9999, seed=1):

```

```

"""B valeurs de I obtenues en mélangeant y sur les zones."""
rng = np.random.default_rng(seed)
z = np.asarray(y, dtype=float) - np.mean(y)
idx = np.argsort(rng.random((B, len(z))), axis=1)           # B permutations de 0..n-1
Z = z[idx]                                                  # chaque ligne est un mélange de z
return len(z) / W.sum() * ((Z @ W) * Z).sum(axis=1) / (z @ z)

lignes = []
for nom in ["ventes_hab", "ventes_bruit"]:
    y = deleg[nom].to_numpy()
    I, E, sd, zI = moran_normal(y, W)
    sim = moran_permutations(y, W)
    p_perm = (1 + (sim >= I).sum()) / (len(sim) + 1)
    lignes.append({"variable": nom, "I": I, "E[I]": E, "ecart_type": sd, "z": zI, "p_normale":
↳ stats.norm.sf(zI),
                    "p_permutation": p_perm, "sd_perm": sim.std()})
res = pd.DataFrame(lignes)
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 150):
    print(res.to_string(index=False))

```

| variable     | I       | E[I]      | ecart_type | z       | p_normale | p_permutation | sd_perm |
|--------------|---------|-----------|------------|---------|-----------|---------------|---------|
| ventes_hab   | 0.6207  | -0.006993 | 0.04429    | 14.17   | 6.802e-46 | 0.0001        | 0.04468 |
| ventes_bruit | -0.0214 | -0.006993 | 0.04429    | -0.3253 | 0.6275    | 0.6132        | 0.04445 |

La variable structurée a un indice de Moran nettement positif et une p-valeur proche de la plus petite que l'on puisse obtenir avec  $B = 9999$  permutations (jamais 0 : le +1 au numérateur représente la valeur observée elle-même). Le témoin, lui, est proche de  $-1/(n-1)$  et sa p-valeur est élevée. Remarquons aussi que l'écart-type obtenu par permutations est voisin de celui de la formule normale : deux voies, une même conclusion.

**L'indice ne vaut pas  $\rho$ .** Nous avons fabriqué les données avec  $\rho = 0,9$ , mais  $I \neq 0,9$ . Le paramètre  $\rho$  d'un modèle SAR et l'indice de Moran mesurent la dépendance **sur des échelles différentes** :  $\rho$  gouverne la façon dont un choc se propage de proche en proche, alors que  $I$  résume la ressemblance moyenne entre voisines **directes**. Une forte propagation ( $\rho$  proche de 1) produit de grandes plages, mais la ressemblance entre deux voisines immédiates reste partielle. Ne comparez jamais  $I$  à une probabilité ou à un coefficient du modèle.

On voit mieux ce que mesure  $I$  avec le **diagramme de Moran** (le nuage de la moyenne des voisines contre la valeur de la zone) et l'histogramme de la distribution nulle :

```

BLEU, ORANGE, AQUA, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#e34948"
y = deleg["ventes_hab"].to_numpy()
zc = y - y.mean()
lag = W @ zc
I_obs = moran(y, W)
sim = moran_permutations(y, W)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.4))
ax1.scatter(zc, lag, s=16, color=BLEU, alpha=0.7)
pente, ordonnee = np.polyfit(zc, lag, 1)
xs = np.array([zc.min(), zc.max()])
ax1.plot(xs, pente * xs + ordonnee, color=ORANGE, lw=2)
ax1.axhline(0, color="#999999", lw=0.8); ax1.axvline(0, color="#999999", lw=0.8)
ax1.set_xlabel("écart à la moyenne de la zone z"); ax1.set_ylabel("moyenne des écarts des voisines W
↳ z")
ax1.set_title(f"Diagramme de Moran (pente = I = {pente:.3f})")

ax2.hist(sim, bins=50, color="#b9d0f0", edgecolor="white")
ax2.axvline(I_obs, color=ORANGE, lw=2)
ax2.annotate(f"I observé = {I_obs:.3f}", (I_obs, 400), xytext=(-8, 0), textcoords="offset points",
↳ ha="right", color=ORANGE)

```

```
ax2.set_xlabel("I après mélange des valeurs (9999 permutations)"); ax2.set_ylabel("effectif")
ax2.set_title("Ce que produirait le hasard")
plt.tight_layout()
plt.savefig("figures/ch09-moran.png", dpi=200, bbox_inches="tight")
print("pente du diagramme :", round(pente, 4), "| I :", round(I_obs, 4), "| identiques :",
      ↪ np.isclose(pente, I_obs))
```

```
pente du diagramme : 0.6207 | I : 0.6207 | identiques : True
```

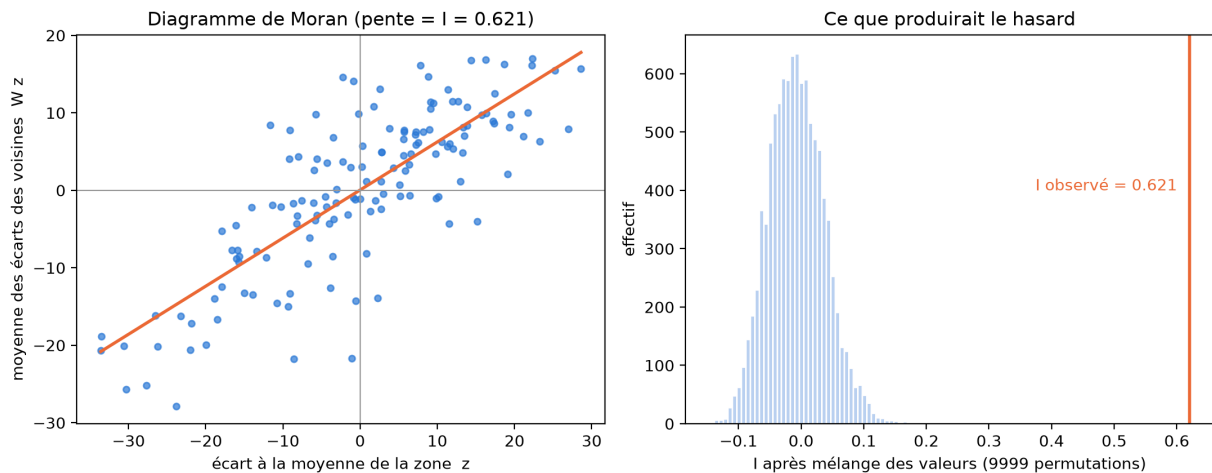


Figure 92: À gauche : diagramme de Moran des ventes par habitant ; la pente de la droite est l'indice de Moran. À droite : distribution de  $I$  quand on mélange les valeurs au hasard sur les zones (hypothèse nulle) et valeur observée.

La pente du diagramme est **égale** à l'indice de Moran, comme annoncé. L'histogramme de droite montre la loi de  $I$  quand les valeurs sont distribuées au hasard : centrée sur un nombre proche de zéro, avec un étalement de l'ordre de 0,05. La valeur observée est loin à droite de cette loi : le hasard ne produit pas un tel alignement.

**La sensibilité au choix de  $W$ .** Le résultat dépend-il de notre définition du voisinage ? Recalculons  $I$  avec cinq définitions, construites à partir des **distances** entre les centres des zones, ce qui est la façon de faire pour des points dans l'espace quelconques.

```
xy = deleg[["x", "y"]].to_numpy()
Dd = np.sqrt(((xy[:, None, :] - xy[None, :, :]) ** 2).sum(axis=2)) # distances euclidiennes (km)

def poids_bande(D, d):
    W = ((D > 0) & (D <= d)).astype(float)
    return W

def poids_knn(D, k):
    W = np.zeros_like(D)
    for i in range(len(D)):
        voisins = np.argsort(D[i])[1:k + 1] # on saute la distance nulle
        ↪ (soi-même)
        W[i, voisins] = 1
    return W

variantes = {
    "tour (distance <= 5 km)": poids_bande(Dd, 5.0),
    "reine (distance <= 7,1 km)": poids_bande(Dd, 7.1),
    "bande de 10 km": poids_bande(Dd, 10.0),
    "4 plus proches voisins": poids_knn(Dd, 4),
    "8 plus proches voisins": poids_knn(Dd, 8),
}
```

```
lignes = []
for nom, Wv in variantes.items():
    Ws = std_lignes(Wv)
    sim_v = moran_permutations(y, Ws, B=999, seed=3)
    I_v = moran(y, Ws)
    lignes.append({"definition": nom, "voisins_moyens": Wv.sum(axis=1).mean(), "I": I_v,
                  "I_temoin": moran(deleg["ventes_bruit"].to_numpy(), Ws),
                  "p_permutation": (1 + (sim_v >= I_v).sum()) / 1000})
with pd.option_context("display.float_format", "{:.3f}").format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))
```

|       | definition             | voisins_moyens | I     | I_temoin | p_permutation |
|-------|------------------------|----------------|-------|----------|---------------|
| tour  | (distance <= 5 km)     | 3.667          | 0.627 | -0.034   | 0.001         |
| reine | (distance <= 7,1 km)   | 7.028          | 0.621 | -0.021   | 0.001         |
|       | bande de 10 km         | 10.361         | 0.572 | 0.009    | 0.001         |
|       | 4 plus proches voisins | 4.000          | 0.618 | -0.013   | 0.001         |
|       | 8 plus proches voisins | 8.000          | 0.607 | 0.011    | 0.001         |

Tous les choix donnent un indice nettement positif et une p-valeur minimale, et le témoin reste proche de zéro. Mais **la valeur de I change** : plus on élargit le voisinage, plus on mélange des zones qui se ressemblent peu, et plus l'indice diminue. Retenons-en une règle : une conclusion du type « il y a de l'autocorrélation » ne devrait pas dépendre du choix de W ; la **valeur de I**, elle, n'a de sens que pour un W donné. On indique donc toujours la définition choisie.

### 9.2.3 L'indice de Geary

L'indice de Moran compare chaque zone à **la moyenne** (par les écarts  $z_i$ ). Celui de Geary compare directement les zones voisines **entre elles** :

$$C = \frac{n-1}{2S_0} \frac{\sum_i \sum_j w_{ij} (y_i - y_j)^2}{\sum_i z_i^2}.$$

Le numérateur est la somme des carrés des **différences** entre voisines, qui sera petite si les voisines se ressemblent. Sans autocorrélation,  $\mathbb{E}[C] = 1$ . Une valeur **inférieure à 1** indique une autocorrélation positive ; supérieure à 1, une autocorrélation négative. L'échelle est donc inversée par rapport à Moran. Geary est plus sensible aux **différences locales** (à l'échelle d'une paire de voisines) alors que Moran, qui passe par la moyenne globale, est plus sensible à la structure d'ensemble.

**Le lien avec le variogramme (9.3).** Le terme  $\frac{1}{2}(y_i - y_j)^2$  est exactement la *semi-variance* d'une paire, la brique du variogramme. L'indice de Geary est donc, à une normalisation près, une moyenne de semi-variances **sur les paires voisines** divisée par la variance totale. C'est une première approximation du variogramme à une seule distance.

```
def geary(y, W):
    y = np.asarray(y, dtype=float)
    z = y - y.mean()
    diff2 = (y[:, None] - y[None, :]) ** 2
    return (len(y) - 1) * (W * diff2).sum() / (2 * W.sum() * (z @ z))

def geary_permutations(y, W, B=999, seed=4):
    rng = np.random.default_rng(seed)
    return np.array([geary(rng.permutation(y), W) for _ in range(B)])

lignes = []
for nom in ["ventes_hab", "ventes_bruit"]:
    yy = deleg[nom].to_numpy()
```

```

C = geary(yy, W)
sim_c = geary_permutations(yy, W)
lignes.append({"variable": nom, "C": C, "moyenne_perm": sim_c.mean(), "p_perm (C petit)": (1 +
↪ (sim_c <= C).sum()) / (len(sim_c) + 1)})
with pd.option_context("display.float_format", "{:.4f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))

```

| variable     | C      | moyenne_perm | p_perm (C petit) |
|--------------|--------|--------------|------------------|
| ventes_hab   | 0.3950 | 1.0033       | 0.0010           |
| ventes_bruit | 0.9909 | 0.9997       | 0.4030           |

Les deux indices racontent la même histoire : la variable structurée a un C bien inférieur à 1, le témoin est proche de 1.

### 9.2.4 Où sont les îlots ? Les indices locaux (LISA)

Un seul nombre pour toute la carte ne dit pas où se trouvent les îlots. Les **indicateurs locaux d'association spatiale** (LISA) décomposent l'indice de Moran en une contribution par zone :

$$I_i = \frac{z_i}{m_2} \sum_j w_{ij} z_j, \quad m_2 = \frac{1}{n} \sum_k z_k^2.$$

Avec une matrice standardisée par ligne, la somme des  $I_i$  vaut exactement  $nI$ . Chaque  $I_i$  est positif quand la zone  $i$  et ses voisines sont du même côté de la moyenne. Le signe de  $z_i$  et celui de la moyenne voisine  $Wz_i$  donnent les **quatre quadrants** du diagramme de Moran :

- **HH** : zone haute entourée de zones hautes (un **îlot de fortes valeurs**) ;
- **LL** : zone basse entourée de zones basses (un **îlot de faibles valeurs**) ;
- **HL / LH** : une zone haute entourée de basses (ou l'inverse), des **valeurs atypiques locales**.

Pour savoir si un  $I_i$  est significatif, on utilise une **permutation conditionnelle** : on garde la valeur de la zone  $i$  fixe, on mélange les autres valeurs sur les autres zones, et on recalcule  $I_i$ . Mais attention : nous faisons **144 tests à la fois**. Comme au volume I (section 3.5.5), sans correction nous attendrions environ 7 faux positifs (5 % de 144) même sur des données sans structure ; nous utiliserons la procédure de **Benjamini-Hochberg**.

```

def lisa(y, W, B=999, seed=2):
    """Moran local, quadrant, et p-valeur bilatérale par permutation conditionnelle."""
    rng = np.random.default_rng(seed)
    y = np.asarray(y, dtype=float)
    n = len(y)
    z = y - y.mean()
    m2 = (z @ z) / n
    lag = W @ z
    Ii = z * lag / m2
    pv = np.empty(n)
    for i in range(n):
        voisins = np.flatnonzero(W[i])
        poids = W[i, voisins]
        r = rng.random((B, n))
        r[:, i] = 2.0 # exclut la zone i du tirage
        tire = np.argsort(r, axis=1)[:n, :len(voisins)] # valeurs tirées pour les voisins
        sim = z[i] * (z[tire] * poids).sum(axis=1) / m2
        p_haut = (1 + (sim >= Ii[i]).sum()) / (B + 1)
        p_bas = (1 + (sim <= Ii[i]).sum()) / (B + 1)
        pv[i] = min(1.0, 2 * min(p_haut, p_bas))
    quadrant = np.where(z > 0, np.where(lag > 0, "HH", "HL"), np.where(lag > 0, "LH", "LL"))
    return Ii, quadrant, pv

```

```
def benjamini_hochberg(p, alpha=0.05):
    """Renvoie un tableau booléen des hypothèses rejetées (FDR contrôlé à alpha)."""
    p = np.asarray(p)
    m = len(p)
    ordre = np.argsort(p)
    ok = p[ordre] <= (np.arange(1, m + 1) / m) * alpha
    rejet = np.zeros(m, dtype=bool)
    if ok.any():
        rejet[ordre[: np.max(np.flatnonzero(ok)) + 1]] = True
    return rejet

lignes = []
resultats = {}
for nom in ["ventes_hab", "ventes_bruit"]:
    Ii, quad, pv = lisa(deleg[nom].to_numpy(), W)
    rej = benjamini_hochberg(pv)
    resultats[nom] = (Ii, quad, pv, rej)
    comptes = pd.Series(np.where(rej, quad, "non significatif")).value_counts()
    lignes.append({"variable": nom, "somme_Ii/n": Ii.sum() / len(Ii), "I_global":
        moran(deleg[nom].to_numpy(), W),
        "p<0.05 brut": int((pv < 0.05).sum()), "significatifs (BH)": int(rej.sum()),
        "HH": int(comptes.get("HH", 0)), "LL": int(comptes.get("LL", 0)),
        "HL": int(comptes.get("HL", 0)), "LH": int(comptes.get("LH", 0))})
with pd.option_context("display.float_format", "{:.4f}".format, "display.width", 170):
    print(pd.DataFrame(lignes).to_string(index=False))
```

| variable     | somme_Ii/n | I_global | p<0.05 brut | significatifs (BH) | HH | LL | HL | LH |
|--------------|------------|----------|-------------|--------------------|----|----|----|----|
| ventes_hab   | 0.6207     | 0.6207   | 49          | 35                 | 11 | 22 | 0  | 2  |
| ventes_bruit | -0.0214    | -0.0214  | 5           | 0                  | 0  | 0  | 0  | 0  |

La première colonne vérifie la propriété annoncée : la moyenne des  $I_i$  est bien l'indice de Moran global. Sur le témoin, 5 zones sur 144 (3 %, soit à peu près les 5 % attendus du hasard) sortent avec la p-valeur brute : c'est le prix des tests multiples. La correction de Benjamini-Hochberg les écarte **toutes** (0 zone significative). Sur la variable structurée, 49 zones sont significatives avant correction et 35 après : 11 îlots de fortes valeurs (HH), 22 de faibles valeurs (LL) et 2 valeurs atypiques (LH). Regardons la carte des zones significatives.

```
couleurs = {"HH": ROUGE, "LL": BLEU, "HL": ORANGE, "LH": AQUA, "ns": "#e8e6df"}
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.8))
for ax, nom in zip(axes, ["ventes_hab", "ventes_bruit"]):
    Ii, quad, pv, rej = resultats[nom]
    cat = np.where(rej, quad, "ns").reshape(12, 12)
    for i in range(12):
        for j in range(12):
            ax.add_patch(plt.Rectangle((j * 5, i * 5), 5, 5, facecolor=couleurs[cat[i, j]],
            edgecolor="white", linewidth=0.8))
    ax.set_xlim(0, 60); ax.set_ylim(0, 60); ax.set_aspect("equal")
    ax.set_title("ventes par habitant (structurées)" if nom == "ventes_hab" else "témoin (bruit sans
    structure)", fontsize=10)
    ax.set_xlabel("x (km)")
axes[0].set_ylabel("y (km)")
poignees = [plt.Rectangle((0, 0), 1, 1, facecolor=couleurs[k]) for k in ["HH", "LL", "HL", "LH",
    "ns"]]
fig.legend(poignees, ["HH : haut entouré de haut", "LL : bas entouré de bas", "HL : haut entouré de
    bas", "LH : bas entouré de haut", "non significatif"],
    loc="lower center", ncol=5, frameon=False, fontsize=8)
plt.tight_layout(rect=(0, 0.07, 1, 1))
plt.savefig("figures/ch09-lisa.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

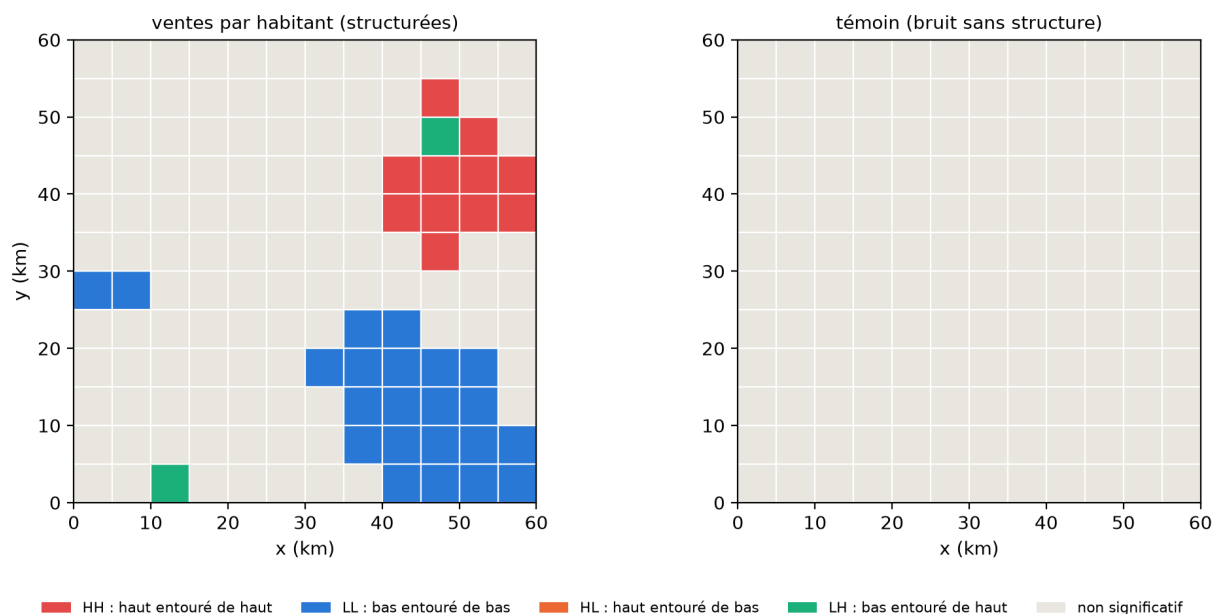


Figure 93: Carte des indicateurs locaux de Moran significatifs (Benjamini-Hochberg à 5 %). À gauche, les ventes par habitant : des îlots de fortes (rouge) et de faibles (bleu) valeurs. À droite, le témoin sans structure spatiale : presque rien n'est significatif.

figure enregistrée

La carte de gauche **localise** les îlots : l'indice global disait *qu'il y en a*, les indices locaux disent *où*. À droite, le témoin est quasiment vide.

**Deux précautions.** (1) Les indices locaux ne sont **pas indépendants** (zones voisines partagent des voisines), donc le taux de faux positifs effectif est mal maîtrisé même avec la correction de Benjamini-Hochberg : on considère les zones signalées comme des **pistes à examiner**, pas comme des découvertes. (2) Les zones **en bordure** de la carte ont moins de voisines : leurs indices sont plus bruités (nous y reviendrons à la section 9.5).

### 9.2.5 Pourquoi cela compte : la régression qui voit des relations qui n'existent pas

Revenons à l'avertissement de 9.1.1 : l'autocorrélation rend les erreurs-types usuelles **trop petites**. Mesurons-le. Nous simulons **deux variables sans aucun lien** (deux champs spatiaux SAR indépendants, avec la même force  $\rho = 0,9$  que plus haut), nous régressons l'une sur l'autre par moindres carrés ordinaires et nous testons la pente à 5 %. Si la méthode est honnête, nous devrions rejeter à tort dans environ 5 % des cas. Nous recommençons 1 000 fois.

```
from scipy import stats

A = np.linalg.inv(np.eye(n) - 0.9 * W)  # (I - rho W)^-1, rho = 0,9
rng = np.random.default_rng(5)
reps = 1000
Y = A @ rng.normal(size=(n, reps))  # 1000 champs « y », indépendants entre eux
X = A @ rng.normal(size=(n, reps))  # 1000 champs « x », indépendants de y

# MCO vectorisée : pente et test de Student sur la pente, pour chacune des 1000 paires
xc, yc = X - X.mean(axis=0), Y - Y.mean(axis=0)
b = (xc * yc).sum(axis=0) / (xc ** 2).sum(axis=0)
res = yc - b * xc
```



```

s2 = (res ** 2).sum(axis=0) / (n - 2)
t = b / np.sqrt(s2 / (xc ** 2).sum(axis=0))
p_mco = 2 * stats.t.sf(np.abs(t), n - 2)
print("MCO : proportion de rejets à 5 % alors qu'il n'y a AUCUN lien :", round((p_mco < 0.05).mean(),
↳ 3))

# Nombre « efficace » d'observations indépendantes pour estimer une moyenne
var_y = Y.var() # variance d'une zone
var_moy = Y.mean(axis=0).var() # variance de la moyenne des 144 zones
print("variance d'une zone :", round(var_y, 2), "| variance de la moyenne de 144 zones :",
↳ round(var_moy, 3),
    "| si indépendantes, ce serait", round(var_y / n, 3))
print("nombre efficace d'observations indépendantes :", round(var_y / var_moy, 1), "sur", n)

```

```

MCO : proportion de rejets à 5 % alors qu'il n'y a AUCUN lien : 0.459
variance d'une zone : 3.9 | variance de la moyenne de 144 zones : 0.706 | si indépendantes, ce serait
↳ 0.027
nombre efficace d'observations indépendantes : 5.5 sur 144

```

Le résultat est sans appel : au lieu des 5 % promis, le test usuel rejette à tort dans **46 %** des cas, presque une fois sur deux. Et pour estimer une moyenne, nos 144 zones valent environ **5,5 observations indépendantes** : la variance de la moyenne est de 0,71, contre 0,027 si les zones étaient indépendantes. Le test usuel suppose des erreurs indépendantes ; ici elles ne le sont pas, et l'erreur-type de la pente est sous-estimée d'un facteur important.

**Le remède, dans son principe.** Il faut tenir compte de la dépendance dans le calcul. Si l'on **connaissait** la structure de covariance  $\Sigma$  des erreurs, on appliquerait les **moindres carrés généralisés** (MCG), c'est-à-dire des moindres carrés sur des données *blanchies*. Pour notre modèle SAR, le blanchiment est explicite : multiplier le vecteur  $y$  par  $(I - \rho W)$  « défait » la propagation. Vérifions que cela rétablit le niveau du test, en supposant ici  $\rho$  connu (c'est le cas idéal ; en pratique on l'estime).

```

T = np.eye(n) - 0.9 * W # opérateur de blanchiment (rho connu)
un = np.ones(n)
rejets = 0
for r in range(reps):
    ys = T @ Y[:, r]
    Xs = np.column_stack([T @ un, T @ X[:, r]]) # constante et x, blanchis
    coef, *_ = np.linalg.lstsq(Xs, ys, rcond=None)
    resid = ys - Xs @ coef
    s2 = resid @ resid / (n - 2)
    cov = s2 * np.linalg.inv(Xs.T @ Xs)
    tt = coef[1] / np.sqrt(cov[1, 1])
    rejets += 2 * stats.t.sf(abs(tt), n - 2) < 0.05
print("MCG avec rho connu : proportion de rejets à 5 % :", rejets / reps)

# Et dans une seule paire (la première) : les résidus des MCO révèlent le problème
Xd = np.column_stack([np.ones(n), X[:, 0]])
beta = np.linalg.lstsq(Xd, Y[:, 0], rcond=None)[0]
residus = Y[:, 0] - Xd @ beta
sim_r = moran_permutations(residus, W, B=999, seed=6)
print("Moran des résidus MCO (première paire) :", round(moran(residus, W), 3),
    "| p-valeur par permutation :", round((1 + (sim_r >= moran(residus, W)).sum()) / 1000, 3))

```

```

MCG avec rho connu : proportion de rejets à 5 % : 0.053
Moran des résidus MCO (première paire) : 0.425 | p-valeur par permutation : 0.001

```

Avec les moindres carrés généralisés, le niveau du test retombe à 5,3 %, tout près des 5 % annoncés. Et, comme le montre la dernière ligne, il existe un **signal d'alarme** simple pour une analyse réelle : calculer l'indice de Moran des **résidus** du modèle. S'il est significatif, les erreurs ne sont pas

indépendantes et les erreurs-types usuelles sont à écarter.

**En pratique.** On ne connaît pas  $\rho$  : on l'estime avec un **modèle autorégressif spatial** (le *spatial lag* ou le *spatial error model*, estimés par maximum de vraisemblance), ou l'on modélise la covariance par un variogramme (section 9.3, krigeage universel), ou l'on utilise des **erreurs-types robustes par blocs spatiaux**. Les outils exacts sortent du cadre de ce chapitre, mais le message est général : *si vos observations sont proches dans l'espace, testez l'autocorrélation des résidus avant de lire vos p-valeurs.*

#### À retenir.

- Pour parler d'autocorrélation, il faut **définir le voisinage** : une matrice de poids  $W$  (contiguïté,  $k$  voisins, bande de distance), presque toujours standardisée par ligne. Ce choix fait partie du résultat.
- **Indice de Moran** :  $I = \frac{n}{S_0} \frac{\sum w_{ij} z_i z_j}{\sum z_i^2}$ . Avec  $W$  standardisée, c'est la pente de « moyenne des voisines » contre « valeur de la zone ». Sous l'hypothèse nulle,  $\mathbb{E}[I] = -1/(n-1)$  ; on le teste par **permutations**.
- **Geary** ( $\mathbb{E}[C] = 1$ ,  $C < 1$  pour une autocorrélation positive) est plus sensible aux différences locales.
- Les **indices locaux** (LISA) disent où sont les îlots (HH, LL) et les valeurs atypiques (HL, LH) ; avec 144 tests, il faut **corriger** (Benjamini-Hochberg).
- Ignorer l'autocorrélation fait **rejeter à tort** bien plus souvent que 5 % : testez l'indice de Moran des résidus avant de croire une p-valeur.

## 9.3 Variogramme et krigeage

**Intuition.** Yasmine veut promettre un délai à une cliente de Zaghouan, où personne n'a encore été livré. Que peut-on dire ? Les livraisons **proches** de cette adresse sont informatives, celles qui sont loin le sont moins, et deux livraisons voisines l'une de l'autre se répètent (elles apportent à peu près la même information). Le **variogramme** mesure à quelle vitesse la ressemblance entre deux mesures s'estompe quand la distance augmente. Le **krigeage** s'en sert pour fabriquer, en tout point, la **meilleure moyenne pondérée** des mesures voisines, avec une **marge d'erreur**.

### 9.3.1 De la corrélation entre voisines à une fonction de la distance

Au 9.2, la ressemblance entre voisines était résumée par un seul nombre (Moran) pour une définition de voisinage fixée. En géostatistique, on décrit toute la courbe : *comment la ressemblance dépend de la distance  $h$  qui sépare deux points.*

Le modèle de base écrit la mesure en un point  $s$  du plan comme la somme d'un niveau moyen, d'un champ qui varie régulièrement et d'un bruit de mesure :

$$Z(s) = \mu + S(s) + \varepsilon(s).$$

On suppose que le champ est **stationnaire** : sa moyenne  $\mu$  est la même partout et la dépendance entre deux points ne dépend que du **vecteur** qui les sépare, pas de l'endroit où ils sont. (L'hypothèse minimale, dite *intrinsèque*, ne demande que la stationnarité des **différences**  $Z(s+h) - Z(s)$ .) On définit alors le **semi-variogramme**

$$\gamma(h) = \frac{1}{2} \mathbb{E}[(Z(s+h) - Z(s))^2],$$

la moitié de l'écart quadratique moyen entre deux mesures séparées par  $h$ . Il est nul à l'origine ( $\gamma(0) = 0$ ) et croît d'ordinaire avec  $h$  : plus deux points sont éloignés, moins ils se ressemblent. Trois nombres décrivent la courbe :

- la **pépité** (*nugget*)  $c_0$  : la valeur de  $\gamma$  juste après 0. Elle regroupe le bruit de mesure et les variations à une échelle plus petite que la distance minimale entre points. Même deux mesures très proches ne se ressemblent pas parfaitement ;
- le **palier** (*sill*) : la valeur de  $\gamma$  quand la courbe se stabilise. C'est la variance totale du processus ( $c_0 + c$ ) ;
- la **portée** (*range*)  $a$  : la distance au-delà de laquelle deux points ne se ressemblent plus davantage qu'au hasard : deux points plus éloignés que la portée sont (presque) indépendants.

**Lien avec la covariance.** Si le processus est de variance finie  $C(0)$  et de covariance  $C(h) = \text{Cov}(Z(s), Z(s+h))$ , alors

$$\gamma(h) = \frac{1}{2} \text{Var}(Z(s+h) - Z(s)) = \frac{1}{2}[C(0) + C(0) - 2C(h)] = C(0) - C(h).$$

Le variogramme est donc la covariance « retournée » :  $\gamma$  monte quand  $C$  descend, et le palier vaut  $C(0)$ . Le variogramme est plus général (il existe aussi quand la variance est infinie) et il se **déduit directement des données**, ce qui explique qu'il soit l'outil standard.

**Un calcul à la main.** Quatre mesures alignées aux abscisses 0, 1, 2 et 3 km, de valeurs 2, 3, 5 et 4. À la distance 1 km, il y a trois paires (0-1, 1-2, 2-3) dont les écarts au carré valent  $(3-2)^2 = 1$ ,  $(5-3)^2 = 4$  et  $(4-5)^2 = 1$  :  $\hat{\gamma}(1) = \frac{1+4+1}{2 \times 3} = 1,0$ . À 2 km, deux paires (0-2, 1-3) :  $\frac{(5-2)^2 + (4-3)^2}{2 \times 2} = \frac{9+1}{4} = 2,5$ . À 3 km, une seule paire :  $\hat{\gamma}(3) = \frac{(4-2)^2}{2} = 2,0$ . Le diviseur est **deux fois le nombre de paires**, d'où le « semi » dans le mot.

### 9.3.2 Le variogramme empirique

Avec  $n$  points, l'estimateur classique de la semi-variance à la distance  $h$  est

$$\hat{\gamma}(h) = \frac{1}{2N(h)} \sum_{(i,j) \in N(h)} (z_i - z_j)^2,$$

où  $N(h)$  est l'ensemble des paires dont la distance est « à peu près »  $h$  (on découpe les distances en **classes** de largeur fixe) et  $N(h)$  leur nombre. Le code suivant fabrique cet estimateur. Nous le validons d'abord sur les quatre points ci-dessus.

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats, optimize

def distances(P, Q=None):
    Q = P if Q is None else Q
    return np.sqrt(((P[:, None, :] - Q[None, :, :]) ** 2).sum(axis=2))

def variogramme_empirique(coords, z, largeur=5.0, hmax=50.0, angle=None, tol=22.5):
    """Semi-variogramme empirique par classes de distance. Si `angle` (en degrés, 0 = est, 90 = nord)
```

```

est donné, on ne garde que les paires orientées dans cette direction (à +/- tol degrés près)."""
n = len(z)
i, j = np.triu_indices(n, 1)
h = distances(coords)[i, j]
g = 0.5 * (z[i] - z[j]) ** 2
garde = np.ones(len(h), dtype=bool)
if angle is not None:
    theta = np.degrees(np.arctan2(coords[j, 1] - coords[i, 1], coords[j, 0] - coords[i, 0])) % 180
    ecart = np.abs(theta - angle)
    garde = np.minimum(ecart, 180 - ecart) <= tol
classes = np.arange(0, hmax + largeur, largeur)
lignes = []
for a, b in zip(classes[:-1], classes[1:]):
    m = garde & (h > a) & (h <= b)
    if m.any():
        lignes.append({"h": h[m].mean(), "gamma": g[m].mean(), "paires": int(m.sum())})
return pd.DataFrame(lignes)

# Validation sur les quatre points de l'exemple à la main (classes de 1 km)
P4 = np.array([[0.0, 0], [1, 0], [2, 0], [3, 0]])
z4 = np.array([2.0, 3, 5, 4])
print(variogramme_empirique(P4, z4, largeur=1.0, hmax=3.0).to_string(index=False))

```

| h   | gamma | paires |
|-----|-------|--------|
| 1.0 | 1.0   | 3      |
| 2.0 | 2.5   | 2      |
| 3.0 | 2.0   | 1      |

Les trois lignes redonnent nos calculs à la main : 1,0 ; 2,5 ; 2,0. Passons aux 200 livraisons. Voici d'abord le code qui les a fabriquées, c'est-à-dire la **vérité** que nous allons chercher à retrouver (nous ne la regarderons qu'après l'analyse). C'est un champ aléatoire gaussien stationnaire de covariance exponentielle, observé avec un bruit de mesure.

```

def livraisons(seed=27, n=200, cote=100.0, sill=1.0, a=12.0, pepite=0.4, moyenne=4.0):
    rng = np.random.default_rng(seed)
    g = np.linspace(2, cote - 2, 25) # grille de prédiction 25 x 25
    gx, gy = np.meshgrid(g, g)
    grille = np.column_stack([gx.ravel(), gy.ravel()])
    obs = rng.uniform(0, cote, size=(n, 2))
    pts = np.vstack([obs, grille])
    C = sill * np.exp(-distances(pts) / a) # covariance exponentielle C(h) = sill *
    ↪ exp(-h/a)
    champ = moyenne + np.linalg.cholesky(C + 1e-8 * np.eye(len(pts))) @ rng.normal(size=len(pts))
    z = champ[:n] + rng.normal(0, np.sqrt(pepite), n) # champ + bruit de mesure (la pépité)
    df = pd.DataFrame({"x": np.round(obs[:, 0], 2), "y": np.round(obs[:, 1], 2), "delai_jours":
    ↪ np.round(z, 3)})
    df["pli"] = rng.permutation(np.arange(n) % 5) # 5 pli pour la validation croisée
    verite = pd.DataFrame({"x": grille[:, 0], "y": grille[:, 1], "champ": champ[n:]})
    return df, verite

liv, verite = livraisons()
fichier = pd.read_csv("donnees/ch09-livraisons.csv")
print("identique au fichier fourni :", np.allclose(liv[["x", "y", "delai_jours"]], fichier[["x", "y",
    ↪ "delai_jours"]]))
coords = liv[["x", "y"]].to_numpy()
z = liv["delai_jours"].to_numpy()
print(f"n = {len(z)} | moyenne = {z.mean():.2f} jours | variance = {z.var(ddof=1):.2f}")

```

```

identique au fichier fourni : True
n = 200 | moyenne = 4.27 jours | variance = 1.21

```

Calculons le variogramme empirique de ces données, avec des classes de 5 km jusqu'à 50 km (la moitié de la taille du domaine : au-delà, les paires deviennent rares et dépendent de la forme du

domaine, c'est la règle de pouce usuelle).

```
emp = variogramme_empirique(coords, z, largeur=5.0, hmax=50.0)
print(emp.round(3).to_string(index=False))
```

| h      | gamma | paires |
|--------|-------|--------|
| 3.328  | 0.473 | 139    |
| 7.817  | 0.784 | 384    |
| 12.655 | 1.057 | 683    |
| 17.572 | 1.265 | 849    |
| 22.550 | 1.250 | 992    |
| 27.537 | 1.190 | 1193   |
| 32.569 | 1.150 | 1278   |
| 37.539 | 1.211 | 1237   |
| 42.466 | 1.373 | 1332   |
| 47.408 | 1.426 | 1343   |

Le tableau montre un variogramme qui **monte puis se stabilise** : le comportement attendu d'un processus stationnaire. Remarquons aussi que le nombre de paires croît avec la distance (il y a bien plus de paires éloignées que de paires proches), si bien que les premières classes, justement les plus importantes pour estimer la pépite, sont aussi les moins fiables.

**Le variogramme empirique est bruité.** Chaque point  $\hat{\gamma}(h)$  est une moyenne de carrés d'écarts, un estimateur sensible aux valeurs extrêmes et fortement **corrélé** d'une classe à l'autre (les mêmes points participent à beaucoup de paires). Le choix de la largeur des classes est un compromis : des classes étroites donnent une courbe précise mais bruitée, des classes larges lissent la forme près de l'origine. Il existe des estimateurs robustes (Cressie-Hawkins, qui utilise la racine des écarts absolus), mais l'estimateur classique reste la référence pour des données à peu près gaussiennes.

### 9.3.3 Modèles de variogramme et ajustement

Une courbe en escalier de 10 points ne peut pas servir directement : il faut une **fonction**  $\gamma(h)$  pour toute distance  $h$ , et pas n'importe laquelle : elle doit garantir qu'une variance calculée avec elle n'est jamais négative (on dit que  $-\gamma$  doit être *conditionnellement définie positive*). On ne choisit donc pas une courbe au hasard : on utilise une famille de modèles **valides**. Trois sont standard, avec  $c_0$  la pépite,  $c$  le palier partiel (palier total  $c_0 + c$ ) et  $a$  le paramètre de portée :

| Modèle             | $\gamma(h)$ pour $h > 0$                                                               | Portée pratique           | Allure près de l'origine        |
|--------------------|----------------------------------------------------------------------------------------|---------------------------|---------------------------------|
| <b>Sphérique</b>   | $c_0 + c \left( \frac{3h}{2a} - \frac{h^3}{2a^3} \right)$ si $h < a$ , sinon $c_0 + c$ | $a$ (atteinte exactement) | linéaire                        |
| <b>Exponentiel</b> | $c_0 + c (1 - e^{-h/a})$                                                               | $3a$ (95 % du palier)     | linéaire                        |
| <b>Gaussien</b>    | $c_0 + c (1 - e^{-(h/a)^2})$                                                           | $\sqrt{3}a$               | <b>parabolique</b> (très lisse) |

La **portée pratique** est la distance où le variogramme atteint 95 % de son palier. Elle vaut  $a$  pour le modèle sphérique,  $3a$  pour l'exponentiel (qui n'atteint jamais exactement son palier) et  $\sqrt{3}a$  pour le gaussien. Le comportement près de l'origine traduit la régularité du phénomène : un variogramme linéaire correspond à un champ continu mais rugueux, un variogramme parabolique à un champ très lisse.

Pour ajuster un modèle, on cherche  $(c_0, c, a)$  qui rapprochent le modèle des points empiriques par **moindres carrés pondérés**. Les classes contenant beaucoup de paires sont plus fiables et celles où  $\gamma$  est petit ont une variance plus faible, d'où les poids (de Cressie)  $N(h)/\gamma_{\text{modèle}}(h)^2$  : on minimise  $\sum_k N_k (\hat{\gamma}_k / \gamma_k(\theta) - 1)^2$ .

```

def gamma_sph(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    g = np.where(h < a, c0 + c * (1.5 * h / a - 0.5 * (h / a) ** 3), c0 + c)
    return np.where(h > 0, g, 0.0)

def gamma_exp(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    return np.where(h > 0, c0 + c * (1 - np.exp(-h / a)), 0.0)

def gamma_gau(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    return np.where(h > 0, c0 + c * (1 - np.exp(-(h / a) ** 2)), 0.0)

MODELES = {"sphérique": gamma_sph, "exponentiel": gamma_exp, "gaussien": gamma_gau}
PORTEE_PRATIQUE = {"sphérique": lambda a: a, "exponentiel": lambda a: 3 * a, "gaussien": lambda a:
    ↪ np.sqrt(3) * a}

def ajuster(emp, modele, v0=None):
    """Ajuste (c0, c, a) par moindres carrés pondérés (poids de Cressie) ; renvoie les paramètres et
    ↪ le critère."""
    h, g, N = emp["h"].to_numpy(), emp["gamma"].to_numpy(), emp["paires"].to_numpy()
    v = g.max() if v0 is None else v0
    def residus(p):
        return np.sqrt(N) * (g / np.maximum(modele(h, *p), 1e-9) - 1)
    meilleur = None
    for a0 in (h.max() / 6, h.max() / 3, h.max() / 1.5): # plusieurs départs : le critère
        ↪ n'est pas convexe
        r = optimize.least_squares(residus, x0=[0.1 * v, v, a0], bounds=([0, 1e-6, 1e-3], [10 * v, 10
        ↪ * v, 10 * h.max()]))
        if meilleur is None or r.cost < meilleur.cost:
            meilleur = r
    return meilleur.x, 2 * meilleur.cost

lignes, ajustements = [], {}
for nom, f in MODELES.items():
    p, crit = ajuster(emp, f)
    ajustements[nom] = p
    lignes.append({"modele": nom, "pépité": p[0], "palier_partiel": p[1], "palier_total": p[0] + p[1],
        "param_a": p[2], "portee_pratique_km": PORTEE_PRATIQUE[nom](p[2]), "critere": crit})
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))

```

| modele      | pépité | palier_partiel | palier_total | param_a | portee_pratique_km | critere |
|-------------|--------|----------------|--------------|---------|--------------------|---------|
| sphérique   | 0.215  | 1.065          | 1.280        | 20.792  | 20.792             | 48.096  |
| exponentiel | 0.038  | 1.277          | 1.315        | 8.136   | 24.407             | 47.196  |
| gaussien    | 0.390  | 0.893          | 1.283        | 10.436  | 18.075             | 48.425  |

Les trois modèles proposent un **palier total** voisin (entre 1,28 et 1,32), mais des décompositions différentes entre pépité et portée : le modèle exponentiel trouve presque une pépité nulle et une portée pratique d'environ 24 km, le gaussien une pépité de 0,39 et une portée de 18 km. Leurs **critères** d'ajustement sont très proches (47,2 pour l'exponentiel, 48,1 et 48,4 pour les deux autres) : l'exponentiel est légèrement meilleur, mais **les données ne permettent pas de trancher** entre les formes. Regardons-les sur la figure.

```

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
hh = np.linspace(0.01, 50, 300)
fig, ax = plt.subplots(figsize=(7.4, 4.6))
ax.scatter(emp["h"], emp["gamma"], s=np.sqrt(emp["paires"]) * 3, color="#444444", zorder=3,
    ↪ label="empirique (taille = nb de paires)")
for (nom, f), couleur in zip(MODELES.items(), [AQUA, ORANGE, VIOLET]):
    ax.plot(hh, f(hh, *ajustements[nom]), color=couleur, lw=2, label=nom)
ax.axhline(z.var(ddof=1), color="#999999", lw=0.8, ls="--")
ax.annotate("variance empirique des données", (50, z.var(ddof=1)), xytext=(-4, 4), textcoords="offset
    ↪ points", ha="right", fontsize=8, color="#666666")

```

```
ax.set_xlabel("distance h (km)"); ax.set_ylabel("semi-variance  $\gamma(h)$  (jours2)")
ax.set_title("Variogramme empirique des délais de livraison et trois modèles ajustés")
ax.set_xlim(0, 52); ax.set_ylim(0, None)
ax.legend(frameon=False, loc="lower right", fontsize=8)
plt.tight_layout()
plt.savefig("figures/ch09-variogramme.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

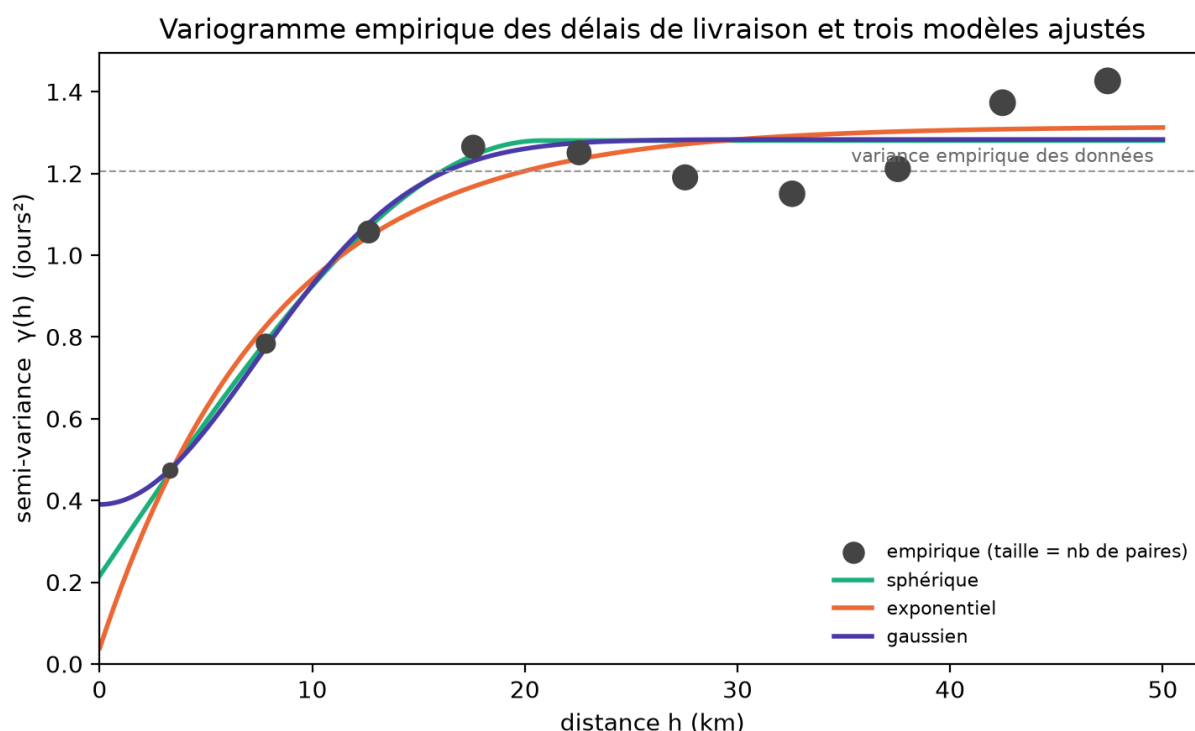


Figure 94: Variogramme empirique (points, taille proportionnelle au nombre de paires) et trois modèles ajustés. La ligne pointillée est la variance empirique des données, voisine du palier.

**La vérité terrain.** Les données sont simulées : nous connaissons la bonne réponse. C'est un champ de covariance exponentielle  $C(h) = 1,0 e^{-h/12}$ , soit un palier partiel  $c = 1,0$  et un paramètre  $a = 12$  (portée pratique 36 km), observé avec une **pépite** de 0,4 (palier total 1,4). Comparons avec la ligne « exponentiel » du tableau : le **palier total** est bien retrouvé (1,32 contre 1,4), mais la **pépite** est très sous-estimée (0,04 contre 0,4) et la **portée pratique** aussi (24 km contre 36 km). Ce qui est facile à estimer, c'est la variance totale ; ce qui est difficile, c'est sa répartition entre bruit de mesure et dépendance spatiale. La suite montre que ce n'est pas une particularité de notre tirage.

**Cet ajustement est-il typique ?** Une seule série de 200 mesures est un seul tirage ; il est légitime de se demander ce qui se passerait avec d'autres tirages du même processus. Faisons l'expérience : nous répétons la simulation avec 30 graines différentes, nous réajustons à chaque fois le modèle exponentiel et nous regardons la dispersion des paramètres estimés.

```
rangs = []
for s in range(100, 130):
    d_s, _ = livraisons(seed=s)
    e_s = variogramme_empirique(d_s[["x", "y"]].to_numpy(), d_s["delai_jours"].to_numpy())
    p, _ = ajuster(e_s, gamma_exp)
```



```

rangs.append({"pepité": p[0], "palier_partiel": p[1], "param_a": p[2], "portee_pratique": 3 *
↪ p[2], "palier_total": p[0] + p[1]})
rangs = pd.DataFrame(rangs)
verite_p = {"pepité": 0.4, "palier_partiel": 1.0, "param_a": 12.0, "portee_pratique": 36.0,
↪ "palier_total": 1.4}
resume = pd.DataFrame({"vérité": pd.Series(verite_p), "médiane": rangs.median(), "10e perc.":
↪ rangs.quantile(0.10), "90e perc.": rangs.quantile(0.90)})
print(resume.round(2).to_string())

```

|                 | vérité | médiane | 10e perc. | 90e perc. |
|-----------------|--------|---------|-----------|-----------|
| pepité          | 0.4    | 0.35    | 0.10      | 0.54      |
| palier_partiel  | 1.0    | 1.04    | 0.78      | 1.38      |
| param_a         | 12.0   | 9.64    | 5.29      | 17.33     |
| portee_pratique | 36.0   | 28.92   | 15.88     | 52.00     |
| palier_total    | 1.4    | 1.38    | 1.05      | 1.63      |

**Ce qu'il faut retenir de cette expérience.** Sur 30 tirages, le palier total est estimé avec une assez bonne précision (médiane 1,38 pour une vérité de 1,4), mais la pépité varie de 0,10 à 0,54 (entre le 10e et le 90e percentile) et la portée pratique de 16 à 52 km, pour des vérités de 0,4 et 36 km. La **médiane** de la portée pratique (29 km) est elle-même inférieure à la vérité : l'ajustement a tendance à la sous-estimer. Notre tirage (pépité 0,04, portée 24 km) est dans la queue basse de la distribution, sans être aberrant. Même avec 200 points, les paramètres du variogramme sont donc **peu précis** : la pépité et la portée sont particulièrement difficiles à identifier, parce que l'une et l'autre se jouent dans les toutes premières classes de distance, qui contiennent peu de paires. Ce n'est pas un défaut de notre code, c'est la nature du problème. Deux conséquences pratiques : (1) ne jamais donner un variogramme ajusté comme un fait certain, (2) comme nous le verrons, le **krigeage** est heureusement assez peu sensible aux petites erreurs de variogramme.

### 9.3.4 Le krigage : la meilleure moyenne pondérée

Nous voulons prédire  $Z(s_0)$  en un point  $s_0$  où nous n'avons pas mesuré. Cherchons un prédicteur de la forme d'une **moyenne pondérée** des  $n$  mesures :

$$\hat{Z}(s_0) = \sum_{i=1}^n \lambda_i Z(s_i), \quad \sum_i \lambda_i = 1.$$

La contrainte  $\sum \lambda_i = 1$  garantit que le prédicteur est **sans biais** quand la moyenne  $\mu$  est constante *mais inconnue* (c'est le **krigeage ordinaire**) :  $\mathbb{E}[\hat{Z}] = \mu \sum \lambda_i = \mu$ . Parmi tous les poids possibles, nous voulons ceux qui **minimisent l'erreur quadratique moyenne**  $\mathbb{E}[(\hat{Z}(s_0) - Z(s_0))^2]$ .

**Dérivation.** Grâce à la contrainte, l'erreur se réécrit  $Z(s_0) - \hat{Z}(s_0) = \sum_i \lambda_i (Z(s_0) - Z(s_i))$  : c'est une combinaison d'**accroissements**, dont on connaît les moments grâce au variogramme. Pour deux accroissements  $a = Z(s_0) - Z(s_i)$  et  $b = Z(s_0) - Z(s_j)$ , on a  $ab = \frac{1}{2}[a^2 + b^2 - (a-b)^2]$  et  $a-b = Z(s_j) - Z(s_i)$ , donc  $\mathbb{E}[ab] = \gamma_{i0} + \gamma_{j0} - \gamma_{ij}$ , en notant  $\gamma_{ij} = \gamma(\|s_i - s_j\|)$ . On en déduit, en utilisant  $\sum \lambda = 1$  :

$$\sigma_K^2(\lambda) = \sum_i \sum_j \lambda_i \lambda_j (\gamma_{i0} + \gamma_{j0} - \gamma_{ij}) = 2 \sum_i \lambda_i \gamma_{i0} - \sum_i \sum_j \lambda_i \lambda_j \gamma_{ij} = 2\lambda^\top \gamma_0 - \lambda^\top \Gamma \lambda,$$

où  $\Gamma = (\gamma_{ij})$  et  $\gamma_0 = (\gamma_{i0})_i$ . On minimise sous la contrainte  $\mathbf{1}^\top \lambda = 1$  avec un **multiplicateur de Lagrange**  $m$  (comme au volume I, section 1.3.5) :  $\mathcal{L} = 2\lambda^\top \gamma_0 - \lambda^\top \Gamma \lambda - 2m(\mathbf{1}^\top \lambda - 1)$ . En annulant le gradient en  $\lambda$  :  $\gamma_0 - \Gamma \lambda - m\mathbf{1} = 0$ . Le système à résoudre est donc

$$\begin{pmatrix} \Gamma & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} \lambda \\ m \end{pmatrix} = \begin{pmatrix} \gamma_0 \\ 1 \end{pmatrix}, \quad \boxed{\hat{Z}(s_0) = \lambda^\top Z}, \quad \boxed{\sigma_K^2 = \lambda^\top \gamma_0 + m}.$$

(Pour la variance :  $\Gamma\lambda = \gamma_0 - m\mathbf{1}$  donne  $\lambda^\top \Gamma\lambda = \lambda^\top \gamma_0 - m$ , donc  $\sigma_K^2 = 2\lambda^\top \gamma_0 - (\lambda^\top \gamma_0 - m) = \lambda^\top \gamma_0 + m$ .)

Trois choses à remarquer dans ce système. (1) Les poids dépendent **des distances entre les points de mesure** (la matrice  $\Gamma$ , qui sait que deux points voisins sont redondants) *et* de la distance de chaque point à la cible ( $\gamma_0$ ) : on n'est pas une simple pondération par l'inverse de la distance. (2) La variance de krigeage  $\sigma_K^2$  ne dépend **que de la géométrie et du variogramme**, pas des valeurs mesurées. (3) Le système est linéaire de taille  $(n + 1)$  : un seul appel à `numpy.linalg.solve`.

**Un exemple à la main.** Prenons le cas le plus simple : une dimension, un variogramme **linéaire**  $\gamma(h) = h$ , deux points de mesure à  $x = 0$  (valeur 10) et  $x = 3$  (valeur 16), et une cible à  $x = 1$ .

Alors  $\Gamma = \begin{pmatrix} 0 & 3 \\ 3 & 0 \end{pmatrix}$  et  $\gamma_0 = (1, 2)^\top$ . Le système s'écrit  $3\lambda_2 + m = 1$ ,  $3\lambda_1 + m = 2$ ,  $\lambda_1 + \lambda_2 = 1$ . En

soustrayant les deux premières équations,  $3(\lambda_1 - \lambda_2) = 1$ , donc  $\lambda_1 = \frac{2}{3}$ ,  $\lambda_2 = \frac{1}{3}$  et  $m = 0$ . La prédiction vaut  $\frac{2}{3} \times 10 + \frac{1}{3} \times 16 = 12$  et la variance  $\lambda^\top \gamma_0 + m = \frac{2}{3} + \frac{1}{3} + 0 = \frac{4}{3}$ . Ce résultat porte un enseignement : **dans ce cas particulier, le krigeage est l'interpolation linéaire** (de 10 à 16 en trois km, soit 12 à 1 km). Vérifions avec une fonction générale.

```
def krigeage_ordinaire(coords, z, cibles, modele, params):
    """Krigage ordinaire : renvoie prédictions, variances de krigeage et poids (une colonne par
    ↪ cible)."""
    n = len(z)
    G = modele(distances(coords), *params)                # Gamma (diagonale nulle)
    A = np.zeros((n + 1, n + 1))
    A[:n, :n] = G
    A[:n, n] = 1.0
    A[n, :n] = 1.0
    g0 = modele(distances(coords, cibles), *params)        # gamma_0, une colonne par cible
    B = np.vstack([g0, np.ones((1, cibles.shape[0]))])
    sol = np.linalg.solve(A, B)
    lam, m = sol[:n], sol[n]
    pred = lam.T @ z
    var = (lam * g0).sum(axis=0) + m
    return pred, var, lam

lineaire = lambda h, pente: np.where(np.asarray(h) > 0, pente * np.asarray(h), 0.0)
pred, var, lam = krigeage_ordinaire(np.array([[0.0, 0], [3, 0]]), np.array([10.0, 16.0]),
    ↪ np.array([[1.0, 0]]), lineaire, (1.0,))
print("poids :", lam.ravel().round(4), "| prédiction :", pred.round(4), "| variance :", var.round(4))
```

```
poids : [0.6667 0.3333] | prédiction : [12.] | variance : [1.3333]
```

Nous retrouvons les poids  $\frac{2}{3}, \frac{1}{3}$ , la prédiction 12 et la variance  $\frac{4}{3}$  du calcul à la main. Revenons aux livraisons, avec le modèle **exponentiel** ajusté. Prédisons d'abord en un point précis, par exemple l'adresse (40, 60), et examinons les poids.

```
mod, par = gamma_exp, ajustements["exponentiel"]
cible = np.array([[40.0, 60.0]])
pred, var, lam = krigeage_ordinaire(coords, z, cible, mod, par)
lam = lam.ravel()
dist = distances(coords, cible).ravel()
ordre = np.argsort(-np.abs(lam))[:6]
print(f"prédiction en (40, 60) : {pred[0]:.2f} jours | écart-type de krigeage : {np.sqrt(var[0]):.2f}
    ↪ jour")
print("somme des poids :", round(lam.sum(), 6), "| poids négatifs :", int((lam < 0).sum()), "sur",
    ↪ len(lam),
```

```
f"| plus petit : {lam.min():.3f} | plus grand : {lam.max():.3f}")
print(pd.DataFrame({"distance_km": dist[ordre], "poids": lam[ordre], "delai":
↳ z[ordre]}).round(3).to_string(index=False))

prédiction en (40, 60) : 4.75 jours | écart-type de krigeage : 0.77 jour
somme des poids : 1.0 | poids négatifs : 29 sur 200 | plus petit : -0.026 | plus grand : 0.532
distance_km  poids  delai
      2.724    0.532    4.753
      4.740    0.158    4.849
      6.420    0.105    3.997
      8.405    0.102    5.156
     11.762    0.054    5.439
     11.745    0.032    3.196
```

Les poids les plus forts reviennent aux livraisons **les plus proches** de la cible (le plus grand, 0,53, est pour celle qui est à 2,7 km), leur somme vaut 1, et 29 des 200 poids sont **négatifs** (mais très petits, comme l'indique le plus petit d'entre eux) : ce n'est pas une anomalie. Quand un point en cache d'autres derrière lui (*effet d'écran*), le krigeage peut donner un poids légèrement négatif au point masqué. Une moyenne ordinaire ne le ferait jamais.

**Le krigeage est un interpolateur exact.** Si la cible est l'un des points de mesure, le système donne le poids 1 à ce point et 0 aux autres : la prédiction redonne la mesure et la variance est nulle. C'est vrai ici même avec une pépite, parce que nous la traitons comme une propriété de la mesure (le point mesuré est connu). Si au contraire la pépite représente un bruit que l'on veut *filtrer*, on utilise une variante qui lisse au lieu d'interpoler (non traitée ici).

### 9.3.5 Une carte de prédiction et une carte d'incertitude

Prédisons maintenant en chacun des 625 points d'une grille régulière (le jeu *verite* contient aussi le **vrai** champ en ces points, grâce à la simulation) et comparons à trois alternatives : la **moyenne globale**, la méthode de l'**inverse de la distance au carré** (IDW, une moyenne pondérée sans variogramme) et le krigeage.

```
grille = verite[["x", "y"]].to_numpy()
pred_k, var_k, _ = krigeage_ordinaire(coords, z, grille, mod, par)

def idw(coords, z, cibles, puissance=2.0):
    d = np.maximum(distances(cibles, coords), 1e-9)
    w = 1.0 / d ** puissance
    return (w * z).sum(axis=1) / w.sum(axis=1)

pred_i = idw(coords, z, grille)
vrai = verite["champ"].to_numpy()
rmse = lambda a, b: float(np.sqrt(np.mean((a - b) ** 2)))
print("RMSE par rapport au VRAI champ sur les 625 points de la grille :")
print(f"  moyenne globale      : {rmse(np.full_like(vrai, z.mean()), vrai):.3f}")
print(f"  inverse de la distance^2 : {rmse(pred_i, vrai):.3f}")
print(f"  krigeage ordinaire       : {rmse(pred_k, vrai):.3f}")
print(f"écart-type de krigeage : de {np.sqrt(var_k.min()):.2f} à {np.sqrt(var_k.max()):.2f} jour
↳ (moyenne {np.sqrt(var_k).mean():.2f})")
```

```
RMSE par rapport au VRAI champ sur les 625 points de la grille :
moyenne globale      : 0.916
inverse de la distance^2 : 0.671
krigeage ordinaire    : 0.645
écart-type de krigeage : de 0.35 à 1.08 jour (moyenne 0.77)
```

Dessignons le vrai champ, la prédiction par krigeage et l'écart-type de krigeage.

```

fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.4))
vmin, vmax = min(vrai.min(), pred_k.min()), max(vrai.max(), pred_k.max())
cartes = [(vrai, "Vrai champ (connu par simulation)", "Oranges", dict(vmin=vmin, vmax=vmax)),
          (pred_k, "Krigeage : prédiction", "Oranges", dict(vmin=vmin, vmax=vmax)),
          (np.sqrt(var_k), "Krigeage : écart-type d'erreur", "Purples", {})]
for ax, (champ, titre, cmap, kw) in zip(axes, cartes):
    im = ax.imshow(champ.reshape(25, 25), origin="lower", extent=(0, 100, 0, 100), cmap=cmap, **kw)
    ax.set_title(titre, fontsize=10)
    ax.set_xlabel("x (km)")
    fig.colorbar(im, ax=ax, shrink=0.78)
    if "écart-type" in titre:
        ax.scatter(coords[:, 0], coords[:, 1], s=5, color="#222222", alpha=0.6)
axes[0].set_ylabel("y (km)")
plt.tight_layout()
plt.savefig("figures/ch09-krigeage.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

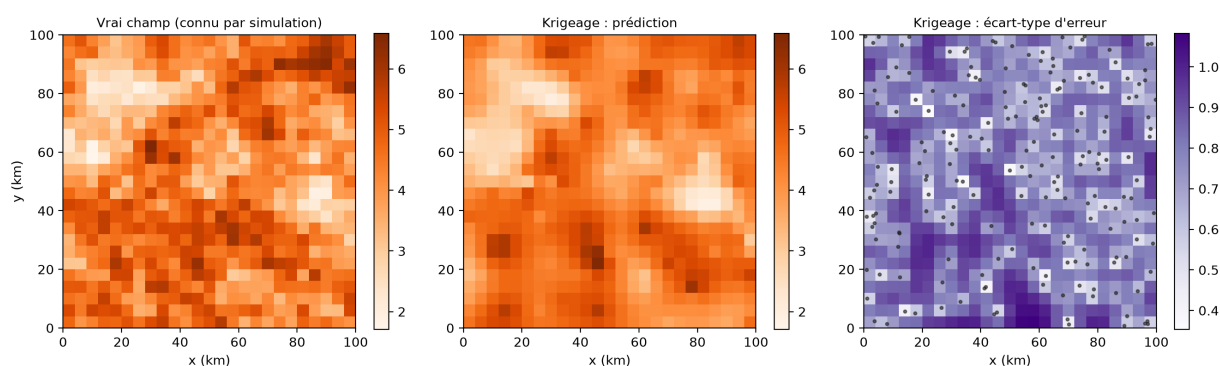


Figure 95: De gauche à droite : le vrai champ de délais (connu car simulé), la prédiction par krigeage ordinaire à partir des 200 mesures, et l'écart-type d'erreur de krigeage (les points noirs sont les 200 livraisons mesurées). L'incertitude est faible près des mesures et monte dans les zones qui en sont éloignées.

Contre le vrai champ, le krigeage obtient une erreur quadratique de 0,645 jour, contre 0,916 pour la moyenne globale (30 % de moins) et 0,671 pour l'inverse de la distance : l>IDW fait presque aussi bien (4 % d'écart), ce qui rappelle qu'une moyenne pondérée bien choisie est déjà un bon prédicteur. L'avantage décisif du krigeage n'est pas là, mais dans la carte de droite. La carte du centre reproduit les grandes structures du vrai champ, en plus lisse (le krigeage est une moyenne : il lisse et sous-estime donc les extrêmes). La carte de droite est une originalité du krigeage : elle dit **où la prédiction est fiable**, avant même d'avoir livré. Dans la pratique, c'est la carte qui guide le choix des **prochains points de mesure** : on va mesurer là où l'incertitude est la plus grande.

De quoi dépend cette incertitude ? Mesurons-le : pour chaque point de la grille, la distance au point de mesure le plus proche, et la façon dont l'écart-type de krigeage évolue avec elle, puis une comparaison entre les points proches d'un bord du domaine et les autres.

```

dmin = distances(grille, coords).min(axis=1) # distance au point mesuré le plus
↳ proche
sd_k = np.sqrt(var_k)
bord = np.minimum.reduce([grille[:, 0], 100 - grille[:, 0], grille[:, 1], 100 - grille[:, 1]])
print("corrélation entre l'écart-type de krigeage et la distance au point le plus proche :",
↳ round(np.corrcoef(sd_k, dmin)[0, 1], 3))
for a_, b_ in [(0, 2), (2, 4), (4, 6), (6, 10)]:
    m = (dmin >= a_) & (dmin < b_)
    print(f" points de grille à {a_}-{b_} km d'une mesure : {int(m.sum()):3d} points, écart-type
↳ moyen {sd_k[m].mean():.2f} jour")

```

```
print(f"écart-type moyen à moins de 8 km d'un bord : {sd_k[bord < 8].mean():.3f} | à l'intérieur :  
↳ {sd_k[bord >= 8].mean():.3f}")
```

```
corrélation entre l'écart-type de krigeage et la distance au point le plus proche : 0.924  
points de grille à 0-2 km d'une mesure : 145 points, écart-type moyen 0.59 jour  
points de grille à 2-4 km d'une mesure : 252 points, écart-type moyen 0.77 jour  
points de grille à 4-6 km d'une mesure : 173 points, écart-type moyen 0.87 jour  
points de grille à 6-10 km d'une mesure : 54 points, écart-type moyen 0.97 jour  
écart-type moyen à moins de 8 km d'un bord : 0.779 | à l'intérieur : 0.770
```

L'écart-type de krigeage est presque une fonction croissante de la distance à la mesure la plus proche (corrélation de 0,92) : de 0,59 jour à moins de 2 km d'une mesure à 0,97 jour entre 6 et 10 km. Aucun **effet de bord** n'apparaît ici (0,78 jour près des bords contre 0,77 à l'intérieur) : avec 200 points répartis uniformément sur le carré, les bords sont aussi bien couverts que le centre. Il se manifesterait avec un échantillonnage plus clairsemé ou concentré au centre, où les points du bord n'auraient de voisins que d'un seul côté.

### 9.3.6 Évaluer honnêtement : la validation croisée

Comparer au vrai champ n'est possible que sur données simulées. Dans une vraie étude, on évalue par **validation croisée** : on retire une partie des mesures, on les prédit avec le reste, et on compare. Notre fichier contient une colonne pli (cinq plis de 40 points). Pour être honnête, il faut **réajuster le variogramme à chaque pli, uniquement sur les données d'entraînement** : sinon, le variogramme aurait « vu » les points à prédire.

```
plis = liv["pli"].to_numpy()
pred_cv = {"moyenne": np.empty(len(z)), "IDW": np.empty(len(z)), "krigeage": np.empty(len(z))}
sig_cv = np.empty(len(z))
for k in range(5):
    tr, te = plis != k, plis == k
    emp_k = variogramme_empirique(coords[tr], z[tr])
    p_k, _ = ajuster(emp_k, gamma_exp) # variogramme réajusté sur
    ↳ l'entraînement
    pk, vk, _ = krigeage_ordinaire(coords[tr], z[tr], coords[te], gamma_exp, p_k)
    pred_cv["krigeage"][te] = pk
    sig_cv[te] = np.sqrt(vk)
    pred_cv["IDW"][te] = idw(coords[tr], z[tr], coords[te])
    pred_cv["moyenne"][te] = z[tr].mean()

tab = pd.DataFrame({nom: {"RMSE (jours)": rmse(p, z), "biais moyen": float(np.mean(p - z))} for nom, p
↳ in pred_cv.items()}).T
print(tab.round(3).to_string())
std_res = (z - pred_cv["krigeage"]) / sig_cv
couvert = np.mean(np.abs(z - pred_cv["krigeage"]) <= 1.96 * sig_cv)
print(f"\nrésidus standardisés du krigeage : moyenne = {std_res.mean():.3f}, variance =  
↳ {std_res.var(ddof=1):.3f} (visé : 0 et 1)")
print(f"couverture des intervalles de krigeage à 95 % : {couvert:.3f}")
```

|          | RMSE (jours) | biais moyen |
|----------|--------------|-------------|
| moyenne  | 1.097        | 0.000       |
| IDW      | 0.994        | 0.040       |
| krigeage | 0.969        | 0.023       |

```
résidus standardisés du krigeage : moyenne = -0.021, variance = 1.710 (visé : 0 et 1)
couverture des intervalles de krigeage à 95 % : 0.895
```

Trois lectures. (1) Les trois méthodes sont comparées sur **les mêmes** 200 prédictions : le krigeage a la plus petite erreur (0,969 jour), devant l>IDW (0,994), elle-même meilleure que la moyenne globale (1,097). L'avantage du krigeage sur l>IDW est mince (2,5 %) : sur ce jeu, la qualité de la prédiction ponctuelle ne départage pas nettement les deux. (2) Le krigeage fournit en plus un

**écart-type d'erreur** : on le contrôle avec les résidus *standardisés*  $(z - \hat{z}) / \sigma_K$ , qui devraient avoir une moyenne proche de 0 et une variance proche de 1 si les barres d'erreur sont bien calibrées, et avec la **couverture** des intervalles  $\hat{z} \pm 1,96 \sigma_K$ . La moyenne des résidus standardisés est bien proche de 0 ( $-0,02$ ), mais leur **variance est de 1,71** et la couverture n'est que de 89,5 % au lieu de 95 %. (3) Les barres d'erreur sont donc **trop optimistes** : trop étroites d'un facteur  $\sqrt{1,71} \approx 1,3$ . La cause est celle que nous avons vue à l'ajustement : un variogramme estimé avec une pépite trop faible (0,04 au lieu de 0,4) annonce une prédiction plus précise qu'elle ne l'est. Le krigeage donne de **bonnes prédictions** même avec un variogramme approximatif, mais ses **barres d'erreur**, elles, héritent de toute l'incertitude du variogramme.

**Remarque sur la mesure d'erreur.** Notre cible, dans cette validation, est la **mesure bruitée**  $Z$  (ce que l'on observerait), qui contient la pépite. Une partie de l'erreur est donc irréductible, quelle que soit la qualité du modèle : même un krigeage parfait ne peut pas prédire le bruit de mesure. C'est pour cela que le RMSE de la validation croisée est supérieur à celui calculé plus haut contre le champ sans bruit.

**Une alternative sans variogramme : un lissage par spline.** On peut aussi prédire le délai comme une fonction lisse  $f(x, y)$  des coordonnées, par une **spline de plaque mince**, comme le fait `mgcv` en R (le modèle additif généralisé de la section 2.5 de ce volume). Voici le même protocole de validation croisée, avec les mêmes plis :

```
library(mgcv)
d <- read.csv("donnees/ch09-livraisons.csv")
pred <- numeric(nrow(d))
for (k in 0:4) {
  tr <- d[d$pli != k, ]
  te <- d[d$pli == k, ]
  g <- gam(delai_jours ~ s(x, y, k = 60), data = tr, method = "REML")
  pred[d$pli == k] <- predict(g, newdata = te)
}
cat("RMSE par validation croisée (spline de plaque mince, mgcv) :", round(sqrt(mean((d$delai_jours -
  ↪ pred)^2)), 3), "\n")
cat("biais moyen :", round(mean(pred - d$delai_jours), 3), "\n")
```

```
Loading required package: nlme
This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
RMSE par validation croisée (spline de plaque mince, mgcv) : 0.974
biais moyen : 0.009
```

La spline obtient une erreur de 0,974 jour, quasiment identique à celle du krigeage (0,969), ce qui n'a rien de surprenant : les deux sont des **lisseurs linéaires** du même type, et le krigeage avec variogramme exponentiel a une interprétation proche. Les avantages propres du krigeage sont (1) un **modèle explicite de la dépendance** (pépite, portée, que l'on peut interpréter), (2) des **variances de prédiction natives** (la spline en fournit aussi, par ses erreurs-types, avec d'autres hypothèses).

### 9.3.7 Les limites : tendance, anisotropie, extrapolation

**Si la moyenne n'est pas constante.** Le variogramme présuppose un processus stationnaire. Si la moyenne varie dans l'espace (une **tendance** : par exemple, les délais augmentent avec l'éloignement du dépôt), le variogramme empirique **monte sans jamais se stabiliser**, parce que les paires éloignées diffèrent aussi par leur niveau moyen. Illustrons-le en ajoutant aux délais une tendance linéaire de 0,04 jour par km vers l'est, puis en l'ôtant par régression avant de calculer le variogramme.



```

z_tend = z + 0.04 * coords[:, 0] # tendance ajoutée : +0,04 jour/km
↳ vers l'est
X = np.column_stack([np.ones(len(z)), coords])
beta = np.linalg.lstsq(X, z_tend, rcond=None)[0]
z_res = z_tend - X @ beta # résidus de la régression sur (x, y)
e_tend = variogramme_empirique(coords, z_tend, largeur=5.0, hmax=70.0)
e_res = variogramme_empirique(coords, z_res, largeur=5.0, hmax=70.0)
e_ref = variogramme_empirique(coords, z, largeur=5.0, hmax=70.0)
print("tendance estimée par régression : ", beta.round(3), "(constante, pente en x, pente en y)")
print(pd.DataFrame({"h": e_ref["h"].round(1), "sans_tendance": e_ref["gamma"], "avec_tendance":
↳ e_tend["gamma"],
                    "tendance_retiree": e_res["gamma"]}).round(2).iloc[[1, 3, 5, 7, 9, 11,
↳ 13]].to_string(index=False))

```

```

tendance estimée par régression : [ 4.325e+00  4.200e-02 -3.000e-03] (constante, pente en x, pente en
↳ y)
   h  sans_tendance  avec_tendance  tendance_retiree
7.8      0.78         0.84         0.78
17.6     1.27         1.41         1.27
27.5     1.19         1.50         1.19
37.5     1.21         1.79         1.22
47.4     1.43         2.36         1.42
57.4     1.35         2.64         1.35
67.4     1.21         3.29         1.19

```

**Si le phénomène dépend de la direction (anisotropie) :** un délai peut varier plus vite le long d'un axe routier que perpendiculairement. On le détecte en calculant des variogrammes **directionnels** (en ne gardant que les paires orientées dans une direction, à une tolérance d'angle près). S'ils diffèrent, un variogramme unique est inadapté.

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))
ax1.plot(e_ref["h"], e_ref["gamma"], "o-", color=AQUA, label="sans tendance")
ax1.plot(e_tend["h"], e_tend["gamma"], "o-", color=ROUGE, label="avec une tendance linéaire non
↳ retirée")
ax1.plot(e_res["h"], e_res["gamma"], "o-", color=BLEU, label="tendance retirée par régression")
ax1.set_xlabel("distance h (km)"); ax1.set_ylabel("semi-variance (jours²)"); ax1.set_title("Une
↳ tendance fait « monter » le variogramme")
ax1.legend(frameon=False, fontsize=8, loc="upper left")
for angle, couleur, nom in [(0, ORANGE, "est-ouest (0°)"), (90, VIOLET, "nord-sud (90°)"]):
    e_dir = variogramme_empirique(coords, z, largeur=7.5, hmax=45.0, angle=angle, tol=22.5)
    ax2.plot(e_dir["h"], e_dir["gamma"], "o-", color=couleur, label=nom)
e_om = variogramme_empirique(coords, z, largeur=7.5, hmax=45.0)
ax2.plot(e_om["h"], e_om["gamma"], "--", color="#444444", label="toutes directions")
ax2.set_xlabel("distance h (km)"); ax2.set_ylabel("semi-variance (jours²)");
↳ ax2.set_title("Variogrammes directionnels")
ax2.legend(frameon=False, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("figures/ch09-limites-variogramme.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

À gauche, la tendance non retirée fait monter le variogramme bien au-delà du palier (de 0,84 à 3,29 entre 8 et 67 km), alors que le retrait de la tendance par régression redonne une courbe presque identique à celle sans tendance. La régression retrouve d'ailleurs la tendance ajoutée : une pente de 0,042 jour par km vers l'est pour 0,04 en vérité, et une pente nulle (−0,003) vers le nord. À droite, les deux directions **ne sont pas superposées** : la courbe nord-sud est au-dessus de la courbe est-ouest à toutes les distances. Faut-il y voir de l'**anisotropie** ? Avec des données réelles, on ne peut pas le savoir d'emblée : des variogrammes directionnels sont calculés sur des secteurs d'angle étroits, donc sur **moins de paires**, et sont bruités. La bonne démarche est de **calibrer** ce que le hasard seul produit. Résumons l'écart par un rapport moyen  $\gamma_{N-S}/\gamma_{E-O}$  sur les classes de



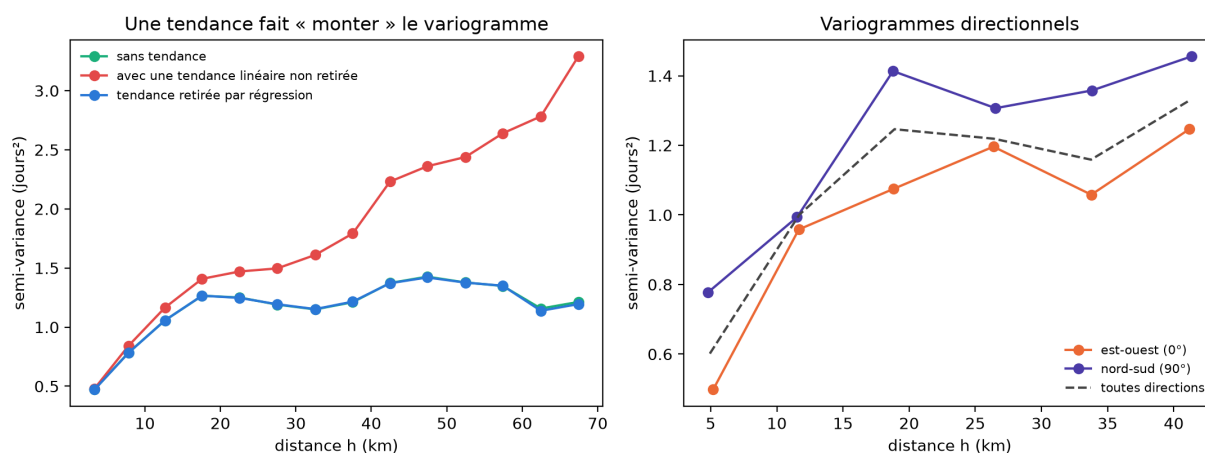


Figure 96: À gauche : variogramme des délais sans tendance (vert), avec une tendance linéaire ajoutée (rouge, qui ne se stabilise pas), et après retrait de la tendance par régression (bleu). À droite : variogrammes directionnels est-ouest et nord-sud, comparés au variogramme toutes directions confondues.

distance, puis calculons le même rapport sur 60 jeux simulés **isotropes** (le même processus, 60 autres graines).

```
def rapport_ns_eo(c, zz):
    ns_ = variogramme_empirique(c, zz, largeur=7.5, hmax=45.0, angle=90, tol=22.5)["gamma"].to_numpy()
    eo_ = variogramme_empirique(c, zz, largeur=7.5, hmax=45.0, angle=0, tol=22.5)["gamma"].to_numpy()
    m_ = min(len(ns_), len(eo_))
    return float(np.mean(ns_[:m_] / eo_[:m_]))

r_obs = rapport_ns_eo(coords, z)
r_sim = []
for s_ in range(100, 160):
    d_s, _ = livraisons(seed=s_)
    r_sim.append(rapport_ns_eo(d_s[["x", "y"]].to_numpy(), d_s["delai_jours"].to_numpy()))
r_sim = np.array(r_sim)
print(f"rapport N-S / E-0 observé : {r_obs:.3f}")
print(f"rapport sur 60 jeux isotropes : médiane {np.median(r_sim):.3f}, 10e-90e percentiles
↳ {np.percentile(r_sim, 10):.3f} - {np.percentile(r_sim, 90):.3f}")
print(f"part des jeux isotropes dont le rapport est au moins aussi grand : {np.mean(r_sim >=
↳ r_obs):.3f}")
```

```
rapport N-S / E-0 observé : 1.243
rapport sur 60 jeux isotropes : médiane 1.020, 10e-90e percentiles 0.908 - 1.167
part des jeux isotropes dont le rapport est au moins aussi grand : 0.050
```

Le rapport observé (1,24) est dans la queue haute de ce que le hasard produit sous isotropie (médiane 1,02 ; 80 % des jeux entre 0,91 et 1,17) : environ 5 % des jeux isotropes l'atteignent ou le dépassent. C'est donc un cas **limite** : trop marqué pour être banal, pas assez pour conclure. Nous *savons* ici, par construction, que le champ est isotrope : l'écart observé est un accident de ce tirage. Avec des données réelles, on aurait hésité, et la réponse prudente serait de comparer des modèles avec et sans anisotropie (par exemple par validation croisée) plutôt que de se fier à un graphique.

#### Quatre mises en garde.

1. **Stationnarité.** Avec une tendance, il faut la modéliser (krigeage universel : tendance par régression et résidu krigé) ou la retirer. Retirer une tendance par régression ordinaire peut sous-estimer légèrement la semi-variance aux grandes distances (ici, la

courbe des résidus est presque confondue avec celle du processus sans tendance).

2. **Anisotropie.** Vérifiez-la sur des variogrammes directionnels avant de supposer qu'une seule portée convient.
3. **Extrapolation.** Le krigeage est un **interpolateur** : loin de toute mesure, ou hors de l'enveloppe des points mesurés, la prédiction revient vers la moyenne et la variance monte vers le palier (nous venons de voir l'écart-type croître avec la distance à la mesure la plus proche). Ne promettez pas un délai dans une zone qu'aucune mesure n'entoure.
4. **Variogramme estimé, non connu.** La variance de krigeage suppose le variogramme **connu**. Comme nous l'avons vu à l'expérience des 30 graines, il est en réalité estimé avec une grande incertitude : l'écart-type de krigeage est donc plutôt optimiste, ce que la validation croisée du 9.3.6 a confirmé (variance des résidus standardisés de 1,71 au lieu de 1).

#### À retenir.

- Le **semi-variogramme**  $\gamma(h) = \frac{1}{2} \mathbb{E}[(Z(s+h) - Z(s))^2]$  décrit comment la ressemblance s'estompe avec la distance ; pour un processus de variance finie,  $\gamma(h) = C(0) - C(h)$ . Ses paramètres : **pépité** (bruit et micro-variations), **palier** (variance totale), **portée** (distance de décorrélation).
- On estime  $\gamma$  par classes de distance (**variogramme empirique**, avec le nombre de paires), puis on **ajuste un modèle valide** (sphérique, exponentiel, gaussien) par moindres carrés pondérés. Pépité et portée sont peu précises, même avec 200 points.
- Le **krigeage ordinaire** est le meilleur prédicteur linéaire sans biais : on résout  $\begin{pmatrix} \Gamma & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix} \begin{pmatrix} \lambda \\ m \end{pmatrix} = \begin{pmatrix} \gamma_0 \\ 1 \end{pmatrix}$ , la prédiction est  $\lambda^\top \mathbf{z}$  et la variance  $\lambda^\top \gamma_0 + m$ .
- On le **valide** par validation croisée *avec réajustement du variogramme à chaque pli*, et on contrôle les barres d'erreur par les résidus standardisés et la couverture.
- Limites : **stationnarité** (tendance), **anisotropie**, **extrapolation**, et un variogramme qui est une estimation, pas une vérité.

## 9.4 Processus ponctuels

**Intuition.** Jusqu'ici, les lieux étaient donnés (les 144 délégations, les adresses de livraison choisies par les clients) et c'était la **valeur** qui était aléatoire. Maintenant, ce sont les **positions** elles-mêmes qui sont le phénomène : où habitent les clients de la boutique de la médina ? Sont-ils éparpillés au hasard, **regroupés** en quartiers (une publicité, un bouche-à-oreille) ou étonnamment **bien espacés** (chacun choisit une adresse à distance des autres) ? Pour répondre, on compare ce que l'on observe à ce que produirait le **hasard complet**.

### 9.4.1 Le hasard complet : le processus de Poisson spatial

Pour reconnaître un écart au hasard, il faut d'abord définir le hasard. Un semis de points est de **hasard spatial complet** (en anglais *complete spatial randomness*, CSR) quand il est un **processus de Poisson homogène** d'intensité  $\lambda$  (nombre moyen de points par  $\text{km}^2$ ) :

1. le nombre de points dans une région  $B$  suit une loi de Poisson de moyenne  $\lambda |B|$ , où  $|B|$  est l'aire de la région ;
2. les nombres de points dans des régions **disjointes** sont indépendants ;
3. conditionnellement au nombre total de points  $n$  dans la fenêtre, les points sont **indépen-**

**dants et uniformes** sur la fenêtre.

La propriété 3 est la plus utile en pratique : fabriquer un semis aléatoire sur une fenêtre, c'est tirer  $n$  points uniformes. Le CSR est notre **hypothèse nulle** : aucune attraction, aucune répulsion entre points, et la même densité partout. Les deux écarts typiques sont l'**agrégat** (points groupés en paquets) et la **régularité** (points qui se repoussent, avec une distance minimale entre eux).

Fabriquons trois semis de la même fenêtre de  $10 \times 10$  km ( $100 \text{ km}^2$ ) autour de la boutique, de 75 à 100 adresses de clients chacun (le nombre de points de l'agrégé est lui-même aléatoire) :

- un semis **aléatoire** (CSR) de 100 points uniformes ;
- un semis **agrégé**, par un processus de **Thomas** : des « centres » (des quartiers, des abonnés d'un même influenceur) tombent au hasard, puis chaque centre engendre un nombre de clients de loi de Poisson, répartis autour de lui selon une loi normale ;
- un semis **régulier**, par **inhibition séquentielle** : on tire des points uniformes l'un après l'autre en refusant ceux qui tomberaient à moins de 0,7 km d'un point déjà accepté.

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

COTE = 10.0 # fenêtre carrée de 10 km de côté
AIRE = COTE ** 2 # 100 km²

def distances(P, Q=None):
    Q = P if Q is None else Q
    return np.sqrt(((P[:, None, :] - Q[None, :, :]) ** 2).sum(axis=2))

def semis_poisson(n, rng):
    """CSR conditionnel à n : n points indépendants et uniformes."""
    return rng.uniform(0, COTE, size=(n, 2))

def semis_thomas(kappa, mu, sigma, rng):
    """Agrégat de Thomas : centres de Poisson (densité kappa), mu descendants en moyenne, étalement
    ↪ sigma."""
    marge = 3 * sigma # on génère aussi des centres hors fenêtre
    n_centres = rng.poisson(kappa * (COTE + 2 * marge) ** 2)
    centres = rng.uniform(-marge, COTE + marge, size=(n_centres, 2))
    nb = rng.poisson(mu, size=n_centres)
    pts = np.repeat(centres, nb, axis=0) + rng.normal(0, sigma, size=(nb.sum(), 2))
    return pts[(pts >= 0) & (pts <= COTE)].all(axis=1) # on ne garde que ce qui tombe dans
    ↪ la fenêtre

def semis_inhibition(n, rmin, rng):
    """Inhibition séquentielle simple : points uniformes refusés s'ils sont à moins de rmin d'un point
    ↪ accepté."""
    pts = []
    while len(pts) < n:
        p = rng.uniform(0, COTE, size=2)
        if not pts or (np.hypot(*(np.array(pts) - p).T) >= rmin).all():
            pts.append(p)
    return np.array(pts)

rng = np.random.default_rng(2025)
semis = {
    "aléatoire (CSR)": semis_poisson(100, rng),
    "agrégé (Thomas)": semis_thomas(kappa=0.08, mu=12, sigma=0.4, rng=rng),
    "régulier (inhibition)": semis_inhibition(100, 0.7, rng),
}
for nom, pts in semis.items():
```

```

print(f"{nom:24s} n = {len(pts):3d} points (intensité estimée {len(pts) / AIRE:.2f} par km²)")

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.3))
for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
    ax.scatter(pts[:, 0], pts[:, 1], s=16, color=couleur, edgecolor="white", linewidth=0.4)
    ax.set_xlim(0, COTE); ax.set_ylim(0, COTE); ax.set_aspect("equal")
    ax.set_title(f"{nom} : {len(pts)} adresses", fontsize=10)
    ax.set_xlabel("x (km)")
axes[0].set_ylabel("y (km)")
plt.tight_layout()
plt.savefig("figures/ch09-semis.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

```

aléatoire (CSR)          n = 100 points (intensité estimée 1.00 par km²)
agrégé (Thomas)         n = 75 points (intensité estimée 0.75 par km²)
régulier (inhibition)   n = 100 points (intensité estimée 1.00 par km²)
figure enregistrée

```

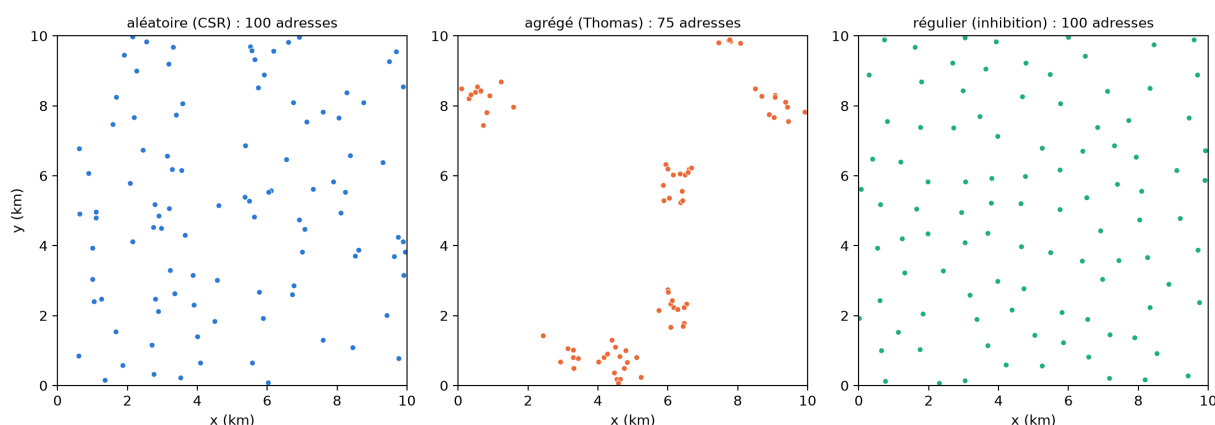


Figure 97: Trois semis de points dans une fenêtre de 10 km × 10 km : hasard complet (gauche), agrégat de Thomas (centre), régulier par inhibition (droite).

Les différences se voient à l'œil : des paquets et des vides au centre, un espacement très uniforme à droite, et au hasard complet à gauche un mélange de petits groupes et de vides qui surprend : **le hasard n'est pas régulier**. C'est un piège classique : nous avons tendance à voir de l'agrégation dans un semis tout à fait aléatoire, parce que des points qui tombent par hasard près les uns des autres nous sautent aux yeux. D'où le besoin de tests.

### 9.4.2 Le test des quadrats

La méthode la plus simple : quadriller la fenêtre en  $m$  cases égales et **compter** les points dans chaque case. Sous le CSR, conditionnellement au nombre  $n$  de points, les comptages suivent une loi multinomiale de probabilités égales : chaque case a la même moyenne  $\bar{c} = n/m$ , et l'on s'attend à une variance voisine de la moyenne (propriété de la loi de Poisson). On forme la statistique de Pearson

$$\chi^2 = \sum_{k=1}^m \frac{(c_k - \bar{c})^2}{\bar{c}}, \quad \text{approximativement } \chi_{m-1}^2 \text{ sous le CSR}$$

(une approximation raisonnable quand  $\bar{c} \geq 5$  environ). Rappelons qu'elle vaut  $(m - 1)$  fois l'**indice de dispersion**  $\text{VMR} = s^2/\bar{c}$  (variance divisée par moyenne). Une valeur **grande** de  $\chi^2$

indique de l'**agrégation** (certaines cases beaucoup plus peuplées que d'autres), une valeur **petite** de la **régularité** (comptages trop uniformes).

**Un exemple à la main.** Quatre cases contenant 1, 1, 1 et 9 points :  $\bar{c} = 3$ , donc  $\chi^2 = \frac{(-2)^2 + (-2)^2 + (-2)^2 + 6^2}{3} = \frac{12+36}{3} = 16$  avec 3 degrés de liberté : bien au-delà de ce que le hasard produit (la valeur critique à 5 % est 7,81). Quatre cases à 3 points chacune :  $\chi^2 = 0$ , une régularité parfaite. La statistique mesure donc l'inégalité des comptages.

```
def comptages(pts, m=4):
    """Comptages par case d'un quadrillage m x m de la fenêtre."""
    bornes = np.linspace(0, COTE, m + 1)
    H, _, _ = np.histogram2d(pts[:, 0], pts[:, 1], bins=[bornes, bornes])
    return H.ravel()

def test_quadrats(pts, m=4):
    c = comptages(pts, m)
    chi2 = ((c - c.mean()) ** 2).sum() / c.mean()
    ddl = len(c) - 1
    return {"chi2": chi2, "ddl": ddl, "VMR": c.var(ddof=1) / c.mean(),
            "p_agregat (chi2 grand)": stats.chi2.sf(chi2, ddl), "p_regulier (chi2 petit)":
            stats.chi2.cdf(chi2, ddl)}

# l'exemple à la main
print("exemple (1, 1, 1, 9) :", round(((np.array([1, 1, 1, 9]) - 3) ** 2).sum() / 3, 2), "| valeur
↳ critique chi2(3) à 5 % :", round(stats.chi2.ppf(0.95, 3), 2))
res = pd.DataFrame({nom: test_quadrats(pts) for nom, pts in semis.items()}).T
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 160):
    print(res.to_string())
print("\ncomptages par case du semis agrégé (4 x 4) :")
print(comptages(semis["agrégé (Thomas)"]).reshape(4, 4).astype(int))
```

```
exemple (1, 1, 1, 9) : 16.0 | valeur critique chi2(3) à 5 % : 7.81
               chi2  ddl   VMR  p_agregat (chi2 grand)  p_regulier (chi2 petit)
aléatoire (CSR)    8.8   15  0.5867                    0.8877                0.1123
agrégé (Thomas)  143.7  15  9.578                    4.342e-23                1
régulier (inhibition)  4   15  0.2667                    0.9977               0.002263
```

```
comptages par case du semis agrégé (4 x 4) :
[[ 1  0  1 11]
 [19  0  0  0]
 [12  2 14  1]
 [ 0  0  0 14]]
```

Le test distingue bien les trois cas : le semis **agrégé** a une statistique très élevée (VMR bien supérieur à 1, p-valeur d'agrégation minuscule), le semis **régulier** une statistique très faible (VMR très inférieur à 1, p-valeur de régularité minuscule) et le semis **aléatoire** une valeur banale.

**Les limites du test des quadrats.** (1) Le résultat dépend de la **taille des cases** : un agrégat de 1 km de diamètre passe inaperçu avec des cases de 5 km. Le test est un test à **une échelle**. (2) Il ne retient que les comptages et jette l'information de **position** à l'intérieur de chaque case. (3) La loi du  $\chi^2$  n'est qu'une approximation quand les comptages sont petits. Les méthodes à base de **distances**, qui suivent, n'ont pas ces défauts.

### 9.4.3 Le plus proche voisin : l'indice de Clark-Evans

Pour chaque point, notons  $d_i$  la distance à son **plus proche voisin**. Sous le CSR d'intensité  $\lambda$ , la probabilité que le plus proche voisin d'un point soit **au-delà** de la distance  $r$  est celle de ne trouver **aucun** point dans un disque d'aire  $\pi r^2$ , soit (loi de Poisson de moyenne  $\lambda \pi r^2$ , valeur en 0) :

$$\mathbb{P}(d > r) = e^{-\lambda\pi r^2} \implies G(r) = \mathbb{P}(d \leq r) = 1 - e^{-\lambda\pi r^2}.$$

On en déduit la distance moyenne au plus proche voisin,  $\mathbb{E}[d] = \int_0^\infty e^{-\lambda\pi r^2} dr = \frac{1}{2\sqrt{\lambda}}$ , et sa variance,  $\text{Var}(d) = \frac{4 - \pi}{4\pi\lambda}$ . L'**indice de Clark-Evans** compare la distance moyenne observée à celle du hasard :

$$R = \frac{\bar{d}_{\text{obs}}}{1/(2\sqrt{\hat{\lambda}})}, \quad \hat{\lambda} = \frac{n}{A},$$

avec  $R = 1$  pour le hasard,  $R < 1$  pour de l'aggrégation (les voisins sont plus proches que prévu),  $R > 1$  pour de la régularité. Comme  $\bar{d}$  est une moyenne de  $n$  distances (supposées à peu près indépendantes), on obtient un test  $z = (\bar{d} - \mathbb{E}[d]) / \sqrt{\text{Var}(d)/n}$  approximativement normal.

**L'effet de bord.** Cette formule suppose un plan infini. Dans une **fenêtre finie**, un point près du bord a moins de voisins possibles (il n'y a rien au-delà), donc son plus proche voisin est en moyenne plus loin :  $\bar{d}$  est **surestimé** et  $R$  est biaisé vers le haut, ce qui peut faire conclure à tort à une régularité. Donnelly (1978) a proposé une correction pour une fenêtre de périmètre  $P$  :  $\mathbb{E}[\bar{d}] = 0,5\sqrt{A/n} + (0,0514 + 0,041/\sqrt{n})P/n$  et  $\text{Var}(\bar{d}) = 0,070A/n^2 + 0,037P\sqrt{A/n^5}$ . Au lieu de la croire sur parole, **vérifions-la par simulation** : nous simulons 4 000 semis CSR de 100 points dans notre fenêtre et nous comparons la moyenne et l'écart-type de  $\bar{d}$  aux deux formules.

```
def plus_proches(pts):
    D = distances(pts)
    np.fill_diagonal(D, np.inf)
    return D.min(axis=1)

rng = np.random.default_rng(1)
n0 = 100
dbar = np.array([plus_proches(semis_poisson(n0, rng)).mean() for _ in range(4000)])
P = 4 * COTE
naif = 0.5 * np.sqrt(AIRE / n0)
don = 0.5 * np.sqrt(AIRE / n0) + (0.0514 + 0.041 / np.sqrt(n0)) * P / n0
sd_naif = np.sqrt((4 - np.pi) / (4 * np.pi * n0 * (n0 / AIRE)))
sd_don = np.sqrt(0.070 * AIRE / n0 ** 2 + 0.037 * P * np.sqrt(AIRE / n0 ** 5))
print(f"{'':28s}{'moyenne de d':>14s}{'écart-type de d':>18s}")
print(f"{'simulation (4000 semis CSR)':28s}{dbar.mean():14.4f}{dbar.std():18.4f}")
print(f"{'formule naïve (plan infini)':28s}{naif:14.4f}{sd_naif:18.4f}")
print(f"{'formule de Donnelly':28s}{don:14.4f}{sd_don:18.4f}")
```

|                             | moyenne de d | écart-type de d |
|-----------------------------|--------------|-----------------|
| simulation (4000 semis CSR) | 0.5224       | 0.0288          |
| formule naïve (plan infini) | 0.5000       | 0.0261          |
| formule de Donnelly         | 0.5222       | 0.0291          |

La formule naïve sous-estime la moyenne (0,500 contre 0,523 simulé : près de 5 % d'écart) et l'écart-type ; celle de Donnelly colle à la simulation. Appliquons l'indice de Clark-Evans aux trois semis avec les deux versions, et ajoutons une troisième voie, qui évite toute formule : une **p-valeur de Monte-Carlo**. On simule  $B = 999$  semis CSR de même taille dans la **même fenêtre** et l'on regarde où tombe notre  $\bar{d}$  : l'effet de bord est automatiquement pris en compte, puisque les semis simulés le subissent aussi.

```
def clark_evans(pts, B=999, seed=7):
    n = len(pts)
    d = plus_proches(pts).mean()
    R_naif = d / (0.5 * np.sqrt(AIRE / n))
    E_don = 0.5 * np.sqrt(AIRE / n) + (0.0514 + 0.041 / np.sqrt(n)) * P / n
    s_don = np.sqrt(0.070 * AIRE / n ** 2 + 0.037 * P * np.sqrt(AIRE / n ** 5))
    z_don = (d - E_don) / s_don
    rng = np.random.default_rng(seed)
    sim = np.array([plus_proches(semis_poisson(n, rng)).mean() for _ in range(B)])
    p_mc = 2 * (1 + min((sim <= d).sum(), (sim >= d).sum())) / (B + 1) # bilatérale
    return {"n": n, "d_moyen": d, "R (naïf)": R_naif, "R (Donnelly)": d / E_don, "z (Donnelly)": z_don,
           "p (Donnelly)": 2 * stats.norm.sf(abs(z_don)), "p (Monte-Carlo)": min(1.0, p_mc)}

res = pd.DataFrame({nom: clark_evans(pts) for nom, pts in semis.items()}).T
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 170):
    print(res.to_string())
```

|                       | n   | d_moyen | R (naïf) | R (Donnelly) | z (Donnelly) | p (Donnelly) | p (Monte-Carlo) |
|-----------------------|-----|---------|----------|--------------|--------------|--------------|-----------------|
| aléatoire (CSR)       | 100 | 0.5353  | 1.071    | 1.025        | 0.4506       | 0.6523       | 0.628           |
| agrégé (Thomas)       | 75  | 0.2026  | 0.3509   | 0.3336       | -10.28       | 8.252e-25    | 0.002           |
| régulier (inhibition) | 100 | 0.829   | 1.658    | 1.587        | 10.53        | 6.024e-26    | 0.002           |

Les trois voies s'accordent pour le semis agrégé (R très inférieur à 1) et pour le semis régulier (R supérieur à 1). Pour le semis aléatoire, le R naïf vaut un peu plus de 1 et pourrait faire croire à une légère régularité : c'est le biais de bord. Avec la correction de Donnelly, il est ramené près de 1.

**La fonction G entière.** L'indice de Clark-Evans résume toutes les distances par leur moyenne. On peut regarder la **fonction de répartition empirique** de  $d_i$ ,  $\hat{G}(r) = \frac{1}{n} \# \{i : d_i \leq r\}$ , et la comparer à la courbe théorique  $1 - e^{-\lambda \pi r^2}$ , avec une **enveloppe de Monte-Carlo** : un bandeau qui contient 95 % des courbes obtenues sur des semis CSR simulés. Si notre courbe sort du bandeau, le hasard complet est mis en défaut. Une courbe  $\hat{G}$  qui **monte plus vite** que le hasard indique des voisins plus proches que prévu (agrégat) ; une courbe qui monte **plus lentement** indique de la répulsion.

```
def G_emp(pts, rs):
    d = plus_proches(pts)
    return np.array([(d <= r).mean() for r in rs])

rs_G = np.linspace(0, 1.6, 81)
rng = np.random.default_rng(11)
fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.9), sharey=True)
lignes = []
for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
    n = len(pts)
    sims = np.array([G_emp(semis_poisson(n, rng), rs_G) for _ in range(499)])
    bas, haut = np.percentile(sims, [2.5, 97.5], axis=0)
    theorique = 1 - np.exp(-(n / AIRE) * np.pi * rs_G ** 2)
    ax.fill_between(rs_G, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %)")
    ax.plot(rs_G, theorique, color="#555555", lw=1, ls="--", label="théorie (plan infini)")
    ax.plot(rs_G, G_emp(pts, rs_G), color=couleur, lw=2.2, label="observé")
    ax.set_title(nom, fontsize=10); ax.set_xlabel("distance r (km)")
    sortie = (G_emp(pts, rs_G) < bas) | (G_emp(pts, rs_G) > haut)
    lignes.append({"semis": nom, "part des r hors enveloppe": sortie.mean()})
axes[0].set_ylabel("G(r)")
axes[0].legend(frameon=False, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("figures/ch09-fonction-G.png", dpi=200, bbox_inches="tight")
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

|                 | semis | part des r hors enveloppe |
|-----------------|-------|---------------------------|
| aléatoire (CSR) |       | 0.000                     |
| agrégé (Thomas) |       | 0.741                     |



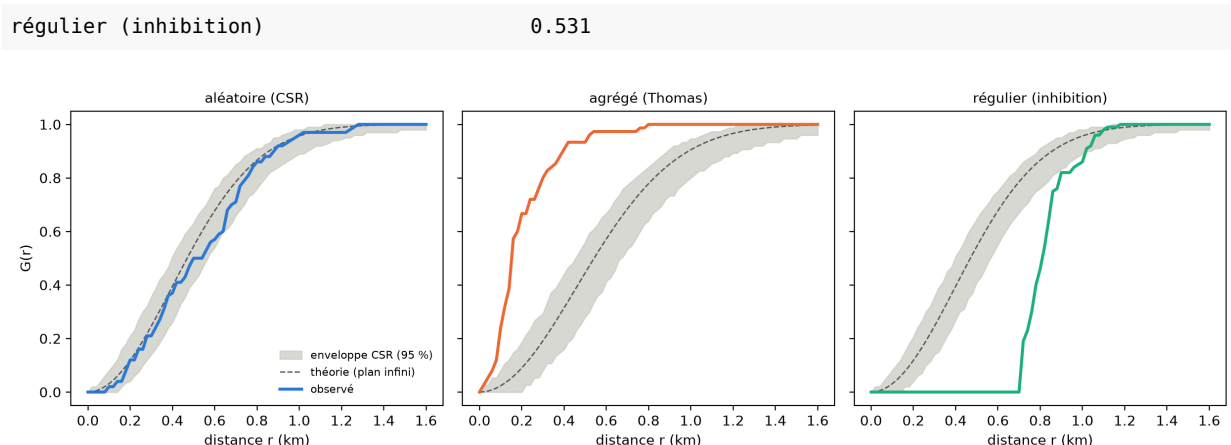


Figure 98: Fonction  $G$  (part des points dont le plus proche voisin est à moins de  $r$ ) pour les trois semis, avec l'enveloppe de Monte-Carlo (bandeau gris, 499 semis CSR de même taille) et la courbe théorique du plan infini (pointillés).

Le semis aléatoire reste dans le bandeau ; l'agrégé monte beaucoup plus vite (ses voisins sont proches) ; le régulier est  **nul**  jusqu'à la distance d'inhibition (0,7 km) puis monte brusquement. Remarquons que le bandeau est un peu décalé par rapport à la courbe théorique : il intègre l'effet de bord. C'est lui, et non la courbe en pointillés, qu'il faut regarder.

#### 9.4.4 La fonction $K$ de Ripley

La fonction  $G$  ne regarde que le **premier** voisin, donc l'échelle la plus petite. Pour étudier plusieurs échelles à la fois, Ripley a proposé la fonction  $K$  :  $K(r)$  est le **nombre moyen de points supplémentaires** que l'on trouve à moins de la distance  $r$  d'un point typique, **divisé par l'intensité  $\lambda$**  :

$$K(r) = \frac{1}{\lambda} \mathbb{E}[\#\{\text{autres points à distance} \leq r \text{ d'un point typique}\}].$$

Sous le CSR, les autres points sont distribués comme un processus de Poisson indépendamment du point typique (propriété de Slivnyak) : le nombre moyen d'autres points dans un disque d'aire  $\pi r^2$  vaut  $\lambda \pi r^2$ , donc

$$K_{\text{CSR}}(r) = \pi r^2.$$

Pour un agrégat, un point typique a **plus** de voisins que prévu à toutes les petites distances :  $K(r) > \pi r^2$ . Pour un semis régulier, il en a **moins** :  $K(r) < \pi r^2$ . On préfère tracer la version « stabilisée »  $L(r) - r = \sqrt{K(r)/\pi} - r$ , qui vaut 0 sous le CSR et dont la variance est à peu près constante en  $r$ . L'estimateur naïf est  $\hat{K}(r) = \frac{A}{n(n-1)} \sum_{i \neq j} \mathbf{1}\{d_{ij} \leq r\}$ .

**Un exemple à la main.** Quatre adresses dans la fenêtre unité ( $A = 1$ ) :  $(0,1;0,1)$ ,  $(0,2;0,1)$ ,  $(0,8;0,8)$ ,  $(0,85;0,9)$ . Les deux premières sont à 0,1 l'une de l'autre, les deux dernières à  $\sqrt{0,05^2 + 0,1^2} \approx 0,112$ , les autres paires à plus de 1. Pour  $r = 0,2$ , il y a 2 paires proches, donc 4 couples ordonnés  $(i,j)$  avec  $i \neq j$  :  $\hat{K}(0,2) = \frac{1}{4 \times 3} \times 4 = 0,333$ , alors que  $\pi r^2 = 0,126$  : le semis est nettement agrégé à cette échelle.

**L'effet de bord, encore.** Un point proche du bord a une partie de son disque hors de la fenêtre : on y voit mécaniquement moins de voisins, et l'estimateur naïf **sous-estime**  $K$  aux grandes distances. La correction la plus simple est la **méthode du bord** (*reduced-sample*) : pour une distance  $r$ , on ne compte que les points situés à plus de  $r$  du bord (leurs disques sont entiers dans la fenêtre), soit  $n_r$  points :

$$\hat{K}(r) = \frac{A}{(n-1)n_r} \sum_{i: b_i \geq r} \#\{j \neq i : d_{ij} \leq r\},$$

où  $b_i$  est la distance de  $i$  au bord. Elle est sans biais, au prix d'une perte de données quand  $r$  grandit : on limite donc  $r$  au quart du côté de la fenêtre environ.

```
def ripley_K(pts, rs, bord=True):
    """Estimateur de K, avec ou sans correction de bord (méthode du bord)."""
    n = len(pts)
    D = distances(pts)
    np.fill_diagonal(D, np.inf)
    b = np.minimum.reduce([pts[:, 0], COTE - pts[:, 0], pts[:, 1], COTE - pts[:, 1]])
    K = np.full(len(rs), np.nan)
    for k, r in enumerate(rs):
        garde = (b >= r) if bord else np.ones(n, dtype=bool)
        nr = garde.sum()
        if nr > 0:
            K[k] = AIRE * (D[garde] <= r).sum() / ((n - 1) * nr)
    return K

# Contrôle sur l'exemple à la main (fenêtre unité) : on neutralise le bord pour retrouver 0,333
P4 = np.array([[0.1, 0.1], [0.2, 0.1], [0.8, 0.8], [0.85, 0.9]])
D4 = distances(P4); np.fill_diagonal(D4, np.inf)
print("K(0,2) de l'exemple à la main :", round(1.0 * (D4 <= 0.2).sum() / (4 * 3), 4), "| pi r^2 =",
      round(np.pi * 0.2 ** 2, 4))

# L'effet de bord, mesuré : moyenne de K sur 300 semis CSR, avec et sans correction, à r = 2 km
rs = np.linspace(0.1, 2.0, 20)
rng = np.random.default_rng(3)
Kn, Kc = [], []
for _ in range(300):
    pts_sim = semis_poisson(100, rng)
    Kn.append(ripley_K(pts_sim, rs, bord=False))
    Kc.append(ripley_K(pts_sim, rs, bord=True))
Kn, Kc = np.array(Kn), np.array(Kc)
print(f"à r = 2 km -> pi r^2 = {np.pi * 4:.2f} | moyenne sans correction : {Kn[:, -1].mean():.2f} |
      moyenne avec correction : {np.nanmean(Kc[:, -1]):.2f}")
```

K(0,2) de l'exemple à la main : 0.3333 | pi r^2 = 0.1257

à r = 2 km -> pi r^2 = 12.57 | moyenne sans correction : 10.42 | moyenne avec correction : 12.30

Sans correction, l'estimateur sous-estime nettement  $K$  à 2 km : 10,4 en moyenne au lieu de  $\pi r^2 = 12,6$ , soit 17 % de trop peu. Avec la méthode du bord, on obtient 12,3 : un écart de 2 %, très inférieur. Passons à l'application : on compare la courbe  $\hat{L}(r) - r$  de chaque semis à une enveloppe obtenue sur 499 semis CSR de même taille. Pour un **test global** (et non « point par point »), on utilise la statistique  $T = \max_r |\hat{L}(r) - r|$ , comparée à sa distribution sous CSR.

```
def L_centre(pts, rs):
    return np.sqrt(ripley_K(pts, rs) / np.pi) - rs

rng = np.random.default_rng(21)
fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.9), sharey=True)
lignes = []
for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
```

```

n = len(pts)
sims = np.array([L_centre(semis_poisson(n, rng), rs) for _ in range(499)])
bas, haut = np.nanpercentile(sims, [2.5, 97.5], axis=0)
obs = L_centre(pts, rs)
T_obs = np.nanmax(np.abs(obs))
T_sim = np.nanmax(np.abs(sims), axis=1)
lignes.append({"semis": nom, "T observé": T_obs, "T sim. (médiane)": np.median(T_sim),
               "p global (Monte-Carlo)": (1 + (T_sim >= T_obs).sum()) / (len(T_sim) + 1),
               "L-r moyen": np.nanmean(obs)})
ax.fill_between(rs, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %, point par point)")
ax.axhline(0, color="#555555", lw=1, ls="--")
ax.plot(rs, obs, color=couleur, lw=2.2, label="observé")
ax.set_title(nom, fontsize=10); ax.set_xlabel("distance r (km)")
axes[0].set_ylabel("L(r) - r (0 = hasard complet)")
axes[0].legend(frameon=False, fontsize=8, loc="lower left")
plt.tight_layout()
plt.savefig("figures/ch09-ripley.png", dpi=200, bbox_inches="tight")
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 170):
    print(pd.DataFrame(lignes).to_string(index=False))

```

|  | semis                 | T observé | T sim. (médiane) | p global (Monte-Carlo) | L-r moyen |
|--|-----------------------|-----------|------------------|------------------------|-----------|
|  | aléatoire (CSR)       | 0.092     | 0.1002           | 0.672                  | -0.03473  |
|  | agrégé (Thomas)       | 1.095     | 0.133            | 0.002                  | 0.7666    |
|  | régulier (inhibition) | 0.7       | 0.1023           | 0.002                  | -0.2289   |

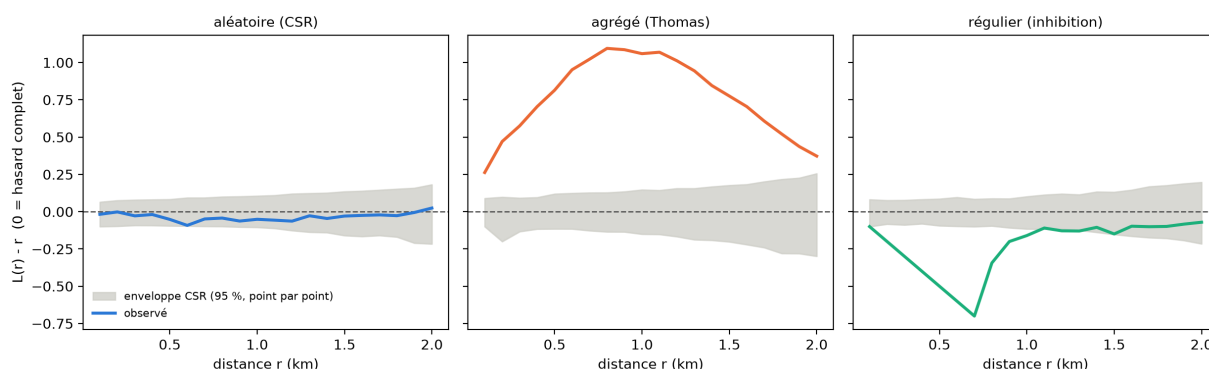


Figure 99: Fonction  $L(r) - r$  de Ripley pour les trois semis, avec l'enveloppe de Monte-Carlo (bandeau gris) obtenue sur 499 semis aléatoires de même taille. Au-dessus de 0 : agrégat ; en dessous : régularité.

La lecture est la suivante. Le semis **aléatoire** reste dans le bandeau à toutes les distances et sa p-valeur globale est banale. Le semis **agrégé** est très au-dessus du bandeau sur toute la plage, avec une forme caractéristique : l'écart se creuse jusqu'à un **maximum vers 1 km**, une échelle de quelques fois l'étalement  $\sigma = 0,4$  km des paquets, puis décroît quand le disque de rayon  $r$  commence à englober plusieurs paquets, dont les voisins ne sont plus « en excès ». Le semis **régulier** est sous le bandeau aux petites distances, avec un **minimum exactement à la distance d'inhibition (0,7 km)**. Ce n'est pas un hasard : tant que  $r < 0,7$  km, aucune paire de points n'est à moins de  $r$ , donc  $\hat{K}(r) = 0$  et  $L(r) - r = -r$  (la droite descendante de la figure) ; au-delà de 0,7 km, les voisins apparaissent et la courbe remonte vers le bandeau.

**Enveloppe point par point  $\neq$  test global.** Un bandeau à 95 % en chaque  $r$  est dépassé quelque part avec une probabilité bien supérieure à 5 % si l'on regarde de nombreuses distances (c'est le même problème que les tests multiples). Pour **conclure**, utilisez la p-valeur **globale** de la statistique  $T$  ; le bandeau sert à **voir** à quelles échelles l'écart se produit.

### 9.4.5 Attention : agrégat ou densité variable ?

Tout ce qui précède suppose que l'**intensité est homogène**. C'est le piège principal des processus ponctuels. Imaginez que les clients habitent plutôt près de la boutique, au centre de la fenêtre, parce que la densité d'habitation y est plus forte. Même si chacun a choisi son adresse **sans tenir compte des autres** (aucune interaction), le semis paraîtra agrégé : beaucoup de points au centre, peu en périphérie. La fonction  $K$  ne distingue pas un **agrégat** (les points s'attirent) d'une **intensité qui varie** (les points s'accumulent là où la densité est forte).

```
def semis_inhomogene(n, rng, etalement=2.5):
    """100 points indépendants, mais avec une densité qui décroît du centre vers les bords."""
    pts = []
    while len(pts) < n:
        p = rng.uniform(0, COTE, size=2)
        if rng.random() < np.exp(-((p - COTE / 2) ** 2).sum() / (2 * etalement ** 2)):
            pts.append(p)
    return np.array(pts)

rng = np.random.default_rng(31)
in的角度 = semis_inhomogene(100, rng)
obs = L_centre(inho, rs)
sims = np.array([L_centre(semis_poisson(100, rng), rs) for _ in range(499)])
T_obs, T_sim = np.nanmax(np.abs(obs)), np.nanmax(np.abs(sims), axis=1)
print(f"semis à densité variable, SANS interaction : T = {T_obs:.3f} | p global CSR = {(1 + (T_sim >=
↳ T_obs).sum()) / 500:.3f}")
q = comptages(inho, 4).reshape(4, 4)
print("comptages par case (4 x 4) :")
print(q.astype(int))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
ax1.scatter(inho[:, 0], inho[:, 1], s=16, color=VIOLET, edgecolor="white", linewidth=0.4)
ax1.set_xlim(0, COTE); ax1.set_ylim(0, COTE); ax1.set_aspect("equal")
ax1.set_title("100 adresses indépendantes, densité forte au centre", fontsize=10); ax1.set_xlabel("x
↳ (km)"); ax1.set_ylabel("y (km)")
bas, haut = np.nanpercentile(sims, [2.5, 97.5], axis=0)
ax2.fill_between(rs, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %)")
ax2.axhline(0, color="#555555", lw=1, ls="--")
ax2.plot(rs, obs, color=VIOLET, lw=2.2, label="observé")
ax2.set_xlabel("distance r (km)"); ax2.set_ylabel("L(r) - r"); ax2.set_title("La fonction K voit un «
↳ agrégat » qui n'en est pas un", fontsize=10)
ax2.legend(frameon=False, fontsize=8, loc="upper left")
plt.tight_layout()
plt.savefig("figures/ch09-inhomogene.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

```
semis à densité variable, SANS interaction : T = 0.687 | p global CSR = 0.002
comptages par case (4 x 4) :
[[ 3  8  2  1]
 [ 4 13 15  7]
 [ 5 10 11  6]
 [ 1  8  5 11]]
figure enregistrée
```

Le test rejette le hasard complet alors qu'**aucun point n'attire les autres**. L'agrégat apparent est entièrement dû à la variation de la densité. Le remède est de comparer non pas à un CSR homogène mais à un **processus de Poisson inhomogène** d'intensité estimée  $\hat{\lambda}(s)$  (par un lissage à noyau, par exemple), et d'utiliser la fonction  $K$  **inhomogène** de Baddeley, Møller et Waagepetersen. Elle ne figure pas dans ce chapitre (nous ne l'avons pas implémentée) ; retenez le principe : **on ne peut parler d'agrégation entre les points qu'après avoir tenu compte de la densité qui varie**. Cette distinction entre *effet du premier ordre* (l'intensité) et *effet du second ordre* (l'interaction) est le cœur de la modélisation des semis de points.

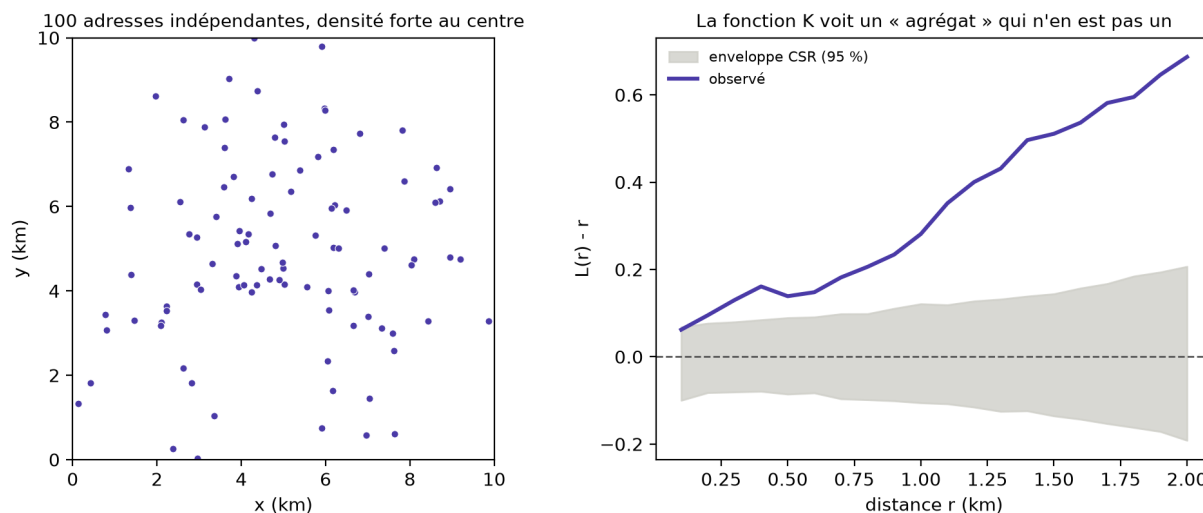


Figure 100: À gauche : 100 adresses indépendantes les unes des autres, mais avec une densité qui décroît du centre vers les bords. À droite : la fonction  $L(r) - r$  sort de l'enveloppe du hasard complet, alors qu'il n'y a aucune interaction entre les points.

**Retour à la question de Yasmine.** Pour savoir si ses clients de la médina sont **vraiment** regroupés autour de la boutique, elle ne peut pas se contenter d'un test de  $K$  : elle doit d'abord se demander si la densité de population varie, c'est-à-dire si les habitants eux-mêmes sont plus nombreux près de la boutique. Un semis de clients qui reproduit simplement la répartition de la **population** ne dit rien sur son comportement. La comparaison utile est avec les habitants **non clients** (un semis de « contrôle ») ou avec une carte de densité de population.

### À retenir.

- Un **semis de points** est un phénomène dont les **positions** sont aléatoires. La référence est le **hasard spatial complet** : un processus de Poisson homogène (comptages de Poisson, indépendance entre régions disjointes, points uniformes conditionnellement à leur nombre).
- **Quadrats** :  $\chi^2 = \sum (c_k - \bar{c})^2 / \bar{c}$  (approximativement  $\chi^2_{m-1}$ ) ; simple, mais dépend de la taille des cases.
- **Plus proche voisin** :  $G(r) = 1 - e^{-\lambda \pi r^2}$  sous le CSR,  $\mathbb{E}[d] = 1/(2\sqrt{\lambda})$  ; indice de Clark-Evans  $R < 1$  pour un agrégat,  $R > 1$  pour de la régularité. Une fenêtre finie biaise  $R$  (**effet de bord**) : corrigez (Donnelly) ou, mieux, utilisez une **p-valeur de Monte-Carlo** dans la même fenêtre.
- **Fonction  $K$  de Ripley** :  $K_{\text{CSR}}(r) = \pi r^2$  ;  $L(r) - r$  au-dessus de 0 pour l'agrégat, en dessous pour la régularité ; **correction de bord** indispensable ; utilisez un test **global** et non un bandeau point par point.
- Une **intensité variable imite un agrégat** : avant de conclure à une interaction, tenez compte de la densité.

## 9.5 Exercices du chapitre 9

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les corrigés réutilisent les fonctions définies dans les sections 9.1 à 9.4 (haversine, moran, variogramme\_empirique, krigeage\_ordinaire, semis\_thomas, ripley\_K...) : le chapitre est exécuté d'un seul tenant.

### Énoncés

**Exercice 1 ★ (quel type de données ?).** Pour chaque situation, dites s'il s'agit de données **géostatistiques**, **surfaciques** ou d'un **semis de points**, et nommez l'outil de ce chapitre qui répond à la question. (a) Yasmine relève la **température** à l'intérieur de 40 entrepôts de stockage répartis dans la région et veut estimer la température dans un entrepôt qu'elle n'a pas visité. (b) Elle dispose du **taux de retour de colis** de chacune des 24 gouvernorats et se demande si les gouvernorats voisins ont des taux semblables. (c) Elle a les **adresses** de tous ses clients de Sousse et se demande si elles se regroupent. (d) Un transporteur mesure le **délai de livraison** à 300 adresses et veut cartographier le délai moyen attendu sur toute la zone.

**Exercice 2 ★ (haversine à la main).** Calculez à la main la distance entre Tunis (36,81°N; 10,18°E) et Sousse (35,83°N; 10,61°E), en utilisant la latitude moyenne pour le facteur  $\cos \varphi$ . Comparez au résultat de la formule de haversine.

**Exercice 3 ★★ (Moran sur quatre zones).** Quatre zones alignées A–B–C–D (chacune voisine de la précédente et de la suivante) ont pour valeurs 1, 2, 3, 4. (a) Calculez à la main l'indice de Moran avec les poids binaires, puis avec les poids standardisés par ligne. (b) Quelle est son espérance sous l'hypothèse nulle ? (c) Il n'y a que  $4! = 24$  façons de ranger ces valeurs sur les zones : calculez la distribution **exacte** de  $I$  et la p-valeur de l'alignement observé.

**Exercice 4 ★★ (lire un indice local).** Dans une carte, la zone  $i$  a un écart à la moyenne  $z_i = +12$  et la moyenne des écarts de ses voisines vaut  $(Wz)_i = -8$ . La variance empirique est  $m_2 = 100$ . (a) Dans quel quadrant se trouve la zone ? (b) Que vaut son indice local  $I_i$  ? (c) Peut-on conclure que c'est une valeur atypique significative ? Que faudrait-il faire ?

**Exercice 5 ★★ (variogramme à la main).** Cinq mesures alignées aux abscisses 0, 1, 2, 3 et 4 km ont pour valeurs 3, 5, 4, 8, 7. Calculez le variogramme empirique aux distances 1, 2, 3 et 4 km. Que pouvez-vous dire de la forme de la courbe et de la fiabilité de ces estimations ?

**Exercice 6 ★★ (krigeage à la main).** En dimension 1, avec le variogramme linéaire  $\gamma(h) = 0,5h$ , on a mesuré 20 en  $x = 0$  et 30 en  $x = 4$ . (a) Écrivez et résolvez le système de krigeage ordinaire pour prédire en  $x = 1$  ; donnez la prédiction et la variance. (b) Si les valeurs mesurées étaient 0 et 100 au lieu de 20 et 30, la variance de krigeage changerait-elle ? Pourquoi ?

**Exercice 7 ★★ (le problème de l'unité spatiale modifiable).** On construit une variable  $x$  corrélée aux ventes\_hab des 144 délégations (la graine est donnée dans le corrigé). On **agrège** ensuite les zones en blocs de  $2 \times 2$ ,  $3 \times 3$ ,  $4 \times 4$  et  $6 \times 6$  cases (moyenne par bloc). Comment évoluent la corrélation entre  $x$  et les ventes, l'écart-type des ventes et l'indice de Moran ? Qu'en conclure pour l'interprétation d'une corrélation calculée sur des zones ?

**Exercice 8 ★★ (l'effet de bord sur les indices locaux).** Sur la grille  $12 \times 12$  avec voisinage « reine », les cases ont 3 voisines (coins), 5 (bords) ou 8 (intérieur). Sans autocorrélation, la variance de l'indice local  $I_i$  devrait varier comme  $1/k$  où  $k$  est le nombre de voisines. Vérifiez-le par simulation et expliquez pourquoi les zones de bord sont plus souvent « significatives » avec une p-valeur naïve.



**Exercice 9 ★★ (nombre efficace d'observations).** Pour un modèle SAR sur la grille  $12 \times 12$  avec  $\rho \in \{0,3; 0,5; 0,7; 0,9; 0,95\}$ , estimez par simulation le **nombre efficace d'observations indépendantes** pour estimer une moyenne, défini par  $n_{\text{eff}} = \text{Var}(y_i) / \text{Var}(\bar{y})$ . Commentez.

**Exercice 10 ★★ (cases vides).** On répartit 100 points au hasard (CSR) dans une fenêtre de  $10 \times 10$  km, divisée en  $4 \times 4$  cases. (a) Quelle est, à la main, la probabilité qu'une case donnée soit vide, et le nombre moyen de cases vides ? (b) Vérifiez par simulation, puis comparez au nombre de cases vides du semis agrégé de la section 9.4. Que cela dit-il ?

**Exercice 11 ★★★ (l'estimateur du variogramme est-il sans biais ?).** Le variogramme empirique est une moyenne de  $\frac{1}{2}(z_i - z_j)^2$ . Montrez que  $\mathbb{E}[\frac{1}{2}(Z(s+h) - Z(s))^2] = \gamma(h)$  pour un processus stationnaire de moyenne  $\mu$ , puis vérifiez par simulation, en moyennant le variogramme empirique de 60 jeux de livraisons indépendants (même processus que 9.3) et en le comparant au variogramme **vrai**  $\gamma(h) = 0,4 + 1,0(1 - e^{-h/12})$ .

**Exercice 12 ★★★ (la fonction K d'un processus de Thomas).** Pour un processus de Thomas de densité de centres  $\kappa$  et d'étalement  $\sigma$  (par coordonnée), on admet que la fonction de corrélation de paires est  $g(r) = 1 + \frac{1}{4\pi\kappa\sigma^2}e^{-r^2/(4\sigma^2)}$ . (a) Montrez que  $K(r) = \pi r^2 + \frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$ . (b) Vérifiez par simulation pour  $\kappa = 0,5$ ,  $\mu = 2$ ,  $\sigma = 0,4$ , puis pour  $\kappa = 0,08$ ,  $\mu = 12$ ,  $\sigma = 0,4$ . Que constatez-vous ?

## Corrigés

**Corrigé 1.** (a) **Géostatistique** : la température existe en tout point de la région et on la mesure en 40 lieux ; on veut prédire ailleurs : variogramme et **krigeage**. (b) **Surfacique** : une valeur par zone, la question est la ressemblance entre zones voisines : matrice de poids et **indice de Moran** (avec 24 zones seulement, on teste par permutations et on indique la définition du voisinage). (c) **Semis de points** : ce sont les positions qui sont le phénomène ; on les compare au hasard complet avec les quadrats, le plus proche voisin et la fonction **K de Ripley**, en pensant à la densité de population qui peut varier (9.4.5). (d) **Géostatistique** : un délai défini en tout point mesuré en 300 adresses ; **variogramme et krigeage** donnent la carte du délai attendu et une carte d'incertitude.

**Corrigé 2.**  $\Delta\varphi = 0,98^\circ$  donne  $0,98 \times 111,2 \approx 109,0$  km vers le sud.  $\Delta\lambda = 0,43^\circ$  : avec  $\cos(36,32^\circ) \approx 0,806$ ,  $0,43 \times 111,2 \times 0,806 \approx 38,5$  km vers l'est. La distance est  $\sqrt{109,0^2 + 38,5^2} \approx 115,6$  km.

```
print("haversine Tunis - Sousse :", round(float(haversine(36.81, 10.18, 35.83, 10.61)), 1), "km")
print("estimation à la main      :", round(float(np.hypot(0.98 * 111.2, 0.43 * 111.2 *
↳ np.cos(np.radians(36.32)))), 1), "km")
```

```
haversine Tunis - Sousse : 115.6 km
estimation à la main      : 115.6 km
```

**Corrigé 3.** (a) Les écarts à la moyenne  $\bar{y} = 2,5$  sont  $z = (-1,5; -0,5; 0,5; 1,5)$  et  $\sum z^2 = 5$ . Les frontières sont A-B, B-C et C-D : produits  $(-1,5)(-0,5) = 0,75$ ,  $(-0,5)(0,5) = -0,25$ ,  $(0,5)(1,5) = 0,75$ , de somme 1,25. **Poids binaires** ( $S_0 = 6$  liens orientés) :  $\sum_{ij} w_{ij} z_i z_j = 2 \times 1,25 = 2,5$  et  $I = \frac{4}{6} \times \frac{2,5}{5} = \frac{1}{3}$ . **Poids standardisés** ( $S_0 = 4$ ) : A et D n'ont qu'une voisine (poids 1), B et C deux (poids  $\frac{1}{2}$  chacune), donc  $z^T W z = z_A z_B + \frac{1}{2} z_B (z_A + z_C) + \frac{1}{2} z_C (z_B + z_D) + z_D z_C = 0,75 + 0,25 + 0,25 + 0,75 = 2$  et  $I = \frac{2}{5} = 0,4$ . (b)  $\mathbb{E}[I] = -1/(n-1) = -\frac{1}{3}$  : avec  $n = 4$  zones seulement, l'espérance sous le hasard est loin de zéro. (c) On énumère les 24 permutations.

```
from itertools import permutations

W4 = std_lignes(np.array([[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]], dtype=float))
W4b = np.array([[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]], dtype=float)
vals = np.array([1, 0, 2, 3, 4])
```



```
print("I (poids binaires)      :", round(moran(vals, W4b), 4), "| I (poids standardisés) :",
      ↪ round(moran(vals, W4), 4))
toutes = np.array([moran(np.array(p), W4) for p in permutations(vals)])
print("nombre de permutations :", len(toutes), "| valeurs distinctes de I :",
      ↪ np.unique(toutes.round(4)).tolist())
print("moyenne de I sur les 24 permutations :", round(toutes.mean(), 4), "(= -1/(n-1) =", round(-1 /
      ↪ 3, 4), ")")
print("p-valeur exacte (I >= observé) :", round(float((toutes >= moran(vals, W4) - 1e-12).mean()), 4))
```

```
I (poids binaires)      : 0.3333 | I (poids standardisés) : 0.4
nombre de permutations : 24 | valeurs distinctes de I : [-0.9, -0.6, -0.5, -0.3, 0.0, 0.3, 0.4]
moyenne de I sur les 24 permutations : -0.3333 (= -1/(n-1) = -0.3333 )
p-valeur exacte (I >= observé) : 0.0833
```

L'espérance exacte sur les permutations est bien  $-\frac{1}{3}$ . L'alignement observé (1, 2, 3, 4) et son image inversée (4, 3, 2, 1) sont les deux seuls arrangements qui atteignent cette valeur maximale : la p-valeur exacte est  $2/24 \approx 0,083$ , **supérieure à 5 %**. Avec quatre zones, aucune autocorrélation, même parfaite, ne peut être significative : c'est le prix du petit nombre de permutations.

**Corrigé 4.** (a)  $z_i > 0$  (zone haute) et  $(Wz)_i < 0$  (voisines basses) : quadrant **HL** (haute entourée de basses). (b)  $I_i = \frac{z_i (Wz)_i}{m_2} = \frac{12 \times (-8)}{100} = -0,96$  : négatif, car la zone s'oppose à ses voisines. (c) Non : un indice local négatif ne dit rien de sa significativité. Il faut une **permutation conditionnelle** (fixer  $z_i$ , mélanger les autres valeurs) pour obtenir une p-valeur, puis **corriger** pour les tests multiples (Benjamini-Hochberg), comme au 9.2.4.

**Corrigé 5.** Distance 1 : paires (3,5), (5,4), (4,8), (8,7) ; carrés des écarts 4, 1, 16, 1, total 22 ;  $\hat{\gamma}(1) = \frac{22}{2 \times 4} = 2,75$ . Distance 2 : (3,4), (5,8), (4,7) ; carrés 1, 9, 9, total 19 ;  $\hat{\gamma}(2) = \frac{19}{2 \times 3} \approx 3,17$ . Distance 3 : (3,8), (5,7) ; carrés 25, 4 ;  $\hat{\gamma}(3) = \frac{29}{2 \times 2} = 7,25$ . Distance 4 : (3,7) ;  $\hat{\gamma}(4) = \frac{16}{2} = 8$ .

```
P5 = np.array([[0.0, 0], [1, 0], [2, 0], [3, 0], [4, 0]])
z5 = np.array([3.0, 5, 4, 8, 7])
print(variogramme_empirique(P5, z5, largeur=1.0, hmax=4.0).round(3).to_string(index=False))
```

| h   | gamma | paires |
|-----|-------|--------|
| 1.0 | 2.750 | 4      |
| 2.0 | 3.167 | 3      |
| 3.0 | 7.250 | 2      |
| 4.0 | 8.000 | 1      |

La courbe est **croissante** (2,75 ; 3,17 ; 7,25 ; 8) et ne montre aucun palier : on ne peut pas estimer de portée. Deux raisons : il y a très peu de données, et ces valeurs montrent une **tendance** (elles augmentent avec l'abscisse), qui fait monter le variogramme (9.3.7). Surtout, le nombre de paires **diminue** avec la distance (4, 3, 2, 1) : l'estimation à 4 km repose sur **une seule paire** et n'a quasiment aucune fiabilité. C'est l'inverse de ce qui se passe en deux dimensions, où les paires lointaines sont nombreuses, mais cela rappelle qu'un variogramme se lit en regardant les effectifs.

**Corrigé 6.** (a)  $\Gamma = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$  (car  $\gamma(4) = 2$ ) et  $\gamma_0 = (\gamma(1), \gamma(3))^T = (0,5; 1,5)^T$ . Les équations sont  $2\lambda_2 + m = 0,5$ ,  $2\lambda_1 + m = 1,5$  et  $\lambda_1 + \lambda_2 = 1$ . En soustrayant,  $2(\lambda_1 - \lambda_2) = 1$ , donc  $\lambda_1 = 0,75$ ,  $\lambda_2 = 0,25$  et  $m = 0$ . Prédiction :  $0,75 \times 20 + 0,25 \times 30 = 22,5$ , variance  $\lambda^T \gamma_0 + m = 0,75 \times 0,5 + 0,25 \times 1,5 = 0,75$ . C'est l'interpolation linéaire (de 20 à 30 sur 4 km, soit 22,5 à 1 km). (b) **Non** : la variance de krigeage  $\lambda^T \gamma_0 + m$  ne dépend que de la **géométrie** (les distances) et du variogramme, jamais des valeurs mesurées. Les poids sont identiques, donc la prédiction vaudrait  $0,75 \times 0 + 0,25 \times 100 = 25$  et la variance resterait 0,75. Le krigeage dit à quel point le *plan d'échantillonnage* est informatif, pas si les données observées sont « surprenantes ».

```
for valeurs in ([20.0, 30.0], [0.0, 100.0]):
    pred, var, lam = krigeage_ordinaire(np.array([[0.0, 0], [4, 0]]), np.array(valeurs),
    ↪ np.array([[1.0, 0]]), lineaire, (0.5,))
    print("valeurs", valeurs, "-> poids", lam.ravel().round(3), "| prédiction", pred.round(3), "|
    ↪ variance", var.round(3))
```

```
valeurs [20.0, 30.0] -> poids [0.75 0.25] | prédiction [22.5] | variance [0.75]
valeurs [0.0, 100.0] -> poids [0.75 0.25] | prédiction [25.] | variance [0.75]
```

**Corrigé 7.** Nous construisons  $x$  comme un mélange de ventes\_hab standardisées et d'un autre champ SAR indépendant, puis nous agrégeons.

```
Wr = std_lignes(contiguite_reine(12, 12))
rng7 = np.random.default_rng(5)
A7 = np.linalg.inv(np.eye(144) - 0.9 * Wr)
y7 = deleg["ventes_hab"].to_numpy()
champ = A7 @ rng7.normal(size=144)
x7 = 0.5 * (y7 - y7.mean()) / y7.std() + 0.87 * champ / champ.std()

def agrege(v, k):
    m = 12 // k
    return v.reshape(12, 12).reshape(m, k, m, k).mean(axis=(1, 3)).ravel()

lignes = []
for k in (1, 2, 3, 4, 6):
    m = 12 // k
    ya, xa = agrege(y7, k), agrege(x7, k)
    I_k = moran(ya, std_lignes(contiguite_reine(m, m))) if m >= 3 else np.nan
    lignes.append({"bloc": f"{k} x {k}", "zones": m * m, "corr(x, ventes)": np.corrcoef(xa, ya)[0, 1],
    ↪ "ecart_type_ventes": ya.std(), "Moran_ventes": I_k, "E[I]": -1 / (m * m - 1) if m >=
    ↪ 3 else np.nan})
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))
```

| bloc  | zones | corr(x, ventes) | ecart_type_ventes | Moran_ventes | E[I]   |
|-------|-------|-----------------|-------------------|--------------|--------|
| 1 x 1 | 144   | 0.597           | 13.468            | 0.621        | -0.007 |
| 2 x 2 | 36    | 0.619           | 11.142            | 0.570        | -0.029 |
| 3 x 3 | 16    | 0.646           | 10.010            | 0.291        | -0.067 |
| 4 x 4 | 9     | 0.774           | 9.350             | -0.054       | -0.125 |
| 6 x 6 | 4     | 0.869           | 8.431             | NaN          | NaN    |

La **corrélation augmente** avec l'agrégation (de 0,60 sur les 144 zones à 0,87 sur les 4 blocs de 6×6), l'**écart-type** des ventes diminue (de 13,5 à 8,4 : les moyennes de blocs lissent les fluctuations) et l'**indice de Moran** chute : de 0,62 à 0,57 (blocs de 2 × 2), à 0,29 (3 × 3, soit 16 zones) puis à -0,05 (4 × 4, soit 9 zones), à comparer à -0,125 sous le hasard pour 9 zones : à cette échelle, l'autocorrélation a pratiquement disparu (et avec 9 zones, un test aurait de toute façon très peu de puissance). Les mêmes personnes, les mêmes ventes, trois « résultats » différents selon le découpage : c'est le **problème de l'unité spatiale modifiable** (MAUP). Une corrélation calculée sur des zones n'est vraie que pour **ce découpage** : on ne peut pas la transposer à l'échelle des individus (sophisme écologique), ni à un autre découpage, sans précaution.

**Corrigé 8.** On simule le hasard en mélangeant les valeurs de ventes\_bruit et on calcule, pour chaque case, l'indice local  $I_i = z_i(Wz)_i / m_2$ .

```
rng8 = np.random.default_rng(8)
zb = deleg["ventes_bruit"].to_numpy() - deleg["ventes_bruit"].mean()
m2 = zb @ zb / 144
perms = np.argsort(rng8.random((3000, 144)), axis=1)
Zs = zb[perms]
Ii_sim = Zs * (Zs @ Wr.T) / m2 # (3000, 144) : indices locaux sous H0
k_vois = contiguite_reine(12, 12).sum(axis=1)
```

```
lignes = []
for kk in (3, 5, 8):
    cols = k_vois == kk
    lignes.append({"voisines": kk, "cases": int(cols.sum()), "variance de I_i": Ii_sim[:,
↪ cols].var(), "variance x k": Ii_sim[:, cols].var() * kk})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

| voisines | cases | variance de I_i | variance x k |
|----------|-------|-----------------|--------------|
| 3        | 4     | 0.333           | 1.000        |
| 5        | 40    | 0.191           | 0.957        |
| 8        | 100   | 0.117           | 0.936        |

La variance de  $I_i$  est environ **2,8 fois plus grande** pour un coin (3 voisines) que pour une case intérieure (8 voisines), à comparer au rapport théorique  $8/3 \approx 2,67$ ; la colonne « variance  $\times k$  » est à peu près constante, ce qui confirme la loi en  $1/k$  : la moyenne des voisines est plus bruitée quand il y a moins de voisines. Une zone de bord ressemble donc plus facilement à une valeur extrême par hasard, et une p-valeur calculée avec la **même** loi pour toutes les zones la déclarerait trop souvent significative. Les **permutations conditionnelles** du 9.2.4 évitent ce piège, car elles tirent exactement le bon nombre de voisines pour chaque zone. Par prudence, on examine séparément les zones de bord.

**Corrigé 9.** Pour chaque  $\rho$ , on simule 4 000 champs SAR et on compare la variance d'une zone à celle de la moyenne des 144 zones.

```
rng9 = np.random.default_rng(9)
lignes = []
for rho in (0.3, 0.5, 0.7, 0.9, 0.95):
    Y9 = np.linalg.inv(np.eye(144) - rho * Wr) @ rng9.normal(size=(144, 4000))
    lignes.append({"rho": rho, "variance d'une zone": Y9.var(), "variance de la moyenne":
↪ Y9.mean(axis=0).var(), "n_eff": Y9.var() / Y9.mean(axis=0).var()})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

| rho  | variance d'une zone | variance de la moyenne | n_eff  |
|------|---------------------|------------------------|--------|
| 0.30 | 1.052               | 0.014                  | 75.715 |
| 0.50 | 1.179               | 0.028                  | 42.577 |
| 0.70 | 1.555               | 0.077                  | 20.203 |
| 0.90 | 3.917               | 0.708                  | 5.532  |
| 0.95 | 8.383               | 2.805                  | 2.989  |

Le nombre efficace d'observations **s'effondre** quand  $\rho$  augmente : d'environ 76 pour  $\rho = 0,3$ , il tombe à 43 pour  $\rho = 0,5$ , à 20 pour  $\rho = 0,7$ , à 5,5 pour  $\rho = 0,9$  (la valeur de la section 9.2.5) et à 3 pour  $\rho = 0,95$ . Même pour une dépendance modérée ( $\rho = 0,5$ ), les 144 zones n'apportent l'information que d'environ 43 observations indépendantes. C'est la raison de la surconfiance des tests usuels.

**Corrigé 10.** (a) Les cases ont une aire de  $2,5 \times 2,5 = 6,25$  km<sup>2</sup> et l'intensité vaut  $\lambda = 1$  par km<sup>2</sup> : le nombre de points d'une case suit approximativement une loi de Poisson de moyenne 6,25, donc  $\mathbb{P}(\text{vide}) = e^{-6,25} \approx 0,0019$ . Plus exactement, conditionnellement à  $n = 100$  points, chaque point tombe hors d'une case donnée avec la probabilité  $\frac{15}{16}$  :  $\mathbb{P}(\text{vide}) = (15/16)^{100} \approx 0,0016$ . Pour 16 cases, le nombre moyen de cases vides est  $16 \times 0,0016 \approx 0,025$  : sous le hasard complet, on s'attend à **ne presque jamais** voir de case vide.

```
print("e^-6,25 =", round(float(np.exp(-6.25)), 5), "| (15/16)^100 =", round((15 / 16) ** 100, 5), "|
↪ 16 x (15/16)^100 =", round(16 * (15 / 16) ** 100, 4))
rng10 = np.random.default_rng(10)
vides = [(comptages(semis_poisson(100, rng10)) == 0).sum() for _ in range(5000)]
print("nombre moyen de cases vides sur 5000 semis CSR :", round(float(np.mean(vides)), 4), "| part des
↪ semis avec au moins une case vide :", round(float(np.mean(np.array(vides) > 0)), 4))
print("cases vides dans le semis agrégé :", int((comptages(semis["agrégé (Thomas)"] == 0).sum()),
↪ "sur 16")
```

```
e^-6,25 = 0.00193 | (15/16)^100 = 0.00157 | 16 x (15/16)^100 = 0.0252
nombre moyen de cases vides sur 5000 semis CSR : 0.0226 | part des semis avec au moins une case vide :
↳ 0.0224
cases vides dans le semis agrégé : 7 sur 16
```

- (b) La simulation confirme le calcul (environ 0,025 case vide en moyenne). Le semis agrégé de 9.4 a **7 cases vides sur 16** : un événement quasi impossible sous le hasard, qui est un autre visage de l'aggrégation. (Attention : ce semis compte 75 points et non 100, ce qui rend les cases vides un peu plus plausibles, sans changer la conclusion.)

**Corrigé 11.** Notons  $\delta = Z(s + h) - Z(s)$ . Par stationnarité,  $\mathbb{E}[Z(s + h)] = \mathbb{E}[Z(s)] = \mu$ , donc  $\mathbb{E}[\delta] = 0$ , et  $\mathbb{E}[\delta^2] = \text{Var}(\delta)$ . Par définition du semi-variogramme,  $\gamma(h) = \frac{1}{2}\mathbb{E}[\delta^2]$ . L'estimateur  $\hat{\gamma}(h)$  est une moyenne de  $\frac{1}{2}\delta^2$  sur des paires dont la distance est  $h$  : son espérance est donc  $\gamma(h)$  ; il est sans biais à  $\mu$  inconnue (ce qui le distingue de la covariance empirique, qui exige d'estimer la moyenne). Le seul biais vient du regroupement par classes (nous comparons à  $\gamma$  à la distance moyenne de la classe) et d'éventuelles tendances. Vérifions par simulation.

```
accumule = []
for graine in range(300, 360):
    d_sim, _ = livraisons(seed=graine)
    accumule.append(variogramme_empirique(d_sim[["x", "y"]].to_numpy(),
    ↳ d_sim["delai_jours"].to_numpy()))
h_moy = np.mean([e["h"].to_numpy() for e in accumule], axis=0)
g_moy = np.mean([e["gamma"].to_numpy() for e in accumule], axis=0)
g_sd = np.std([e["gamma"].to_numpy() for e in accumule], axis=0, ddof=1) / np.sqrt(len(accumule))
vrai_g = gamma_exp(h_moy, 0.4, 1.0, 12.0)
print(pd.DataFrame({"h": h_moy, "gamma_moyen": g_moy, "erreur_type_moyenne": g_sd, "gamma_vrai":
↳ vrai_g, "ecart_%": 100 * (g_moy / vrai_g - 1)}).round(3).to_string(index=False))
```

| h      | gamma_moyen | erreur_type_moyenne | gamma_vrai | ecart_% |
|--------|-------------|---------------------|------------|---------|
| 3.324  | 0.632       | 0.011               | 0.642      | -1.592  |
| 7.745  | 0.867       | 0.014               | 0.876      | -0.956  |
| 12.636 | 1.033       | 0.019               | 1.051      | -1.751  |
| 17.593 | 1.164       | 0.022               | 1.169      | -0.416  |
| 22.548 | 1.242       | 0.027               | 1.247      | -0.389  |
| 27.549 | 1.303       | 0.030               | 1.299      | 0.277   |
| 32.529 | 1.329       | 0.032               | 1.334      | -0.326  |
| 37.509 | 1.367       | 0.037               | 1.356      | 0.809   |
| 42.512 | 1.404       | 0.036               | 1.371      | 2.385   |
| 47.503 | 1.405       | 0.037               | 1.381      | 1.719   |

La moyenne sur 60 jeux colle au variogramme vrai à toutes les distances, à moins de 2,5 % près, et les écarts restent de l'ordre de l'erreur-type de la moyenne (colonne suivante) : rien ne montre de biais. Mais regardons la colonne des erreurs-types : elle **augmente avec la distance**. C'est la signature d'une corrélation forte entre classes de distance voisines et de grandes fluctuations d'un jeu à l'autre aux distances élevées : une moyenne de 60 jeux est précise, mais **un seul jeu** de données réelles est un tirage unique, d'où la dispersion observée en 9.3.3 sur les paramètres ajustés.

**Corrigé 12.** (a) Par définition,  $K(r) = \int_0^r 2\pi s g(s) ds$  (le nombre moyen d'autres points à moins de  $r$ , divisé par  $\lambda$ , est l'intégrale de la fonction de corrélation de paires sur le disque). Donc  $K(r) = \pi r^2 + \frac{2\pi}{4\pi\kappa\sigma^2} \int_0^r s e^{-s^2/(4\sigma^2)} ds$ . Or  $\int_0^r s e^{-s^2/(4\sigma^2)} ds = 2\sigma^2(1 - e^{-r^2/(4\sigma^2)})$ , d'où  $K(r) = \pi r^2 + \frac{1}{2\kappa\sigma^2} \cdot 2\sigma^2(1 - e^{-r^2/(4\sigma^2)}) = \pi r^2 + \frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$ . Remarquons que  $K$  ne dépend ni de  $\mu$  ni de l'intensité totale : l'excès par rapport au hasard est  $\frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$ , et il tend vers  $1/\kappa$  (le nombre moyen de centres par  $\text{km}^2$  est  $\kappa$ , donc  $1/\kappa$  est l'aire d'« un agrégat »). (b) Simulons.

```
def K_thomas_theorique(r, kappa, sigma):
    return np.pi * r ** 2 + (1 / kappa) * (1 - np.exp(-r ** 2 / (4 * sigma ** 2)))

rs12 = np.array([0.25, 0.5, 1.0, 1.5, 2.0])
for kappa, mu, sigma, B in [(0.5, 2, 0.4, 500), (0.08, 12, 0.4, 300)]:
    rng12 = np.random.default_rng(12)
    Ks, effectifs = [], []
    while len(Ks) < B:
        pts12 = semis_thomas(kappa, mu, sigma, rng12)
        if len(pts12) > 10:
            Ks.append(ripley_K(pts12, rs12))
            effectifs.append(len(pts12))
    Ks = np.array(Ks)
    theo = K_thomas_theorique(rs12, kappa, sigma)
    tab = pd.DataFrame({"r": rs12, "pi r^2 (hasard)": np.pi * rs12 ** 2, "K théorique": theo, "K
    ↪ simulé (moyenne)": np.nanmean(Ks, axis=0)})
    tab["écart_%"] = 100 * (tab["K simulé (moyenne)"] / tab["K théorique"] - 1)
    print(f"kappa = {kappa}, mu = {mu}, sigma = {sigma} ({B} semis) | nombre de points : moyenne
    ↪ {np.mean(effectifs):.0f}, "
        f"écart-type {np.std(effectifs):.0f}, coefficient de variation {np.std(effectifs) /
        ↪ np.mean(effectifs):.0%}")
    print(tab.round(2).to_string(index=False))
    print()
```

```
kappa = 0.5, mu = 2, sigma = 0.4 (500 semis) | nombre de points : moyenne 101, écart-type 17,
    ↪ coefficient de variation 16%
```

| r    | pi r <sup>2</sup> (hasard) | K théorique | K simulé (moyenne) | écart_% |
|------|----------------------------|-------------|--------------------|---------|
| 0.25 | 0.20                       | 0.38        | 0.38               | -0.15   |
| 0.50 | 0.79                       | 1.43        | 1.43               | -0.19   |
| 1.00 | 3.14                       | 4.72        | 4.68               | -0.82   |
| 1.50 | 7.07                       | 9.01        | 8.81               | -2.25   |
| 2.00 | 12.57                      | 14.56       | 13.96              | -4.13   |

```
kappa = 0.08, mu = 12, sigma = 0.4 (300 semis) | nombre de points : moyenne 97, écart-type 31,
    ↪ coefficient de variation 32%
```

| r    | pi r <sup>2</sup> (hasard) | K théorique | K simulé (moyenne) | écart_% |
|------|----------------------------|-------------|--------------------|---------|
| 0.25 | 0.20                       | 1.36        | 1.46               | 7.63    |
| 0.50 | 0.79                       | 4.83        | 5.16               | 6.94    |
| 1.00 | 3.14                       | 13.02       | 13.67              | 4.95    |
| 1.50 | 7.07                       | 19.20       | 19.05              | -0.77   |
| 2.00 | 12.57                      | 25.04       | 23.21              | -7.34   |

Pour le premier réglage (agrégats faibles et nombreux : en moyenne 2 descendants par centre), la simulation retrouve la théorie à moins de 1 % jusqu'à  $r = 1$  km, puis s'en écarte de  $-2,3\%$  à 1,5 km et de  $-4,1\%$  à 2 km. Pour le second (agrégats forts : 12 descendants par centre), les écarts sont nettement plus importants et de **signe variable** :  $+7,6\%$  à 0,25 km,  $+5\%$  à 1 km,  $-7,3\%$  à 2 km. **Cela ne signifie pas que la formule est fausse**, mais que l'estimateur  $\hat{K}$  est biaisé quand le nombre total de points varie beaucoup d'un semis à l'autre : le coefficient de variation de  $n$  est de 16 % pour le premier réglage et de 32 % pour le second (voir la ligne imprimée au-dessus de chaque tableau). Une cause probable est que l'estimateur divise par un nombre de points aléatoire, lui-même corrélé aux comptages de voisins : l'espérance d'un rapport n'est pas le rapport des espérances. Ajoutons que, pour les grandes distances, la méthode du bord n'utilise qu'une petite fraction des points (à 2 km, seuls ceux situés à plus de 2 km du bord, soit 36 %). **Morale : ne pas surinterpréter  $\hat{K}$  pour des semis très agrégés ni aux grandes distances**, ce que résume la règle de ne pas dépasser le quart du côté de la fenêtre.

## Bilan du chapitre 9

Vous savez maintenant :

- distinguer les **trois familles** de données spatiales (géostatistique, surfacique, semis de points) en se demandant *qu'est-ce qui est aléatoire ?* ;
- repérer un point et **mesurer une distance** correctement (haversine, projection locale) et dessiner une carte honnête sans fond de carte ;
- **définir un voisinage** par une matrice de poids et mesurer l'**autocorrélation** avec les indices de **Moran** et de **Geary**, les tester par **permutations**, et localiser les îlots avec les indices locaux (LISA) en **corrigeant** les tests multiples ;
- expliquer pourquoi ignorer l'autocorrélation fait **rejeter à tort** (46 % de faux positifs au lieu de 5 % dans notre simulation) et la tester sur les **résidus** d'un modèle ;
- estimer et ajuster un **variogramme** (pépité, palier, portée), connaître ses incertitudes, et **prédire par krigeage** avec une variance d'erreur, en validant par **validation croisée** ;
- étudier un **semis de points** par quadrats, plus proche voisin et **fonction K de Ripley**, avec une **enveloppe de Monte-Carlo** et une **correction de bord**.

Ce chapitre a aussi rassemblé les **limites** qui reviennent à chaque étape :

| Limite                                         | Où elle apparaît                                                  | Précaution                                                                                                       |
|------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Effet de bord</b>                           | plus proche voisin (biais de $R$ ), fonction $K$ , indices locaux | correction (Donnelly, méthode du bord), enveloppe de Monte-Carlo dans la même fenêtre, examiner les bords à part |
| <b>Problème de l'unité spatiale modifiable</b> | corrélations et indices calculés sur des zones                    | indiquer le découpage, tester plusieurs échelles, ne pas transposer à l'individu                                 |
| <b>Non-stationnarité</b>                       | variogramme, krigeage                                             | retirer ou modéliser la tendance (krigeage universel)                                                            |
| <b>Anisotropie</b>                             | variogramme                                                       | variogrammes directionnels, <b>calibrés</b> par simulation                                                       |
| <b>Densité variable</b>                        | fonction $K$                                                      | $K$ inhomogène ; ne pas confondre premier et second ordre                                                        |
| <b>Variogramme estimé, pas connu</b>           | barres d'erreur du krigeage                                       | validation croisée avec réajustement, prudence sur les intervalles                                               |

**Si vous ne deviez retenir qu'une idée.** *L'espace n'est pas une colonne de plus dans le tableau : il change la nature de l'information.* Des observations voisines ne sont pas indépendantes, donc : définissez le voisinage, mesurez la dépendance, vérifiez-la dans les résidus, et attribuez-lui une incertitude honnête.

**Pour aller plus loin** (hors de ce chapitre) : les modèles de régression spatiale (*spatial lag* et *spatial error*, par maximum de vraisemblance), le krigeage universel et le co-krigeage, les modèles géostatistiques bayésiens, la fonction  $K$  inhomogène et les modèles de Cox log-gaussiens pour les semis de points, la statistique spatio-temporelle. Ces extensions reposent exactement sur les trois idées que vous venez de pratiquer : un **voisinage**, une **fonction de dépendance** et une **comparaison à un hasard bien défini**.

# Projet du volume : le plan 2026 de Dar Jasmin

« Prévoir, c'est facile : il suffit de se tromper de façon honnête, et de dire de combien. »

Dans le volume I, le projet de clôture assemblait des mesures simples. Ici, nous assemblons des **modèles** : une série temporelle pour prévoir, un modèle linéaire généralisé pour comprendre qui rachète et combien chaque client dépense, un modèle de survie pour savoir combien de temps un client reste. À la fin, ces trois résultats se combinent en une **décision** : l'offre de bienvenue vaut-elle son coût ?

**Comment lire ce projet.** Il n'introduit aucune notion nouvelle : chaque étape renvoie à la section où la méthode est expliquée. Le plus profitable : lire le cahier des charges (P.1), fermer le livre, essayer de répondre vous-même, puis comparer. Les données sont **simulées** (voir l'introduction du volume) : en P.8, nous dévoilerons ce qui avait été programmé et nous verrons ce que nos modèles en ont retrouvé.

## P.1 Le cahier des charges

Début janvier 2026, Yasmine vous écrit :

« Bonjour ! 2025 est bouclée, je prépare le budget de 2026. J'ai quatre questions.

1. Combien vais-je vendre en 2026, mois par mois ? J'ai besoin d'une fourchette, pas seulement d'un chiffre : je dois prévoir ma trésorerie.
2. L'offre de bienvenue que je tire au sort pour les nouveaux clients, est-ce qu'elle les fait revenir ? Et de combien ?
3. Un client, ça me rapporte combien par an ? Ça dépend de l'âge, du canal ?
4. Et combien de temps un client reste-t-il ? Au fond, je voudrais savoir ce que vaut un client, et si l'offre de bienvenue (elle me coûte environ 10 DT par client) vaut la peine. Merci ! »

La méthode suit six étapes. Chacune s'appuie sur un chapitre du volume.

| Étape                   | Question                         | Outil                                      | Chapitre          |
|-------------------------|----------------------------------|--------------------------------------------|-------------------|
| 1. Contrôler            | Les données sont-elles fiables ? | vérifications, assertions                  | volume I (projet) |
| 2. Prévoir              | Les ventes de 2026               | SARIMA avec variables exogènes, rétro-test | 4                 |
| 3. Comprendre le rachat | Effet de l'offre de bienvenue    | régression logistique                      | 2 (2.2, 2.4)      |



| Étape        | Question                                            | Outil                                 | Chapitre                            |
|--------------|-----------------------------------------------------|---------------------------------------|-------------------------------------|
| 4. Valoriser | Combien rapporte un client ?                        | modèle Tweedie / Gamma                | 2 (2.3, 2.6), 1                     |
| 5. Durer     | Combien de temps reste un client ?                  | Kaplan-Meier, Cox                     | 5                                   |
| 6. Décider   | Que vaut un client ?<br>L'offre est-elle rentable ? | combinaison des modèles + incertitude | 5, 6 (bootstrap du volume I, 3.3.5) |

## P.2 Étape 1 : charger et contrôler

```
import warnings
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")          # les avertissements de convergence sont examinés un par un
↳ dans le texte

clients = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
ventes = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"], index_col="mois")
ventes.index.freq = "MS"

controles = {
    "2 000 clients, identifiants uniques": clients["id_client"].is_unique and len(clients) == 2000,
    "aucune valeur manquante (clients)": int(clients.isna().sum().sum()) == 0,
    "offre_bienvenue ne vaut que 0 ou 1": set(clients["offre_bienvenue"]) == {0, 1},
    "dépense nulle si et seulement si aucune commande": bool(((clients["depense_annuelle"] == 0) ==
↳ (clients["nb_commandes_an"] == 0)).all()),
    "durée de relation positive ou nulle": bool((clients["duree_mois"] >= 0).all()),
    "120 mois consécutifs sans trou": len(ventes) == 120 and bool((ventes.index ==
↳ pd.date_range("2016-01-01", periods=120, freq="MS")).all()),
    "chiffre d'affaires strictement positif": bool((ventes["ca"] > 0).all()),
}
for nom, ok in controles.items():
    print("OK " if ok else "ÉCHEC", nom)
assert all(controles.values())

OK 2 000 clients, identifiants uniques
OK aucune valeur manquante (clients)
OK offre_bienvenue ne vaut que 0 ou 1
OK dépense nulle si et seulement si aucune commande
OK durée de relation positive ou nulle
OK 120 mois consécutifs sans trou
OK chiffre d'affaires strictement positif
```

Comme au volume I, un contrôle qui échoue **arrête** le programme : on préfère un plantage bruyant à un rapport faux.

## P.3 Étape 2 : prévoir les ventes de 2026 (question 1)

### P.3.1 Regarder, puis fixer des repères

Une série temporelle se **dessine** avant de se modéliser (chapitre 4, section 4.1). Sur l'échelle logarithmique, la saisonnalité devient additive et la tendance presque linéaire :

```
y = np.log(ventes["ca"])
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].plot(ventes.index, ventes["ca"], color="#2a78d6", lw=1.6)
ax[0].axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-06-30"), color="#e34948", alpha=0.15)
ax[0].text(pd.Timestamp("2020-03-15"), ventes["ca"].max() * 0.93, "2020", color="#e34948",
           fontweight="bold", fontstyle="italic", fontfamily="serif",
           fontsize=9)
ax[0].set_title("Chiffre d'affaires mensuel (DT)")
ax[0].set_ylabel("DT")
ax[1].plot(ventes.index, y, color="#2a78d6", lw=1.6)
ax[1].set_title("Même série en logarithme")
ax[1].set_ylabel("log(DT)")
plt.tight_layout()
plt.savefig("figures/ch10-serie-ventes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

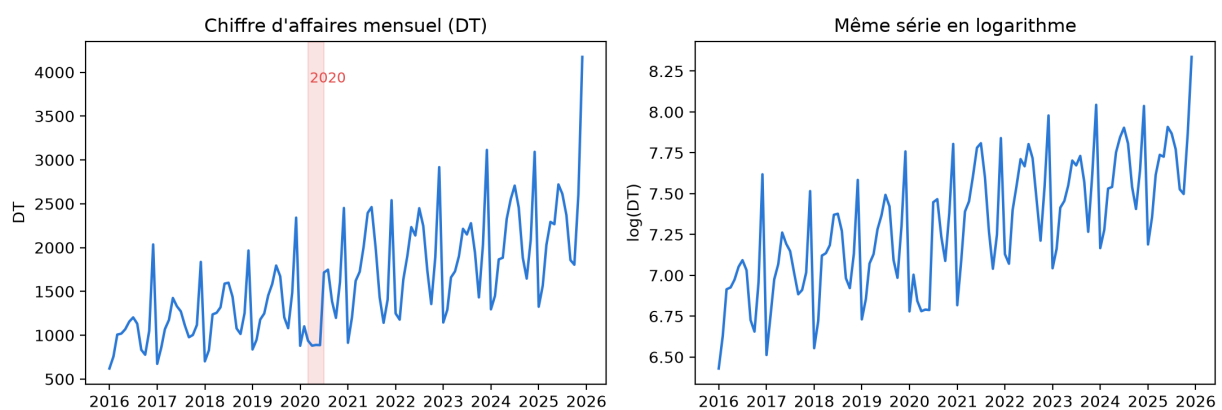


Figure 101: À gauche : chiffre d'affaires mensuel de Dar Jasmin de 2016 à 2025 (la bande rouge marque l'arrêt du printemps 2020). À droite : la même série en logarithme, où la saisonnalité est de même amplitude à tous les niveaux.

Pour juger un modèle de prévision, il faut un **repère** : une méthode si simple que ne pas la battre serait inquiétant. Le plus utile ici est la **prévision naïve saisonnière** : « chaque mois ressemblera au même mois de l'an passé ».

On s'entraîne sur 2016-2023 et on teste sur 2024-2025 : le modèle ne voit jamais les mois qu'il doit prévoir (chapitre 4, section 4.3). La mesure d'erreur est le **MAPE**, l'erreur absolue moyenne en pourcentage.

```
X = ventes[["promo", "covid"]].astype(float)
train, test = slice(None, "2023-12-01"), slice("2024-01-01", None)

def mape(reel, prevu):
    return float(np.mean(np.abs(reel - prevu) / reel) * 100)

naif_saisonnier = ventes["ca"].shift(12)[test]
print(f"MAPE du naïf saisonnier sur 2024-2025 : {mape(ventes['ca'][test], naif_saisonnier):.2f} %")
```

MAPE du naïf saisonnier sur 2024-2025 : 9.52 %

### P.3.2 Ajuster et comparer des modèles SARIMA

Quatre candidats, tous avec une composante saisonnière annuelle et les deux variables exogènes promo (promotion ce mois-ci) et covid (arrêt de 2020) :

```
candidats = {
    "SARIMA(1,1,0)(0,1,1)12": dict(order=(1, 1, 0), seasonal_order=(0, 1, 1, 12), trend="n"),
    "SARIMA(1,1,1)(0,1,1)12": dict(order=(1, 1, 1), seasonal_order=(0, 1, 1, 12), trend="n"),
    "SARIMA(1,0,0)(0,1,1)12 + constante": dict(order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="c"),
    "SARIMA(2,0,0)(0,1,1)12 + constante": dict(order=(2, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="c"),
}
lignes, ajustes = [], {}
for nom, spec in candidats.items():
    res = sm.tsa.SARIMAX(y[train], exog=X[train], **spec).fit(dispatch=False)
    prevu = np.exp(res.get_forecast(24, exog=X[test]).predicted_mean)
    ajustes[nom] = res
    lignes.append({"modèle": nom, "AIC": round(res.aic, 1), "MAPE test (%)":
    ↪ round(mape(ventes["ca"][test], prevu), 2)})
comparaison = pd.DataFrame(lignes)
print(comparaison.to_string(index=False))
```

|  | modèle                             | AIC    | MAPE test (%) |
|--|------------------------------------|--------|---------------|
|  | SARIMA(1,1,0)(0,1,1)12             | -164.5 | 6.93          |
|  | SARIMA(1,1,1)(0,1,1)12             | -176.7 | 8.23          |
|  | SARIMA(1,0,0)(0,1,1)12 + constante | -185.0 | 8.36          |
|  | SARIMA(2,0,0)(0,1,1)12 + constante | -184.0 | 8.29          |

**L'AIC n'est comparable qu'entre modèles de même ordre de différenciation.** Les deux modèles « avec constante » n'ont pas été différenciés ( $d = 0$ ) : leur AIC est calculé sur la série elle-même, alors que celui des deux premiers l'est sur la série différenciée. Comparer ces AIC n'a pas de sens, même si le tableau les met côte à côte. C'est le **MAPE sur la période de test**, mesuré sur des données que le modèle n'a pas vues, qui arbitre ici (chapitre 4, section 4.3).

On retient le meilleur modèle sur le test (son MAPE, 6,93 %, bat nettement le repère naïf saisonnier, 9,52 %), puis on regarde si ses résidus ressemblent à du bruit blanc : si ce n'est pas le cas, le modèle a laissé du signal derrière lui.

```
meilleur = comparaison.sort_values("MAPE test (%)").iloc[0]["modèle"]
res = ajustes[meilleur]
print("modèle retenu :", meilleur)
from statsmodels.stats.diagnostic import acorr_ljungbox
residus = res.resid[13:] # on écarte les premiers résidus, dominés par l'initialisation
lb = acorr_ljungbox(residus, lags=[12, 24], return_df=True)
print(lb.round(3).to_string())
```

```
modèle retenu : SARIMA(1,1,0)(0,1,1)12
  lb_stat lb_pvalue
12   13.760    0.316
24   25.206    0.395
```

Les p-valeurs de Ljung-Box (chapitre 4, section 4.1) sont élevées : on ne détecte pas d'autocorrélation résiduelle. Le modèle a capté la structure.

### P.3.3 Prévoir 2026

On réajuste le modèle retenu sur **les 120 mois**, puis on prévoit les 12 mois de 2026. Il faut fournir les valeurs futures des variables exogènes : pas de nouvel arrêt (covid = 0), et une hypothèse sur les promotions. Yasmine prévoit une promotion en décembre, comme en 2025 :

```
spec = candidats[meilleur]
final = sm.tsa.SARIMAX(y, exog=X, **spec).fit(dispatch=False)
futur = pd.date_range("2026-01-01", periods=12, freq="MS")
X_futur = pd.DataFrame({"promo": 0.0, "covid": 0.0}, index=futur)
X_futur.loc["2026-12-01", "promo"] = 1.0
pred = final.get_forecast(12, exog=X_futur)
ic = pred.conf_int(alpha=0.05)
prevision = pd.DataFrame({
    "prévision": np.exp(pred.predicted_mean),
    "bas_95": np.exp(ic.iloc[:, 0]),
    "haut_95": np.exp(ic.iloc[:, 1]),
}).round(0)
prevision.index = prevision.index.strftime("%Y-%m")
print(prevision.to_string())
print()
total_2025 = ventes.loc["2025", "ca"].sum()
total_2026 = prevision["prévision"].sum()
print(f"total 2025 : {total_2025:,.0f} DT    total 2026 prévu : {total_2026:,.0f} DT    ({100 *
↵ (total_2026 / total_2025 - 1):+.1f} %)")
```

|         | prévision | bas_95 | haut_95 |
|---------|-----------|--------|---------|
| 2026-01 | 1517.0    | 1306.0 | 1762.0  |
| 2026-02 | 1835.0    | 1504.0 | 2241.0  |
| 2026-03 | 2407.0    | 1893.0 | 3060.0  |
| 2026-04 | 2531.0    | 1923.0 | 3331.0  |
| 2026-05 | 2838.0    | 2091.0 | 3852.0  |
| 2026-06 | 3087.0    | 2212.0 | 4309.0  |
| 2026-07 | 3215.0    | 2245.0 | 4603.0  |
| 2026-08 | 2972.0    | 2026.0 | 4360.0  |
| 2026-09 | 2324.0    | 1549.0 | 3487.0  |
| 2026-10 | 2023.0    | 1320.0 | 3101.0  |
| 2026-11 | 2681.0    | 1714.0 | 4194.0  |
| 2026-12 | 4599.0    | 2883.0 | 7335.0  |

total 2025 : 27,630 DT    total 2026 prévu : 32,029 DT    (+15.9 %)

Les bornes à 95 % s'écartent à mesure que l'on s'éloigne dans le futur : l'incertitude s'accumule. Voici la prévision en image :

```
fig, ax = plt.subplots(figsize=(10, 3.9))
recent = ventes.loc["2023":]
ax.plot(recent.index, recent["ca"], color="#2a78d6", lw=1.8, label="observé")
ax.plot(futur, np.exp(pred.predicted_mean), color="#eb6834", lw=1.8, label="prévision 2026")
ax.fill_between(futur, np.exp(ic.iloc[:, 0]), np.exp(ic.iloc[:, 1]), color="#eb6834", alpha=0.18,
↵ label="intervalle à 95 %")
ax.set_ylabel("chiffre d'affaires (DT)")
ax.set_title("Prévision du chiffre d'affaires mensuel de 2026")
ax.legend(frameon=False, loc="upper left")
plt.tight_layout()
plt.savefig("figures/ch10-prevision-2026.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

**Ce que l'intervalle ne couvre pas.** L'intervalle à 95 % suppose que le mécanisme reste le même en 2026. Il ne couvre ni une nouvelle crise, ni une concurrence nouvelle, ni un chan-

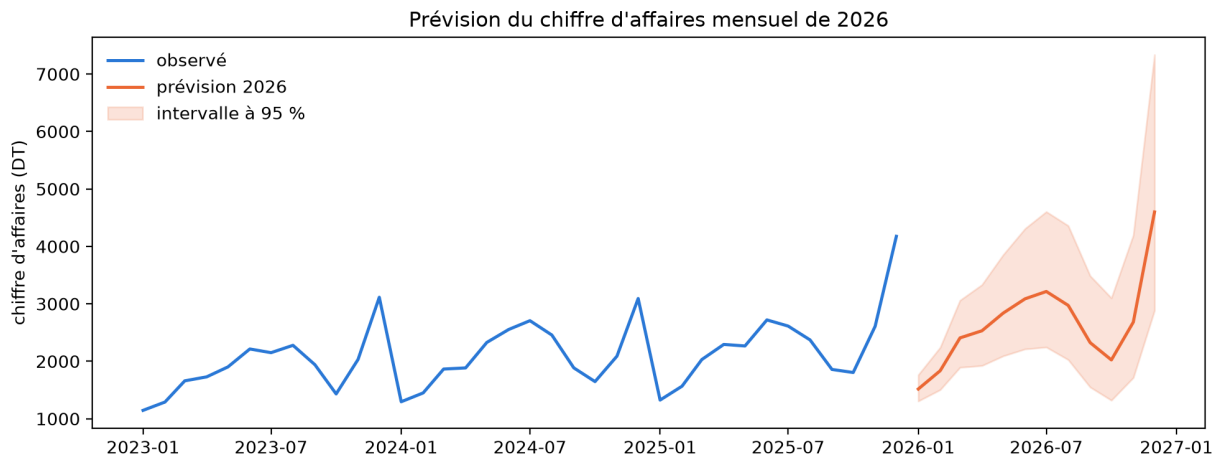


Figure 102: Chiffre d'affaires observé depuis 2023 et prévision mois par mois pour 2026, avec intervalle de prévision à 95 %.

gement de stratégie. De plus, la prévision est faite sur le logarithme puis ramenée à l'échelle d'origine : c'est une **prévision de la médiane** du mois, un peu inférieure à sa moyenne (volume I, section 2.3 : l'espérance d'une exponentielle dépasse l'exponentielle de l'espérance). Pour la trésorerie, on regardera donc plutôt la borne basse que la valeur centrale.

## P.4 Étape 3 : l'offre de bienvenue fait-elle revenir les clients ? (question 2)

La variable à expliquer est binaire (`rachat_12m`) : une **régression logistique** (chapitre 2, section 2.2). Surtout, l'offre a été **tirée au sort** : c'est la situation idéale où l'on peut lire l'effet comme un effet **causal** (chapitre 7 explique pourquoi). Les autres variables (`âge`, `canal`) servent à affiner, pas à identifier.

D'abord la comparaison brute, puis le modèle :

```
print(clients.groupby("offre_bienvenue")["rachat_12m"].agg(clients="count",
↳ taux_rachat="mean").round(3).to_string())
print()
logit = smf.logit("rachat_12m ~ offre_bienvenue + I(age - 36) + C(canal_acquisition)",
↳ clients).fit(dis=0)
tab = pd.DataFrame({"coef": logit.params, "rapport_de_cotes": np.exp(logit.params),
"IC95_bas": np.exp(logit.conf_int()[0]), "IC95_haut": np.exp(logit.conf_int()[1]),
"p": logit.pvalues}).round(3)
print(tab.to_string())
```

|                                   | clients | taux_rachat |        |                  |          |           |
|-----------------------------------|---------|-------------|--------|------------------|----------|-----------|
| offre_bienvenue                   |         |             |        |                  |          |           |
| 0                                 | 985     | 0.448       |        |                  |          |           |
| 1                                 | 1015    | 0.569       |        |                  |          |           |
|                                   |         |             | coef   | rapport_de_cotes | IC95_bas | IC95_haut |
| Intercept                         |         |             | 0.031  | 1.031            | 0.847    | 1.255     |
| C(canal_acquisition)[T.Instagram] |         |             | -0.463 | 0.629            | 0.502    | 0.789     |
| C(canal_acquisition)[T.Site]      |         |             | -0.168 | 0.846            | 0.669    | 1.069     |
| offre_bienvenue                   |         |             | 0.493  | 1.638            | 1.370    | 1.957     |
| I(age - 36)                       |         |             | -0.015 | 0.985            | 0.976    | 0.993     |
|                                   |         |             |        |                  |          | p         |
|                                   |         |             |        |                  |          | 0.761     |
|                                   |         |             |        |                  |          | 0.000     |
|                                   |         |             |        |                  |          | 0.161     |
|                                   |         |             |        |                  |          | 0.000     |
|                                   |         |             |        |                  |          | 0.000     |

Un coefficient s'interprète sur l'échelle **logit** ; on le rend lisible en l'exponentiant, ce qui donne un

**rapport de cotes** (*odds ratio*). Mais un rapport de cotes n'est pas une différence de probabilité, et c'est la seconde que Yasmine veut : « de combien l'offre augmente-t-elle la probabilité de racheter ? ». Nous la calculons en **prédisant deux mondes** : le monde où chaque client a reçu l'offre, et celui où aucun ne l'a reçue (l'« effet marginal moyen »), puis nous en donnons un intervalle de confiance par **bootstrap** (volume I, section 3.3.5).

```
def effet_offre(df):
    m = smf.logit("rachat_12m ~ offre_bienvenue + I(age - 36) + C(canal_acquisition)", df).fit(dis=0)
    return float((m.predict(df.assign(offre_bienvenue=1)) -
                  m.predict(df.assign(offre_bienvenue=0))).mean())

ame = effet_offre(clients)
rng = np.random.default_rng(2026)
boot = np.array([effet_offre(clients.sample(len(clients), replace=True, random_state=int(s)))
                  for s in rng.integers(0, 2*31 - 1, 300)])
ame_bas, ame_haut = np.percentile(boot, [2.5, 97.5])
print(f"effet moyen de l'offre sur la probabilité de rachat : {100 * ame:+.1f} points")
print(f"intervalle de confiance à 95 % (bootstrap, 300 rééchantillons) : [{100 * ame_bas:+.1f} ; {100 *
    * ame_haut:+.1f}] points")
```

```
effet moyen de l'offre sur la probabilité de rachat : +12.1 points
intervalle de confiance à 95 % (bootstrap, 300 rééchantillons) : [+7.6 ; +16.3] points
```

Il reste à vérifier que le modèle est **utilisable**. Deux contrôles (chapitre 2, section 2.4) : son pouvoir de discrimination (l'aire sous la courbe ROC) et sa **calibration** (quand le modèle dit « 60 % », voit-on environ 60 % de rachats ?).

```
from sklearn.metrics import roc_auc_score
p = logit.predict(clients)
print(f"AUC : {roc_auc_score(clients['rachat_12m'], p):.3f}")
decile = pd.qcut(p, 5, labels=False)
calib = clients.assign(p=p, groupe=decile).groupby("groupe").agg(
    clients=("p", "size"), proba_moyenne_prevue=("p", "mean"), taux_observed=("rachat_12m",
    "mean")).round(3)
print(calib.to_string())
```

```
AUC : 0.597
      clients  proba_moyenne_prevue  taux_observed
groupe
0           410                0.386            0.371
1           394                0.462            0.497
2           398                0.512            0.515
3           400                0.560            0.538
4           398                0.630            0.631
```

**Lire une AUC modeste.** Une AUC proche de 0,6 signifie que le modèle ne trie pas très bien les clients individuellement : savoir qu'un client est jeune, d'Instagram, et qu'il a reçu l'offre aide peu à prédire **son** rachat, parce que l'essentiel de la variabilité tient à des facteurs que nous n'observons pas. Cela ne contredit pas la qualité de l'estimation de l'**effet moyen** de l'offre : on peut estimer très précisément une différence moyenne tout en prédisant mal chaque individu. Prédire et expliquer sont deux tâches distinctes.

## P.5 Étape 4 : combien rapporte un client ? (question 3)

La dépense annuelle est positive, asymétrique, avec **13 % de zéros** (les clients sans commande) : ni la normale, ni le modèle Gamma seul (qui exige des valeurs strictement positives) ne conviennent. Deux solutions du chapitre 2 :

1. le modèle de **Tweedie** (section 2.6), qui gère les zéros et la partie positive d'un seul tenant ;

2. un modèle **en deux parties** (section 2.3 et 2.6) : une logistique pour « le client passe-t-il au moins une commande ? », puis un Gamma sur la dépense des acheteurs.

```
clients["achete"] = (clients["nb_commandes_an"] > 0).astype(int)
formule = "~ I(age - 36) + C(canal_acquisition) + offre_bienvenue"

tweedie = smf.glm("depense_annuelle " + formule, clients,
                  family=sm.families.Tweedie(var_power=1.5, link=sm.families.links.Log())).fit()
partiel1 = smf.logit("achete " + formule, clients).fit(dis=0)
partiel2 = smf.glm("depense_annuelle " + formule, clients[clients["achete"] == 1],
                  family=sm.families.Gamma(sm.families.links.Log())).fit()

deux_parties = partiel1.predict(clients) * partiel2.predict(clients)
print(f"dépense annuelle moyenne observée      : {clients['depense_annuelle'].mean():.1f} DT")
print(f"prévue par le modèle de Tweedie         : {tweedie.predict(clients).mean():.1f} DT")
print(f"prévue par le modèle en deux parties     : {deux_parties.mean():.1f} DT")
print(f"corrélation entre les deux prévisions    : {np.corrcoef(tweedie.predict(clients),
  ↳ deux_parties)[0, 1]:.3f}")
print()
coefs = pd.DataFrame({"Tweedie": tweedie.params, "Gamma (acheteurs)": partiel2.params}).round(3)
coefs["effet Tweedie (%)"] = ((np.exp(coefs["Tweedie"]) - 1) * 100).round(1)
coefs.loc["Intercept", "effet Tweedie (%)"] = np.nan      # l'ordonnée à l'origine ne se lit pas en
  ↳ pourcentage
print(coefs.to_string())
print()
print(f"effet de l'offre sur la probabilité d'au moins une commande : coef logit =
  ↳ {partiel1.params['offre_bienvenue']:+.3f}, p = {partiel1.pvalues['offre_bienvenue']:.2f}")
```

```
dépense annuelle moyenne observée      : 247.0 DT
prévue par le modèle de Tweedie         : 247.0 DT
prévue par le modèle en deux parties     : 247.0 DT
corrélation entre les deux prévisions    : 1.000
```

|                                   | Tweedie | Gamma (acheteurs) | effet Tweedie (%) |
|-----------------------------------|---------|-------------------|-------------------|
| Intercept                         | 5.766   | 5.880             | NaN               |
| C(canal_acquisition)[T.Instagram] | -0.555  | -0.499            | -42.6             |
| C(canal_acquisition)[T.Site]      | -0.151  | -0.152            | -14.0             |
| I(age - 36)                       | 0.008   | 0.008             | 0.8               |
| offre_bienvenue                   | -0.014  | -0.010            | -1.4              |

```
effet de l'offre sur la probabilité d'au moins une commande : coef logit = -0.027, p = 0.84
```

Les deux approches donnent des prévisions pratiquement identiques. Les coefficients d'un modèle à lien log se lisent en **pourcentages** :  $\exp(\text{coef}) - 1$  est la variation relative de la dépense moyenne pour une unité de la variable. Le tableau montre un résultat **nuancé et important pour la suite** : l'offre de bienvenue n'a pratiquement **aucun effet** sur la dépense annuelle (ni sur la probabilité de passer commande), alors que le canal d'acquisition en a un net.

```
profils = pd.DataFrame({"age": [25, 25, 40, 40], "canal_acquisition": ["Instagram", "Boutique",
  ↳ "Instagram", "Boutique"],
  ↳ "offre_bienvenue": [0, 0, 0, 0]})
profils["valeur_annuelle_DT"] = tweedie.predict(profils).round(1)
print(profils.to_string(index=False))
```

| age | canal_acquisition | offre_bienvenue | valeur_annuelle_DT |
|-----|-------------------|-----------------|--------------------|
| 25  | Instagram         | 0               | 167.5              |
| 25  | Boutique          | 0               | 291.8              |
| 40  | Instagram         | 0               | 189.2              |
| 40  | Boutique          | 0               | 329.8              |



## P.6 Étape 5 : combien de temps un client reste-t-il ? (question 4)

Ici, la durée est **censurée** : 1 023 clients sur 2 000 sont encore là en décembre 2025. Les ignorer, ou les traiter comme des départs, biaiserait tout (chapitre 5, section 5.1). On commence par l'**estimateur de Kaplan-Meier** de la fonction de survie, écrit à la main (section 5.2) : à chaque date de départ, on multiplie la survie par la proportion de clients qui ont survécu parmi ceux qui étaient encore là.

```
def kaplan_meier(durees, evenements):
    """Retourne les instants de départ et la survie S(t) correspondante."""
    durees, evenements = np.asarray(durees, float), np.asarray(evenements, int)
    instants = np.unique(durees[evenements == 1])
    s, courbe = 1.0, []
    for t in instants:
        a_risque = np.sum(durees >= t)
        depart = np.sum((durees == t) & (evenements == 1))
        s *= 1 - depart / a_risque
        courbe.append(s)
    return instants, np.array(courbe)

def survie_en(t, instants, courbe):
    """S(t) : valeur de l'escalier en t (1 avant le premier départ)."""
    i = np.searchsorted(instants, t, side="right") - 1
    return 1.0 if i < 0 else courbe[i]

def survie_mediane(instants, courbe):
    sous = np.where(courbe <= 0.5)[0]
    return float(instants[sous[0]]) if len(sous) else np.nan

for nom, g in clients.groupby("offre_bienvenue"):
    t, s = kaplan_meier(g["duree_mois"], g["churn"])
    print(f"offre = {nom} : médiane de survie = {survie_mediane(t, s):.1f} mois ; "
          f"survie à 12 mois = {survie_en(12, t, s):.3f} ; à 24 mois = {survie_en(24, t, s):.3f} ; à "
          f"↪ 36 mois = {survie_en(36, t, s):.3f}")
```

```
offre = 0 : médiane de survie = 28.1 mois ; survie à 12 mois = 0.790 ; à 24 mois = 0.573 ; à 36
↪ mois = 0.393
offre = 1 : médiane de survie = 36.6 mois ; survie à 12 mois = 0.858 ; à 24 mois = 0.691 ; à 36
↪ mois = 0.509
```

Vérifions notre implémentation contre celle de `statsmodels`, puis ajustons le **modèle de Cox** (section 5.3), qui estime l'effet de plusieurs variables à la fois sur le risque de départ :

```
from statsmodels.duration.survfunc import SurvfuncRight
g1 = clients[clients["offre_bienvenue"] == 1]
sf = SurvfuncRight(g1["duree_mois"], g1["churn"])
t_h, s_h = kaplan_meier(g1["duree_mois"], g1["churn"])
ecarts = [abs(survie_en(t, t_h, s_h) - float(sf.surv_prob[max(np.searchsorted(sf.surv_times, t,
    ↪ side="right") - 1, 0]))
            for t in range(6, 61, 6)]
print(f"écart maximal avec statsmodels sur la grille 6, 12, ..., 60 mois : {max(ecarts):.1e}")

from statsmodels.duration.hazard_regression import PHReg
exog = pd.get_dummies(clients[["offre_bienvenue", "age", "canal_acquisition"]],
    ↪ columns=["canal_acquisition"], drop_first=True, dtype=float)
cox = PHReg(clients["duree_mois"], exog, status=clients["churn"]).fit()
resume = pd.DataFrame({"coef": cox.params, "rapport_de_risques": np.exp(cox.params),
    ↪ "IC95_bas": np.exp(cox.params - 1.96 * cox.bse), "IC95_haut": np.exp(cox.params
    ↪ + 1.96 * cox.bse),
    ↪ "p": cox.pvalues}, index=exog.columns).round(3)
print(resume.to_string())
```

écart maximal avec statsmodels sur la grille 6, 12, ..., 60 mois : 2.2e-16

|                             | coef   | rapport_de_risques | IC95_bas | IC95_haut | p   |
|-----------------------------|--------|--------------------|----------|-----------|-----|
| offre_bienvenue             | -0.406 | 0.666              | 0.587    | 0.756     | 0.0 |
| age                         | -0.014 | 0.986              | 0.981    | 0.992     | 0.0 |
| canal_acquisition_Instagram | 0.635  | 1.887              | 1.600    | 2.226     | 0.0 |
| canal_acquisition_Site      | 0.326  | 1.385              | 1.164    | 1.648     | 0.0 |

Un **rapport de risques** inférieur à 1 signifie que la variable *réduit* le risque instantané de départ. L'offre de bienvenue réduit le risque de départ d'environ un tiers ; les clients arrivés par Instagram ou par le site partent plus vite que ceux de la boutique (référence).

## P.7 Étape 6 : que vaut un client, et l'offre est-elle rentable ?

Assemblons les pièces. La **valeur d'un client** sur un horizon donné est la somme de ce qu'il dépensera tant qu'il reste, **actualisée** (un dinar dans cinq ans vaut moins qu'un dinar aujourd'hui). Si  $A$  est la **marge** annuelle (la part de la dépense qui reste après le coût des produits : c'est elle qui paie l'offre, pas le chiffre d'affaires) et  $S(t)$  la probabilité de rester au moins  $t$  ans, alors, avec un taux d'actualisation  $\delta$  :

$$\text{valeur} = A \int_0^{\tau} S(t) e^{-\delta t} dt$$

L'intégrale est la **durée de vie moyenne restreinte actualisée** : le nombre d'années pondérées qu'un client passe chez nous dans l'horizon  $\tau$ . Nous prenons  $\tau = 5$  ans et  $\delta = 8\%$  (hypothèses de calcul, à discuter avec Yasmine), un taux de marge brute de  $40\%$  (hypothèse de calcul, à vérifier avec la comptabilité), appliqué à la dépense annuelle moyenne prévue par le modèle de Tweedie.

```
def duree_actualisee(instants, courbe, horizon_mois=60, taux=0.08):
    """Intègre S(t) e^(-taux t) sur [0, horizon] (t en années), par la méthode des rectangles sur une
    ↪ grille fine."""
    grille = np.linspace(0, horizon_mois, 6001)
    S = np.array([survie_en(t, instants, courbe) for t in grille])
    f = S * np.exp(-taux * grille / 12)
    return float(np.sum((f[:-1] + f[1:]) / 2 * np.diff(grille)) / 12)

depense_annuelle = tweedie.predict(clients).mean()
taux_marge = 0.40
marge_annuelle = taux_marge * depense_annuelle
cout_offre = 10.0
resultats = {}
for nom, g in clients.groupby("offre_bienvenue"):
    t, s = kaplan_meier(g["duree_mois"], g["churn"])
    d = duree_actualisee(t, s)
    resultats[nom] = d
    print(f"offre = {nom} : durée de vie actualisée sur 5 ans = {d:.2f} années ; valeur du client =
    ↪ {marge_annuelle * d:.0f} DT")
gain = marge_annuelle * (resultats[1] - resultats[0])
print(f"\ngain brut de l'offre : {gain:.0f} DT par client ; coût : {cout_offre:.0f} DT ; gain net :
    ↪ {gain - cout_offre:.0f} DT par client")
```

```
offre = 0 : durée de vie actualisée sur 5 ans = 2.23 années ; valeur du client = 220 DT
offre = 1 : durée de vie actualisée sur 5 ans = 2.65 années ; valeur du client = 262 DT
```

```
gain brut de l'offre : 42 DT par client ; coût : 10 DT ; gain net : 32 DT par client
```

Le gain net est-il vraiment positif, ou le hasard de l'échantillon pourrait-il l'expliquer ? Le **bootstrap** donne une réponse : on rééchantillonne les clients, on recalcule tout, et on regarde la disper-

sion du gain net.

```
def gain_net(df):
    d = {}
    for nom, g in df.groupby("offre_bienvenue"):
        t, s = kaplan_meier(g["duree_mois"], g["churn"])
        d[nom] = duree_actualisee(t, s)
    return marge_annuelle * (d[1] - d[0]) - cout_offre

rng = np.random.default_rng(99)
sim = np.array([gain_net(clients.sample(len(clients), replace=True, random_state=int(s))) for s in
    ↪ rng.integers(0, 2**31 - 1, 200)])
gain_bas, gain_haut = np.percentile(sim, [2.5, 97.5])
print(f"gain net par client : {gain_net(clients):.0f} DT    IC95 bootstrap (200 rééchantillons) :
    ↪ [{gain_bas:.0f} ; {gain_haut:.0f}] DT")
print(f"part des rééchantillons où l'offre est rentable : {np.mean(sim > 0):.2f}")
```

```
gain net par client : 32 DT    IC95 bootstrap (200 rééchantillons) : [18 ; 44] DT
part des rééchantillons où l'offre est rentable : 1.00
```

## P.8 Le rapport pour Yasmine, et ce qui avait été programmé

Comme au volume I, le rapport est **généré** à partir des résultats déjà calculés, sans aucun nombre recopié à la main.

```
def fr(x, d=0):
    return f"{x:,.{d}f}".replace(",", " ").replace(".", ",")

t_unique = ventes.loc["2025", "ca"].sum()
rapport = f"""\PLAN 2026 : DAR JASMIN
{"=" * 60}

1. Ventes 2026
- Chiffre d'affaires prévu : {fr(total_2026)} DT (2025 : {fr(t_unique)} DT, soit {100 * (total_2026
    ↪ / t_unique - 1):+.1f} %).
- Mois le plus fort : décembre ({fr(prevision['prévision'].iloc[-1])} DT) ; le plus faible :
    ↪ {prevision['prévision'].idxmin()[-2:]/2026}.
- Précision attendue : erreur moyenne d'environ {mape(ventes['ca'][test],
    ↪ np.exp(ajustes[meilleur].get_forecast(24, exog=X[test]).predicted_mean):.0f} % mois par mois
    ↪ lors du rétro-test.

2. L'offre de bienvenue
- Elle augmente la probabilité de racheter de {100 * ame:.0f} points (intervalle à 95 % : de {100 *
    ↪ ame_bas:.0f} à {100 * ame_haut:.0f} points).
- Elle réduit le risque de départ d'environ {100 * (1 - np.exp(cox.params[0])):.0f} %.
- Elle ne change pas la dépense annuelle d'un client.

3. Valeur d'un client (horizon 5 ans, actualisé à 8 %)
- Dépense annuelle moyenne : {fr(depense_annuelle)} DT (marge supposée de 40 %) ; clients du canal
    ↪ Boutique : plus rentables que ceux d'Instagram.
- Gain net de l'offre : {fr(gain_net(clients))} DT par client (intervalle à 95 % : de
    ↪ {fr(gain_bas)} à {fr(gain_haut)} DT).

Hypothèses et limites : dépense annuelle supposée constante tant que le client reste ; marge brute de
    ↪ 40 % ; coût de l'offre de 10 DT ;
pas de nouvel arrêt d'activité en 2026 ; données d'une seule boutique.
"""
print(rapport)
```

```
PLAN 2026 : DAR JASMIN
```

```
=====
```

## 1. Ventes 2026

- Chiffre d'affaires prévu : 32 029 DT (2025 : 27 630 DT, soit +15.9 %).
- Mois le plus fort : décembre (4 599 DT) ; le plus faible : 01/2026.
- Précision attendue : erreur moyenne d'environ 7 % mois par mois lors du rétro-test.

## 2. L'offre de bienvenue

- Elle augmente la probabilité de racheter de 12 points (intervalle à 95 % : de 8 à 16 points).
- Elle réduit le risque de départ d'environ 33 %.
- Elle ne change pas la dépense annuelle d'un client.

## 3. Valeur d'un client (horizon 5 ans, actualisé à 8 %)

- Dépense annuelle moyenne : 247 DT (marge supposée de 40 %) ; clients du canal Boutique : plus rentables que ceux d'Instagram.
- Gain net de l'offre : 32 DT par client (intervalle à 95 % : de 18 à 44 DT).

Hypothèses et limites : dépense annuelle supposée constante tant que le client reste ; marge brute de 40 % ; coût de l'offre de 10 DT ; pas de nouvel arrêt d'activité en 2026 ; données d'une seule boutique.

**Relisez ce rapport comme Yasmine** : aucun mot technique ne devrait la gêner. L'annexe technique, c'est le reste de ce projet.

## Ce qui avait été programmé

Les données étant simulées, nous pouvons maintenant comparer ce que nos modèles ont **retrouvé** à ce que le générateur contenait (script `build/donnees2.py`) :

| Quantité                                                        | Valeur programmée | Estimée ici                                         |
|-----------------------------------------------------------------|-------------------|-----------------------------------------------------|
| Effet de l'offre sur le logit du rachat                         | +0,55             | voir P.4 ( $\approx +0,49$ , intervalle compatible) |
| Effet du canal Instagram sur le logit du rachat (réf. Boutique) | -0,30             | voir P.4 ( $\approx -0,46$ , intervalle compatible) |
| Effet de l'offre sur la dépense (log)                           | 0                 | voir P.5 ( $\approx -0,01$ )                        |
| Forme de Weibull de la durée de relation                        | 1,35              | non estimée ici (chapitre 5, section 5.4)           |
| Tendance de la série (log, par mois)                            | +0,0075           | cohérente avec la croissance de 2016 à 2025         |

**Pourquoi les estimations ne sont-elles pas exactement les valeurs programmées ?** Deux raisons, qui valent bien au-delà de ce projet. D'abord le **hasard d'échantillonnage** : les intervalles de confiance de P.4 contiennent les valeurs programmées (par exemple, celui de l'effet d'Instagram contient -0,30), ce qui est exactement ce que la théorie promet. Ensuite, la **variabilité non observée** : le générateur fait dépendre le rachat de deux « goûts » latents (produits et service, ceux que révélera l'analyse factorielle du chapitre 3) que notre modèle ne contient pas. Omettre des variables qui influencent la réponse **atténue** les coefficients d'une régression logistique, même si ces variables n'ont aucun lien avec l'offre : l'effet estimé de l'offre (+0,49) est un peu inférieur au +0,55 programmé. Ce phénomène, la *non-collapsibilité* du rapport de cotes (chapitre 2, section 2.2), n'existe pas en régression linéaire. Dans la vie réelle, on n'a jamais la valeur programmée pour comparer : il faut connaître le piège.

## P.9 Un détour par de vraies données

Les modèles ci-dessus ont été testés sur des données simulées. Une dernière vérification : ces méthodes fonctionnent-elles sur des **données réelles** ? Deux jeux sont embarqués dans les bibliothèques, donc disponibles hors ligne : la concentration de CO<sub>2</sub> dans l'atmosphère mesurée à Mauna Loa (Hawaï), et une étude de récidive de détenus libérés (le jeu « Rossi », étudié en analyse de survie).

```
co2 = sm.datasets.co2.load_pandas().data["co2"].resample("MS").mean().interpolate()
yc = np.log(co2)
m = sm.tsa.SARIMAX(yc[:"1999-12"], order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(dispatch=False)
f = np.exp(m.get_forecast(len(yc["2000-01":])).predicted_mean)
reel = co2["2000-01":]
print(f"C02 : {len(co2)} mois ({co2.index[0]:%Y-%m} à {co2.index[-1]:%Y-%m}), prévision hors
↳ échantillon de {len(reel)} mois")
print(f"erreur moyenne en pourcentage : {mape(reel.values, f.values):.2f} %")
print(f"naïf saisonnier : {mape(reel.values[12:], co2.shift(12)['2000-01:'].values[12:]):.2f} %")
```

```
C02 : 526 mois (1958-03 à 2001-12), prévision hors échantillon de 24 mois
erreur moyenne en pourcentage : 0.13 %
naïf saisonnier : 0.40 %
```

Même méthode, même code qu'en P.3 : un SARIMA saisonnier (1,1,1)(0,1,1)<sub>12</sub> sur la série en logarithme, entraîné sur les données jusqu'en 1999 et évalué sur la suite.

Le jeu Rossi : 432 détenus libérés ont été suivis un an ; l'événement est une nouvelle arrestation (arrest), les durées sont en semaines (week), censurées à 52 semaines pour ceux qui n'ont pas été réarrêtés.

```
from lifelines.datasets import load_rossi
rossi = load_rossi()
X_r = rossi[["fin", "age", "race", "wexp", "mar", "paro", "prio"]].astype(float)
cox_r = PHReg(rossi["week"], X_r, status=rossi["arrest"]).fit()
res_r = pd.DataFrame({"rapport_de_risques": np.exp(cox_r.params), "p": cox_r.pvalues},
↳ index=X_r.columns).round(3)
print(f"{len(rossi)} détenus, {int(rossi['arrest'].sum())} réarrestations observées ({100 * (1 -
↳ rossi['arrest'].mean()):.0f} % de censure)")
print(res_r.to_string())
```

```
432 détenus, 114 réarrestations observées (74 % de censure)
   rapport_de_risques    p
fin                0.685  0.048
age                0.944  0.009
race               1.369  0.308
wexp               0.860  0.476
mar                0.649  0.257
paro               0.919  0.664
prio               1.095  0.001
```

Les rapports de risques se lisent comme en P.6 : l'aide financière (fin) est associée à un risque de nouvelle arrestation inférieur d'environ 31 % (rapport de risques de 0,685, p = 0,048, à la limite de la significativité), chaque année d'âge supplémentaire à un risque inférieur de 5,6 %, et chaque condamnation antérieure (prio) à un risque supérieur d'environ 9,5 %.

**Lire avec prudence.** Dans cette étude, seule l'aide financière (fin) a été attribuée **au hasard** ; les autres variables (âge, antécédents, mariage...) sont seulement observées. Seul l'effet de l'aide se lit donc causalement ; les autres coefficients décrivent des associations. Cet exemple est ici pour montrer que le code vu dans ce volume fonctionne tel quel sur des données réelles, pas pour tirer des conclusions de politique pénale.

## P.10 Limites, et la suite

- **Une boutique, un jeu simulé.** La vraie vie a des variables oubliées, des erreurs de saisie, et des phénomènes que nous n'avons pas programmés.
- **Des hypothèses fortes dans la valeur client :** dépense constante dans le temps, taux de marge (40 %) et taux d'actualisation (8 %) fixés, coût de l'offre (10 DT) connu. La rentabilité de l'offre dépend de ces choix : une étude sérieuse ferait varier ces hypothèses (analyse de sensibilité).
- **La prévision est conditionnelle.** Elle suppose que le futur ressemble au passé, y compris pour les variables exogènes que nous avons dû fixer (promo, covid).
- **Prédire n'est pas expliquer.** L'AUC modeste de P.4 le rappelle : un modèle peut estimer correctement un effet moyen sans bien prédire chaque cas.

**À retenir.** Une étude de modélisation complète enchaîne : *contrôler* → *regarder* → *modéliser* → *vérifier le modèle hors échantillon* → *quantifier l'incertitude* → *traduire en décision* → *reconnaître les limites*. Le volume III, consacré à l'apprentissage automatique, reprend ce cycle avec d'autres modèles, plus flexibles, pour la **prédiction**, et la même exigence : savoir de combien l'on se trompe.

# Points clés et auto-évaluation

« Une équation ne devient une compétence que le jour où l'on sait dire, sans notes, à quoi elle sert et quand elle ment. »

Ce dernier chapitre a deux rôles : **fixer l'essentiel** de chaque chapitre, et vous donner un moyen **honnête** de savoir si le volume a fait son travail : quarante-deux questions (trente-six sur les chapitres principaux, six sur les chapitres facultatifs), leurs réponses, et une grille d'auto-évaluation.

## Les points clés, chapitre par chapitre

| Chapitre               | Ce que vous devez emporter                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Régression linéaire | $\hat{\beta} = (X^T X)^{-1} X^T y$ est une <b>projection orthogonale</b> ; Gauss-Markov dit pourquoi c'est le meilleur estimateur linéaire sans biais. Un coefficient se lit « toutes choses égales par ailleurs ». On <b>vérifie</b> (résidus, levier, Cook, VIF) avant de <b>croire</b> les tests. Choisir un modèle : AIC, BIC, validation croisée, jamais le seul $R^2$ . Régulariser (Ridge, Lasso), résister aux aberrations (Huber), tenir compte des groupes (effets mixtes). |
| 2. GLM                 | Une <b>loi</b> de la famille exponentielle, un <b>prédicteur linéaire</b> , un <b>lien</b> : trois choix, un cadre. Estimation par <b>maximum de vraisemblance</b> (IRLS). Logistique pour 0/1 (rapport de cotes $\neq$ risque relatif), Poisson et binomiale négative pour les comptages, Gamma pour les montants, Tweedie pour les montants avec zéros. On vérifie : déviance, résidus, calibration.                                                                                |
| 3. Analyse multivariée | L' <b>ACP</b> diagonalise la matrice de covariance : elle <i>résume</i> . L' <b>analyse factorielle</b> <i>modélise</i> des facteurs cachés. <b>K-means</b> et classification hiérarchique regroupent sans étiquettes, mais <b>rendent toujours un résultat, même sur du bruit</b> : on vérifie par silhouette, stabilité, analyse parallèle.                                                                                                                                         |
| 4. Séries temporelles  | L'ordre des observations compte. <b>Stationnarité</b> et <b>autocorrélation</b> sont le langage de base ; ARIMA et SARIMA modélisent la mémoire du passé et la saisonnalité. On évalue en <b>prévoyant des données non vues</b> (rétro-test), contre un repère naïf.                                                                                                                                                                                                                  |



| Chapitre                  | Ce que vous devez emporter                                                                                                                                                                                                                                                                                                                       |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5. Survie                 | La <b>censure</b> rend biaisées les moyennes naïves. $S(t) = e^{-H(t)}$ relie survie, risque et risque cumulé. <b>Kaplan-Meier</b> estime la survie sans loi ; <b>Cox</b> mesure l'effet de variables sur le risque (hypothèse : risques proportionnels) ; les modèles paramétriques extrapolent.                                                |
| 6. Bayésien et simulation | <b>A posteriori</b> $\propto$ <b>vraisemblance</b> $\times$ <b>a priori</b> . Monte-Carlo remplace l'intégrale par une moyenne (erreur en $1/\sqrt{n}$ ) ; <b>MCMC</b> simule les lois qu'on ne sait pas calculer. Un échantillonneur se <b>diagnostique</b> ( $\hat{R}$ , ESS, traces) ; un modèle se <b>vérifie</b> (prédictive a posteriori). |
| ⊗ 7. Causal               | Un effet causal est une différence entre deux mondes dont un seul est observé. La <b>randomisation</b> élimine le biais de sélection. En observationnel : DAG, score de propension, différence de différences, variable instrumentale. Chacune remplace l'information manquante par une <b>hypothèse</b> .                                       |
| ⊗ 8. Plans d'expériences  | Un bon plan (randomisation, répétition, blocs, facteurs variés <b>ensemble</b> ) simplifie l'analyse. ANOVA, plans factoriels $2^k$ , fractionnaires (alias, résolution), surfaces de réponse.                                                                                                                                                   |
| ⊗ 9. Spatial              | Des observations voisines ne sont pas indépendantes. <b>Moran</b> , <b>variogramme</b> , <b>krigage</b> , <b>fonction K</b> : un voisinage, une fonction de dépendance, une comparaison à un hasard bien défini.                                                                                                                                 |
| Projet                    | Contrôler $\rightarrow$ regarder $\rightarrow$ modéliser $\rightarrow$ vérifier hors échantillon $\rightarrow$ quantifier l'incertitude $\rightarrow$ décider $\rightarrow$ reconnaître les limites.                                                                                                                                             |

**Cinq idées qui traversent tout le volume** (reprises de l'introduction).

1. Un modèle = un **signal** et un **bruit** explicitement séparés.
2. **Maximum de vraisemblance** : choisir les paramètres qui rendent les données les plus plausibles. Régression, GLM, ARIMA, survie en sont des cas ; le bayésien en est le prolongement.
3. **Un modèle s'évalue sur ce qu'il n'a pas vu.**
4. **Les hypothèses se vérifient.**
5. **Association n'est pas causalité.**

## Quarante-deux questions

**Mode d'emploi.** Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente bonnes réponses sur les trente-six premières signalent un volume bien assimilé.

### Régression linéaire (chapitre 1)

1. Que sont les équations normales, et que représente géométriquement la solution des moindres carrés ?

2. Dans une régression de  $\ln(\text{panier})$  sur le canal, le coefficient de « Boutique » (par rapport à « Instagram ») vaut 0,22. De combien de pourcents le panier est-il plus élevé en boutique ?
3. Quelle est la différence entre un intervalle de confiance pour la **réponse moyenne** et un intervalle de **prédiction** ? Lequel est le plus large, et pourquoi ?
4. Deux variables explicatives ont une corrélation de 0,95. Quel est le facteur d'inflation de la variance (VIF) de chacune, et que cela signifie-t-il ?
5. Pourquoi ne faut-il pas choisir le modèle qui a le plus grand  $R^2$  ?
6. Lequel de Ridge et de Lasso peut annuler exactement des coefficients, et pourquoi ?

### Modèles linéaires généralisés (chapitre 2)

7. Quels sont les trois ingrédients d'un GLM ?
8. Une régression logistique donne, pour l'offre de bienvenue, un coefficient de 0,49. Le taux de rachat sans offre est de 44,8 %. Quel est le taux avec offre, selon le modèle ?
9. Qu'est-ce que la surdispersion d'un comptage, et que faire ?
10. Pourquoi une régression Gamma à lien logarithmique convient-elle à des montants positifs ?
11. Que mesure la déviance, et comment compare-t-on deux modèles emboîtés ?
12. Vos dépenses annuelles contiennent 13 % de zéros exacts et une partie positive asymétrique. Citez deux modèles adaptés.

### Analyse multivariée (chapitre 3)

13. Les valeurs propres de la matrice de corrélation de quatre variables sont 2,4, 1, 0,4 et 0,2. Quelle part de la variance la première composante résume-t-elle ?
14. Quand faut-il standardiser les variables avant une ACP ?
15. Quelle différence de nature entre l'ACP et l'analyse factorielle ?
16. Comment choisir le nombre de composantes ou de facteurs ?
17. Que minimise l'algorithme des k-means, et pourquoi le lance-t-on plusieurs fois ?
18. Pourquoi une silhouette élevée ne suffit-elle pas à prouver qu'il y a des groupes ?

### Séries temporelles (chapitre 4)

19. Qu'est-ce qu'une série faiblement stationnaire ?
20. Quelle est l'allure de l'ACF d'un AR(1) avec  $\varphi = 0,8$  ? Quelle est sa valeur au retard 3 ?
21. Dans un test de Dickey-Fuller augmenté, quelle est l'hypothèse nulle, et que conclut-on d'une p-valeur de 0,40 ?
22. Pourquoi ne peut-on pas comparer par l'AIC un modèle différencié et un modèle qui ne l'est pas ?
23. Qu'est-ce qu'un rétro-test (*rolling origin*) et pourquoi compare-t-on toujours à un repère naïf ?
24. Un modèle SARIMA a des résidus dont la statistique de Ljung-Box donne  $p = 0,002$ . Que faire ?

### Analyse de survie (chapitre 5)

25. Pourquoi la durée moyenne calculée sur les seuls clients partis est-elle biaisée ?
26. Un client part à taux constant de 0,05 par mois. Quelle est la durée médiane de la relation ?
27. Cinq clients ont les durées observées 2, 3<sup>+</sup>, 5, 7, 8<sup>+</sup> mois (<sup>+</sup> : censuré). Calculez à la main l'estimateur de Kaplan-Meier à 2, 5 et 7 mois.
28. Un rapport de risques de 0,67 pour l'offre de bienvenue : que signifie-t-il, et quelle hypothèse suppose le modèle de Cox ?

29. Quelle est l'hypothèse nulle du test du log-rank ?
30. Pourquoi « 1 – Kaplan-Meier » surestime-t-il l'incidence d'une cause en présence de risques concurrents ?

### Statistique bayésienne et simulation (chapitre 6)

31. Prior Beta(1, 1), puis 12 rachats sur 20 clients : quelle est la loi a posteriori, et sa moyenne ?
32. Différence entre un intervalle de crédibilité à 95 % et un intervalle de confiance à 95 % ?
33. Un écart-type de simulation de 0,5 : combien de tirages Monte-Carlo pour que l'erreur-type de la moyenne soit de 0,001 ?
34. Dans Metropolis-Hastings, avec une proposition symétrique, quelle est la probabilité d'accepter un candidat  $x'$  depuis  $x$  ? Pourquoi la constante de normalisation de la loi cible n'est-elle pas nécessaire ?
35. Quatre chaînes MCMC donnent un  $\hat{R}$  de 1,4. Que faire ?
36. Qu'est-ce qu'une vérification prédictive a posteriori ?

### Chapitres facultatifs (7, 8, 9)

37. Faut-il ajuster sur une cause commune ? Sur un effet commun (collision) ? Pourquoi ?
38. Pourquoi la randomisation permet-elle une lecture causale de l'écart de moyennes ?
39. Une variable instrumentale (un rappel envoyé au hasard) augmente la dépense moyenne de 3 DT et la probabilité d'ouvrir le courriel de 0,6. Quel est l'estimateur de Wald ?
40. Trois groupes de 10 observations :  $SC_{\text{inter}} = 24$  et  $SC_{\text{intra}} = 60$ . Quelle est la statistique  $F$  de l'ANOVA ?
41. Combien d'essais faut-il pour un plan factoriel complet à trois facteurs à deux niveaux, et que calcule-t-on pour l'effet principal d'un facteur ?
42. Un indice de Moran de +0,4 avec  $n = 50$  : que cela indique-t-il, et quelle est son espérance sous l'indépendance spatiale ?

## Vérifier les réponses chiffrées

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np
from scipy import stats

# Q2 : coefficient d'un modèle en logarithme
print(f"Q2  exp(0,22) - 1 = {100 * (np.exp(0.22) - 1):.1f} %")

# Q4 : VIF pour deux variables de corrélation 0,95
r = 0.95
print(f"Q4  VIF = 1 / (1 - r^2) = {1 / (1 - r**2):.2f}")

# Q8 : rapport de cotes -> probabilité
p0, coef = 0.448, 0.49
cotes = p0 / (1 - p0) * np.exp(coef)
print(f"Q8  rapport de cotes = {np.exp(coef):.3f} ; probabilité avec offre = {cotes / (1 + cotes):.3f}")

# Q13 : part de variance de la première composante (matrice de corrélation : somme des valeurs propres = nombre de variables)
vp = np.array([2.4, 1.0, 0.4, 0.2])
print(f"Q13  {vp[0] / vp.sum():.0%} de la variance")

# Q20 : ACF de l'AR(1)
print("Q20 ACF aux retards 1, 2, 3 :", [round(0.8**k, 3) for k in (1, 2, 3)])
```

```

# Q26 : médiane d'une durée exponentielle
print(f"Q26 ln(2) / 0,05 = {np.log(2) / 0.05:.2f} mois")

# Q27 : Kaplan-Meier à la main
durees = np.array([2, 3, 5, 7, 8]); evenements = np.array([1, 0, 1, 1, 0])
S = 1.0
for t in sorted(durees[evenements == 1]):
    a_risque = np.sum(durees >= t)
    S *= 1 - 1 / a_risque
    print(f"Q27 t = {t} : {a_risque} à risque, S = {S:.4f}")

# Q31 : bêta-binomiale
a, b = 1 + 12, 1 + 8
print(f"Q31 posteriori Beta({a}, {b}), moyenne = {a / (a + b):.3f}, IC crédible 95 % =
↳ [{stats.beta.ppf(0.025, a, b):.3f} ; {stats.beta.ppf(0.975, a, b):.3f}]")

# Q33 : taille de simulation
print(f"Q33 n = (0,5 / 0,001)^2 = {(0.5 / 0.001) ** 2:,.0f}")

# Q39 : Wald
print(f"Q39 3 / 0,6 = {3 / 0.6:.1f} DT")

# Q40 : F de l'ANOVA
k, n = 3, 30
F = (24 / (k - 1)) / (60 / (n - k))
print(f"Q40 F = {F:.2f}, p = {stats.f.sf(F, k - 1, n - k):.4f}")

# Q42 : espérance de l'indice de Moran
print(f"Q42 E[I] = -1/(n-1) = {-1 / 49:.4f}")

Q2 exp(0,22) - 1 = 24.6 %
Q4 VIF = 1 / (1 - r^2) = 10.26
Q8 rapport de cotes = 1.632 ; probabilité avec offre = 0.570
Q13 60% de la variance
Q20 ACF aux retards 1, 2, 3 : [0.8, 0.64, 0.512]
Q26 ln(2) / 0,05 = 13.86 mois
Q27 t = 2 : 5 à risque, S = 0.8000
Q27 t = 5 : 3 à risque, S = 0.5333
Q27 t = 7 : 2 à risque, S = 0.2667
Q31 posteriori Beta(13, 9), moyenne = 0.591, IC crédible 95 % = [0.384 ; 0.782]
Q33 n = (0,5 / 0,001)^2 = 250,000
Q39 3 / 0,6 = 5.0 DT
Q40 F = 5.40, p = 0.0106
Q42 E[I] = -1/(n-1) = -0.0204

```

## Corrigé

1. Les **équations normales**  $X^T X \beta = X^T y$  expriment que le résidu est **orthogonal** à toutes les colonnes de  $X$ . Géométriquement,  $X\hat{\beta}$  est la **projection orthogonale** de  $y$  sur le sous-espace engendré par les colonnes de  $X$ . (1.1)
2.  $e^{0,22} - 1 \approx 24,6\%$  : en boutique, le panier est environ **un quart plus élevé**, toutes choses égales par ailleurs. Dans un modèle en logarithme, un coefficient  $\beta$  se lit comme un effet **multiplicatif**  $e^\beta$ , et non comme une différence en dinars. (1.1)
3. L'intervalle sur la **réponse moyenne** encadre la valeur moyenne de  $y$  pour des valeurs données de  $x$  ; l'intervalle de **prédiction** encadre une **nouvelle observation** individuelle. Le second est plus large : il ajoute la variance du bruit individuel  $\sigma^2$  à l'incertitude sur la moyenne. (1.2)
4.  $VIF = 1/(1 - r^2) \approx 10,26$  : la variance du coefficient est multipliée par plus de dix à cause de la colinéarité. On interprète mal chaque coefficient séparément, même si les prédictions restent

correctes. (1.3)

5. Le  $R^2$  **ne peut qu'augmenter** quand on ajoute des variables, même du bruit pur : il récompense le sur-ajustement. On choisit avec l'AIC, le BIC, le  $R^2$  ajusté, ou, mieux, une **validation croisée** sur des données non utilisées pour l'ajustement. (1.4)

6. Le **Lasso** (pénalité  $\ell_1$ ), parce que la géométrie de la contrainte  $\sum |\beta_j| \leq t$  a des **coins** sur les axes : la solution tombe souvent dessus. Ridge (pénalité  $\ell_2$ , contrainte sphérique) rétrécit tous les coefficients sans jamais les annuler exactement. (1.5)

7. Une **loi** de la famille exponentielle pour la réponse, un **prédicteur linéaire**  $\eta = \mathbf{x}^\top \beta$ , et une **fonction de lien**  $g$  qui relie la moyenne au prédicteur :  $g(\mu) = \eta$ . (2.1)

8. Environ 57 % : les cotes sans offre valent  $0,448/0,552 \approx 0,81$ , multipliées par  $e^{0,49} \approx 1,63$ , puis reconverties en probabilité. Le rapport de cotes de 1,63 n'est **pas** un rapport de probabilités : une cote multipliée par 1,63 ne multiplie pas la probabilité par 1,63. (2.2)

9. Il y a **surdispersion** quand la variance observée dépasse la moyenne, alors que la loi de Poisson impose variance = moyenne. Les erreurs-types sont alors trop optimistes. On passe à une loi **binomiale négative**, ou à un Poisson avec erreurs-types robustes (quasi-vraisemblance). (2.3)

10. Les montants sont **strictement positifs** et leur variabilité **croît avec le niveau** (écart-type proportionnel à la moyenne), ce que fait la loi Gamma ( $\text{Var} = \phi\mu^2$ ). Le lien logarithmique garantit des moyennes positives et donne des effets **multiplicatifs**. (2.3)

11. La **déviance** est  $2(\ell_{\text{saturé}} - \ell_{\text{modèle}})$  : l'écart de vraisemblance au modèle parfait. Pour deux modèles **emboîtés**, la différence de déviances suit approximativement une loi du  $\chi^2$  dont les degrés de liberté valent le nombre de paramètres en plus (test du rapport de vraisemblance). (2.4)

12. Un modèle de **Tweedie** (avec  $1 < p < 2$ ), qui mêle masse en zéro et partie positive continue ; ou un **modèle en deux parties** (logistique pour « dépense nulle ou non », puis Gamma sur les dépenses positives). (2.6)

13.  $2,4/4 = 60\%$  (la somme des valeurs propres d'une matrice de corrélation vaut le nombre de variables). (3.1)

14. Quand les variables ont des **unités ou des échelles différentes** (dinars, âges, notes) : sans standardisation, la variable de plus grande variance domine l'ACP. Avec des variables de même nature et de même échelle, on peut travailler sur la matrice de covariance. (3.1)

15. L'ACP **résume** : elle cherche les combinaisons de variables de variance maximale, sans modèle. L'analyse factorielle **modélise** : elle suppose que les corrélations viennent de facteurs cachés, avec une part de bruit propre à chaque variable ( $\Sigma = \Lambda\Lambda^\top + \Psi$ ). (3.2)

16. Éboulis des valeurs propres (le coude), critère de Kaiser (valeurs propres  $> 1$ , à manier avec prudence), et surtout **analyse parallèle** (comparaison à des données sans structure), ajoutés à l'interprétabilité. En analyse factorielle, on dispose en plus d'un **test d'ajustement**. (3.1 et 3.2)

17. La **somme des carrés intra-classes** (inertie intra). L'algorithme converge vers un **minimum local** qui dépend de l'initialisation : on le lance plusieurs fois (avec k-means++) et on garde le meilleur. (3.3)

18. Parce qu'un nuage **sans structure** mais asymétrique peut aussi obtenir une silhouette élevée : une méthode de classification rend toujours des groupes. Il faut comparer à une référence sans groupes et tester la **stabilité** (rééchantillonnage, indice de Rand ajusté). (3.3)

19. Une série dont l'**espérance** est constante, la **variance** constante et dont l'**autocovariance** ne dépend que du décalage entre les dates, pas des dates elles-mêmes. (4.1)

20. Une décroissance **géométrique** :  $\rho(k) = \varphi^k$ , soit 0,8, 0,64 et 0,512 aux retards 1, 2 et 3. (4.1 et 4.2)

21. L'hypothèse nulle est la **présence d'une racine unitaire** (non-stationnarité). Une p-valeur de 0,40 ne permet pas de la rejeter : la série est compatible avec une marche aléatoire ; on la différencie. (« Ne pas rejeter » n'est pas « prouver ».) (4.1)

22. Parce que la vraisemblance porte sur **des données différentes** : la série différenciée a moins d'observations et une autre échelle. L'AIC ne se compare qu'entre modèles ajustés **à la même série**. Pour arbitrer entre ordres de différenciation, on compare des **prévisions hors échantillon**. (4.2 et 4.3)

23. On prévoit à plusieurs **origines successives** : on ajuste sur le passé jusqu'à  $t$ , on prévoit  $t + 1, \dots, t + h$ , on avance  $t$  et on recommence, pour mesurer des erreurs sur des données jamais vues. Le **repère naïf** (la dernière valeur, ou la même saison l'an passé) fixe le seuil à battre : un modèle sophistiqué qui ne le bat pas ne sert à rien. (4.3)

24. Une p-valeur de 0,002 signale une **autocorrélation résiduelle** : le modèle a laissé du signal. On ajoute des termes AR/MA ou saisonniers, on traite une rupture ou un choc non modélisé, puis on relance les diagnostics. (4.2)

25. Parce qu'on ne regarde que les clients **partis tôt** : les clients fidèles, qui restent encore au moment de l'analyse, sont exclus alors que ce sont eux qui ont les durées les plus longues. La moyenne est donc sous-estimée. (5.1)

26. Pour un risque constant  $\lambda$ ,  $S(t) = e^{-\lambda t}$ , et la médiane vaut  $\ln 2 / \lambda \approx 13,86$  mois. (5.1)

27. À 2 mois, 5 clients à risque, 1 départ :  $S = 0,8$ . À 5 mois, il reste 3 clients à risque (5, 7 et 8<sup>+</sup>), 1 départ :  $S = 0,8 \times \frac{2}{3} \approx 0,5333$ . À 7 mois, il en reste 2 à risque, 1 départ :  $S = 0,5333 \times \frac{1}{2} \approx 0,2667$ . Le client censuré à 3 mois n'est plus à risque après, mais **il a compté** dans le dénominateur jusqu'à sa sortie. (5.2)

28. Le risque instantané de départ des clients avec offre vaut **67 % de celui** des clients sans offre, à chaque instant, soit **33 % de moins**. Le modèle de Cox suppose les **risques proportionnels** : ce rapport est le même à toutes les dates. On le vérifie (graphique log-log, test de Grambsch-Therneau). (5.3)

29. Que les **fonctions de survie des groupes sont égales** à toutes les dates. (5.2)

30. Parce que « 1 – Kaplan-Meier » traite les départs pour les **autres causes** comme des censures, comme si ces clients allaient encore pouvoir partir pour la cause étudiée. Or ils ne le peuvent plus : l'incidence est donc surestimée. On utilise l'estimateur d'**Aalen-Johansen** de l'incidence cumulée. (5.5)

31. Beta(13, 9) : on ajoute les succès à  $a$  et les échecs à  $b$ . La moyenne vaut  $13/22 \approx 0,591$  ; l'intervalle de crédibilité à 95 % est donné par le code ci-dessus. (6.1)

32. L'intervalle de **crédibilité** dit : « *étant donné les données et l'a priori*, le paramètre a 95 % de chances d'être dans cet intervalle ». L'intervalle de **confiance** dit : « *la méthode* encadre la vraie valeur dans 95 % des échantillons possibles ». Le premier est une probabilité sur le paramètre, le second une propriété de la procédure. (6.1)

33.  $(0,5/0,001)^2 = 250\,000$  tirages : l'erreur décroît en  $1/\sqrt{n}$ , donc pour la diviser par dix il faut cent fois plus de tirages. (6.2)

34. On accepte avec la probabilité  $\min(1, \pi(x')/\pi(x))$ . Le rapport  $\pi(x')/\pi(x)$  **simplifie la constante de normalisation**, inconnue en général (c'est l'intégrale du produit vraisemblance  $\times$  a priori) : elle apparaît au numérateur et au dénominateur. (6.3)

35. Un  $\hat{R}$  de 1,4 (bien supérieur à 1,01) indique que les chaînes **n'ont pas convergé vers la même loi**. Ne pas utiliser les résultats : allonger les chaînes, revoir la paramétrisation, le pas de la proposition, les valeurs initiales, ou le modèle lui-même (multimodalité). (6.4)

36. On **simule des jeux de données** à partir du modèle ajusté (en tirant les paramètres dans leur loi a posteriori), et on regarde si les données observées ressemblent à ces répliques sur une statistique bien choisie (variance, nombre de zéros, maximum). Si les données réelles sortent de la distribution des répliques, le modèle ne reproduit pas un aspect important. (6.4)

37. Sur une **cause commune** (variable de confusion) : oui, on ajuste, pour bloquer le chemin de confusion. Sur un **effet commun** (collision) : **non**, car conditionner sur un effet commun **ouvre** un chemin artificiel entre ses deux causes et crée une association qui n'existe pas. (7.1)

38. Parce que, l'affectation étant faite au hasard, les groupes sont **comparables en moyenne sur tout**, y compris sur ce qu'on n'observe pas. L'écart de moyennes mesure alors uniquement l'effet du traitement : le **biais de sélection** est nul en espérance. (7.1)

39.  $3/0,6 = 5$  DT : l'effet du rappel sur la dépense (forme réduite) divisé par son effet sur l'ouverture du courriel (première étape). C'est l'effet moyen **pour les « complaisants »**, c'est-à-dire ceux dont le comportement change à cause du rappel. (7.4)

40.  $F = \frac{24/2}{60/27} = 5,40$ , avec 2 et 27 degrés de liberté ; la p-valeur est donnée par le code. (8.2)

41.  $2^3 = 8$  essais. L'effet principal d'un facteur est la **différence entre la moyenne des réponses au niveau haut et la moyenne au niveau bas**, calculée sur les 4 essais de chaque niveau. (8.3)

42. Un indice **positif** indique une **autocorrélation spatiale positive** : des zones voisines se ressemblent plus que ne le voudrait le hasard. Son espérance sous indépendance est  $-1/(n-1) = -1/49 \approx -0,0204$ . On teste ensuite l'écart par **permutations**. (9.2)

## Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer / je sais le faire / à revoir**.

| Compétence                                                       | Où la retravailler |
|------------------------------------------------------------------|--------------------|
| Écrire un modèle linéaire, l'estimer et démontrer ses propriétés | 1.1                |
| Interpréter les coefficients (indicatrices, logarithmes)         | 1.1                |
| Tester, calculer intervalles de confiance et de prédiction       | 1.2                |
| Diagnostiquer un modèle (résidus, levier, colinéarité)           | 1.3                |
| Choisir un modèle sans tricher                                   | 1.4                |
| Régulariser, résister aux aberrations, modéliser des groupes     | 1.5, 1.6, 1.7      |
| Choisir loi et lien d'un GLM, estimer et vérifier                | 2.1 à 2.4          |
| Modéliser des zéros en excès, assouplir un effet                 | 2.5, 2.6           |
| Réduire la dimension (ACP, analyse factorielle)                  | 3.1, 3.2           |
| Classer sans étiquettes et vérifier qu'il y a des groupes        | 3.3                |
| Diagnostiquer la stationnarité, lire une ACF                     | 4.1                |
| Ajuster un SARIMA et évaluer des prévisions                      | 4.2, 4.3           |
| Traiter des durées censurées (Kaplan-Meier, Cox)                 | 5.1, 5.2, 5.3      |



| Compétence                                           | Où la retravailler |
|------------------------------------------------------|--------------------|
| Ajuster des modèles de durée paramétriques           | 5.4                |
| Raisonner à la Bayes, choisir un a priori            | 6.1                |
| Simuler (Monte-Carlo, MCMC) et diagnostiquer         | 6.2, 6.3, 6.4      |
| Formuler une question causale et choisir une méthode | 7.1 à 7.4          |
| Concevoir une expérience, analyser un plan           | 8.1 à 8.4          |
| Mesurer l'autocorrélation spatiale, krigeage         | 9.2, 9.3           |
| Mener une étude de modélisation de bout en bout      | Projet du volume   |

## Et maintenant ?

Vous savez maintenant **construire, vérifier et interpréter** des modèles statistiques. Le **volume III : Apprentissage automatique** reprend le même cycle (données → modèle → évaluation hors échantillon → décision) avec des modèles plus flexibles (arbres, forêts, boosting, réseaux), dont l'objectif premier est la **prédiction**. Tout ce que vous avez appris ici ne devient pas inutile : la validation croisée, la régularisation, la vérification de modèles, la différence entre prédire et expliquer en sont le fondement.

**À retenir, tout simplement.** Un modèle est une carte : il simplifie pour servir. Votre métier, désormais, consiste à savoir **ce que la carte montre, ce qu'elle cache, et comment vérifier** que vous n'avez pas oublié le fleuve.